VLA Interview Library · 八卷总目录

点击目录条目可跳转到合订 PDF 中对应卷首或章节起始页；页码为合订版页码。

VOL 01 · Foundations 4

知识库 · VOL 01 Foundations：机器人、深度学习与强化学习的共同语言 6

题卡 · VOL 01 Foundations：核心面试题卡（Batch 1） 17

来源与事实复核 · VOL 01 Foundations Batch 1：独立事实复核与公式边界 51

来源与事实复核 · VOL 01 Foundations Batch 2：MDP / TD / 机器人控制独立事实复核 53

来源与事实复核 · VOL 01 Foundations Batch 3：Impedance / MPC / Learned Policy 独立事实复核 56

来源与事实复核 · VOL 01 Foundations：39 卡 Completion Fact Reconciliation 59

VOL 02 · IL / RL / Post-training 63

知识库 · VOL 02 IL / RL / Post-training：从示范数据到可验证反馈 65

题卡 · VOL 02 IL / RL / Post-training：核心面试题卡 71

来源与事实复核 · VOL 02 IL / RL / Post-training：来源计划 113

来源与事实复核 · VOL 02 IL / RL / Post-training：独立事实与实现边界复核 115

VOL 03 · VLA / Action 118

知识库 · VOL 03 VLA / Action：从多模态条件到可执行动作（完整卷） 120

题卡 · VOL 03 VLA / Action：核心面试题卡（完整卷） 164

来源与事实复核 · VOL 03 Source Audit C：ACT CVAE、Action Chunk 与 Temporal Ensemble Claim-to-Code 189

来源与事实复核 · VOL 03 Source Audit C：ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核 202

来源与事实复核 · VOL 03 Source Audit F：VLA 架构、适配、延迟与动作执行 Claim-to-Code 208

来源与事实复核 · VOL 03 Source Audit F：VLA 架构、Mask、适配与执行独立事实复核 221

来源与事实复核 · VOL 03 VLA / Action：44-Card Completion Reconciliation 228

来源与事实复核 · VOL 03 Source Audit D：Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code 232

来源与事实复核 · VOL 03 Source Audit D：Diffusion Policy 独立事实复核 244

来源与事实复核 · VOL 03 Source Audit E：Flow Matching 基础、π0 特化与 ODE Claim-to-Code 250

来源与事实复核 · VOL 03 Source Audit E：Flow Matching / π0 Time − Sign 独立事实复核 270

来源与事实复核 · VOL 03 Source Audit A：OpenVLA Action Token 与 I/O Claim-to-Code 276

来源与事实复核 · VOL 03 Source Audit A：OpenVLA Endpoint / Token / Mask 独立事实复核 285

来源与事实复核 · VOL 03 Source Audit B：π0 / π0.5 / FAST / openpi Claim-to-Code 288

来源与事实复核 · VOL 03 Source Audit B：π0 / π0.5 / FAST / OpenPI 独立事实复核 304

来源与事实复核 · VOL 03 VLA / Action：来源计划 311

VOL 04 · World Model / Sim2Real 317

知识库 · VOL 04 World Model / Sim2Real：Foundations、评估与证据（Batches 1 − 2） 319

题卡 · VOL 04 World Model / Sim2Real：核心面试题卡（Foundations + Evaluation） 344

来源与事实复核 · VOL 04 Batch 1：独立事实复核记录（WM-039..044） 356

来源与事实复核 · VOL 04 Source Audit E：Benchmark / Evidence Claim-to-Protocol 360

来源与事实复核 · VOL 04 Source Audit C：Cosmos Policy / WAM Claim-to-Code 372

来源与事实复核 · VOL 04 Cosmos Policy / WAM：1 epoch gate 与 full10k 运行族对账 382

来源与事实复核 · VOL 04 Audit C：Cosmos Policy / WAM 独立事实复核 386

来源与事实复核 · VOL 04 Audit D：Sim2Real 独立事实复核 395

来源与事实复核 · VOL 04 Audit B：UWM 独立事实复核 406

来源与事实复核 · VOL 04 Source Audit B：UWM Video / Action Diffusion Claim-to-Code 410

来源与事实复核 · VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 Foundations Claim-to-Code 416

来源与事实复核 · VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 独立事实复核 427

VOL 05 · Engineering / Evaluation 431

知识库 · VOL 05 Engineering / Evaluation：数据、训练、部署与闭环诊断（Batch 1） 433

题卡 · VOL 05 Engineering / Evaluation：核心面试题卡（Batch 1） 440

来源与事实复核 · VOL 05 Batch 1：独立官方文档事实复核 445

来源与事实复核 · VOL 05 Batch 2A：独立官方文档事实复核 448

来源与事实复核 · VOL 05：Completion Audit / Independent Fact Review 452

来源与事实复核 · VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结 456

VOL 06 · Perception / Grounding / Navigation 461

知识库 · VOL 06 Perception / Grounding / Navigation：Point、Box、Text 与坐标协议（PER-001） 464

知识库 · VOL 06 Perception / Grounding / Navigation：2D → 3D Lift 与 Frame Contract（PER-002） 470

知识库 · VOL 06 Perception / Grounding / Navigation：多视角、时序与输入 Schema（PER-003） 476

知识库 · VOL 06 Perception / Grounding / Navigation：VLN 与 ObjectNav（PER-004） 482

知识库 · VOL 06 Perception / Grounding / Navigation：Navigation Policy Architecture（PER-005） 490

知识库 · VOL 06 Perception / Grounding / Navigation：Image Preprocess、Patch Token 与 Camera Identity（PER-006） 497

知识库 · VOL 06 Perception / Grounding / Navigation：Open-vocabulary Grounding Pipeline（PER-007） 508

知识库 · VOL 06 Perception / Grounding / Navigation：Point / Box / Mask / Keypoint / Affordance 表示选择（PER-008） 519

知识库 · VOL 06 Perception / Grounding / Navigation：Grounding Matching、Negative 与 Loss Reduction（PER-009） 530

知识库 · VOL 06 Perception / Grounding / Navigation：Grounding Metric、Strict Parser 与 Protocol Comparability（PER-010） 541

知识库 · VOL 06 Perception / Grounding / Navigation：Pseudo-label、Teacher Noise、Confidence Calibration、Leakage 与 Curriculum（PER-011） 552

知识库 · VOL 06 Perception / Grounding / Navigation：Segmentation、Depth、Keypoint 与 Manipulation Affordance（PER-012） 565

知识库 · VOL 06 Perception / Grounding / Navigation：Point Cloud、Voxel、BEV、Occupancy 与 Sampling（PER-013） 577

知识库 · VOL 06 Perception / Grounding / Navigation：3D Grounding、Spatial Relation 与 Open-vocabulary 3D Scene（PER-014） 592
知识库 · VOL 06 Perception / Grounding / Navigation：Perception→Action、Reachability、Uncertainty 与 Failure Gate（PER-015） 607
知识库 · VOL 06 Perception / Grounding / Navigation：Localization、Odometry、SLAM、Map Frame 与 Drift（PER-016） 625
知识库 · VOL 06 PER-017：Belief、Memory、Frontier 与 Active Exploration 641
知识库 · VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias 654
知识库 · VOL 06 PER-019：Navigation Evaluation、Failure Taxonomy 与 Fail-Closed Protocol 664
知识库 · VOL 06 PER-020：Robustness、Safety、Fallback 与风险门 673
题卡 · VOL 06 Perception / Grounding / Navigation：PER-001 原子题卡 681
题卡 · VOL 06 Perception / Grounding / Navigation：PER-002 原子题卡 683
题卡 · VOL 06 Perception / Grounding / Navigation：PER-003 原子题卡 685
题卡 · VOL 06 Perception / Grounding / Navigation：PER-004 原子题卡 687
题卡 · VOL 06 Perception / Grounding / Navigation：PER-005 原子题卡 689
题卡 · VOL 06 Perception / Grounding / Navigation：PER-006 原子题卡 691
题卡 · VOL 06 Perception / Grounding / Navigation：PER-007 原子题卡 693
题卡 · VOL 06 Perception / Grounding / Navigation：PER-008 原子题卡 695
题卡 · VOL 06 Perception / Grounding / Navigation：PER-009 原子题卡 697
题卡 · VOL 06 Perception / Grounding / Navigation：PER-010 原子题卡 699
题卡 · VOL 06 Perception / Grounding / Navigation：PER-011 原子题卡 701
题卡 · VOL 06 Perception / Grounding / Navigation：PER-012 原子题卡 703
题卡 · VOL 06 Perception / Grounding / Navigation：PER-013 原子题卡 705
题卡 · VOL 06 Perception / Grounding / Navigation：PER-014 原子题卡 707
题卡 · VOL 06 Perception / Grounding / Navigation：PER-015 原子题卡 710
题卡 · VOL 06 Perception / Grounding / Navigation：PER-016 原子题卡 713
题卡 · VOL 06 Perception / Grounding / Navigation：PER-017 原子题卡 716
题卡 · VOL 06 Perception / Grounding / Navigation：PER-018 原子题卡 719
题卡 · VOL 06 Perception / Grounding / Navigation：PER-019 原子题卡 721
题卡 · VOL 06 Perception / Grounding / Navigation：PER-020 原子题卡 723
来源与事实复核 · VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias 独立事实复核 725
来源与事实复核 · VOL 06 PER-020：Robustness、Safety、Fallback 与风险门独立事实复核 732
来源与事实复核 · VOL 06 Perception / Grounding / Navigation：Coverage-Gap Audit 737

VOL 07 · Personal Project Deep Dives 744

来源与事实复核 · VOL 07：五项目证据、Ownership 与面试叙事 Reconciliation 746
口述与练习 · 五项目串联：从实现证据反推的 VLA 项目总述口述稿 749
口述与练习 · PROJ-WAM-001/002 从项目实现反推的证据约束口述稿 752
题卡 · PROJ-WAM：Cosmos Policy / WAM 项目原子题卡（Training Failure + Diagnostic + Ownership Batch 1） 761
项目证据 · Cosmos / WAM Claim − Evidence Inventory（WAM-01） 764
项目证据 · Cosmos / WAM Ownership Ledger：多目标训练、RoboCasa 评测与诊断边界 771
项目证据 · WM-019 WAM P0 Bundle Audit：CoffeePressButton 775
项目证据 · Cosmos Policy full10k trained checkpoint：RoboCasa 24×5 P0 评测证据审计 777
项目证据 · Cosmos Policy RoboCasa Official-HF Baseline：P0 评测证据审计 782
项目证据 · Cosmos Policy TurnOffMicrowave 本地训练失败：P0 First-Divergence 审计 786
项目证据 · Cosmos Policy / WAM Checkpoint Save-Load Gate：P0 证据审计 792
项目证据 · WM-019 WAM Diagnostic Artifact Manifest（CoffeePressButton） 798
项目证据 · WM-019 WAM Action-Trace Paired Diagnostic（CoffeePressButton） 801
项目证据 · WM-019 WAM Future-Image Metric Paired Diagnostic（CoffeePressButton） 803
项目证据 · WM-019 WAM Matched-Checkpoint Protocol Bridge（iter_000010000） 805
口述与练习 · PROJ-UWM-001..004 从项目实现反推的证据约束口述稿 807
题卡 · VOL 07 UWM / LIBERO 项目原子题卡（Protocol Boundary Batch 1 + Ownership） 815
项目证据 · UWM / LIBERO Claim − Evidence Inventory（UWM-01） 818
项目证据 · UWM / LIBERO Ownership Ledger：协议迁移、数据管线与结果边界 824
项目证据 · UWM / LIBERO P0 Bundle Audit：Protocol / Data Pipeline / Remote Result 830
项目证据 · UWM RoboCasa ActionHeadFix 30 Epoch：Train-to-Closed-Loop P0 证据审计 832
来源与事实复核 · UWM libero_90.pt 本地 Immutable Checkpoint Manifest Gate 838
来源与事实复核 · VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结 842
口述与练习 · PROJ-OPENPIVLA-001 + Batch 2 从项目实现反推的证据约束口述稿 847
题卡 · PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 1） 851
题卡 · PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 2） 855
项目证据 · OpenPiVLA Claim − Evidence Inventory（OPV-01） 858
项目证据 · OpenPiVLA Ownership Ledger：个人贡献、文件归属与面试口径 865
项目证据 · OpenPiVLA P0 Bundle Audit：Formal Trainer / Action Route / Eval Lineage 871
来源与事实复核 · OpenPiVLA Action JSON Parser / Round-trip Artifact Gate 873
来源与事实复核 · OpenPiVLA ALOHA Action-Level Eval：Checkpoint Load / Serve / L1 Gate 878
来源与事实复核 · OpenPiVLA 本地 Artifact Gate：JSON/CE 动作表示与 H/D shape contract 881
来源与事实复核 · OpenPiVLA / LIBERO Checkpoint Load − Serve − Client Lineage Gate 884
来源与事实复核 · OpenPiVLA / LIBERO Matched Failure Probe Readiness Gate 887
来源与事实复核 · OpenPiVLA Official Checkpoint Immutable Manifest Gate 891

来源与事实复核 · OpenPiVLA 本地 Runtime Rollout Artifact Gate：ALOHA 与 LIBERO 895
口述与练习 · PROJ-HARNESS-001..008 从项目实现与 P0 Audit 反推的证据约束口述稿 898
题卡 · PROJ-HARNESS：Robot Harness 项目原子题卡（Direct-Evidence Batch 1） 902
项目证据 · Robot Harness Claim – Evidence Inventory（RH-01） 906
项目证据 · Robot Harness Ownership Ledger：Owner 决策、Codex 实现与结果归属 912
项目证据 · Robot Harness P0 Bundle Audit：Runtime / Post-training / Eval 918
项目证据 · Robot Harness 五/八模型 × 十套件聚合证据门 920
项目证据 · Robot Harness 四模型共同支持集 Raw Bundle Manifest 925
口述与练习 · PROJ-OPD-001..009 从项目实现反推的证据约束口述稿 931
题卡 · PROJ-OPD：OPD / VLM Grounding 项目原子题卡（Direct-Evidence Batch 1 + Ownership） 934
项目证据 · OPD / VLM Grounding Claim – Evidence Inventory（OPD-01） 939
项目证据 · OPD / VLM Grounding Ownership Ledger：数据、路由、训练与结果归属 945
项目证据 · OPD / VLM Grounding P0 Bundle Audit：Data Route / Distillation / Holdout 951
项目证据 · OPD Training Objective Lineage Gate：top1 CE、Full-Vocabulary KL 与 Rollout Source 953

VOL 08 · Coding / System Design 957

知识库 · VOL 08 CODE-001：数值稳定的 softmax / log-softmax 实现 959
知识库 · VOL 08 CODE-002：Scaled Dot-Product Attention 手写与调试 961
知识库 · VOL 08 CODE-003：LayerNorm / RMSNorm 手写与调试 963
知识库 · VOL 08 CODE-004：Masked MSE / Huber Action Loss 965
知识库 · VOL 08 CODE-005：Homogeneous Transform Composition 968
知识库 · VOL 08 CODE-006：2-Link Planar Forward Kinematics 970
知识库 · VOL 08 CODE-007：PID Update with Anti-windup 972
知识库 · VOL 08 CODE-008：Bellman TD(0) Update 976
知识库 · VOL 08 SYS-001：可复现的 VLA 训练—评估平台 979
知识库 · VOL 08 SYS-002：Policy Serving 与 Real-time Control Loop 982
知识库 · VOL 08 SYS-003：Robot Fleet 与 Multi-tenant GPU Scheduling 987
知识库 · VOL 08 SYS-004：Fault Recovery、Safety 与 Observability 992
题卡 · VOL 08 CODE-VLA-001 原子手撕题卡：数值稳定的 softmax 997
题卡 · VOL 08 CODE-VLA-002 原子手撕题卡：Scaled Dot-Product Attention 998
题卡 · VOL 08 CODE-VLA-003 原子手撕题卡：LayerNorm / RMSNorm 999
题卡 · VOL 08 CODE-VLA-004 原子手撕题卡：Masked MSE / Huber Loss 1000
题卡 · VOL 08 CODE-VLA-005 原子手撕题卡：Homogeneous Transform Composition 1001
题卡 · VOL 08 CODE-VLA-006 原子手撕题卡：2-Link Planar Forward Kinematics 1002
题卡 · VOL 08 CODE-VLA-007 原子手撕题卡：PID Update with Anti-windup 1003
题卡 · VOL 08 CODE-VLA-008 原子手撕题卡：Bellman TD(0) Update 1004
题卡 · VOL 08 SYS-001 原子题卡：可复现的 VLA 训练—评估平台 1005
题卡 · VOL 08 SYS-002 原子题卡：Policy Serving 与 Real-time Control Loop 1006
题卡 · VOL 08 SYS-003 原子题卡：Robot Fleet 与 Multi-tenant GPU Scheduling 1007
题卡 · VOL 08 SYS-004 原子题卡：Fault Recovery、Safety 与 Observability 1008
来源与事实复核 · VOL 08 Coding / System Design：Coverage-Gap Audit 1009

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 01 · Foundations

数学、Transformer、优化、MDP / TD 与控制基础

> 本卷由 6 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 01 Foundations：机器人、深度学习与强化学习的共同语言

题卡

1. 02 · VOL 01 Foundations：核心面试题卡（Batch 1）

来源与事实复核

1. 03 · VOL 01 Foundations Batch 1：独立事实复核与公式边界
2. 04 · VOL 01 Foundations Batch 2：MDP / TD / 机器人控制独立事实复核
3. 05 · VOL 01 Foundations Batch 3：Impedance / MPC / Learned Policy 独立事实复核
4. 06 · VOL 01 Foundations：39 卡 Completion Fact Reconciliation

Part 1 / 6 · `interview_materials/vla/knowledge/vol_01_foundations.html` · SHA-256
`D422CA9FAD9536F743350130461FB11C98A8D75E1833B6756129699F773CF23A`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：
`d422ca9fad9536f743350130461fb11c98a8d75e1833b6756129699f773cf23a`；生成自：
`interview_materials\vla\knowledge\vol_01_foundations.md`

VOL 01 Foundations：机器人、深度学习与强化学习的共同语言

状态：`batch1+2+3_fact_reviewed_candidate / WARN` 报告日期：2026-07-16 证据截止：2026-07-16 适用岗位：VLA / 具身智能算法 / 多模态机器人策略 本卷不展开：VLA 专属 action head、Flow Matching 细节、World Model 训练细节和个人项目完整复盘；这些内容分别放到 VOL 03、VOL 04、VOL 07。

问题（Question）

本卷要让你能用同一套语言回答四类问题：

- **数据/模型问题**：observation、state、action、token、mask 和 loss 在张量层面是什么？
- **机器人问题**：坐标变换、Jacobian、控制频率、延迟和 action denormalization 为什么会影响闭环？
- **训练问题**：Transformer、归一化、MSE、交叉熵、Bellman target 和 advantage 各自监督什么？
- **评估问题**：离线 proxy、online success、reward、value 和 success predicate 为什么不能混为一谈？

口述目标

面试官问“你做 VLA 需要哪些基础”时，先给结论：

> VLA 不是只把图像和语言喂给大模型；它把观测和任务条件编码成策略输入，再把输出变成带坐标、尺度、时序和控制约束的动作。因而基础要同时覆盖表示学习、序列建模、机器人几何、控制延迟和闭环评估。

背景与动机

VLA 面试常见的失分不是完全不会某个名词，而是把不同层次混在一起：把 `state` 当成 `observation`，把离线动作 MSE 当成真实成功率，把 action token 当成控制器动作，把 reward 当

成 value，或者把一个归一化配置当成算法创新。

本卷建立一条最小因果链：

```
传感器 / 图像 / 语言 / proprioception
                ↓
       observation 与 state 表示
                ↓
   几何坐标、Transformer 表示、策略输出
                ↓
  normalization / denormalization / controller
                ↓
       robot closed-loop rollout 与评估
```

这条链既能解释一般 VLA，也能连接你的 OpenPiVLA、UWM、WAM 和 OPD 项目；但具体项目 claim 仍必须回到本地页面和日志，而不是由基础公式自动推出。

基础概念与术语

术语	口语解释	严谨边界	常见误区
Observation	机器人此刻能读到的传感器输入	o_t，可能是不完整、有噪声的状态观测	直接把 observation 当完整 state
State	足以决定未来转移的系统状态	s_t，在 MDP 中定义转移与奖励	只要有一张图就一定是 state
Proprioception	机器人自身关节/末端/速度等本体感知	状态向量的一部分，维度和坐标系需明确	不说明单位、顺序和时间戳
Action	策略发给控制器的控制量	a_t，可为位置、速度、力矩或 chunk	忽略 action 轴顺序、尺度和执行频率
Policy	从条件到动作的映射	$\pi_\theta(a_t \mid o_{\le t}, c)$	把 action head 等同整个 policy
Closed loop	执行动作后重新观测并纠偏	rollout 中 $o_t \to a_t \to o_{t+1}$ 的循环	把一次性序列预测当稳定执行
Protocol	评估时固定的初始化、归一化、步数和成功判定	benchmark 的可比性条件	只比较最终百分比

几何、状态与控制

Observation、State、Action 的层次

通俗地说，图像、关节角和任务文本是“模型看见的东西”，state 是描述系统未来演化所需的信息，action 是控制器真正执行的量。真实机器人通常只能获得部分 state，因此策略需要在历史 observation 或记忆上做推断。

专业地说，若系统满足 Markov 性质，状态转移可以写为 $p(s_{t+1} \mid s_t, a_t)$。实际 VLA 的输入往往是 o_t、历史帧、语言条件 c 和 proprioception 的组合，因此更安全的面试表述是“策略条件输入”，不要未经证据声称它拥有完整 state。

齐次坐标与坐标变换

在机器人学里，旋转和平移常合并到齐次变换矩阵：

$$T_{ab} = \begin{bmatrix} R_{ab} & p_{ab} \\ 0_{1\times 3} & 1 \end{bmatrix}, \qquad \tilde{p}_a = T_{ab}\tilde{p}_b.$$

变量解释：$R_{ab} \in SO(3)$ 表示坐标系 b 到 a 的旋转，p_{ab} 是原点平移，$\tilde{p} = [p_x, p_y, p_z, 1]^\top$ 是齐次点。直觉是先把点放到统一的坐标协议里，才能比较 action、目标框或末端位姿。实现映射是矩阵乘法；但你当前仓库没有提供 OpenPiVLA 的具体坐标变换代码，因此只能把它作为通用基础，不能声称某个本地函数一定采用此轴序。

矩阵乘法顺序不可随意交换：

$$T_{ac} = T_{ab}T_{bc}, \qquad T_{ab}T_{bc} \neq T_{bc}T_{ab}.$$

这解释了为什么相机坐标、机器人 base、末端坐标和世界坐标的命名必须写清楚；顺序错时，模型可能在数值上输出“合理”的动作，却在物理空间中朝错误方向运动。

Forward Kinematics 与 Jacobian

给定关节角 q，forward kinematics 计算末端位姿 $x = f(q)$。局部速度关系由 Jacobian 给出：

$$\dot{x} = J(q)\dot{q}.$$

变量解释：$\dot{q}$ 是关节速度，$\dot{x}$ 可包含末端线速度和角速度，$J(q)$ 是随姿态变化的 Jacobian。直觉是它把“关节空间的小动作”映射到“末端空间的小动作”；接近奇异位形时，J 的 rank 或条件数退化，在 inverse/伪逆映射中，为实现某些末端速度可能需要很大的关节速度，并放大测量噪声。实现上需要明确 Jacobian 的轴序、单位和参考坐标系；VLA 输出若是末端增量，仍需经过控制器或 IK 才能执行。

Action 归一化、反归一化与执行

连续 action 常在训练前做逐维归一化：

$$\hat{a}_t^{(i)} = \frac{a_t^{(i)} - \mu_i}{\sigma_i + \epsilon}, \qquad a_t^{(i)} = (\sigma_i + \epsilon)\hat{a}_t^{(i)} + \mu_i.$$

这里 μ_i, σ_i 是第 i 个动作维度的统计量，$\hat{a}$ 是模型空间，a 是控制器空间。直觉是让不同单位和量级的轴在训练中有相近的数值尺度；实现中必须保证训练和推理使用同一版本 normalizer、轴顺序、

gripper 约定和 clipping 规则。`denormalization` 正确不等于控制安全，还要检查控制频率、动作范围和执行窗口。

PID 与延迟

最基础的 PID 控制器可以写为：

$$u_t = K_P e_t + K_I \sum_{k=0}^{t} e_k \Delta t + K_D \frac{e_t - e_{t-1}}{\Delta t}.$$

e_t 是目标与当前状态的误差，u_t 是控制输出，Δt 是控制周期。P 项响应当前误差，I 项消除稳态偏差，D 项抑制变化过快；实现时需考虑积分饱和、导数噪声、derivative kick、滤波和输出限幅。VLA 负责高层动作并不意味着可以忽略底层控制器：推理延迟、观测延迟和 replan 间隔会改变闭环有效频率。

Impedance Control、MPC 与 Learned Policy

这三类方法通常不在同一层竞争。impedance control 规定接触中的力—位移动态关系；MPC 用模型和约束滚动求解有限时域动作；learned policy/VLA 从高维观测与语言条件预测目标、动作或 action chunk。实际系统常让 VLA 给 subgoal/reference，再由 MPC、IK 或 impedance/controller 执行。

令 $e = x - x_d$，一种目标阻抗关系可写为：

$$M_d \ddot{e} + D_d \dot{e} + K_d e = F_{ext}.$$

M_d, D_d, K_d 是期望惯性、阻尼和刚度，F_{ext} 是外部交互力。直觉是规定机器人受力后的动态响应，而不是只追踪位置。实现上必须说明力/位姿 frame、传感器、底层 torque/position loop、采样周期、饱和和滤波；“用了 impedance”不等于天然安全或稳定。

有限时域 MPC 可抽象为：

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) + \ell_f(x_H), \quad x_{k+1} = f(x_k, u_k), \quad x_k \in \mathcal{X},\ u_k \in \mathcal{U}.$$

H 是规划 horizon，f 是动力学模型，ℓ, ℓ_f 是阶段/终端代价，$\mathcal{X}, \mathcal{U}$ 是状态和控制约束。MPC 每次只执行第一步或短前缀后重新估计和求解；它能显式处理约束，但会受模型失配、状态估计、solver deadline 和 terminal design 影响。

learned policy 可写为 $\pi_\theta(a_t \mid o_{\le t}, c)$ 或 chunk distribution。它擅长视觉/语言条件与复杂数据分布，却仍依赖 action normalization、frame/unit、controller rate、replan、safety limit 和 success predicate。模型输出可能是直接 action、末端目标、MPC reference 或 residual；接口语义不同，不能仅按维度判断。

场景	合理组合	首要检查
接触、插入、表面跟随	policy 给目标，impedance/admittance 处理力—位移关系	force sensing、stiffness/damping、delay
动力学和约束较可信	policy/world model 给 reference/model，MPC 做 constrained replan	model error、state estimate、solver deadline
高维视觉语言与多任务	learned policy/VLA 负责条件理解，controller 负责执行	data coverage、OOD、action contract
长时序且模型不准	subgoal/chunk + 短 horizon replan + recovery/fallback	observation age、failure detector、fallback

深度学习与 Transformer

MSE、Huber 与交叉熵

连续动作回归常用带 mask 的均方误差：

$$\mathcal{L}_{\text{MSE}} = \frac{\sum_{t,d} m_{t,d} (\hat{a}_{t,d} - a_{t,d})^2}{\sum_{t,d} m_{t,d} + \epsilon}.$$

$m_{t,d}$ 表示第 t 步第 d 个动作维度是否有效。它与 action padding、缺失标签或只训练部分 head 直接对应；不能只写“算 MSE”，还要说明 mask 和 reduction。Huber loss 在小误差区近似二次、大误差区近似一次，通常更抗 outlier。离散 token 或工具调用则常用 cross-entropy，监督对象是目标 token 的类别，而不是连续坐标本身。

Scaled Dot-Product Attention

Transformer 的核心注意力为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Q, K, V 分别是 query、key、value，d_k 是 key 的通道维度，M 是 causal/padding 等 mask。除以 $\sqrt{d_k}$ 可以抑制点积随维度增长造成的 softmax 饱和；mask 负责禁止无效位置或未来 token。实现映射是 `scaled_dot_product_attention` 或等价的矩阵运算；在 VLA 中，图像、语言、proprio 和 action token 的拼接顺序及 mask 决定信息能否跨模态流动。

LayerNorm 与 RMSNorm

LayerNorm 对一个 token 的特征维度做标准化：

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} + \beta.$$

它减去均值并除以标准差；γ, β 是可学习的缩放与偏置。RMSNorm 去掉均值中心化，只使用均方根：

$$\mathrm{RMSNorm}(x) = g \odot \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}}.$$

两式的变量和实现都对应最后一维的 hidden state。面试不要绝对化地说 RMSNorm 一定更好：它减少了计算和参数，但是否更稳定要看 backbone、精度和训练设置。Qwen3-VL 的具体 norm 配置属于上游模型事实，应以官方模型卡/源码为准；本卷只准备原理。

Pre-Norm、残差与 LoRA

Transformer block 中，pre-norm 常把归一化放在子层之前，post-norm 放在残差相加之后。回答时重点讲梯度路径和深层训练稳定性，不要只背位置。

LoRA 将可训练更新限制为低秩矩阵：

$$W' = W + \frac{\alpha}{r} BA,$$

其中 W 是冻结的原权重，$A \in \mathbb{R}^{r \times d}$、$B \in \mathbb{R}^{d' \times r}$，$r \ll \min(d, d')$。直觉是用低秩子空间近似任务更新；它节省显存和优化参数，但低秩假设可能限制任务适应能力。OpenPiVLA 页面中的 Qwen3-VL + LoRA 属于你的本地训练路径，具体注入层和 rank 仍需以项目代码/配置核对。

MDP、RL 与策略评估

MDP 与 Bellman 方程

Markov Decision Process 可表示为 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$。给定策略 π，状态价值为：

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi,\, s' \sim P}\left[r(s,a) + \gamma V^{\pi}(s')\right].$$

$P(s' \mid s, a)$ 是转移分布，r 是即时奖励，γ 是折扣因子。这里公式把 $r(s,a)$ 视为条件期望 reward；若 reward 还依赖 s'，应把它放进对转移的期望。Bellman 方程表达“当前价值等于即时收益加折扣后的未来价值”；它不等于某个模型的训练 loss，必须先说明 value target 的构造。

动作价值和优势函数为：

$$Q^\pi(s,a) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a], \qquad A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

优势表示某个动作相对于当前状态平均水平好多少。PPO/GRPO 等算法会用到 advantage 或相对优势，但 WAM 中的 value head、离线 MSE 和在线 reward 不是同一对象。

TD 误差与 on/off-policy

一步 TD 误差可写成：

$$\delta_t = r_t + \gamma(1 - d_t^{term})V_\phi(s_{t+1}) - V_\phi(s_t),$$

其中 $d_t^{term} \in \{0, 1\}$ 只表示真实环境终止。真实终止通常停止 bootstrap；time-limit `truncated` 通常仍应从 $V_\phi(s_{t+1})$ bootstrap，不能无条件把两者压成一个 `done`。on-policy 方法使用当前或近当前策略采样的数据，分布更新更贴近当前 policy 但样本复用受限；off-policy 方法可以复用 replay/history，但要处理行为策略与目标策略的分布差异。off-policy 不等于 offline RL，前者仍可持续与环境交互。

为什么离线 proxy 不等于 online success

离线 action MSE 只衡量数据集状态上的动作接近程度：

$$\mathbb{E}_{(o,a)\sim D}\left[\|\pi_\theta(o) - a\|^2\right].$$

online success 还受到动作误差累积、接触动力学、视觉分布偏移、控制延迟、重规划和 success predicate 的影响。因此 WAM 面试回答必须把 policy MSE 说成离线代理指标，而不是“策略成功率提升”；UWM 的 0.744 目前只保留为 user-confirmed remote final，必须带 strict fixed-init 等协议与 raw-bundle 缺口，不能写成本地 artifact-verified benchmark。

Pipeline 与伪代码

训练 step：带 mask 的连续 action 监督

```
def masked_action_step(batch, policy, optimizer, normalizer):
    # Read images, language, proprioception, action labels, and valid masks.
    obs = batch["observation"]
    action = batch["action"]
    mask = batch["action_mask"]

    # Normalize action with the same statistics used by the dataset contract.
    target = normalizer.normalize(action)

    # Encode multimodal inputs and predict the action representation.
    pred = policy(obs)

    # Only valid time/dimension entries contribute to the regression objective.
    loss = ((pred - target) ** 2 * mask).sum() / (mask.sum() + 1e-6)

    # Update the trainable parameters and clear gradients for the next step.
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    return loss
```

这是知识层伪代码，不声称与你本地 OpenPiVLA trainer 的函数名一致；真实字段、action window 和 collator 需要在 VOL 07 的项目证据中逐项核对。

闭环执行：重新观测、反归一化与重规划

```
def closed_loop_rollout(env, policy, normalizer, task, replan_steps):
    # Reset the environment with the benchmark's declared initialization protocol.
    obs, info = env.reset(task)
    terminated, truncated, success = False, False, False

    while not (terminated or truncated or success):
        # Build the policy input from images, language, and proprioception.
        model_obs = build_observation(obs, task)

        # Predict a normalized action chunk, not yet a controller-space command.
        normalized_chunk = policy(model_obs)

        # Convert back to controller units and enforce the declared action limits.
        action_chunk = normalizer.denormalize(normalized_chunk)
        action_chunk = clip_to_controller_range(action_chunk)

        # Execute only a short prefix, then re-observe to reduce open-loop drift.
        for action in action_chunk[:replan_steps]:
            obs, reward, terminated, truncated, info = env.step(action)
            success = bool(info.get("success", False))
            if terminated or truncated or success:
                break

    # Keep task success separate from termination/truncation and preserve traces.
    return success, info.get("trace", [])
```

闭环的关键不是“模型输出了一个 chunk”，而是 chunk 执行多少步、何时重新观测、如何反归一化、如何判定终止和如何记录失败。它直接连接 OpenPiVLA 的 action window/controller 讨论和 UWM 的 strict eval 讨论。

实现映射与项目挂钩

基础概念	本地项目证据	当前能安全说的内容	尚待核对
action window / normalization / controller	`interview_materials/03_openpivla_action_modeling.html`	你理解并复原了动作表示、归一化、chunk 和 controller 的闭环关系	具体源码函数、轴顺序和归一化文件路径
offline action MSE vs online rollout	`interview_materials/04_cosmos_wam_multitask_training.html`	WAM 页把 policy MSE 作为离线代理，并主动区分 online success	每个 MSE 数字的完整数据划分和脚本路径
fixed-init / warmup / normalizer	`interview_materials/05_uwm_libero_eval_protocol.html`	UWM 的 strict 协议是可比性前提	所有 checkpoint 与 normalizer checksum
Transformer / VLM backbone	`interview_materials/02_opd_vlm_grounding.html`、简历 HTML	可以讲多模态输入、grounding 输出协议和数据分布	Qwen3-VL 具体 layer/config 需官方源码或本地 config
schema / reward / eval	`interview_materials/01_robot_harness_tool_training.html`	可解释三者在工具调用训练中的不同角色	runtime 代码、训练字段覆盖统计和完整 reward checker

公源/本地设计边界

公有设计 / 上游公开流程

- Transformer attention、LayerNorm、RMSNorm、MDP、Bellman 和经典机器人几何属于公开基础知识。
- OpenVLA、openpi、Diffusion Policy、ACT、LIBERO 等仓库和论文的具体接口与 benchmark 以其官方版本为准。

本地证据

- 五个现有项目 HTML、简历 HTML 和 `output/resume_restructure_plan.md` 是当前工作区可直接读取的证据。
- 它们支持你如何叙述项目、哪些指标出现过以及哪些风险已经被主动承认。

本地推断

- 从项目页可以推断你需要把 action normalization、评估协议和数据分布作为统一的解释链；这不是某个单一源码函数的直接证明。

尚未核实

⚠️ 当前工作区没有 OpenPiVLA、WAM、UWM 的完整源码、normalizer 文件和训练日志，因此本卷不把伪代码中的字段名、函数名或张量 shape 写成本地实现事实。

方法对比与选择建议

问题	选择/解释	面试防线
连续动作监督	MSE 简单稳定；Huber 更抗 outlier；是否有效取决于 action normalization 和闭环执行	不把 MSE 下降说成成功率提升
离散 token 监督	Cross-entropy 适合 token 序列，复用语言模型解码；动作精度和串行延迟需要额外讨论	不说 token 化“没有精度损失”
on-policy vs off-policy	前者数据贴近当前策略，后者复用数据但有 distribution shift	先问数据来自哪个 policy
pre-norm vs post-norm	关注深层梯度路径与训练稳定性，不背绝对结论	具体 backbone 配置要查源码
高层 policy vs 低层 controller	policy 负责任务条件下的动作意图，controller 负责时序、限幅、动力学和安全	明确接口和单位
offline proxy vs online eval	proxy 便宜、可定位，online 更接近任务目标但受 protocol 影响	同时报告数据划分和 rollout 协议

风险与后续建议

1. **基础概念不能只背公式**：每个公式都要能说明变量、shape、代码位置和失败模式。
2. **坐标协议是高危点**：仍需补一张 action axis/order/coordinate checklist，连接 OpenPiVLA 和 UWM；本轮只修复公式转义，不把轴序写成本地事实。
3. **缺源码时要主动降级**：当前本卷只写通用机制，不把伪代码说成已验证的本地实现。
4. **题卡需要口述校准**：本卷清单中的 P0 题先进入题库，P1/P2 根据第一次 drill 的表现调整。
5. **下一卷衔接**：VOL 02 进入 BC、DAgger、PPO、GRPO、SFT/DPO 和数据分布；VOL 03 再展开 VLA action head、Diffusion/Flow Matching。

参考文献（References）

1. Vaswani et al., “Attention Is All You Need,” 2017.
2. Ba, Kiros, Hinton, “Layer Normalization,” 2016.
3. Zhang and Sennrich, “Root Mean Square Layer Normalization,” 2019.
4. Sutton and Barto, *Reinforcement Learning: An Introduction*, 2nd ed.
5. Lynch and Park, *Modern Robotics*, Chapter 4: Forward Kinematics.
6. PyTorch, `torch.nn.LayerNorm` documentation.
7. PyTorch, `scaled_dot_product_attention` documentation.
8. OpenVLA official repository.
9. Physical Intelligence, `openpi` official repository.
10. LIBERO official repository.
11. 本地项目证据：`interview_materials/01_robot_harness_tool_training.html`、`02_opd_vlm_grounding.html`、`03_openpivla_action_modeling.html`、`04_cosmos_wam_multitask_training.html`、`05_uwm_libero_eval_protocol.html`。

12. PyTorch, `torch.nn.MSELoss` documentation.
13. PyTorch, `torch.nn.HuberLoss` documentation.
14. PyTorch, `torch.nn.CrossEntropyLoss` documentation.
15. Gymnasium, Handling Time Limits.
16. Lynch and Park, *Modern Robotics*, Chapter 5: Velocity Kinematics and Statics.
17. Åström and Murray, *Feedback Systems*.
18. VOL 01 Batch 2 MDP / TD / Control Independent Fact Review.
19. Hogan, Impedance Control: An Approach to Manipulation, Part I.
20. Rawlings, Mayne, Diehl, *Model Predictive Control: Theory, Computation, and Design*.
21. Chi et al., Diffusion Policy.
22. Kim et al., OpenVLA.
23. VOL 01 Batch 3 Advanced Control Independent Fact Review.

Part 2 / 6 · `interview_materials/vla/questions/vol_01_foundations.html` · SHA-256
`068330C88ABBEE213EB279FF7C2542B8DEC8BD76BF9BAA7CD6AA3EA81A85E56A`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：
`068330c88abbee213eb279ff7c2542b8dec8bd76bf9baa7cd6aa3ea81a85e56a`；生成自：
`interview_materials\vla\questions\vol_01_foundations.md`

VOL 01 Foundations：核心面试题卡（Batch 1）

状态：`drafted` 题数：39 难度：L1 12 题 / L2 23 题 / L3 4 题 练习状态：全部 `unseen` 说明：P0 表示本仓库必须掌握，不代表公开面经频次。

A. 表示、优化与损失

▼ `L1` `P0` **VLA-FND-001 · 为什么机器人策略和 VLM 都需要向量化表示？embedding 维度代表什么？**

30 秒回答：模型只能处理数值张量，所以图像 patch、文本 token、proprioception 和 action 都要先向量化。embedding 维度是表示空间的通道数，不等于类别数；维度增大通常提高容量，但也增加显存、计算和过拟合风险。多模态进入共享 backbone 前还必须对齐 shape、位置与 mask。

机制展开（约 2 分钟）：图像由视觉编码器变成 patch/token 序列，文本由 tokenizer 与语言 embedding 变成 token，proprioception/action window 则通过连续特征投影或 action head 的输入映射进入同一计算图。所谓“对齐”不只是把最后一维设成相同大小，还包括时间/空间位置编码、模态类型标识、padding/valid mask，以及 batch、sequence、feature 三个轴的约定。维度选择要和 attention 的计算量、数据量、冻结/微调策略一起看，不能把更大的 embedding 直接说成更强的语义理解。

项目挂钩：OpenPiVLA 中图像、指令和 action window 最终都要进入统一训练 contract。

追问：多模态 embedding 如何对齐？拼接和 cross-attention 有什么区别？

易错：embedding 的单个维度通常没有稳定、可解释的一一语义。

来源：[1][27][41]　**练习状态**：unseen

▼ L1 P0 VLA-FND-002 · 点积、余弦相似度和欧氏距离分别适合什么场景?

30 秒回答：点积同时受方向和模长影响，常用于 attention logits；余弦相似度先归一化、主要比较方向，适合语义 embedding；欧氏距离衡量绝对空间距离，适合坐标或已统一尺度的特征。选择前要确认归一化、模长意义和量纲是否一致。

机制展开（约 2 分钟）：若 x, y 未归一化，点积 $x^\top y = \|x\|\|y\| \cos\theta$ 会把方向和模长混在一起；这正是 attention 中 logits 既受相似方向又受 feature scale 影响的原因。余弦相似度显式除以两个范数，适合比较语义方向，但会丢掉幅值信息；欧氏距离对坐标偏移敏感，必须先确认 frame、单位和 normalization。机器人 state/action 的不同维度若量纲不统一，直接用欧氏距离会让大尺度轴支配结果；OPD 的 box/point 几何误差应使用几何合同，而不是拿语义余弦替代。

项目挂钩：OPD 的 box/point 几何误差不能直接用语义余弦相似度代替。

追问：为什么 CLIP 常用归一化后的点积?

易错：不能在未统一量纲的机器人 state/action 上直接比较欧氏距离。

来源：[1][27][44]　**练习状态**：unseen

▼ L2 P0 VLA-FND-003 · gradient、Jacobian 和 Hessian 分别是什么?

30 秒回答：gradient 是标量函数的一阶导数，Jacobian 是向量输出对向量输入的一阶偏导矩阵，Hessian 是标量函数的二阶导数矩阵。训练里 gradient 用于更新参数；机器人里 Jacobian 把关节速度映射为末端速度；Hessian 描述局部曲率。三者输入输出类型不同，不能混为一谈。

机制展开（约 2 分钟）：对标量 $f(x)$，gradient 可写成 $\nabla_x f$；对向量 $y(x)$，Jacobian 为 $J_{ij} = \partial y_i / \partial x_j$；Hessian 为 $H_{ij} = \partial^2 f / (\partial x_i \partial x_j)$。机械臂在当前关节位姿 q 附近满足 $\dot{x} = J(q)\dot{q}$，因此 Jacobian 的小奇异值会让伪逆放大观测噪声和控制误差，常用阻尼最小二乘或限幅来稳定 IK。VLA 训练中的 $\nabla_\theta \mathcal{L}$ 是损失对模型参数的梯度，不等于机器人运动 Jacobian；action denormalization、controller 和环境 step 还可能包含不可微或安全限幅环节。面试时要先说明变量的输入输出，再谈数值稳定性。

项目挂钩：动作空间和 controller contract 的解释需要区分参数梯度与机器人运动 Jacobian。

追问：为什么 Jacobian 条件数大时 IK 不稳定?

易错：Jacobian 不是“更大的 gradient”，要先看输入输出是否都是向量。

来源：[27][37][41]　**练习状态**：unseen

▼ L2 P0 VLA-FND-004 · LayerNorm、BatchNorm、RMSNorm 如何取舍?

30 秒回答：BatchNorm 按通道使用 batch/空间统计，依赖 batch 稳定性且训练、推理统计不同；LayerNorm 对每个样本或 token 的特征维归一化，适合 Transformer；RMSNorm 只按均方根缩放、不减均值，形式更简。选型首先服从 pretrained backbone 和兼容性，不能脱离统计轴说谁绝对更好。

机制展开（约 2 分钟）：BatchNorm 在训练时使用当前 mini-batch 的均值/方差，并维护 running statistics 供推理使用，所以 batch 很小、跨设备统计不同或数据分布漂移时容易不稳。LayerNorm 对单个 token 的 hidden dimension 计算均值和方差，不依赖其他样本，适合变长序列；RMSNorm 可写为 $y_i = \gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$，保留均值方向，只控制均方根尺度。实际迁移 Qwen3-VL 等 pretrained backbone 时，norm 类型、Pre-Norm/Post-Norm 位置、ϵ、权重名称和 dtype 都属于 checkpoint contract，随意替换会破坏加载与训练行为。它们处理的是网络 activation；action normalization 则把物理动作按数据统计映射到模型尺度，两者目的和证据完全不同。

项目挂钩：Qwen3-VL 的具体 norm 类型应查官方模型配置，不凭印象回答。

追问：为什么小 batch 训练时 BatchNorm 容易不稳定?

易错：归一化层与 action normalization 不是同一件事。

来源：[2][3][6][28]　**练习状态**：unseen

▼ L2 P0 VLA-FND-006 · MSE、MAE、Huber loss 什么时候使用?

30 秒回答：MSE 对误差平方求平均，梯度平滑但会放大 outlier；MAE 使用绝对误差，对 outlier 更稳但零点不可微；Huber 在小误差区用二次项、大误差区用一次项，是两者折中。连续 action 选 loss 前还要固定维度尺度、normalization、valid mask 和 reduction，否则数值不能比较。

机制展开（约 2 分钟）：令误差 $e = \hat{y} - y$，MSE 为 $\mathbb{E}[e^2]$，MAE 为 $\mathbb{E}[|e|]$。Huber loss 写成：

$$\ell_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta, \\ \delta\left(|e| - \frac{1}{2}\delta\right), & |e| > \delta. \end{cases}$$

变量解释：δ 决定二次区与一次区的切换点；小误差保持平滑梯度，大误差的梯度被限制，降低异常值主导。对 action tensor，正确 reduction 应按 valid mask 的有效元素数归一，而不是把 padding 或不存在的 action 维度算进分母；不同轴还应先统一物理单位或使用训练 normalizer。gripper 若是离散开合语义，可能更适合分类/BCE 或单独加权，而不是和位姿轴无条件共享 MSE。即使 teacher-forced policy MSE 下降，也只说明离线标签拟合更好；闭环还受

observation shift、采样、反归一化、controller 和 success predicate 影响，不能直接推出 WAM task success。

项目挂钩：WAM 的 policy MSE 是离线代理，不自动推出闭环成功率。

追问：gripper 二值维度是否应该和连续位姿用同一种 loss?

易错：loss 越小不一定任务成功率越高。

来源：[11][12][29][42]　练习状态：unseen

B. Transformer 与多模态输入

▼ `L1` `P0` **VLA-FND-010 · softmax 如何把 logits 变成概率？怎样保证数值稳定?**

30 秒回答：softmax 把 logits 变成和为 1 的概率分布；实现时先减去最大 logit，因为整体平移不改变结果，却能避免指数溢出。训练交叉熵通常直接用 `log_softmax` 或 fused cross-entropy，避免先算概率再取 log。temperature 和 mask 必须在 softmax 前处理。

机制展开（约 2 分钟）：原式是 $p_i = \exp(z_i) / \sum_j \exp(z_j)$。令 $m = \max_j z_j$，稳定实现为 $p_i = \exp(z_i - m) / \sum_j \exp(z_j - m)$，它利用了 log-sum-exp 的平移不变性；极端 logits 仍应关注 dtype、溢出和 underflow。temperature T 通常使用 z_i / T：T 越大分布越平，越小越尖锐；它改变采样/分类分布，不等于自动完成 calibration。无效 token 或 padding 应在 softmax 前加 mask（通常置为 $-\infty$ 或使用框架 masked kernel），而不是先归一化后再把概率清零。交叉熵可直接计算 `log_softmax(z) - target`，数值更稳定。VLA 中 action token、tool token 和文本 token 共用分类 logits 时，必须说明 vocabulary mask、padding mask 和 temperature 的作用域；softmax 输出也不等于已校准的置信度。

项目挂钩：action token、tool token 和 VLM 文本输出都依赖稳定的分类 logits。

追问：temperature 如何改变分布尖锐度?

易错：softmax 输出是模型分布，不是置信度已校准的证明。

来源：[1][13][30]　练习状态：unseen

▼ `L1` `P0` **VLA-FND-011 · cross-entropy 与 MSE 的监督对象有什么区别?**

30 秒回答：cross-entropy 监督离散类别或 token 的概率分布，MSE 监督连续数值误差。文本、action token 和工具选择通常用 cross-entropy；连续 action、state 或 value regression 通常用 MSE。把动作离散化后，必须额外评估 decode 回连续动作后的误差、控制接口和闭环结果。

机制展开（约 2 分钟）：对 one-hot 目标 y 和 logits z，交叉熵可写为 $\mathcal{L}_{CE} = -\sum_k y_k \log p_k$，其中 $p = \mathrm{softmax}(z)$；它关心正确类别相对其他类别的概率质量。连续回归常写为

$\mathcal{L}_{MSE} = \frac{1}{N}\sum_i m_i(\hat{a}_i - a_i)^2$，其中 m_i 是 valid/action mask，分母必须与有效元素一致。动作离散化会引入 bin width、边界和 vocabulary/sequence length 的选择：bin 更多可能降低量化误差，却增加 token 数、稀疏监督和 decode 复杂度；CE 变好不保证反归一化后的连续 action 更准。反过来，MSE 的数值也受 action normalization、维度权重和 reduction 影响。OpenPiVLA 的 assistant-side action token 与 WAM 的连续 action MSE 属于不同监督协议，不能直接比较 loss 数值；必须在同一 decode、单位、mask、控制器和 success predicate 下比较下游结果。

项目挂钩：OpenPiVLA 的 assistant-side action token 与 WAM 连续 action MSE 属于不同监督协议。

追问：离散 bin 越多是否一定更精确?

易错：训练 loss 不同，数值大小不能横向直接比较。

来源：[8][9][11][13][41][42]　练习状态：unseen

▼ L2 P0 VLA-FND-012 · attention 为什么除以 $\sqrt{d_k}$?

30 秒回答：若 query/key 各维独立且方差相近，点积的方差会随 d_k 增长。logits 过大时 softmax 饱和、梯度变小；除以 $\sqrt{d_k}$ 把尺度拉回稳定范围。它控制的是 logits 的数值尺度，不是为了让概率和为 1。

机制展开（约 2 分钟）：scaled dot-product attention 可写为：

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

若 q_i, k_i 独立、零均值且方差为 1，则 $q^\top k = \sum_{i=1}^{d_k} q_i k_i$ 的方差约为 d_k，标准差约为 $\sqrt{d_k}$。不缩放时 logits 随 head dimension 变大，softmax 容易进入近似 one-hot 区域，反向梯度变小；除法把典型标准差恢复到常数级。M 是 attention mask，应在 softmax 前加入（无效位置通常为 $-\infty$），不能先得到概率再清零后重新假设归一化。temperature 也能改变 logits 尖锐度，但它是显式分布温度，不能替代对 d_k 的结构性缩放。VLA 中 $Q/K/V$ 可能混合图像、文本、proprio 或 action tokens，仍要先核对 batch、head、sequence、head-dimension 的 shape；缩放不能修复错误的 token mask 或模态位置编码。

项目挂钩：VLM backbone 处理图像、文本和动作条件时都依赖 attention 的稳定尺度。

追问：temperature 与 $1/\sqrt{d_k}$ 有什么联系?

易错：softmax 本身就保证和为 1，缩放解决的是饱和问题。

来源：[1][7]　练习状态：unseen

▼ L2 P0 **VLA-FND-013 · self-attention、cross-attention、causal mask 和 padding mask 如何区分?**

30 秒回答：self-attention 的 Q/K/V 来自同一序列，cross-attention 的 query 与 key/value 来自不同序列或模态。causal mask 禁止看未来位置，padding mask 屏蔽补齐 token；loss mask 只决定哪些目标计入损失，不一定改变 attention。多模态 VLA 中必须分清 attention mask、condition mask 和 loss mask。

机制展开（约 2 分钟）：若 $Q \in \mathbb{R}^{B\times h\times L_q\times d}$、$K,V \in \mathbb{R}^{B\times h\times L_k\times d}$，attention score 的形状是 $B \times h \times L_q \times L_k$，通常计算 $\mathrm{softmax}(QK^\top/\sqrt{d}+M)V$。self-attention 取 $L_q = L_k$ 且来自同一 token 序列；cross-attention 允许 query 来自语言/动作流，而 key/value 来自图像或另一模态。causal mask 是时间方向的上三角约束；padding mask 根据有效长度屏蔽 key/value 位置；二者可叠加。loss mask 则在 logits/target reduction 阶段过滤监督项，即使某个 token 不计 loss，它仍可能被模型作为上下文读取。WAM 的 world/value/action target mask、sample routing 与 Transformer attention mask 是不同控制面，必须分别记录 broadcast shape、取值约定和作用阶段；只做 loss mask 可能仍让无效条件污染上下文，反过来只做 attention mask 也不会自动改变 loss 分母。

项目挂钩：WAM 的 world/value mask 与 Transformer attention mask 不应混成同一概念。

追问：只做 loss mask、不做 attention mask 会发生什么?

易错：mask 的值约定和 broadcast shape 要查具体实现。

来源：[1][7][42]　**练习状态**：unseen

▼ L2 P0 **VLA-FND-016 · 图像、文本、proprio 和 action token 如何进入同一个模型?**

30 秒回答：图像、文本、proprio 和 action 先经各自 encoder/tokenizer，再投影到兼容的 hidden size，通过拼接、自注意力、cross-attention 或独立 expert 交互。必须同时固定模态标识、位置/时间、有效 mask 和 action loss 区域；hidden size 相同不等于语义已经对齐。

机制展开（约 2 分钟）：可以把每种输入写成 $z_m = P_m(E_m(x_m))$，其中 E_m 是模态编码器、P_m 把输出投影到共享 hidden size；之后再由 token order、modality/type embedding、position/time embedding 和 attention mask 定义信息流。图像通常是 patch/token 序列，文本要遵守 tokenizer 与 chat template，proprio 可压成单 token 或按时间/关节拆成多个 token，action 既可能是连续 head 输入，也可能是离散 token 目标。多模态拼接降低接口复杂度但依赖长序列和 mask；cross-attention 可隔离 query/key/value 角色但增加模块和 shape 合同；独立 expert 便于专门处理 action，却需要明确交互层和梯度路由。OpenPiVLA 的 image/instruction/action window 不能只说“拼起来”，还要核对 chat template、时间对齐、padding、action formatter 与哪些 assistant positions 进入 causal-CE。

项目挂钩：OpenPiVLA 需要对齐 image/instruction/action window 与 Qwen chat template。

追问：proprio 作为单 token 和多 token 的 trade-off 是什么？

易错：hidden size 对齐不等于语义已经对齐。

来源：[8][9][41]　练习状态：unseen

C. MDP、RL 与策略评估

▼ L1 P0 **VLA-FND-021 · MDP 的 S, A, P, R, γ 分别是什么？**

30 秒回答：MDP 用 S, A, P, R, γ 描述状态空间、动作空间、转移分布、奖励和折扣因子。机器人里 observation 可能不是完整 state，action 必须说明位置/速度/力矩、单位与控制频率；reward、done 和 success predicate 也要分别定义。语言指令通常是条件，不自动等于 state。

机制展开（约 2 分钟）：策略通常写成 $\pi(a_t \mid s_t)$，回报为 $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$。$P(s'|s,a)$ 描述环境动力学，R 可以是 dense shaping、cost 或 sparse task reward；γ 决定远期回报的权重。实际机器人往往只能得到 o_t，若 o_t 不满足 Markov 性，就需要历史窗口、recurrent state、belief 或显式 memory；带语言指令时更安全地写成 $\pi(a_t \mid o_{\le t}, l)$，而不是把 l 直接当作物理 state。动作还要绑定 frame、单位、维度、chunk/horizon、controller 与 frequency。episode 的真正 terminal 应停止 bootstrap，time-limit truncated 通常仍可 bootstrap；具体实现要看环境 API。UWM/LIBERO 的 binary success、WAM 的 value/world target 和训练 proxy reward 不是同一个量，比较时必须保留 protocol tuple。

项目挂钩：UWM 的 rollout success 与 WAM 的 value target 不能只用“奖励”统称。

追问：部分可观测时为什么需要历史或 memory？

易错：语言指令通常是条件，不自动等于 MDP state。

来源：[4][42][43]　练习状态：unseen

▼ L1 P0 **VLA-FND-022 · value、Q 和 advantage 有什么区别？**

30 秒回答：$V^\pi(s)$ 是按策略从状态 s 出发的期望回报；$Q^\pi(s,a)$ 固定第一步动作后的期望回报；$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ 表示该动作比策略平均水平好多少。advantage 常用于策略更新，value 常作 baseline，target 取法取决于算法和 termination 语义。

机制展开（约 2 分钟）：三者关系为 $V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)]$ 与 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$。一步 TD 误差常写成 $\delta_t = r_t + \gamma(1-d_t)V(s_{t+1}) - V(s_t)$，其中 d_t 必须区分真正 terminal 与 time-limit truncated；GAE 再对多步 δ 做带 λ 的加权累积。减去只依赖 state 的 baseline 不改变 policy-gradient 的期望，却能降低方差。Q/value 还可以通过 Monte Carlo、TD、n-step 或

lambda-return 构造，不能把一个实现的 target 当成所有 RL/VLA 的默认。WAM 的 value/world auxiliary head 若存在，只能按项目合同解释为辅助预测或 ranking 信号；它不自动等于控制动作，也不证明 online RL policy improvement。面试时要同时说清输入、target、mask、bootstrap 和评估协议。

项目挂钩：WAM 的 value head 是辅助目标，不能说它直接输出控制动作。

追问：为什么减 baseline 不改变 policy gradient 的期望?

易错：value 不是当前即时 reward。

来源：[4][42]　练习状态：unseen

▼ L2 P0 **VLA-FND-023 · Bellman expectation equation 与 optimality equation 有什么不同?**

30 秒回答：Bellman expectation equation 固定策略 π，对下一动作按 π 求期望，回答“这条策略有多好”；optimality equation 对动作取最大值，定义 V^*, Q^* 并隐含 greedy policy improvement，回答“最优能有多好”。连续高维动作不能枚举 max，通常由 actor 或候选动作搜索近似；benchmark success 也不能证明达到 Bellman optimum。

机制展开（约 2 分钟）：对 state value，可写成 $(\mathcal{T}^\pi V)(s) = \mathbb{E}_{a\sim\pi, s'\sim P}[r + \gamma V(s')]$ 与 $(\mathcal{T}^* V)(s) = \max_a \mathbb{E}_{s'\sim P}[r + \gamma V(s')]$；对 action value，则是 $(\mathcal{T}^\pi Q)(s,a) = \mathbb{E}[r + \gamma \mathbb{E}_{a'\sim\pi} Q(s',a')]$ 和 $(\mathcal{T}^* Q)(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q(s',a')]$。$\mathcal{T}^\pi$ 只评估固定策略，$\mathcal{T}^*$ 把评估与改进耦合起来。连续动作中的 actor-critic 用 critic 估计 Q，再让 actor 近似 $\arg\max_a Q(s,a)$ 或最大化其期望；这不是解析枚举，也受函数逼近、数据覆盖和 critic 误差限制。许多 BC、diffusion 或 flow-matching VLA 只是条件动作生成器，除非训练目标和运行证据明确包含 Bellman backup，不能称为在求最优 Q。WAM value proxy 与 LIBERO success 统计同样不是 Bellman optimality 的解或证明。

项目挂钩：WAM value 估计与 LIBERO success 统计都不是在求解析的 Bellman 最优解。

追问：actor-critic 如何绕过连续动作的显式 max?

易错：Bellman 方程不是某个神经网络结构。

来源：[4][42][43]　练习状态：unseen

▼ L2 P0 **VLA-FND-024 · on-policy 与 off-policy 的核心差异是什么?**

30 秒回答：on-policy 用当前或近当前目标策略 π 产生的数据更新，估计分布直接，但策略一变数据就迅速变旧，样本复用有限；off-policy 允许行为策略 $\mu \neq \pi$，可复用 replay、历史或其他策略数据，样本效率更高，却要处理分布修正、OOD 动作和估计偏差。off-policy 仍可持续交互，不等于只有固定数据的 offline RL。

机制展开（约 2 分钟）：on-policy estimator 的轨迹来自 $\tau \sim \pi_\theta$，PPO 只允许有限轮次、有限策略偏移地复用 rollout；off-policy 数据则来自 $d^\mu(s)\mu(a \mid s)$。若行为概率可得且 support 覆盖，可用 importance ratio $\rho_t = \pi(a_t \mid s_t)/\mu(a_t \mid s_t)$ 重加权，但长轨迹的 ratio 乘积方差很高，clip、truncate 或 per-decision correction 又会引入 bias–variance trade-off，而且 $\mu(a \mid s) = 0$ 的动作无法靠重加权补出来。Q-learning 用 $r + \gamma \max_{a'} Q_{target}(s', a')$ 构造与行为动作不同的 target，是典型 off-policy 思路；深度实现还需 replay、target network、double-Q 或 conservative regularization 控制 bootstrap 与 OOD 误差。offline RL 是 off-policy 学习的更强约束场景：数据集固定，不能主动补覆盖。Robot Harness 当前只是固定 candidate group 上的 reward-shaped weighted token CE；它不是 canonical online GRPO，也不能仅因复用离线样本就称为标准 off-policy RL。

项目挂钩：canonical online GRPO 与当前 Harness 的 offline reward-shaped grouped weighting 承担不同训练角色，不能混称为同一实现。

追问：importance sampling 为什么能修正分布差异?

易错：off-policy 不等于 offline RL；前者仍可持续交互。

来源：[4][45]　**练习状态**：unseen

▼ L2 P0 VLA-FND-025 · policy gradient 为什么需要 advantage/baseline?

30 秒回答：REINFORCE 用 return 给动作加权，但同一状态下整条轨迹的随机性会让梯度方差很大。减去只依赖 state 的 baseline 不改变期望梯度，却能减少这种共同噪声；advantage 就是动作相对该状态平均水平的增量。实际通常用 value network、TD/GAE 来估计它，而不是把 advantage 当作已知标签。

机制展开（约 2 分钟）：策略梯度可写成 $\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t]$。若换成 $G_t - b(s_t)$，因为 baseline 不依赖当前采样动作，有 $\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] = 0$，所以期望梯度不变，但方差通常下降。令 $V^\pi(s)$ 近似状态平均回报，则 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ 表示该动作相对基准的贡献；一步 TD 可用 $\delta_t = r_t + \gamma(1-d_t)V(s_{t+1}) - V(s_t)$，GAE 再对多个 δ 做 λ 加权。baseline 过弱，降方差有限；过强或 target/termination/mask 写错，会把偏差带进更新。GRPO 的组内相对信号可以起到 baseline-like 的作用，但不能直接说它等于机器人 value head；Robot Harness 的 reward-shaped weighted token CE 也不能据此改称 canonical online GRPO。

项目挂钩：GRPO 使用组内相对信号，但不能把它和机器人 value head 直接等同。

追问：为什么 baseline 不能依赖当前采样动作?

易错：advantage 不是天然已知，需要估计。

来源：[4][45]　**练习状态**：unseen

▼ L2 P0 **VLA-FND-029 · 为什么 offline action MSE 降低不一定提高 online success?**

30 秒回答：offline action MSE 只比较数据集状态上的动作误差；closed loop 中一个小误差可能改变后续 observation，使策略进入未见状态并累积偏差。online success 还受 action denormalization、控制频率/延迟、接触动力学、replan 和 success predicate 影响，所以 MSE 是诊断 proxy，不是成功率的充分条件。

机制展开（约 2 分钟）：连续动作的 masked MSE 可写成 $\mathcal{L}_{MSE} = \frac{\sum_i m_i(\hat{a}_i - a_i)^2}{\max(1, \sum_i m_i)}$，但这个数值依赖 action normalization、维度权重、mask、reduction 和数据状态分布。即便 teacher-forced 或 fixed-window MSE 下降，部署时仍要经过反归一化、chunk 切片、控制器执行和环境 step；误差会改变下一帧视觉/proprio，导致 $d^{\pi}(s)$ 偏离离线数据分布。接触任务还可能存在“位置误差很小但接触时序失败”，而低频重规划、延迟或错误的 success predicate 也会让相同动作误差对应不同成功结果。更接近 online 的离线诊断应至少固定 checkpoint、数据 split、state coverage、normalizer、action horizon、seed 和 protocol，并报告 per-task/per-init 的 failure slice；最终仍需 matched closed-loop rollouts。WAM 的离线 MSE 趋势只能支持 proxy 或 root-cause 候选，历史改善比例在没有同 checkpoint、同脚本和原始 artifact 时不能升级为成功率提升。

项目挂钩：WAM 的离线 MSE 趋势只能作为 proxy；历史改善比例仍需同数据、同 checkpoint、同脚本 artifact 复核，不能写成成功率提升。

追问：如何设计更接近在线表现的离线评估?

易错：离线指标没有价值是另一个极端；它适合快速定位和消融。

来源：[4][10][39][42]　**练习状态**：unseen

D. 机器人几何、控制与闭环

▼ L1 P0 **VLA-FND-030 · observation、state、proprioception、action 如何区分?**

30 秒回答：observation 是传感器实际读到的量，可能有噪声且不完全；state 是在 Markov 假设下足以决定下一步转移分布的系统状态；proprioception 是关节角、速度、末端位姿、夹爪等本体感知，通常只是 observation/state 的一部分；action 是策略发给控制器的命令。回答时必须补时间戳、单位、坐标系、维度和控制频率。

机制展开（约 2 分钟）：环境可写成 $s_{t+1} \sim P(\cdot \mid s_t, a_t)$，传感器再由 $o_t \sim O(\cdot \mid s_t)$ 产生 observation。真实机器人常只得到多视角图像、深度和 proprio，因此单帧 o_t 未必满足 Markov 性；可用历史窗口、recurrent memory、state estimator 或 belief $b_t(s) = p(s_t \mid o_{\le t}, a_{<t})$ 补偿。proprio 的“末端位姿”还要说明是在 base/world/camera frame，下游是否已做 normalization。action 可能是关节位置/速度/力矩、末端位姿 delta、夹爪指令或 action

chunk；策略输出通常还要经过 denormalization、限幅、坐标变换和 low-level controller，才真正影响动力学。OpenPiVLA/UWM 的 image、instruction、proprio、action window 不能笼统叫输入输出：要冻结 shape、frame、unit、timestamp、valid mask、chunk/horizon 与 controller contract，否则 offline loss、server response 和 closed-loop rollout 无法对齐。

项目挂钩：OpenPiVLA 的 observation/state normalization 不能只叫“预处理”。

追问：只有单帧图像时是否满足 Markov 性质?

易错：可观测到的信息不等于完整 state。

来源：[4][5][35][41]　**练习状态**：unseen

▼ L1 P0 **VLA-FND-031 · 世界、相机、base、末端和关节坐标为什么必须区分?**

30 秒回答：同一个点或动作在不同 frame 下数值不同：视觉观测通常在 camera frame，机器人控制常在 base/world、末端 frame 或 joint space。必须明确变换方向、旋转和平移顺序、单位、轴约定和时间戳；否则数值看似合理，执行方向仍可能整体反掉。相机内参负责像素与相机射线，外参负责 camera 与机器人/world frame 的关系。

机制展开（约 2 分钟）：齐次变换可写成 $T_B^A = \begin{bmatrix} R_B^A & t_B^A \\ 0 & 1 \end{bmatrix}$，若点在 frame B 中表示为 $\tilde{p}_B = [p_B, 1]^\top$，则 $\tilde{p}_A = T_B^A \tilde{p}_B$，组合时必须按实际链路顺序相乘，例如 $T_C^W = T_B^W T_E^B T_C^E$ 只在每个下标方向定义一致时成立。旋转可用矩阵、四元数或 axis-angle 表示，但要固定左/右乘、主动/被动旋转、角度单位和 quaternion 排列。相机内参 K 描述焦距、主点和像素尺度；外参 T_C^B 描述相机相对 base 的位姿，二者不能互换。操作 action 还要区分绝对 pose、delta pose、joint command 与 controller 输入，保证 transform、normalizer 和 gripper convention 一致。OPD 的 box/point 像素坐标、OpenPiVLA 的末端 action、UWM 的 controller action 不是同一空间，必须在接口处显式记录 frame contract 和 round-trip 检查。

项目挂钩：OPD 的图像坐标与 OpenPiVLA 的机器人 action 坐标不是同一个空间。

追问：相机外参和内参分别负责什么?

易错：坐标系命名中的上下标方向不能凭感觉解释。

来源：[5][35][44]　**练习状态**：unseen

▼ L2 P0 **VLA-FND-032 · 齐次变换矩阵为什么能同时表示旋转和平移?**

30 秒回答：把三维点扩成齐次坐标 $[x, y, z, 1]^\top$，就能用 $T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$ 的一次矩阵乘法同时完成旋转和平移。多个 frame 通过矩阵连乘组合，但顺序不可交换；实现前还要固定 column/row vector、主动/被动旋转和下标方向。

机制展开（约 2 分钟）：在 column-vector 约定下，$\tilde{p}_A = T_B^A \tilde{p}_B$，其中 $R \in SO(3)$ 满足正交和单位行列式，p 是平移；这让刚体位姿落在 $SE(3)$ 的统一接口中。若 $T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$，则 $T^{-1} = \begin{bmatrix} R^\top & -R^\top p \\ 0 & 1 \end{bmatrix}$，不能只把平移取负而忽略旋转。组合链例如 $T_C^W = T_B^W T_E^B T_C^E$ 必须从右到左作用在点上；row-vector 实现或右乘约定会改变写法。工程中还要处理单位（m/mm、rad/deg）、axis order、quaternion 排列、时间同步和数值正交化。VLA action contract 如果把 world/base/end-effector frame 混用，offline parser 可能仍能通过 shape 检查，但 closed-loop 会出现系统性方向或尺度偏差，因此应加入 transform round-trip、identity 和 inverse consistency 测试。

项目挂钩：action contract 若混淆 world/base/end-effector frame，会出现系统性偏差。

追问：T_{ab}^{-1} 如何写？

易错：旋转矩阵不能随意当作欧拉角向量相加。

来源：[5][35][41]　**练习状态**：unseen

▼ L2 P0 VLA-FND-033 · FK 与 IK 的输入输出和难点是什么？

30 秒回答：FK 输入关节位置/姿态 q，沿机器人链计算末端 pose，通常是确定映射；IK 反过来给定目标末端 pose，求满足 $f(q) = T^*$ 的关节配置，可能多解、无解或处于奇异附近。数值 IK 还受初值、关节限位、碰撞和连续性约束影响。VLA 输出 7D 末端 action 仍需 IK/controller 才能变成关节命令，不能直接说成电机 torque。

机制展开（约 2 分钟）：FK 可写成 $T_E^W(q) = T_B^W \prod_i T_i(q_i)$，输入是关节配置和固定 link/工具外参，输出是某个 frame 下的末端位姿。IK 则求解 $q^* = \arg\min_q d(T_E^W(q), T^*)$，其中距离可同时包含位置、旋转、平滑、关节限位和碰撞 penalty；冗余机械臂自由度多于任务约束时会有多解，目标超出 workspace 或违反 limit 时无解。数值方法常用 Jacobian transpose、伪逆或阻尼最小二乘，且必须检查 residual、limit、collision 和 step continuity；找到一个 IK 解不代表整条轨迹动态可行。对于策略输出的末端 delta pose，要先按 frame 和 unit 累积/变换，再交给 IK、impedance 或其他 low-level controller；控制频率、插值、夹爪通道和安全限幅也会改变实际执行。面试时要把“policy action → controller input → joint command → simulator/robot step”接口说完整，不能把 7D/12D action 维度直接等同于执行器物理量。

项目挂钩：不要把 policy 输出的 7D action 直接说成电机 torque。

追问：为什么 redundant arm 的 IK 有无限多解？

易错：IK 成功不代表路径无碰撞或动态可行。

来源：[5][38][41]　**练习状态**：unseen

▼ L2 P0 VLA-FND-034 · Jacobian 如何连接关节速度和末端速度？奇异位形有什么影响?

30 秒回答：局部关系是 $\dot{x} = J(q)\dot{q}$，其中 Jacobian 把关节速度映射为末端 linear/angular velocity（twist），每一列对应一个关节的局部贡献。接近奇异位形时 rank 降低或条件数变差，某些末端方向不可控，伪逆会放大关节速度和噪声；可用阻尼伪逆、限速或改变姿态缓解。

机制展开（约 2 分钟）：对末端 pose 映射 $x = f(q)$，Jacobian 是 $J(q) = \partial x / \partial q$，速度控制常求 $\dot{q} = J^{\#}\dot{x}^{*}$。普通伪逆在最小二乘意义下追踪目标，但当最小奇异值接近 0 时，$J^{\#}$ 会把观测噪声、量化误差或 policy jerk 放大到关节速度；这不是单纯数值 bug，而是机构在该姿态的可控方向退化。阻尼最小二乘可写成 $\dot{q} = J^{\top}(JJ^{\top} + \lambda^{2}I)^{-1}\dot{x}^{*}$，通过 λ 换取少量 tracking bias 来降低速度爆炸；实际还要加入 joint-limit null-space、velocity/acceleration limit、碰撞约束和 controller rate。VLA action 的异常幅度可能来自模型，也可能来自 frame/normalization 错误、IK/Jacobian 放大或低层控制器，因此诊断要分别记录 raw action、denormalized action、IK residual、singular values、joint velocity 与最终 env step，不能只看 policy 输出。

项目挂钩：action jerk 或异常幅度可能来自模型，也可能来自 IK/Jacobian 放大。

追问：阻尼最小二乘为何更稳?

易错：奇异不是单纯数值 bug，而是机构在该姿态的可控方向退化。

来源：[37][41]　**练习状态**：unseen

▼ L1 P0 VLA-FND-035 · position、velocity、torque control 有什么区别?

30 秒回答：position control 给目标位置/角度，由底层回路产生速度或力矩；velocity control 给目标速度；torque control 直接给关节力矩，表达力强但对动力学模型、实时频率、接触稳定和安全限幅要求更高。VLA action 的 7D/14D 只是接口维度，必须说明最终 controller 语义和单位。

机制展开（约 2 分钟）：三者主要区别在控制层级、闭环带宽和对模型/安全的依赖：position/velocity 往往由成熟 servo 或 impedance loop 负责稳定性，policy 只需输出较高层 reference；torque control 让策略直接影响动力学，但噪声、延迟、负载变化和接触冲击会更直接地传到机器人。末端 delta pose 通常先在指定 frame 中反归一化，再经 IK/Jacobian 或 impedance controller 转成 joint target，最后由 position/velocity/torque loop 执行；同样的 7D 向量如果被解释为 pose delta、joint velocity 或 torque，系统行为完全不同。选型要看任务接触性、控制频率、动力学可信度、action horizon、sim2real gap 和 safety fallback；面试时应说明 saturate、rate limit、workspace/torque limit、E-stop 与 termination。OpenPiVLA 的 action contract 只有在与 LIBERO/ALOHA 的 controller、normalizer、gripper convention 和 step frequency 对齐后，才有可比的 offline/closed-loop 结果。

项目挂钩：OpenPiVLA 的 action contract 要与 LIBERO/ALOHA controller 匹配。

追问：末端 delta pose 最终如何变成关节命令？

易错：action 维度相同不代表控制语义相同。

来源：[9][35][36][37][41]　练习状态：unseen

▼ L1 P0 **VLA-FND-036 · PID 的 P/I/D 各解决什么问题？**

30 秒回答：P 对当前误差响应，增大通常更快但可能振荡；I 累积历史误差以消除稳态偏差，但会积分饱和；D 看误差变化趋势、提供阻尼，却容易放大测量噪声。实现必须同时处理采样周期、输出限幅、anti-windup、导数滤波和延迟。

机制展开（约 2 分钟）：离散 PID 可写成 $u_t = K_p e_t + K_i \sum_{k \le t} e_k \Delta t + K_d (e_t - e_{t-1})/\Delta t$，其中 $e_t = r_t - y_t$。P 主要决定局部刚度/响应速度；I 抵消持续偏差，但当输出饱和时继续累积会产生 windup，解除饱和后出现大 overshoot；D 提前感知变化、增加阻尼，但对高频 measurement noise 和 $1/\Delta t$ 敏感，工程上常对 measurement 求导并加低通滤波，避免 setpoint jump 造成 derivative kick。anti-windup 可用 conditional integration（饱和且误差继续推向饱和时暂停积分）或 back-calculation，把 $u_{sat} - u_{raw}$ 反馈到积分状态；还要记录积分限幅、输出 rate limit、单位、采样时间和 reset/termination 时的状态清理。闭环抖动不能只归因于 policy：action chunk、server latency、控制频率、normalizer、接触和 PID 参数都会改变误差序列，因此应分别记录 raw/commanded action、controller output、saturation、tracking error 与 timestamp。

项目挂钩：闭环轨迹抖动不一定是模型错误，也可能是 controller 参数和延迟问题。

追问：为什么 D 项常对测量值而不是误差求导？

易错：PID 不是“调大三个系数直到效果好”。

来源：[36]　练习状态：unseen

▼ L2 P0 **VLA-FND-037 · 控制频率、推理延迟和 action chunk 如何共同影响稳定性？**

30 秒回答：控制频率决定动作多久更新一次，推理/通信延迟会让策略基于过时 observation；action chunk 减少模型查询但执行越长越接近 open loop。短 replan 纠偏及时却更耗算力和带宽，长 chunk 更高效但误差积累更大。评估必须同时报告 policy query rate、controller rate、端到端 latency、预测 horizon 和实际执行前缀。

机制展开（约 2 分钟）：设 controller period 为 Δt_c，policy query period 为 Δt_p，端到端延迟为 τ，则 observation 到 action 生效期间系统可能已经执行了约 $\tau/\Delta t_c$ 个旧动作；chunk 长度 H 还决定了两次 replan 之间的最大 open-loop 步数。若 τ 接近或超过控制周期，直接同步调用会阻塞控制 loop；常见设计是 observation buffer + inference worker + timestamped action queue，控制线程按 deadline 取最新可用前缀，过期或缺失时触发 hold/zero/fallback，而不是

无限等待。chunk 执行可以用 early termination、receding-horizon overlap 或 uncertainty-triggered replan 折中；但 action overlap、队列丢弃、server retry、clock drift 和 reset/termination 语义必须记录。模型预测 horizon 不等于 controller 实际执行步数，尤其在 OpenPiVLA/UWM 的 action window、warmup、denormalization 和 controller slicing 之间。面试时要把 query rate、controller rate、latency percentiles、queue depth、executed prefix、termination 与 success predicate 放在同一 closed-loop protocol 中比较。

项目挂钩：OpenPiVLA 的 action window 与 UWM warmup/rollout protocol 都属于闭环合同。

追问：延迟大于控制周期时如何设计异步 pipeline?

易错：模型预测 horizon 不等于控制器一次实际执行的步数。

来源：[9][10][25][36][41][43]　**练习状态**：unseen

▼ `L2` `P0` **VLA-FND-038 · denormalization、clipping 和 gripper 维度为什么会造成系统性失败?**

30 秒回答：policy 输出通常仍在模型的 normalized space；执行前必须用训练时完全相同的方法、逐维统计量和维度顺序恢复 controller unit。normalizer 版本、axis/frame/unit 或 gripper 开闭编码错，会在每一步制造同方向的 scale/bias；clipping 虽能把数值压回合法范围，却可能让大量维度饱和并掩盖合同错误。诊断要做 action round-trip、逐维 sentinel、normalizer checksum，并同时记录 clipping rate 与最终 controller command。

机制展开（约 2 分钟）：对逐维 affine normalizer，可写成 $z_i = (a_i - c_i)/s_i$、$a_i = s_i z_i + c_i$；这里 z_i 是模型空间 action，a_i 是物理/控制器空间 action，c_i, s_i 可以来自 mean/std、quantile range 或其他训练时冻结的统计协议。公式可逆不代表实现一定正确：checkpoint 必须绑定 normalizer method/revision、dimension layout、absolute/delta 语义、frame、unit、valid dims 与 gripper convention。若 x/y 轴互换、角度 rad/deg 混用、不同数据集统计量复用，或者把 gripper 的 `[-1,1]`、`[0,1]`、开合宽度、binary open/close 符号混写，shape 检查仍可能通过，但闭环会出现稳定的方向、尺度或夹爪状态错误。随后 $a'_i = \mathrm{clip}(a_i, l_i, u_i)$ 只能提供 safety saturation，不能修复 calibration；高 saturation ratio、长期贴边或 clipping 前后差值很大反而是 normalizer/schema 错配信号。定位时先对数据集 action 做 `denormalize(normalize(a))` round-trip，再让每个维度依次通过 one-hot/sentinel，核对 axis、sign、unit 与 gripper；然后用记录的 expert action 绕过 policy 走同一 adapter/controller，最后比较 raw model action、denormalized action、clipped action、controller command 和实际 state delta。这样才能区分模型预测错误、action adapter bug 与 controller/environment 问题。现有 OpenPiVLA inventory 仍把 axis/unit/normalizer/parser 标为 semantics-open，UWM inventory 也把 normalizer checksum 和 RoboCasa action order 列为缺口，因此可以把它们讲成评估前置 gate 和诊断风险，不能直接宣称已经证明某次失败的唯一根因。

项目挂钩：OpenPiVLA/UWM 的 normalizer、action order 与 gripper contract 是评估前置条件；当前证据只支持合同审计和风险定位，不支持把 normalizer 错配写成已闭合 root cause。

追问：怎样区分模型失败与 denormalization bug?

易错：动作“在范围内”不代表语义和轴顺序正确。

来源：[9][10][35][41][43]　练习状态：unseen

E. 补齐基础与研究深度

▼ L1　P0　**VLA-FND-005 · 期望、方差、协方差和相关系数有什么区别?**

30 秒回答：期望 $\mathbb{E}[X]$ 描述随机变量的平均位置；方差 $\mathrm{Var}(X)$ 描述单变量围绕均值的平方波动；协方差 $\mathrm{Cov}(X,Y)$ 描述两个变量的线性共同变化，但受量纲影响；相关系数 ρ_{XY} 再除以两边标准差，通常落在 $[-1,1]$。在机器人 action 统计里必须逐维保留单位、valid mask 和数据切分；零相关不等于独立，相关也不代表因果。

机制展开（约 2 分钟）：令 $\mu_X=\mathbb{E}[X]$，则 $\mathrm{Var}(X)=\mathbb{E}[(X-\mu_X)^2]=\mathbb{E}[X^2]-\mu_X^2$，$\mathrm{Cov}(X,Y)=\mathbb{E}[(X-\mu_X)(Y-\mu_Y)]$，而 $\rho_{XY}=\mathrm{Cov}(X,Y)/(\sigma_X\sigma_Y)$。方差有平方量纲，协方差的量纲是两变量单位之积，所以位置、旋转和 gripper 维度不能靠原始数值大小直接比较；相关系数消除了尺度，却只刻画线性关系，而且当某一维 $\sigma=0$ 时没有定义。对随机向量 a，协方差矩阵 $\Sigma=\mathbb{E}[(a-\mu)(a-\mu)^\top]$ 必须半正定，因为任意向量 v 都满足 $v^\top\Sigma v=\mathrm{Var}(v^\top a)\geq 0$。工程上要说明统计量来自 train split，按 action layout 与任务/机器人域计算，并正确处理 padding、invalid dimensions、episode 长度和时间相关性；样本方差采用 n 还是 $n-1$ 也要与实现一致。逐维 normalizer 通常只使用边缘均值/尺度，不会自动消除维度间相关性；若混入不同 frame、unit、absolute/delta 语义或 gripper 编码，得到的均值和协方差即使数值稳定也没有统一物理含义。OpenPiVLA/UWM 当前 inventory 只支持把 normalizer revision、action order、valid mask 和 train/eval split 当作审计项，尚无证据支持声称已得到某组个人项目 action 统计或由它闭合了失败根因。

项目挂钩：OpenPiVLA/UWM 的 action normalization 应按维度绑定 layout、unit、mask 与统计版本；当前证据只支持合同审计，不支持把某个统计异常写成已验证 root cause。

追问：为什么协方差矩阵必须半正定?

易错：相关系数接近 0 只说明线性相关弱，不代表完全独立。

来源：[27][41][43]　练习状态：unseen

▼ L2 P0 VLA-FND-007 · KL divergence 的方向为什么重要?

30 秒回答：$D_{KL}(P\|Q) = \mathbb{E}_{x\sim P}[\log P(x) - \log Q(x)]$ 由 P 决定期望区域；Q 若漏掉 P 有质量的 mode 会被强罚，所以近似优化常更偏 mode-covering。反向 $D_{KL}(Q\|P)$ 只在 Q 自己覆盖的区域取期望，更可能选中 P 的一个 mode。它不对称、没有三角不等式，不是普通距离；代码里必须说明谁是 target、谁是 student、输入是 logits/log-prob/prob、mask 和 reduction。

机制展开（约 2 分钟）：KL 的定义是 $D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0$，等号只在几乎处处 $P = Q$ 时成立。方向重要的根源是权重分布不同：forward KL 用 P 加权，若 $P(x) > 0$ 而 $Q(x) \to 0$，代价趋于无穷；reverse KL 用 Q 加权，Q 没覆盖的 P mode 几乎不进入期望，但 Q 若把质量放到 $P(x) = 0$ 的区域同样会发散。“mode-covering/mode-seeking”是多峰近似族受限时的常见行为，不是无条件定理；若模型族足以精确表示 P，两种方向的最优解都可为 P。实现层面要先对齐同一 token/动作 support：teacher logits z_T、student logits z_S 经 log-softmax 得到 $\log p_T, \log p_S$，teacher-to-student 蒸馏常计算 $\sum_v p_T(v)(\log p_T(v) - \log p_S(v))$。PyTorch `KLDivLoss` 默认把 `input` 解释为 log-prob、`target` 解释为 probability；`log_target=True` 时 target 才是 log-prob，且 `batchmean` 才与“按 batch 求和再除 batch size”的 KL 定义对齐。token mask、temperature、vocab revision、padding 和 denominator 任一不一致，都可能让“方向正确”的公式落到错误实现。

项目挂钩：OPD formal `point/train_opd_online_vl.py` 计算 $D_{KL}(q_\beta\|p_S)$：veto beta=0 时为 teacher // student，beta>0 时 target logits 还加入 detached student logits；根目录同名 legacy trainer 则是 teacher top1 hard-label CE。两份实现必须按完整路径和 run summary 分开，详见 OPD Training Objective Lineage Gate。

追问：为什么 KL 不满足对称性和三角不等式？`KLDivLoss(mean)` 与 `batchmean` 有什么差别？为什么 mode-covering/mode-seeking 只能作为近似族受限时的经验描述?

易错：把 forward/reverse 名称记反；把 logits 直接传给 `KLDivLoss`；忽略 target 是否为 log；把 mode-covering/mode-seeking 说成无条件定理；把 KL 当作 metric。

来源：[14][31][44] **练习状态**：unseen

▼ L2 P0 VLA-FND-008 · bias-variance trade-off 如何影响策略学习和评估?

30 秒回答：bias 是学习器平均预测相对真实目标的系统性误差，variance 是训练数据、初始化或优化随机性变化时模型输出的波动。更大模型通常能降低近似 bias，但不保证只降低 bias，也可能放大估计方差和优化不稳定。机器人评估还要把 training-seed variance、task/init 差异和固定 policy 的 episode 二项噪声分开；多跑同一 checkpoint 只能估计后者，不能替代多个独立训练 run。

机制展开（约 2 分钟）：在平方误差和固定输入 x 下，若训练集为随机变量 $\mathcal{D}$，经典分解为

$$\mathbb{E}_{\mathcal{D},\epsilon}\left[(y-\hat{f}_{\mathcal{D}}(x))^2\right]=\sigma_\epsilon^2+\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]-f(x)\right)^2+\mathrm{Var}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)].$$

第一项是不可约噪声，第二项是 squared bias，第三项是 estimator variance。它依赖损失、数据生成过程和模型族，不能把训练误差直接叫 bias、测试误差直接叫 variance。正则、更强先验、数据增强或早停常以增加部分 bias 换取更低 variance；更多独立覆盖通常降低估计方差；增大模型容量可能降低 approximation bias，但有限数据下的方差、优化路径和分布外行为未必单调。对 VLA policy，还要把 action-supervision bias、dataset coverage、closed-loop compounding error 和 evaluator noise 分层。固定 checkpoint、task/init protocol 下，二元 success 的条件方差约为 $p(1-p)/n$；它只描述 episode sampling，不包含换训练 seed 后 policy 本身的变化。可信比较应保存 training/data/env/rollout/bootstrap seed，先在同 task/init 上做 paired episode 比较，再把独立 training run 作为顶层 replicate，报告逐任务分布和区间，而不是把同一 checkpoint 的几百个 episode 当成几百个训练样本。

项目挂钩：UWM 的 `186/250` 是用户确认的 fixed-pruned-init summary，五个 task 使用 task-specific checkpoints，本地缺多 training-seed panel 和 250 条 raw episode，因此既不能估 training variance，也不能把它与论文三 seed/OOD protocol直接比较。WAM full10k 与 official-HF path 的 checkpoint/config lineage也未做 matched join，`0/118` 与 `84/118` 不能解释为 bias–variance 分解或训练因果效果。

追问：增加模型规模通常只会降低 bias 吗？同一 checkpoint 跑 250 个 episode 为什么不能估计 training-seed variance？paired init 能消除哪些方差、不能消除哪些？

易错：bias–variance 不是“训练误差 vs 测试误差”的同义词；episode 数不等于独立 training runs；不同协议的百分比不能拿来估计方差或 bias。

来源：[4][27][42][43][46]　**练习状态**：unseen

▼ L1 P0 VLA-FND-009 · 为什么深度网络需要非线性激活？GELU 和 SiLU 有何特点？

30 秒回答：连续仿射层之间若没有非线性，任意深度都可合并为一个仿射变换，深度不会增加这部分网络的函数表达力。GELU 是 $x\Phi(x)$ 的高斯 CDF 自门控，SiLU 是 $x\sigma(x)$ 的 sigmoid 自门控；二者都平滑、保留小负值，但不存在脱离架构与训练协议的必然优劣，微调预训练模型时应服从原 checkpoint/config。

机制展开（约 2 分钟）：若每层只有 $h_{k+1}=W_kh_k+b_k$，则多层复合仍满足

$$h_L=W_{\text{eff}}x+b_{\text{eff}},$$

其中 W_{eff} 是各层权重的连乘，偏置也能合并。逐元素非线性 ϕ 把下一层变成 $h_{k+1} = \phi(W_k h_k + b_k)$，从而产生随输入变化的门控和曲率，网络才可组合出复杂分段或平滑决策边界。这里要限定口径：Transformer 即使拿掉 MLP 激活，attention softmax 和 normalization 仍可能提供非线性；“退化为单层”说的是纯仿射链，不是所有现代网络的完整计算图。

两种平滑激活及其梯度可写为

$$\begin{aligned} \mathrm{GELU}(x) &= x\Phi(x), & \mathrm{GELU}'(x) &= \Phi(x) + x\varphi(x), \\ \mathrm{SiLU}(x) &= x\sigma(x), & \mathrm{SiLU}'(x) &= \sigma(x) + x\sigma(x)(1-\sigma(x)). \end{aligned}$$

这里 Φ 和 φ 分别是标准高斯 CDF 与 PDF，σ 是 logistic sigmoid。GELU 常用 exact 或 tanh approximation；SiLU 也称 Swish（无可学习缩放的常用形式）。二者在负半轴允许有限负输出，零点附近梯度连续，不像 ReLU 在全部负输入上梯度恒为 0；但它们在很负的区域梯度仍会趋近 0，所以“平滑”不等于消除梯度消失。SwiGLU 则是带两路线性投影和乘法门的 MLP 结构，不能与单个 SiLU 激活混为一谈。

项目挂钩：本地 `Project` 快照中，`uwm_robocasa_transfer/uwm_robocasa_train.py` 在 state/right-view residual projection 和 phase/control/contact/progress auxiliary heads 使用 `nn.SiLU()`；`cosmos_policy_deepspeed_train.py` 也使用 `nn.SiLU()`，但类注释明确它只是 objective-compatible DeepSpeed proxy，不是完整 Cosmos Policy。面试时可说“我从实现核对了新增模块/代理模型的激活与 checkpoint compatibility”，不能说完整上游 backbone 都采用 SiLU，也不能把某次训练或闭环结果因果归因于它。[49]

追问：ReLU 的 dying neuron 如何出现？为什么不能在加载预训练权重后随意把 GELU 换成 SiLU？

易错：GELU/SiLU 平滑不等于一定收敛更快或永不饱和；SiLU 与 SwiGLU 不是同一层级的概念；不要由局部 adapter/proxy 实现外推完整 backbone。

来源：[15][16][27][47][48][49]　练习状态：unseen

▼ `L2` `P0` VLA-FND-014 · 位置编码解决什么问题？RoPE 如何提供相对位置信息？

30 秒回答：不含位置相关 mask/encoding 的 content-only self-attention 对 token 置换是等变的，能看内容关系却不知道顺序和距离。RoPE 按位置把 query/key 的每个二维通道对旋转；因为 $R_m^\top R_n = R_{n-m}$，旋转后的 QK 内积自然依赖相对位移 $n-m$。它不是简单相加绝对位置，也不自动保证长上下文外推。

机制展开（约 2 分钟）：对没有位置相关 mask 的 self-attention，若 P 是 token 维的置换矩阵，则

$$\mathrm{SA}(PX) = P\,\mathrm{SA}(X).$$

这表示输入 token 怎样重排，输出只会跟着同样重排；模型没有单独的“第几个 token”信号。causal mask 本身会引入前后可见性的偏序，但仍不等于给出了可泛化的相对距离表示。位置编码的作用，是把序号、二维 patch 坐标或其他位置结构注入 attention 计算。

RoPE 对 head dimension 中第 j 个二维通道对使用位置 m 与频率 θ_j 形成旋转：

$$R(m\theta_j) = \begin{bmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{bmatrix}, \qquad \tilde{q}_m^{(j)} = R(m\theta_j) q_m^{(j)}, \quad \tilde{k}_n^{(j)} = R(n\theta_j) k_n^{(j)}.$$

其中 m, n 是 position id，θ_j 是第 j 个频率。旋转不改变 Q/K tensor shape；关键是正交旋转满足

$$\left(\tilde{q}_m^{(j)}\right)^\top \tilde{k}_n^{(j)} = \left(q_m^{(j)}\right)^\top R((n-m)\theta_j) k_n^{(j)},$$

所以 attention logit 同时保留内容相似度并显式依赖相对位移。标准 RoPE 旋转 Q/K，是因为它们决定 attention routing；V 是按这些权重聚合的内容。旋转 V 并非数学上绝对禁止，但会把内容放进位置相关坐标系，需要额外一致性设计，不能当作标准 RoPE 的默认做法。

实现时还要核对 `position_ids`、padding/packing 重置规则、head dimension、频率表、最大训练长度与 scaling recipe。长上下文失败可能来自未见过的相位组合、频率分布或 scaling 不匹配；RoPE 的相对内积形式不等于无限外推保证。对 VLA，多模态 token 序列中的 image patch、instruction 和 action token 都有序，但 token position 不是机器人 timestamp、控制频率或传感器同步；二维/时间位置还需要模型自己的映射规则。

项目挂钩：本地 `Project` HEAD `6c524d9` 的 Cosmos/WAM Transformer Engine shim 用 `_rotate_half` 和 `tensor * cos + rotate_half(tensor) * sin` 应用预计算 RoPE，并处理 `bshd/sbhd` layout 与维度检查；这能支持“我核对了旋转 kernel 的 tensor contract”。但 shim 不生成频率，也未证明完整模型的 Q/K call site、长上下文能力或指标效果。OpenPiVLA formal path 只确认加载 Qwen3-VL，本地没有冻结 base `config.json` 的 RoPE 参数，因此不能宣称 exact multimodal RoPE 配置。[41][42][50]

追问：为什么标准 RoPE 主要旋转 Q/K 而不直接旋转 V？packed sequence 的 `position_ids` 若不重置会发生什么？一维 token position 为什么不能替代图像二维坐标或机器人 timestamp？

易错：self-attention 是置换等变而非简单“置换不变”；相对位置进入 QK 内积不代表模型只含相对信息；RoPE 不等于无限长外推，也不能完成传感器时间对齐。

来源：[1][17][41][42][50]　练习状态：unseen

▼ L2　P0　**VLA-FND-015 · pre-norm 和 post-norm 的训练差异是什么？**

30 秒回答：pre-norm 写成 $x_{l+1} = x_l + F_l(N(x_l))$，归一化在子层前，残差主干保留显式 identity path，深层优化通常更稳；post-norm 是 $x_{l+1} = N(x_l + F_l(x_l))$，每层输出被重新归一化，但反向梯度必须经过 norm Jacobian，对初始化、学习率和 warmup 往往更敏感。它讨论的是 norm 相对子层/残差的位置，不是 LayerNorm 与 RMSNorm 的区别。

机制展开（约 2 分钟）：令 F_l 表示 attention 或 MLP 子层，N 表示 LayerNorm/RMSNorm 一类归一化，则两种残差块可抽象为

$$x_{l+1}^{\text{pre}} = x_l + F_l(N(x_l)),$$
$$x_{l+1}^{\text{post}} = N(x_l + F_l(x_l)).$$

对应的局部 Jacobian 结构是

$$\frac{\partial x_{l+1}^{\text{pre}}}{\partial x_l} = I + J_{F_l} J_N, \qquad \frac{\partial x_{l+1}^{\text{post}}}{\partial x_l} = J_N(I + J_{F_l}).$$

pre-norm 中的 I 给残差流提供不经过 norm 的直接梯度项；post-norm 则在每个 block 输出端乘上 J_N，深层连乘更依赖初始化和优化 recipe。Xiong 等人的理论与实验结论是在其假设和设置下：Post-LN 初始化时靠近输出层的期望梯度可能较大，warmup 有助于稳定；Pre-LN 梯度更平衡，并可在相应设置下减少 warmup 依赖。面试时不能把它升级成“Pre-LN 永远不需要 warmup”。[32]

代价也要讲清：pre-norm 的 raw residual stream 不在每个 block 末端重新归一化，x_l 会累加各层增量，尺度可能随深度增长，因此通常还要 final norm、合理初始化或 residual scaling；这不等于必然发散。post-norm 每层输出尺度受控，但优化可能更敏感。最终收敛质量还取决于深度、初始化、残差缩放、optimizer、学习率、warmup、精度和数据，不能只靠 norm 位置排序。

项目挂钩：OpenPiVLA formal trainer 直接 `from_pretrained` 加载 `Qwen3VLForConditionalGeneration`，LoRA 默认只命中 `q_proj,k_proj,v_proj,o_proj`，没有重写 decoder norm/residual 顺序。与本地环境记录的 Transformers `4.57.1` 对应的官方 Qwen3-VL 代码中，`input_layernorm → self_attn → residual add`，再 `post_attention_layernorm → MLP → residual add`，所以两个子层都是 pre-norm；变量名 `post_attention_layernorm` 只是“位于 attention 之后”，因为它在 MLP 之前，不能误判为 Post-LN。当前缺远端 runtime source

hash 与 base config snapshot，因此这是 version-matched official-code mapping，不是运行时 binary 证明，也不能把上游结构说成个人设计。[41][51]

Cosmos/WAM 本地 `RMSNorm` compatibility shim 只实现均方根归一化算子，没有 frozen full-block call site；它能证明 norm operator contract，不能证明完整 Cosmos block 是 pre-norm 还是 post-norm，更不能解释训练指标。[42][51]

追问：为什么 `post_attention_layernorm` 仍可属于 pre-norm？pre-norm residual stream 的尺度为什么可能增长？能否把预训练 Post-LN checkpoint 直接改成 Pre-LN 后继续加载？

易错：pre/post 说的是 norm 相对子层与 residual add 的位置；不要与 LayerNorm/RMSNorm 类型混淆，也不要把“通常更稳”说成“必然更优或无需 warmup”。

来源：[1][2][3][32][41][42][51]　**练习状态**：unseen

▼ L2 P0 VLA-FND-017 · LoRA 为什么能减少可训练参数？低秩假设的代价是什么？

30 秒回答：LoRA 冻结 $W_0 \in \mathbb{R}^{d_{out}\times d_{in}}$，只学习 BA 的 rank-r 更新，使可训练参数从 $d_{out}d_{in}$ 降到 $r(d_{in}+d_{out})$。它主要节省参数梯度和 optimizer states，不会让 base weights 或 activation 自动消失。代价是更新被限制在低秩子空间；rank、target modules、alpha、dropout 和数据若不匹配，会欠拟合或漏掉关键适配方向。

机制展开（约 2 分钟）：标准 LoRA 线性层可写成

$$
\begin{aligned}
y &= W_0x + sBAx, \qquad s=\frac{\alpha}{r},\\
A &\in \mathbb{R}^{r\times d_{in}}, \qquad B\in\mathbb{R}^{d_{out}\times r}, \qquad r \ll \min(d_{in},d_{out}),\\
N_{\text{LoRA}} &= r(d_{in}+d_{out}) \ll d_{out}d_{in}.
\end{aligned}
$$

这里 W_0 冻结，A,B 和允许训练的其他参数进入 optimizer；α/r 是常见 scaling，具体实现还要检查是否启用 rsLoRA 等变体。LoRA 的效率来自“不为整块 W_0 保存 weight gradient 和 Adam 一、二阶状态”，不是不经过 base network：前向仍需要 base weights，反向仍要穿过冻结层计算 adapter 梯度，activation memory 也受 batch、sequence、checkpointing 和精度影响。QLoRA 的 base quantization 是另一项机制，不能与普通 LoRA 混为一谈。

低秩假设是“任务所需的有效权重增量近似落在低维子空间”。若真实更新的有效 rank 大于 r，或者只把 adapter 放进 Q/K/V/O 而任务主要需要 MLP、vision encoder、cross-modal connector 或 action head 改变，LoRA 会形成容量瓶颈。提高 rank 或扩大 target modules 能增强表达，但会增加 trainable params、optimizer memory、adapter storage 和过拟合风险；α 改变更新尺度，dropout 改变训练时支路，不能只报一个 rank 就说 recipe 相同。

推理侧也要分清：未 merge 时需要额外计算 BAx，可能有 kernel/serving 开销；merge 后用 $W' = W_0 + sBA$ 可消除额外支路，但 dense weight 的形状、基础参数规模与主要 dense FLOPs 没变。LoRA 的优势是训练和多 adapter 存储/切换，不是天然模型压缩。

项目挂钩：当前 OpenPiVLA formal trainer 使用 PEFT `LoraConfig(r=16, alpha=32, dropout=0.05, bias="none")`，默认 target 为 `q_proj,k_proj,v_proj,o_proj`；代码按 `requires_grad` 统计并打印 trainable/total params。committed report 记录 `6,422,528 / 2,133,954,560 ≈ 0.301%`，但 raw start log、adapter binary/hash 与 fresh-load smoke 未同步，所以该数字只能说 `report-supported`，不能说本地已由 checkpoint 复算。更关键的是归档目标 YAML 为 `lora: False`，其中 `lora_r: 32` 只是未启用字段；当前 sanitized LoRA formal path 不能倒推成历史原 run 也使用 LoRA。[41][52]

面试 ownership 只能说“核对/接入了 Qwen3-VL + PEFT LoRA formal path、target modules 和参数账本”，不能把 LoRA 算法或 Qwen upstream architecture 说成个人创新，也不能把 6.42M 参数直接等同于 benchmark 效果。[41][52]

追问：为什么 LoRA 明显减少 optimizer memory，却不按相同比例减少总训练显存？只在 attention projection 注入可能漏掉什么？merge 后为什么不等于模型被压缩？如何证明一个 adapter checkpoint 可独立加载？

易错：可训练参数少不等于 base parameters、activation、推理 FLOPs 同比例下降；配置中出现 `lora_r` 不等于 LoRA 已启用；不要把当前 adapter recipe 归属于 lineage 未闭合的历史 run。

来源：[18][41][51][52]　**练习状态**：unseen

▼ L2 P0 VLA-FND-018 · KV cache 为什么能加速自回归推理？

30 秒回答：自回归第 t 步只新增一个 token，历史 token 在各层的 K/V 不再变化；KV cache 保存它们，新步只算当前 token 的 Q/K/V，再让 q_t 关注缓存前缀，避免每一步重跑整个 prefix。它主要加速 decode，不消除串行 token 数；代价是 cache 显存随 layer、batch、KV heads、上下文长度和 dtype 线性增长，长上下文/高并发时可能由 compute-bound 变成 memory-bound。

机制展开（约 2 分钟）：对 decoder 第 l 层的第 t 个 token，cache 更新和 attention 可写为

$$
\begin{aligned}
K_{1:t}^{(l)} &= \mathrm{concat}\left(K_{1:t-1,\mathrm{cache}}^{(l)}, k_t^{(l)}\right),\\
V_{1:t}^{(l)} &= \mathrm{concat}\left(V_{1:t-1,\mathrm{cache}}^{(l)}, v_t^{(l)}\right),\\
o_t^{(l)} &= \mathrm{softmax}\left(\frac{q_t^{(l)} K_{1:t}^{(l)\top}}{\sqrt{d_h}} + M_t\right) V_{1:t}^{(l)}.
\end{aligned}
$$

这里 M_t 是 causal/padding mask，d_h 是 head dimension。没有 cache 时，每个 decode step 会再次计算历史 token 的 layer states 和 K/V projection；有 cache 时历史 K/V 直接复用。但当前 query 仍要与长度 t 的 K 做 attention，所以单步 attention 成本仍随上下文增长；生成 T 个 token 仍有 T 次串行 forward，KV cache 不会把 autoregressive decoding 变成一次并行 action head。

常规 dense cache 的近似字节数为

$$M_{\mathrm{KV}} \approx 2LBTH_{\mathrm{kv}}d_h\,b,$$

其中系数 2 对应 K 和 V，L 是 decoder layers，B 是含 beam/并发展开后的 batch，T 是已缓存序列长度，H_{kv} 是 KV heads，b 是每元素字节数。GQA/MQA 通过减少 H_{kv} 降低 cache；quantized/offloaded/sliding-window cache 则以精度、带宽或有效上下文换显存。cache 必须和相同模型权重、prefix、position ids、mask 与 batch reorder 对齐，否则复用会产生错误结果。

prefill 与 decode 也要分开：prefill 仍要并行处理完整 prompt 并建立首份 cache；cache 的主要收益出现在后续逐 token decode。teacher-forcing 训练一次并行计算整段 labels，没有跨 decode step 的前缀复用，保留 cache 反而增加状态/显存并常与 gradient checkpointing 冲突，所以训练代码通常关闭 `use_cache`。encoder-only 一次处理完整输入，也没有同样的 decoder self-attention 逐步追加场景；encoder-decoder 可以另行缓存 encoder/cross-attention K/V，概念相近但生命周期不同。

项目挂钩：OpenPiVLA formal trainer 在加载 Qwen3-VL 后显式 `model.config.use_cache=False`，这属于 full-chat causal-CE 训练合同；formal package 又缺 generation/parser/controller，所以当前没有 formal action-token decode latency 可以声称。OPD raw-holdout eval 则显式设置 `model.config.use_cache=True`，并调用 `generate(..., use_cache=True)`；这能证明 eval generation 请求了 cache，但没有 cache-on/off matched benchmark、peak-memory trace 或机器人控制延迟，因此不能量化加速。OPD full-vocab teacher/student loss forward 仍显式 `use_cache=False`。[41][44][53]

对 VLA 还要先判断 action 输出范式：离散 action text/token 的逐 token generation 可受益于 KV cache；一次并行连续 action head、diffusion denoising 或已生成的 action chunk 不会因为“有 KV cache”就减少自身 denoising/控制步数。[41][42][43]

追问：为什么 KV cache 降低重复计算却可能让 decode 更 memory-bound？GQA/MQA 如何改变 cache 大小？training 为什么通常关闭 cache？encoder-decoder 的 cross-attention cache 与 decoder self-attention cache 有何不同？

易错：cache 不是把生成并行化，也不消除当前 token 对全部历史 K/V 的读取；`use_cache=True` 只能证明请求了功能，不能替代 on/off latency、显存和输出一致性实验。

来源：[19][41][42][43][44][51][53]　练习状态：unseen

▼ L3 P0 VLA-FND-019 · 梯度消失/爆炸如何定位?

30 秒回答：我不会从最终 loss 直接猜“梯度爆炸”。先冻结代码、checkpoint、数据顺序、精度和第一个异常 step，用同一 batch 重放；在 optimizer step 前，FP16 要先 unscale，再同时看 loss、scaler 是否跳步、全局及逐层 grad norm、参数/梯度非有限值。爆炸表现为 norm 突增或 NaN/Inf，消失表现为早层长期接近零，但还要排除空 mask、错误分母、detach、学习率/调度器、累积与 `zero_grad` 顺序。最后用 FP32/小 batch 重放定位；clipping 只能止血，不能代替根因修复。

机制展开（约 2 分钟）：深层网络的反向传播是 Jacobian 连乘。若 $h_{k+1} = f_k(h_k)$，则

$$\frac{\partial \mathcal{L}}{\partial h_l} = \left(\prod_{k=l}^{L-1} J_k^\top \right) \frac{\partial \mathcal{L}}{\partial h_L}, \qquad J_k = \frac{\partial h_{k+1}}{\partial h_k}.$$

这里 J_k 是第 k 层局部 Jacobian；连乘的有效奇异值长期小于 1 会让早层梯度衰减，长期大于 1 会放大梯度。残差路径、归一化、激活饱和、初始化和序列长度都会改变这条链，所以“模型很深”只是风险因素，不是根因证据。

我的定位顺序是：第一，固定 run identity，并找到 first bad step，而不是只比较两个 run 的最终 loss。第二，在 backward 后、optimizer step 前记录未裁剪的 global norm；混合精度若用了 `GradScaler`，必须先 `unscale_(optimizer)`，否则看到的是放大后的梯度。第三，把统计下钻到 trainable parameter group/layer：记录 grad L2、max、p99、零梯度比例，并配对看 activation、weight、optimizer state 是否 finite；global norm 正常也可能掩盖某个早层长期接近零。第四，检查 loss contract 和数据：有效 token/action mask 分母是否为零、label 是否全 ignore、异常图像/动作是否进入 batch、loss reduction 是否随序列长度突变，以及梯度累积时是否过早 `zero_grad`、少除或多除 accumulation steps。第五，保存首个坏 batch，用 FP32、batch=1、关闭可选 head/AMP 的 matched replay 二分定位；若 FP32 正常而低精度失败，才把精度路径升级为强嫌疑。

全局范数裁剪可写为

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2 + \epsilon}\right),$$

其中 g 是所有待更新参数梯度拼接后的向量，τ 是阈值。它保留方向并限制范数；PyTorch `clip_grad_norm_` 的返回值是裁剪前 total norm，因此适合记录，但要注明统计时点。持续触发 clipping 说明优化仍在撞阈值，后续应通过 matched rerun 检查学习率/warmup、loss 尺度、数据 outlier、残差/归一化或初始化，而不能把“训练没 NaN”写成根因已解决。[27][32][33][34]

项目挂钩：OpenPiVLA formal trainer 对 `requires_grad` 参数在 accumulation 边界调用 `clip_grad_norm_`，随后记录其返回的 `grad_norm`，再 `optimizer.step()` 与 `zero_grad()`；默认阈值是 1.0，但代码没有逐层 activation/grad trace，也没有显式 non-finite gradient gate，所以只能证明“有全局止血与稀疏日志”，不能证明某次 run 没有梯度异常。UWM/LIBERO 与 UWM/RoboCasa 的本地非 DeepSpeed AMP 路径都先 `scaler.unscale_` 再裁剪，符合统计/裁剪顺序，但返回 norm 被丢弃；RoboCasa 的 DeepSpeed 分支绕过这段本地 clip，实际行为还要对账当次 DeepSpeed config。formal OPD 明确拒绝 non-finite loss，TrainingArguments 传入 `max_grad_norm`，回调检查已记录的 loss/grad_norm/lr；这仍不等于 layer-wise 根因定位。当前没有可复算的 matched bad-step bundle，因此 WAM step-1500 save API failure、UWM DataLoader stall 或闭环 eval 失败都不能反推为梯度爆炸。[41][43][44][54]

追问：`clip_grad_norm_` 返回的是裁剪前还是裁剪后 norm？为什么 FP16 要先 unscale 再裁剪？global norm 正常但 loss 不降，还要查哪些 trainable-parameter、mask/target、optimizer/scheduler 和数据问题？如何构造 first-bad-batch 的 matched replay？

易错：clipping 是 containment，不是 diagnosis；loss spike、OOM、DataLoader stall、checkpoint 保存异常或 online success 低都不能单独证明梯度爆炸。只有日志里出现 `grad_norm` 字段，也不等于已经记录了逐层、未裁剪、正确 unscale 时点的梯度证据。

来源：[27][32][33][34][41][43][44][54]　练习状态：unseen

▼ L3 P0 VLA-FND-020 · 多任务共享 backbone 时如何判断梯度冲突？

30 秒回答：我会在同一个参数快照上，把每个任务对共享参数的梯度单独算出来，再看 pairwise cosine、梯度范数比例和各任务验证指标。余弦为负表示一阶局部冲突：沿任务 i 的下降方向会抬高任务 j；但零梯度时 cosine 无定义，正余弦也可能被数量级失衡掩盖。mask、route 或独立 head 只隔离样本/输出路径；只要 backbone 仍被多个任务更新，就要用 matched task batches 实测，不能从 loss 大小或指标涨跌直接下结论。

机制展开（约 2 分钟）：只在真正共享的参数 θ_s 上定义任务梯度 $g_i = \nabla_{\theta_s}\mathcal{L}_i$。同一参数快照、同一 train/eval mode 和固定随机性下，pairwise 诊断为

$$c_{ij} = \frac{g_i^\top g_j}{\|g_i\|_2 \|g_j\|_2}, \qquad \mathcal{L}_j(\theta_s - \eta g_i) \approx \mathcal{L}_j(\theta_s) - \eta g_j^\top g_i.$$

这里 $c_{ij} < 0$ 表示当前点附近的 first-order conflict；$c_{ij} > 0$ 是局部对齐，不保证长期 joint training 更好。若任一 $\|g_i\|$ 接近零，应报告“cosine undefined/低信号”，而不是解释成正交。工程上我会在 optimizer step 前，对每个 active task/route 分别调用 `autograd.grad(loss_i, shared_params)`，不让梯度先累积；同时记录 active mask/token/sample 分母、global 与 layer-

wise norm、cosine 分布、负余弦频率和 norm ratio。大模型算不起全参数时可固定一组共享 block 或 LoRA 参数做 probe，但必须写清采样参数集合，不能把局部 probe 冒充全模型结论。

若训练实际使用 $g = \sum_i w_i g_i$，任务 i 的一阶变化和 PCGrad-style 投影可写成

$$\Delta \mathcal{L}_i \approx -\eta g_i^\top \left(\sum_j w_j g_j \right), \qquad \tilde{g}_i = g_i - \frac{\min(0, g_i^\top g_j)}{\|g_j\|_2^2 + \epsilon} g_j.$$

第一式说明调 loss weight 只会重标梯度，不能保证所有 $g_i^\top g > 0$；所以总 loss 下降不等于每个任务改善。第二式在 pairwise dot product 为负时去掉冲突分量，但会改变优化问题，也不保证有限步、非凸训练中的 Pareto 改善。是否采用 reweight、gradient surgery、任务 adapter/branch 或 stop-gradient，必须用固定数据比例、相同训练预算和每任务指标做 matched ablation，而不是只比较 aggregate loss。[20]

项目挂钩：WAM 正式合同是共享 diffusion backbone 上的 action/future/value latent slots、condition/loss mask 与 policy/world/value sample routing，并非三个独立 head；本地 online-text 入口继续调用 official trainer/model/dataloader，没有 per-role `autograd.grad` 或 gradient-cosine 记录。另一个 MetaX DeepSpeed 文件确实用共享 proxy trunk、四个显式 head 和加权 MSE，但文件自述为兼容 adapter/proxy，不能反推正式 Cosmos checkpoint 的优化行为。要验证 WAM 冲突，应在同一 checkpoint 上各取 policy/world/value matched routed batch，对相同 diffusion blocks 分别求梯度。formal OPD 则强制一个 microbatch 内 route 一致，只计算该 routed teacher 的 full-vocabulary KL，再乘 sample weight；五条 route 是跨 step 共享 student 的多域训练，不是五个 loss 同时反传。应在同一 student snapshot 上各取一个 matched route batch 测共享参数梯度；route count、loss weight、不同 pool 的最佳 checkpoint 分散都不是 gradient conflict 的直接证据。[42][44][55]

追问：为什么 loss 数值更大不一定梯度更大？为什么 $c_{ij} < 0$ 只是局部一阶证据？route-homogeneous microbatch 如何做跨 route matched gradient probe？简单调权重为什么不能保证 Pareto 改善？PCGrad 可能牺牲什么？

易错：loss 曲线相反、不同任务最佳 checkpoint 不同、mask 非重叠或 route 分开都不能单独证明共享参数梯度冲突；head-specific 参数没有跨 head 冲突，但 shared backbone 仍可能有。计算 cosine 前要对齐同一参数快照、未混合的任务梯度、有效 mask 分母和参数集合。

来源：[20][42][44][55]　练习状态：unseen

▼ `L2` `P0` **VLA-FND-026 · PPO clipping 的直觉是什么？**

30 秒回答：PPO 用新旧策略的概率比 $r_t(\theta)$ 加权 advantage，但把 surrogate objective 的贡献限制在 $[1-\epsilon, 1+\epsilon]$ 的邻域，避免一次更新把策略推离采样 rollout 的旧策略太远。clipping 是近

似 trust region，不是严格 KL 约束；还要看 value loss、entropy、advantage normalization 和 rollout freshness。

机制展开（约 2 分钟）：先定义 $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{old}}(a_t \mid s_t)$，PPO-Clip 常写成 $L^{CLIP} = \mathbb{E}_t[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)]$。取 `min` 是为了在策略朝“有利方向”更新过头时采用更保守的项：当 $A_t > 0$ 时，ratio 太大不会继续增加收益；当 $A_t < 0$ 时，ratio 太小也不会让惩罚无限放大。注意这只改变 surrogate 的梯度贡献，不是直接截断 action，也不保证实际 KL 永远小于阈值。训练仍依赖相对新鲜的 on-policy rollout、正确的 old log-prob、advantage/return 的 termination 处理，以及 value clipping 或 entropy 等辅助项；ϵ 太小会限制学习，太大则失去保护。当前 Robot Harness 的固定 candidate group、reward-shaped weighted token CE 与 PPO 的 rollout/ratio/clipped objective 不同，不能因为都出现“相对 reward”就称为 canonical online GRPO 或 PPO。

项目挂钩：canonical online GRPO 不能直接称为 PPO；当前 Harness 本地脚本又是 offline GRPO-style continuation，回答时要分别说明采样、objective 与 reward/checker。

追问：为什么 clipped objective 要取 min?

易错：clip ratio 不是把 action 本身截断。

来源：[21][45]　**练习状态**：unseen

▼ L2 P0 VLA-FND-027 · reward、return、value target 和 success predicate 如何区分?

30 秒回答：reward 是每一步给出的反馈或代价，return 是从当前时刻累积折扣后的回报，value target 是用来回归 value function 的某种估计目标，success predicate 则是评估器根据状态/轨迹判定任务是否成功的布尔规则。四者可能有关，但分母、时间范围和实现位置不同；尤其要区分真正 terminal、time-limit truncated，以及训练 reward 和 benchmark success。

机制展开（约 2 分钟）：单步奖励记为 r_t，有限时域 return 可写为 $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$；value target 则可能是 Monte Carlo 的 G_t、一步 TD 的 $r_t + \gamma(1-d_t)V(s_{t+1})$、n-step 或 GAE-derived target，具体取法取决于算法。这里的 d_t 只应在真正终止时阻断 bootstrap；时间上限导致的 truncated 通常仍保留下一状态价值。success predicate 不是把 reward 求平均，而是由任务状态、接触条件、物体位姿、轨迹终止或脚本阈值组成的可复现实验判定，success rate 再对 episode 级布尔结果取均值并绑定分母、seed、初始化和环境版本。于是 shaped reward 可以上升而 success 不变：策略可能更快靠近目标却没有完成接触，或 reward 只覆盖代理目标；反过来 sparse success reward 也可能长期几乎不变而动作质量已改善。WAM 的 value/world auxiliary、RoboCasa 的训练/控制 reward 与 LIBERO 的 success predicate 必须分别记录 input、target、mask、reduction 和评估协议，不能把任一 proxy 直接写成任务成功率。

项目挂钩：WAM value head、RoboCasa reward 和 LIBERO success rate 必须分开讲。

追问：为什么一个任务可以 reward 上升但 success 不变?

易错：success rate 不是 reward 的简单平均。

来源：[4][10][40][42][43]　练习状态：unseen

▼ L3 P0 **VLA-FND-028 · offline policy 的 distribution shift 与 extrapolation error 如何导致闭环失败?**

30 秒回答：offline policy 只在数据分布上被监督；闭环执行中一个小动作误差就可能把状态推到训练分布之外，随后 policy 或 critic 对 OOD state-action 的估计不可靠，错误继续累积。缓解要从数据覆盖、行为约束、保守估计、recovery 数据和 matched OOD eval 入手，不能只靠把模型变大。

机制展开（约 2 分钟）：设行为数据来自 $d^{\beta}(s)\beta(a \mid s)$，而部署策略诱导的状态分布是 $d^{\pi}(s)$。即使单步行为克隆误差很小，只要 $d^{\pi} \neq d^{\beta}$，策略就会在未覆盖状态上外推；在长时序中，早期错误改变后续 observation，形成 compounding error。offline RL 还会遇到 critic 在数据外动作上的 extrapolation error：若 bootstrap 使用了不可靠的 $Q(s', a')$，过高估计会把策略推向 dataset support 之外。数据聚合或 intervention recovery 可以扩大状态覆盖，behavior regularization/BC constraint 与 conservative Q 可以限制危险外推，短 horizon、uncertainty/risk gate 和失败后 reset 也能缩短误差链；但每种缓解都要绑定具体 protocol，不能把“加入 recovery 数据”直接写成已解决。UWM/LIBERO 中接触精度、长时序 recovery coverage 目前只能作为候选诊断假设；升级为根因需要 raw episode trace、matched intervention、同初始化/seed 的对照和可复算的失败分层。

项目挂钩：接触精度和长时序 recovery coverage 只是 UWM 的候选诊断假设，需 raw episode trace 与 matched intervention 才能升级为根因。

追问：为什么行为克隆在短任务上可能仍然有效?

易错：distribution shift 不是只发生在真实机器人，仿真 closed loop 同样存在。

来源：[22][39][43]　练习状态：unseen

▼ L3 P0 **VLA-FND-039 · 什么时候使用 impedance control、MPC 或 learned policy?**

30 秒回答：impedance control 适合接触、插入和表面跟随，因为它规定机器人受外力后的期望刚度、阻尼和惯性；MPC 适合动力学、状态估计和约束较可信，且 solver 能在控制 deadline 内完成的场景；learned policy/VLA 更擅长从图像、语言和历史观测处理多任务变化，但依赖数据覆盖且本身不提供稳定性保证。三者通常分层组合：VLA 给 subgoal/reference/chunk，MPC 处理有限时域约束，impedance/servo 负责接触执行和安全边界。

机制展开（约 2 分钟）：impedance 的典型目标关系可写成 $M_d\ddot{e} + D_d\dot{e} + K_d e = F_{ext}$，其中 $e = x - x_d$，M_d, D_d, K_d 分别是期望惯性、阻尼和刚度；它不是直接预测任务动作，而是塑造末端受力后的动态响应。实际使用必须说明 force/torque sensing、frame、采样周期、滤波、

saturation，以及底层是 torque loop、position-based impedance 还是 admittance；“有 compliance”不等于天然安全。MPC 则在每个时刻求解 $\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) + \ell_f(x_H)$，满足 $x_{k+1} = f(x_k, u_k)$、$x_k \in \mathcal{X}$、$u_k \in \mathcal{U}$，通常只执行第一步或短前缀再重估。它能显式纳入 workspace、velocity、collision 或 force constraint，但受 model mismatch、state-estimation error、terminal design 和 solver deadline 约束。learned policy 可写成 $\pi_\theta(a_t \mid o_{\le t}, c)$ 或 action-chunk distribution，擅长视觉语言条件和复杂数据分布，却仍要经过 normalizer、frame/unit conversion、controller、rate limit 和 failure fallback。工程上应按风险分层：接触任务可用 policy 给目标、impedance/admittance 执行；约束可信时由 policy/world-model 给 reference，MPC 做 receding-horizon control；模型不准时缩短 horizon、提高 replan、加入 uncertainty/robust constraint 或 residual model，并保留可验证的安全控制器。不能因为 WAM/UWM 名为 world model 就直接把它当作可用于 MPC 的 calibrated dynamics model；还需要 state/action rollout contract、closed-loop error、uncertainty、constraint interface 和 deadline 证据。当前 OpenPiVLA/UWM 材料只证明 learned-policy 与 controller/action contract 的讨论，未发现个人项目接入 impedance/MPC 的实现和运行证据。

项目挂钩：OpenPiVLA/UWM 只支持 learned policy、world/action prediction 与 controller contract 的分析；当前证据不支持声称个人项目已部署 impedance/MPC，也不能把 WAM/UWM world model 自动等同于 MPC dynamics model。

追问：如果动力学模型不准，MPC 还值得用吗?

易错：三者不是互斥模型类别，常见答案是分层组合而不是二选一。

来源：[23][24][25][26][41][43]　练习状态：unseen

参考来源

1. Vaswani et al., Attention Is All You Need, 2017.
2. Ba et al., Layer Normalization, 2016.
3. Zhang and Sennrich, RMSNorm, 2019.
4. Sutton and Barto, Reinforcement Learning: An Introduction, 2nd ed.
5. Lynch and Park, Modern Robotics, Chapter 4.
6. PyTorch, LayerNorm documentation.
7. PyTorch, scaled dot-product attention documentation.
8. OpenVLA official repository.
9. Physical Intelligence openpi.
10. LIBERO official repository.
11. PyTorch, MSELoss documentation.
12. PyTorch, HuberLoss documentation.

13. PyTorch, CrossEntropyLoss documentation.
14. PyTorch, KLDivLoss documentation.
15. PyTorch, GELU documentation.
16. PyTorch, SiLU documentation.
17. Su et al., RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021.
18. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models, 2021.
19. Hugging Face Transformers, KV cache documentation.
20. Yu et al., Gradient Surgery for Multi-Task Learning, 2020.
21. Schulman et al., Proximal Policy Optimization Algorithms, 2017.
22. Fujimoto et al., Off-Policy Deep Reinforcement Learning without Exploration, 2019.
23. Hogan, Impedance Control: An Approach to Manipulation, Part I, 1985.
24. Rawlings, Mayne, Diehl, Model Predictive Control: Theory, Computation, and Design.
25. Chi et al., Diffusion Policy, 2023.
26. Kim et al., OpenVLA, 2024.
27. Goodfellow, Bengio, Courville, Deep Learning, 2016.
28. Ioffe and Szegedy, Batch Normalization, 2015.
29. PyTorch, L1Loss documentation.
30. PyTorch, Softmax documentation.
31. Minka, Divergence Measures and Message Passing, 2005.
32. Xiong et al., On Layer Normalization in the Transformer Architecture, 2020.
33. PyTorch, Automatic Mixed Precision Examples.
34. PyTorch, `clip_grad_norm_` documentation.
35. ROS REP-103, Standard Units of Measure and Coordinate Conventions.
36. Åström and Murray, Feedback Systems.
37. Lynch and Park, Modern Robotics, Chapter 5.
38. Lynch and Park, Modern Robotics, Chapter 6.
39. Ross, Gordon, Bagnell, DAgger, 2011.
40. Gymnasium, Handling Time Limits.
41. OpenPiVLA Claim–Evidence Inventory.
42. WAM Claim–Evidence Inventory.
43. UWM/LIBERO Claim–Evidence Inventory.
44. OPD Claim–Evidence Inventory.
45. Robot Harness Claim–Evidence Inventory.

46. Agarwal et al., *Deep RL at the Edge of the Statistical Precipice*, 2021；实现参考 `google-research/rliable@3ccd9f4`。

47. Hendrycks and Gimpel, *Gaussian Error Linear Units (GELUs)*, 2016。

48. Ramachandran, Zoph, Le, *Searching for Activation Functions*, 2017。

49. 本地实现快照，`C:\Users\mousongzhe\Project` HEAD `6c524d9c6c6226075f100cdac395fef2fdc37671`：`wm/uwm_robocasa_transfer/uwm_robocasa_train.py`（SHA-256 `EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE`）与 `wm/cosmos_policy_deepspeed_train.py`（SHA-256 `2A95D52DE64A5B46BC7C72C30B20BEC07C715FBF4FB590BF5DEA239CBA858680`）；仅支持局部 transfer/proxy 模块事实，不支持完整 upstream backbone 或性能因果归因。

50. 本地 RoPE implementation snapshot，`C:\Users\mousongzhe\Project` HEAD `6c524d9c6c6226075f100cdac395fef2fdc37671`：`wm/compat_transformer_engine/transformer_engine/pytorch/attention/__init__.py`（SHA-256 `375A646579C12E9D69EDAB96B4B01D583022407E826BDE241B6AA3C7C413AF63`）与 `scripts/muxi_write_te_torch_compat.py`（SHA-256 `7DC7994532D4EFEF725249326847567790343 80E3889D72CBFCBF74A0ACCE66A`）；支持 Cosmos smoke compatibility shim 的 rotate-half、cos/sin application、layout/dimension contract，不支持完整模型频率生成、Q/K call site、Qwen3-VL exact config 或性能归因。

51. Qwen3-VL norm/residual implementation lineage：Hugging Face Transformers `v4.57.1`（commit `8cb5963`），该文件下载内容 SHA-256 `DD63ED3B124232735B3DCA1BFA28F9D6B0D3F7182AFCB75DDE8F3E724B2B22DA`；本地 `muxi/metax_pkgs/pip-list.json`（SHA-256 `2A73054308392E14531544E471C790E8B500E449DB9442EB80C865B9197A170C`）记录 Transformers `4.57.1`，OpenPiVLA `formal_train.py`（SHA-256 `51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94`）加载该模型类并只向 Q/K/V/O 注入 LoRA；Cosmos `RMSNorm` shim（SHA-256 `1D2CB070B7C9FE21438C35ED059E99C08174410548284 7061AEB9F2CE1F539FA`）只支持算子语义。上述证据不替代远端 runtime source/config hash，也不证明完整 Cosmos block placement 或指标因果。

52. OpenPiVLA LoRA run-contract snapshot：`formal_train.py`（SHA-256 `51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94`）定义 `r=16`、`alpha=32`、`dropout=0.05`、`bias=none`、Q/K/V/O targets，并打印 `requires_grad` 参数账本；committed formal report（SHA-256 `D74126A3896F68AD44B75C7A88F1B7B9A90CB55DF0C6C6BEB90A5748E9E8E074`）记录 `trainable_params=6422528`、`total_params_seen=2133954560`（约 `0.301%`），同时说明历史目

标 YAML 的 `lora: False`。该证据支持 local trainer contract/report summary，不支持 raw log 复算、adapter binary/load smoke、历史 run lineage、个人 authorship 或性能因果。

53. KV-cache official/local contract：Hugging Face Transformers `v4.57.1` cache documentation（2026-07-22 抓取内容 SHA-256 `FCB47131204FBF721CB59014CF3E5CF1307471F1F3E2DB13579820B38436014A`）；OpenPiVLA `formal_train.py`（SHA-256 `51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94`）设置训练 `use_cache=False`；OPD `point/eval_qwen_vl_raw_holdout.py`（SHA-256 `29574ADCE53B254BC33ED39C370642707F570000F7F3FB8F037DBD1F5E0D206B`）在 eval generation 同时设置 config 与 call-site `use_cache=True`；formal OPD trainer（SHA-256 `7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`）对 teacher/student full-vocab loss forward 使用 `use_cache=False`。这些证据支持代码路径区分，不支持 cache on/off 加速比、peak memory、输出 parity 或机器人闭环延迟。

54. 本地梯度稳定性实现快照：`C:\Users\mousongzhe\Project` HEAD `6c524d9c6c6226075f100cdac395fef2fdc37671` 下，OpenPiVLA `formal_train.py`（SHA-256 `51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94`）实现 accumulation-boundary global clipping 与 `grad_norm` 日志；UWM/LIBERO `experiments/uwm/train.py`（SHA-256 `CFB4010A1DCA274C904295A7A937D2E1C5C3B08101A036D738F9C2B1560B4879`）和 UWM/RoboCasa `uwm_robocasa_train.py`（SHA-256 `EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE`）在本地 AMP 路径先 unscale 再 clip；`C:\Users\mousongzhe\muxi` HEAD `756e27e16adcfb3ccbfe98c2920dda42095b127f` 下，formal OPD trainer（SHA-256 `7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`）检查 non-finite loss，`expert_sft.py`（SHA-256 `2A1D37F5912C91B952C529B3B80C8609D7E3105DC57BBDB2D906EA288B58EA98`）向 TrainingArguments 传入 `max_grad_norm` 并对已记录的 loss/grad_norm/lr 做 finite callback。上述证据只支持实现合同，不支持实际 run 的逐层梯度/激活轨迹、爆炸根因、跨硬件数值因果或个人逐文件 ownership。

55. 本地多目标/routing 实现边界快照：`C:\Users\mousongzhe\Project` HEAD `6c524d9c6c6226075f100cdac395fef2fdc37671` 下，official-path `wm/cosmos_policy_train_with_consolidated_online_text.py`（SHA-256 `FBE879A3AD6E3E77B258AE971480338CF41D6EC3378E29C9DE05209CF9D38DF8`）保留 official trainer/model/dataloader 并启用 online Reason1；`wm/cosmos_policy_deepspeed_train.py`（SHA-256 `2A95D52DE64A5B46BC7C72C30B20BEC07C715FBF4FB590BF5DEA239CBA858680`）自述为 MetaX-safe adapter，使用 proxy trunk/heads、sample masks 与显式加权 MSE，不能替代正式 Cosmos diffusion loss。`C:\Users\mousongzhe\muxi` HEAD `756e27e16adcfb3ccbfe98c2920dda42095b127f` 下，formal OPD `point/train_opd_online_vl.py`（SHA-256 `7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`）强制 route-

homogeneous microbatch、选择单个 routed teacher，并对 weighted full-vocabulary KL 反传。上述文件与定向源码扫描未发现 per-task/route `autograd.grad`、gradient cosine 或 PCGrad 记录；该证据支持“当前未实现直接冲突测量”的代码边界，不支持实际 run 存在/不存在冲突、Pareto 改善、性能因果或个人逐文件 ownership。

Part 3 / 6 · `interview_materials/vla/sources/vol_01_batch1_independent_fact_review.html` · SHA-256 `FE2934579E5CB23A59926DA49B63CBE6F761CB96467ABAC62C57D512B6F0838C`

返回本卷目录

Canonical Markdown SHA-256：`fe2934579e5cb23a59926da49b63cbe6f761cb96467abac62c57d512b6f0838c`；生成自：`interview_materials\vla\sources\vol_01_batch1_independent_fact_review.md`

VOL 01 Foundations Batch 1：独立事实复核与公式边界

状态：`independent_fact_review / batch1 / WARN` 审查日期：2026-07-19 范围：VOL 01 knowledge 中表示、attention、归一化、损失与基础机器人几何的高频入口；不宣称整卷 39 张题卡已审查或已口述。 结论先行：核心机制与 primary sources 一致，但 canonical knowledge 曾出现 `\theta`、`\mid`、`\in`、`\tilde` 被写成控制字符/缺失反斜杠的问题；已回写修复。Qwen3-VL/本地 LoRA、OpenPiVLA 轴序和 UWM normalizer 仍保持未核实边界。

1. 冻结来源

ID	来源	类型	覆盖
S01	Vaswani et al., Attention Is All You Need	primary paper	scaled dot-product attention、残差与位置编码的基础语义
S02	Ba et al., Layer Normalization	primary paper	LayerNorm 的按样本/特征归一化与训练时行为
S03	Zhang & Sennrich, RMSNorm	primary paper	去均值 RMS 归一化与 re-scaling invariance 动机
S04	PyTorch `scaled_dot_product_attention`	official docs	Q/K/V shape、scale、mask API 边界
S05	PyTorch `LayerNorm`	official docs	normalized shape、affine、epsilon
S06	PyTorch `MSELoss`、`HuberLoss`、`CrossEntropyLoss`	official docs	reduction、连续回归与离散 token 监督边界
S07	Lynch & Park, Modern Robotics Chapter 4	primary textbook/site	齐次变换、FK 与 frame composition
S08	OpenVLA official repository、openpi official repository	official-code	仅用于确认项目接口应回到官方源码；不支持本地 ownership

2. Claim-to-source 复核

Claim	Verdict	保留边界
attention 使用 $QK^\top/\sqrt{d_k}$	`PASS / primary-verified`	除法是尺度控制；mask 语义取决于 bool/additive API
LayerNorm 减均值，RMSNorm 不减均值	`PASS / primary-verified`	具体 backbone 的 norm 选择不能由基础原理推出
MSE/Huber/CrossEntropy 监督对象不同	`PASS / official-docs`	reduction、mask、ignore index 和 dtype 仍是实现合同
齐次变换可组合旋转和平移	`PASS / primary-verified`	必须声明 T_B^A 方向、列向量约定、单位与时间戳
LoRA 是低秩更新	`PASS / method-level`	本地 rank、注入层、alpha 和 ownership 未核实
OpenVLA/openpi 的具体 action 轴序	`UNVERIFIED`	需要 frozen commit、配置和本地 artifact，不能从基础页推出

3. 本次修复

1. 将 Policy 术语中的 `pi_\theta(a_t\mid o_{\le t},c)` 恢复为完整 LaTeX。
2. 将状态转移中的 `p(s_{t+1}\mid s_t,a_t)` 恢复为完整 LaTeX。
3. 将齐次坐标说明中的 `R_{ab}\in SO(3)` 与 `\tilde p` 恢复为完整 LaTeX。
4. 保留 `done`、timeout、normalizer、axis order 和 controller 的实现风险，不把通用伪代码写成项目代码证据。

4. 复核边界

- 本页只审查 VOL 01 knowledge 的高频公式与基础术语，不代表 39 张题卡全部独立事实审查完成。
- 所有 39 张题卡继续 `unseen`；没有真实口述就不更新 weak/pass/strong，也不创建 coaching state。
- Qwen3-VL 具体 normalization、LoRA 注入层、OpenPiVLA action schema、UWM normalizer/checkpoint 仍需项目证据。
- 浏览器 `file://` 视觉 QA 受当前环境策略阻断，静态 HTML 校验单独记录。

5. 下一步

优先做一题真实口述（VOL 02 `VLA-RL-011` 或 VOL 01 `VLA-FND-012`）；若继续 source-first，则审查 VOL 01 的 MDP/TD/机器人控制 batch，而不是一次性重写整卷。

参考文献

以上 S01–S08 已在本页逐条冻结；本地项目映射仍以 `interview_materials/01..05_*.html` 与 VOL 07 evidence ledger 为准。

Part 4 / 6 · `interview_materials/vla/sources/vol_01_batch2_mdp_td_control_independent_fact_review.html` · SHA-256 `C7E75C811DB1ACC88A478235C40BB79B3897DA0CEB37749F828ED2A4109EA4CE`

返回本卷目录

Canonical Markdown SHA-256：`c7e75c811db1acc88a478235c40bb79b3897da0ceb37749f828ed2a4109ea4ce`；生成自：`interview_materials\vla\sources\vol_01_batch2_mdp_td_control_independent_fact_review.md`

VOL 01 Foundations Batch 2：MDP / TD / 机器人控制独立事实复核

状态：`independent_fact_review / batch2 / WARN` 审查日期：2026-07-19 范围：VOL 01 knowledge 与 `VLA-FND-021..039` 中的 MDP、Bellman、TD、on/off-policy、termination/truncation、Jacobian、PID、控制频率和闭环执行边界。 结论先行：MDP/Bellman/on-off-policy 的核心定义可保留；TD 与 rollout 必须把 `terminated` 和 `truncated` 分开；Jacobian 奇异性的风险应表述为 inverse/伪逆放大关节速度与噪声，而不是笼统说“小关节误差导致末端不稳定”。Robot Harness 只能绑定本地 offline GRPO-style continuation，WAM/UWM 指标只能按 evidence gate 降级表述。

1. 冻结来源

ID	来源	类型	本次支持的 claim	不支持的 claim
So1	Sutton & Barto, Reinforcement Learning: An Introduction, 2nd ed.	primary textbook	MDP、return、value/Q/advantage、Bellman、TD、on/off-policy	不支持某个本地项目使用了特定 RL 算法
So2	Gymnasium, Handling Time Limits	official docs	`terminated` / `truncated`、timeout bootstrap	不支持所有环境都采用同一 success predicate
So3	Lynch & Park, Modern Robotics Chapter 5	primary textbook/site	twist、Jacobian、singularity、velocity kinematics	不支持本地 action axis/frame
So4	Lynch & Park, Modern Robotics Chapter 6	primary textbook/site	numerical IK、多解、初值与 singularity 边界	不支持 IK 成功等于碰撞/动力学可行
So5	ROS REP-103, Standard Units of Measure and Coordinate Conventions	official standard	SI units、right-handed frame、axis convention	不支持某个项目已遵守该规范
So6	Åström & Murray, Feedback Systems	primary textbook/site	feedback、PID、delay、stability 的基础关系	不支持任意 PID 参数或真实控制稳定性
So7	VOL 02 Independent Fact Review	local fact review	GAE/PPO/GRPO 与 local Harness implementation 边界	不支持 online robot rollout 或个人 ownership
So8	WAM Inventory、UWM Inventory	local evidence audit	指标、协议和 artifact 缺口	不支持将历史 proxy 升级为 benchmark success

2. Claim-to-source 复核

Claim	Verdict	必须保留的边界
V^π, Q^π, A^π 的定义与 Bellman expectation relation	`PASS / primary-verified`	即时 reward 与期望 reward 的记号需说明；Bellman relation 不是单一网络 loss
on-policy 与 off-policy 的差异	`PASS / primary-verified`	off-policy 不等于 offline RL；数据复用需要分布修正/保守性边界
TD terminal mask	`FIX_REQUIRED`	true termination 通常停止 bootstrap；time-limit truncation 通常仍 bootstrap
Jacobian singularity	`PASS WITH WORDING FIX`	rank/condition 问题主要在 inverse mapping 中放大 joint velocity/noise
PID 的 P/I/D 直觉	`PASS / textbook-consistent`	D 项会放大噪声；anti-windup、滤波、采样周期和 delay 是实现合同
chunk 减少 policy query、长执行前缀更开环	`PASS / engineering inference`	prediction horizon、execution horizon、controller rate 和 observation age 必须分开
Harness 的“online GRPO”项目挂钩	`FAIL / OVERCLAIM`	当前本地直接证据只支持 offline reward-shaped grouped weighting
WAM 约 45% MSE 改善	`WARN / protocol-limited`	当前 evidence gate 不支持把历史比例写成已核验强结果
UWM 0.744 / 74.4%	`WARN / user-confirmed remote`	不是本地 raw bundle 验证的 benchmark；必须带协议和来源标签

3. 回写动作

1. TD residual 改用 d_t^{term}，并明确 time-limit truncation 的 bootstrap 语义。
2. 闭环伪代码改为显式处理 `terminated`、`truncated` 与 `success`，避免一个无语义 `done` 混合训练和评估边界。
3. Jacobian 文字改为 inverse/伪逆的速度与噪声放大，不把 singularity 误写成 forward kinematics 的普遍误差放大。
4. `VLA-FND-024/026` 的 Harness 项目挂钩改为 canonical online GRPO 与本地 offline GRPO-style 的对照。
5. `VLA-FND-029` 的 WAM 历史比例降级为“离线 proxy 趋势需同协议 artifact 复核”；`VLA-FND-028` 的 UWM 根因改为诊断假设。

4. 边界与下一步

- 本批完成后，VOL 01 knowledge 的两批高频基础事实可进入 candidate 状态，但整卷 39 卡仍未逐卡独立审查，也没有真实口述。
- `VLA-FND-039` 的 impedance/MPC/learned-policy 选择仍需单独控制方法 batch；本页只保留分层组合的工程直觉。

- 所有卡保持 `unseen`；不创建 `coaching_state.md`，不把静态审查当 mastery。
- 浏览器 `file://` 视觉 QA 保持 `BLOCKED`；完成后仍需 SHA、单 H1、公式和本地链接静态校验。

参考文献

本页使用 S01–S08 作为冻结入口；具体项目数字与 ownership 必须回到 VOL 07 inventory/ledger，不由基础知识页反向证明。

Part 5 / 6 · `interview_materials/vla/sources/vol_01_batch3_advanced_control_independent_fact_review.html` · SHA-256 `53E9F12E8EBCFC9C4A627271F83A75E7EB0A7E7F18DAEBA412576E693C31EF54`

返回本卷目录

Canonical Markdown SHA-256：`53e9f12e8ebcfc9c4a627271f83a75e7eb0a7e7f18daeba412576e693c31ef54`；生成自：`interview_materials\vla\sources\vol_01_batch3_advanced_control_independent_fact_review.md`

VOL 01 Foundations Batch 3：Impedance / MPC / Learned Policy 独立事实复核

状态：`independent_fact_review / batch3 / WARN` 审查日期：2026-07-19 范围：VOL 01 knowledge 的 advanced-control 入口与 `VLA-FND-039`；目标是建立控制方法选择边界，不把通用方法反向写成个人项目实现。 结论先行：impedance control、MPC 与 learned policy 解决的层级不同，常见系统是组合而不是互斥选择。只有在 action/controller contract、动力学模型、replan 频率、约束与本地 runtime artifact 齐全时，才能声称项目实际实现了某种组合。

1. 冻结来源

ID	来源	类型	支持	不支持
So1	Hogan, Impedance Control: An Approach to Manipulation, Part I, 1985	primary paper	通过期望动态关系连接运动误差与交互力	不支持 impedance 天然安全或任意接触稳定
So2	Rawlings, Mayne, Diehl, Model Predictive Control: Theory, Computation, and Design	primary textbook	finite-horizon optimization、模型/约束、receding horizon	不支持模型失配下性能必然优于 policy
So3	Chi et al., Diffusion Policy, 2023	primary paper	视觉条件 action sequence policy、receding-horizon execution	不支持其替代底层 controller 或安全约束
So4	Kim et al., OpenVLA, 2024	primary paper	vision-language-conditioned robot action policy	不支持本地 OpenPiVLA action schema/ownership
So5	Lynch & Park, Modern Robotics	primary textbook/site	kinematics、Jacobian、force/velocity 与 controller 接口	不支持项目实际控制频率或 torque loop
So6	VOL 03 Action Knowledge	local knowledge audit	action chunk、replan、normalization 的跨卷入口	不支持本地 runtime 已闭合
So7	OpenPiVLA Inventory、UWM Inventory	local evidence audit	项目 action/controller 与 artifact 缺口	不支持从项目页推断 impedance/MPC 已实现

2. 方法边界

2.1 Impedance control

令 $e = x - x_d$，一种目标阻抗关系可写为：

$$M_d\ddot{e} + D_d\dot{e} + K_d e = F_{ext}.$$

M_d, D_d, K_d 是期望惯性、阻尼和刚度，F_{ext} 是外部交互力。直觉是系统不只追踪位置，还规定“受力后应该有多硬、多软、多阻尼”。这不等于天然安全：离散时间、延迟、力传感噪声、饱和、接触模型与底层 torque/position loop 都会影响稳定性。

2.2 Model Predictive Control

标准 finite-horizon MPC 可抽象为：

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) + \ell_f(x_H), \quad x_{k+1} = f(x_k, u_k), \quad x_k \in \mathcal{X},\ u_k \in \mathcal{U}.$$

每次根据当前观测/状态解长度 (H) 的受约束优化，只执行第一步或短前缀，再重新估计和求解。优势是约束显式、代价可解释；风险是模型失配、状态估计误差、求解延迟与 horizon/terminal design。

2.3 Learned policy / VLA

learned policy 直接学习 $\pi_\theta(a_t \mid o_{\le t}, c)$ 或 action chunk，擅长把高维视觉和语言条件压到动作分布。它仍需要 action normalization、frame/unit、controller rate、safety limit、replan 和 success predicate。模型输出可作为末端目标、subgoal、MPC reference、residual 或直接 action；这些接口不是同一语义。

3. 选择与组合

场景	首要方法	组合建议	主要 gate
接触、插入、表面跟随	impedance/admittance	learned policy 给目标，低层控制力—位移关系	force sensing、stiffness/damping、delay
约束明确、动力学可用	MPC	policy/world model 提 reference/model，MPC 做 constrained replan	model error、solver deadline、state estimate
高维视觉语言、任务变化大	learned policy/VLA	controller/MPC/impedance 承担执行和约束	data coverage、OOD、action contract

场景	首要方法	组合建议	主要 gate
长时序且模型不准	hybrid	policy 产 subgoal/chunk，短 horizon 重规划与 recovery	observation age、failure detector、fallback

回答“什么时候用哪个”时先声明输出接口和闭环层级，再比较模型可信度、接触强度、约束、延迟、数据覆盖和安全预算。不能只以“方法更先进”作选择依据。

4. 项目与真实性边界

- OpenPiVLA/UWM 页面支持讨论 learned policy 与 controller 的接口，但当前 inventory 不证明候选人实现了 impedance 或 MPC。
- WAM/world model 可以为 planning/MPC 提预测或评分，但必须有 local planner runtime、objective、constraint、latency 与 selected-action trace 才能声称实际接入。
- `VLA-FND-039` 只能作为通用系统设计题；个人回答须使用“如果系统采用……”或绑定已核验 artifact。
- 39 张 VOL 01 题卡继续 `unseen`；本页不改变 mastery，也不创建 coaching state。

5. 下一步

将 impedance/MPC 公式、变量、pipeline 与 evidence boundary 回写 VOL 01 knowledge，并把 `VLA-FND-039` 的来源从仅 Modern Robotics 扩展到 primary control/policy sources。完成静态校验后，VOL 01 可进入“knowledge fact-review candidate”，但仍不能进入 `reviewed/drilled`。

参考文献

本页冻结 S01–S07；项目实现与个人 ownership 仍以 VOL 07 inventories/ledgers 为唯一入口。

Part 6 / 6 · `interview_materials/vla/sources/vol_01_completion_independent_fact_review.html` · SHA-256 `5B64360C4012115D01355FBADAC601305C4D5E80CC5B0F547BB82C0044F6A632`

返回本卷目录

Canonical Markdown SHA-256：`5b64360c4012115d01355fbadac601305c4d5e80cc5b0f547bb82c0044f6a632`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_01_completion_independent_fact_review.md`

VOL 01 Foundations：39 卡 Completion Fact Reconciliation

状态：`completion_fact_reconciliation / 39_cards / WARN` 审查日期：2026-07-19 范围：`VLA-FND-001..039` 的稳定 ID、短答、来源编号、项目挂钩、追问、易错点和练习状态。 结论先行：39 张卡均有可解析结构和直接来源入口，Batch 1–3 发现的公式、timeout、Jacobian、GRPO、WAM/UWM 与 advanced-control 边界已回写。这里的 completion 只表示“题卡事实链已 reconciliation”，不表示候选人已口述掌握；39 张卡继续全部 `unseen`，VOL 01 verdict 仍为 `WARN`。

1. 审查合同

每张卡必须同时满足：稳定唯一 ID、机制级短答、至少一个 primary/official source、项目挂钩有 local evidence 或明确 caveat、一个 follow-up、一个 pitfall、答案不超过 350 字、练习状态与真实口述一致。

来源分四层：

1. `primary/official`：论文、教材、官方 API/标准；支持通用机制。
2. `official-code`：OpenVLA/openpi/LIBERO；支持上游接口，不支持本地 ownership。
3. `local-evidence-audit`：五项目 inventory；支持当前证据边界，不自动支持强结果。
4. `unverified`：具体 Qwen3-VL config、OpenPiVLA axis、UWM normalizer、WAM matched metric 与个人逐文件 ownership。

2. 39 卡逐项 reconciliation

ID	核心机制来源	项目挂钩审查	Verdict
VLA-FND-001	Transformer + Deep Learning	OpenPiVLA 仅作 multimodal/action contract 入口	`PASS/WARN`
VLA-FND-002	Transformer + Deep Learning	OPD 几何误差与语义相似度分层	`PASS/WARN`

ID	核心机制来源	项目挂钩审查	Verdict
VLA-FND-003	Deep Learning + Modern Robotics Ch.5	参数导数与机器人 Jacobian 分开	PASS/WARN
VLA-FND-004	LayerNorm/RMSNorm/BatchNorm primary + PyTorch	Qwen3-VL exact norm 仍查 config	PASS/WARN
VLA-FND-005	Deep Learning 概率基础	action 逐维统计是 contract，不是项目结果	PASS/WARN
VLA-FND-006	PyTorch MSE/Huber/L1	WAM MSE 只作 proxy	PASS/WARN
VLA-FND-007	PyTorch KL + Minka divergence	DPO/KL 方向只讲机制	PASS/WARN
VLA-FND-008	RL/Deep Learning + WAM/UWM inventories	seed/denominator 仍需 raw artifact	PASS/WARN
VLA-FND-009	GELU/SiLU docs + Deep Learning	backbone activation 需 frozen config	PASS/WARN
VLA-FND-010	Transformer + PyTorch CE/Softmax	token/action logits 只讲数值机制	PASS
VLA-FND-011	OpenVLA/openpi + PyTorch losses	token vs continuous action 与项目 evidence 分层	PASS/WARN
VLA-FND-012	Transformer + SDPA docs	attention scale 为通用机制	PASS
VLA-FND-013	Transformer + SDPA + WAM inventory	attention mask 与 loss/routing mask 分开	PASS/WARN
VLA-FND-014	RoFormer	多模态时序只作机制，不推本地 layout	PASS/WARN
VLA-FND-015	Transformer/LN + Xiong et al.	pre/post norm 不包装为训练创新	PASS/WARN
VLA-FND-016	OpenVLA/openpi + OpenPiVLA inventory	模态拼接/独立 expert 不混称	PASS/WARN
VLA-FND-017	LoRA + OpenPiVLA inventory	rank/target modules/ownership 仍未核实	PASS/WARN
VLA-FND-018	HF KV cache docs	cache 不消除串行 token steps	PASS
VLA-FND-019	Transformer/LN + PyTorch AMP/clip docs	跨硬件问题保持诊断树	PASS/WARN
VLA-FND-020	PCGrad + WAM/OPD inventories	mask/routing 不自动等于 gradient conflict 解决	PASS/WARN
VLA-FND-021	Sutton–Barto + WAM/UWM inventories	reward/value/success 分层	PASS/WARN
VLA-FND-022	Sutton–Barto + WAM inventory	value head 不等于 action	PASS/WARN
VLA-FND-023	Sutton–Barto + WAM/UWM inventories	benchmark success 不等于 Bellman optimum	PASS/WARN

ID	核心机制来源	项目挂钩审查	Verdict
VLA-FND-024	Sutton–Barto + Harness inventory	canonical online 与 local offline GRPO-style 分开	PASS/WARN
VLA-FND-025	Sutton–Barto + Harness inventory	group-relative signal 不等于 robot value head	PASS/WARN
VLA-FND-026	PPO + Harness inventory	PPO/GRPO/local offline objective 分层	PASS/WARN
VLA-FND-027	Sutton–Barto/Gymnasium + WAM/UWM inventories	terminated/truncated/success 分开	PASS/WARN
VLA-FND-028	BCQ/DAgger + UWM inventory	contact/recovery 只作诊断假设	PASS/WARN
VLA-FND-029	Sutton–Barto/DAgger + WAM inventory	历史 MSE 比例已移除，保留 proxy 口径	PASS/WARN
VLA-FND-030	RL/Modern Robotics/REP-103 + OpenPiVLA inventory	observation/state/action contract 分层	PASS/WARN
VLA-FND-031	Modern Robotics/REP-103 + OPD inventory	image frame 与 robot action frame 分开	PASS/WARN
VLA-FND-032	Modern Robotics/REP-103 + OpenPiVLA inventory	方向、列/行向量、单位仍需本地 schema	PASS/WARN
VLA-FND-033	Modern Robotics Ch.4/6 + OpenPiVLA inventory	IK 成功不等于安全可执行	PASS/WARN
VLA-FND-034	Modern Robotics Ch.5 + OpenPiVLA inventory	singularity 以 inverse amplification 表述	PASS/WARN
VLA-FND-035	openpi/REP-103/Feedback/Modern Robotics + inventory	7D/14D 不等于 torque 语义	PASS/WARN
VLA-FND-036	Feedback Systems	PID 是底层机制，不冒充项目实现	PASS/WARN
VLA-FND-037	openpi/LIBERO/Diffusion Policy/Feedback + inventories	query/controller rate 与 execution prefix 分开	PASS/WARN
VLA-FND-038	openpi/LIBERO/REP-103 + inventories	normalizer/axis/gripper checksum 仍缺	PASS/WARN
VLA-FND-039	Hogan/Rawlings/Diffusion Policy/OpenVLA + inventories	不声称本地接入 impedance/MPC	PASS/WARN

3. 本次来源修复

- 新增 Deep Learning、BatchNorm、L1Loss、Softmax、Minka divergence、Pre-LN、AMP、gradient clipping、REP-103、Feedback Systems、Modern Robotics Ch.5/6、DAgger 与 Gymnasium timeout。
- 五个 local inventory 进入题卡来源表，用于约束项目 hook；Git author、项目页存在或官方实现均不当作个人 ownership。

- 45 个来源编号全部定义；所有题卡来源编号必须解析到该表，不允许悬空编号。

4. 完成度与未完成项

已完成

- 39/39 stable IDs；39/39 source blocks；39/39 project hooks；39/39 follow-ups；39/39 pitfalls。
- Batch 1–3 knowledge 与题卡边界回写。
- canonical SHA、HTML H1、公式污染、控制字符、details balance、本地链接和 review JSON 可解析性校验。

仍未完成

- 39/39 真实口述均未发生，因此练习状态继续 `unseen`。
- Qwen3-VL exact config、OpenPiVLA axis/normalizer/controller、WAM matched metric、UWM raw episode/checkpoint 与个人 ownership 仍依赖 VOL 07 evidence gate。
- 浏览器 `file://` 视觉 QA 仍 `BLOCKED`。

5. 下一步

VOL 01 不再继续批量生成新卡。下一 bounded slice 应是一次真实口述（优先 `VLA-FND-030`、`012`、`021`、`029`、`037` 中的一题）或转入 VOL 03 独立事实 completion audit；没有真实回答前不创建 coaching state，不更新 mastery。

参考入口

- Batch 1 Independent Fact Review
- Batch 2 MDP / TD / Control Review
- Batch 3 Advanced Control Review
- OpenPiVLA Inventory
- WAM Inventory
- UWM Inventory
- OPD Inventory
- Robot Harness Inventory

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 02 · IL / RL / Post-training

行为克隆、离线强化学习、PPO / GRPO 与机器人后训练

本卷由 4 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 02 IL / RL / Post-training：从示范数据到可验证反馈

题卡

1. 02 · VOL 02 IL / RL / Post-training：核心面试题卡

来源与事实复核

1. 03 · VOL 02 IL / RL / Post-training：来源计划
2. 04 · VOL 02 IL / RL / Post-training：独立事实与实现边界复核

Part 1 / 4 · `interview_materials/vla/knowledge/vol_02_il_rl_posttraining.html` · SHA-256 `B60889E43234CB48C26AE768BF773BA1AE1BC7345E694811588537BC52F85D7D`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`b60889e43234cb48c26ae768bf773ba1ae1bc7345e694811588537bc52f85d7d`；生成自：`interview_materials\vla\knowledge\vol_02_il_rl_posttraining.md`

VOL 02 IL / RL / Post-training：从示范数据到可验证反馈

状态：`fact_reviewed_candidate / WARN` 报告日期：2026-07-19 证据截止：2026-07-16 适用岗位：VLA / 具身智能算法 / 大模型后训练 / 机器人策略学习
本卷边界：解释 imitation learning、policy gradient、offline RL、SFT/DPO/GRPO 和 schema-aware reward 的通用机制；个人项目的具体 run、指标和贡献边界仍以 VOL 07 与现有项目页为准。

问题（Question）

本卷回答一条连续的问题链：

```
示范数据有什么监督信号?
        ↓
策略如何利用 reward / preference / group feedback 更新?
        ↓
离线数据之外的动作为什么危险?
        ↓
schema、reward checker、holdout eval 如何各司其职?
        ↓
怎样把训练结果与真实机器人闭环分开解释?
```

面试官最常用的转场是：

- “你做了 SFT/GRPO，具体优化的 loss 是什么？”
- “为什么不用 PPO / DPO / offline RL？”
- “reward 是怎么定义的，如何防止 reward hacking？”
- “tool accuracy 或 proxy MSE 提升，为什么能/不能说明真实任务变好？”

一句话口径

IL 解决“模仿给定行为”，RL/post-training 解决“根据反馈重新加权行为”；但反馈的可靠性、数据支持范围和评估协议决定了结果是否可信。schema 是输出契约，reward 是训练信号，holdout eval 是验收协议，三者不能混成一个指标。

背景与动机

机器人策略训练同时受三种约束：示范昂贵、在线交互风险高、策略输出必须可执行。单纯 BC 可以快速复用示范，却会遇到协变量偏移；纯 online RL 反馈更接近任务目标，却需要大量安全交互；大模型 post-training 又引入 preference、reference policy 和格式约束。

因此回答算法选择时，先说清楚四件事：

1. **数据从哪里来**：expert demo、旧 policy rollout、人工 preference、规则 checker 还是环境 reward。
2. **更新什么对象**：连续 action、离散 token、value、reward model 还是 adapter。
3. **反馈覆盖什么范围**：只判断格式/工具合法性，还是能判断任务成功、接触质量和长期回报。
4. **在哪里验收**：in-domain、OOD、closed-loop、真机，还是仅离线 proxy。

模仿学习与数据分布

Behavior Cloning 与 covariate shift

BC 把专家轨迹中的 (o_t, a_t^*) 当监督样本，最小化：

$$\mathcal{L}_{BC}(\theta) = \mathbb{E}_{(o,a^*)\sim D_E}\left[\ell\left(\pi_\theta(o), a^*\right)\right].$$

D_E 是专家数据分布，ℓ 可以是连续 action 的 MSE/Huber 或 token action 的 cross-entropy。直觉是策略在“专家访问过的状态”上学得像；一旦自己的小误差把系统带到 D_E 之外，后续状态会越来越陌生，误差会复利累积。实现上要记录 observation history、action window、padding/mask 和 train/eval split，不能只说“拿轨迹训练”。

DAgger：让策略访问它会犯错的状态

DAgger 让当前策略执行，收集它实际访问的状态，再由专家标注动作并加入数据集：

```
D_0 = expert demonstrations
for iteration k:
    train π_k on aggregated D_{k-1}
    roll out π_k to collect visited observations
    query expert labels on those observations
    D_k = D_{k-1} ∪ newly labeled data
```

它的核心不是某个 optimizer trick，而是改变训练分布，使策略访问的状态被纳入监督。代价是需要在线专家、人工或高质量 oracle，真实机器人上的标注成本和安全风险都很高。VLA 项目中，recovery/intervention 数据、失败轨迹和 synthetic perturbation 可以起到部分相似作用，但不能不加证据地称为 DAgger。

Action chunking 与误差累积

若每步动作误差期望为 ϵ，长时序 rollout 可能出现随时间增长的累积误差。action chunking 一次预测 H 步、只执行 $K \le H$ 步后重新观测，可把策略查询次数从 T 降到约 T/K，同时保留纠偏机会。H 决定预测视野，K 决定开环执行长度；二者不是同一个参数。VOL 03 会专门讨论 action representation，VOL 05 会讨论延迟和评估协议。

数据质量、覆盖与 suboptimal demo

“更多 demo”不是无条件答案。需要分别检查：

- 覆盖：任务、对象、视角、初始状态、失败恢复是否出现；
- 一致性：action 轴顺序、单位、时间同步和语言条件是否统一；
- 质量：轨迹是否含卡顿、错误 gripper、异常 reset 或 label leakage；
- 目标：suboptimal demo 是噪声、可学习的次优策略，还是需要保留的 recovery 经验。

过滤掉所有“看起来不完美”的轨迹可能丢失恢复分布；全部保留又可能把错误行为变成监督。面试中要先说明过滤规则和评估依据，再说数据量。

Policy Gradient、GAE 与 PPO

REINFORCE 与 baseline

策略梯度可写为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\right],$$

其中 G_t 是从 t 开始的 return。直接使用 return 方差很大；减去只依赖状态的 baseline $b(s_t)$ 不改变期望梯度，却能减少方差。用 $A_t = G_t - b(s_t)$ 后，baseline 通常由 value network 估计。机器人场景还要面对 reward 稀疏、长 horizon 和昂贵 rollout。

GAE：在 TD 与 Monte Carlo 之间插值

GAE 用 TD residual 的折扣和估计 advantage：

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma(1-d_t^{term})V(s_{t+1}) - V(s_t).$$

γ 控制长期回报折扣，λ 在低方差/高 bias 的 TD 和高方差/低 bias 的 Monte Carlo 之间插值，d_t^{term} 只表示真实终止。时间上限截断 `truncated` 通常仍允许 bootstrap，不能把它和 `terminated` 无条件合成一个 `done`。实现时要检查 episode boundary、timeout、value bootstrap 和 advantage normalization；GAE 不是 PPO 专属，但常与 PPO 配套。

PPO clipped objective

PPO 用旧策略与新策略的概率比：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, \qquad \mathcal{L}^{CLIP} = \mathbb{E}_t\left[\min\left(r_t\hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right].$$

当更新让 ratio 偏离过远时，clip 项限制其继续获益，近似 trust-region；它不是严格约束，也不替代 value loss、entropy regularization 或 rollout freshness。面试时要区分 ratio clipping 与 action clipping：前者限制策略更新比率，后者限制控制量范围。

PPO 训练诊断

除了 policy loss，还应观察：

- `approx_kl`：新旧策略偏离程度；
- `clip_fraction`：多少样本触发 clip；
- `value explained variance`：value 是否跟上回报；
- entropy：策略是否过早塌缩；
- reward/success 与 episode length：是否出现 reward hacking 或短路。

如果 policy loss 正常但 success 不涨，先查 reward/checker、action parser、终止逻辑和 rollout 分布，再调模型。仅看 loss 曲线无法证明闭环策略正确。

Offline RL：保守性与行为约束

为什么离线数据外的动作危险

离线数据只有 ((s,a,r,s')) 的有限支持。策略如果选择数据中很少出现的动作，critic 需要在缺乏证据的区域外推 Q 值；一旦 Q 过估，actor 会主动追逐这些虚假的高价值动作。机器人上，这种 OOD action 可能直接导致碰撞或不可恢复状态。

CQL：惩罚 OOD Q 过估

CQL 的核心思想是让数据外动作的 Q 保守，离散动作的简化项可写成：

$$\mathcal{L}_{CQL} = \alpha \left(\mathbb{E}_s \log \sum_a \exp Q(s,a) - \mathbb{E}_{(s,a)\sim D} Q(s,a) \right).$$

第一项鼓励降低所有候选动作的过高 Q，第二项保留数据动作的价值。连续动作需要采样或重要性近似；具体实现有多种变体。面试时不要说 CQL“保证安全”，它只是减少一类 OOD 过估风险。

IQL：不显式查询数据外动作

IQL 通过 expectile value learning 和 advantage-weighted behavior cloning 提取策略，避免在训练中直接对数据外动作做 max-Q 查询。直觉是先估计数据支持范围内哪些行为更好，再用加权 BC 复制高优势行为。它适合静态数据，但效果仍取决于 reward、数据覆盖和 value 估计。

Offline RL 的诊断分层

当 offline RL 或行为策略失败时，按层定位：

1. **数据层**：任务覆盖、action normalization、终止/timeout、reward label；
2. **critic/value 层**：Q 过估、target 失真、OOD action 比例；
3. **policy 层**：动作分布、行为约束、采样温度；
4. **执行层**：denormalization、controller、接触和闭环恢复。

这套分层与 UWM/WAM 的失败分析兼容，但不自动说明你的项目使用了 CQL/IQL。

SFT、DPO、GRPO 与 schema-aware reward

SFT：对已确定的目标序列做 teacher forcing

SFT 对目标 token 序列做 masked cross-entropy：

$$\mathcal{L}_{SFT} = -\frac{1}{\sum_t m_t} \sum_t m_t \log \pi_\theta(y_t \mid x, y_{<t}),$$

x 是图像/文本/状态等条件，y_t 是目标 token，m_t 决定哪些位置计入 loss。工具调用中，m_t 可只覆盖 assistant/tool JSON；VLA 中则要明确 action token 是否与文本共同训练。SFT 学的是数据中给出的行为，不会自动学会未覆盖的失败恢复。

DPO：用 preference pair 隐式吸收 reward

DPO 对 chosen y_w 与 rejected y_l 的相对 log-ratio 优化：

$$\mathcal{L}_{DPO} = -\mathbb{E} \log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)} \right] \right).$$

reference policy 提供锚点，β 控制偏离程度。DPO 不代表没有 reward 假设，而是把 preference 关系直接写进分类式目标；chosen/rejected 的质量和 pair 构造决定结果。

GRPO：组内相对反馈

GRPO 对同一 prompt 采样一组回答/轨迹，用组内 reward 归一化形成相对优势的简化形式：

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G}) + \epsilon}.$$

随后仍需结合 policy ratio、clip、reference/KL 或具体实现的正则项进行更新。面试时应说“group-relative feedback 减少了独立 value model 的依赖/提供相对信号”，不要把所有 GRPO 实现说成完全相同，也不要把组内分数直接当真实 task success。

Schema、reward、holdout eval 三层分工

对于工具调用或结构化机器人 action，可以把设计分成三层：

层	适合放什么	不应声称什么
Schema / output contract	工具名、字段类型、robot scope、JSON 可解析性、动作维度	schema 合法不等于任务正确
Reward / checker	可程序化判分的工具选择、参数、scope、顺序和禁止动作	checker 分数不等于真实物理成功
Holdout eval	in-domain、OOD、安全边界、业务分布、闭环任务	holdout 一次通过不等于泛化已证明

可写成一个设计模板：

$$R = \sum_j w_j R_j,$$

其中 R_j 是不同约束项，w_j 是权重。这个公式只表达 reward decomposition 的通用机制；你现有 Robot Harness 页面支持“schema-aware reward”这一项目方向，但具体权重和 checker 字段仍需以本地数据/代码证据为准。

Pipeline 与伪代码

SFT → preference/online feedback 的训练路径

```
def post_training_pipeline(dataset, base_policy, checker, evaluator):
    # 1. Split demonstrations and preference/rollout data before training.
    sft_data, preference_data, holdout = split_by_episode_and_task(dataset)

    # 2. Train the base behavior with masked teacher-forcing supervision.
    policy = sft(base_policy, sft_data, loss_mask="assistant_or_action_slots")

    # 3. Score candidates with a checker/reward function, keeping raw traces.
    scored = score_with_checker(policy, preference_data, checker)

    # 4. Apply DPO/GRPO only to the feedback format that is actually available.
    policy = preference_or_group_update(policy, scored)

    # 5. Evaluate holdout separately from the reward used during training.
    metrics = evaluator(policy, holdout)
    return policy, metrics
```

关键是先固定 split 和协议，再把 reward/checker 与 holdout 分离；否则可能出现训练 checker 变高但独立评估不变的 reward hacking。

一轮 GRPO 风格更新的抽象

```
def group_relative_update(policy, prompts, rollout_env, optimizer):
    # Sample multiple trajectories for each prompt/task condition.
    groups = [rollout_env.sample(policy, prompt, num_samples=G) for prompt in prompts]

    # Compute rewards from the declared checker/environment, preserving traces.
    rewards = [rollout_env.score(group) for group in groups]

    # Normalize within each group so the update uses relative rather than raw scale.
    advantages = normalize_within_group(rewards)

    # Compute the implementation-specific clipped/KL-regularized policy objective.
    loss = policy_objective(policy, groups, advantages)
```

```
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    return loss, rewards
```

伪代码没有假定你的 Harness 使用哪一种 exact GRPO loss；它只映射“采样—判分—组内相对化—更新—独立 eval”的数据流。

实现映射与项目挂钩

通用机制	本地材料	当前安全表达	未核实/不能直接推出
SFT / GRPO-style / schema-aware reward	`C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\train_grpo.py`、`configs/reward_spec.json`、`eval_grpo_tasks.py`；另见 Robot Harness Inventory	本地存在 reward-shaped grouped weighting、11 类 reward component、task-family weights 和字段 coverage evaluator；可讲“GRPO-style continuation”	不能把它说成 canonical online multi-candidate GRPOTrainer、真实物理 reward 或个人独立 ownership
action/world/value 多目标	`interview_materials/04_cosmos_wam_multitask_training.html`	可以解释 action imitation、world/value 辅助目标和 mask/routing 的潜在互助/干扰	不能把通用 PPO/GRPO 结论写成 WAM 使用的算法
DPO / preference sample	`output/mousongzhe_resume_optimized.html`	可以说参与样本与偏好样本整理、SFT/DPO 实验和错误归因	不能虚构 preference pair 数、reward model 或独立 ownership
OPD ablation	`interview_materials/02_opd_vlm_grounding.html`	可用作“如何设计因果 ablation”的项目转场	现有材料没有完整 SFT-only/single-teacher 因果表
offline proxy / online rollout	`interview_materials/05_uwm_libero_eval_protocol.html`、WAM 页	可以区分 fixed-init strict eval、离线 proxy 和闭环成功	不能从基础算法公式推出具体 UWM 根因

公源/本地设计边界

公有设计 / 上游公开流程

- DAgger、GAE、PPO、CQL、IQL、DPO、GRPO 的论文目标和基本假设。
- OpenVLA、openpi、Cosmos Policy 等公开仓库的上游接口与 recipe，具体版本以 Source Scout 冻结为准。

本地证据

- 五个项目页、简历 HTML 和 `resume_restructure_plan.md` 支持项目范围、指标和当前已承认的风险。
- Robot Harness 页支持 schema/reward/holdout 的项目叙事，但不替代完整训练代码。

本地推断

- 由项目页可推断你需要把训练信号和评估协议分开讲；这是一条叙事/诊断原则，不是某个官方算法的新颖性 claim。

当前仍未核实

⚠️ Harness 的 GRPO-style trainer、reward spec 和字段 evaluator 已能在外部 Project 工作区读取，但 canonical online GRPO、多候选环境 rollout、同 scorer base→SFT→GRPO 因果 bundle 与个人逐文件 ownership 仍未闭合；WAM/UWM 训练源码与完整 rollout logs 仍缺。因此本卷的通用 GRPO 公式与本地脚本必须分层表述。

方法对比与选择建议

方法	优点	主要风险	面试时先问
BC / SFT	简单、稳定、易扩展到 token/连续 action	covariate shift、示范偏差	数据覆盖和 loss mask
DAgger	直接补策略访问态	需要专家在线标注，真实机器人昂贵	expert oracle 是否可用
PPO / online RL	直接优化环境反馈	rollout 成本、reward hacking、训练不稳	reward/checker 和安全边界
CQL / IQL	可在静态数据上保守改进	reward/critic/OOD 误差	数据支持范围
DPO	不必显式训练 reward model，适合 preference pair	pair 质量、reference、KL 偏移	chosen/rejected 如何构造
GRPO	组内相对反馈，适合可采样/可判分任务	reward scale、group variance、实现差异	group、checker、clip/KL 细节

风险与后续建议

1. 不要把“用了 GRPO”当作结果解释；先说明 group、reward、policy objective 和独立 eval。
2. 不要把 schema 合法、tool accuracy 或 proxy MSE 说成真实物理成功的充分条件。
3. DPO/GRPO 的答案必须区分通用论文机制与个人项目实际使用的实现。
4. VOL 02 下一批题卡应优先绑定 Robot Harness 的 schema/reward/holdout 与“offline GRPO-style ≠ online GRPO”边界、WAM 的 proxy/online gap 和 OPD 的 ablation 缺口。
5. VOL 03 再展开 Diffusion/Flow Matching；不要在本卷用几句带过复杂 action head。

参考文献（References）

1. Ross, Gordon, Bagnell, A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, 2011.
2. Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation, 2016.
3. Schulman et al., Proximal Policy Optimization Algorithms, 2017.
4. Kumar et al., Conservative Q-Learning for Offline Reinforcement Learning, 2020.
5. Kostrikov et al., Offline Reinforcement Learning with Implicit Q-Learning, 2022.
6. Rafailov et al., Direct Preference Optimization, 2023.
7. Shao et al., DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024.
8. Hugging Face TRL, official training library.
9. 本地项目证据：`interview_materials/01_robot_harness_tool_training.html`、`02_opd_vlm_grounding.html`、`03_openpivla_action_modeling.html`、`04_cosmos_wam_multitask_training.html`、`05_uwm_libero_eval_protocol.html`。
10. VOL 02 独立事实与实现边界复核：primary source、GAE timeout、CQL partial objective 与本地 GRPO-style trainer 的冻结口径。

Part 2 / 4 · `interview_materials/vla/questions/vol_02_il_rl_posttraining.html` · SHA-256 `0BEBE009B33CCD1E7B4E1D07EDA80B98B62A1B76326FA248BDE1057F4112F886`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256: `0bebe009b33ccd1e7b4e1d07eda80b98b62a1b76326fa248bde1057f4112f886`；生成自：`interview_materials\vla\questions\vol_02_il_rl_posttraining.md`

VOL 02 IL / RL / Post-training：核心面试题卡

状态：`drafted` 当前题数：40 / 40 难度：L1 5 题 / L2 23 题 / L3 12 题 练习状态：全部 `unseen` 说明：题库层使用短答案；深度公式和算法边界见 `knowledge/vol_02_il_rl_posttraining.md`。

A. 模仿学习与数据分布

▼ `L1` `P0` **VLA-RL-001 · Behavior Cloning 是什么？为什么会有 covariate shift?**

30 秒回答：BC 把专家轨迹拆成 (o_t, a_t^*)，在专家分布 $d_E(o)$ 上监督学习 $\pi_\theta(a \mid o)$。训练时只见专家做对后的状态；部署时小误差会改变下一状态，使 $d_\pi(o) \neq d_E(o)$，后续错误放大，即 policy-induced covariate shift 与 compounding error。增加同分布 demo 只降低 d_E 上误差，未覆盖恢复状态；应先排 parser/normalizer/controller，再用 policy rollout + expert correction、recovery/intervention data 或安全 DAgger 补 d_π 支持。

约 2 分钟机制展开：BC 的监督目标只对 expert occupancy measure 加权，而真实闭环轨迹由当前策略反复改变未来状态分布：

$$L_{\mathrm{BC}}(\theta) = \mathbb{E}_{(o,a^*)\sim d_E}[\ell(\pi_\theta(o), a^*)], \qquad p_\pi(\tau) = p(s_0)\prod_{t=0}^{T-1}\pi_\theta(a_t \mid o_t)P(s_{t+1} \mid s_t, a_t), \qquad \Pr(\text{at least one error}) = 1-(1-\epsilon)^T \approx T\epsilon.$$

d_E 是专家轨迹诱导的 observation/state 分布，d_π 是 learned policy 闭环执行后的分布，ϵ 是专家分布上的单步错误率。最坏情况下，一个早期错误把策略带到无标签状态，此后每步成本继续累积，经典 supervised reduction 可得到 $O(T^2\epsilon)$ 量级的性能差；这是 worst-case scaling，不是每个机器人任务都精确按平方恶化。[1]

“covariate shift”在这里不是简单的相机 OOD，也不是标签函数必然变化；核心是 policy action 通过环境动力学改变 future input distribution。即便 train/validation action MSE 很低，只要 validation window 仍来自 expert trajectory，它仍没测试“偏离后能否恢复”。反过来，rollout 失败也不能直接归因 BC shift：action parser、normalizer、维序、controller、initial state 或 success checker 错误会制造相同表象。

诊断层	观测/实验	支持什么	不能直接推出什么
action contract	round-trip、GT replay、preclip→env action、dim/order	排除接口/尺度错误	state distribution shift 已发生
expert support	demo action/state quantile、kNN、density、coverage	expert 数据覆盖范围	policy 在同 state 下必然失败
closed-loop drift	matched-init rollout 的 state embedding distance、first-error time、recovery rate	policy-induced occupancy drift	根因一定是数据量不足
intervention	在 policy 访问状态请求 expert correction，再重训	新 d_π 状态是否能被修复	任意 augmentation 等价 DAgger

为什么“更多 demo”不一定够：如果新增数据仍集中在成功轨迹中心、相同 initial state 和相同 expert controller，它只提高高密度区域的样本量，不增加偏航、接触失败、遮挡、停滞和恢复状态的 support。应按 task/scene/object/init/failure type 看 coverage，并区分随机图像增强、action noise 与真正的 policy-visited state + expert relabel；只有后者直接改变训练 occupancy。[1]

缓解顺序：第一，锁 action/eval contract；第二，对离线轨迹做 episode-group split，加入有物理意义的小扰动和 recovery demonstrations；第三，在安全边界内 rollout，用 uncertainty、停滞、接触异常或人工接管触发 expert 标注，聚合后重新训练；第四，使用短 receding-horizon/action chunk、uncertainty fallback 和 safety controller 限制错误传播。RL 只有在 reward/termination/安全反馈可信时才进入，不是 BC 失败的自动修复按钮。

项目实现口述：我会这样讲：“WAM 当前 proxy 从预先存在的 demo 与 archived rollout directories 构造 `RoboCasaDataset`，用 `demonstration_sampling_prob`、`success_rollout_sampling_prob` 和 `p_world_model` 调 mixture，训练循环就是固定 DataLoader 的 action/world/value MSE；没有当前 policy collector 或 buffer insertion。UWM LIBERO 也是固定 train loader，间歇/独立 env rollout 只写 success/video，transition 不回流 optimizer。因此它们可以称 supervised fine-tune/IL + online evaluation，不能说已经用 DAgger 修复 covariate shift。”[16][17][19]

“Cosmos action diagnostics 比较 demo/base/trained 的逐维 mean/std/mean-z，GT replay 还检查 normalizer 与 7D→12D assembly；这些能提示 action marginal support 或接口风险，但没有 matched state、state embedding occupancy 和 intervention retrain，所以不能把 0-success 单因归成 $d_\pi \neq d_E$。UWM RoboCasa 的 preclip/env-action/clip trace同样先用于排接口，再讨论闭环分布。”[18][20]

最小补强实验：固定 checkpoint、normalizer、controller、task/seed/init/horizon，保存每步 observation embedding、raw/env action、expert nearest-neighbor distance、uncertainty、first-divergence 与 stop reason；选一批失败前后状态让 expert relabel，做 original-BC vs BC+recovery 的同预算 matched rollout。只有 recovery arm 在相同协议下改善 first-error time/success，才支持"补 d_π coverage 有效"。

ownership 与结论防线：现有 artifact 支持说明固定数据监督目标、archived rollout sampling、接口/support 诊断和缺少 feedback loop；不支持声称已实现 DAgger、已直接测量 state occupancy shift、已因果证明 compounding error 是当前失败根因，或把上游数据/模型设计写成个人原创。

追问：为什么增加同分布 demo 不一定消除 shift？$O(T^2\epsilon)$ 是定理边界还是经验定律？action marginal OOD 与 state-conditional shift 有什么区别？怎样区分 parser bug 和 covariate shift？action chunking 为什么只能缓解不能消除？什么样的数据才能称 DAgger/recovery？

易错：把任意视觉 OOD 都叫 covariate shift；把 offline validation MSE 当 closed-loop 保证；看到 0 success 就归因 BC；只加成功 demo；把 archived rollout directory 当 online collector；用 action mean-z 证明 state shift；把随机 augmentation 叫 DAgger；没有 expert relabel 却声称 recovery learning；默认 RL 一定解决分布偏移。

来源：[1][9][16][17][18][19][20]　**练习状态**：unseen

▼ L1 P0 VLA-RL-002 · BC、DAgger、GAIL、IRL 的差异是什么？

30 秒回答：BC 在固定专家数据上回归动作，便宜但只覆盖 d_E；DAgger 让当前策略访问 d_π 并请专家纠正，再聚合为监督数据；GAIL 不对 policy 状态逐个查询专家动作，而用判别器给策略 rollout 提供 occupancy-matching 信号，再以 RL 更新；IRL 先从示范反推 reward，再求最优策略。四者主要区分固定/交互数据、专家动作/轨迹信号、隐式/显式 reward 及是否需要 RL。机器人上通常先 BC，再按安全预算选择 DAgger；GAIL/IRL 的交互与内层优化更贵。

约 2 分钟机制展开：先看四个问题：最终学的是 policy 还是 reward；策略是否真的进入环境采新状态；专家返回逐状态动作还是只给轨迹；policy update 是监督学习还是 RL。

$$\begin{aligned}\text{BC:}\quad & \min_\theta \ \mathbb{E}_{(o,a^*)\sim D_E}\left[-\log \pi_\theta(a^* \mid o)\right],\\ \text{DAgger:}\quad & D_{k+1} = D_k \cup \{(o, \pi_E(o)) : o \sim d_{\pi_k}\}, \qquad \pi_{k+1} = \mathrm{BC}(D_{k+1}).\end{aligned}$$

BC 的数据集在训练前已冻结；DAgger 的 loss 仍可只是 supervised BC，真正的变化是数据协议：第 k 轮用当前策略 π_k 访问状态，再由 expert/oracle 给这些状态动作标签并版本化聚合。只有失败轨迹、随机扰动或旧 rollout，没有 current-policy state、expert correction 与下一轮 learner ingestion，都不能叫完整 DAgger。[1]

$$\begin{aligned}\text{GAIL:}\quad & \min_\pi \max_D \ \mathbb{E}_{(s,a)\sim d_\pi}[\log D(s,a)] + \mathbb{E}_{(s,a)\sim d_E}[\log(1 - D(s,a))] - \lambda H(\pi),\\ \text{IRL:}\quad & \hat{r} \in \arg\max_r p(D_E \mid r), \qquad \pi_{\hat{r}} \in \arg\max_\pi \mathbb{E}_\pi\left[\sum_t \gamma^t \hat{r}(s_t, a_t)\right].\end{aligned}$$

这里的 GAIL 判别器约定 D 输出"来自 policy"的概率，换标签约定只会交换两项。GAIL 用 discriminator-derived signal 和 RL 直接匹配 expert/policy occupancy，通常不产出一个可脱离判别器复用、可解释的显式 reward；IRL 则把 reward 当主要产物，再规划或 RL 求策略，但同一行为可能由多组 reward 解释，存在 reward ambiguity / shaping non-identifiability。[31][32]

方法	学习对象与监督	数据闭环	主要代价与高危点
BC	policy；expert action label	固定 expert dataset	covariate shift；便宜、稳定、最常用起点
DAgger	policy；policy-visited state 上的 expert action	rollout → query expert → aggregate → retrain	人工/oracle 成本、安全风险、round mixing 与标签不一致
GAIL	policy + discriminator；expert/policy occupancy 对抗信号	反复采 policy rollout 并做 adversarial RL	sample inefficiency、训练不稳、discriminator shortcut、真实机器人探索风险
IRL	reward，再由 reward 求 policy	reward inference 与 planning/RL 双层问题	reward 不可辨识、动力学/规划假设、计算和验证成本

选择条件：有高质量 state-action demonstrations 时先做 BC；闭环偏离明显且能安全请求 expert correction 时，优先小批量 safe DAgger/recovery。只有离线专家轨迹、没有可在线查询的 oracle 和可信 task reward，并且有充足 simulator/interaction budget 时才考虑 GAIL；需要把目标函数迁移到新动力学、解释行为偏好或用于后续规划时才更有理由做 IRL。真实机器人较少直接从 GAIL 起步，不是它"理论上无效"，而是每轮 discriminator/policy 更新都需要新 rollout，探索安全、样本成本和 proxy shortcut 很难承受。

项目实现口述：我会这样讲："WAM 虽然有 `rollout_data_dir`、`success_rollout_sampling_prob`，但这些目录在训练前已经存在；代码只构造一次 `RoboCasaDataset` 和固定 DataLoader，再优化 action/world/value MSE，没有 current-policy collector、expert query 或版本化 buffer ingestion，所以属于 fixed-mixture supervised IL，不是 DAgger。UWM LIBERO 的 `collect_rollout` 只返回 success rate 和 video，主训练仍从固定 train loader 取 batch，rollout transition 不回流 dataset/replay/loss，也不能因为训练中调用了 env 就称 online imitation。"[17][19]

"同样，本地定向扫描没有 GAIL discriminator、expert/policy occupancy loss 或 adversarial policy update；WAM 的 value head 只是对数据中 `value_function_return` 做 regression，并没有从 expert optimality 反推 reward，所以也不是 IRL。当前最安全的项目表述是 BC/监督微调 +

online evaluation，并把 safe recovery aggregation 作为待补实验。”[17][19]

如果要把 recovery 升级为 DAgger：至少保存 policy/checkpoint SHA、round、seed/init、触发状态、expert correction、intervention reason、dataset version 与 ingestion manifest；每轮只允许预注册的一次聚合/重训，并用未回流的 matched rollout 比较 first-error time、recovery rate、success 与 intervention rate。仅把失败视频加入 holdout 或做 action noise augmentation，仍不满足 DAgger 合同。

ownership 与结论防线：当前证据支持按代码数据流判断“实现了哪种学习闭环”和设计下一步 safe DAgger；不支持声称已经实现 DAgger/GAIL/IRL、已在线请求专家、已学出 transferable reward、已获得 adversarial imitation uplift，或把上游算法设计写成个人原创。

追问：DAgger 为什么还是监督学习？GAIL 为什么说是 implicit reward 而 IRL 是 explicit reward？两者都匹配 expert behavior，优化目标有什么本质区别？没有 expert 在线标注时能否近似 DAgger？IRL 的 reward ambiguity 怎样验证？真实机器人为什么更常 BC→recovery，而不是直接 GAIL？

易错：把 DAgger 说成一个 loss；把任意失败轨迹/旧 rollout 叫 policy aggregation；把“训练时跑了 env”叫 online learning；把 discriminator 当可解释 task reward；把 value/return regression 叫 IRL；忽略 GAIL 仍需要 RL 与策略 rollout；没有动力学和独立任务指标却声称 IRL reward 已被识别。

来源：[1][17][19][31][32]　**练习状态**：unseen

▼ L2 P0 VLA-RL-003 · DAgger 为什么能缓解 compounding error？代价是什么？

30 秒回答：DAgger 不是换一个 loss，而是把训练分布改成策略会访问的分布：第 k 轮让 π_k rollout 到偏航/恢复状态，请 expert/oracle 给这些状态纠正动作，聚合 D_{k+1} 再监督训练。这样 d_E 外的“无标签状态”被补上，compounding error 的 support 缺口变小；代价是 rollout、专家接管、同步/安全/标签一致性和重训成本。没有 current-policy state、expert relabel 和下一轮 ingestion，只有失败视频或噪声扩增，都不算完整 DAgger。

约 2 分钟机制展开：BC 的单步误差通常在 d_E 上测，部署时策略一旦偏离，后续 observation 来自 d_{π_k}，而这些状态可能没有 expert label。DAgger 每轮显式把这两个分布接上：

$$D_{k+1} = D_k \cup \{(o, \pi_E(o)) : o \sim d_{\pi_k}\}, \qquad \epsilon_k = \mathbb{E}_{o \sim d_{\pi_k}}[\ell(\pi_\theta(o), \pi_E(o))].$$

D_k 是到第 k 轮为止的聚合数据，d_{π_k} 是当前策略闭环访问的 observation/state 分布，$\pi_E(o)$ 是专家在该状态的纠正动作。只要 expert label 可靠、训练能降低自己的 on-policy loss、状态观测和动作合同稳定，策略在“自己会走到的错误区域”上就不再完全无监督；这解释了它为什么能缓解而不是神奇地消除 compounding error。[1]

代价与失效模式：

成本/风险	机制	需要记录的证据
expert/oracle 成本	每个偏航状态都要接管、查询或规划	intervention time、expert latency、每步/每段标注成本
真实机器人安全	访问 d_{π_k} 可能触碰物体、越界或卡死	safety envelope、急停、触发器、invalid/timeout 分栏
交互与重训成本	每轮都要 rollout、写入新数据、再训练和评估	round、policy/checkpoint SHA、dataset version、effective updates
标签一致性	不同人/不同时间的纠正动作可能冲突	expert identity、agreement、action variance、conflict policy
触发偏差	只采“被触发”的失败，可能遗漏渐进漂移或未触发 OOD	trigger recall、failure taxonomy、matched non-trigger holdout
expert-policy mismatch	专家动作不可执行、延迟大或与 controller 合同不同	raw/env action、normalizer、controller trace、replay check

DAgger 的收益也依赖聚合策略：全量保留旧数据更稳定但数据膨胀；只保留最新 intervention 更贴近 d_{π_k} 但可能遗忘专家基本技能；expert 与 learner 混合执行要预先写清 mixture schedule，不能把一次接管视频直接等同于完整算法。

没有在线 expert 时的近似：可以用 simulator reset/扰动后由 scripted oracle 重标注，或从真实人工接管日志构造 recovery set；这些只有在状态确实来自当前/候选策略访问、并且有对应 expert correction 与下一轮 ingestion 时，才可称 DAgger-like。纯 action noise、随机图像增强、旧 policy 的失败轨迹和只做 holdout/failure analysis 都只能称 coverage augmentation 或 recovery dataset，不能升级为 DAgger。

项目实现口述：我会这样讲：“Cosmos/WAM 的 `rollout_data_dir` 是训练前已有的 archived data，`RoboCasaDataset` 初始化后由 DataLoader 采样，`treat_success_rollouts_as_demos=False` 也没有改变成 current-policy collector；训练代码没有 expert query、intervention state 记录或 round-wise dataset append。UWM 的 `maybe_collect_rollout` 在固定 step 调环境，只把 success rate 和 video 写到日志，LIBERO standalone eval 也只产 episode/summary，没有 transition→replay→optimizer 的回流。所以这些 rollout 不能被包装成 DAgger。”[17][19]

最小可补实验：先冻结一个 policy/checkpoint 和安全触发器；每条 intervention 记录 `round/policy_sha/seed/init/obs/raw_action/env_action/expert_action/reason/dataset_version`，按 task/scene/object/failure type 分层；只做一次预注册 aggregation，训练后用未回流的 matched initial state 比较 first-error time、recovery rate、success 和 intervention rate。若没有真实 expert correction，就把结果命名为 recovery/holdout，不写 DAgger。

ownership 与结论防线：当前证据支持解释 DAgger 的数据闭环和设计 safe intervention ledger；不支持声称已经完成在线专家标注、DAgger round、compounding-error 因果消融或真实机器人 uplift，也不支持把普通数据清洗/失败视频写成个人算法创新。

追问：为什么 DAgger 仍是 imitation learning 而不是 RL？如果 expert 本身有噪声，聚合会不会放大错误？为什么只保留最新 recovery 数据可能遗忘基本技能？触发器漏检怎样影响结论？没有在线 oracle 时，哪种 simulator 方案仍满足 DAgger-like 合同？如何把 intervention rate 纳入 success 的公平比较？

易错：把 DAgger 说成一个 loss；把当前策略 rollout 误写成 expert rollout；把旧失败视频、随机扰动或 action noise 叫 policy-state aggregation；没有 expert label 却说完成 DAgger；忽略 dataset version、checkpoint lineage、safety invalid 和未回流 holdout；把一次接管成功写成 compounding error 已被证明消除。

来源：[1][17][19]　**练习状态**：unseen

▼ L2 P0 VLA-RL-004 · action chunking 为什么可能降低误差累积?

30 秒回答：Action chunking 一次预测 H 步，但运行时可只执行前 K 步再重新观测。它可能通过学习时序相关动作、减少逐步独立预测噪声和 policy inference/通信次数，把查询次数从 T 降到约 T/K；若 $K < H$，就是 receding-horizon 折中。但 K 越大，观察越陈旧、越开环，动力学和接触误差会在 chunk 内累积；K 越小则推理延迟、抖动和 deadline miss 增多。因此必须分开报告 model horizon H、execution prefix K、control Δt 与 replan latency，chunking 不是稳定性保证。

约 2 分钟机制展开：模型输出动作块，但环境仍逐步执行低层动作。核心不是“输出了几步”，而是何时重新读取 observation 并请求新 chunk：

$$A_t = \pi_\theta(o_t) \in \mathbb{R}^{H\times d}, \qquad a_{t+j} = A_t[j] \;\; (j = 0, \dots, K-1), \qquad o_{t+K} \longrightarrow A_{t+K}, \qquad K \le H.$$

H 是模型一次预测的 action horizon，d 是动作维度，K 是实际执行前缀；episode 有 T 个低层控制步时，policy query 约为 $\lceil T/K \rceil$，不是环境控制步变成 T/K。同步阻塞式 runner 仍会在 queue 为空时等待 inference；只有异步预取下一 chunk 时，当前 queue 才提供约 $K\Delta t$ 的 latency-hiding budget，并且必须冻结 timestamp、staleness 与过期策略。[33]

为什么“可能”降低误差累积：相邻动作往往强相关，联合预测可表达“接近→抓取→抬起”的短期结构，减少每步独立采样带来的方向抖动；减少 policy query 还能降低网络 jitter、模型 latency 和 frame/action 不一致。ACT 进一步用 temporal ensemble 融合多个重叠 chunk 对同一时刻的预测，但这和“只执行当前 chunk 前 K 步”是两个机制，不能混写。[33]

为什么也可能更差：

因素	K 较小	K 较大	应观察指标
feedback freshness	频繁重新观测，接触/遮挡后能快纠偏	observation/context 陈旧，chunk 内近似开环	first-divergence、recovery latency
inference/通信	query 多，P95 latency 与 deadline miss 风险高	query 少，吞吐更友好	query count、TTFT/P95、deadline miss
action smoothness	相邻 chunk 可能跳变或抖动	同一 chunk 内通常更连贯	jerk、chunk-boundary discontinuity
model/dynamics error	误差较快被新 observation 修正	模型偏差、滑移和接触变化持续累积	state/action deviation、collision/clip rate
safety/stop	能更快响应 safety predicate	chunk queue 若不抢占可能继续执行旧动作	stop latency、discarded queued actions

因此应把 H 与 K 分开消融：先固定 checkpoint、model horizon、normalizer、controller、task/seed/init 和 control rate，只扫 $K \in \{1, 2, 4, 8, H\}$；再固定 K 比较不同 H。只比较 $H = K$ 的多组配置，会把模型预测难度和执行开环长度混在一起。

项目实现口述：我会这样讲：“OpenPiVLA 的本地 LIBERO runner 在 action queue 为空时同步请求 `client.infer(...)["actions"]`，只把 `chunk[:replan_steps, :7]` 入队并逐步 `popleft`；代码默认 $K = 5$，但归档 client log 没有完整 argv，因此不能把默认值冒充该 run 的 effective config。另一个 LeRobot evaluator 的 `--action-horizon=50` 只是构造 GT/pred window 做离线 L1，对环境没有执行，不是 $K = 50$ rollout。已有 task-0 run 是 70 steps、model/no-op 都 0 reward/false success，不能据此说 chunking 提升了稳定性。”[34]

“Cosmos/WAM 归档 action-debug 为 `chunk_size=32`、`num_open_loop_steps=16`、action `32×7`，即 $H = 32, K = 16$；UWM RoboCasa 则用 `action_len=16`，`exec_horizon` 控制实际前缀。其 matched 54-scenario sweep 在 $K = 1/2/4/5/8$ 时 success 为 `2/1/0/1/1`，总体低且非单调，只能说明执行前缀是敏感协议因子，不能宣布最佳 K 或因果稳定性收益。”[18][34]

最小验证实验：对同一 checkpoint 和同一批 initial states 扫 K，每步保存 observation hash/timestamp、chunk ID、chunk index、raw/denorm/env action、inference latency、queue age、clip/contact、stop reason；报告 task-macro success、first-error time、recovery/stop latency、jerk、query count 和 P95 latency。若要测试 temporal ensemble，另设 prefix-only 与 overlap-ensemble 两个 arm，不能把它隐含进 K sweep。

ownership 与结论防线：现有证据支持说明本地 H/K/queue 数据流、离线 action-window 与 closed-loop execution 的区别，以及 UWM 小样本 K sweep；不支持声称 action chunking 已因果提升成功率、代码默认值等于所有实际 run、ACT temporal ensemble 已在本地启用，或把上游 action-chunk 架构写成个人原创。

追问：为什么 query 次数减少不等于控制步减少？H 和 K 如何分别影响训练难度与执行稳定性？为什么 $K=1$ 也可能更差？temporal ensemble 与 receding-horizon prefix execution 有什么区别？异步推理时怎样处理过期 chunk？接触任务和自由空间移动应使用相同 K 吗？

易错：把 action dimension 当 horizon；把离线 `action_horizon` 当 controller execution window；模型输出 H 步就默认执行 H 步；只报平均 latency 不看 deadline miss；把 smoother action 当 success；跨 checkpoint/normalizer/seed 比较 K；把 0-success 或单次成功强归因于 chunk length；把 prefix queue 叫 temporal ensemble。

来源：[18][33][34]　**练习状态**：unseen

▼ `L2` `P0` VLA-RL-005 · 专家轨迹覆盖不足时，如何构造 recovery / intervention 数据？

30 秒回答：先冻结 checkpoint、seed、init 和安全边界，让当前策略 rollout；在停滞、偏航、接触/越界、低置信或即将 timeout 时保存触发前后窗口，由 expert/teleop 接管给出可执行纠正动作。每条样本绑定 policy/checkpoint、obs、raw/env action、reason、task/scene、expert latency 和 dataset version，按 failure type 分层并按 episode split。只有 policy-visited state + expert relabel + 下一轮 ingestion 才叫 DAgger；失败视频或噪声增强只是 recovery/augmentation。

约 2 分钟机制展开：先把样本分成四档，避免“有失败数据”就过度包装：

数据等级	必须满足的事实	安全命名
failure record	有 observation/video、action、success/stop 或接触 proxy，但没有纠正标签	failure log / diagnostic trace
recovery demonstration	从失败前状态开始，存在 expert/teleop 的可执行纠正动作和恢复后上下文	recovery demo
intervention dataset	由明确触发器从当前/候选 policy rollout 采到 takeover 段，保存 source、时间和 expert identity	intervention data
DAgger round	intervention state 被 expert relabel，写入版本化 aggregate dataset，并用于下一轮 learner retrain/eval	DAgger-like / DAgger（需合同闭合）

一个可审计的 intervention window 至少应保存：

$$W_t = \{o_{t-L_{pre}:t+L_{post}}, a^{raw}_{t-L_{pre}:t+L_{post}}, a^{expert}_{t:t+L_{post}}, m^{src}, r^{trigger}, q^{safety}, v^{dataset}\}.$$

L_{pre} 和 L_{post} 分别覆盖触发前和接管后的上下文；a^{raw} 与 a^{expert} 要能映射到同一 normalizer、action order、controller/env contract；m^{src} 标记 learner、expert、scripted oracle 或 invalid；$r^{trigger}$ 是触发原因；q^{safety} 记录急停、越界、碰撞、timeout 等安全状态；$v^{dataset}$ 用来证明哪一轮训练消费了它。没有这些字段，后续无法区分“模型动作错”“接口错”与“接管本身改变了任务”。

触发器设计：停滞可用末端速度/状态进展阈值，偏航可用目标相对位姿或 expert nearest-state 距离，接触异常可用 contact/force/clip proxy，低置信可用 ensemble disagreement 或 action variance；但触发器必须在 held-out failure 和 near-miss 上测 recall/precision。过于保守会漏掉渐进 drift，过于激进会把正常接触切成 takeover，增大专家成本并改变数据分布。触发器本身不能使用最终 success label 事后回看，否则会产生 hindsight leakage。

采样与隔离：先保留干净 expert base，再按 task/scene/object/init/failure type 分层加入 recovery；不要把同一 episode 的前缀放 train、接管段放 test。至少按 episode/scene 或 policy checkpoint 分组切分，normalizer 只从 train 拟合；同一 intervention 产生的连续窗口要 group ID 去重。训练混合比例、expert/learner action mask、无效/急停样本处理和 replay priority 应预注册，并保留一批从未回流的 matched initial-state holdout。

没有真实 expert 时：可以先用 simulator scripted controller/oracle 生成 recovery label，或对人工接管日志做离线 relabel；这能形成 recovery dataset，只有当状态确实来自当前策略访问且样本进入下一轮 learner，才升级为 DAgger-like。action noise、图像增强、旧 policy 失败视频、把 contact-rich window 提高 loss weight，都不能替代 expert correction。反之，也不应把所有失败样本删除：可恢复失败和接管前状态正是闭环 support 的重要部分。

项目实现口述：我会这样讲：“WAM 的 base/trained bundle 有 `manifest.json`、`actions.jsonl`、raw video、`success=false` 和 `num_open_loop_steps=16`，可以做 first-divergence 与 failure taxonomy，但没有 expert action、接管时间或下一轮 dataset append。Cosmos TurnOffMicrowave 的 1 episode `0/1`、500 steps、32 次 query 也只是可审计失败，不是 recovery demo。UWM RoboCasa 的 18 tasks×3 scenarios、接触统计和 `success` 是 rollout diagnostics；公开 LeRobot 只提供 action/state/reward/done 等字段，contact/task state 仍是 proxy。现有记录因此只能支撑 failure analysis / recovery dataset 设计，不能说已经有 intervention 或 DAgger 数据。”[18][34]

最小补强实验：在 simulator 先固定一个 checkpoint、task/scene/init/seed 和 safety envelope，设置一个停滞/偏航触发器；采集 learner-only、expert takeover、scripted-oracle 三类窗口，做 action-source mask 与 normalizer/controller round-trip；按 episode group split，训练 `base BC` 与 `BC+recovery` 两个 matched arm，只在未回流 holdout 上比较 first-error time、recovery rate、success、intervention rate、invalid/急停率。若没有 expert label，就把结论限定为“recovery set 可行性”，不写 DAgger uplift。

ownership 与结论防线：当前证据支持设计数据 schema、触发器、分层采样和安全评估；不支持声称已经采集 teleop/expert takeover、已完成 recovery retraining、已测 intervention 成本或成功率提升，也不支持把上游失败视频、接触 proxy 或普通数据清洗写成个人原创算法。

追问：触发器太保守或太激进分别怎样影响数据分布？为什么接管段不能随机切窗？如何避免同一 episode 泄漏到 test？expert 动作和 learner 动作的 loss mask 怎么写？接管后状态是否仍属于同一 task success predicate？没有 teleop 时 simulator oracle 的可信度如何验证？为什么 contact-

rich window 加权不等于 recovery learning?

易错：把失败轨迹叫 expert trajectory；把视频叫 intervention label；把 reward/done/contact proxy 当 task ground truth；把 action noise 当 DAgger；只采成功 episode；按 frame 随机切分造成 episode leakage；忽略 normalizer/action order/controller；把一次接管或单 episode success 写成 recovery uplift。

来源：[1][9][18][34]　**练习状态**：unseen

▼ L2 P0 VLA-RL-006 · 数据质量、数量和多样性如何取舍?

30 秒回答：先过质量门，再谈数量和多样性：检查 schema、时间同步、action 轴、normalizer、label 与 episode 边界；数量看 unique episodes 和有效 windows，不看重叠 frame 数；多样性按 task/scene/object/init/view/control phase/failure type 统计 coverage 和长尾。训练要 group split、train-only stats、防重复泄漏；可恢复失败不能全删。用 held-out、bad-case、closed-loop 和成本做过滤/加权消融，不能从 demo 总数或 train loss 推出泛化。

约 2 分钟机制展开：把三者分开：质量回答“这一条样本是否可信、可执行、可对齐”；数量回答“有效 support 有多大”；多样性回答“support 是否覆盖部署会遇到的条件”。一个简单的 coverage 记法是：

$$\text{coverage}(g) = \frac{|\{\text{observed groups in } g\}|}{|\{\text{target groups in } g\}|}, \qquad N_{\text{eff}} \approx \frac{N}{1 + (m-1)\rho}.$$

这里 g 可以是 task×scene×object×init×failure type，N 是窗口数，m 是同一 episode/轨迹产生的相关窗口数，ρ 是它们的平均相关性；第二式只是提醒“连续重叠窗口不等于独立样本”，不是精确统计估计。报告 raw windows、unique episodes、unique initial states、group coverage 和 long-tail counts，而不是只报总帧数。

维度	先检查什么	常见假象	对应证据/指标
schema / label quality	obs/action shape、时间戳、轴序、单位、normalizer、mask、episode boundary、success 定义	loss 能下降但 action 轴错、跨 episode 拼窗、proxy label 当真值	schema validator、round-trip、missing/NaN/clip、label audit
quantity	unique episode、task、init、有效 action window、重复率	一个长 episode 切出很多相邻 frame，数量大但 support 不增	unique/group counts、duplicate rate、effective sample proxy
diversity / coverage	task/scene/object/view/init/control phase/failure type	只增加成功中心或单一 task，平均 loss 变好但 OOD/recovery 不变	group coverage、分层直方图、entropy/long-tail recall
temporal / causal integrity	action-observation 对齐、chunk 边界、future leakage、source lineage	随机 frame split 看似泛化，实际同 episode 泄漏	episode/group split、timestamp delta、future-field audit
closed-loop relevance	stop reason、contact/interaction、first divergence、recovery state	训练 MSE 或 contact proxy 被当作任务成功	matched rollout、bad-case taxonomy、success/invalid/stop breakdown

数量与多样性的取舍：先建立干净 baseline，再按缺口分层补数据；不把所有新样本混成一个 pool。相同 task 的新 episode 主要增加数量，相同数量但新 object/initialization/failure phase 才可能增加 coverage；成功轨迹中心的数据能降低 in-support noise，却未必提供恢复能力。长尾样本可通过分层采样、质量标签或有限 loss weight 进入训练，但权重不能替代缺失的 state/action support，也不能把 proxy reward 变成可靠 label。

质量过滤的原则：硬删除明显 schema 破坏、时间错位、越界且无法解释、重复泄漏和无法回放的样本；对可恢复失败、接管前状态、困难物体和 suboptimal 但可执行动作保留 `quality/failure/source` 标签，分别做全量、过滤、加权三臂 held-out/rollout ablation。过滤规则要在 train 外的 bad-case 集上验证，不能先看 test 成功再反推“低质量”。

项目实现口述：我会这样讲：“WAM 的 `RoboCasaDataset` 混合 demonstration、archived rollout 和 world-model 采样概率，首先要查 action shape/axis、时间窗口、normalizer 与 success proxy；`success_rollout_sampling_prob` 改的是采样 mixture，不自动增加 state coverage。UWM RoboCasa 迁移的 18 tasks、9126 episodes、133457 sequences 看起来数量大，但 `contact_window_weight` 和 reward/done 只是接触/成功 proxy，full-h4 的接触也没有转成 task success，所以必须按 task、scenario、init、phase 和 failure type 重算 coverage。Harness 的 8916 条样本有 task-family 分布 `3735/2748/2300/120/13`，但 validation 为空；OPD 的多 teacher/source route 与 raw holdout 也要防 source skew 和 leakage。它们都支持数据审计与分层设计，不支持从样本量直接宣称效果。”[18][24][27][29][30]

最小验证实验：固定 checkpoint、optimizer update/token budget、seed 和评测协议，做三臂：`raw`、`schema-cleaned`、`stratified quality+coverage`；三者保持相同 unique-episode/train-group 预算，另留 episode-level held-out 与 failure/OOD holdout。报告 action/schema error、duplicate/leakage、group coverage、long-tail recall、offline loss、per-group metric、closed-loop success、first-error/invalid/stop 和数据/训练成本；只有 coverage 与闭环指标在 matched protocol 下同时改善，才支持数据设计有效。

ownership 与结论防线：当前证据支持我解释质量/数量/多样性的可测指标、项目数据分布和下一步分层实验；不支持声称已完成全量数据清洗、已证明某个 sampling weight 提升泛化、已关闭 OPD/Harness validation gap，或把公开数据规模和上游 manifest 写成个人原创成果。

追问：为什么 unique episode 比 frame 数重要？如何定义 coverage 的 target groups？连续窗口相关性怎样影响有效样本数？contact-rich 采样为什么不等于 recovery data？只保留成功轨迹会删掉什么？source/task skew 如何检测？train-only normalizer 为什么是数据质量问题？如果 schema-cleaned loss 变差但 rollout 变好，如何解释?

易错：把数据量当质量；把相邻 frame 当独立样本；随机 frame split；只报 aggregate success/平均 loss；用 reward/done/contact proxy 当 ground truth；把采样概率当新数据；把失败/困难轨迹全删；没有 held-out 就声称 diversity 提升；把多 teacher/source route 的覆盖写成无偏分布。

来源：[1][9][18][24][27][29][30][34]　练习状态：unseen

▼ L2 P1 VLA-RL-007 · demonstration 中的 suboptimal action 是否应该过滤?

30 秒回答：不应按 return 一刀切。先硬清理传感/时间错位、action 轴或 normalizer 错、越界且不可回放、重复泄漏和不可解释停顿；可恢复失败、纠偏动作、困难物体和 suboptimal-but-executable 状态应保留质量标签。再比较全量、过滤、质量/return 加权三臂，在 episode-group held-out 与闭环 rollout 上看 per-group 指标、recovery 和成本；WAM/UWM 的 proxy 分数低不能单独证明动作无用。

约 2 分钟机制展开：先回答“坏在哪里”，再决定删不删：

类别	例子	默认处理	不能直接推出
interface-corrupt	shape/axis/order/unit、timestamp、normalizer、跨 episode 拼窗错误	hard reject 或修复后重新生成	模型学不会该任务
unsafe/invalid	越界、NaN、控制器拒绝、急停、无法回放	quarantine，单独统计，不作为普通正例	该状态没有训练价值
noisy-but-valid	轻微抖动、次优路径、低 return 但动作可执行	保留 `quality/source`，可降权或分层	低 return 等于错误标签
recovery/hard-state	偏航后的纠偏、遮挡、接触、困难初态	保留甚至提高覆盖优先级	一定应该模仿完整轨迹
label/proxy-conflict	success/reward/done/contact 与 task predicate 或 expert judgement 冲突	标记 conflict，做独立 holdout/人工复核	proxy 分数低就应删除

return/advantage 只能是一个 feature，不是唯一质量标签。可以用一个带上下限的 sample weight：

$$w_i = \mathrm{clip}(w_{\mathrm{schema}}(i) \cdot w_{\mathrm{coverage}}(g_i) \cdot f(r_i,\, q_i), w_{\min}, w_{\max}), \qquad \mathcal{L} = \frac{\sum_i w_i\, \ell_i}{\sum_i w_i + \varepsilon}.$$

其中 w_{schema} 只允许通过硬质量门的样本进入，$w_{coverage}$ 用于补长尾 group g_i，r_i 是 return/reward proxy，q_i 是人工或规则质量标签；权重需在 train 内拟合并在 held-out 上验证，不能用 test success 反推 w_i。如果标签本身不可信，降权只是隐藏问题，不能替代修复或重新标注。

为什么不只保留成功 episode：成功轨迹通常覆盖任务中心和 expert 的常规路径，却删除了偏航、接触、遮挡、停滞和 recovery 状态；同时“成功”可能来自宽松 scorer、偶然碰撞或错误 proxy。相反，所有失败也不应保留为正例：不可执行/接口损坏样本应隔离，失败轨迹中的可恢复片段可以作为 hard-state 或 intervention candidate。正确做法是把 `outcome`、`action_validity`、`recovery_value`、`source` 分开标记。

三臂验证：固定 checkpoint、seed、group split、optimizer updates/token budget 和 normalizer，比较 `all-valid`、`hard-filtered`、`quality+return-weighted`；另设 full task、long-tail/failure 和 OOD holdout。报告 schema/action error、per-group offline loss、coverage、成功/失败类型、first-error/recovery、invalid/stop、数据保留率和计算成本。若过滤后平均 loss 下降但 long-tail/recovery 下降，说明清洗过度；若加权只提高 proxy、闭环不变，说明标签或目标不对齐；若全量优于过滤，不能据此说噪声无害，还要看是否保留了有价值的困难状态。

项目实现口述：我会这样讲：“WAM 的 `RoboCasaDataset` 会混合 demonstration、archived rollout 和 world-model 采样；`success_rollout_sampling_prob` 不是质量标签，manifest 的 reward/done/success 也不能替代 action validity 和 task predicate。UWM RoboCasa 的 `contact_window_weight` 是监督 emphasis，full-h4 中 45/54 有 robot-env contact 但 success 为 0/54，说明接触或低 loss proxy 不能直接当好动作；公开 LeRobot 的 reward/done 还缺真实接触与对象状态。OPD 里严格 parser/format conflict、source route 和 raw-holdout 去重应分开审计；Harness 的 reward components 也是协议 proxy，不能拿 tool/scope 分数决定物理 action 是否该保留。我的结论会是：先清合同损坏，再保留并标记可恢复/困难样本，用 matched 三臂实验决定过滤或加权。”[5][9][18][24][27][29][30][34]

ownership 与结论防线：当前证据支持设计 action-validity/quality/failure/source 标签、全量/过滤/加权对照和项目 proxy 边界；不支持声称已经完成全库清洗、证明某个 return threshold 最优、证明 sampling weight 提升闭环、或把上游数据规则写成个人原创算法。

追问：return 低但动作可执行时为何不删？怎样区分接口错误和真正 suboptimal action？quality weight 会不会让长尾样本梯度过大？如何避免按 frame 的 quality label 泄漏 episode？如果 filtered offline loss 降低但 success 降低，优先查哪里？contact-rich window 为什么不能直接当正例？success proxy 和 expert label 冲突如何仲裁？

易错：只保留成功轨迹；把低 return 当错误标签；用接触/reward/done proxy 代替 task success；把 action noise 叫 recovery；在 test 上调过滤阈值；忽略 normalizer/axis/time alignment；把 loss 降低当作数据质量证明；把数据规模或过滤规则写成个人创新。

来源：[5][9][10][18][24][27][29][30][34]　练习状态：unseen

▼ L3 P1 VLA-RL-008 · 如何检测 trajectory leakage、task overlap 和 train/eval contamination?

30 秒回答：先按原始 episode/trajectory 切分，再 window/augmentation；为 episode、scene、task template、object、image/action/prompt 建稳定 ID/hash，查 exact、近重复 cluster 与语义 overlap。normalizer/阈值只用 train，dev 选 checkpoint，final test 锁定。UWM 当前先构造全窗

口 dataset/normalizer 后 random-split window，`val_ratio>0` 有同 episode 与 stats 泄漏风险；已审计 30-epoch run `val_ratio=0` 是无 validation，不是已发生泄漏。[35]

约 2 分钟机制展开：先冻结"泛化轴"，再判断 overlap。比如目标是 same-task new-object，则 task template 可共享、object instance 必须隔离；目标是 unseen-task，则 template 和语言语义簇也要隔离。没有预先定义 split axis，看到相同词或物体就叫泄漏，同样不严谨。

$$\text{overlap}_k = \frac{|H_k(D_{train}) \cap H_k(D_{eval})|}{\max(1, |H_k(D_{eval})|)}, \qquad k \in \{episode, image, scene, object, prompt, trajectory\}.$$

H_k 是按第 k 类稳定字段或 canonical hash 生成的集合。exact hash 检查完全重复；perceptual image hash、prompt embedding、action-sequence distance 或 trajectory fingerprint 用于 near-duplicate clustering；semantic overlap 则输出 task×object×scene 矩阵，不自动判 FAIL，而是与预注册泛化轴对照。

泄漏层级	典型问题	检测方式	正确修复
source/episode leakage	同一 episode 的相邻窗口分到 train/eval	episode/source ID、window parent、timestamp range	先按 raw episode group split，再 windowing
frame/image near duplicate	相邻帧、同图不同 crop/路径、视频复用	SHA/perceptual hash、video+frame ID、cluster split	按 cluster/原视频分组，不只删 exact row
task/object/scene contamination	eval object instance 或 scene config 已在 train	template/object/scene/config hash overlap matrix	按目标泛化轴重新分组；合法 overlap 单独报告
transform/statistics leakage	全量数据拟合 normalizer、词表、阈值或过滤规则	artifact input manifest、train ID lineage、stats SHA	只用 train 重算并重新训练/评估
model-selection leakage	用 final test 选 checkpoint、调 parser 或阈值	run ledger、query count、selection rule	dev 选型；final 冻结后一次性评测
protocol contamination	不同 n/scorer/denominator 的结果被拼成 uplift	model/data/scorer/protocol tuple	重新做 matched evaluation；不要误称数据泄漏

为什么删除 eval 重复还不够：如果泄漏样本、全量 normalizer 或 test-derived threshold 已经影响训练/选型，只在最终表里删重复行不能撤销信息流；要从 raw source 重新 group split、重算 train-only artifacts、重训并重新选择 checkpoint。近重复也应先 cluster 再 split，否则同一视频的不同 crop/window 仍会跨边界。

项目实现口述：我会这样讲："UWM RoboCasa 的 dataset 先发现 episodes、构造所有 sequence windows并可在初始化时对 `self.episodes` 计算 action stats；训练脚本随后才对 dataset index 调 `random_split`。因此默认 `val_ratio>0` 路径有相邻 window 同 episode、全量 stats 污染 val 的结构风险。已审计 actionheadfix run 显式 `val_ratio=0, eval_every=0`，所以当前能说的是没有 held-out validation，而不是已经观察到 train/val leakage。"[18][35]

"OPD raw-holdout 是相对正例：10k 行按 image-hash 做 train/eval 阻断，source-line、fingerprint、image+prompt+answer 重复和 train-image leakage 均为 0；但有 1168 个重复图片路径、2917 行位于这些图片上，这是多标注共享底图，不等于重复样本，需要在 image-level/annotation-level 两列报告。Harness 的 base `n=1333`/GRPO ladder `n=1000` 且 scorer backend 不同，是 protocol mismatch，不是 trajectory leakage；OpenPiVLA 97.4 缺 raw episodes/checkpoint/protocol，当前无法完成 contamination audit。"[24][29][34][35]

最小审计产物：输出一份 split manifest，逐 group 保存 `source/episode/task_template/object/scene/image/video_frame/prompt/action` ID/hash、parent window、split、normalizer/checkpoint/scorer SHA；生成 exact/near/semantic overlap 表和 train→dev→final lineage DAG。对 UWM 做 episode-group split + train-only normalizer 的 clean rerun，与现有 window-random/default path matched 比较 offline、per-task、closed-loop 和置信区间。

ownership 与结论防线：当前证据支持指出 UWM split/normalizer 的结构风险、OPD dedupe 正例和 Harness/OpenPiVLA 的协议/证据缺口；不支持声称已证实某次 UWM run 发生 leakage、已完成 clean retrain、OpenPiVLA 97.4 无污染，或把 OPD 上游数据来源与全部去重实现写成个人独立成果。

追问：同一 task template、不同 object instance 算不算泄漏？为什么 window random split 风险高？normalizer 为什么也是信息泄漏？共享图片但不同标注如何处理？near-duplicate threshold 谁来调？final test 被查看一次后还能继续称 final 吗？n/scorer 不同为什么是 protocol mismatch 而不是 contamination？发现泄漏后为什么必须重训？

易错：先 window 后随机切分；只查行级 exact duplicate；把合法技能重叠全叫泄漏；用全量数据算 normalizer/阈值；从 final test 选 checkpoint；删掉 eval 重复却不重训；把不同 n/scorer 的分数差叫数据污染；没有 raw episodes 仍宣称 zero leakage。

来源：[9][11][18][24][29][34][35]　**练习状态**：unseen

B. Policy Gradient、GAE 与 PPO

▼ `L1` `P0` **VLA-RL-009 · REINFORCE 的目标函数和高方差来源是什么？**

30 秒回答：REINFORCE 最大化 on-policy 期望回报，用 score-function 梯度 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)(G_t - b(s_t))$ 提高高回报动作概率。高方差来自动作/转移采样、长 horizon 的 credit assignment、稀疏或 noisy reward、有限且相关的 rollout。state-only baseline 期望为零，不改变梯度；reward-to-go、更多独立轨迹和归一化可降方差，GAE/bootstrap 则做 bias–variance 取舍。Harness V2 用组内均值/std 替代 critic；同组等分时 advantage=0，这是无学习信号，不是收敛。

约 2 分钟机制展开：令轨迹 $\tau = (s_0, a_0, \dots, s_T)$ 由当前策略与环境共同采样，G_t 是从时刻 t 开始的 discounted reward-to-go。REINFORCE 用 log-derivative trick 把不可直接求导的采样过程写成：

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k, \qquad \nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big(G_t - b(s_t)\big)\right].$$

γ 是 discount factor，$b(s_t)$ 是不能依赖当前 sampled action 的 baseline。直觉是：回报高于 baseline 的动作提高概率，低于 baseline 的动作降低概率。使用 reward-to-go 而不是把整条轨迹总回报乘到每个 action 上，会去掉 t 之前、理论上不受 a_t 影响的随机 reward，因此不改期望而减少无关噪声。[36]

baseline 不改变期望梯度，是因为在给定状态下：

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0.$$

连续动作时把求和换成积分。这个结论要求 baseline 与当前 sampled action 条件独立；学一个 state-value baseline 本身不必让 policy gradient 有偏，但若把 Monte Carlo return 换成不准确的 TD/bootstrap advantage，或用未经证明的 action-dependent baseline、reward clipping/shaping，就可能引入 bias 或改变优化目标。[2][36]

高方差来源	典型表现	优先手段	边界
policy / dynamics / initial-state sampling	同 checkpoint、不同 seed 的 return 与梯度方向波动	更多独立轨迹、paired seed、分层采样	同一长轨迹的 window 不是独立样本
长 horizon 与 credit assignment	早期 action 乘上大量未来随机 reward	reward-to-go、state baseline、短 rollout + 合法 bootstrap	截短 horizon 可能丢失长期目标
sparse / delayed / heavy-tail reward	大量零回报，少数轨迹主导 update	reward normalization、可信 dense component、success/failure 分栏	shaping/checker 必须防 shortcut 与目标漂移
noisy checker / proxy reward	同 completion 重判翻转，advantage 符号不稳	scorer repeat、agreement/flip/tie、hard-case audit	proxy 分数不等于机器人 ground-truth return
有限且相关的 rollout batch	梯度置信区间宽、batch 间 oscillation	扩大 unique group、报告 effective sample 与 gradient norm	单纯 gradient accumulation 不产生新轨迹
stale / off-policy sample	log-prob 与数据生成策略不匹配	on-policy freshness 或 importance correction	这是 bias/合同错误，不只是高方差

项目实现口述：我会这样讲："Robot Harness V2 对每个 prompt 用当前模型采样 K 个 completion，经 schema-aware checker 得到 R_i，再计算组内相对 advantage，并在 completion token 上优化 clipped ratio loss：

$$A_i = \frac{R_i - \bar{R}}{s_R + 10^{-4}}, \qquad \rho_{i,j} = \exp(\log \pi_\theta(y_{i,j}) - \log \pi_{\text{old}}(y_{i,j})), \qquad L = -\frac{1}{N_{\text{tok}}} \sum_{i,j} \min(\rho_{i,j} A_i, \text{clip}(\rho_{i,j}, 1-\epsilon, 1+\epsilon) A_i).$$

这条路径是 critic-free group-relative policy gradient：组均值相当于 prompt-local baseline，组 std 做尺度归一化，代码还可加 reference log-prob penalty。最终 run 记录的是 `num_generations=8`、1007 steps；报告明确晚期很多组的 8 个候选同分，因此 $s_R = 0$、全部 $A_i = 0$、policy loss 为零。这个现象直接说明 group reward 没有区分度，不能解释成策略已收敛。"[27]

"但它也不是原始机器人 trajectory REINFORCE：采样单位是 language completion，reward 是 terminal schema/checker proxy，没有真实 environment transition、物理 effect 或 step-wise reward-to-go。早期出现 `grad_norm=nan` 后通过降低 learning rate/reward scale、缩短 prompt/completion 与收紧 grad norm 完成 restart；这只能称数值稳定性修复，不能在缺 reward/advantage/gradient 对照时归因于 Monte Carlo 高方差。旧版 Harness 更不是 on-policy estimator：它从 reference/empty/wrong-tool/reversed 固定候选算离线 sample weight，再优化 weighted reference-token CE。"[16][27]

"WAM 与 UWM 当前本地训练分别是多头 MSE 和 diffusion/noise MSE 的 fixed-dataset supervised learner；env rollout 主要写 success/video，没有 log π、reward-to-go 或 rollout→learner 回流。因此我能用它们解释 sparse success 与昂贵 rollout 为什么会让未来 online RL 难，但不能说这两个复现已经运行 REINFORCE。"[18][19]

最小诊断实验：固定 model/checkpoint、prompt holdout、checker revision、temperature、token budget 和 update 数，对 $K = 2/4/8$ 做 matched sweep；逐 prompt 保存 raw reward、组均值/std、zero-variance group rate、advantage quantile、completion entropy、token KL、clip fraction、gradient norm 与 scorer repeat flip/tie。比较 raw return、只减均值、均值+std normalization 三臂，并报告 unique prompt/completion 数、训练/判分成本和独立 holdout。只有 reward diversity、梯度稳定性与 held-out 指标同时改善，才能说 variance-reduction 设计有效。

ownership 与结论防线：当前证据支持解释 V2 的 policy sampling、group normalization、clipped token loss、equal-reward zero-signal 和旧版 offline-weighted-CE 边界；不支持把 checker reward 当真实机器人 return、把 NaN 单因归成 REINFORCE 方差、声称已做上述 K 消融、声称 WAM/UWM 使用 policy gradient，或把脚本逐文件实现写成个人独立成果。

追问：score-function trick 为什么成立？reward-to-go 为什么比 full-trajectory return 方差低？state baseline 为什么无偏，action-dependent baseline 呢？加大 gradient accumulation 为什么不等于更多 rollout？组内 std 很小时会怎样？equal-reward group 是“低方差”还是“零信号”？GAE 在哪里引入 bias–variance 取舍？off-policy 数据不校正是方差问题还是 bias？checker repeat flip 如何进入梯度不确定性？

易错：把 REINFORCE 等同所有 policy gradient；说任意 baseline 都无偏；把 reward normalization、clipping、shaping 都说成不改目标；把相关 window 当独立轨迹；只看平均 reward 不看 group std；把 equal-reward zero loss 当收敛；把 numerical NaN 自动归因高方差；把旧版 weighted CE 或 WAM/UWM supervised loss 叫 on-policy RL；把 schema reward 叫物理成功。

来源：[2][7][16][18][19][27][36]　**练习状态**：unseen

▼ L2 P0 VLA-RL-010 · actor-critic 为什么比纯 Monte Carlo 更实用？

30 秒回答：Actor–critic 用 critic 估计 V/Q，以 TD/GAE advantage 替代纯 Monte Carlo return，可提前更新并降低长轨迹方差。bootstrap/函数逼近会带来 bias，critic 追不上 actor 会给错方向；要查终止 mask、explained variance 与两路梯度。VLA action chunk 还要按实际 K 聚合 reward、用 γ^K bootstrap。本地 WAM value head 只回归离线 return，未给 action head advantage；Harness GRPO 也没有 critic，都不能称 actor–critic。

约 2 分钟机制展开：纯 episodic Monte Carlo 要等轨迹结束后得到 G_t，目标对真实 return 无 bootstrap bias，但长 horizon 会把后续动作、环境和 reward 的全部随机性传回早期 action。Actor–critic 引入参数化 critic $V_\phi(s)$ 或 $Q_\phi(s,a)$，用 temporal-difference error 同时提供低方差 learning signal：

$$y_t^{\mathrm{TD}} = r_t + \gamma m_t V_{\bar{\phi}}(s_{t+1}), \qquad \delta_t = y_t^{\mathrm{TD}} - V_\phi(s_t), \qquad \hat{A}_t \in \{\delta_t, \hat{A}_t^{(n)}, \hat{A}_t^{\mathrm{GAE}}\}.$$

$m_t = 0$ 只对应真实 MDP terminal；time-limit `truncated` 若状态本可继续，通常仍令 $m_t = 1$。$\bar{\phi}$ 表示用于 target 的 critic 参数，可能与当前 ϕ 相同、延迟或由 target network 提供，取决于算法。一步 TD 能立刻更新但更依赖 critic；n-step/GAE 延长真实 reward 链，在 bias 与 variance 间插值。[2][37]

典型 stochastic actor–critic 的两条损失是：

$$L_{\mathrm{actor}} = -\mathbb{E}_t\Big[\log \pi_\theta(a_t \mid s_t)\,\mathrm{stopgrad}(\hat{A}_t)\Big], \qquad L_{\mathrm{critic}} = \frac{1}{2}\mathbb{E}_t\big(V_\phi(s_t) - y_t\big)^2.$$

`stopgrad` 防止 actor loss 通过 advantage target 反向“改答案”。Actor 依赖 critic 的相对动作/状态评价，critic 又在 actor 持续改变的 occupancy 上学习，所以这是 two-time-scale、non-stationary optimization：critic 太慢会给 stale advantage，太快或过拟合 noisy reward 会把错误放大给 actor。value loss 下降只说明拟合当前 target，不证明 target、bootstrap mask、action ranking 或 closed-loop success 正确。[37]

estimator	何时可更新	variance	主要 bias / 风险	必看诊断
Monte Carlo G_t	episode 结束后	高，尤其长 horizon / sparse reward	无 bootstrap bias；仍受有限样本和 policy sampling 影响	return CI、trajectory 数、reward scale
one-step TD	每个 transition	较低	critic approximation、bootstrap、错误 terminal mask	TD residual、target drift、explained variance
n-step / GAE	收集短 rollout 后	可调	n, γ, λ、尾部 value 与截断语义	advantage mean/std、bootstrap fraction
group-relative，无 critic	同 prompt/group 判分后	依赖组内多样性	checker proxy、group composition、同分零信号	group reward std、tie/flip、zero-advantage rate

VLA / action chunk 的额外合同：若 actor 每次预测 K 个低层动作并开环执行到 $t+K$，critic 的一个 decision step 不能仍偷用单步 target。应先聚合执行期间 reward，再在下一个重规划状态 bootstrap：

$$R_t^{(K)} = \sum_{j=0}^{K-1} \gamma^j r_{t+j}, \qquad \delta_t^{(K)} = R_t^{(K)} + \gamma^K m_{t+K} V_\phi(s_{t+K}) - V_\phi(s_t).$$

如果中途 success、碰撞或急停，K 要换成实际执行长度并正确设置 mask。还要先确认 policy 是否有可计算的 action/chunk log-prob；对只有 deterministic regression、diffusion denoising loss 或外部 sampler 的 VLA，不能因为加了 value head 就直接套 stochastic actor loss。

项目实现口述：我会这样讲：“Cosmos/WAM 本地 `CosmosPolicyDeepSpeedProxy` 共享 `encoder→trunk`，再接 action、future-proprio、future-image 和 value heads。dataset 以 `return_value_function_returns=True` 提供 `value_function_return`；代码只对 batch target 做 masked value MSE，再与 `16×` action MSE 和 world losses 相加，`AdamW(model.parameters())` 更新全部共享参数。它没有 s_{t+1}、TD residual、behavior log-prob，也没有用 value/advantage 重加权 action loss，所以准确说法是 supervised auxiliary value-return regression，不是 critic 驱动 actor。”[17][21]

“这个 value head 可能通过共享 trunk 改变 action representation，但当前没有 stop-gradient、separate critic、branch-gradient cosine 或 matched closed-loop ablation；因此不能从 value loss 下降推断 action policy 改善，也不能把它说成我实现了 actor–critic。Harness V2 虽然有

`advantage` 和 clipped policy loss，但 advantage 来自同一 prompt completion 的 group mean/std，没有 $V_\phi(s)$、TD target 或时间递推，是 critic-free GRPO-style update。UWM 则是 fixed-dataset diffusion/noise MSE，rollout 只做 success/video 评测并不回流 learner。”[19][21][27]

最小补强实验：先在 simulator/shadow 环境冻结 action parser、normalizer、controller、task/seed/init、reward/termination 和 checkpoint lineage；每个 decision 保存 observation、完整 action chunk、实际执行 K、逐步 reward、next observation、`terminated/truncated`、old log-prob 与 policy version。做同 rollout budget 的 Monte Carlo value baseline、one-step TD、GAE 三臂，并加 shared trunk vs critic stop-gradient/separate encoder 隔离；报告 return/success、invalid/stop、critic explained variance、return-bin calibration、TD/advantage quantile、actor/critic gradient norm/cosine、KL/entropy 和成本，而不是只比 value loss。

ownership 与结论防线：当前证据支持解释 WAM 的 masked auxiliary return regression、共享参数路径、Harness critic-free group advantage 与 UWM learner/eval 数据流；不支持声称本地已有 TD/GAE actor–critic trainer、critic 提升了 WAM action success、shared value branch 已因果有益/有害、diffusion policy 已具可用 log-prob、真实机器人 online RL uplift，或逐文件个人独立实现。

追问：actor–critic 一定比 Monte Carlo 无偏吗？critic 是 V 还是 Q 时 actor signal 有何不同？为什么 value loss 低仍可能给错 policy gradient？`terminated` 与 `truncated` 怎样设置 bootstrap？actor/critic 学习率为什么常需两时间尺度？action chunk 为何出现 γ^K？中途 terminal 怎么处理？shared encoder 的 value gradient 可能怎样帮助或伤害 action head？diffusion VLA 没有 tractable log-prob 时怎么办？Harness group advantage 为什么不是 critic？

易错：看到 value head 就叫 actor–critic；把 batch return regression 当 TD；把 reset 用的 `done` 直接当 bootstrap mask；说 critic 只降 variance 不引入 bias；只看 value MSE 不看 calibration/advantage；actor loss 反传进 advantage target；按低层单步 target 训练 K-step chunk critic；默认所有 diffusion policy 都有可用 log-prob；把 Harness group baseline 叫 value network；从 shared-head loss 推断闭环因果收益。

来源：[2][17][18][19][21][27][37]　**练习状态**：unseen

▼ `L2` `P0` VLA-RL-011 · GAE 如何在 bias 和 variance 之间插值?

30 秒回答：GAE 将 TD residual 按 $(\gamma\lambda)^l$ 加权。$\lambda=0$ 是一步 TD，低方差但更依赖 critic/bootstrap；$\lambda\to1$ 接近 discounted Monte Carlo，通常减 critic bias、增轨迹方差。γ 管 horizon，λ 管 residual mixing。`terminated` 后不 bootstrap，time-limit `truncated` 通常仍 bootstrap。本地无 GAE trainer：WAM 回归 return，Harness 算 group advantage，OpenPiVLA 仅作 eval 退出。

机制展开（约 2 分钟）：先定义一步 TD residual。令 m_t 是 critic 的 bootstrap mask，而不是笼统的 episode-loop `done`：

$$\delta_t = r_t + \gamma m_t V_\phi(s_{t+1}) - V_\phi(s_t).$$

对有限轨迹，GAE 再把从当前时刻开始的 residual 做指数加权：

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

当 $\lambda=0$ 时只剩 δ_t，随机 reward 的方差较小，但误差更受 V_ϕ 和一步 bootstrap 支配；当 $\lambda\to1$ 时会纳入更长的 reward/residual 链，趋近 discounted Monte Carlo，通常降低 critic-bootstrap bias，却把环境、动作和长轨迹随机性带进来。这里不是无条件的“λ 越大 bias 越小”：$\gamma<1$ 本身改变优化 horizon，错误终止 mask、有限 rollout 截断和不准的尾部 value 仍会引入偏差。GAE 只是 advantage estimator，可被 PPO 等 actor-critic 使用，并非 PPO 专属。[2]

终止语义是实现高频坑。真正 MDP terminal 后没有可继续的未来，令 $m_t=0$；若只是外部 time limit 截断、底层任务仍可继续，通常令 $m_t=1$ 并用 $V(s_{t+1})$ bootstrap。训练循环可以用 `done = terminated or truncated` 决定何时 reset，但不能直接把这个 `done` 当 critic mask，否则会把 time-limit state 错当 terminal，系统性压低 value/advantage。[12]

本地代码提供的是三种相邻但不同的语义：WAM proxy 从 batch 读取 `value_function_return` 做 value-head regression，没有构造 TD residual 或 actor advantage；Harness 的 `advantage = ref_score - mean_reward` 是同一样本合成候选组的相对权重，没有时间递推；OpenPiVLA true-rollout eval 用 `terminated or truncated` 停止 episode，并主要从 `terminated` / `info` 判 success，它是评测控制流，不是 critic 训练。故当前能说“我核对了 return、group advantage 与 rollout termination 的语义边界”，不能说“我在这些项目里实现或跑过 GAE”。[16][17]

项目挂钩：如果面试官要求把 GAE 接到机器人 online post-training，我会先锁定 reward scale、γ/λ、rollout horizon、`terminated` / `truncated` mask 与尾部 value，再记录 advantage mean/std、explained variance、KL 和 success；当前五项目没有可归属的 GAE run，不能拿 WAM value head 或 Harness group weighting 代替。

追问：GAE 是 PPO 专属吗？$\lambda=1$ 是否等于无偏 Monte Carlo？为什么 reset 的 `done` 不能直接作为 critic bootstrap mask？WAM 的 value return、Harness 的 group advantage 分别缺少 GAE 的哪一步？

易错：把 γ 和 λ 都解释成“学习率”；把 `terminated` 与 time-limit `truncated` 合成同一个 bootstrap mask；看到 value head 或变量名 `advantage` 就声称项目使用了 GAE。

来源：[2][12][16][17]　练习状态：unseen

▼ L2 P0 **VLA-RL-012 · PPO clipped objective 如何限制策略更新?**

30 秒回答：PPO 用 rollout 时冻结的 old policy 算概率比 $r_t = \pi_\theta/\pi_{old}$，再对 $r_t\hat{A}_t$ 与 clipped 项取 min：正 advantage 限制继续增概率的收益，负 advantage 限制继续降概率的收益，有害方向仍受惩罚。它不是硬 KL 约束，也不裁动作值，要联合看 approx KL 与 clip fraction。Harness 没有 old log-prob/ratio；OpenPiVLA 的 `np.clip` 只是 env action bound，二者都不是 PPO。

机制展开（约 2 分钟）：rollout buffer 必须保存采样动作 a_t 在冻结行为策略 $\pi_{\theta_{old}}$ 下的 old log-prob。更新时用当前策略重算同一动作的 log-prob，得到 importance ratio：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} = \exp(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{old}}(a_t \mid s_t)).$$

这里 $r_t > 1$ 表示当前策略提高了该采样动作的概率，$r_t < 1$ 表示降低。PPO 的常见 clipped surrogate 是：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big].$$

实现通常最小化它的负值。若 $\hat{A}_t > 0$，提高该动作概率是有利方向，ratio 超过 $1+\epsilon$ 后收益被封顶；若错误地降低概率，unclipped 项仍更差。若 $\hat{A}_t < 0$，降低该动作概率是有利方向，ratio 低于 $1-\epsilon$ 后收益被封顶；若继续提高坏动作概率，unclipped 项仍继续惩罚。这就是负 advantage 时仍取 `min` 的原因：clipping 截断“过度获益”的方向，不替有害更新兜底。[3]

它只是在采样动作的 surrogate 上形成近似 trust-region，不保证每个状态的 KL 都受硬约束。多轮 minibatch epoch 后参数仍可能漂移，旧 rollout 也会让 ratio 失真；因此还要监控 approx KL、clip fraction、entropy、value loss、explained variance 与 return，必要时按 KL early-stop。`clip fraction` 是 ratio 落出区间的样本比例，不是动作越界率或成功率。[3]

本地有两个容易混淆的实现。Harness 先把 reference 相对合成组均值的 advantage 变成 $w = \text{clip}(1 + A, 0.2, 2.0)$，再最小化 weighted reference token CE；代码没有 rollout-time old log-prob、current/old ratio 或 PPO surrogate。OpenPiVLA true-rollout 的 `_clip_action` 则把模型动作 reshape/pad/truncate 后，用 `np.clip(action, env.action_space.low, high)` 裁到控制边界，并记录 `clip_fraction`；它改变的是送进环境的数值动作，不是策略分布概率比。前者是 sample-weight clipping，后者是 actuator-bound clipping，均不能包装成 PPO。[16][17]

项目挂钩：当前能诚实口述的是“我从实现合同区分了 policy-ratio、sample-weight 与 action-bound 三种 clipping”。不能说自己已跑 PPO。若要接入机器人 PPO，至少要冻结 policy-generated rollout，保存 observation/action/reward/`terminated`/`truncated`/value/old-log-prob，计算 GAE，再做 ratio-clipped minibatch update，并保存 KL、clip fraction、entropy、value 与独立 rollout 指标。

追问：为什么 advantage 为负时仍取 `min`？ratio 已超出 clip 区间时梯度一定为零吗？ratio 均值接近 1 为什么 KL 仍可能很大？OpenPiVLA 的动作 `clip_fraction` 与 PPO clip fraction 分别统计什么？

易错：把 clip 当作参数或 KL 的硬约束；认为 ratio 一出区间所有方向都停止更新；把 Harness sample weight clipping 或 OpenPiVLA action clipping 说成 PPO；没有 rollout-time old log-prob 却声称计算了可信 ratio。

来源：[3][16][17]　练习状态：unseen

▼ L2 P1 **VLA-RL-013 · PPO 中 value loss、policy loss、entropy loss 如何平衡?**

30 秒回答：PPO 常最小化 $-L_{clip} + c_v L_V - c_H \mathcal{H}$：policy 项利用 advantage，value 项拟合 return，entropy 防止过早确定化。系数没有通用比例，因为三项的单位、reduction、mask 和共享参数都不同。先核对 return/advantage，再联合看 return、KL、clip fraction、explained variance、entropy 与共享层梯度。WAM proxy 是多头 MSE，Harness 是 weighted CE；即使出现 `value_loss`，二者也不是 PPO。

机制展开（约 2 分钟）：用“最小化 loss”的符号约定，一种常见复合目标可写成：

$$\mathcal{L}_{\text{train}} = -L^{\text{CLIP}}(\theta) + c_v\,\mathbb{E}_t\Big[(V_\phi(s_t) - \hat{V}_t)^2\Big] - c_H\,\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))].$$

L^{CLIP} 是要最大化的策略 surrogate，所以进入最小化目标时取负；L_V 让 critic 拟合 return/value target $\hat{V}_t$；entropy $\mathcal{H}$ 越大表示策略越分散，因此 entropy bonus 在 loss 中取负。c_v, c_H 只是权重，不存在跨任务通用的“1:0.5:0.01 最优比例”；还要说明 value clipping、entropy 定义、action-dimension reduction 与 advantage normalization 的具体实现。[3]

若 actor、critic 共享 backbone 参数 ψ，真正作用于共享表示的是加权梯度和：

$$\nabla_\psi \mathcal{L}_{\text{train}} = -\nabla_\psi L^{\text{CLIP}} + c_v \nabla_\psi L_V - c_H \nabla_\psi \mathcal{H}.$$

所以不能只比较三项打印出来的 raw scalar：不同 target 单位、batch mask、按 token/step/dimension 的 reduction 都会改变量级；某项数值大也不等于它对共享层梯度最大。应同时记录未加权/加权 loss、有效样本数、shared-trunk 与各 head 的 gradient norm，必要时看 gradient cosine。若 value explained variance 上升、policy return 也提升，即使 value MSE 看起来大，也不应机械降权；先检查 reward/return scale 与 target noise。[3]

调参顺序应先语义、后尺度、再系数：先核对 reward、`terminated` / `truncated`、GAE、advantage mean/std 与 value target；再核对每项 mask/reduction、return normalization 和数值范围；最后结合 rollout return/success、approx KL、clip fraction、entropy、explained variance 和梯度诊断调 c_v, c_H。critic 主导且 actor 停滞时，可降低 c_v、调整 critic 学习率/更新次数、value clipping，或分离 optimizer/backbone；entropy 过快塌缩可增大或调度 c_H，持续过高且动作随机则反向调低。系数变化必须用 matched-seed rollout 验证，不能靠单项 loss 好看下结论。

本地 WAM proxy 明确构造 action、future-proprio、future-image 和 value 四个 head：默认总损失为 $16L_{action} + L_{future_proprio} + L_{future_image} + L_{value}$，后三项按各自 sample mask/denominator 归一；其中 value target 来自 batch `value_function_return`。它没有 stochastic-policy log-prob、advantage、PPO surrogate 或 entropy bonus，只能用于讲多目标 scale/mask/shared-trunk 权衡。Harness 的 `RewardWeightedTrainer` 则只优化 sample-weighted reference token CE，也没有 critic/entropy。二者都不能作为本地 PPO run。[16][17]

项目挂钩：项目口述可以说“我核对过多目标 loss 的 coefficient、mask、denominator 与 shared representation 风险，并知道如何用 per-component loss/gradient 和 rollout 指标做消融”；不能把 WAM 的 `value_loss` 叫 PPO critic，也不能把默认 `action_loss_multiplier=16` 包装成算法创新或已证明最优。

追问：value loss 很大但 return 上升，是否必须降 c_v？为什么 raw loss 更大不等于 shared gradient 更强？entropy 高是否一定代表探索有效？何时分离 actor/critic optimizer 或 backbone？

易错：直接比较三项 raw scalar；忽略符号、系数、mask、reduction 和有效样本数；只看 entropy 不看 action/return；看到 value head 就声称 actor-critic/PPO；把默认 loss coefficient 当个人算法贡献。

来源：[3][16][17]　**练习状态**：unseen

▼ L3 P1 VLA-RL-014 · PPO ratio、advantage normalization 和 clipping fraction 如何诊断训练？

30 秒回答：ratio 看 current/old policy 对采样动作的概率变化；均值接近 1 仍要看分位数、approx KL 和 clip fraction，因为正负偏移会抵消。advantage normalization 稳定 batch 尺度，但不会修复错 reward、GAE 或 mask。KL/clip fraction 都高常见于学习率或 epoch 过大、rollout 过旧；两者都低且 return 不涨则查 advantage/梯度。OpenPi 的 clip fraction 是动作越界，WAM 的 fraction 是 mask 覆盖，Harness 无 PPO ratio/KL，三者不能混用。

机制展开（约 2 分钟）：先在同一批 rollout sample 上定义三个量。N 是有效、未被 padding mask 掉的 step 数：

$$r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, \qquad \tilde{A}_t = \frac{\hat{A}_t - \mu_A}{\sigma_A + \epsilon_A}, \qquad f_{clip} = \frac{1}{N}\sum_{t=1}^{N} \mathbf{1}(|r_t - 1| > \epsilon_{clip}).$$

ratio 的经验均值应大致围绕 1，但单看均值会漏掉长尾：一部分样本 ratio 很大、另一部分很小可以互相抵消。至少记录 mean/std、p05/p50/p95/p99、max 与非有限值；若是 token policy，还要说清按 token 还是 sequence 聚合。f_{clip} 是 ratio 落出 $[1-\epsilon_{clip}, 1+\epsilon_{clip}]$ 的有效样本比例，不等于这些样本的梯度全部为零；是否进入 clipped plateau 还取决于 advantage 符号与更新方向。[3]

KL 提供分布层面的补充。对 old-policy rollout，forward KL 可写为：

$$D_{KL}(\pi_{\theta_{old}} \| \pi_\theta) = \mathbb{E}_{s,\, a\sim\pi_{\theta_{old}}}[\log \pi_{\theta_{old}}(a \mid s) - \log \pi_\theta(a \mid s)].$$

实际 `approx_kl` 是有限 batch estimator，会有噪声，必须注明方向和 reduction。ratio 均值接近 1 不推出 KL 小，因为 KL 对 log-ratio 非线性敏感。若 approx KL、ratio tail 和 clip fraction 同时上升，通常说明每批更新过猛、minibatch epoch 太多，或 rollout policy 已过旧；应降低学习率/epoch、缩短采样到更新延迟，必要时 KL early-stop，而不是只把 clip range 调宽。[3]

advantage normalization 把当前统计域内的 advantage 调成近似零均值、单位方差，能减小 reward scale 对梯度步长的影响，但不会修复错误 reward、termination mask、GAE 或 action log-prob。要先应用 padding/valid mask，再算 μ_A, σ_A；还要说明按整批 rollout 还是每个 minibatch/group 标准化。减均值会改变绝对零点，甚至翻转接近均值的小 advantage；$\sigma_A \approx 0$ 时更表示 batch 缺少可辨别信号，不能靠除以 epsilon 制造学习价值。

诊断要做联表而非单指标裁决：高 KL + 高 clip fraction + return 震荡，先降更新强度/提高 freshness；低 KL + 低 clip fraction + return 不涨，检查 advantage std、正负比例、reward/GAE/mask、学习率与梯度是否过弱；高 clip fraction 但 return 不涨，优先怀疑 advantage 噪声或步长过大；同时结合 entropy、explained variance、value bias、success 与 bad-case rollout，避免把 surrogate 健康误当任务成功。

本地三个 “fraction” 不能串线。OpenPiVLA true-rollout 的 `clip_fraction` 是每步被 `env.action_space` 上下界修改的动作坐标比例，并汇总为 `clip_fraction_mean`；WAM proxy 的 `world_model_fraction/value_fraction` 是相应 sample mask 的 batch 均值；Harness 把 sample weight 裁到 $[0.2, 2.0]$，报告只给 weighted-CE train loss/reward summary，没有 rollout-time old log-prob、ratio、approx KL 或 PPO clip fraction。因此这些日志都不能用于声称“本地 PPO 更新稳定”。[16][17]

项目挂钩：项目口述可以说“我先按 numerator/denominator 和统计对象审计指标语义，再做 ratio-tail、KL、clip fraction、advantage 与 return 联合诊断”；不能把动作越界率、mask 覆盖率、sample-weight clipping 或普通 train loss 改名成 PPO 监控证据。

追问：ratio 均值接近 1 为什么 KL 仍可能很大？clip fraction 高是否表示对应样本梯度都为零？advantage 应按 rollout batch 还是 minibatch 标准化？σ_A 接近零时怎么办？

易错：只看 ratio mean；把 clip fraction 当动作越界率或成功率；标准化前未应用 valid mask；认为 normalization 能修复错误 reward；没有 old log-prob/KL 日志却声称完成 PPO 稳定性诊断。

来源：[3][16][17]　**练习状态**：unseen

▼ L2 P1 VLA-RL-015 · on-policy rollout freshness 为什么影响 PPO 稳定性？

30 秒回答：PPO 的 ratio 分母必须是生成该 batch 的 behavior policy。actor 与 learner 版本延迟越大，ratio、KL、clip fraction 越易恶化；old log-prob 只能校正已采样动作的 likelihood，不能把旧策略访问的状态分布变成当前分布。因此同一 rollout 只做有限 epoch，并绑定 policy version、age 与 behavior log-prob。Harness 是离线 weighted CE，OpenPiVLA 只执行 JSON 动作序列，均没有该 freshness loop。

约 2 分钟机制展开：PPO 对采样动作使用 importance ratio：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{beh}}}(a_t \mid s_t)}.$$

这里的 θ_{beh} 必须对应真正生成这条 trajectory 的 checkpoint，而不是任意一个名为 old 的模型。learner 每更新一次，current policy 与 behavior policy 的 action likelihood 和访问状态分布都可能继续分离：ratio 长尾、approx KL 与 PPO clip fraction 上升后，更多样本只能提供被截断或高方差的 surrogate 信号。clipping 只限制已采样 action 上的目标，不会补回 current policy 会访问、旧 rollout 却没有覆盖的 state/action support。[3]

保留 rollout-time old log-prob 能计算 sampled-action correction，却不能重建 current-policy state visitation；事后用新模型重算 current log-prob 同样不能修复这一点。因此 PPO 可以对一个刚采完的 batch 做有限个 minibatch epoch，但每多一轮都在消费 freshness budget，不能把无限历史池继续叫标准 on-policy PPO。异步系统至少同时看版本延迟、wall-clock age 与权重退化：

$$\Delta k = k_{\text{learner}} - k_{\text{actor}}, \qquad N_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

其中 Δk 是 learner/actor policy-version lag，w_i 是诊断用 importance weight；N_{eff} 是 ratio 极端时的工程诊断，不是 PPO 论文规定的唯一 freshness 标准。完整 rollout contract 应保存 policy version 或 checkpoint SHA、采样时间、behavior log-prob、env/checker revision、learner update index，并预设可接受的 version lag/age。若高 lag 同时伴随高 KL 或高 clip fraction，就应丢弃过旧 batch、缩短 actor 队列、减少每批 epoch 或放慢 learner；若必须长期 replay，则改用有明确 off-policy correction 的方法，不能只靠 PPO clipping。[3]

本地 Harness 在 `GRPODataset` 初始化时从离线 reference 与 empty/wrong-tool/reversed 负例构造 candidate group，再把 group-relative advantage 裁成 sample weight、优化 reference token CE；它没有 policy version、collection timestamp、behavior log-prob、environment trajectory 或 rollout→learner 数据环。OpenPiVLA true-rollout 则从 `--actions-json` 读取已导出的 `reference_actions/model_actions`，裁到动作边界后逐步送入 SimplerEnv；脚本没有 model inference、optimizer 或 learner update。前者是 offline weighted CE，后者是 eval execution loop；环境里“执行了动作”不等于训练数据是 on-policy。[16][17]

项目挂钩：我会口述为：“我从本地数据流审计了 PPO freshness 所需合同，并确认现有 Harness 和 OpenPiVLA eval 都不满足 rollout-to-learner 闭环；若接 online PPO/GRPO，我会先给每条 trajectory 固化 checkpoint SHA、behavior log-prob、采样时间和 env/checker 版本，再以 lag、KL、clip fraction、ESS 与 task success 联合设 gate。”不能说“我已经实现或跑过 PPO freshness 管理”。

追问：为什么 old log-prob 不能修复 state-distribution mismatch？同一 rollout 为什么可以做有限 epoch、却不能无限 replay？异步 actor 最少要携带哪些版本字段？高 KL、高 clip fraction 与高 version lag 同时出现时先改什么？

易错：把任意旧 checkpoint 当 behavior policy；事后重算 log-prob 就声称数据重新 on-policy；认为 PPO clipping 能纠正 missing support；看到环境执行循环就叫 online learner；把 `num_generations` 或 JSON action sequence 当 current-policy rollout。

来源：[3][16][17]　练习状态：unseen

▼ L3　P2　**VLA-RL-016 · 连续动作 PPO 如何处理 squashed Gaussian、log-prob 和 action bounds?**

30 秒回答：有界连续策略可先采 $u \sim \mathcal{N}(\mu, \sigma)$，再令 $a = c + d \tanh u$。合法 log-prob 必须包含 tanh Jacobian 与 bound scale；old/current 必须对同一动作走同一变换。若直接 Gaussian 后由环境 clip，clip 是 many-to-one，不能把 clipped action 代回未裁 Gaussian 当 log-prob。UWM 是 affine denorm→adapter→env clip，OpenPi 是 JSON action→env clip，均无 policy log-prob，不是 PPO。

约 2 分钟机制展开：设每维环境下界为 ℓ、上界为 h，常见有界策略是先在无界 latent space 采样，再做可逆 squash 与 affine scale：[13]

$$u \sim \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2(s))\big), \qquad z = \tanh u, \qquad a = c + d \odot z, \quad c = \frac{h+\ell}{2},\ d = \frac{h-\ell}{2}.$$

这里 c 是 action center，$d > 0$ 是逐维 half-range；固定维或离散 gripper/control mode 不应硬塞进同一个连续 density。因为 tanh 改变体积，物理动作空间中的 density 必须用 change of variables：

$$\log \pi_\theta(a \mid s) = \sum_i \big[\log \mathcal{N}\big(u_i; \mu_i, \sigma_i^2\big) - \log\big(1 - z_i^2\big) - \log d_i\big], \qquad u = \mathrm{atanh}((a - c) \oslash d).$$

第一项是 latent Gaussian log-density，第二项是 tanh Jacobian，第三项是 action-bound affine scale。固定 bounds 下，Jacobian/scale 对同一物理动作的 old/current log-ratio 会代数抵消，但这不允许采样和重算走不同路径：rollout 最稳妥地保存 pre-tanh u、physical action、old log-prob、bounds/normalizer version，并让 current policy 用同一 contract 评估。靠近 $|z| = 1$ 时 `atanh` 与 $\log(1 - z^2)$ 数值敏感，通常保存 u、限制 log-std，并用稳定恒等式 $\log(1 - \tanh^2 u) = 2(\log 2 - u - \mathrm{softplus}(-2u))$ 或受控 epsilon。[13]

“Gaussian 后让环境 clip”是另一套接口。$a_{env} = \mathrm{clip}(u, \ell, h)$ 是 many-to-one：所有越界 latent action 都映到边界，执行分布在边界带 point mass。若 rollout 把 clipped action 覆盖写回、再用普通 Gaussian density 在该值上算 old/current log-prob，概率对象已经错位；若始终保存 preclip sample 及其 log-prob，则 PPO 优化的是 latent/preclip policy，数学上可保持内部一致，但会有大量 action aliasing、边界梯度与控制效果错配，必须监控 action-bound clip fraction。它与 PPO ratio clipping 完全不是一件事。[3]

本地 UWM RoboCasa 数据管线令 `offset=(amax+amin)/2`、`scale=max((amax-amin)/2, eps)`，再用 `action_norm=(action-offset)/scale`；rollout 的真实顺序是 `model.sample → normalizer.reconstruct → auxiliary override → action adapter → np.clip(env.action_spec)`。训练侧用 diffusion scheduler 加噪并最小化 action-noise MSE，再以 affine denorm 后的 x_0 MSE 诊断；没有 tanh-squashed Gaussian、policy log-prob 或 PPO ratio。OpenPiVLA true-rollout 只把 JSON 中的动作 reshape 后裁到 `env.action_space`；Cosmos wrapper 只设置 `unnormalize_actions=True` 与 `deterministic=True`，没有在该 wrapper 内实现 inverse transform。因此本地证据支持“我审计并跑通了动作变换边界”，不支持“这些项目使用连续动作 PPO”。[17][18]

项目挂钩：我会口述为：“我把 normalized action、physical action、adapter 后 preclip action、env action 四层分别落档；UWM 的 affine reconstruct 与 env clip 只执行一次，并用 clip fraction 查尺度/维序错误。若接 PPO，我会新增 pre-tanh latent、old log-prob、log-std、bounds 与 normalizer SHA，保证 sampling/evaluate-actions 同构。”不能把 diffusion sample、确定性 eval 或普通 clipping 说成 squashed-Gaussian PPO。

追问：tanh Jacobian 为什么在 PPO ratio 中可能抵消、却仍应正确实现？环境 clip 后保留 preclip log-prob 何时仍内部一致？为什么把 clipped action 代回 Gaussian density 是错的？gripper/control mode 是否应与连续 EEF 共用一个 Gaussian head?

易错：漏掉 affine scale；先 tanh 又按数据 stats 重复压缩；sample 与 evaluate-actions 使用不同 bounds/normalizer；把动作边界 clipping、log-std clamp、数值 epsilon 和 PPO ratio clipping 混成一件事；看到 `model.sample` 就推断存在 policy log-prob。

来源：[3][13][17][18]　练习状态：unseen

C. Offline RL 与机器人数据

▼ L1　P0　**VLA-RL-017 · offline RL 与 online RL 的核心区别是什么?**

30 秒回答：offline/online 首先看训练数据闭环：offline 在优化期间只消费固定 behavior dataset，不能用当前策略补覆盖；online 让当前或行为策略采新轨迹，并把 transition/reward 回流下一次更新。固定数据上的 BC/MSE 不因有 reward 字段就自动成为 offline RL；评测 `env.step` 也不让训练变 online。本地 WAM、UWM、Harness 都是固定数据训练，UWM/OpenPi rollout 只评测，当前没有 online learner。

约 2 分钟机制展开：offline setting 的关键不是“文件在本地”，而是第 k 次更新看到的训练支持集仍由历史 behavior policy β 决定：[4][5]

$$\mathcal{D}_{\text{train}}^{(k)} = \mathcal{D}_{\text{off}} \sim d_\beta(s, a, r, s'), \qquad \forall k.$$

shuffle、重复 epoch、重采样或从名为 `rollout_data_dir` 的目录读取轨迹，都不会新增 state-action support。优势是零额外交互、可复现且安全；代价是策略在 $d_\beta(s,a)$ 很小处缺少可靠监督，bootstrapped Q 容易外推过估计，所以 CQL/IQL 等 offline RL 方法会加 conservatism、behavior support 或 expectile/advantage filtering。[4][5]

online setting 则形成采集—更新反馈环：[10]

$$\tau_k \sim P^{\pi_k}, \qquad \mathcal{D}_{k+1} = \mathcal{D}_k \cup \tau_k, \qquad \theta_{k+1} = \mathrm{Update}(\theta_k, \mathcal{D}_{k+1}).$$

其中 P^{π_k} 是策略 π_k 与环境动力学共同产生的 trajectory distribution。online 不等于 on-policy：PPO 强调 fresh rollout，SAC/DQN/AWAC 可以 online 采集却用 replay 做 off-policy 更新；offline/online 描述数据是否在训练中刷新，on/off-policy 描述更新数据与目标策略的关系。online 能主动补失败区域，却要支付环境成本、探索和安全代价，并管理 policy version、buffer contamination 与独立 final holdout。[3][10]

还要区分 “offline training” 与狭义 “offline RL algorithm”。固定 demo 上做 action MSE、diffusion noise prediction 或 token CE 是 offline supervised/IL setting；只有 reward/return、value estimation 与 policy improvement 真正进入目标，才更适合称 offline RL。存在 `reward`、`value_function_return` 或 `model.sample` 字段不是充分证据。反过来，训练间歇执行 env rollout 但只记录 success/video、没有把 transition/reward 写回 dataset/replay/loss，也只是 online evaluation，不是 online learning。

本地 WAM proxy 从预先存在的 demo/rollout directories 构造 `RoboCasaDataset`，随后只做 `cycle(dataloader) → backward → step`；目录中的 success rollout 是 archived data，不是 learner 现场采集。UWM LIBERO `train.py` 从固定 `train_loader` 优化；`train_robomimic.py` 虽可间隔 `env.step`，返回值只写 `rollout/success_rate` 与 video，训练仍继续取原 loader batch。独立 `uwm_libero_rollout_eval.py` 载入 checkpoint、执行并写 episodes/summary，没有 optimizer。Harness 从固定 JSONL 建 `GRPODataset` 后跑 weighted CE；OpenPiVLA true-rollout 只消费 JSON action。故这些经历可以讲 offline pipeline、interleaved/standalone evaluation 与数据边界，不能说已完成 online RL fine-tune。[16][17][18][19]

项目挂钩：我会口述为：“我把数据分成 archived demonstrations/rollouts、train/validation loader、interleaved rollout monitor 和 independent final eval；只有新 episode 带 policy version、reward/checker revision 写入 replay，并实际影响后续 optimizer update，才算训练闭环 online。”若要把现有 UWM/WAM 扩成 offline-to-online，我会先冻结 offline checkpoint 与 buffer schema，再做小批安全采集、明确插入规则和 off-policy correction，并把 final eval 永久隔离。[10]

追问：online RL 是否一定 on-policy？训练中每 1k step 跑环境 success 算不算 online learning？已有 success rollout 放进固定 dataset 为什么仍是 offline？action MSE 加一个 return field 就能称 offline RL 吗？

易错：把 `env.step`、`rollout_data_dir` 或 `model.sample` 当 online RL 充分条件；把所有固定数据训练统称 offline RL；认为 replay buffer 只属于 offline；把 evaluation episode 回流训练后仍称独立 holdout；没有 buffer insertion/update lineage 却声称完成 offline-to-online。

来源：[3][4][5][10][16][17][18][19]　**练习状态**：unseen

▼ `L2` `P0` **VLA-RL-018 · offline RL 的 extrapolation error 从哪里来?**

30 秒回答：extrapolation error 不是普通泛化差，而是 critic 只在固定数据覆盖的 (s,a) 上受约束，Bellman target/actor 却查询数据外 action；网络给出虚假高 Q，actor 再偏向它，误差自我强化。先排 action transform：本地 UWM/Cosmos 已有 preclip/clip、维序、demo mean-z 与 GT replay 诊断，但 WAM 仅回归 state value、UWM 无 Q critic，所以现有失败不能归为 offline-RL extrapolation，也不能声称用过 CQL/IQL。

约 2 分钟机制展开：典型 offline actor-critic 只在数据集 transition 上回归 Bellman target：[4]

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[Q_\phi(s,a) - \Big(r + \gamma\, \mathbb{E}_{a'\sim\pi(\cdot|s')} Q_{\bar{\phi}}(s',a')\Big)\Big]^2.$$

数据 loss 只直接约束 $(s,a)\in\mathcal{D}$，但 target 中的 $a'\sim\pi$ 和 actor improvement 会查询数据未覆盖的 action。函数逼近器在这些点仍会输出数值；若噪声、bootstrap bias 或 max/actor optimization 把其中一个错高值选出来，下一轮 target 又把它传播到更多 state，形成 bootstrapping + OOD action + policy optimization 的反馈环。它不同于单纯“一次预测没见过所以不准”。[4][5]

$$a_{\mathrm{ood}}(s) = \arg\max_a Q_\phi(s,a), \qquad d_\beta(s,a_{\mathrm{ood}}) \approx 0, \qquad Q_\phi(s,a_{\mathrm{ood}}) \gg G(s,a_{\mathrm{ood}}).$$

这里 d_β 是 behavior dataset support，G 是同协议真实 rollout/Monte-Carlo return；最后一个不等式是需要用新交互或可靠评估验证的 failure signature，不是只看 critic 数值就能宣布成立。CQL 通过对数据外 action 的 Q 加保守压力，IQL 尽量不在 value fitting 中直接最大化数据外 action，再以 advantage-weighted behavior extraction 得到 policy；二者都不能修复 action parser、normalizer 或 controller 错。[4][5]

诊断要按三层走。第一层先做 action contract：锁定 observation、normalizer SHA、action dim/order、adapter、preclip/env action、controller 与 reset state，检查 round-trip，并在匹配初态下 replay dataset action。第二层做 support：比较 policy action 与 dataset 的逐维 quantile、mean-z、kNN distance/behavior likelihood、clip fraction 与 ensemble disagreement。第三层才校准 critic：对同一 state 比较 dataset action、

policy action、nearest-data action 的 Q，检查双 critic/ensemble 分歧，并将预测 Q/advantage 与锁协议 rollout return 分桶对账。若 bypass/修正 parser 后 failure 消失，是接口问题；若 parser round-trip 正确、policy action 低 support、高 predicted Q、低 realized return，才更像 extrapolation。

本地实现正好提供前两层而非第三层。Cosmos `cosmos_policy_action_diagnostics.py` 汇总 demo/base/trained action 的 mean/std/min/max，并计算 policy mean 相对 demo std 的 mean-z；`cosmos_policy_gt_action_replay_diagnostics.py` 检查 7D→12D action dim、全局 min/max normalization 与 GT env replay，同时明确 reset 初态可能不匹配，所以 replay 失败不是 demo failure 证明。UWM RoboCasa 记录 adapter 前 preclip action、env action、维序与 clip fraction。WAM proxy 的 `value_head(features)` 只回归 `value_function_return`，不是 $Q(s,a)$；UWM diffusion policy 与 OpenPi eval 也没有 action-Q critic。故我能说“先排除了/审计了 action pipeline 与 support proxy”，不能说“本地 critic 已证实过估”或“实施了 CQL/IQL”。[17][18][20]

项目挂钩：我会口述为：“遇到 rollout 失败，我不会先讲 extrapolation；先用 GT replay、normalizer round-trip、preclip→env action 和 dimension-order 排接口，再用 demo mean-z/quantile/clip fraction 看 support。只有接入 action-conditioned critic 后，出现低 support + 高 Q + 低同协议 realized return，并经 nearest-data-action/critic-ensemble 对照复现，我才把根因升级为 offline-RL extrapolation。”

追问：为什么 critic validation MSE 低仍可能有 extrapolation error？如何区分 OOD action、OOD state 与 parser mismatch？GT demo replay 失败为什么不能直接证明 controller 错？WAM value head 为什么不能拿来检查 action-Q overestimation？

易错：看到动作越界或 0 success 就归因 critic；把 state value 当 action Q；只看 dataset validation TD loss；用不匹配 reset state 做 GT replay 后强归因；把 mean-z、clip fraction 当 extrapolation 的充分证据；没有 realized return 校准却声称 Q 已过估。

来源：[4][5][17][18][20]　**练习状态**：unseen

▼ L2 P0 VLA-RL-019 · CQL、IQL 如何避免过度相信 OOD action？

30 秒回答：CQL 在 Bellman/TD loss 外，对候选动作的 Q 加 conservative regularizer，主动压低数据外高 Q；IQL 不对 OOD action 做 actor 的 max-Q，而用 expectile 回归数据动作的 value，再用 $Q - V$ 做 advantage-weighted BC。CQL 可能过保守，IQL 受数据 support 限制；本地 WAM/Harness/UWM 都没有这些 objective，不能说项目用了 CQL/IQL。

约 2 分钟机制展开：CQL 的核心是完整 Q objective 中的保守正则项，而不是单独替代 Bellman loss。离散动作的常见形式为：[4]

$$\mathcal{L}_{\mathrm{CQL}}^{\mathrm{reg}} = \alpha \left(\mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_{a} \exp Q_\phi(s,a) \right] - \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[Q_\phi(s,a) \right] \right).$$

第一项把候选动作的高 Q 往下压，第二项保留数据动作相对优势；真实训练还要与 TD/Bellman error、target network、动作采样和优化器组合。连续动作通常用 policy/uniform/noise proposal 加 importance correction 近似 log-sum-exp，不能把离散公式直接当作任意机器人实现。α 太小压不住 OOD 过估，太大则会把数据中罕见但真正有价值的动作也压掉，并增加候选动作采样与调参成本。[4]

IQL 的路径不同：它避免在 value fitting 中对数据外 action 做显式 $\max_a Q(s,a)$，先用数据 action 的 Q 与 V 做 expectile regression：

$$\mathcal{L}_V^\tau(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\rho_\tau \Big(Q_{\hat{\phi}}(s,a) - V_\psi(s) \Big) \right], \qquad \rho_\tau(u) = |\tau - \mathbf{1}[u<0]| u^2.$$

$\tau > 0.5$ 会更偏向数据中较高 Q 的上尾；它不是“计算所有 OOD action 后再挑最大”。得到 V 后，策略仍从数据动作做 advantage-weighted behavior cloning：

$$\mathcal{L}_\pi(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\mathrm{clip}\Big(\exp\Big(\beta [Q_{\hat{\phi}}(s,a) - V_\psi(s)] \Big), 0, w_{\max} \Big) \log \pi_\theta(a \mid s) \right].$$

β 控制 advantage 权重温度，$w_{\max}$ 是工程上防止少量高估样本支配梯度的上限。IQL 因而通常更贴近数据 support，但高 τ、高 β、错误 reward 或错误 Q 仍会放大数据内的坏动作；它不是自动安全证明。[5]

本地实现不能只看变量名对号入座。WAM 的 `value_head` 回归 batch `value_function_return`，没有 CQL 的 candidate-action log-sum-exp、IQL 的 expectile indicator 或 $Q - V$ AWR policy loss；Harness 的 `advantage=reference_reward-group_mean` 只变成 `[0.2,2.0]` sample weight，再进入 reference token CE，negative candidates 也没有自己的 policy log-prob；UWM 是 masked MSE/diffusion noise objective。故本地只能讲“我理解并审计 CQL/IQL 的 support 约束”，不能把 value head、group advantage 或 reward-weighted CE 改名成 CQL/IQL。[16][17][18]

项目挂钩：我会口述为：“如果面对 UWM/WAM 的离线轨迹，我先按 loss/dataflow 证明它不是 CQL/IQL；若设计 offline baseline，CQL 要记录 α、候选动作 proposal、data-action Q 与 conservative gap，IQL 要记录 τ、β、weight clipping、Q-V 分布和同协议 realized return，再做 BC/IQL/CQL 对照。”

追问：CQL 的 log-sum-exp 项为什么能压 OOD Q？连续机器人动作如何近似它？IQL 为什么不需要对 OOD action 求 max？τ、β、α 分别过大时会发生什么？Harness 的 group advantage 为什么不是 IQL advantage？

易错：把 CQL 正则项说成完整 loss；把 IQL expectile 当 quantile regression；把所有 advantage-weighted CE 叫 IQL；忽略 reward scale 对 β 权重的影响；没有 Q-V、expectile 或 candidate-action evidence 却声称实现了 CQL/IQL。

来源：[4][5][16][17][18]　练习状态：unseen

▼ `L2` `P1` **VLA-RL-020 · behavior constraint、advantage-weighted regression 和 conservatism 如何取舍？**

30 秒回答：三者控制 support 的位置不同：behavior constraint 直接限制策略偏离 behavior，最稳但上限接近数据行为；AWR 只提高数据内高-advantage 动作的回归权重，依赖 $Q - V$ 可信；conservative Q 主动压候选/OOD 动作的 Q，能防 actor 钻 critic 漏洞但可能过度悲观。我按 coverage、reward/value 可信度和交互安全预算选择。本地 WAM/UWM/Harness 的采样或监督权重不具备这些目标，不能改名成 offline RL。

约 2 分钟机制展开：三者不是“保守程度从小到大”的同一旋钮，而是分别约束 policy、dataset-action regression 与 critic。behavior constraint 的典型抽象是：[4][10]

$$\max_{\pi} \mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi(\cdot|s)}[Q(s,a)]$$
$$\text{s.t. } \mathbb{E}_{s\sim\mathcal{D}} D_{\mathrm{KL}}(\pi(\cdot \mid s) \,\|\, \beta(\cdot \mid s)) \le \epsilon.$$

其中 β 是拟合或记录到的 behavior policy，ϵ 是允许偏离预算。它直接限制策略更新，适合数据窄、示范质量高或真实交互风险高的场景；但若 behavior 本身差且数据里没有更好动作，约束越紧，性能上限越接近 behavior。KL 方向、behavior model 误差和连续动作密度估计都会影响“留在 support 内”的真实性，不能把一个系数当安全证明。

advantage-weighted regression 不要求 actor 对任意候选动作做 max-Q，而在数据动作上做加权似然。AWR/AWAC/IQL policy extraction 家族的常见形式是：[5][10]

$$\mathcal{L}_{\mathrm{AWR}}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\mathrm{clip}\left(\exp\left(\frac{A(s,a)}{\lambda}\right), 0, w_{\max}\right) \log \pi_\theta(a \mid s)\right].$$

这里 A 可由 return-minus-value 或 $Q - V$ 估计，λ 是温度，$w_{\max}$ 抑制极端权重。它适合数据覆盖尚可、质量有高低且 advantage 排序可信的情况；reward/Q/V 偏差、温度过低或少量大权重会让梯度被少数样本支配。除权重直方图与 clipping rate 外，我还会看有效样本量：[10]

$$N_{\mathrm{eff}} = \frac{\left(\sum_{i=1}^{N} w_i\right)^2}{\sum_{i=1}^{N} w_i^2}.$$

若 N_{eff}/N 很低，表面上用了整批数据，实际更新却近似由少量样本决定。AWR 只对数据动作监督，并不意味着参数化 policy 在连续空间输出的每个动作都自动处于 support 内。

conservative Q 则改 critic：用 CQL 类正则压低候选动作相对数据动作的 Q，适合 actor 明显利用 critic 过估、数据含噪且能定义合理 candidate-action proposal 的场景。代价是可能压掉罕见但真实优质的动作；因此要同时看 data-action Q、candidate-action Q、conservative gap、proposal 覆盖和同协议 realized return，而不是只看 TD loss。[4]

我的选择顺序是：窄而高质量、强安全约束的数据先保留 BC/behavior-constrained baseline；覆盖尚可、质量混合且 return/value 可校准时再比较 AWR/IQL/AWAC；出现“低 support + 高 predicted Q + 低 realized return”并排除 parser/normalizer 问题后，才引入 conservative Q。若 reward 或 critic 本身不可信，我不会硬上 AWR/CQL，而会先补数据、BC、checker 和评估。对照实验固定 dataset、seed、base checkpoint、更新预算与 rollout 协议，只改变 support-control mechanism。

本地实现中的三类“权重”语义不同。Cosmos/WAM 的 `demonstration_sampling_prob`、`success_rollout_sampling_prob` 与 `p_world_model` 在 archived dataset 构造阶段控制 mixture/routing，`value_head` 只回归 `value_function_return`；UWM RoboCasa 用固定 `action_dim_weights`、contact-window `loss_weight` 和 composite coefficient 加权 diffusion action-noise MSE/辅助监督；Harness 从 reference 与 empty/wrong-tool/reversed 合成组算 `reference_reward_group_mean`，裁成 `[0.2,2.0]` sample weight，最终只优化 reference token CE。三者都没有 behavior-policy KL、action-conditioned $Q - V$ policy extraction 或 candidate-action conservative penalty。[16][17][18]

项目挂钩：我会口述为：“我先沿数据流看权重乘在哪里：WAM 是 archived data mixture，UWM 是 contact/dimension/noise-MSE weighting，Harness 是 synthetic-group-derived reference CE weighting。它们都不是 behavior constraint、AWR 或 CQL。若要补 offline-RL baseline，我会在固定协议下分别记录 KL/support、advantage 权重 ESS，或 conservative gap，再与 BC 和真实 rollout return 对账。”

追问：数据很窄但 behavior 很差时，为什么三者都难取得可信提升？KL constraint 为什么可能受 behavior-model error 影响？AWR 的 N_{eff} 很低说明什么？CQL 过度保守如何从 data/candidate Q 与 realized return 中识别？Harness 的 `advantage` 为什么不是 $Q - V$ advantage？

易错：把三者当成一个“保守系数”；把所有 sample/loss weight 叫 AWR；认为 dataset-action regression 自动保证生成动作 in-support；把 conservative Q 等同于更安全或成功率更高；reward/critic 未校准就按 advantage 加权；没有 KL、$Q - V$ 或 candidate-action penalty 的代码证据却声称项目实现了 offline RL。

来源：[4][5][10][16][17][18]　**练习状态**：unseen

▼ `L2` `P1` **VLA-RL-021 · 机器人 offline RL 的 reward、终止和 timeout 如何定义?**

30 秒回答：我先把任务成功、训练 reward 和 episode end 拆开：success 由可复现 predicate 决定；reward 可以含稀疏成功、进度、安全惩罚和 step cost，但 shaped return 不能替代成功率。`terminated` 表示 MDP 内终态，target 不 bootstrap；外部时限导致的 `truncated` 只结束采样，仍应 bootstrap。固定有限时域若本来就是 MDP 且剩余时间进 observation，则到 horizon 反而是 termination。本地 LIBERO/RoboCasa 脚本压缩或丢弃了这些标志，只能讲评测口径，不能声称已实现正确 RL target。

约 2 分钟机制展开：第一步先定义 task success predicate $g(s)$，例如“物体进入目标区域且约束持续若干步”，并把它与训练 reward 分开。一个机器人训练信号可以写成：

$$r_t = w_s \mathbf{1}[g(s_{t+1})] + w_p \Delta p_t - w_c \mathbf{1}[\text{collision}_t] - w_{\text{step}}.$$

其中 Δp_t 是进度 shaping，碰撞项和 step cost 约束安全与效率。权重会改变优化问题，所以模型选择仍要锁定原始 success predicate、horizon 和安全事件；不能用 shaped return 上升代替 closed-loop success。对离线数据还要记录 reward 版本、checker 版本与生成方式，否则同一轨迹在不同 reward recipe 下不可直接比较。

第二步区分“为什么 episode 停止”。Gymnasium 的基准语义是：`terminated=True` 表示到达 MDP 定义的终态，如成功、失败、跌倒；`truncated=True` 表示 MDP 外的采样条件中止，如外部 `TimeLimit`。两者都会结束当前 episode，但 learner mask 不同：[12]

$$\begin{aligned} e_t &= \text{terminated}_t \vee \text{truncated}_t, \\ m_t^{\text{boot}} &= 1 - \text{terminated}_t, \\ y_t &= r_t + \gamma m_t^{\text{boot}} V(s_{t+1}). \end{aligned}$$

这里 e_t 控制 collector reset，m_t^{boot} 控制 TD target。若只是外部超时且 next observation 有效，episode 虽结束，仍保留 $V(s_{t+1})$；若是 MDP 内失败终态则不 bootstrap。一个容易漏掉的例外是 finite-horizon MDP：如果“第 H 步必结束”本就是任务定义，且 remaining time 进入 observation 以保持 Markov 性，那么到 H 是 termination，不是外部 truncation。[12]

安全停机也要按原因分类：任务定义内的碰撞/跌倒可作为 terminal failure；人工急停、渲染器故障或通信中断通常是外部截断，若 next state 不可信还应标为 invalid 并从 target 构造中排除。最小 transition schema 应至少保存 `reward/success/terminated/truncated/termination_reason/env_step/policy_step/horizon`；action chunk 场景必须区分一次 policy inference 与实际执行的 env steps。

本地实现揭示了三种语义压缩。UWM LIBERO eval 用 `max_episode_length=rollout_length(+dummy_steps)`，消费四元组 `obs,reward,done,info`，循环到单一 `done`，最终只从 `info["success"]` 取成功并写 `success/policy_steps`；脚本本身无法恢复 done 是成功、失败还是 timeout。UWM RoboCasa 的 `step_env` 对五元组直接丢弃 reward、terminated、truncated，对四元组也丢弃 reward/done；外层只在 `check_success` 或 `steps>=horizon` 时停止，记录中没有 stop reason。LeRobot 数据适配又把 `reward>0` 或通用 `done` 合并成 success/contact proxy，因此 timeout/failure 也可能被启发式标成 success-near window。对照之下，OpenPi SimplerEnv eval 至少接收 `terminated` 与 `truncated`，但 summary 仍折叠成 `done=terminated or truncated`，且它只是评测，不生成 RL target。[17][18][19]

项目挂钩：我会口述为：“我在 LIBERO/RoboCasa 协议对齐时，不只对 horizon 数字，还追到 wrapper 返回值。LIBERO 只剩单一 done，RoboCasa rollout 直接丢了 termination flags，LeRobot 还把 done 当 success proxy；所以当前结果能支持锁定协议的 success-rate eval，但不能支持 critic bootstrap 或 timeout-as-failure 的结论。若补 offline RL，我会先改 schema，分别保存 terminated/truncated/stop reason，再用 `1-terminated` 构造 target mask。”

追问：固定 horizon 什么时候是 termination，什么时候是 truncation？任务超时但接近成功时 success、reward 与 target 分别如何处理？人工急停和 simulator crash 能否一律当 truncation？action chunk 下 `policy_steps` 为什么不能替代 `env_steps`？LIBERO 的一个 `done` 为什么不足以决定 bootstrap？LeRobot 把 done 当 success proxy 会造成什么偏差？

易错：把 success、reward 和 done 当同一字段；所有 timeout 都不 bootstrap；所有到 horizon 都 bootstrap；用 shaped return 代替成功率；把安全停机、基础设施错误和 MDP failure 混成一类；只有 episode-level summary、没有 next state 仍强行 bootstrap；看到本地脚本能结束 rollout 就声称训练端 termination mask 正确。

来源：[12][17][18][19]　**练习状态**：unseen

▼ L3 P1 VLA-RL-022 · 如何判断 offline RL 失败来自 critic、数据覆盖还是 action parser?

30 秒回答：我做三层单变量干预。先锁初态、controller、normalizer 和 horizon，用 dataset action 检查 round-trip 与匹配初态 replay；失败先查 parser。接口通过后，看 policy action 的 quantile/mean-z/kNN/clip，并与 BC/nearest-data action 对照，偏离大才支持 coverage。最后在同一 state 比 dataset/policy action 的 Q、ensemble 分歧和 realized return：in-support Q 也失真才偏 critic；低 support+高 Q+低 return 是 coverage×critic exploitation。

约 2 分钟机制展开：核心是做 counterfactual intervention，而不是把相关性当归因。每一轮固定 observation、reset state、checkpoint、normalizer、controller、seed 与 horizon，只替换一段数据流。

第一层查 action contract。模型输出真正进入环境前通常经历：

$$a_t^{\text{env}} = \text{clip}\big(T_{\text{adapter}}\big(T_{\text{denorm}}(\hat{a}_t;\eta_{\text{norm}})\big),\ell,u\big), \qquad e_{\text{rt}} = \|T_{\text{norm}}(T_{\text{denorm}}(z)) - z\|_{\infty}.$$

这里 η_{norm} 是 normalizer 版本/统计，T_{adapter} 决定维序、控制语义与 7D→12D 映射，$[\ell,u]$ 是环境边界。先检查 shape、axis/order、单位、delta/absolute control、round-trip error、preclip→env action 与 clip fraction；再在完全匹配的 initial state/controller 上 replay dataset action。若 bypass normalizer/adapter 后行为改变，或 GT action 在匹配初态仍执行成错误轴/方向，parser 假设最强。初态不匹配的 GT replay 失败只能算线索，不能定罪。

第二层查 coverage，而且要把 state-OOD 与 action-OOD 拆开。动作侧可先用：[4][5]

$$z_j = \frac{\mu_{\pi,j} - \mu_{\mathcal{D},j}}{\sigma_{\mathcal{D},j} + \epsilon}, \qquad d_{k\text{NN}}(s,a) = \min_{(s_i,a_i)\in\mathcal{D}_k} \|\phi(s,a) - \phi(s_i,a_i)\|_2.$$

逐维 quantile violation、$|z_j|$、behavior likelihood、kNN distance 与 clipping 都只是 support proxy。要配合干预：同一状态下将 policy action 投影到 nearest-data action，或换成 matched BC/nearest-neighbor policy；若 parser 已通过，BC/nearest action 能执行，而原 policy 大量低 support 且失败，才更支持 coverage/policy-extraction 问题。若所有动作都在 support 内但 observation embedding 远离数据，应归到 state coverage，而不是 action parser。

第三层才查 critic。对相同状态同时查询 dataset action、policy action 和 nearest-data action，做 ensemble/双 critic 分歧，并按 support bucket 与同协议 Monte-Carlo return 校准：[4][5]

$$\Delta_Q(B) = \mathbb{E}[Q_\phi(s,a) - G_t \mid (s,a) \in B].$$

若数据内 action 的 Q 排序、Bellman target 和 realized return 已明显错，critic 假设更强；若只有低-support policy action 显示高 Q、critic disagreement 大且 realized return 低，则是 coverage 触发的 critic exploitation，不能硬拆成单因。critic validation loss 低只说明 dataset query 上拟合，不能证明 actor query 正确。

归因表应这样读：

单变量干预	观察	当前最强假设	仍需排除
matched-state GT action / parser bypass	动作轴、尺度或控制效果立即恢复	parser / normalizer / controller	reset state、env revision
parser 通过，BC/nearest action 成功，policy action 低 support	原 policy 失败	data coverage / policy extraction	observation OOD、checkpoint mismatch
in-support dataset action 的 Q 也与 return/排序冲突	actor 是否更换都存在	critic target / reward / termination	checker、bootstrap mask、reward scale
低-support policy action 高 Q、低 return、ensemble 分歧大	actor 借 critic 漏洞	coverage × critic exploitation	parser 与 state-OOD

本地证据只覆盖前两层。Cosmos action artifact 汇总 CoffeePressButton 5,868 个 demo 7D 动作与 base/trained 各 300-step trace；base 的第 4/6 维 mean-z 约为 11.76/8.95，trained 对应约为 −5.95/ − 3.00。这是明显的 marginal support warning，但 trace 数、state 条件和 protocol 未构成因果对照。GT replay artifact 的源轨迹是 102 × 12，env action dim 为 12，执行 80 步未成功；manifest 同时明确 HDF5 initial state 可能不匹配 `env.reset`，所以不能据此宣布 parser/controller 错。UWM RoboCasa 实现进一步校验 normalizer metadata/action order，并记录 adapter 前动作、env action 和 clip stats。WAM 只有从 state feature 回归 return 的 value head，UWM/OpenPi 也没有 action-conditioned Q，因此本地没有第三层 critic calibration 证据。[17][18][20]

项目挂钩：我会口述为：“我在 0-success 排查中先把接口、coverage、critic 分层。现有 Cosmos/UWM artifact 已支持检查 dim/order、normalizer、adapter、GT replay、mean-z 和 clip fraction；其中 mean-z 偏移只说明 support warning，初态不匹配的 GT replay 失败也不是 parser 定论。由于本地没有 action-Q 与 realized-return calibration，我只能说完成了 parser/support 审计，不能把失败归因成 critic extrapolation。”

追问：GT replay 失败但 reset state 不匹配时下一步如何补证？为什么 marginal mean-z 很大仍不能证明 action-OOD？如何区分 state-OOD 与 action-OOD？nearest-data-action 投影恢复成功说明了什么、又不能说明什么？没有环境回放时 parser 的最小证据是什么？为什么 WAM state-

value head 不能用于比较 dataset/policy action 的 Q?

易错：看到 0 success 就先怪 critic；把 GT replay 的初态错配忽略；把 mean-z/clip fraction 当 OOD 充分证据；把 state value 当 action Q；不同 state/checkpoint/protocol 下比较 Q 与 return；BC 失败就直接说数据差；只看 validation TD/MSE；没有 action-Q、ensemble 与 return calibration 却声称已证明 extrapolation root cause。

来源：[4][5][17][18][20]　**练习状态**：unseen

▼ L3 P2 VLA-RL-023 · 如何把视觉 representation learning 与 offline RL value learning 隔离?

30 秒回答：我把因果链拆成 encoder→head→loss，固定数据、split、seed、head 和预算，比较 pretrained/random、encoder frozen/trainable，并加 value stop-gradient 分支。共享表征同时测 probe、value calibration、encoder drift、各分支梯度 cosine 和同协议闭环 success；不能只看 value loss。当前 WAM/UWM 都默认把 `model.parameters()` 交给 AdamW，未见 frozen-encoder、stop-gradient 或 probe artifact，所以只能讲 shared multi-task 结构，不能声称已完成视觉因果消融。

约 2 分钟机制展开：先明确变量：视觉 encoder 输出 $z = f_\theta(o)$，policy/action head 为 $\pi_\phi(z)$，value head 为 $V_\psi(z)$。共享 encoder 的联合目标可抽象为：

$$\mathcal{L}(\theta, \phi, \psi) = \mathcal{L}_\pi(\pi_\phi(f_\theta(o))) + c_v \mathcal{L}_V(V_\psi(f_\theta(o))) + c_{\text{aux}} \mathcal{L}_{\text{aux}}.$$

若 value 分支和 policy 分支都回传到 θ，value target 的噪声、reward/termination 错误或 loss coefficient 都可能改变视觉特征。做隔离时可以把 value 路径改为 $V_\psi(\text{sg}(z))$，其中 sg 是 stop-gradient；这只切断 value→encoder 的梯度，不会让 value head 消失，也不能自动证明 policy feature 更好。

实验最小合同是一个 2×2：固定数据版本、train/val split、action/value head、optimizer family、训练步数、batch、seed 与 eval harness，只改变 encoder initialization（random/pretrained）和 update mode（frozen/trainable）。在 pretrained+trainable 分支再加 value-gradient-on/off，才能把“预训练视觉收益”和“critic 是否改坏共享表征”拆开。不要同时换 reward recipe、action normalization、camera augmentation 或 checkpoint，否则差值没有单因素含义。

指标要分三层。表征层做 held-out observation-only probe 或冻结 encoder 后的线性 action/proprio probe，并记录 encoder drift：

$$D_{\text{enc}} = \frac{\left\| \theta_{\text{enc}} - \theta_{\text{enc}}^{(0)} \right\|_2}{\left\| \theta_{\text{enc}}^{(0)} \right\|_2 + \epsilon}.$$

value 层用锁定的 realized return / 正确 termination mask 做 calibration、explained variance、Bellman residual 和按 observation-support 分桶的误差；不能把 value MSE 下降直接当成更好的视觉特征。policy 层至少报告 held-out action loss、逐任务失败类型和同初态/同 horizon 的 closed-loop success。共享层还要记录 policy/value/aux 的 gradient norm 与 cosine，避免 raw loss scalar 代替真正的参数影响。[15]

归因可以按以下分支解释：

对照结果	更合理的解释	还不能推出
pretrained frozen 的 probe 和 policy 都高于 random frozen	初始化表征有用	端到端微调一定更好
pretrained trainable 高于 frozen，且 encoder drift 与 policy success 同步	task adaptation 可能带来收益	收益一定来自 value 分支
value-on 使 value calibration 变好但 policy success/梯度 cosine 变差；value-off 恢复	value→encoder interference 的证据	已证明所有训练不稳定都由 critic 造成
probe 提升、value loss 下降，但闭环 success 不变或下降	representation/proxy 与控制目标脱钩	预训练视觉无效

本地实现更适合讲“为什么要做这个消融”。WAM proxy 的 `encoder→trunk` 共享 `action/future-proprio/value/future-image` 四个 head，loss 是 action、world 与 `value_function_return` 的 masked multi-head MSE，且 `AdamW(model.parameters())` 没有冻结或 stop-gradient 分支。UWM RoboCasa 构造 `UWMObservationEncoder`，`--pretrained-vision` 只影响轻量初始化时是否请求外部视觉权重，optimizer 仍接收全部 model parameters；UWM LIBERO 加载可选 pretrain checkpoint 后也对全部 parameters 建 AdamW。定向扫描和现有日志没有 frozen-encoder、value stop-gradient、linear-probe、encoder-drift 或 per-branch gradient-cosine artifact。因此当前项目能证明 shared multi-task implementation 与消融缺口，不能说“视觉表征提升了 offline-RL value”或“value head 已破坏 backbone”。[9][17][19][21]

项目挂钩：我会口述为：“我不会用 world/value loss 下降包装视觉收益。先在同一数据和 evaluator 下做 pretrained/random × frozen/trainable，再对 pretrained trainable 加 value stop-gradient；同时看 probe、value calibration、encoder drift、梯度 cosine 和闭环成功。如果只做了 shared encoder 的训练而没有这些对照，我会把它写成实现结构和待补实验，而不是因果结论。”

追问：为什么 frozen encoder 的 probe 更高仍可能 success 不涨？value stop-gradient 会不会让 value head 学不到？random frozen 为什么不是公平 baseline？如何固定 camera augmentation 与 action normalization？encoder drift 很大但 success 上升说明什么？只用 linear probe 能否判断 representation 对长时序控制有用?

易错：把 value loss 下降当视觉收益；同时更换 encoder、reward、数据和 head 仍声称单因素消融；把 pretrained checkpoint 加载叫 frozen；把 `--pretrained-vision` 当作 freeze 开关；把 stop-gradient 当作删除 value learning；只报 probe/MSE 不报同协议闭环 success；没有梯度或 drift artifact 却声称 value 干扰已被证明。

来源：[4][5][9][15][17][19][21] 练习状态：unseen

D. SFT、DPO、GRPO 与后训练

▼ `L1` `P0` **VLA-RL-024 · SFT 在 VLA 或 tool calling 中监督什么?**

30 秒回答：SFT 用 teacher-forcing 监督输出，不是把整段上下文都当 label。文本、工具 JSON、离散 action token 可用 causal CE；连续 action 通常要单独的 regression/diffusion head。关键是 prompt/context、padding 与 assistant target 的 loss mask，以及截断后是否还保留完整 target。Harness 将 system/tool schema 放上下文，把 assistant 文本或 `<tool_call>` JSON 作为 target；本地只支持 tool-call SFT，不支持连续机器人 action head、runtime memory 或线上成功学习。

约 2 分钟机制展开：给定条件输入 x 和目标序列 $y_{1:T}$，SFT 的 causal teacher-forcing 目标是：

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{\sum_{t=1}^{T} m_t} \sum_{t=1}^{T} m_t \log p_\theta(y_t \mid x, y_{<t}), \qquad m_t \in \{0,1\}.$$

$m_t = 0$ 的位置可以是 system、user、tool schema、历史 assistant context 或 padding；$m_t = 1$ 的位置才是本轮需要模仿的 assistant target。模型仍能看到被 mask 的上下文，只是这些 token 不贡献监督梯度。Causal LM 内部通常对 logits/labels 做一位 shift，所以要检查 chat template、BOS/EOS、tool-call special token 与 label 对齐，不能只看原始 token 数。

在 tool calling 中，schema 通常是输入条件，目标是规范化的函数名与 JSON arguments；例如 Harness 的 `prep_messages` 把 system rules 和 Available tools JSON 放入 prompt，`render_tool_calls` 将 assistant calls 序列化成 `<tool_call> ... </tool_call>`，`ToolSFTCollator` 再用 full sequence 与 generation prompt 的长度差把 prompt/padding 设为 `-100`。这监督的是“在给定 schema 下生成协议正确的调用字符串”，不是执行工具、验证副作用或学习 runtime memory。

VLA 里要先区分 target 类型：

target	常见监督	需要额外说明
文本/解释	token CE	只学示范文本风格与内容
tool name + JSON arguments	schema-aware token CE	JSON 可解析不等于参数语义正确或工具执行成功
离散 action token	token CE	tokenizer/codebook 与 action mask 必须固定
连续 action / action chunk	MSE、diffusion noise 或 flow objective	不是把向量转字符串就自动变成 tool SFT

还要记录有效 target 比例：

$$\rho_{\mathrm{target}} = \frac{\sum_t m_t}{\sum_t \mathbf{1}[\mathrm{attention}_t = 1]}.$$

若 ρ_{target} 很低，loss 可能主要反映少量格式 token；若 prompt 被误标成 label，模型会学会复述用户/schema，格式指标却可能虚高。`max_seq_length` 截断还可能把 assistant target 切掉；本地 collator 的“保留一个 valid token” fallback 只能防止空 label 报错，不能证明样本仍是有效 SFT，应单独统计 truncation、target token 数、JSON parse rate、argument exact match 和 tool execution success。

参数更新范围也要单独记录。`--freeze-vision-tower` 是视觉参数的 trainability 开关，LoRA 是低秩 adapter 参数化；二者都不改变 target mask，也不代表视觉塔学到了机器人控制。Harness 的一次报告记录了 57,223 条 train/dev/test 数据（51,467/2,902/2,854），project sandbox 新增 1,837、Robo2VLM 高层辅助 5,000；LoRA+gradient-checkpointing+frozen-vision 的 300-step run train loss 为 0.50627，但模型 dev/test tool-call eval 尚未完成。[22]

项目挂钩：我会口述为：“我审计 SFT 时先看样本 schema 和 label mask，再看模型 trainability。Harness 的实际 target 是 assistant tool-call JSON/文本，system rules、tool schema 和 user goal 是上下文；project trajectory 被转成 tool calls，高层 VQA 是辅助文本 target。它能证明 schema-aware SFT 数据流和一次训练 run，不能说连续 VLA action head、工具副作用、memory/subagent 或真实机器人成功已经被训练。”

追问：为什么把 prompt 也设成 label 会让格式指标虚高？tool schema 是输入还是 target？JSON parse success 与 argument exact match、tool execution success 有什么差别？`freeze_vision_tower` 与 LoRA 分别改变什么？截断后只剩一个 label token 为什么仍可能让训练跑通但结论失真？连续 action chunk 为什么不能直接套 token CE?

易错：把 SFT 说成“所有 token 无差别训练”；把可解析 JSON 当作工具执行成功；把 schema/tool description 当作模型学到的行为；把 LoRA 或 freeze 当作数据/算法创新；把 300-step loss 下降写成线上能力；没有 target-token、truncation 和执行级评估却声称 SFT 学会了机器人任务。

来源：[8][9][22] 练习状态：unseen

▼ L2 P0 **VLA-RL-025 · DPO 不显式训练 reward model，如何利用 preference pair？**

30 秒回答：DPO 把同一条件下的 chosen/rejected 变成相对监督：比较 policy 的 completion log-prob 差与 reference 的差，用 negative log-sigmoid 让 chosen 相对 rejected 的 margin 超过 reference。因此它不单独拟合标量 reward model，但仍依赖 latent-reward/偏好噪声假设。实现先锁同 prompt、completion-only mask、pair 去重与交换、截断和长度偏置，再用 held-out pair accuracy 与任务指标验收。现有证据只支持我参与偏好样本整理、实验设计和错误归因，不支持 pair 数、DPO uplift 或独立 ownership。

约 2 分钟机制展开：一条偏好样本写成 (x, y_w, y_l)：x 是相同的 prompt、图像、schema 或状态条件，y_w 是 chosen，y_l 是 rejected。先定义 completion-only 的序列 log-prob 差，再与冻结 reference 的差比较：

$$s_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x), \qquad \mathcal{L}_{\text{DPO}} = -\mathbb{E} \log \sigma(\beta[s_\theta - s_{\text{ref}}]).$$

当 $s_\theta - s_{\text{ref}} > 0$ 时，当前 policy 相对 reference 更偏向 chosen；loss 会继续增大这一相对 margin。DPO 在 Bradley–Terry 偏好模型与 KL-regularized policy objective 的推导下消去了显式 reward-model 拟合步骤，不等于“不存在隐式 reward 假设”，也不等于 chosen/rejected 可以没有共同条件。β 调节相对 logit 的尺度和偏离 reference 的权衡，不应直接叫 learning rate。[6]

实现中，同一 batch 至少包含 `prompt/chosen/rejected`。policy 对两个 completion 分别算 log-prob，reference 在 no-grad 下算对应 log-prob；prompt/padding 不应进入 completion score。chat template、BOS/EOS、tokenizer 与截断必须一致，否则比较的是两套序列合同。标准 sequence log-prob 通常是 completion token log-prob 的和，因此长度、固定格式和拒答模板都可能成为 shortcut，不能只看 train loss。

偏好对进入训练前应做四层审计：

层	必查项	失败后果
pair 语义	同一 x、偏好准则明确、过滤 tie/双错/双对	label 不可辨识或强行制造偏好
shortcut	长度、格式、关键词、拒答模板与生成来源	模型学表面风格而非正确性
split	按题目、轨迹或任务族分组去重	同 prompt 改写跨 train/test 泄漏
eval	held-out pair accuracy/margin + task correctness、安全和格式	preference win 不能证明任务能力提升

tool calling 中可以对同一 schema/user goal 构造正确 tool/arguments 与真实错误调用；如果 rejected 永远只是非法 JSON，模型只会学格式捷径。VLA 中若 action 是离散 token，可在固定 action tokenizer 下比较 completion log-prob；连续 deterministic regression 或 diffusion action head 若没有可比较的 policy log-density，不能直接套标准 token DPO，必须另定义可计算的 preference objective。

本地证据也要做反向核验。优化简历只支持“参与偏好样本整理、SFT/DPO 实验设计及错误归因”。另一个 `muxi/ds_test` 快照只有 `dpo_config.yaml` 的 `beta: 0.1` 和 `run_dpo.sh` 启动骨架；本地没有 `train_dpo.py`、pair 数据、DPO output 或 run log，launcher 也未冻结 DPO-specific trainer/collator。因此这些文件只能证明曾规划 smoke 接口，不能证明 DPO objective 或运行结果已经闭合。[23]

项目挂钩：我会口述为：“在广州皓岚经历里，我的可核验参与范围是训练/偏好样本整理、SFT/DPO 实验设计和错误样例归因。回答偏好对时，我会先讲同题同上下文、chosen/rejected 的判定准则、tie/双错过滤、按题目去重切分和长度/格式 shortcut，再讲 DPO loss；当前没有 raw pair manifest、trainer 与 run bundle，所以不报 pair 规模、提升数字或独立 ownership。其他仓库里的 DPO 配置骨架也不会拿来替代这段实习证据。”

追问：为什么 DPO 仍有 reward 假设？为什么 prompt token 不应进入 completion score？chosen 更长时 sequence log-prob 会有什么偏置？双错 pair 是否应该保留？pair accuracy 上升但数学正确率下降如何定位？tool-call rejected 只含非法 JSON 会学到什么 shortcut？连续 diffusion action 为什么不能直接套 token DPO？

易错：把 DPO 说成 chosen 正类/rejected 负类的独立 CE；认为无需 reward model 就无需偏好假设；不同 prompt 的答案强行配对；把 reference 当 rejected；只看 loss 或 pair win 不看任务正确率；有 `beta` 配置和启动脚本就声称跑通过 DPO；把无 log-density 的连续 action head 直接套标准 DPO。

来源：[6][23] 练习状态：unseen

▼ L2 P0 **VLA-RL-026 · DPO 的 reference policy 和 KL 约束分别做什么？**

30 秒回答：reference policy 是 DPO 前冻结的能力锚点；chosen/rejected 在 policy 与 reference 下的 completion log-prob 差形成 DPO logit。它不是 rejected、reward model 或随训练更新的 old policy。KL 是推导中限制 policy 偏离 reference 的分布约束，标准 DPO 用 reference log-ratio 隐式吸收，不等于调用 `KLDivLoss`。理论上 beta 越大锚定越强，但必须结合 log-ratio/KL proxy 与任务指标调参。本地仅见 beta 配置骨架，无 reference checkpoint、trainer 或 KL 日志，不能声称跑通。

约 2 分钟机制展开：DPO 从 KL-regularized reward maximization 出发。对每个条件 x，抽象目标是：

$$\max_{\pi} \mathbb{E}_{y\sim\pi(\cdot|x)}[r(x,y)] - \beta D_{\mathrm{KL}}(\pi(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)), \qquad \pi_r(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x) e^{r(x,y)/\beta}.$$

π_{ref} 通常是 DPO 开始前冻结的 SFT/base checkpoint，提供“原模型认为哪些输出合理”的相对尺度；β 是理论 KL penalty 系数。由最优策略关系可把 reward 写成 reference-relative log-ratio 加只依赖 x 的配分项，chosen/rejected 做差时配分项抵消：

$$r(x, y_w) - r(x, y_l) = \beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)} \right].$$

这解释了两层职责：reference 进入每个 pair 的相对 logit，KL 则是原始优化问题中防止策略为局部 preference 任意漂移的分布级约束。标准 DPO empirical loss 已通过 reference log-ratio 使用这层关系，通常不再单独对 vocab distribution 调 `KLDivLoss`；若某个变体另加 explicit KL，必须把 estimator、采样分布、reduction 和系数作为额外 objective 说明，不能只看到变量名 `kl` 就断言等价。[6]

真正的 forward KL 是在当前 policy 采样分布下取期望：

$$D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}}) = \mathbb{E}_{y\sim\pi_\theta} [\log \pi_\theta(y \mid x) - \log \pi_{\mathrm{ref}}(y \mid x)].$$

若只在固定 preference dataset 的 chosen/rejected 上平均 `policy_logp - ref_logp`，采样分布不是 π_θ，它只是 dataset log-ratio/drift proxy，不是无偏的 policy KL。面试中我会同时记录 reference identity、pair margin 与能力保持指标，而不是把单个 `kl` 数字当安全证明。

训练合同	最低记录	为什么
reference identity	model/checkpoint SHA、tokenizer、chat template、冻结方式	版本错配会让 log-ratio 失去共同尺度
pair log-prob	policy/ref 的 chosen/rejected completion log-prob、mask、length	区分真实 preference margin 与长度/模板 shortcut
drift	sampled KL 或明确标注 dataset proxy、response length、held-out SFT/perplexity	检查是否牺牲基础能力换 pair win
task eval	数学正确率、tool argument/execute success 或同协议 rollout	pair accuracy 不能替代任务结果
beta sweep	固定数据/checkpoint/seed 的多档 beta 与 early-stop 规则	beta 不是通用常数，也不是 learning rate

理论上更大的 β 对偏离 reference 的惩罚更强；在 DPO logit 中它也放大 reference-relative margin，因此达到同一 preference odds 所需的 policy log-ratio 偏移更小。更小的 β 允许更大漂移，但可能放大噪声 preference；实际优化还受 learning rate、长度、数据可分性和训练步数影响，所以不能只凭 beta 大小预测最终 KL。

实现上可让冻结 reference 模型在线算 log-prob，也可在 reference、tokenizer、template、mask 和 dataset 全部不可变时预计算 ref log-prob；PEFT 场景还可能通过禁用训练 adapter 得到 base reference。无论哪种方式，reference 必须在训练中保持语义不变，不能把 current policy 的旧 batch、rejected 文本或另一个 tokenizer 的模型冒充 reference。

本地 source `[23]` 正好构成负向门禁：Muxi smoke config 虽写 `beta: 0.1`，但没有 reference checkpoint/revision、DPO-specific trainer/collator、pair data、policy/ref log-prob、KL/drift 指标或 run log；优化简历也只支持偏好样本整理、实验设计和错误归因。因此当前不能回答“我的 reference 是哪个 checkpoint”“beta sweep 结果如何”或“KL 控制在多少”，只能讲应有的训练合同与尚缺 artifact。

项目挂钩：我会口述为：“我把 reference、pair 和 eval 分开审计。reference 要冻结 checkpoint、tokenizer、template 和 completion mask；训练看 policy/ref 的 chosen/rejected log-prob 与 reward margin；能力保持看明确采样分布的 KL 或 drift proxy，再回到数学正确率/工具执行等任务指标。广州皓岚材料支持我参与偏好样本整理、实验设计和错误归因，但当前没有 reference manifest、beta sweep 与 run bundle，所以不报 checkpoint、KL 数字或 DPO uplift；其他仓库里单独一行 `beta: 0.1` 也不能补齐这些证据。”

追问：reference policy 与 PPO old policy 有何区别？为什么 rejected 不是 reference？DPO 是否还要显式加 KL loss？固定 preference data 上的平均 log-ratio 为什么不是严格 KL？beta 越大为什么理论上更保守，却不能单独预测最终漂移？reference 能否预计算 log-prob？LoRA 下如何构造 reference？reference checkpoint 和 tokenizer 不匹配会怎样？

易错：把 reference 当 rejected、reward model 或随 step 更新的 old policy；认为 DPO 的 KL 必须调用 `KLDivLoss`；把 dataset log-ratio proxy 报成精确 policy KL；把 beta 当 learning rate；reference 也参与 optimizer 更新；只给 model path 不冻结 tokenizer/template/mask；看到 `beta` 配置就虚构 reference checkpoint、KL 曲线或已完成 run。

来源：[6][23] **练习状态**：unseen

▼ `L2` `P0` **VLA-RL-027 · GRPO 为什么使用 group-relative reward？与 PPO 有什么关系？**

30 秒回答：PPO 通常用 critic/GAE 估计每个时间步的 advantage；GRPO 改为对同一 condition 的一组 rollout 做 reward 相对化，从而减少独立 critic，但要多采样、要求组内可比且依赖 checker。两者都可用 old-policy ratio 和 clipped surrogate，GRPO 还常带 reference/KL。若组内 reward 全相同，canonical group signal 为零；当前 Harness 却令 `weight=1` 并继续 reference SFT CE，这正说明本地只是 group-relative offline weighting，不是把 PPO critic 删除后的 online GRPO。

机制展开（约 2 分钟）：PPO 的 advantage 常由 critic 和 GAE 给出。令 d_t 表示真正终止 mask，则

$$\delta_t = r_t + \gamma(1-d_t)V_\phi(s_{t+1}) - V_\phi(s_t), \qquad \hat{A}_t^{\text{PPO}} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

critic 提供跨状态的 baseline 与 temporal credit assignment，但也引入 value approximation、bootstrap 和额外训练成本。GRPO 不用独立 critic 预测这个 baseline，而是在同一条件 x 的 G 条 rollout 内构造

$$\hat{A}_i^{\text{GRPO}} = \frac{R_i - \bar{R}}{s_R + \epsilon}, \qquad \rho_{i,t} = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid x, y_{i,<t})}, \qquad \mathcal{J}_{clip} = \mathbb{E}\left[\min(\rho\hat{A}, \text{clip}(\rho, 1-\varepsilon, 1+\varepsilon)\hat{A})\right].$$

这里 $\bar{R}, s_R$ 是同组 reward 均值与标准差。GRPO 与 PPO 的亲缘性在后半段：都限制 current policy 相对 rollout/old policy 的更新；差异在 advantage 来源。PPO 的 $\hat{A}_t$ 通常随时间步变化，GRPO 的 sequence reward 常广播到回答 token，具体 token credit、KL estimator 与 reduction 仍依实现而定。critic-free 省掉 value model，却把成本转到每个 condition 的 G 次采样与 checker；若组内难度不一致，相对排名会混入任务难度。若 $R_1 = \cdots = R_G$，分子为零，canonical policy-gradient 项应为零或该组被跳过，最多只剩实现中的 KL/其他正则。[3][7]

本地 Harness 的 `build_sft_row` 只计算 reference 的 $a_b = r_b^{ref} - \bar{r}_b$，再令 $w_b = \text{clip}(1 + a_b, 0.2, 2.0)$；`RewardWeightedTrainer` 仍最小化 reference token CE。于是合成候选同分时 $a_b = 0, w_b = 1$，更新不会消失，而会退化成普通 SFT。negative candidates 只影响 reference 权重，其自身 log-prob 从未进入 objective；`num_generations` 也只改变合成组均值，不证明 policy exploration。[16]

项目挂钩：当前项目能诚实口述的是“我对 schema/tool constraint 定义了 component reward，并运行/审计了 group-relative sample weighting continuation”。不能说“实现了 PPO 去 critic”或“完成 online GRPO”：代码没有 current-policy rollout、old log-prob、ratio/clip，也只训练 reference trajectory；报告还明确声明不是 full online GRPOTrainer。若要升级，需保存同 condition 的 policy-generated group、checker revision/reasons、old-policy log-prob、训练 checkpoint 与独立 same-scorer eval bundle。[9][16]

追问：PPO 的 critic baseline 与 GRPO 的 group baseline 分别解决什么？为什么 group 必须同 condition？reward std 为零时 canonical 与本地 Harness 各发生什么？GRPO 没有 critic后如何处理 token credit？为什么 `num_generations=4` 不等于四条 policy rollout？

易错：GRPO 不是“PPO 删除 critic”这一处代码改动；还必须有 policy-generated comparable group、可靠 checker、relative advantage、old-policy ratio/clip 与独立评估。组内相对第一也只表示比同组更好，不等于 reward 为正、任务成功或安全。

来源：[3][7][9][16]　**练习状态**：unseen

▼ `L2` `P0` **VLA-RL-028 · online GRPO 的 rollout、group、checker 和 policy update 如何连接?**

30 秒回答：online GRPO 固定 rollout policy，对同一 condition 采一组轨迹；同版 checker 打分，组内标准化成 advantage，再用 old-policy ratio/clip 和 reference/KL 更新，最后做独立 holdout。组内难度要可比，reward 方差为零时应跳过或稳健处理。本地 Harness 只给 reference 与合成负例离线打分，再按相对 reward 加权 reference token CE；没有 policy rollout、old log-prob 或 clipped objective，不能称 canonical online GRPO。

机制展开（约 2 分钟）：在第 k 轮先固定 $\pi_{\theta_{old}}$。对同一条件 x 采样 G 条完整输出/轨迹 y_i，并用同一个 checker revision 计算 $r_i = R(x, y_i)$。常见的 group-relative advantage 和简化 clipped objective 为

$$y_i \sim \pi_{\theta_{old}}(\cdot \mid x), \qquad \hat{A}_i = \frac{r_i - \bar{r}}{\text{std}(r_{1:G}) + \epsilon}, \qquad \rho_{i,t} = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid x, y_{i,<t})},$$

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_i \frac{1}{|y_i|}\sum_t \min\Big(\rho_{i,t}\hat{A}_i, \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{ref})\right].$$

第一式把绝对 reward 变成同组相对信号；第二式只是常见抽象，具体 GRPO 对 token/sequence reduction、KL estimator、clip 和 zero-variance group 的处理必须绑定实现版本。完整数据流是 rollout → raw trace → checker component/reason → group advantage → policy update → 新 checkpoint，再由新 policy 进入下一轮。checker 既要版本化，也要保留 component score、拒绝原因和原始输出；否则 reward 上升时无法区分 policy 变好、checker 改版或 reward hacking。独立 eval 不参与训练，并固定 scorer、split、seed、sample count 与 checkpoint。

本地 Harness 的实际更新则是

$$w_b = \mathrm{clip}\left(1 + r_b^{ref} - \frac{1}{G}\sum_{i=1}^{G} r_{b,i}, 0.2, 2.0\right), \qquad \mathcal{L}_{local} = \frac{\sum_b w_b \bar{\ell}_{CE,b}}{\sum_b w_b}.$$

这里 candidate group 来自 reference、empty、wrong-tool、reversed 与重复 empty，而不是当前 policy 采样；只有 reference trajectory 被转成 SFT row，w_b 只改变其 teacher-forcing CE 权重。它没有学习每个候选的 policy ratio，也没有 canonical online group update，所以算法名称必须降级。[7][16]

项目挂钩：`train_grpo.py` 明确打印 `offline_grouped_reward_weighted_policy_update`，`candidate_group` 构造合成负例，`RewardWeightedTrainer` 对 reference token CE 做 sample weighting；`reward_spec.json` 提供 task-family/component 权重。`eval_grpo_tasks.py` 只统计 reward-family、user goal 和 reference trajectory 字段 coverage，不加载模型、不生成 rollout、也不判断真实 robot success。`grpo_report.md` 记录一个 200-step completed continuation，同时直写“不是 full online GRPOTrainer run”。因此面试中可说我审计/运行了 schema-aware reward-shaped continuation 与失败链，但 canonical online rollout、同 scorer base/SFT/GRPO causal bundle、物理 success 和逐文件 ownership 仍未闭合。[9][16]

追问：为什么同组样本必须共享 condition/难度？group reward 方差为零会怎样？old policy 与 reference policy 有什么区别？checker 被钻空子时如何用 raw trace、component score 和独立 eval 发现？为什么本地 `num_generations=4` 仍不等于四条 online rollout？

易错：有 candidate group、relative reward 或 `GRPO` 文件名，不等于存在 online GRPO；要逐项核对候选是否由 policy 采样、是否保存 old log-prob、是否有 ratio/clip/KL policy objective、checker 是否独立于 eval，以及新 policy 是否进入下一轮 rollout。

来源：[7][8][9][16] **练习状态**：unseen

▼ L2 P0 VLA-RL-029 · schema、reward、holdout eval 为什么不能混为一谈？

30 秒回答：schema 定义可解析性、字段类型、允许工具和 scope；reward/checker 对 tool/order/args、澄清、禁止副作用等训练信号打分；holdout 在冻结 checkpoint 后以固定 split、n、scorer、OOD/安全/任务指标独立验收。三者分别是输出合同、优化 proxy 和验收协议：合法 JSON 不等于语义正确，reward 高不等于机器人成功。Harness 本地只对离线 reference 与合成 negative 做 reward-weighted CE；coverage eval 或 project sandbox 100% 都不能叫 online RL/物理成功。

约 2 分钟机制展开：先把三层放在不同时间点：

层	典型职责	本地实现映射	不能直接推出
schema / output contract	JSON 可解析、tool name、字段类型、enum、参数结构、允许 scope 的表达	OpenAI-compatible `messages + tools`、`allowed_tools`、tool schema；runtime 还会做 scope/mutex/cancel hard boundary	字段合法就完成了任务
reward / checker	对候选输出按任务族计算组件分数、hard penalty、拒绝原因，供训练权重或离线比较	`score_candidate` 读取 reference trajectory、allowed/forbidden tools、scope 与 task family；`reward_spec.json` 给权重和 penalty	分数是物理 success 或可部署安全证明
holdout eval	与训练数据隔离，冻结 checkpoint、scorer、n、seed 和 protocol，验收泛化/安全/任务指标	`eval_grpo_tasks.py` 只做数据字段与 reward-family coverage；模型能力由独立 suite/metrics 报告	coverage 通过等于模型能力通过

本地 reward 可以抽象成：

$$R(y \mid x) = \sum_i w_i c_i(y, x) + \sum_j p_j \mathbf{1}[h_j(y, x) \text{ 被触发}], \qquad p_j < 0.$$

其中 c_i 是 tool name/order、required coverage、argument semantic、clarification、state success、cancellation compensation、scope 等组件，w_i 按 task family 变化；h_j 是 forbidden tool、缺失澄清仍执行、scope 忽略或取消不补偿等 hard failure。它是可解释的 checker/reward contract，不是新的物理环境 reward 定理。

关键实现边界在 `train_grpo.py`：`candidate_group` 不是让当前 policy 采样，而是构造 reference、空调用、错误工具和 reversed trajectory；`score_candidate` 为这些合成候选计算组件；`build_sft_row` 用 `reference_reward - group_mean` 得到 advantage，再把 sample weight 裁到 `[0.2, 2.0]`；`RewardWeightedTrainer` 最终仍优化 weighted token CE。也就是说 reward 影响的是离线样本权重，代码没有 environment rollout、old log-prob、policy ratio 或真实副作用执行。

holdout 应该在模型选择之后才使用，并把协议写死。可抽象为：

$$M_H = \frac{1}{|H|}\sum_{(x,y)\in H} \mathbf{1}[\mathrm{predicate}_{\mathrm{frozen}}(x, y) = 1], \qquad H \cap D_{\mathrm{train}} = \varnothing.$$

这里 H 不只是“另一个文件夹”，还要记录任务族、去重键、scorer/revision、checkpoint、n、seed、controller 或 effect-sink。`eval_grpo_tasks.py` 的 `reward_family_covered_rate`、`has_user_goal_rate` 和 `has_reference_trajectory_rate` 只证明评测输入合同完整；如

果 `eval_sft_tool_call.py` 没有 `--predictions`，它会用 gold 做 `GOLD_DRY_RUN` 格式检查，更不能当作模型 holdout 结果。

项目证据适合这样分层：Robot Harness inventory 记录 8B full pipeline 的 SFT holdout $n = 1333$、project sandbox $n = 130$、safety $n = 289$、browser deferred $n = 930$ 和 cross-source $n = 1820$；project sandbox 的 100% 只覆盖 tool/schema/scope/argument proxy，safety 多步工具链仍弱，browser 没有真实 runner。32B ladder 的 95.6% 使用 $n = 1000$ 与 token-overlap fallback，而早期 base 使用 $n = 1333$ 和另一 semantic backend，不能拼成严格 base→GRPO 因果 uplift。[9][16][24]

因此约束放置要看失败语义：语法、类型和 enum 优先放 schema；需要上下文、任务状态、scope、取消补偿或副作用判断的规则放 checker/runtime；holdout 不负责“教模型”，只负责独立验收。某些 scope 字段可同时出现在 schema、reward 和 runtime，但三处的职责不同，不能因为字段名相同就合并指标。

项目挂钩：我会口述为：“Robot Harness 的核心是把高层工具调用变成分层合同。schema 先保证输出可解析和字段结构，reward/checker 再按 task family 检查工具选择、参数、scope、澄清和禁止副作用，holdout 最后用冻结 scorer 和独立 suite 验收。我的安全贡献边界是约束取舍、数据/reward/eval 设计和结果归因；Codex 主要落地脚本。project sandbox 的高分只是结构化协议 proxy，safety 仍弱、browser 仍是 deferred，所以我不会说真实机器人成功或 online RL 已闭环。”

追问：scope 为什么可能同时出现在 schema、reward 和 runtime？ `eval_grpo_tasks.py` 通过率高为什么不代表模型强？为什么 gold dry-run 不能证明 holdout？reward 和 holdout 是否必须用不同 checker？project sandbox 100% 缺少哪一层执行证据？如何设计 schema 合法但语义错误的 adversarial case？为什么 `num_generations=4` 不等于四条 online rollout？

易错：把 JSON parse success 当 task success；把 reward/checker 分数当环境回报；把 data coverage evaluator 当模型评测；把 project sandbox 100% 当真实机器人成功；把不同 n/scorer/checkpoint 的 95.6% 拼成因果 uplift；把 `train_grpo.py` 的合成 group 说成 policy rollout；把所有 scope/安全规则塞进 schema，忽略 runtime 和 context-dependent checker。

来源：[9][16][24]　**练习状态**：unseen

▼ L2 P0 VLA-RL-030 · reward hacking 在工具调用和机器人任务中分别是什么？

30 秒回答：reward hacking 不是“模型故意作弊”，而是策略找到了 proxy reward 与真实目标的缺口。工具调用里常见合法 JSON 却选错 tool/args、只做高权重局部步骤，或靠永远拒绝/no-call 绕过风险；机器人里常见刷 shaped progress、利用 termination/success predicate，甚至抖动、碰撞换 dense reward，但任务未完成。只有 reward 高、独立 effect-sink/success 低且 trace 显示策略利用 checker 缺口，才能强称 hacking；否则只叫风险信号。

约 2 分钟机制展开：先把优化目标与验收目标分开。训练只能看到 proxy R_{proxy}，真正关心的是工具副作用或机器人闭环成功 J_{true}：

$$R_{\text{proxy}}(\tau) \uparrow \quad \not\Rightarrow \quad J_{\text{true}}(\tau) \uparrow .$$

场景	典型 shortcut	独立验收必须看什么
工具调用	parser escape；schema 合法但 tool/args 错；多步任务只做一个高分步骤；统一拒绝或 no-call；迎合 token-overlap/semantic scorer	原始调用链、真实工具执行结果、required steps、目标对象与副作用
机器人	shaped distance/progress 上升但 success=false；动作饱和、抖动或碰撞刷 dense reward；利用 reset/termination/success bug；高层 tool 合法但 controller 未执行	环境 state transition、锁定的 success predicate、stop reason、碰撞/安全、相同 seed/task 的 episode outcome

判断时要分三档：① 只有低 success，是能力不足或分布外失败；② reward 高但独立 success 低，是 proxy gap / hacking signature；③ paired/adversarial trace 中策略稳定选择 checker 漏洞，换独立 scorer 或 effect-sink 后优势消失，才接近因果证明。诊断时保留 raw trace，筛出 reward-high/success-low 样本，做只改变漏洞维度的 paired case，并在 common-support 上比较训练 scorer、独立 scorer 与人工复核。

修复顺序不是“继续训更久”，而是先修目标：语法/enum 放 schema，context、scope 与副作用放 checker/runtime hard gate，终局 success 由独立 effect-sink 判定；再加入 adversarial holdout、scorer ensemble/人工抽检和 rollback gate。dense shaping 只提供学习信号，不能覆盖终局失败。

项目实现口述：我会这样讲：“Robot Harness 的 schema-aware scorer 会识别 malformed/latent tool-call，区分 execution/query tool，并检查 required/critical/semantic fields、澄清/拒答、robot scope 和禁止副作用。但它仍是离线规则/语义 scorer，不是真实执行器。8B V3 safety suite 的 289 条里，合法 JSON/tool-call 与 scope respected 都是 100%，而 tool name、required-tool coverage、semantic argument、argument exact 只有 44.29%、49.13%、17.65%、2.77%。像空参数、多步任务只做一步、`click[description]` 变成 `click[buy now]`、查询联系人却输出 `remove_contact`，都说明格式 proxy 与任务语义脱节；它们是 hacking 风险证据，不足以单独证明模型主动钻漏洞。我的边界是负责约束取舍、数据/reward/eval 设计与结果归因，Codex 主要实现脚本；我不声称已完成物理 effect-sink 或 native online GRPO。”

证据防线：合法率/scope 高而语义低只能证明 checker 覆盖不完整；没有真实 browser/robot effect-sink、matched checkpoint 与独立 scorer，就不能把回退归因成 reward hacking。反过来，低 success 但 reward 也低通常只是 policy 弱，不是 hacking。

追问：如何构造 reward-high/success-low paired case？为什么统一拒绝也可能是 shortcut？怎样区分 parser bug、checker bug 与 policy bug？dense shaping 与终局 success 冲突时谁优先？scorer ensemble 为什么仍可能共因失败？机器人 success predicate 本身被钻时怎么验收？

易错：把低任务成功直接叫 reward hacking；心理化成模型“有意作弊”；拿合法 JSON、scope 100% 或 project sandbox 高分代替真实 effect-sink；只加 semantic reward 权重却不做 adversarial holdout；修 checker 后沿用旧 reward 分数做前后因果对比。

来源：[14][24][25]　**练习状态**：unseen

▼ `L3` `P1` **VLA-RL-031 · 多目标 reward 如何防止格式正确但语义错误?**

30 秒回答：不能只把 JSON/schema 合法性和 semantic 分数做任意加权和，否则格式高分可能补偿错 tool/args。应先把可解析、allowed tool、scope、critical/required fields 和禁止副作用设为 feasibility gate，再对 tool/order/coverage、非关键参数语义和最终 effect 分层打分；关键目标用 lexicographic、乘法 gate 或 constrained reward，保留 soft component 给 early policy 学习。训练记录每个分量与拒绝原因，并用“格式相同、语义一对一翻转”的 adversarial holdout 和独立 effect-sink 验收。

约 2 分钟机制展开：问题不在“目标多”，而在目标之间能否互相补偿。普通加权和

$$R_{\text{sum}}(y \mid x) = \sum_i w_i r_i(y \mid x) + \sum_k p_k(y \mid x)$$

允许一个候选输出用高 `format/schema` 分抵消低 `tool/argument/effect` 分。更稳的合同是先定义硬约束 $g_j \in \{0,1\}$，再只在可行集合里优化 soft quality：

$$g(y) = \prod_j g_j(y), \qquad R(y) = g(y) R_{\text{soft}}(y) - \sum_j \lambda_j \big(1 - g_j(y)\big).$$

这里 g_j 可对应 parse、allowed tool、required/critical fields、scope、forbidden side effect；R_{soft} 再评价非关键语义相似度、顺序、coverage、回答质量与 verified effect。若某些错误绝不能被补偿，就用 constrained/lexicographic：先满足 safety 与 effect，再比较 semantic quality，而不是继续调大某个权重。

层级	适合 hard gate / constraint	适合 soft score	反例
结构	parse、tool-call envelope、字段类型	可选字段完整度	合法 JSON 但调用不存在的工具
任务	allowed/required tool、关键步骤与 order 前置条件	多步 coverage、冗余度	只完成高权重的一步
参数	required/enum/ID/scope/金额等 critical fields	自然语言描述、非关键语义相似度	`click[description]` 变成 `click[buy now]`
执行与安全	forbidden side effect、runtime permission、终局 success predicate	进度、成本、延迟	tool/args 看似正确但 effect 未发生
restraint	缺信息时禁止 execution tool	澄清质量	统一 no-call 或假澄清刷安全分

硬门槛确实可能让 early policy 大量同分、梯度稀疏，所以不要把所有维度都二值化。做法是保留 component-level dense feedback、分阶段提高阈值、对合法但语义错的候选给可区分的负分，并监控 per-component pass rate、reward variance、zero-reward rate 与 hard-reject reason。课程式放宽只能用于训练，最终 holdout 的 gate 与 success predicate 必须冻结，不能边看 test 边调。

项目实现口述：我会这样讲：“Robot Harness V2 的 `reward_spec.json` 有 21 个组件，按 task family 选择权重；`score_arguments` 对 required/critical/enum 字段做 exact gate，对 semantic fields 算相似度并以 0.72 判定，`state_success` 又合取 order、critical、semantic、forbidden、scope 和 restraint。安全类错误另有 7 个 hard penalty。128 条 tool-restraint paired check 中，good 均值 1.0、显式 bad 与 malformed latent call 都是 -5.3423，平均 gap 6.3423，未惩罚样本为 0，说明该 checker 能抓住这类固定反例。但 `final_reward_from_components` 本质仍是 task-family weighted sum 加 penalties，且这 128 条是脚本构造的 checker test，不是模型 holdout；8B safety 里格式/scope 100% 而 semantic argument 仅 17.65%，说明多目标拆分已经做了，但独立 effect-sink 和不可补偿约束仍没闭环。”

设计取舍与防线：现有实现最能证明的是“分量可观测、部分安全错误有 hard penalty、固定 restraint 反例可区分”；不能证明权重 Pareto 最优、semantic scorer 不可投机、线上执行成功或 reward 改动带来因果 uplift。我的口述边界仍是负责组件选择、约束放置、数据/eval 设计与结果归因，Codex 主要落地代码与报告。

追问：为什么 semantic score 不能直接做 hard gate？加权和何时仍可接受？怎样用 Lagrangian 或 lexicographic 实现 constraint？阈值 0.72 如何校准？如何构造格式完全相同的 paired negative？component 全通过但 effect 失败怎么办？hard gate 导致组内 reward 同分时如何恢复学习信号？

易错：只把 semantic 权重调大；把 schema 合法当 feasibility 全通过；所有分量都二值化导致 early policy 无区分度；把 hard penalty 叫 runtime enforcement；拿 128 条 scripted checker test 当模型 holdout；忽略不同 task family 权重和 scorer revision，直接比较总 reward。

来源：[7][14][24][25][26]　**练习状态**：unseen

▼ L3 P1 VLA-RL-032 · preference 数据质量和 label noise 如何影响 DPO/GRPO?

30 秒回答：DPO 的 chosen/rejected 必须共享同一 prompt 和约束，偏好差异可解释；重复、近同义、长度/格式 shortcut、无明显胜负和翻转错标都会把 margin 学歪。GRPO 不用静态 pair，但 checker 就是在线标注器：候选级噪声会误排组内样本，同组 reward 全相等则 advantage 为零。应按 prompt/group 去重切分，保留原始输出、标注理由、annotator/checker 版本和 component reward，统计 tie、flip、agreement、margin 与分层分布，再用人工复核和独立 holdout 区分 policy 退化与 label drift。

约 2 分钟机制展开：DPO 对一个条件 x 和偏好对 (y_w, y_l) 优化

$$\ell_{\text{DPO}} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right).$$

若把 winner/loser 翻转，括号内方向也反转；若两条实际上等价，梯度只会放大标注者偏好、长度或格式 shortcut。不同 prompt 拼成 pair 更糟，因为模型可能学条件差异而不是响应质量。因此 pair contract 至少要冻结 prompt、tool schema/context、采样模型与温度、chosen/rejected 原文、偏好理由、annotator/revision、置信度和去重键。

GRPO 的数据质量问题从“pair 标签”变成“同条件 group + reward checker”：

$$\hat{A}_i = \frac{\tilde{r}_i - \bar{\tilde{r}}}{\text{std}(\tilde{r}) + \epsilon}, \qquad \tilde{r}_i = r_i + \eta_i.$$

同组共享的常数偏移会被中心化消掉，但候选相关噪声 η_i 会改变排序和 advantage 符号；若 scorer 饱和使 $\text{std}(\tilde{r}) = 0$，整组 $\hat{A}_i = 0$，没有 policy signal。group 内还必须共享 prompt、上下文、采样预算和可比难度；跨任务 reward scale 或 scorer revision 不能直接混排。

噪声/偏差	DPO 表现	GRPO 表现	诊断与修复
label flip / checker misrank	chosen-rejected 梯度反向	advantage 符号错误	gold audit、paired adversarial、人工复核 top-disagreement
tie / 无明显差异	随机放大微小表面差异	组内 reward 同分、零梯度	显式 tie/abstain；扩大有效候选差异，不伪造胜负
长度/格式 shortcut	学会更长或固定模板	format reward 压过 task reward	长度匹配 pair、component 分数、独立 task/effect 指标
duplicate / leakage	样本权重被重复放大，holdout 泄漏	重复 prompt/group 主导更新	语义+轨迹去重，按 group/task/source 切分
source/task skew	偏好只代表少数域	某些 task family 主导 reward	分层采样、per-source/task 指标与最小覆盖门槛
scorer drift	新旧 pair 口径不一致	同一 policy 因 checker 版本变分	冻结 scorer SHA，双 scorer 回放，禁止直接拼总分

实践上先做数据审计，再调 beta、学习率或 group size：统计 pair 有效率、tie/flip/agreement、chosen/rejected 长度与格式差、prompt/group 重复率、reward std=0 比例、组内 rank stability，以及人工标签与 checker 的 confusion matrix。模型 checkpoint、采样参数、数据版本与 checker 版本都锁住后，才能用 replay 判断“模型退化”还是“标注口径漂移”。

项目实现口述：我会这样讲：“Robot Harness V2 的 `OnlineGRPOTrainer` 对每个 prompt 重复采样 `num_generations=8`，逐候选解析 tool call，用 schema-aware scorer 记录 component、hard penalty、scorer backend 和 completion preview，再按组均值/标准差算 advantage；所以 checker 不是外围评测，而是直接决定更新方向。代码上同组同分会得到零 advantage，报告生成器也记录了后期出现 equal-reward/zero-loss group。数据 manifest 有 8,916 条，task family 明显不均衡：no-forbidden 3,735、project robot 2,748、clarification 2,300，而 bound-scope 只有 13；manifest 的 `validation` 为空，没有 agreement、flip、tie 或 rank-stability 指标，所以我不会说数据质量已闭环。旧版 reference/empty/wrong-tool/reversed 合成组只能算启发式 negative，也不能冒充人工 preference。”

实习与 ownership 防线：广州皓岚的现有简历证据只支持“参与训练/偏好样本整理、SFT/DPO 实验设计、错误样例归因与质量校验标准”；没有 pair 数、chosen/rejected schema、annotator agreement、DPO-specific trainer/run、beta sweep 或 uplift artifact。安全说法是讲我会如何审计和归因，不把 Harness 代码说成实习代码，也不把 Codex-primary Harness 实现说成我独立手写。

追问：随机 label flip 对 DPO 最优解有什么影响？为什么同组常数 reward 偏移通常不影响 GRPO？reward noise 经组内标准化后为什么可能被放大？如何定义 pair 有效信息量？tie 应删除、降权还是单独建模？怎样用双 scorer replay 发现 drift？group size 增大能否解决 checker 噪声？

易错：只报 pair 数不报有效率；把不同 prompt 的响应强行配对；默认 chosen 永远正确；把 checker 当客观真值；把 equal reward 当训练稳定；混用不同 scorer 的 reward；用旧版合成 negative 冒充人工偏好；为实习补造 DPO run 指标或个人实现。

来源：[6][7][23][24][25][26][27] **练习状态**：unseen

▼ L3 P2 VLA-RL-033 · 如何把连续动作评价转成 policy optimization 的反馈?

30 秒回答：先冻结 task、初始状态、horizon、终止语义和 success predicate，以 episode 成败为主目标，再用 progress、接触、安全和成本构造 step reward；critic/GAE 将 return 变成 action/chunk advantage。可计算连续密度时用 PPO/actor-critic；Diffusion/Flow likelihood 不便时，用

Q/advantage-weighted BC、chunk preference 或 critic guidance。动作 MSE、world/value loss 只是辅助 proxy，最终以闭环 success 和 safety 验收，dense shaping 不得补偿终局失败。

约 2 分钟机制展开：第一步不是选 PPO 还是 GRPO，而是把环境反馈合同补全。轨迹 $\tau = (s_0, a_0, r_0, \dots, s_T)$ 至少要记录 task/seed、checkpoint、action normalization、controller、reward components、`terminated`、`truncated`、stop reason 与 success。终局成功可写成主奖励，dense progress 与成本只作 shaping：

$$r_t = w_s \mathbf{1}[t = T \wedge \text{success}] + w_p \Delta\Phi_t - w_c c_t - w_{\text{safe}} v_t.$$

这里 $\Delta\Phi_t$ 必须来自可验证状态变化；若希望不改变最优策略，可用 potential-based shaping $F_t = \gamma\Phi(s_{t+1}) - \Phi(s_t)$。安全违规通常作为 constraint/hard termination，而不是允许 progress 高分抵消。

第二步做 temporal credit assignment。用 critic 时，bootstrap mask m_t 只在真正 terminal 时为 0，时间截断仍应按协议决定是否 bootstrap：

$$\delta_t = r_t + \gamma m_t V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda m_t \hat{A}_{t+1}.$$

对 action chunk $a_{t:t+H-1}$，可以共享 episode/chunk advantage，或训练 chunk-level $Q(s_t, a_{t:t+H-1})$；必须记录 replan/exec horizon，不能把同一个回报重复算给重叠 chunk。若同组比较，多条 rollout 要共享 task、初始状态、horizon 和难度，否则 group-relative advantage 混入环境差异。

第三步选择能与策略参数匹配的优化器：

policy / 数据条件	可用反馈入口	主要风险
Gaussian/可计算连续密度	PPO clipped ratio、actor-critic	action squashing/log-prob、on-policy freshness
Diffusion/Flow action chunk，density 可追踪	对采样/denoising trajectory 做 policy gradient 或近似 likelihood update	计算昂贵，ratio 与 credit 定义复杂
只有离线 transition + critic	AWAC/IQL 风格 advantage-weighted BC、Q-guided sampling	OOD action 与 critic extrapolation
只有 episode/chunk 排序	同状态 chunk preference、DPO-style preference tuning	pair 可比性、length/horizon shortcut
只有 demonstration action	BC/diffusion-noise MSE 辅助项	只学“像示范”，不等于闭环成功

无论选哪条，训练与验收都要分别记录 terminal success、dense return、cost/safety、critic calibration、advantage 分布、action support、per-task success 和失败类型；若 success 不涨，先查 reward/termination/parser/controller，再查优化器。

项目实现口述：我会这样讲：“现有三个实现还没有完成连续动作 RL 闭环。Cosmos/WAM proxy 从 batch 读 `value_function_return`，优化 `16×action MSE + future proprio/image MSE + value MSE`；没有 env collector、TD target 或 policy ratio，所以 value head 是监督回归，不是已经训练好的 critic。UWM-RoboCasa 对 normalized action 加 diffusion noise，优化 action-noise/dynamics/x_0/contact/progress 等 loss，contact-rich window 只是 sample weight；rollout 的 `step_env` 明确丢弃 reward、terminated、truncated，只保留 obs/info，再按 success 或 horizon 停止，无法算正确 return。UWM-LIBERO 训练间 rollout 只把 success/video 写日志，独立 eval 也只保存 success 与 policy steps，没有把 transition 回流 dataset/replay/optimizer。因此我能讲的是怎样补 `transition ledger → return/GAE或Q → policy update → frozen rollout eval`，不能说现有 WAM/UWM 已做 online policy optimization。”

最小补强方案：先让 rollout 保存 $(s_t, a_t, r_t, s_{t+1}, terminated, truncated, success)$、chunk/replan index、checkpoint 和 controller；再做 frozen reward replay 与 return/advantage 单测，训练独立 critic 并检查 calibration；最后只选一种 policy update 做小规模 matched-seed ablation。未完成这些补做前，简历仍写 supervised fine-tune/eval/protocol alignment，不写 RL uplift。

追问：稀疏 success 与 dense progress 冲突时谁优先？为什么 timeout 不一定把 bootstrap mask 设 0？action chunk 的 reward 怎样避免重复归因？Diffusion policy 没有方便的 log-prob 怎么做 RL？value-return MSE 为什么不等于 critic 已可用？如何用 matched initial state 降低 rollout 方差？offline Q 如何防止 OOD action 过估计？

易错：把 action MSE 当 reward；把数据集里的 return regression 当 actor-critic；让 shaped progress 抵消 success=false；丢失 terminated/truncated 后仍算 GAE；把 rollout success 日志当数据回流；忽略 normalization/controller/exec horizon；没有 matched task/seed 就做 group-relative 比较；把“可补做方案”说成已完成。

来源：[2][3][5][7][9][10][17][18][19][20][28]　**练习状态**：unseen

E. 训练稳定性、评估与项目转场

▼ `L2` `P0` **VLA-RL-034 · reward scale、advantage scale 和 gradient scale 如何共同影响训练?**

30 秒回答：reward scale 决定 return/critic target 和组内可分性；advantage scale 决定动作对 policy loss 的推动；gradient scale 还经过 loss 权重、reduction、batch、KL、学习率和 clipping。组内 advantage normalization 对正仿射 reward 变换近似不敏感，但不能修复同分、稀疏、误排

序或 nonlinear penalty。调参沿 component reward→group std/tie→raw/normalized advantage→pre/post-clip gradient→update norm→success 定位，不能只看 total loss。

约 2 分钟机制展开：先分清三层变量。若把同一组 reward 做 $r'_i = cr_i + b$，组内标准化为

$$\hat{A}'_i = \frac{c(r_i - \bar{r})}{|c|\,\sigma_r + \epsilon}.$$

当 $c > 0$ 且 $|c|\sigma_r \gg \epsilon$ 时，平移 b 被中心化消掉，正缩放 c 也近似消掉；但 near-tie 时 ϵ 占主导，scale 会改变 advantage 大小。同组全相等仍是零 advantage；负缩放会翻转排序，clip、hard penalty、task-specific weights 和 scorer saturation 也不是仿射变换，normalization 无法修复。

若没有 advantage normalization，reward 缩放会线性进入 return、TD residual 和 policy advantage；critic 的 MSE target 还可能按平方尺度变化。即使做了 normalization，raw reward 仍影响候选排序、tie rate、critic target、reward clipping 和跨组/跨任务校准，因此"归一化后 reward scale 完全无关"是错的。

最终参数梯度还要经过另一条链：

$$g = \nabla_\theta L_\pi + c_V \nabla_\theta L_V + \sum_k \lambda_k \nabla_\theta L_k + \beta \nabla_\theta L_{\text{ref}}, \qquad g_{\text{clip}} = g \min\left(1, \frac{G_{\max}}{\|g\|_2}\right).$$

这里 c_V, λ_k, β 是 value/auxiliary/reference loss 权重；实际梯度还受 token/action horizon 的 denominator、mask 覆盖率、batch/gradient accumulation、共享参数、optimizer moments、mixed precision、学习率和 weight decay 影响。loss 数值相等不代表梯度范数相等，乘数 16 也不保证共享 encoder 的该分支梯度恰好放大 16 倍。

层级	直接观测	常见异常	优先修复
reward	per-component quantile、group mean/std、tie/rank、task 分布	饱和、同分、单分量支配、跨任务尺度漂移	checker/权重/按任务校准，不先改 optimizer
advantage	raw/normalized 分布、正负比、zero rate、time/chunk 分解	全零、极端值、错误符号、长 horizon 漂移	return/termination/normalization/critic
loss	policy/value/aux/KL、reduction denominator、mask coverage	某分支数值大但无梯度，或小 loss 大梯度	核对 reduction 与 coefficient
gradient	分支/共享层 pre-clip norm、cosine、clip fraction	长期被裁、NaN、分支冲突、update 过小	学习率、loss balance、clip、数值稳定性
outcome	held-out return、success、安全、失败类型	loss 正常但 success 不涨	回到反馈/协议链，不继续盲调 scale

项目实现口述：我会这样讲："Harness V2 先把 schema reward 乘 `reward_scale`，再按每组 mean/std 和 `1e-4` 算 advantage；policy loss 还除以 active completion token 数，加上 `beta=0.003` 的 reference loss，Trainer 最后用 `max_grad_norm=1.0`。因此 final run 的 `reward_scale=0.5` 对非同分组大多会被 group normalization 近似抵消，但会影响 near-tie；equal-reward group 仍直接变成 zero-loss。仓库还记录 earlier run 在 step 6 出现 `grad_norm=nan`，成功重启同时改了 prompt/completion 长度、lr、reward scale、max-grad-norm 和 ZeRO 设置，所以不能把稳定性因果归给某一个旋钮。Cosmos/WAM 把 action MSE 乘 16 再加 masked future/value MSE；UWM 又有 action/dynamics/x_0/contact/progress 权重和 per-sample `loss_weight`。这些都是 objective scale，必须测 per-branch/shared-layer gradient 才知道谁真正主导。"

调参顺序与 ownership 防线：先锁数据、checkpoint、seed 和 eval，冻结 reward/checker revision；再逐层记录 reward→advantage→loss→pre/post-clip gradient→update norm→success，单因素改 scale。当前 artifact 支持我解释和归因这些尺度链，不支持声称 `0.5`、`16×` 或某个 clip 值是最优配置，也不支持把多变量 restart 写成单因素 uplift。

追问：为什么正 reward scaling 经组内标准化后只近似不变？同组加常数为什么通常消失？reward scale 如何影响 critic 即使 policy advantage 已归一化？loss 乘 16 为什么不保证 gradient norm 乘 16？长期 clip fraction 高说明什么？如何区分分支梯度冲突与单纯尺度不匹配？Adam 是否完全对全局梯度缩放不敏感?

易错：把 reward normalization、advantage normalization 和 gradient clipping 当同一操作；只看 total loss；把零 loss 当稳定；跨 task family 共用 reward 统计；忽略 reduction denominator/mask；同时改 reward scale、lr、clip 后做单因素归因；把 WAM 的 `16×` 或 Harness 的 `0.5` 宣称成经验最优。

来源：[2][3][7][15][17][18][27]　**练习状态**：unseen

▼ `L3` `P0` **VLA-RL-035 · policy loss 正常但 success 不涨时先查什么?**

答：我不会先调学习率。policy loss 下降只说明当前样本、mask 和 surrogate objective 被优化，不保证 success 合同正确。先锁 task/seed/初态/horizon/termination/success predicate 与 checkpoint lineage；再逐步 trace raw action→parse→denorm→adapter→clip→env；随后查 reward/checker、advantage/KL/梯度和数据覆盖。只有协议与反馈链都对齐，才讨论模型容量或训练步数。

机制展开：要先接受一个基本事实：训练目标与最终任务指标不是同一个随机变量。

$$L_\pi \downarrow \not\Rightarrow\ J_{\text{success}} \uparrow.$$

L_π 是当前 batch、mask、reduction 和 reward/checker 下的 policy surrogate；J_{success} 是冻结评测协议下的闭环成功率。两者之间隔着 checkpoint、推理、动作执行、环境反馈和成功判定，只要任一合同漂移，loss 都可能“正常”，success 仍不涨。[3][7][9]

我会按下表排查，顺序从“是否在测同一个系统”逐层走到“是否真的学不到”，而不是先换模型：

层级	必查合同 / artifact	最小异常信号	先做什么
eval / success	task、seed、初态、horizon、terminated/truncated、success predicate	同一轨迹被不同脚本判成不同结果	checker replay；固定协议重算 success
checkpoint lineage	model/checkpoint SHA、normalizer、prompt/obs wrapper、controller revision	eval 实际加载旧权重或错配 stats	current/frozen checkpoint 在同 seed、同初态对照
action pipeline	raw output、parse、denorm、adapter、preclip、env action、控制频率	shape/维序/尺度错误，clip fraction 异常	单步/单 chunk trace；round-trip 与 gold-action replay
feedback	raw reward components、penalty、group std/tie、success/reward 对照	reward-high/success-low，或同组零方差	独立 success predicate；逐分量 replay
optimization	advantage、mask/denominator、KL、pre/post-clip gradient、update norm	loss 降但有效 action token 无梯度或 update 近零	冻结 batch 做梯度与参数差分
data / perception	task/scene/camera/state/action support、失败类型	held-out 初态或动作落在训练 support 外	matched-state rollout、分桶 success 与 action support

完整链路要能被单 episode trace 还原：

$$o_t \xrightarrow{\pi_{\theta,k}} \hat{a}_t^{\text{raw}} \xrightarrow{\text{parse/denorm/adapter/clip}} a_t^{\text{env}} \xrightarrow{\text{env.step}} (o_{t+1}, r_t, \text{terminated}_t, \text{truncated}_t, \text{info}_t) \xrightarrow{\text{checker}} y_{\text{success}}.$$

其中 k 是明确的 checkpoint/version。若 trace 缺少 r_t、终止原因或 model/normalizer/controller lineage，就不能从 loss 与 episode success 建立可审计的因果链。

最小隔离实验：第一，用同一初态 replay gold action；若 gold 也失败，优先查 reset/controller/env/checker，而不是 policy。第二，用同一 seed 对 current 与 frozen checkpoint，并记录权重 SHA，排除“评了旧模型”。第三，保存一个 policy step 的 raw→env action 全链和 clip 统计；必要时接 mock controller，只测 parser/adapter。第四，把已保存 trajectory 离线重放给训练 checker 与独立 success predicate；若 reward 高而独立 success 低，说明反馈目标或判定存在 proxy gap。只有这些实验都通过，才进入 optimizer/data/model 层。

项目实现口述：我会这样讲：“Harness V2 里我不会把 loss 正常或 equal-reward group 的 zero-loss 当 task success；已有 safety holdout 在 n=289 上 valid JSON/tool-call 和 scope respected 都是 100%，但 semantic argument 只有 17.65%，这正说明格式/范围 proxy 可以很好而真实语义仍失败。Cosmos/WAM 的 action-distribution 与 GT replay 能检查 dim、normalizer、controller 和 reset，但那次 GT replay 的 initial state 可能不匹配，所以失败不能单独证明 parser/controller 错。UWM RoboCasa rollout 的 `step_env` 丢弃 reward、terminated、truncated，只按 success/horizon 停；LIBERO eval 只落 success、policy steps 和 video，没有 transition 回流 learner。因此这些实现支持我讲 feedback/eval chain audit，不支持声称 WAM/UWM 已完成 online policy optimization。”[18][19][20][25][27]

调参和 ownership 防线：先冻结 evaluator/checker、checkpoint、normalizer、controller、task/seed/初态，再做一次单因素隔离；没有相同协议的 before/after，不把 success 变化归给某个 optimizer 旋钮。当前 artifact 支持 implementation-derived 诊断与口述，不自动证明候选人独立实现了相关文件，也不证明某个 root cause 已被因果确认。

追问：如何用 gold-action replay 区分 policy 与 controller/checker bug？reward-high/success-low 说明哪几类错位？为什么同组 zero-loss 不是稳定训练证据？如何证明 eval 用的是当前 checkpoint？GT replay 失败为什么不能单独证明 action parser 错？缺少 terminated/truncated 会怎样污染 return/GAE 与 success 归因？

易错：一看到 success 不涨就调学习率或换模型；只检查 reward 总分不看 component/tie；只保存 post-clip action；把 success、reward、done、terminated、truncated 混为一谈；GT replay 不锁初态；拿格式合法率代替语义/物理成功；把 evaluation rollout 误称在线训练；没有 checkpoint/normalizer/controller lineage 却做因果归因。

来源：[3][7][9][18][19][20][25][27]　**练习状态**：unseen

▼ L2 P1 **VLA-RL-036 · SFT → preference tuning → online RL 的阶段边界如何划分？**

30 秒回答：我不把三阶段当必经流水线，而按反馈合同升级。SFT 用 demonstration 建立可解析、可执行的行为底座；只有同一条件下存在可靠 chosen/rejected、但环境交互昂贵时，才做 DPO 等 preference tuning；只有 policy 能采 fresh rollout，reward/success/termination 可审计，且具备安全预算与 rollback，才进入 online RL。每阶段必须过独立 held-out/OOD/safety gate，信号不够就停在前一层。

约 2 分钟机制展开：三阶段最本质的区别不是脚本名，而是训练数据由谁产生、监督信号是什么、能否对当前 policy 的分布纠偏：

$$L_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y^*)\sim D_{\text{demo}}} \log \pi_\theta(y^* \mid x),$$
$$L_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D_{\text{pref}}} \log \sigma\left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right),$$
$$J_{\text{online}}(\theta) = \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}},\,\mathcal{E}}\left[\sum_t r_t\right].$$

D_{demo} 是示范数据，D_{pref} 是同条件偏好对，π_{ref} 是冻结 reference，τ 是由有版本号的 behavior policy 在环境 $\mathcal{E}$ 中新采的轨迹。SFT 只逼近已有 target；DPO 在数据 support 内调整相对偏好；online RL 才让当前策略访问新输出/状态并由 reward 回传。若所谓 RL 仍只消费固定 reference target，或 rollout 只写 success/video、没有影响后续 optimizer update，就没有形成 online learning。[6][7][8]

阶段	进入条件	必须冻结的合同	退出门	不该升级时
SFT / BC	有可执行示范，目标 schema/action 可监督	data source/split、token/action mask、normalizer、target format	held-out 可解析/可执行；关键任务覆盖；连续动作还要 offline loss 与小规模 rollout	target/动作轴错误、覆盖不足、held-out 泄漏
preference tuning	SFT 已有行为底座；同 prompt/state 可比较；chosen/rejected 质量可复核	pair construction、reference checkpoint、annotator/checker revision、completion mask	held-out pair accuracy/margin；flip/tie/agreement；长度/格式 shortcut；任务指标非退化	没有可靠 pair，错误来自接口/coverage，或偏好只是单一格式规则
online RL	fresh rollout 可采；reward/success/termination 可信；old-policy/log-prob 或算法等价 lineage 完整	policy/checker/env/controller version、seed/初态、transition schema、安全预算	reward diversity、KL/update 稳定；independent success/OOD/safety；防 reward hacking；可 rollback	checker 可被钻、transition 缺字段、交互危险、没有独立 effect-sink

“进入阶段”与“完成训练 run”也不是同一件事。online trainer 跑完只能证明 update loop 存在；要退出该阶段进入 shadow/production，还必须让独立 success/effect-sink、安全和 OOD gate 通过。反过来，preference tuning 不是必选中间层：如果 SFT 后已有低成本、低风险、可审计的自动环境反馈，而人工 pair 不比 reward 更可靠，可以直接进入受限 online RL；如果 reward/runner 不成熟，则宁可停在 SFT 或 preference 层。[6][7]

最小阶段决策树：

1. 输出还不能稳定 parse/执行，或 action support 不够：先补 SFT/BC 数据，不上 DPO/RL。
2. 输出都可执行，但同一 context 下“选哪个更好”需要人类/规则相对判断，且 rollout 贵：构造同 prompt/state pair，先做 preference tuning。
3. 环境能对 policy-sampled trajectory 给可信反馈，且 freshness、termination、安全、rollback 可审计：从冻结 SFT checkpoint 做小预算 online RL。
4. reward 高但 independent success 低、同组大量 tie、safety gate 失败：冻结或回滚到上一阶段，修 checker/数据，不靠继续训练掩盖合同问题。

项目实现口述：我会这样讲：“Harness 的阶段必须按 artifact 分开。SFT 实现是 prompt/tool schema 只做上下文、assistant 文本或 `<tool_call>` JSON 做 masked teacher-forcing target；现有证据支持语言工具调用 SFT，不支持连续机器人 action head 或视觉能力学习。广州皓岚材料只支持我参与偏好样本整理、SFT/DPO 实验设计和错误归因；本地 `beta=0.1` 配置/通用 launcher 没有 pair schema、DPO trainer、run 或 uplift，不能说 DPO 已复现。旧 Harness `train_grpo.py` 是固定 reference 加合成负例后做 reward-weighted CE，也不是 online GRPO。V2 `train_grpo_online.py` 才从 SFT checkpoint 对同 prompt 采 `gen=8` completion，用 schema-aware checker 做组内 advantage、clipped token loss 和 reference loss，报告完成 1007 steps；所以可说在语言 tool-call/checker 合同下完成了 policy-sampled online GRPO。可是 safety holdout 的 semantic argument 仍只有约 17.65%，browser 没有真实 runner，project sandbox 也只是协议 proxy，因此训练 run 已完成，真实机器人/browser release gate 没过。”[22][23][24][25][27]

WAM/UWM 则处在另一条路线：Cosmos/WAM 的 action/world/value 是固定数据上的多头 MSE，UWM 是 diffusion action-noise 与辅助监督；RoboCasa/LIBERO rollout 没把完整 transition/reward 回流 learner。它们可以口述为 supervised fine-tune/IL + online evaluation/protocol alignment，不能迁移 Harness V2 的 online-GRPO 阶段结论。[17][18][19]

阶段与 ownership 防线：可安全写的是“我能按 dataflow 判定每条 run 属于 SFT、preference、offline weighting、online policy optimization 还是 evaluation”。除用户已确认的经历外，Codex-primary 文件仍按设计/验收/复核边界口述；不同项目、不同 checkpoint、不同 scorer/n 的指标不拼成一条个人因果 uplift。

追问：什么情况下跳过 DPO 直接从 SFT 进 online RL？为什么 SFT held-out 很高仍可能不满足 RL entry gate？固定数据 reward-weighted CE 为什么不是 online RL？V2 Harness 为什么可称 policy-sampled online GRPO、又为什么不能称真实机器人 RL？preference pair 与 checker 都有噪声时如何选？训练 run 完成与 release gate 通过有什么区别？

易错：认为阶段越多越先进；把 DPO 当 SFT 和 RL 的必经层；看到 reward 字段或 `env.step` 就称 online RL；把固定 reference CE 改名 GRPO；没有 policy/checker/env lineage；只看 pair accuracy/reward 不看独立 success；把语言工具调用 checker 的 online RL 外推为物理机器人闭环；把配置文件或 Codex-primary 实现写成个人已完成 run。

来源：[6][7][8][9][17][18][19][22][23][24][25][27]　**练习状态**：unseen

▼ L3 P1 VLA-RL-037 · 如何设计 SFT-only、single-teacher、OPD/GRPO ablation 支持因果归因?

30 秒回答：我先定义要归因的因子，不按最终模型名比较。固定 base checkpoint、input IDs、train/eval split、scorer、update/token 预算和 seed；teacher 只改 none/A/B/routed-multi，objective 只改 hard target/full-vocab KL，GRPO 必须从同一 pre-GRPO checkpoint 分叉 on/off。用多 seed 和 paired per-sample bootstrap 报主效应、交互与失败类型。现有 OPD 缺 SFT-only、single-teacher 和 raw predictions，Harness GRPO 又跨模型/任务，不能拼成 uplift。

约 2 分钟机制展开：第一步不是列模型，而是写 estimand——到底想测 teacher 来源、teacher 数量/routing、hard-label 与 full-vocab objective、teacher-rollout 与 student-rollout，还是 GRPO update。一个可识别的分解可以写成：

$$M_{i,j,k,s} = \mu + \alpha_i^{\text{teacher}} + \beta_j^{\text{objective}} + \gamma_k^{\text{GRPO}} + (\alpha\beta)_{ij} + (\beta\gamma)_{jk} + b_s + \epsilon_{i,j,k,s}, \qquad \Delta_{\text{OPD}} = M_{\text{student rollout}} - M_{\text{teacher rollout}}, \qquad \Delta_{\text{GRPO}} = M_{R=1} -$$

M 是同一 evaluator 下的主指标与逐任务向量，$i/j/k$ 分别是 teacher、distillation objective/rollout、GRPO 开关，b_s 是 seed 或配对样本 block。只有计算差值的两个 cell 除目标因子外完全一致，Δ 才能解释为该因素的效应；否则只是 package comparison。

推荐实验矩阵：

Arm	Parent / 数据	唯一变化	能回答什么
A0 · SFT-only	同一 base；同 input IDs、split、token/update 预算	ground-truth / hard target，无 teacher	行为底座与数据本身
A1/A2 · single-teacher	从同一 base 分叉；A、B 都覆盖 common-support rows	只换 teacher identity	teacher 质量差异；不能与不同 domain 数据量混淆
A3 · routed multi-teacher teacher-rollout	同 rows、同 formal KL、同预算	single→metadata-routed multi	teacher mixture/routing 的系统效应
A4 · OPD student-rollout	从 A3 相同 parent 分叉；同 route、KL、prefix/veto、steps	只换 rollout source	on-policy student distribution 的 OPD 效应
A5 · +GRPO / no-GRPO	从完全相同的 pre-GRPO checkpoint 克隆	只开 fresh rollout policy update	GRPO 主效应；若 A0/A4 都做 on/off，才能估 interaction

五个 expert 若各自只覆盖不同 domain，直接比较 single-teacher 与 routed multi 会同时改变 task support。应先在所有 teacher 都能打分的 common-support subset 上估 teacher/routing 效应，再把 full-mixture 结果报告为 system-level coverage，不冒充纯“teacher 数量”效应。teacher logits/outputs、route metadata、input IDs 和 loss mask 最好预先缓存并 hash，避免每个 arm 重新生成不同伪标签。

必须冻结的合同：

1. 同一个 base/parent checkpoint SHA；OPD student-rollout 与 teacher-rollout要从同一 coldstart checkpoint 分叉。
2. 同一 unique input、source/pool/format 配比、synthetic ratio、expanded-sample count、effective target tokens 和 optimizer updates；数据量若是研究因子，要另做 factorial。
3. 同一 objective 实现、prefix/veto schedule、batch/accumulation、LR、checkpoint selection rule；比较 top1 CE 与 full-vocab KL 时只切 objective。
4. 同一 raw-holdout IDs、prompt/parser/scorer hash、denominator 与 checkpoint-selection policy；保存逐样本 prediction，做 paired bootstrap CI、McNemar/sign test 或按任务 seed 汇总。
5. final test 只在预注册 checkpoint 上跑一次；多 seed 至少报告 mean/CI、逐 pool/format、worst-group、失败类型和训练成本，不只报最优 seed。

四个 run 的取舍：若主问题是“OPD 与 GRPO 是否交互”，四个 run 应做 $\{\text{SFT}, \text{OPD}\} \times \{\text{GRPO off}, \text{GRPO on}\}$ 的 2×2，single-teacher 留到下一批。若必须保留 teacher chain，则做 SFT-only、single-teacher、matched teacher-rollout multi-teacher、matched student-rollout OPD；这批只能估 teacher/multi/rollout 的顺序对比，不能再声称 GRPO 效应。把 SFT、single-teacher、OPD、OPD+GRPO 各跑一次虽然能展示产品路线，但只能可靠估最后一段 OPD→OPD+GRPO，不能估完整 interaction。

项目实现口述：我会这样讲：“OPD 当前最重要的不是编一个提升数字，而是先修 lineage。根目录 legacy trainer 是 teacher rollout + teacher top1 hard-label CE；`point/` formal trainer 才支持 teacher/student rollout 和完整词表 $D_{KL}(q_\beta \| p_S)$。coldstart 从 base 做 500-step teacher rollout，OPD5000 从 coldstart 继续 5000-step student rollout，所以两者还同时改了 parent、预算和 rollout source，不能用差值证明 student rollout 更好。五 teacher 又是 metadata route，不是 learned router。现有 10k raw-holdout 已有 image-hash leakage gate 和 checkpoint 表，但缺 SFT-only、single-teacher、no-synthetic、raw predictions 与本地 checkpoint binary，因此只能讲组合路线结果和补实验设计。”[9][29]

Harness V2 GRPO 是另一条语言 tool-call/checker 线：若要测 GRPO，必须从同一 Harness SFT checkpoint 克隆 no-GRPO/+GRPO，并在同一 suite、n、scorer、raw predictions 下配对比较。它与 OPD grounding 的 model、data、objective 和 metric 都不同，不能把 OPD ckpt 与 Harness GRPO ckpt 放进一个表后称“OPD/GRPO ablation”。[22][24][25][27]

因果与 ownership 防线：当前 artifact 支持我指出缺失 cell、设计 matched run 和解释 legacy/formal objective；不支持声称这些 ablation 已经跑完、OPD 单独带来现有全部指标、GRPO 在 grounding 上有效，或相关代码均为个人独立实现。

追问：算力只够四个 run 时怎样选？为什么 single-teacher 必须看 common support？teacher-rollout 与 student-rollout 怎样保证同 parent？为什么 effective target tokens 比 raw row 数更重要？没有 raw predictions 为什么不能做 paired CI？2×2 中如何解释 OPD×GRPO interaction？checkpoint selection 用最优点会引入什么 bias？

易错：不同 checkpoint、数据量、steps 或 scorer 的差值被写成单因素收益；用 specialized teacher 的不同 domain 直接比较；每个 arm 重新生成不同伪标签；只对齐 epoch 不对齐 token/update budget；从 coldstart 与 OPD5000 差值归因 rollout source；把 metadata route 叫 learned router；把跨项目 Harness GRPO 当 OPD 对照；只报 best seed/aggregate，不保留逐样本和 worst-group。

来源：[6][7][9][22][24][25][27][29]　**练习状态**：unseen

▼ L3 P1 VLA-RL-038 · 辅助目标何时帮助 policy，何时造成 gradient interference?

30 秒回答：我只在共享参数上判断，不看 auxiliary loss 是否好看。辅助标签与控制因果相关、无未来泄漏/shortcut、mask/尺度正确，且 $g_\pi^\top g_k$ 长期为正或不显著冲突时，才可能帮助 policy；若负余弦持续、$\lambda_k\|g_k\|$ 压过主梯度，或 aux loss 降而闭环 success 不涨，就是干扰信号。做 policy-only、aux-on、aux-stop-grad/separate-trunk matched ablation，联合看 success、分支 norm/cosine、clip 和 update；单 batch 负 cosine 不等于立刻上 PCGrad。

约 2 分钟机制展开：设 ϕ 是 policy 与辅助 head 的共享参数，主任务梯度为 $g_\pi = \nabla_\phi L_\pi$，第 k 个辅助梯度为 $g_k = \nabla_\phi L_k$。组合更新对主任务的一阶影响是：

$$L = L_\pi + \sum_k \lambda_k L_k, \qquad L_\pi\left(\phi - \eta\left[g_\pi + \sum_k \lambda_k g_k\right]\right) \approx L_\pi(\phi) - \eta\|g_\pi\|_2^2 - \eta\sum_k \lambda_k g_\pi^\top g_k, \qquad c_k = \frac{g_\pi^\top g_k}{\|g_\pi\|_2\|g_k\|_2}, \quad \rho_k = \frac{\lambda_k\|g_k\|_2}{\|g_\pi\|_2}.$$

c_k 是共享层 gradient cosine，描述局部方向一致性；ρ_k 是加权 auxiliary 相对主梯度的 dominance ratio。$c_k > 0$ 时 auxiliary 在该 batch 的一阶方向通常协同；$c_k < 0$ 只表示局部冲突，只有它在任务/seed/训练阶段上持续存在、ρ_k 又足够大，并且 aux-on 让主指标变差，才能升级为 harmful interference。cosine 接近零也不代表无用：辅助任务可能通过长期表征、正则化或数据覆盖产生二阶收益。[15]

辅助目标可能帮助的条件：

维度	有益信号	高风险信号	必查证据
target 语义	预测与动作决策相关的接触、进度、可达性、未来状态	标签是启发式 shortcut，或与真实 success 不一致	label rule、confusion matrix、失败样例
因果/时间	训练 target 在推理时可由当前 observation 推断	使用未来帧、终局 return 或 privileged state 造成 leakage	timestamp、observation boundary、deployment input
mask / coverage	auxiliary 在可靠样本上激活且 denominator 稳定	mask 稀疏/错位，少数样本产生大梯度	mask fraction、effective tokens/samples、分层 norm
表征	与 policy 需要的几何、接触、动态特征一致	强迫 encoder 保留与 action 无关或冲突的不变性	layer-wise cosine、probe、encoder drift
尺度	ρ_k 可控，shared update 不被单分支支配	raw loss 单位不同、长期 clip、主梯度被淹没	pre/post-clip norm、coefficient、update norm
outcome	main held-out、闭环 success/safety 同步提升	auxiliary metric 变好而 action/success 持平或下降	matched ablation、逐任务/失败类型

最小诊断实验：先固定 dataset/split、base checkpoint、seed、updates、batch order、LR、clip 和 evaluator，在同一 batch 对各 loss 单独 backward，分别记录 shared encoder early/late layer 与 trunk 的 $\|g_k\|$、c_k、ρ_k 和 clip fraction；再跑四个 arm：policy-only、policy+aux、policy+aux 但 auxiliary 对 shared trunk stop-gradient、policy+aux separate trunk。若 aux-on 提升而 stop-grad 消失，说明收益来自共享表征；若 separate trunk 恢复主任务，说明共享冲突更可信。最后以闭环主指标验收，不能用 total loss。

PCGrad、GradNorm、uncertainty weighting 或动态 λ_k 是后置干预。单 batch 负 cosine 可能来自噪声、合理的 Pareto trade-off 或训练早期探索；PCGrad 会删除冲突分量，也可能抹掉对泛化有用的约束。只有原始加权、scale-matched baseline 和 stop-grad/separate-trunk 对照都表明持续冲突时，才比较 gradient surgery，并报告主任务收益与辅助退化。[15]

项目实现口述：我会这样讲："WAM 当前是 objective-compatible proxy，不是完整 Cosmos 大模型：共享 encoder/trunk 后接 action、future-proprio、future-image 和 value 四个 head；total loss 是 `16×action MSE` 加按 `world_model_sample_mask` 归一化的 future losses 和按 `value_function_sample_mask` 归一化的 return MSE，AdamW/DeepSpeed 更新全部参数。这个结构确实存在共享干扰的可能，但当前日志只有各 loss 与 mask fraction，没有 per-branch gradient、stop-gradient 或 closed-loop matched ablation，所以不能说 world/value 已帮助或伤害 action。"[17][21]

"UWM RoboCasa 的 weighted branch 又在共享 observation encoder/noise network 上组合 action-noise、dynamics、denormalized x_0、gripper phase、control mode、contact 和 progress loss，并允许 action-dimension/sample weighting；optimizer 同样接全部 model parameters。这里 contact/progress 可能提供动作阶段监督，也可能把 heuristic label/scale 带进 shared gradient，必须按 arm 实测。Harness V2 的 reference 项则是 current/reference token log-prob 的 squared difference 乘 beta，它是限制 policy 漂移的 anchor，不是标准 KL 名字本身；也应测它与 policy gradient 的 cosine、KL proxy 和 held-out，而不能因训练稳定就断言有益。"[18][21][27]

ownership 与结论防线：现有 artifact 支持解释 loss dataflow、shared-parameter 风险和补实验；不支持声称已采集 WAM/UWM/Harness 的分支梯度、已验证 PCGrad、已证明某个 auxiliary 的因果收益，或把这些实现写成候选人逐文件独立完成。

追问：gradient cosine 为负是否一定要用 PCGrad？为什么 raw auxiliary loss 很小仍可能主导梯度？stop-gradient 与 separate trunk 分别隔离什么？未来状态预测什么时候构成 leakage？$c_k \approx 0$ 为什么仍可能长期有益？多个 auxiliary 两两冲突但都与 policy 正相关怎么办？gradient clipping 会怎样掩盖 dominance？

易错：辅助 loss 总和更低就断言表示更好；只在 head 参数上算 cosine；不乘 λ_k 比 gradient norm；忽略 mask denominator 和 sample weighting；单 batch 负 cosine 就上 PCGrad；未来标签泄漏；aux-on 同时改数据/预算；只看 offline MSE 不看闭环 policy；把 WAM value return regression 当 action-Q；把 squared log-prob reference loss 直接称精确 KL。

来源：[9][15][17][18][21][27]　练习状态：unseen

▼ L2 P1 **VLA-RL-039 · 如何设计 held-out、OOD、online rollout 三层评估?**

30 秒回答：三层回答不同问题。held-out 要按原 episode/task/scene 分组隔离但保持 ID 分布，用于 checkpoint 选择和离线 proxy；OOD 在锁定其他条件时逐轴改变对象、场景、指令、视角、动力学或组合，衡量哪种 shift 失效；online rollout 冻结 model/normalizer/controller、初态/seed、horizon、success/termination 与 invalid policy，报告闭环 success、安全和失败类型。OOD-test 与 final rollout 不参与调参、不回流训练；三层分别报告逐任务、置信区间和协议，不能互相替代。

约 2 分钟机制展开：先给每个结果绑定完整 protocol tuple，而不是只写一个 success rate：

$$\mathcal{P} = (\text{model SHA}, \text{data/split SHA}, \text{task}, \text{init}, \text{seed}, \text{obs/action transform}, \text{controller}, \text{horizon}, \text{success}, \text{termination}, \text{invalid policy}, \text{scorer}), \qquad \hat{p}_{\text{macro}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \hat{p}_t$$

$\mathcal{P}$ 是不可缺字段的评测合同，$y_{t,i}$ 是 task t 第 i 个 episode 的 success，macro success 防止样本多的任务吞掉长尾；Δ_j 只在第 j 个 OOD 轴变化、其余协议匹配时才有可解释性。若同时换对象、场景、相机、指令和 controller，差值只能叫 package shift，不能归因某一泛化能力。

层级	核心问题	数据/环境隔离	主要指标	能否用于调参
ID held-out	是否过拟合；离线 proxy 是否泛化到同分布未见样本	先按 source episode/task/scene/object group split，再 window/augmentation；train-only normalizer	action NLL/MSE、schema/grounding、逐任务 proxy、calibration	dev 可以；locked held-out test 不可以
OOD	哪个分布轴失效	预注册 object/scene/instruction/view/dynamics/embodiment/composition 轴；保留 matched ID anchor	每轴 drop、worst-group、failure taxonomy、CI	OOD-dev 可；final OOD-test 不可以
online rollout	闭环 action 是否完成任务且安全	锁 checkpoint、policy version、initial state/seed、controller、action transform、horizon、success/stop reason	task-macro/micro success、return、safety/cost、latency、termination/invalid/error	training rollout/monitor 可回流；final rollout 永不回流

held-out 的关键是防 window leakage：机器人连续轨迹不能先切窗口再随机分 train/test，同一 episode、相邻帧、同一初始化和增强副本必须在同一 split。OOD 则要建立 shift matrix，先做单轴和少量预注册组合；组合 OOD 可以测整体鲁棒性，但不承担根因解释。online rollout 还要区分 policy step 与 env step、terminated/truncated/timeout/infrastructure error，并保留 episode-level trace；只有 aggregate success 没有 stop reason，不能支持 return/bootstrap 或细粒度失败归因。[11][12][18][19]

选择与统计纪律：checkpoint 只能看 train/ID-dev，最多用 OOD-dev 做设计 gate；最终 ID-test、OOD-test 和 online final 在模型/threshold/parser/controller 全冻结后一次性运行。比较两个模型时尽量复用 task/seed/init 的 paired episodes，报告 task-macro、micro denominator、Wilson/bootstrap CI、paired difference 与逐任务失败，不用一次高波动 rollout 排名。若 online episode 曾进入 replay、reward tuning 或 checkpoint selection，就把它改名 training/validation rollout，不能继续称 final test。

项目实现口述：我会这样讲："UWM/LIBERO 目前有三套不能混的协议。项目 fixed-pruned-init 的 `186/250=0.744` 是用户确认的远端 summary，但缺本地 target checkpoint、250 条 episode JSONL 和完整失败视频；reset-style 的 `0.928` 更容易，只能做 protocol sensitivity；UWM Table II 还要求 OOD 初始化偏移、background removal 和多 seed，所以 0.744 不能写成 Table II 复现。现有 LIBERO eval 脚本只保存 success、policy steps 和可选 video，单一 done 无法恢复 terminated/truncated/stop reason，因此能支持 success-rate eval，不能直接支持 RL target。"[11][19][30]

"Cosmos/WAM 还要把 proxy training、自己的 checkpoint eval 与 official-HF baseline 分栏。official-HF 24 task×5 episode 的 raw log 重算为 `86/120`；按本地审计排除两个 length=1 artifact success 后 trusted 为 `84/118`。这只能证明我跑通并审计了固定 inference/eval harness；官方权重在 bundle 中只有远端 path，没有本地 binary/hash，也没有与 1 epoch/full10k checkpoint 做 matched 24×5，所以不能叫个人训练成绩或 uplift。后续比较必须复用相同 task/seed/layout/controller/scorer/invalid policy 与原始 episode denominator。"[9][30]

最小补证据动作：UWM 先把 fixed-init 与 reset-style 在同 checkpoint 上做 paired seed/init run，保存 checkpoint/normalizer/config SHA、250 条 episode JSONL、stop reason 和视频索引；WAM 对 official baseline 与目标 checkpoint做 matched 24×5，保留全部 episode manifest、scorer/invalid-policy version 和 paired difference。两者都先锁 protocol，再讨论模型。

ownership 与结论防线：当前证据支持区分三层评测、解释 protocol sensitivity、重算已有 denominator 和设计 matched eval；不支持把 user-confirmed summary 当本地可重放 bundle、把 official baseline 当个人模型、把跨协议差值写成 causal uplift，或从运行路径推断逐文件 ownership。

追问：OOD 同时改变五个因素为什么解释价值下降？OOD-dev 与 OOD-test 如何分工？final rollout 进过 replay 后为什么必须改名？macro 与 micro success 何时差异很大？如何对两个 policy 做 paired interval？只有 success/video 没有 stop reason 会丢失什么？invalid episode 应从 numerator 还是 denominator 处理？

易错：把 validation、OOD 和 online 当同一个测试集；先 window 再随机 split；final test 反复选 checkpoint；只报 aggregate/micro；同时改变五个 OOD 轴却做单因归因；不同 seed/init/controller/scorer 横比；把 timeout、failure 和 infrastructure error 合并；训练 rollout 回流后仍称独立评测；把 UWM 0.744/0.928/Table II 或 WAM raw/trusted/baseline/trained 混成一个数字。

来源：[9][11][12][18][19][30]　练习状态：unseen

▼ L3 P2 **VLA-RL-040 · 真实机器人交互预算只有 100 episodes，如何分配?**

30 秒回答：对 offline-to-online 适配，我先用模拟器、gold-action replay 和 safety checker 做零真机预检，再预注册 `10+40+20+30`：10 条接口/急停 smoke，不进主结果；40 条分四批采 recovery/intervention，每批间最多一次离线更新；20 条做两次冻结 checkpoint gate；最后 30 条锁 model/normalizer/controller、task/seed/init/horizon 做 final。训练侧轨迹可进 replay，final 永不回流；abort 也计原始预算并记录原因。30 条二项结果置信区间很宽，只支持大效应和失败审计，不包装成精确 benchmark。

约 2 分钟机制展开：这不是通用黄金比例，而是一份面向安全适配的默认预算。先做不消耗真机 episode 的 preflight：action dim/order/单位、normalizer round-trip、controller mock、workspace/速度/力矩界、success/termination checker、gold-action replay、通信超时与 emergency stop 全部通过，才开放真实交互。

$$N = 10_{\text{smoke}} + 40_{\text{collect}} + 20_{\text{gate}} + 30_{\text{final}} = 100, \qquad h_{95} \approx 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{N_{\text{final}}}}, \qquad h_{95}\big|_{\hat{p}=0.5,N=30} \approx 0.179.$$

h_{95} 是普通正态近似的 95% 半宽，仅用于直觉；正式小样本报告应使用 Wilson 或 exact interval。即使 final 30 条观察到 50% success，误差量级仍约 ±18 个百分点；若再分成 5 个 task，每 task 只有 6 条，逐任务不确定性更大。因此这套预算优先发现接口/安全/大幅行为差异，不能证明 3–5pp 的小提升。

阶段	Episodes	用途与 gate	是否进 replay	允许改变什么
zero-episode preflight	0	sim/mock/gold replay、limits、checker、急停演练	既有 demo 可用	修接口，不碰 final protocol
smoke	10	覆盖关键 task/controller/边界；任何严重 safety event 触发 campaign stop	合法轨迹可在标注后作为 training-side	只修 parser/normalizer/controller/safety
recovery collection	40	4×10；按停滞、偏航、接触异常、低置信触发 intervention，保存接管前状态与纠正动作	是	每批间最多一次预注册 update
intermediate gate	20	2×10 frozen checkpoint；看安全、success、失败类型，决定 continue/rollback	默认不在当前选择周期回流	可选 checkpoint/停机，不偷看 final
locked final	30	一个 checkpoint；预注册 task×seed/init；完整 trace 与 paired baseline	永不回流	不改 model、threshold、parser、controller

安全和记账合同：每次 task attempt 从 robot enable/首次 action 开始计一条；human intervention、急停、越界、通信中断、基础设施错误和 timeout都写入 immutable ledger。任务 failure、safety termination、external truncation 与 infrastructure invalid 分栏；是否补跑 invalid 必须预注册，同时报告 raw attempted denominator 与 protocol-valid denominator，不能静默删掉失败。连续 action chunk 要同时记录 policy step、env step、preclip/env action、force/contact、stop reason、operator intervention 和 video locator。[12]

自适应更新纪律：40 条采集可以像安全版 DAgger 一样按 10 条分批，优先收集当前 policy 实际访问的 recovery 状态，而不是随机堆成功轨迹。[1] 每批只允许预注册的一次数据处理/训练更新，并保留 policy version→episode IDs→replay insertion→checkpoint lineage。中期 gate 若后来进入 replay，可以，但它从那一刻起只能叫 training-side data；任何使用它更新的 checkpoint 不能再拿同一 10 条做 validation，仍需靠 untouched final 30 验收。

任务配额按风险和目标预注册：smoke 覆盖所有 controller/action mode 与最高风险任务；collection 对失败高且可安全接管的 task 过采样；gate/final 用固定 task-macro 配额，不能临时避开难任务。若目标只是评估而不是适配，应把大部分 40+20 转给 locked evaluation；若硬件风险更高，应增加 smoke/gate、减少训练采集，而不是机械套比例。

项目实现口述：我会主动说明：“这 100 条是真机实验设计，不是现有项目成绩。当前 Cosmos/WAM official-HF 的 24×5、UWM LIBERO 的 250 条 summary、RoboCasa 的 8×54 raw sweep都来自模拟环境。它们能帮助我设计 task/seed/horizon/scorer/invalid-policy ledger，但不能证明我已经做过 100 次真实机器人交互。尤其 UWM RoboCasa `step_env` 丢弃 reward/terminated/truncated，LIBERO 也只落 success/policy steps/video；真机前必须先补完整 transition、stop reason、safety event 和 policy/checkpoint lineage。”[18][19][30]

“我还不会拿现有模拟 success 当真机先验直接算小幅 uplift。WAM official baseline 缺与训练 checkpoint 的 matched eval，UWM fixed/reset/Table-II 也协议不同；因此 30 条 final 会优先做同 task/seed/init 的 paired baseline，并把结论限制为当前 robot/controller/protocol 下的大效应或失败类型。”[9][30]

最小实施清单：预注册 episode definition、task quota、seed/init list、success/termination/invalid policy、force/workspace limits、operator权限、campaign stop rule、允许更新次数、checkpoint/normalizer/controller SHA、replay insertion rule、final freeze time和统计方法；没有这些字段，`10+40+20+30` 只是数字排版。

ownership 与结论防线：当前证据支持我从模拟项目设计真机预算和安全 ledger，不支持声称预算已执行、真机 success 已获得、现有 sim episode 等于 real-world evidence，或把未来补实验写成个人既成经历。

追问：中期 gate 用掉的 episode 能否进 replay？abort 是否计入 100 条？为什么 30 条 final 不能证明 5pp uplift？五个 task 怎么分 30 条？什么时候应把 40 条 collection 转成 evaluation？intervention 轨迹如何避免只学会人工接管前的坏动作？基础设施 invalid 能否补跑？

易错：边看 final 边调策略；静默删除急停/timeout；把 invalid 当 failure 或反过来混报；用 episode 数代替 env-step/风险暴露；40 条采集不记录 policy version；每个 batch 无限调参；final 回流 replay；只报 micro success；把模拟 RoboCasa/LIBERO rollout 说成真机；用 30 条结果宣称小幅稳定提升。

来源：[1][9][12][18][19][30]　**练习状态**：unseen

参考来源

1. Ross, Gordon, Bagnell, DAgger, 2011.
2. Schulman et al., GAE, 2016.
3. Schulman et al., PPO, 2017.
4. Kumar et al., CQL, 2020.
5. Kostrikov et al., IQL, 2022.
6. Rafailov et al., DPO, 2023.
7. Shao et al., DeepSeekMath / GRPO, 2024.
8. Hugging Face TRL.
9. 本地项目页：`01_robot_harness_tool_training.html`、`02_opd_vlm_grounding.html`、`04_cosmos_wam_multitask_training.html`、`05_uwm_libero_eval_protocol.html`。
10. Nair et al., AWAC: Accelerating Online Reinforcement Learning with Offline Datasets, 2020.
11. Liu et al., LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning, 2023.
12. Farama Foundation, Handling Time Limits, Gymnasium official documentation.
13. Haarnoja et al., Soft Actor-Critic Algorithms and Applications, 2019.
14. Amodei et al., Concrete Problems in AI Safety, 2016.
15. Yu et al., Gradient Surgery for Multi-Task Learning, 2020.
16. Robot Harness 本地 GRPO-style 实现快照：`C:\Users\mousongzhe\agent\remote_training_pipeline_scripts` 下，`train_grpo.py`（SHA-256 `5BF8D38FB6AEB9A21D125CC50031EDE25EEEE52A54F139876F9CA59E33490852`）构造 reference/empty/wrong-tool/reversed 合成 group，以 reference reward 减组均值形成 advantage、截断为 sample weight，并在 `RewardWeightedTrainer` 中优化 weighted reference token CE；`configs/reward_spec.json`（SHA-256 `DBDF3569A4F5C90E55520AEE56E5B32DA2CE447A12580717B86A060236F88F6B`）定义 task-family/component 权重；`eval_grpo_tasks.py`（SHA-256 `DD6DA1F8CF62F79A5DCF619382ABBD100883D4021E446F19AC25124C6F9816CA`）只检查数据与 reward-family coverage；`reports/grpo_report.md`（SHA-256 `121D3781415FA784CB99DA609E0998844EB032E5E32DDAF811B433F6BA22F9BC`）记录 200-step completed continuation 并明确不是 full online GRPOTrainer。该快照支持 reward-shaped offline grouped weighting 的实现合同，不支持 policy-sampled online rollout、old-logprob ratio/clip/KL objective、真实机器人 success、同 scorer 因果 uplift或逐文件个人 ownership。
17. GAE / PPO 相邻语义的本地实现快照：`C:\Users\mousongzhe\Project\wm\cosmos_policy_deepspeed_train.py`（SHA-256 `2A95D52DE64A5B46BC7C72C30B20BEC07C715FBF4FB590BF5DEA239CBA858680`）从 batch 读取 `value_function_return`，以独立 mask/denominator 计算 value、future-proprio、future-image MSE，与默认乘数 16 的 action MSE 相加，并记录 `world_model_fraction/value_fraction` mask 均值；`C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\train_grpo.py`（SHA-256 `5BF8D38FB6AEB9A21D125CC50031EDE25EEEE52A54F139876F9CA59E33490852`）在 `GRPODataset` 初始化时从离线 reference 与 empty/wrong-tool/reversed 负例构造 group，计算 reference 相对合成组均值的 sample advantage、裁成 sample weight，再优化 weighted reference token CE；它没有 policy version、collection timestamp、behavior log-prob、environment trajectory 或 rollout→learner 数据环。`C:\Users\mousongzhe\Project\openpivla_deployment\scripts\run_simpler_env_true_rollout.py`（SHA-256 `E96F6D444FFD73295F2CC0D116DFD9AEA4C67A62913932136BC9342A1F2E6BEA`）从 `--actions-json` 消费已导出的 `reference_actions/model_actions`，用 `np.clip` 裁到 `env.action_space` 后逐步送入 SimplerEnv，以 `terminated or truncated` 结束 eval episode并从 `info` / `terminated` 推断 success；它没有 model inference、optimizer 或 learner update。该快照只支持 masked multi-head MSE、mask coverage、value-return regression、group-relative sample weighting、外部 action-sequence eval、rollout termination/success 与 action-bound clipping 的实现边界，不支持本地 PPO/GAE trainer、on-policy freshness 管理、old-logprob ratio/clipped surrogate、PPO clip fraction/approx-KL、entropy bonus、GAE hyperparameter、PPO/GAE run 指标或逐文件个人 ownership。
18. 连续动作变换、监督权重与 RoboCasa episode-end 的本地 UWM / Cosmos 实现快照：`C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_dataset.py`（SHA-256 `64DA146524120A0DD110378CF903A898D1DAAA26507611E588E8221BE0BC0303`）以数据集逐维 min/max 构造 midpoint offset 与 half-range scale，并实现 affine normalize/reconstruct；`C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_lerobot_dataset.py`（SHA-256

536CC0E77247A15F8B2E0873A51BAA747CD6D636FFE0D8BB92A7605159886BB7）将 reward>0 或通用 done 合并成 success/contact proxy，并将 contact-rich window 的 loss_weight 设为 1 + contact_window_weight；
C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_train.py（SHA-256 EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE）用 diffusion scheduler 给 normalized action 加噪，以固定 action-dimension 与 sample weight 加权 action-noise MSE，并组合 dynamics、x_0 与辅助监督；
C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_rollout.py（SHA-256 E43D3751F8D079DAC54C902B5B222A13242FF5A9CE0F3975FB00D630DA53775E）执行 model.sample → reconstruct → auxiliary override → adapter → np.clip(env.action_spec)，其 step_env 丢弃 reward 与四/五元组的 done/terminated/truncated，外层仅按 check_success 或 steps>=horizon 停止；C:\Users\mousongzhe\Project\wm\cosmos_policy_robocasa_official_eval_wrapper.py（SHA-256 45CB2D44A547C645D15AE890DCFF023569CF126FC20E85A88B7160AED2CF5E42）只在 eval config 设置 unnormalize_actions=True、dataset stats path 与 deterministic=True，inverse-transform 实现在外部 upstream 依赖。该快照支持 affine action normalization/denormalization、contact/dimension-weighted diffusion-noise objective、adapter/aux override、env-bound clipping、success-or-horizon eval 与 episode-end 字段缺失边界，不支持本地 behavior-policy KL、$Q - V$ advantage weighting、CQL candidate-action penalty、正确 termination bootstrap mask、通用 done 的 success 语义、tanh-squashed Gaussian、policy log-prob/PPO ratio、Cosmos upstream inverse-transform 的精确公式、这些算法设计的个人原创性或逐文件 ownership。

19. offline learner / online evaluation 与 LIBERO episode-end 的本地数据流快照：
C:\Users\mousongzhe\Project\wm\cosmos_policy_deepspeed_train.py（SHA-256 2A95D52DE64A5B46BC7C72C30B20BEC07C715FBF4FB590BF5DEA239CBA858680）从预先存在的 data_dir/rollout_data_dir 构造 RoboCasaDataset，之后循环 DataLoader batch 并 backward/step，没有 env collector 或 buffer insertion；
C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\repo_patch\experiments\uwm\train.py（SHA-256 CFB4010A1DCA274C904295A7A937D2E1C5C3B08101A036D738F9C2B1560B4879）从固定 train/validation loader 优化；
C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\repo_patch\experiments\uwm\train_robomimic.py（SHA-256 842302703CB09A75A34611CD6B289D18C0268B710BCC2C0F9D0138192E2640BD）可在训练间隔执行 env rollout，但只把 success rate/video 写日志，没有将 transition/reward 写回 dataset、replay 或 loss；
C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\uwm_libero_rollout_eval.py（SHA-256 05E912C2096EFF4BFE8D061E4C334517B03DE80847793D83025A2AA3E18F3943）以 max_episode_length=rollout_length(+dummy_steps) 创建环境，消费四元组 obs,reward,done,info 并循环到单一 done，最终只写 info["success"]、policy steps 与 episode/summary，未保存 terminated/truncated/stop reason，也没有 optimizer update。该快照支持 fixed-dataset training、archived rollout sampling、interleaved/standalone online evaluation、single-done success-rate protocol 与 learner-dataflow 边界，不支持本地 online RL、采集数据回流、off-policy correction、由单一 done 推断 timeout/termination、正确 TD bootstrap mask、evaluation-to-training causal uplift 或逐文件个人 ownership。

20. critic 归因前的本地 action-pipeline 诊断快照：C:\Users\mousongzhe\Project\wm\cosmos_policy_action_diagnostics.py（SHA-256 71C20E8C525FC5F7FD3B720E19E917D5D54412E33FEEEB33EEC6FF27B5FF17F2）汇总 demo/base/trained action 的逐维 mean/std/min/max、step L2 与 policy mean 相对 demo std 的 mean-z；对应 outputs/cosmos_policy_action_diagnostics_20260606/action_distribution_diagnostics.json（SHA-256 0EE4101BEDD74623D147F1294B7DBCB333C49D89AD7F9F3F0D22ABF0D70ED55F）冻结 CoffeePressButton 5,868 个 demo 7D action、base/trained 各 300-step trace，以及 base 第 4/6 维 mean-z 11.7577/8.9503、trained -5.9451/-2.9966。
C:\Users\mousongzhe\Project\wm\cosmos_policy_gt_action_replay_diagnostics.py（SHA-256 CDD140108F5B475012FC85BA579E163C3B0D818BC3F49D5A2B10CFC7E7CAE5BD）检查 HDF5 action shape、7D→12D env assembly、global-stat min/max normalization 与 ground-truth env replay；汇总 gt_replay_and_dataset_diagnostics.json（SHA-256 B67402AA7F78F33EEBA93C045C47AF2D64126657B1222BD2789E3A783A37B2FD）及 per-run manifest.json（SHA-256 8DEE556538839EE83020494DCAE3CB386236EE5C4AB1E793B6D920455A9DAFE2）记录源 action shape 102×12、env action dim 12、replayed steps 80、success false，并明确 HDF5 initial state 可能与 env.reset 不匹配，失败不能单独证明 demo/parser/controller 错。结合 source [18] 的 UWM normalizer metadata/action order、preclip/env-action/clip-fraction 记录，该快照支持 marginal action-distribution warning、normalizer、dimension/order、controller 与 reset-state 诊断，不支持 state-conditional support、action-conditioned Q、critic calibration、offline-RL extrapolation 的直接观测、CQL/IQL run、因果 root cause 或逐文件个人 ownership。

21. shared visual encoder / value-isolation 的本地实现快照：C:\Users\mousongzhe\Project\wm\cosmos_policy_deepspeed_train.py（SHA-256 2A95D52DE64A5B46BC7C72C30B20BEC07C715FBF4FB590BF5DEA239CBA858680）的 proxy 使用 encoder→trunk→action/future-proprio/value/future-image heads，默认 AdamW(model.parameters())，未见 freeze/stop-gradient/linear-probe 分支；
C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_train.py（SHA-256 EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE）构造 UWMObservationEncoder 并以 AdamW(model.parameters()) 更新，--pretrained-vision 只控制轻量初始化时是否请求外部视觉权重，没有 encoder freeze 选项；
C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\repo_patch\experiments\uwm\train.py（SHA-256 CFB4010A1DCA274C904295A7A937D2E1C5C3B08101A036D738F9C2B1560B4879）加载可选 pretrain checkpoint 后同样对全部 model.parameters() 建立 AdamW。定向扫描未发现本地 frozen-encoder、value stop-gradient、linear-probe、encoder-drift 或 per-branch gradient-cosine artifact；该快照支持 shared-parameter/loss 与“因果消融尚缺”的边界，不支持视觉表征因果收益、value 破坏表征、独立 critic 训练或个人原创性。

22. Harness schema-aware SFT 的本地实现/运行快照：C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\train_sft.py（SHA-256 D5131EF7733B79D17CEEB7741C30C5E36471E1A9C8B75B3B1D20AF1943410EA8）把 system/tool schema 与 prompt 送入 chat template，把完整序列与

generation prompt 分别 tokenize，以 prompt token 和 padding mask `labels=-100`，只对 assistant target（文本或 `<tool_call>` JSON）做 causal-LM teacher-forcing；支持 `--freeze-vision-tower`、LoRA 与 `max_seq_length`，过度截断时保留一个 valid token 的 fallback。`prepare_sft_data.py`（SHA-256 `AF75E07ED5F2483686D0E4DFADFE23D7F50BD31FE3455FCC3BC7354AB5E85589`）将 OpenAI tool calls、ARL-SCOUT project trajectory 与高层 Robo2VLM VQA 混成 OpenAI-compatible messages/tools 数据，过滤低层 point/box target；`manifests/sft_data_manifest.json`（SHA-256 `2C15D6695AF57937BD0C48306AFC39870B7EBDDF1263AF85C0378A230F3FC619`）记录 train/dev/test `51467/2902/2854`、project sandbox 新增 1837、Robo2VLM 辅助 5000；`reports/sft_report.md`（SHA-256 `A35528A2494B459A1E62FB9FD4F1A2A08737AEF1A64F9BA1C56A746345CBD901`）记录 LoRA+gradient-checkpointing+frozen-vision 的 300-step run，train loss `0.50627`，但仅完成格式 dry-run，未完成模型 dev/test tool-call eval。该快照支持 token/JSON SFT 数据流、loss-mask 与运行边界，不支持 VLA 连续动作 head 训练、runtime memory/subagent 学习、线上机器人成功或个人逐文件 ownership。

23. DPO 个人材料与本地 smoke 骨架的证据边界：`output/mousongzhe_resume_optimized.html`（SHA-256 `5374D4B6F6617394381BB0C17E81C298CCEF34BF6C3FC329E6E6BF04D01F1D4C`）只直接支持广州皓岚经历中的训练/偏好样本整理、SFT/DPO 实验设计、错误归因与质量校验标准，不给出 pair 数、trainer、run 指标或逐文件 ownership。另一个本地仓库 `C:\Users\mousongzhe\muxi` 在 commit `756e27e16adcfb3ccbfe98c2920dda42095b127f` 下只有 `ds_test/configs/dpo_config.yaml`（SHA-256 `B281A11076EF65563B1FA20C75B7F7CC3780F7FBAC9E14B639E7F1613811F844`）与 `ds_test/scripts/run_dpo.sh`（SHA-256 `243DB3BA8E9132C53A5FEBCDCB2B07209094799FEBA9FF6C86B73BD8BEA5480D`）：前者声明 `beta: 0.1` 和 smoke 路径，后者调用通用 `transformers.trainer` module；定向文件/内容扫描未发现 `train_dpo.py`、`DPOTrainer`、chosen/rejected pair schema、本地 smoke data、DPO output 或 run log。该快照只支持“存在 DPO smoke 配置/启动骨架”和证据缺口，不支持 DPO-specific objective/collator 已实现、smoke 已通过、广州皓岚实习与该仓库同源、DPO uplift 或个人独立 ownership。

24. Robot Harness schema/reward/holdout evidence freeze：`interview_materials/vla/evidence/projects/robot_harness_claim_evidence_inventory.md`（SHA-256 `71C3B62F52204574C1B854B554258DABC1856F5D9044EB9628280A32500A797F`）冻结 62,223 SFT、8,916 canonical GRPO、1,200 restraint mix、8B/32B suite 的 n/scorer/protocol 边界、project sandbox proxy、safety/browser failure 与 Codex-primary ownership；`robot_harness_ownership_ledger.md`（SHA-256 `66E6A93DC1C512E0F5603355CBE910D6D7DB3AD0CA64AD41B0C494339BBA7BD7`）冻结 owner/reviewer/implementation/verification/result 双轴合同。对应远端实现快照为 `remote_training_pipeline_scripts/train_grpo.py`（SHA-256 `5BF8D38FB6AEB9A21D125CC50031EDE25EEEE52A54F139876F9CA59E33490852`）、`configs/reward_spec.json`（SHA-256 `DBDF3569A4F5C90E55520AEE56E5B32DA2CE447A12580717B86A060236F88F6B`）、`eval_grpo_tasks.py`（SHA-256 `DD6DA1F8CF62F79A5DCF619382ABBD100883D4021E446F19AC25124C6F9816CA`）、`eval_sft_tool_call.py`（SHA-256 `0B8AECAC309A61FFA8CF797FDD372069FD79E764A19456FEB673A071835EC7CB`）、`finalize_grpo_eval_reports.py`（SHA-256 `B3C9E2418615C883EA76CCFEA96C64385E5C571660800B6AF3D50334B4549653`）和 `reports/grpo_report.md`（SHA-256 `121D3781415FA784CB99DA609E0998844EB032E5E32DDAF811B433F6BA22F9BC`）。这些文件支持 schema-aware offline reward-weighted CE、字段 coverage evaluator、gold dry-run 与 protocol-bounded holdout/report 解释，不支持 native online GRPO、物理机器人 success、同 scorer/n 的严格因果 uplift 或逐文件个人 ownership。

25. Robot Harness reward-hacking 风险实现/评估快照：`C:\Users\mousongzhe\agent\remote_training_pipeline_v2\scripts\schema_aware_reward.py`（SHA-256 `A402D22299C1AF0BAB2D08E83E4389206882559294C3B1F55F9DF55AC0F85E1A`）处理 malformed/latent tool-call、execution/query tool、澄清/拒答、required/critical/semantic fields、robot scope 与禁止副作用；`reports/eval_report.md`（SHA-256 `542B4C8833225DF5D0BCD851B8CC6ACE97F881013E6546BFEAAC3090DD401F4D`）和 8B V3 `safety_side_effect_eval.metrics.json`（SHA-256 `C858C5777D08CB8AD398A28490FFB9B8FF925EAFC48F6AD1E3A2FBB384EA25B7`）冻结 n=289、valid JSON/tool-call 100%、scope respected 100%、forbidden-tool 0%、tool-name 44.29%、required-tool coverage 49.13%、semantic-argument 17.65%、argument-exact 2.77% 及空参数、错工具、错语义参数和多步缺失案例；同 run 的 `browser_deferred_eval.metrics.json`（SHA-256 `B18E9F45E37035D0A2796EC0F537BA452883063D3F8F8F885949787EB2DBB5A6`）与 `v3_full_grpo_schemaaware_native_tools_eval_report.md`（SHA-256 `B647FC5C0E2A0C7FD510D2CF3F42A64972F26C0C87194FB7D0290832F2D29D1B`）冻结 deferred/browser 与 suite 汇总边界。这些证据支持格式/scope proxy 与工具语义间的 gap、reward-hacking 风险诊断和 failure case 口述，不支持模型主动利用漏洞的因果证明、真实 browser/robot effect-sink、物理成功、native online GRPO 或个人逐文件独立实现。

26. Robot Harness V2 多目标 reward 与 restraint paired-check 快照：`C:\Users\mousongzhe\agent\remote_training_pipeline_v2\configs\reward_spec.json`（SHA-256 `BB4C3D7A62E78890186035975B9657C8201A2D5C7977B48D27A6D0767E2D83C3`）列出 21 个组件、9 类 task-family 权重和 7 个 hard penalties；`scripts/schema_aware_reward.py`（SHA-256 `A402D22299C1AF0BAB2D08E83E4389206882559294C3B1F55F9DF55AC0F85E1A`）实现 required/critical/enum exact、semantic threshold 0.72、state-success conjunction，并以 task-family weighted sum 加 penalty 聚合；`scripts/check_tool_restraint_reward.py`（SHA-256 `1C3FFEA7C94D3E304BFBCF1D3780E927E7A36FCBDAE059D688706AFEC840C459`）、`reports/tool_restraint_reward_check.md`（SHA-256 `C8780EE0DEA8F7534FB6946BE42B3431EE8BDF3E192C2C3290E604582DF8C8AD`）与 JSON（SHA-256 `ADE12649FF8CDDA4F39193DA968DD4D2D24A62BE53169F5C60A4D695DFAD6143`）冻结 128 个 scripted good/explicit-bad/malformed-latent paired cases，good/bad/malformed 均值为 1.0000/-5.3423/-5.3423、平均 gap 6.3423，未惩罚 bad/malformed 均为 0。该快照支持 component observability、task-conditioned aggregation、部分不可补偿 safety penalty 与固定反例 checker regression，不支持 Pareto/threshold 最优性、真实模型 holdout、线上 runtime enforcement、独立 effect-sink、reward 改动因果 uplift、native online GRPO 或个人逐文件独立实现。

27. Robot Harness V2 online GRPO group/noise 实现与运行快照：`C:\Users\mousongzhe\agent\remote_training_pipeline_v2\scripts\train_grpo_online.py`（SHA-256

EC66C0BB4F34210C76B4EB6942FA28FE3BBE1138213DB63118DF59FAA39CB987）对每个 prompt 采样 num_generations 个 completion，逐候选调用 schema-aware scorer，记录 component/penalty/backend/preview，以组内 mean/std 得到 advantage 并进入 clipped token policy loss；同组 reward 全相等时 advantage 为零。manifests/grpo_data_manifest.json（SHA-256 AE249F88EEE84FC91FEB0BC4800F29710A64BC08577F4E7FA7B6AA642A2AC795）冻结 8,916 条及 train/dev/test 8052/439/425、task-family 3735/2748/2300/120/13，但 validation 为空；reports/grpo_report.md（SHA-256 9B0F9365B8E44D171E9E2DC65B1886BFE8DEC9480A4DFFEA4E96D6795DB7DA5B）冻结 8B gen=8、1007 steps 与 reward range/分布；scripts/generate_final_reports.py（SHA-256 FC80551CAF7112A8CFEE03F4990C7E0692A7B4736721BDF4D650662E3CF4EC8A）记录 late equal-reward/zero-loss group 的诊断语义。该快照支持语言工具调用场景的 policy-sampled group、group-relative normalization、checker 噪声传播、同分退化与数据分布缺口，不支持 reward 是客观真值、preference agreement/flip/tie 已测、真实 browser/robot environment return、物理成功、与广州皓岚实习同源或个人逐文件独立实现。

28. Ng, Harada, Russell, Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping, ICML 1999；支持 potential-based shaping 的策略不变性条件，不支持任意 dense reward、局部实现正确性或项目成功归因。

29. OPD teacher/objective/ablation 本地证据门：interview_materials/vla/evidence/projects/opd_claim_evidence_inventory.md（SHA-256 F16B08E2E4C8DE58C0EECC4AEB7FF48CA90D6024A6F77F33D34424540825EBC4）冻结五 expert 数据/metadata route、coldstart/OPD5000 lineage、10k raw-holdout、checkpoint/pool/format 表以及 SFT-only/single-teacher/no-synthetic/raw-prediction 缺口；opd_training_objective_lineage_gate.md（SHA-256 5B9498966531981C21FF89980978BF16AA40D6FBD6D3F9C820FF2C70BFF7D1F4）按 full SHA 拆开 legacy teacher-rollout top1 CE 与 formal teacher/student-rollout full-vocabulary $D_{KL}(q_\beta \| p_S)$，并冻结 coldstart-500 teacher rollout、OPD5000 student rollout、veto/prefix 和缺失 checkpoint binary/raw predictions 的边界。该证据支持设计 matched ablation 与解释现有 confound，不支持 SFT-only/single-teacher/rollout-source/OPD 的单因素 uplift、grounding GRPO 效果或逐文件个人 ownership。

30. UWM/WAM protocol-separated evaluation 本地证据门：interview_materials/vla/evidence/projects/uwm_libero_claim_evidence_inventory.md（SHA-256 48616E5E88E87AA70D196983021C7D627AB6FE3B4ABFD8FAFF3AAAE436B7494D）冻结 repo reset、project fixed-pruned-init、UWM Table II OOD 三套协议、user-confirmed 186/250=0.744、reset-style 0.928 诊断、缺失 target checkpoint/250 episodes 与 RoboCasa raw-rollout边界；cosmos_policy_robocasa_baseline_p0_audit.md（SHA-256 39E59144D6CE36135A09314294C5E399BC1F2D686A55486C919DA5F5A63972B8）冻结 official-HF 24×5 raw 86/120、本地 trusted 84/118、protocol tuple、action-debug integrity 与缺失 checkpoint binary/paired trained eval。该证据支持 protocol-sensitive 分栏、denominator 重算和 matched-eval设计，不支持 Table II 可比复现、official baseline 个人成绩、训练 uplift或逐文件 ownership。

31. Ho & Ermon, Generative Adversarial Imitation Learning, NeurIPS 2016；支持 adversarial occupancy matching、discriminator-derived cost 与 model-free policy optimization 的方法定义，不支持本地已实现 GAIL、真实机器人样本效率、稳定性或项目 uplift。

32. Ng & Russell, Algorithms for Inverse Reinforcement Learning, ICML 2000；支持从示范行为反推 reward 再求策略及 reward ambiguity 的基本边界，不支持本地已识别唯一 reward、已有动力学/规划器、迁移收益或个人算法原创。

33. Zhao et al., Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, RSS 2023；支持 ACT 的 action chunking、有效预测步数缩短与 temporal ensemble 机制，不支持任意 VLA/chunk 长度必然稳定、真实机器人通用最优 K 或本地项目 uplift。

34. Action-horizon / execution-prefix 本地证据快照：C:\Users\mousongzhe\Project\openpivla_deployment\scripts\run_libero_true_rollout.py（SHA-256 1C7F74DE83FD373A27FEDC70D863D5F0BF164B4512ED63E1D76552A0F22C3016）在 deque 为空时请求 action chunk、只入队前 replan_steps×7D 并逐步执行；eval_lerobot_action_server.py（SHA-256 B1B7FB9B7A86355B9C1843CDE971B38FD26A2DAB1F18550128671F79F1AB4F39）的 action_horizon=50 只用于 GT/pred 离线 L1 window；openpivla_libero_checkpoint_load_lineage_gate.md（SHA-256 02B6D0A5ECC380C992B92EBC03E7BBE6EEAF47E30B9A22020B3F7A43F8C4648A）冻结 70-step、0-reward/false-success 与 effective argv 缺失边界。uwm_robocasa_rollout.py（SHA-256 E43D3751F8D079DAC54C902B5B222A13242FF5A9CE0F3975FB00D630DA53775E）以 exec_horizon 截取 sampled chunk；uwm_robocasa_actionheadfix_30epoch_train_eval_p0_audit.md（SHA-256 41B9CDDDB95163A369719E007EA11AD3A7776E79F0901F80DCACD34FAA83F117）冻结同一 54-scenario 的 K=1/2/4/5/8 sweep 与 success 2/1/0/1/1。cosmos_policy_robocasa_official_eval_wrapper.py（SHA-256 45CB2D44A547C645D15AE890DCFF023569CF126FC20E85A88B7160AED2CF5E42）和 cosmos_policy_full10k_trained_eval_p0_audit.md（SHA-256 DB83888B1279E548EBCA6426009448936690 5C632C5CC4F9E44F834BC12D66CF）冻结 chunk_size=32、num_open_loop_steps=16、32×7 action trace。该证据支持 H/K/离线-window/queue 区分和 protocol-sensitive sweep，不支持 chunking 因果 uplift、最佳 K、所有 run 的 effective defaults、temporal ensemble 已启用或逐文件 ownership。

35. Split / leakage / lineage 本地证据快照：C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_train.py（SHA-256 EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE）先构造完整 dataset，再以 random_split(dataset index) 切 train/val；uwm_robocasa_lerobot_dataset.py（SHA-256 536CC0E77247A15F8B2E0873A51BAA747CD6D636FFE0D8BB92A7605159886BB7）在构造时从 self.episodes 建 sequence windows并可计算全 episode action stats。该结构在 val_ratio>0 时存在同 episode adjacent-window 与 stats contamination 风险；已审计 30-epoch run 的 val_ratio=0/eval_every=0 只证明无 held-out validation，不证明发生 leakage。opd_claim_evidence_inventory.md（SHA-256 F16B08E2E4C8DE58C0EECC4AEB7FF48CA90D6024A6F77F33D34424540825EBC4）冻结 10k raw-holdout 的 0 source-line/fingerprint/image+prompt+answer duplicate、0 train-image leakage，以及 1168 duplicate image paths/2917 affected annotation rows 的合法多标注边界；robot_harness_claim_evidence_inventory.md（SHA-256 71C3B62F52204574C1B854B554258DABC1856F5D9044EB9628280A32500A797F）冻结 base/ladder 的 n/scorer mismatch；openpivla_claim_evidence_inventory.md（SHA-256 13023578D1B7343083BEB588C7F8CE7B1BDC267E2BC834204D309106E8EA132A）冻结 97.4 缺 raw

episode/checkpoint/protocol 的不可审计边界。该证据支持 leakage-risk、dedupe-positive、legal-overlap 与 protocol-mismatch 区分，不支持已证实 UWM 某 run 泄漏、clean retrain、OpenPiVLA zero-contamination 或逐文件独立 ownership。

36. Williams, Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning, Machine Learning 8, 1992；支持 REINFORCE score-function estimator、episodic return weighting与无偏 reinforcement comparison 的方法边界，不支持任意 learned baseline/bootstrapping 无偏、local GRPO 等价 vanilla REINFORCE、机器人样本效率或本地项目 uplift。
37. Konda & Tsitsiklis, Actor-Critic Algorithms, NeurIPS 1999；支持以 TD critic 向 actor 提供近似梯度信息、two-time-scale actor–critic 与函数逼近条件的方法边界，不支持任意 nonlinear/shared-encoder critic 稳定、local WAM value head 等价 actor–critic、VLA chunk target 已实现或本地项目 uplift。

Part 3 / 4 · `interview_materials/vla/sources/vol_02_il_rl_posttraining_sources.html` · SHA-256 `B9476798E485231768CD4B11BB929DAED305C762AC8C61A877C1F3ECDA0876B7`

返回本卷目录

Canonical Markdown SHA-256: `b9476798e485231768cd4b11bb929daed305c762ac8c61a877c1f3ecda0876b7`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_02_il_rl_posttraining_sources.md`

VOL 02 IL / RL / Post-training：来源计划

调研日期：2026-07-16；registry reconciliation：2026-07-19 状态：`source_registry_reconciled / seven_core_methods_independently_reviewed / supporting_sources_registered / local_gates_warn` 目标：为 IL、PPO、offline RL、DPO、GRPO 和 schema-aware reward 建立 primary-source ledger。边界：七个核心方法与 Gymnasium/PyTorch 文档已独立复核；AWAC、LIBERO、SAC、AI Safety、PCGrad 仍是 supporting registry，不宣称达到相同 review 深度。Robot Harness 本地实现是 reward-shaped offline GRPO-style continuation，不是 canonical online GRPO。

1. Primary papers

主题	来源	计划支持的 claim	状态
DAgger	A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning	expert relabeling、covariate shift 缓解和在线数据聚合	independently reviewed / WARN
GAE	High-Dimensional Continuous Control Using Generalized Advantage Estimation	bias/variance、λ、advantage estimator	independently reviewed；terminated/truncated boundary patched / WARN
PPO	Proximal Policy Optimization Algorithms	ratio、clipping、trust-region 近似	independently reviewed / WARN
CQL	Conservative Q-Learning for Offline Reinforcement Learning	OOD action、conservative Q、offline extrapolation	independently reviewed；regularizer≠full objective / WARN
IQL	Offline Reinforcement Learning with Implicit Q-Learning	advantage-weighted policy extraction、避免显式 OOD action evaluation	independently reviewed / WARN
AWAC	Accelerating Online Reinforcement Learning with Offline Datasets	advantage-weighted regression、offline-to-online policy extraction	URL verified
DPO	Direct Preference Optimization	preference pair、reference policy、KL-regularized objective	independently reviewed / WARN
GRPO	DeepSeekMath	group-relative reward、critic-free/relative advantage 叙事边界	independently reviewed；local implementation separated / WARN
LIBERO	Benchmarking Knowledge Transfer for Lifelong Robot Learning	task/suite split、机器人评估与 contamination 审计语境	registered/supporting；VOL 04/05 protocol pages carry deeper review
SAC	Soft Actor-Critic Algorithms and Applications	tanh-squashed Gaussian 与 log-prob 变量变换；仅作连续动作分布机制来源	registered/supporting
AI Safety	Concrete Problems in AI Safety	reward hacking / reward gaming 的目标错配边界	registered/supporting
PCGrad	Gradient Surgery for Multi-Task Learning	多目标 gradient interference 的诊断与一种缓解方法	registered/supporting；VOL 04 auxiliary-gradient pages carry deeper review

2. Official documentation / implementations

来源	用途	状态
Hugging Face TRL	SFT、DPO、GRPO trainer 的 API 和配置边界	registered only；exact version/API call 未绑定本地项目
PyTorch KLDivLoss	KL input/target/reduction 语义	official docs independently reviewed
Gymnasium: Handling Time Limits	terminated / truncated 与 value bootstrap 语义	official docs independently reviewed
OpenVLA	VLA imitation / fine-tuning context	frozen/reviewed in VOL 03；只作跨卷语境
Physical Intelligence openpi	policy fine-tuning / action expert context	frozen/reviewed in VOL 03；不证明 Harness 实现
NVlabs/cosmos-policy	action/world/value multi-objective context	frozen/reviewed in VOL 04；不证明本地 WAM run

3. 本地项目映射

VOL 02 主题	本地材料	不能直接推出的结论
SFT / GRPO / schema-aware reward	Robot Harness Inventory + Ownership Ledger	local trainer 为 offline grouped reward weighting；tool proxy 不等于机器人 success；scorer/n/checkpoint/ownership 仍 WARN
action/world/value	WAM Inventory + Ownership Ledger	proxy MSE 不等于 online gain；checkpoint/denominator/ownership 未闭合
OPD / teacher ablation	OPD Inventory	缺 matched SFT-only/single-teacher raw ablation，不能单因果归因

VOL 02 主题	本地材料	不能直接推出的结论
DeepSeek 数据/偏好样本	`output/mousongzhe_resume_optimized.html`	只能讲有证据的样本整理、实验与错误归因；不虚构规模、结果与 ownership

4. Registry reconciliation

闭环项	当前证据	状态
DAgger/GAE/PPO/CQL/IQL/DPO/GRPO 核心机制	独立事实复核	`checked / WARN`
terminated/truncated、CQL partial objective、local GRPO boundary	knowledge/questions + independent review	`patched and checked`
VOL 02 knowledge / cards	consolidated knowledge + `VLA-RL-001..040`	`40/40 / all unseen`
AWAC/LIBERO/SAC/AI Safety/PCGrad	primary URLs 与跨卷 supporting usage	`registered/supporting`
TRL exact API/version 与本地调用	未绑定 local code/revision	`pending`
Harness same-scorer base/SFT/GRPO causal comparison	n/scorer/checkpoint/raw bundle 不齐	`WARN`
candidate ownership 与口述	ownership ledger 未确认；无真实回答	`PENDING`

5. 下一 bounded slice

1. 不再生成 VOL 02 题卡；40 张保持 `unseen`。
2. 优先进行 `VLA-RL-011`、`VLA-RL-027` 或 `VLA-RL-028` 单题口述；有真实回答后才创建 coaching state。
3. 无候选人回答时，按相同流程闭合 VOL 04 source registry，消除最后一个 registry-only 例外。
4. 后续若需要提高 VOL 02 source depth，再单独复核 supporting sources 与 TRL exact version，不在本 registry closure 中伪造完成。

Part 4 / 4 · `interview_materials/vla/sources/vol_02_il_rl_posttraining_independent_fact_review.html` · SHA-256 `CF1F14AF83AFA65B11509DA51AF4C486CE84BC0E89654A33F7977497FBC0492A`

返回本卷目录

Canonical Markdown SHA-256：`cf1f14af83afa65b11509da51af4c486ce84bc0e89654a33f7977497fbc0492a`；生成自：`interview_materials\vla\sources\vol_02_il_rl_posttraining_independent_fact_review.md`

VOL 02 IL / RL / Post-training：独立事实与实现边界复核

状态：`independent_fact_review / source_checked / local_grpo_boundary_checked / WARN` 审查日期：2026-07-19 证据截止：2026-07-19 范围：VOL 02 knowledge、40 张题卡、primary papers、官方文档与 Robot Harness 本地 GRPO-style trainer。 结论先行：DAgger、GAE、PPO、CQL、IQL、DPO、GRPO 的核心机制与现有知识页基本一致；需要明确修正三处容易混淆的口径：GAE 的 terminated/truncated mask、CQL 公式只是保守正则项而非完整 objective、当前 Harness 本地实现是 `offline_grouped_reward_weighted_policy_update`，不能直接称 DeepSeekMath 式 online GRPO。所有题卡继续保持 `unseen`，项目指标与个人 ownership 仍由 VOL 07 evidence gate 约束。

1. 问题与审查方法

本复核回答四个问题：

1. VOL 02 的公式、术语和方法边界是否与 primary paper 一致?
2. 哪些内容是论文/官方文档事实，哪些只是面试抽象或本地推断?
3. Robot Harness 的 GRPO、reward、schema 和 eval 是否能绑定到真实本地代码?
4. 哪些修正必须回写 knowledge 或题卡，才能避免高压追问时过度声称?

判定标签：`primary-verified`（论文/官方文档直接支持）、`official-code`（官方实现支持）、`local-direct`（本地文件可读取）、`inference`（由证据推断）、`blocked`（缺 runtime/raw/ownership）。本页不把“公式正确”升级为“个人项目已实现”。

2. 冻结来源矩阵

ID	来源	类型	本次核对的范围	Verdict
S01	DAgger, arXiv `1011.0686`	primary paper	expert relabeling、aggregated dataset、no-regret/data-distribution 叙事	`primary-verified`
S02	GAE, arXiv `1506.02438`	primary paper	TD residual、γ/λ bias-variance、advantage estimate	`primary-verified`
S03	PPO, arXiv `1707.06347`	primary paper	old/new policy ratio、clipped surrogate、trust-region 近似边界	`primary-verified`
S04	CQL, arXiv `2006.04779`	primary paper	conservative Q regularizer、OOD action overestimation	`primary-verified`
S05	IQL, arXiv `2110.06169`	primary paper	expectile value、advantage-weighted extraction、避免显式 OOD max	`primary-verified`
S06	DPO, arXiv `2305.18290`	primary paper	chosen/rejected、reference policy、KL-regularized log-ratio	`primary-verified`
S07	DeepSeekMath / GRPO, arXiv `2402.03300`	primary paper	group-relative reward、组内相对优势、实现差异边界	`primary-verified`
S08	Gymnasium time-limit handling	official docs	`terminated` 与 `truncated`、bootstrap 语义	`official-code`
S09	PyTorch `KLDivLoss`	official docs	input/target/reduction 语义；用于 DPO/KL 面试对照	`official-code`
S10	VOL 02 source plan	local canonical source	来源 URL、项目映射与待核列表	`local-direct`
S11	`C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\train_grpo.py`	local-direct	reward-shaped grouped weighting、Trainer loss、LoRA/DeepSpeed 参数	`local-direct`
S12	`...\configs\reward_spec.json` / `eval_grpo_tasks.py`	local-direct	task-family weights、hard penalty 设计、格式/字段覆盖评估	`local-direct`
S13	Robot Harness Claim–Evidence Inventory	local evidence audit	数据量、run lineage、scorer、safety/browser 与 ownership 边界	`local-direct / WARN`

3. Claim-to-source 复核矩阵

Claim	复核结论	必须保留的边界
BC 在 expert distribution 上拟合 action	`primary-verified / textbook-consistent`	不自动解决 covariate shift；action window、padding、split 仍是实现合同
DAgger 通过策略访问态再让 expert 标注	`primary-verified`	需要在线 expert/oracle；recovery 数据只能说“类似作用”，不能直接称 DAgger
GAE 是 TD residual 的 γλ 加权和	`primary-verified`	`d_t` 必须区分 true termination 与 time-limit truncation；timeout 通常需要 bootstrap
PPO clipping 近似 trust region	`primary-verified`	clip 只覆盖 surrogate policy term，不是严格 KL 约束，也不替代 value/entropy/rollout freshness
CQL 惩罚数据外动作的 Q 过估	`primary-verified`	α(log-sum-exp − data-Q) 是核心正则化示意，不是完整 CQL loss 或安全保证

Claim	复核结论	必须保留的边界
IQL 不显式对数据外 action 做 max-Q	`primary-verified`	仍依赖 reward、expectile value、数据覆盖和 advantage-weighted extraction
DPO 用 chosen/rejected 相对 reference log-ratio	`primary-verified`	β 与 reference/KL 锚点必须绑定；pair 质量决定结果，不代表无 reward 假设
GRPO 使用同 prompt 的 group-relative reward	`primary-verified`	具体实现可能有 clip/KL、std=0 处理、采样/过滤差异，不能把所有 GRPO 代码视为相同
schema 是输出契约，reward/checker 是训练信号，holdout 是验收协议	`inference / engineering contract`	这是面试系统设计抽象，不是某篇论文的统一算法定理
Robot Harness 有 GRPO trainer	`local-direct`	本地脚本日志写明 `offline_grouped_reward_weighted_policy_update`；它不是已核验的 online multi-candidate GRPOTrainer
Robot Harness reward 约束 tool/scope/cancel 等组件	`local-direct`	reward_spec 是高层工具协议 proxy；不等于真实物理成功、安全闭环或个人独立 ownership
Robot Harness 95.6% ladder 是 GRPO uplift	`protocol-limited / WARN`	base/SFT/GRPO 的 n、scorer 与 checkpoint lineage 不完全相同，不能做无条件因果归因

4. 关键公式与实现核对

4.1 GAE 的 termination / truncation 修正

令 d_t^{term} 表示环境真正终止，d_t^{trunc} 表示时间上限截断。GAE 的 TD residual 可写为：

$$\delta_t = r_t + \gamma(1 - d_t^{term})V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}.$$

这里 d_t^{term} 决定是否允许 bootstrap；time-limit truncation 通常不能当作真实 terminal。实现映射：batch 必须保存 `terminated` 与 `truncated`，不能把 Gymnasium 两个 flag 合成一个无语义的 `done`。当前 VOL 02 原文已有“termination/timeout”提醒，本复核把变量边界正式写清。

4.2 CQL 只是完整 objective 的核心正则项

离散动作的常见核心项可写为：

$$\mathcal{L}_{CQL}^{reg} = \alpha \left(\mathbb{E}_s \log \sum_a \exp Q(s,a) - \mathbb{E}_{(s,a)\sim D} Q(s,a) \right).$$

第一项压低候选动作的过高 Q，第二项保留数据动作的相对价值；真正训练还要和 Bellman/TD 误差、target network、优化器和实现变体组合。当前 VOL 02 的“简化项”表述是可接受的，但题卡答案不得说 CQL 单独保证安全。

4.3 本地 Harness 的 GRPO-style 更新

本地 `train_grpo.py` 先从 reference trajectory 构造 reference、empty、wrong-tool、reversed 等候选，按 `reward_spec.json` 计算 reward，再用样本权重进入 token CE：

$$\mathcal{L}_{local} = \frac{\sum_i w_i \, \ell_{CE}(y_i \mid x_i)}{\sum_i w_i + \epsilon}, \qquad w_i = \mathrm{clip}(1 + (r_i - \bar{r}), 0.2, 2.0).$$

这描述的是当前本地脚本的 reward-shaped offline continuation，不是 DeepSeekMath 论文的完整 online group policy optimization。实现证据来自脚本 SHA `5BF8D38FB6AEB9A21D125CC50031EDE25EEEE52A54F139876F9CA59E33490852`；它能支持“本地实现了 GRPO-style reward weighting”，不能支持“已运行标准 online GRPO 并证明真实机器人收益”。

5. 本地实现与项目 ownership 边界

5.1 直接可读的本地证据

- `train_grpo.py`：`GRPODataset`、candidate group、`score_candidate`、`RewardWeightedTrainer.compute_loss`、LoRA/DeepSpeed/Trainer 参数。
- `reward_spec.json`：11 类 reward components、task-family weights 和 hard-penalty 字段；SHA `DBDF3569A4F5C90E55520AEE56E5B32DA2CE447A12580717B86A060236F88F6B`。
- `eval_grpo_tasks.py`：只验证数据字段和 reward-family coverage，不运行真实 robot/environment。
- `grpo_report.md`：记录 200-step completed run 与三次失败/中断尝试，但明确注明这是 reward-shaped GRPO-style continuation；报告 SHA `121D3781415FA784CB99DA609E0998844EB032E5E32DDAF811B433F6BA22F9BC`。

5.2 不能从这些文件推出的结论

1. 不推出 online multi-sample environment rollout 已闭合。
2. 不推出 schema reward 等价于真实物理 success 或 safety。
3. 不推出 95.6% ladder 是同 scorer、同 n、同 checkpoint lineage 的 base→GRPO 因果提升。
4. 不推出 Git author 或 Codex 生成记录等于候选人逐文件独立 ownership。

个人项目口述应使用：Robot Harness Inventory 与 Ownership Ledger 的分层口径。

6. 修正动作与验证状态

动作	结果	状态
对照 DAgger/GAE/PPO/CQL/IQL/DPO/GRPO primary source	核心机制一致；补充边界	`PASS`
对照 Gymnasium terminated/truncated	已回写 knowledge 与题卡，显式区分 timeout bootstrap	`PASS WITH WARNING`
对照本地 `train_grpo.py`	确认是 reward-shaped offline GRPO-style continuation	`PASS WITH WARNING`
对照 reward/eval scripts	证明 schema/reward/holdout 是不同层，不是物理 success	`PASS`
对照 Robot Harness inventory/ledger	数据、scorer、n、ownership 保持 WARN	`PASS WITH WARNING`
真实口述	尚未进行；40 张题卡继续 `unseen`	`PENDING`

7. 结论与下一步

VOL 02 的通用知识可以进入 `fact_reviewed_candidate` 候选状态，但不能进入 `reviewed/drilled`：仍没有用户真实口述。GAE timeout 与 local GRPO-style implementation 两处边界已回写并完成静态校验。下一小步不是再生成更多题，而是做一题口述，例如 `VLA-RL-011`、`VLA-RL-027` 或 `VLA-RL-028`。所有 40 张题卡继续 `unseen`，不创建 `coaching_state.md`。

参考文献（References）

1. Ross et al., DAgger, arXiv:1011.0686。
2. Schulman et al., GAE, arXiv:1506.02438。
3. Schulman et al., PPO, arXiv:1707.06347。
4. Kumar et al., CQL, arXiv:2006.04779。
5. Kostrikov et al., IQL, arXiv:2110.06169。
6. Rafailov et al., DPO, arXiv:2305.18290。
7. Shao et al., DeepSeekMath / GRPO, arXiv:2402.03300。
8. Gymnasium, Handling Time Limits。
9. PyTorch, `KLDivLoss`。
10. VOL 02 source plan。
11. Robot Harness Claim–Evidence Inventory。
12. Robot Harness Ownership Ledger。

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 03 · VLA / Action

VLA 架构、动作表示、token / diffusion / flow 与闭环控制

> 本卷由 16 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 03 VLA / Action：从多模态条件到可执行动作（完整卷）

题卡

1. 02 · VOL 03 VLA / Action：核心面试题卡（完整卷）

来源与事实复核

1. 03 · VOL 03 Source Audit C：ACT CVAE、Action Chunk 与 Temporal Ensemble Claim-to-Code
2. 04 · VOL 03 Source Audit C：ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核
3. 05 · VOL 03 Source Audit F：VLA 架构、适配、延迟与动作执行 Claim-to-Code
4. 06 · VOL 03 Source Audit F：VLA 架构、Mask、适配与执行独立事实复核
5. 07 · VOL 03 VLA / Action：44-Card Completion Reconciliation
6. 08 · VOL 03 Source Audit D：Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code
7. 09 · VOL 03 Source Audit D：Diffusion Policy 独立事实复核
8. 10 · VOL 03 Source Audit E：Flow Matching 基础、π0 特化与 ODE Claim-to-Code
9. 11 · VOL 03 Source Audit E：Flow Matching / π0 Time–Sign 独立事实复核
10. 12 · VOL 03 Source Audit A：OpenVLA Action Token 与 I/O Claim-to-Code
11. 13 · VOL 03 Source Audit A：OpenVLA Endpoint / Token / Mask 独立事实复核
12. 14 · VOL 03 Source Audit B：π0 / π0.5 / FAST / openpi Claim-to-Code
13. 15 · VOL 03 Source Audit B：π0 / π0.5 / FAST / OpenPI 独立事实复核
14. 16 · VOL 03 VLA / Action：来源计划

Part 1 / 16 · `interview_materials/vla/knowledge/vol_03_vla_action.html` · SHA-256
`84DD9D5C12035509BCE0AC4C39B315973E46EE4711F95489E5EE02F12EAB919F`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：
`84dd9d5c12035509bce0ac4c39b315973e46ee4711f95489e5ee02f12eab919f`；生成自：
`interview_materials\vla\knowledge\vol_03_vla_action.md`

VOL 03 VLA / Action：从多模态条件到可执行动作（完整卷）

状态：`completion_reconciled_candidate / source_audits_warn / WARN` 报告日期：2026-07-19 证据截止：2026-07-19 完整覆盖：`VLA-ARCH-001..012`、`VLA-ACT-001..032`，共 44 题 适用岗位：VLA / 具身智能算法 / 多模态机器人策略 / Robot Learning 最后一批增量：补齐 RT-2/OpenVLA/Octo/π0 landscape、sequence/attention/loss mask、适配策略、异构动作、延迟与证据审计，以及 decode→controller 的执行闭环和最小单测。 项目边界：OpenPiVLA 的具体 checkpoint、Qwen/π 路径、action 轴序、horizon、normalization asset、97.4% protocol 与个人贡献仍须本地 artifact 证明。

目录

1. 问题（Question）
2. 知识地图位置与批次边界
3. VLA、VLM 与传统 visuomotor policy
4. VLA 输入输出合同与时间对齐
5. Backbone、action expert、policy head 与 controller
6. OpenVLA：逐标量 action token 路线
7. π0：连续 action expert 与 Flow Matching
8. VLA 架构路线、mask、适配与部署证据
9. Action representation 总览
10. Normalization / denormalization
11. FAST：频域 action token
12. ACT：CVAE、Action Chunk 与 Temporal Ensemble
13. Diffusion Policy：条件动作扩散与 Receding Horizon
14. Flow Matching：CFM、OT Path、ODE 与 π0
15. Action dimension、padding、mask 与 slicing
16. 平移、旋转、gripper 与 base action

17. 模型输出到 controller 与最小闭环单测
18. 如何选择 action representation
19. 训练与闭环推理 Pipeline
20. 实现证据与 44 题覆盖
21. OpenPiVLA 项目挂钩与真实性边界
22. Bad Case 与最小检查清单
23. 后续工作
24. 参考文献（References）

问题（Question）

本卷回答一条从“模型类别”到“机器人执行”的最短闭环：

```
# Full-volume chain: each arrow remains an auditable interface.
RGB / language / proprioception / masks / timestamps
                          ↓
       VLM backbone or visuomotor encoder
                          ↓
 scalar/FAST token / ACT CVAE / Diffusion / Flow expert
                          ↓
        decode → denormalize → valid-dim slice
                          ↓
       controller units → execute → observe again
```

面试官常以这些问题连续追问：

- VLA 相比 VLM 多了什么，为什么不能把 action token 当普通文本？
- 多视角图像、语言和 proprioception 如何进入模型，时间是谁对齐的？
- OpenVLA 与 π0 的 action output 为什么一个是 token、一个是 continuous flow？
- FAST 为什么先做 DCT，再做 BPE？token 少是否就一定更准？
- ACT 的 posterior latent 为什么训练可见 action、推理却固定 $z = 0$？temporal ensemble 融合的究竟是哪几个预测？
- Diffusion Policy 的 $T_o/T_p/T_a$、diffusion steps 与真实 replan interval 为什么是五条不同时间轴？
- 训练 action dimension 与机器人执行 dimension 不同时，padding、loss mask 和 slicing 怎样证明正确？
- translation、rotation、gripper、base velocity 的单位和统计能否混在一起归一化？
- 如果 offline loss 正常但 rollout 0-success，应先查模型还是 action interface？

一句话总口径

> VLA 的核心不是“让 VLM 输出一个动作字符串”，而是建立从多模态观测到可执行控制量的完整策略合同：输入必须有视角、时间与 mask；输出必须有表示、shape、尺度与 axis 语义；训练目标、

> decode、denormalization、slicing、replan 和 controller 必须前后一致。

知识地图位置与批次边界

前置知识

- VOL 01：observation/state/action、坐标系、控制频率、Transformer、mask 与 closed loop。
- VOL 02：BC/SFT 的监督语义、token CE、连续 loss、offline proxy 与 online success。

完整卷做什么

范围	本批深度	依据
VLA definition / I/O	机制级	RT-2、OpenVLA、π0 与 frozen repos
OpenVLA scalar token	公式 + code mapping	Source Audit A
π0 flow action expert	公式 + paper/code convention	Source Audit B
FAST tokenizer	DCT/round/BPE + frozen Hub code	Source Audit B
ACT CVAE / chunk / temporal ensemble	公式 + code mapping	Source Audit C
Diffusion Policy	epsilon/sample target、scheduler、horizon、rollout	Source Audit D
Flow Matching / π0	CNF/CFM、OT path、ODE、time/sign 与 padding/debug	Source Audit E
RT-2 / OpenVLA / Octo / π0 landscape	backbone、action representation、training route	Source Audit F
sequence / mask / adaptation / latency	attention/loss、full/head/LoRA、timing decomposition	Source Audit F
output → controller / tests	decode、unnorm、slice、axis/unit、queue、safety	Source Audit F
continuous / token / CVAE / diffusion / flow taxonomy	机制级对照	primary papers + Source Audits
action semantics / normalization / valid dimensions	tensor contract + design checklist	openpi docs/code + robotics representation source

仍不提前声称什么

1. ACT、Diffusion、Flow 的 upstream implementation 不证明本地 OpenPiVLA 采用相同 head、scheduler、solver 或 rollout。
2. Source Audit E 的通用 CFG example 不证明 frozen openpi Pi0 已启用 guidance。
3. 不用 openpi example config 证明本地 OpenPiVLA 的 H、D、K 或 controller。
4. 不因知识页生成就创建 mastered/pass 状态；后续题卡仍从 unseen 开始。

VLA、VLM 与传统 visuomotor policy

先讲直觉

- **VLM** 回答“图里有什么、任务是什么意思”，输出通常是文本或语义表示。
- **传统 visuomotor policy** 直接把传感器观测映射到机器人动作，常为任务专用，未必继承大规模语言视觉预训练。
- **VLA** 把视觉、语言和机器人动作放进一个端到端策略合同，既利用 VLM 的语义先验，又必须输出能经过机器人接口执行的 action。

RT-2 将机器人动作表达为 text tokens，与视觉语言任务共同训练，并把这类模型称为 Vision-Language-Action model。[1] OpenVLA 延续离散 action token 路线，而 π0 让 VLM context 条件化一个连续 action expert。[2][4]

严谨定义

给定观测历史 $o_{\le t}$、任务条件 c 和可能的机器人 state/proprioception q_t，策略建模：

$$\pi_\theta(A_t \mid o_{\le t}, c, q_t), \qquad A_t = [a_t, \dots, a_{t+H-1}].$$

变量解释：π_θ 是整个 policy，不只是 action head；$A_t \in \mathbb{R}^{H\times d}$ 可以是 H 步连续动作，也可以先被编码成 token sequence。H=1 时是单步 action。实现映射：OpenVLA 原始旗舰路径实际为单图 + 语言 → 单步 d 维动作；π0 paper/openpi 支持多图、state 和 H 步 chunk。[2][3][4][6]

VLA 与 VLM 的分界不在模型名字，而在**是否存在可训练、可解码、可执行、可评估的 action contract**。如果模型只输出“move left”而没有 axis、unit、controller 和闭环执行定义，它仍是语义 planner/VLM component，不是完整低层 VLA policy。

面试对比表

维度	VLM	传统 visuomotor policy	VLA
输入	image/video + text	image/state/history	image/text/state/history 的组合
输出	text、box、embedding	continuous/discrete robot action	token action、continuous head 或生成式 action chunk
预训练	web vision-language 常见	task/robot data 为主	VLM prior + robot trajectory adaptation
语言泛化	核心能力	可选	通常是关键条件
执行合同	通常没有 controller action	必须有	必须有，且更容易被 token/normalization 掩盖
评估	VQA/caption/grounding	task success / control metrics	既要语义泛化，也要 closed-loop success

这张表是职责分类，不意味着 VLA 在任何任务上都优于小型 visuomotor policy。数据不足、延迟严格或语言无关时，较小的专用 policy 可能更合适。

VLA 输入输出合同与时间对齐

输入合同

通用 VLA observation 可以写为：

$$o_t = \left(\{I^v_{\tau_v}, m_v, \tau_v\}_{v=1}^{V}, \ell, q_{\tau_q}, h_t\right).$$

变量解释：$I^v_{\tau_v}$ 是第 v 路相机在时间 τ_v 的图像；m_v 是该 view 是否有效的 mask；ℓ 是语言指令；q_{τ_q} 是 proprioception；h_t 是可选历史/记忆。直觉是“多模态输入”不只是拼字段，还必须说明每个字段何时采样、是否缺失。openpi model boundary 接收已经对齐和 resize 的 arrays；时间同步通常应在数据转换或环境 wrapper 完成，不能假设 Transformer 自动修正 timestamp mismatch。[6][12]

最小字段表：

字段	必须说明	常见失败
image/view	camera name、shape、颜色空间、resize/crop/flip、mask	训练/推理视角或翻转不一致
language	instruction 来源、模板、tokenizer、截断长度	prompt 模板改变 action positions
proprio	axis order、unit、frame、timestamp、normalization	位置/速度混淆；state 与图像错帧
history	window、stride、padding、causal rule	history 泄漏未来或顺序颠倒
action target	type、H、d、control frequency、absolute/delta	label 与执行频率不匹配
dataset identity	norm stats key、embodiment/control mode	使用错误 checkpoint assets

时间对齐

若以 control tick t_k 为监督锚点，可定义：

$$I^v_k = I^v_{\arg\min_\tau |\,\tau - t_k\,|}, \quad q_k = \mathrm{interp}(q, t_k), \quad A_k = [a(t_k), a(t_k + \Delta t), \ldots, a(t_k + (H-1)\Delta t)].$$

变量解释：图像可用 nearest timestamp，连续 proprio 可插值，action label 按控制周期 Δt 构造 H 步窗口。该式是通用数据设计合同，不是对 OpenVLA/openpi 某个未读 converter 的事实声明。实现中应记录最大时间差阈值、缺帧 mask 和 episode 边界；窗口越界时不可把下一 episode 的动作拼进来。

两个已核实的具体实例

- OpenVLA frozen flagship：`image_primary[0]` + language + `action[0]`；默认 `load_proprio=False`、`window_size=1`、`future_action_window_size=0`，即单图、无 proprio/history、单步动作。[2][3][12]

- π0/openpi：论文为 2–3 张图 + language + proprio，H=50；frozen PioConfig 默认有三个 image slots、state [B,32]、actions [B,50,32]，但 config 可覆盖。[4][6][12]

Backbone、action expert、policy head 与 controller

职责不能混写

组件	主要输入	主要输出	负责什么	不负责什么
vision encoder / VLM backbone	image + language	contextual tokens	视觉语义、语言条件与预训练知识	不自动定义机器人 axis/unit
projector / modality adapter	visual/state features	shared-width tokens	对齐 embedding space	不等于完整 action policy
action tokenizer	continuous action / chunk	token IDs 或 inverse	非参数化/轻量表示转换	不负责 closed-loop control
policy head	context embedding	action logits/continuous params	直接预测 action distribution	不一定能表达复杂多模态
action expert	context + noisy/latent action	velocity/noise/action	独立建模 robot-specific action	仍需要 normalizer 与 controller
output transform	model output	dataset action	decode、denorm、slice、axis conversion	不学习环境 dynamics
controller	dataset/env action	actuator command	position/velocity/torque loop、安全限制	不解释任务语言

OpenVLA 没有独立 continuous action head：Llama LM head 预测 tail-vocabulary action tokens，ActionTokenizer 做 scalar ↔ token 映射。[2][3][12] π0 则以 VLM prefix 条件化 300M action expert，expert 输出 H×D flow velocity，再由 solver 得到连续 chunk。[4][6][12]

Policy 是整条链

$$\pi_{\text{exec}} = \mathcal{C} \circ \mathcal{S} \circ \mathcal{U} \circ \mathcal{D} \circ f_\theta \circ \mathcal{P},$$

变量解释：$\mathcal{P}$ 是输入 preprocess/normalize，f_θ 是模型，$\mathcal{D}$ 是 token/flow decode，$\mathcal{U}$ 是 unnormalize，$\mathcal{S}$ 是有效维度与 axis 选择，$\mathcal{C}$ 是 controller。直觉是任何一个函数错，closed-loop policy 都会错；只验证 f_θ 的 loss 不能证明 π_{exec} 正确。openpi Policy.infer 的 transform 顺序直接体现了这一组合。[6][12]

OpenVLA：逐标量 action token 路线

架构与 I/O

OpenVLA flagship 基于 Prismatic 7B：视觉侧融合 DINOv2 与 SigLIP patch features，经 projector 输入 Llama-2 7B；LM head 自回归生成动作 token。[2][3][12]

```
# OpenVLA flagship path at the audited commit.
one primary RGB + task prompt
        ↓
DINOv2/SigLIP features → projector → Llama-2
        ↓
d tail-vocabulary token IDs
        ↓
scalar centers → dataset q01/q99 unnormalize
        ↓
one d-dimensional action
```

注意：生成 d 个 token 表示“一个动作的 d 个维度”，不是 H=d 的未来时间窗口。frozen dataset default 的 `future_action_window_size=0` 与论文把 action chunking 列为未来方向相一致。[2][3][12]

q01/q99 normalization

对真实 action 第 i 维：

$$\tilde{a}_i = \text{clip}\left(2\frac{a_i - q_i^{01}}{q_i^{99} - q_i^{01} + \varepsilon} - 1, -1, 1\right).$$

变量解释：q_i^{01}, q_i^{99} 是当前 dataset/embodiment 的分位统计；$\tilde{a}_i$ 进入 scalar tokenizer。OpenVLA frozen preprocessing 对受 mask 管理的维度显式 clip 到 [-1,1]。直觉是压缩 outlier 影响；代价是超出区间的不同物理动作可能饱和到同一 endpoint。[3][12]

scalar → token

frozen code 构造 256 个均匀 threshold/edge values（论文则抽象为 256 bins）：

$$e_j = -1 + \frac{2j}{255}, \quad j = 0, \ldots, 255, \qquad t_i = V - \text{digitize}(\tilde{a}_i, e).$$

变量解释：V 是 base tokenizer vocabulary size；`np.digitize` 在 clip 后返回 $b_i \in \{1, \ldots, 256\}$，所以 $t_i = V - b_i$ 恰好占用末尾 256 个 token IDs。直觉是把每个 normalized scalar 变成 LLM 可预测的“词”。实现风险是 token offset、special-token collision 和 tokenizer version；不能只看输出 ID 在词表范围内。[3][12]

frozen decoder 使用 255 个相邻 endpoint center：

$$c_j = \frac{e_j + e_{j+1}}{2}, \quad j = 0, \dots, 254, \qquad \hat{\hat{a}}_i = c_{\mathrm{clip}(V - t_i - 1, 0, 254)}.$$

变量解释：论文写 256-way bins、索引 0..255；该 commit 实际用 256 个 thresholds 编码、255 个相邻 centers 解码。编码的 $b = 255$ 与 $b = 256$ 都被 clip 到 center index 254，因此两个最高 token IDs 重建为同一个 0.996078 . . . center。面试中应把 paper abstraction、encoder token count 与 decoder reconstruction cardinality 分层说明。[2][3][12]

masked token CE

$$\mathcal{L}_{\mathrm{OpenVLA}} = -\frac{\sum_k m_k \log p_\theta(y_k \mid y_{<k}, I)}{\sum_k m_k}.$$

变量解释：m_k 只让 action tail 和配置允许的 stop/EOS label 参加 loss；prompt 与插入的 image patch positions 使用 IGNORE_INDEX。连续 L1 是 argmax token decode 后的 metric，不是优化目标，也不等于 online success。[3][12]

quantization 的三类代价

1. **Resolution**：有限 centers 带来不可消除的 reconstruction error。
2. **Saturation**：q01/q99 外被 clip 后失去幅度差异。
3. **Distribution shift**：换 robot/dataset 却沿用旧 stats，会让相同 token 对应错误物理尺度。

π0：连续 action expert 与 Flow Matching

I/O 与架构

π0 paper 定义：

$$o_t = [I_t^1, \dots, I_t^n, \ell_t, q_t], \qquad A_t = [a_t, \dots, a_{t+H-1}] \in \mathbb{R}^{H \times D}.$$

变量解释：论文实验用 2–3 路 RGB、language、joint state，H=50；frozen openpi 默认 D=32、H=50。D 是 model action dimension，不代表每个 robot 都有 32 个真实 actuator dimensions。[4][6][12]

VLM 产生 image/language prefix；π0 的 suffix 含一个 continuous state token 与 H 个 noisy-action tokens。action expert 输出每个 noisy-action token 的 D 维 velocity。采样时 prefix KV cache 只算一次，每个 integration step 重算 suffix。[6][12]

paper convention

$$A^\tau = \tau A + (1-\tau)\epsilon, \qquad u_{\text{paper}} = A - \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

变量解释：$\tau = 0$ 是 noise、$\tau = 1$ 是 data；网络学习沿直线路径的 velocity。训练目标是 $v_\theta(A^\tau, o)$ 与 $A - \epsilon$ 的 MSE，作用张量为 [B,H,D]。[4]

$$\mathcal{L}_{\text{flow}} = \mathbb{E}\left[\frac{1}{HD}\|v_\theta(A^\tau, o) - (A-\epsilon)\|_2^2\right].$$

变量解释：frozen compute_loss 先对 D 求 mean，train.py 再对 H 和 batch 求 mean；公开主路径未显示额外 valid-dimension mask。若某个本地项目 D 大于真实 d，是否对 padding dimensions 监督为零或使用 mask，必须读本地 batch/loss，而不能由 output slicing 反推。[6][12]

code convention

openpi 反向参数化时间：

$$x_t = t\epsilon + (1-t)A, \qquad u_{\text{code}} = \epsilon - A.$$

变量解释：代码从 t=1 noise 以 $dt = -1/N$ 积分到 t=0 data，默认 N=10。令 $\tau = 1 - t$，则 path point 相同，且 $u_{\text{code}} = -u_{\text{paper}}$、$d\tau = -dt$，所以最终 ODE 轨迹等价。[6][12]

π0.5 不等于 PiOFAST

> 2026-07-19 Audit B 独立复核 已直接核对 π0.5 paper 的 FAST-pretrain/flow-posttrain hybrid、frozen `Pi0(pi05=True)` 与 `Pi0FAST` 分叉；FAST Hub normalization、model default 与 `pi0_fast_libero` 的 D=7/H=10 override 也已分层。local OpenPiVLA 仍需 artifact，相关题卡保持 `unseen/WARN`。

π0.5 paper 的完整 recipe 在预训练用 FAST action-token CE，post-training 加连续 action expert，并联合优化 CE + flow loss；推理使用 text token 后接 10 步 flow sampling。[5] frozen openpi 的 `Pi0(pi05=True)` 直接证实 state 进入离散 prompt、timestep 用 adaRMSNorm 注入，但审计的 `compute_loss` 只返回 flow MSE。独立 `Pi0FAST` class 才实现 FAST-token CE。[6][12]

因此：

- π0.5 paper = hybrid training design；
- frozen Pi0(pi05=True) = released flow architecture path；

- PioFAST = independent autoregressive FAST model type。

三者不能只按名称合并。

VLA 架构路线、mask、适配与部署证据

> 本节对应 `VLA-ARCH-007..012`。四方法比较、Octo frozen code 与 openpi serving path 来自 Source Audit F，并已完成 独立事实复核；比较机制，不横比不同 protocol 的 success rate。[19][20][21][27]

VLA-ARCH-007：RT-2、OpenVLA、Octo、π0 不是同一 head 的尺寸变化

直觉：四者都从视觉/任务条件产生机器人动作，但“动作放在哪里、用什么 loss 学、推理要循环什么”完全不同。先说 action representation，再谈 backbone size，能避免只背模型名称。

方法	backbone / condition	action representation	训练主路线	推理循环
RT-2	PaLI-X 5B/55B 或 PaLM-E 12B	pose/gripper 等连续维 uniform 256 bins 后成为 action tokens，另有 termination	预训练 VLM 上混合 web + robot data co-fine-tune	自回归生成 action token；论文部署用 cloud TPU
OpenVLA	DINOv2 + SigLIP → projector → Llama-2 7B	当前动作每个 scalar 占一个 tail-vocab token	OXE robot data 上 full fine-tune；official repo 另支持 LoRA	生成 (d) 个 token → scalar decode → q01/q99 unnorm
Octo	27M/93M block-wise transformer；image tokenizer + frozen T5-base	action readout latent 条件化 diffusion head，公开 pretrain config 为 H=4、D=7	800k OXE trajectories 预训练；新域 head-only/head-MLP-only/full FT	backbone 一次产生 readout；diffusion head 多步去噪 chunk
π0	PaliGemma 2B VLM + 约 300M action expert	$H \times D$ noisy continuous chunk 的 flow velocity	cross-embodiment pretrain + task post-train；openpi 提供 full/LoRA 示例	prefix KV 一次；expert 做多步 Euler/ODE update

RT-2 的论文 tokenization 还随 VLM 而变：PaLI-X 复用整数 token，PaLM-E 覆写 256 个低频 token。OpenVLA 的 frozen implementation 则使用 Llama tail vocab 和特定 255-center decode nuance。它们同属“action as token”范式，但不能交换 tokenizer 细节。[1][2][3][21]

回答收口：RT-2 强调 web knowledge co-fine-tuning 与 action-as-text；OpenVLA 将这条路线开放到 7B/OXE；Octo 用小型 modular transformer/readout/diffusion 做可适配 generalist policy；π0 用大 VLM context + 小 continuous expert 做 flow chunk。项目选型要再看 data、latency、H/K 与 controller，而不是把年份当排序。

VLA-ARCH-008：attention-visible 与 loss-visible 是两件事

直觉：token 在序列里出现，只说明它可能参与计算；是否能看见别的位置由 attention mask 决定，是否直接产生监督由 label/loss mask 决定。

对自回归 action-token 路线：

$$\mathcal{L}_{\mathrm{AR}} = -\frac{\sum_i m_i \log p_\theta(y_i \mid y_{<i}, c)}{\sum_i m_i + \epsilon}.$$

变量与实现：y_i 是 token，c 是图像/任务条件，m_i 是 label mask。OpenVLA prompt labels 与插入的 visual patch labels 是 `IGNORE_INDEX`，action tail 与默认 stop token 才参加 CE；因而“patch 被 action token attend”与“patch 自己有 CE label”完全不同。[2][3][21]

三种典型 layout：

```
# OpenVLA: action values themselves are autoregressive labels.
[image patches / prompt] -> [a_1] [a_2] ... [a_d] [EOS?]

# Octo: readout tokens are latent queries; a separate diffusion loss supervises actions.
[task prefix] [obs(t-1), readout(t-1)] [obs(t), readout(t)] -> H x D

# pi0: prefix conditions a continuous noisy-action suffix under a block mask.
[image + language prefix] [state + noisy action_1 ... noisy action_H] -> velocity
```

代码块说明：Octo readout 只读取 task/observation/自身 readout，不被 observation 反向读取；π0 action suffix 是连续 latent，不是 LM action label。位置改变会影响 positional embedding、causal context、KV cache 与 loss alignment；一次 off-by-one 就可能把 prompt/EOS 当 action，或让 action 看见未来 label。[6][20][21]

Octo 的 timestep/action target mask 是正例：

$$M_{b,w,h,d} = T_{b,w} \wedge A_{b,w,h,d}, \qquad \mathcal{L} = \mathrm{continuous_loss}(\hat{z}, z; M).$$

变量与实现：(T) 排除 history padding，(A) 排除 action padding；(b,w,h,d) 是 batch/window/horizon/action dimension。frozen MSE 与 diffusion head 都使用 conjunction。[20] 相比之下，frozen Pio flow loss 对 configured D 全维 mean；不能泛称“所有 VLA 都自动 mask padding”。[18][21]

VLA-ARCH-009：freeze、full、LoRA 与独立 expert 的选择

直觉：这不是四个互斥模型名，而是“允许哪些参数适应新数据”的容量预算。数据少、domain 接近时先限制更新；camera/objects/dynamics 变化大时才逐步解冻。

方式	主要更新	合适情形	风险
freeze VLM / head-only	projector/readout/action head	新 action space、小数据、视觉语义接近	backbone domain gap 无法纠正
full FT	vision/projector/LLM/action modules	数据足、视觉与控制都明显变化	显存高、遗忘、过拟合
LoRA / adapter	selected linear layers 的低秩增量	需要更多容量但显存有限	rank/target modules 与 base revision 约束能力

方式	主要更新	合适情形	风险
independent action expert	大 VLM 提供 context，小 expert 专门建模 action	continuous chunk、需要独立 target/width	context/expert interface 与联合训练更复杂

LoRA 可写为：

$$W' = W + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times i},\ B \in \mathbb{R}^{o \times r}, \quad r \ll \min(i, o).$$

变量与实现：(W) 是 frozen base weight，(r) 是 rank。OpenVLA official finetune 使用 PEFT `all-linear` LoRA；openpi LIBERO low-memory config 同时选择 PaliGemma/action-expert LoRA variant 和匹配 freeze filter；Octo 则明确提供 `head_only`、`head_mlp_only`、`full`。[2][7][20][21]

证据边界：OpenVLA flagship 论文结果来自 full fine-tuning；repo 支持 LoRA 不代表主结果或本地 checkpoint 使用 LoRA。项目回答必须补 trainable-parameter log、base/adapter revisions、rank、target modules、merge/load order。

VLA-ARCH-010：异构动作要统一 schema，也要统一监督语义

令 embodiment e 的真实动作宽度为 d_e，模型宽度为 $D \geq d_e$，P_e 把真实维写进统一空间，$S_e = P_e^\top$ 取回：

$$\bar{a}^{(e)} = P_e a^{(e)}, \qquad \hat{a}^{(e)} = S_e \hat{\bar{a}}, \qquad m_j^{(e)} = \mathbf{1}[j \in \mathrm{Im}(P_e)].$$

变量与实现：P_e 只定义维度位置，不会自动统一 absolute/delta、frame、unit、rotation 或 gripper polarity。openpi `PadStatesAndActions` 只做 zero-pad；Octo `action_pad_mask` 则是 loss 中的显式有效维正例。[7][20][21]

masked objective：

$$\mathcal{L}_{\text{valid}} = \frac{\sum_{b,h,j} m_{b,h,j}\, \ell\left(\hat{\bar{a}}_{b,h,j}, \bar{a}_{b,h,j}\right)}{\sum_{b,h,j} m_{b,h,j} + \varepsilon}.$$

变量与实现：m 应覆盖 timestep 与 action dimension；若直接对 D 维 mean，padding zeros 也会产生“逼近 0”的监督，并把有效维梯度按 d_e/D 稀释或改变。推理 slice 并不能修复训练 target 已被污染的问题。[18][20][21]

跨机器人至少逐组记录：translation axis/frame/unit/type，rotation representation/order/unit，gripper range/polarity，base semantics，control frequency/dt，以及 modality/timestep/action

masks。只写“统一成 32D”不是 action unification 设计。

VLA-ARCH-011：延迟是 pipeline 向量，不是一个数字

远程 chunk query 的端到端时间可拆为：

$$T_{\text{e2e}} = T_{\text{capture}} + T_{\text{pre}} + T_{\uparrow} + T_{\text{vision/prefill}} + NT_{\text{step}} + T_{\text{post}} + T_{\downarrow} + T_{\text{queue}}.$$

变量与实现：token policy 的 (N) 是 action token length；FAST 是可变压缩序列长度；diffusion/flow 是 denoise/NFE steps。是否缓存 vision/prefix、每 step 跑大 backbone 还是小 expert，必须看源码。[12][21]

- RT-2 论文在其 multi-TPU cloud protocol 下报告 55B 为 1–3 Hz、5B 约 5 Hz；这包含历史网络/服务条件，不能外推本地 GPU。[1]
- FAST 论文的指定 4090 测量中，π0 flow 约 100 ms/chunk、π0-FAST 约 750 ms/chunk；Source Audit B 将差异映射到 10 次小 expert 与 30–60 次大 VLM token decode。[7][12]
- openpi `Policy.infer` 的 `policy_timing.infer_ms` 包围 model sampling；server `infer_ms` 包围完整 policy infer，`prev_total_ms` 才进一步覆盖上一轮 recv/unpack/send。它们仍未拆出 vision、每步 sampler 和 queue。[6][21]

测量时报告 warm-up、device sync、batch/dtype、payload bytes、H/K、NFE/token length、cache、p50/p95；在异步 GPU 上直接用未同步 wall clock 会低估 kernel 时间。

VLA-ARCH-012：从 paper 一直核到 runtime trace

直觉：模型名是线索，不是合同。真实 I/O 需要四层证据闭环：

```
# Each layer answers a different question; none may be skipped.
paper/version
  -> repo commit + license + model/loss/infer path
  -> checkpoint manifest + config + tokenizer + norm assets
  -> runtime request/response + transforms + queue + controller trace
```

代码块说明：paper 支持方法定义；repo 支持该 commit 的实现；checkpoint 决定实际权重/资产；runtime 才证明线上收发、denorm、轴序和执行。

openpi `create_trained_policy` 展示了正确的资产绑定：从 checkpoint 加载 model weight，norm stats 默认取 checkpoint `assets/<asset_id>`；input 依次做 repack/data transform、normalize、model transform；output 依次做 model transform、unnormalize、data/repack transform。[6][21] OpenVLA 的 `unnorm_key` 同样决定 action dim 和 q01/q99；多 dataset stats 不显式选 key 会拒绝推理。[2][3]

最小 manifest 应有：paper/repo revision、model class、weight hash/shape、base+adapter、tokenizer/processor revision、image views/resize、state/action schema、H/D/K、norm

asset/hash、sampler/NFE、output transform、controller mode/frequency、eval checkpoint/task/trial/seed。对 OpenPiVLA，这些仍是待取证项，不能由上游 Source Audit F 自动补齐。

Action representation 总览

五条路线各自建模什么

路线	抽象分布/目标	优点倾向	主要风险	本批证据深度
deterministic continuous regression	$\hat{A} = f_\theta(o)$，MSE/Huber	简单、低 decode latency	多模态 label 下均值化	通用机制
scalar discrete token	$p(y_{1:d} \mid o)$	复用 LM next-token infra	量化、offset、AR latency	OpenVLA 深入
CVAE action chunk	$p(A \mid o) = \int p(A \mid o, z)p(z \mid o)dz$	latent 表达 sequence variation	posterior collapse、train/infer latent gap	ACT paper/code 深入
diffusion action	conditional denoising / score	适合多峰、高维 continuous action	多步 sampling、scheduler contract	Diffusion Policy paper/code 深入
flow action	conditional velocity field + ODE	连续多模态、solver 明确	time convention、integration latency	π0 特化深入

表中“优点倾向”不是跨 benchmark 排名。数据、网络规模、H、controller frequency 和 implementation 都会改变结果。

deterministic regression

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_{(o,A)\sim\mathcal{D}} \left[\| f_\theta(o) - A \|_2^2 \right].$$

变量解释：若同一个 o 在数据中对应左右绕行等多个可行动作，MSE 的条件最优解倾向于条件均值，可能生成物理上不属于任何 mode 的动作。若任务近似单峰、action 定义稳定且延迟严格，direct regression 仍是重要 baseline。

CVAE 定义级边界

$$\mathcal{L}_{\text{CVAE}} = \mathbb{E}_{q_\phi(z|o,A)} \left[-\log p_\theta(A \mid o, z) \right] + \beta D_{\text{KL}} \left(q_\phi(z \mid o, A) \,\|\, p(z \mid o) \right).$$

变量解释：z 是 sequence/style latent；训练 encoder 可看见 action target，推理只能从 prior 采样或取固定 latent。冻结 ACT path 具体采用 `[CLS]+qpos+actions` posterior、learned action queries，并在推理固定 $z = 0$；详见后续 ACT 小节。[9][14]

diffusion 定义级边界

Diffusion Policy 将 visuomotor policy 表示为 conditional denoising diffusion process，并强调 multimodal action distribution、visual conditioning 与 receding-horizon control。[10] Batch 2A 后续小节展开 frozen epsilon/sample target、global condition、DDPM/DDIM 与 $T_o/T_p/T_a$。

flow 与 diffusion 不只是“步数不同”

- diffusion 常学习 noise、score 或 data parameterization，并依赖 noise schedule 与 stochastic/deterministic sampler；
- flow matching 学给定 probability path 的 velocity field，通过 ODE solver transport noise 到 data；
- 两者都可迭代生成 continuous chunk，但训练 target、time convention 与 solver 不同。

本页后续 Flow Matching 小节从 Source Audit E 回填一般 CNF/CFM、OT path、ODE 与 π0 特化；仍不把 current supporting package 冒充 2022 论文历史实现。

Normalization / denormalization

为什么它是 0-success 高风险接口

神经网络只在 normalized space 学习；controller 在 physical units 执行。若训练与推理的 stats key、axis order 或 action type 不一致，模型可在 normalized space 得到很低 loss，却输出错误量级或错误轴的物理动作。

openpi 的 quantile mapping 为：

$$\tilde{a}_i = 2\frac{a_i - q_i^{01}}{q_i^{99} - q_i^{01} + \varepsilon} - 1, \qquad a_i = \frac{\tilde{a}_i + 1}{2}\left(q_i^{99} - q_i^{01} + \varepsilon\right) + q_i^{01}.$$

变量解释：该 frozen transform 不显式 clip；因此 q01/q99 外值可落在 [-1,1] 之外。Policy factory 从 checkpoint assets 加载训练期 norm stats，输入 sample 前 normalize，output transform 后 unnormalize。[6][12]

OpenVLA 与 openpi 的差别：

维度	OpenVLA audited path	openpi audited path
stats	dataset-specific q01/q99	checkpoint asset norm stats
quantile mapping	对受管维度 clip [-1,1]	quantile transform 无显式 clip
tokenization	normalize 后 scalar bin	flow 直接用 continuous normalized；FAST 再 DCT/BPE
inverse	token center 后 q01/q99 inverse	model output transform 后 inverse

维度	OpenVLA audited path	openpi audited path
extra padded dims	dataset-specific	stats 未覆盖的尾维保持原值，再 output slice

最小 inverse-consistency test

$$\max_{a \in \mathcal{T}} \|\text{unnorm}(\text{norm}(a)) - a\|_\infty < \delta.$$

变量解释：$\mathcal{T}$ 至少包含训练分位内样本、q01/q99 endpoint、outlier、constant dimension 与 padding dimension；δ 按物理单位设定。对于有 clip 的 pipeline，outlier 不应期待严格可逆，而应显式测试 saturation behavior。

常见穿帮说法

- 错：“归一化只是数值稳定性，不影响策略语义。”
- 对：“归一化 stats 与 axis order 共同定义 token/continuous output 的物理意义，是 checkpoint contract。”
- 错：“FAST 会自动把任何机器人 action 处理到统一尺度。”
- 对：“FAST Hub processor 从 DCT 开始；调用者先做 dataset/embodiment normalization。”

FAST：频域 action token

FAST 的 quantile normalization、DCT/round/BPE 顺序、paper vocab=1024 与 Hub vocab=2048、以及 processor 不自动做 normalization，已在 Audit B 独立复核 直接核对；这里的上游默认值不等于本地 tokenizer revision。

它解决什么

逐 timestep、逐 scalar tokenization 会让高频 action window 产生大量相邻高度相关 token；autoregressive model 可能靠复制最近 token 获得低 loss，而没有学到整段动作形状。FAST 先把时间序列变到频域，让低频整体趋势先出现，再用 BPE 压缩重复系数。[7][8]

算法

对 normalized action chunk $\tilde{A} \in \mathbb{R}^{H \times D}$，每维沿时间轴做 DCT：

$$C_j^i = \text{DCT}(\tilde{a}_{1:H}^i), \qquad \bar{C}_j^i = \text{round}(\gamma C_j^i).$$

变量解释：i 是 action dimension，j 是 frequency index，γ 控制量化精度/压缩率。FAST paper 单数据集实验使用 scale 10；frozen FAST+ Hub processor 也使用 scale 10。[7][8][12]

按 frequency-first 展开并做 BPE：

$$z = [\bar{C}_0^1, \ldots, \bar{C}_0^D, \bar{C}_1^1, \ldots, \bar{C}_1^D, \ldots], \qquad y_{1:L} = \mathrm{BPE}_\Phi(z).$$

变量解释：先排所有维度的低频 coefficient，再进入高频；L 是可变 token length；Φ 是唯一 learned 的 BPE vocabulary。frozen Hub code 对 [H,D] 的 DCT array 做 row-major flatten，正好得到 frequency-first 顺序。[7][8][12]

inverse decode：

$$\hat{\tilde{A}} = \mathrm{IDCT}\left(\frac{\mathrm{unflatten}\big(\mathrm{BPE}^{-1}(y;\Phi); H, D\big)}{\gamma} \right).$$

变量解释：decode 恢复 normalized H×D chunk，之后仍需 checkpoint-matched unnormalize。Hub decoder 需要 H 与 D；reshape 失败时 frozen code 会返回 zero coefficients，因此部署应增加显式 error/telemetry，而不是静默执行零动作。[8][12]

paper / artifact / model 的默认值要分开

对象	scale	vocab	H / D
FAST paper 单数据集实验	10	1024	1-second chunk，依 control frequency
frozen FAST+ Hub	10	2048	decode 时传入或缓存
openpi PioFASTConfig default	external	external	H=32, D=32
openpi LIBERO example config	external	external	H=10, D=7

不能用最后一行证明本地 OpenPiVLA H=10。

compression 为什么不等于控制误差更小

至少分开五个量：

1. token count / compression ratio；
2. normalized reconstruction error；
3. next-token training loss；
4. physical-unit trajectory error；
5. closed-loop task success。

BPE 减少重复 zero tokens、缩短 AR sequence，论文消融在其任务上提升 policy performance；但更短 token sequence 不在数学上保证更小 reconstruction error，更不保证跨 controller 的 success 提升。[7]

ACT：CVAE、Action Chunk 与 Temporal Ensemble

> 本节对应 `VLA-ACT-011..017`。直接证据来自 ACT arXiv v1 与 `tonyzhaozh/act@742c753...`；它是 continuous action-chunk CVAE，不是 action-token VLA，也不能证明个人 OpenPiVLA 使用 ACT head。[9][14]

VLA-ACT-011：H、K、R 与 control period

先把四个时间量拆开：

量	记号	回答什么	frozen ACT
prediction horizon / chunk size	H	一次 forward 输出几步	`num_queries`
execution window	K	一个预测 chunk 实际连续执行几步	无独立 CLI field
query / replan interval	R	相邻两次 policy inference 间隔几步	`query_frequency`
control period	Δt	每个环境 action 的物理周期	sim `DT=0.02`

若 episode 有 T 个 control steps，无 temporal aggregation 且 $K = R = H$，policy query 数近似：

$$N_{\text{query}} \approx \left\lceil \frac{T}{H} \right\rceil .$$

变量与实现：H 是 model tensor 的时间维；K/R 属于 rollout。冻结 ACT 的无 aggregation 分支每 H 步 query 一次并消费整段；开启 aggregation 后 $R = K = 1$，但 model H 不变。[14]

概念上 K 与 R 仍可不同：异步 controller 可先提交较长 queue，却在 queue 未耗尽前生成新计划。不能只看到 `chunk_size=100` 就说“100 步后才闭环”。

VLA-ACT-012：CVAE-Transformer 如何并行输出 chunk

训练期 posterior encoder 与 decoder 的数据流：

```
# The posterior sees proprioception and the supervised action chunk.
[CLS] + projected qpos + projected actions[0:H]
                  |
        Transformer encoder
                  |
           mu, logvar [B,32]
                  |
        reparameterized z [B,32]
```

```
# The decoder consumes current observation and H learned query slots.
shared multi-camera ResNet + qpos token + z token
                    |
          Transformer encoder/decoder
                    |
          H learned action queries
                    |
             actions [B,H,14]
```

代码映射：posterior 不看图像，使用 `[CLS]+qpos+actions`；相机共享 ResNet18，feature maps 沿 spatial width 拼接；decoder 的 `nn.Embedding(num_queries, hidden_dim)` 是 H 个 learned output slots，一次 forward 并行得到：

$$\mathrm{ACT}_\theta(o_t, z) = \left(\hat{a}_t^{(t)}, \ldots, \hat{a}_{t+H-1}^{(t)}\right) \in \mathbb{R}^{H\times 14}.$$

变量说明：上标 (t) 是 query time，下标是目标物理时刻；14D 是该双臂 setup 的公开 action shape，不是所有机器人通用维数。论文描述 fixed/sinusoidal queries，但 frozen code 实际使用 learned `nn.Embedding`，回答实现时必须保留此差异。[9][14]

VLA-ACT-013：style latent 的 train / infer gap

训练 posterior 为：

$$q_\phi(z \mid q_t, A_t) = \mathcal{N}(\mu_\phi, \mathrm{diag}(\exp \ell_\phi)), \quad z = \mu_\phi + \exp(\tfrac{1}{2}\ell_\phi) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

变量与实现：$A_t \in \mathbb{R}^{H\times 14}$，$z \in \mathbb{R}^{32}$，$\ell_\phi$ 是 `logvar`。target chunk 帮助 posterior 压缩 demonstration style，reparameterization 让 reconstruction gradient 通过 sampling。[14]

推理时 frozen code 不运行 posterior，也不从 prior 随机采样，而是：

$$z_{\text{test}} = 0 = \mathbb{E}_{\mathcal{N}(0,I)}[z].$$

这使 evaluation deterministic。它的边界是：训练期 latent 能表达多样性，不等于部署会随机探索多个 mode。若面试回答“ACT 推理采多个 z 再挑最好动作”，与该 commit 不符。[9][14]

KL 与 reconstruction 的总目标为：

$$\mathcal{L}_{\mathrm{ACT}} = \mathcal{L}_{L1}^{\text{code}} + \beta \frac{1}{B} \sum_b \frac{1}{2} \sum_{j=1}^{32} \left(\mu_{b,j}^2 + \exp \ell_{b,j} - 1 - \ell_{b,j}\right).$$

变量与实现：β 是 `kl_weight`；公开 recipe 使用 10，但不是跨任务常数。论文 Algorithm 1 写 MSE，方法正文与 frozen `policy.py` 用 L1；安全口径必须说“paper algorithm 存在不一致，正文/代码是 L1 + KL”。[9][14]

VLA-ACT-014：temporal ensemble 融合相同 target time

query time q 会预测 $q, \dots, q+H-1$。当前 target time t 可由这些历史 query 覆盖：

$$\mathcal{Q}_t = \{q \mid \max(0, t-H+1) \le q \le t\}.$$

变量与实现：每个 $q \in \mathcal{Q}_t$ 都产生对**同一个** a_t 的估计 $\hat{a}_t^{(q)}$；代码把 chunk 写入二维 `all_time_actions[q,q:q+H]` 后读取 column t。这不是把 a_{t-1}, a_t, a_{t+1} 做普通时域平滑。[14]

按 oldest-to-newest 排列 $q_0 < \cdots < q_{n-1}$：

$$w_i = \exp(-mi), \qquad a_t^{\text{exec}} = \frac{\sum_i w_i \hat{a}_t^{(q_i)}}{\sum_i w_i}.$$

变量与代码映射：$m = 0.01$ 是 frozen decay；`np.arange` 从 oldest row 开始，因此 w_0 最大，**最旧预测权重最高**。快速变化环境中这也可能保留 stale plan；“新预测权重最大”是对此实现的错误复述。[9][14]

VLA-ACT-015：chunking、replan 与 ensemble 如何共同决定闭环

frozen rollout 有两种极端：

模式	model H	query interval R	execute K	新 observation 如何进入
`temporal_agg=False`	不变	H	H	chunk 边界
`temporal_agg=True`	不变	1	1	每个 control step

因此 chunking 是**预测表示**，replanning 是**反馈频率**，ensemble 是**重叠预测融合**。一次输出 H 步不等于必须 open-loop 执行 H 步；反过来，每步 query 也不意味着 model horizon 变成 1。[14]

```
def act_closed_loop_step(obs, t, policy, prediction_table, temporal_agg):
    # Query every step with aggregation, otherwise only at chunk boundaries.
    if temporal_agg or t % policy.horizon == 0:
        chunk = policy(obs, z=zeros())  # Frozen inference uses z=0.

    if temporal_agg:
        # Store each query's predictions under their physical target times.
        add_overlapping_chunk(prediction_table, query_time=t, chunk=chunk)

        # Fuse predictions that all target the current physical time t.
        normalized_action = ensemble_same_target_time(prediction_table[t])
    else:
        # Execute one cached slot; fresh observation changes the next chunk only.
```

```
        normalized_action = chunk[t % policy.horizon]

    # Denormalize to absolute joint-position targets before env.step.
    return denormalize(normalized_action)
```

伪代码映射 frozen rollout 的两条 path；真实项目要另报异步 queue、safety clip、action semantics 与 K/R 是否解耦。

VLA-ACT-016：chunk 太短或太长

H	主要收益	主要代价
太短 / $H=1$	反馈及时、单次输出小	query 多、长程 coherence 弱、易受局部噪声影响
中等	sequence coherence 与 responsiveness 折中	需要按任务/控制频率调
太长	policy calls 少、覆盖长动作片段	stale tail、长序列建模难、query slots 计算/显存增加

若无 aggregation，最坏 observation staleness 约为：

$$\Delta t_{\text{stale,max}} \approx (H-1)\Delta t.$$

变量与直觉：Δt 是 control period；扰动刚发生在 chunk 首步后，最迟到下一 chunk 才进入 policy。ACT 论文的特定 ablation 在 $H=100$ 附近最好、200/400 略降，只能说明其协议中的折中，不能背成通用最优 horizon。[9][14]

开启 temporal ensemble 可把 feedback interval 降到 1，却把 inference 从约 T/H 次提高到 T 次，并引入 overlapping buffer 与 stale-weight 风险。选 H 时必须同时报 $K, R, \Delta t$ 和 policy latency。

VLA-ACT-017：ACT 与 token、Diffusion / Flow 的归纳偏置

路线	训练 target	生成计算	多模态/随机性	部署重点
ACT	continuous chunk L1 + CVAE KL	一次 parallel query decode	train latent；frozen test $z=0$ deterministic	H/K/R、temporal ensemble
OpenVLA	scalar token CE	AR 生成 d 个 token	categorical decoding	bin/offset/单步
FAST	frequency/BPE token CE	AR variable-length tokens	token decoding	tokenizer H/D/revision
Diffusion Policy	epsilon/sample MSE	K_d scheduler steps	Gaussian trajectory init	$T_o/T_p/T_a/K_d$
π0 Flow	velocity MSE	N_f ODE/Euler steps	Gaussian action init	time/sign/NFE/H/K

ACT 的强归纳偏置是“用 H 个 query 一次给出连续 chunk，并可对重叠 chunk 融合”；它没有量化/AR token latency，也没有 diffusion/flow 的多步 sampler，但 frozen test latent 固定为零，部署

多模态性弱于随机生成路线。方法选择必须用同协议的 latency、physical error 与 closed-loop success，而不是按“生成式”标签排序。[14][15][16]

Diffusion Policy：条件动作扩散与 Receding Horizon

> 本节对应 `VLA-ACT-018..022`。直接证据来自 Diffusion Policy arXiv v5、`real-stanford/diffusion_policy@5ba07ac...` 与其固定的 `diffusers=0.11.1`；warm-start、loss denominator、boundary padding 与 inference-step 数已独立复核，local runtime 仍 WARN。[10][15][26]

VLA-ACT-018：为什么适合多模态连续动作

Diffusion Policy 直接建模整个 continuous trajectory 的条件分布：

$$p_\theta(A_t \mid O_t), \qquad A_t \in \mathbb{R}^{T_p \times D_a}, \quad O_t = (o_{t-T_o+1}, \ldots, o_t).$$

变量与直觉：T_o 是 observation horizon，T_p 是 prediction horizon，D_a 是 action dimension。随机 Gaussian trajectory 经过 conditional denoising 得到一条连贯 action sequence；不同 initial noise 可落到不同动作 mode。[10][15]

它不是每个 timestep 独立采样：

$$p_\theta(A_t \mid O_t) \neq \prod_{h=0}^{T_p-1} p_\theta(a_{t+h} \mid O_t).$$

实现映射：UNet/Transformer 都以 `[B,T_p,D_a]` 为整体对象，temporal network 在同一次 denoising 过程中耦合相邻 action。多模态性来自 sequence-level stochastic generation，不等于线上必须保持随机；固定 generator 或 deterministic DDIM 会改变部署行为。[15]

VLA-ACT-019：conditional DDPM 的 noisy action、target 与 condition

clean trajectory A^0 在随机 diffusion timestep k 加噪：

$$A^k = \sqrt{\bar{\alpha}_k} A^0 + \sqrt{1-\bar{\alpha}_k}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

变量与实现：$\bar{\alpha}_k$ 来自 scheduler 的 beta/noise schedule；frozen policy 用 `noise_scheduler.add_noise(trajectory, noise, timesteps)` 构造 `noisy_trajectory`，shape 不

变。[10][15]

默认 epsilon objective：

$$\mathcal{L}_\epsilon = \mathbb{E}\left[\left\|\epsilon - \epsilon_\theta(O_t, A^k, k)\right\|_2^2\right].$$

变量与实现：condition O_t 在 default image-hybrid path 经 vision encoder 展平成 `global_cond`；trajectory 只含 action channels。frozen policy 也支持 `prediction_type="sample"` 时 target 为 clean A^0，但 policy code 没实现 `v_prediction` target，即使底层 diffusers scheduler 支持该 branch。[15]

VLA-ACT-020：schedule、prediction type 与 horizon 是三组合同

frozen policy target 为：

$$y = \begin{cases} \epsilon, & \texttt{prediction_type=epsilon}, \\ A^0, & \texttt{prediction_type=sample}. \end{cases}$$

变量与边界：prediction type 决定 model output 的监督语义；scheduler 决定如何从 A^0 构造 A^k 以及如何解释 model output 做 reverse step。只改 config 字符串而不改 target builder，会训练错误 objective。[15]

必须同时报告五条时间轴：

时间量	field	作用
observation horizon	`n_obs_steps = T_o`	condition 用几帧
prediction horizon	`horizon = T_p`	diffusion trajectory 长度
action slice	`n_action_steps = T_a`	policy 返回几步
replan interval	wrapper / `steps_per_inference`	多久重新观测/query
diffusion iterations	`num_inference_steps = K_d`	一次 query 内去噪几次

noise schedule 控制不同 k 的 signal-to-noise；T_p 控制 sequence length；K_d 控制 reverse approximation compute。三者互相关联但不能互换。frozen boundary padding 重复 episode 首尾 target，且 policy 拒绝 `valid_mask`，重复值会作为普通 target 进入 MSE。[15]

VLA-ACT-021：receding-horizon 的 action slice 与执行

论文定义：

$$O_t = (o_{t-T_o+1}, \ldots, o_t), \qquad A_t = (a_t, \ldots, a_{t+T_p-1}).$$

policy 生成 T_p 步后，从最后一帧 observation 对齐：

$$\hat{A}_t^{\text{return}} = \text{Unnormalize}\left(X^{(0)}_{:\, T_o-1:T_o-1+T_a,\, :D_a}\right).$$

变量与实现：slice start 是 `T_o-1`，T_a 是返回 action 数。simulation 的 `MultiStepWrapper` 通常顺序执行整段 T_a 后再给新 observation；real script 可一次提交更长 timestamped queue，却由 `steps_per_inference` 更早触发新 query。提交长度不等于重规划间隔。[15]

```
def diffusion_receding_horizon(obs_history, policy, env):
    # Encode exactly To recent observations as the diffusion condition.
    condition = policy.encode_observations(obs_history[:policy.To])

    # One query starts from a fresh Gaussian action trajectory [Tp, Da].
    trajectory = randn([policy.Tp, policy.action_dim])
    for k in policy.scheduler.timesteps:
        # Predict epsilon or clean sample according to the checkpoint contract.
        model_output = policy.model(trajectory, k, global_cond=condition)
        trajectory = policy.scheduler.step(model_output, k, trajectory).prev_sample

    # Return Ta actions aligned with the most recent observation.
    actions = unnormalize(trajectory[policy.To - 1:policy.To - 1 + policy.Ta])

    # The rollout wrapper decides how many returned actions execute before replan.
    return env.step(actions)
```

伪代码表达 frozen global-conditioning/simulation 主链；real robot 的 timestamp queue 与 query schedule 需要单独建模，不能由 `T_a` 推断。

VLA-ACT-022：去噪步数、sampler 与 warm-start

一次 inference 可写成：

$$X^{(K_d)} \sim \mathcal{N}(0, I), \qquad X^{(k-1)} = \text{SchedulerStep}\left(X^{(k)}, f_\theta(X^{(k)}, k, O_t)\right).$$

变量与实现：K_d 是 denoising iterations；DDPM 通常保留 stochastic reverse transition，DDIM 可在抽稀 time grid 上做更确定的更新。减少 K_d 会降低 query compute，但改变 reverse approximation，不保证 success 单调不变。[10][15]

版本边界必须原样保留：

- paper main text 的 real-world 例子写 100 train / 10 DDIM eval；
- paper v5 real table rows 出现 16 eval；

- frozen real configs 写 8；
- `eval_real_robot.py` runtime 又强制 16。

paper 称 receding-horizon 可 warm-start 下一次 inference；frozen major policy classes 每次从 `torch.randn` 重启，并拒绝 `past_action`。所以只能说“paper 提供设计选项，当前审计 commit 未实现”。如果自行实现 warm-start，要对齐已执行 prefix、新 observation、trajectory shift 和 scheduler noise level，并用 latency/stability ablation 证明收益。[15]

Flow Matching：CFM、OT Path、ODE 与 π0

本节对应 `VLA-ACT-023..030`。基础定义来自 Flow Matching arXiv v2；现代 `facebookresearch/flow_matching@11568d3...` 只作 path/solver API 补证，不冒充 2022 历史实现；π0/openpi 特化沿用 Source Audit B/E 的 frozen 版本。[16][17][18] 2026-07-19 的 独立事实复核 已直接核对 Theorem 2、paper `sigma_min` endpoint、time/sign/dt、Beta 换元与 10-step oracle；项目 runtime 与口述仍 `WARN/unseen`。

VLA-ACT-023：probability path 与 velocity field

Continuous Normalizing Flow（CNF）用 vector field 定义 sample trajectory：

$$\frac{d}{dt}\phi_t(x) = v_t(\phi_t(x)), \qquad \phi_0(x) = x, \qquad p_t = [\phi_t]_* p_0.$$

变量与直觉：p_0 是 source/noise distribution，p_t 是中间 probability path，$\phi_t(x)$ 是一个 sample 的 ODE trajectory，v_t 告诉当前位置下一瞬间往哪里走。path 是分布随时间的变化，不是一条具体 action curve。[17]

Flow Matching 让 network v_θ 拟合生成 path 的 target field u_t：

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x\sim p_t}\left[\|v_\theta(x,t) - u_t(x)\|_2^2\right].$$

变量与实现：直接 marginal u_t 通常难算；Conditional Flow Matching（CFM）对每个 data endpoint x_1 采 conditional path $p_t(x \mid x_1)$ 与可计算的 $u_t(x \mid x_1)$。论文 Theorem 2 在 $p_t(x) > 0$ 的假设下证明两项目标相差与 θ 无关的常数，因此 gradients identical；这不等于逐 sample target 或 loss 数值相同。[16][17]

“simulation-free training”表示训练可直接采 path point/target，不需先积分 ODE；inference 仍需 solver。

VLA-ACT-024：线性 OT path 与 target velocity

基础论文的 Gaussian conditional OT path 是：

$$x_t = [1 - (1 - \sigma_{\min})t]x_0 + tx_1, \qquad \dot{x}_t = x_1 - (1 - \sigma_{\min})x_0.$$

变量与边界：$x_0 \sim \mathcal{N}(0, I)$，$x_1 \sim q$；当 $t = 1$ 时仍有 $\sigma_{\min} x_0$，所以 paper endpoint 是 data point 周围的小 Gaussian。不能说原论文所有 OT 公式都令 $\sigma_{\min} = 0$。[16][17]

现代 package 的 `CondOTProbPath` 使用 exact endpoint，即 $\sigma_{\min} = 0$ 特例：

$$x_t = (1 - t)x_0 + tx_1, \qquad u_t = x_1 - x_0.$$

变量与实现：`CondOTScheduler` 返回 $\alpha_t = t, \sigma_t = 1 - t, \dot{\alpha}_t = 1, \dot{\sigma}_t = -1$；`AffineProbPath.sample` 产生 `x_t` 与 `dx_t`。continuous example 手写 velocity MSE：

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x_0,x_1}\left[\|v_\theta(x_t, t) - (x_1 - x_0)\|_2^2\right].$$

公式映射：x_1 在 VLA 中可替换成 normalized $H \times D$ action chunk，x_0 是同形状 Gaussian。conditional target 对一对 endpoints 是常量，不等于 marginal field 在全空间是常量。[16][18]

VLA-ACT-025：ODE solver 如何从 noise 得到 action chunk

给定 learned velocity，Euler update 为：

$$x_{k+1} = x_k + \Delta t\, v_\theta(x_k, t_k, c), \qquad t_{k+1} = t_k + \Delta t.$$

变量与实现：c 是 observation condition；$x_k \in \mathbb{R}^{B \times H \times D}$ 是当前 noisy action state。现代 package 的 `ODESolver` 可调用 Euler、midpoint、Heun 或 adaptive `dopri5`；fixed step 要报 grid/step size，adaptive solver 要报 tolerance 与实际 NFE。[18]

openpi 的 code convention 从 $t = 1$ noise 以 $\Delta t = -1/N$ 走到 $t = 0$ data，默认 $N = 10$：

$$x_{k+1} = x_k - \frac{1}{N} v_\theta^{\text{code}}(x_k, t_k, o).$$

变量与边界：10 是一次 query 内的 flow integration steps，不是 action horizon H、执行前缀 K 或 control steps。prefix image/language KV cache 只计算一次，每个 Euler step 重算 state/noisy-

action/time suffix。[6][16]

VLA-ACT-026：Diffusion epsilon 与 Flow velocity

维度	Frozen Diffusion Policy	π0 / openpi Flow
time	discrete scheduler index k	continuous t/τ
noisy sample	scheduler `add_noise`	explicit affine interpolation
default target	epsilon	path velocity
inference	DDPM/DDIM scheduler step	ODE/Euler step
compute count	K_d denoising iterations	N_f field evaluations
horizon	$T_p/T_a/R$	$H/K/R$

两项常见 loss：

$$\mathcal{L}_\epsilon = \mathbb{E}\|\epsilon - \epsilon_\theta(O, A^k, k)\|_2^2, \qquad \mathcal{L}_v = \mathbb{E}\|u_t - v_\theta(o, x_t, t)\|_2^2.$$

变量与直觉：两者都可从 Gaussian trajectory 开始、都可能使用 MSE，也都能建模 continuous multimodal chunks；但 target、time parameterization 和 sampler contract 不同。Flow Matching 论文还允许 diffusion probability paths，所以“完全无关”与“只是少几步 diffusion”都错误。[15][16][17]

采样步数不是定义：Diffusion 可用少步 DDIM，Flow 也可用很多 ODE steps。延迟必须按 model size、NFE、vision/prefix、transport 与 hardware 实测。

VLA-ACT-027：π0 action expert 的 condition 与 noisy action

π0 将 data endpoint 具体化为 normalized action chunk：

$$A^\tau = \tau A + (1-\tau)\epsilon, \qquad u_{\text{paper}} = A - \epsilon, \quad A, \epsilon \in \mathbb{R}^{B\times H\times D}.$$

变量与架构：A 是 target chunk，ϵ 是 Gaussian source；图像与语言进入 VLM prefix，state、noisy action 与 flow time 进入 action-expert suffix，`action_out_proj` 输出同形状 velocity。[4][6][16]

frozen openpi 使用反向坐标：

$$x_t = t\epsilon + (1-t)A, \qquad u_{\text{code}} = \epsilon - A, \qquad t = 1 - \tau.$$

变量与等价性：$d\tau = -dt$，同时 $u_{\text{code}} = -u_{\text{paper}}$，所以 $u_{\text{code}}dt = u_{\text{paper}}d\tau$。只说时间反向而漏掉 velocity sign，不足以证明 trajectory 等价。[16]

π0 的 timestep 也不是 uniform：若 $y \sim \text{Beta}(1.5, 1)$、$\tau = 0.999(1 - y)$，则 openpi time：

$$t = 1 - \tau = 0.001 + 0.999y.$$

这直接映射 code 的 `Beta(1.5,1)*0.999+0.001`，强调 high-noise training region；它是 recipe weighting，不是 Flow Matching 的普遍定义。[4][6][16]

VLA-ACT-028：valid dimensions、padding 与 loss mask

若真实 action dimension 是 d，model capacity 是 $D \geq d$，理想 valid-mask velocity loss 为：

$$\mathcal{L}_{\text{masked}} = \frac{\sum_{b,h,j} m_{b,h,j}(v_{b,h,j} - u_{b,h,j})^2}{\sum_{b,h,j} m_{b,h,j} + \varepsilon_0}.$$

变量与实现要求：m 同时表达有效 horizon slot 与有效 action axis；它必须来自 dataset/action schema，不能由 inference output slicing 反推。[16]

frozen Pio 实际先对 configured D 直接 mean：

$$\ell_{b,h} = \frac{1}{D}\sum_{j=1}^{D}(v_{b,h,j} - u_{b,h,j})^2,$$

训练脚本再对 batch/horizon mean。若 `PadStatesAndActions` 将真实 d 补到 D，padding targets 也进入 gradient 与 denominator；模型可学零，但大量容易维会稀释 effective-action loss。`PadStatesAndActions`、attention mask、token loss mask、action valid mask 与 output slice 是五个不同层次。[6][16]

VLA-ACT-029：conditional dropout / guidance

现代 flow package 的 image example 在训练随机丢 label condition，推理组合 conditional/unconditional velocities：

$$v_{\text{cfg}} = (1 + s)v_{\text{cond}} - s\,v_{\text{uncond}}.$$

变量与实现：s 是 guidance scale；$s \neq 0$ 时每次 ODE function evaluation 实际要两次 model forward。这个 example 支持 CFG 机制，不证明 frozen openpi Pio action path 已实现 condition dropout/guidance。[16][18]

迁移到 action policy 前必须定义 drop 什么：

- 只 drop language，保留 images/state；
- drop task condition 但保留安全/本体状态；
- 或完全 unconditional branch。

风险包括 double NFE latency、过大 velocity/action magnitude、破坏 smoothness、弱化视觉反馈或生成与 controller constraints 不一致的动作。验收要比较 task compliance、diversity、safety、success 与 latency，不只看 velocity loss。

VLA-ACT-030：loss 正常但 rollout 失败的四层定位

层	Flow 必查	Diffusion 必查	共用接口
A target/path	endpoint、time、velocity sign	add-noise、prediction type	shape/dtype/mask
B sampler	grid、dt、NFE、solver	scheduler version、Kd、step order	fixed-noise reproducibility
C transforms	norm、d/D、valid mask	norm、padding、clip_sample	axis/unit/action type
D controller	H/K/R、queue	Tp/Ta/R、timestamp queue	camera/state time、安全与终止

先用 oracle target 测算法不变量。openpi time convention 下，固定 A, ϵ：

$$x_t = t\epsilon + (1-t)A, \qquad u_t = \epsilon - A.$$

变量与测试：用 oracle $v = u_t$ 从同一 noise 做完整 Euler integration，应恢复 A；失败说明 target/sign/grid 有错，尚未轮到模型 capacity。之后依次做 tiny-batch overfit、norm round-trip、per-axis sentinel 和 bypass-model controller test。[16]

```
def debug_generative_action_policy(batch, policy, controller):
    # A. Check target/path algebra with fixed action, noise and time.
    assert_oracle_solver_recovers_target(batch.actions)

    # B. Freeze observation/noise and sweep solver or scheduler steps.
    endpoints = sweep_sampling_steps(policy, fixed_noise=True)
    assert_no_sign_or_step_order_failure(endpoints)

    # C. Verify checkpoint stats, valid dimensions, axes and units.
    assert_normalization_roundtrip(batch.action_schema)
    run_per_axis_sentinel_tests(policy.output_transform)

    # D. Bypass the model with a known-safe action fixture.
    assert_controller_queue_and_replan(controller, batch.safe_fixture)
```

伪代码是可执行检查顺序，不代表本地 OpenPiVLA 已完成这些 tests。只有 checkpoint、action schema、solver/scheduler、H/K/R 与 eval artifacts 齐全后，才能把 failure 归因到个人项目模型。[16]

Action dimension、padding、mask 与 slicing

model D 与 environment d

令真实 action $a \in \mathbb{R}^d$，model action dimension 为 D 且 $d \le D$。zero-pad 与 output selection 可写为：

$$a^{model} = Pa = \begin{bmatrix} I_d \\ 0 \end{bmatrix} a, \qquad a^{env} = Sa^{model}, \quad S = [I_d \quad 0], \quad SP = I_d.$$

变量解释：P 对应输入/target padding，S 对应 output slicing。openpi `PadStatesAndActions` 将尾维补零；LIBERO output transform 返回前 7 维。`Pi0Config` 默认 D=32，`Pi0FASTConfig` 默认 D=32/H=32；官方 `pi0_fast_libero` training config 才显式 override 为 D=7/H=10。[6][12]

training loss mask 是另一件事

理想的 valid-dimension masked loss 可写为：

$$\mathcal{L}_{\text{valid}} = \frac{\sum_{b,h,i} m_{bhi} \left(\hat{a}_{bhi} - a_{bhi}\right)^2}{\sum_{b,h,i} m_{bhi} + \varepsilon}.$$

变量解释：$m_{bhi} = 1$ 表示该 batch/example、horizon、dimension 有真实语义；mask 可同时处理 embodiment-specific dims、episode-end horizon padding 和缺失 label。实现映射必须指出 mask 来自哪个 batch field。frozen π0 main loss 对 D 直接 mean，output slicing 发生在推理 transform；所以“有 slicing”不能证明“训练已 mask padding loss”。[6][12]

四个不同 mask

Mask	管什么	错误后果
image mask	view 是否真实存在	zero image 被当有效 view
token input mask	padding token 是否参加 attention	无效 token 污染 context
token loss mask	哪些 output token 参加 CE	prompt/visual/token padding 被错误监督
action valid-dim/horizon mask	哪些 H×D target 有物理语义	padding dims 或越界 future actions 污染 loss

openpi LIBERO 对缺失 right-wrist image 的 mask 在 Pio 与 PioFAST 路径不同，这是 frozen code behavior；本地模型必须按实际 architecture 验证，不能照抄 bool 值。[6][12]

平移、旋转、gripper 与 base action

先报 action type，再报数值

同一个 7D/8D vector 可表示完全不同控制语义：

- joint position / joint delta / joint velocity；
- end-effector position + orientation；
- gripper absolute position、binary open/close 或 delta；
- mobile base x-y velocity；
- torque/force。

openpi norm_stats 文档说明，其默认 π layout 将前 6/14 维用于左右 arm joint angles、gripper 分列，mobile base 使用 dim 14–15 velocity；DROID 的 8D 却是 7 维 joint velocity + gripper，15 Hz。[6] 因而 shape 相同不等于 action contract 相同。

translation

必须说明：

- absolute pose 还是 $\Delta x, \Delta y, \Delta z$；
- world/base/tool/camera frame；
- meter、centimeter 还是 normalized unit；
- 每 control tick 的 delta 还是每 second velocity。

将位置 delta 当 velocity 会额外乘/除控制周期，造成量级错误。

rotation

常见表示：

表示	维度	优点	风险
Euler angles	3	直观	singularity、周期跳变、轴顺序
axis-angle / rotation vector	3	适合小旋转增量	接近 π 的不连续与 branch
quaternion	4	避免 Euler singularity	unit norm、q 与 -q 双覆盖
6D rotation	6	对 neural regression 更连续	需正交化还原 rotation matrix

Zhou 等指出 3D rotation 在 4D 或以下欧氏表示中存在连续性问题，并提出 5D/6D continuous representation 用于学习。[11] 这不是说所有 policy 都必须用 6D；小角度 delta control、现有

controller API 与 dataset compatibility 仍可能使 axis-angle/quaternion 更合适。

若使用 quaternion $r \in \mathbb{R}^4$，至少检查：

$$\hat{r} = \frac{r}{\|r\|_2 + \varepsilon}, \qquad R(\hat{r}) = R(-\hat{r}).$$

变量解释：第一式保证 unit quaternion；第二式说明双覆盖，同一个旋转有两个相反符号表示。dataset 若不做 hemisphere/sign canonicalization，MSE 会把等价旋转误当作相距很远。

gripper

gripper 常为 absolute [0,1]、binary 或 position/velocity。不要把它与 arm delta 用完全相同的 loss/normalization 假设。OpenVLA OXE materialization 可对最后 gripper 维采用不同 normalization mask；openpi 文档将 gripper position 定义为 0 fully open、1 fully closed，但具体 dataset transform 仍需核对。[3][6][12]

base action

mobile base 常用 v_x, v_y, ω_z 或只用 x-y velocity。必须说明 robot/body frame、最大速度、安全裁剪与是否和 arm 同频执行。public openpi default layout 只列 x-y base velocity，不能据此推断本地项目也有 ω_z。[6]

按语义组做统计

$$\tilde{a}_i = \mathcal{N}_{g(i)}(a_i; \eta_{g(i)}),$$

变量解释：$g(i)$ 把 dimension i 分到 translation、rotation、gripper、base 等语义组，η_g 是对应 stats/scale。实现中仍通常逐维保存数值，但审计时按语义组检查单位、outlier 与 inverse behavior，避免把不同物理量只因相邻排列而当作同分布。

模型输出到 controller 与最小闭环单测

本节对应 `VLA-ACT-031..032`。上游 decode/unnorm/queue、Octo mask 与 openpi broker/DROID example 已独立复核；本地 OpenPiVLA 的 axis、unit、controller mode 与 safety policy 仍需项目 artifact。[21][27]

VLA-ACT-031：decode、unnorm、slice、schema 与 safety 必须有序

直觉：模型输出只是 model-space tensor。只有经过正确 tokenizer/solver、正确 normalization asset、正确维度和物理语义，再被 controller 接收，才是可执行动作。

令 raw model output 为 z，decoder 为 $\mathcal{D}$，反归一化为 $\mathcal{U}$，有效维选择为 S_e，frame/unit/absolute-delta 映射为 $\mathcal{R}_e$，安全投影为 $\Pi_{\mathcal{A}_e}$：

$$a^{\text{ctrl}} = \Pi_{\mathcal{A}_e}(\mathcal{R}_e(S_e\,\mathcal{U}(\mathcal{D}(z)))).$$

变量与实现：$\mathcal{A}_e$ 是 controller 可接受的动作集合。顺序不能任意交换：normalized-space clip 与 physical velocity clip 不同；先 slice 再用 D 维 stats 与先 unnorm 再 slice 只有在 stats padding 规则匹配时才可能等价。[2][6][20][21]

路线	已核实输出链	仍不属于模型的部分
OpenVLA	生成 d 个 token → tail-vocab decode → q01/q99 unnorm	axis/unit/frame/controller
Octo	diffusion action → embodiment D slice → mean/std 或 p01/p99 unnorm	robot-specific controller wrapper
openpi flow	sampled H×D → model outputs → checkpoint unnorm → dataset/repack outputs	local eval client 的 K/axis/safety
PioFAST	AR tokens → FAST inverse → H×D → checkpoint unnorm/output transforms	同上

若 dataset 是部分 delta、controller 接 absolute，公开 openpi 式向量情况可写：

$$a^{\text{abs}}_{t:t+H-1} = a^{\text{data}}_{t:t+H-1} + m_\Delta \odot q_t, \qquad a^{\text{ctrl}} = s_e \odot R_e a^{\text{abs}}.$$

变量与实现：m_Δ 选择 delta dimensions，q_t 是当前 state，R_e 处理轴序/坐标，s_e 处理单位。真实 rotation 可能需要 $SO(3)$ composition，不能默认向量加法。openpi `AbsoluteActions` 只能证明按 mask 给前若干维加 state；本地 (R_e, s_e) 仍需 wrapper 与 sentinel trace。[7][21]

openpi DROID example 是一条具体而非通用的 execution contract：远程返回 `[10,8]`，client 按 index 取一行、二值化 gripper、clip `[-1,1]`、调用 `env.step` 并按数据频率 sleep。[21] 面试中可以用它解释 chain，但不能把它的 8D schema 套到 OpenPiVLA。

VLA-ACT-032：最小闭环测试要让错误可辨识

直觉：全 0/random tensor 很难发现 transpose、错轴和重复行。用 prime/one-hot sentinel，让每个 horizon row 和 action dimension 都不同，再逐层断言。

测试	sentinel / 故障注入	必须断言
parser boundary	min/max token、EOS、越界/短序列	只接受 action token；严格得到 H×D
norm round-trip	q01/mean/q99、gripper mask	`unnorm(norm(a))=a`；masked 维不变

测试	sentinel / 故障注入	必须断言
padding	valid dims 放互异质数，padding 放大常数	loss/output 对 padding sentinel 不敏感；slice 只保留 d
axis/unit	每组维度 one-hot	controller 轴、符号、m/cm、rad/deg 精确匹配
FIFO chunk	row 编码为 0..H-1	连续 K 次严格输出 0..K-1，不重复/跳步
replan/reset	第 K 步新 chunk，episode 中途 reset	只在定义边界切换；旧 queue 不泄漏
stale RPC	延迟、乱序、重复 response ID、断连	旧 response 被丢弃；超时进入 fail-safe
safety	NaN/Inf/超限 velocity/workspace pose	reject/投影规则明确，保留原值与触发日志
mock controller	完整 request→一次 `step`	最终 dtype/shape/value/dt 全部匹配

openpi frozen `ActionChunkBroker` 在 cache 为空时 query，随后按 `_cur_step` slice，到 configured horizon 清空；manual test 只断言每步 shape。[9][21] 因此项目 test 仍需补 request ID、axis/unit、NaN、安全和组合顺序。

```
def test_closed_loop_contract(policy_stub, broker, parser, output_transform, mock_env):
    # Unique rows and dimensions expose transpose, wrong-slice and repeated-row bugs.
    sentinel = make_prime_sentinel_chunk(horizon=4, model_dim=8, valid_dim=7)
    policy_stub.return_value = {"actions": sentinel, "request_id": 42}

    # Execute K=2 actions; FIFO must consume rows 0 and 1 exactly once.
    a0 = broker.infer(make_observation(request_id=42))["actions"]
    a1 = broker.infer(make_observation(request_id=42))["actions"]
    assert not_equal(a0, a1)

    # Apply the declared parse -> denorm -> slice -> schema -> safety sequence.
    c0 = output_transform(parser(a0))
    c1 = output_transform(parser(a1))
    assert c0.shape == c1.shape == (7,)
    assert all_finite_and_within_limits(c0, c1)

    # The final assertion is at the controller boundary, not only model tensors.
    mock_env.step(c0)
    assert mock_env.last_action == expected_controller_sentinel(row=0)
```

代码块说明：真实 suite 还要固定 norm stats hash、tokenizer revision、frame transform、controller mode、control dt 与 timeout policy。合成单测证明接口不变量，不替代 benchmark rollout。

如何选择 action representation

决策顺序

1. 先定 action contract：type、d、H、control frequency、controller。
2. 再看 conditional action 是否明显多模态。
3. 确定 inference latency 和 replan budget。
4. 检查 robot data 规模与 VLM token infrastructure。
5. 评估 normalized reconstruction 与 physical-unit error。
6. 最后才选 head/tokenizer，不以论文名替代需求。

选择表

场景	优先考虑	原因	必查反例
近单峰、严格低延迟、数据有限	direct regression	pipeline 最短、debug 简单	mode averaging、uncertainty
已有 autoregressive VLM、action 低频/短	scalar token	复用 LM head/CE	quantization、AR latency
高频 1-second chunks、需要 token route	FAST	频域压缩减少相关冗余	tokenizer revision、H/D reshape
示范存在 sequence style variation	CVAE/ACT family	latent sequence model	posterior collapse、latent inference
continuous action 多峰、采样预算充足	diffusion	generative continuous distribution	schedule/steps/latency
想用 continuous transport + ODE/action expert	flow	velocity target + solver	time convention、solver stability

不能用单个指标选型

FAST paper 报告其 π0-FAST AR inference 比 π0 flow 慢；这是特定 hardware/model 的 measurement，不是“flow 永远更快”。direct regression loss 低也可能 mode-average；token CE 低也可能只复制相邻动作；diffusion/flow offline sample quality 好也可能因 denorm/controller 错误而 0-success。[7][10][12]

面试回答模板

> 我会先固定 action type、H、d、控制频率与 replan budget，再判断条件分布是否多模态。若近单峰且延迟严格，continuous regression 是强 baseline；若要复用 VLM autoregressive head，考虑 scalar token 或 FAST，但要核 tokenizer、量化与 AR latency；若需要连续多模态生成，比较 diffusion 与 flow 的 target、sampling steps 和 action expert 成本。最后用 physical-unit reconstruction、latency 和 closed-loop success 联合验收。

训练与闭环推理 Pipeline

Batch 构造与训练

```
def build_and_train_vla_step(raw_episode, tick, model, representation, norm_stats):
    # 1. Align camera views and proprioception to one control tick.
    images, image_masks = align_views(raw_episode, tick)
    state = interpolate_proprio(raw_episode, tick)

    # 2. Build an H-step target without crossing the episode boundary.
    actions, horizon_mask = build_action_window(raw_episode, tick)

    # 3. Normalize with the exact stats attached to this dataset/checkpoint.
    norm_state = normalize_state(state, norm_stats)
    norm_actions = normalize_actions(actions, norm_stats)

    # 4. Convert only the action target into the selected representation.
    if representation == "openvla_scalar":
        targets, loss_mask = scalar_action_tokens(norm_actions, horizon_mask)
```

```
    elif representation == "fast":
        targets, loss_mask = fast_frequency_tokens(norm_actions, horizon_mask)
    elif representation == "act_cvae":
        # ACT posterior may see qpos/actions during training; padded slots need masks.
        targets, loss_mask = act_chunk_targets(norm_actions, horizon_mask)
    elif representation == "diffusion":
        # Diffusion target semantics depend on scheduler prediction_type.
        targets, loss_mask = sample_diffusion_target(norm_actions, horizon_mask)
    elif representation == "pi0_flow":
        targets, loss_mask = sample_flow_target(norm_actions, horizon_mask)
    else:
        targets, loss_mask = continuous_targets(norm_actions, horizon_mask)

    # 5. Model inputs keep view/state masks separate from target loss masks.
    output = model(images, image_masks, norm_state, targets)
    loss = masked_objective(output, targets, loss_mask)
    return loss
```

伪代码是通用审计模板：OpenVLA 原始路径没有多视角/state/H>1；ACT posterior 的 train/infer 输入不同；Diffusion target 必须与 scheduler prediction type 一致；π0 flow 不把 actions 变成 token；PioFAST 的 tokenization 与 loss mask 由 transforms 构造。真实实现应替换函数名并链接具体 code。[3][6][12][14][15]

闭环推理

```
def closed_loop_step(env, policy, checkpoint_assets, replan_steps):
    # 1. Observe fresh images/state; preserve camera names and timestamps.
    raw_obs = env.get_observation()

    # 2. Reuse training-time preprocessing and checkpoint normalization stats.
    model_obs = policy.input_transforms(raw_obs, checkpoint_assets)

    # 3. Sample either tokens or a continuous action chunk.
    raw_model_output = policy.model.sample_actions(model_obs)

    # 4. Decode representation, unnormalize, select valid dimensions and convert axes/units.
    action_chunk = policy.output_transforms(raw_model_output, checkpoint_assets)

    # 5. Execute only K actions, then re-observe; H and K are different contracts.
    for action in action_chunk[:replan_steps]:
        checked_action = safety_clip_and_validate(action)
        obs, reward, done, info = env.step(checked_action)
        if done or info.get("success", False):
            break
    return info
```

变量映射：model horizon H 由 checkpoint/config 决定；replan_steps=K 由 rollout client 决定。openpi LIBERO example 默认 K=5，而 Pio/PioFAST configs 可有不同 H。ActionChunkBroker 又可消费整段 H 后再 infer，证明 H 和 K 不是同义词。[6][12]

实现证据与 44 题覆盖

frozen implementation map

机制	关键路径	已核实事实
OpenVLA data contract	`prismatic/vla/datasets/datasets.py`	primary image、language、current action；default no proprio/history/future

机制	关键路径	已核实事实
OpenVLA tokenizer	`prismatic/vla/action_tokenizer.py`	tail vocab、256 token IDs、255 reconstruction centers nuance
OpenVLA inference	`prismatic/models/vlas/openvla.py`	generated length from dataset action dim、decode、unnorm
OpenVLA training	`base_strategy.py` / VLM labels	token CE optimized；continuous L1 metric
π0 shape	`src/openpi/models/pi0_config.py`	default three images、state [B,32]、actions [B,50,32]
π0 flow	`src/openpi/models/pi0.py`	code convention、flow MSE、10-step default、prefix KV cache
π0.5 state/time	`pi0_config.py` / `pi0.py` / `tokenizer.py`	discrete state prompt、adaRMSNorm
PioFAST CE	`src/openpi/models/pi0_fast.py`	prefix-LM attention、postfix masked next-token CE
FAST integration	`tokenizer.py` / `transforms.py`	Hub processor、tail vocab mapping、token decode → H×D
policy transforms	`policy_config.py` / `policy.py`	checkpoint stats、normalize/sample/unnormalize/output order
LIBERO action interface	`libero_policy.py` / `examples/libero/main.py`	8D state、7D output、K=5 default、image rotation
ACT posterior / decoder	`detr/models/detr_vae.py` / `transformer.py`	`[CLS]+qpos+actions`、32D latent、H learned queries、`[B,H,14]`；论文 fixed/sinusoidal query 与 frozen code 有差异
ACT loss / rollout	`policy.py` / `imitate_episodes.py`	masked L1 但 full `B×H×D` mean、KL、`z=0`、整段执行与 temporal ensemble；oldest row 权重最大、zero sentinel 有边界
Diffusion training	`diffusion_unet_hybrid_image_policy.py`	`[B,Tp,Da]`、epsilon/sample target、global condition、masked MSE
Diffusion inference	policy / scheduler / runners	Gaussian restart、DDPM/DDIM、`To-1` action slice、sim/real replan distinction
Flow Matching path/solver	`flow_matching/path/affine.py` / `solver/ode_solver.py`	exact CondOT path、velocity target、Euler/adaptive ODE API
RT-2 / Octo landscape	RT-2 §3.2–3.3、Octo §III、`octo_pretrain_config.py`	action-as-token、800k OXE、27M/93M、T5/image tokenizers、H=4/D=7 diffusion config
Octo mask/readout	`octo_module.py` / `action_heads.py`	block-wise attention、readout isolation、timestep/action conjunction mask、embodiment slice
Adaptation modes	OpenVLA `vla-scripts/finetune.py`、Octo `finetune_config.py`、openpi `training/config.py`	full/head-only/head-MLP-only/LoRA boundaries；不替代本地 trainable log
Serving/timing /queue	openpi `policy.py`、`websocket_policy_server.py`、`action_chunk_broker.py`	model/server/total timing、ordered output transforms、chunk cache/slice/reset
Controller contract tests	Source Audit F test matrix	parser、norm、padding、axis/unit、FIFO/replan、stale RPC、safety 与 mock step 设计

详细 frozen commit 与逐行解释见 Source Audit A、Source Audit B、Source Audit C、Source Audit D、Source Audit E 与 Source Audit F。

44 题覆盖矩阵

ID	本页答案位置	当前证据状态	后续缺口
VLA-ARCH-001	VLA/VLM/visuomotor 对比	source-ready	更多 landscape 对比在 ARCH-007
VLA-ARCH-002	I/O 字段表与 policy contract	source-ready	local OpenPiVLA contract
VLA-ARCH-003	component responsibility table	source-ready	local model class
VLA-ARCH-004	view/state/timestamp alignment	design-ready + upstream examples	local converter timestamps
VLA-ARCH-005	OpenVLA normalize/token/CE/decode	source-ready / WARN	independent endpoint review
VLA-ARCH-006	π0 prefix/suffix/flow/solver	source-ready / WARN	independent convention review
VLA-ARCH-007	RT-2/OpenVLA/Octo/π0 landscape	independent-fact-reviewed / WARN	local architecture mapping
VLA-ARCH-008	token/readout/suffix layout、attention/loss mask	independent-fact-reviewed / WARN	local layout sentinel test
VLA-ARCH-009	freeze/full/head/LoRA/action expert	independent-fact-reviewed / WARN	target data/hardware ablation
VLA-ARCH-010	unified D/d、Octo valid mask、padding risk	independent-fact-reviewed / WARN	local positive mask evidence
VLA-ARCH-011	latency components、NFE/token/transport/queue	independent-fact-reviewed / WARN	local synchronized timing
VLA-ARCH-012	paper→repo→checkpoint→runtime audit	independent-fact-reviewed / WARN	local manifest/trace
VLA-ACT-001	five-route taxonomy	source-ready / WARN	independent cross-method review
VLA-ACT-002	checkpoint norm contract	source-ready	local norm assets/tests
VLA-ACT-003	OpenVLA quantile/bin trade-offs	source-ready / WARN	distribution-shift experiment
VLA-ACT-004	tail vocab offset/collision/decode	source-ready / WARN	tokenizer version test
VLA-ACT-005	FAST motivation	source-ready	independent review
VLA-ACT-006	DCT/round/BPE/inverse	source-ready / WARN	reconstruction test

ID	本页答案位置	当前证据状态	后续缺口
VLA-ACT-007	compression vs control metrics	source-ready	local closed-loop ablation
VLA-ACT-008	P/S、valid mask、slicing	source-ready as contract	local loss mask
VLA-ACT-009	action type/unit/rotation/gripper/base	design-ready	local controller schema
VLA-ACT-010	representation decision table	design-ready	JD / hardware-specific constraints
VLA-ACT-011	ACT H/K/R/Δt	source-ready / WARN	local rollout values
VLA-ACT-012	ACT posterior、decoder、learned queries	independent-fact-reviewed / WARN	local ACT usage and oral drill
VLA-ACT-013	CVAE posterior、KL、test $z = 0$	independent-fact-reviewed / WARN	latent ablation
VLA-ACT-014	overlapping target-time ensemble	independent-fact-reviewed / WARN	local queue and sentinel test
VLA-ACT-015	chunk / replan / ensemble modes	independent-fact-reviewed / WARN	local queue mapping
VLA-ACT-016	horizon trade-off	independent-fact-reviewed / WARN	target hardware/task sweep
VLA-ACT-017	ACT/token/Diffusion/Flow comparison	independent-fact-reviewed / WARN	comparable local ablation; Diffusion audit remains WARN
VLA-ACT-018	sequence-level conditional diffusion	independent-fact-reviewed / WARN	fixed-noise diversity test
VLA-ACT-019	noisy action、epsilon/sample target、condition	independent-fact-reviewed / WARN	local valid-mask evidence
VLA-ACT-020	schedule/prediction type/five time axes	independent-fact-reviewed / WARN	v-pred extension if selected
VLA-ACT-021	`To-1` slice、sim/real replan	independent-fact-reviewed / WARN	local controller queue
VLA-ACT-022	DDPM/DDIM、steps、warm-start gap	independent-fact-reviewed / WARN	target hardware telemetry
VLA-ACT-023	CNF path、sample trajectory、velocity field	source-ready	independent theorem review
VLA-ACT-024	paper $\sigma_{\min}$ 与 exact-linear CFM	source-ready / WARN	hand derivation
VLA-ACT-025	ODE/Euler、time grid、NFE	source-ready	target hardware telemetry
VLA-ACT-026	Diffusion epsilon vs Flow velocity	source-ready / WARN	controlled local comparison

ID	本页答案位置	当前证据状态	后续缺口
VLA-ACT-027	π0 prefix/suffix、time/sign、Beta recipe	source-ready / WARN	local model class
VLA-ACT-028	masked ideal vs frozen all-D mean	source-ready / WARN	positive local mask evidence
VLA-ACT-029	generic flow CFG mechanism	source-ready with warning	action-policy implementation
VLA-ACT-030	four-layer target→controller debug	source-ready as framework	local test artifacts
VLA-ACT-031	decode→unnorm→slice→schema→safety→controller	independent-fact-reviewed / WARN	local axis/unit/controller
VLA-ACT-032	parser/FIFO/replan/stale/safety/mock tests	independent-fact-reviewed / WARN	local executable tests

“source-ready”仍不是“reviewed”。Source Audit C 的 ACT、Audit D 的 Diffusion Policy 与 Audit F 的 architecture/execution 已完成独立事实复核（见 ACT 独立复核、Diffusion 独立复核 与 Architecture/Execution 独立复核）；local runtime 与口述训练仍为 WARN/PENDING。

OpenPiVLA 项目挂钩与真实性边界

可以安全连接的上游机制

- OpenVLA 提供 scalar action-token baseline 和 token/CE/unnorm 审计方法。
- FAST 提供 action-window DCT/round/BPE 表示。
- π0/openpi 提供 multi-view/state/action expert、flow sampling、normalization 与 output transform 的公开候选机制。
- ACT 提供 CVAE action chunk、learned query 与 same-target-time temporal ensemble 的对照机制。
- Diffusion Policy 提供 conditional trajectory diffusion、scheduler 与 receding-horizon 的对照机制。
- openpi LIBERO example 提供 H/K、image transform、state/action slicing 的接口检查范式。
- Octo 提供 block-wise mask、readout 与有效 action-dimension mask 的正例；可用于审查“padding 是否真的不进 loss”。
- Source Audit F 提供 paper→repo→checkpoint→runtime 的证据链、延迟分层和 parser/queue/safety 单测模板。

不能自动写成本地事实

Claim	为什么不能由本页证明	需要的本地 artifact
使用 Qwen3-VL 或某个 π checkpoint	upstream model 不等于 local load path	config、checkpoint manifest、startup log
使用 FAST+ revision ec4d7aa...	default Hub string 未 pin local cache	tokenizer config/cache/revision
horizon=10	official example 可覆盖 config	dataset target、model output shape、eval config
action axes/units	shape 不能推出 controller semantics	schema、norm stats、output transform
formal JSON/token supervision	OpenVLA/FAST 都不等于 local formatter	prompt formatter、labels、loss mask、parser
97.4%	upstream 不含该 local eval	checkpoint ID、suite、tasks、trials、seed、summary
本人创新 action expert/FAST	公开设计属于 upstream	local diff、设计记录、run ledger、ownership

项目口述安全模板

> 上游机制上，我会把 OpenVLA scalar token、FAST frequency token 和 π0 flow action expert 分开解释。对我的项目，我只在本地 config、checkpoint、formatter、norm stats 与 rollout summary 能对应时，才说明实际采用哪条路径。当前整理页中的 97.4%、horizon 和 Qwen/π 关系仍需 artifact reconciliation，因此不会用 upstream repo 代替项目证据。

Bad Case 与最小检查清单

offline loss 正常、online 0-success 的排查顺序

1. **Action semantics**：position/velocity/delta、axis order、frame、unit。
2. **Normalization**：asset_id、q01/q99、clip policy、inverse consistency。
3. **Dimension**：D、d、padding、valid loss mask、output slicing。
4. **Tokenizer/decode**：revision、offset、EOS、H/D reshape、malformed fallback。
5. **Observation**：camera name、flip/resize、image mask、state timestamp。
6. **Chunk loop**：H、K、queue reset、control frequency、episode termination。
7. **Model target**：CE label positions、ACT L1/KL 与 pad denominator、Diffusion prediction type/scheduler、Flow time/velocity sign。
8. **最后才谈 capacity/data**：先排除接口错误，再归因模型不足。

最小单测矩阵

Test	输入	断言
norm round-trip	q01/q99 内 action	inverse error < δ
saturation	q01/q99 外 action	clip/no-clip behavior 与设计一致
axis impulse	单维非零动作	只有预期 actuator/axis 响应

Test	输入	断言
padding	d<D 的 target	extra dims 的 loss/output 行为有显式断言
token round-trip	scalar / FAST known chunk	decode shape、误差、offset 正确
missing view	zero image + mask false/true	architecture-specific mask 正确
chunk queue	H>K、episode reset	执行正好 K 步并清空旧 queue
stale RPC / safety	乱序 response、NaN/Inf、超限 action	旧 response 丢弃；fail-safe/投影日志正确
temporal ensemble	三个 query 覆盖同一 target time	ordering、weights 与 valid sentinel 正确
diffusion scheduler	fixed clean/noise/timestep	add-noise target 与 reverse prediction type 一致
controller rate	constant velocity/delta	单位与 Δt 转换正确

高危答案

- “VLA 就是 VLM 多一个 action head。”
- “OpenVLA 一次生成 H 步，因为会生成很多 token。”
- “FAST 是把每个 action scalar 分 bin。”
- “π0.5 和 PioFAST 是同一个模型。”
- “padding 后推理 slice 掉，所以训练 loss 不会受影响。”
- “ACT 推理会随机采多个 style latent，再挑最优 chunk。”
- “temporal ensemble 给最新预测最高权重，或平滑相邻物理时刻。”
- “Diffusion Policy 的 horizon、返回 action 数、replan interval 和去噪步数是同一个量。”
- “paper 提 warm-start，所以 frozen repo 默认复用上一段 trajectory。”
- “8D action 就是 xyz + rotation + gripper。”
- “offline action loss 下降说明机器人 success 会提升。”

后续工作

本页已完成 VOL 03 的 44/44 知识节点，但仍是 `review_pending / WARN`，不等于已掌握。下一步顺序：

1. 44-card completion reconciliation 已完成；A–F source audits 的独立事实复核仍保持 WARN/PENDING，不提前升 reviewed。
2. 对应 44 道原子题卡全部保持 unseen，等待真实口述。
3. Source Batch C 已完成：ACT paper/repo 的 CVAE latent、chunk query、masked L1 + KL、full-mean padding dilution、learned query 与 temporal ensemble ordering 已独立复核；见 ACT Claim-to-Code 与 ACT 独立复核。
4. Source Batch D 已完成：Diffusion Policy paper/repo 的 target、scheduler、horizon、loss denominator、boundary padding、warm-start 边界与 receding-horizon 已独立复核；见

Diffusion Policy Claim-to-Code 与 Diffusion 独立复核。

5. Source Batch E 已完成：Flow Matching foundation、CFM/OT path、ODE、π0/openpi 时间换元与 Diffusion 对照；见 Flow Matching / π0 Claim-to-Code。
6. Source Batch F 已完成：RT-2/OpenVLA/Octo/π0 architecture、mask、adaptation、latency、checkpoint audit、transform/broker/controller test 已独立复核；见 Architecture & Execution Claim-to-Code 与 独立复核。
7. 下一阶段只做一题 `VLA-ARCH-010/031`、ACT/Diffusion 或 OpenPiVLA ownership 真实口述；不再新增 VOL 03 卡，不把页面生成计入掌握分数。

参考文献（References）

1. Brohan et al., RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control, arXiv v1, 2023.
2. Kim et al., OpenVLA: An Open-Source Vision-Language-Action Model, arXiv v3, 2024.
3. openvla/openvla, frozen commit `c8f03f48af692657d3060c19588038c7220e9af9`.
4. Black et al., π0: A Vision-Language-Action Flow Model for General Robot Control, arXiv v4, 2024.
5. Black et al., π0.5: a Vision-Language-Action Model with Open-World Generalization, arXiv v1, 2025.
6. Physical-Intelligence/openpi, frozen commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`, Apache-2.0.
7. Pertsch et al., FAST: Efficient Action Tokenization for Vision-Language-Action Models, arXiv v1, 2025.
8. physical-intelligence/fast, frozen revision `ec4d7aa71691cac0b8bed6942be45684db2110f4`, Apache-2.0.
9. Zhao et al., Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, ACT, arXiv v1, 2023.
10. Chi et al., Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, arXiv v5, 2024.
11. Zhou et al., On the Continuity of Rotation Representations in Neural Networks, CVPR 2019.
12. 本仓库逐行证据页：OpenVLA Source Audit A；π0 / π0.5 / FAST / openpi Source Audit B。
13. 本地项目入口：OpenPiVLA：FAST 动作表示、Metax 迁移与策略评估复现。该页不是缺失 raw artifacts 的替代证据。
14. ACT CVAE、Action Chunk 与 Temporal Ensemble Source Audit C，frozen commit `742c753c0d4a5d87076c8f69e5628c79a8cc5488`。
15. Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Source Audit D，frozen commit `5ba07ac6661db573af695b419a7947ecb704690f`。

16. Flow Matching 基础、π0 特化与 ODE Source Audit E，用于 ACT/Diffusion/Flow 方法边界与 Batch 2B 知识/题卡。
17. Lipman et al., Flow Matching for Generative Modeling, arXiv `2210.02747v2`, 2023。
18. facebookresearch/flow_matching, frozen commit `11568d37f8d5a080e12aa7b5305d9c35ae07d136`, CC BY-NC 4.0；后续维护包，不作为 2022 历史实现。
19. Octo Model Team et al., Octo: An Open-Source Generalist Robot Policy, arXiv `2405.12213v2`, 2024；§III-A Architecture。
20. octo-models/octo, frozen commit `241fb3514b7c40957a86d869fecb7c7fc353f540`, MIT；block-wise transformer、action head、mask 与 fine-tune config。
21. VLA 架构、适配、延迟与动作执行 Source Audit F，用于 `VLA-ARCH-007..012` 与 `VLA-ACT-031..032` 的 claim-to-code 边界。
22. OpenPiVLA Claim–Evidence Inventory。
23. OpenPiVLA Ownership Ledger。
24. VOL 03 44-Card Completion Reconciliation。
25. ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核，2026-07-19。
26. Diffusion Policy 条件扩散独立事实复核，frozen commit `5ba07ac6661db573af695b419a7947ecb704690f`，2026-07-19。
27. VLA 架构、Mask、适配与执行独立事实复核，RT-2/Octo/openpi direct source hashes 与 probes，2026-07-19。

Part 2 / 16 · `interview_materials/vla/questions/vol_03_vla_action.html` · SHA-256
`EE42DF0EA7C34A12A2173E95B4C0C446CF23139266818C3E6E8CBBFDE5955C9E`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：
`ee42df0ea7c34a12a2173e95b4c0c446cf23139266818c3e6e8cbbfde5955c9e`；生成自：
`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_03_vla_action.md`

VOL 03 VLA / Action：核心面试题卡（完整卷）

状态：`completion_reconciled_candidate / source_audits_warn / 44 unseen / WARN` 当前题数：44 / 44 覆盖范围：`VLA-ARCH-001..012`、`VLA-ACT-001..032` 难度：L1 2 题 / L2 18 题 / L3 24 题 优先级：P0 26 题 / P1 17 题 / P2 1 题 练习状态：全部 `unseen` 使用方式：先闭卷说 30 秒版本，再展开机制；未真实口述前不得改为 pass/strong。 深度来源：VOL 03 完整知识卷；Source Audit A–F 的 public paper/code facts 均已完成独立切片，但本页不替代项目 raw artifacts 或真实口述。 项目证据总门：OpenPiVLA Claim–Evidence Inventory / Ownership Ledger；下列所有项目挂钩均受其 checkpoint、schema、eval 与 ownership 缺口约束。

A. VLA 架构与输入输出合同

▼ `L1` `P0` VLA-ARCH-001 · VLA 与 VLM、传统 visuomotor policy 的本质区别是什么？

面试意图：检查你能否区分语义模型、低层策略和完整执行合同。

30 秒回答：VLM 主要从图像文本产生语义输出；传统 visuomotor policy 直接从观测到动作；VLA 则把视觉语言先验接入可执行 action contract。区别不只是多一个 head，输出还必须经过表示、反归一化和 controller，在闭环中真正执行。

机制展开（约 2 分钟）：RT-2/OpenVLA 把动作编码成 token，让 VLM 用 next-token prediction 学 policy；π0 则用 VLM context 条件化连续 action expert。完整执行策略还包含 preprocess、decode、denormalization、valid-dimension slicing 与 controller。若模型只输出“向左移动”而没有物理 action schema，它更像 semantic planner，不是完整低层 VLA。语言无关、数据少或延迟严格时，小型专用 policy 也可能更合适。

项目挂钩：讲 OpenPiVLA 时先核实本地究竟是 scalar token、FAST、flow 还是自定义 Qwen formatter，再谈项目路线。

追问树：什么时候传统 policy 更合适？语义 planner + 低层 policy 算不算端到端 VLA？VLA 的 online 指标应如何设计？

危险答案：把 VLA 简化成“VLM 后面接一个 action head”，忽略执行接口。

来源：[1][2][4][12][13]　练习状态：unseen

▼ L2 P0 **VLA-ARCH-002 · 一个 VLA 的 observation/text/proprio/action 输入输出合同应包含哪些字段?**

面试意图：判断你是否会把“多模态”落实成可复现的 tensor 和机器人接口。

30 秒回答：输入要报 view、图像预处理/mask、语言模板、proprio 轴序单位与时间戳、可选 history；输出要报 action type、$H \times d$、absolute/delta、控制频率、norm/decode、执行前缀 K 和 controller。H 与 K 必须分开。

机制展开（约 2 分钟）：以 control tick 为锚点，图像可取最近时间戳，连续 proprio 插值，action label 构造 H 步窗口且不能跨 episode。OpenVLA 原始旗舰是单主视角 + 语言 → 单步 d 维动作，无 proprio/history；π0 论文是 2–3 图 + language + proprio → H=50 chunk。模型接收到的通常是已对齐数组，时间同步应由 dataset converter 或 environment wrapper 证明。

项目挂钩：本地 OpenPiVLA 仍需补 config、dataset schema、norm asset、output transform 和 rollout client，不能从项目名猜合同。

追问树：时间对齐在哪里做？缺一张 wrist image 怎么处理？H 与 K 为什么不是同一个参数?

危险答案：只报“图像、文本、动作”，不报 shape、时间、mask、单位和执行语义。

来源：[2][3][4][6][12][13]　练习状态：unseen

▼ L2 P0 **VLA-ARCH-003 · VLM backbone、action expert、policy head 与 controller 分别负责什么?**

面试意图：检查你能否沿模型边界追到物理执行，而不是混淆模块职责。

30 秒回答：VLM backbone 编码视觉语言；policy head 直接给 action logits/参数，action expert 可独立建模 noisy/latent action；output transform 做 decode、denorm、slice；controller 才产生位置、速度或力矩闭环。整条链才是 policy。

机制展开（约 2 分钟）：OpenVLA 没有独立 continuous head，Llama LM head 预测 tail-vocab action tokens，ActionTokenizer 做 scalar 映射。π0 用 PaliGemma prefix 条件化较小 action expert，expert 输出 H×D flow velocity，再由 ODE solver 得到 chunk。无论哪条路，backbone 都不自动定义 axis/unit；controller 也不负责理解任务语言。接口错误与模型能力不足应分层排查。

项目挂钩：项目叙述要把上游 VLM、本人修改的 action formatter/head、policy server 和 LIBERO controller 分开标 ownership。

追问树：projector 算 action head 吗？action tokenizer 是否可训练？为什么 output transform 也属于 policy contract？

危险答案：把 action expert、tokenizer、controller 都称为“action head”。

来源：[2][3][4][6][12][13]　练习状态：unseen

▼ L2 P0 **VLA-ARCH-004 · 多视角图像、语言与 proprioception 如何进入同一策略？时间对齐在哪里发生？**

面试意图：判断你是否理解 modality fusion 之前的数据同步和缺失模态处理。

30 秒回答：先按 control tick 对齐相机与 proprio：图像取最近时间戳、连续 state 可插值，缺失 view 用 mask；再编码、投影并由 attention 融合。模型通常只看到已对齐数组，因此同步责任在 dataset converter 或 environment wrapper，不是 Transformer。

机制展开（约 2 分钟）：每路 view 必须固定 camera name、颜色空间、resize/flip 和有效 mask；language 还要固定 prompt template 与 truncation；proprio 要报 axis、frame、unit 和 timestamp。窗口构造时需限制最大时间差、处理 episode-end padding，禁止 future leakage。openpi frozen model 有三个 image slots 和 masks；LIBERO 缺右腕图时填 zero image，但 Pi0/PiFAST 的 mask 规则不同，不能无条件照抄。

项目挂钩：核对 OpenPiVLA 训练图像与 LIBERO eval 是否同样旋转、resize、camera-key 映射和 state 时序。

追问树：不同相机频率怎么办？zero image 与 mask false 有何区别？history window 如何避免跨 episode？

危险答案：说“把多模态 token concat 就完成对齐”，完全不谈 timestamp。

来源：[4][6][12][13]　练习状态：unseen

▼ L3 P0 **VLA-ARCH-005 · OpenVLA 如何把视觉语言特征映射成离散 action token，再还原连续动作？**

面试意图：检查你能否把 architecture、tokenizer、loss 和 inference 连成一条 code-level 链。

30 秒回答：OpenVLA 将 DINOv2+SigLIP 特征投影进 Llama-2；动作按 dataset q01/q99 归一化、clip，再逐维映射到词表尾部。训练做 masked token CE；推理生成 d 个 token，解码并用同一 stats 反归一化成单步 d 维动作。

机制展开（约 2 分钟）：论文写 256 bins；frozen code 实际构造 256 个 thresholds，digitize 得到 1..256 并映射尾部 256 IDs。decoder 只有 255 个相邻 centers，$b = 255, 256$ 都 clip 到 center 254，因此最高两个 token IDs 重建相同。生成 d 个 token 是一个动作的 d 个维度，不是 H 步 chunk；连续 L1 是 metric，优化目标是 masked token CE。

项目挂钩：OpenPiVLA 若使用 FAST 或 JSON serialization，必须明确说它不是原始 OpenVLA scalar tokenizer。

追问树：为什么是 256 IDs/255 centers？EOS 是否参加 loss？换 dataset 时为什么必须换 unnorm key？

危险答案：说 OpenVLA 直接用 L1 回归连续 action，或把 d 个 token 解释为 d 个时间步。

来源：[2][3][12][13]　练习状态：unseen

▼ L3 P0 **VLA-ARCH-006 · π0 的 action expert 与 flow-matching action generation 如何连接?**

面试意图：检查你能否解释 VLM context、noisy action、velocity target 与 ODE sampling。

30 秒回答：π0 用 VLM prefix 条件化 state 和 noisy action chunk；action expert 输出 H×D velocity，训练拟合 flow target；推理缓存 prefix KV，经默认 10 个 Euler steps 将高斯噪声变为连续 chunk。

机制展开（约 2 分钟）：论文写 $A^\tau = \tau A + (1-\tau)\epsilon$, $u = A - \epsilon$，从 $\tau = 0$ noise 到 1 data。openpi 写 $x_t = t\epsilon + (1-t)A$, $u = \epsilon - A$，从 t=1 以负 dt 到 0；令 $\tau = 1 - t$，path point 相同，velocity 与时间方向同时取反，因此等价。frozen loss 对 D、H、batch 求 mean，是否 mask padding dimensions 仍需按具体项目核对。

项目挂钩：不能仅因项目名含 π 就声称本地使用相同 action expert、H=50 或 10-step solver。

追问树：为什么 action tokens 可双向 attention？paper/code 时间约定为何等价？prefix cache 节省哪部分计算?

危险答案：只说“flow 比 diffusion 少几步”，不报 path、target 和 solver。

来源：[4][6][12][13]　练习状态：unseen

▼ L2 P1 **VLA-ARCH-007 · RT-2、OpenVLA、Octo、π0 的 backbone、action representation 与训练路线如何比较?**

面试意图：检查你能否比较机制，而不是只背模型规模或 benchmark 排名。

30 秒回答：RT-2/OpenVLA 是语言 action token；Octo 是 readout + diffusion；π0 是 VLM context + continuous flow expert。训练路线和数据 protocol 不同，先比较 action/loss/sampler，再谈 backbone，不能直接横比 success rate。

机制展开（约 2 分钟）：RT-2 是 PaLI-X/PaLM-E 的 action-as-token 路线，OpenVLA 延续 scalar tail-vocab token；Octo 用 27M/93M modular transformer、image/T5 tokenizer、readout 与 H×D diffusion；π0 用约 2B PaliGemma context 加约 300M action expert 输出

velocity。回答先报 action unit、loss、sampler，再报 backbone；若没有 local checkpoint，不把项目名归到任何一条。

项目挂钩：OpenPiVLA 要用 model class、config、checkpoint 和 output shape 证明是 FAST、Pio flow 或自定义 Qwen path。

追问树：哪条支持多模态连续动作？Octo 为什么不是 VLM LM head？RT-2 的 cloud 频率能否外推本地？

危险答案：把“更大 backbone”当成四种方法的唯一差异，或直接比较未经对齐 protocol 的 success rate。

来源：[1][2][4][20][21][22][27]　练习状态：unseen

▼ `L3` `P1` **VLA-ARCH-008 · VLA 的 token sequence layout、attention mask 和 action positions 为什么会改变 loss 语义？**

面试意图：确认你能把信息流和梯度流分开解释。

30 秒回答：attention mask 决定能看谁，label/loss mask 决定谁有梯度。OpenVLA 监督 action tail，Octo 监督 readout 后的 diffusion，π0 用 prefix 条件化连续 suffix；错一位就可能把 prompt/EOS 当动作或泄露未来。

机制展开（约 2 分钟）：先写 masked CE，再画三种 layout。OpenVLA prompt 与 visual patch labels 为 `IGNORE_INDEX`；Octo observation 只看 task/历史 observation，readout 只读前文和自身，action loss 另用 timestep/action mask；π0 suffix 的 H 个 noisy action 是连续 expert 输入，不是自回归 token labels。因此“被 attend”不等于“被 supervise”。

项目挂钩：对本地 formatter 要打印 input IDs、attention mask、loss mask、action positions 和 H/D reshape 的 sentinel batch。

追问树：为什么 image patch 不应有 CE？Octo readout 为什么不能被 observation 写回？如何查 off-by-one？

危险答案：看到 action 出现在序列里就默认所有位置参加 next-token loss。

来源：[2][3][6][20][21][22][27]　练习状态：unseen

▼ `L2` `P1` **VLA-ARCH-009 · 冻结 VLM、全量微调、LoRA/adapter 与独立 action expert 如何取舍？**

面试意图：判断你能否按数据、domain gap、显存和遗忘风险做决策。

30 秒回答：新 action head、近似视觉域且数据少时先 head-only/adapter；视觉和动力学明显变时逐步解冻，数据足再 full FT。LoRA 省显存但受 rank/target module 限制；独立 action

expert 解耦连续控制容量，却增加 context/采样接口。repo 支持某路线不等于论文主结果或本地训练采用它。

机制展开（约 2 分钟）：OpenVLA flagship 是 full 7B FT，但脚本支持 all-linear LoRA；Octo 有 head-only、head-MLP-only、full；openpi 有 full 与 PaliGemma/action-expert LoRA config。要核对 trainable-parameter log、base/adapter revision、rank、target modules、merge 顺序、action expert width 和 joint loss，而不是只看配置名。

项目挂钩：若 OpenPiVLA 的 config、显存日志或 diff 缺失，只能给候选方案，不能说本人做过 LoRA 或冻结策略。

追问树：什么时候解冻 vision encoder？LoRA rank 太小的症状？为什么 π0 需要小 expert？

危险答案：把 LoRA 当作所有 VLA 的默认训练方式，或把 action head 参数变化叫架构创新。

来源：[2][7][20][21][22][27]　**练习状态**：unseen

▼ `L3` `P1` **VLA-ARCH-010 · 多机器人异构 action space 如何统一表示，并避免 padding 维污染 loss？**

面试意图：检查你是否能把“统一 D 维”落实为 schema、mask、loss 与 slicing 合同。

30 秒回答：用 P 把真实 d 维写入统一 D 维，用 valid mask 计算 loss，推理再用 S slice 回 d。P/S 不会统一 axis、unit、absolute/delta、rotation 或 gripper polarity，还要有 schema transform。

机制展开（约 2 分钟）：Octo 是公开正例：`timestep_pad_mask & action_pad_mask` 进入 MSE/diffusion loss，sample 时用 embodiment action dim 限制 padded diffusion 维。openpi `PadStatesAndActions` 只 zero-pad，而 frozen Pi0 loss 对 configured D mean，属于负例边界。故“推理 slice 掉”不能证明训练 target 未被 padding 污染，denominator 也必须按 valid 元素解释。

项目挂钩：本地需要 d/D、P/S、mask 来源、per-axis loss 和 padding sentinel test；没有这些 artifact 就不能声称跨机器人统一完成。

追问树：gripper 是否单独统计？padding zero target 与 mask 掉的梯度差别？怎么处理 mobile base？

危险答案：只说“把短向量补零，最后 slice”，不提 loss mask 和单位/轴序。

来源：[6][7][20][21][22][27]　**练习状态**：unseen

▼ `L2` `P1` **VLA-ARCH-011 · VLA 延迟主要来自视觉编码、LLM decoding、action sampling 还**

是通信？怎样拆分？

面试意图：判断你能否设计可复现 timing，而不是凭经验猜瓶颈。

30 秒回答：分层测 capture/preprocess、网络、vision/prefix、token/NFE、postprocess、queue 和 controller，报 warm-up 后 p50/p95。RT-2/FAST 数值只属于指定硬件；openpi timing 仍未拆 vision、sampler、queue。

机制展开（约 2 分钟）：token 路线的 N 近似 action token length，FAST 是可变压缩长度，flow/diffusion 是 NFE/denoise steps；是否缓存 prefix、每步跑大 backbone 还是小 expert 决定曲线。要同步 CUDA/JAX device，记录 payload bytes、H/K、dtype、batch、cache、网络 RTT 和 controller wait。否则一次 wall clock 无法区分模型慢还是 transport 慢。

项目挂钩：OpenPiVLA 若没有 timing log，只能给 instrumentation plan，不能把 RT-2 的 1–3 Hz 或 FAST 的 100/750 ms 写成本地结果。

追问树：为什么 flow 10 step 仍可能比 FAST 快？怎样测 prefix cache？p95 突刺如何定位？

危险答案：只测 `policy.infer` 或只看 GPU kernel，就宣称端到端 latency。

来源：[1][7][12][21][22][27]　练习状态：unseen

▼ L3 P1 VLA-ARCH-012 · 如何从论文和仓库核对一个 VLA 的真实 I/O、checkpoint、normalization 与推理合同？

面试意图：考查你是否能建立 paper→repo→checkpoint→runtime 的证据链。

30 秒回答：冻结论文/repo 版本，读 model/loss/infer；核对 checkpoint、model class、tokenizer、norm、H/D/K 和 transforms，最后用 sentinel request 回放到 mock controller。缺证据就标未核实，不用模型名或 benchmark 补齐。

机制展开（约 2 分钟）：openpi policy loader 从 checkpoint 取 weights 与 norm assets，并按 input repack→normalize→model transform、model output→unnormalize→data/repack output 组合；OpenVLA `unnorm_key` 决定 action dim 和 q01/q99。审计表还要记录 base/adapter revision、image resize/views、sampler/NFE、axis/unit、controller mode、task/trial/seed。论文支持方法，repo 支持实现，checkpoint/runtime 才支持“这一次运行”。

项目挂钩：OpenPiVLA 的 checkpoint、Qwen/π class、FAST revision、norm asset、action schema、97.4% summary 和 ownership 都要回到本地文件，不能由本页上游证据代答。

追问树：多 dataset norm stats 怎么选？如何证明 output shape 不是 accidental？缺 checkpoint manifest 怎么说？

危险答案：拿官方 example config、论文 H=50 或项目页面指标冒充本地运行合同。

来源：[2][3][5][6][7][14][20][21][22][23][24][27]　练习状态：unseen

B. Action representation、tokenization 与 FAST

> 2026-07-19 独立复核已核对 π0.5 hybrid 与 `Pi0FAST` 分叉、FAST paper/Hub 默认值、normalization 外置、token mask/fallback、D/H config override 与 LIBERO replan example；以下题卡仍只代表公共机制，项目 hook 继续要求本地 artifact，状态保持 `unseen/WARN`。

▼ `L1` `P0` **VLA-ACT-001 · 连续回归、离散 token、CVAE、Diffusion 与 Flow action head 各自建模什么?**

面试意图：判断你能否先按分布和训练目标分类，再讨论算法名字。

30 秒回答：连续回归给确定性 action；离散 token 建模 categorical sequence；CVAE 用 latent 表达序列变化；Diffusion 学条件去噪/score；Flow 学 probability path 的 velocity 并用 ODE 采样。选型看多模态性、延迟、数据和控制频率。

机制展开（约 2 分钟）：MSE regression 在多峰标签下可能输出条件均值；token route 可复用 LM CE，但有量化和 autoregressive latency；CVAE 有 posterior collapse 与 train/infer latent gap；Diffusion/Flow 都能生成连续多模态 chunk，但 target、time parameterization 和 sampler 不同。本卷后续卡分别展开 ACT 与 Diffusion，不把 π0 flow 当作所有生成式 policy 的模板。

项目挂钩：OpenPiVLA 选型回答应先确认本地 supervision/decoder，再按 latency、H、control rate 和 closed-loop evidence解释。

追问树：为什么 MSE 会 mode-average? CVAE 推理时 z 从哪里来? Diffusion 与 Flow 的 target 有何差异?

危险答案：用“连续/离散”两类覆盖所有方法，或把 Flow 称为换名字的 Diffusion。

来源：[4][7][9][10][12][13]　练习状态：unseen

▼ `L2` `P0` **VLA-ACT-002 · action normalization 与 denormalization 为什么是 VLA 最容易 0-success 的接口之一?**

面试意图：检查你是否理解 normalized loss 与 physical execution 之间的断层。

30 秒回答：模型在 normalized space 学习，controller 在物理单位执行；stats、axis order、action type 或 inverse 任一错位，都可能 loss 正常却执行错误。因此 normalization 是 checkpoint contract，必须与 output interface 一起版本化。

机制展开（约 2 分钟）：OpenVLA 对受管维度做 q01/q99 映射并 clip；openpi quantile transform 不显式 clip，policy 从 checkpoint assets 加载训练 stats，sample 后再 unnormalize。最小测试包括 norm→unnorm round-trip、q01/q99 endpoint、outlier saturation、constant

dimension、padding dimension 和 axis impulse。FAST Hub processor 从 DCT 开始，不替调用者完成 normalization。

项目挂钩：OpenPiVLA 需要保存实际 asset_id/norm file，并在 LIBERO client 前验证 denorm 后的 7D/目标 action schema。

追问树：clip 与 no-clip 各有什么代价？为什么换 robot 不能沿用旧 stats？constant dimension 怎么处理？

危险答案：说 normalization 只是优化稳定性，与机器人行为语义无关。

来源：[2][3][6][7][8][12][13]　练习状态：unseen

▼ L3 P0 **VLA-ACT-003 · quantile/bin action discretization 如何编码？分辨率、饱和和分布漂移有什么代价？**

面试意图：考查你能否从公式解释 tokenization 的物理误差，而非只背“256 bins”。

30 秒回答：先用每维 q01/q99 把 action 映射到 $[-1, 1]$，再按网格 digitize 成 token；推理取 center 并反归一化。代价是有限 bin 的 resolution error、quantile 外的 saturation，以及换数据/stats 后同一 token 对应错误物理尺度。

机制展开（约 2 分钟）：OpenVLA paper 写 256 bins；frozen path 用 256 thresholds 与尾部 256 token IDs，但 decode table 只有 255 个相邻 centers，最高两个编码索引都落到最后 center。单维 normalized quantization error 还会被物理 quantile range 放大。审计时要画每维 histogram/饱和率，测 encode→decode 的 normalized/physical error，并用 held-out/OOD stats 检查 shift。

项目挂钩：若本地项目改用 FAST/JSON token，不能继续拿 OpenVLA scalar-bin 误差解释本地结果。

追问树：bin 数越多一定越好吗？gripper 需要同样分 bin 吗？如何测 saturation rate？

危险答案：只说“256 bins 精度足够”，不说明 range、center 和 controller unit。

来源：[2][3][12][13]　练习状态：unseen

▼ L3 P0 **VLA-ACT-004 · action token 如何占用语言词表？token offset、vocab collision 和 decode 边界如何检查？**

面试意图：判断你是否能审计 action vocabulary，而不把 token ID 当无语义整数。

30 秒回答：常见做法是复用 VLM 词表尾部并定义可逆 offset。必须固定 tokenizer/version、action-token 数、special-token 保留区、BOS/EOS 和 loss positions；再用边界值与随机 action 做 encode→ID range→decode round-trip，排除碰撞。

机制展开（约 2 分钟）：OpenVLA 用 $t_i = V - b_i$ 占末尾 256 IDs；openpi FAST 将 BPE token 映到 PaliGemma tail，同时跳过最后 128 special-token slots。两条映射不同。检查至少覆盖最小/最大 bin、未知/EOS、生成截断、错误 unnorm key、tokenizer vocab size变化，以及 malformed FAST sequence。loss mask 还要确保 prompt/image padding 不被当 action target。

项目挂钩：若 OpenPiVLA 用 assistant-side JSON/token，需单独验证 formatter、labels、parser 与 vocab contract，不能称为原生 FAST/OpenVLA offset。

追问树：为什么 tokenizer 升级会破坏 checkpoint？EOS 是否属于 action loss？FAST token length 为什么可变？

危险答案：只要 ID 没越界就认为 decode 正确。

来源：[3][6][8][12][13]　**练习状态**：unseen

▼ L2 P0 VLA-ACT-005 · FAST 解决了逐标量 action tokenization 的什么效率问题？

面试意图：检查你是否理解 FAST 的问题设定，而非只会展开缩写。

30 秒回答：高频 action chunk 若逐 timestep/逐 scalar token 化，会产生大量相邻相关 token，模型可能靠复制取得低 CE。FAST 先用 DCT 把轨迹集中到少数低频系数，再量化并用 BPE 合并重复 pattern，减少冗余并提高 token 信息量。

机制展开（约 2 分钟）：DCT 是解析变换，把平滑轨迹能量集中到低频；frequency-first flatten 先预测各维整体趋势；round 产生稀疏整数，BPE 再压缩大量 zero 和常见 coefficient pattern。BPE 是 learned component，但 FAST processor 不负责 q01/q99 normalization。它主要改善 token sequence 与学习信号，不保证 physical reconstruction 或 closed-loop success 必然更好。

项目挂钩：本地若声称“FAST 提升”，至少要提供 token length、reconstruction、latency 和 rollout ablation，而不只报最终成功率。

追问树：为什么先低频更稳定？去掉 BPE 会怎样？FAST 与 OpenVLA scalar token 的基本单位是什么？

危险答案：把 FAST 说成对每个 scalar 做更细的 binning。

来源：[7][8][12][13]　**练习状态**：unseen

▼ L3 P0 VLA-ACT-006 · FAST 的频域变换、量化与 BPE 压缩如何形成 action token 序列？

面试意图：考查你能否按 shape 和顺序复述完整可逆 pipeline。

30 秒回答：对 normalized $H \times D$ chunk 的每维沿时间做 DCT，系数乘 scale 后 round；按 frequency-first 展平，再用 BPE 编成可变长 token。解码反向执行 BPE inverse、reshape、除 scale、IDCT，最后再 denormalize。

机制展开（约 2 分钟）：frozen Hub code 对 [B,H,D] 在 time axis 做 DCT，row-major flatten 实现“所有维度低频在前”。FAST paper 单数据集实验是 scale=10、vocab=1024；frozen FAST+ Hub artifact 是 scale=10、vocab=2048；openpi PiOFASTConfig default H=32,D=32，LIBERO example H=10,D=7。默认值必须连同对象/version 回答，reshape 失败应显式报错。

项目挂钩：核对 OpenPiVLA 保存的 tokenizer revision、scale/vocab、H/D 与 norm stats，避免依赖未 pin 的 Hub default。

追问树：为什么 flatten order 重要？scale 如何影响重建/压缩？BPE 是有损还是无损步骤？

危险答案：把 paper 1024、Hub 2048 和 model H/D 混成一个“FAST 默认配置”。

来源：[7][8][12][13]　**练习状态**：unseen

▼ L2 P1 VLA-ACT-007 · FAST 的 token compression 为什么不等于物理控制误差一定更小？

面试意图：判断你能否区分表示压缩、离线误差和闭环成功率。

30 秒回答：token 少只说明 sequence 更短，不直接约束 IDCT 重建误差；normalized error 还会被 quantile range 放大成物理误差，controller/replan 又会改变闭环结果。因此要分报压缩率、重建、CE、延迟、物理轨迹误差和成功率。

机制展开（约 2 分钟）：round scale 决定有损精度，BPE 主要无损合并整数 pattern；低频优先可能改善 AR 学习，但 tokenizer reconstruction 好不代表 policy 预测对，policy offline 对也不代表 denorm/axis/controller 对。FAST 论文的 no-BPE 消融说明 BPE 在其任务上有帮助，不能推成所有 robot/checkpoint 的因果保证。选型还要与 scalar token、flow 的同协议 baseline 比较。

项目挂钩：97.4% 若无 tokenizer ablation、相同 checkpoint/protocol 和失败分解，不能单因果归因给 FAST。

追问树：应该画哪些曲线？physical error 如何按 semantic group 报告？token 更短为何可能重建更差？

危险答案：把压缩率提升直接翻译为成功率提升。

来源：[7][8][12][13][14]　**练习状态**：unseen

▼ L3 P0 VLA-ACT-008 · action dimension、padding、mask 和 output slicing 如何写成可验证的张量合同？

面试意图：检查你是否会区分 model capacity、有效监督和环境动作。

30 秒回答：设真实 action 为 d 维、model 为 D 维，输入用 $P = [I_d; 0]$ 补到 D，输出用 $S = [I_d, 0]$ 取回 d，且 $SP = I_d$。训练还需 valid mask 排除无语义维度和越界 horizon；推理

slicing 不能证明 loss 已 mask。

机制展开（约 2 分钟）：要分别记录 image mask、token input mask、token loss mask 和 action valid-dim/horizon mask。openpi `PadStatesAndActions` 可补 state/action，LIBERO output transform 切前 7 维；frozen π0 main loss 对 D 直接 mean，所以某个项目若 D>d，要证明 padding target 为零是否有意，或提供实际 loss mask。还需测试 episode-end window、不同 embodiment 的有效维和 batch broadcasting。

项目挂钩：OpenPiVLA 必须给出 local D、LIBERO d、padding 位置、loss 维度和 output parser 单测。

追问树：padding target 监督为零与 mask 掉有何不同？rotation/gripper 可否不连续排列？multi-embodiment mask 怎么存？

危险答案：说“最后 slice 掉 padding，所以训练不会受影响”。

来源：[6][12][13]　**练习状态**：unseen

▼ L2 P1 VLA-ACT-009 · 平移、旋转、gripper 与 base action 应如何表示和归一化？

面试意图：判断你是否会先定义物理语义，再谈网络输出。

30 秒回答：先报 action 的 type、frame、unit 和时间语义：translation 区分 absolute/delta/velocity；rotation 说明表示；gripper 说明 absolute/binary/delta；base 说明 velocity frame。统计逐维保存、按语义组审计。

机制展开（约 2 分钟）：translation 要区分 meter、每 tick delta 与每 second velocity；rotation 要处理 Euler singularity、axis-angle branch、quaternion unit norm/双覆盖或 6D 正交化；gripper 常为 [0,1] absolute，可能需要独立 mask/loss；base 要说明 v_x, v_y, ω_z、frame 和安全限幅。相同 8D shape 可分别表示 pose delta 或 7 joint velocities+gripper，shape 对齐不等于 controller contract 对齐。

项目挂钩：本地 OpenPiVLA action schema 未核实前，只能列待查 axis/unit，不能从 LIBERO 7D 或 openpi 默认 layout 反推。

追问树：quaternion 为什么要 canonicalize sign？delta 与 velocity 如何用 Δt 转换？gripper 为什么常单独处理？

危险答案：看到 7D/8D 就直接说 xyz + rotation + gripper。

来源：[3][6][11][12][13]　**练习状态**：unseen

▼ L3 P1 VLA-ACT-010 · 如何按多模态性、延迟、数据量和 controller 频率选择 action

representation?

面试意图：考查你能否从需求做系统选型，而不是按论文热度排序。

30 秒回答：先锁 action type、d、H、控制频率和 replan budget，再判断 action 是否多模态。近单峰、低延迟先做 regression；复用 VLM AR head 可选 scalar/FAST token；连续多峰再比较 CVAE、Diffusion、Flow，并用重建、延迟和闭环成功联合验收。

机制展开（约 2 分钟）：scalar token 简单但有量化/AR latency；FAST 适合高频 chunk token route，但依赖 H/D/tokenizer revision；CVAE 关注 latent train/infer gap；Diffusion/Flow 有迭代 sampling，需比较 target、steps、action expert 和通信开销。数据有限时复杂生成 head 未必占优；controller 高频时必须看每 chunk 推理、执行 K 与网络抖动。所有选择都应保留 direct regression 或简单 token baseline。

项目挂钩：项目回答用同一 protocol 比较候选路线；没有 ablation 时只讲设计理由，不把最终分数强归因给表示。

追问树：多模态性如何从数据诊断？控制频率升高时 FAST/Flow 怎样变化？为什么需要 simple baseline?

危险答案：说“最新模型一定最好”，或只比较 offline loss。

来源：[4][7][9][10][12][13][14]　练习状态：unseen

C. ACT、Action Chunking 与 Temporal Ensemble

▼ `L2` `P0` **VLA-ACT-011 · action chunk、prediction horizon、execution window 与 replan interval 有何区别?**

面试意图：检查你是否会把 model shape、执行队列和反馈频率分开。

30 秒回答：H 是一次预测的 chunk 长度，K 是实际执行的前缀，R 是两次 policy query 间隔，Δt 是控制周期。它们可以不同；一次输出 H 步不等于必须 H 步后才重新观测。

机制展开（约 2 分钟）：冻结 ACT 无 temporal aggregation 时把 $K = R = H$：每 H 步 query 并消费整段；开启后 $K = R = 1$，但 model H 不变。概念上异步 queue 还能让 K 与 R 解耦。episode T 步时，整段执行约需 $\lceil T/H \rceil$ 次 inference；反馈 staleness 与 R、Δt 共同决定。

项目挂钩：OpenPiVLA 必须从 local model config、client queue 和 controller loop 分别取 H/K/R，不能用 official example 代证。

追问树：K<R 或 R<K 可否存在？H=50、K=5 表示什么？网络延迟应算在哪一层?

危险答案：把 horizon、返回 action 数、replan 和控制频率称为同一个参数。

来源：[9][15][25] 练习状态：unseen

▼ L3 P0 VLA-ACT-012 · ACT 的 CVAE-Transformer 如何一次预测 action chunk?

面试意图：考查你能否按 train/infer 输入与 tensor shape 解释 ACT，而非只说“Transformer + chunk”。

30 秒回答：训练 posterior 用 `[CLS]+qpos+target actions` 得到 32D latent；decoder 融合共享多相机特征、qpos、latent 和 H 个 learned queries，一次并行输出 `[B,H,14]`，不是自回归 token decoding。

机制展开（约 2 分钟）：posterior encoder 为加速不看图像；decoder 的相机共享 ResNet18，feature 沿 width 拼接，latent/qpos 作为额外 token。`nn.Embedding(num_queries,hidden_dim)` 提供 H 个输出槽，action head 每槽输出 14D。论文把 query 描述成 fixed/sinusoidal，但 frozen commit 实际 learned；`is_pad_head` 未进入 loss/推理。

项目挂钩：若本地使用 Qwen/π action expert，ACT 只能作为 action-chunk baseline，不能写成本人架构。

追问树：posterior 为什么不看图像？H 个 query 是否 autoregressive？14D 能否迁移到 LIBERO 7D?

危险答案：说 ACT 逐步生成 H 个 token，或说 frozen queries 是固定正弦位置。

来源：[9][15][25] 练习状态：unseen

▼ L3 P1 VLA-ACT-013 · ACT 的 latent/style variable 在训练和推理时如何使用?

面试意图：检查你是否理解 CVAE posterior、KL 和部署多模态性的边界。

30 秒回答：训练时 posterior 看 qpos 和 target chunk，输出 $\mu, \log \sigma^2$，重参数采样 z，并用 L1 + βKL；冻结推理不跑 posterior，也不随机采样，而固定 $z = 0$，因此 evaluation 是 deterministic。

机制展开（约 2 分钟）：z 为 32D style latent，KL 将 posterior 拉向单位高斯。训练 action target 可见、推理不可见形成 train/infer gap；固定 prior mean 提高可复现性，却不等于部署会探索多个 demonstration mode。论文 Algorithm 1 写 MSE，但方法正文与 frozen `policy.py` 用 masked L1；公开 $\beta = 10$ 不是通用常数。

项目挂钩：项目若声称 CVAE 多模态部署，要给出实际 z sampling/selection code 和 rollout ablation。

追问树：posterior collapse 怎么看？为什么 $z = 0$ 是 prior mean？随机采 z 有何安全风险？

危险答案：说 ACT 推理会采多个 style latent 后自动选出最好 chunk。

来源：[9][15][25] 练习状态：unseen

▼ L2 P1 **VLA-ACT-014 · temporal ensembling 如何融合不同时间预测的同一步动作?**

面试意图：判断你是否能区分 same-target ensemble 与相邻动作 smoothing。

30 秒回答：每个 query 预测未来 H 步；当前物理时刻 t 会被过去最多 H 个 chunk 覆盖。temporal ensemble 只融合这些对同一 a_t 的估计，不平均 a_{t-1}, a_t, a_{t+1}。

机制展开（约 2 分钟）：代码把 query q 的 chunk 写到 `all_time_actions[q,q:q+H]`，再读 column t。按 oldest-to-newest 排列后用 $w_i = e^{-mi}$ 加权；frozen `m=0.01` 且 oldest row 对应 i=0，所以最旧预测权重最大。这样避免普通时域 smoothing 的 phase shift，但快速变化时也可能保留 stale plan。

项目挂钩：若本地实现 overlapping chunks，要用 synthetic table 验证 query/target 轴、权重顺序和有效 sentinel。

追问树：为什么最旧预测权重更大？m 越大怎样？zero sentinel 有什么风险?

危险答案：说它给最新预测最高权重，或说它平滑相邻物理时刻的动作。

来源：[9][15][25] 练习状态：unseen

▼ L3 P0 **VLA-ACT-015 · chunking、replanning 与 temporal ensembling 如何共同影响闭环性?**

面试意图：检查你能否从预测表示走到在线反馈与计算代价。

30 秒回答：chunking 决定一次预测 H 步，replan 决定多久吸收新观测，ensemble 融合重叠 chunk 对同一时刻的估计。ACT 可 H 步 query/整段执行，也可每步 query/ensemble；H 不随之变成 1。

机制展开（约 2 分钟）：整段模式 query 少、chunk 内 sequence coherence 强，但反馈只在边界进入；aggregation 模式每步 query，执行当前融合 action，闭环更强却把 inference 次数从约 T/H 提到 T，并维护 H 长 overlap buffer。平滑不等于安全：多个一致但错误的预测仍会被融合。真实系统还要报告 K/R、queue reset 与 latency。

项目挂钩：将 OpenPiVLA 的 action chunk queue 与 ACT 两种模式做结构对照，但不假定采用 temporal ensemble。

追问树：每步 query 是否一定更好？异步 server 怎么做 overlap？episode reset 要清什么状态?

危险答案：说 action chunking 天然 open-loop，或 temporal ensemble 不增加 inference 成本。

来源：[9][15][25] 练习状态：unseen

▼ L2 P1 VLA-ACT-016 · chunk 太长或太短分别导致什么误差、延迟和吞吐问题?

面试意图：考查你是否能把 horizon 选型转成可测 trade-off。

30 秒回答：H 太短会频繁 query、缺少长程 coherence；H 太长会让 chunk 尾部 stale、长序列更难建模并增加 query-slot 计算。最优值依赖控制周期、K/R、扰动和 latency，不能背成固定 100。

机制展开（约 2 分钟）：无 aggregation 时最大反馈等待约 $(H-1)\Delta t$，query 数约 T/H。H=1 反馈最快但 policy calls 最大；大 H 降低调用数，却让早期误差污染后半段。ACT 论文在其特定 ablation 中 H=100 最好、200/400 略降，只证明该协议的折中。开启 aggregation 可降反馈间隔，却恢复每步 inference。

项目挂钩：本地 sweep 必须固定 checkpoint、controller、task/trials，联合画 latency、physical error 和 success。

追问树：控制频率翻倍时 H 怎么理解？K<H 能缓解什么？如何测 stale tail?

危险答案：把 ACT 的 H=100 写成所有机器人和 VLA 的最优 horizon。

来源：[9][15][25]　练习状态：unseen

▼ L3 P1 VLA-ACT-017 · ACT 与 autoregressive token、Diffusion/Flow policy 的归纳偏置如何比较?

面试意图：判断你是否能比较 target、生成计算、多模态和闭环，而非按论文新旧排序。

30 秒回答：ACT 用 L1+KL 一次并行出 continuous chunk，冻结推理 z=0；token 路线用 CE 自回归；Diffusion 预测 epsilon/sample 走 scheduler；Flow 预测 velocity 解 ODE。选型要联合看多模态、NFE/AR 延迟和 H/K/R。

机制展开（约 2 分钟）：ACT 无量化和多步 sampler，适合低 inference-step chunking，但 test 随机性弱；scalar/FAST token 复用 LM，却有量化或 AR token latency；Diffusion/Flow 从 Gaussian 初始化，显式建连续多峰，但每 query 多次 network evaluation。比较需固定数据、backbone、action schema、controller 和 trial protocol，不能用跨论文成功率下结论。

项目挂钩：OpenPiVLA 只在 local model/target/decoder 取证后选择对应比较列，未做 ablation 时只讲设计理由。

追问树：哪条路线最容易 mode-average？ACT 为什么不等于 deterministic regression？Flow 与 Diffusion 只差步数吗?

危险答案：说 ACT 是 diffusion 的单步版本，或说生成式方法天然闭环更强。

来源：[9][10][15][16][17]　练习状态：unseen

D. Diffusion Policy 与 Receding-Horizon

▼ `L2` `P0` **VLA-ACT-018 · Diffusion Policy 为什么对多模态连续动作有吸引力?**

面试意图：检查你是否理解 sequence-level stochastic generation，而非只会说“扩散能多模态”。

30 秒回答：它把整个 $T_p \times D_a$ continuous trajectory 作为联合随机变量，从 Gaussian trajectory 条件去噪。不同初始噪声可落到不同动作 mode，temporal network 又让同一 mode 内的动作序列保持连贯。

机制展开（约 2 分钟）：policy 建模 $p_\theta(A_t \mid O_t)$，不是逐时刻独立分布乘积；UNet/Transformer 每步处理 `[B,Tp,Da]`。这避免 deterministic MSE 在多峰标签下只给条件均值，但不保证线上必须随机：固定 seed、DDIM、smoothing 和执行窗口都会改变 diversity。还需用 closed-loop success 验证 mode 是否可执行。

项目挂钩：本地若声称多模态收益，要固定 observation，用多 noise sample 展示连贯候选并做 success/安全分析。

追问树：联合 trajectory 与逐步独立有何区别？固定 seed 还多模态吗？怎样避免 unsafe mode?

危险答案：说每个 action timestep 独立采一个 Gaussian，所以能多模态。

来源：[10][16][26]　练习状态：unseen

▼ `L3` `P0` **VLA-ACT-019 · conditional DDPM action objective 中的 noisy action、noise target 与 condition 是什么?**

面试意图：考查你能否把加噪公式、target 和 tensor condition 对到代码。

30 秒回答：对 clean trajectory A^0 随机选 k，用 schedule 得到 $A^k = \sqrt{\bar{\alpha}_k}A^0 + \sqrt{1-\bar{\alpha}_k}\epsilon$。默认模型以最近 T_o 帧视觉/state 为 condition，预测同形状 epsilon，做 MSE。

机制展开（约 2 分钟）：frozen image-hybrid 默认把前 To 帧过 encoder 后 flatten 为 `global_cond`，diffusion trajectory 只含 action channels `[B,Tp,Da]`。`add_noise` 构造 noisy sample，prediction type=epsilon 时 target 是 noise；sample 时 target 是 clean trajectory。condition mask 与 valid padding mask 不同，default global action mask 全 false。

项目挂钩：审计本地 target builder 时固定一组 clean/noise/k 手算 `add_noise` 与 MSE target。

追问树：$\bar{\alpha}_k$ 从哪里来？condition 是否拼进 action channel？sample target 有何变化？

危险答案：说默认 MSE 直接回归 clean action，或把 observation 当 noise target。

来源：[10][16][26]　练习状态：unseen

▼ L3 P1 VLA-ACT-020 · noise schedule、prediction type 和 action horizon 如何影响训练与采样?

面试意图：判断你是否会分开 scheduler、监督参数化和机器人时间轴。

30 秒回答：schedule 决定每个 k 的信噪比与 reverse 系数；prediction type 决定模型学 epsilon 还是 clean sample；T_p 决定 trajectory 长度。它们还要与 T_o, T_a、replan interval 和去噪步数 K_d 分开。

机制展开（约 2 分钟）：frozen policy 只实现 epsilon/sample target；dependency 虽支持 v_prediction，policy target builder 没有对应分支。Tp 变长增加 sequence compute 并可能让尾部 stale，Kd 决定每次 query 的 scheduler iterations。boundary sampler 会重复首尾 target，且 policy 拒绝 valid_mask，所以 padding 行为也会改变 loss。必须锁 diffusers 版本与 runtime override。

项目挂钩：本地 manifest 应一起记录 scheduler config、prediction type、Tp/To/Ta、Kd 和 controller replan。

追问树：只把 config 改成 v_prediction 会怎样？Tp 与 Kd 哪个是控制步？padding 怎么进 loss?

危险答案：把 diffusion timesteps、prediction horizon 和 action execution steps 混为一谈。

来源：[10][16][26]　练习状态：unseen

▼ L2 P0 VLA-ACT-021 · receding-horizon action diffusion 如何执行 chunk 并重新观测?

面试意图：检查你能否从 $T_o/T_p/T_a$ 走到真实 wrapper/queue。

30 秒回答：policy 用最近 T_o 帧生成 T_p 步 trajectory，从 index $T_o - 1$ 起切出 T_a 步并反归一化；simulation 通常执行这段后重观测。real queue 可能提交更长，但 query interval 另由调度器决定。

机制展开（约 2 分钟）：global path 从 `[B,Tp,Da]` Gaussian 开始，完成 Kd 次 reverse step 后 slice `To-1:To-1+Ta`。PushT runner 把返回 batch 交给 `MultiStepWrapper` 逐步执行；real script 可提交 timestamped actions，并按 `steps_per_inference` 重新 query。返回 15 步不等于一定 15 步后才重规划。

项目挂钩：OpenPiVLA client 要分别记录 model H、returned slice、submitted queue 和 actual executed-before-replan。

追问树：为什么 slice 从 To-1 开始？sim/real 有何区别？controller 超时如何处理?

危险答案：看到 horizon=16 就断言每次执行 16 步。

来源：[10][16][26]　练习状态：unseen

▼ L2　P1　**VLA-ACT-022 · 去噪步数、sampler 与 warm start 如何影响策略延迟和稳定性？**

面试意图：判断你是否会保留 paper/config/runtime 差异并做部署权衡。

30 秒回答：K_d 越少通常更快，但 reverse approximation 可能变差；DDPM/DDIM 的随机性和 time grid 不同。论文说可 warm-start，但 frozen policy 每次从 `torch.randn` 重启并拒绝 `past_action`。

机制展开（约 2 分钟）：DDIM 可用抽稀 inference grid，不能说训练也只剩 8 步。版本中 paper 正文/表格、config、runtime 分别出现 10/16/8/16 steps，回答必须带对象。warm-start 若补做，要 shift 未执行 trajectory、匹配新 observation 和 scheduler noise level；它可能降抖动/计算，也可能把 stale plan 带进下一轮。

项目挂钩：本地用 fixed observation/noise sweep Kd，记录真正 model forwards、latency、trajectory error 与 closed-loop success。

追问树：DDIM 是否必然 deterministic？warm-start 从哪个 noise level 开始？NFE 与 controller step 怎么拆？

危险答案：说代码默认 warm-start，或把 100 train timesteps 当成 100 个机器人控制步。

来源：[10][16][26]　练习状态：unseen

E. Flow Matching、π0 与生成式 Action Debug

2026-07-19 独立复核已直接核对 Theorem 2、paper `sigma_min` endpoint、π0/OpenPI time-sign-dt、shifted Beta 换元与 10-step oracle；以下八卡的通用机制口径通过，但项目 runtime 与真实口述仍为 `WARN/unseen`。

▼ L2　P0　**VLA-ACT-023 · Flow Matching 的 probability path 与 velocity field 直觉是什么？**

面试意图：检查你能否区分分布路径、sample trajectory、target field 和网络输出。

30 秒回答：probability path p_t 描述分布从 noise 到 data 如何变化；velocity field $u_t(x)$ 描述位于 x 的样本此刻该往哪里走。模型 v_θ 拟合这个速度，推理解 ODE transport noise。

机制展开（约 2 分钟）：CNF 满足 $d\phi_t(x)/dt = v_t(\phi_t(x))$，且 $p_t = [\phi_t]_* p_0$。直接 marginal field 难算，CFM 对每个 data endpoint 设计 conditional path/velocity；Theorem 2 在 $p_t(x) > 0$ 的假设下证明两项目标相差参数无关常数，因此梯度相同，不是逐样本 loss 相同。“simulation-free”只指训练无需先跑 ODE，采样仍要 solver。

项目挂钩：本地 action head 必须找到 path builder、velocity target、time sampler 和 solver，不能只凭项目名认定是 Flow。

追问树：path 与 trajectory 有何区别？CFM 为什么可训练？推理为什么仍迭代？

危险答案：说 Flow Matching 直接回归 clean action，或 simulation-free 表示推理一步完成。

来源：[17][18][19]　练习状态：unseen

▼ L3 P0 VLA-ACT-024 · 线性插值路径下 target velocity 和 flow-matching loss 如何推导？

面试意图：考查你能否从 path 求导，并保留 paper/package endpoint 差异。

30 秒回答：exact-linear path 是 $x_t = (1-t)x_0 + tx_1$，对 t 求导得 $u_t = x_1 - x_0$，因此 CFM loss 是 $\mathbb{E}\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$。

机制展开（约 2 分钟）：现代 `CondOTProbPath` 用 $\alpha = t, \sigma = 1 - t$。但基础论文 OT path 是 $x_t = [1 - (1 - \sigma_{\min})t]x_0 + tx_1$，target 为 $x_1 - (1 - \sigma_{\min})x_0$，endpoint 仍有小 Gaussian noise。线性 target 对一对 endpoints 为常量，不代表 marginal field 全局常量。

项目挂钩：把 x_1 换成 normalized $H \times D$ action chunk 前，先核本地是否 exact endpoint、time weighting 和 valid mask。

追问树：$\sigma_{\min}$ 为何保留？target 为什么不依赖 t？conditional 与 marginal field 怎么区别？

危险答案：说所有 Flow Matching 都是 exact-linear path，或原论文必然令 $\sigma_{\min} = 0$。

来源：[17][18][19]　练习状态：unseen

▼ L3 P1 VLA-ACT-025 · Flow policy 推理时如何通过 ODE solver 从 noise 得到 action chunk？

面试意图：检查你是否会把 velocity、time grid、NFE 与 action horizon 分开。

30 秒回答：先采同形状 Gaussian action chunk，再沿 time grid 多次算 $v_\theta(x_k, t_k, c)$，用 Euler 等更新 $x_{k+1} = x_k + \Delta t v_\theta$，终点才是 normalized action chunk。

机制展开（约 2 分钟）：solver 可 fixed-step Euler/Heun，也可 adaptive method；要报 grid、step、tolerance 和 NFE。openpi 从 code time t=1 noise 以负 dt 到 0 data，默认 10 Euler steps，并缓存视觉语言 prefix、每步重算 action suffix。10 是 integration steps，不是 H、K 或 controller steps。

项目挂钩：本地记录 solver runtime override、fixed-noise output、每 NFE latency、unnorm 和执行前缀。

追问树：高阶 solver 能修复 target sign 吗？NFE 与 H 有何关系？adaptive solver 如何复现？

危险答案：把 10 次 Euler 说成一次输出 10 个 action，或漏掉终点仍需 unnormalize。

来源：[4][6][17][18][19]　练习状态：unseen

▼ L3 P0 **VLA-ACT-026 · Diffusion noise prediction 与 Flow velocity prediction 的目标和采样有何区别?**

面试意图：判断你是否能按 target/time/sampler 对照，而不是只比较步数。

30 秒回答：冻结 Diffusion Policy 在离散 k 上默认预测 epsilon，由 DDPM/DDIM scheduler 更新；Flow 在连续 t 上预测 probability path velocity，由 ODE solver 更新。两者都可从 Gaussian chunk 开始，但合同不同。

机制展开（约 2 分钟）：Diffusion objective 是 $\|\epsilon - \epsilon_\theta(O, A^k, k)\|^2$；Flow 是 $\|u_t - v_\theta(o, x_t, t)\|^2$。FM 可选 diffusion probability path，所以两类理论有联系，但 prediction parameterization 和 sampler 不能混用。步数不是本质：DDIM 可少步，Flow 也可多 NFE；延迟还取决于 model size 和 prefix。

项目挂钩：比较本地路线时固定 backbone/data/action schema/controller，分别锁 scheduler 或 solver 和 sampling budget。

追问树：都用 MSE 为什么 target 不同？Flow 一定更快吗？v_prediction 属于哪类参数化？

危险答案：说 Flow 只是少几步的 Diffusion，或二者完全无关。

来源：[10][16][17][18]　练习状态：unseen

▼ L3 P0 **VLA-ACT-027 · π0 如何把 VLM context、state 与 noisy action 输入 action expert?**

面试意图：考查你能否把 π0 architecture、tensor shape、time recipe 与 openpi code 对齐。

30 秒回答：图像和语言组成 VLM prefix；state、`[B,H,D]` noisy actions 与 flow time 组成 suffix，action expert 输出同形状 velocity。采样时 prefix KV 缓存，每个 Euler step 只重算 suffix。

机制展开（约 2 分钟）：paper 写 $A^\tau = \tau A + (1-\tau)\epsilon$, $u = A - \epsilon$，从 noise 0 到 data 1；openpi 写 $x_t = t\epsilon + (1-t)A$, $u = \epsilon - A$，从 t=1 负步长到 0。令 $\tau = 1 - t$，time 和 velocity 同时翻转而 trajectory 等价。code 的 shifted Beta time也在该换元下对应论文 recipe。

项目挂钩：OpenPiVLA 必须用 local class/forward 证明采用 action expert，不能由 π/FAST 名称或 Qwen backbone 推断。

追问树：state 在 π0/π0.5 怎样不同？为什么 prefix 可缓存？Beta time强调哪一区域?

危险答案：只说 paper/code 时间相反，不说明 velocity sign 与 $d\tau = -dt$。

来源：[4][6][17]　练习状态：unseen

▼ L3 P0 **VLA-ACT-028 · action loss mask、有效维度与 padded model dimension 如何避免错误监督?**

面试意图：检查你是否会区分 padding、attention mask、loss mask 和 output slicing。

30 秒回答：真实 d 维补到 model D 维后，训练应由 action-valid mask 决定哪些 horizon/axis 进 loss；推理 slice 只控制输出，不能替代训练 mask。mask denominator 也要按有效元素归一化。

机制展开（约 2 分钟）：理想 loss 是 $\sum m(v-u)^2/(\sum m+\epsilon)$。frozen Pio 对 configured D 直接 mean，训练再平均 batch/H；若 D>d，padding zero targets 仍进 gradient 和 denominator，可能稀释有效轴误差。`PadStatesAndActions`、image/token attention mask、token loss mask、action valid mask 和 output slice 是不同层。

项目挂钩：本地需给出 d/D、mask 来源、episode-end slots、per-axis loss 和 output parser 单测。

追问树：监督 padding 为零与 mask 掉有何区别？denominator 为什么重要？multi-embodiment 怎么存 mask?

危险答案：说推理最后 slice 掉 padding，所以训练不会受影响。

来源：[6][17]　练习状态：unseen

▼ L2 P2 **VLA-ACT-029 · conditional dropout / guidance 在 action generation 中何时有意义，风险是什么?**

面试意图：判断你能否从通用 CFG 机制落到 action safety/latency，而不虚构 openpi 功能。

30 秒回答：训练可随机 drop task condition，推理用 $v_{\text{cfg}}=(1+s)v_{\text{cond}}-sv_{\text{uncond}}$ 强化条件遵循；但每 NFE 约双倍 forward，过强 guidance 还可能放大动作、破坏平滑或安全。

机制展开（约 2 分钟）：现代 flow package 的 image example 支持 class dropout/CFG，但 frozen openpi Pio 没有对应 action branch。迁移时要定义只 drop language、保留 image/state，还是完全 unconditional；同时评估 task compliance、diversity、action magnitude、smoothness、success 和真实 model forwards。不能只复制 guidance scale。

项目挂钩：除非 local config/code/ablation 存在，只把 CFG 作为可补做设计，不写成已完成贡献。

追问树：为什么不能 drop proprio？CFG scale 越大越好吗？NFE counter 会漏算什么?

危险答案：因为官方 flow package 有 CFG，就说 π0/openpi 默认启用 guidance。

来源：[17][19]　练习状态：unseen

▼ `L3` `P1` **VLA-ACT-030 · Flow/Diffusion loss 正常但 rollout 失败时，如何区分 target、scheduler、normalization 与 controller 问题?**

面试意图：考查你能否按 causal distance 调试 0-success，而不是直接归因数据或模型容量。

30 秒回答：按四层查：先手算 target/path，再用固定 noise 查 solver/scheduler；随后做 norm、mask、axis/unit round-trip；最后 bypass model 验证 H/K/R、queue 与 controller。每层先建立不变量。

机制展开（约 2 分钟）：Flow 用 oracle velocity 验证 endpoint、time/sign/NFE；Diffusion固定 clean/noise/k 核 `add_noise`、prediction type 和 step order。再 overfit tiny batch、测 q01/q99 或 min/max inverse、d/D/padding、per-axis sentinel。最后直接执行已知安全 action，查 camera/state timestamp、queue、clip和 success predicate。loss 只证明 normalized target fit。

项目挂钩：把每层结果存为 test/log artifact并绑定 exact checkpoint；缺证据时只说待核实，不能把 upstream bug 当本人修复。

追问树：为什么先 oracle 再训练？步数增大仍发散先查什么？如何隔离 controller?

危险答案：看到 loss 正常就跳到“数据不够”，或先调 learning rate 而不查 action contract。

来源：[16][17]　练习状态：unseen

▼ `L3` `P0` **VLA-ACT-031 · 模型输出如何经过 slicing、denormalization、坐标/单位转换进入 controller?**

面试意图：检查你能否把 model-space tensor 追到物理执行边界。

30 秒回答：依次做 token/FAST/flow decode、checkpoint unnorm、valid-dim slice、absolute/delta 与 axis/frame/unit 映射，再安全投影进 controller。每步都要有 shape、单位和 sentinel trace；output shape 不是 controller 语义。

机制展开（约 2 分钟）：可写成 $a^{ctrl} = \Pi(R(S(U(D(z)))))$。OpenVLA 是 token→normalized scalar→q01/q99；Octo 会按 embodiment action dim slice 后 unnorm；openpi 按 model/data output transforms→Unnormalize→repack，DROID example 还二值化 gripper、clip `[-1,1]`、按频率 `env.step`。delta→absolute 要用正确 state/frame，rotation 可能要在 $SO(3)$ 上 composition；local axis/unit/controller 仍需 artifact。

项目挂钩：OpenPiVLA 的 output transform、norm stats、axis order、H/K 与 controller wrapper 任何一项缺失，都只能说待核实。

追问树：为什么不能先 clip 再 denorm？gripper 要不要单独处理？如何证明 slice 不是错轴?

危险答案：说模型直接输出 controller command，或用 upstream DROID 的 8D 例子代替本地 schema。

来源：[2][6][7][14][20][21][22][27]　练习状态：unseen

▼ `L3` `P0` **VLA-ACT-032 · 如何为 action parser、chunk queue、replan 与安全裁剪设计最小闭环单测?**

面试意图：判断你是否能用可辨识输入测试组合接口，而不是只 assert tensor shape。

30 秒回答：用 prime/one-hot sentinel 测 token、norm、padding、axis/unit、FIFO、K 边界、reset、stale RPC、NaN/超限和 mock step。断言每步只消费一次、旧 chunk 不泄漏、无效维不进 loss/输出、最终动作满足安全规则。

机制展开（约 2 分钟）：先构造 H×D 每行每维不同的 chunk；parser 检查 EOS/越界，unnorm 检查逆变换，slice 检查 valid d，axis impulse 检查 frame/unit；broker 在 K 步后重新 query，乱序 response 依 request id 丢弃，NaN/Inf 进入 fail-safe。openpi broker 的公开测试只验证 chunk shape，因此项目还必须补这些不变量和一次真实/仿真 controller `step`。

项目挂钩：测试产物要绑定 tokenizer/norm/config/checkpoint revision，并记录输入 sentinel、输出 action、控制 dt、触发 safety 规则和日志路径。

追问树：H=10/K=5 怎么测？怎样测试 reset 清空 queue？为什么不能依赖 `np.clip` 处理 NaN?

危险答案：只写一个 shape assert，或把成功 rollout 当作 parser/安全单测。

来源：[6][7][14][20][21][22][27]　练习状态：unseen

参考来源

1. Brohan et al., RT-2, 2023.
2. Kim et al., OpenVLA, arXiv v3, 2024.
3. openvla/openvla, frozen commit `c8f03f48af692657d3060c19588038c7220e9af9`.
4. Black et al., π0, arXiv v4, 2024.
5. Black et al., π0.5, arXiv v1, 2025.
6. Physical-Intelligence/openpi, frozen commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`.
7. Pertsch et al., FAST, arXiv v1, 2025.
8. physical-intelligence/fast, frozen revision `ec4d7aa71691cac0b8bed6942be45684db2110f4`.
9. Zhao et al., ACT, 2023.
10. Chi et al., Diffusion Policy, arXiv v5.

11. Zhou et al., On the Continuity of Rotation Representations in Neural Networks, CVPR 2019.
12. VOL 03 完整知识卷。
13. OpenVLA Source Audit A；π0 / π0.5 / FAST / openpi Source Audit B。
14. 本地项目入口：OpenPiVLA。该页不替代缺失的 config、checkpoint、norm stats、diff 与 eval summary。
15. ACT CVAE、Action Chunk 与 Temporal Ensemble Source Audit C。
16. Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Source Audit D。
17. Flow Matching 基础、π0 特化与 ODE Source Audit E。
18. Lipman et al., Flow Matching for Generative Modeling, arXiv `2210.02747v2`。
19. facebookresearch/flow_matching, frozen commit `11568d37f8d5a080e12aa7b5305d9c35ae07d136`, CC BY-NC 4.0。
20. Octo Model Team et al., Octo: An Open-Source Generalist Robot Policy, arXiv v2；octo-models/octo@241fb351, MIT。
21. VLA 架构、适配、延迟与动作执行 Source Audit F。
22. openpi serving / queue / DROID execution：`policy.py`、`action_chunk_broker.py`、`examples/droid/main.py`。
23. OpenPiVLA Claim–Evidence Inventory。
24. OpenPiVLA Ownership Ledger。
25. ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核，本仓库二次 paper/frozen-code 复核与 synthetic probes，2026-07-19。
26. Diffusion Policy 条件扩散独立事实复核，本仓库二次 paper/frozen-code 复核与 synthetic probes，2026-07-19。
27. VLA 架构、Mask、适配与执行独立事实复核，RT-2/Octo/openpi direct source hashes 与 probes，2026-07-19。

Part 3 / 16 · `interview_materials/vla/sources/vol_03_act_claim_to_code.html` · SHA-256
`6C3C40330D13B1BB186B09EA744006667630CD4BD6AE1651EC1FDC38E8DC5A2A`

返回本卷目录

Canonical Markdown SHA-256：
`6c3c40330d13b1bb186b09ea744006667630cd4bd6ae1651ec1fdc38e8dc5a2a`；生成自：
`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_act_claim_to_code.md`

VOL 03 Source Audit C：ACT CVAE、Action Chunk 与 Temporal Ensemble Claim-to-Code

审计日期：2026-07-19 状态：`independent_fact_reviewed / cvae_loss_mask_query_ensemble_checked / local_runtime_pending / WARN` 论文版本：*Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*，arXiv `2304.13705v1` 官方仓库：`tonyzhaozh/act` frozen commit `742c753c0d4a5d87076c8f69e5628c79a8cc5488` License：MIT 对应题目：`VLA-ACT-011..017` 本页问题：ACT 到底如何用 CVAE-Transformer 一次预测 action chunk？训练期 latent、padding 和 loss 如何实现？不使用 / 使用 temporal ensemble 时，prediction horizon、执行窗口、查询频率和闭环性分别是什么？

1. 结论先行

1. **ACT 是连续 action-chunk policy，不是 action tokenizer，也不是带语言输入的 VLA。**论文与 frozen code 的主合同是多视角 RGB + 当前双臂关节位置 → 未来 H 步绝对目标关节位置；该 commit 把 state/action dimension 都硬编码为 14。[1][2]
2. `chunk_size`、`num_queries` **和模型 prediction horizon 在这份实现里是同一个配置 H，但它不自动等于执行窗口。**不开 temporal aggregation 时，代码每 H 步查询一次并顺序执行整段，等价于 $K = H$；开启后每一步查询并融合覆盖当前时刻的多个 chunk，等价于 $K = 1$。[2]
3. **训练期 CVAE encoder 只看当前 qpos 与目标 action chunk，不看图像。**输入顺序是 `[CLS] + qpos + H actions`，长度 $H + 2$；`[CLS]` 输出产生 posterior 的 `mu/logvar`，再用 reparameterization 得到 32D style latent。[1][2]
4. **推理期不是“从 prior 随机采样风格”。**CVAE encoder 被丢弃，frozen code 把 z 固定为全零，即单位高斯 prior 的均值，因此同一 observation 的输出是确定的。[1][2]
5. **decoder 用一个共享 ResNet18 逐相机提特征，将各相机 feature map 沿宽度拼接，再把 z 与 qpos 作为额外 encoder tokens。**H 个 query slot 经 Transformer decoder 和线性 action head 并行输出 $[B, H, 14]$，不是逐步自回归生成。[2]

6. **实际优化目标是 masked elementwise L1 + `kl_weight × KL`。**但源码用 mask 乘零后对完整 $B \times H \times D$ 张量 `.mean()`，分母仍包含 padded slots；它不是按有效元素数重新归一化。`is_pad_head` 虽被定义并前向计算，却没有进入该 commit 的 loss。[2]
7. **论文内部存在 reconstruction loss 表述差异。**Algorithm 1 写 `MSE`，方法正文明确写使用 L1，frozen code 也是 `F.l1_loss`；面试中应以"正文 + 官方实现为 L1，并说明算法框误写/版本差异"为安全口径。[1][2]
8. **temporal ensemble 只融合"不同查询对同一物理时刻的预测"，不是对相邻时刻动作做低通滤波。**论文与代码都把 oldest prediction 排在权重序列首位，并用 $\exp(-mi)$ 使较老预测权重更大；它换来更平滑的动作，但需要每个控制步都做一次 policy inference。[1][2]
9. **ACT 不能替代本地 OpenPiVLA 证据。**ACT 的 14D 绝对 joint target、$H = 100$ 示例、50 Hz、CVAE、temporal aggregation 和公开成功率都不能迁移成个人项目的 action schema、horizon、replan 或指标。[1][2]

2. 证据层级、冻结对象与适用边界

层级	本页使用方式	能支持	不能支持
arXiv v1	方法 Section 4、Algorithms 1/2、Appendix architecture 与 ablation	ACT 的公开动机、CVAE/action-chunk/temporal-ensemble 定义、论文实验	frozen commit 的精确 tensor/mask 行为；个人项目事实
frozen official repo	commit `742c753...` 的 `detr_vae.py`、`policy.py`、`imitate_episodes.py`、`utils.py`	张量布局、loss、normalization、query/execute 分支	论文未归档的真实机器人外部 `aloha_scripts` 全路径
同源推断	从上述论文与代码联合推导 H/K、有效 loss 分母和 row ordering	implementation contract 与安全面试口径	未运行的性能、未给出的 controller 语义
本地项目	本页不读取个人项目 raw artifact	只能提出取证清单	OpenPiVLA checkpoint、axis order、97.4%、个人贡献

冻结信息：

- arXiv API 返回唯一版本 `2304.13705v1`，published/updated 为 `2023-04-23T19:10:53Z`。[1]
- 官方 repo `HEAD` 已核对为 `742c753c0d4a5d87076c8f69e5628c79a8cc5488`，commit date `2024-01-28`；根目录 `LICENSE` 为 MIT。[2]
- 本页审计的是上述历史 commit，不把 2026 年可能存在的 fork、后续 ACT++/ALOHA 2 或第三方重实现混入同一实现结论。

3. 任务 I/O 与数据构造

3.1 论文中的 real-robot contract

论文 Section 4 将 observation 写成当前 follower joint positions 加 4 路 RGB，相机为 top、front、left wrist、right wrist；action 是下一时刻两个 leader arms 的绝对目标关节位置，共 14D。policy 在当前观测下预测未来 action sequence：[1]

$$\pi_\theta(a_{t:t+H-1} \mid o_t, z), \qquad o_t = (I_t^{1:C}, q_t), \qquad a_t \in \mathbb{R}^{14}.$$

变量与直觉：C 是相机数，$q_t \in \mathbb{R}^{14}$ 是当前双臂 joint positions，H 是 chunk/prediction horizon，z 是 CVAE style latent。模型不是只猜 a_t，而是一次输出 $H \times 14$ 的连续目标关节序列。实现映射：`state_dim=14` 和 `num_queries=chunk_size` 在 `imitate_episodes.py#L49-L68`，输出 head 在 `detr_vae.py#L47-L54` 与 `#L131-L139`。

边界：paper 的 real setup 是 4 cameras；frozen repo 自带的 simulation task configs 只有 `camera_names=['top']`，见 `constants.py#L5-L32`。真实机器人 task config 来自仓库外部 `aloha_scripts`，因此不能把“此 repo 所有运行默认 4 相机”写成源码事实。

3.2 一条训练样本怎样产生

`EpisodicDataset.__getitem__` 随机选择 `start_ts`，只读取该时刻各相机图像和 qpos，并读取从该时刻起的所有 action；尾部不足 episode length 的位置补零并生成 `is_pad`。[2]

Tensor	frozen code 语义	进入 policy 前的 shape
`image_data`	当前时刻、按 `camera_names` 堆叠、像素除以 255	`[C,3,H_img,W_img]`
`qpos_data`	当前关节位置，dataset mean/std 标准化	`[14]`
`action_data`	从 `start_ts` 起的绝对目标 joint positions，补到 episode length 后标准化	`[T_episode,14]`
`is_pad`	真实 target 为 `False`，尾部补零为 `True`	`[T_episode]`

随后 `ACTPolicy` 只截前 `num_queries=H` 个 action 与 mask，所以监督 target 是 `[B,H,14]`，见 `policy.py#L18-L35`。完整数据读取、padding 与 normalization 在 `utils.py#L23-L76`。

3.3 normalization 与执行前反归一化

令训练集每维均值和标准差为 $\mu_q, \sigma_q, \mu_a, \sigma_a$，代码使用：

$$\tilde{q}_t = \frac{q_t - \mu_q}{\max(\sigma_q, 10^{-2})}, \qquad \tilde{a}_{t:t+H-1} = \frac{a_{t:t+H-1} - \mu_a}{\max(\sigma_a, 10^{-2})}.$$

变量与实现：所有量均按 14 个 action/state dimensions 独立统计；标准差下界防止近常数维除零。统计在 `utils.py#L79-L108`。该函数在 train/val split 后仍对全部 `num_episodes` 计算统计，见

`utils.py#L111-L128`；这是此 commit 的数据边界，不应泛化为严格 holdout normalization recipe。

推理时再做：

$$\hat{a}_t = \hat{\tilde{a}}_t \odot \sigma_a + \mu_a.$$

代码将反归一化结果直接命名为 `target_qpos` 并传入 `env.step`，见 `imitate_episodes.py#L267-L273`。这证明当前实现目标是 joint-position control，不证明任何本地 VLA 的坐标轴、单位或 gripper 语义。

4. CVAE encoder：style latent 如何训练、如何消失

4.1 posterior 输入不是完整 observation

训练时 encoder 输入序列为：

```
[CLS] learned token       [B, 1, D]
current qpos projection   [B, 1, D]
target action projection  [B, H, D]
             concatenate → [B, H+2, D]
                           |
                           v
                Transformer encoder
                           | take CLS output
                           v
              mu, logvar ∈ R^[B,32]
```

论文明确说明为加速训练，CVAE encoder 不输入图像，只条件于 proprioception 和 target action sequence；Appendix 又给出 `[CLS] + joints + actions` 的 $H+2$ 布局。[1] 代码逐行对应 `detr_vae.py#L66-L110`。源码的局部 shape comment 写过 `seq+1`，但实际 `cat` 和 position table 都是 `H+2`；面试应以操作和论文附录为准。

posterior 写为：

$$q_\phi(z \mid q_t, a_{t:t+H-1}) = \mathcal{N}(\mu_\phi, \mathrm{diag}(\exp(\ell_\phi))), \qquad z = \mu_\phi + \exp(\tfrac{1}{2}\ell_\phi) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

变量与直觉：ℓ_ϕ 是 `logvar`，$z \in \mathbb{R}^{32}$。target action chunk 帮助 encoder 把人类 demonstration 的风格/多模态因素压进 latent；reparameterization 让梯度通过采样。代码映射：`reparametrize#L17-L20` 与 `DETRVAE.forward#L85-L110`。

`is_pad` 在 posterior encoder 中作为 `src_key_padding_mask`，并在前面补两个恒为 `False` 的 CLS/qpos mask，因此 padded future actions 不参与 self-attention，见 `detr_vae.py#L97-L105`。

4.2 推理为什么固定 $z = 0$

当 `actions is None` 时，代码不运行 posterior encoder，而是：

$$z_{test} = 0 = \mathbb{E}_{p(z)}[z], \qquad p(z) = \mathcal{N}(0, I).$$

变量与直觉：z_{test} 是 prior mean，不是一次随机 sample。这样部署输出可重复，但训练期 CVAE 的“能表达多种 demonstration style”不等于测试时会主动探索多模态轨迹。论文 Appendix 明确称该选择使 evaluation deterministic；代码在 `detr_vae.py#L111-L115` 和 `policy.py#L36-L38` 对齐。

面试防线：不要说“ACT 推理时从 CVAE 随机采样多个 action chunk 再选一个”；此 frozen path 只用一个零 latent、一次 deterministic forward。

5. CVAE decoder：相机、qpos、latent 和 action queries 怎样汇合

5.1 observation encoder

每个相机图像先经过同一个 `backbones[0]`；得到的 feature maps 和 2D position embeddings 沿 width 维拼接。然后将 projected latent 与 projected qpos 作为两个额外 token 放在图像 tokens 前面：

```
def act_decode(images, qpos, z, action_queries):
    # All cameras reuse the same ResNet18 weights in this frozen commit.
    camera_features = [shared_resnet(img) for img in images]

    # Concatenate camera feature maps along the spatial-width axis.
    visual_memory = concat_width(camera_features)

    # Prepend projected style and proprioception as two encoder tokens.
    memory_input = prepend(project(z), project(qpos), visual_memory)

    # Decode H learned query slots in parallel; no autoregressive token loop.
    hidden = transformer(memory_input, queries=action_queries)
    return linear_action_head(hidden)  # [B, H, 14]
```

实现链：相机共享 backbone 与 width concat 在 `detr_vae.py#L116-L131`；latent/qpos prepend 与图像 flatten 在 `transformer.py#L49-L77`。图像在 dataset 先除 255，进入 policy 后再做 ImageNet mean/std normalize，见 `policy.py#L18-L23`。

5.2 H 个 query slot 是并行输出槽

模型定义 `query_embed = nn.Embedding(num_queries, hidden_dim)`；decoder 的零初始 target 叠加这些 learned query positions，并对 encoder memory 做 cross-attention。最终 `action_head` 对每个 query 输出一个 14D action，因此：

$$\mathrm{ACT}_\theta(o_t, z) = \left(\hat{a}_t^{(t)}, \hat{a}_{t+1}^{(t)}, \ldots, \hat{a}_{t+H-1}^{(t)}\right) \in \mathbb{R}^{H\times 14}.$$

变量与记号：上标 (t) 表示“在 query time t 做出的预测”，下标表示它要执行的物理时刻。实现映射：query/action heads 在 `detr_vae.py#L47-L54`，Transformer decoder 的 self-/cross-attention 在 `transformer.py#L204-L250`。

paper/code boundary：论文正文/Appendix 把 query 描述为 fixed position / sinusoidal embeddings；此 commit 明确使用可学习 `nn.Embedding`。二者不能在实现面试中混成“代码使用固定正弦 action queries”。

5.3 `is_pad_head` 不能被讲成已训练终止头

模型同时定义 `is_pad_head = nn.Linear(hidden_dim,1)` 并返回 `is_pad_hat`，但 `ACTPolicy` 不计算它的 BCE/CE，也不在推理使用它。因而本 commit 的 chunk 长度由固定 `H=num_queries` 决定；有效尾部依赖 dataset `is_pad` mask 训练，而不是由模型预测动态停止。[2]

6. 训练目标、padding mask 与 paper/code 差异

6.1 frozen code 的 reconstruction loss

令 $p_{b,h} \in \{0,1\}$ 表示 target slot 是否为 padding，$D_a = 14$。源码的精确 L1 为：

$$\mathcal{L}_{L1}^{code} = \frac{1}{BHD_a}\sum_{b=1}^{B}\sum_{h=1}^{H}\sum_{d=1}^{D_a}(1-p_{b,h})\,\left|a_{b,h,d}-\hat{a}_{b,h,d}\right|.$$

变量与直觉：mask 确实让 padded target 的 numerator 为零；但随后 `.mean()` 仍除以完整 BHD_a，而不是 $D_a\sum_{b,h}(1-p_{b,h})$。所以靠近 episode 尾部、valid slots 较少的样本，其 reconstruction contribution 会被额外缩小。直接证据是 `policy.py#L23-L34`。

这与常见的 valid-count normalization 不同：

$$\mathcal{L}_{L1}^{valid} = \frac{\sum_{b,h,d}(1-p_{b,h})|a_{b,h,d}-\hat{a}_{b,h,d}|}{D_a\sum_{b,h}(1-p_{b,h})+\epsilon}.$$

此式只作为对照，并不是 frozen code。它说明“有 loss mask”与“按有效 token/action 数无偏归一化”是两个独立问题；面试中不能看到 `~is_pad` 就默认后者成立。

6.2 KL 与总 loss

posterior 对单位高斯 prior 的 KL 实现为：

$$\mathcal{L}_{KL} = \frac{1}{B}\sum_{b=1}^{B}\frac{1}{2}\sum_{j=1}^{32}\left(\mu_{b,j}^2 + \exp(\ell_{b,j}) - 1 - \ell_{b,j}\right),$$

$$\mathcal{L}_{ACT} = \mathcal{L}_{L1}^{code} + \beta\mathcal{L}_{KL}.$$

变量与实现：j 遍历 32D latent，ℓ 是 `logvar`，β 对应 CLI/config 的 `kl_weight`。KL 在 `policy.py#L71-L84`，总 loss 在 `policy.py#L27-L35`。README 示例和论文 hyperparameter table 都使用 `kl_weight/beta=10`、`chunk_size=100`，但它们是公开 recipe，不是跨任务最优常数。[1][2]

6.3 论文 Algorithm 1 的 MSE 不等于实际代码

位置	reconstruction 表述	判定
Paper Algorithm 1	`MSE(a_hat, a)`	论文算法框原文
Paper Section 4.3 / line after architecture	“We use L1 loss … instead of L2”	论文方法正文
Frozen `policy.py`	`F.l1_loss(..., reduction='none')`	实际审计实现

因此安全表述是：“ACT v1 论文算法框存在 MSE/L1 不一致；方法正文和 commit `742c753...` 的训练路径使用 masked L1，再加 beta-weighted KL。”不要只背算法框，也不要删掉版本差异。

7. Horizon、执行窗口、replan 与 temporal ensemble

7.1 四个容易混淆的量

量	本页记号	ACT frozen code	含义
Prediction horizon / chunk size	H	`policy_config['num_queries']`	一次 forward 输出多少个未来 action slots
Execution window	K	无独立 CLI 参数	一次 query 后实际连续执行多少步才使用新观测重新规划
Query/replan interval	R	`query_frequency`	两次 policy inference 相隔多少 control steps
Control period	Δt	sim `DT=0.02`	环境/机器人执行一步的物理周期；不是模型 horizon

此 commit 中 K 与 R 被 rollout 分支绑在一起，但概念上仍应分开：不开 ensemble 时 $K = R = H$；开 ensemble 时 $K = R = 1$。模型 H 在两种模式中不变。

7.2 模式 A：整段执行

默认 `temporal_agg=False` 时：

```
# Query once at t = 0, H, 2H, ...
if t % H == 0:
    chunk = policy(qpos_t, image_t)  # [1, H, 14]

# Execute every predicted action in order before the next policy query.
raw_action = chunk[:, t % H]
```

这对应 `imitate_episodes.py#L191-L194` 与 `#L246-L261`。它不是"一次输出 H 步，因此整个 episode 永远 open-loop"；更准确地说，**每个 chunk 内不吸收新 observation，chunk 边界重新闭环**。只有 $H = T_{episode}$ 时才退化为整条 episode open-loop。

7.3 模式 B：每步查询 + overlapping-chunk ensemble

开启 `temporal_agg=True` 后，`query_frequency` 被改为 1。query time q 产生对未来时刻 t 的预测 $\hat{a}_t^{(q)}$；对当前时刻 t，有效 query 集为：

$$\mathcal{Q}_t = \{q \mid \max(0, t - H + 1) \le q \le t\}.$$

变量与直觉：最多有 H 个历史 chunk 覆盖当前时刻，且它们都在预测**同一个 a_t**。代码将每次输出写入 `all_time_actions[q, q:q+H]`，再读取 column `t`，见 `imitate_episodes.py#L217-L254`。

按 oldest-to-newest 排序 $q_0 < q_1 < \cdots < q_{n-1}$ 后：

$$w_i = \exp(-mi), \qquad a_t^{exec} = \frac{\sum_{i=0}^{n-1} w_i \hat{a}_t^{(q_i)}}{\sum_{i=0}^{n-1} w_i}.$$

变量与实现：m 是 decay rate；论文明确 w_0 对应 oldest action，代码 row 过滤后仍保持 query-time 升序，并固定 `m/k=0.01`，见 `imitate_episodes.py#L251-L259`。因此较旧预测权重最大；m 越小权重越平，新的 observation 相对占比越高。

不是相邻时刻平滑：它没有把 a_{t-1}, a_t, a_{t+1} 混在一起，而是把多个 `query time` 对同一 target time t 的估计做 ensemble，避免普通时域 smoothing 引入时间偏移。

7.4 rollout 伪代码

```
def act_rollout(env, policy, stats, horizon, temporal_agg):
    # Reset environment; qpos and images are the current observation only.
    ts = env.reset()
    prediction_table = {}  # target_time -> predictions ordered oldest to newest

    for t in range(env.max_steps):
        # Normalize qpos/images exactly as in training.
        qpos_t, images_t = preprocess(ts.observation, stats)

        if temporal_agg:
            # Replan every control step while keeping model horizon H unchanged.
            chunk = policy(qpos_t, images_t)  # deterministic z=0, [H, 14]
```

```
            for offset, pred in enumerate(chunk):
                prediction_table.setdefault(t + offset, []).append(pred)

            # Fuse predictions for the same physical target time, oldest first.
            normalized_action = exponential_ensemble(prediction_table[t], m=0.01)
        else:
            # Query every H steps, then execute the entire cached chunk.
            if t % horizon == 0:
                chunk = policy(qpos_t, images_t)
            normalized_action = chunk[t % horizon]

        # Recover absolute target joint positions and execute one control step.
        target_qpos = denormalize_action(normalized_action, stats)
        ts = env.step(target_qpos)
```

这段伪代码表达 frozen rollout 的信息流，不声称是逐行可运行复制。真实代码还以全零 tensor 作为“尚未填充”的 sentinel，并通过 `torch.all(action != 0)` 判断有效行；若模型恰好输出含精确零分量的 normalized action，该行可能被误判为空。它是此 commit 的实现风险，不等于论文方法必须如此设计。[2]

8. Chunk 长短、闭环性、延迟与吞吐的 trade-off

8.1 H 太短

- $H = 1$ 是单步 policy：每个控制步都要查询，反馈最及时，但长任务的有效决策 horizon 最大，行为更容易受局部预测噪声和 compounding error 影响。[1]
- inference 次数约为 T/H；无 temporal ensemble 时 H 越小，单位 episode 的 policy calls 越多，吞吐压力越高。
- 单步输出缺少 chunk-level sequence coherence，难以利用一个 demonstration 片段中持续动作/暂停等 temporal structure。

8.2 H 太长

- 不 replanning 时，新的 observation 最多要等 H 个 control steps 才被吸收；环境扰动或早期 action error 会让 chunk 后半段 stale。
- 长序列更难建模，decoder query 数增加；可推断地，query self-attention 与 cross-attention 的计算/显存负担也会随 H 增加，但本页没有做 runtime benchmark。
- 论文在四个 simulation/data settings、关闭 temporal ensemble 的 ablation 中报告：平均 success 从 $H = 1$ 的 1% 升到 $H = 100$ 的 44%，更长的 200/400 又轻微下降；作者归因于反应性不足与长序列建模困难。[1] **这不是“100 对所有机器人最优”的证据。**

8.3 temporal ensemble 的额外取舍

收益	代价 / 风险
每步吸收新 observation，避免只在 chunk 边界突变	policy inference 从每 H 步一次变为每步一次
融合相同 target time 的多次估计，减少模型噪声	oldest prediction 权重最大，快速变化场景可能保留 stale plan

收益	代价 / 风险
不增加训练成本	需要维护 overlapping prediction buffer；实现 sentinel/mask 要可靠
动作更平滑且无相邻时刻平均的 phase bias	平滑不等于安全，仍可能把多个一致但错误的预测融合

论文报告 temporal ensemble 对 ACT 的平均增益约 3.3%，但相应实验 separately tuned；因此它是该论文协议内的 ablation，不应直接迁移为本地成功率预期。[1]

9. ACT 与 token、Diffusion / Flow 路线的归纳偏置

路线	Action 表示 / 目标	生成方式	主要归纳偏置	本页边界
ACT	标准化连续 $H \times D_a$；masked L1 + CVAE KL	H 个 learned queries 一次并行回归；test $z = 0$	chunk coherence、低 inference step 数、训练期 latent regularization	本页 paper/code verified
OpenVLA scalar token	每个 action scalar 离散为 token；next-token CE	自回归生成一个当前动作的各维 token	复用 LM vocabulary/objective，量化连续值	见 OpenVLA audit
FAST token	action window 频域变换、量化、BPE；masked token CE	自回归生成压缩后的 action tokens	利用机器人轨迹的时频结构和重复模式	见 π0/FAST/openpi audit
π0 flow action	noisy continuous action chunk 上的 velocity MSE	多步 ODE/Euler integration	连续多模态生成、VLM context + action expert	见 π0/FAST/openpi audit
Diffusion Policy	noisy continuous action trajectory 的 epsilon/sample MSE	scheduler 多步去噪；$T_o/T_p/T_a$ 分离	见 Diffusion Policy audit	source-ready / WARN

选择时不能只问“谁更先进”，而应问：

1. action 是否需要离散 token 与语言模型统一接口；
2. 是否需要显式多模态生成，还是允许 deterministic prior-mean deployment；
3. 一次 forward 的低延迟是否比 iterative generation 更重要；
4. H、K、controller frequency 和 observation latency 如何共同决定闭环；
5. 数据量、动作平滑性、action axes 与 normalization 是否和该表示匹配。

10. Claim-to-code 总表

Claim	论文证据	frozen code 证据	判定	面试口径
ACT 输入 language instruction	无	无 text encoder/input	rejected	ACT 本身是 visuomotor imitation policy，不是 VLA
一次输出未来 H 步连续 joint targets	Section 4.3	`num_queries` + `action_head`	direct	`[B,H,14]`，并行 query slots

Claim	论文证据	frozen code 证据	判定	面试口径
H 自动等于执行窗口	论文区分 naive chunk 与 per-step ensemble	rollout 有两分支	rejected	模型 horizon 与执行/重规划分开报
posterior encoder 输入 RGB	Section 4.2 明确省略 image	只 project qpos/actions	rejected	encoder 看 `[CLS]+qpos+target chunk`
推理从 prior 随机采样 z	Appendix 明确设零	zero tensor	rejected	test $z=0$，deterministic
代码用固定正弦 action queries	paper 如此描述	`nn.Embedding`	paper/code mismatch	此 commit 是 learned queries
实际 reconstruction 是 MSE	Algorithm 1 写 MSE；正文写 L1	`F.l1_loss`	mismatch, code=L1	报版本差异，训练路径按 L1
padded targets 完全按有效数归一化	未给实现分母	mask 后完整 `.mean()`	rejected	numerator masked，denominator 含 padded slots
`is_pad_head` 是训练好的 stop head	无此 claim	head 返回但不进 loss	rejected	固定长度 chunk；不要称动态 stop
temporal ensemble 平均相邻时刻动作	明确否定	读取 prediction table 同一 column	rejected	融合不同 query 对同一 target time 的预测
新预测权重最大	paper w_0 oldest	row 旧→新，exp 衰减	rejected	oldest prediction 权重最大
不开 ensemble 就完全不是闭环	chunk boundary 接收新 observation	每 H 步 query	rejected	chunk 内 open-loop、边界闭环
开 ensemble 不增加成本	仅称无额外 training cost	inference 每步一次	rejected	training 不变，inference 更贵
论文 4 cameras 是 sim repo 默认	real paper 4 view	sim config top-only	rejected as universal	报具体 task/config
ACT 公开结果证明本地 OpenPiVLA	无	无	rejected	另做本地 artifact contract

11. 对 `VLA-ACT-011..017` 的支持状态

Question ID	本页覆盖	状态	仍缺
VLA-ACT-011	$H/K/R/\Delta t$ 定义与两个 rollout 分支	source-ready	本地 OpenPiVLA 的实际 H/K/config
VLA-ACT-012	CVAE encoder、decoder、queries、shape 与 loss	source-ready	独立事实审查
VLA-ACT-013	posterior/reparameterization、test $z=0$	source-ready	真实口述训练
VLA-ACT-014	same-target-time exponential ensemble、weight ordering	source-ready with warning	sentinel edge-case test
VLA-ACT-015	chunk 内/边界闭环、per-step replan 与 ensemble	source-ready	更一般 rollout abstraction 的项目映射
VLA-ACT-016	H 长短、calls、staleness、论文 ablation	source-ready	本地 latency/profile 与控制频率
VLA-ACT-017	ACT 对 OpenVLA/FAST/π0 的归纳偏置	partial/source-ready	Diffusion Policy 独立 source batch

`source-ready` 只表示可据来源写后续题卡，不表示候选人已掌握；题卡生成后仍必须保持 `unseen`，直到真实口述。

12. 与个人项目的真实性边界

ACT 上游事实	可以怎样帮助 OpenPiVLA 面试	不能怎样迁移
$H = num_queries$ 输出 chunk	用作区分 model horizon / execution prefix / replan 的反例	不能证明本地 `horizon=10` 或执行几步
mean/std action normalization	提醒 checkpoint、stats 与 controller 必须匹配	不能替代本地 q01/q99 / norm asset
14D absolute joint targets	作为 action schema 需要显式报告的例子	不能推断本地 axis/order/unit
temporal aggregation $K = 1$	帮助解释“chunk policy 仍可逐步闭环”	不能声称本地使用 ACT ensemble
masked L1 + KL	训练 objective 的对照	不能证明本地 token CE / flow loss
paper ablation / public success	说明方法在特定协议的观察	不能支撑本地 97.4% 或个人贡献

进入项目强回答前仍需本地取证：model class/checkpoint、action dimension 与 axis order、normalization stats、model horizon H、执行前缀 K、policy query 条件、controller frequency、rollout task/trial/split/seed、success predicate 与 eval summary。

13. 面试安全口径与危险答案

13.1 30 秒安全骨架

“ACT 把单步行为克隆改成连续 action chunk 预测。训练时，一个 CVAE encoder 用当前 qpos 和未来动作段推断 style latent，decoder 再结合多视角图像、qpos 和 latent，用 H 个 Transformer queries 并行回归 $H \times 14$ 的绝对 joint targets；loss 在官方实现里是 masked L1 加 beta-KL。推理时 encoder 丢弃并固定 $z = 0$。不开 temporal ensemble 时每 H 步重规划；开启后每步查询，并对不同 chunk 预测的同一时刻动作做指数加权，所以 prediction horizon 与 execution/replan interval 不能混为一谈。”

13.2 高危说法

- “ACT 就是把 action token 一次生成 100 个。”——它输出连续向量，不是 token。
- “CVAE 推理时随机采样，所以每次轨迹不同。”——此官方路径固定 $z = 0$。
- “chunk size 100 就代表机器人 100 步都不看图。”——只适用于不开 temporal ensemble 的整段执行分支。
- “temporal ensemble 对最近预测权重最大。”——论文与代码的序列约定恰好是 oldest 最大。
- “mask 后 loss 就严格只除 valid slots。”——源码 denominator 仍含 padded slots。
- “论文说 MSE，所以代码也是 MSE。”——正文和 frozen code 是 L1。

- “ACT 的 80M、50Hz、4 相机、14D 都是所有 action-chunk policy 的定义。”——这些是特定论文/实现配置。
- “ACT 证明我的 OpenPiVLA 采用了 action chunking 并得到 97.4%。”——缺本地 artifact。

14. 风险、未决项与下一步

1. **核心高风险事实已独立复核**：ACT CVAE、L1/KL、padding denominator、learned query、temporal ensemble ordering、sentinel 与 H/K/R 已在 独立复核页 直接核对，公共事实层为 `PASS WITH WARNING`；local project artifacts、controller、ownership 与 oral drill 仍使本页整体保持 `WARN/PENDING`。
2. **视觉 QA 已完成**：通过 localhost 静态服务检查桌面 `1440×1000` 与窄屏 `800×1000`；确认一处 H1、academic palette/serif、desktop sticky TOC、narrow static TOC、公式无 `<em>` 污染且无页面级横向溢出。
3. **paper/code mismatch 必须保留**：MSE/L1、fixed/sinusoidal query vs learned embedding 都不应被“统一润色”掉。
4. **mask 需要测试**：可用不同 valid lengths 的合成 batch 验证 `.mean()` 缩放；本页当前是逐行确定的代码结论，尚未执行单元测试。
5. **temporal table sentinel 需要测试**：精确零 action component 的有效行可能被 `torch.all(... != 0)` 过滤；不把它夸大为公开论文失败原因。
6. **VLA-ACT-017 仍有缺口**：Diffusion Policy 已完成 source freeze；仍需基础 Flow Matching 独立审计，才能完成 ACT / token / diffusion / flow 的严格对比。
7. **本地项目仍待取证**：ACT upstream 只进入知识与面试对照，不进入个人贡献或指标证明。

参考文献（References）

1. Zhao et al., Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, arXiv `2304.13705v1`, 2023；arXiv abs/API version entry。
2. Tony Z. Zhao et al., tonyzhaozh/act, frozen commit `742c753c0d4a5d87076c8f69e5628c79a8cc5488`, MIT License；核心路径：`detr/models/detr_vae.py`、`detr/models/transformer.py`、`policy.py`、`imitate_episodes.py`、`utils.py`。
3. VOL 03 OpenVLA Action Token 与 I/O Claim-to-Code，本仓库 Source Audit A，2026-07-16。
4. VOL 03 π0 / π0.5 / FAST / openpi Claim-to-Code，本仓库 Source Audit B，2026-07-16。
5. VOL 03 Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code，本仓库 Source Audit D，2026-07-16。
6. VOL 03 ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核，本仓库二次 paper/frozen-code 复核与 synthetic probes，2026-07-19。

Part 4 / 16 · `interview_materials/vla/sources/vol_03_act_independent_fact_review.html` · SHA-256 `FA7E2F654760642A6C834B0C1A48F553ED500DF725D520C8189EF0C073B8C3B0`

返回本卷目录

Canonical Markdown SHA-256：`fa7e2f654760642a6c834b0c1a48f553ed500df725d520c8189ef0c073b8c3b0`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_act_independent_fact_review.md`

VOL 03 Source Audit C：ACT CVAE / Action Chunk / Temporal Ensemble 独立事实复核

状态：`independent_fact_reviewed / audit_c / cvae_loss_mask_query_ensemble_checked / PASS WITH WARNING` 审查日期：2026-07-19 范围：ACT v1 论文与 frozen repo 的 CVAE 输入、latent train/infer、L1/KL、padding denominator、query embedding、temporal ensemble 与 H/K/R rollout。 结论先行：ACT 的论文与官方代码都支持“连续 action chunk + CVAE style latent + Transformer queries”的主机制，但实现层必须保留三类差异：Algorithm 1 写 reconstruction MSE，正文与 frozen code 使用 L1；论文图示的 fixed/sinusoidal query 与 frozen code 的 learned `nn.Embedding` 不同；`is_pad` 乘到逐元素 L1 numerator 后，代码直接对完整 `B×H×D` 张量取 mean，padding 会稀释 loss。temporal ensemble 融合的是不同 query 对同一 target time 的预测，oldest row 权重最大；含任意零分量的 prediction row 会被 sentinel 过滤。以上均为 public paper/code facts，不证明本地 OpenPiVLA 采用 ACT 或任何指标。

1. 直接复核来源与冻结对象

ID	来源	本次直接读取内容	SHA / Verdict
Po 1	ACT v1 PDF	Section IV、Algorithms 1/2、Fig. 4/5、loss/query/ensemble ablation	`2E2FE25860F5F9CEE9E655A2714E9F2264A7A5078BCD267C16D3F39AF461345A` / `primary-verified`
Co 1	`detr_vae.py`	posterior input、reparameterization、zero latent、query embedding、pad head	`499D968A3704156E5E29425A29691C3FA93421481D3609ED6B600B21801270F1` / `official-code`
Co 2	`policy.py`	masked L1、KL、loss denominator、test path	`352EFCA415EE0E31073A27CA91560195702D1E9B545E573A3A668A53F06DD127` / `official-code`
Co 3	`utils.py`	episode tail padding、is_pad、mean/std、all-episode stats	`71315DE9E613024E781F089C2F46BAA6A94BE79DAA97A45C8601829FC1BADCEC` / `official-code`
Co 4	`imitate_episodes.py`	query frequency、temporal table、sentinel、postprocess、50 rollouts	`F2B057CD497F3FEF20BDC4197879D1C02E1C751F83154CCE0183FCC10F7E2540` / `official-code`
Co 5	`constants.py`	simulation camera/task defaults	`2E27D4412E2B7E1F2E4148105FDF514D8C52EC6C8F1C86EF0549C475E8B91D86` / `official-code`

ID	来源	本次直接读取内容	SHA / Verdict
Co 6	`transformer.py`	latent/proprio prefix、camera concat、query decoder	`C6D2A1E7138E6CA62DB7CC61BFC99857FB2A3A9375BA7B304310CA264F357EE2` / `official-code`
Oo 1	本次 synthetic probes	masked mean dilution、temporal weights、zero sentinel、H/K query steps	`local-evidence / PASS`

冻结 commit：`742c753c0d4a5d87076c8f69e5628c79a8cc5488`，MIT。本文不把 repo 外部 `aloha_scripts`、第三方 ACT++ 或用户本地 fork 混入实现结论。

2. ACT 的 public I/O 与 chunk contract

论文 real-robot setup 使用多视角 RGB、当前 follower qpos 与 14D absolute joint target；frozen repo 的 simulation config 可只有 top camera，因此“ACT repo 默认总是四相机”不成立。一次 policy forward 输出 `[B,H,14]` 连续 action chunk：

$$\pi_\theta(a_{t:t+H-1} \mid o_t, z), \qquad o_t = (I_t^{1:C}, q_t), \qquad a_t \in \mathbb{R}^{14}.$$

变量与直觉：`H` 是 prediction/chunk horizon，`C` 是实际 camera count；`q_t` 与 `a_t` 是当前 joint state 与未来 joint target。代码中的 `num_queries` 同时决定 query slot 数与输出时间长度，但不能单独证明执行窗口 `K` 或 replan interval `R`。

3. CVAE：posterior 看什么，推理时如何消失

训练期 posterior encoder 的输入是：

```
[CLS] learned token      [B,1,D]
current qpos projection  [B,1,D]
target action projection [B,H,D]
                         → encoder sequence [B,H+2,D]
```

代码由 `encoder_action_proj`、`encoder_joint_proj`、`cls_embed` 与 `latent_proj` 产生 $\mu, \log\sigma^2$，再用：

$$z = \mu_\phi + \exp\left(\frac{1}{2}\log\sigma_\phi^2\right) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

变量与实现：`z\in\mathbb R^{32}` 是 style latent；posterior 不读取 image，image 只进入 decoder policy path。推理时不传 action，因此代码跳过 posterior，直接构造全零 `z=0`，即单位高斯 prior

mean；这使 frozen path 在相同输入下 deterministic。不能说 ACT inference 默认随机采样多个 style。

decoder 通过 ResNet 特征、qpos、latent 与 `H` 个 query embedding 经过 Transformer decoder，最后由 `action_head` 映射到 `[B,H,14]`。论文图示/文字使用 fixed/sinusoidal position embedding 的表述，而该 frozen commit 的 `query_embed=nn.Embedding(num_queries, hidden_dim)` 是 learned query；这是必须保留的 paper/code mismatch。

4. L1/KL、padding 与 loss denominator

论文 Algorithm 1 写 reconstruction MSE，但正文 Section IV-C 说明使用 L1，frozen `policy.py` 实际执行：

$$\mathcal{L}_{\text{L1,code}} = \text{mean}_{b,h,d}\left[m_{b,h}\ \left|\hat{a}_{b,h,d} - a_{b,h,d}\right|\right],$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{L1,code}} + \lambda_{\text{KL}}\, D_{\text{KL}}(q_\phi(z \mid q, a)\,\|\,\mathcal{N}(0, I)).$$

变量与实现：`m_{b,h}=1` 表示 non-padding timestep；代码把 `is_pad.unsqueeze(-1)` 取反后乘到逐元素 L1，但随后直接调用 tensor `.mean()`，分母仍是完整 `B\times H\times D`。`is_pad_head` 由 decoder 返回，却没有进入该 commit 的 loss。

理想的 valid-element reduction 应为：

$$\mathcal{L}_{\text{valid}} = \frac{\sum_{b,h,d} m_{b,h}\left|\hat{a}_{b,h,d} - a_{b,h,d}\right|}{\sum_{b,h,d} m_{b,h} + \varepsilon}.$$

这两个公式的区别不是“mask 是否存在”，而是 denominator 是否按有效 action elements 归一化。被 mask 的 padding 不参与 numerator，但仍会稀释 gradient scale；不能把 frozen code 的行为称为严格 valid-only mean。

5. 数据 padding 与 normalization 边界

`EpisodicDataset` 随机选 episode 内 `start_ts`，读取当前 qpos/图像和从该时刻开始的 action；episode 尾部不足 `episode_len` 的位置以零填充并置 `is_pad=True`。`get_norm_stats` 在 train/val split 之前对传入的全部 `num_episodes` 计算 qpos/action mean/std，并将 std clip 到至少 `1e-2`：

$$\tilde{q} = \frac{q - \mu_q}{\max(\sigma_q, 10^{-2})}, \qquad \tilde{a} = \frac{a - \mu_a}{\max(\sigma_a, 10^{-2})}.$$

变量与实现：这说明当前 commit 的 normalization stats 不是严格只由 train split 计算；它是实现事实，不应推广成推荐的 holdout protocol。推理时 `imitate_episodes.py` 用 dataset_stats 反归一化并把结果命名为 `target_qpos` 传给 `env.step`，证明该实现的 controller target 是 joint-position path。

6. Temporal ensemble：同一 target time，oldest 权重最大

不开 temporal aggregation 时，代码：

- `query_frequency=num_queries=H`；
- 在 `t=0,H,2H,...` query；
- 对当前 chunk 读取 `all_actions[:,t % H]`；
- chunk 内 open-loop，chunk 边界重新观察。

开启 temporal aggregation 时：

- `query_frequency=1`，每个 control step query；
- 第 `q` 次 query 的 `H` 个预测写入 `all_time_actions[q,q:q+H]`；
- 当前物理时刻 `t` 读取的是 column `t`，融合不同 query 对同一 `a_t` 的预测；
- `exp_weights=np.exp(-0.01*np.arange(n))`，row 顺序 oldest→newest，因此 oldest 权重最大。

若有效预测数为 `n`，权重是：

$$w_i = \frac{\exp(-mi)}{\sum_{j=0}^{n-1} \exp(-mj)}, \qquad i = 0 \text{ 表示 oldest row.}$$

变量与实现：这不是把 `a_{t-1},a_t,a_{t+1}` 做普通时域平滑，而是对同一目标时刻的多次估计加权。论文称没有额外 training cost，但 inference 从每 `H` 步一次变为每步一次；不能把“无额外训练成本”说成“无额外部署成本”。

7. Synthetic probes

本次用独立 NumPy probe 检查边界，不安装或修改 ACT 运行环境：

Probe	结果	含义
`B=1,H=4,D=14`，仅 1 个 valid element error=1	code-style full mean = `1/56=0.0178571429`，valid-only mean = 1	padding/维度会产生 56 倍 dilution
两样本 valid lengths=4 与 1	full mean=0.625，valid-only mean=1	batch 内不同 episode 长度也改变 effective scale
temporal weights `n=3,m=0.01`	[0.336672, 0.333322, 0.330006]	oldest row 最大
prediction row 含一个零分量	`torch.all(row != 0)` 为 false，整行被过滤	sentinel 不是 per-dimension mask
`H=3`、10 control steps	no-ensemble query steps [0,3,6,9]；ensemble [0..9]	H、K、R 的查询语义分离

probe 验证的是 reduction、权重和分支语义，不证明任何 learned policy success、controller safety 或个人项目 bug。

8. 对 VLA-ACT-011..017 的回写结论

Question ID	独立复核结论	必须保留的边界
`VLA-ACT-011`	`H/K/R/\Delta t` 与两种 rollout 分支通过	local OpenPiVLA H/K 未证
`VLA-ACT-012`	CVAE posterior/decoder/query/shape 通过	ACT 不是 language VLA
`VLA-ACT-013`	posterior train、reparameterization、test `z=0` 通过	不推断本地随机 rollout
`VLA-ACT-014`	same-target-time ensemble 与 oldest-first weight 通过	sentinel edge-case 仍是实现风险
`VLA-ACT-015`	chunk boundary 与 per-step ensemble 通过	local queue/controller 未证
`VLA-ACT-016`	H trade-off、calls、staleness 与 paper ablation 通过	论文 H=100 不是通用最优
`VLA-ACT-017`	ACT/FAST/π0/Diffusion 表示对照可用	不横比跨 protocol success

本次不改变 44 张 VOL 03 卡或全局题卡的 `unseen` 状态，也不创建 `coaching_state.md`。

9. 不能推出的本地项目事实

- 不推出 OpenPiVLA 使用 ACT 的 CVAE、14D absolute joint target 或 temporal ensemble。
- 不推出本地 `H=100`、`K=1`、`R=1`、50 Hz、4 cameras、mean/std stats 或 97.4%。
- 不把论文 Algorithm 1 的 MSE/L1 表述差异、learned query、mask denominator 或 sentinel edge-case 写成本人发现/修复。
- 不把 ACT public success、paper ablation 或 official code author 映射为候选人 ownership。

10. 下一步

Audit C 的 ACT CVAE、L1/KL、padding denominator、query embedding、temporal ensemble 与 H/K/R public facts 已完成独立复核，公共事实层 verdict 为 `PASS WITH WARNING`。这只表示 paper/frozen-code/synthetic-probe 证据链已闭合；local project artifacts、controller、ownership

与 oral drill 仍保持 `WARN/PENDING`。下一 bounded slice 优先进行一题 ACT 真实口述，或收集本地 OpenPiVLA runtime artifact。

参考入口

- VOL 03 ACT Claim-to-Code
- VOL 03 π0 / FAST / OpenPI Independent Review
- VOL 03 Diffusion Policy Claim-to-Code
- VOL 03 Completion Reconciliation
- OpenPiVLA Inventory
- OpenPiVLA Ownership Ledger

Part 5 / 16 · `interview_materials/vla/sources/vol_03_architecture_execution_claim_to_code.html` · SHA-256 `03339A51A09812DB5D78E87BFB1F947D00C8E5EFE677CAA87EA76D7A5CBA2B5F`

返回本卷目录

Canonical Markdown SHA-256：`03339a51a09812db5d78e87bfb1f947d00c8e5efe677caa87ea76d7a5cba2b5f`；生成自：`interview_materials\vla\sources\vol_03_architecture_execution_claim_to_code.md`

VOL 03 Source Audit F：VLA 架构、适配、延迟与动作执行 Claim-to-Code

审计日期：2026-07-19 状态：`independent_fact_reviewed / landscape_mask_transform_queue_checked / local_runtime_pending / WARN` 论文范围：RT-2 `2307.15818v1`、OpenVLA `2406.09246v3`、Octo `2405.12213v2`、π0 `2410.24164v4` 官方仓库：`openvla/openvla@c8f03f48...`、`octo-models/octo@241fb351...`、`Physical-Intelligence/openpi@15a9616a...` 对应题目：`VLA-ARCH-007..012`、`VLA-ACT-031..032` 本页问题：四类代表性 VLA/GRP 的 backbone、action representation 与训练路线如何比较？sequence layout、attention/loss mask、适配策略、异构动作、延迟和 checkpoint contract 怎样逐项核实？模型输出又怎样经过 decode、反归一化、切片、坐标/单位转换、chunk queue 与安全约束进入 controller？

1. 结论先行

1. **RT-2、OpenVLA、Octo 与 π0 不是同一条 action head 路线的不同尺寸**。RT-2/OpenVLA 把离散动作写进语言 token 空间；Octo 用 block-wise transformer 的 readout token 条件化 diffusion action head；π0 用 VLM prefix 条件化约 300M continuous action expert，并通过 Flow Matching 生成 chunk。[1][2][3][4]
2. **sequence layout 同时决定“谁能看谁”和“谁被监督”**。attention mask 决定信息流，label/loss mask 决定梯度流；action token 出现在序列里不等于它参加 loss，反之 readout token 不必是 action label，也能通过独立 action loss 得到梯度。[2][3][5][6]
3. **冻结、全量微调、LoRA 与独立 action expert 是四种不同的容量—显存—遗忘权衡**。OpenVLA flagship 是全量微调；其官方 finetune 脚本支持 all-linear LoRA。Octo 官方 recipe 提供 `head_only`、`head_mlp_only`、`full`；openpi 同时给出 full 与 VLM/action-expert LoRA 配置。不能把“repo 支持 LoRA”说成“论文主结果由 LoRA 得到”。[2][5][7]
4. **异构 action space 必须同时有表示映射和有效维 mask**。Octo 是本批正例：`timestep_pad_mask & action_pad_mask` 进入 MSE/diffusion loss，推理又用 embodiment action dimension 限制

padded diffusion 维；openpi π0 frozen loss 对配置的全部 D 维求 mean，是此前审计的负例边界。[5][6][8]

5. **延迟不能只报一个 `infer_ms`**。至少要拆 capture/preprocess、vision/prefix、token decode 或 NFE、output transform、serialization/network、queue wait 与 controller tick。openpi 同时记录 model sampling、server infer 和上一请求 total time，可作为分层 instrumentation 起点，但这些字段仍不能自动分离视觉编码与每步 sampler。[7][9]
6. **真实推理合同不在论文标题里**。需要形成 `paper → repo commit → config/model class → checkpoint manifest → norm stats → input/output transforms → rollout client → controller` 的闭环；只核对模型权重或只看到 success rate 都不够。[2][5][7]
7. **动作执行是有序函数复合**：decode、denormalize、valid-dimension slicing、absolute/delta 恢复、轴序/坐标/单位映射、安全投影、queue/replan，任何一步换序都可能产生“loss 正常、rollout 0-success”。OpenVLA、Octo 与 openpi 都能证明 decode/unnorm 的一部分；本地 OpenPiVLA 的 axis、unit 和 controller 仍未取证。[2][6][7]
8. **最小闭环单测必须使用可辨识 sentinel，不是只检查 shape**。需要验证 parser/token 边界、norm round-trip、padding 不变量、轴序和单位、chunk FIFO、replan 边界、stale response、安全裁剪与一次 mock-controller execution。[6][7][10]

2. 版本、来源与证据边界

来源	冻结版本	本页用途	不能证明什么
RT-2 paper	arXiv `2307.15818v1`	PaLI-X/PaLM-E、256-bin action tokens、web+robot co-fine-tuning、cloud serving frequency	公开 repo 的具体 tensor/layout；本地模型实现
OpenVLA paper + repo	arXiv `2406.09246v3`；`c8f03f48af692657d3060c19588038c7220e9af9`	Prismatic-7B、action-only labels、full FT / LoRA、token decode 与 q01/q99 unnorm	本地 OpenPiVLA 是同一 checkpoint 或同一 tokenizer
Octo paper + repo	arXiv `2405.12213v2`；`241fb3514b7c40957a86d869fecb7c7fc353f540`，MIT	block-wise attention、readout/diffusion head、三种 FT mode、action padding mask、sample/unnorm	Octo 指标可与 RT-2/OpenVLA/π0 直接横比
π0 paper + openpi	arXiv `2410.24164v4`；`15a9616a00943ada6c20a0f158e3adb39df2ccac`，Apache-2.0	VLM + action expert、LoRA、output transforms、timing、websocket、chunk broker、DROID execution	私有 π0 data recipe；本地 OpenPiVLA 的 checkpoint/controller
本仓 Source Audits A/B/E	2026-07-16 canonical pages	OpenVLA token 端点、π0 time convention、FAST/flow latency、padding loss 边界	independent fact review 或个人项目事实

Octo clone 在本次审计中执行 `git rev-parse HEAD` 得到 `241fb351...`，最后提交日期为 2024-07-30；不能把“2026-07-16 访问到的 HEAD”误写成“2026 年新实现”。RT-2 本页只使用论文与项目页，不声称审计到官方训练源码。

3. VLA-ARCH-007：四条代表路线如何比较

3.1 landscape table

方法	backbone / condition encoder	action representation	原始训练路线	推理关键循环	面试边界
RT-2	PaLI-X 5B/55B 或 PaLM-E 12B	6-DoF pose delta、gripper、termination；连续维 uniform 256 bins 后变成整数 token	预训练 VLM 上用 robot action data 与原 web data co-fine-tune	自回归生成 action token；大模型由 multi-TPU cloud service 提供	两种 VLM tokenizer 的 action-token 占位方式不同；不是公开 OpenVLA code
OpenVLA	Prismatic-7B：DINOv2 + SigLIP → projector → Llama-2 7B	当前动作每个 scalar 一个 tail-vocabulary token	OXE robot data 上全量微调 7B；官方另有 LoRA finetune 脚本	生成 d 个 token → 255-center nuance decode → dataset q01/q99 unnorm	flagship 基础路径是单主视角、单步、无 proprio/history
Octo	27M/93M transformer；轻量 CNN image tokenizer + frozen T5-base language encoder	learned action readout → diffusion action head，默认公开 pretrain config 为 H=4、max D=7	800k OXE trajectories 预训练；新域可 head-only / head-MLP-only / full FT	transformer 一次产生 readout embedding；diffusion head 迭代去噪 chunk	它是 generalist robot policy，不是把大型生成式 VLM LM head 直接当 action head
π0	PaliGemma 2B VLM + 约 300M action expert	H×D continuous action chunk 的 flow velocity	跨 embodiment 预训练，再对目标数据 post-train；公开 openpi 提供 full/LoRA 示例	prefix KV 一次，action suffix 做多步 Euler/ODE 更新	paper data mixture 与公开 repo recipe 不等价；controller 在模型外

RT-2 的 action token 和 co-fine-tuning 证据来自论文 §3.2：连续维 uniform 256 bins；PaLI-X 复用整数 token，PaLM-E 覆写 256 个低频 token；web 与 robot data 同批混合并提高 robot sampling weight。[1] §3.3 报告论文部署下 55B 为 1–3 Hz、5B 约 5 Hz，这些是 cloud TPU protocol 的历史测量，不是现代单机基准。[1]

OpenVLA 的架构、全量微调与 code endpoint 已在 Source Audit A 核实。Octo 论文明确给出 27M/93M checkpoint、800k Open X-Embodiment trajectories、language/goal-image instruction、diffusion head 与 action chunk。[3] frozen pretrain config 将 T5-base encoder 设为 `finetune_encoder=False`，action readout 接 `DiffusionActionHead(H=4,D=7)`，并冻结 `*hf_model*`。[5]

π0 的 prefix/suffix、action expert 和 Flow Matching 已在 Source Audit B 与 Source Audit E 核实。比较时应对齐任务、机器人、数据、trial、controller 和 success predicate；上表只比较机制，不做成功率排名。

4. VLA-ARCH-008：sequence layout、attention 与 loss 语义

4.1 三种 layout 不能只画成同一条 token 串

```
# RT-2 / OpenVLA: action itself occupies autoregressive label positions.
[image patches / prompt tokens] -> [a_1 token] [a_2 token] ... [a_d token] [EOS?]

# Octo: action readout is a latent query; continuous action is supervised in a separate head.
[task prefix] [obs(t-1), readout(t-1)] [obs(t), readout(t)] -> diffusion H x D

# pi0: condition prefix and continuous noisy-action suffix use a custom block mask.
[image + language prefix] [state token + noisy action_1 ... noisy action_H] -> H x D velocity
```

代码块说明：三者都使用 transformer，但 action 的“位置”分别是 LM label、readout latent 和 continuous suffix latent。把它们都叫 action token 会掩盖监督目标与 attention 方向。

对自回归 token 路线，令 y_i 为第 i 个 token，$m_i \in \{0,1\}$ 为 label mask，则：

$$\mathcal{L}_{\mathrm{AR}} = -\frac{\sum_i m_i \log p_\theta(y_i \mid y_{<i}, c)}{\sum_i m_i + \epsilon}.$$

变量与实现：c 是图像/任务条件；causal attention 决定 y_i 可读取哪些前缀，m_i 决定哪些位置产生 CE 梯度。OpenVLA `RLDSBatchTransform` 把 prompt prefix 设为 `IGNORE_INDEX=-100`，默认保留 action tokens 与 stop token；插入的 image patches 也补 `IGNORE_INDEX`。[2] 因此 attention-visible 不等于 loss-visible。

Octo 的 frozen `octo_module.py` 将 task 作为 prefix；同 timestep 的 observation groups 互相注意并看历史；readout 可读取 task/observation/自身 readout，却不被 observation 写回。其源码注释甚至记录 v1.5 修复过 attention mask 的 off-by-one bug，说明 mask 不是展示性元数据。[5]

Octo 的 action loss 使用两类 mask：

$$M_{b,w,h,d} = T_{b,w} \wedge A_{b,w,h,d}, \qquad \mathcal{L}_{\mathrm{Octo}} = \mathrm{continuous_loss}(\hat{z}, z; M).$$

变量与实现：(T) 是 observation-history `timestep_pad_mask`，(A) 是 `action_pad_mask`；(b,w,h,d) 分别为 batch、history window、action horizon、model action dimension。frozen MSE 与 diffusion head 都先构造这个 conjunction，再调用 `continuous_loss`。[6] 这既阻止 padded history 产生监督，也阻止异构 embodiment 的 padded action dimensions 污染 target。

π0 / PioFAST 的 prefix/postfix attention、AR mask 与 token loss mask 已在 Source Audit B 精确展开；特别是 frozen Pio continuous loss 对配置 D 全维 mean，没有 Octo 式有效维 mask。面试时应说“不同实现的 mask 正例/负例”，不能泛称“VLA 都会自动 mask padding”。

5. `VLA-ARCH-009`：冻结、全量、LoRA 与 action expert 的取舍

5.1 它们分别在改什么

路线	更新参数	优点	主要风险	官方证据例
freeze backbone / head-only	只训 projector、readout 或 action head	显存小、语义先验稳定、少数据不易过拟合	domain gap 无法进入视觉/语言表征；head 容量成为瓶颈	Octo `head_only` 冻结 `octo_transformer.*`
head-MLP-only	冻结 transformer 与 readout attention，只训 head MLP	比 head-only 更强约束，便于快速适配 action space	不能改变 readout 如何从 context 聚合	Octo `head_mlp_only` 额外冻结 MAP probe/attention
full fine-tuning	更新 vision/projector/LLM/action modules	最大任务适配容量；可调整视觉细粒度特征	显存与通信大、灾难遗忘、少数据过拟合	OpenVLA flagship；Octo `full`；openpi LIBERO full config
LoRA / adapter	对选定线性层学习低秩增量	训练参数和 optimizer state 小，checkpoint 易管理	rank/target module 可能限制容量；base revision 与 adapter 必须配对	OpenVLA all-linear LoRA；openpi VLM + action-expert LoRA
independent action expert	保留大 VLM context，另训较小 continuous expert	robot action 的宽度/目标与 VLM 解耦；多步 flow 的单步计算较小	context bottleneck、expert 与 backbone 联调复杂	π0 2B VLM + 300M expert

LoRA 对冻结权重 $W \in \mathbb{R}^{o\times i}$ 学习低秩增量：

$$W' = W + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r\times i},\ B \in \mathbb{R}^{o\times r}.$$

变量与实现：$r \ll \min(i, o)$ 是 rank，α 是 scale。OpenVLA official finetune 用 PEFT `target_modules="all-linear"`；openpi LIBERO low-memory 配置把 PaliGemma 与 action expert 都换成 LoRA variant，并用匹配 config 的 `get_freeze_filter()` 冻结非 LoRA 权重。[2][7] 这证明“支持怎样适配”，不证明某个本地 checkpoint 实际采用该路线。

5.2 面试决策顺序

1. 先看 dataset size/diversity、视觉 domain shift、语言变化、action schema 变化和显存预算。
2. 若只是新 action 维度、视觉近似且数据少，先 head-only / adapter；若 camera、objects、dynamics 变化明显，考虑解冻视觉/projector。
3. 若语义与控制都显著变化且有足够数据，再 full FT；同时保留 upstream/task eval 监控遗忘。
4. 对 LoRA 必报 base checkpoint revision、target modules、rank、merge 状态和加载顺序。
5. 对 action expert 必报 context interface、expert width、训练 target、是否共同更新 VLM，以及 sampling NFE。

6. `VLA-ARCH-010`：跨 embodiment action 统一与 padding loss

设 robot e 的真实动作 $a^{(e)} \in \mathbb{R}^{d_e}$，模型统一宽度为 $D \geq d_e$。用 embedding matrix $P_e \in \{0,1\}^{D\times d_e}$ 写入统一空间，用 selection matrix $S_e = P_e^\top$ 取回：

$$\bar{a}^{(e)} = P_e a^{(e)}, \qquad \hat{a}^{(e)} = S_e \hat{\bar{a}}, \qquad m_j^{(e)} = \mathbb{1}[j \in \mathrm{Im}(P_e)].$$

变量与实现：P_e 只能表示“维度放到哪里”，不能自动统一 axis、unit、absolute/delta、rotation 或 gripper semantics；这些还需要 schema transform。openpi `PadStatesAndActions` 只做 zero-pad，不能据此假设 loss mask 存在。[7]

正确的 masked continuous objective 应明确 normalization 和有效维：

$$\mathcal{L}_{mathrmvalid} = \frac{\sum_{b,h,j} m_{b,h,j} \ell(\hat{\bar{a}}_{b,h,j}, \bar{a}_{b,h,j})}{\sum_{b,h,j} m_{b,h,j} + \epsilon}.$$

变量与实现：m 同时排除 padding timestep 与 padding dimension；Octo 的 `action_pad_mask` 是直接正例。[6] 若对全部 D 维 mean，zero padded axes 也会产生“逼近 0”的监督，并随 d_e/D 改变有效维梯度占比。此前 frozen π0 审计只看到 all-D mean，因此本地若声称避免 padding loss，必须提供自己的 mask path。

6.1 schema 统一检查表

字段	必须记录	常见错误
translation	x/y/z axis、frame、m 或 cm、absolute/delta/velocity	只按列号 pad，未统一 frame/unit
rotation	Euler/quaternion/6D、顺序、弧度/角度、delta composition	quaternion sign、Euler order 或单位不一致
gripper	position/velocity/binary、open polarity、range	把 continuous gripper 当普通 z-score 维
base	vx/vy/yaw-rate 或 pose、body/world frame	mobile-base 维与 arm 维错位
timestep	control frequency、label horizon、timestamp	同一个 delta 在不同 dt 下物理量不同
mask	timestep/action/modality valid mask	只 pad input，不 mask target/loss

7. `VLA-ARCH-011`：延迟如何分层而不是猜瓶颈

对一次远程 chunk query，端到端延迟写为：

$$T_{\mathrm{e2e}} = T_{\mathrm{capture}} + T_{\mathrm{pre}} + T_{\uparrow} + T_{\mathrm{vision/prefill}} + N\,T_{\mathrm{decode/step}} + T_{\mathrm{post}} + T_{\downarrow} + T_{\mathrm{queue}}.$$

变量与实现：(N=d) 可表示 OpenVLA/RT-2 的 action-token 数；对 PioFAST 是可变 FAST token length；对 flow/diffusion 是 NFE/denoise steps。`vision/prefill` 是否可缓存、每步是否只跑 action expert，取决于真实实现。openpi `Policy.infer` 的 `policy_timing.infer_ms` 只包围

`sample_actions`；server 的 `server_timing.infer_ms` 包围完整 policy infer，`prev_total_ms` 还包括 recv/unpack/send 的上一轮总时长。[7][9]

RT-2 论文的 55B 1–3 Hz / 5B 约 5 Hz 包含其 cloud serving protocol，说明 network 不能从大模型部署中忽略，但不能直接外推 openpi。FAST 论文在指定 4090 实现下报告 π0 flow 约 100 ms/chunk、π0-FAST 约 750 ms/chunk；解释是后者用 2B backbone 自回归生成约 30–60 tokens，前者做 10 次较小 expert step。[11] 这同样只是 source-specific measurement。

7.1 可执行 timing pseudocode

```
def timed_policy_query(raw_obs, client, policy, controller):
    # Synchronize device events; otherwise asynchronous GPU work makes timers invalid.
    t0 = sync_clock()
    obs = preprocess_and_resize(raw_obs)
    t_pre = sync_clock()

    # Measure serialization/transport separately from server-side model timing.
    response = client.infer(obs)
    t_roundtrip = sync_clock()

    # Read server/model fields, then time local parse, denorm, slicing and safety.
    model_ms = response["policy_timing"]["infer_ms"]
    server_ms = response["server_timing"]["infer_ms"]
    action_chunk = parse_and_validate(response["actions"])
    env_actions = output_transform_and_safety(action_chunk)
    t_post = sync_clock()

    # Queue wait and controller step need their own timestamp at the consumer side.
    controller.enqueue(env_actions)
    return make_timing_record(t0, t_pre, t_roundtrip, t_post, model_ms, server_ms)
```

代码块说明：真实 CUDA/JAX 计时需 device synchronization 与 warm-up；至少报告 p50/p95、payload bytes、H/K、NFE/token length、batch、dtype、device、是否缓存 prefix。只报一次 wall clock 不足以定位瓶颈。

8. `VLA-ARCH-012`：paper / repo / checkpoint / I/O 证据审计

8.1 四层证据不能互相替代

层	要回答的问题	最小 artifact	典型误判
paper	方法声称什么、实验 protocol 是什么	版本号、section/table、公式	用 paper 抽象当 frozen code 细节
repo	某 commit 实际实现什么	commit、license、model class、loss/infer path	用 main HEAD 解释旧 checkpoint
checkpoint	权重对应什么模型与资产	manifest/config、weight shape/hash、tokenizer、norm stats	只凭目录名猜 backbone/H/D
runtime	真正收发和执行什么	startup log、request/response schema、transforms、rollout trace	模型 output shape 对就认为 controller 正确

openpi `create_trained_policy` 是正例：先从 checkpoint 判断 JAX/PyTorch weight，再由 training config 构造 model/data config；norm stats 默认从 checkpoint `assets/<asset_id>` 读取，随后固定

input transforms → normalize → model transforms，以及 model outputs → unnormalize → data/repack outputs 的顺序。[7] OpenVLA 的 `unnorm_key` 同样决定生成 token 数和 q01/q99 stats；多 dataset checkpoint 不显式选择 key 会报错。[2]

```
def audit_vla_release(paper, repo, checkpoint, runtime_sample):
    # Freeze provenance before reading implementation claims.
    assert paper.version and repo.commit and repo.license

    # Reconstruct the model from the exact checkpoint config, not a similarly named default.
    manifest = load_checkpoint_manifest(checkpoint)
    assert hash_file(checkpoint.weights) == manifest.weight_sha256
    assert manifest.model_class == runtime_sample.startup_model_class

    # Verify tokenizer, normalization assets and tensor contract as one bundle.
    assert manifest.tokenizer_revision == runtime_sample.tokenizer_revision
    assert manifest.norm_asset_id == runtime_sample.norm_asset_id
    assert runtime_sample.output_shape == (manifest.horizon, manifest.action_dim)

    # Trace one sentinel action through decode -> unnorm -> schema -> controller.
    trace = replay_sentinel(runtime_sample)
    assert trace.controller_action == trace.expected_controller_action
    return trace
```

代码块说明：这是审计框架，不声称本地已有这些字段。对 OpenPiVLA，checkpoint、Qwen/π model class、tokenizer revision、norm asset、H/D/K、axis/unit、rollout trace 与 97.4% eval summary 仍待本地 artifact。

9. `VLA-ACT-031`：模型输出到 controller 的有序链

9.1 一般形式

令模型原始输出为 z，token/FAST/flow decoder 为 $\mathcal{D}$，反归一化为 $\mathcal{U}$，有效维选择为 S_e，absolute/delta 与 frame/unit 映射为 $\mathcal{R}_e$，安全投影为 $\Pi_{\mathcal{A}_e}$：

$$a^{\mathrm{ctrl}} = \Pi_{\mathcal{A}_e}(\mathcal{R}_e(S_e\,\mathcal{U}(\mathcal{D}(z)))).$$

变量与实现：$\mathcal{A}_e$ 是 robot/controller 允许的动作集合。函数顺序不能随意交换：例如在 normalized space clip 与在物理 velocity space clip 的含义不同；delta action 必须在正确 state/frame 上恢复。openpi 的 `Policy.infer` 明确在 model sampling 后执行 output transform，`policy_config` 把 `model_transforms.outputs`、`Unnormalize`、dataset outputs 和 repack outputs 依次组合。[7]

具体公开实现边界：

- OpenVLA：生成数量由 dataset stats 的 action dim 决定；token decode 后按 q01/q99 与 optional mask unnormalize。它没有证明 controller axis/unit。[2]
- Octo：`sample_actions` 用 `embodiment_action_dim=len(stats["mean"])` 限制 diffusion padded 维，再 slice 到 stats 长度并用 mean/std 或 p01/p99 unnormalize。[6]

- openpi：FAST path 先 `ExtractFASTActions`；continuous/FAST 共同经过 checkpoint norm stats 的 `Unnormalize`；`AbsoluteActions` 可按 mask 把 delta 加回 state；具体 data output transform 还可能重排 robot schema。[7]
- openpi DROID example：server 返回 `[10,8]`，client 取 chunk 当前行、二值化 gripper、clip 到 `[-1,1]`，再 `env.step`，并按数据采集频率 sleep。[10] 这是 DROID example 的合同，不是通用 controller 标准。

9.2 axis/unit 映射必须显式

若 dataset delta a^{data} 与 controller action 的轴序/单位关系为矩阵 R_e 和 scale s_e，且部分维为 absolute 恢复，则可写：

$$a^{\text{abs}}_{t:t+H-1} = a^{\text{data}}_{t:t+H-1} + m_\Delta \odot q_t, \qquad a^{\text{ctrl}} = s_e \odot R_e a^{\text{abs}}.$$

变量与实现：m_Δ 只在 delta dimensions 为 1，q_t 是当前 state；真实 rotation 可能需要群上的 composition，不能总用向量加法。openpi `AbsoluteActions` 只对 mask 覆盖的前 dims 加 state，是可核代码例；本地 axis matrix、unit scale 与 rotation semantics 仍需 controller wrapper 证明。[7]

10. `VLA-ACT-032`：parser、queue、replan 与 safety 的最小闭环测试

10.1 必测矩阵

Test ID	输入 sentinel / 扰动	应检查的不变量	能抓住的错误
T1 parser boundary	最小/最大合法 token、EOS、越界 token、短 chunk	非 action token 被拒绝；shape 恰为 H×D	vocab offset、EOS 混入、silent reshape
T2 norm round-trip	每维 `q01/mean/q99` 与 gripper mask	`unnorm(norm(a))≈a`；masked dim 原值保留	stats key/revision、广播维错误
T3 padding sentinel	valid dims 放互异质数，padding 放大常数	slice 后只保留 d；loss 对 padding sentinel 不变	padding loss 污染、wrong-axis slice
T4 frame/unit	x/y/z/rot/gripper 各放独立 one-hot	controller 对应轴、符号、单位精确匹配	frame rotation、cm/m、degree/radian、gripper polarity
T5 FIFO chunk	chunk 行编码为 step index 0..H-1	连续 K 次输出严格 0..K-1，无重复/跳步	off-by-one、queue reset 错误
T6 replan	第 K 步注入新观测/新 chunk	只在定义边界切换；旧 chunk 不泄漏	H/K 混淆、stale chunk
T7 stale/RPC	延迟、乱序、断连、重复 response ID	超时 fail-safe；旧 response 不覆盖新 plan	网络乱序与无 request id
T8 safety	NaN/Inf/超限 velocity/workspace pose	reject 或投影；记录原值与触发规则	`np.clip` 吞掉 NaN、错误空间裁剪

Test ID	输入 sentinel / 扰动	应检查的不变量	能抓住的错误
T9 mock controller	完整 request → one mock `step`	controller 收到预期 dtype/shape/value/time	单模块都通过但组合换序

openpi frozen `ActionChunkBroker` 只在 cache 为空时 query inner policy，按 `_cur_step` slice，并在达到 configured action horizon 后清空；manual test 只验证每步 shape。[9] 这提供最小行为实现，但没有覆盖 request ID、stale response、NaN、axis/unit 或 safety。因此项目级 test 必须扩展。

```
def test_closed_loop_contract(policy_stub, parser, output_transform, broker, mock_env):
    # Each row and dimension is unique, so transpose/slice/queue bugs cannot hide behind zeros.
    sentinel = make_prime_sentinel_chunk(horizon=4, model_dim=8, valid_dim=7)
    policy_stub.return_value = {"actions": sentinel, "request_id": 42}

    # Execute K=2 rows; the broker must consume rows 0 and 1 exactly once.
    a0 = broker.infer(make_observation(request_id=42))["actions"]
    a1 = broker.infer(make_observation(request_id=42))["actions"]
    assert not_equal(a0, a1)

    # Parse, unnormalize, slice, map axes/units and apply safety in the declared order.
    c0 = output_transform(parser(a0))
    c1 = output_transform(parser(a1))
    assert c0.shape == c1.shape == (7,)
    assert all_finite_and_within_limits(c0, c1)

    # A mock controller checks the final physical contract, not only intermediate tensors.
    mock_env.step(c0)
    assert mock_env.last_action == expected_controller_sentinel(row=0)
```

代码块说明：使用质数/one-hot sentinel 是为了让 transpose、错轴、padding 与重复行产生可观察差异；真实测试还需固定 norm stats、frame transform、controller mode 和 control dt。

11. Claim-to-code matrix

Claim	paper	frozen repo	判定	关键证据
RT-2 有 PaLI-X/PaLM-E 两个 backbone family	direct	本页未审计 repo	paper-direct	RT-2 §3.1 / appendix
RT-2 action 连续维为 uniform 256 bins 并映射成 token	direct	本页未审计 repo	paper-direct	RT-2 §3.2
RT-2 主 recipe 混合 web + robot co-fine-tuning	direct	本页未审计 repo	paper-direct	RT-2 §3.2
OpenVLA flagship 是 DINOv2+SigLIP / Llama-2 7B，full FT	direct	direct	source-ready / WARN	Source Audit A
OpenVLA loss 只监督 action tail 与可选 stop，patch labels ignore	paper overview	direct	source-ready	`datasets.py` / `modeling_prismatic.py`
Octo 是 27M/93M transformer generalist policy，800k OXE	direct	checkpoint config compatible	source-ready	Octo abstract / §III
Octo block-wise attention 的 readout 只读 condition	direct	direct	source-ready	`octo_module.py`
Octo diffusion loss 排除 padded timestep/action dims	method overview	direct	source-ready	`action_heads.py`

Claim	paper	frozen repo	判定	关键证据
Octo 支持 head-only/head-MLP-only/full FT	paper + docs	direct config	source-ready	`finetune_config.py`
π0 是 2B VLM + 300M action expert continuous flow	direct	direct config/path	source-ready / WARN	Source Audits B/E
openpi 支持 full 与 LoRA LIBERO adaptation	不属于 π0 paper 主结果	direct	repo-supported option	`training/config.py`
openpi output transforms 从 model decode 到 unnorm/data outputs 有序组合	architecture overview	direct	source-ready	`policy_config.py`
openpi timing 已完全拆分 vision、expert、network 与 queue	no	no	rejected	当前只有 model/server/previous total 三层
本地 OpenPiVLA 采用上述任一 checkpoint/H/D/K/controller	no	no	unverified	需要本地 artifacts

12. 公源 / 本地项目边界

本地 claim	最小证据	当前状态
base 是 OpenVLA、Pio、PioFAST、π0.5 或自定义 Qwen route	startup config、model class、checkpoint manifest、forward/loss path	pending
frozen/full/LoRA/action-expert adaptation	train config、trainable-parameter log、adapter/base revisions	pending
H/D/K 与 action schema	dataset target、model output、queue/replan config、controller wrapper	pending
padding 不污染 loss	actual loss mask、synthetic padding sentinel test	pending
end-to-end latency	synchronized p50/p95 timing ledger + payload/device/NFE/token length	pending
decode/denorm/slice/axis/unit 正确	tokenizer/norm assets、output transform、mock controller trace	pending
97.4% 与 checkpoint 对应	eval summary、checkpoint ID/hash、task/trial/split/seed/protocol	pending
personal contribution	local diff、设计记录、run ledger、ownership boundary	pending

本页所有架构、loss、transform 与 serving 实现都属于上游公开设计。除非本地 diff 与运行 artifact 另行证明，不得写成本人的原创贡献，也不得用 upstream example 的 `[10,8]`、H=50 或某个 norm asset 推断本地值。

13. 题目覆盖与剩余验证

Question ID	本页支持	状态	仍缺
VLA-ARCH-007	RT-2/OpenVLA/Octo/π0 landscape	independent-fact-reviewed / WARN	local architecture mapping
VLA-ARCH-008	token/readout/suffix layout、attention/loss masks	independent-fact-reviewed / WARN	local layout sentinel test
VLA-ARCH-009	freeze/full/head/LoRA/expert decision	independent-fact-reviewed / WARN	target hardware/data ablation
VLA-ARCH-010	unified D/d、Octo positive mask、openpi negative boundary	independent-fact-reviewed / WARN	local mask evidence
VLA-ARCH-011	latency equation、RT-2/FAST context、openpi timings	independent-fact-reviewed / WARN	local synchronized timing
VLA-ARCH-012	paper→repo→checkpoint→runtime audit	independent-fact-reviewed / WARN	local manifest and trace
VLA-ACT-031	decode→unnorm→slice→schema→safety→controller	independent-fact-reviewed / WARN	local axis/unit/controller
VLA-ACT-032	parser/queue/replan/safety test matrix	independent-fact-reviewed / WARN	local executable tests

RT-2/Octo 论文措辞、Octo `continuous_loss` denominator、openpi output transform、broker 与 DROID example 已在 独立事实复核页 核对。下一步只进行一题真实口述或补本地 manifest/trace；所有 8 张卡继续 `unseen`。

14. 参考文献（References）

1. Brohan et al., RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control, arXiv v1, 2023；§3.2 Robot-Action Fine-tuning；§3.3 Real-Time Inference。
2. Kim et al., OpenVLA, arXiv v3；`openvla/openvla@c8f03f48`；Source Audit A。重点代码：`datasets.py#L44-L67`、`modeling_prismatic.py#L388-L410`、`finetune.py#L98-L103`、`#L172-L182`、`openvla.py#L80-L103`。
3. Octo Model Team et al., Octo: An Open-Source Generalist Robot Policy, arXiv v2, 2024；§III-A Architecture、§III-C Training Objective。
4. Black et al., π0, arXiv v4；Source Audit B；Source Audit E。
5. `octo-models/octo@241fb351`, MIT. 重点：`octo_module.py#L24-L63`、`#L138-L168`、`#L241-L268`、`octo_pretrain_config.py#L45-L62`、`#L107-L125`。
6. Octo action contract：`action_heads.py#L175-L207`、`#L466-L526`、`#L540-L610`、`octo_model.py#L174-L250`。
7. `Physical-Intelligence/openpi@15a9616a`, Apache-2.0. 重点：`policy_config.py#L16-L64`、`#L75-L94`、`transforms.py#L115-L181`、`#L204-L244`、`#L328-L337`、

`training/config.py#L651-L698`。

8. Source Audit E：Flow Matching / π0，frozen Pio all-D mean 与有效维 mask 边界。
9. openpi serving / queue：`policy.py#L68-L105`、`websocket_policy_server.py#L48-L72`、`websocket_client_policy.py#L46-L54`、`action_chunk_broker.py#L10-L50`。
10. openpi DROID example：`examples/droid/main.py#L113-L156`；manual shape/broker test：`policy_test.py#L9-L34`。
11. Pertsch et al., FAST, arXiv v1；latency 条件与路径已在 Source Audit B 核对。
12. VLA 架构、Mask、适配与执行独立事实复核，RT-2/Octo/openpi direct source hashes 与 synthetic probes，2026-07-19。

Part 6 / 16 · `interview_materials/vla/sources/vol_03_architecture_execution_independent_fact_review.html` · SHA-256 `65942F733EAA17B9AC444DF27C6A70CB04E13051B7D77452C8DA1A587B9A2F6E`

返回本卷目录

Canonical Markdown SHA-256：`65942f733eaa17b9ac444df27c6a70cb04e13051b7d77452c8da1a587b9a2f6e`；生成自：`interview_materials\vla\sources\vol_03_architecture_execution_independent_fact_review.md`

VOL 03 Source Audit F：VLA 架构、Mask、适配与执行独立事实复核

状态：`independent_fact_review / landscape_mask_transform_queue_checked / WARN` 审查日期：2026-07-19 范围：RT-2、OpenVLA、Octo、π0/openpi 的架构路线、sequence/mask、适配、checkpoint/norm/transform、timing、chunk broker 与 DROID controller example。 结论先行：四条路线确实使用不同的 action 监督合同；Octo 的 `timestep_pad_mask & action_pad_mask` 是有效维 mask 的正例，openpi 的 `Policy.infer` 明确执行 checkpoint norm stats 与 input/output transform 链，`ActionChunkBroker` 只在缓存耗尽时重新 query，DROID example 的 `[10,8]` 与 `open_loop_horizon=8` 是特定 example contract。RT-2 的 1–3 Hz/5 Hz 是 cloud-serving 测量，不能迁移为本地 latency；OpenVLA/Octo/π0 上游实现也不能证明 OpenPiVLA 的 checkpoint、axis/unit、controller 或个人贡献。

1. 直接复核来源与冻结对象

ID	来源	本次核对	SHA / Verdict
Po1	RT-2 v1 PDF	§3.2 action 256 bins/token、PaLI-X/PaLM-E tokenization、co-fine-tune、cloud serving frequency	`0A62DC36BBBFDEA232AD45A0AB75C82E9CF535B55333FF22C02D5DA1897CCAC` / `primary-verified`
Po2	Octo v2 PDF	readout/block-wise mask、diffusion head、27M/93M、Open X-Embodiment mixture、20-step head	`73BFF297CFAFE523319162124E6B7F96919C0930E0A380F307255C4C7464AC93` / `primary-verified`
Co1	`octo_module.py`	timestep padding、observation/task/readout token masks	`E458062721FD1E72FAD00F7833B749E82D0DF60AA7927557362BDF6463299FBF` / `official-code`
Co2	`action_heads.py`	`masked_mean`、timestep/action conjunction、embodiment dimension slice、diffusion mask	`480F374788571DB11F1E3CA2922BEAEC52F752001AE5AC6D284E0F1DFEC40D11` / `official-code`
Co3	`octo_pretrain_config.py`	default `action_horizon=4`, `action_dim=7`	`531A2F056EBCF995FE37DFB99634354D58F167F423C1A97BDF3C5545E425E0DD` / `official-code`
Co4	`octo_model.py`	sample action output and `embodiment_action_dim` /unnormalization path	`93D96015BC43EE92910127421E17697D372F0BBC1A4D3589A0ADDB22EFB819E0` / `official-code`
Co5	`openpi/policy.py`	input transform、model timing、output transform、`policy_timing.infer_ms`	`A2AC5236E23B11D0F5A840D4A8FEF148D89DBA921D5452EC5DFCD5A6FFA46F20` / `official-code`

ID	来源	本次核对	SHA / Verdict
Co6	`policy_config.py`	checkpoint type detection、model/config load、checkpoint norm stats、ordered transforms	`AAF42AB04A33B6C91D2447926211646C33EBD8EBFA427F026D7B6C4F7C45EC52` / `official-code`
Co7	`transforms.py`	Unnormalize、Delta/Absolute action transforms、pad/repack boundary	`A1B94E9E72849A18834778F229C6BB389A495EB7FBE0AA800EDEA728B9424FF4` / `official-code`
Co8	`training/config.py`	full/LoRA examples、action horizon/dim overrides、delta masks	`C34231F888DBA5DC981D808765B3D98775AB845B04313BF53FCECD567C520B0D` / `official-code`
Co9	`action_chunk_broker.py`	cache slice、requery at exhaustion、reset	`019B9D69CB26CC681331A2F95DBE60A2CC7B02D6D309C5D15B6457AB39798234` / `official-code`
C10	`examples/droid/main.py`	`open_loop_horizon=8`、`[10,8]` chunk、gripper binarize、clip、control frequency	`4BDBDBDDE068EBB30BA90D62BF8C51DF156B4BAAE5C5AC5870C08F7B03D4AFC2` / `official-code`
C11	`policy_test.py`	broker shape/half-chunk smoke semantics	`9DBB20963600367A7E7ABFC97D1E326916777C57A87B39B1BC4070BA64676683` / `official-code`
Io1	本仓 Source Audits A/B/E	OpenVLA endpoint、π0/FAST、Flow time/sign 与 all-D mask 边界	`internal-source-reviewed`
Oo1	本次 synthetic probes	Octo mask mean、openpi delta/absolute round-trip、broker requery、DROID prefix	`local-evidence / PASS`

冻结版本：Octo commit `241fb3514b7c40957a86d869fecb7c7fc353f540`（MIT）、OpenPI commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`（Apache-2.0）。本页不把 repo example、Git author、上游 checkpoint 或公开 metric 归为个人贡献。

2. Landscape：action 监督合同不同于模型规模

路线	action 表示	监督/生成	本次可核事实	不可推出
RT-2	6-DoF/夹爪/终止等维度 uniform 256 bins → token	VLM next-token + robot/web co-fine-tune；动作 token 受限采样	§3.2 明确 256 bins、PaLI-X 与 PaLM-E 使用不同 token 复用方式	本地 tokenizer、controller、单机 latency
OpenVLA	每个 scalar 使用尾部 action token	action label 的 causal CE；decoder unnorm 依 dataset stats	见 Audit A；旗舰路径与 LoRA 脚本需分开	本地 Qwen route 或个人改造
Octo	learned readout → continuous diffusion action head	transformer 一次 forward，multi-step denoise 在小 head 内完成；默认 pretrain config `H=4,D=7`	论文与 frozen config/readout/head 直接支持	Octo 指标可跨协议比较、任意 embodiment 都是 7D
π0/openpi	continuous $H \times D$ action expert 或 FAST tokens	VLM prefix + flow/FAST；输出 transform 与 broker 在模型外	`Policy`/transforms/broker/DROID example 直接支持	本地 checkpoint/H/K/axis/unit

RT-2 论文的 55B 1–3 Hz、5B 约 5 Hz 是其 cloud-serving protocol 的历史测量；它们不是 OpenVLA/π0 或本地 OpenPiVLA 的 hardware benchmark。Octo 的 27M/93M、800k OXE 和 20 diffusion steps 属于公开方法/recipe边界，不代表本地训练配置。

3. Sequence、attention mask 与 loss mask

3.1 Octo 的 readout 与 block-wise mask

Octo 的 `octo_module.py` 将 `timestep_pad_mask` 标记为 `(batch, horizon)`，`False` 表示 padding timestep。观察 token 的 mask 会结合 tokenizer mask 与 timestep mask；task token 有自己的 pad mask；readout token 使用独立的 readout mask。论文同时说明：observation token 只能因果读取同一或更早 timestep 与 task token，readout 能读取前面的 observation/task/readout，但 observation/task 不读取回 readout。

这意味着：

- attention-visible：某 token 是否能读取另一个 token；
- timestep-valid：该 observation window 是否真实存在；
- action-valid：该 embodiment 的 action dimension 是否存在；
- loss-visible：该元素是否进入 continuous/diffusion loss。

它们不是同一个 boolean。

3.2 Octo 的有效维 loss

`action_heads.py` 的 `masked_mean` 先将 mask broadcast 到 loss shape，再计算：

$$\text{masked_mean}(x, m) = \frac{\text{mean}(x \odot m)}{\max(\text{mean}(m), 10^{-5})}.$$

变量与直觉：当 (m) 是 0/1 mask 时，这等价于只对有效元素平均（只要有效比例不为 0）。连续 head 与 diffusion head 都构造：

$$M_{b,w,h,d} = T_{b,w} \wedge A_{b,w,h,d},$$

再将 (M) 传入 loss；推理 `sample_actions` 可用 `embodiment_action_dim` 把 padded diffusion dimensions 的 mask 设为 False，并最终 slice 到 stats 长度。

这与此前 frozen Pio 的 all-D mean 是有意保留的正例/负例对照；不能泛称“所有 VLA 都正确 mask padding”。

4. 适配方式：full、head-only、LoRA、action expert

Octo paper/repo 提供轻量 diffusion head 与 readout 结构，公开配置支持更换 observation/task/action head；其 finetune choices 与 OpenVLA/openpi 的 LoRA 例子说明的是

“repo 支持的 adaptation route”，不是某个 paper 主结果默认使用 LoRA。

openpi `training/config.py` 直接列出：

- full finetune 配置（例如 `pi0_fast_full_droid_finetune`）；
- PaliGemma 与 action expert 的 LoRA variant；
- action dimension/horizon 随 dataset 调整的配置。

LoRA 的事实仍需按 base checkpoint、target modules、rank、merge state 和 optimizer state 绑定。看到 `lora` 字段不能证明本地 checkpoint 真的用该 config。

5. Checkpoint → norm → transform → runtime 合同

openpi `create_trained_policy` 的顺序是可直接核验的：

1. 检查 checkpoint 是否有 `model.safetensors`，据此选择 PyTorch/JAX load path；
2. 用 training config 构造 model/data config；
3. 若未显式传入 norm stats，从 checkpoint `assets/<asset_id>` 读取；
4. input transforms：repack → prompt → data transforms → normalize → model transforms；
5. `Policy.infer` 调用 `sample_actions`；
6. output transforms：model outputs → unnormalize → data outputs → repack outputs。

normalization 的 z-score 反变换为：

$$x_{\text{physical}} = x_{\text{model}}(\sigma + 10^{-6}) + \mu.$$

quantile path 则用 q01/q99 做 ([-1,1]) 到物理区间的逆映射。`DeltaActions` / `AbsoluteActions` 按 boolean mask 对前若干维加减 state；这只完成 declared transform，不自动决定 axis、frame、rotation 或 gripper semantics。

6. Timing：`infer_ms` 不是端到端 latency

openpi `Policy.infer` 的 `policy_timing.infer_ms` 只包围 `_sample_actions`，不包括输入 transform、网络传输、server serialization 或 controller queue。要得到端到端值，仍需另记：capture、preprocess、upload、server/model、download、parse/unnorm、queue wait 和 controller tick。一次上游字段不足以证明视觉编码、action expert NFE 或网络分别耗时多少。

7. Chunk broker 与 DROID example

`ActionChunkBroker` 的行为是：缓存为空时调用 inner policy；每次返回当前 `_cur_step` 行；当 `_cur_step >= action_horizon` 时清空缓存；`reset()` 清空 policy 与 cache。它假设所有 array action fields 第一维是 chunk length，但不做 request-id/stale-response/NaN/axis/unit 安全检查。

DROID example 另有一层 rollout contract：

```
# Query a fresh chunk when the configured execution prefix is exhausted.
if actions_from_chunk_completed == 0 or actions_from_chunk_completed >= open_loop_horizon:
    pred_action_chunk = policy_client.infer(request_data)["actions"]  # [10, 8]

# Execute only the selected prefix row, then binarize gripper and clip.
action = pred_action_chunk[actions_from_chunk_completed]
action[-1] = 1.0 if action[-1] > 0.5 else 0.0
action = np.clip(action, -1, 1)
env.step(action)
```

这里的 `[10,8]` 表示 10 行 action、8 个维度（7 joint velocity + 1 gripper position）；`open_loop_horizon=8` 表示执行 8 行后再 query，剩余 2 行不一定被使用。这是该 example 的 H/K 合同，不是 openpi 的 universal default，也不是本地 OpenPiVLA 证据。

8. Synthetic probes

Probe	结果	结论
Octo `x=[1,3,5,7]`，mask `[1,1,0,0]`	masked mean `2.0`	action/timestep mask 可排除 padded dims
Octo `T=[1,0]` 与 `A` 全 valid	合并 mask 第二 window 全 0	padded timestep 不产生 loss
openpi z-score round-trip：`mu=[10,20]`，`std=[2,4]`，`x=[14,12]`	normalize `[2,-2]` 后 inverse `[14,12]`（浮点误差内）	norm asset 必须匹配
`DeltaActions` / `AbsoluteActions` mask `[1,0,1]`，state `[10,20,30]`，delta `[1,2,3]`	abs `[11,2,33]`，再 delta `[1,2,3]`	transform 顺序/掩码可单测
`ActionChunkBroker(action_horizon=2)`，inner chunk rows `[0,1,2,3]`	输出 `0,1` 后第二次 query；reset 清空 cache	broker execution prefix 与 model chunk 可不同
DROID example chunk `[10,8]`，`open_loop_horizon=8`	只消费 rows `0..7`，第 8 次边界 query	example 不等于执行完整 H

这些 probes 验证 mask、transform、broker 与 example branch semantics，不证明任何 learned policy success、latency 或本地项目实现。

9. 对 `VLA-ARCH-007..012` 与 `VLA-ACT-031..032` 的回写结论

Question ID	独立复核结论	必须保留的边界
`VLA-ARCH-007`	RT-2/OpenVLA/Octo/π0 landscape 与 action contract 可核	不做跨 protocol 指标排名
`VLA-ARCH-008`	attention/readout/loss/action mask 分层可核	本地 mask 未证
`VLA-ARCH-009`	full/head/LoRA/action expert 的 repo support 可核	不把 support 当本地 choice/个人设计

Question ID	独立复核结论	必须保留的边界
`VLA-ARCH-010`	Octo conjunction mask 与 openpi all-D 对照可核	本地 D/d/valid mask 未证
`VLA-ARCH-011`	RT-2/FAST/openpi timing scopes 可核	不迁移 source latency 到本地
`VLA-ARCH-012`	paper→repo→checkpoint→runtime 五层审计合同可核	本地 manifest/trace 仍缺
`VLA-ACT-031`	decode→unnorm→slice→schema→controller 顺序可核	本地 axis/unit/controller 未证
`VLA-ACT-032`	parser/FIFO/replan/stale/safety test matrix 可核	上游 broker test 只覆盖 shape，项目 tests 未执行

10. 项目真实性边界与安全口径

上游 facts 可以帮助面试回答“为什么 mask、为什么要把 norm stats 和 transform 绑到 checkpoint、为什么 action chunk 的 H 与执行 prefix K 分开”。它们不能证明 OpenPiVLA 采用 Octo mask、openpi broker、DROID `[10,8]`、某个 LoRA config、RT-2 latency 或任何 upstream checkpoint。

30 秒安全回答

“VLA 的 action head 不能只按 backbone 名字分类：RT-2/OpenVLA 是离散 token，Octo 是 readout 条件化的 diffusion head，π0 是 VLM prefix 条件化的 continuous action expert。Octo 直接把 timestep padding mask 和 embodiment action mask 合并进 loss，而 openpi 的 runtime 还必须把 checkpoint、norm stats、input/output transforms 和 broker 一起审计。DROID example 里模型输出 `[10,8]`，但只按 `open_loop_horizon=8` 消费，说明 model chunk 和 execution prefix 不是一回事。对我的项目，我只有在本地 manifest、parser、norm、axis/unit、queue 和 controller trace 齐全时，才把这些上游机制映射成项目事实。”

11. 风险、未决项与下一步

1. **核心 public facts 已独立复核**：RT-2 action token、Octo readout/mask/diffusion、openpi transform/broker/timing/DROID example 已核对。
2. **视觉 QA**：`file://` 浏览器策略限制仍记录为 `BLOCKED`；静态 HTML 校验不等价于视觉验收。
3. **本地项目 gate**：OpenPiVLA checkpoint、tokenizer/norm assets、D/d/H/K、axis/unit、controller trace、latency ledger 与 ownership 仍待本地证据。
4. **下一 bounded slice**：进行一题 `VLA-ARCH-010/031` 或 OpenPiVLA ownership 真实口述；不新增题卡、不创建 `coaching_state.md`。

参考文献（References）

1. Brohan et al., RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control，arXiv `2307.15818v1`。
2. Kim et al., OpenVLA 与本仓 Source Audit A。

3. Octo Model Team et al., Octo: An Open-Source Generalist Robot Policy，arXiv `2405.12213v2`。
4. Black et al., π0 与本仓 Source Audits B/E。
5. octo-models/octo@241fb351，MIT。
6. Physical-Intelligence/openpi@15a9616a，Apache-2.0。
7. VLA 架构、适配、延迟与动作执行 Claim-to-Code，本仓先前 source freeze；本页为独立复核。

Part 7 / 16 · `interview_materials/vla/sources/vol_03_completion_fact_reconciliation.html` · SHA-256 `B5EB09FDD0615AD02EC79231B2457B32800AAB9F53487E9A3C251BC05923D53A`

返回本卷目录

Canonical Markdown SHA-256：`b5eb09fdd0615ad02ec79231b2457b32800aab9f53487e9a3c251bc05923d53a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_completion_fact_reconciliation.md`

VOL 03 VLA / Action：44-Card Completion Reconciliation

状态：`completion_reconciled_candidate / 44_cards / source_audits_warn / WARN` 审查日期：2026-07-19 范围：`VLA-ARCH-001..012`、`VLA-ACT-001..032` 的题卡结构、source audit 映射、项目证据边界与静态 HTML 合同。 结论先行：44 张卡均已映射到 primary/frozen source 与 Source Audit A–F，OpenPiVLA inventory/ownership 已设为所有项目 hook 的总门；Audit A 的 OpenVLA endpoint/token/mask、Audit B 的 π0.5/FAST/OpenPI、Audit E 的 theorem/endpoint/time-sign/Beta 已完成独立复核，C/D/F 及 A/B/E 的 local runtime gate 仍为 `WARN/PENDING`。本页不能把 VOL 03 升级为 `reviewed/drilled`，44 张卡继续全部 `unseen`。

1. Reconciliation 合同

每张卡必须包含：稳定唯一 ID、面试意图、30 秒回答、机制展开、项目挂钩、追问树、危险答案、来源和真实练习状态。项目挂钩必须受 OpenPiVLA Inventory 与 Ownership Ledger 约束。

Source Audit A–F 的角色：

Audit	主要范围	当前边界
A OpenVLA	tokenization、255/256 endpoint、CE/metric mask、unnorm	endpoint/mask independent-reviewed；local checkpoint/schema 未核实
B π0/π0.5/FAST/openpi	action expert、time/sign、FAST revision、transforms	pi05/FAST/OpenPI public facts independent-reviewed；local model class/H/D/solver 未核实
C ACT	CVAE、L1/KL、chunk、temporal ensemble	paper/code gap、padding denominator、learned query、oldest weight 与 sentinel 已 independent-reviewed；local use 保持 WARN
D Diffusion Policy	epsilon/sample target、scheduler、horizon、receding horizon	paper/code warm-start、loss denominator、boundary padding、step-count 与 real queue 已 independent-reviewed；local scheduler 保持 WARN
E Flow Matching/π0	path/velocity、endpoint、ODE、mask、CFG	theorem/endpoint/time-sign/Beta independent-reviewed；historical package/local implementation 保持 WARN

Audit	主要范围	当前边界
F Architecture/ Execution	RT-2/OpenVLA/π0、mask、adaptation、latency、controller	RT-2/Octo/openpi mask/transform/broker/example 已 independent-reviewed；local timing/axis/tests 未运行

2. 44 卡逐项映射

ID	主要 evidence chain	必须保留的边界	Verdict
VLA-ARCH-001	RT-2/OpenVLA/π0 + A/B	semantic planner、policy、controller 分层	PASS/WARN
VLA-ARCH-002	A/B + inventory	view/state/time/H/K 不能由项目名反推	PASS/WARN
VLA-ARCH-003	A/B + inventory/ledger	backbone/head/expert/controller ownership 分开	PASS/WARN
VLA-ARCH-004	B/F	camera/mask/time alignment 是 converter/wrapper contract	PASS/WARN
VLA-ARCH-005	A	paper 256-way 与 frozen 255-center decode 分开	PASS/WARN
VLA-ARCH-006	π0/B/E	action expert、H、solver 不反推本地实现	PASS/WARN
VLA-ARCH-007	RT-2/OpenVLA/Octo/π0/F	不做跨协议指标排名	PASS/WARN
VLA-ARCH-008	A/B/F	attention-visible、label-visible、action mask 分开	PASS/WARN
VLA-ARCH-009	A/F	full/head/LoRA/expert 选择不冒充项目贡献	PASS/WARN
VLA-ARCH-010	B/F	d/D、padding、loss denominator 与 output slice 分开	PASS/WARN
VLA-ARCH-011	RT-2/FAST/openpi/F	source latency 不写成本地 timing	PASS/WARN
VLA-ARCH-012	A–F + inventory/ledger	paper→repo→checkpoint→runtime→ownership 五层审计	PASS/WARN
VLA-ACT-001	π0/FAST/ACT/Diffusion + B–E	supervision/decoder 先于方法标签	PASS/WARN
VLA-ACT-002	A/B/F	normalization asset、axis 与 controller 前置	PASS/WARN
VLA-ACT-003	A	scalar bins 不解释 FAST/JSON 路线	PASS/WARN
VLA-ACT-004	A/B	vocab offset 与 custom formatter 分开	PASS/WARN
VLA-ACT-005	FAST/B	token compression 不等于 success uplift	PASS/WARN
VLA-ACT-006	FAST/B	DCT/quantization/BPE 与 pinned revision 绑定	PASS/WARN
VLA-ACT-007	FAST/B + project page	97.4% 不归因 FAST，protocol/artifact 缺口保留	PASS/WARN
VLA-ACT-008	B/F	H×D、valid d、padding/mask/slice 可验证	PASS/WARN
VLA-ACT-009	rotation + B/F	translation/rotation/gripper/base frame 不混用	PASS/WARN
VLA-ACT-010	ACT/Diffusion/FAST + B–E	选型理由不代替 matched ablation	PASS/WARN
VLA-ACT-011	ACT/C	prediction H、execution K、replan R 分开	PASS/WARN
VLA-ACT-012	ACT/C	CVAE chunk 只作方法/baseline，不冒充本地架构	PASS/WARN
VLA-ACT-013	ACT/C	train posterior 与 inference z=0 分开	PASS/WARN
VLA-ACT-014	ACT/C	oldest-weight ordering 与 query/target 轴显式	PASS/WARN
VLA-ACT-015	ACT/C	chunk、replan、temporal ensemble 三机制分开	PASS/WARN

ID	主要 evidence chain	必须保留的边界	Verdict
VLA-ACT-016	ACT/C	horizon sweep 需 fixed checkpoint/protocol	PASS/WARN
VLA-ACT-017	ACT/Diffusion/Flow + C–E	inductive bias 比较不等于本地 ablation	PASS/WARN
VLA-ACT-018	Diffusion/D	多模态生成直觉不自动证明 safety/success	PASS/WARN
VLA-ACT-019	Diffusion/D	noisy action/noise target/condition 与 code target 绑定	PASS/WARN
VLA-ACT-020	Diffusion/D	schedule/prediction type/horizon/steps 分开	PASS/WARN
VLA-ACT-021	Diffusion/D	returned batch、executed prefix、re-observation 分开	PASS/WARN
VLA-ACT-022	Diffusion/D	paper 10/16、frozen config 8 与 warm-start gap 保留	PASS/WARN
VLA-ACT-023	Flow/E	probability path、velocity field、historical code 分开	PASS/WARN
VLA-ACT-024	Flow/E	exact-linear 与 sigma-min endpoint 分开	PASS/WARN
VLA-ACT-025	π0/openpi/E	time direction、velocity sign、NFE/solver 显式	PASS/WARN
VLA-ACT-026	Diffusion/Flow/D/E	target、time、sampler、budget 同协议比较	PASS/WARN
VLA-ACT-027	π0/B/E	local class/forward 才能证明 action expert	PASS/WARN
VLA-ACT-028	openpi/B/E/F	all-D mean 与 valid-dim mask 不混称	PASS/WARN
VLA-ACT-029	Flow/E	CFG 仅作候选设计，未写成本地贡献	PASS/WARN
VLA-ACT-030	Diffusion/Flow/D/E	target→solver→norm/schema→controller 逐层排查	PASS/WARN
VLA-ACT-031	A/B/F	decode→unnorm→slice→frame/unit→safety 顺序明确	PASS/WARN
VLA-ACT-032	openpi/F	sentinel/FIFO/replan/stale RPC/NaN 测试是设计，未本地执行	PASS/WARN

3. 本次修复

1. 六个 Source Audit review 的 12 位截断 SHA 修复为完整 SHA-256，并与 canonical Markdown/HTML metadata 对齐。
2. 题卡增加 OpenPiVLA inventory 与 ownership ledger 总门；项目页存在、Git author 或上游实现不视为个人 ownership。
3. `VLA-ACT-031` 的 controller transform 与 (SO(3)) 改为有效 MathJax。
4. `VLA-ACT-032` 的追问改为“为什么不能依赖 `np.clip` 处理 NaN”，避免暗示 clip 会完成 sanitization。
5. π0.5、inventory、ownership 与 ACT/Diffusion/Architecture 三个独立复核入口纳入题卡来源表；27 个来源编号均需有定义和使用记录。

4. 可完成与不可完成

本页可以证明

- 12 个 ARCH + 32 个 ACT stable IDs 完整；44 张卡结构、长度、来源与项目 hook 可静态核验。

- 所有方法都能追到 primary/frozen source 或 A–F claim-to-code audit。
- 本地 OpenPiVLA 强 claim 继续受 checkpoint/schema/eval/ownership gate 约束。

本页不能证明

- Audit A/B/E 的局部 `independent_fact_review` 不会自动让 C/D/F 或 local project gate 变成 PASS；未复核项与本地 runtime/ownership 仍是 WARN。
- OpenPiVLA 的 model class、checkpoint、FAST revision、action axes、H/D/K、normalizer、solver、controller、97.4% protocol 或个人 ownership 已闭合。
- 44 张卡已经会讲；没有真实口述，全部继续 `unseen`。

5. 下一步

完成确定性校验后，VOL 03 状态仍为 `completion_reconciled_candidate / source_audits_warn / 44 unseen / WARN`。Audit A–F 的 public paper/code facts 均已完成独立切片；local checkpoint/schema/controller/latency/ownership 与真实口述仍未闭合。下一 bounded slice 改为一题真实口述或本地 runtime artifact，不得继续批量生成 VOL 03 卡。

参考入口

- OpenVLA Audit A
- OpenVLA Endpoint / Token / Mask Independent Review
- π0 / π0.5 / FAST / openpi Audit B
- ACT Audit C
- ACT Audit C 独立事实复核
- Diffusion Policy Audit D
- Diffusion Policy Audit D 独立事实复核
- Flow Matching / π0 Audit E
- Flow Matching / π0 Time–Sign Independent Review
- π0 / π0.5 / FAST / OpenPI Independent Review
- Architecture / Execution Audit F
- Architecture / Execution Audit F 独立事实复核
- OpenPiVLA Inventory
- OpenPiVLA Ownership Ledger

Part 8 / 16 · `interview_materials/vla/sources/vol_03_diffusion_policy_claim_to_code.html` · SHA-256 `2DB632245846A102B3895CD8EE38EC53D22F90EAAD0AB88D45C1BD9BBC66450D`

返回本卷目录

Canonical Markdown SHA-256：`2db632245846a102b3895cd8ee38ec53d22f90eaad0ab88d45c1bd9bbc66450d`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_diffusion_policy_claim_to_code.md`

VOL 03 Source Audit D：Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code

审计日期：2026-07-16 状态：`drafted / upstream_paper_and_code_reviewed / independent_review_pending` 论文版本：*Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*，arXiv `2303.04137v5` 官方仓库：`real-stanford/diffusion_policy` frozen commit `5ba07ac6661db573af695b419a7947ecb704690f` License：MIT；关键依赖：Hugging Face `diffusers=0.11.1` 对应题目：`VLA-ACT-018..022`；`VLA-ACT-017` 的 diffusion 对照 本页问题：Diffusion Policy 为什么能表示多模态连续动作？训练时 noisy action、noise target、condition 与 mask 如何组织？observation / prediction / execution horizon 如何进入代码？receding-horizon、DDIM、去噪步数和 warm-start 在论文与 frozen repo 中是否一致？

1. 结论先行

1. **Diffusion Policy 把 visuomotor policy 写成条件 action-sequence diffusion，而不是逐维 token 分类。**时刻 t 接收最近 T_o 步 observation，建模未来 action sequence 的条件分布 $p_\theta(A_t \mid O_t)$；论文记作 T_p，官方代码叫 `horizon`。[1][2]
2. **训练目标是随机 timestep 的 noise/sample MSE。**代码对标准化 trajectory 加 scheduler noise；`prediction_type=epsilon` 时 target 是同形状 Gaussian noise，`sample` 时 target 是 clean trajectory。该 policy 代码只显式支持 epsilon/sample，不自动处理 `v_prediction`。[2][3]
3. **默认 image hybrid 路径是 global conditioning。**前 T_o 帧经视觉 encoder 成为扁平 `global_cond`，diffusion trajectory 只保留 action channels，初始采样为 $[B, T_p, D_a]$ 的 Gaussian noise；默认 global path 的 action condition mask 全 False。[2]
4. **模型 horizon、返回 action 数和真正 replanning interval 必须分开。**`horizon=T_p`；`n_action_steps=T_a` 决定 `predict_action` 返回多少步；simulation 的 `MultiStepWrapper` 一次执行 T_a 步后再 query，real eval 可提交更长 action slice、每 `steps_per_inference` 步重算。[2]

5. **官方 action slice 从最后一帧 observation 对齐。** `start=n_obs_steps-1`，`end=start+n_action_steps`；例如 $T_p = 16, T_o = 2, T_a = 8$ 时，完整 trajectory 有 16 步，返回 index 1 到 8 的 8 步。[2]
6. **paper 说 receding-horizon 可以 warm-start 下一次 inference；该 frozen commit 没实现。** 主要 policy 都从 `torch.randn` 重新开始，并在入口拒绝 `past_action`。所以"论文支持 warm-start"不能讲成"这个 commit 默认用上一段 action 初始化"。[1][2]
7. **论文正文、论文表格和 code 的 inference step 数不是统一常数。** 论文正文给 DDIM 100 train / 10 eval 的 real-world 例子；v5 表格的 real rows 为 16 eval；frozen real configs 为 8，`eval_real_robot.py` 又运行时设为 16。[1][2]
8. **多模态来自 sequence-level stochastic generation，而非每个时间点独立采样。** 随机 Gaussian 初值和 DDPM/DDIM transition 提供多样性；一次对整段 $T_p \times D_a$ trajectory 去噪，使相邻动作共享同一生成过程。[1]

2. 证据层级、冻结对象与适用边界

层级	本页使用方式	能支持	不能支持
arXiv v5	formulation、horizons、network/noise schedule、DDIM 与 ablation	方法定义、论文符号、公开实验和 warm-start 设计意图	frozen repo 的实际 default 与个人项目事实
frozen official repo	policy、dataset、sampler、runner、real eval、YAML	tensor shape、mask、normalization、action slice、执行/重规划路径	个人 OpenPiVLA 是否采用该实现
frozen dependency	diffusers 0.11.1 DDPMScheduler/DDIMScheduler	`add_noise`、prediction type、`set_timesteps`、`step` 语义	不能替代 policy 自己的 loss/mask
同源推断	paper/code 联合推导 $T_o/T_p/T_a$、latency 与 stochasticity	面试解释的 interface contract	未运行的 latency/success

冻结信息：

- arXiv API 的唯一 latest entry 是 `2303.04137v5`，更新时间 2024-03-14；v5 是 extended journal version。[1]
- 官方 repo HEAD 为 `5ba07ac6661db573af695b419a7947ecb704690f`，commit message 为 `Done adapting mujoco image dataset`；根目录 LICENSE 为 MIT。[2]
- repo `conda_environment.yaml` 固定 `diffusers=0.11.1`、PyTorch 1.12.1、Hydra 1.2.0。[2]
- 代表实现为 `DiffusionUnetHybridImagePolicy` 与 `DiffusionTransformerHybridImagePolicy`；lowdim policy 共享 scheduler、loss、horizon 和 inference contract。

3. 论文符号与三个 horizon

论文定义：

$$O_t = (o_{t-T_o+1}, \ldots, o_t), \qquad A_t = (a_t, \ldots, a_{t+T_p-1}),$$

其中 T_o 是 observation horizon，T_p 是 action prediction horizon，T_a 是 action execution horizon。policy 用 O_t 预测 T_p 步，只执行 T_a 步，随后重新观察并规划。[1]

变量与直觉：T_o 决定历史长度，T_p 决定一次 diffusion 生成多长 trajectory，T_a 决定多久才吸收新 observation。实现映射：`n_obs_steps`、`horizon`、`n_action_steps` 在 `train_diffusion_unet_hybrid_workspace.yaml#L12-L20` 与 policy constructor `diffusion_unet_hybrid_image_policy.py#L22-L40` 对应。

概念	frozen field	作用	不能混同
observation horizon T_o	`n_obs_steps`	encoder 使用 `obs[:,:To]`	不等于全部 episode history
prediction horizon T_p	`horizon`	diffusion trajectory 时间维	不等于返回 action 数
execution window T_a	`n_action_steps`	action slice 长度	不自动等于 real query interval
replan interval R	simulation 的 action batch；real CLI `steps_per_inference`	多少控制步后再次 inference	不由 `horizon` 单独决定
diffusion steps K	`num_inference_steps`	一次 query 内的 denoising iterations	不是机器人控制步数

4. Conditional DDPM：训练目标与前向加噪

4.1 论文 objective

论文以 clean action sequence A_t^0 为样本，随机选 diffusion timestep k，采样 ϵ，让网络根据 observation、noisy action 和 k 预测噪声：

$$A_t^k = \sqrt{\bar{\alpha}_k}\, A_t^0 + \sqrt{1-\bar{\alpha}_k}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

$$\mathcal{L}_\epsilon = \mathbb{E}_{A_t^0, O_t, k, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(O_t, A_t^k, k)\right\|_2^2\right].$$

变量与直觉：$\bar{\alpha}_k$ 由 beta/noise schedule 决定；较小 k 保留更多 action signal，较大 k 更接近 Gaussian。网络学习去噪方向而不是直接回归 clean action。论文 Section DDPM/DDPM Training 见 HTML；repo 的 `add_noise` 映射到 diffusers v0.11.1 `scheduling_ddpm.py#L329-L350`。

论文把反向 transition 写成：

$$A_t^{k-1} = \alpha_k \left(A_t^k - \gamma_k \epsilon_\theta(O_t, A_t^k, k) + \mathcal{N}(0, \sigma_k^2 I) \right).$$

这表示网络输出与 scheduler 共同定义 reverse update；不能把 ϵ_θ 单独当最终 action。系数、variance 和 clipping 由 DDPM/DDIM scheduler 完成。[1][2]

4.2 Frozen training path

```
def training_step(batch, policy):
    # Normalize observations and the action trajectory.
    nobs = policy.normalizer.normalize(batch["obs"])
    nactions = policy.normalizer["action"].normalize(batch["action"])

    # Encode the first To observations and build the trajectory contract.
    cond_data, trajectory, global_cond = build_condition(nobs, nactions)

    # Sample one diffusion timestep and Gaussian noise per batch item.
    noise = torch.randn_like(trajectory)
    timesteps = torch.randint(
        0, policy.noise_scheduler.config.num_train_timesteps,
        (trajectory.shape[0],), device=trajectory.device
    )
    noisy = policy.noise_scheduler.add_noise(trajectory, noise, timesteps)

    # Re-impose hard conditions after adding noise.
    condition_mask = policy.mask_generator(trajectory.shape)
    noisy[condition_mask] = cond_data[condition_mask]

    # Predict epsilon or the clean sample and apply the masked MSE.
    pred = policy.model(noisy, timesteps, global_cond=global_cond)
    target = noise if policy.noise_scheduler.config.prediction_type == "epsilon" \
        else trajectory
    loss = ((pred - target) ** 2 * (~condition_mask)).mean()
    return loss
```

直接代码证据：normalization/trajectory 在 `diffusion_unet_hybrid_image_policy.py#L284-L314`，noise/timestep/add_noise 在 `#L316-L337`，target 与 MSE 在 `#L339-L351`。

4.3 `prediction_type` 与 mask

$$y_\theta = \begin{cases} \epsilon, & \texttt{prediction_type} = \texttt{epsilon}, \\ A_t^0, & \texttt{prediction_type} = \texttt{sample}. \end{cases}$$

变量与实现：policy 只处理 epsilon/sample，其他字符串直接 `ValueError`。diffusers scheduler 虽有 `v_prediction` 分支，policy 的 `compute_loss` 没实现对应 velocity target；依赖库能力不等于当前 policy 能力。证据为 `diffusion_unet_hybrid_image_policy.py#L339-L345` 与 `scheduling_ddpm.py#L278-L289`。

`loss_mask = ~condition_mask` 后用完整 tensor mean；conditioned dimensions 的 numerator 被置零，但 denominator 仍是 trajectory 总元素数。policy 还 assert batch 不带 `valid_mask`。默认

`obs_as_global_cond=True` 时 `obs_dim=0`、`action_visible=False`，action channels 全部参与 MSE；non-global/inpainting path 才把前 T_o observation-feature dimensions 固定并排除。[2]

4.4 为什么 sequence diffusion 能表达多模态

$$p_\theta(A_t \mid O_t) \neq \prod_{h=0}^{T_p-1} p_\theta(a_{t+h} \mid O_t).$$

一次去噪对象是整段联合 trajectory，不是每个时间点独立的 categorical/Gaussian head。同一噪声初值和 temporal network 让“向左绕/向右绕”等 mode 形成连贯 sequence；这是论文设计解释，不是 repo 显式计算的概率分解。[1]

5. Observation conditioning 与网络实现

5.1 CNN / UNet hybrid

论文 CNN policy 用视觉 encoder 生成 O_t latent，通过 FiLM 将 observation feature 和 diffusion timestep 注入 1D temporal U-Net。repo 的 `ConditionalUnet1D` 将 timestep embedding 与 global visual condition 拼接，并在 residual blocks 生成 per-channel scale/bias：

```
image sequence [B, To, C, H, W] + lowdim obs
            |
            v
ResNet18 observation encoder (GroupNorm / fixed crop)
            |
            v
global_cond [B, To * Do]
            |
random noisy action trajectory [B, Tp, Da]
            |
            v
ConditionalUnet1D + timestep FiLM
            |
            v
predicted epsilon/sample [B, Tp, Da]
```

`obs_as_global_cond=True` 时 `input_dim=action_dim`、`global_cond_dim=obs_feature_dim*n_obs_steps`，见 `diffusion_unet_hybrid_image_policy.py#L129-L146`。FiLM scale/bias 在 `conditional_unet1d.py#L14-L66`，global feature concat 在 `#L173-L201`。

论文 visual encoder 为每个 camera 使用 separate ResNet-18、空间 softmax 和 GroupNorm；代码由 robomimic config 建立 encoder，并可用 `obs_encoder_group_norm=True`、`eval_fixed_crop=True` 替换 BN/随机 crop。camera 数、action dimensions、absolute/velocity semantics 由 task `shape_meta` 决定。

5.2 Time-series diffusion transformer

Transformer variant 将 noisy action sequence 作为 T_p 个 token，扩散 timestep embedding 作为 condition；视觉 observation sequence 变为 condition embedding，decoder cross-attend 到 condition memory，输出每个 action token 的 residual。`TransformerForDiffusion.forward` 的 contract 是 input `[B,T,input_dim]`、cond `[B,T_o,cond_dim]`、output `[B,T,input_dim]`，见 `transformer_for_diffusion.py#L270-L344`。

默认 transformer hybrid config 是 $T_p = 10, T_o = 2, T_a = 8$、causal attention、8 layers、256 embedding、DDPM 100 train / 100 inference；real hybrid 改为 $T_p = 16, T_o = 2, T_a = 8$、DDIM 8 inference。[2] 论文建议先用 CNN；action sequence 变化快时再试 transformer，但 transformer 对超参更敏感。[1]

6. Inference：随机初始化、denoising loop 与 action slice

6.1 Global-conditioned inference

`predict_action` 的顺序：

1. normalize observation；
2. 只取 `obs[:, :To]` 过 vision encoder；
3. 建立全零 `cond_data`，形状 `[B,Tp,Da]`，`cond_mask` 全 False；
4. 以同形状 Gaussian noise 初始化 trajectory；
5. `scheduler.set_timesteps(num_inference_steps)`，依次执行 `scheduler.step(...).prev_sample`；
6. 从 `nsample[...,:Da]` unnormalize，再从 `start=To-1` 切出 `Ta` 步。

$$X^{(K)} \sim \mathcal{N}(0, I), \qquad X^{(k-1)} = \mathrm{SchedulerStep}\left(X^{(k)}, f_\theta(X^{(k)}, k, O_t)\right),$$

$$\hat{A}_t = \mathrm{Unnormalize}\left(X^{(0)}_{:,\, To-1:To-1+T_a,\, :D_a}\right).$$

变量与实现：K 是 diffusion steps，T_p 是 trajectory time，T_a 是返回 action 数。UNet hybrid random init/scheduler loop 在 `#L175-L212`，condition/action slice 在 `#L215-L278`。Transformer hybrid 同构路径见 `#L183-L219` 与 `#L222-L290`。

6.2 Paper warm-start 与 code random restart

论文称 receding-horizon formulation 可以用上一段 action prediction warm-start 下一轮 inference。[1] frozen repo 的主要 policy paths 都是：

```
def conditional_sample(condition_data, condition_mask):
    # Every query in this commit starts from fresh Gaussian noise.
    trajectory = torch.randn(condition_data.shape, device=condition_data.device)

    # The scheduler determines descending diffusion timesteps.
    scheduler.set_timesteps(num_inference_steps)
    for t in scheduler.timesteps:
        # Re-impose hard conditions before each denoising step.
        trajectory[condition_mask] = condition_data[condition_mask]
        model_output = model(trajectory, t, global_cond=global_cond)
        trajectory = scheduler.step(model_output, t, trajectory).prev_sample
    return trajectory
```

`predict_action` 还 assert `past_action` 不在 observation dict，因此没有 previous `action_pred` 初值或 warm-start buffer。[2]

说法	判定
论文提出 warm-start 作为 receding-horizon 可用选项	direct paper claim
frozen repo 默认 eval 使用 warm-start	rejected
code stochasticity 只来自 scheduler step	rejected；还有 initial `torch.randn`
固定 generator/seed 可复现同一输入	同源推断，需目标设备/版本实测

6.3 DDPM、DDIM、去噪步数

路径	scheduler	train timesteps	config inference steps	备注
UNet hybrid	DDPMScheduler	100	100	squared-cosine、epsilon、fixed_small
UNet DDIM hybrid	DDIMScheduler	100	8	inference grid 抽稀
real UNet hybrid	DDIMScheduler	100	8	frozen real config
`eval_real_robot.py` diffusion branch	loaded scheduler	checkpoint-dependent	强制 16	runtime override
paper main text example	DDIM	100	10	3080 / 0.1s 叙述
paper v5 real table rows	task rows	100	16	与 main text 不完全一致

DDIM 将 train grid 与 inference steps 解耦；减少 K 降低 query compute，但改变 reverse approximation。`DDIMScheduler.set_timesteps` / `step` 见 `scheduling_ddim.py#L196-L214` 与 `#L249-L334`。

7. Normalization、padding 与数据切片

7.1 默认 $[-1,1]$ normalizer

PushT image dataset 对 action 与 agent position 统计 min/max，`LinearNormalizer.fit(..., mode='limits')` 将每维映射到 $[-1,1]$；图像先除以 255，再做 $x \mapsto 2x-1$。[2]

$$\tilde{x}_i = \frac{2(x_i - x_i^{\min})}{x_i^{\max} - x_i^{\min}} - 1, \qquad x_i = \frac{\tilde{x}_i + 1}{2}(x_i^{\max} - x_i^{\min}) + x_i^{\min}.$$

变量与直觉：每个 axis 独立 scale；near-constant dimension 特殊处理避免除零。实现见 `pusht_image_dataset.py#L60-L68`、`normalize_util.py#L6-L35` 与 `normalizer.py#L182-L261`。`clip_sample=True` 又会在 scheduler 侧限制 clean prediction，因此 stats 与 clipping 是联合 contract。

7.2 Boundary padding 不是 valid loss mask

PushT dataset 以 `sequence_length=horizon` 建 SequenceSampler，用 `pad_before=To-1`、`pad_after=Ta-1` 让 episode 边界也能采样。[2] sampler 在边界重复第一/最后一个真实 frame/action，见 `sampler.py#L121-L153`。

因此 default boundary padding 是 repeated values，不是 NaN 或显式 valid mask；policy 又要求 batch 不带 `valid_mask`，这些 repeated targets 会按普通 actions 进入 MSE。不能见到 `pad_after` 就说 padded positions 不监督。

7.3 Normalizer 数据范围

`PushTImageDataset.get_normalizer` 从完整 replay buffer 的 action/state 拟合；sampler 的 train/validation episode mask 不会自动限制 stats。[2] 这是该 dataset implementation 的复现边界，不等于所有 dataset 都有 validation leakage；要先报具体 dataset class。

8. Receding-horizon rollout：simulation 与 real robot

8.1 Simulation

`PushTImageRunner` 每次 query 得到 `[B,T_a,D_a]`，再把整段 action 传给 `MultiStepWrapper.step`；wrapper 内部逐 action 调 env，执行 T_a 步后返回最近 T_o 个 observation。[2]

```
def simulation_rollout(env_wrapper, policy):
    # Reset and receive the latest To observations.
    obs = env_wrapper.reset()
    policy.reset()
    while not done:
        # K denoising steps produce Tp, then policy returns Ta actions.
        action_chunk = policy.predict_action(obs)["action"]
        # Wrapper executes Ta actions before returning a new observation stack.
        obs, reward, done, info = env_wrapper.step(action_chunk)
```

证据：query/step 在 `pusht_image_runner.py#L176-L214`，wrapper action loop 在 `multistep_wrapper.py#L101-L124`。

8.2 Real robot

`eval_real_robot.py` diffusion branch 将 `num_inference_steps` 设为 16，并把 `n_action_steps` 改成 `horizon-n_obs_steps+1`，尽量返回从最后 observation 对齐后的整段 action。[2] 外层 `steps_per_inference` 是 CLI 参数，默认 6；它决定多久再读 real observation。action 带 timestamp 提交给 controller，超时则只提交最后一项。[2]

所以有三个长度：

1. diffusion trajectory T_p，例如 16；
2. `predict_action` slice，可能是 $T_p - T_o + 1 = 15$；
3. 外层 query interval，默认 6，还受 observation/inference/controller latency 影响。

“每次执行 15 步后才重规划”是错误的；script 可能提交 15 个未来 targets，但下一次 query 由 6-step schedule 和实时预算决定。证据：`eval_real_robot.py#L90-L118` 与 `#L294-L345`。

9. Horizon、去噪步数与 latency trade-off

9.1 T_p/T_a 太短

sequence-level consistency 变弱；若 T_a 同步变短，query 次数增加、inference overhead 上升；但 observation 刷新更快，对扰动更 responsive。

9.2 T_p/T_a 太长

long trajectory 提供更强 sequence modeling buffer；若 T_a 也变长，replan 变稀疏，stale action 和 latency 风险上升。U-Net/Transformer 时间维与每次 denoising 计算共同影响显存和吞吐。

论文把 action horizon 说成 temporal consistency 与 responsiveness 的 trade-off，并把 latency 定义为“最后一帧 observation 到第一条可执行 action 之间的 step 数”。[1] 该延迟不只由 `num_inference_steps` 决定。

9.3 K 太少或太多

K 小会降低 query latency，但 reverse approximation 误差可能变大；K 大给更多 denoising compute，却延长闭环周期。K 与 T_p/T_a 正交；scheduler、prediction type、clip_sample 和 beta schedule 必须一起报。

9.4 warm-start 取舍

若实现 warm-start，可把上一轮 trajectory 时间平移后作为下一轮初值，理论上减少随机重采样；但要处理已执行 prefix、时间对齐与新 observation 冲突。本页不把它写成 frozen repo 行为。

10. 与 ACT、token、flow 路线的归纳偏置

方法	训练 target	一次生成	stochasticity	closed-loop 接口
ACT	continuous chunk masked L1 + CVAE KL	一次 forward，test $z=0$	默认 deterministic	chunk boundary / temporal ensemble
OpenVLA	scalar token CE	autoregressive token	token decoding	原始单步 action
FAST	frequency-space token CE	autoregressive FAST tokens	token decoding	window/replan 由 config
Diffusion Policy	epsilon/sample MSE	K 次 scheduler step	Gaussian init + scheduler	T_o, T_p, T_a + controller
π0 flow	velocity MSE	ODE/flow solver	noise init / solver	openpi H/K contract

“能表示多模态”不等于部署必须随机；是否固定 generator、如何 smoothing、如何选择 T_a，仍是 evaluation/controller contract。

11. Claim-to-code 总表

Claim	paper	frozen code	判定	安全口径
默认直接回归 clean action	noise MSE	target 按 prediction type	rejected	默认 epsilon；sample 可选
每时刻独立 diffusion	action trajectory	`[B,Tp,Da]`	rejected	联合 sequence
horizon 就是执行步数	区分 T_p, T_a	slice/runner 分开	rejected	四条时间轴分开
global cond 拼进 action channel	visual condition	flattened global cond	rejected	trajectory action-only
所有路径都 inpainting	有 global/non-global	default global	rejected	先报 cond path
policy 已支持 v_prediction	dependency 有	policy 无 target	rejected	依赖能力不等于 policy
padding 自动不进 loss	无统一 claim	repeat + no valid_mask	rejected	dataset-specific
random noise 只在训练	inference 也 random	`conditional_sample`	rejected	两端都有
warm-start 是默认	paper says can	code rejects past_action	rejected	design option only
DDIM 固定 10 steps	paper example	config 8/runtime 16	rejected	报版本/config
$T_a = 1$ 才闭环	fixed horizon allowed	batch execute	rejected	consistency/responsiveness trade-off
46.9% 可迁移本地	specific benchmarks	no local artifact	rejected	不证明个人指标

12. 对 `VLA-ACT-018..022` 的支持状态

ID	覆盖	状态	仍缺
VLA-ACT-018	conditional diffusion、多模态、temporal consistency	source-ready	independent review
VLA-ACT-019	noisy action、target、condition/mask、MSE	source-ready	手算 scheduler
VLA-ACT-020	schedule、prediction type、T_p、K	source-ready with warning	v-pred extension
VLA-ACT-021	$T_o/T_p/T_a$、sim/real replan	source-ready	本地 controller
VLA-ACT-022	DDPM/DDIM、latency、warm-start gap	source-ready with warning	hardware telemetry

source-ready 不等于 candidate-ready；题卡不在本页生成，练习状态不改变。

13. 与个人项目的真实性边界

上游事实	可用于提醒	不可迁移
T_o, T_p, T_a	拆本地 H/K/replan	不证明本地数值
normalizer + clipping	stats/scheduler/controller 一致	不替代本地 asset
global condition	encoder/denoise loop 对照	不证明本地 Qwen path
random restart	paper/code 差异案例	不证明本地行为
public latency	recipe 语境	不证明本地 GPU
timestamp scheduling	action slice/queue 分开	不替代本地 client

进入 OpenPiVLA 强回答前仍需：policy class、checkpoint、scheduler/prediction type、normalizer、action axes、model horizon、executed prefix、replan trigger、controller frequency、latency、task/trial/seed/protocol 和 success summary。

14. 面试安全口径与危险答案

14.1 30 秒安全骨架

“Diffusion Policy 把未来 action chunk 当作联合连续变量，用 conditional DDPM 学 noisy action 的噪声残差。训练时随机选 diffusion timestep，把标准化 trajectory 加 Gaussian noise，condition 是最近 T_o 步 image/state；默认 epsilon prediction 用 noise MSE。推理从 Gaussian trajectory 开始，运行 K 次 scheduler step 得到 T_p 步，再只返回/执行 T_a 步并重观测。horizon、action slice、replan interval 和 diffusion iterations 是不同时间轴；论文提 warm-start，但 frozen repo 每次从 random noise 重启。”

14.2 高危说法

- “MSE 直接回归 clean action。”——默认 target 是 epsilon。
- “每步只生成一个 action。”——生成整个 $T_p \times D_a$ sequence。

- “horizon=16 就执行 16 步。”——config 可能返回 8，real eval 还异步重规划。
- “100 inference steps 是机器人控制步。”——是 denoising iterations。
- “代码 warm-start 上一段 action。”——当前 commit random init。
- “mask 去掉所有 padding。”——sampler 可能重复边界值。
- “DDIM 把训练也改成 8 steps。”——通常只抽稀 inference。
- “46.9% 说明本地一定更好。”——protocol 不可偷换。

15. 风险、未决项与下一步

1. 独立审查未完成：本页由同一 agent 完成 freeze，review 保持 WARN。
2. 版本差异必须保留：warm-start vs random restart；10/16/8 steps；paper symbols vs real scheduling。
3. **padding 需合成 test**：确认各 dataset repeated targets 是否进 loss。
4. **scheduler 要锁版本**：记录 diffusers、scheduler、prediction type、clip_sample、runtime steps。
5. 尚未进入题卡：`VLA-ACT-018..022` 先 source-ready。
6. 下一主交付物：基础 Flow Matching → π0，核对 probability path、velocity target、ODE solver 与 openpi 时间方向。
7. 本地仍待取证：upstream 只作方法对照，不进入个人指标/贡献。

参考文献（References）

1. Chi et al., Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, arXiv `2303.04137v5`, 2024；arXiv abs/API。
2. real-stanford/diffusion_policy, frozen commit `5ba07ac6661db573af695b419a7947ecb704690f`, MIT；核心路径为 policy、dataset、sampler、runner 与 `eval_real_robot.py`。
3. Hugging Face diffusers v0.11.1 schedulers，supporting dependency。
4. ACT Claim-to-Code，本仓库 Source Audit C。
5. OpenVLA Claim-to-Code，本仓库 Source Audit A。
6. π0 / FAST / openpi Claim-to-Code，本仓库 Source Audit B。

Part 9 / 16 · `interview_materials/vla/sources/vol_03_diffusion_policy_independent_fact_review.html` · SHA-256 `C3F8AD6AF61BF70C60AA472F43AED3C7A592087661FDBE04DEF05664E3BC38DF`

返回本卷目录

Canonical Markdown SHA-256：`c3f8ad6af61bf70c60aa472f43aed3c7a592087661fdbe04def05664e3bc38df`；生成自：`interview_materials\vla\sources\vol_03_diffusion_policy_independent_fact_review.md`

VOL 03 Source Audit D：Diffusion Policy 独立事实复核

状态：`independent_fact_review / horizon_loss_padding_queue_checked / WARN` 审查日期：2026-07-19 范围：Diffusion Policy v5 与 `real-stanford/diffusion_policy` frozen commit 的训练 target、condition mask、normalization/padding、$T_o/T_p/T_a$、采样步数、warm-start、sim/real rollout 与 synthetic probes。 结论先行：论文与 frozen code 支持“条件 action-sequence diffusion + receding-horizon execution”的主机制，但必须保留四类边界：代码的 loss 通过 `loss_mask` 置零后仍对完整元素取 mean；episode 边界 sampler 重复首/尾 frame/action，而不是 valid-length mask；论文提出 warm-start 设计意图，frozen policy 入口拒绝 `past_action` 且从 Gaussian 重启；DDIM inference steps、返回 action 数和 real queue/requery interval 是不同合同。以下均为 public paper/code facts，不证明本地 OpenPiVLA 的 scheduler、horizon、latency 或 success。

1. 直接复核来源与冻结对象

ID	来源	本次核对	SHA / Verdict
Po 1	Diffusion Policy v5 PDF	Section 2–4、Table 7/8、Appendix B；$T_o/T_p/T_a$、warm-start、DDIM 10/16、action horizon trade-off	`B65C474B696A4802D8F1457D86B637CE2C5521412570D3AA928CD54563BABC8F` / `primary-verified`
Co 1	`diffusion_unet_hybrid_image_policy.py`	Gaussian init、global/inpainting condition、slice、loss target/mask、`past_action` assert	`D8EAD767ED3B1501C358AA3DD97AB479EBDF77A62EC9AB7081D6A2211C2D2E76` / `official-code`
Co 2	`diffusion_transformer_hybrid_image_policy.py`	Transformer path、`pred_action_steps_only`、同一 loss contract	`E4166882BF0FC2E36E97BEB27CDF701286230ABCECA1EFAEA9AAE9A114A008E7` / `official-code`
Co 3	`sampler.py`	`pad_before/pad_after` 与 episode 边界填充	`3DC7AFB686A418ABF4CEEE4648DAFB7E1BE901BB70B1539530F9F9964CAD1688` / `official-code`
Co 4	`pusht_image_runner.py`	sim `policy.predict_action` → `env.step(action)`、每次执行 batch 后重新观测	`84FAA24E803635FE1F07B1907EF04FCB74D07C6ECFCE5F2FF598601019232F03` / `official-code`
Co 5	`multistep_wrapper.py`	`n_action_steps` 的 wrapper action batch 与 observation deque	`2923E73C13FF9F782451AB61BED43A802670BBF29419471F82878C690F26A342` / `official-code`
Co 6	`eval_real_robot.py`	`num_inference_steps=16`、timestamped action queue、over-budget fallback、clip	`CD7C0F1F46DCE8606EBB9B2B2618811870C4D41205CE4AD6369AC5D7AC0A1B7C` / `official-code`

ID	来源	本次核对	SHA / Verdict
Co7	`conda_environment.yaml`	frozen dependency declaration	`C945BCAC09284CF897D3E1837139EA83F991E4B2298DBD3A8C4A0718A34994EA` / `official-code`
Do1	DDPM scheduler v0.11.1	`prediction_type` branches and scheduler step API	`238D19CBE8F105106121F67ABFCA507BB208A72B14C6595DABA4829BB2001CA5` / `dependency`
Do2	DDIM scheduler v0.11.1	inference timestep decimation/step semantics	`073AF47F32CA3DA9F5BB9285A774643C66978DE7987805A69283B5F65F163F79` / `dependency`
Oo1	本次 synthetic probes	action slice、mask dilution、boundary repeat、unsupported v-pred	`local-evidence / PASS`

冻结 repo commit：`5ba07ac6661db573af695b419a7947ecb704690f`，MIT；依赖声明包含 `diffusers=0.11.1`。本页不把其它 fork、用户本地 checkpoint 或论文结果升级为个人项目事实。

2. Action-sequence diffusion 与三条 horizon

论文定义最近 T_o 步 observation，预测 T_p 步 action，仅执行 T_a 步后重新观察：

$$O_t = (o_{t-T_o+1}, \ldots, o_t), \qquad A_t = (a_t, \ldots, a_{t+T_p-1}),$$

变量与直觉：T_o 是 observation history 长度，T_p 是一次 diffusion trajectory 的时间长度，T_a 是送入执行器的 action prefix。它们不是 diffusion solver 的步数，也不能自动推出 real requery interval R。代码将它们映射到 `n_obs_steps`、`horizon` 与 `n_action_steps`；UNet 与 Transformer hybrid 都保留这个输出合同。

在标准 hybrid image path，输入 observation 经 encoder 压成 `global_cond`，trajectory 只有 action channels：

```
# Encode the latest To frames and flatten them as global condition.
features = obs_encoder(obs[:, :To].reshape(B * To, ...))
global_cond = features.reshape(B, -1)

# Diffuse a B x Tp x Da action trajectory, then return an execution prefix.
cond_data = torch.zeros(B, Tp, Da)
cond_mask = torch.zeros_like(cond_data, dtype=torch.bool)
sample = conditional_sample(cond_data, cond_mask, global_cond=global_cond)
action_pred = action_normalizer.unnormalize(sample[..., :Da])
action = action_pred[:, To - 1: To - 1 + Ta]
```

代码证据是 `diffusion_unet_hybrid_image_policy.py` 的 `start = To - 1` 与 `end = start + n_action_steps`。若走 `obs_as_global_cond=False`，observation feature 会拼到 trajectory 作为 inpainting condition；这不是“所有 Diffusion Policy 都只有 global condition”的通用定义。

3. 训练 target、condition mask 与 denominator

标准化 clean trajectory X_0 在随机 timestep k 加噪：

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

变量与直觉：X_k 是网络输入，ϵ 或 clean X_0 是由 scheduler `prediction_type` 选择的 target。frozen policy 只接受 `epsilon` 与 `sample`；遇到其它字符串（例如 `v_prediction`）直接 `ValueError`。依赖 scheduler 的能力不能替代 policy 的 `compute_loss` 分支。

代码先构造 `condition_mask`，将被条件化的位置重新写回 `cond_data`，然后：

$$\mathcal{L}_{\text{code}} = \text{mean}_{b,\tau,d}\left[(1 - m_{b,\tau,d})\left(\hat{y}_{b,\tau,d} - y_{b,\tau,d}\right)^2\right].$$

变量与实现：`F.mse_loss(..., reduction='none')` 后乘 `loss_mask = ~condition_mask`，再先对每个 batch 展平取 mean、最后 batch mean。被条件化元素的 numerator 为零，但 denominator 仍包含完整 tensor；因此这是 masked numerator + full-element denominator，不是 valid-count normalization。当前 commit 还明确 `assert 'valid_mask' not in batch`，不能把它说成支持 episode valid mask。

4. Boundary padding：重复边界值而非 valid-length mask

`SequenceSampler.sample_sequence` 在 sample 越过 episode 边界时分配完整 `sequence_length`，将左侧位置填成 `sample[0]`，右侧位置填成 `sample[-1]`，再把真实片段写回中间。以 episode 长度 2、`sequence_length=4`、右侧越界 2 为例，action 序列是：

```
真实:      [a0, a1]
采样结果:  [a0, a1, a1, a1]
valid mask: 没有随 sampler 一起返回
```

因此“边界位置已 padding，所以 loss 自动忽略”是错误答案；这个实现选择的是 boundary-repeat 数据合同。是否把重复值视作真实监督，必须结合训练 pipeline 另外证明，不能仅凭 `pad_after` 推断 valid-only loss。

5. Sampling、solver steps 与 action execution

推理 `conditional_sample` 每次先用与 $T_p \times D_a$ 同形状的 Gaussian 初始化，再调用 `scheduler.set_timesteps(num_inference_steps)`，对每个 scheduler timestep 运行一次网络和 `scheduler.step`。因此：

量	frozen field	语义
T_o	`n_obs_steps`	输入 observation 数
T_p	`horizon`	生成 trajectory 长度
T_a	`n_action_steps`	返回 action prefix 长度
K_d	`num_inference_steps`	一次 query 的 denoising/network forwards
R	runner/real timestamp loop	多久重新获取 observation 并 query

论文正文给出 DDIM 100 training iterations / 10 inference iterations 的 0.1s 例子；Table 7/8 的 real rows 使用 16 eval iterations；frozen `eval_real_robot.py` 在 diffusion 分支把 `policy.num_inference_steps = 16`。这些对象属于不同版本/配置，不能压成“Diffusion Policy 默认 10 步”或“16 步机器人控制”。

6. Warm-start：paper design ≠ frozen implementation

论文在 closed-loop action-sequence prediction 段落提出，receding horizon 可以用上一段 action sequence prediction warm-start 下一轮 inference。frozen UNet/Transformer hybrid 的 `conditional_sample` 始终 `torch.randn(...)`，`predict_action` 入口还执行 `assert 'past_action' not in obs_dict`。因此安全口径是：

1. warm-start 是论文提出的可选设计意图；
2. 该 frozen commit 的默认 inference 从 fresh Gaussian 开始；
3. 若本地实现 warm-start，仍需处理已执行 prefix、时间平移、noise level、新 observation 冲突与可复现性。

不要把“论文提到 warm-start”升级成“官方 frozen repo 默认启用 warm-start”。

7. Simulation 与 real execution 的边界

PushT runner 调用 `policy.predict_action`，把返回的 action batch 传给 `MultiStepWrapper.step`；wrapper 逐步执行 batch 并收集新 observation。这个路径把 T_a 作为一次环境 wrapper 的执行窗口。

real script 则把 policy 输出转成 timestamped target poses：用 observation timestamp 与控制周期 Δt 构造未来时间戳，删除已经过期的 action；若 inference 超过时间预算，只保留最后一个 target 并安排到下一个可用时刻。`steps_per_inference` 参与 cycle timing，不能仅凭提交的 action 数量断言“恰好执行 T_a 步后才 replan”。真实的 executed prefix 取决于 timestamp filtering、latency 与 queue。

8. Synthetic probes

Probe	结果	结论
$T_o=2, T_p=16, T_a=8$	`start=1,end=9`，返回 indices `[1..8]`	observation 对齐与 action prefix 分离
loss mask：总元素 8、仅 1 个元素误差 1	full mean `1/8=0.125`；valid-only mean `1`	mask 不等于 valid-count denominator
episode `[a0,a1]`，右 padding 2	`[a0,a1,a1,a1]`	sampler 重复边界值，不生成 valid mask
`prediction_type='v_prediction'`	policy branch raises `ValueError`	frozen policy 未实现 v-target
$T_p=16, T_a=8, K_d=16$	每次 query 最多 16 次 scheduler/model step，返回 8 行 action	solver steps 与 execution rows 分离
real action timestamps 中一半已过期	只保留未来 entries；全过期时保留最后一行	queue/replan 受 latency 与 wall-clock 影响

这些 probes 验证的是 shape、reduction、padding 与 branch semantics，不证明任何 learned checkpoint success、真实 latency 或个人项目 bug。

9. 对 `VLA-ACT-018..022` 与 `VLA-ACT-017` 的回写结论

Question ID	独立复核结论	必须保留的边界
`VLA-ACT-017`	ACT/token/Diffusion/Flow 的 target 与 sampler 对照可用	跨协议 success 不可直接排名
`VLA-ACT-018`	sequence-level conditional diffusion 与多模态直觉通过	不推断本地 diversity/success
`VLA-ACT-019`	epsilon/sample target、condition mask、full-mean reduction 通过	本地 valid mask 待取证
`VLA-ACT-020`	schedule、prediction type、T_p、K_d 分离通过	frozen policy 不支持 v-pred
`VLA-ACT-021`	$T_o/T_p/T_a$、sim wrapper 与 real timestamp queue 通过	本地 H/K/R 未证
`VLA-ACT-022`	DDPM/DDIM、10/16 step 版本差异与 warm-start gap 通过	不把论文 latency 迁移到本地硬件

10. 项目真实性边界与面试安全口径

上游 Diffusion Policy 可以作为 action diffusion、horizon 拆分与 0-success interface debugging 的对照；它不能证明 OpenPiVLA 使用同一 scheduler、`prediction_type`、$T_o/T_p/T_a$、warm-start、action axis、normalizer 或 real queue。要把这些写入个人项目回答，仍需本地 config、checkpoint、transform、controller trace、timestamped rollout 和 eval summary。

30 秒安全回答

“Diffusion Policy 把未来 T_p 步连续动作作为一个 trajectory，用 observation 条件化的 diffusion process 学 noisy action 的噪声或 clean sample。推理从同形状 Gaussian trajectory 开始，运行 K_d 次 scheduler step，再从 T_o-1 起切出 T_a 步执行；(T_o, T_p, T_a, K_d, R) 是不同时间轴。论文提出可用上一段预测 warm-start，但这个 frozen commit 的 policy 每次从 Gaussian 重启，并且 real script 还要经过 timestamp queue，所以我不会把 action batch 长度直接等同于 replan interval。”

11. 风险、未决项与下一步

1. **核心 public facts 已独立复核**：training target/mask、boundary repeat、horizon/slice、solver steps、warm-start gap 与 sim/real queue 已核对。
2. **视觉 QA**：当前环境的 `file://` 浏览器导航受策略限制，静态检查可以通过但不宣称浏览器视觉验收。
3. **本地 runtime**：OpenPiVLA scheduler、checkpoint、normalizer、action schema、H/K/R、controller 与 success protocol 仍待 artifact。
4. **下一 bounded slice**：进行一题真实 ACT/Diffusion 口述，或补一个本地 OpenPiVLA runtime artifact；不批量新增 VOL 03 题卡。

参考文献（References）

1. Chi et al., Diffusion Policy: Visuomotor Policy Learning via Action Diffusion，arXiv `2303.04137v5`。
2. real-stanford/diffusion_policy，frozen commit `5ba07ac6661db573af695b419a7947ecb704690f`，MIT。
3. Hugging Face diffusers v0.11.1，DDPM/DDIM scheduler source。
4. VOL 03 Diffusion Policy Claim-to-Code，本仓库先前 source freeze；本页为独立复核，不替代本地项目证据。

Part 10 / 16 · `interview_materials/vla/sources/vol_03_flow_matching_pi0_claim_to_code.html` · SHA-256 `8E7D1607F544BF7850A9D57C3033E2A401496B5DEDC834B9851C4C5F3D9FD04A`

返回本卷目录

Canonical Markdown SHA-256：`8e7d1607f544bf7850a9d57c3033e2a401496b5dedc834b9851c4c5f3d9fd04a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_flow_matching_pi0_claim_to_code.md`

VOL 03 Source Audit E：Flow Matching 基础、π0 特化与 ODE Claim-to-Code

审计日期：2026-07-19 状态：`independent_fact_reviewed / theorem_endpoint_time_sign_checked / local_runtime_pending / WARN` 基础论文：*Flow Matching for Generative Modeling*，arXiv `2210.02747v2` 后续官方包：`facebookresearch/flow_matching` frozen commit `11568d37f8d5a080e12aa7b5305d9c35ae07d136`，CC BY-NC 4.0 VLA 特化：*π0: A Vision-Language-Action Flow Model for General Robot Control*，arXiv `2410.24164v4`；`Physical-Intelligence/openpi` frozen commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`，Apache-2.0 对应题目：`VLA-ACT-023..030`；补全 `VLA-ACT-026` 的 Diffusion / Flow 对照 本页问题：Flow Matching 的 probability path、marginal / conditional vector field 与训练目标是什么？线性路径为什么只是特例？π0 如何把这套生成建模落到 `[B,H,D]` action chunk，openpi 为什么反向积分仍然等价？Flow 与 Diffusion 的 target、sampler、mask、guidance 和故障定位应如何严格区分？

1. 结论先行

1. **Flow Matching（FM）不是“直接回归最终 action”，而是回归给定 probability path 上的时间依赖速度场。**训练期随机取路径上的点 x_t，让 $v_\theta(x_t, t)$ 拟合目标 vector field；推理期再解 ODE 把 source/noise 推到 target/data。[1]
2. **原始 FM objective 依赖通常不可直接计算的 marginal vector field。**Conditional Flow Matching（CFM）改为采样 data-conditioned path 和 conditional velocity；论文 Theorem 2 证明 FM 与 CFM 对参数 θ 的梯度相同，而不是声称两项 loss 的逐样本数值相同。[1]
3. **线性插值 $x_t = (1-t)x_0 + tx_1$、target $x_1 - x_0$ 是 conditional OT path 的零端点噪声特例，不是全部 Flow Matching。**原论文 Eq. 20–23 实际保留 $\sigma_{\min} > 0$；当前官方包的 `CondOTProbPath` 才直接使用 $\sigma_t = 1 - t, \alpha_t = t$。[1][2]
4. **“simulation-free”只描述训练不需要先数值模拟 CNF trajectory。**采样仍要调用 ODE solver；solver method、步长、时间网格、NFE 和误差容限都会影响质量与 latency。[1][2]

5. **当前 `facebookresearch/flow_matching` 是后续维护的通用 PyTorch 包，不是 2022 论文发布时的历史实现快照。**它可补证现代 path / solver API，但不能倒推原论文当年的 exact code、default 或实验 recipe。[2]
6. **π0 把 data sample x_1 具体化为 normalized action chunk $A \in \mathbb{R}^{B\times H\times D}$，把 context 具体化为多视角图像、语言和 proprioception。**action expert 在每个 flow step 接收 noisy action 与 timestep，输出同形状 velocity。[3][4]
7. **π0 paper 与 openpi 的时间方向相反，但轨迹等价。**paper 从 $\tau = 0$ noise 到 $\tau = 1$ action，target 为 $A - \epsilon$；code 从 $t = 1$ noise 以负步长到 $t = 0$ action，target 为 $\epsilon - A$。令 $\tau = 1 - t$，时间和速度同时翻转，ODE 增量不变。[3][4]
8. **timestep sampling 也不能漏报。**基础 FM 论文写 $t \sim \mathcal{U}[0, 1]$；π0 使用 shifted Beta recipe 强调 high-noise 区域。openpi 的 `0.001 + 0.999 * Beta(1.5,1)` 是在反向时间坐标下对论文 recipe 的直接实现，不是无缘由偏离。[3][4]
9. **Flow 与 Diffusion 可以共享 Gaussian path 语境，但不应混成同一训练接口。**本仓库冻结的 Diffusion Policy 默认预测 discrete scheduler timestep 的 epsilon；π0 预测 continuous flow time 的 velocity，并用显式 Euler ODE update。FM 论文还明确允许 diffusion probability paths，因此“Flow 与 Diffusion 完全无关”和“Flow 只是少步 Diffusion”都不准确。[1][5]
10. **上游 loss 正常不能证明本地 rollout 正确。**openpi flow loss 对 configured D 维直接 mean，没有 action-valid-dimension mask；padding、norm stats、axis/unit、H/K、solver sign 和 controller queue 都可能让 offline loss 看似正常但 closed-loop 失败。[4]

2. 证据层级、冻结对象与适用边界

层级	冻结对象	本页使用	不能支持
基础论文	arXiv `2210.02747v2`，updated 2023-02-08	CNF、FM/CFM、gradient equivalence、Gaussian / OT conditional path	当前 package API、π0 action tensor、本地项目
后续官方包	`facebookresearch/flow_matching` commit `11568d...`	`AffineProbPath`、`CondOTProbPath`、velocity target、ODE solver、supporting CFG example	2022 paper 的历史 code/default；商业复用授权
π0 论文	arXiv `2410.24164v4`	action path、action expert、shifted Beta timestep、10-step flow sampling	frozen openpi 的逐行实现、本地 OpenPiVLA
openpi	commit `15a9616...`	`[B,H,D]` shape、loss、prefix/suffix、time reversal、Euler loop	用户项目的 checkpoint、axis、H/K、97.4%
Diffusion Audit D	paper/repo/dependency 已冻结	epsilon target、scheduler、$T_o/T_p/T_a/K$ 对照	用 diffusion 实现替代 flow 实现
同源推断	上述公式与代码联合推导	time/sign 等价、mask 污染、debug invariant	未运行的本地 success、latency 和个人贡献

版本与许可边界：

- Flow Matching v2 的 title / revision 已按 arXiv 页面冻结；正文 Section 2–4 与 Appendix B continuity equation 已逐式核验。[1]
- 当前 `flow_matching` clone HEAD 为 `11568d37f8d5a080e12aa7b5305d9c35ae07d136`，commit date 2026-01-05；README 说明它属于后来的 *Flow Matching Guide and Codebase*，并同时覆盖 continuous / discrete flow matching。[2]
- 该 package 根 LICENSE 是 Attribution-NonCommercial 4.0；所以它适合作为学习与审计来源，不能默认当作无限制商用依赖。[2]
- π0 / openpi 的完整 architecture、FAST、normalization 与 rollout 边界已在 Source Audit B 冻结；本页只重放与基础 FM 直接相关的最短证据链。[3][4]

3. CNF：路径、速度场与 continuity equation

3.1 从 ODE 定义 flow

Continuous Normalizing Flow（CNF）用时间依赖 vector field $v_t : \mathbb{R}^d \to \mathbb{R}^d$ 定义 flow map ϕ_t：

$$\frac{d}{dt}\phi_t(x) = v_t(\phi_t(x)), \qquad \phi_0(x) = x.$$

变量与直觉：$x \sim p_0$ 是 source sample，$\phi_t(x)$ 是沿速度场走到时间 t 的位置；实现中 v_t 就是神经网络或其 wrapper 的输出。公式来自基础论文 Eq. 1–2。[1]

flow map 将 source density push forward 成 probability path：

$$p_t = [\phi_t]_* p_0, \qquad p_1 \approx q.$$

其中 q 是目标数据分布。直觉上，模型不是直接记住一个终点函数，而是学习一组随时间变化的“往哪里走”的指令。实现映射：ODE solver 从 source sample 和 time grid 出发，多次调用 velocity model。[1][2]

3.2 continuity equation 是一致性条件

若 v_t 真正生成 p_t，二者满足：

$$\frac{\partial p_t(x)}{\partial t} + \operatorname{div}(p_t(x) v_t(x)) = 0.$$

变量与直觉：第一项是某点概率质量随时间变化，divergence 项是流入/流出的净通量；它表达“概率质量没有凭空产生或消失”。这是理论一致性条件，不是 π0 训练代码额外计算的一项 divergence loss。基础论文正文引用 continuity equation，完整式见 Appendix B Eq. 26。[1]

3.3 probability path 不是单条 sample trajectory

- p_t：时间 t 上整个随机变量的 density。
- $\phi_t(x_0)$：给定一个 source sample 的 deterministic ODE trajectory。
- $u_t(x)$：能生成目标 path 的 target vector field。
- $v_\theta(x,t)$：神经网络对 target field 的近似。

把这四者混在一起会导致两类常见错误：把 `x_t` 当成 loss target，或把一次 Euler 更新当成完整 probability distribution。

4. 从 FM 到 CFM：为什么可以逐 data sample 训练

4.1 Flow Matching objective

给定预先选择的 marginal probability path p_t 及其 target field u_t，基础 FM objective 为：

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x\sim p_t}\left[\|v_\theta(x,t) - u_t(x)\|_2^2\right].$$

变量与实现含义：t 是 continuous time，x 是 path sample，u_t 是监督 velocity。难点是 marginal p_t 与 u_t 通常包含对全部 data 的积分，不能为每个 minibatch point 直接算出。论文 Eq. 5 使用 uniform time。[1]

4.2 conditional path 如何组成 marginal path

令 $x_1 \sim q$ 为 data sample，为每个 x_1 设计 conditional probability path $p_t(x \mid x_1)$，则：

$$p_t(x) = \int p_t(x \mid x_1) q(x_1)\, dx_1.$$

变量与直觉：每个 data point 携带一条从 source/noise 到其邻域的 conditional path；把所有 data-conditioned paths 混合，就得到 marginal path。论文 Eq. 6–7 要求 p_0 是 source，p_1 逼近 target q。[1]

若 $u_t(x \mid x_1)$ 生成每条 conditional path，则 marginal vector field 可写为：

$$u_t(x) = \int u_t(x \mid x_1) \frac{p_t(x \mid x_1) q(x_1)}{p_t(x)} \, dx_1.$$

变量与直觉：权重是给定当前位置 x 时不同 data endpoint 的 posterior responsibility；所以多条 conditional velocity 在同一点会按概率混合。论文 Theorem 1 证明这个 field 生成前述 marginal path。[1]

4.3 Conditional Flow Matching objective

CFM 直接采样 x_1、conditional path point x 和可计算的 conditional velocity：

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}[0,1],\, x_1 \sim q, \\ x \sim p_t(\cdot \mid x_1)}} \left[\| v_\theta(x, t) - u_t(x \mid x_1) \|_2^2 \right].$$

变量与实现：minibatch data 是 x_1，新采样 noise 构造 x_t，解析式或 path API 给出 $u_t(x_t \mid x_1)$。基础论文 Eq. 9 / Theorem 2 证明，在 objectives 有限的条件下，FM 与 CFM 对 θ 的梯度相同。[1]

必须保留的限定：

- identical gradients 不等于每个 sample 的 target 相同；
- identical gradients 不等于两个 loss scalar 在日志中逐值相等；
- 结论依赖 conditional path / field 的构造确实生成所声明的 marginal path；
- π0 改用非 uniform timestep sampling，相当于选择了 task-specific time weighting，不能再无条件称为原始 uniform objective。

5. Gaussian conditional path 与 OT 特例

5.1 一般 affine Gaussian path

基础论文考虑：

$$p_t(x \mid x_1) = \mathcal{N}\big(x \mid \mu_t(x_1),\, \sigma_t(x_1)^2 I\big),$$

并用 $x_0 \sim \mathcal{N}(0, I)$ 的 affine map 采样：

$$\psi_t(x_0) = \sigma_t(x_1) x_0 + \mu_t(x_1).$$

变量与直觉：μ_t 控制 conditional mean，σ_t 控制 noise scale；不同选择可得到 diffusion path 或 non-diffusion path。实现映射：现代 package 的 `AffineProbPath.sample` 正是 `x_t = sigma_t * x_0 + alpha_t * x_1`。[1][2]

沿这个 sample map 的 conditional target velocity 是：

$$\dot{x}_t = \frac{d}{dt}\psi_t(x_0) = \dot{\sigma}_t(x_1)x_0 + \dot{\mu}_t(x_1).$$

变量与实现：`PathSample.dx_t` 保存这项 target；训练 example 用 `MSE(model(path_sample.x_t,t), path_sample.dx_t)`。这是 velocity prediction，不是 epsilon prediction 的同名替换。[2]

若只知道当前位置 x，论文还给出：

$$u_t(x \mid x_1) = \frac{\dot{\sigma}_t}{\sigma_t}(x - \mu_t) + \dot{\mu}_t.$$

这个表达把 conditional velocity 写成 x, x_1, t 的函数；$\sigma_t = 0$ 的端点需要按具体 path / solver 处理，不能机械在端点做除法。公式对应基础论文 Eq. 15。[1]

5.2 论文的 OT conditional path

基础论文的 OT path 取：

$$\mu_t(x_1) = tx_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min})t,$$

因此 sample path 与 target velocity 为：

$$x_t = [1 - (1 - \sigma_{\min})t]x_0 + tx_1, \qquad \dot{x}_t = x_1 - (1 - \sigma_{\min})x_0.$$

变量与边界：$t = 0$ 是 $x_0 \sim \mathcal{N}(0, I)$；$t = 1$ 仍有 $\sigma_{\min}x_0$，所以 conditional endpoint 是以 x_1 为均值的小 Gaussian，而不是严格 delta。对应论文 Eq. 20–23。[1]

5.3 常背的线性公式是 $\sigma_{\min} = 0$ 特例

当 $\sigma_{\min} = 0$：

$$x_t = (1 - t)x_0 + tx_1, \qquad u_t = x_1 - x_0.$$

变量与直觉：每对 noise/data endpoint 沿直线匀速移动；target 不依赖 t，但网络输出仍依赖当前位置和 condition，因为不同 endpoint pairs 在空间中会重叠。不能从“conditional target 常量”推出“marginal field 是全局常量”。[1]

当前官方 package 的 `CondOTScheduler` 明确返回：

```
# Modern package exact-endpoint convention.
alpha_t = t
sigma_t = 1 - t
d_alpha_t = 1
d_sigma_t = -1
```

代码块映射：`AffineProbPath.sample` 随后得到 `x_t=(1-t)x_0+tx_1`、`dx_t=x_1-x_0`。[2] 这可支持面试中的线性推导，但答题时要注明它是 package 的 exact-endpoint `CondOT` convention；不要反写成原论文所有公式都令 $\sigma_{\min} = 0$。

6. 后续官方包：path、loss 与 solver 的真实接口

6.1 `AffineProbPath` 返回什么

frozen package 的 `PathSample` 包含：

字段	语义	shape
`x_0`	source/noise sample	`[B,...]`
`x_1`	target/data sample	`[B,...]`
`t`	sampled continuous time	`[B]`
`x_t`	conditional path point	`[B,...]`
`dx_t`	conditional velocity target	`[B,...]`

`AffineProbPath.sample` 先从 scheduler 取 $\alpha_t, \sigma_t, \dot{\alpha}_t, \dot{\sigma}_t$，再计算：

$$x_t = \sigma_t x_0 + \alpha_t x_1, \qquad \dot{x}_t = \dot{\sigma}_t x_0 + \dot{\alpha}_t x_1.$$

实现证据为 `affine.py#L57-L92` 与 `scheduler.py#L104-L114`。变量说明：package 用 α_t 乘 target、σ_t 乘 source；这套命名必须跟具体 scheduler 一起读。

6.2 continuous FM MSE 不在 `flow_matching.loss` 中自动替你做

当前 package 的 continuous example 直接写：

```
def continuous_fm_step(model, path, x_1):
    # Sample source noise and one continuous time per data item.
    x_0 = torch.randn_like(x_1)
    t = torch.rand(x_1.shape[0], device=x_1.device)
```

```
    # The path object provides both the interpolated point and velocity target.
    sample = path.sample(x_0=x_0, x_1=x_1, t=t)

    # Continuous FM uses ordinary velocity MSE in this example.
    pred = model(sample.x_t, t)
    return ((pred - sample.dx_t) ** 2).mean()
```

代码块映射：`examples/image/training/train_loop.py` 用 `CondOTProbPath`、`path_sample.dx_t` 与 tensor MSE。[2] package 的 `flow_matching.loss` 目录此 commit 主要提供 discrete mixture path 的 generalized KL；所以“import 一个 FlowMatchingLoss 就自动处理连续 CFM”不是这个 frozen API 的事实。

6.3 ODE solver

frozen `ODESolver.sample` 接收 initial state、time grid、method、step size 与 tolerances，然后调用 `torchdiffeq.odeint`。Euler 情况为：

$$\hat{x}_{k+1} = \hat{x}_k + \Delta t\, v_\theta(\hat{x}_k, t_k).$$

变量与实现：Euler 只是 `method="euler"` 时的一阶近似；package 还可调用 `dopri5`、`midpoint`、`heun3` 等 torchdiffeq methods。`time_grid` 可升序或降序，方向必须与 velocity convention 配套。[2]

关键边界：

- fixed-step solver 要记录 step size / grid；adaptive solver 要记录 `atol`、`rtol` 与实际 NFE；
- training path sample 不需要先跑 ODE，但 inference 需要；
- solver 更高阶不自动修复错误 velocity target、time sign 或 normalization；
- package 默认 API 不等于 π0 的 deployment default，π0/openpi 手写固定 10-step Euler。

6.4 当前 package 的 CFG 只是 supporting example

image example 在训练时以 `class_drop_prob` 丢弃 label condition，推理时组合 conditional / unconditional velocities：

$$v_{\text{cfg}} = (1 + s)v_{\text{cond}} - s\, v_{\text{uncond}}.$$

变量与实现：s 是 `cfg_scale`；当 $s \neq 0$ 时，每次 ODE function evaluation 实际做两次 model forward。证据见 `train_loop.py#L68-L73` 与 `eval_loop.py#L36-L66`。[2]

这不是 action-policy 直接证据：frozen openpi Pio path 未实现 condition dropout / CFG。若迁移到 VLA，必须定义究竟 drop language、images、state 还是全部 condition，并评估 double forward

latency、velocity magnitude、action safety 与 task compliance；不能只复制一个 scale。

7. π0：把 CFM 特化为 action chunk generation

7.1 data、noise、condition 与 output shape

对 π0：

- data endpoint：normalized action chunk $A \in \mathbb{R}^{B\times H\times D}$；
- source：$\epsilon \sim \mathcal{N}(0, I)$，shape 同 A；
- condition：多视角图像、语言 prompt、proprioceptive state；
- model output：action expert velocity，shape `[B,H,D]`；
- inference endpoint：仍是 normalized action chunk，之后才 unnormalize、slice 并进入 controller。

论文路径为：

$$A^\tau = \tau A + (1-\tau)\epsilon, \qquad u_{\text{paper}} = A - \epsilon.$$

变量与直觉：$\tau = 0$ 是 noise，$\tau = 1$ 是 action；这是上一节 exact linear path 令 $x_0 = \epsilon, x_1 = A$ 的直接实例。π0 用 observation o 对 velocity model 条件化。[3]

训练目标可写为：

$$\mathcal{L}_{\pi_0} = \mathbb{E}\left[\frac{1}{HD}\|v_\theta(A^\tau, \tau, o) - (A-\epsilon)\|_2^2\right].$$

变量与代码映射：论文写 action flow objective；openpi `compute_loss` 对最后 action dimension D 先 mean，`scripts/train.py` 再对 batch 与 horizon mean，最终覆盖 configured `[B,H,D]` 元素。若 D 含 zero-padding，这个公式也会监督 padded elements，除非另有 mask；frozen Pio path 未见该 mask。[3][4]

7.2 VLM prefix 与 action suffix

frozen openpi 的信息流：

```
# Prefix is condition and is cached during sampling.
images + language
        |
        v
PaliGemma/VLM prefix tokens ------------------+
                                              |
state token + noisy actions [B,H,D] + time ---+--> action expert
                                                      |
```

```
                                                                  v
                                                          velocity [B,H,D]
```

代码块说明：`action_in_proj` 把每步 D 维 noisy action 投到 expert width，time embedding 与 action token 融合；`action_out_proj` 投回 D 维 velocity。采样时 prefix KV cache 只建一次，每个 Euler step 重算 suffix。[4]

注意 π0 与 π0.5 分支差异已在 Source Audit B 详述；本页的基础公式不证明 frozen `Pi0(pi05=True)` 完整实现 π0.5 hybrid CE objective。

7.3 shifted Beta timestep 与 openpi 的反向坐标

基础 FM 论文使用 uniform time。π0 Appendix 改为强调 high-noise 区域的 shifted Beta：

$$y \sim \mathrm{Beta}(1.5, 1), \qquad \tau = s(1-y), \qquad s = 0.999.$$

变量与直觉：较大的 y 映射到较小 τ，也就是更接近 noise 的 action path point；这改变了不同 flow times 在训练 loss 中的权重。π0 还截掉 $\tau > s$ 的极低噪声末端。[3]

令 openpi code time $t = 1 - \tau$，则：

$$t = 1 - s(1-y) = 0.001 + 0.999y.$$

这正对应 `time = Beta(1.5,1) * 0.999 + 0.001`。所以 code 的 Beta sample 不是和 π0 paper 冲突；它是在反向时间坐标中的等价写法。[3][4]

8. openpi 时间反转、loss 与 Euler sampling

8.1 code training convention

frozen `Pi0.compute_loss` 构造：

$$x_t = t\epsilon + (1-t)A, \qquad u_{\text{code}} = \epsilon - A.$$

变量与实现：$t = 1$ 是 noise，$t = 0$ 是 action；`noise`、`x_t`、`u_t` 均为 `[B,H,D]`。action expert 的直接输出 v_t 与 u_{code} 做 MSE。[4]

8.2 与 paper 的严格等价

令 $\tau = 1 - t$：

$$x_{\text{code}}(t) = t\epsilon + (1-t)A = (1-\tau)\epsilon + \tau A = x_{\text{paper}}(\tau),$$

$$u_{\text{code}} = \epsilon - A = -u_{\text{paper}}, \qquad d\tau = -dt.$$

变量与直觉：同一路径上的 point 没变；velocity 和 time differential 都取反，因此：

$$u_{\text{code}}\, dt = u_{\text{paper}}\, d\tau.$$

这才是“反向时间仍等价”的完整理由。只说 `t=1` 到 `0` 而不说 velocity sign，会留下方向错误。[3] [4]

8.3 explicit Euler

`sample_actions` 默认 $N = 10$，令 $\Delta t = -1/N$：

$$x_{k+1} = x_k + \Delta t\, v_\theta(x_k, t_k, o), \qquad t_{k+1} = t_k + \Delta t.$$

变量与实现：初值在代码中名为 `noise`，time 从 1 开始；循环结束返回 time 0 的 action estimate。`num_steps` 是一次 policy query 内的 flow integration steps，不是 action horizon H、执行 prefix K 或机器人 control steps。[4]

```
def openpi_flow_sample(observation, model, num_steps=10):
    # Cache image/language prefix once for this policy query.
    prefix_cache = model.encode_prefix(observation)

    # Initialize the whole normalized action chunk from Gaussian noise.
    x = randn(shape=[batch, action_horizon, action_dim])
    t = 1.0
    dt = -1.0 / num_steps

    while t >= -dt / 2:
        # Recompute state/noisy-action/time suffix at each flow step.
        velocity = model.action_expert(
            prefix_cache, observation.state, noisy_actions=x, time=t
        )

        # Explicit Euler follows code-time velocity toward data time 0.
        x = x + dt * velocity
        t = t + dt

    # Still normalized model-space [H,D]; policy transforms run afterward.
    return x
```

伪代码映射 `src/openpi/models/pi0.py` 的 `embed_suffix`、`compute_loss` 与 `sample_actions`；unnormalize、dataset output transform 和 execution queue 不在 model method 内完成。[4]

9. Diffusion noise prediction 与 Flow velocity prediction

9.1 不能只用“都是加噪再还原”回答

维度	Frozen Diffusion Policy	π0 / openpi Flow
training time	discrete scheduler index k	continuous flow time t/τ
path construction	scheduler `add_noise` with $\bar{\alpha}_k$	explicit linear interpolation
default target	Gaussian epsilon	endpoint displacement / velocity
model output	scheduler interprets epsilon/sample	ODE vector field
inference init	Gaussian trajectory	Gaussian action chunk
update	DDPM/DDIM `scheduler.step`	explicit Euler $x + \Delta t v$
time count	diffusion iterations K_d	flow NFE / steps N_f
horizon	T_p plus action slice T_a	H plus execution prefix/replan K
conditioning	visual/state global or inpainting path	VLM prefix + state/action suffix
frozen warm-start	paper option，code random restart	random init unless noise supplied

Diffusion Audit D 的默认 objective：

$$\mathcal{L}_\epsilon = \mathbb{E}\left[\|\epsilon - \epsilon_\theta(O, A^k, k)\|_2^2\right].$$

π0 flow objective：

$$\mathcal{L}_v = \mathbb{E}\left[\|u_t - v_\theta(o, x_t, t)\|_2^2\right].$$

变量与区别：两者都可用 MSE，也都可从 Gaussian 初始化，但 target 的定义、time parameterization 和 sampler update contract 不同。FM 基础论文允许用 diffusion probability paths，所以“path family”与“prediction parameterization / sampler”应分别比较。[1][4][5]

9.2 采样步数不是本质定义

- Flow 可以用很多 ODE steps；Diffusion 也可以用少步 DDIM。
- π0 默认 10 steps、Diffusion Policy frozen config 的 8/16/100 都是具体 recipe，不是方法定义。
- latency 要拆 vision/prefix encode、每次 model NFE、solver/scheduler overhead、transport 与 controller queue。
- 质量要同时看 offline velocity/noise error、action reconstruction、closed-loop success 与 safety，不可只比较 step 数。

10. Action valid dimensions、padding 与 guidance

10.1 frozen Pio loss 没有 action-valid-dimension mask

若 environment action $a \in \mathbb{R}^d$ 被 padding 到 model dimension $D \geq d$，理想 masked flow loss 可写为：

$$\mathcal{L}_{\text{masked}} = \frac{\sum_{b,h,j} m_{b,h,j}(v_{b,h,j} - u_{b,h,j})^2}{\sum_{b,h,j} m_{b,h,j} + \varepsilon_0}.$$

变量与实现要求：$m_{b,h,j} = 1$ 才是有效 timestep / action axis；ε_0 防止空 mask 除零。mask 必须来自 dataset schema、sequence validity 或 action-dimension metadata，不能由 output slicing 反推。[4]

frozen openpi Pio 实际先：

$$\ell_{b,h} = \frac{1}{D}\sum_{j=1}^{D}(v_{b,h,j} - u_{b,h,j})^2,$$

训练脚本再对 b, h mean。变量与风险：configured D 中的 padded zero targets 也进入 denominator 和 gradient；`PadStatesAndActions` 与推理 output slice 不能替代 loss mask。它未必造成训练错误——模型可以学输出零——但会改变有效维 loss 权重，并可能让平均 loss 被大量容易 padding 维稀释。[4]

10.2 mask 的四个层次

mask / slice	作用阶段	能解决	不能解决
image/prompt attention mask	model input	缺失 modality / token padding	action loss validity
action timestep mask	training loss	episode/chunk padding	action axis 语义
action dimension mask	training loss	$d < D$ 的无效轴	controller reorder
output slicing / transform	inference	model D 到 env d	训练期 padding gradient

10.3 conditional dropout / guidance 的 action 风险

Flow package image example足以支持 CFG 机制公式，但不足以证明 openpi action policy 已用 CFG。若要为 action generation 补做：

1. 明确 dropout unit：只 drop language，还是同时 drop images/state；
2. 训练 conditional 与 unconditional branches 使用同一 action/noise/time；

3. 记录实际 model forwards，而不是只报 solver NFE；
4. 检查 guidance 后 normalized action magnitude、axis safety 和 temporal smoothness；
5. 比较 task compliance、diversity、success 与 latency，而不是只看 velocity MSE。

因此 `VLA-ACT-029` 当前是 source-ready with warning：有通用 flow CFG supporting code，没有 frozen action-policy positive evidence。

11. Loss 正常但 rollout 失败：四层 invariant debugging

11.1 Layer A：target / path algebra

先用固定 A, ϵ, t 做 hand-check：

$$x_t = t\epsilon + (1-t)A, \qquad u_t = \epsilon - A.$$

检查 endpoint、shape、dtype、time distribution 和 target sign；用 oracle field $v = u_t$ 做一次完整 Euler integration，确认能从同一 noise 回到 A。这一步失败，问题在 target/path/solver，不应先调 controller。[4]

11.2 Layer B：solver / scheduler

Flow 必查	Diffusion 必查
time grid 升降序	train/inference scheduler 是否匹配
velocity sign 与 dt	prediction type epsilon/sample/v
Euler steps / adaptive NFE	beta schedule / DDIM steps
endpoint 与 off-by-one	`set_timesteps` / step order
fixed noise reproducibility	generator / stochastic variance

先 overfit tiny batch，再用相同 observation 和固定 noise 对比：一步误差、十步 endpoint、不同 NFE 收敛趋势。若增加 flow steps 反而稳定发散，优先怀疑 sign、scale 或 time conditioning。

11.3 Layer C：normalization、mask 与 action schema

执行：

- normalize → unnormalize round-trip；
- q01/q99 或 min/max 的 checkpoint / dataset ID 对齐；
- 每轴 sentinel：只激活一个 action dimension，追到 controller；
- d/D、padding、loss denominator 与 output slice 对齐；

- horizon padding / episode boundary 是否进入 loss；
- action unit、position/velocity、delta/absolute 与 gripper convention。

offline loss 对 normalized tensor 正常，只能证明模型拟合了这个 tensor contract；它不证明 tensor 与真实 controller contract 一致。

11.4 Layer D：online controller / queue

固定一段已知安全 action，不经过模型直接走 output transform 和 controller，检查：

- H 是 model output horizon，K 是执行/replan prefix；
- control frequency、timestamp、queue drop 和 network latency；
- axis reorder、clip、安全限幅与 termination；
- camera resize/flip、state timestamp 与 action observation alignment；
- rollout summary 绑定 exact checkpoint、task、seed、trial count。

```
def debug_flow_rollout(batch, policy, controller):
    # A. Verify path algebra with fixed data/noise/time and an oracle velocity.
    assert_oracle_integration_recovers_action(batch.actions)

    # B. Freeze observation and noise; compare solver endpoints across step counts.
    endpoints = sweep_flow_steps(policy, batch.observation, fixed_noise=True)
    check_convergence_and_time_sign(endpoints)

    # C. Round-trip every valid action axis through normalization and transforms.
    assert_action_transform_roundtrip(batch.action_schema)
    run_per_axis_sentinel_tests(policy.output_transform)

    # D. Bypass the model and execute a known-safe controller sequence.
    assert_controller_contract(controller, batch.safe_action_fixture)
```

伪代码说明：四层按因果距离从模型 objective 到真实控制排列；每层产生可保存的 test artifact。它是面试中的 debug framework，不代表这些 tests 已在本地 OpenPiVLA 跑过。

12. Claim-to-code 总表

Claim	基础论文	frozen package / openpi	判定	安全面试口径
FM 直接回归 clean data/action	velocity field	`dx_t` / `u_t` MSE	rejected	回归 path velocity
FM training 要先解 ODE	simulation-free training	path sample 解析构造	rejected	inference 才解 ODE
FM 完全不需要 solver	CNF sampling	`odeint` / Euler loop	rejected	sampling 需要
CFM loss 与 FM 每项数值相等	identical gradients	conditional sample MSE	rejected	对 θ 梯度相同
所有 FM 都是线性插值	general Gaussian paths	多 scheduler	rejected	linear CondOT 是特例
原论文 OT endpoint 必为 exact data	保留 $\sigma_{\min}$	package exact endpoint	rejected	报 paper/package 差异

Claim	基础论文	frozen package / openpi	判定	安全面试口径
current package 是 2022 paper 历史代码	无	2026 maintained package	rejected	只作现代 API 补证
`flow_matching.loss` 自动提供 continuous MSE	objective in paper	continuous example手写 MSE	rejected	path 给 target，用户算 MSE
π0 flow target 是 epsilon	$A - \epsilon$ velocity	code $\epsilon - A$	rejected	是 time-reversed velocity
paper/code 反向所以不等价	paper 0→1	code 1→0	rejected	time 与 velocity 同时翻转
openpi Beta time 偏离 π0	shifted Beta	reverse-coordinate exact mapping	rejected	公式换元后对齐
10 steps 是 action horizon	10 integrations	`num_steps`	rejected	flow NFE 与 H/K 分开
padded D 不影响 loss	无该保证	Pi0 all-D mean	rejected as universal	需 explicit mask/evidence
output slice 等于 loss mask	无	transform 与 loss 分离	rejected	train/infer 分层
package CFG 证明 π0 已用 guidance	generic image example	openpi 无对应 branch	rejected	仅 supporting mechanism
Flow 比 Diffusion 快是定义事实	无	recipe-dependent	rejected	报 model/NFE/hardware
upstream 可证明本地 97.4%	无	无 local eval	rejected	必须绑定本地 artifact

13. 对 `VLA-ACT-023..030` 的支持状态

ID	覆盖	状态	仍缺
VLA-ACT-023	probability path、CNF、velocity field、continuity intuition	source-ready	independent review
VLA-ACT-024	CFM、linear/OT path、target velocity 与 MSE 推导	source-ready with paper/package boundary	手算题与口述
VLA-ACT-025	ODE solver、Euler、time grid、π0 10-step loop	source-ready	target hardware telemetry
VLA-ACT-026	epsilon prediction vs velocity prediction、scheduler vs ODE	source-ready	同 checkpoint controlled comparison
VLA-ACT-027	VLM prefix、state/noisy action/time suffix、`[B,H,D]` output	source-ready	local OpenPiVLA model class
VLA-ACT-028	all-D mean、理想 mask、padding/slice 分层	source-ready with warning	positive local mask implementation
VLA-ACT-029	generic flow CFG training/inference mechanism	source-ready with warning	action-policy implementation与安全实验

ID	覆盖	状态	仍缺
VLA-ACT-030	target/solver/norm/controller 四层 debug	source-ready as framework	本地 tests、logs 与 failure traces

`source-ready` 只表示可进入后续知识页与题卡起草；本页不创建题卡，也不改变 `unseen` 练习状态。

14. 与个人 OpenPiVLA 项目的真实性边界

上游可确认	可用于本地核验问题	不能直接迁移
π0 linear flow path	本地 `x_t` / target / time convention	不证明本地 model 是 Pio
openpi `[B,H,D]` action expert	查本地 forward / loss / config	不证明 H、D 数值
shifted Beta timestep	查本地 sampler / seed	不证明本地采用同分布
default 10 Euler steps	查 runtime override	不证明本地 latency
prefix cache / suffix recompute	查本地 architecture	不证明 Qwen action head
all-D mean risk	查 mask / padding test	不证明已发生污染
package CFG example	设计 advanced ablation	不证明本地 guidance
public solver behavior	设计 oracle integration test	不证明本地 97.4%

进入项目强回答前仍需本地 artifact：

1. exact model class、base checkpoint 与 startup manifest；
2. data action shape、axis/order/unit、normalization stats 与 output transform；
3. `compute_loss` / target builder / timestep sampler / valid mask；
4. model H, D、真实 d、execution prefix K 与 control frequency；
5. solver type、runtime steps、fixed-noise test 与 latency breakdown；
6. exact eval checkpoint、task/split、trials、seed、success summary；
7. upstream/local diff 与本人设计、实现、实验贡献边界。

在这些证据缺失时，安全说法是：“我能用 Flow Matching / π0 的公开机制解释候选 action head，并已把需要核验的 target、time、mask、normalization 与 controller contract 列成检查项”；不能说“我主导设计了 π0 action expert”或“上游默认证明本地达到 97.4%”。

15. 面试安全口径与危险答案

15.1 30 秒安全骨架

“Flow Matching 先选一条从 noise distribution 到 data distribution 的 probability path，再训练网络预测这条路径上的 velocity field。直接的 marginal field 难算，所以 CFM 对每个 data sample 构造

conditional path；论文证明 CFM 与 FM 对参数的梯度相同。最常用的 CondOT 特例是 $x_t = (1-t)x_0 + tx_1$，target 是 $x_1 - x_0$，但它不是全部 FM。π0 把 x_1 换成整个 action chunk，用图像、语言和 state 作 condition；openpi 反向时间积分，因此 target 和步长一起取反，默认 10 次 Euler。与 Diffusion Policy 相比，它预测 continuous velocity 并解 ODE，而 frozen Diffusion Policy 默认预测 epsilon 并走 scheduler；两者的 horizon、sampling steps 和 controller replan 都必须分开。”

15.2 2 分钟展开锚点

1. CNF ODE 与 p_t；
2. FM marginal target 难算；
3. CFM conditional sample 与 gradient equivalence；
4. Gaussian affine path → CondOT linear special case；
5. π0 `[B,H,D]`、shifted Beta time、VLM/action expert；
6. openpi time/sign/Euler 等价；
7. Diffusion epsilon/scheduler 对照；
8. mask/norm/controller 的 closed-loop 防线。

15.3 高危说法

- “Flow Matching 直接预测 clean action。”——它预测 velocity。
- “CFM 和 FM loss 数值完全一样。”——论文证明 gradient equivalence。
- “所有 Flow Matching 都是线性插值。”——一般 Gaussian / diffusion paths 都可用。
- “原论文 CondOT 就严格到 data endpoint。”——Eq. 20 保留 $\sigma_{\min}$。
- “simulation-free 表示推理也不用迭代。”——sampling 仍解 ODE。
- “openpi 和 π0 paper 方向相反，所以代码错了。”——time 和 velocity 同时翻转。
- “Beta timestep 是 openpi 自己改坏的。”——反向换元后与 π0 recipe 对齐。
- “10 是 action horizon。”——是 flow integration steps。
- “Flow 本质上一定比 Diffusion 少步/更快。”——取决于 model、solver、NFE 和硬件。
- “padding 最后 slice 掉就不影响训练。”——loss 已可能监督 padding。
- “官方 flow package 有 CFG，所以 π0 默认有 guidance。”——没有对应 frozen action path 证据。
- “loss 降了说明 controller 没问题。”——norm、axis、H/K 和 queue 仍可错。

16. 风险、未决项与下一步

1. 核心高风险事实已独立复核：Theorem 2、paper `sigma_min` endpoint、π0/OpenPI time-sign-dt、shifted Beta 与 deterministic oracle 已在 独立复核页 闭合，公共事实层为 `PASS WITH`

`WARNING`；local runtime、mask、controller、ownership 与 oral drill 仍使本页整体保持 `WARN/PENDING`。

2. **arXiv source tar 不可靠**：下载对象达到服务器 Content-Length，但本地 tar 读取报告 truncated；公式最终以完整 ar5iv/arXiv HTML v2 的 equation anchors 核验，不能声称 TeX source 已成功解包。
3. **current package 不是 historical implementation**：只把它用于 modern path/solver/CFG API evidence。
4. **$\sigma_{\min}$ 边界要保留**：题卡中同时给 paper form 与 exact-linear special case。
5. **CFG 只到 supporting evidence**：进入 action strong answer 前需本地实现与 safety/latency ablation。
6. **mask 仍缺 positive local evidence**：Source Audit B/E 只能证明 frozen Pi0 all-D mean 与风险，不能虚构已实现 valid-dim mask。
7. **题卡已分批生成**：累计 Batch 2B 已完成本页对应的 `VLA-ACT-023..030`，8 张新卡均保持 `unseen`；后续只做 independent review 与真实口述，不倒填 mastery。
8. **个人项目继续等待 artifact**：checkpoint、action schema、solver、H/K、97.4% 与个人贡献不得由 upstream 代证。
9. **视觉 QA 已完成**：通过 localhost 静态服务检查桌面 `1440×1000` 与窄屏 `800×1000`；确认一处 H1、academic palette/serif、desktop sticky TOC、narrow static TOC、公式无 `<em>` 污染且无页面级横向溢出。`arXiv source tar` 的 truncated 仍保留为独立证据风险。

参考文献（References）

1. Lipman et al., Flow Matching for Generative Modeling, arXiv `2210.02747v2`, 2023；重点为 Section 2 CNF、Section 3 FM / CFM、Section 4 conditional paths 与 Appendix B continuity equation。
2. facebookresearch/flow_matching, frozen commit `11568d37f8d5a080e12aa7b5305d9c35ae07d136`, CC BY-NC 4.0；核心路径：`affine.py`、`scheduler.py`、`ode_solver.py`、`continuous train example`、`CFG eval wrapper`。
3. Black et al., π0: A Vision-Language-Action Flow Model for General Robot Control, arXiv `2410.24164v4`, 2024；重点为 model/action flow 与 timestep sampling appendix。
4. Physical-Intelligence/openpi, frozen commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`, Apache-2.0；核心路径：`pi0.py`、`pi0_config.py`、`train.py`。
5. Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code，本仓库 Source Audit D，2026-07-16。
6. π0 / π0.5 / FAST / openpi Claim-to-Code，本仓库 Source Audit B，2026-07-16。

7. ACT CVAE、Action Chunking 与 Temporal Aggregation Claim-to-Code，本仓库 Source Audit C，2026-07-16。
8. Flow Matching / π0 Time–Sign 独立事实复核，本仓库二次 primary/frozen-source 复核与 deterministic oracle，2026-07-19。

Part 11 / 16 · `interview_materials/vla/sources/vol_03_flow_pi0_time_sign_independent_fact_review.html` · SHA-256 `EA075428DA74B129B654CEE3FA4DDA7E6C0E90F3B5672E6CF1046CE5B5A62D2E`

返回本卷目录

Canonical Markdown SHA-256：`ea075428da74b129b654cee3fa4dda7e6c0e90f3b5672e6cf1046ce5b5a62d2e`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_flow_pi0_time_sign_independent_fact_review.md`

VOL 03 Source Audit E：Flow Matching / π0 Time–Sign 独立事实复核

状态：`independent_fact_reviewed / audit_e / theorem_endpoint_time_sign_checked / PASS WITH WARNING` 审查日期：2026-07-19 冻结版本：Flow Matching paper arXiv `2210.02747v2`；π0 paper arXiv `2410.24164v4`；OpenPI commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`。 结论先行：Flow Matching v2 的 Theorem 2 证明 FM 与 CFM objectives 在假设下相差一个与参数无关的常数，因此参数梯度相同；原论文 OT conditional path 在 data endpoint 保留 `sigma_min` 噪声。π0 paper 与 frozen OpenPI 使用互为反向的时间坐标和速度符号，配合相反积分步长后轨迹完全等价；10-step deterministic oracle 在两套约定下都恢复了同一 action endpoint。

1. 直接复核来源与固定摘要

ID	来源	本次直接读取内容	固定摘要 / Verdict
Po1	Flow Matching v2 PDF	Eq. 9、Theorem 2、Eq. 20–23	SHA-256 `5EEB39BA516396924ABA4787452F9D0ABDEE88467A4D0C264D9F66CAD0C5EE14` / `primary-verified`
Po2	π0 v4 PDF	action path、velocity target、Euler sampling、shifted Beta	SHA-256 `FBDFBA56258BDBD220207BA63410C6C943E43A73E97A7A5A47CA0BC74204F821` / `primary-verified`
Co1	`openpi/models/pi0.py`	`compute_loss`、`sample_actions`、time/sign/dt	raw file SHA-256 `24B32D7C6ED5E409459AFA8CA5AF96A4DAE09C2DCDF48E828907E411F4F12342` / `official-code / frozen`
Oo1	本次 deterministic oracle	paper/code 10-step endpoint 与 Beta coordinate identity	`local-evidence / PASS`

这些摘要只固定本次下载对象与 frozen raw file；它们不替代 arXiv revision、Git commit 或本地项目 checkpoint 的身份记录。

2. Theorem 2：相差常数与梯度相同

Flow Matching 的 marginal objective 与 Conditional Flow Matching objective 分别写为：

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,\, x\sim p_t} \| v_\theta(x,t) - u_t(x) \|_2^2,$$

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\, x_1\sim q,\, x\sim p_t(\cdot|x_1)} \| v_\theta(x,t) - u_t(x \mid x_1) \|_2^2.$$

变量与直觉：`FM` 监督通常不可直接计算的 marginal vector field；`CFM` 对每个 data endpoint 采 conditional path 与解析 velocity。论文 Theorem 2 的原始限定是：若对所有 $x \in \mathbb{R}^d, t \in [0,1]$ 都有 $p_t(x) > 0$，则两项目标相差一个与 θ 无关的常数，因此

$$\nabla_\theta \mathcal{L}_{\mathrm{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\mathrm{CFM}}(\theta).$$

面试中应说“期望目标相差参数无关常数、优化梯度相同”，不能说“每个样本的 target 或 loss 数值相同”。原 Audit E 的 gradient-equivalence 口径正确，本次补全了 theorem assumption 与 constant boundary。

3. 原论文 OT path 的 `sigma_min` endpoint

Flow Matching v2 Eq. 20 选择：

$$\mu_t(x_1) = t x_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min}) t.$$

对应 sample path 与 conditional target 为：

$$x_t = [1 - (1 - \sigma_{\min}) t] x_0 + t x_1, \qquad \dot{x}_t = x_1 - (1 - \sigma_{\min}) x_0.$$

变量与端点：$x_0 \sim \mathcal{N}(0, I)$ 是 source noise，x_1 是 data sample；当 $t = 1$ 时 $x_1 + \sigma_{\min} x_0$，即以 x_1 为均值、标准差为 $\sigma_{\min}$ 的小 Gaussian。常背的

$$x_t = (1-t) x_0 + t x_1, \qquad \dot{x}_t = x_1 - x_0$$

是令 $\sigma_{\min} = 0$ 的 exact-endpoint 特例。原 Audit E 已正确区分 paper convention 与后续 package 的 exact-linear `CondOTProbPath`，本次直接 PDF 复核未发现需要反向修正的事实错误。

4. π0 paper 与 OpenPI code 的时间坐标

π0 paper 以 $\tau = 0$ 表示 noise、$\tau = 1$ 表示 action：

$$A^\tau = \tau A + (1-\tau)\epsilon, \qquad u_{\text{paper}} = A - \epsilon.$$

变量与实现：$A, \epsilon \in \mathbb{R}^{B\times H\times D}$，论文从 $A^0 = \epsilon$ 用正步长 Euler 积分到 $A^1 = A$，实验使用 10 steps、$\delta = 0.1$。

frozen `pi0.py` 在 `compute_loss` 中使用：

$$x_t = t\epsilon + (1-t)A, \qquad u_{\text{code}} = \epsilon - A,$$

并在 `sample_actions` 中从 $t = 1$ noise 以 $\Delta t = -1/N$ 积分到 $t = 0$ action。令

$$t = 1 - \tau, \qquad dt = -d\tau, \qquad u_{\text{code}} = -u_{\text{paper}},$$

则每个 infinitesimal displacement 满足

$$u_{\text{code}}\, dt = u_{\text{paper}}\, d\tau.$$

因此三件事必须一起检查：time coordinate、velocity target sign、solver step sign。只看到 paper/code 时间相反不能判定 code 错误；若只反转其中一项，才会把 endpoint 推离 action。

5. Shifted Beta 换元

π0 Appendix 令 $s = 0.999$，其采样可以写成：

$$y \sim \text{Beta}(1.5, 1), \qquad \tau = s(1-y).$$

OpenPI 的反向 time coordinate 因而是：

$$t = 1 - \tau = 1 - s + sy = 0.001 + 0.999y.$$

这与 frozen code 的 `jax.random.beta(..., 1.5, 1) * 0.999 + 0.001` 精确对应。固定 $y \in \{0, 0.1, 0.5, 0.9, 1\}$ 的 probe 均得到 $t + \tau = 1$；这证明坐标映射，不证明任意本地模型沿用该 sampler。

6. Deterministic oracle probe

本次用固定向量

$$A = [0.25, -0.50, 0.75, 1.00], \qquad \epsilon = [-1.00, 0.50, 0.25, -0.75]$$

运行两个常速度 10-step Euler oracle：

```
# Paper coordinate: tau goes from 0 to 1 and velocity is A - eps.
x_paper = eps.copy()
for _ in range(10):
    x_paper += 0.1 * (A - eps)

# OpenPI coordinate: t goes from 1 to 0 and velocity is eps - A.
x_code = eps.copy()
for _ in range(10):
    x_code += -0.1 * (eps - A)
```

检查	结果
paper endpoint	`[0.25, -0.5, 0.7500000000000001, 1.0000000000000002]`
code endpoint	`[0.25, -0.5, 0.7500000000000001, 1.0000000000000002]`
两者最大绝对 endpoint error	`2.220446049250313e-16`
每步 displacement 是否相同	`True`
五个固定 Beta coordinate 的 $t + \tau$	均为 `1.000000`

这里的误差仅来自 float64 round-off。probe 验证的是代数与积分方向，不是 learned velocity accuracy、真实 solver latency 或 closed-loop success。

7. 对 ACT-023..030 的回写结论

题目	本次结论	保留边界
`VLA-ACT-023`	FM/CFM gradient equivalence 正确；补 theorem positivity assumption 与 constant boundary	不等于逐样本 loss 相同
`VLA-ACT-024`	paper `sigma_min` endpoint 与 exact-linear 特例区分正确	后续 package 不是 2022 historical implementation

题目	本次结论	保留边界
`VLA-ACT-025`	Euler/NFE/H/K 分离正确	oracle 不能代替 runtime trace
`VLA-ACT-026`	diffusion target/scheduler 与 flow velocity/ODE 分离正确	不做无 matched protocol 的速度排名
`VLA-ACT-027`	π0/OpenPI time、velocity、dt 与 Beta 换元正确	local OpenPiVLA class/sampler 未证实
`VLA-ACT-028`	frozen Pi0 all-D mean 的风险描述保持 WARN	本次未补 positive local valid-dim mask
`VLA-ACT-029`	generic CFG 仍仅是 supporting example	frozen OpenPI action path 未实现 CFG
`VLA-ACT-030`	oracle-first 四层调试顺序可保留	未运行本地 controller/rollout tests

8. 不能推出的项目事实

- 不推出 OpenPiVLA 使用 frozen `Pi0` class、同一 timestep sampler、10-step Euler 或相同 H, D。
- 不推出本地 checkpoint、normalizer、axis/unit、valid mask、execution prefix K、controller 或 97.4% protocol 已闭合。
- 不把 upstream time/sign convention、同一 agent 的推导或本次 oracle 归为候选人的个人设计贡献。
- 不改变 44 张 VOL 03 卡或全局 280 张卡的 `unseen` 状态，也不创建 `coaching_state.md`。

9. 下一步

Source Audit E 的 theorem/endpoint/time-sign/Beta 高风险事实已完成独立复核，公共事实层为 `PASS WITH WARNING`；VOL 03 项目层仍因 local runtime、mask、controller、ownership 和 oral drill 缺口保持 `WARN/PENDING`。下一 bounded slice 优先 Source Audit B 的 π0/π0.5/FAST/OpenPI 其余高风险项，或进行一题真实口述。

参考入口

- Flow Matching / π0 Source Audit E
- π0 / π0.5 / FAST / OpenPI Source Audit B
- VOL 03 Completion Reconciliation
- VOL 03 Knowledge
- VOL 03 Questions
- OpenPiVLA Inventory

- OpenPiVLA Ownership Ledger

Part 12 / 16 · `interview_materials/vla/sources/vol_03_openvla_claim_to_code.html` · SHA-256 `8A66CE90E10BF60E015EE46CC78D9B1756048F386A38DF679607B0CABD17974F`

返回本卷目录

Canonical Markdown SHA-256：`8a66ce90e10bf60e015ee46cc78d9b1756048f386a38df679607b0cabd17974f`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_openvla_claim_to_code.md`

VOL 03 Source Audit A：OpenVLA Action Token 与 I/O Claim-to-Code

审计日期：2026-07-16 状态：`independent_fact_reviewed / endpoint_mask_checked / project_boundary_pending / WARN` 论文版本：OpenVLA arXiv `2406.09246v3` 官方仓库：`openvla/openvla` frozen commit `c8f03f48af692657d3060c19588038c7220e9af9` 对应题目：`VLA-ARCH-001..005`、`VLA-ACT-002..004` 本页问题：OpenVLA 到底接收什么、训练什么、生成什么、怎样把 token 还原为机器人动作？哪些事实不能迁移到 OpenPiVLA / FAST？

1. 结论先行

1. **OpenVLA 不是“VLM 外接连续 action head”**。冻结代码中的 `OpenVLA` 继承 `PrismaticVLM`，由 DINOv2 + SigLIP 视觉特征、投影器和 Llama-2 语言模型组成；动作通过语言模型末尾词表 token 自回归生成。[1][2]
2. **旗舰 OpenVLA 的基础 I/O 是单主视角 RGB + 语言 → 单步连续动作向量**。默认数据路径只加载 `image_primary` 和语言，`load_proprio=False`、`window_size=1`、`future_action_window_size=0`。[1][2]
3. **训练优化目标是 masked next-token cross-entropy**，不是连续动作 L1。代码中的连续 L1 用于把预测/标签 token 解码后做监控；默认配置还可把 stop/EOS token 纳入 CE，但 action accuracy mask 不把 EOS 算作 action token。[1][2]
4. **动作先按数据集每维 q01/q99 归一化到 [-1,1]，再逐标量离散**。论文称每维 256 bins；冻结实现使用 256 个均匀网格点、255 个 bin center，并把上端点折到最后 center，同时仍占用末尾 256 个词表 token。[1][2]
5. **推理生成 token 数等于所选 `unnorm_key` 的动作维数**；随后 token → normalized scalar → q01/q99 反归一化。模型输出的不是 controller 命令语义本身，坐标轴、单位和执行频率仍属于环境/机器人接口。[2]
6. **OpenVLA 2024 的 tokenization 不是 FAST**。OpenVLA 是“每个动作维度一个离散 token”的标量方案；FAST 是后续的 action-window 频域/BPE 路线。两者不能因为都输出 token 就混写。

[1][3]

7. **本地 OpenPiVLA 页不能用 OpenVLA 源码补证据**。其中的 FAST、Qwen3-VL、assistant-side JSON/token、action horizon 10 与归档 97.4% 都需要 openpi/FAST 或本地 artifact 单独证明。[3]

2. 证据层级与审计范围

层级	本页使用方式	能支持	不能支持
论文原文	OpenVLA v3 的 `3.2`、`3.3`、`3.4`、`3.5`、`6`	方法定义、旗舰模型、公开实验和局限	frozen commit 的逐行行为；个人项目事实
frozen official repo	commit `c8f03f...` 的 Python/README/config	tokenizer、mask、I/O、normalization、训练/推理路径	本地 OpenPiVLA 实际采用了同一实现
本地项目页	`../../03_openpivla_action_modeling.html`	当前整理出的口述口径和待补证据项	未附原始 config/log 的强指标与因果归因
同源推断	由论文与 frozen code 联合推出	tensor contract、公式到代码的映射	未出现的 action axis、controller、真实 rollout 行为

本页只审计原始 OpenVLA 主路径。OpenVLA-OFT、π0/π0.5、openpi、FAST、ACT、Diffusion Policy 与 Flow Matching 分属后续 source batch，除边界提示外不在这里提前总结其实现。

3. 架构与输入输出合同

3.1 旗舰架构

论文与 README 将 `openvla-7b` 描述为：Prismatic 7B VLM，视觉侧融合 DINOv2 与 SigLIP，语言侧使用 Llama-2。[1][2] 冻结代码把这一声明落实为：

```
RGB image
  ├─ DINOv2 ViT patches ─┐
  └─ SigLIP ViT patches ─┴─ concatenate channel features
                                    │
                                    v
                          fused MLP projector
                                    │
                                    v
task prompt tokens + projected image patches + Llama-2
                                    │
                                    v
                          autoregressive action tokens
                                    │
                                    v
                       scalar decode + denormalization
```

代码证据：

- `conf/models.py#L491-L497`：224px DINO-SigLIP、Llama-2 7B 与 fused GELU MLP 配置。

- `dinosiglip_vit.py#L20-L29` 与 `#L142-L160`：两个视觉 encoder 的具体标识和 patch feature 拼接。
- `prismatic.py#L38-L75`：vision backbone、LLM backbone 与 projector 三部分。
- `prismatic.py#L367-L420`：视觉 patch 被投影后插在 BOS 与其余文本 token 之间，patch labels 设为 `IGNORE_INDEX`。

面试安全说法：OpenVLA 的“action head”不是独立连续 MLP；在原始路径里，Llama-2 的 LM head 直接预测被复用为 action vocabulary 的 token，`ActionTokenizer` 负责非参数化的连续值 ↔ token 映射。

3.2 训练样本 I/O

`RLDSBatchTransform` 直接读取：

字段	frozen code 行为	shape / 语义
`observation.image_primary[0]`	变为 PIL image 后走 DINO/SigLIP transform	单张主视角 RGB
`task.language_instruction`	decode、lowercase，写入固定问句 prompt	一条任务语言
`action[0]`	q99 normalization 后逐维 tokenization	当前时刻动作向量 `a∈R^d`
proprio / history / extra view	默认不加载	原始旗舰路径不存在

直接证据：

- `datasets.py#L38-L67`：image、language、current action 和 labels 构造。
- `datasets.py#L91-L105`：只加载 primary camera、`load_proprio=False`、`window_size=1`、`future_action_window_size=0`。
- 论文 `Section 6`：明确将多图、proprio 与 observation history 列为未来工作。

因此，`VLA-ARCH-004` 的正确回答不能说“OpenVLA 已把多视角和 proprio 全部融合”；它应以 OpenVLA 作为反例，再由 Octo/π 系列等后续来源补完整比较。

3.3 推理 I/O

`predict_action` 的公开合同是：

```
def predict_action(image, instruction, unnorm_key):
    # 1. Build the same language prompt used during training.
    prompt = build_prompt(instruction.lower())

    # 2. Transform one RGB image and run autoregressive generation.
    token_ids = generate(
        image=image,
        prompt=prompt,
        max_new_tokens=get_action_dim(unnorm_key),
    )

    # 3. Decode exactly d final tokens to normalized scalar actions.
    normalized_action = decode_last_d_tokens(token_ids)
```

```
    # 4. Select dataset statistics and map normalized dimensions back to units.
    return q99_unnormalize(normalized_action, stats=norm_stats[unnorm_key])
```

这段伪代码映射到 `openvla.py#L35-L103`。动作维数来自 `openvla.py#L121-L131` 中所选数据集统计的 `q01` 长度，而不是一个跨机器人固定常数。

4. Action normalization 与 tokenizer 的精确数学

4.1 q01/q99 归一化

令原始动作 `a∈R^d`，训练集每维统计为 `q_i^{01}`、`q_i^{99}`，归一化 mask 为 `m_i∈{0,1}`。冻结实现等价于：

$$\tilde{a}_i = \begin{cases} \mathrm{clip}\left(2\frac{a_i - q_i^{01}}{q_i^{99} - q_i^{01} + \epsilon} - 1,\ -1,\ 1\right), & m_i = 1, \\ a_i, & m_i = 0. \end{cases}$$

变量与直觉：`d` 是当前数据集 action dimension；q01/q99 把大部分训练动作铺到 [-1,1]，并通过 clipping 降低极端值对离散分辨率的影响。实现映射在 `data_utils.py#L48-L103`，统计计算在 `#L240-L260`。OXE 路径默认不归一化最后的 gripper 维，见 `materialize.py#L31-L43`；具体 gripper 标准化仍受各 dataset transform 约束。

4.2 normalized scalar → token ID

令 `V` 为基础 tokenizer 的词表大小。代码构造 256 个均匀网格点

$$e_j = -1 + \frac{2j}{255}, \qquad j = 0, \ldots, 255,$$

然后对每个归一化标量做 `np.digitize`：

$$b_i = \mathrm{digitize}(\mathrm{clip}(\tilde{a}_i, -1, 1), e), \qquad t_i = V - b_i.$$

这里 `b_i∈{1,\ldots,256}`，因此 `t_i` 落在词表末尾 256 个 ID。直觉是把每个动作维当作一个“特殊词”，使 LLM 能以标准 next-token prediction 生成动作。实现证据是 `action_tokenizer.py#L13-L47`。

4.3 token ID → normalized scalar

代码以相邻网格点中心

$$c_j = \frac{e_j + e_{j+1}}{2}, \qquad j = 0, \ldots, 254$$

作为重建值，并计算

$$j_i = \mathrm{clip}(V - t_i - 1, 0, 254), \qquad \hat{\tilde{a}}_i = c_{j_i}.$$

变量与实现：`j_i` 是 255 个 center 的索引；由于 `np.digitize` 可能返回 256，代码把最后一个索引折叠到 `c_{254}`，见 `action_tokenizer.py#L49-L68`。

论文—代码精确边界：

- 论文 `3.2` 的抽象表述是“每维 256 bins、得到 0...255 的整数、覆盖末尾 256 tokens”。[1]
- frozen code 的实际重建表有 255 个 center；最大端点对应的两个离散索引最终映射到同一个最后 center。[2]
- 因而面试中可说“论文层是 256-way action token；这个 commit 的 decode 有 255 个 center 并处理上端点”，不要无条件说“存在 256 个不同连续重建值”。

4.4 反归一化

$$\hat{a}_i = \begin{cases} \frac{\hat{\tilde{a}}_i + 1}{2}(q_i^{99} - q_i^{01}) + q_i^{01}, & m_i = 1, \\ \hat{\tilde{a}}_i, & m_i = 0. \end{cases}$$

该公式直接映射 `openvla.py#L89-L103`。它只恢复数据集 action units；代码没有证明各轴最终如何进入某个 controller，因此 controller axis/order/频率仍需具体 eval 或机器人 wrapper 证据。

5. 训练目标、labels 与监控指标

令 `y_1,\ldots,y_T` 是 prompt、可选 stop token 与 action tokens 组成的语言序列，`M_k` 表示第 `k` 个 label 是否参与 loss。训练目标为：

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{\sum_k M_k} \sum_{k=1}^{T} M_k \log p_\theta(y_k \mid y_{<k}, I),$$

其中 `I` 是投影后的图像 patch 条件。`RLDSBatchTransform` 把 prompt 部分设为 `IGNORE_INDEX=-100`；`PrismaticVLM.forward` 又给插入的视觉 patch 位置补 `IGNORE_INDEX`。默认

`predict_stop_token=True` 时，代码保留 `len(action)+1` 个尾部 labels，意味着 action token 加 stop/EOS；关闭后再 mask 最后一位。[2]

实现链：

1. `datasets.py#L44-L67`：prompt/action token 序列和 prefix mask。
2. `prismatic.py#L388-L420`：图像 patch 插入及 patch-label mask。
3. `base_strategy.py#L285-L332`：`output.loss.backward()` 是优化路径；action token accuracy 与 continuous L1 在其后计算并提交为 metrics。

关键面试防线：

- 不能说“OpenVLA 用 L1 直接回归连续动作”；它用 token CE 训练。
- 也不能说“训练 loss 包含所有文本位置”；prompt 与 image patch labels 被 mask。
- 连续 L1 是将 argmax token 解码后的离线量化/预测误差指标，不等于 closed-loop success。

6. 全量微调、单图限制与 action chunk 边界

6.1 旗舰训练是全量微调

论文 `3.4` 说明最终训练中微调全部 7B 参数，并认为 vision encoder fine-tuning 对精细控制重要。[1] frozen config 的旗舰 OXE 变体继承 `freeze_vision_backbone=False`、`freeze_llm_backbone=False`；`train.py` 将其解析为 `vla-full-train`。[2]

- `conf/vla.py#L63-L91`
- `conf/vla.py#L124-L136`
- `train.py#L159-L180`

LoRA、冻结视觉 encoder、只解冻最后一层是支持的适配选项，不是旗舰预训练 recipe 的同义词。

6.2 base OpenVLA 不是 chunk policy

论文的对照实验把 matched Diffusion Policy 配置成与 OpenVLA 一样：单图、无 proprio、无历史、单步相对动作；`Section 6` 又把 action chunking 列为未来改进方向。[1] frozen dataset config 的 `future_action_window_size=0` 与之对齐。[2]

所以：

- “OpenVLA 支持自回归生成多个 action dimensions”不等于“它预测未来 H 步 action chunk”。
- 生成 `d` 个 token 是一个 `d` 维当前动作，不是 `H×d` 的时间窗口。
- OpenPiVLA 页面里的 `horizon=10` 必须由 openpi/FAST 或本地 eval config 证明，不能回填成 OpenVLA 机制。

7. Claim-to-code 总表

Claim	论文证据	frozen code 证据	判定	面试口径
视觉侧融合 DINOv2 + SigLIP，语言侧 Llama-2 7B	Abstract / `3.4`	`conf/models.py`、`dinosiglip_vit.py`	direct	“Prismatic VLM backbone，融合两套视觉特征后投影进 Llama-2”
输入为 image + language	`3.2`	`RLDSBatchTransform`、`predict_action`	direct	“旗舰路径是单 RGB 主视角 + 任务语言”
默认包含 proprio / history / multi-view	`6` 明确列为未来工作	`load_proprio=False`、primary-only、window=1	rejected	不得这样描述原始 OpenVLA
每个动作维度离散为 action token	`3.2`	`ActionTokenizer`	direct with nuance	论文 256-way；代码精确讲 256 token IDs / 255 centers
占用 Llama 词表末尾 256 token	`3.2`	`V-b_i` 映射	direct	“复用 least-used tail tokens，不新增完整 256 special tokens”
训练用 action-token CE	`3.2`	masked labels + `output.loss.backward`	direct	CE 是优化目标；L1 是 metric
生成长度是固定 7	README Bridge 示例是 7-DoF	`get_action_dim(unnorm_key)`	rejected as universal	维度由 dataset norm stats 决定
q01/q99 normalization	`3.2`	`BOUNDS_Q99` 与 unnormalize	direct	训练、checkpoint 与 eval 必须共享同一 stats key
原始 OpenVLA 使用 action chunk	`6` 列为未来方向	`future_action_window_size=0`	rejected	单步动作；chunk 属其他路线/后续改造
OpenVLA code 证明本地 FAST/Qwen/97.4%	无	无	rejected	只能进入 OpenPiVLA 独立证据合同

8. 与 OpenPiVLA / FAST 的真实性边界

名称	本页已核实的 action 表示	与本地项目的关系	当前状态
OpenVLA	q01/q99 scalar normalization → 每维离散 token → LM next-token CE → 单步动作	只能提供 action-token 对照和审计方法	upstream verified
FAST	action window 的频域变换、量化与 BPE	本地页主要口述机制；不能由 OpenVLA 证明	下一个 source batch
openpi / π0	本页不展开；需核对 flow action expert、state/action mask、solver	本地项目命名和 policy/eval contract 的主要上游候选	下一个 source batch
本地 Qwen formal reproduction	assistant-side action JSON/token + LoRA	工程折中与训练语义恢复，不等同于 OpenVLA tokenizer 或原 action expert	local artifact pending
`224_h10_40000` / 97.4%	与 OpenVLA paper/repo 无直接关系	strongest archived eval，需 checkpoint + summary + protocol	local evidence pending

对本地项目最重要的改写原则：

1. 讲“action token”时先报具体 tokenizer 名称，不把 OpenVLA scalar bins、FAST 和 JSON serialization 混成同一算法。

2. 讲“horizon”时说明它是 dataset target horizon、model output horizon 还是 controller execution/replan interval。
3. 讲“97.4%”时先报 run/checkpoint/protocol，再讲方法；不能用上游 architecture 代替本地归因证据。

9. 对 Phase 0 题目的支持状态

Question ID	OpenVLA 证据覆盖	状态	仍缺
VLA-ARCH-001	VLM 通过可执行 action token 成为 policy	ready for OpenVLA example	传统 policy 与其他 VLA 对照
VLA-ARCH-002	单图、语言、dataset-specific action stats 与 output	ready for OpenVLA subtype	generic proprio/multi-view schema
VLA-ARCH-003	vision backbone + projector + LLM；无独立 continuous head	partial	π0 action expert / continuous heads
VLA-ARCH-004	直接证明原 OpenVLA 缺 multi-view/proprio/history	negative evidence ready	Octo/π 系列正例
VLA-ARCH-005	tokenization、mask、generation、decode、unnormalize 全链	ready	independent fact review
VLA-ACT-002	q99 normalize / dataset-key unnormalize	ready for OpenVLA	openpi normalizer 与 controller
VLA-ACT-003	quantile、网格、token、endpoint nuance	ready	量化误差实验与分布漂移案例
VLA-ACT-004	tail-vocab mapping、begin index、decode	ready	FAST vocab/BPE 对照

这些题仍然只是 `source-ready`，不是候选人已经掌握；正式题卡生成后仍应标为 `unseen`。

10. 风险与下一步

1. **独立审查已完成**：paper v3、frozen tokenizer/dataset/model/strategy 与 endpoint probe 已在 独立复核页 直接核对，公共事实层为 `PASS WITH WARNING`；本页整体仍因本地项目边界保持 `WARN/PENDING`。
2. **paper/code 数值表述要分层**：256 bins 是论文抽象；255 centers 是此 commit 的 decode 行为。
3. **checkpoint/config 仍是变量**：Hugging Face exported implementation 与 Prismatic path 均存在，具体面试答案要报所使用路径。
4. **OpenPiVLA raw artifacts 未进入本页**：不能据现有项目 HTML 确认 action axis、horizon、normalizer、controller 或 97.4% checkpoint。
5. π0、FAST 与 frozen `openpi` 已在后续 Audit B/E/F 页面分别复核；下一步只收集本地 OpenPiVLA checkpoint/action schema/normalization/controller artifact 或进行真实口述，不把 OpenVLA 上游合同迁移为个人项目事实。

参考文献（References）

1. Kim et al., OpenVLA: An Open-Source Vision-Language-Action Model, arXiv v3, 2024.
2. openvla/openvla, frozen commit `c8f03f48af692657d3060c19588038c7220e9af9`, accessed 2026-07-16.
3. 本地项目页：OpenPiVLA：FAST 动作表示、Metax 迁移与策略评估复现，当前为整理材料而非原始 artifact manifest。

Part 13 / 16 · `interview_materials/vla/sources/vol_03_openvla_endpoint_independent_fact_review.html` · SHA-256 `71AD60A297FF77A0A3191D49EB6A96B26D08B24EECED16ECB5581FBD63EBD873`

返回本卷目录

Canonical Markdown SHA-256：`71ad60a297ff77a0a3191d49eb6a96b26d08b24eeced16ecb5581fbd63ebd873`；生成自：`interview_materials\vla\sources\vol_03_openvla_endpoint_independent_fact_review.md`

VOL 03 Source Audit A：OpenVLA Endpoint / Token / Mask 独立事实复核

状态：`independent_fact_reviewed / audit_a / endpoint_mask_checked / PASS WITH WARNING` 审查日期：2026-07-19 冻结版本：OpenVLA paper arXiv `2406.09246v3`；official repo commit `c8f03f48af692657d3060c19588038c7220e9af9`。结论先行：论文明确写“每维 256 bins、整数 0..255、覆盖 Llama 末尾 256 tokens”；frozen code 则构造 256 个 threshold values 与 255 个 centers，encode 仍会产生 256 个 token IDs，但 decode 只有 255 个不同 center，最高两个编码索引都重建为最后 center。训练 CE 的 label mask 与 action accuracy/L1 metric mask 也不是同一掩码。

1. 直接复核来源

ID	来源	本次直接读取内容	Verdict
P01	OpenVLA paper v3 §3.2	256 bins、整数 0..255、overwrite 256 least-used tokens、action-token-only CE	`primary-verified`
C01	`action_tokenizer.py`	256 thresholds、255 centers、digitize、tail-vocab mapping、decode clip	`official-code / frozen`
C02	`datasets.py`	action+EOS labels、prompt `IGNORE_INDEX`、`predict_stop_token`	`official-code / frozen`
C03	`prismatic.py`	image patches 插入与 patch-label `IGNORE_INDEX`	`official-code / frozen`
C04	`base_strategy.py`	CE backward、action-token accuracy mask、decode 后 L1 metric	`official-code / frozen`
C05	`openvla.py`	generate d tokens、decode、unnorm key、q01/q99 inverse	`official-code / frozen`

2. 256 token IDs / 255 centers 的精确推导

代码设置：

$$e_j = -1 + \frac{2j}{255}, \quad j = 0, \dots, 255, \qquad c_j = \frac{e_j + e_{j+1}}{2}, \quad j = 0, \dots, 254.$$

因此 `self.bins`/thresholds 有 256 个值，`self.bin_centers` 有 255 个值。对 clip 后的 normalized scalar，NumPy 默认 `digitize` 返回：

$$b = \text{digitize}(\tilde{a}, e) \in \{1, \dots, 256\}, \qquad t = V - b.$$

V 是 tokenizer vocabulary size，所以 $t \in \{V - 256, \dots, V - 1\}$，确实使用末尾 256 个 token IDs。decode 做：

$$j = \text{clip}(V - t - 1, 0, 254), \qquad \hat{a} = c_j.$$

于是 (b=255) 与 (b=256) 都变成 (j=254)。本次 deterministic probe 得到：

输入	digitize b	decode index j	reconstructed center
-1	1	0	-0.9960784314
0	128	127	0
0.9921568627	255	254	0.9960784314
1	256	254	0.9960784314

因此必须同时说清三件事：paper abstraction 是 256 bins；encoder/token vocabulary 的 cardinality 是 256；frozen decoder 的 unique reconstruction centers 是 255。不能把任何一个数字单独替代另外两个。

3. Label mask 与 metric mask

`RLDSBatchTransform` 将 prompt labels 设为 `IGNORE_INDEX`，保留 `len(action)+1` 个尾部 label；默认 `predict_stop_token=True` 时这包含 action tokens 与 stop/EOS。`PrismaticVLM.forward` 还将插入的 image patch labels 设为 `IGNORE_INDEX`。

训练优化使用模型返回的 causal-LM CE；而 `base_strategy.py` 的 action accuracy/L1 metric 使用：

$$m_k^{metric} = \mathbf{1}[y_k > V - (256 + 1)].$$

该阈值恰好选中 (V-256..V-1) 的 256 个 action token IDs，排除 prompt、image patches 和 EOS。连续 L1 是 argmax token decode 后的监控 metric，不是训练 objective，也不是 closed-loop success。

4. 回写结论

下游	修正
VOL 03 knowledge	“256 endpoints”改为“256 thresholds/edge values”；显式写 b=255/256 alias 到 center 254
VLA-ARCH-005	30 秒答案不变；机制答案补 paper/code/cardinality 三层
VLA-ACT-003	quantization 代价补 highest-two-token decode alias
Source Audit A review	independent endpoint/mask review 改为 PASS WITH WARNING；local project gate 不变

5. 不能推出的项目事实

- 不推出 OpenPiVLA 使用原始 OpenVLA `ActionTokenizer`；FAST、JSON/custom formatter 必须分别取证。
- 不推出本地 tokenizer revision、checkpoint、action dimension、q01/q99、unnorm key、controller 或 97.4% protocol 已闭合。
- 不推出 255-center nuance 是个人发现或个人修复；这是 frozen official code 的直接行为。
- 不改变 44 张 VOL 03 卡的 `unseen` 状态，也不创建 coaching state。

6. 下一步

Source Audit A 的 endpoint/token/mask 高风险项已完成独立复核，公共事实层 verdict 为 `PASS WITH WARNING`；这不关闭本地 tokenizer/checkpoint/stats/controller/97.4%/ownership 与口述 gate。下一 bounded slice 优先进行一题真实口述，或收集一个本地 OpenPiVLA immutable artifact。

参考入口

- OpenVLA Source Audit A
- VOL 03 Knowledge
- VOL 03 Questions
- OpenPiVLA Inventory
- OpenPiVLA Ownership Ledger

Part 14 / 16 · `interview_materials/vla/sources/vol_03_pi0_fast_openpi_claim_to_code.html` · SHA-256 `27FBB3E860864DB0206DFD29E550B8E20170BA77F053C57D0C541B516E3A715F`

返回本卷目录

Canonical Markdown SHA-256：`27fbb3e860864db0206dfd29e550b8e20170ba77f053c57d0c541b516e3a715f`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_pi0_fast_openpi_claim_to_code.md`

VOL 03 Source Audit B：π0 / π0.5 / FAST / openpi Claim-to-Code

审计日期：2026-07-19 状态：independent_fact_reviewed / pi05_fast_norm_rollout_checked / local_runtime_pending / WARN 论文版本：π0 arXiv 2410.24164v4；π0.5 arXiv 2504.16054v1；FAST arXiv 2501.09747v1 官方仓库：Physical-Intelligence/openpi frozen commit 15a9616a00943ada6c20a0f158e3adb39df2ccac FAST Hub：physical-intelligence/fast frozen revision ec4d7aa71691cac0b8bed6942be45684db2110f4 对应题目：VLA-ARCH-002..004、006、008、010..012；VLA-ACT-001..002、005..008、010..011、023..025、027..028、031..032 本页问题：π0 的连续 flow action expert、π0.5 的混合训练、π0-FAST 的自回归 token 路线以及 openpi 的 normalization / rollout 到底如何连接？哪些公开事实仍不能证明本地 OpenPiVLA？

1. 结论先行

1. **π0 的公开主路径是多图 + 语言 + proprioception → 连续 action chunk**。论文使用 2–3 张 RGB、语言命令与关节状态，预测长度 50 的动作块；冻结 openpi 默认配置则给出 3 个图像槽、32 维 model action space 和 H=50 的张量合同。[P1][R1]
2. **π0 的 action expert 学的是 flow velocity，不是 action token CE**。论文从 noise time 0 积分到 data time 1；openpi 采用相反但等价的时间参数，从 t=1 的 noise 以负步长积分到 t=0 的 data，默认 10 步。[P1][R1]
3. **π0.5 论文不是简单的“π0 改了 state token”**。论文预训练时用 FAST action token 做 autoregressive next-token prediction，post-training 再加入连续 action expert，并联合优化 token CE 与 flow loss；推理用文本自回归输出后接 10 步 flow sampling。[P2]
4. **冻结 openpi 的 Pi0(pi05=True) 只直接实现 flow loss**。它确实把 state 放进离散 prompt token，并用 adaRMSNorm 注入 flow timestep，但 compute_loss 没有 π0.5 论文的 FAST action-token CE 分支。不能说该 commit 完整复现了论文的 hybrid pre-training objective。[R1]
5. **openpi 的 PiFAST 是另一条独立自回归路线**。它把 prompt、离散 state 和 FAST actions 排成 prefix/postfix token sequence，只对 postfix 做 next-token CE，推理逐 token 生成后再通过

FAST decoder 恢复 H×D 连续动作。[R1]

6. **FAST 的算法顺序是 quantile normalization → per-dimension temporal DCT → scale-and-round → frequency-first flatten → BPE**。BPE 是唯一 learned component；DCT 和反 DCT 是解析变换。官方 Hub processor 本身不做 normalization，调用者必须先把动作处理到合适尺度。[P3][H1]
7. **论文默认值、Hub tokenizer 与 model config 不是同一个“默认”**。FAST 单数据集实验使用 scale=10、BPE vocab=1024；冻结 FAST+ Hub processor 是 scale=10、vocab=2048；openpi 的 PiOFASTConfig 默认 H=32、D=32，而 LIBERO 示例配置显式改为 H=10、D=7。[P3][H1][R1]
8. **model horizon、executed prefix 与 replan interval 必须分开**。openpi LIBERO client 默认每次取 chunk 的前 K=5 步执行后重新推理；ActionChunkBroker 则可把整个 action horizon 逐步消费完才请求新 chunk。H 和 K 不是同一参数。[R1]
9. **normalization stats 是 checkpoint contract，不是可选装饰**。policy 构造时从 checkpoint assets 取训练期 stats；推理 pipeline 在 sample 前 normalize、sample 后 unnormalize，再做 dataset-specific output slicing。[R1]
10. **以上仍不能证明本地 OpenPiVLA 的 97.4%、Qwen JSON/token、horizon 10 或个人贡献**。这些必须由本地 checkpoint、config、diff、norm stats、rollout client 与 eval summary 建立独立 artifact manifest。

2. 证据层级与冻结边界

层级	冻结对象	本页能支持	本页不能支持
论文	π0 v4、π0.5 v1、FAST v1	方法定义、论文约定、公开训练/推理路线	某个未来 commit 或本地 fork 的逐行行为
official repo	openpi commit 15a9616…, Apache-2.0	JAX model、transform、policy、LIBERO client 的当前实现	论文未发布的训练数据；本地项目实际采用情况
official model artifact	FAST Hub revision ec4d7aa…, Apache-2.0	processor 源码、scale、vocab、DCT/BPE encode/decode	openpi checkpoint 是否固定加载了该 revision
同源推断	paper ↔ frozen code 对照	时间约定等价、shape 流、mask 语义	local run 的 checkpoint、指标、axis/order
本地项目材料	03_openpivla_action_modeling.html	待核对 claim 与项目口述入口	缺 raw config/log/diff 时的强归因

冻结 openpi commit 的提交时间为 2026-06-16T00:14:00Z。本页所有 GitHub 代码链接都固定到该 commit；FAST Hub 链接固定到 revision ec4d7aa…。访问日期均为 2026-07-16。

3. π0 的 observation、action chunk 与 action expert

3.1 论文 I/O 合同

令批大小为 B，图像数为 n，动作维数为 D，预测长度为 H。π0 论文把观测和动作块写成：

$$o_t = [I_t^1, \ldots, I_t^n, \ell_t, q_t], \qquad A_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}].$$

变量与直觉：每个 I_t^i 是当前时刻的一路 RGB，ℓ_t 是语言 token，$q_t \in \mathbb{R}^{D_q}$ 是 proprioceptive joint state，$A_t \in \mathbb{R}^{H \times D}$ 是未来动作块。论文实验使用 n=2 或 3、H=50。[P1]

实现映射：冻结 openpi 的 PioConfig 默认 D=32、H=50，输入规格有 base_0_rgb、left_wrist_0_rgb、right_wrist_0_rgb 三个 image slot，state shape 为 [B,32]，action shape 为 [B,50,32]。[R1]

这不是“每个机器人天然都有 32 个执行轴”。32 是 model-side unified dimension；真实机器人可使用较少维度，再通过 padding 和 output transform 恢复 dataset contract。

3.2 prefix / suffix 信息流

冻结代码把 π0 分成两个 token group：

```
# Information-flow sketch: prefix is cached; suffix changes with flow time.
prefix: image tokens from each valid view + language tokens
    │
    ├── cached once during sampling
    │
suffix: one continuous state token + H noisy-action tokens
    │
    └── recomputed at every flow integration step
              ↓
      action_out_proj → velocity [B,H,D]
```

该代码块的注释含义：prefix 提供视觉语言条件；suffix 承载随 flow time 改变的动作状态。采样时 prefix 的 KV cache 只计算一次，每个 Euler step 只重算 suffix。[R1]

π0 的 image/language prefix 内部是 full attention；suffix 的 H 个 action tokens 在同一 action block 内可互相注意。论文明确称 action expert 对 action tokens 使用 full bidirectional attention，因此一次建模整个 chunk，而不是按时间步自回归生成。[P1]

3.3 backbone 与 action expert

论文和冻结配置都把 PaliGemma 视觉语言模块与较小 action expert 配对：PaliGemma 侧使用 2B Gemma 变体，action expert 约 300M。冻结代码以 action_in_proj 将 D 维 noisy action 映射到 expert width，再以 action_out_proj 返回 D 维 velocity。[P1][R1]

面试安全说法：

- VLM backbone 负责把图像与语言变成条件 context。
- action expert 负责在这个 context 下建模连续 H×D noisy action 的 velocity field。

- controller 不在 action expert 内；output transform、axis/unit conversion、replan queue 与 env.step 属于 policy/eval interface。

4. Flow Matching：论文约定与代码约定为什么等价

4.1 π0 论文约定

令真实动作块为 $A \in \mathbb{R}^{B\times H\times D}$，高斯噪声为 $\epsilon \sim \mathcal{N}(0, I)$，论文时间为 $\tau \in [0,1]$。线性 probability path 为：

$$A^\tau = \tau A + (1-\tau)\epsilon, \qquad u_{\text{paper}} = A - \epsilon.$$

变量与直觉：$\tau = 0$ 是 noise，$\tau = 1$ 是 data；沿直线路径的目标 velocity 不随 τ 改变。π0 用网络 $v_\theta(A^\tau, o)$ 预测该速度。[P1]

训练目标为：

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_{A,o,\epsilon,\tau}\left[\frac{1}{HD}\|v_\theta(A^\tau, o) - (A-\epsilon)\|_2^2\right].$$

变量与实现含义：平方误差先作用于每个 action dimension；论文公式写总体范数，冻结 model 的 compute_loss 先在 D 维取 mean，训练脚本再对 batch 与 H 取 mean，最终等价于对有效 [B,H,D] 元素求平均。该公式只描述公开 π0 路径；若真实项目对 padding 维做 mask，则必须额外写出 mask 来源。[P1][R1]

4.2 openpi 代码约定

冻结代码直接写：

$$x_t = t\epsilon + (1-t)A, \qquad u_{\text{code}} = \epsilon - A, \quad t \in [0,1].$$

代码从 t=1 的 noise 出发，以 $dt = -1/N$ 迭代到 t=0 的 data：

$$x_{t+dt} = x_t + dt\, v_\theta^{\text{code}}(x_t, o), \qquad N = 10 \text{ by default}.$$

变量与实现映射：noise shape 为 [B,H,D]；num_steps 默认 10；prefix KV 先缓存，suffix 在每一步重算；最终返回 x_0。对应 Pi0.compute_loss 与 Pi0.sample_actions。[R1]

4.3 等价变量替换

令论文时间 $s = 1 - t$，则：

$$x_{\text{code}}(t) = t\epsilon + (1-t)A = sA + (1-s)\epsilon = x_{\text{paper}}(s),$$

$$u_{\text{code}} = \epsilon - A = -u_{\text{paper}}, \qquad ds = -dt.$$

因此 $v_{\text{code}} = -v_{\text{paper}}$，而 $v_{\text{code}}dt = v_{\text{paper}}ds$。直觉上，代码把时间轴和速度方向同时翻转，轨迹上的点与最终 data endpoint 不变。冻结源码还直接注释其 convention 与 π0 paper 相反。[R1]

面试中不要只说“一个从 0 到 1，一个从 1 到 0”；必须补上 target velocity 也取反，否则会被追问为什么 ODE 仍然等价。

5. π0 与 π0.5：论文差异、代码差异和未覆盖项

5.1 π0.5 论文的 hybrid objective

π0.5 论文将同一 action supervision 同时表示为：

- FAST 离散 action tokens，由大 VLM 自回归预测；
- 连续 noisy action chunk，由小 action expert 做 flow matching。

论文联合目标可概括为：

$$\mathcal{L}_{\pi_{0.5}} = \mathcal{H}\big(x_{1:M}, f_\theta^\ell(o,\ell)\big) + \alpha\|\omega - A - f_\theta^a(A^{\tau,\omega}, o, \ell)\|_2^2.$$

变量与直觉：$\mathcal{H}$ 是 text、semantic output 与 FAST action token 的 next-token cross-entropy；f_θ^a 是 action expert velocity；α 是两项目标的权衡。论文先以 $\alpha = 0$ 做 autoregressive pre-training，再在 post-training 加入随机初始化的 action expert，并报告 $\alpha = 10$ 的联合训练。[P2]

推理不需要自回归生成 FAST actions：论文描述为先生成所需 text/subtask token，再用条件 flow action expert 做 10 步采样。这正是“token route 帮助训练、flow route 保证较快连续动作推理”的设计意图。[P2]

5.2 论文中的 state 与 timestep 注入

与 π0 相比，π0.5：

1. 将 proprioceptive state 离散化后放入 text/prefix token，而不是单独连续 state suffix token；

2. 不再把 flow timestep 与每个 noisy action embedding 拼接，而是经单独 MLP 后用 adaptive RMSNorm 注入 action expert 各层。[P2]

冻结 openpi 对这两点有直接代码：

- PaligemmaTokenizer 在 pi05 格式中将 normalized state 用 256 bins 离散，组成 Task: ..., State: ...; Action: prompt。
- PioConfig(pi05=True) 默认 discrete_state_input=True，max_token_len 从 48 变为 200。
- Pio.embed_suffix 在 pi05 路径不添加 continuous state token，timestep MLP 的结果作为 adarms_cond；π0 路径则把 time embedding 与每个 action token 拼接。[R1]

5.3 必须保留的 paper / repo 缺口

Claim	π0.5 paper	openpi Pio(pi05=True)	判定
state 作为离散 prompt token	direct	direct，且可由 config 关闭	implemented with config nuance
adaRMSNorm 注入 flow timestep	direct	direct	implemented
action expert 预测连续 chunk	direct	direct	implemented
FAST action tokens 同时参加 CE	direct	Pio.compute_loss 未实现该分支	**paper-only in this audit**
联合 CE + α flow loss	direct	Pio.compute_loss 只返回 flow MSE	**not reproduced by this path**
论文完整 pretrain/post-train data recipe	direct overview	repo code不足以重建私有数据	unverified

因此，冻结 repo 可以支持“released π0.5-style flow architecture 的 state/timestep 改造”，不能单独支持“完整复现论文 π0.5 hybrid pre-training”。独立的 PioFAST class 也不能自动当作 Pio(pi05=True) 内部的 CE branch；它们是不同 ModelType、不同 loss 和不同 inference path。[R1]

6. FAST：从连续 action chunk 到频域 token

6.1 normalization 不在 Hub processor 内

令第 i 个 action dimension 的训练集 1%/99% quantile 为 q_i^{01}, q_i^{99}。论文和 openpi 的 quantile normalization 形式为：

$$\tilde{a}_i = 2\frac{a_i - q_i^{01}}{q_i^{99} - q_i^{01} + \varepsilon} - 1.$$

变量与边界：$\tilde{a}_i$ 是输入 tokenizer/model 的归一化值；openpi 的 Normalize._normalize_quantile 没有显式 clip，所以超出训练分位范围的值可能落在 [-1,1] 外。FAST paper/README 推荐预归一化到约 [-1,1]，但冻结 Hub processor 的 __call__ 从 DCT 开始，不读取 norm stats。[P3][R1][H1]

这意味着正确 pipeline 是“dataset/checkpoint normalizer → FAST processor”，不是“FAST processor 自己完成所有尺度处理”。

6.2 DCT、量化与 frequency-first flatten

对 $\tilde{A} \in \mathbb{R}^{H\times D}$ 的每个 action dimension i，FAST 在时间轴做 DCT：

$$C_j^i = \mathrm{DCT}(\tilde{a}_{1:H}^i), \qquad \bar{C}_j^i = \mathrm{round}(\gamma C_j^i).$$

变量与直觉：j 是频率 index，低 j 表示慢变化趋势；γ 越大，保留的小系数越多、重建更精细但压缩更弱。论文单数据集实验使用 $\gamma = 10$。Hub processor 对 shape [B,H,D] 在 axis=1 做 orthonormal DCT，再乘 scale 并 round。[P3][H1]

随后按频率优先展开：

$$z = [\bar{C}_0^1, \ldots, \bar{C}_0^D, \bar{C}_1^1, \ldots, \bar{C}_1^D, \ldots], \qquad y_{1:L} = \mathrm{BPE}_\Phi(z).$$

变量与实现映射：z 先排列所有维度的低频 coefficient，再进入高频；BPE 字典 Φ 合并常见整数片段和大量重复 zero。Hub 源码的 DCT 输出是 [H,D]，C-order elem.flatten() 正好实现 frequency-major flatten。[P3][H1]

解码为：

$$\hat{A} = \mathrm{IDCT}\left(\mathrm{unflatten}_{H,D}\left(\mathrm{BPE}_\Phi^{-1}(y_{1:L})\right)/\gamma\right).$$

变量与边界：FAST 解码恢复的是 normalized H×D chunk，之后仍需 checkpoint-matched unnormalize 和 dataset output transform。Hub decoder 若 token 序列无法 reshape 成 [H,D]，冻结实现会退化为全零 coefficient；这类错误不应在 rollout 中静默忽略。[H1]

6.3 BPE 是 learned component，但 compression 不等于控制质量

FAST 的 DCT、round 与 IDCT 不需要训练；BPE vocabulary 是唯一 learned component。论文消融显示去掉 BPE 会留下大量重复 zero token、显著增加 autoregressive steps，并在其评测任务上降低 policy performance。[P3]

但以下推论不成立：

- token 更少 ⇒ 物理重建误差必然更小；
- reconstruction MSE 更低 ⇒ closed-loop success 必然更高；

- FAST+ 在一个 embodiment 上可用 ⇒ 不需要核对 axis、unit、frequency 与 norm stats。

compression、reconstruction、next-token learning signal、autoregressive latency 和 online control 是五个不同指标层。

6.4 三组容易混淆的默认值

对象	scale	BPE vocab	H / D	解释
FAST paper 单数据集实验	10	1024	1-second chunks，依 robot frequency	实验 recipe
frozen FAST+ Hub processor	10	2048	decode 时显式传或缓存	发布 artifact 的 processor_config
openpi PioFASTConfig 默认	由 external tokenizer	由 external tokenizer	H=32，D=32	model class default
openpi pio_fast_libero 示例	external tokenizer	external tokenizer	H=10，D=7	dataset-specific example config

这些数值必须连同对象名称一起回答。尤其不能用 LIBERO example 的 H=10 证明本地 OpenPiVLA 也使用 horizon 10。

7. PioFAST：自回归训练与 decode path

7.1 token layout 与 masks

冻结 openpi 的 FASTTokenizer 构造：

```
# Token-layout sketch: prefix is condition; postfix is the autoregressive target.
prefix  = BOS + "Task: ..., State: <256-bin state>;\\n"
postfix = "Action: " + <FAST action tokens mapped into PaliGemma tail vocab> + "|" + EOS
```

代码块说明：prefix token 使用 bidirectional prefix attention；postfix 使用 causal attention；loss_mask 只在 postfix 为 true。FAST action token 通过 PaliGemma vocabulary 末尾区域映射，另跳过 128 个 special-token slots。[R1]

这与 OpenVLA 的逐标量 tail-vocab token 不同：

- OpenVLA：每个当前动作维度对应一个 scalar token；
- FAST：整个时间窗口先做 DCT/round/BPE，token 数 L 可变；
- π0 flow：不生成离散 action token，网络每步输出 H×D velocity。

7.2 masked next-token CE

令序列 token 为 $y_{1:T}$，postfix loss mask 为 $m_k \in \{0, 1\}$。PioFAST 的训练目标是：

$$\mathcal{L}_{\text{FAST-CE}} = -\frac{\sum_{k=1}^{T-1} m_{k+1} \log p_\theta(y_{k+1} \mid y_{\le k}, I)}{\max(1, \sum_{k=1}^{T-1} m_{k+1})}.$$

变量与实现：I 是图像 token 条件；target 和 loss mask 都右移一位以匹配 next-token prediction；prefix 与 padding 不参加 loss。PiOFAST.compute_loss 返回每个 batch example 的平均 masked CE，训练脚本再取 batch mean。[R1]

7.3 推理先生成 token，output transform 再恢复 action

PiOFAST.sample_actions 的直接返回值其实是最多 256 个 autoregressive output token，不是最终 controller action。Policy.infer 把它暂存为 actions 字段，随后 ModelTransformFactory 注册的 ExtractFASTActions 才执行：

1. 从生成文本中抽取 Action: ... | 区间；
2. 映回 FAST token IDs；
3. 以指定 action_horizon 与 action_dim 做 FAST decode；
4. 进入 Unnormalize；
5. 进入 dataset-specific output transform 和 slicing。[R1]

因此只阅读 sample_actions 函数会误判它的返回合同。必须继续追到 policy output transform。

7.4 latency 的来源

FAST 论文在 NVIDIA 4090 的报告值是：π0 flow 约 100 ms/chunk，而 π0-FAST 约 750 ms/chunk；其解释是 FAST 通常需要 autoregressively decode 30–60 个 action tokens，并使用完整 2B backbone，而 flow π0 使用 10 次较小 300M action expert integration。[P3]

这些是论文硬件与实现下的公开测量，不是所有 deployment 的固定延迟。真实面试回答应拆成 vision encode、prefix prefill、per-token/per-step decode、network transport 与 controller queue，并用本地 timing log 证明本地数值。

8. Normalization、padding、slicing 与执行合同

8.1 checkpoint-matched normalization

冻结 policy_config 明确从 checkpoint_dir/assets 加载训练时 norm stats。输入顺序是：

```
# Policy transform order: arrows denote the exact inference data flow.
repack → dataset input transform → normalize → model input transform
      → sample_actions
      → model output transform → unnormalize
      → dataset output transform → repack output
```

代码块说明：norm stats 不是仅在数据预处理脚本里用一次，而是 train/infer 必须共享的 policy artifact；错用 asset_id、漏 unnormalize 或先 slice 再错误匹配 stats 都可能产生 offline loss 正常但 online 0-success。[R1]

8.2 zero-padding 与 selection matrix

真实机器人 action $a \in \mathbb{R}^d$ 可嵌入 model action $a^{model} \in \mathbb{R}^D$，其中 $d \le D$：

$$a^{model} = Pa = \begin{bmatrix} I_d \\ 0 \end{bmatrix} a, \qquad a^{env} = Sa^{model}, \quad S = [I_d \quad 0].$$

变量与代码映射：P 对应 PadStatesAndActions 的尾部 zero-pad，S 对应 dataset output transform 的前 d 维 slicing；理论上 $SP = I_d$。LIBERO output transform 返回前 7 维，PioConfig 默认 model D=32；PioFAST 的 LIBERO 示例则直接把 model D 配成 7。[R1]

风险边界：zero padding 不自动保证 padded dimensions 不参与 loss。公开 π0 compute_loss 对 D 维直接 mean；如果 model D 大于真实 d，是否需要 loss mask、是否训练 padding 为零，必须按具体 data/config 路径核对。不能把 output slicing 当作训练期 loss masking 的证据。

8.3 quantile unnormalize

对有统计的前 d 维，openpi 反归一化为：

$$\hat{a}_i = \frac{\hat{\tilde{a}}_i + 1}{2}\left(q_i^{99} - q_i^{01} + \varepsilon\right) + q_i^{01}.$$

变量与实现：$\hat{\tilde{a}}$ 是 flow sample 或 FAST decode 后的 normalized action。若 model 输出还有 stats 未覆盖的 padding dimensions，Unnormalize 保留这些额外维原值，随后 dataset output transform 再切片。[R1]

公式中的乘法应理解为 $(\hat{\tilde{a}}_i + 1)/2$ 乘以 quantile range；这是 normalized unit 到 dataset action unit 的恢复，仍不等于 controller-specific axis conversion 或安全裁剪。

8.4 action axis 与 unit 不能靠维数猜

openpi norm_stats 文档的默认 π action layout 是：

- dim 0–5：left arm joint angles；
- dim 6：left gripper；
- dim 7–12：right arm joint angles；
- dim 13：right gripper；

- dim 14–15：mobile base x-y velocity。

Franka 路径使用前 7 维 joint + 第 8 维 gripper；DROID 又是前 7 维 joint velocity + 第 8 维 gripper，15 Hz。[R1]

这说明同样的 8D shape 可以有 position 或 velocity 等不同语义。shape 对齐只是必要条件，不是 controller contract 完成的充分条件。

9. Action horizon、execution window 与 replan

令 model 预测：

$$\hat{A}_t = [\hat{a}_t, \dots, \hat{a}_{t+H-1}] \in \mathbb{R}^{H \times d},$$

每次只执行前 K 步：

$$\text{execute}\left(\hat{A}_t[0:K]\right), \qquad 1 \le K \le H,$$

然后以新观测重新 query policy。变量含义：H 是 model output horizon；K 是 execution/replan interval。K 小时闭环更强但 query 更频繁；K 大时吞吐更好但更容易沿旧计划累积误差。[R1]

冻结 LIBERO example 的默认 K=5、50 trials/task、seed=7，并在每次 queue 为空时取新 chunk 的前 K 步。它还将 base/wrist 图像旋转 180°以匹配训练 preprocessing，构造 8D state，output transform 只返回前 7 action dims。[R1]

另一个 ActionChunkBroker 会逐步消费整个 configured action_horizon，耗尽后才重新 infer。两种 queue policy 都是公开示例；本地项目到底采用哪一种，必须读本地 client。

```
def rollout_step(env, client, task, replan_steps):
    # 1. Read fresh RGB observations and proprioceptive state.
    obs = build_libero_observation(env, task)

    # 2. Policy transforms normalize state and build model-specific tokens.
    #    Flow returns HxD normalized actions; FAST first returns tokens then decodes HxD.
    action_chunk = client.infer(obs)["actions"]

    # 3. Output transforms have already unnormalized and sliced dataset dimensions.
    assert len(action_chunk) >= replan_steps

    # 4. Execute only K steps, then use a new observation for the next query.
    for action in action_chunk[:replan_steps]:
        _, _, done, info = env.step(action.tolist())
        if done:
            return info
    return info
```

伪代码映射冻结 examples/libero/main.py 与 policy transform 顺序；真实项目若在 client 外另做 denormalize、clip 或 axis reorder，必须改写并附本地源码证据。

10. 三条 action 路线对照

维度	OpenVLA scalar tokens	FAST / PioFAST	π0 flow action expert
基本单位	当前动作的每个 scalar dimension	H×D chunk 的频域/BPE token sequence	H×D continuous noisy action
时间结构	原始旗舰单步	先压缩整个 1-second chunk	整个 chunk 联合建模
训练目标	masked token CE	masked autoregressive token CE	velocity MSE
推理	生成 d 个 token	生成可变 L 个 token，再 FAST decode	N 次 ODE/Euler update
多模态连续 action	量化 token 分布	token sequence 分布	continuous generative field
主要延迟	LM decoding	完整 VLM 的多 token AR decoding	多次较小 action expert
normalization	dataset q01/q99 + scalar bins	external norm stats + FAST processor	external norm stats
必查风险	token offset、bin endpoint	H/D reshape、BPE revision、truncation	time convention、solver、valid dims
本地 OpenPiVLA 关系	只作 scalar-token 对照	可能是上游机制候选	可能是上游 action expert 候选

面试选择路线时不要只谈“谁更先进”。至少同时回答：action 多模态性、控制频率、H/K、允许延迟、训练数据量、reconstruction tolerance、checkpoint assets 与 controller contract。

11. Claim-to-code 总表

Claim	paper / artifact	frozen code	判定	面试口径
π0 输入多图、语言、proprio	π0 v4 direct	PioConfig 三图 + state	direct	论文 2–3 图；repo 默认三槽
π0 输出 H=50 action chunk	π0 v4 direct	PioConfig default H=50	direct default	config 可覆盖，不是所有 checkpoint 固定 50
π0 flow paper/code 时间相反	π0 v4 0→1	code 1→0	equivalent with sign change	同时报 time 和 target velocity 取反
π0 默认 10 Euler steps	π0 v4 direct	sample_actions num_steps=10	direct default	deployment 可覆盖
π0.5 state token + adaRMSNorm	π0.5 v1 direct	Pio pi05 branch	direct with config nuance	released architecture difference
π0.5 联合 FAST CE + flow loss	π0.5 v1 direct	Pio compute_loss 无 CE branch	paper-only here	不得称 frozen path 完整复现
PioFAST 用 FAST token CE	FAST/openpi direct	PioFAST.compute_loss	direct	独立 ModelType，不等于 Pio pi05

Claim	paper / artifact	frozen code	判定	面试口径
FAST 是 per-scalar bins	FAST v1 rejects	Hub processor DCT/BPE	rejected	它是 window frequency tokenization
FAST processor 自带 q01/q99 stats	README 要求预归一化	processor 从 DCT 开始	rejected	normalizer 在 processor 外
FAST 默认 vocab 永远 1024	paper 单数据集 1024	Hub artifact 2048	rejected as universal	回答时注明对象/version
openpi quantile normalize 会 clip	未声明	transform 无 clip	rejected	超界值可落在 [-1,1] 外
D=32 代表 32 个真实控制轴	无	Pad/Slice path	rejected	D 是 model capacity，d 是 dataset action
output slicing 等于 loss mask	无	loss 与 output transform 分离	rejected	训练有效维必须另核对
H 等于 replan interval	无	LIBERO H 与 K 分离	rejected	H model output；K executed prefix
LIBERO example 证明本地 H=10	无	只是 official example config	rejected	需本地 config/checkpoint
upstream code 证明本地 97.4%	无	无	rejected	需 eval summary + protocol + checkpoint

12. 对 Phase 0 题目的支持状态

Question ID	本页支持	状态	仍缺
VLA-ARCH-002	multi-view/text/state → H×D contract	source-ready	generic history/time alignment 对照
VLA-ARCH-003	VLM context、action expert、output transform、controller 边界	source-ready	Octo / RT-2 comparison
VLA-ARCH-004	三图槽、mask、离散/连续 state	partial	更广 multi-view alignment
VLA-ARCH-006	π0 action expert + flow train/infer	source-ready	independent formula review
VLA-ARCH-008	PiOFAST prefix/postfix/AR/loss masks	source-ready	other VLA layouts
VLA-ARCH-010	D/d、padding、slicing、loss risk	source-ready	explicit masked-loss positive example
VLA-ARCH-011	FAST vs flow latency component	partial	local timing instrumentation
VLA-ARCH-012	paper/repo/artifact/checkpoint audit method	source-ready	local manifest
VLA-ACT-002	checkpoint norm stats、normalize/unnormalize	source-ready	local norm asset
VLA-ACT-005..007	FAST motivation、algorithm、trade-off	source-ready	independent fact review
VLA-ACT-008	H×D、P/S、valid dimension boundary	source-ready	local action_dim/mask

Question ID	本页支持	状态	仍缺
VLA-ACT-011	H、K、chunk queue	source-ready	ACT temporal ensemble later
VLA-ACT-023..025	probability path、velocity、ODE solver	source-ready for π0	foundational Flow Matching paper
VLA-ACT-027	context/state/noisy action expert input	source-ready	PyTorch parity if needed
VLA-ACT-028	padding pollution risk	partial / negative evidence	positive mask implementation
VLA-ACT-031..032	decode→unnorm→slice→queue	partial	controller axis/safety tests

source-ready 只表示来源足够起草知识页，不代表候选人已掌握。后续题卡创建后仍必须标为 unseen，直到真实口述训练。

13. 本地 OpenPiVLA 证据缺口

公开 π0/FAST/openpi 可以给出上游机制，但以下 claim 必须补本地 artifact：

本地 claim	需要的最小证据	当前状态
base checkpoint / backbone	load config、checkpoint manifest、startup log	pending
是 Pi0、Pi05、PiFAST 还是自定义 Qwen path	model class、forward/compute_loss、local diff	pending
action H=10	dataset target shape、model config、inference output shape	pending
action dimension / axis / unit	dataset schema、norm stats、output transform、controller wrapper	pending
FAST revision / scale / vocab	tokenizer config、cached revision、training log	pending
JSON/action token supervision	formatter、labels/loss mask、decode/parser tests	pending
replan / queue policy	local rollout client、K、control frequency	pending
97.4%	exact checkpoint、suite/tasks、trials、seed/split、summary JSON	pending
personal contribution	upstream/local diff、设计记录、run ledger	pending

在这些证据收集完成前，最安全的项目表述是“基于 π/FAST/openpi 公开机制建立 action modeling 与 eval pipeline 的候选解释；本地具体实现和指标归因仍按 artifact 核实”，不能把 upstream design 自动写成本人创新。

14. 风险与下一步

1. 核心高风险事实已独立复核：π0.5 hybrid、FAST normalization/DCT/BPE、OpenPI model split、padding/slicing、norm stats 与 LIBERO replan 已在 独立复核页 直接核对，公共事实层为 `PASS WITH WARNING`；local checkpoint、tokenizer revision、axis/unit、H/K、controller、eval bundle 与 ownership 仍使本页整体保持 `WARN/PENDING`。

2. **π0.5 hybrid gap 是高优先级复核项**：需独立确认 frozen openpi 的 released Pi05 training path 是否在其他文件另有 combined CE；当前已读主 compute_loss 和 transform factory，未见该分支。
3. **FAST artifact revision 必须固定**：openpi 默认字符串没有 pin Hub revision；复现时需要 cache/lockfile 或显式 revision。
4. **normalization 无 clip 的边界需单测**：测试 q01/q99 外动作、constant dimension、padding dimension 和 inverse consistency。
5. **decode failure 不应静默全零**：本地部署应让 malformed FAST sequence 触发 error/telemetry，而不是无声执行零动作。
6. **视觉 QA 已完成**：通过 localhost 静态服务检查桌面 `1440×1000` 与窄屏 `800×1000`；确认一处 H1、academic palette/serif、desktop sticky TOC、narrow static TOC、公式无 `<em>` 污染且无页面级横向溢出。
7. **下一步**：优先进行一题 π0/FAST/Flow 真实口述，或收集本地 OpenPiVLA checkpoint/tokenizer/stats/controller artifact；题卡继续保持 `unseen`。

参考文献（References）

1. [P1] Black et al., π0: A Vision-Language-Action Flow Model for General Robot Control, arXiv v4, 2024.
2. [P2] Black et al., π0.5: a Vision-Language-Action Model with Open-World Generalization, arXiv v1, 2025.
3. [P3] Pertsch et al., FAST: Efficient Action Tokenization for Vision-Language-Action Models, arXiv v1, 2025.
4. [R1] Physical-Intelligence/openpi, frozen commit 15a9616a00943ada6c20a0f158e3adb39df2ccac, accessed 2026-07-16. 核心路径：

- pi0_config.py
- pi0.py
- pi0_fast.py
- tokenizer.py
- transforms.py
- training/config.py
- policy_config.py
- libero_policy.py
- LIBERO client
- normalization notes

1. [H1] physical-intelligence/fast, frozen revision ec4d7aa71691cac0b8bed6942be45684db2110f4, Apache-2.0, accessed 2026-07-16. processor source / processor config.
2. 本地项目页：OpenPiVLA：FAST 动作表示、Metax 迁移与策略评估复现。当前仅作为 claim 入口，不作为缺失 raw artifact 的替代证据。

Part 15 / 16 · `interview_materials/vla/sources/vol_03_pi0_fast_openpi_independent_fact_review.html` · SHA-256 `E19A70F7B653F84B412EF1916BD1F70AF194F6DD396E64C5891DC29FD0EEE4B7`

返回本卷目录

Canonical Markdown SHA-256：`e19a70f7b653f84b412ef1916bd1f70af194f6dd396e64c5891dc29fd0eee4b7`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_03_pi0_fast_openpi_independent_fact_review.md`

VOL 03 Source Audit B：π0 / π0.5 / FAST / OpenPI 独立事实复核

状态：`independent_fact_reviewed / audit_b / pi05_fast_norm_rollout_checked / PASS WITH WARNING` 审查日期：2026-07-19 范围：π0.5 hybrid 训练边界、FAST action tokenizer、OpenPI model/config/transform、normalization 与 LIBERO rollout example。 结论先行：π0.5 v1 明确把 pre-training 的 FAST 离散 token 目标与 post-training 的连续 flow action expert 分成两阶段；frozen OpenPI 中 `Pi0(pi05=True)` 的主 flow loss 与 `Pi0FAST` 的 autoregressive CE 是两条不同模型路径，不能互称为完整 π0.5 hybrid 复现。FAST paper 的 quantile normalization、DCT、scale-and-round、frequency-first flatten、BPE 顺序得到直接复核；FAST Hub processor 与 OpenPI tokenizer 不负责 quantile normalization。OpenPI 的 32 维 model space、7 维 LIBERO output slice、norm-stat checkpoint contract 与 replan=5 只是官方配置/example，不是本地 OpenPiVLA 事实。

1. 直接复核来源与冻结对象

ID	来源	本次直接读取内容	SHA / Verdict
Po1	π0.5 v1 PDF	Fig. 3、Sec. IV、Eq. (1)、pre/post training	`6A1029FD8AB6944B74CF22F5E5D30E60BC15699D964B2900AF799B807A34B64C` / `primary-verified`
Po2	FAST v1 PDF	Sec. V-B、Algorithm 1、scale/vocab、quantile/DCT/BPE	`3739B31F5FECDDE371509FF5BB13619979734E894A255A9B264253F4CC53934A` / `primary-verified`
Co1	OpenPI `pi0_fast.py`	`Pi0FASTConfig`、token CE、loss mask、AR decode	`627332A555046F2744EF144C45F8BD3361A9820CF68854A4528D0FDBBA7E9AA3` / `official-code`
Co2	OpenPI `pi0_config.py`	Pi0/Pi05 defaults、D/H、discrete state flag	`D321858C30B398126035EA2B6DA09E1C422EEEEBB5EBF8DF631DAF9DC77647FA` / `official-code`
Co3	OpenPI `tokenizer.py`	FAST prefix/postfix、128 special-token skip、mask、truncation、decode fallback	`965BE8B3C393A6811875BBC32DA9E01CC2F87802DE801CF7293E049748C` / `official-code`
Co4	OpenPI `transforms.py`	quantile/z-score normalize、unnormalize、pad、FAST extract	`A1B94E9E72849A18834778F229C6BB389A495EB7FBE0AA800EDEA728B9424FF4` / `official-code`
Co5	OpenPI `libero_policy.p`	valid/padded camera mask、7D output slice	`D8679D99FACC3FEF48DA07406081B20709ED872FCEFDF7DE67EDE99422D623FA` / `official-code`

ID	来源	本次直接读取内容	SHA / Verdict
	`y`		
Co 6	OpenPI LIBERO example	180° image flip、resize、replan=5、50 trials、success logging	`D53748C95804ADA559E3CA867305B8F755D8651C4C7F4E2723EF80AEA55A3943` / `official-code`
Co 7	OpenPI `training/config.py`	`pi0_fast_libero` 的 action_dim=7、action_horizon=10 与 checkpoint config override	`C34231F888AB5DC981D808765B3D98775AB845B04313BF53FCECD567C520B0D` / `official-code`
Ho 1	FAST Hub processor	scale=10、vocab=2048、DCT/BPE encode/decode	`6F021CA1F4C1B194AB6FA399D80BAF3D642EADB17EFB8F73301E4AC401522C20` / `official-artifact`
Ho 2	FAST Hub config	`scale=10`、`vocab_size=2048`、`min_token=-354`	`F40CFBB1020858FE1D48C0F946B0C1315A90D6E84AA82710036F24F4C167706A` / `official-artifact`
Do 1	OpenPI normalization notes	stats 与 checkpoint、action axes、单位、频率	`62E4A39EA7FCDAE53B4466942DB348C3E38FC016B7F7A5273CAFED649EBE614B` / `official-doc`
Oo 1	本次 deterministic probes	DCT roundtrip、quantile out-of-range、pad/slice、replan queue	`local-evidence / PASS`

所有 GitHub 代码均固定在 OpenPI commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`；FAST Hub 固定在 `ec4d7aa71691cac0b8bed6942be45684db2110f4`。这些来源支持公开机制和官方 example，不支持用户本地 checkpoint、fork、指标或 ownership。

2. π0.5：论文 hybrid 目标与 OpenPI 两条路径

2.1 论文的两阶段合同

π0.5 v1 的 Fig. 3 与 Sec. IV 写明：pre-training 把 robot action 表示成 FAST 离散 token，并以 autoregressive next-token prediction 训练；post-training 添加 action expert，用 flow matching 生成连续 action chunk。其联合目标可概括为：

$$\mathcal{L}_{\pi 0.5} = \mathcal{L}_{\mathrm{CE}}(x_{1:M}, y_{1:M}) + \alpha\, \mathbb{E}_{\tau,\omega} \left\| \omega - a_{t:t+H} - f_\theta^a(a_{t:t+H}^{\tau,\omega}, o_t, \ell) \right\|_2^2.$$

变量与实现：`\mathcal L_{\mathrm{CE}}` 覆盖文本/语义/FAST action token 的离散 next-token 目标；`f^a_\theta` 是较小 action expert 的 flow vector field；`\alpha` 是 trade-off。论文 post-training 报告 `\alpha=10.0`、先做 280k pre-training 再追加 80k post-training；这是论文 recipe，不是本地运行证据。[P01]

2.2 frozen OpenPI 的分叉

OpenPI `Pi0(pi05=True)` 的配置差异是 state 进入离散 prompt、flow timestep 用 adaRMSNorm；但该类的 `compute_loss` 仍是连续 flow loss 路径。独立读取的 `Pi0FAST` 则在 `compute_loss` 中构造 shifted next-token targets，只对 `token_loss_mask[:,1:]` 做 CE reduction：

$$\ell_{\mathrm{CE}} = -\frac{\sum_k m_k \log p_\theta(y_k \mid y_{<k})}{\max\left(\sum_k m_k, 1\right)}.$$

变量与实现：`m_k` 来自 FAST postfix mask；prefix state/language 与 padding 不参加 CE。`Pi0FAST.sample_actions` 是 autoregressive token decode，遇到 EOS 可提前停止，之后由 FAST tokenizer 解码成连续动作；它不是 `Pi0(pi05=True)` 的 flow expert sampler。

因此安全结论是：

- 论文 hybrid objective：`primary-verified / paper-level`；
- frozen `Pi0(pi05=True)`：`flow path direct evidence`，不能称完整 hybrid；
- frozen `Pi0FAST`：`FAST token CE direct evidence`，不能自动补出 Pio.5 paper 的联合 flow loss；
- 本地 OpenPiVLA 采用哪一类，必须由 model class、forward/compute_loss、tokenizer 与 checkpoint manifest 证明。

3. FAST：归一化、DCT、量化与 BPE

FAST paper Sec. V-B 的顺序是：

$$a \xrightarrow{\text{quantile normalize}} \tilde{a} \xrightarrow{\text{DCT along time}} C \xrightarrow{\text{round}(\gamma C)} \bar{C} \xrightarrow{\text{frequency first flatten}} T \xrightarrow{\text{BPE}} \bar{T}.$$

变量与直觉：每个 action dimension 的训练集 1%/99% quantile 映射到 [-1,1]；DCT 把平滑时间序列变成低频优先的系数；`\gamma` 控制量化误差/压缩率；BPE 合并常见 coefficient 字符串。论文 single-dataset 实验使用 `\gamma=10`、BPE vocab 1024，并强调 BPE 是唯一 learned component，DCT 与 inverse DCT 是解析变换。[P02]

Hub revision 的 processor 默认则是：

对象	值	证据
`scale`	10	H01/H02
`vocab_size`	2048	H01/H02
`min_token`	-354	H02
`time_horizon/action_dim`	可由构造器或 decode 参数提供	H01

`processing_action_tokenizer.py` 的 `__call__` 从输入 action chunk 直接做 DCT、round、flatten、BPE；其中没有 q01/q99 normalization。OpenPI 的 `FASTTokenizer` 还负责把 FAST token

映射到 PaliGemma vocabulary，跳过最后 128 个 special-token slots，并构造 prefix/postfix、AR mask、loss mask。故“FAST processor 自带 normalization”是错误答案。

4. FAST token mask、截断与 malformed decode

OpenPI `FASTTokenizer.tokenize` 直接把：

- prefix：`Task: ... State: ...;`，`loss_mask=False`；
- postfix：`Action: <FAST tokens>|`，`loss_mask=True`；
- padding：`token_mask/ar_mask/loss_mask` 均为 false/zero；
- 超过 `max_len`：截断并记录 warning。

`extract_actions` 在生成文本缺少 `Action:` 时返回全零 action；FAST processor decode 失败时也捕获异常并构造全零 DCT 矩阵。它们是官方 fallback 行为，不等于安全策略。真实部署应记录 malformed token、fallback 次数、零动作是否触发 controller safety stop，而不是把 fallback 当成功。

5. OpenPI action dimension、padding、normalization 与 rollout

5.1 model dimension 不是 physical dimension

frozen config 默认：

Model config	`action_dim`	`action_horizon`	语义
`Pi0Config`	32	50	continuous flow model-side space
`Pi0FASTConfig` default	32	32	FAST AR model-side space
`pi0_fast_libero` config override	7	10	official LIBERO fine-tune example

`PadStatesAndActions` 把 state/action 在最后一维补到 model dimension；`LiberoOutputs` 再只取前 7 维。`training/config.py` 的 `pi0_fast_libero` 则显式把 PioFAST 的 model `D=7,H=10` 覆盖为 LIBERO 任务规格。这个 config override、output slice 与 model default 仍是三件事，不能反推本地项目同样采用；output slice 也不能反推训练 loss 已经 mask padding。该 frozen Pio flow path 对 configured D 做 mean 的风险已在 Audit E 记录，本页不把 negative evidence 升级成 local bug。

5.2 normalization 是 checkpoint contract

OpenPI docs 明确：state/action normalization stats 由 training data 计算并和 checkpoint 一起保存；可以复用预训练 stats，也可以重新计算新数据 stats。`Normalize` 的 quantile 路径为：

$$\tilde{x} = \frac{x - q_{01}}{q_{99} - q_{01} + \epsilon} \cdot 2 - 1, \qquad \text{Unnormalize}(\tilde{x}) = \frac{\tilde{x} + 1}{2}(q_{99} - q_{01} + \epsilon) + q_{01}.$$

变量与实现：`q_{01},q_{99}` 来自 stats artifact；代码对输入先按现有维度切 stats，对输出按 tensor 宽度补 mean=0/std=1 或拼接未归一化尾部。代码没有在 quantile normalization 中自动 clip，因此 q01/q99 外动作可得到 [-1,1] 之外的 normalized value；是否 clip、在哪一层 clip，必须由具体项目 transform/controller 证明。

5.3 LIBERO example 的 replan 与视觉合同

官方 example 默认 `replan_steps=5`、`num_trials_per_task=50`、seed=7；每次 policy query 后只把 chunk 前 5 步放入 queue，执行完再重新读取 observation。image 先旋转 180°、resize/pad 到 224，再送入 client；`LiberoInputs` 把不存在的右腕图像补零，并为 Pio 与 Pio-FAST 使用不同 image-mask 语义。它们都是 example protocol，不是候选人项目的真实 eval protocol。

```
def libero_example_step(obs, client, action_plan, replan_steps=5):
    # 1. Match training image orientation and resize contract.
    image = resize_with_pad(rotate_180(obs["agentview_image"]), 224, 224)

    # 2. Build state/action input keys expected by the frozen policy wrapper.
    element = build_libero_element(image, obs["robot0_eef_pos"], obs["robot0_eef_quat"])

    # 3. Re-query only when the previous execution prefix is consumed.
    if not action_plan:
        chunk = client.infer(element)["actions"]
        action_plan.extend(chunk[:replan_steps])

    # 4. Execute one denormalized, dataset-dimension action and log success.
    return action_plan.popleft()
```

代码映射：上述顺序来自 frozen `examples/libero/main.py`、`libero_policy.py` 与 transform factory；本地若另有 denormalize、axis reorder、clip 或 async queue，必须以本地源码覆盖该 example。

6. Deterministic probes

本次没有把官方依赖安装到工作区，只用独立 NumPy/SciPy probe 检查公式边界：

Probe	结果	含义
FAST DCT/round/idct，输入 [8,2]、scale=10	shape 保持 [8,2]，max abs reconstruction error `0.0487787431`	round 是有损量化；DCT shape 不等于 token length
quantile normalize q01=[-1,2], q99=[1,6]	q01/q99 附近约 0，value=[2,8] 得到约 [2,2]	frozen formula 不自动 clip
quantile inverse	max abs round-trip error 0	stats 与 inverse 公式相配时可逆（忽略数值 epsilon）
7D→32D pad→slice	tail 25 维全 0，slice round-trip=True	shape mapping 成立，不证明 loss mask

Probe	结果	含义
queue consume 5	12 control steps 触发 3 次 query，首 chunk 执行 5 步	K/replan 是 rollout contract，不是 H

Probe 只验证公式、shape 与 queue 逻辑；不证明 learned model、checkpoint、LIBERO success 或个人项目实现。

7. 对下游题目的回写

题目范围	独立复核结论	必须保留的边界
`VLA-ARCH-002..004`	multi-view/state/token/action expert 的 public I/O 口径通过	local view/time alignment 未证
`VLA-ARCH-006`	Pi0/Pi05/FAST 三条 model path 分开	local class/checkpoint 未证
`VLA-ARCH-008`	FAST prefix/postfix、AR/loss mask 与 image mask 分开	不把 token mask 当 continuous action mask
`VLA-ARCH-010`	D/d、pad/slice 与 model capacity 分开	positive local valid-dim mask 未证
`VLA-ARCH-011`	FAST AR、flow NFE、prefix/cache、replan queue 分层	无 local timing
`VLA-ACT-001/002/005..008`	action representation、normalization、FAST processor 与 error boundary 通过	local normalizer/tokenizer revision 未证
`VLA-ACT-010/011`	H、K、replan 与 chunk queue 分开	local H/K 未证
`VLA-ACT-023..025/027/028/031..032`	与 Audit E 的 flow、padding、decode/controller 口径一致	local runtime/controller/ownership 未证

本次不改变 23 张相关卡或全局题卡的 `unseen` 状态，也不创建 `coaching_state.md`。

8. 不能推出的本地项目事实

- 不推出 OpenPiVLA 使用 `Pi0`、`Pi0FAST`、Pi05 hybrid 或自定义 Qwen/JSON 路线。
- 不推出本地 action `H=10`、`D=32`、7D slice、FAST revision、scale/vocab、normalizer、replan=5 或 97.4%。
- 不推出官方 example 的 50 trials、seed=7、图像旋转、LIBERO success predicate 已用于用户项目。
- 不把 π0.5 论文的 280k/80k、`\alpha=10` 或公开性能写成本人训练结果。
- 不把 upstream code、Git author 或本次 probe 写成候选人个人 ownership。

9. 下一步

Audit B 的 π0.5 hybrid、FAST、OpenPI shape/norm/rollout 高风险事实已完成独立复核，公共事实层为 `PASS WITH WARNING`；整体项目层仍因 local checkpoint、tokenizer revision、stats、

axis/unit、H/K、controller、eval bundle 与 ownership 缺失保持 `WARN/PENDING`。下一 bounded slice 优先独立复核 Audit C 的 ACT paper/code gap，或进行一题真实口述。

参考入口

- VOL 03 π0 / π0.5 / FAST / OpenPI Claim-to-Code
- VOL 03 π0 / π0.5 / FAST / OpenPI 独立事实复核
- VOL 03 Flow Matching / π0 Time–Sign 独立复核
- VOL 03 Completion Reconciliation
- OpenPiVLA Inventory
- OpenPiVLA Ownership Ledger

Part 16 / 16 · `interview_materials/vla/sources/vol_03_vla_action_sources.html` · SHA-256 `4C89E5CEF978C8E7F841EA946D93222F2419A10D69333F0A3F5EEC178E24E4DD`

返回本卷目录

Canonical Markdown SHA-256：`4c89e5cef978c8e7f841ea946d93222f2419a10d69333f0a3f5eec178e24e4dd`；生成自：`interview_materials\vla\sources\vol_03_vla_action_sources.md`

VOL 03 VLA / Action：来源计划

调研日期：2026-07-16；registry reconciliation：2026-07-19 状态：`source_registry_reconciled / audits_A_F_public_facts_reviewed / local_gates_warn` 目标：为 VLA 架构、action token、FAST、ACT、Diffusion Policy、Flow Matching 和 openpi action expert 建立 primary-source ledger。 边界：Audit A–F 的 public paper/code facts 已分别完成 claim-to-code 与 independent review；这不证明本地 OpenPiVLA checkpoint、action parser/controller、97.4 lineage、个人 ownership 或口述 mastery。

1. Primary papers / technical reports

主题	来源与冻结版本	计划支持的 claim	当前状态
OpenVLA	OpenVLA: An Open-Source Vision-Language-Action Model, arXiv v3	VLM-to-action token、action discretization、open VLA landscape	Audit A endpoint/token/mask independently checked / PASS WITH WARNING
RT-2	RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control, arXiv v1	web/VLM knowledge 与 action-as-token 路线	Audit F action-token/serving boundary independently checked / WARN
Octo	Octo: An Open-Source Generalist Robot Policy, arXiv v2	generalist policy、task/action readout 对照	Audit F readout/mask independently checked / WARN
π0	π0: A Vision-Language-Action Flow Model for General Robot Control, arXiv v4	VLM context、action expert、flow-matching action generation	Audit E time/sign + Audit B/F config/rollout independently checked / WARN
π0.5	π0.5: a Vision-Language-Action Model with Open-World Generalization, arXiv v1	π 系列后续架构与 open-world 语境；不替代项目 checkpoint 证据	Audit B hybrid boundary independently checked / WARN
FAST	FAST: Efficient Action Tokenization for Vision-Language-Action Models, arXiv v1	frequency-space transform、quantization、BPE action compression	Audit B paper/Hub/defaults/mask independently checked / WARN
ACT	Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, arXiv v1	CVAE action chunking、temporal ensembling	Audit C paper/code gaps independently checked / PASS WITH WARNING

主题	来源与冻结版本	计划支持的 claim	当前状态
Diffusion Policy	Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, arXiv v5	conditional action diffusion、receding-horizon control	Audit D target/mask/horizon/runtime independently checked / PASS WITH WARNING
Flow Matching	Flow Matching for Generative Modeling, arXiv v2	probability path、conditional vector field、ODE sampling 基础	Audit E theorem/endpoint/time-sign independently checked / WARN

2. Official repositories frozen at 2026-07-16

Official repo	Frozen HEAD / branch	License evidence	本卷用途
openvla/openvla	`c8f03f48af692657d3060c19588038c7220e9af9` / `main`	MIT（Source Scout）	action tokenizer、model/prompt contract、inference decode
octo-models/octo	`241fb3514b7c40957a86d869fecb7c7fc353f540` / `main`	MIT（LICENSE 已复核）	block-wise attention、readout/diffusion head、FT mode、action padding mask
Physical-Intelligence/openpi	`15a9616a00943ada6c20a0f158e3adb39df2ccac` / `main`	Apache-2.0（LICENSE 已复核）	π policy config、action expert、normalization 与 policy I/O
real-stanford/diffusion_policy	`5ba07ac6661db573af695b419a7947ecb704690f` / `main`	MIT	action diffusion training/inference implementation
tonyzhaozh/act	`742c753c0d4a5d87076c8f69e5628c79a8cc5488` / `main`	MIT	CVAE chunk、temporal ensemble 与 evaluation path
facebookresearch/flow_matching	`11568d37f8d5a080e12aa7b5305d9c35ae07d136` / `main`	CC BY-NC 4.0	后续维护包；只补证现代 probability path / ODE solver / CFG API，不冒充 2022 历史实现
QwenLM/Qwen3-VL	`96588727e44c78b25ba03ea03b8e12f7e64fd0da` / `main`	Apache-2.0	仅作本地 VLM backbone 语境；不能证明 action head

3. 逐源阅读与实现映射队列

Batch	必须回答	证据动作	未完成前禁止的表述
A：OpenVLA	action token 对应哪些维度、如何离散/反离散、哪些 token 参与 loss?	论文公式 → repo tokenizer/model/inference 路径 → tensor shape 表	“所有 action-token VLA 都按 OpenVLA 实现”
B：π0 / π0.5 / openpi + FAST	observation/state/noisy action 怎样进入 action expert? flow 与 FAST 的 target、mask、normalization、solver/decode 在哪里?	π0/π0.5/FAST 论文 → frozen openpi + FAST Hub → 两条 train/infer 对照	“OpenPiVLA 与公开 π0/π0.5 完全相同”或“压缩率更高必然提升成功率”
C：ACT	CVAE latent、chunk query、temporal ensemble 的训练/推理差异是什么?	论文 → ACT repo model/eval path	“action chunking 等于 ACT”
D：Diffusion Policy	condition、prediction target、scheduler、horizon 与 receding-horizon 执行如何映射?	论文 → official workspace/policy/config path	“所有连续生成 head 都是 diffusion”

Batch	必须回答	证据动作	未完成前禁止的表述
E：Flow Matching	probability path、target velocity 与 ODE integration 如何映射到 π0 action tensor?	Flow Matching 基础 → π0 特化 → openpi code	“flow matching 只是少几步的 diffusion”
F：架构与执行	RT-2/OpenVLA/Octo/π0 如何比较? mask、FT、异构动作、延迟、checkpoint 与 controller 如何闭环?	四篇论文 → frozen OpenVLA/Octo/openpi → action parser/queue/test matrix	“repo example 即本地合同”或“shape 对就代表可执行”

Batch A：OpenVLA（public facts independently reviewed）

- OpenVLA Action Token 与 I/O Claim-to-Code：覆盖 `VLA-ARCH-001..005`、`VLA-ACT-002..004` 的上游论文与 frozen commit。
- 已冻结路径：`prismatic/vla/action_tokenizer.py`、`prismatic/vla/datasets/datasets.py`、`prismatic/models/vlas/openvla.py`、`prismatic/extern/hf/modeling_prismatic.py`、`prismatic/vla/datasets/rlds/utils/data_utils.py`、`prismatic/conf/vla.py`。
- 独立复核 已确认 256-bin paper abstraction、encoder 256 IDs、decoder 255 centers、endpoint alias 与 CE/metric mask；本地 tokenizer/checkpoint/stats/controller/ownership 仍 WARN。

Batch B：π0 / π0.5 / openpi + FAST（public facts independently reviewed）

- π0 / π0.5 / FAST / openpi Claim-to-Code：覆盖 flow paper/code 时间约定、π0.5 hybrid objective 边界、FAST DCT/量化/BPE、PioFAST masked CE、normalization、padding/slicing 与 H/K rollout contract。
- openpi 已冻结到 `15a9616a00943ada6c20a0f158e3adb39df2ccac`；FAST Hub 已冻结到 `ec4d7aa71691cac0b8bed6942be45684db2110f4`。
- 关键 warning：π0.5 论文联合 FAST-token CE 与 flow loss；当前审计的 `Pi0(pi05=True)` 主 loss 路径只直接实现 flow MSE，不能称该 commit 完整复现论文 hybrid pre-training。
- 独立复核 已确认 π0.5 two-stage boundary、FAST paper/Hub defaults、prefix/postfix mask、normalization、padding/output slice 与 LIBERO example；本地 formal Qwen route 已另证为 JSON/full-chat CE，不能称 FAST 等价。

Batch C：ACT（public facts independently reviewed）

- ACT CVAE、Action Chunk 与 Temporal Ensemble Claim-to-Code：覆盖 `VLA-ACT-011..017` 的上游 source contract。
- 已核对 arXiv `2304.13705v1` 与 `tonyzhaozh/act@742c753...`；明确 train-time posterior、test-time `z=0`、`[B,H,14]`、masked L1 + KL、整段执行和 per-step temporal aggregation。
- 关键 warning：论文 Algorithm 1 的 MSE 与正文/代码 L1 不一致；论文的 fixed/sinusoidal query 描述与 frozen code 的 learned `nn.Embedding` 不一致；masked L1 的 denominator 仍含 padded slots。

- 独立复核 已完成 posterior/`z=0`、L1/KL denominator、learned query、temporal weight、zero sentinel 与 H/K/R probes，公共事实层为 `PASS WITH WARNING`；本地 OpenPiVLA action schema/controller 仍 WARN。

Batch D：Diffusion Policy（public facts independently reviewed）

- Diffusion Policy 条件扩散、Horizon 与 Receding-Horizon Claim-to-Code：覆盖 `VLA-ACT-018..022`，并补 `VLA-ACT-017` 的 diffusion 对照。
- 已核对 arXiv `2303.04137v5`、`real-stanford/diffusion_policy@5ba07ac...` 与 frozen `diffusers=0.11.1` scheduler。
- 关键 warning：paper 提出的 warm-start 未出现在 frozen policy；paper main text / table / config / runtime 的 inference steps 分别出现 10 / 16 / 8 / 16；boundary padding 是 repeat-edge target，不是 valid-mask padding。
- 独立复核 已完成 epsilon/sample target、full-mean mask、boundary repeat、To/Tp/Ta/Kd/R、warm-start gap、10/16 steps 与 sim/real queue probes；本地 scheduler/normalizer/controller 仍 WARN。

Batch E：Flow Matching / π0（public facts independently reviewed）

- Flow Matching 基础、π0 特化与 ODE Claim-to-Code：覆盖 `VLA-ACT-023..030`，并完成 `VLA-ACT-026` 的 Diffusion epsilon / Flow velocity 对照。
- 已核对 arXiv `2210.02747v2`、`facebookresearch/flow_matching@11568d3...`、π0 v4 与 `openpi@15a9616...`。
- 关键 warning：基础论文 OT path 保留 $\sigma_{\min}$，后续 package 的 `CondOTProbPath` 才使用 exact endpoint；current package 不是 2022 历史实现且 license 为 CC BY-NC 4.0；openpi Pi0 loss 对 configured D 维直接 mean。
- 独立复核 已完成 Theorem 2、`sigma_min` endpoint、`t=1-τ`、velocity/sign/dt 与 10-step oracle；本地 target/solver/valid-dim/controller 仍 WARN。

Batch F：VLA 架构与动作执行（public facts independently reviewed）

- VLA 架构、适配、延迟与动作执行 Claim-to-Code：覆盖 `VLA-ARCH-007..012` 与 `VLA-ACT-031..032`。
- 已核对 RT-2 v1、OpenVLA v3、Octo v2、π0 v4，以及 `octo-models/octo@241fb351...`、既有 frozen OpenVLA/openpi commits。
- 关键 warning：Octo 提供 `timestep_pad_mask & action_pad_mask` 的有效维正例，而 frozen Pi0 仍是 all-D mean；openpi timing 只分到 model/server/previous-total，未自动拆 vision、sampler、network 与 queue；上游 DROID `[10,8]` example 不能证明本地 controller contract。

- 独立复核 已完成 RT-2 token/frequency、Octo readout/valid-count mask、openpi checkpoint→norm→transform、timing、chunk broker 与 DROID H/K probes；本地 checkpoint/latency/controller 仍 WARN。

4. OpenPiVLA 本地证据待核对表

Claim / contract	当前边界	进入 VOL 03 正文前需要的证据
base checkpoint / backbone	local formal source 使用 Qwen3-VL + LoRA；immutable formal checkpoint/load smoke 仍缺	checkpoint manifest、processor/base revision、加载日志
action dimension / axis order / unit	recipe/smoke 为 D=72，但 formal JSON 每行 `limit=16`；axis/unit 未核	strict schema、parser、policy config、output transform
action horizon / executed prefix / replan	formal H=15；official WebSocket artifact 为 H=4/D=7 的平行 lineage	formal generation/parser、rollout client 与 controller loop
normalization / denormalization	formal JSON route 未发现 normalizer/controller round-trip	norm stats、transform code、sentinel 回放
archived 97.4%	归档最强 eval，不等于当前 formal training 直接所得	eval summary、checkpoint ID、task/trial/split/protocol
personal contribution	上游架构不是个人原创	local diff、设计记录、run ledger、本人负责边界

5. 来源质量门

1. 论文支持方法定义；官方 repo 支持实现和接口；本地 artifact 才支持个人项目事实。
2. 每个显示公式必须写变量 shape、目标张量、mask 和 train/infer 映射。
3. 同名术语先核实现：action horizon、chunk、window、replan、control step 不默认同义。
4. 对比表只在 checkpoint、数据、任务、trial、controller 和 success protocol 可比时讨论指标。
5. 若源码路径或项目 artifact 缺失，标为 `未核实`，不以“通常实现”补齐本地事实。

6. Registry reconciliation 与下一批工作

1. Audit A–F 的 public paper/code independent review 均已完成；A/C/D/F 为 PASS WITH WARNING，B/E 的 claim-to-code registry 仍保留 WARN（其独立事实切片已单独记录），local/runtime/oral warning 不因此消失。
2. `VLA-ARCH-001..012` 与 `VLA-ACT-001..032` 的 knowledge/cards 44/44 已完成；44 张题卡全部 `unseen`。
3. OpenPiVLA local gates 已新增 formatter、full-sequence CE、72→16 lossy serialization、parser absence 与 official/Qwen parallel-lineage 证据；checkpoint、normalizer、controller、97.4 lineage 与 ownership 仍开放。

4. 下一 bounded slice 优先做候选人 ownership confirmation，或补 immutable formal checkpoint/generation bundle；无候选人回答时继续关闭 VOL 03 source registry 生命周期。

7. 本 registry 能证明与不能证明什么

结论	状态
九篇核心 paper/technical report 与七个 official repo 的用途和冻结版本已登记	`checked`
Audit A–F public facts 均有独立 review 页面	`checked`
VOL 03 knowledge/cards 44/44、stable ID 无缺口	`checked / all unseen`
本地 OpenPiVLA formal Qwen parser/controller round-trip	`missing`
archived 97.4 与 formal Qwen/official checkpoint 的同一 lineage	`unverified`
candidate 对 trainer/formatter/diagnostics 的个人 ownership	`pending confirmation`
浏览器 desktop/narrow visual QA	`PASS WITH WARNING (localhost 1440×1000 / 800×1000; local runtime/ownership/oral gates remain open)`

因此本 registry 的 verdict 保持 `WARN`：来源和 public facts 已闭环，项目真实性与训练掌握尚未闭环。

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 04 · World Model / Sim2Real

世界模型、联合 video-action 建模、规划与仿真到真实迁移

> 本卷由 12 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 04 World Model / Sim2Real：Foundations、评估与证据（Batches 1–2）

题卡

1. 02 · VOL 04 World Model / Sim2Real：核心面试题卡（Foundations + Evaluation）

来源与事实复核

1. 03 · VOL 04 Batch 1：独立事实复核记录（WM-039..044）
2. 04 · VOL 04 Source Audit E：Benchmark / Evidence Claim-to-Protocol
3. 05 · VOL 04 Source Audit C：Cosmos Policy / WAM Claim-to-Code
4. 06 · VOL 04 Cosmos Policy / WAM：1 epoch gate 与 full10k 运行族对账
5. 07 · VOL 04 Audit C：Cosmos Policy / WAM 独立事实复核
6. 08 · VOL 04 Audit D：Sim2Real 独立事实复核
7. 09 · VOL 04 Audit B：UWM 独立事实复核
8. 10 · VOL 04 Source Audit B：UWM Video / Action Diffusion Claim-to-Code
9. 11 · VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 Foundations Claim-to-Code
10. 12 · VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 独立事实复核

Part 1 / 12 · `interview_materials/vla/knowledge/vol_04_world_model_sim2real.html` · SHA-256
`900F3D6EB22BBB3924A6427CF889B322EE47EFBB19A528598F95EAA98F074F9E`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：
`900f3d6eb22bbb3924a6427cf889b322ee47efbb19a528598f95eaa98f074f9e`；生成自：
`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_04_world_model_sim2real.md`

VOL 04 World Model / Sim2Real：Foundations、评估与证据（Batches 1–2）

状态：`drafted / aggregate_knowledge_44_of_44 / cards_44_of_44_unseen / review_warn` 报告日期：2026-07-19 证据截止：2026-07-16 基础页覆盖：`VLA-WM-001..010`、`VLA-WM-039..044`，16/44 个知识节点；独立原子知识页 `VLA-WM-011..038`，28/44 个节点 VOL 04 aggregate：44/44 个知识节点，所有题卡仍 `unseen` 当前目标：先建立 world-model 定义/I-O、概率 dynamics、RSSM、rollout/uncertainty/actionability 基础，再把 strict protocol、metric ladder、消融、失败回写和 evidence audit 连成诊断闭环。 项目边界：UWM 74.4%/92.8%、WAM 84/118 与离线 MSE 目前只是本地材料中的 claim queue；本页不把它们升级为已核验个人结果。

问题（Question）

当前基础页回答 16 个相互依赖的问题；WM-011..038 由同目录独立原子知识页承载，VOL 04 总入口按 aggregate 计数。

Batch 2：World Model Foundations（本增量）

1. `VLA-WM-001`：world model 与 policy、environment simulator、reward/value model 有什么边界?
2. `VLA-WM-002`：observation、state、latent、reward、termination 应建模哪些变量?
3. `VLA-WM-003`：deterministic 与 probabilistic dynamics 分别表达什么?
4. `VLA-WM-004`：encoder、transition、decoder、reward 与 continuation head 如何连接?
5. `VLA-WM-005`：one-step prediction 好为什么不能保证 multi-step rollout?
6. `VLA-WM-006`：teacher forcing、open-loop 与 scheduled sampling 的分布差异是什么?
7. `VLA-WM-007`：aleatoric 与 epistemic uncertainty 如何影响 planning?
8. `VLA-WM-008`：video generation 何时是 actionable world model?
9. `VLA-WM-009`：auxiliary representation 与 queryable simulator 有何差异?
10. `VLA-WM-010`：model bias 和 compounding error 如何造成“想象成功、执行失败”?

Batch 1：评估、证据与失败归因（已完成知识与题卡）

1. `VLA-WM-039`：checkpoint、normalizer、init、warmup、horizon 与 rollout 数怎样组成 strict protocol?
2. `VLA-WM-040`：offline/open-loop/world-model proxy 与 closed-loop success 怎样建立联系而不夸大?
3. `VLA-WM-041`：action/future/value loss 下降但 online success 不变时，怎样做可归因消融?
4. `VLA-WM-042`：怎样检测 hallucination、physical inconsistency、action insensitivity 与 causal inconsistency?
5. `VLA-WM-043`：失败轨迹怎样分层归因并回写数据、训练或评估合同?
6. `VLA-WM-044`：怎样从 paper、repo、checkpoint、runtime 到 ownership 核对真实训练和 rollout 合同?

16 个问题组成“先定义可预测世界，再验证它能否安全用于决策”的链条：

```
# Foundations establish the contract; evaluation gates test its external validity.
define observation/state/latent/action/reward/continuation
          ↓
separate posterior filtering from prior rollout
          ↓
measure one-step → open-loop → closed-loop gaps
          ↓
freeze protocol + artifacts
          ↓
separate offline / open-loop / proxy / simulator / real metrics
          ↓
run action-only / +world / +value / all paired ablations
          ↓
intervene on actions and test physical / causal consistency
          ↓
localize the first failure and route it to the correct owner
          ↓
audit paper → repo → checkpoint → runtime → personal contribution
```

一句话总口径

> World-model 评估的核心不是多报几个 loss，而是证明“同一模型、同一协议、同一初始条件下，某个可干预机制改善了可复现的闭环结果”；任何缺失的 checkpoint、normalizer、init、controller 或 episode record 都会切断这条证据链。

知识地图位置与当前批次边界

前置知识

- VOL 01：MDP/POMDP、随机变量、置信区间、坐标系、控制频率和 simulator/controller 边界。
- VOL 02：BC、offline proxy、value、distribution shift 与消融的基本因果口径。
- VOL 03：action shape/axis/unit、normalization、chunk/replan、decode→controller 执行闭环。
- VOL 04 Source Audit A–E：world-model probability contract、UWM/Cosmos 实现、Sim2Real 与 benchmark protocol；Foundations 与 Batch 1 已有独立 frozen-source 复核。[1][2][3][4][5][16][17]

当前两个批次做什么

范围	深度	主要依据
world-model contract	observation/state/latent/action/reward/continuation	Audit A + Foundations independent review
latent dynamics	deterministic/probabilistic、posterior/prior、RSSM heads	World Models、PlaNet、DreamerV3 paper/repo
rollout validity	one-step、teacher-forced、open-loop、scheduled sampling	Audit A、PlaNet/DreamerV3
uncertainty/actionability	aleatoric/epistemic、intervention、auxiliary-vs-simulator	Audit A/B/C
strict protocol	12-field tuple + run/episode manifest	Audit E、LIBERO/UWM/Cosmos frozen runners
metric ladder	teacher-forced、open-loop、WM proxy、sim、real	Audit A/B/E、WorldGym
multi-objective ablation	2×2 world/value factorial + paired eval	Audit B/C/E
actionable consistency	action intervention、causal alignment、constraint violation	Audit A/B/C
failure trace	first-failure taxonomy + artifact routing	Audit C/D/E
evidence audit	paper→repo→checkpoint→runtime→ownership	Audit A–E

当前仍不做什么

1. 不展开 `VLA-WM-011..038` 的完整训练目标、planning/value、多目标 routing 与 Sim2Real 方法课；它们属于后续独立批次。
2. 不把 LIBERO/RoboCasa simulation 写成 Sim2Real，也不声称用户做过真实机器人部署。[4]
3. 不把项目页、resume 或口述数字当作 raw artifact；缺少 checkpoint、config、normalizer、episode trace 时统一标为 `claim-only`。[5]
4. 当前只生成 16/44 个知识节点；`VLA-WM-001..010` 与 `VLA-WM-039..044` 均已有题卡，16 张全部保持 `unseen`。

VLA-WM-001：World Model 不是 Policy、Value 或环境本体

先讲直觉

世界模型（world model）是在模型内部回答“如果在当前情境下执行这段动作，后面可能发生什么”的条件预测器。它可以预测未来 observation、latent/state、reward 或 continuation，但它不自动决定该做什么；决定动作的是 policy 或 planner。真实 environment/simulator 负责产生实际 transition，learned world model 只是由数据拟合出的近似。[1][17]

概率合同

令 $o_{\le t}$ 是历史观测，$a_{t:t+H-1}$ 是候选动作序列，g 是任务/语言条件。一个通用合同可写成：

$$p_\theta\left(o_{t+1:t+H}, s_{t+1:t+H}, r_{t:t+H-1}, c_{t+1:t+H} \mid o_{\le t}, a_{t:t+H-1}, g\right).$$

变量解释：s 是可选的真实或 latent state，r 是一步 reward，$c \in \{0,1\}$ 是 continuation；并非所有实现都预测全部变量。直觉是先明确“条件是什么、未来变量是什么、预测多长”。实现映射：dataset

schema、encoder/transition/heads、action horizon 与 loss mask 必须能回到这组变量；只有视频输出而没有 action contract 时，不能默认它可用于控制。[1][2][3][17]

模块	输入→输出	主要职责	不能自动证明
policy	observation/task → action	选择或生成动作	对未来动力学建模正确
world model	history/action → future variables	预测动作后果	已找到最优动作
environment/simulator	current state/action → actual next state	产生真实/仿真 transition	learned model 的泛化
reward model	state/transition/task → immediate reward	给一步偏好/任务信号	长期 return 已校准
value model	state/policy → expected future return	评价长期结果	world transition 正确
success classifier	trace/final state → success label	评估或终止判断	可替代 reward/value/controller

项目与答题边界

- WAM/Cosmos 的 world/value/action slot 是上游公开设计；没有 local checkpoint/diff 时不能称为个人提出。[3]
- UWM 的 video/action diffusion 可具备 world-model 角色，但项目名或视频质量不证明 local checkpoint 的 action consequence 正确。[2]
- 面试中先说“在这个项目里 world head 是 auxiliary 还是 rollout-time query”，再谈收益。

VLA-WM-002：选择 Observation、State、Latent、Reward 与 Termination

POMDP 视角

机器人通常只有部分可观测信息。可用一组分解表达：

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t), \qquad r_t \sim p(r_t \mid s_t, a_t), \qquad c_t \sim p(c_t \mid s_t).$$

变量解释：s_t 是 Markov state，o_t 是相机/proprio 等 observation，a_t 是执行动作，r_t 是一步 reward，c_t 表示 episode 是否继续。直觉是环境在 state 上转移，传感器只露出 observation。实现映射：真实 simulator state 可能只用于评估，模型内部以 encoder 产生的 z_t 或 recurrent belief (h_t, z_t) 近似 state；字段必须在 dataset 与 eval manifest 中分开。[1][17]

变量取舍表

变量	何时必须	何时可省	省略风险
observation	几乎总是，作为感知输入/重建目标	state-only 控制实验	无法审计相机/遮挡/视角 shift
privileged state	simulator training/eval、几何/接触诊断	真实部署不可得	把 privileged input 泄漏到 policy
latent/belief	高维 observation、部分可观测、多步 rollout	小型 fully observed state	latent 可能丢任务细节或 collapse
reward	model-based RL、planning scoring	纯 representation auxiliary	无法评价候选 trajectory
continuation/termination	variable-length episode、return bootstrapping	固定无终止 horizon	imagined rollout 穿过 terminal

变量	何时必须	何时可省	省略风险
future observation/state	world prediction 或 consistency	只学 reward/value 的抽象模型	无法做状态级 intervention 诊断

最小 I/O 合同

一次样本至少记录 obs_keys/history、action shape/axis/unit、state/privileged flags、future horizon、reward semantics、termination vs truncation、mask/padding。termination 是任务/环境终止，truncation 是评测时限；把二者都当 $c = 0$ 会错误截断 value target。

VLA-WM-003：Deterministic 与 Probabilistic Dynamics

两种合同

deterministic dynamics 输出一个点：

$$\hat{s}_{t+1} = f_\theta(s_t, a_t).$$

probabilistic dynamics 输出分布：

$$\hat{s}_{t+1} \sim p_\theta(s_{t+1} \mid s_t, a_t).$$

变量解释：f_θ 常对应均值/单一路径，p_θ 可由 Gaussian、mixture、categorical latent 或 diffusion 表示多种 future。直觉是前者回答“最可能下一步是什么”，后者回答“有哪些可能、各有多不确定”。实现映射：输出 head 决定参数形状，loss 分别可能是 MSE、NLL、KL 或生成式 denoising；采样次数、temperature 与 seed 属于 rollout contract。[1][2][17]

能表达什么，不能表达什么

设计	优势	主要风险	合适场景
deterministic	训练/推理简单、速度快、轨迹稳定	对多未来取平均，接触/遮挡可能被模糊	低噪声 state dynamics、短 horizon
stochastic single model	能表达 aleatoric 多模态	likelihood 失配、采样方差	多未来、视频/接触 uncertainty
ensemble	可用 disagreement 估计 epistemic	计算/存储成本，成员相关性	OOD 检测、风险敏感 planning
stochastic latent + ensemble	同时覆盖多模态与知识不足	校准困难、planner 易钻漏洞	高风险候选筛选，需严格校准

高频误区：stochastic latent 不自动等于 epistemic uncertainty；单模型可能只学到数据噪声。deterministic model 也不是“没有不确定性”，只是没有在输出合同中显式表达。

VLA-WM-004：Latent World Model 与 RSSM 的完整数据流

Encoder、Posterior、Prior 与 Heads

DreamerV3 风格 RSSM 可写成：[1][17]

$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim q_\phi(z_t \mid h_t, o_t), \qquad \hat{z}_t \sim p_\phi(z_t \mid h_t).$$

变量解释：h_t 是 deterministic recurrent state，z_t 是 stochastic latent；posterior q 在训练/filtering 时读取当前 observation，prior p 在 imagination 时只依赖历史与动作。直觉是“看见现实时校正 belief，想象未来时不能偷看未来图像”。实现映射：frozen danijar/dreamerv3 reimplementation 的 RSSM.observe、RSSM.imagine、RSSM.loss 分别对应 filtering、prior rollout 与 dyn/rep KL；它不是 Google/DeepMind 官方代码。[1][17]

heads 在 feature $x_t = [h_t, z_t]$ 上预测：

$$p_\phi(o_t \mid x_t), \qquad p_\phi(r_t \mid x_t), \qquad p_\phi(c_t \mid x_t).$$

变量解释：decoder/reconstruction head 保留 observation 信息，reward head 提供一步任务信号，continue head 控制 imagined return 的 discount/termination。实现映射：每个 head 要有独立 target、loss scale 与 mask；不能把 total loss 下降解释为每个 head 都变好。[1][17]

```
def rssm_train_and_imagine(batch, model):
    # 1. Encode real observations and update posterior states with o_t.
    tokens = model.encoder(batch["observation"])
    posterior = model.rssm.observe(tokens, batch["action"], batch["reset"])

    # 2. Fit reconstruction, reward, continuation, dynamics and representation losses.
    losses = model.world_model_losses(posterior, batch)

    # 3. Start from replay posterior states, then roll the prior without future observations.
    imagined = model.rssm.imagine(posterior.start_states, model.policy,
                                  horizon=model.imag_horizon)

    # 4. Predict imagined reward/continuation for actor-critic or diagnostic rollout.
    return losses, model.reward(imagined), model.continue_head(imagined)
```

这段是机制映射，不是用户项目执行日志。项目若声称使用 RSSM，必须补 local module path、config、checkpoint 与 tensor trace。

VLA-WM-005：One-Step 好不代表 Multi-Step 可靠

Teacher-Forced 与 Autoregressive 的输入分布不同

one-step evaluation 每步用真实 history；open-loop rollout 从第二步开始使用模型自己的预测。若局部误差为 e_t，局部 transition 对 state perturbation 的放大因子为 L，可用分析上界理解：

$$\|\hat{s}_{t+h} - s_{t+h}\| \lesssim \sum_{j=0}^{h-1} L^{h-1-j} e_{t+j}.$$

变量解释：h 是 rollout horizon，e_{t+j} 是每个局部 prediction error，L 是局部 sensitivity 的直觉量而非固定常数。当 $L > 1$ 或 rollout 进入训练分布外 state，小误差会放大。实现映射：评测必须输出 error_by_horizon[h]，同时记录 teacher-forced/open-loop、sampler seed、action sequence 与 reset；不能只报平均 one-step MSE。[1][17]

常见断点

one-step 看起来好	multi-step 失败原因	最小检查
pixel MSE 低	背景占比大、任务对象误差被平均	task-object pose/error crop
latent loss 低	latent 丢 action/contact 信息	action probe + contact probe
reward error 低	rollout state OOD、reward shortcut	expert-state vs rollout-state error
future video逼真	action ignoring 或 sign/scale错	same-state action swap
短 horizon稳定	compounding 与 terminal 漏建模	horizon curve + continuation audit

VLA-WM-006：Teacher Forcing、Open-Loop 与 Scheduled Sampling

三种训练/评估分布

teacher forcing 使用真实前一步 x_t 预测 x_{t+1}；open-loop 使用 $\hat{x}_t$ 继续预测；scheduled sampling 在训练时随机混用二者：

$$\tilde{x}_t = b_t x_t + (1 - b_t)\hat{x}_t, \qquad b_t \sim \text{Bernoulli}(\rho_t).$$

变量解释：ρ_t 是使用真实输入的概率，可随训练下降；图像/离散 latent 中“加法”只是选择器的简写，不一定做数值插值。直觉是逐步暴露模型自己的错误状态。实现映射：training batch 要记录 teacher/predicted mask、stop-gradient 规则、schedule 与跨 episode padding；否则无法判断模型是否真正见过 rollout-state distribution。

Protocol	输入来源	主要用途	风险
teacher forcing	全部真实 history	稳定监督、component fit	exposure bias
scheduled sampling	真实/预测混合	缓解 train-rollout gap	schedule bias、目标不一致
open-loop eval	模型预测 history	测 compounding	不代表 controller 可执行
closed-loop eval	环境真实 observation 回流	测 policy+model+controller	难以单独归因 world head

```
def evaluate_rollout_modes(model, episode, horizons):
    # 1. Reuse the same start state, actions and sampler seeds across modes.
    one_step = model.teacher_forced_predict(episode.observation, episode.action)

    # 2. Feed predictions back only in open-loop mode to expose compounding error.
    open_loop = model.autoregressive_rollout(episode.start, episode.action,
                                             max(horizons), fixed_seed=0)
```

```
    # 3. Report each horizon separately; never collapse both modes into one MSE.
    return compare_by_horizon(one_step, open_loop, episode.target, horizons)
```

scheduled sampling 是设计选项，不是保证；若 action、termination 或 observation schema 本身错误，混入预测输入只会训练模型适应错误分布。

VLA-WM-007：Aleatoric 与 Epistemic Uncertainty

两类不确定性的来源

- **Aleatoric**：环境/观测本身存在不可消除的多结果，例如遮挡后物体位置、接触滑移或人类行为随机性；更多同分布数据也不一定消失。
- **Epistemic**：数据覆盖不足或模型知识有限，例如新物体、新相机、未见 action magnitude；补相应数据或模型 ensemble 可能降低。

总预测方差可用 law of total variance 作分析分解：

$$\mathrm{Var}(Y \mid x) = \mathbb{E}_\theta[\mathrm{Var}(Y \mid x, \theta)] + \mathrm{Var}_\theta(\mathbb{E}[Y \mid x, \theta]).$$

变量解释：第一项近似成员内部的 aleatoric predictive variance，第二项近似不同模型/参数间的 epistemic disagreement。直觉是区分“世界本来就多解”和“模型没见过”。实现映射：ensemble member/checkpoint、采样 seed、distribution head 与 calibration split 必须冻结；单次 diffusion sample 的变化不能直接当 epistemic uncertainty。[1][17]

Planning 中怎样使用

可用风险敏感目标表达：

$$a^* = \arg\max_a \Big(\mathbb{E}[\hat{R}(a)] - \beta U(a)\Big).$$

变量解释：$\hat{R}(a)$ 是 predicted return，$U(a)$ 是 uncertainty/constraint risk，$\beta \geq 0$ 是风险权重。直觉是高均值候选若来自模型不确定区域，应降权或触发 fallback。实现映射：需要 calibration curve、coverage/abstention、N sweep 与真实 return 对照；未校准 disagreement 可能奖励多样性而非风险。

VLA-WM-008：Video Generator 何时是 Actionable World Model

必须通过的四道门

1. **Action condition**：候选 action sequence 以明确 axis/unit/time 进入 transition，而不是只用文本描述动作。

2. **Sensitivity**：固定 observation 与 sampler seed，改变 direction/magnitude/gripper/order 时 future 有可辨识差异。
3. **Alignment**：这种差异与 simulator/真实 intervention 的对象 pose、joint/contact 方向一致。
4. **Decision utility**：生成 future 能在冻结协议下改善候选筛选、planning 或 failure detection，而非只提高视觉分数。

条件分布可写为：

$$p_\theta(o_{t+1:t+H} \mid o_{\le t}, a_{t:t+H-1}, g).$$

变量解释：g 是任务条件，a 必须是实际可执行动作合同。直觉是同一画面下“向左”和“向右”要产生不同且方向正确的未来。实现映射：保存 paired actions/videos、sampler seed、normalizer、horizon 与 environment clone state；只有 FVD/PSNR 或主观逼真度时，最多称 action-conditioned video predictor。[1][2][3][16][17]

证据	可以说	不能说
action token/latent进入模型	模型结构上有 action condition	模型真的使用 action
action swap 产生不同视频	有 action sensitivity	变化方向正确
cloned-state alignment	局部动作后果与环境一致	长 horizon/真实物理全部正确
proxy rank 有效	可筛选所测候选	可替代 simulator/real eval

UWM/Cosmos 的上游结构可以作为实现例子，但当前 local 项目没有 checkpoint-level sensitivity/alignment artifact，不能说本人已经验证 actionable dynamics。

VLA-WM-009：Auxiliary Representation 与 Queryable Simulator

两种使用方式

维度	Auxiliary world objective	Queryable simulator
训练目的	用 future/reconstruction 改善共享表示	学可供候选 action 查询的 transition
推理时是否调用	可以完全不用 world head	必须对一个或多个 action rollout
输出要求	latent feature 对 policy 有帮助即可	future/reward/continuation 有稳定合同
action coverage	与训练 policy 分布一致即可起作用	需覆盖 planner proposal/OOD action
核心证据	action-only vs +world online消融	prediction/calibration + planning uplift
主要风险	gradient conflict、shortcut、额外数据混杂	model exploitation、latency、compounding

auxiliary 训练常写成：

$$\mathcal{L} = \mathcal{L}_{policy} + \lambda_w \mathcal{L}_{world}.$$

变量解释：λ_w 是 world auxiliary weight；只有 same-data/same-budget 的 action-only 与 +world 消融，才能把差异归因给目标。实现映射：必须记录 sample routing、loss mask、gradient scope 与每项 loss，不能用 total loss 证明 policy 改善。[3][16]

queryable simulator 则在推理时显式比较候选：

$$a^* = \arg\max_{a \in \mathcal{A}} \widehat{V}\left(\widehat{T}(s_t, a)\right).$$

变量解释：$\widehat{T}$ 是 learned transition，$\widehat{V}$ 是 learned value/scorer，$\mathcal{A}$ 是 policy proposal set。实现映射：必须记录 proposal N、world/value checkpoint、denoise/rollout steps、selector 与 latency；world head 只在训练中出现时不能称 model-based planning。[3]

VLA-WM-010：Model Bias、Compounding 与“想象成功”

三层误差会被 Planner 放大

planner 不是被动使用 world model，而会主动寻找评分最高的 action：

$$\hat{a} = \arg\max_{a \in \mathcal{A}} \widehat{V}\left(\widehat{T}^{(H)}(s_t, a)\right).$$

变量解释：$\widehat{T}^{(H)}$ 是 H 步 imagined transition，$\widehat{V}$ 是 scorer。直觉是 optimizer 会寻找模型最乐观的位置；候选越多，越可能选中 hallucination、物理漏洞或 value overestimate。实现映射：需要 random/median/top selector、N sweep、horizon sweep、selected-vs-unselected real outcome 与 latency，不可只报 best-of-N 最终分数。[3][16]

断点	典型症状	诊断
transition bias	action effect 方向/幅度错	cloned-state intervention
compounding	短时对、长时漂	error/violation by horizon
reward/value bias	失败 future 得高分	calibration + rank trace
proposal OOD	planner 提出训练外动作	action density/disagreement
terminal/contact漏建模	穿过 terminal、无接触抓取	continuation/contact constraint
controller mismatch	imagined delta 无法物理执行	output-transform + tracking trace
no recovery	首次偏离后持续恶化	first-divergence + recovery-state eval

```
def diagnose_imagination_gap(model, planner, cloned_env, start_state):
    # 1. Freeze candidates so planner and environment evaluate identical actions.
    candidates = planner.propose(start_state, fixed_seed=0)

    # 2. Save imagined futures, values and uncertainty for every candidate.
    imagined = [model.rollout(start_state.obs, a) for a in candidates]
    scores = planner.score(imagined)

    # 3. Execute selected and control candidates from cloned simulator states.
    real = [cloned_env.rollout(start_state.clone(), a) for a in candidates]

    # 4. Compare model/environment deltas by horizon and locate first divergence.
    return compare_transition_value_and_constraints(imagined, scores, real)
```

最小防守口径

“我不会因为 imagined video 或 value 看起来好就说 world model 可用于规划。我会固定 candidate/action contract，先做 horizon-wise transition 与 action intervention，再看 value calibration 和 selected-vs-control closed loop；最后核对 controller、contact 与 recovery。缺 local checkpoint/episode trace 时，这是一套设计方案，不是我已经跑出的结果。”

Foundations：从定义到可诊断 Rollout 的最小 Pipeline

```
# Each arrow is an interface that needs a schema and an evaluation gate.
observation/history/task
        ↓ encoder + posterior filtering
belief state (h_t, z_t)
        ↓ action-conditioned prior transition
future latent/state/observation
        ↓ reward + continuation + optional value
candidate return / risk
        ↓ planner or policy selector
controller action
        ↓ same-protocol environment rollout
error-by-horizon + first-divergence trace
```

这条 pipeline 同时解释了十个基础问题：WM-001/002 定义变量与模块，003/004 定义 dynamics 与 latent data flow，005/006 解释 train-rollout gap，007 定义风险，008/009 决定 world model 是否可查询，010 解释为何 optimizer 会放大模型漏洞。

VLA-WM-039：Strict Protocol 是可重放合同

先讲直觉

strict 不是“我更严格地跑了一遍”，而是任何人拿到同一批 artifact 都能回答三个问题：用了什么模型、从什么状态开始、怎样执行与判成功。如果两个结果只共享 task 名，却使用不同 init distribution、warmup、chunk、controller 或 denominator，它们不是同一实验。

12-field protocol tuple

一次闭环评测写成：

$$\mathcal{P} = (S, I, W, O, A, K, C, H, Y, R, N, M).$$

变量解释：S 是 suite/task；I 是 initial-state distribution；W 是 warmup；O 是 observation/preprocess；A 是 action/normalizer；K 是 chunk 与 replan；C 是 controller/timing；H 是 horizon；Y 是 success predicate；R 是 randomness；N 是 trials/aggregation；M 是 checkpoint、代码与 runtime。直觉上，这 12 个字段共同定义“同一场考试”。实现映射：它们应进入 run manifest，而不是散落在命令、README 和口述里。[5]

字段	必须冻结	常见伪对齐
S suite/task	task IDs、task class/BDD hash、subset	只写“LIBERO-10”
I init	state file/hash/index、layout/style、object split	都是 50 rollouts 就认为相同
W warmup	action vector、步数、是否计 horizon	5 个 zero7 与 10 个 close-gripper noop 混用
O observation	camera keys、history、flip/crop/resize/JPEG	只对齐最终分辨率
A action	dim/order/unit/frame、stats、clip、gripper sign	都是 7D 就认为相同
K chunk	predicted H、executed K、queue/requery	chunk 16 但每次只执行 1 与执行 16 混用
C controller	config/hash、Hz、physics step、latency	都叫 OSC_POSE 就认为参数相同
H horizon	task max step、timeout、early stop	env horizon 与 eval horizon 混写
Y success	exact predicate、threshold、invalid rule	事后删 episode 后仍用原分母名
R randomness	train/eval/reset/sampler seed、determinism	只记录一个 global seed
N sampling	trials/task、seeds、macro/micro、CI	单 seed 与三 seed均值比较
M model/runtime	checkpoint SHA、stats/config/code/env/hardware	model ID 代替具体 tensor

成功率与 paired effect

固定 task t 后，二元成功率是：

$$\hat{p}_t = \frac{1}{N_t} \sum_{i=1}^{N_t} y_{t,i}, \qquad y_{t,i} \in \{0, 1\}.$$

变量解释：N_t 是该 task 的有效 episode 数，$y_{t,i}$ 必须由冻结的 success predicate 产生。直觉是分子和分母都要能回到 episode ID；只保存百分比会失去 invalid episode、init 与失败类型。实现映射：summary JSON 应由 immutable episode JSONL 聚合，而不是手工录入。

比较两个 checkpoint A/B 时，优先在相同 task/init/seed 上做 paired evaluation：

$$\hat{\Delta} = \frac{1}{N}\sum_{i=1}^{N}\left(y_i^B - y_i^A\right), \qquad \widehat{\mathrm{SE}}(\hat{\Delta}) = \sqrt{\frac{1}{N(N-1)}\sum_{i=1}^{N}(d_i - \bar{d})^2},$$

其中 $d_i = y_i^B - y_i^A$，$\bar{d} = \hat{\Delta}$。paired 设计让每个 init 充当自己的对照，减少 init 难度差异；二元小样本时应使用 paired bootstrap 或 exact paired test，而不是把正态近似当保证。实现映射：A/B 的 episode record 必须共享 `pair_id=(task, init_id, seed)`，否则不能计算这组差值。

UWM 三协议不能混名

协议	Init/reset	Warmup	Trial/seed	当前证据结论
UWM Table II v3	50 个 OOD init；扩大 object range 0.03；移除背景物体	公开 runner 未闭合	50/task × 3 finetune seeds	paper 内部 setting；0.79 属于该 setting [6]
frozen UWM repo rollout	`env.seed(e)+env.reset()`	无	template 50/task	reset-style；不是 Table II [2][7]
本地项目 fixed-init claim	项目页称 pruned init	项目页称 5×zero7	项目页称 50/task	缺 raw artifacts；74.4% 暂为 claim-only [5][14]

任务名相同不消除第一列的分布差异。正确表达是“`fixed-pruned-init-50-v1` 下的待核验结果”，而不是“Table II strict aligned”。[5][6]

Run 与 episode manifest

层级	最小字段	作用
run	run ID、owner、purpose、git diff hash	说明谁在何种代码状态执行
model	model/revision、checkpoint SHA/bytes、config SHA	防止同名 checkpoint 漂移
stats	action/proprio stats SHA、tokenizer/T5 revision	固定输入输出尺度与条件
runtime	Python、Torch、CUDA/MACA、MuJoCo、robosuite、hardware	解释数值与 sampler 差异
episode	task/init/seed、success/step、exception、video/trace SHA	让百分比可回放和清洗
protocol	$S..M$ 的结构化值	决定哪些 run 可比较

```
def freeze_eval_run(cfg, checkpoint, episodes):
    # 1. Hash every mutable model/config/stat asset used at inference.
    model_manifest = hash_assets(checkpoint, cfg.model, cfg.normalizer)

    # 2. Serialize the full protocol tuple; unknown is a blocker, not a wildcard.
    protocol = build_protocol_tuple(cfg)
    assert all(protocol[field] is not None for field in REQUIRED_PROTOCOL_FIELDS)

    # 3. Attach task/init/seed and raw artifact hashes to every episode.
    records = [attach_pair_id_and_hashes(ep) for ep in episodes]

    # 4. Derive summary from records so deleted/invalid episodes remain auditable.
    summary = aggregate_without_dropping_history(records)
    return {"model": model_manifest, "protocol": protocol,
            "episodes": records, "summary": summary}
```

VLA-WM-040：Metric Ladder 不允许跨层替代

五层证据阶梯

层级	例子	主要证明	不能直接证明
L0 component fit	action/future/value L1/MSE、KL	某 target 在 frozen split/noise 下被拟合	causal dynamics、controller success
L1 teacher-forced	one-step image/latent/reward error	真实历史条件下的局部预测	自回归稳定性
L2 open-loop	horizon-wise rollout error、action sensitivity	模型使用自己预测后的误差累积	真实环境可执行性
L3 WM proxy	WorldGym/VLM score、imagined value/rank	learned environment 中的筛选/排序能力	simulator 或真实 success 等价
L4 simulator closed loop	LIBERO/RoboCasa task success	固定 simulator/controller protocol 下的行为	real-world transfer
L5 real closed loop	real robot success、constraint/recovery trace	指定 hardware/protocol 下的部署证据	其他机器人/环境泛化

从 L0 到 L5 是逐层增加外部有效性，不是指标必然单调。一个模型可能 image MSE 更低，却因 action ignoring、contact shortcut 或 value miscalibration 在 L4 更差。[1][2][3]

one-step 好为何不能外推 long rollout

若第 t 步局部误差是 e_t，局部 dynamics 对 state 误差的放大因子近似为 L，可用分析框架表示：

$$\|\hat{s}_{t+h} - s_{t+h}\| \lesssim \sum_{j=0}^{h-1} L^{h-1-j} e_{t+j}.$$

变量解释：h 是 rollout horizon，L 不是固定的论文常数，而是局部 sensitivity 的直觉量。当 $L > 1$ 或预测进入 OOD state，小 one-step error 也会被放大。实现映射：评估必须输出 `error_by_horizon[h]`，并同时标注 teacher-forced 或 autoregressive；不能只报一个平均 MSE。[1]

如何验证 proxy 与真实 success 的关系

不要在单个 checkpoint 上说“相关”。至少准备多个真实不同的 policy variants $j = 1..J$，在同一 task distribution 上分别得到 proxy score q_j 与 simulator/real success p_j，再看 rank consistency：

$$\rho_s = \mathrm{Corr}\left(\mathrm{rank}(q_1, \ldots, q_J), \mathrm{rank}(p_1, \ldots, p_J)\right).$$

变量解释：ρ_s 是 Spearman-style rank correlation，用于回答 proxy 能否排序候选，而不是两者数值是否相等。实现映射：variants 必须在 checkpoint、规模、训练阶段等维度预先定义；每个 q_j, p_j 都要有

uncertainty/seed。即使 ρ_s 高，也只支持所测 policy/task 分布；WorldGym 的 VLM scorer、checkpoint 和 API version 仍需冻结。[11][12]

不夸大的三档结论

证据	安全结论	危险结论
action MSE 降	“held-out action fitting 改善”	“policy success 提升”
future error 降	“该 horizon 的预测更接近 target”	“学会真实物理”
proxy rank 与 sim rank相关	“可用于所测候选的前置筛选”	“可替代 simulator/real eval”
sim success 提升	“该 frozen sim protocol 下更好”	“已完成 Sim2Real”
real paired trials 提升	“指定 hardware/task/protocol 下改善”	“泛化到所有真实场景”

VLA-WM-041：Offline 下降但 Online 不变时怎样消融

先把“多目标有效”拆成四个训练臂

对 action 主任务，world/value 的最小 2×2 factorial 是：

Arm	Action loss	World loss	Value loss	回答的问题
A00 action-only	on	off	off	主任务基线
A10 +world	on	on	off	world 的边际作用
A01 +value	on	off	on	value 的边际作用
A11 all	on	on	on	联合与 interaction

四个 arm 必须共享 data split、initial checkpoint、train steps/tokens、optimizer budget、batch composition、action labels 与 eval protocol。若 `+world` 额外看更多 robot/video data，应把“objective effect”和“data amount effect”再拆开，不允许把二者都归因于 world loss。[2][3]

Factorial effect

对任一结果 Y（offline loss、proxy 或 online success）可写一个解释性模型：

$$Y_{w,v} = \beta_0 + \beta_W w + \beta_V v + \beta_{WV} wv + \varepsilon, \qquad w, v \in \{0, 1\}.$$

变量解释：w 与 v 表示是否启用 world/value；β_W 和 β_V 是在另一目标关闭时的边际 effect；β_{WV} 是联合目标超出简单相加的 interaction。直觉是 A11 更好不能自动说明两个 auxiliary 都有用。实现映射：每个 cell 至少多个 training seeds，并在相同 paired episodes 上评估；binary success 可用 paired bootstrap/分层模型，而不是只拟合一次线性回归。

Offline→online 断点矩阵

观察	更可能的断点	下一实验
action MSE ↓，success↔	normalizer/axis、distribution shift、recovery缺失	action round-trip、expert-state vs rollout-state error
world loss ↓，action/success↔	world 表示未进入 action decision、目标过易	frozen encoder probe、action intervention
value MSE ↓，best-of-N ↓	OOD overestimate、optimizer's curse	reliability/rank、random/median/top selector、N sweep
A10 ↑、A01↔、A11 ↓	world有益但 value冲突	gradient cosine、value label audit、weight sweep
all offline均 ↓、sim success ↓	protocol/checkpoint 或 closed-loop drift	same-run manifest diff、horizon/K sweep
sim ↑、real↔	reality gap/controller/contact	paired real trace、SysID/latency/contact tests

必须同时报告的四类结果

1. **Component metrics**：action/world/value 各自的 loss，不能只报 total loss。
2. **Representation/consistency probes**：action sensitivity、horizon error、value calibration。
3. **Closed-loop**：same-protocol paired task success、episode length、failure family。
4. **Cost**：train compute、inference latency、planning N 与 executed replan K。

```
def run_factorial_ablation(base_cfg, seeds, paired_eval_set):
    # 1. Create action-only, +world, +value and all with identical budgets.
    arms = make_2x2_arms(base_cfg, factors=["world_loss", "value_loss"])
    results = []
    for arm in arms:
        for seed in seeds:
            # 2. Train from the same initialization and record every config delta.
            ckpt = train_with_manifest(arm, seed, init=base_cfg.initial_checkpoint)

            # 3. Evaluate component metrics and consistency before closed-loop rollout.
            offline = evaluate_components(ckpt, base_cfg.frozen_holdout)
            probes = run_action_world_value_probes(ckpt)

            # 4. Reuse identical task/init pair IDs for online comparison.
            online = evaluate_paired(ckpt, paired_eval_set)
            results.append(record(arm, seed, offline, probes, online))

    # 5. Estimate main/interaction effects with seed and episode uncertainty.
    return summarize_factorial_effects(results)
```

VLA-WM-042：检测 Hallucination 与物理/因果不一致

四个概念先分开

现象	定义	典型症状
hallucination	生成了数据/环境不支持的对象、状态或成功线索	物体凭空出现、门未开却预测完成
physical inconsistency	违反几何、接触、连续性或约束	穿透、无支撑悬浮、夹爪未闭却抓起
action insensitivity	不同有效动作产生近似相同 future	action swap 后视频几乎不变
causal inconsistency	预测会变化，但变化方向与真实 intervention 不一致	向左动作预测向右移动、gripper sign反转

画面“好看”只能反驳明显视觉崩坏，不能反驳后三项。[1][2][3]

Action sensitivity 不是 action correctness

固定 observation o_t，构造语义上有差异的 action sequences a 与 a'，定义 horizon h 的 sensitivity：

$$S_{act}(h) = \mathbb{E}\left[\frac{d_\phi\left(\hat{o}_{t+h}(o_t, a), \hat{o}_{t+h}(o_t, a')\right)}{\|a - a'\|_2 + \epsilon}\right].$$

变量解释：d_ϕ 可以是 task-relevant feature/pose distance，ϵ 防止分母为零。S_{act} 过低提示 action ignoring；过高也可能是噪声放大。实现映射：action pair 应覆盖 direction、magnitude、gripper、noop 与时间置换，并保存相同 sampler seed 的 paired videos，避免 sampling noise 冒充 sensitivity。

Causal alignment 需要真实或 simulator intervention

从同一可克隆 state 分别执行 baseline action a^0 和候选 action a，比较模型与环境中的 task-relevant change：

$$C(h) = \mathbb{E}\left[\cos\left(\Delta\hat{\phi}_h(a, a^0), \Delta\phi_h^{env}(a, a^0)\right)\right].$$

其中 $\Delta\hat{\phi}_h = \phi(\hat{o}_{t+h}(a)) - \phi(\hat{o}_{t+h}(a^0))$，环境项同理。直觉是模型不仅要“对 action 有反应”，还要沿正确方向反应。实现映射：LIBERO 可从同一 simulator state clone 两条 rollout；feature 应优先使用 object pose、joint/contact state 等可解释量，而不是只用像素 embedding。该测试证明局部 intervention alignment，不证明任意长 horizon 因果正确。

Physical violation 与 horizon curve

令 $g_j(x) \le 0$ 表示第 j 个物理/任务约束，例如穿透深度、支撑关系、关节限位或对象守恒，可定义：

$$V_{phys}(h) = \mathbb{E}\left[\sum_j w_j \, \mathbf{1}\{g_j(\hat{x}_{t+h}) > 0\}\right], \qquad E(h) = \mathbb{E}[d(\hat{x}_{t+h}, x_{t+h})].$$

变量解释：w_j 是约束严重度，V_{phys} 统计 violation，$E(h)$ 画出 error 随 horizon 的变化。实现映射：能取得 simulator state 时直接用 pose/contact/joint；只有视频时，几何或 VLM judge 只能作为 proxy，必须标记 scorer version 与 uncertainty。[4][11][12]

最小 consistency suite

Test	固定	改变	观测	失败说明
action swap	observation、sampler seed	action direction	future delta	action ignoring / sign error
magnitude sweep	state、axis	action magnitude	monotonic pose change	saturation / scaling mismatch
gripper counterfactual	arm trajectory	open/close sign	contact/grasp state	gripper semantics错误

Test	固定	改变	观测	失败说明
noop test	state	valid noop vs motion	background/object motion	uncontrolled hallucination
temporal permutation	action multiset	order	endpoint/trajectory	time conditioning失效
cloned-state env pair	exact state	candidate action	model-vs-env delta	causal inconsistency
long-horizon rollout	start state	horizon h	$E(h), V_{phys}(h)$	compounding / physics drift
value consistency	same candidates	N/selector	rank vs real return	OOD value overestimate

```
def consistency_suite(world_model, cloned_env, start_state, action_pairs):
    # 1. Reuse the same state and sampler seeds for every counterfactual pair.
    reports = []
    for action_a, action_b in action_pairs:
        pred_a = world_model.rollout(start_state.obs, action_a, fixed_seed=0)
        pred_b = world_model.rollout(start_state.obs, action_b, fixed_seed=0)

        # 2. Measure whether the model reacts to the action at each horizon.
        sensitivity = feature_delta_by_horizon(pred_a, pred_b)

        # 3. Execute both actions from cloned simulator states for causal alignment.
        real_a = cloned_env.rollout(start_state.clone(), action_a)
        real_b = cloned_env.rollout(start_state.clone(), action_b)
        alignment = causal_alignment(pred_a, pred_b, real_a, real_b)

        # 4. Count task-specific geometry/contact/constraint violations.
        violations = physical_constraint_checks(pred_a, pred_b)
        reports.append((sensitivity, alignment, violations))
    return aggregate_with_horizon_and_seed_uncertainty(reports)
```

VLA-WM-043：Failure Trace 要定位第一处断点

不从 success=0 直接猜 backbone

最终失败通常是上游小误差级联的结果。应定位 first observable divergence：第一次相机/状态不一致、第一次 action 反向/饱和、第一次 controller 跟踪失败、第一次接触丢失、第一次规划选错或第一次无法恢复。只看最后一帧会把 recovery failure 错归因给 perception。

推荐顺序：

```
# Stop at the first layer that differs from the frozen contract.
protocol / checkpoint
        ↓
observation and prompt
        ↓
action transform / gripper
        ↓
chunk queue / controller / timing
        ↓
free-space dynamics
        ↓
contact / object interaction
        ↓
planning / value selection
        ↓
recovery / fallback / success predicate
```

Failure taxonomy 与证据

Layer	最小 trace	常见 failure	先做什么
protocol	run manifest、task/init/seed	不同分布或 denominator	protocol diff + paired rerun
checkpoint	SHA、load keys、stats SHA	错 tensor/EMA/normalizer	strict load + tensor stats
observation	raw/preprocessed frame、prompt、history	view/flip/crop/time mismatch	image pair + timestamp replay
action	normalized/raw/env action	axis/unit/gripper/clip错误	round-trip + expert action replay
chunk/control	query/queue/command/state timestamps	stale chunk、延迟、controller mismatch	K sweep + tracking error
dynamics	predicted/real free-space pose	action consequence失真	cloned-state intervention
contact	contact force、grasp/object state	slip、penetration、wrong compliance	contact/gripper trace
planning/value	candidates、future/value/rank	hallucination被高 value选中	N/selector/calibration sweep
recovery	first error、replan、fallback event	无法从 OOD state返回	recovery-state dataset
success/infra	predicate、timeout、exception	false positive/negative、漏计	exact predicate + invalid rule

回写不是把所有失败都加回训练集

Failure family	回写目标	不应做的事
observation	修 converter/wrapper；补 view/time hard cases	用更多模型容量掩盖 flip bug
action contract	修 stats/axis/unit；加 round-trip unit tests	把错误动作当新标签训练
action coverage	补转折、gripper、recovery states	只重复 easy demos
world dynamics	补 action-diverse transition 与 long rollout supervision	只优化像素细节
contact	补高频 contact/gripper state 与 hard negatives	仅凭视频主观标物理正确
value/planning	补 OOD failure/value label；做 calibration	直接增大 N 追逐高估值
recovery	从 first deviation 后采集 corrective trajectories	只从专家初始分布重跑
protocol/infra	修 harness、manifest、validity rule	把无效 episode 当模型失败

```
def classify_and_route_failure(episode, frozen_protocol):
    # 1. Reject protocol/checkpoint mismatches before model diagnosis.
    assert_manifest_matches(episode.run_manifest, frozen_protocol)

    # 2. Find the earliest timestamp where an interface or behavior diverges.
    first = find_first_divergence(episode.raw_obs, episode.model_obs,
                                  episode.model_action, episode.env_action,
                                  episode.controller_state)

    # 3. Attach evidence, not only a human-readable label.
    family = classify_with_traces(first, episode.contact,
                                  episode.candidates, episode.success_predicate)

    # 4. Route only confirmed failures to code, data, objective or eval owners.
    destination = ROUTING_TABLE[family]
    return create_failure_record(first, family, destination,
                                 artifact_hashes=episode.hashes)
```

VLA-WM-044：从 Paper 到 Ownership 的 Evidence Audit

五层链条

层	能证明	不能跨越
paper	方法定义、实验 setting、论文报告结果	当前 repo default、checkpoint tensor
official repo	frozen commit 的代码路径和默认/示例 config	论文实际 run、用户 local diff
checkpoint/assets	某组 tensor、stats、tokenizer/T5 与 license	环境能加载、成功率
runtime/eval	exact environment/controller/protocol 下的 episode records	个人原创机制、其他 protocol 泛化
ownership	local diff、design note、commands/logs、负责范围	上游公开结构归为个人创新

强 claim 需要链条连续。例如“我复现 UWM Table II”至少需要 paper OOD setting → 对应 env patch → checkpoint/config/stats → 3-seed 50-init episode records → local execution/ownership。仅有 task 名、README command 和一个成功率不够。[2][5][6]

Evidence labels

Label	定义	推荐语气
`primary`	paper、benchmark paper、官方技术报告	“论文定义/报告……”
`official-code`	frozen official repo、docs、model/dataset card	“该 commit 的代码路径……”
`local-evidence`	local code/config/log/checkpoint/eval artifact	“本地 artifact 显示……”
`inference`	多项证据支持但未直接记录	“由同源证据推断……”
`unverified`	缺 source、revision、artifact 或相互矛盾	“尚未核实，不能确认……”

Claim ledger

每条面试 claim 建议记录：

字段	问题
claim	具体说了什么，是否含数字、因果或 ownership?
source	primary/official/local 的 URL/path/revision?
supports	source 真正支持哪一部分?
does not support	最容易被错误外推的是什么?
protocol	$\mathcal{P}$ 是否完整且与 baseline 相同?
artifact	checkpoint、stats、episodes、trace 是否有 SHA?
ownership	上游、本人、团队分别做了什么?
verdict	verified / directional / claim-only / blocked

```
def audit_interview_claim(claim, evidence_graph):
    # 1. Parse method, metric, protocol and ownership subclaims separately.
    atoms = decompose_claim(claim)
    verdicts = []
    for atom in atoms:
        # 2. Walk primary → code → checkpoint → runtime → ownership evidence.
        chain = evidence_graph.find_chain(atom)

        # 3. Missing revisions or artifacts stop the chain; intent is not evidence.
        if chain.has_gap() or chain.has_protocol_mismatch():
            verdicts.append((atom, "claim-only", chain.gaps()))
```

```
        else:
            verdicts.append((atom, "verified", chain.artifact_hashes()))

    # 4. Phrase the final answer at the weakest supported atom, not the strongest one.
    return build_truth_bounded_answer(verdicts)
```

一个典型的错误升级

```
# Each gap below requires a new source or runtime artifact.
paper reports 0.79 on five named tasks
        ≠ repo default runner reproduces 0.79
        ≠ my fixed-init run is protocol-equivalent
        ≠ my checkpoint achieved the paper result
        ≠ I invented UWM's architecture
```

每个 `≠` 都需要新的证据层。面试中主动指出边界通常比给出一个无法防守的强数字更可信。

跨六题的最小执行 Pipeline

```
def evaluate_world_model_policy(candidate_runs, source_ledger):
    # Gate 1: audit paper/repo/checkpoint/runtime and freeze protocol tuples.
    audited = [audit_run_sources(run, source_ledger) for run in candidate_runs]
    comparable = select_runs_with_identical_protocol(audited)

    # Gate 2: compute component, teacher-forced, open-loop and proxy metrics separately.
    ladder = evaluate_metric_ladder(comparable)

    # Gate 3: if an auxiliary objective is claimed, require the 2x2 factorial arms.
    effects = run_or_load_world_value_factorial(comparable)

    # Gate 4: intervene on actions and compare predictions with cloned env rollouts.
    consistency = run_action_and_physics_consistency_tests(comparable)

    # Gate 5: run same-init closed-loop episodes and record the first divergence.
    episodes = paired_closed_loop_eval(comparable)
    failures = [classify_and_route_failure(ep, comparable.protocol)
                for ep in episodes if not ep.success]

    # Gate 6: emit claim ledger; unsupported personal claims remain blocked.
    return build_evidence_report(ladder, effects, consistency,
                                 episodes, failures, source_ledger)
```

这段 pipeline 的顺序很重要：protocol 不同就不进入 score comparison；没有 2×2 arm 就不归因 auxiliary；没有 intervention 就不声称 causal dynamics；没有 runtime/ownership artifact 就不升级个人贡献。

UWM / WAM 项目挂钩与真实性边界

当前可以安全讲什么

项目	安全口径	仍缺证据
UWM/LIBERO	“我把 fixed-init 与 reset-style 分开，并通过 source audit 发现它们都不能直接称为 UWM Table II setting。”	checkpoint/config/stats SHA、250 episode records、exact init/horizon/chunk、OOD env patch
Cosmos/WAM	“我把 action/world/value component fit、direct policy 与 planning selection 分开，知道 auxiliary 收益需要 action-only/+world/+value/all 消融。”	local checkpoint、split/metric tensors、24-task raw episodes、controller/fork manifest

项目	安全口径	仍缺证据
WorldGym/WM proxy	“它可作为 deployment 前筛选 proxy，但要用多 policy variants 验证 rank consistency。”	local WM checkpoint、scorer/API revision、sim/real paired validation
Sim2Real	“我会按 observation→control→dynamics→contact 做 paired trace。”	任何真实 hardware/calibration/latency/contact/rollout artifact

当前不能说什么

1. “我的 74.4% 已严格复现 UWM Table II。”——论文 OOD init 与项目 fixed-pruned-init 不等价。[5][6][14]
2. “92.8% 是正式 UWM benchmark。”——项目页缺 same-checkpoint protocol manifest，最多是 reset-style diagnostic claim。[5][14]
3. “84/118 是我的 WAM 训练结果。”——项目页把它描述为 official HF baseline 的本地 smoke，而且两条 episode 的清洗规则未由 raw records核验。[5][15]
4. “world/value loss 下降证明辅助目标提升 policy。”——没有 factorial arm 与 same-protocol online result，不能做因果归因。[3]
5. “LIBERO/RoboCasa success 说明完成 Sim2Real。”——simulation 与真实硬件 evidence 是不同层。[4]

90 秒项目防守稿

“我会先区分方法、协议和个人证据。对 UWM，论文 Table II 的五任务均值来自 OOD initialization、背景物体改造和三次 finetune seed；frozen repo 默认 rollout 是 random reset，而我现有材料记录的是 fixed-pruned-init，所以不能只因任务相同就叫 strict reproduction。我会把本地 setting 命名成独立 protocol，补 checkpoint、normalizer、init ID、chunk/horizon 与 250 条 episode record，只和完全相同 protocol 的 baseline 做 paired comparison。对 WAM，action/world/value 的离线 loss、direct-policy success 和 planning uplift 是三层证据；若要说 auxiliary 帮助 action，必须做 action-only、+world、+value、all 四臂消融，再配 action intervention、value calibration 和 same-init closed-loop trace。缺 artifact 的数字我只当 claim queue，不当个人已验证结果。”

十六题覆盖矩阵

ID	本页核心位置	已给出的机制	项目挂钩	状态
VLA-WM-001	World model boundary	probability contract、policy/simulator/reward/value 边界	UWM/Cosmos/WAM claim boundary	knowledge drafted / card unseen
VLA-WM-002	Variable contract	observation/state/latent/reward/continuation 与 POMDP	dataset/eval schema	knowledge drafted / card unseen
VLA-WM-003	Dynamics	deterministic/probabilistic、ensemble 与 uncertainty	world prediction risk	knowledge drafted / card unseen
VLA-WM-004	RSSM data flow	encoder、posterior/prior、decoder/reward/continue heads	DreamerV3 reimplementation boundary	knowledge drafted / card unseen

ID	本页核心位置	已给出的机制	项目挂钩	状态
VLA-WM-005	Rollout validity	one-step、compounding、horizon error	UWM/WAM open-loop proxy	knowledge drafted / card unseen
VLA-WM-006	Train/eval gap	teacher forcing、scheduled sampling、open-loop	data/rollout contract	knowledge drafted / card unseen
VLA-WM-007	Uncertainty	aleatoric/epistemic、risk-sensitive planning	calibration/OOD	knowledge drafted / card unseen
VLA-WM-008	Actionability	action condition、sensitivity、causal alignment	UWM/Cosmos world head	knowledge drafted / card unseen
VLA-WM-009	Usage boundary	auxiliary representation vs queryable simulator	WAM planning claim	knowledge drafted / card unseen
VLA-WM-010	Model bias	planner exploitation、value bias、recovery	UWM/WAM failure diagnosis	knowledge drafted / card unseen
VLA-WM-039	Strict Protocol	12-field tuple、paired effect、run/episode manifest	UWM three-protocol split	knowledge drafted / card unseen
VLA-WM-040	Metric Ladder	L0–L5、compounding、rank validation	WorldGym/UWM/WAM	knowledge drafted / card unseen
VLA-WM-041	Offline→Online Ablation	2×2 factorial、interaction、断点矩阵	WAM/UWM auxiliary	knowledge drafted / card unseen
VLA-WM-042	Consistency Tests	action sensitivity、causal alignment、physical violation	UWM/Cosmos world head	knowledge drafted / card unseen
VLA-WM-043	Failure Trace	first divergence、taxonomy、data routing	LIBERO/RoboCasa 0-success	knowledge drafted / card unseen
VLA-WM-044	Evidence Audit	five-layer chain、labels、claim ledger	五项目 truth boundary	knowledge drafted / card unseen

知识与题卡覆盖不等于面试掌握。本批 16 个知识节点的上游关键事实已完成对应 source review，16 张题卡均为 unseen、尚未真实口述；因此不能标记 drilled 或 mastered，整体 review 仍为 WARN。

Bad Case 与最小检查清单

Bad case	最小检查	需要避免的解释
task 名相同但结果差 4.6pp	完整 $\mathcal{P}$ diff	直接归因 checkpoint质量
total loss 降、success 不变	component loss + factorial arms	“多任务一般有用”
video好看、action无效	action swap + cloned-state alignment	“画面真实所以 dynamics正确”
value MAE低、N 越大越差	calibration + N/selector sweep	“采样不够多”
fixed-init高、random-reset低/反之	init distribution + same checkpoint	选更高分作为正式成绩
0-success	protocol→action→controller first divergence	先怪 world-model backbone
两条 episode被删除	raw records + predeclared invalid rule	事后清洗仍称官方分数
simulator成功	real paired trials + safety trace	“已完成 Sim2Real”

后续工作

1. 已完成 `VLA-WM-001..010` foundations 与 `VLA-WM-039..044` evaluation 的知识页和 16 张题卡；全部 `unseen`。
2. 下一步优先真实口述或补 UWM/WAM raw artifacts；不生成 `WM-011..038`。
3. 若找到用户的 UWM/WAM run bundle，优先建立 checkpoint/config/stats/episode manifest，不先改强回答。
4. 本增量在渲染与静态校验后停止；后续仅在新的 source-ready 小批被明确选择后继续，不一次性生成剩余 28 个知识节点。

参考文献（References）

1. VOL 04 Source Audit A：World Model Foundations，World Models/PlaNet/Dreamer、one-step/open-loop/closed-loop 与 metric boundary；canonical Markdown。
2. VOL 04 Source Audit B：UWM Video / Action Diffusion，frozen UWM model、metric、normalizer、wrapper 与 reset-style 边界；canonical Markdown。
3. VOL 04 Source Audit C：Cosmos Policy / WAM，action/future/value routing、best-of-N、offline/online 与 eval contract；canonical Markdown。
4. VOL 04 Source Audit D：Sim2Real，六层 reality gap、paired test、failure attribution 与真实性边界；canonical Markdown。
5. VOL 04 Source Audit E：Benchmark / Evidence，12-field protocol、UWM/Cosmos/RoboCasa/WorldGym 可比性与 local manifest；canonical Markdown。
6. Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets”, arXiv `2504.02792v3`, Table II 与 Appendix A。
7. WEIRDLabUW, `unified-world-model` frozen commit `b7d4067c8859775e5b08f681e76d538015cdd445`: eval_robomimic.py, wrappers.py；frozen tree 未发现 LICENSE。
8. Lifelong Robot Learning, `LIBERO` frozen commit `8f1084e3132a39270c3a13ebe37270a43ece2a01`: metric.py, eval config, MIT。
9. Kim et al., “Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning”, arXiv `2601.16163v1`, HTML。
10. NVIDIA, `cosmos-policy` frozen commit `18a2accadf4e7a3531e56754102af5a24d2316da`: LIBERO runner, RoboCasa runner, Apache-2.0 code。
11. Quevedo et al., “WorldGym: World Model as An Environment for Policy Evaluation”, arXiv `2506.00613v3`, arXiv。
12. `world-model-eval/world-model-eval` frozen commit `709665b29f5c95124e7dbc05b0d4d47cd353de97`: utils.py, world_model.py；frozen tree 未发现 LICENSE。
13. MuJoCo official docs, Modeling / XML Reference；robosuite 1.5 Controllers，accessed 2026-07-16。

14. 本地待核验材料：UWM / LIBERO 项目页。该页的 74.4%/92.8% 不替代 raw run artifacts。
15. 本地待核验材料：Cosmos / WAM 项目页。该页的 MSE、84/118 与 0-success narrative 不替代 checkpoint/episode evidence。
16. VOL 04 Batch 1 独立事实复核；canonical Markdown，状态：`review`。
17. VOL 04 Foundations 独立事实复核；canonical Markdown，状态：`review`。

Part 2 / 12 · `interview_materials/vla/questions/vol_04_world_model_sim2real.html` · SHA-256
`23F6B5AC581229CD7A2210296268FD8669A841588ADAE0E5F0E2AF3A38C86636`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：
`23f6b5ac581229cd7a2210296268fd8669a841588adae0e5f0e2af3a38c86636`；生成自：
`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_04_world_model_sim2real.md`

VOL 04 World Model / Sim2Real：核心面试题卡（Foundations + Evaluation）

状态：`drafted / aggregate_cards_44_of_44 / WARN` 更新日期：2026-07-19 基础页题卡：`VLA-WM-001..010`、`VLA-WM-039..044`（16 / 44） 独立原子题卡：`VLA-WM-011..038`（28 / 44） VOL 04 aggregate cards：44 / 44；练习状态全部 `unseen` 难度：L1 1 题 / L2 7 题 / L3 8 题 优先级：P0 11 题 / P1 5 题 练习状态：全部 `unseen` 使用方式：先闭卷说 30 秒版本，再展开机制与诊断；未真实口述前不得改为 pass/strong。 深度来源：VOL 04 Foundations + Evaluation 知识页；本页不替代 UWM/WAM 的 checkpoint、config、normalizer 或 episode artifacts。

A. World Model 基础与范式

▼ `L1` `P0` VLA-WM-001 · 什么是 world model？它与 policy、environment simulator、reward/value model 分别是什么关系？

面试意图：检查你能否先划清模块职责，再讨论 architecture 或项目收益。

30 秒回答：world model 是给定历史与候选动作、预测未来 observation/state/latent、reward 或 continuation 的条件模型。policy 选动作，真实 environment/simulator 产生 transition，reward 给即时信号，value 估计长期回报；world model 本身不等于规划器。

机制展开（约 2 分钟）：先报概率合同 `history + action + task -> future variables`。World Models 用 VAE+MDN-RNN 学 latent future，PlaNet 用 latent dynamics 在线规划，Dreamer 从 posterior belief 出发用 prior imagination 训练 actor/critic。一个视频生成器只有在动作合同、horizon、reward/termination 或 decision utility 可验证时才承担 world-model 角色；否则它只是视觉预测器。learned model 也不等于环境本体。

诊断 / 消融：逐项问推理时是否查询 future、由谁选 action、reward/value 是否独立、预测是否进入 controller；用 no-world、auxiliary-world、planning-world 三组合同对照。

项目挂钩：讲 WAM/UWM 时先说明 world head 是训练辅助还是 rollout-time query；上游结构和项目页数字都不是个人已验证结果。

追问树：没有像素 decoder 还能叫 world model 吗？reward model 算 world model 吗？world model 与 simulator 的关键差别是什么？

危险答案：把 world model 等同于“能生成视频”，或说它天然包含 policy、value 和 controller。

来源：[1][2][17][18][19][20]　练习状态：unseen

▼ L2　P0　**VLA-WM-002 · world model 应建模 observation、state、latent、reward、termination 中哪些变量？**

面试意图：判断你是否会按用途设计 I/O，而不是机械堆齐所有 head。

30 秒回答：没有固定全家桶。感知侧输入 observation；高维或部分可观测任务常用 latent/belief；planning/RL 通常需要 reward 与 continuation。privileged simulator state 只可作监督或诊断，不能泄漏到部署输入；每个字段还要冻结时序、mask、horizon 和终止语义。

机制展开（约 2 分钟）：POMDP 中真实 state 转移、observation 由 state 产生；模型可用 encoder 与 recurrent belief 近似 state。decoder 保留可重建信息，reward head 给一步信号，continuation 决定 imagined return 是否跨过 terminal。纯 auxiliary model 可只学 latent future；queryable simulator 则必须输出 planner 真正需要的 future/reward/termination。privileged simulator state 只能在明确 flag 下作 target/probe。

诊断 / 消融：做字段级 ablation：去 decoder、reward、continuation 或 privileged target；分别报告 representation probe、open-loop error、return 与 closed-loop success。

项目挂钩：核对 UWM/WAM 数据中 camera、proprio、action、future/value slot 与 mask；缺 local schema 时不猜哪些字段真正训练。

追问树：termination 与 truncation 为什么不能合并？latent 是否必须能解码图像？privileged state 怎样避免泄漏？

危险答案：说 world model 必须同时预测像素、state、reward、value 和 done，或把 simulator state 当部署输入。

来源：[1][2][17][20]　练习状态：unseen

▼ L2　P1　**VLA-WM-003 · deterministic dynamics 与 probabilistic dynamics 各自能表达什么不确定性？**

面试意图：检查你能否区分点预测、多未来、数据噪声与模型知识不足。

30 秒回答：deterministic dynamics 给点估计，稳定但会平均多未来；probabilistic dynamics 输出分布，可表达 aleatoric 多模态。epistemic 来自覆盖不足，通常需 ensemble 或 posterior/disagreement；单个随机模型不自动代表 epistemic uncertainty。

机制展开（约 2 分钟）：deterministic 常以 MSE 拟合条件均值，适合低噪声 state 和短 horizon；Gaussian/mixture/categorical latent/diffusion 可表示多未来，但要冻结采样 seed、temperature 与

likelihood/denoise 合同。ensemble 的成员差异可作为 epistemic proxy，但若成员训练高度相关或未校准，disagreement 也不等于真实风险。选择取决于是否存在遮挡、接触、多解和 planning 风险。

诊断 / 消融：比较 deterministic、single stochastic、ensemble 的 NLL/coverage、horizon error、calibration 与 risk-sensitive planning success，不能只看 sample 多样性。

项目挂钩：UWM/video diffusion 的采样随机性最多先证明多样性；没有 ensemble/calibration 和环境对照，不能称本地 epistemic risk 已解决。

追问树：为什么 MSE 会模糊多未来？aleatoric 能否靠更多数据消除？ensemble disagreement 什么时候失真？

危险答案：说 deterministic 模型没有任何不确定性，或把 diffusion 的 sampling noise 直接称为 epistemic uncertainty。

来源：[1][2][17][19][20]　**练习状态**：unseen

▼ `L3` `P0` VLA-WM-004 · latent world model 的 encoder、transition、decoder、reward 与 continuation head 如何连接?

面试意图：考查你能否把 RSSM 的 filtering、imagination、loss 与 actor/critic 数据流连起来。

30 秒回答：encoder 编码 observation；posterior 用当前观测更新 latent belief，prior 仅靠历史 latent 和 action 想象未来。decoder、reward、continuation head 从 belief 预测观测、回报与是否继续；训练对齐 prior/posterior，规划时不能读取未来观测。

机制展开（约 2 分钟）：DreamerV3 风格 RSSM 由 deterministic state `h` 与 stochastic state `z` 构成：`observe` 做 posterior filtering，`imagine` 做 action-conditioned prior rollout，dyn/rep KL 用不同 stop-gradient 方向，free-nats 防止过度压 KL。world heads 分别有 target、scale、mask；actor/critic 从 replay posterior start 出发在 prior 中 rollout reward/continue。danijar repo 是 reimplementation，不是论文官方代码。

诊断 / 消融：检查 posterior-vs-prior gap、KL/free-nats、各 head loss、posterior collapse；做 future-observation leakage test 和 prior-only horizon rollout。

项目挂钩：项目若声称使用 RSSM，必须给 local module/config/checkpoint/tensor trace；不能用 Dreamer upstream code 代替本人实现证据。

追问树：posterior/prior 为什么分开？continuation 怎样影响 return？decoder 去掉会怎样？PlaNet overshooting 能否直接归给 DreamerV3?

危险答案：让 imagination 读取未来 observation，或称 danijar/dreamerv3 为 Google/DeepMind 官方代码。

来源：[1][2][17][20][21]　**练习状态**：unseen

▼ L2 P0 VLA-WM-005 · one-step prediction 好为什么不代表 multi-step rollout 可靠?

面试意图：检查你是否理解 teacher-forced 分布与自回归状态漂移。

30 秒回答：one-step 每次站在真实 history 上；open-loop 从第二步开始吃自己的预测。局部误差会被 dynamics sensitivity 反复传播，rollout 还会进入训练外 state；背景占优、action ignoring 或 continuation/contact 漏建模都可能让平均 MSE 很低但长时轨迹失真。

机制展开（约 2 分钟）：把第 j 步局部误差记为 `e_j`、局部放大因子记为 `L`，h-step error 会累积近似加权项；当 L>1 或 state OOD 时增长更快。pixel MSE 可能被静态背景主导，latent/reward loss 也只证明对应 target fit。正确评估要画 teacher-forced 与 autoregressive `error_by_horizon`，固定 action/seed/reset，并配 task-object pose、contact、action sensitivity 和 continuation error。

诊断 / 消融：做 horizon 1/4/8/... sweep、真实-history vs predicted-history 对照、action swap、对象区域/状态指标与 terminal audit。

项目挂钩：UWM/WAM 离线 future MSE 下降只能说相应 split/horizon 拟合改善，不能直接升级为 LIBERO success。

追问树：为什么背景会掩盖错误？L 是什么直觉量？open-loop 好是否就等于 closed-loop 好？

危险答案：只报一个平均 one-step MSE，就声称 world model 可做长时 planning。

来源：[1][2][17][19][20]　练习状态：unseen

▼ L3 P1 VLA-WM-006 · teacher forcing、open-loop rollout 与 scheduled sampling 的分布差异是什么?

面试意图：判断你能否从训练输入来源解释 exposure bias，而不是只背术语。

30 秒回答：teacher forcing 始终喂真实前一步，训练稳定但模型没见过自己的错误状态；open-loop 始终回灌预测，用来暴露 compounding；scheduled sampling 按 schedule 混用真实/预测输入，试图缩小 gap，但会引入新的训练分布和目标不一致，不能替代 open/closed-loop 评测。

机制展开（约 2 分钟）：scheduled sampling 可用 Bernoulli mask 决定每步取 ground truth 还是 model prediction，必须记录 teacher probability、stop-gradient、padding 与 episode boundary。它可能让模型适应有限预测噪声，也可能破坏序列目标的一致性；若 action、termination 或 schema 本身错，混入预测输入只会适应错误。评测要在同 start/action/seed 下分别跑 teacher-forced、autoregressive 和 closed loop。

诊断 / 消融：对 schedule、prediction detach 与 rollout horizon 做 sweep；分报 one-step、open-loop、closed-loop，不把三种 MSE 合并。

项目挂钩：若项目只保存 teacher-forced future loss，就不能宣称训练已覆盖 rollout-state distribution；需要 config 和真实 open-loop trace。

追问树：scheduled sampling 为什么有 bias？预测输入是否 detach？closed loop 与 open loop 的 observation 来源有何不同？

危险答案：说 scheduled sampling 一定消除 exposure bias，或把 teacher-forced validation 当 autoregressive rollout。

来源：[1][2][17][19]　练习状态：unseen

▼ L3 P1 **VLA-WM-007 · aleatoric 与 epistemic uncertainty 如何区分，分别怎样影响 planning？**

面试意图：考查你能否把 uncertainty 分类落实为校准、拒答和风险敏感决策。

30 秒回答：aleatoric 是同一条件下环境本身有多种结果，更多同分布数据也未必消失；epistemic 是模型没覆盖或不知道，补数据/ensemble 可能降低。planning 中前者需要对多 outcome 做期望或约束，后者应触发 pessimism、abstention、fallback 或补采数据；两者都必须校准。

机制展开（约 2 分钟）：单模型分布内部 variance 可近似 aleatoric，不同 ensemble 成员预测均值的 variance 可近似 epistemic，但这是估计框架。风险目标可写成 predicted return 减 uncertainty penalty；beta 决定保守度。必须验证 coverage、reliability、OOD detection 与 success/constraint trace。高 disagreement 可能来自成员不稳定，低 disagreement 也可能是共同过拟合，不能把数值直接当安全保证。

诊断 / 消融：做 in-distribution/OOD calibration、ensemble size、risk beta、abstention threshold 与 fallback 对照，同时报告成功率和 constraint violation。

项目挂钩：当前 UWM/WAM 没有 local ensemble/calibration artifact；安全说法是会这样设计，而不是已解决不确定性。

追问树：diffusion 多样性属于哪类？更多数据对两类 uncertainty 的作用？pessimism 过强会怎样？

危险答案：把 sample variance 当完整 epistemic uncertainty，或声称 uncertainty penalty 天然保证安全。

来源：[1][2][17][19][20]　练习状态：unseen

▼ L2 P0 **VLA-WM-008 · video generation model 何时可以叫 actionable world model，何时只是视觉预测器？**

面试意图：检查你能否从“画面逼真”推进到 action intervention 和 decision utility。

30 秒回答：至少过四关：候选动作以真实 axis/unit/time 进入模型；固定起点换动作时 future 有差异；差异方向与 simulator/真实 intervention 对齐；这些 future 能改善候选筛选或失败发现。只有视频质量、FVD/PSNR 或文本动作条件，最多说明视觉预测，不证明 actionable dynamics。

机制展开（约 2 分钟）：先做 action swap、magnitude、gripper、noop、order tests，复用 observation 与 sampler seed；再从 cloned state 执行动作，比 object pose、joint/contact delta。sensitivity 只证明模型有反应，alignment 才说明方向对；短时 alignment 也不证明长 horizon。

decision utility 还要比较 proxy rank、planning selector 与 same-protocol closed-loop，同时记录 latency 和 failure cases。

诊断 / 消融：action-free、shuffled-action、correct-action 三组；pixel/feature/pose/contact 与 closed-loop 指标分层报告。

项目挂钩：UWM/Cosmos 是上游 action-conditioned 例子；没有 local checkpoint 和 intervention trace，不能说本人已验证物理正确。

追问树：action sensitivity 高为什么仍可能错？文本条件算 action 吗？FVD 能证明什么？

危险答案：说“生成视频很真实，所以模型理解动作和物理”。

来源：[1][2][3][4][17]　**练习状态**：unseen

▼ `L2` `P1` **VLA-WM-009 · world model 作为 policy auxiliary representation 与作为可查询 simulator 有何区别？**

面试意图：判断你是否会区分训练辅助目标与 test-time model-based planning。

30 秒回答：auxiliary 训练时加 future/world loss，推理可不用 world head，收益靠公平的 action-only vs +world 消融。queryable simulator 推理时 rollout 候选的 future/reward/termination 后选择，更依赖 OOD action 覆盖、校准与 latency。

机制展开（约 2 分钟）：auxiliary 目标可改善共享表示，也可能因 gradient conflict、shortcut 或额外数据混杂而负迁移；必须记录 routing、loss mask、weight 与 gradient scope。queryable 模式要记录 proposal N、world/value checkpoint、rollout/denoise steps、selector、replan 和 cost。训练时出现 world loss 不等于 model-based control；planning uplift 也不能只归因 world head，除非 value、N 与 protocol 有消融。

诊断 / 消融：action-only、+world、planning-world 三臂；再分 world/value/N/selector 与 latency，比较同 init closed loop。

项目挂钩：WAM 项目叙述先说明现有 run 是 multi-task pretraining、direct policy 还是 planning；缺 runtime trace 时不混写。

追问树：无 decoder 的 latent auxiliary 算 world model 吗？为什么 planning 更怕 OOD action？额外视频数据怎样控制？

危险答案：看到 future loss 就说系统在推理时用 world model 规划。

来源：[1][2][4][17]　**练习状态**：unseen

▼ `L3` `P0` **VLA-WM-010 · model bias 与 rollout compounding error 如何导致“想象中很好、真实执行失败”？**

面试意图：考查你能否解释 planner exploitation，并设计候选级别的归因实验。

30 秒回答：transition 偏差随 horizon 累积并把 rollout 推入 OOD；planner 又会选择 world/value 最乐观的候选，N 越大越易命中幻觉、物理漏洞或 value 高估。再叠加 normalizer、controller、contact 与 recovery mismatch，imagined return 高不等于真实成功。

机制展开（约 2 分钟）：把候选固定后保存 imagined future、value、uncertainty，并从 cloned env state 执行 selected 与 random/median control。按 horizon 比 transition delta、constraint violation 和 rank；做 N、selector、horizon sweep。如果 selected candidate 的 model score 升而真实 return 降，是 optimizer's curse 信号。再查 action transform/controller tracking、terminal/contact 和首次偏离后的 recovery，避免把接口失败全归给 backbone。

诊断 / 消融：random/median/top selector、N/horizon、oracle-state/value、model bypass 与 controller replay；记录 first divergence。

项目挂钩：WAM/UWM 的 0-success 或高 proxy 低 success 只能先按这条链诊断；无 raw episodes 时不声称已定位 root cause。

追问树：为什么 N 增大可能更差？怎样区分 transition bias 与 value bias？receding horizon 能完全解决吗？

危险答案：认为更多候选和更长 rollout 必然更好，或只凭最后失败帧归因 world-model backbone。

来源：[1][2][4][16][17][19][20]　**练习状态**：unseen

E. World Model 评估与失败归因

▼ `L3` `P0` **VLA-WM-039 · UWM / LIBERO 的 checkpoint、normalizer、init、warmup、horizon 与 rollout 数怎样构成 strict protocol?**

面试意图：判断你能否把“严格评测”落实成可重放合同，而不是只对齐 task 名和 rollout 数。

30 秒回答：strict protocol 是模型、初态和执行判定的完整合同。我会冻结 task/init、warmup、观测预处理、action/normalizer、H/K、controller/Hz、horizon、success predicate、随机性、trial 聚合，以及 checkpoint/code/runtime；任何字段不一致都先视为不同协议。

机制展开（约 2 分钟）：把协议写成 $\mathcal{P} = (S, I, W, O, A, K, C, H, Y, R, N, M)$，每个 run 保存 checkpoint/config/stats SHA，每个 episode 保存 task、init、seed、success、step、异常与 trace hash。比较 A/B 时复用同一 `pair_id=(task, init, seed)`，从 immutable episode records 聚合结果。UWM 论文 Table II OOD init、官方仓库 random-reset runner 与本地 fixed-pruned-init claim 分属三个 setting，不能因任务和 50 rollouts 相同就混名。

诊断 / 消融：先做 protocol manifest diff；若任一字段未知就阻断分数比较。补齐后只改一个因素并做 same-init paired rerun，同时保留 invalid episode 与预先声明的清洗规则。

项目挂钩：现阶段可讲“我识别并拆开了三种协议”；74.4%/92.8% 仍缺 checkpoint、stats、init、chunk/horizon 和逐 episode 记录，只能标为 `claim-only`。

追问树：为什么 H 与 K 都要冻结？warmup 是否计入 horizon？macro/micro average 怎样选？paired episode 为什么能减少 init 难度噪声？

危险答案：说“用了相同 LIBERO 任务、50 次 rollout，所以就是 strict reproduction”。

来源：[1][3][6][7][8][9][14]　练习状态：unseen

▼ `L2` `P0` **VLA-WM-040 · world model open-loop prediction metric 与 closed-loop policy success 怎样建立但不夸大关联?**

面试意图：检查你能否分清 proxy 的证据层级，并设计它与真实行为指标的验证关系。

30 秒回答：分层报告 component、one-step、open-loop、WM proxy、sim success 和 real success。低层变好只证明对应 target/horizon；要在多个 policy variant 上检验 proxy 与同协议闭环的排名和不确定性，不能用单点替代真实评测。

机制展开（约 2 分钟）：one-step error 在 rollout 中会被局部 dynamics sensitivity 与 OOD state 逐步放大，因此至少画 `error_by_horizon` 并标明 teacher-forced 还是 autoregressive。proxy 验证需要预先定义多个真实不同的 checkpoint，在同一 task distribution 上得到 proxy q_j 与 closed-loop success p_j，检查 rank consistency、seed/CI 和 bad cases。即使排序稳定，也只支持已测 policy/task/scorer 版本，不能声称数值等价或替代 sim/real。

诊断 / 消融：做 horizon sweep、action-conditioned counterfactual 和多 checkpoint rank test；分别报告 action/future/value component metric、proxy、sim 与 real，不用 total loss 掩盖断点。

项目挂钩：WorldGym 可作为 deployment 前筛选框架；UWM/WAM 的离线 MSE 或生成质量不能自动升级为 LIBERO success，更不能写成 Sim2Real。

追问树：为什么单个 checkpoint 不能谈相关性？Spearman 高说明什么、不说明什么？image MSE 更低但 success 更差有哪些原因?

危险答案：说“open-loop 指标越好，closed-loop success 必然越高”，或用 simulator success 证明真实机器人泛化。

来源：[1][2][3][4][6][12][13]　练习状态：unseen

▼ `L3` `P0` **VLA-WM-041 · offline future/action/value loss 下降但 online success 不变时，如何设计消融?**

面试意图：考查你是否能把多目标收益拆成可归因实验，而不是从 total loss 直接推断策略提升。

30 秒回答：做 action-only、+world、+value、all 的 2×2 消融，四臂共享数据、初始化、预算和协议，并在相同 init 上 paired eval。分报 component loss、action sensitivity、value calibration、闭环结果与开销，定位 offline→online 断点及 interaction。

机制展开（约 2 分钟）：对结果 $Y_{w,v}$ 分解 world 主效应、value 主效应和 $world \times value$ interaction；A11 更好不代表两个 auxiliary 单独都有效。若 +world 额外使用更多视频，需再控制

data amount。action MSE 降但 success 不变先查 norm/axis 与 rollout-state shift；value MSE 降但 best-of-N 变差查 OOD overestimate；A10 好、A11 差则查 gradient conflict、label 与 weight。每个 cell 要多 train seed，并在同一 episode pair 上报告 online uncertainty。

诊断 / 消融：补 gradient cosine/weight sweep、expert-state 对 rollout-state error、random/median/top selector 与 planning-N sweep；同时 diff run manifest，避免 checkpoint/protocol 漂移冒充目标冲突。

项目挂钩：WAM 的 MSE、84/118 和 0-success 目前缺 local checkpoint 与 raw episodes；只能用它们说明要做哪组消融，不能据此声称 auxiliary 已带来收益。

追问树：为什么 all 更好不能归因两个 auxiliary？如何控制额外数据量？value optimizer's curse 是什么？四臂预算怎样对齐？

危险答案：只比较 action-only 与 all，看到 total loss 下降就说 world/value 对 policy 都有帮助。

来源：[1][3][4][6][10][11][15]　**练习状态**：unseen

▼ L3 P1 **VLA-WM-042 · 如何检测 hallucination、physical inconsistency、action insensitivity 与 causal inconsistency?**

面试意图：判断你能否从“视频看起来合理”推进到可干预、可度量的 world-model 一致性测试。

30 秒回答：hallucination 是生成无依据状态；physical inconsistency 是违反几何或接触约束；action insensitivity 是换动作而 future 几乎不变；causal inconsistency 是会变但方向错误。固定起点和噪声做反事实，再从 cloned simulator state 执行动作并对齐环境变化方向。

机制展开（约 2 分钟）：action swap、magnitude sweep、gripper 正反、noop 和时间置换先测模型是否使用 action；再从同一 simulator state 执行 baseline/candidate action，比较预测与环境中的 object pose、joint/contact delta，而不是只比像素。按 horizon 统计 error 与 penetration、悬浮、关节越界、对象守恒等 violation。高 sensitivity 也可能是噪声放大，只有与环境 intervention 方向一致才支持局部 causal alignment，仍不证明长时真实物理。

诊断 / 消融：每对反事实复用 observation、start state 与 sampler seed；按 axis、magnitude、gripper、order 和 horizon 分桶，并区分视频/VLM proxy 与 simulator-state ground truth。

项目挂钩：可为 UWM/Cosmos world head设计这套 test suite；没有 local checkpoint 和执行 trace 前，不能声称已验证物理正确或真实机器人因果性。

追问树：action sensitivity 高为什么仍可能错？只有视频时怎样标记物理 judge？为什么要 clone state？如何检测 gripper sign 反转？

危险答案：用 PSNR、FVD 或“视频看起来逼真”直接证明模型理解动作和物理。

来源：[1][2][3][4][6][7][10]　**练习状态**：unseen

▼ L2 P0 **VLA-WM-043 · 失败轨迹怎样按 perception、dynamics、contact、planning、controller 和 recovery 分层回写数据?**

面试意图：检查你是否会定位第一处可观测断点，并把失败路由到正确的数据、代码、目标或评测 owner。

30 秒回答：0-success 时别猜 backbone；按 protocol/checkpoint→observation→action/controller→dynamics/contact→planning/value→recovery 找断点。接口 bug 修代码与单测，覆盖缺口补数据，world/contact/value/recovery 各回写自己的目标。

机制展开（约 2 分钟）：每个 episode 对齐 raw/model observation、normalized/raw/env action、queue/controller state、predicted/real pose、contact、candidate/value 和 success trace。先排 protocol、checkpoint 与 normalizer，再找最早时间戳：flip/时序错、axis/gripper 错、stale chunk、跟踪误差、接触丢失、value 选错或恢复失败。只路由证据确认的 family；action contract 错误不能当新标签训练，recovery 缺口应采 first-deviation 后的 corrective trajectory。

诊断 / 消融：做 manifest diff、image/timestamp replay、action round-trip、expert action replay、K/latency sweep、cloned-state intervention、contact trace、N/selector/calibration sweep与 predicate audit。

项目挂钩：WAM 的 0-success 或 UWM 单条失败先当待分类 episode；没有完整 trace 时只描述诊断计划，不把它包装成本人已修复的模型问题。

追问树：为什么先查 protocol? first divergence 与最终 failure label 有何区别? 哪些失败该补数据、哪些只该修 harness?

危险答案：看到最后抓取失败就归因 perception/backbone，或把所有失败轨迹无差别加入训练集。

来源：[1][4][5][6][9][11]　练习状态：unseen

▼ L3 P0 **VLA-WM-044 · 如何从 paper/repo/checkpoint/eval harness 核对 world model 的真实训练和 rollout 合同?**

面试意图：考查你能否建立完整证据链，并把上游方法、运行结果与个人贡献分开。

30 秒回答：把 claim 拆成方法、配置、指标、协议和 ownership，再核对 paper→repo→checkpoint/assets→runtime→local diff。每层注明支持与不支持什么；revision、artifact 或 protocol 有缺口就降为 claim-only，并按最弱证据原子表述。

机制展开（约 2 分钟）：paper 证明方法与论文 setting，不证明当前 default；repo 证明某 commit 的路径，不证明论文 run；checkpoint/stats 证明资产存在，不证明能加载或成功；runtime 需要 exact controller/protocol 与 episode records；ownership 还需 local diff、design note 和命令/日志。ledger 含 claim、source、supports、does-not-support、protocol、artifact SHA、ownership、verdict，并区分 primary、official-code、local-evidence、inference、unverified。

诊断 / 消融：逐原子寻找 evidence chain；对缺口提出最小补件，如 env patch、stats SHA、load log、paired episode JSONL 或 local diff。只比较协议相同的 run，并对数字、因果和个人贡献分别给 verdict。

项目挂钩：74.4%/92.8%、WAM MSE/84/118 与 upstream UWM/Cosmos 结构当前不能串成“我的严格复现/创新”；安全说法是已完成 source audit，raw artifacts 仍待补。

追问树：repo default 能否证明论文实验？checkpoint 能加载是否证明分数？团队成果怎样标个人 ownership？哪类证据可把 claim-only 升级？

危险答案：把论文数字、官方 baseline、本地 smoke test 和个人贡献合成一条强叙事。

来源：[1][2][3][4][5][6][7][10][14][15]　**练习状态**：unseen

参考来源

本页是 VOL 04 的基础与评估入口；WM-011..038 的长知识页和折叠题卡位于同目录的独立原子文件，主入口与总索引按 aggregate 计数，不重复粘贴正文。

1. VOL 04 Batch 1 完整知识页；canonical Markdown。
2. Source Audit A：World Model Foundations。
3. Source Audit B：UWM Video / Action Diffusion。
4. Source Audit C：Cosmos Policy / WAM。
5. Source Audit D：Sim2Real。
6. Source Audit E：Benchmark / Evidence。
7. Zhu et al., Unified World Models, arXiv `2504.02792v3`。
8. WEIRDLabUW, `unified-world-model@b7d4067`，frozen tree 未发现 LICENSE。
9. Lifelong Robot Learning, `LIBERO@8f1084e`，MIT。
10. Kim et al., Cosmos Policy, arXiv `2601.16163v1`。
11. NVIDIA, `cosmos-policy@18a2acc`，Apache-2.0 code。
12. Quevedo et al., WorldGym, arXiv `2506.00613v3`。
13. `world-model-eval@709665b`，frozen tree 未发现 LICENSE。
14. 本地待核验材料：UWM / LIBERO 项目页；数字不替代 raw run artifacts。
15. 本地待核验材料：Cosmos / WAM 项目页；数字不替代 checkpoint 与 episode evidence。
16. VOL 04 Batch 1 独立事实复核；该记录只核对上游 frozen source，不替代本地 runtime artifacts 或真实口述。
17. VOL 04 Foundations 独立事实复核；该记录核对 World Models、PlaNet、DreamerV3 paper/repo 与实现边界。
18. Ha & Schmidhuber, World Models, 2018。
19. Hafner et al., PlaNet, 2019。

20. Hafner et al., DreamerV3, 2023/2025。

21. `danijar/dreamerv3@e3f0224`，MIT；README 明示为 reimplementation。

> Part 3 / 12 · `interview_materials/vla/sources/vol_04_batch1_independent_fact_review.html` · SHA-256 `115EED1C2C3D0D98ADC46ACDE2E8EFFE12F0305B2F7EA2A81C7FA3FFAC2503C1`

返回本卷目录

> Canonical Markdown SHA-256：`115eed1c2c3d0d98adc46acde2e8effe12f0305b2f7ea2a81c7fa3ffac2503c1`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_batch1_independent_fact_review.md`

VOL 04 Batch 1：独立事实复核记录（WM-039..044）

> 状态：`reviewed_slice / WARN` 复核日期：2026-07-16 证据截止：2026-07-16 复核范围：`VLA-WM-039..044` 的知识页、6 张题卡，以及它们引用的 UWM、Cosmos Policy、WorldGym primary/official-code 来源。 结论先行：上游协议边界的关键事实已由独立 HTTP 读取的 frozen paper/repo 复核；本地 74.4%、92.8%、WAM 84/118 与 MSE 仍没有 raw artifacts，因此不升级为 verified personal evidence。

问题（Question）

本记录回答：六张题卡中的强事实是否能回到独立 primary/official-code 来源？哪些内容是论文/代码直接支持，哪些只是本仓库设计的分析框架，哪些仍被本地证据缺口阻断？

本次复核不是重新执行 UWM、Cosmos 或 WorldGym，也不是浏览器视觉 QA；它只审查 frozen source 的文字与代码路径，并把结果回写到 review JSON。

复核方法与证据等级

等级	本次含义	允许的面试口径
`primary-checked`	直接读取论文 HTML 的方法/实验原文	“论文报告/定义……”
`official-code-checked`	直接读取 frozen commit 的代码或 README	“该 commit 的路径/默认值……”
`framework`	本仓库提出的评估设计、统计或失败归因方案	“我会这样设计/诊断……”
`claim-only`	只在本地项目页、简历或口述中出现，没有 raw artifact	“项目材料记录为……待核验”
`blocked`	缺 checkpoint、normalizer、环境补丁、episode 或 ownership 证据	不升级为复现/个人成果

复核入口被冻结为：UWM paper `arXiv 2504.02792v3`、UWM repo `b7d4067c8859775e5b08f681e76d538015cdd445`、Cosmos Policy repo `18a2accadf4e7a3531e56754102af5a24d2316da`、WorldGym repo `709665b29f5c95124e7dbc05b0d4d47cd353de97`。所有链接均在参考文献中给出。

1. UWM Table II：关键协议事实已复核

UWM v3 HTML 的实验段落直接写明：每个方法在 multitask dataset 上预训练 100K gradient steps、下游任务 finetune 10K steps；每个 environment 使用 3 个 random seeds，并在 50 个不同 initializations 上评估；Table II 的均值跨 initializations，置信区间跨 random seeds。[1]

同一论文的 OOD 描述写明：为制造未见初态，将每个 object 的 initialization range 增加 `0.03`，并移除 background objects 造成视觉分布变化。[1] Table II 的 UWM average 为 `0.79`，所以题卡把它称为论文内部的 OOD protocol mean 是有 primary 支持的；它不是本地 checkpoint 的成绩。

这里有一个重要的统计边界：论文的 `±` 是跨 finetune seed 的区间语义，不应被题卡改写成一次 250 episode 的 binomial confidence interval。当前题卡没有把它写成 binomial CI，保持正确。

2. UWM frozen runner：reset-style 与 Table II 不等价

在 frozen `experiments/uwm/eval_robomimic.py` 中，rollout loop 明确执行 `env.seed(e)` 后 `obs = env.reset()`，然后直接进入 model sampling；`config.rollout_length` 传给环境 wrapper。该入口没有 Table II OOD object-range/background-removal patch，也没有显式 warmup action。[2]

`environments/robomimic/wrappers.py` 进一步显示：`reset()` 调基础环境并用同一 observation 填充 history buffer；`step(actions)` 按 action sequence 逐步执行，并以 `_max_episode_length` 约束长度；`reset_to_state` 虽然存在，但不是上述 eval entrypoint 的默认路径。[3]

因此 `VLA-WM-039` 中“官方仓库 random-reset runner、不是 Table II”是 `official-code-checked`；“本地 fixed-pruned-init 74.4%”仍是 `claim-only`。两者不能因都使用 50 trials 就合并。

3. Cosmos Policy：代码默认值与本地 WAM 证据分离

Cosmos frozen LIBERO runner 的 config 注释和默认字段可直接核对：`chunk_size=16`、`num_open_loop_steps=16`、action denoising steps 为 5、future/value prediction 各为 1、`num_trials_per_task=50`；代码还明确检查训练 chunk 与 eval chunk 是否一致。[4]

这些是上游代码合同，不是本地 WAM 的训练结果。`VLA-WM-041` 只把 action-only / +world / +value / all 作为可归因的 factorial 设计；它没有声称论文已经提供这四臂消融，也没有把 WAM 的 84/118 当作已核验结果。

4. WorldGym：proxy scorer 的真实边界已复核

WorldGym frozen runner `run_openvla.py` 从每个 trial 的 `trial_png` 初始化 256×256 frame，调用 VLA 产生 action，再由 world model 生成视频；随后把生成视频交给 `predict()`。[5]

`utils.py` 的 `predict()` 将视频帧编码后发送给 `gpt-4o`，要求返回离散 `Final Score`，默认进行多次采样并解析投票；`aggregate_model_results()` 再把每个 trial score 聚合为均值与标准误。[6] 这直接支持“WorldGym 是 generated-video/VLM scorer proxy”的说法，但不支持它等价于 simulator success 或真实机器人 success。

5. 六张题卡逐题复核矩阵

题卡	独立核对结果	事实/框架边界	当前动作
`VLA-WM-039`	`PASS WITH WARNING`	UWM paper OOD、repo reset、Table-II gap 已核对；12-field tuple 与 paired effect 是本仓库 framework	保持题卡；74.4%/92.8% 继续 claim-only
`VLA-WM-040`	`PASS WITH WARNING`	WorldGym PNG→video→GPT-4o scorer 已核对；L0–L5、horizon curve、rank test 是分析框架	保持题卡；不把 proxy 当 sim/real
`VLA-WM-041`	`PASS WITH WARNING`	Cosmos action/future/value eval defaults 已核对；2×2 factorial 与 interaction 是设计建议	保持题卡；WAM 数字不升级
`VLA-WM-042`	`FRAMEWORK / SOURCE-CONSISTENT`	action swap、cloned-state causal alignment、physical violation 是测试设计，不是已执行结果	明确“需 runtime”
`VLA-WM-043`	`FRAMEWORK / SOURCE-CONSISTENT`	first-divergence taxonomy 与 routing 是诊断框架	没有 episode trace 不命名具体 root cause
`VLA-WM-044`	`PASS WITH WARNING`	paper→repo→checkpoint→runtime→ownership 是证据审计框架，source 层级已核对	缺 checkpoint/runtime/ownership 时保持 claim-only

6. 回写与未解决项

本次没有发现需要改写题卡核心事实的错误；修订重点是把“已核对的上游事实”和“本仓库设计的实验框架”分开记录。题卡仍全部 `unseen`，因为事实复核不等于候选人口述掌握。

仍然阻断强结论的证据包括：

1. 用户本地 UWM checkpoint、normalizer、config、OOD environment patch、init IDs 和逐 episode JSONL。
2. WAM 的 local checkpoint、action/world/value split、prediction/label artifacts、controller/fork manifest 与 raw episodes。
3. WorldGym scorer/API version、local world-model checkpoint 与 simulator/real paired validation。
4. 浏览器视觉 QA 与候选人的真实口述记录。

```
def phrase_claim_at_weakest_evidence(atoms, evidence_graph):
    # Split a sentence into method, metric, protocol and ownership atoms.
    verdicts = [evidence_graph.lookup(atom) for atom in atoms]

    # A missing checkpoint/runtime/protocol edge blocks a strong reproduction claim.
    if any(v in {"claim-only", "blocked"} for v in verdicts):
        return "按最弱证据逐原子表述，并保留待核验边界"

    # Only a continuous primary→code→runtime→ownership chain permits a strong claim.
    return "可以在对应 protocol 范围内给出已核验口径"
```

复核结论

本批 6 张题卡的关键上游事实现在有独立 primary/official-code 依据；统计、consistency、failure taxonomy 和 evidence chain 中明确标注为 framework 的部分没有被伪装成论文结论或本地实验结果。总体 verdict 仍为 `WARN`，原因是本地运行 artifacts、浏览器视觉 QA 和真实口述尚未完成。

参考文献（References）

1. Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets”, arXiv `2504.02792v3`，HTML。
2. WEIRDLabUW, `unified-world-model` frozen commit `b7d4067c8859775e5b08f681e76d538015cdd445`, `eval_robomimic.py`。
3. 同上 frozen commit，`wrappers.py`。
4. NVIDIA, `cosmos-policy` frozen commit `18a2accadf4e7a3531e56754102af5a24d2316da`, `run_libero_eval.py`。
5. `world-model-eval/world-model-eval` frozen commit `709665b29f5c95124e7dbc05b0d4d47cd353de97`, `run_openvla.py`。
6. 同上 frozen commit，`utils.py`。
7. 本仓库 VOL 04 Audit E，canonical Markdown。
8. 本仓库 VOL 04 Batch 1 知识页 与 6 张题卡。

Part 4 / 12 · `interview_materials/vla/sources/vol_04_benchmark_evidence_claim_to_protocol.html` · SHA-256 `58A5636E953F44C2AA27E3ACF65C621460186DAF3732B916EAC70E6FFAB2DE6A`

返回本卷目录

Canonical Markdown SHA-256：`58a5636e953f44c2aa27e3acf65c621460186daf3732b916eac70e6ffab2de6a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_benchmark_evidence_claim_to_protocol.md`

VOL 04 Source Audit E：Benchmark / Evidence Claim-to-Protocol

审计日期：2026-07-16 状态：drafted / primary_protocols_frozen / local_artifacts_pending / independent_review_pending 覆盖题目：VLA-WM-039..044 冻结对象：LIBERO、UWM、Cosmos Policy、RoboCasa、WorldGym 的 paper/repo/eval 入口与项目 claim 边界 核心规则：任务名相同不代表 protocol 相同；只有 protocol tuple 与 artifact manifest 同时闭合，成功率才允许横向比较。

1. 结论先行

1. **“strict”不是形容词，而是一份可重放合同。**最少要固定 task/suite、initial state、warmup、camera/preprocess、action schema/normalizer、chunk/replan、controller、horizon、success predicate、seed/determinism、trial count、checkpoint/runtime。[1][2]
2. **LIBERO frozen 官方代码的默认 lifelong eval 是 20 个 fixed `pruned_init`、5 次零动作 warmup、600 policy steps。**环境默认是 20 Hz、`OSC_POSE`、7D action、两路 128×128 RGB；这些是该 commit 的默认值，不是所有 LIBERO 论文的统一协议。[1]
3. **UWM Table II 的 0.79 不是上述默认 fixed-init protocol。**论文 v3 在五个 LIBERO-10 任务上扩大每个物体的初始化范围 0.03、移除背景物体，按 50 个 OOD initializations、3 个 finetune seeds 报告均值与区间。[2]
4. **frozen UWM repo 没有给出 Table II 的 OOD 环境改造入口。**仓库 `eval_robomimic.py` 使用 `env.seed(e) + env.reset()`，没有 `set_init_state`、没有 warmup，默认 rollout length 1000，并整段执行 model sample；`reset_to_state` 只出现在 inverse-dynamics ablation。[3]
5. **因此，当前项目页的 74.4% 只能暂记为“本地 fixed-init claim”，不能写成“与 UWM Table II strict aligned”。**项目材料写的是 `pruned_init + 5-step zero action + 50/task`；论文 Table II 写的是 OOD shifted initialization。二者任务相同，但 initial-state distribution 不同。[2][3][10]
6. **92.8% reset-style 只能作为 protocol sensitivity diagnostic。**当前 workspace 没有该结果的 checkpoint、episode summary、seed/init trace 或同 checkpoint paired run，不能把项目页数字升级为已核验事实。[10]
7. **Cosmos LIBERO 又是第三套协议。**frozen runner 使用默认 initial states 的前 50 个、10 次带闭夹爪的 dummy action、suite-specific 220–520 step horizon、chunk 16 / open-loop 16；官方

README 要求 seeds 195/196/197。[4]

8. **Cosmos RoboCasa 使用的是一个未在安装命令中 pin 的 fork。** `ROBOCASA.md` 安装 `moojink/robocasa-cosmos-policy`；2026-07-16 观察到 fork HEAD 为 v0.2.0，而官方 `robocasa/robocasa` frozen HEAD 是 v1.0.1。两者不能混成一个环境版本。[5][6][7]
9. **RoboCasa 的 84/118=71.19% 是“官方 HF checkpoint 的本地 harness smoke evidence”，不是官方 paper protocol，也不是本地训练收益。** 当前只在项目页出现 24 tasks × 5 episodes 与两条剔除；缺少逐 episode artifact，保持待核验。[10]
10. **WorldGym 的“success rate”是 world-model rollout 上的 VLM proxy。** frozen code 从真实初始帧生成视频，用 GPT-4o 五路投票打 0/0.5/1 分，默认 rollout length 40；它不能与 LIBERO/RoboCasa simulator predicate 或真实机器人成功率直接混分。[8][9]
11. **本轮只核验上游协议，不冒充本地复现。** 没有下载用户 checkpoint、normalizer、dataset statistics，也没有运行 GPU rollout；所有个人数字继续位于 evidence queue。
12. **Audit E 之后可以开放通用知识页，但项目数字仍受 VOL 07 证据门禁。** 知识答案可以讲 protocol 设计；个人回答必须区分“论文值、上游 runner、项目页 claim、已核验 artifact”。

2. 来源冻结与许可边界

编号	来源与冻结版本	本页采用的证据	不能由它证明
[1]	`Lifelong-Robot-Learning/LIBERO` @ `8f1084e3132a39270c3a13ebe37270a43ece2a01`，`master`，MIT	task suite、pruned init、default eval、environment/controller/camera/success wrapper	UWM/Cosmos 论文一定使用默认 config；用户本地环境版本
[2]	UWM arXiv `2504.02792v3`，Table II 与 Appendix A	五任务数值、50 OOD init、3 seeds、+0.03 init shift、移除背景物体	frozen repo 中存在 exact OOD eval code；用户 74.4% 与论文可比
[3]	`WEIRDLabUW/unified-world-model` @ `b7d4067c8859775e5b08f681e76d538015cdd445`，无 LICENSE 文件	reset-style runner、wrapper、shape、rollout defaults、inverse ablation	代码复用许可；Google Drive checkpoint revision；Table II runtime
[4]	`NVlabs/cosmos-policy` @ `18a2accadf4e7a3531e56754102af5a24d2316da`，Apache-2.0	LIBERO/RoboCasa eval CLI、init/warmup/chunk/horizon/success/seed	HF 权重商用许可；本地 baseline 或训练结果
[5]	`moojink/robocasa-cosmos-policy` observed HEAD `edd9a328b3ec98050f42d194c1419307a79c4d87`，`main`，MIT，package 0.2.0	Cosmos 安装目标在访问日的可观察语义	Cosmos paper/runtime 实际 fork commit；安装命令未 pin
[6]	`robocasa/robocasa` @ `b4684e6ee37d377cc392e98302a6b916d588b415`，`main`，MIT，package 1.0.1	当前官方 API、controller/eval helper、determinism test 的对照	Cosmos runner 与 v1.0.1 兼容；旧 fork 结果可直接迁移
[7]	Cosmos `ROBOCASA.md` / `LIBERO.md` frozen 于 [4]	官方命令、50 trials、3 seeds、runtime/hardware note	用户真正执行的 CLI、完整日志、paper 数值复现
[8]	WorldGym arXiv `2506.00613v3`	world model 作为 proxy environment、VLM reward 的论文主张	simulator/real success 等价；本地 checkpoint 可用
[9]	`world-model-eval/world-model-eval` @ `709665b29f5c95124e7dbc05b0d4d47cd353de97`，无 LICENSE	runner、action conversion、GPT-4o scorer、aggregate	代码复用许可；Google Drive checkpoint hash；外部 API 可重

编号	来源与冻结版本	本页采用的证据	不能由它证明
	文件		放
[10]	本地项目页、resume 和 restructure plan	待核验个人 claim queue	原始 checkpoint、episode trace、贡献 ownership

冻结 commit 只解决“本页看的是哪一版源码”，不解决 dataset/checkpoint/runtime 的版本。尤其是 Cosmos 的 RoboCasa 安装命令没有 commit pin；本页记录 observed fork HEAD 只是访问证据，不反推论文环境。

3. 可比性的 protocol tuple

把一次评测写成 protocol tuple：

$$\mathcal{P} = (S, I, W, O, A, K, C, H, Y, R, N, M),$$

其中：

字段	含义	最小证据
S	suite、task list、task version	task IDs、BDD/registry hash、subset
I	initial-state distribution	init file/hash、state index、layout/object split
W	warmup/stabilization	action vector、步数、是否计入 horizon
O	observation contract	camera keys、resolution、flip/crop/JPEG/history
A	action contract	dim/order/unit/frame、normalizer、clip、gripper sign
K	chunk/execution	predicted horizon、executed steps、replan/queue
C	controller/timing	controller config、control Hz、physics Hz、latency
H	episode horizon	task-specific max step、timeout、early termination
Y	success predicate	exact function、threshold、debounce、invalid episode rule
R	randomness	train/eval/reset/sampler seed、determinism flags
N	sampling plan	trials/task、seeds、macro/micro aggregation、CI
M	model/runtime	checkpoint SHA、config、stats、code/env/hardware

单任务成功率只是：

$$\hat{p}_t = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathbb{1}\{Y(\tau_{t,i}) = 1\}.$$

如果 $\mathcal{P}_a \neq \mathcal{P}_b$，$\hat{p}_a - \hat{p}_b$ 同时混入 model effect 与 protocol effect。任务名、分母和 success 函数相同仍不足以比较；initial-state distribution、chunk 和 controller 任一项不同都可能改变闭环轨迹。

4. LIBERO frozen default 到底是什么

4.1 Task 与 initial state

frozen benchmark 将 `LIBERO_SPATIAL`、`LIBERO_OBJECT`、`LIBERO_GOAL`、`LIBERO_90`、`LIBERO_10` 注册为 suite；每个 task 绑定 BDDL 文件与 `.pruned_init`。`get_task_init_states` 通过 `torch.load` 载入固定 simulator states。[1]

official lifelong metric 的 default contract 是：

字段	Frozen default	代码入口
suite task order	10-task suite 支持 21 种 task order；default index 0	`benchmark/__init__.py`
init	`.pruned_init`；按 episode index modulo state count	`metric.py:evaluate_one_task_success`
trials	`n_eval=20`，并行进程最多 20	`configs/eval/default.yaml`
warmup	5 次全零 7D action	`metric.py`
camera	agentview + eye-in-hand，128×128	`configs/data/default.yaml`
low-dim	gripper qpos + joint pos	`configs/data/default.yaml`
action/control	7D；`OSC_POSE`；20 Hz	`env_wrapper.py:ControlEnv`
horizon	eval max 600；env default horizon 1000	eval config + env wrapper
success record	runner 累积 environment `done`；wrapper 的 `check_success` 转发 `_check_success`	`metric.py` + `env_wrapper.py`
seed	global default 10000；env 行为仍需 runtime 版本核验	`configs/config.yaml`

`evaluate.py` 的独立入口也固定 20 个环境、5 次 zero step，但其 warmup 后没有把返回 observation 赋回 `obs`；`metric.py` 则正确更新 `obs`。因此只说“用了官方 evaluate.py”还不够，必须记录 exact entrypoint 与 local diff。

4.2 “官方 LIBERO”不是唯一数字协议

LIBERO repo 提供 task、state 与 baseline helper；后续论文可以定义新的 OOD init、不同 trial count、不同 action chunk 和 horizon。合规表述是“基于 LIBERO 环境的某 protocol”，而不是把所有论文的 setting 都叫“LIBERO 官方 strict”。

5. UWM：Table II、repo reset 与项目 fixed-init 是三套协议

5.1 论文 Table II

UWM v3 Table II 的五个 task 与数值为：[2]

Task	UWM paper mean	当前项目页 claim	差值只作描述
Book-Caddy	0.91	44/50 = 0.88	-0.03
Soup-Cheese	0.93	40/50 = 0.80	-0.13
Bowl-Drawer	0.80	38/50 = 0.76	-0.04
Moka-Moka	0.68	33/50 = 0.66	-0.02
Mug-Mug	0.65	31/50 = 0.62	-0.03
Average	0.79	186/250 = 0.744	-0.046

最后一列不能解释为模型性能差距，因为 protocol 不同。论文明确写：LIBERO-90 pretrain 100K steps；五个 LIBERO-10 task 各用 50 demos finetune 10K steps；3 个 random seeds；50 个不同 OOD initializations；对象 initialization range 增加 0.03；删除背景物体造成视觉 shift。Table II 的 `±` 是跨 seed 的区间语义，不是单次 250 rollout 的 binomial CI。[2]

5.2 Frozen UWM repo 的 rollout

`eval_robomimic.py` 的实际路径是：dataset HDF5 metadata → one task env → `env.seed(e)` → `env.reset()` → observation history 重复填充 → model sample → wrapper 整段执行 action sequence → `check_success` 或 1000-step truncation。[3]

字段	UWM repo runner	与 Table II 的缺口
init	random reset；wrapper seed 只调用 `np.random.seed`	没有 +0.03 / background removal manifest
trials	50	数量相同不代表 init 相同
warmup	无	论文未由 repo entry 明示
camera	two RGB 128，wrapper vertical flip；model resize 240	OOD visual modification未实现
chunk	sample 返回整段，wrapper for-loop 执行到 success	action horizon/endpoint依赖 model config
horizon	1000	Table II exact horizon 未由论文表格或 repo entry闭合
success	`env.check_success()`	predicate family一致，environment version仍需固定
checkpoint	`models.pt`；README Google Drive 无 hash	不能绑定 paper seed/checkpoint

`LIBEROEnvWrapper.reset_to_state` 存在，但只在 inverse-dynamics trajectory-tracking ablation 中由 `demo["states"][0]` 调用。它不能证明 policy rollout 使用 fixed init。LIBERO-90 pretraining 还在 `maybe_collect_rollout` 中被直接跳过。[3]

5.3 当前项目 claim 的正确层级

workspace 的 UWM 项目页 与 resume restructure plan 写了：五任务、50/task、`.pruned_init`、5-step zero action、186/250=74.4%；另有 reset-style 232/250=92.8%。本轮没有发现对应 raw summary/checkpoint/normalizer/init index/video/action trace，所以证据层级是：

Claim	当前等级	允许表述	禁止表述
74.4% fixed-init	`CLAIM_ONLY`	“当前项目材料记录为 186/250，待 artifact 复核”	“已严格复现 Table II”

Claim	当前等级	允许表述	禁止表述
92.8% reset-style	`CLAIM_ONLY / DIAGNOSTIC`	“项目材料记录的 protocol sensitivity 对照”	“UWM 正式 benchmark 92.8%”
0.79 Table II	`UPSTREAM_VERIFIED`	“UWM v3 的 OOD protocol mean”	“我的 checkpoint 官方成绩”
74.4 vs 0.79	`NON_COMPARABLE`	“同任务名下的方向性上下文”	“只低 4.6pp，已接近复现”

恢复可比性的两条路径二选一：复现 Table II 的 environment modifications + 3 seeds；或把本地 protocol 明确命名为 `fixed-pruned-init-50-v1`，只和同一 protocol 下的 baseline/checkpoint 做 paired comparison。

6. Cosmos LIBERO protocol

frozen Cosmos README 的 public command 使用 checkpoint `nvidia/Cosmos-Policy-LIBERO-Predict2-2B`、dataset stats、T5 embeddings、two cameras/proprio、chunk 16、open-loop 16、5 denoise steps、deterministic true、seeds 195/196/197；每 suite 10 tasks × 50 episodes。[4]

字段	Frozen Cosmos LIBERO runner	边界
init	official task init states；默认取 `[episode_idx]`，50 trials	需要 state count ≥ 50；没有 modulo/bounds guard
env seed	`env.seed(0)`	注释称 fixed state 下仍影响 object position
warmup	10 次 `[0,0,0,0,0,0,-1]`	与 LIBERO default 5 次全零不同；gripper语义也不同
camera	agentview + wrist，environment 256，vertical flip，JPEG path	model resize另由 model family决定
action	stats unnormalize；chunk 16；execute 16	config 与 train chunk有 assert
horizon	spatial 220、object 280、goal 300、LIBERO-10 520、LIBERO-90 400	与 default 600 不同
success	runner 把 `done` 直接标成 success；frozen LIBERO BDDL base 将 `done` 设为 `_check_success()`	必须同时冻结 benchmark version，不能只看 runner
determinism	deterministic 模式每 policy step 重置 RNG 到 0	与只固定 reset seed不同
aggregation	50/task；README 说对 seeds 195/196/197 平均	单 seed log不能替代论文 aggregate

这意味着 Cosmos-LIBERO、LIBERO default、UWM Table II、UWM repo reset-style 至少是四套 protocol。可以比较架构思想，不能直接比较成功率。

7. Cosmos RoboCasa protocol 与版本陷阱

7.1 Environment fork

Cosmos `ROBOCASA.md` 明确 clone `moojink/robocasa-cosmos-policy`，但没有 SHA。访问日 observed fork HEAD `edd9a...` 的 package version 是 0.2.0、MuJoCo pin 3.2.6；官方 `robocasa/robocasa` frozen `b468...` 是 package 1.0.1、MuJoCo pin 3.3.1。官方 v1.0.1 可以解释当前 API，但不能作为 Cosmos run 的环境 provenance。[5][6][7]

7.2 Runner contract

字段	Frozen Cosmos RoboCasa	必须保存的 artifact
task set	runner列出 24 tasks	exact task list/registry hash
trials	50/task default；README要求3 seeds	per-task per-seed summary
scenes	5 个 `(layout,style)`；每 10 episodes 切一个	episode→scene mapping
object split	`B` held-out objects	fork commit + object registry
env seed	deterministic 时 `seed = base_seed × episode_idx × 256`	seed log；episode 0 为 0
warmup	10 次全 0 的 environment-dimensional action	warmup是否计入视频/horizon
camera	left/right/wrist，224×224，vertical flip，JPEG input path	raw/preprocessed frame pair
controller	pickle `robocasa_controller_configs.pkl`	pickle SHA + decoded JSON summary
action	policy 7D；environment 12D 时追加 `[0,0,0,0,-1]` mobile base	action axis/unit/gripper/mobile schema
chunk	predict 32，execute first 16 then requery	queue/replan trace
horizon	task-specific 300–1000	exact task horizon
success	`env._check_success()`	task class/predicate commit
invalid episode	data collection时少于5步只是不保存，仍已计入 success list	denominator与剔除规则须另写

当前 WAM 项目页写 `raw 86/120=71.67%`、`trusted 84/118=71.19%`。若“trusted”删除两个 early-success/bug episode，这已经改变分母；必须保存两条 episode ID、原始 predicate、删除理由和预注册规则。事后看视频再删 episode 只能叫 cleaned diagnostic，不能叫官方 success rate。[10]

8. WorldGym：world-model proxy 不等于 closed-loop benchmark

WorldGym v3 的目标是用 action-conditioned video world model 作为真实部署前 proxy；frozen repo runner 从一张 trial PNG 初始化 world model，policy 每步读取生成历史，world model 生成下一帧，再由 GPT-4o 对完整生成视频评分。[8][9]

字段	Frozen code	风险
initial state	root 下 PNG + JSON instruction	静态帧不包含真实 dynamics state
action	policy-specific转换后 pad到10D	conversion可能改变 scale/gripper semantics
rollout	default 40，world model chunk size 1	error autoregressively accumulation
reward	GPT-4o `n=5` votes；0/0.5/1	API/model version、prompt与解析会漂移
tie break	优先 1，再 0.5，再 0	optimistic tie bias
aggregation	trial fractional score mean	非 simulator binary predicate
checkpoint	Google Drive约9GB，无 hash	runner commit不能绑定 tensor
license	frozen tree无 LICENSE	暂不执行或复用代码

正确关系是 `offline action/video metrics → WorldGym proxy → simulator closed loop → real closed loop` 的证据阶梯，不是四个可互换的指标。WorldGym 可以筛选候选策略与发现 failure mode，但仍需

simulator/real paired validation。

9. Strict / reset-style / paper / proxy 可比矩阵

名称	Init / reset	Warmup	Trial / seed	Horizon / chunk	当前可比结论
LIBERO frozen default	first 20 pruned states	5 × zero7	20/task，default seed 10000	600，policy-specific single action	只与相同 config 比
UWM Table II v3	50 OOD init；+0.03 range；remove background	paper未由公开runner闭合	50/task × 3 finetune seeds	exact eval runner未提供	上游论文内部可比
UWM repo rollout	`env.seed(e)+reset`	none	50/task，runner seed 0	1000，execute sampled sequence	reset-style；不等于 Table II
项目 fixed-init claim	pruned init	5 × zero7	50/task，seed artifact缺失	chunk/horizon artifact缺失	74.4%待核；不与0.79直比
项目 reset claim	random reset	项目页未闭合	50/task，artifact缺失	artifact缺失	92.8%仅 diagnostic
Cosmos LIBERO	first 50 default states	10 × close-gripper noop	50/task × seeds 195/196/197	220–520，16/16	只与 Cosmos同 protocol 比
Cosmos RoboCasa	generated scene/object split B	10 × zero12	50/task × 3 seeds	300–1000，32/16	需 frozen fork/controller
项目 HF RoboCasa smoke	项目页称 24×5	项目页未闭合	raw120 / trusted118	artifact缺失	harness smoke；非官方 aggregate
WorldGym	real initial PNG	none	discovered PNG trials / retries	generated 40，chunk1	VLM proxy；不与 env success 混分

10. Local evidence manifest

10.1 Run-level manifest

每个可对外数字必须有一个 immutable run manifest：

分组	必填字段
identity	`run_id`、开始/结束时间、owner、purpose、git dirty diff hash
model	upstream model ID/revision、checkpoint path、SHA-256、bytes、config SHA
stats	action/proprio normalizer SHA、dataset stats revision、T5/tokenizer revision
code	policy repo SHA、eval harness SHA、benchmark/fork SHA、local patch SHA
runtime	Python、PyTorch、CUDA/MACA、MuJoCo、robosuite、driver、GPU/accelerator
protocol	$S, I, W, O, A, K, C, H, Y, R, N$ 的结构化值
outputs	raw episode JSONL、summary JSON、videos、action traces、stderr/stdout SHA

10.2 Episode-level record

字段	例子 / 说明
task identity	suite、task ID、language、BDD/task class hash
episode identity	episode index、seed、init state index/hash、layout/style/object split
observation	camera keys、first-frame SHA、preprocessed-frame SHA
action	normalized/raw action、timestamp、chunk index、executed count、clip flag
result	success bool、predicate output、step、timeout、exception
failure	perception、action-scale、gripper、contact、recovery、protocol、infra
artifacts	video path/SHA、trace path/SHA、checkpoint SHA

10.3 当前 workspace 证据缺口

本轮只扫描 `C:\Users\mousongzhe\cv`。发现 resume、project HTML 与 roadmap claim，未发现 UWM/WAM raw checkpoint、normalizer、per-episode JSON/CSV、rollout videos、exact eval config、controller dump 或 local diff。因此：

项目 claim	当前证据	下一件最小补证据
UWM 186/250	resume + project page	per-task summary + 250 episode IDs + checkpoint/config/stats SHA
UWM 232/250	project page	same checkpoint paired protocol manifest + init distribution diff
UWM 对齐 Table II	paper task names相同	OOD env patch、50 init manifest、3 seeds；否则改名 local fixed-init
WAM 84/118	project page	raw 120 records、2 deleted IDs、predeclared validity rule、HF revision
WAM local 0-success	project narrative	load log、checkpoint SHA、action/gripper stats、failure videos
MSE 0.09718→0.05349	project narrative	split/metric script、prediction/label tensors、paired checkpoint

11. 失败分类与归因顺序

Failure family	先查 artifact	可下结论前需要
protocol	init/warmup/horizon/success/denominator diff	same checkpoint paired rerun
checkpoint	SHA、missing/unexpected keys、EMA/normalizer	load log + tensor stats
observation	camera key、flip/crop/JPEG/history	raw/preprocessed frame pair
action	axis/unit/frame/gripper/clip	normalize round-trip + action trace
chunk/control	predicted vs executed、queue、controller Hz	timestamped command/state trace
perception	target localization、occlusion、language mismatch	image overlays + policy attention/probe
contact	grasp pose、friction、collision、gripper force	simulator state/contact trace
recovery	first error后是否重规划、OOD state	failure timeline + counterfactual reset
infrastructure	exception、timeout、OOM、dropped episode	stderr + exit code + denominator rule

诊断顺序应先排 protocol/checkpoint/observation/action contract，再讨论模型机制。否则“world model 没学到 contact”可能只是图像上下翻转或 gripper sign 错。

12. Protocol gate 伪代码

```
# Protocol comparison gate: compare fields before comparing scores.
REQUIRED = [
    "suite", "tasks", "init", "warmup", "observation", "action",
    "chunk", "controller", "horizon", "success", "randomness",
    "trials", "checkpoint", "runtime",
]

def compare_runs(run_a, run_b):
    # Missing values are not wildcards; they block a strong comparison.
    missing = [k for k in REQUIRED if not run_a.get(k) or not run_b.get(k)]
    if missing:
        return {"verdict": "UNKNOWN", "missing": missing}

    mismatches = {
        k: (run_a[k], run_b[k])
        for k in REQUIRED
        if canonicalize(run_a[k]) != canonicalize(run_b[k])
    }
    if mismatches:
        return {
            "verdict": "NON_COMPARABLE",
            "mismatches": mismatches,
            "allowed_claim": "protocol sensitivity or directional context only",
        }

    # Score comparison opens only after artifact and protocol identity checks pass.
    assert sha256_exists(run_a["episodes"])
    assert sha256_exists(run_b["episodes"])
    return paired_or_interval_test(run_a, run_b)
```

这个 gate 故意把 unknown 当 blocker。面试中最危险的不是承认 unknown，而是把 unknown 默认为相同。

13. 面试回答骨架

13.1 30 秒：什么叫 strict eval

“我不会把 strict 当成‘跑得更认真’，而是把评测写成可重放 tuple：task、init、warmup、camera、action/normalizer、chunk/replan、controller、horizon、success、seed、trials 和 checkpoint/runtime。只有这些字段相同，成功率差值才主要反映模型。以 UWM 为例，论文 Table II 是五个 LIBERO-10 任务上的 OOD 初始化，还扩大 object init range 并移除背景物体；repo 默认 runner 则是 random reset。我的 fixed-pruned-init 结果必须单独命名，不能只因任务相同就说严格复现 0.79。”

13.2 90 秒：怎样讲当前 UWM 项目

“这个项目目前最可信的贡献方向是评估协议工程，而不是宣称超过论文。项目材料记录五个任务 fixed-init 186/250，以及 random-reset 232/250；我会把后者只当 protocol sensitivity diagnostic。进一步 source audit 后，我发现论文 v3 的 0.79 使用 50 个 OOD initialization、三次 finetune seed，并做 +0.03 object-init shift 和背景物体移除，所以本地 pruned-init 不能叫 Table II strict aligned。下一步我会冻结 checkpoint、normalizer、250 条 episode trace、init index、warmup、chunk/horizon 与 harness diff；若复现论文环境改造，就报告 three-seed OOD result；否则把协议正式命名为 fixed-pruned-init-50-v1，只和同协议 baseline 做 paired comparison。”

13.3 RoboCasa 71.19% 的防守口径

“71.19% 是官方 HF checkpoint 在我这套 harness 上的 smoke result，不是我的训练结果，也不是官方 50 trials × 3 seeds protocol。项目页记录 raw 86/120 和 trusted 84/118，说明我还必须给出被剔除的两条 episode、预先定义的 invalid rule、fork/controller/checkpoint revision；在这些 artifact 齐全前，我只用它证明 harness 能产出非零可信行为，不把它当正式 benchmark。”

14. 尚未核实与下一步

- 已核实：LIBERO default eval code；UWM v3 Table II 与 Appendix OOD setting；UWM frozen reset-style runner；Cosmos LIBERO/RoboCasa frozen runners；RoboCasa official/fork version split；WorldGym frozen proxy scorer。
- 尚未核实：用户 local repos/runs、checkpoint tensor、normalizer、dataset revision、250/250 episode artifacts、RoboCasa 120 raw episodes、controller pickle semantic dump、browser visual QA、独立 fact review、真实口述。
- 当前 verdict：`WARN`。原因不是上游来源不足，而是个人数字与 runtime artifact 尚未闭合，且“Table II strict aligned”表述已被 source audit 发现 protocol mismatch。
- 后续动作：WM-039..044 的第一批 benchmark/evidence 知识页与 6 张题卡已完成；下一步独立事实复核并补 local run artifacts。所有题卡保持 `unseen`，项目数字不进入强答案，直到 VOL 07 evidence gate 通过。

15. 参考文献（References）

1. Lifelong Robot Learning, `LIBERO` frozen commit `8f1084e3132a39270c3a13ebe37270a43ece2a01`: metric.py, eval config, environment wrapper, benchmark registry, MIT.
2. Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets”, arXiv `2504.02792v3`, Table II and Appendix A, accessed 2026-07-16.
3. WEIRDLabUW, `unified-world-model` frozen commit `b7d4067c8859775e5b08f681e76d538015cdd445`: eval_robomimic.py, wrappers.py, template config; no license file found in frozen tree.
4. NVIDIA, `cosmos-policy` frozen commit `18a2accadf4e7a3531e56754102af5a24d2316da`: LIBERO runner, LIBERO utils, RoboCasa runner, Apache-2.0 code.
5. `moojink/robocasa-cosmos-policy`, observed HEAD `edd9a328b3ec98050f42d194c1419307a79c4d87`, setup.py, eval helper, MIT; observed 2026-07-16, not a verified paper pin.
6. RoboCasa, official repo frozen commit `b4684e6ee37d377cc392e98302a6b916d588b415`, setup.py, eval_utils.py, determinism test, MIT.
7. NVIDIA Cosmos Policy frozen docs: LIBERO.md, ROBOCASA.md.
8. Quevedo et al., “WorldGym: World Model as An Environment for Policy Evaluation”, arXiv `2506.00613v3`, abstract, 2025.
9. `world-model-eval/world-model-eval` frozen commit `709665b29f5c95124e7dbc05b0d4d47cd353de97`: README, utils.py, world_model.py; no license file found in frozen tree.

10. 本地待核验材料：UWM/LIBERO 项目页、Cosmos/WAM 项目页、resume restructure plan、optimized resume。

Part 5 / 12 · `interview_materials/vla/sources/vol_04_cosmos_policy_wam_claim_to_code.html` · SHA-256
`F7FB2C7C68D1003763B24812FA5494F4BB361DBCB1BD4342FC1D658FBE74CC76`

返回本卷目录

Canonical Markdown SHA-256：
`f7fb2c7c68d1003763b24812fa5494f4bb361dbcb1bd4342fc1d658fbe74cc76`；生成自：
`interview_materials\vla\sources\vol_04_cosmos_policy_wam_claim_to_code.md`

VOL 04 Source Audit C：Cosmos Policy / WAM Claim-to-Code

审计日期：2026-07-16 状态：drafted / paper_v1_full_text_reviewed / repo_frozen_and_line_audited / hf_model_revisions_frozen / independent_fact_reviewed / WARN 覆盖题目：VLA-WM-008..009、VLA-WM-022..030、VLA-WM-040..044 frozen repo：NVlabs/cosmos-policy @ 18a2accadf4e7a3531e56754102af5a24d2316da（commit date 2026-01-22）

1. 结论先行

1. **Cosmos Policy 不是在视频 backbone 后外挂 action/world/value 三个 head。**论文和 frozen code 都把当前 proprio、action chunk、未来 proprio、future images 与 value 编排进同一 video latent sequence；低维量通过 latent injection 覆写 placeholder frame，训练与采样继续复用 Cosmos-Predict2 的 diffusion transformer。[1][2]
2. **“同一模型有三种角色”由 conditioning 与 loss routing 决定。**base training 的示范/rollout sample 约按 50% policy、25% world model、25% value function 分流；policy、world、value 分别对不同 latent prefix 条件化并预测不同 target slot，而不是仅靠三个 loss 名称。[1][2]
3. **direct policy 与 planning 是两个不同推理合同。**direct eval 一次并行生成 action/future/value，但只执行 action；planning 先采样 action，再条件生成 future state，再评分 value，best-of-N 选择最高分候选。[1][2]
4. **可用 planning 需要 rollout distribution 与独立 planning checkpoint。**论文明确 base demonstration-only model 的 world/value 对失败分布覆盖不足；其 ALOHA planning 实验使用 rollout 数据细化 world/value，并 dual-deploy 原 policy checkpoint 与 refined planning checkpoint。[1]
5. **latent injection 不是“语义 token”。**action $K \times D_a$ 被 flatten/repeat 到 $16 \times 28 \times 28$ latent volume；解码时把可整除的完整 action copies reshape 后求均值。value 则把整个 value frame 求均值，再从 $[-1, 1]$ 映射到 $[0, 1]$。[1][2]
6. **LIBERO 与 RoboCasa 的执行合同不同。**LIBERO 是 16×7 action chunk 并默认全 16 步 open-loop；RoboCasa 是 32×7 chunk、默认执行前 16 步，再在环境侧拼接 5 维 inactive mobile-base action 形成 12D controller action。[2][3][4][5]

7. **官方 direct-policy 分数不能写成本地 WAM 结果。**论文/模型卡报告 LIBERO 98.5% 和 RoboCasa 67.1%，它们绑定官方 checkpoint、task set、50 trials、三 seeds 与指定 runtime；任何本地 45%、71.19%、0-success 或 offline MSE 必须由本地 artifact 单独证明。[1][6][7]
8. **论文、代码与权重许可证不同。**arXiv v1 是 CC BY 4.0，frozen GitHub tree 是 Apache-2.0，而两个 HF policy model card 声明 NVIDIA One-Way Noncommercial License；不能用 repo license 推断 checkpoint 可商用。[1][2][6][7]
9. **当前审计与后续独立复核均未运行 GPU inference 或新 rollout。**已核对 paper→repo→model card→eval path，并在独立复核中下载固定 config/statistics、重哈希本地 diagnostic；checkpoint tensor、完整 dataset revision、runtime reproduction 与候选人 ownership 仍未闭合，review 必须保持 WARN。[8]

2. 来源与证据边界

编号	来源	冻结版本	可支持	不能支持
[1]	Kim et al., Cosmos Policy paper	arXiv 2601.16163v1，提交 2026-01-22；CC BY 4.0	latent injection、三类概率合同、50/25/25 routing、rollout refinement、best-of-N、论文 protocol/metric	用户本地 checkpoint、日志、0-success、个人贡献
[2]	NVlabs/cosmos-policy	commit 18a2accadf4e7a3531e56754102af5a24d2316da；Apache-2.0	dataset slot、mask、loss、sampler、get_action、planning、eval wrapper 的 frozen code 行为	命令未执行；main/commit 与任一缓存权重不自动兼容
[3]	同 commit 的 README / LIBERO.md	同上	官方 quickstart、model ID、16-step chunk、5-step denoise、500 trials/suite、训练命令	README command 不证明本机复现
[4]	同 commit 的 ROBOCASA.md	同上	24 tasks、50 demos/task、32-step chunk、16-step execution、50 trials/task、seeds	文档默认值不等于用户项目 protocol
[5]	Frozen LIBERO/RoboCasa eval source	同上	reset/warmup/init/camera/controller/action queue/success predicate	尚未与外部 benchmark commit 做独立复核
[6]	nvidia/Cosmos-Policy-LIBERO-Predict2-2B model card/API	revision cb689ec0e3347c13667d70a78a3447388f5c3bb8	model I/O、official checkpoint files、98.5% protocol、NSCLv1	未下载 checkpoint；card 的数字仍是上游声明
[7]	nvidia/Cosmos-Policy-RoboCasa-Predict2-2B model card/API	revision 4b2a04c80d97202f86127ebec80461e8016ec1dc	model I/O、official checkpoint files、67.1% protocol、NSCLv1	未下载 checkpoint；不能证明某个本地 cache revision
同源推断	[1]–[7] 联合分析	evidence cutoff 2026-07-16	claim-to-code、shape、mask、protocol 与风险边界	不替代 runtime experiment、独立事实审查和视觉 QA

基础权重 `nvidia/Cosmos-Predict2-2B-Video2World` 的 HF API revision 冻结为 `f50c09f5d8ab133a90cac3f4886a6471e9ba3f18`。训练 config 的 `load_path` 指向其中 `model-480p-16fps.pt`，但本轮没有下载该权重或验证文件 hash。[2]

3. 概率合同：policy、world model 与 value

令 $s_t = (o_t, p_t, c)$ 表示当前多视角 observation o_t、proprio p_t 与任务文本 c，$a_{t:t+K-1} \in \mathbb{R}^{K \times D_a}$ 是 action chunk，$s' \approx s_{t+K}$ 是未来 observation proxy，G_{t+K} 是 Monte Carlo return。论文把三种训练角色写成：[1]

$$\begin{aligned} &\text{policy: } p_\theta(a, s', G \mid s), \\ &\text{world: } p_\theta(s', G \mid s, a), \\ &\text{value: } p_\theta(G \mid s, a, s'). \end{aligned}$$

变量解释：这里的 s' 实际只包含 $t+K$ 时刻的 proprio 与 camera observations，并非可直接访问的完整 simulator state；G 是终局稀疏奖励按 γ^{H-t} 折扣后的 empirical return。实现映射：同一 latent sequence 保持固定布局，`num_conditional_frames`、input mask 与 sample mask 改变哪部分是 clean condition、哪部分参与 denoising target。[1][2]

这三条合同不能合并成“一个 joint loss 自动学会 planning”。base policy 可以并行生成三个 output，但只有在 world/value 对候选 action 的 OOD future 可靠且 value 已校准时，才可用于 action selection。论文自己也把 demonstration-only direct policy 与 rollout-refined planning model 分开。[1]

4. Latent sequence 与注入/提取

4.1 Frozen layouts

Suite	latent sequence（去掉 tokenizer 内部 spatial token）	state_t	action / proprio
LIBERO	blank, current proprio, wrist, primary, action, future proprio, future wrist, future primary, value	9	action 16×7; proprio 9D
RoboCasa	blank, current proprio, wrist, left primary, right primary, action, future proprio, future wrist, future left, future right, value	11	action 32×7; proprio 9D

每个语义 frame 对应 $C' \times H' \times W' = 16 \times 28 \times 28$ latent volume。Wan2.1 tokenizer 的 temporal compression factor 是 4，所以 config 以一张首 blank 加每个 semantic frame 的四张重复 image 组织 33-frame LIBERO input 或 41-frame RoboCasa input。[1][2]

4.2 Action latent injection

令归一化 action chunk 为 $A \in [-1, 1]^{K \times D_a}$，latent volume 元素数为 $M = C'H'W'$，action 元素数为 $L = KD_a$。frozen code 的注入可写为：

$$z_a = \text{reshape}\left(\text{repeat}(\text{vec}(A), \lceil M/L \rceil)_{1:M}\right) \in \mathbb{R}^{C' \times H' \times W'}.$$

变量解释：`replace_latent_with_action_chunk` 先 flatten (A)，按 ceiling repeat 填满 latent，再覆写 action latent index。它没有 action-specific learned encoder；学习发生在共享 diffusion transformer

中。[2]

提取时，`extract_action_chunk_from_latent_sequence` 只取可组成完整 action copies 的前 qL 个元素，其中 $q = \lfloor M/L \rfloor$，reshape 成 $q \times K \times D_a$ 后平均：

$$\hat{A} = \frac{1}{q} \sum_{j=1}^{q} \text{reshape}(z_a^{(j)}, K, D_a).$$

实现含义：注入时可能存在最后一个不完整重复片段，提取时它被丢弃；这应在极端 (K, D_a) 组合与换机器人时单测。输出随后按 dataset `actions_min/actions_max` 反归一化，不能跳过 checkpoint 对应的 statistics 文件。[2]

4.3 Value 与 future image

value scalar 被扩展到整个 value frame；采样后对该 frame 的全部元素求均值，并计算 $\hat{V} = \text{clip}((\bar{z}_v + 1)/2, 0, 1)$。future images 需要先把 blank/proprio/action 注入位置换回 VAE 编码前的 clean placeholder latent，再用 tokenizer decode；因此“保存了 future image”不代表执行时这些 latent 会送入 controller。[1][2]

5. 训练 routing、conditioning 与 loss

5.1 Dataset routing

LIBERO/RoboCasa dataset 把索引空间组织为 demos、success rollouts、failure rollouts。demo sample 的 `rollout_data_mask=0`；rollout sample 为 1。若 `return_value_function_returns=True`，rollout sample 再随机选择 world-model 或 value-function 角色。base config 令 demonstration sampling probability 0.5、success rollout sampling probability 0.5，而每个 rollout sample 内 world/value 默认各占一半，对应论文约 50/25/25 的 batch 角色比例。[1][2]

future index 固定为 (t+K)，episode 尾部 clamp 到最后一帧；action chunk 不足时重复最后动作补齐。value label 使用 future index 对应的 return。尾部 padding 没有独立 valid-step mask，因而边界样本会把 repeated terminal actions 作为真实 target；这是 frozen behavior，不应当作通用设计最优解。[2]

5.2 Conditioning mask

角色	clean condition	target/主要用途
policy demo	current state (s)	action、future state、value 可联合预测
world rollout	current state + action	future proprio/images；value 可作为 auxiliary
value rollout	full prefix (s,a,s')	value frame
rollout state-value variant	future state only	(V(s'))，mask current state/action
rollout Q variant	current state + action only	(Q(s,a))，mask future state

`CosmosPolicyVideo2WorldModel.get_data_and_condition` 对 world sample 把 action frame 置为 clean condition；对 value sample 把除 value frame 之外的 frames 置 clean。V/Q 变体再对 current/action 或 future slots 做 input masking。这里 `mask=1` 表示 clean condition，`mask=0` 表示需 denoise/predict；与某些 attention mask 语义相反，面试时必须说清 mask 类型。[2]

5.3 EDM loss 与 target mask

frozen training 先把 action/proprio/value 写入 clean latent x_0，按 HybridEDMSDE 产生 noisy x_t，denoiser 输出 $\hat{x}_0$。主项可抽象为：

$$\mathcal{L}_{EDM} = \frac{1}{BCTHW} \sum_{b,t,c,h,w} m_{b,t}\, w(\sigma_{b,t}) \left(x_{0,bcthw} - \hat{x}_{0,bcthw}\right)^2.$$

变量解释：$m_{b,t}$ 是 latent-frame loss mask，$w(\sigma)$ 是 noise-level weight。`mask_loss_for_action_future_state_prediction=True` 时 demo 仅保留 action slot、world sample 仅保留 future state slots、value sample 仅保留 value slot；默认 config 未在本轮确认该 flag 被主实验打开，因此不能把可选分支写成所有 checkpoint 的既定 loss。[2]

代码还可对 action slot 乘 `action_loss_multiplier`。各 action/future/value L1/MSE 同时被记录用于诊断，但主反向 loss 是 noise-weighted latent MSE，而不是日志里的 unweighted component L1。README 的“训练到 action L1 约 0.010/0.015”是经验阈值，不等于 closed-loop early stopping 充分条件。[2][3][4]

另有一个必须保留为 WARN 的 frozen-code 风险：`mask_value_prediction_loss_for_policy_prediction` 分支会构造 `final_mask_B_T`，但随后决定是否把 mask 乘到 `kendall_loss` 的条件没有列入这个 flag。主 LIBERO/RoboCasa config 没有打开该 flag，所以不能据此否定官方 checkpoint；但如果个人 WAM 配置单独启用它，应先做非零 mask/gradient 单测，不能仅凭配置名声称 auxiliary value loss 已排除。[2]

6. Sampling：parallel、autoregressive 与 denoising steps

6.1 Sampler contract

`CosmosPolicySampler` 在 `sample_clean=True` 且请求 steps (>1) 时，将 ODE solver 的 timestamp 数减一，最后再做一次 clean denoise；请求 1 step 时直接在首 sigma 上调用 denoiser。因此 CLI 的 `num_denoising_steps_action=5` 表示总 denoiser schedule contract，不应简单解读为“solver loop 五次”。[2]

LIBERO/RoboCasa inference config 将 SDE range 覆盖为 $\sigma_{min} = 4, \sigma_{max} = 80$，默认 direct policy 用 5 denoising steps。论文报告 H100 上 5-step action/future/value parallel generation 约 0.61 s/chunk、1-step 约 0.16 s/chunk；这些是指定硬件 wall-clock，不是本机 latency 事实。[1][2]

6.2 Direct policy

`get_action` 构造完整 latent sequence，以 current state slots 为 condition，一次采样得到 action/future/value。action 被反归一化后执行；future image/value 可用于可视化，但 direct eval 的 action selection 不读取它们。故“模型输出 value”不等于“在线 rollout 使用 value”。[1][2][5]

6.3 Autoregressive planning

planning path 的顺序是：

```
def best_of_n_step(observation, task, N):
    # 1. Policy checkpoint samples N candidate action chunks using distinct seeds.
    proposals = [policy.sample_action(observation, task, seed=i) for i in range(N)]

    # 2. Planning checkpoint conditions on each proposal and predicts future state samples.
    futures = [planning_model.sample_future(observation, action) for action in proposals]

    # 3. Predict and aggregate values; paper uses future/value ensembles for ALOHA planning.
    scores = [aggregate_value(planning_model.sample_value(future)) for future in futures]

    # 4. Execute only the configured open-loop prefix, then observe the real environment again.
    best = proposals[argmax(scores)]
    return best[:num_open_loop_steps]
```

代码映射：`get_action` → `get_future_state_prediction` → `get_value_prediction` / `get_qvalue_prediction` → `max(seed_to_return_dict, key=value)` → action queue。论文 ALOHA planning 使用 (N=8)、每个 action 三个 future samples、每个 future 五个 value samples；repo 支持 serial/parallel worker pool、ensemble aggregation 与有限 search depth。[1][2][5]

7. Best-of-N 与 value 风险

best-of-N 的选择规则是：

$$a^* = \arg\max_{a^{(i)} \sim \pi(\cdot|s),\ i=1..N} \widehat{V}\left(\hat{s}'^{(i)}\right), \qquad \hat{s}'^{(i)} \sim \hat{T}(\cdot \mid s, a^{(i)}).$$

变量解释：(pi) 是原 policy checkpoint，(hat T,widehat V) 来自 refined planning checkpoint。它不是 CEM/MPPI：没有迭代更新 action distribution，只对 policy proposals 排序；search depth 默认为 1，论文也明确主要使用 one-layer search。[1][2]

风险来自两层 optimizer's curse：候选越多，越容易选择 world-model hallucination 或过高 value estimate。论文用 rollout refinement、future/value ensemble 与 majority-mean 缓解，但不能消除 OOD calibration 问题。最小审计应同时报告：proposal diversity、future error、value calibration、rank correlation、selected-vs-random action success、N/latency 曲线与 failure traces。[1]

repo 的 `aggregate_value_predictions` 支持 average、LCB、success_vote、majority_mean 等 scheme；默认 LIBERO/RoboCasa config 是 average。论文 ALOHA planning 的 majority-mean 不能自动外推为所有官方 checkpoint 的默认 runtime。[1][2]

8. LIBERO / RoboCasa eval 合同

8.1 LIBERO

1. 使用 official suite task 与 default init states；每 task 取 `initial_states[episode_idx]` 后调用 `env.set_init_state`。
2. reset 后先执行 10 个 dummy actions 让物体稳定。
3. observation 为 agentview、wrist、9D proprio；图片按配置 vertical flip、JPEG、resize 与 center crop。
4. action queue 空时查询；默认 chunk 16、open-loop 16。
5. `env.step(action.tolist())` 返回 `done` 即记 success；需在独立 benchmark audit 中确认 wrapper 的 done/success 等价和版本。
6. 每 suite 默认 10 tasks×50 episodes=500 trials；paper/model card 的 98.5% 是四 suites、三 seeds 汇总口径。[1][3][5][6]

8.2 RoboCasa

1. controller 为 OSC_POSE，robot 是 PandaMobile，object split B；默认 5 个 layout/style scenes，每 10 个 episode 切一个 scene。
2. 每 episode 重建 env，seed 与 scene selection 进入 env kwargs；先 10 个 zero actions warmup。
3. observation 是 left/right third-person、wrist 与 9D proprio。
4. policy 输出 7D manipulation action；环境若要求 12D，则拼 `[0,0,0,0,-1]` 作为 inactive mobile-base controls。
5. default chunk 32，执行前 16 步再 requery；success 由 `env._check_success()`。
6. paper/model card 的 67.1% 是 24 tasks×50 trials×3 seeds=3600 trials；单 task 的 50 trials 不能直接称为 24-task average。[1][4][5][7]

8.3 Protocol comparability

必须冻结 checkpoint/model revision、dataset stats、T5 cache、repo commit、Python/PyTorch/CUDA、GPU、task/suite、init state、seed、determinism、camera flip/JPEG/crop、chunk/open-loop horizon、denoise steps、controller、max steps、success predicate、trial count。缺任一项时，跨报告差异最多是 directional context。

9. Checkpoint、资产与许可证边界

Artifact	Frozen revision / file manifest	License / 风险
GitHub code	18a2acc...；LICENSE、README、model/dataset/eval source	Apache-2.0；个别源文件头部出现与根 license 不同的历史版权措辞，复用仍需法务审查
LIBERO policy model	cb689ec...；`.pt`、config.json、stats JSON、T5 pickle、README	model card 声明 NSCLv1、非商用
RoboCasa policy model	4b2a04...；`.pt`、config.json、stats JSON、T5 pickle、README	model card声明 NSCLv1、非商用

Artifact	Frozen revision / file manifest	License / 风险
Base Predict2 model	f50c09...；含 `model-480p-16fps.pt`	gated=auto；本轮未下载；2026-07-20 独立复核确认 model card 为 NVIDIA Open Model License。[8]
LIBERO/RoboCasa modified datasets	repo docs 给出 HF IDs	API/raw card 访问在本轮返回 401；数据条款与 artifact revision 尚未冻结

`snapshot_download(repo_id=...)` 在 code 中未传 revision，因此真实运行会解析当时的 remote HEAD。严格复现应改为显式 revision 或记录 cache snapshot SHA，并对 `.pt`、config、stats、T5 embedding 建 manifest/hash。[2]

10. Offline proxy、online success 与 WAM 项目边界

证据	可以证明	不能证明
action/future/value latent L1/MSE	frozen split/noise 下 component fitting	controller execution、causal dynamics、task success
future image visualization	某候选的视觉预测	contact correctness、proprio/action axis、calibrated transition
value MAE/MSE	labeled state distribution 上的 regression	top-N selection calibration、OOD ranking
direct-policy success	固定 protocol 下 action execution	world/value auxiliary 的独立因果收益
planning uplift	policy/planning/model/protocol 全部固定时的 selection gain	单独归因于 world、value 或 N，除非有消融

WAM 项目数字必须留在 VOL 07 evidence layer：held-out MSE 需要 split、metric script、prediction/label artifact；官方 baseline 需要 model revision、24×5 或论文 protocol 的 summary；0-success 需要 local checkpoint load log、stats、action/gripper distribution、rollout video 与 harness hash。上游 latent injection、三目标 routing、best-of-N 均不能写成本人原创。

11. 失败诊断与最小测试矩阵

症状	首要证据	可能原因	最小实验
loss 正常、0 success	action hist、stats SHA、controller trace	normalizer/axis/gripper/controller mismatch	dataset action round-trip + one-step env replay
future image好看、动作无效	action sensitivity、contact trace	action ignoring / visual shortcut	固定 observation，置换 action，测 future delta
value loss低、best-of-N 更差	calibration/reliability、rank correlation	optimizer's curse / OOD overestimate	N sweep + random/median/top selector 对照
direct policy可用、planning失败	policy/planning checkpoint manifest	planning model distribution shift	original vs refined world/value blind eval
LIBERO高、RoboCasa低	suite I/O/controller/protocol	camera/action/chunk/contact difference	per-layer contract diff，而非只比平均分
换硬件/torch 后掉点	env lockfile、determinism log	numeric sampling sensitivity	three-seed paired rerun

症状	首要证据	可能原因	最小实验
chunk 前半好、后半漂	per-step action error/success	open-loop horizon过长	$K_{exec} \in \{1, 4, 8, 16\}$ paired sweep

还应加入四个 deterministic unit tests：action latent injection→extraction round trip；value frame→scalar round trip；normalization→denormalization round trip；LIBERO 7D / RoboCasa 7→12D controller mapping。它们只验证接口，不替代 closed-loop rollout。

12. 面试答题骨架与当前边界

12.1 30 秒回答

“Cosmos Policy 的关键不是给视频模型加三个 head，而是把 proprio、action chunk、future observation 和 value 都编码成同一 latent video sequence 的 frame。通过 condition mask 和 sample routing，同一 diffusion backbone 可以学 $p(a, s', V \mid s)$、$p(s', V \mid s, a)$ 和 $p(V \mid s, a, s')$。direct policy 一次并行生成三者但只执行 action；planning 则用原 policy 采样 N 个 action，再用 rollout-refined planning checkpoint 自回归预测 future/value，选最高 value。工程审计重点是 latent slot、loss mask、action stats、chunk/replan、checkpoint revision、init/warmup/controller 与 value calibration。论文官方分数不是我的本地项目结果。”

12.2 尚未核实

- 已核实：arXiv v1 full text、paper license、frozen repo commit/date/root license、latent injection/extraction、routing/mask/loss/sampler、LIBERO/RoboCasa config/eval path、两个 HF policy model revision/file list/model-card license；2026-07-20 独立复核重新命中13/13 official files、三份 HF revision、10/10 probes、9/9 local artifacts，并补核 base model license。[8]
- 尚未核实：checkpoint tensor/local hash、modified dataset card/revision、真实 inference、official result reproduction、external benchmark commit 与本地 WAM result/ownership。
- 用户本地待取证：WAM config/checkpoint、training/held-out metric artifact、official baseline cache revision、0-success rollouts、personal diff/ownership。
- 本页不把任何 WAM 本地数字写成 verified result，不把 Cosmos 上游设计写成本人贡献。
- 下一步：Audit C public facts 已独立复核；若继续文档复核，只选择 Source Audit D（Sim2Real）。VOL 04 aggregate knowledge/cards 已为44/44且全部 `unseen`，真实口述前不改变 mastery。

13. 参考文献（References）

1. Kim et al., “Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning”, arXiv 2601.16163v1, 2026. arXiv / HTML。
2. NVlabs/cosmos-policy, frozen commit `18a2accadf4e7a3531e56754102af5a24d2316da`. GitHub。重点文件：`policy_text2world_model.py`、`policy_video2world_model.py`、`cosmos_sampler.py`、`cosmos_utils.py`、dataset/config files。
3. Cosmos Policy LIBERO guide at the frozen commit. LIBERO.md。

4. Cosmos Policy RoboCasa guide at the frozen commit. ROBOCASA.md。
5. Frozen eval sources: LIBERO / RoboCasa。
6. NVIDIA, Cosmos-Policy-LIBERO-Predict2-2B model card, revision `cb689ec0e3347c13667d70a78a3447388f5c3bb8`. Hugging Face。
7. NVIDIA, Cosmos-Policy-RoboCasa-Predict2-2B model card, revision `4b2a04c80d97202f86127ebec80461e8016ec1dc`. Hugging Face。
8. VOL 04 Audit C 独立事实复核：paper、13个fixed-commit文件、HF revisions/config/statistics、10个deterministic probes与本地WAM artifacts已重新获取和reconcile，verdict为`PASS WITH WARNING`。

> Part 6 / 12 · `interview_materials/vla/sources/vol_04_cosmos_policy_wam_full10k_run_reconciliation.html` · SHA-256 `2E3B1E1C4CD87CDBD2AC19E6869A91FCBB762C86EBA7B1D66C15FDCAD4FBF378`

返回本卷目录

> Canonical Markdown SHA-256：`2e3b1e1c4cd87cdbd2ac19e6869a91fcbb762c86eba7b1d66c15fdcad4fbf378`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_cosmos_policy_wam_full10k_run_reconciliation.md`

VOL 04 Cosmos Policy / WAM：1 epoch gate 与 full10k 运行族对账

> 状态：`local_evidence_reconciliation / run_families_separated / WARN` 复核日期：2026-07-21 范围：只对账本地 `Project/report/cosmos_wam_repro` 中的独立 full10k WAM 复现与 `cosmos_policy_eval` 下的 1 epoch gate；不把两个 run family 合并成单一因果结果，也不改变题卡 `unseen` 或 ownership。

1. 结论先行

仓库此前把两条不同的 Cosmos/WAM 运行族压缩成“1 epoch 未完成”一条叙事，容易漏掉一条重要事实：

1. `cosmos_policy_eval/muxi_deepspeed_1epoch_20260623` 的 8-GPU、`max_steps=9907` gate，确实在 save-2k 主线遇到 cgroup OOM；no-cycle 修复后的正式 run 又在 step 1500 的 `state_dict(destination=...)` 保存接口处失败。该 run family 没有可用于本轮 eval 的可加载 checkpoint。
2. 独立的 `report/cosmos_wam_repro/cosmos_policy_wam_repro_full10k_20260619.log` 记录了另一个 batch-1、单卡 GPU7 的 full10k run：step 10000 loss=`0.0543`，checkpoint 写入远端路径，trainer `Done with training` 且 `EXIT:0`。
3. 同一 full10k report 登记了 CoffeePressButton、1 episode、`max_steps_override=120` 的 RoboCasa smoke，结果 `0/1`；后续独立 remote-artifact bundle 又冻结了同一 `iter_000010000` path 下的 24×5 formal eval：24/24 process `exit_code=0`，raw `2/120`，两条 CoffeeServeMug length=1 immediate success 按当前本地 policy 排除后 trusted `0/118`。这支持“full10k 训练与正式评测链路均跑通并可审计”，但不能升级为官方 benchmark、成功率 uplift、官方 scorer 等价或本地 checkpoint binary/hash 已核验。

因此面试中应明确说“两个 run family”：**1 epoch gate 的保存失败**与**独立 full10k repro 的远端 checkpoint path + 1×1 smoke + 24×5 formal eval**不能互相替代。

2. Run-family 对账表

run family	本地证据入口	训练事实	eval 事实	当前能否称为完整结果
1 epoch gate	`Project/report/cosmos_policy_eval/muxi_deepspeed_1epoch_20260623`	8 GPU、global batch 128、`max_steps=9907`；save-2k 在 step 1903 后 cgroup OOM；no-cycle 正式 run 在 step 1500 保存接口失败	本轮没有对应可加载 checkpoint/eval bundle	`NO / WARN`
full10k repro	`Project/report/cosmos_wam_repro` + `Project/report/cosmos_policy_eval/remote_artifacts`	单卡 GPU7，`MAX_ITER=10000`；step 10000 loss=`0.0543`；日志记录 checkpoint saved 与 `EXIT:0`	CoffeePressButton 1×1 smoke `0/1`；同一远端 `iter_000010000` path 下24×5 raw `2/120`、local-policy trusted `0/118`	`PARTIAL / WARN`
官方/其他 baseline	同目录 report 与独立 eval 产物	不是上述两条训练 run 的 continuation	只能作外部 protocol 参照	`NO causal join`

3. Full10k 证据冻结

3.1 原始文件与 SHA-256

文件	SHA-256	支持的事实
`C:\Users\mousongzhe\Project\report\cosmos_wam_repro\cosmos_policy_wam_repro_full10k_20260619.log`	`403604E8A1C2F77041E5E1ED43B80AE0BDD2630911D8DB1645B6FB6D0F0443EC`	`MAX_ITER=10000`、step 10000 loss、checkpoint path、`Done with training`、`EXIT:0`
`C:\Users\mousongzhe\Project\report\cosmos_wam_repro\remote_probe_summary.txt`	`0005153BD899CC16BC7769B47C01B3AA201AAC22BE762424E48222A181C17BC2`	checkpoint 完成摘要、loss JSONL 已回传、CoffeePressButton `0/1` 与 eval 输出路径
`C:\Users\mousongzhe\Project\report\cosmos_wam_repro\cosmos_wam_repro_report.html`	`12BE6C510EC5AF583B571B80B92FC98EB7F63217B543F7CD62CA05B9DF3075D6`	report 对 full10k、loss breakdown、eval smoke 的结构化汇总
`C:\Users\mousongzhe\Project\report\cosmos_wam_repro\loss_breakdown_full10k.jsonl`	`B6CEB4459C8CB5A00AADF3AE7F00F440DC359111B143FE44D5AF21CCB639BDDE`	full10k loss JSONL；report 说明每 iteration 有两个 callback 副本并按 iteration 去重

3.2 可复核的训练指标

- run name：`cosmos_policy_wam_repro_full10k_20260619`。
- `MAX_ITER=10000`、`BATCH_SIZE=1`、`NUM_WORKERS=1`，日志指向远端 `/data/msz/wm/outputs/...`。
- step `10000`：训练日志显示 `Loss: 0.0543`；report 同时给出 full10k 首末与平均分支诊断：demo action MSE `0.677 → 0.026`、world image MSE `0.787 → 0.159`、value MSE `0.707 → 0.003`，全程平均约 `0.087/0.128/0.069`。
- `Saving last checkpoint file iter_000010000` 后记录 `Saved checkpoint to .../checkpoints/iter_000010000`、`Done with training`、`EXIT:0`。

这些数字是 full10k 这条 run 的训练/诊断证据，不是 8-GPU 1 epoch gate 的最终 loss，也不是闭环成功率。

4. Full10k eval：从 1×1 smoke 到 24×5 formal audit

`remote_probe_summary.txt` 与 report 一致登记：CoffeePressButton、1 episode、`max_steps_override=120`、success `0/1`，并回传 `ENV_EVAL-CoffeePressButton-full10k-official-eval-smoke.txt`、`full10k_rollout.mp4` 与 `full10k_with_future_img.mp4`。后续 Full10k Trained Eval Po Audit 对 `Project/report/cosmos_policy_eval/remote_artifacts` 做了独立只读审计：24 unique tasks、120 episodes、24/24 formal `exit_code=0`；24 个 task log 重算为 raw `2/120`；两条 raw success 都是 CoffeeServeMug episode 1/2 的 `SUCCESS:length=1`，按当前本地 policy 排除后为 trusted `0/118`。

合并这两层证据后，可以支持四件事：

1. full10k checkpoint reference 能进入一次最小 RoboCasa eval smoke；
2. 同一远端 `iter_000010000` path 能进入 24×5 formal eval，status/log/summary task set 与 denominator 对齐；
3. 24 个 action-debug JSONL 共 4066 行，全部 `32×7`、finite，并指向同一远端 checkpoint path；
4. raw `2/120` 与 local-policy trusted `0/118` 两种口径可独立重算，且 denominator 变化有明确的两条 length=1 artifact 记录。

它仍不支持：把 `0/118` 称为 RoboCasa 官方统一 scorer 的 benchmark、与 official-HF `84/118` 做严格因果差值、full10k 相对 base 的 uplift、checkpoint binary/hash 与 fresh-process reload 的本地重算，或个人独立 ownership。目录中的 14 个本地 media 也不是完整 120 episode bundle。

5. 与 1 epoch gate 的关系

`vol_04_cosmos_policy_1epoch_evidence_freeze` 仍然正确描述 8-GPU `9907`-step gate 的 OOM 与保存接口失败；本页只是补充另一条独立的 full10k repro。两者不能用字符串“full10k”或“iter_000010000”直接 join，因为：

- run name、日期、目录和 batch/硬件不同；
- 1 epoch gate 的远端输出位于 `cosmos_policy_eval/muxi_deepspeed_1epoch_20260623`，full10k 位于 `cosmos_wam_repro`；
- 没有共同的 checkpoint hash、config SHA、dataset revision、normalizer/statistics manifest 和同一 eval denominator。

推荐的 interview phrasing 是：

> “我有两条需要分开讲的证据：一条是 8 卡、global batch 128 的 9907-step 单 epoch gate，定位了 dataloader cgroup OOM 和 state_dict 保存兼容性；另一条是独立的 full10k WAM repro，日志显示 step 10000 loss 0.0543、远端 checkpoint 保存成功，先在 CoffeePressButton 1×1 smoke 中得到 0/1，后续同一路径的 24×5 formal eval 重算为 raw 2/120、按当前本地 policy trusted 0/118。后者证明训练和评测链路可运行并可审计，不等于官方 benchmark 或 uplift，也不能替代前一条 gate 的 save/load 闭环。”

6. 仍未闭合的 Po 字段

字段	full10k 当前状态	下一动作
checkpoint binary/hash	只有远端 path 与 saved log；本地未保存 binary/hash	回传不可变 manifest 与 SHA-256
reload smoke	当前 report 未给出新进程 load parity	1-step save→销毁进程→load→fixed-input parity
eval denominator	24×5 status/log/summary 已对齐，raw `2/120`、local-policy trusted `0/118`；完整 episode JSONL 与 scorer version 仍缺	固定 scorer/invalid policy、termination/error 与120条 raw episode records
config/data/normalizer SHA	report 有脚本和摘要，但未形成 run-specific immutable manifest	记录 code/config/data/stats revision
ownership	report/日志存在不等于候选人逐文件 ownership	按 WAM ownership ledger 逐项确认

7. 结论与禁止升级

当前可以升级的项目叙事是“完成了一条独立 full10k 训练—1×1 smoke—24×5 formal eval 的审计链，并保留了 8-GPU 1 epoch gate 的系统排障链”。仍禁止写成：

1. “我完成并严格复现了 8-GPU 9907-step 1 epoch，并得到 full10k 的 loss/成功率”；
2. “Cosmos Policy full10k 在 RoboCasa 官方 benchmark 成功率为 0%”——当前只能说 raw `2/120`、按本地 policy trusted `0/118`；
3. “训练 loss 下降导致 benchmark uplift”；
4. “远端 checkpoint path 就是本地已验证权重”；
5. “报告中的脚本、日志或 Git author 自动等于个人独立实现”。

References

1. `C:\Users\mousongzhe\Project\report\cosmos_wam_repro\cosmos_wam_repro_report.html`（本地 report，full10k 与 eval smoke 汇总）。
2. `C:\Users\mousongzhe\Project\report\cosmos_wam_repro\remote_probe_summary.txt`（full10k checkpoint/eval 摘要）。
3. `C:\Users\mousongzhe\Project\report\cosmos_wam_repro\cosmos_policy_wam_repro_full10k_20260619.log`（原始训练日志）。
4. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623`（8-GPU 1 epoch gate source freeze）。
5. Cosmos Policy 1 Epoch Evidence Freeze。
6. WAM Claim–Evidence Inventory。
7. Cosmos Policy full10k Trained Eval P0 Audit。

Part 7 / 12 · `interview_materials/vla/sources/vol_04_cosmos_policy_wam_independent_fact_review.html` · SHA-256 `55118A9EB36991EDC760401E2A3021520F2AB9884970EA0B04F91278624FB094`

返回本卷目录

Canonical Markdown SHA-256：`55118a9eb36991edc760401e2a3021520f2ab9884970ea0b04f91278624fb094`；生成自：`interview_materials\vla\sources\vol_04_cosmos_policy_wam_independent_fact_review.md`

VOL 04 Audit C：Cosmos Policy / WAM 独立事实复核

状态：`independent_fact_reviewed / Audit C / public_facts_reconciled / PASS WITH WARNING` 复核日期：2026-07-20 证据截止：2026-07-20 范围：重新获取 arXiv 论文、固定 GitHub commit、固定 Hugging Face revision 与本地 WAM diagnostic artifacts；复核 slot/layout、routing、condition/target/metric mask、EDM loss、sampler、direct/planning、LIBERO/RoboCasa protocol、license 和个人项目边界。 Bounded slice：本页只完成 Audit C 独立复核及必要状态同步；不新增知识节点或题卡，不启动 PER-019，不修改 269 张题卡的 `unseen` 状态，不创建 `coaching_state.md`。

1. 结论先行

原 Cosmos Policy / WAM Claim-to-Code Audit 的主要公开事实成立：统一 latent-video sequence、policy/world/value 三角色、约 50/25/25 base routing、action/value latent injection、condition 与 target mask 分层、noise-weighted latent EDM loss、direct parallel generation、dual-checkpoint best-of-N planning、LIBERO 16×7 与 RoboCasa 32×7→执行16步→7D扩展12D、官方 98.5%/67.1% protocol 均可在重新获取的 paper/code/model card 中定位。

本轮 verdict 为 **PASS WITH WARNING**，而不是无条件 PASS，原因有四类：

1. `mask_value_prediction_loss_for_policy_prediction` 会构造 `final_mask_B_T`，但最终把 mask 乘到 `kendall_loss` 的条件没有包含该 flag；若它单独开启，配置名不等于 mask 真正生效。
2. 两个约 3.91 GB policy checkpoint、T5 pickle 与 base checkpoint 未下载或加载；本轮只冻结 policy revision、远端对象 ETag/大小、两份 config 与 statistics 的本地 SHA。
3. 本地 CoffeePressButton base/trained 两侧均为 `success=false` 的单 episode diagnostic；checkpoint 仍只是远端路径，不能证明 binary identity、模型 uplift、完整 0-success benchmark 或历史成功率。
4. 候选人的逐文件设计、编写、运行、验收角色仍未确认；上游 latent layout、routing、planning 与官方指标不得写成个人原创或个人结果。

另有两项边界被本轮补强：base `Cosmos-Predict2-2B-Video2World` 固定 revision 的模型卡明确是 NVIDIA Open Model License，而两份 policy model card 是 NSCLv1；官方 LIBERO/RoboCasa shape 下 latent volume 12544 恰好分别被 $16 \times 7 = 112$ 和 $32 \times 7 = 224$ 整除，因此这两种 shape 没有不完整 action tail，通用实现仍需保留整除/尾部测试。

2. 独立来源冻结

层	本轮重新获取	Identity	结果与边界
primary paper	`cosmos_policy.pdf`	arXiv `2601.16163v1`；21,715,508 bytes；SHA-256 `05696475C9B5481AFBF88673A65F80A46CF3FB87A5755C1410E83D0F468CC6E0`；22页、0空白页	best-of-N、rollout refinement、dual deployment、98.5%/67.1%、0.61s/0.16s、6000/3600 trials 可定位；不支持本地 WAM 结果
official repository	GitHub recursive tree + 13 files	`NVlabs/cosmos-policy@18a2accadf4e7a3531e56754102af5a24d2316da`；tree 664 entries、`truncated=false`	13/13 下载文件的 computed Git blob 与 tree blob 相同；repo 根 license 为 Apache-2.0
LIBERO policy model	API、README、config、statistics、8 个 remote HEAD	revision `cb689ec0e3347c13667d70a78a3447388f5c3bb8`	revision exact match；6 siblings；model card 为 NSCLv1；checkpoint 未下载
RoboCasa policy model	API、README、config、statistics、8 个 remote HEAD	revision `4b2a04c80d97202f86127ebec80461e8016ec1dc`	revision exact match；6 siblings；model card 为 NSCLv1；checkpoint 未下载
base model	API + README	revision `f50c09f5d8ab133a90cac3f4886a6471e9ba3f18`	revision exact match；`gated=auto`；NVIDIA Open Model License；本轮未下载权重
local WAM	manifests、traces、videos、log、offline metric	`C:\Users\mousongzhe\Project\wm\outputs`	9/9 指定文件 SHA 重新命中；仅支持 local-direct diagnostic 与 path-reference bridge

2.1 Fixed-commit 代表文件

文件	bytes	SHA-256	Git blob
`LICENSE`	11,356	`7382B1CF711E7A7C7AF9816E0E07B49B91E7149B7C6D347225F98800FCB1F1C2`	`4c9ad980682246bd6ab0d2bae82232be6dbdcbd4`
`README.md`	5,933	`D2ABD5F64EB132B6EDDAAF3E4CCD8C6EE6C905AA6F1810B004CAE32C88228CA5`	`f7e8dfb762ac595c4612343f88352351936d5a81`
`LIBERO.md`	6,591	`827B6FC57913E0D54702904AAB193F7410D6534BCB90DC39DB23E0294DF0FE77`	`6640360c6b4d0fe5cedf8418212ba0946e0f7616`
`ROBOCASA.md`	6,977	`97C518597D3D8BEC56C66161BC01FA1DE04EA2CB7C8D55B5F63057C062E24A27`	`3d816e64b8a5dc0f44dfb47655430c6b5dd093bd`
`models/policy_text2world_model.py`	45,535	`D2D6AF812E2241FA0D3D8C68620447BD0F23D236412ADA9BAD5C849DF5B848F5`	`efbafee498b91e3865b135599179949e159f8c75`
`models/policy_video2world_model.py`	48,775	`4F5314FDEA8F3F05D210F99923CA07E3C05FE9E0756B71F77B201F4B52E460EB`	`625bd0aeb1c4fb0ceee20a3b7404f1342ea1ff4d`
`modules/cosmos_sampler.py`	5,621	`D428B578159D7D2EFDC962A16FA168A627E831BD6D96EBEC4D8C8C4987156D82`	`ef9e42cc321942a7bfbff008dbe44cb700f15617`
`experiments/robot/cosmos_utils.py`	109,448	`A95C74F33F519F4C0CE048C3BA2694989D3A7C431BF0F324A274413CDE4C0B47`	`00c2845bc9f45e4ef4b333e5e88897b59df142db`
`datasets/libero_dataset.py`	44,590	`CBEBD356916DE3F7FC6BC084A6F719B6A8A58A9EF496F7F104AE5980F5F177E8`	`200aecfefe022d2768b85b8a35b4fb9c7706a8c8`
`datasets/robocasa_dataset.py`	58,913	`966D8496A98FCBADCE31C6EEC397BAE5439F461D85789B828EEF866695ABE893`	`63d21e4f3fc7405ac2a7bc20ab46188373c6f9e8`

文件	bytes	SHA-256	Git blob
`experiments/robot/libero/run_libero_eval.py`	46,874	`4A103710D9057E484B415DA46D5B40C054ABEB736E751AC80AD5A879C5B4AC10`	`8ced47fd42f3c1be0707dca874e18c2707a4a3a8`
`experiments/robot/robocasa/run_robocasa_eval.py`	58,807	`32AA3B78FF33C9D159A91BBA880953F467740F611D756A2B48223DE80D8E59D7`	`982df9b724e97648a7b7d46bc8678aab2833ec3f`
`config/experiment/cosmos_policy_experiment_configs.py`	20,835	`16B8C0D1E2112EBF09CC5393F3F38F07CBC82B9EE2844AD79F35FD6532A566FB`	`ba4f98edcb9ed9f9ee0c9985058c71de3c56528a`

这张表证明下载内容与 fixed Git tree 相同；它不证明依赖、dataset、checkpoint 或本地 fork 与该 commit 等价。

3. Slot / Layout / Injection 复核

config 与 eval source 分别明确记录：LIBERO `state_t=9`，顺序为 blank/current proprio/wrist/primary/action/future proprio/future wrist/future primary/value；RoboCasa `state_t=11`，额外包含左右两路 third-person view。两份固定 model config 又分别记录 action `16×7` 与 `32×7`，proprio 均为 9D。[2][3][4]

令 action chunk 为 $A \in [-1,1]^{K\times D_a}$，单个 latent frame 的元素数为 $M = C'H'W' = 16\times 28\times 28 = 12544$，action 元素数为 $L = KD_a$。注入实现为：

$$z_a = operatornamereshape\left(\mathrm{repeat}(\mathrm{vec}(A), \lceil M/L \rceil)_{1:M}\right).$$

变量解释：K 是 chunk horizon，D_a 是 action 维数；实现映射到 `replace_latent_with_action_chunk` 的 flatten、ceiling repeat、slice 与 overwrite。它没有 action-specific learned encoder，不能描述成独立 action head。

提取只使用完整 copies，并跨 copies 求均值：

$$\hat{A} = \frac{1}{q}\sum_{j=1}^{q} \mathrm{reshape}(z_a^{(j)}, K, D_a), qquadq = \left\lfloor \frac{M}{KD_a} \right\rfloor.$$

变量解释：LIBERO 的 $q = 112$、RoboCasa 的 $q = 56$，两者 tail 都为 0；独立 round-trip probe 的最大浮点误差分别约 2.44×10^{-15} 与 1.22×10^{-15}。若更换 K, D_a, C', H', W'，仍需检查 `num_action_elements <= latent_elements`、整除性与尾部语义。

value frame 的提取是对整帧求均值，调用侧再按 (operatorname{clip}((v+1)/2,0,1)) 映射到 ([0,1])。probe 对输入 ([-0.8,0.2,1.0]) 得到 ([0.1,0.6,1.0])。这验证编码/解码算术，不验证 value calibration 或 success probability。

4. Routing、Condition Mask 与 Loss

4.1 角色概率

base config 对 demo 与 rollout 的 sampling probability 都设为 0.5；rollout 内 LIBERO 固定 `p_world_model=0.5`，RoboCasa 默认 `p_world_model=0.5`。因此 recipe-level 期望比例为：

$$P(\text{policy}) = 0.5, qquad P(\text{world}) = 0.5 \times 0.5 = 0.25, qquad P(\text{value}) = 0.5 \times 0.5 = 0.25.$$

变量解释：这是 epoch structure 与随机 role selection 的期望，不是某个实际 batch 的精确计数；runtime 必须记录 realized ratio。rollout refinement 的 ALOHA planning recipe 则是 10% policy、其余 90%在world/value间均分，不能与 base 50/25/25混写。[1][2]

4.2 Condition 与 target 的分层

`get_data_and_condition` 对 world sample 把 action slot 设为 clean condition；对 value sample 先把全部 frames 设为 clean，再把 value slot 置为需生成。V/Q variant 分别 mask current/action 或 future state。这里 `condition_video_input_mask=1` 表示 clean condition，不能当成 attention mask 或 loss mask。[2]

主训练路径先注入 action、proprio、future proprio 与 value，采样 x_t，denoiser 输出 $\hat{x}_0$，再形成 noise-level weighted latent MSE：

$$\mathcal{L}_{\text{EDM}} = \frac{1}{BCTHW} \sum_{b,c,t,h,w} m_{b,t}\, w(\sigma_{b,t}) \left(x_{0,bcthw} - \hat{x}_{0,bcthw}\right)^2.$$

变量解释：$m_{b,t}$ 是按 latent frame 广播的 target mask/multiplier，$w(\sigma)$ 是 noise-level weight。实现先构造 per-element `edm_loss_B_C_T_H_W`，再按条件乘 `final_mask_B_T`；action/future/value L1/MSE 是 component diagnostics，不等于主反向目标。

4.3 已确认的 flag 风险

`mask_value_prediction_loss_for_policy_prediction=True` 时，代码在 490–539 行构造并更新 `final_mask_B_T`；但 555–560 行的 apply-mask 条件只包含：

- `mask_loss_for_action_future_state_prediction`；
- `mask_current_state_action_for_value_prediction`；
- `mask_future_state_for_qvalue_prediction`；
- `action_loss_multiplier != 1`。

它没有列入 `mask_value_prediction_loss_for_policy_prediction`。所以该 flag 若单独为 true，构造的 mask 不会因它自身进入最终乘法。当前默认 config 两个 target-mask flags 都为 false，本问题不能据此

否定官方 checkpoint；个人配置若开启该 flag，必须做 masked/unmasked loss 与 gradient probe。

5. Sampler、Direct Policy 与 Planning

`CosmosPolicySampler.forward` 在请求 steps 大于1时先减1个 solver NFE，随后补一次 clean denoise；请求1步时直接在首 sigma 调 denoiser。因此请求5步的合同是 `4 solver NFE + 1 clean denoise`，不是“五次相同 solver loop”。10/10 deterministic probes 中对此计算已单独通过。[2]

direct eval 的 `get_action` 可在一次 generation 中并行产生 action/future/value，但 action selection 不依赖 future/value。planning path 则是：

```
def audited_best_of_n(observation, task, n):
    # Policy checkpoint proposes N action chunks with distinct seeds.
    proposals = [policy_get_action(observation, task, seed=i) for i in range(n)]

    # A separately loaded planning checkpoint conditions on each proposal.
    futures = [planning_get_future(observation, action) for action in proposals]

    # Value predictions rank proposals; only the selected execution prefix reaches the controller.
    scores = [aggregate_value(planning_get_value(future)) for future in futures]
    return proposals[argmax(scores)][:num_open_loop_steps]
```

代码与论文都支持 action→future→value→argmax 的顺序，且 `get_planning_model` 与主 model 分开加载。论文 ALOHA 实验使用 (N=8)、每 action 3 个 future samples、每 future 5 个 value samples与8张 H100；repo 的默认 LIBERO/RoboCasa runtime 则是 `N=1`、ensemble关闭、aggregation=`average`。论文设定不能冒充所有默认配置。[1][2]

best-of-N 可写为：

$$a^* = \arg\max_{a^{(i)} \sim \pi(\cdot|s)} \widehat{V}\left(\widehat{T}(s, a^{(i)})\right).$$

变量解释：(pi) 是 proposal policy，(widehat T,widehat V) 属于 refined planning model；它是排序 policy proposals，不是 CEM/MPPI 的迭代分布更新。候选越多越可能放大 world/value 过估计，因此 N、rank correlation、selected-vs-random、latency 与 failure trace 必须一起报告。

6. LIBERO / RoboCasa 与官方数字

合同项	LIBERO	RoboCasa	独立结论
model output	16×7 action；9D proprio；两路 camera	32×7 action；9D proprio；三路 camera	config、card 与 code 一致
execution	default open-loop 16；deque maxlen=16	先 slice `actions[:16]` 再入 queue	“chunk horizon”与“执行 prefix”分开
reset/warmup	official init state；10个 dummy actions	每 episode 重建 env；按 episode 每10条切 scene；10个 zero actions	不能只报 task 名

合同项	LIBERO	RoboCasa	独立结论
controller	`env.step(action.tolist())`	PandaMobile + OSC_POSE；7D 后拼 `[0,0,0,0,-1]` 成12D	7→12 probe 通过
success	eval wrapper 的 `done`	`env._check_success()`	外部 benchmark/version 仍需独立冻结
official denominator	4 suites×10 tasks×50 episodes×3 seeds = 6000	24 tasks×50 trials×3 seeds = 3600	两项 denominator probe 通过
official score	98.5%	67.1%	都是上游 checkpoint/protocol 声明，不是用户 WAM 结果

论文还报告同一H100上5-step并行 action/future/value 约0.61s/chunk、1-step约0.16s/chunk；这些 wall-clock 只绑定指定硬件与实现，不能写成本机或本地部署 latency。[1]

7. Model Asset 与许可证边界

Model	revision	本轮本地下载	remote-only object	License
LIBERO policy	`cb689ec...`	`config.json` SHA `6C246E65...2C31F43`；statistics SHA `5B119A98...C580D3`；README SHA `3BD88247...405506`	checkpoint 3,913,017,345 bytes，ETag `c86c9427...fa8d5`；T5 41,957,938 bytes，ETag `f4e95822...ef05b`	NSCLv1，来自 model card正文
RoboCasa policy	`4b2a04c...`	`config.json` SHA `B1690DC2...73521`；statistics SHA `5B9882C8...6DB87`；README SHA `1DC2D061...F7AF9`	checkpoint 3,913,017,345 bytes，ETag `5abf2715...d2992`；T5 463,599,495 bytes，ETag `2feda9de...9fceb`	NSCLv1，来自 model card正文
base Predict2	`f50c09f...`	API + README；README SHA `738F95DB...DFFBE`	26 siblings，`gated=auto`，权重未下载	NVIDIA Open Model License

ETag 是固定 revision 下的远端对象 identity，不等于本轮已本地读取并复算的 binary SHA。严格运行 manifest 仍需 checkpoint/config/statistics/T5 的本地或不可变远端 identity，并避免 `snapshot_download` 不传 revision 时解析未来 HEAD。

论文 CC BY 4.0、GitHub code Apache-2.0、base model NVIDIA Open Model License、policy checkpoint NSCLv1 必须分列；任何一层的 license 都不能自动覆盖其他 artifact。

8. 本地 WAM 再哈希与 Truth Boundary

Local artifact	重新命中	直接支持	仍不能支持
base manifest	`63A51867...BD33A86`，3,901 bytes	CoffeePressButton、episode0、seed195/env_seed0、300 steps、open-loop16、denoise5、`success=false`	base checkpoint binary 已验证
trained manifest	`809AE4F8...EF55CC`，3,918 bytes	同 protocol 字段、`success=false`、远端 iter10000 path	trained checkpoint binary 已验证
base/trained traces	`C6A3623B...88525D` / `9C6A4F00...82CF2D`	各300×7 policy、300×12 env、finite、success true count均0、reward sum均0	目标动作误差、模型好坏、完整成功率

Local artifact	重新命中	直接支持	仍不能支持
base/trained videos	`D469A8B2...2F388` / `59176353...2877E`	两条失败 episode 文件存在	matched suite denominator、root cause
rollout log	`19448961...27143`	action chunk/dim/runtime线索	checkpoint与结果 ownership
offline manifest/metric	`8C4872F7...EC71` / `C9AB7F04...EC9D`	offline metric 与 trained rollout checkpoint path 逐字相同；12 future frames	binary identity、matched episode/seed/horizon、proxy-success 因果

独立解析两条 action trace 得到：rows 均为300，(t) 序列完全相同，policy shape均为300×7，env shape均为300×12，无 NaN/Inf，逐步 `success_after_step=True` 计数均为0，reward sum均为0；两条 policy trace 之间 MAE 为 `0.3100783610894134`，RMSE 为 `0.3906446458988869`。这是 trace-to-trace 几何差异，不是 prediction-to-target error。

两份 manifest 的 checkpoint 字段仍为 `/data/msz/...` 路径。offline `iter_000010000/0000_metrics.json` 与 trained rollout manifest 的路径字符串 exact match，只能建立 checkpoint-reference bridge；offline 输入是成功 regen-demo、seed7、4 denoise steps与12 future frames，rollout 是另一 environment episode、policy seed195/env seed0、5 action steps与300 env steps，不能做相关或因果推断。

8.1 当前安全口述

> “我把 Cosmos/WAM 拆成上游统一 latent contract、训练 routing/mask、RoboCasa controller protocol 与本地诊断四层。上游 slot、50/25/25 routing、direct/planning 和公开分数不是我的原创或本地结果。本地能复核到一组 CoffeePressButton base/trained manifest、300×7→300×12 action traces 与失败视频，但两侧都失败，checkpoint 仍是远端路径，只有 path fingerprint。所以我把它用于接口与失败诊断，不把 trace 差异写成 uplift，也不恢复历史 MSE/成功率。”

候选人的实现/运行/验收 ownership 仍应按 WAM Ownership Ledger 逐项回答；在真实确认前，只能用“本地快照包含”“材料记录”“我按上游合同梳理/对齐”等受限动词。

9. 十个独立 Deterministic Probes

Probe	结果
LIBERO 16×7 action inject→extract	PASS；112完整copies、tailo
RoboCasa 32×7 action inject→extract	PASS；56完整copies、tailo
value frame mean + $[-1,1] \to [0,1]$	PASS；`[-0.8,0.2,1.0]→[0.1,0.6,1.0]`
base recipe role algebra	PASS；`0.5/0.25/0.25`
RoboCasa statistics normalize→denormalize	PASS；max error约 `1.11e-16`
RoboCasa 7D→12D controller append	PASS；尾部 `[0,0,0,0,-1]`
LIBERO official denominator	PASS；6000
RoboCasa official denominator	PASS；3600

Probe	结果
requested 5-step sampler contract	PASS；4 solver NFE + 1 clean denoise
best-of-N selector	PASS；`argmax([0.1,0.8,0.3])=1`

10/10 probes 只验证确定性算术与接口合同，不代替 GPU inference、checkpoint load、environment rollout、value calibration 或候选人 ownership。

10. Source → Knowledge → Card Reconciliation

Claim cluster	Source Audit C	WM-025 Knowledge	VLA-WM-025 Card	Verdict / correction
unified latent sequence	明确不是三个独立head	先纠正“三个head”	30秒回答一致	PASS
layout/injection	9/11 slots、16×7/32×7、repeat/mean	action/future/value slot分开	mechanism answer一致	PASS；补充官方shape tail=0
routing	base约50/25/25	sample routing与mask分层	诊断要求打印role	PASS；实际batch需记录realized ratio
condition/target/metric mask	代码路径与flag边界	四类开关明确	易错点明确	PASS WITH WARNING；保留value-aux apply-mask flag风险
EDM/component metrics	noise-weighted latent MSE	主loss不等于component L1	30秒与诊断一致	PASS
direct/planning	parallel direct与dual-checkpoint planning分开	卡片聚焦WM-025，不扩写planning	不混称value=planning	PASS
benchmark/license	protocol与四层license分开	项目边界不引用官方结果	不把公开结构写成个人设计	PASS WITH WARNING；base license现已补核
local project	raw checkpoint/result/ownership不足	CoffeePressButton仅partial	保持受限安全口述	PASS WITH WARNING；两侧失败、路径不是binary hash

不需要重写 VLA-WM-025 的30秒答案；面试时应追加两个版本限定：“官方 LIBERO/RoboCasa shape 恰好无 action tail，但换 shape 要重测”；“value-aux mask flag 单独开启时必须查 final apply branch 与 gradient”。题卡继续 `unseen`。

11. 审查状态与下一步

Audit C 的 paper/code/model-card public facts 已完成独立再获取、Git blob identity、10 probes、local rehash 与 source→knowledge→card reconciliation，可从 `independent_review_pending` 提升为 `independent_fact_reviewed / PASS WITH WARNING`。这不改变 VOL 04 的整卷 `WARN`，也不表示本地 WAM run 或个人 ownership 已通过。

仍开放：

- 下载/固定 policy/base checkpoint 与 T5 object 仅在许可证、空间和真实复现需要时进行，并建立 immutable manifest；

- 冻结 modified LIBERO/RoboCasa dataset revisions 与数据条款；
- 对 external benchmark commit、reset/success semantics 做独立版本核对；
- 对 value-aux mask flag 跑真实 tensor loss/gradient test；
- 补 local checkpoint binary/hash、processor/normalizer/config、完整 episode denominator与同协议 paired rerun；
- 由候选人逐项确认 WAM 的设计、编写、运行、审查和工具/协作者边界；
- 真实口述前不改变 mastery，不创建 coaching state。

下一 bounded slice 优先进行一题真实口述；若继续文档复核，则只选择 VOL 04 Audit D Sim2Real，不启动 PER-019。

References

1. Kim et al., Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning, arXiv v1, 2026。
2. NVlabs, cosmos-policy@18a2acc，重点复核 model、dataset、sampler、config 与 eval paths。
3. NVIDIA, Cosmos-Policy-LIBERO-Predict2-2B@cb689ec。
4. NVIDIA, Cosmos-Policy-RoboCasa-Predict2-2B@4b2a04c。
5. NVIDIA, Cosmos-Predict2-2B-Video2World@f50c09f。
6. Audit C Claim-to-Code 与 WM-025 Source Freeze。
7. WM-025 Knowledge 与 VLA-WM-025 Atomic Card。
8. WAM Claim–Evidence Inventory、WAM Ownership Ledger、Artifact Manifest、Action-Trace Diagnostic 与 Matched-Checkpoint Bridge。

Part 8 / 12 · `interview_materials/vla/sources/vol_04_sim2real_independent_fact_review.html` · SHA-256 `278F1B69BD8F414DDB597AE848AE4D97517C1C2F07355601959D4E7CC8D45136`

返回本卷目录

Canonical Markdown SHA-256：`278f1b69bd8f414ddb597ae848ae4d97517c1c2f07355601959d4e7cc8d45136`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_sim2real_independent_fact_review.md`

VOL 04 Audit D：Sim2Real 独立事实复核

状态：`independent_fact_reviewed / Audit D / public_facts_reconciled / PASS WITH WARNING` 复核日期：2026-07-20 证据截止：2026-07-20 范围：重新获取 10 份固定 arXiv PDF，固定 MuJoCo / OpenCV / robosuite 的官方源码与文档快照，复核 visual/sensor、dynamics、control/timing、contact/compliance、task/protocol、safety/recovery 六层 reality gap，以及 DR、SysID、SimOpt、UP-OSI、RMA、residual、chance constraint、recovery 与五项目真实性边界。 Bounded slice：本页只完成 Audit D 独立复核与必要状态同步；不新增知识节点或题卡，不启动 PER-019，不修改 269 张题卡的 `unseen` 状态，不创建 `coaching_state.md`。

1. 结论先行

原 Sim2Real Claim-to-Mechanism Audit D 的主干事实成立：Sim2Real 不能缩成“加 domain randomization”，至少要把视觉/传感器、动力学、控制/时序、接触/柔顺、任务/协议与安全/恢复分成六个可测合同。Domain randomization（DR）不是越宽越好；System identification（SysID）与 DR 不是二选一；SimOpt 更新的是能解释 real/sim behavior discrepancy 的 simulator-parameter distribution；UP-OSI 显式估计 dynamics parameter，RMA 估计 task-relevant latent，二者都不等于 rollout 内更新 policy weights；residual policy、chance constraint 与 learned recovery 也都带有前提，不能包装成通用稳定性或零风险保证。[1]–[13]

本轮 verdict 为 **PASS WITH WARNING**，不是无条件 PASS，原因有五类：

1. 六层 taxonomy、range provenance、latency decomposition、matched test、queue flush 与 layered safety state machine 中有工程综合；它们与一手来源一致，但不是某篇论文原封不动的定理或标准。
2. MuJoCo `stable` HTML 只固定了响应内容；页面元数据没有给出可验证的 build commit。本轮另固定 `3.10.0@28009f9` 的 RST 源码，但不声称 `stable` HTML 与该 tag 是同一 build。
3. OpenCV 的 reprojection error 只衡量标定点拟合；它不自动证明部署时 hand-eye extrinsics、mounting、timestamp、resize/crop 与 action frame 正确。[12]
4. residual policy 不自动继承 base controller 的稳定性；未校准 ensemble/disagreement 只能叫 risk heuristic；Recovery RL 是 learned shield，不是 zero-risk theorem。[7]–[9]

5. 五项目目前只有 simulator、offline data、接口、诊断与高层取消/补偿证据；没有形成“真机 calibration + latency/contact measurement + controller/limits + real rollout + incident/E-stop + ownership”的协议完整 bundle。

因此，安全口述仍是“我能按六层合同设计 Sim2Real 诊断与实验矩阵，并在本地 simulator/接口材料上做证据审计”，不能说“已完成真机 Sim2Real 部署”“已证明安全”或把公开方法写成个人原创。

2. 独立来源冻结

2.1 十份固定 PDF

本轮从固定 arXiv revision 重新下载 10 份 PDF，使用本地 bytes 计算 SHA-256，并逐页抽取文本检查纯空白页；合计 **124 页、0 个纯空白页**。SimOpt 第 3 页、Minitaur 第 5 页与 RMA 第 2 页另经 PNG 视觉抽查，双栏顺序、公式、图注和正文均可读。

Paper	固定 revision	bytes	pages / blank	SHA-256
UP-OSI	`1702.02453v3`	1,012,357	10 / 0	`F4ABCAA147175A471B6FA217758AECD80D42DF11F6F19B70A7DFBA66021D24CA`
Domain Randomization	`1703.06907v1`	4,066,798	8 / 0	`3FC98C5F4CEA050686D45858E647E1B704492121FDAE80F8FCD5D560C107F11F`
Cautious MPC	`1705.10702v4`	1,817,694	12 / 0	`2A38976CA6B6ADFB6DD9CF4F85AB9039E46666C2DD263F887E7F54DDCBD0BC36`
Dynamics Randomization	`1710.06537v3`	2,845,918	8 / 0	`A15CB24630A11218865027024AE2353F80F46D40371B42DCCB2B24E9D00D62D8`
Minitaur Sim2Real	`1804.10332v2`	1,062,958	11 / 0	`455EA9154D6C5E905601F91E71EC9F8FD0A348E7BE639E1019FFE1D941B7FF76`
Dexterous In-Hand Manipulation	`1808.00177v5`	4,590,851	27 / 0	`DA6EC44DD0941D674EC58334FF630F83D7BD19E6062AB2837CCFA0B3B94F6F73`
SimOpt	`1810.05687v4`	5,492,359	10 / 0	`5DDE7AD391342C29CDCB80C3728389A59712E6798A21FC5A8F4B5D82E4BAF245`
Residual RL	`1812.03201v2`	2,959,124	8 / 0	`C6279120F9E0A799A9D7A943A5624A5738250E588F99C5FC8A9FAC28E2039641`
Recovery RL	`2010.15920v2`	7,929,814	15 / 0	`3404EF792695647E3900D0E36E608FCF23D261E75F45D16C277206064F2D9AF5`
RMA	`2107.04034v1`	16,534,863	15 / 0	`D8660DA881851756287EFAB24D85F5C629BA97878F70297EB429A8E8A9A57846`

这里只引用事实与方法边界，不复用论文图表或代码；PDF identity 不证明任何本地项目执行过对应方法。

2.2 MuJoCo、OpenCV 与 robosuite

来源	固定 identity	本轮重新获取	结论与边界
MuJoCo source	tag `3.10.0` → commit `28009f9105cd92784b7b0b30c0605a5e29107a77`	`modeling.rst` SHA `F13C1C15...41893` / blob `5492C407...DE4`；`XMLreference.rst` SHA `A6EA3CE3...FD6` / blob `72842F40...B86`；`computation/index.rst` SHA `19D8F088...DE4D` / blob `A0D890F1...3F6`	支持 contact、friction、`solref/solimp`、solver、timestep 与 sensor-delay 语义；不证明本地 simulator build
MuJoCo stable HTML	accessed 2026-07-20	Modeling `50523078...15DA`；Computation `94228BFD...3AC`；XML Reference `C816A678...229B`	响应 SHA 与既有 source freeze 一致；页面只标记 `stable`，不与 `3.10.0` build 等同
OpenCV calibration source	commit `f7dd7df170add1655de75b5e95f20f41d2a88293`	`py_calibration.markdown` SHA `72F48A26...03C4` / blob `423F46A2...D6DB`	支持 pinhole intrinsic/extrinsic、radial/tangential distortion 与 reprojection error；不证明部署标定正确
robosuite	tag `v1.5.1` → commit `51cc01785bab80ffeed20da15e67d7dd4140e76a`	controller RST SHA `843563FE...2314`；gripper base SHA `B4582D44...6CB`；Panda gripper SHA `DF0449A9...AAB8D`	支持 action split、policy/simulation frequency 与 Panda `-1=open, 1=closed`；不能外推到真实 Panda 或任意 LIBERO/RoboCasa fork
licenses	fixed repository files	MuJoCo/OpenCV LICENSE SHA `CFC7749B...23D30`；robosuite LICENSE SHA `177978CB...D9CBA`	MuJoCo/OpenCV 为 Apache-2.0，robosuite 为 MIT；论文、代码、模型与数据许可不能互相替代

GitHub 匿名 REST API 本轮命中共享出口限流，因此 tag 解析使用 `git ls-remote`，文件通过 immutable commit URL 获取并计算 Git blob；没有把 mutable branch HEAD 当作固定版本。

3. Paper-to-claim 独立复核

Claim	直接定位	独立结论	不能外推
visual DR factors 与 ablation	Tobin pp.1, 3, 5	camera、lighting、pose、texture、distractor 与 ablation 可定位	object localization / grasp demo 不证明任意 VLA transfer
95 个 dynamics parameters、timestep/latency、recurrent implicit SysID	Peng p.4	95 是特定 pushing setup；episode/step cadence 与 memory 边界成立	不能抄作默认参数表或保证 target support
real/sim trajectory discrepancy 与 KL-bounded distribution update	SimOpt p.3	SimOpt 更新 distribution，并明确说过宽 distribution 可能含 infeasible scenarios 或过度保守	不等于恢复唯一真实物理参数；需要安全 real rollout authority
actuator、latency、measurement + margin、robustness–optimality	Tan pp.1, 5, 7	延迟会缩小真实闭环稳定区；实测中心与 DR coverage 互补	Minitaur 数值与范围不能直接迁移到 manipulation
history → dynamics parameter → universal policy	UP-OSI pp.1–2	显式 OSI 估计有语义的 parameter context	论文实验主要在 simulation；不是 policy-weight update
privileged extrinsics latent、history adaptation、异步部署	RMA pp.1–3	训练期 privileged encoder 与部署期 10 Hz adaptation / 100 Hz base policy 可定位	legged latent 不自动适配 image VLA；stale latent 仍需协议
conventional/base controller + learned residual	Residual RL pp.1–3	真实 contact-rich assembly 支持 residual decomposition	论文没有给任意 residual 的通用稳定性保证
task policy / recovery policy 与 future-risk threshold	Recovery RL pp.1–3	learned risk critic 与 recovery switch 成立	empirical safety improvement 不等于 zero-violation theorem

Claim	直接定位	独立结论	不能外推
nominal dynamics + GP residual + chance constraint	Cautious MPC pp.1–3	uncertainty propagation 与概率约束成立	依赖 GP/model/distribution assumptions；不是任意 ensemble variance 即可
calibration + extensive randomization + memory	OpenAI pp.3–6	tendon/contact/sensor mismatch 与 recurrent adaptation 边界成立	Shadow Hand recipe 不直接推广到 Panda/gripper

两处原审计措辞需要继续保留限定语：SimOpt 是 **behavior/distribution adaptation**，不是 exact parameter recovery；Residual RL 的真实硬件结果是方法可行性证据，不是 stability inheritance 证明。

4. 六层 reality-gap 合同

用一个最小闭环把 sensing、dynamics、controller 与 delay 分开：

$$o_t^{\text{real}} = h_{\psi_r}\left(x_{t-\tau_s}^{\text{real}}\right) + \epsilon_t, \qquad x_{t+1}^{\text{real}} = f_{\xi_r}\left(x_t^{\text{real}}, \mathcal{C}_{\kappa_r}(a_{t-\tau_a})\right).$$

变量解释：h_{ψ_r} 是真实 sensor/image-formation chain，f_{ξ_r} 是 dynamics/contact，$\mathcal{C}_{\kappa_r}$ 是 low-level controller/actuator；τ_s, τ_a 分别是 sensing 与 action-path delay。实现映射：Audit D 的 trace 必须把 camera timestamp、policy、network、queue、controller apply 与下一帧关联。该式是工程抽象，不是某篇论文的原式。

层	主要 mismatch	first evidence	不能拿什么替代
Visual / sensor	intrinsics/extrinsics、distortion、exposure、blur、occlusion、timestamp、preprocess	calibration bundle、replay、factor ablation	只看 color jitter 或 sim image quality
Dynamics	mass、inertia、damping、motor strength、payload、drift	free-space tracking、SysID residual、parameter manifest	接触失败时盲调所有 mass/friction
Control / timing	controller type/frame/unit/gain、frequency、latency、ZOH、chunk queue	capture-to-apply trace、target/actual action、frequency sweep	只报一次 inference latency
Contact / compliance	sliding/torsional/rolling friction、softness、solver、timestep、pad/object deformation	press/slip/hold/lift micro-bench	单一 friction scalar
Task / protocol	reset/init/layout/object/language/horizon/success predicate	immutable protocol tuple、matched seeds/trials	同名 task 或 sim score
Safety / recovery	OOD、constraint、uncalibrated risk、stale action、collision	limit registry、shield/recovery/fallback/E-stop trace	action clipping 或训练 reward

六层是独立诊断维度，但不是互不作用：camera timestamp 会进入 timing，controller gain 会改变 contact，task reset 会改变 contact exposure。正确做法是记录 first divergence，再确定下一层 probe。

5. DR、SysID 与 SimOpt

5.1 “越宽越好”被一手来源否定

固定 simulator distribution $p_\phi(\xi)$ 时，policy 学到的是分布内期望：

$$\theta^*(\phi) = \arg\max_\theta \mathbb{E}_{\xi\sim p_\phi}\left[J(\pi_\theta; f_\xi)\right].$$

变量解释：ξ 是 simulator parameters，J 是 closed-loop return。实现映射：必须保存参数单位、support、相关性、采样 cadence、rejection 与 realized samples。SimOpt p.3 和 Tan p.5/p.7 都明确支持 coverage 与 nominal optimality 的取舍，所以“分布越宽越稳”是错误答案。[3][4]

5.2 SysID 给中心，DR 覆盖 residual uncertainty

可测且可辨识的 mass、latency、motor response 先用测量或 fit 定中心；population variation、wear、temperature、payload、不可辨识耦合与 model-form error 再用条件分布覆盖。fit trajectory 好只证明 behavior match，不证明每个 physical parameter 都是真值。

SimOpt 的 outer loop 可写为：

$$\phi_{k+1} = \arg\min_\phi \mathbb{E}_{\xi\sim p_\phi}\left[D\left(\tau^{\text{real}}, \tau^{\text{sim}}(\xi)\right)\right] \quad \text{s.t.} \quad D_{\text{KL}}(p_\phi \,\|\, p_{\phi_k}) \le \varepsilon.$$

变量解释：D 比较 real/sim observable trajectories，KL constraint 限制 distribution update。实现映射：这对应 SimOpt Algorithm 1 的 real rollout、sim rollout、cost 与 update distribution；它不要求返回唯一真实 ξ_r。本地若没有安全 real rollout、同步 observation 与 immutable checkpoint，就没有执行 SimOpt 的 authority。[3]

6. Camera、timing、contact 与 controller

6.1 Calibration error 不等于 deployment correctness

Pinhole projection 为：

$$s\begin{bmatrix}u\\v\\1\end{bmatrix} = K\,[R\mid t]\begin{bmatrix}X_w\\1\end{bmatrix}, \qquad K=\begin{bmatrix}f_x & 0 & c_x\\0 & f_y & c_y\\0 & 0 & 1\end{bmatrix}.$$

变量解释：K 是 intrinsics，$[R\mid t]$ 是 world-to-camera extrinsics，(u,v) 是 image coordinates。实现映射：OpenCV 还给出 radial/tangential distortion 与 reprojection error；部署还需 hand-eye frame、resize/crop 后的 K、mounting revision 和 timestamp。reprojection error 低不能替代这些 checks。[12]

6.2 Latency 必须拆 critical path

$$\tau_a = \tau_{\text{capture/transport}} + \tau_{\text{preprocess}} + \tau_{\text{policy}} + \tau_{\text{network}} + \tau_{\text{queue}} + \tau_{\text{controller/actuator}}.$$

变量解释：每项都应记录 distribution 与 tail，而不是只相加平均值。实现映射：chunked VLA 还要保存 observation version、action age、H、执行前缀 K_e、deadline、stale/drop/flush/ack。Tan 直接证明未建模 latency 会使真实 feedback oscillate；robosuite source 证明 policy reference 会在多次 simulation/controller step 中保持，但它只是 simulator 语义。[4][13]

6.3 Contact 不是一个 friction scalar

MuJoCo fixed source 明确说明：geom 的 tangential/torsional/rolling 三类 friction 会扩成 contact 的五个 coefficients；`solref`、`solimp`、priority/mixing、cone、solver、timestep 与 no-slip settings 都能改变结果。Panda gripper source 的 `-1=open, 1=closed` 只绑定 robosuite `v1.5.1`；真实 robot driver、另一个 simulator 或本地 adapter 必须重新做 sign/range/aperture/force/current round trip。[11][13]

因此 first-divergence 顺序应是：先 action/parser/controller free-space，再 actuator/latency，再接触触发、press/slip/hold/lift；不能用调 friction 掩盖 axis、unit 或 controller mismatch。

7. Adaptation、residual 与 safety 边界

方法	rollout/test-time 更新对象	policy weights 当场改变?	最小 authority	主要警告
Fixed DR	simulator sample	否	simulator only	support/range 可能错误
SimOpt	simulator distribution	否	少量安全 real trajectories	behavior metric shortcut、real cost
UP-OSI	explicit dynamics parameter context	否	history + sim-supervised parameter	identifiability / model assumption
RMA	task-relevant latent context	否	history + privileged sim teacher	stale/OOD latent、legged→VLA gap
Residual policy	executed action correction	residual weights通常离线训练	reliable base + bounded authority	不自动继承稳定性
DAgger / real fine-tune	dataset / model weights	训练轮次改变	expert/reward + safe collection + rollback	hardware/data cost、catastrophic forgetting

Residual composition 可写为：

$$u_t = \text{sat}\big(u_{\text{base}}(s_t) + \alpha\, u_{\text{res},\theta}(s_t, o_t)\big).$$

变量解释：α 与 saturation 只限制 residual authority，不构成稳定性证明。实现映射：需要 residual norm/rate/energy、base-only/residual-on paired arms、fallback 与 saturation trace；当前五项目没有这组 bundle。[7]

对概率安全约束：

$$\Pr(g_j(x_{t+k}, a_{t+k}) \le 0,\ \forall j, k) \ge 1 - \delta.$$

变量解释：g_j 是 joint/workspace/collision/force 等 constraints，δ 是允许风险。实现映射：Cautious MPC 用 GP residual 与 uncertainty propagation 支撑概率声明；VLA 的 ensemble spread 若没有 reliability/risk-coverage/false-safe calibration，只能当 heuristic。[9]

Recovery RL 的 learned switch 可抽象为：

$$a_t = \begin{cases} \pi_{\text{task}}(s_t), & \widehat{Q}_{\text{risk}}(s_t, a_t^{\text{task}}) \le \epsilon_{\text{risk}}, \\ \pi_{\text{rec}}(s_t), & \text{otherwise.} \end{cases}$$

变量解释：risk critic 预测 future constraint violation，recovery 尝试返回 recoverable region。实现映射：risk event 必须同时阻止当前 proposal、flush/ack queued chunk，并按 hysteresis、timeout、fallback/E-stop state machine 处理；论文不提供任意 VLA 的 zero-risk guarantee。[8]

8. 十二个确定性 probes

本轮对几何、接口、范围与 safety state 做了 12 个纯确定性 probe；**12/12 PASS**。这些 probe 验证 contract arithmetic，不代替真实 calibration、simulator rollout、real robot、uncertainty calibration 或 ownership。

Probe	结果
六层 label 唯一性	PASS；6 个 layer / 6 个 unique names
pinhole identity projection	PASS；$(0, 0, 1) \to (0, 0)$
zero-distortion round trip	PASS；$(0.1, -0.2)$ 不变
MuJoCo 3→5 friction expansion	PASS；tangential/rolling 分别复制
Panda gripper sign contract	PASS；fixed source 含 `-1 => open, 1 => closed`
composite action split + multi-clock text	PASS；fixed controller RST 同时命中
latency additive decomposition	PASS；`4+3+2+5=14 ms`
stale-chunk TTL	PASS；age `1.0 s` 超过 `0.5 s`，拒绝
overwide-DR toy counterexample	PASS；期望 reward 在 $\sigma = 0$ 最大，不随宽度单调增加
SimOpt Gaussian KL ball	PASS；小更新接受、大更新拒绝
UP-OSI/RMA/SimOpt update-object classifier	PASS；parameter / latent / distribution，不是 weights
Gaussian one-sided chance threshold	PASS；$z = 1.645$ 对应约 95%

核心复核逻辑可压缩为：

```
def audit_sim2real_contract(trace, limits):
    # 先验证 observation/action 的 frame、unit、version 与时间戳。
    assert trace.camera_contract_ok and trace.action_roundtrip_ok

    # action age 超过 TTL 时拒绝整段 chunk；风险事件必须 flush queue。
    if trace.now - trace.action_timestamp > limits.ttl:
        trace.action_queue.clear()
        return "STALE_REJECT"

    # free-space 未闭合前，不允许把失败归因到 friction/contact。
    if not trace.free_space_tracking_ok:
        return "CHECK_CONTROLLER_ACTUATOR_LATENCY"

    # contact 只在 sign/range/controller 已闭合后分解。
    if trace.contact_first_divergence:
        return "RUN_PRESS_SLIP_HOLD_LIFT_MICROBENCH"

    # uncertainty 未校准时只能作为 heuristic，不写 chance guarantee。
    return "RISK_HEURISTIC" if not trace.risk_calibrated else "GUARDED_EXECUTION"
```

9. 五项目本地证据与 truth boundary

本轮重新计算 Evidence Manifest 与五份 claim–evidence inventory 的 SHA，并做有界 filename/content 复核。以下“缺失”只指审计范围内没有协议完整 bundle，不代表用户其他机器、远端或未归档介质一定不存在。

项目	本轮 identity / 直接发现	当前支持	不支持
Robot Harness	inventory SHA `71C3B62F...A797F`；项目根 3,726 个文件中有 25 个 calibration/hardware filename hit，均位于 `third_party/sensebot_robotsystem`	高层 scope、mutex、cancel、timeout、best-effort compensation；第三方机器人资产存在	不能把 third-party 文件名写成候选人 Sim2Real 实现；无本项目 calibration→controller→real rollout→incident bundle
OPD / VLM Grounding	inventory SHA `C002B227...9AFA`；317 个文件 filename scan 0 个 Sim2Real/hardware hit	token-space teacher/student rollout、grounding 数据与评估	不是 robot-state DAgger、SysID、real adaptation 或 hardware deployment
OpenPi VLA	inventory SHA `13023578...132A`；`aloha_sim_real_rollout` summary 明确为 `pi0_aloha_sim`、`gym_aloha/AlohaTransferCube-v0`、3 episodes、2/3 simulator success	official-checkpoint simulator diagnostic 与 controller/action interface	文件名中的 `real_rollout` 不是 real hardware；不能绑定 formal Qwen、97.4 或 Sim2Real transfer
Cosmos / WAM	inventory SHA `2362311B...AFB4`；base/trained manifests SHA `63A51867...3A86` / `809AE4F8...55CC`，均 `kind=cosmos_policy_robocasa_action_rollout`、offscreen renderer、`success=false`	RoboCasa simulator controller/config、300-step failure diagnostic、offline real-demo target videos	“real RoboCasa renders”/`real_primary` 不是 physical robot rollout；无 checkpoint binary、calibration、real latency/contact/safety
UWM / LIBERO	inventory SHA `C35C41D6...8A9`；156 个项目文件，contact filename hits 是 RoboCasa diagnostic contact sheets	simulator finetune/rollout、action adapter、protocol 与 failure instrumentation	不支持 physical contact、Sim2Real uplift、real recovery 或 safety completion

Evidence Manifest 本轮 SHA 为 `8C870E35736ECCF6201BECF19A1B8E6C3B2367DC2250662687249 47B70169083`。五项目 ownership ledger 都仍需候选人逐项确认：Git author、文件存在、上游机制、Codex/工具实现或第三方资产不能自动升级为个人独立设计与实现。

当前安全口述：

> “我把 Sim2Real 拆成 visual/sensor、dynamics、control/timing、contact、task/protocol、safety/recovery 六层，先做 frame/unit/timestamp/action round trip，再做 free-space、latency/actuator 与 contact micro-bench，最后才做 matched real rollout。公开论文支持 DR、SysID、SimOpt、OSI/RMA、residual 和 recovery 的机制边界；我的项目材料目前主要是 simulator、接口和诊断证据，没有协议完整的真机 bundle，所以我讲设计与审计方法，不把它说成已经完成硬件迁移。”

10. Source → Knowledge → Card reconciliation

Cluster	Source / Knowledge / Card	独立结论	保留 warning
WM-031 taxonomy	六层/first-divergence	PASS AS SYNTHESIS	原子 source 早期写“五层”；安全/恢复作为第六响应层后形成六层 Audit D 合同，需说明是工程综合
WM-032 visual DR	relevant support、label/geometry consistency、overwide gate	PASS	Tobin/ADR/SimOpt 不能证明本地 visual transfer
WM-033 SysID + DR	nominal center、residual uncertainty、identifiability、matched arms	PASS	fit 好不等于参数真；无本地 real excitation bundle
WM-034 timing/controller	multi-clock、action age、actuator/controller mismatch	PASS	single inference time 不能替 capture-to-apply latency；无本地真实 tail
WM-035 camera/sensor	K, R, t、distortion、photometric/temporal/noise/occlusion	PASS	reprojection error 不等于 deployment extrinsics 正确
WM-036 contact/gripper	friction/compliance/solver/gripper round trip	PASS	robosuite sign 是 simulator/version-specific；无 real contact trace
WM-037 adaptation	residual / UP-OSI / RMA / SimOpt / DAgger 分层	PASS	不把 token rollout、offline finetune 或 sim rollout叫 real online adaptation
WM-038 safety	constraint、calibration、CBF/training/recovery/fallback/E-stop	PASS WITH WARNING	learned risk、CPO、clipping、高层 cancel 都不是 physical zero-risk guarantee
WM-040..044	metric ladder、matched ablation、consistency、first failure、evidence audit	PASS	offline/sim proxy 不能替 closed-loop real success；ownership仍是独立 gate

知识页与题卡无需重写；它们已保留上述 boundary。`VLA-WM-031..038` 原子题卡和 `VLA-WM-040..044` 聚合题卡继续保持 `unseen`，独立 fact review 不等于 oral mastery。

11. 审查状态与下一步

Audit D 的 public paper/source facts、official semantics、12 probes、local truth boundary 与 source→knowledge→card reconciliation 已完成，可从 `independent_review_pending` 提升为 `independent_fact_reviewed / PASS WITH WARNING`。这不改变 VOL 04 整卷 `WARN`，也不证明任何项目完成真机 Sim2Real。

仍开放：

- 收集每个目标平台的 camera serial/intrinsics/extrinsics/hand-eye、mounting revision、timestamp 与 preprocessing manifest；

- 收集 physics/controller/policy/firmware Hz、capture-to-apply latency distribution、actuator fit/held-out residual 与 queue stale/drop/flush/ack trace；
- 收集 material/object/gripper/contact/solver manifest，以及 press/slip/hold/lift/transport matched micro-benches；
- 在安全 authority 下补 fixed-checkpoint、fixed-init/controller 的 sim/real paired trials，报告 denominator、置信区间与 failure family；
- 补 constraint registry、calibrated risk、shield/recovery/fallback/E-stop、near-miss/incident/reset/approval log；
- 由候选人确认五项目逐文件设计、实现、运行、审查与工具/协作者角色；真实口述前不改变 mastery，不创建 coaching state。

下一 bounded slice 优先进行一题真实口述；若继续文档事实复核，则 VOL 04 Audit A–E 已全部完成独立 review，应转向 repository completion / oral gate，而不是生成新的 Sim2Real 节点或启动 PER-019。

References

1. Tobin et al., *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, arXiv v1, 2017。
2. Peng et al., *Sim-to-Real Transfer of Robotic Control with Dynamics Randomization*, arXiv v3, 2018。
3. Chebotar et al., *Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience*, arXiv v4, 2019。
4. Tan et al., *Sim-to-Real: Learning Agile Locomotion For Quadruped Robots*, arXiv v2, 2018。
5. Yu et al., *Preparing for the Unknown: Learning a Universal Policy with Online System Identification*, arXiv v3, 2017。
6. Kumar et al., *RMA: Rapid Motor Adaptation for Legged Robots*, arXiv v1, 2021。
7. Johannink et al., *Residual Reinforcement Learning for Robot Control*, arXiv v2, 2018。
8. Thananjeyan et al., *Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones*, arXiv v2, 2021。
9. Hewing et al., *Cautious Model Predictive Control using Gaussian Process Regression*, arXiv v4, 2019。
10. OpenAI et al., *Learning Dexterous In-Hand Manipulation*, arXiv v5, 2019。
11. Google DeepMind, MuJoCo `3.10.0@28009f9`，并参照固定响应的 Modeling、Computation 与 XML Reference。
12. OpenCV, `py_calibration.markdown@f7dd7df`。
13. ARISE Initiative, robosuite `v1.5.1@51cc017`，重点复核 controller 与 Panda gripper source。
14. VOL 04 Sim2Real Claim-to-Mechanism Audit D。

15. WM-031 Reality-gap、WM-032 Visual DR、WM-033 Dynamics SysID、WM-034 Timing、WM-035 Camera、WM-036 Contact、WM-037 Adaptation 与 WM-038 Safety。
16. VOL 04 Aggregate Knowledge 与 Question Bank。
17. Evidence Manifest 与五项目 claim–evidence inventories。

Part 9 / 12 · `interview_materials/vla/sources/vol_04_uwm_independent_fact_review.html` · SHA-256 `4E16E0B024E17E7A6022026DF54ED4C1BBA4B6604266E61DBC5AA17F17C3FB74`

返回本卷目录

Canonical Markdown SHA-256：`4e16e0b024e17e7a6022026df54ed4c1bba4b6604266e61dbc5aa17f17c3fb74`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_uwm_independent_fact_review.md`

VOL 04 Audit B：UWM 独立事实复核

状态：`independent_review / UWM / WARN` 复核日期：2026-07-17 冻结 revision：`WEIRDLabUW/unified-world-model@b7d4067c8859775e5b08f681e76d538015cdd445` 目的：独立重新获取官方 raw 文件与项目页，复核 Audit B 对 `VLA-WM-015..020` 的关键实现断言；不复核用户项目数字，不下载 checkpoint，不生成题卡。

复核问题与结论

本轮只问：UWM 是否真的在共享网络中同时处理 action/video，是否使用独立 timestep，action-free video 的 mask 到底改变了什么，以及默认 shape、训练/评估和 checkpoint 边界是否被 source audit 正确描述?

结论：核心架构断言可以由官方项目页与冻结 commit 的 raw 文件独立支持；`action_mask` 的边界也被代码直接证实，但它不是 action loss mask。checkpoint、runtime、完整论文公式、许可证与本地项目结果仍未闭合，因此 verdict 保持 `WARN`。本复核足以解锁下一 bounded slice：单独生成 `VLA-WM-015` 知识与题卡；不能把 UWM 方法审计升级为个人项目结果。

来源与可重复指纹

访问方式是只读 HTTP GET；raw 文件按返回字节计算 SHA-256。网页内容是访问时快照，不等同 Git commit 文件。

Source	类型	访问/冻结	SHA-256	直接支持
`configs/model/uwm.yaml`	official-code	frozen commit	`9C2EB19F65ED823C06F3D9C6A1520FF62393BA6DC4B1820E50AEA0F4BA62D152`	`action_len=16`、`use_low_dim=False`、`use_language=False`、train/inference steps
`models/uwm/uwm.py`	official-code	frozen commit	`ADD93268D82B784CC733C84F8A09E98115C0ACC370FC9894BC9373976EF39E26`	Dual timestep、共享 token sequence、action mask timestep、loss、sampling modes
`experiments/uwm/train.py`	official-code	frozen commit	`7971ECEAB5495C4BD534F48AF5BE075751465CCB3E0E0F6549CAA9C5483E8707`	validation metric ladder、`action_mask` 传递、pretrain load
`experiments/uwm/train_robomimic.py`	official-code	frozen commit	`71F8CBD2FAC379A69221B4224699E3777443F042D37279BC5BA5666284D07678`	rollout hook、pretraining skip、checkpoint load

Source	类型	访问/冻结	SHA-256	直接支持
`environments/robomimic/wrappers.py`	official-code	frozen commit	`981088E44B22E1B7E68EB2679DFBE931D6839E33B43F2B45DA24913A9D2374AD`	LIBERO wrapper、`reset_to_state`、`check_success`
`README.md`	official-code	frozen commit	`84FDD2F1E3E3E90E33B65148F196EBE9DD7024514D0F911BD7D173F5D9406B16`	LIBERO-90 pretrain/finetune 命令、Google Drive checkpoint 链接
UWM official project page	primary web	fetched 2026-07-17	`6C6155C17F807F55CCA22D40AF877CFFBB166D0116A3F3C04D72C2577EE623C6`	action/video coupling、independent timestep、action-free video、four inference modes
arXiv abstract page `2504.02792v3`	primary web	fetched 2026-07-17	`1B71E15DE9AE075D10810403ACB1D69C1D9B8C3A72FA288531932CDD841BA5CD`	论文版本与摘要级方法定位；本轮未使用 full-text equation

许可证探测：冻结 commit 的 `LICENSE`、`COPYING`、`NOTICE` raw 路径均返回 404；不能假定该树有可复用许可证。

Claim-by-claim 复核

Claim	复核证据	结果	边界
共享 diffusion transformer 同时处理 action 与 next-observation video	`DualNoisePredictionNet` 定义 action encoder、3D video patchifier、registers、共享 AdaLN blocks；输出再切成 action/video 两段	`PASS`	这是上游实现事实，不是个人创新
两路 timestep 独立进入条件	`DualTimestepEncoder.forward(t1,t2)` 分别做 sinusoidal embedding 后拼接；`UnifiedWorldModel.forward` 独立采样 action/next_obs timestep	`PASS`	只能说明代码合同，未做 runtime numerical smoke
action-free video 把 action 置为 fully noised	official page 明确 missing actions as noise；代码 `action_t[~action_mask] = num_train_steps - 1`	`PASS WITH WARNING`	代码还用 zero action 作为输入张量；“fully noised”指 timestep，不等于移除 action loss
`action_mask` 会屏蔽 action loss	代码随后直接执行 `F.mse_loss(action_noise_pred, action_noise)`，没有 sample mask 或 valid-axis denominator	`FAIL CLOSED / claim rejected`	source audit 的“不是 action loss mask”断言被独立确认
inference modes 由 modality timestep slice 控制	`sample_forward_dynamics` 固定 `action_t=timesteps[-1]`；`sample_inverse_dynamics` 固定 `next_obs_t=timesteps[-1]`；marginal/joint 方法分别迭代对应分支	`PASS`	scheduler endpoint 的数值含义仍依赖 pinned diffusers runtime
frozen defaults 是 action_len 16、train steps 100、inference steps 10	`configs/model/uwm.yaml` 与 `uwm.py` defaults 同时出现	`PASS`	不是方法常数；dataset config 可覆盖
默认配置不是 language-conditioned VLA	config 明确 `use_low_dim=False`、`use_language=False`	`PASS WITH WARNING`	其他实验 config 若 override，必须单独记录；不能泛化所有 UWM run
validation metrics 只形成 proxy ladder	`train.py` 记录 action MSE（marginal/inverse/joint）与 image MSE（marginal/forward/joint），并对每项单独 `F.mse_loss`	`PASS`	不等于 closed-loop success 或 causal validity
pretraining checkpoint 可由 README 复现	README 给出命令与 Google Drive folder；train code load `ckpt["model"]` 及 normalizers	`PASS WITH WARNING`	没有 checkpoint hash/revision，命令未在本机执行

Claim	复核证据	结果	边界
LIBERO wrapper 证明 strict fixed-init benchmark	wrapper 有 `reset_to_state` 与 `check_success`，训练脚本有 rollout hook	`FAIL CLOSED / claim rejected`	wrapper/API 不等于某次 strict init、seed、horizon 或本地成功率

关键实现核对

1. Sequence 与 shape

冻结 config 给出 `action_len=16`，`action_dim` 来自 dataset shape metadata；模型的 positional sequence 是 action tokens + video tokens + registers。因而 `[B,16,D_a]` 是该 config 的 action contract，而不是 UWM 的普适 action dimension。DROID/LIBERO 的 `D_a` 必须回到各自 dataset config，不能从 `action_len` 推出。

2. 训练 loss 与 mask

`forward` 为 action 和 next observation 各采样一个 timestep、各自加噪并预测 noise；最后是两个全张量 MSE 的和。对 action-free video，`action_mask` 只改变 action branch 的 timestep。要声称“视频样本不产生 action gradient”，必须补一个 gradient test 或显式 masked-loss A/B；当前没有这项 runtime artifact。

3. Sampling mode

官方项目页将 UWM 描述为 policy、forward dynamics、inverse dynamics 和 video prediction 的统一框架；raw code 进一步显示这些 mode 通过固定另一分支的 timestep、只迭代目标分支实现。这个结论是“方法/代码映射”，不是说每种 mode 都在用户本地 checkpoint 上校准过。

4. 训练、评估与 checkpoint

`train.py` 的 validation 会对 action/image proxy 做分项 MSE；`train_robomimic.py` 的 rollout 逻辑在 pretraining 阶段跳过，finetune 才可能调用环境。checkpoint 保存 model、optimizer、scheduler、scaler、normalizer 和 step，但 README 只提供 Google Drive folder。故 paper → repo 已核对，repo → checkpoint/runtime 仍为 open。

独立复核中未完成的项目

未完成项	原因	处理
full-paper equations	本轮只取 arXiv abstract 页面	公式只保留 code/project-page 可支持的抽象，不写摘要外数字
diffusers endpoint numerical smoke	当前环境未固定官方训练 runtime/checkpoint	题卡中把 `timesteps[0]/[-1]` 写成需核验的 implementation detail
action-free gradient ablation	没有 upstream dataset/checkpoint/runtime	保持“timestep mask ≠ loss mask”边界
checkpoint hash/contents	README 只有 Google Drive folder	不把公开链接当本地 artifact

未完成项	原因	处理
strict LIBERO result	wrapper 与 benchmark protocol 不是同一证据	UWM 74.4/92.8 仍留在 VOL 07 evidence
license	frozen tree 未找到 LICENSE/COPYING/NOTICE	复用前向上游确认
browser visual QA	当前 app 的 `file://` 策略限制	review 标记 BLOCKED/PENDING，不用静态检查冒充

允许进入下一节点的口径

`VLA-WM-015` 可以以如下范围进入知识页：

> UWM 的 frozen implementation 在共享 diffusion transformer 中拼接 action 与 next-observation video token，使用独立 timestep，并通过固定一侧 timestep 实现 policy/forward/inverse/video/joint sampling；action-free video 的代码行为是把 action timestep 设到最大训练步，但当前 action MSE 没有显式 sample mask。所有 checkpoint、runtime、许可证和用户项目结果另行标注。

不允许升级为：

- “UWM 已证明本地 74.4% checkpoint”；
- “视频样本完全不产生 action loss”；
- “README 的 checkpoint 链接就是可复算权重”；
- “wrapper API 等价于官方 strict benchmark”；
- “共享 architecture 属于个人原创”。

参考资料

1. UWM Source Audit B：原始 claim-to-code 审计与未决项。
2. UWM 官方项目页：方法与 action-free video 语义。
3. UWM frozen repository：本轮 raw 文件来源。
4. arXiv 2504.02792v3：论文版本与摘要定位。
5. VOL 04 knowledge map：当前 18/44 knowledge 边界与后续节点。

Part 10 / 12 · `interview_materials/vla/sources/vol_04_uwm_video_action_claim_to_code.html` · SHA-256
`AD1C1DB7112F294C2867DF4B7021E5AFFFF13B9A1CE4F5513CEF4776655F31DB`

返回本卷目录

Canonical Markdown SHA-256：
`ad1c1db7112f294c2867df4b7021e5afffff13b9a1ce4f5513cef4776655f31db`；生成自：
`interview_materials\vla\sources\vol_04_uwm_video_action_claim_to_code.md`

VOL 04 Source Audit B：UWM Video / Action Diffusion Claim-to-Code

审计日期：2026-07-16 状态：drafted / paper_abstract_and_official_site_reviewed / repo_frozen_and_line_audited / independent_review_complete / WARN 覆盖题目：VLA-WM-015..020、VLA-WM-039..044 frozen repo：WEIRDLabUW/unified-world-model @ b7d4067c8859775e5b08f681e76d538015cdd445（commit date 2025-10-08）

1. 结论先行

1. **UWM 不是在视频模型旁边外挂 action head。**它在一个共享 diffusion transformer 中同时建模 action chunk 和 next-observation latent video；action 与 video 使用独立 diffusion timestep，官方摘要/项目页将其描述为 policy、forward dynamics、inverse dynamics 和 video prediction 的统一框架。[1][2]
2. **训练 schema 有 robot tuple 与 action-free video tuple。**robot 数据提供 (o, a, o')，视频数据只提供 (o, o')。官网方法页把缺失 action 描述为 fully noised，并将 action diffusion timestep 手动设到 T。[2]
3. **独立 timestep 定义了 inference mode，而不是一句“多任务学习”。**一个分支固定在高噪声、另一分支迭代得到 marginal；给定 clean action 预测未来是 forward dynamics；给定 clean future 反推 action 是 inverse dynamics；两个分支一起迭代是 joint sample。[1][2][3]
4. **frozen default 不是 language-conditioned VLA。**configs/model/uwm.yaml 设置 use_low_dim=False 和 use_language=False；默认 LIBERO 输入是两路 RGB，DROID 是否使用 low-dim 也由模型配置控制。[3]
5. **工程默认合同是 num_frames=19、obs_num_frames=2、action_len=16、100 training steps、10 DDIM inference steps。**这些值只属于 frozen config，不是方法常数。[3]
6. **当前代码里的 action_mask 不是 action loss mask。**它只把 video 样本的 `action_t` 设为最大噪声 timestep；随后 action loss 仍是全量 `F.mse_loss(action_noise_pred, action_noise)`。因此不能说 video 样本 action loss 已排除。[3]
7. **离线 MSE 不能替代闭环 success。**仓库 validation 同时记录 marginal/inverse/joint action MSE 与 marginal/forward/joint image MSE；每项只对应一个 conditional proxy。[3]

8. **README 的 LIBERO 命令与 Google Drive 链接不等于本地 checkpoint 或 strict 结果。** train_robomimic.py 对 libero_90 预训练直接跳过 rollout；checkpoint 没有 hash/revision。[3][4]
9. **frozen tree 没有 LICENSE/COPYING/NOTICE。** 代码可见性不等于明确复用许可，需向上游确认。[3]

2. 来源与证据边界

编号	来源	可支持	不能支持
[1]	arXiv 2504.02792v3 abstract，提交 2025-04-03、修订 2025-05-23；快照：%TEMP%/vol04-papers/uwm.html	统一 action/video diffusion、独立 timestep、四种 mode、DROID/LIBERO 方向	本轮未稳定获得 full-text equation/PDF/source tar；不写摘要外 exact 数字
[2]	官方 weirdlabuw.github.io/uwm 项目页；快照：%TEMP%/vol04-papers/uwm_site.html	tuple schema、fully-noised action、独立 timestep、DROID 实验方向	官网视频/文字不是本地运行证据
[3]	frozen repo WEIRDLabUW/unified-world-model @ b7d4067c8859775e5b08f681e76d538015cdd445	模型、config、dataset、normalizer、train/eval/wrapper 的逐行行为	commit 是论文 v3 之后的实现；无许可证/ checkpoint revision
[4]	frozen repo README	安装、LIBERO-90 pretrain/finetune、DROID pretrain/cotrain/finetune、checkpoint 链接	命令未在本机执行，不证明本地结果
同源推断	[1]–[4] 联合分析	timestep mode table、shape/horizon、metric comparability、mask 风险	未运行模型、未下载 checkpoint、未做 rollout

3. 统一 diffusion 合同

令 o 是当前 observation feature，$a \in \mathbb{R}^{H \times D_a}$ 是 action chunk，y 是 next-observation latent，t_a, t_y 是两路 timestep。共享预测器可抽象为：

$$\left(\hat{\epsilon}_a, \hat{\epsilon}_y\right) = F_\theta\left(\mathrm{concat}(E_a(a_{t_a}), E_y(y_{t_y}), R), E_o(o), e_t(t_a, t_y)\right).$$

变量解释：E_a 是 action MLP，E_y 是 3D video patchifier，R 是 register tokens，E_o 是 observation encoder，e_t 是两个 sinusoidal timestep embedding；输出分别与 action/video noise 同形状。代码映射是 DualNoisePredictionNet、DualTimestepEncoder、AdaLN attention blocks。[3]

DDPM-style noise prediction 的通用表达为：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \qquad \mathcal{L}_{noise} = \mathrm{MSE}(\hat{\epsilon}_\theta(x_t, t, c), \epsilon).$$

其中 x_0 可以是 action 或 VAE latent video，c 是 observation 与另一 modality 的 noisy token，$\bar{\alpha}_t$ 来自 scheduler。frozen code 用 diffusers DDIMScheduler 的 add_noise/step；公式是通用 diffusion 记号，不是 UWM 的独有新公式。[2][3]

robot tuple 的代码目标可整理为：

$$\mathcal{L} = \mathrm{MSE}(\hat{\epsilon}_a, \epsilon_a) + \mathrm{MSE}(\hat{\epsilon}_y, \epsilon_y).$$

当前 UnifiedWorldModel.forward 对两项各调用一次 F.mse_loss 后相加，没有显式 modality weight、valid-axis mask 或 per-sample denominator。[3]

4. 架构、数据与 shape

4.1 视觉与可选条件

UWMObservationEncoder 做 image transform、ViT/ResNet feature、可选 low-dim/text 拼接，并用 VAEDownsample 将 future images 转成 latent video。MultiViewVideoPatchifier 用 3D Conv patchify latent video，输出经 linear decoder/unpatchifier 回到 latent shape。[3]

默认模型配置为 ResNet、use_low_dim=False、use_language=False，所以没有证据时不能称其为 language VLA；若本地 config 覆盖了这些字段，需把覆盖文件和 checkpoint 一起记录。

4.2 sequence / action contract

process_batch 使用 action_start=obs_horizon-1、action_end=action_start+action_horizon：在默认 num_frames=19、obs_num_frames=2、action_len=16 下，当前 history 是前 2 帧，action 是索引 1–16 的 16 步，future observation 是剩余 2 帧。

LIBERO template action shape [7]（position/quaternion/gripper 注释）；DROID config shape [10]（3 position + 6 rotation + gripper 注释）。因此 action tensor 可能是 [B,16,7] 或 [B,16,10]，不能把 7D 当成 UWM 固定维度。[3]

4.3 shared transformer flow

共享数据流是：current RGB history → UWMObservationEncoder → global condition；noisy action → action MLP；noisy next latent video → 3D patchifier；两者与 registers、position embedding 拼接后进入 AdaLN DiT；最后分为 action decoder 和 video unpatchifier。共享 blocks 使 modality 交互，但输出 decoder 分离；没有 value/reward head，不能把它写成 Cosmos/WAM 的 action/world/value 三头。[3]

5. action-free video 与 mask 风险

官方语义是把视频样本缺失 action 当作 fully noised。[2] frozen code 路径是：webvideo dataset 返回 zero action 与 action_mask=False；mixture dataset 给 robot/video 样本写 true/false；train.py 把 sample-level mask 传给 model.forward；uwm.py 执行 action_t[~action_mask] = num_train_steps - 1，但未对 action_loss 做 sample mask。

这支持的强 claim 是“action-free video 走最大 action noise timestep”，不支持“video action loss 被完全排除”。最小审计应固定 seed，检查 video 样本 action loss 是否有非零梯度，并比较当前实现与显式 `where(action_mask, ...)` 的 loss、gradient、rollout 消融；该测试尚未运行，review 保持 WARN。

6. 独立 timestep 与 inference modes

mode	action branch	next-observation branch	frozen method
policy / marginal action	action 从 noise 迭代；future branch 固定高噪声	nuisance sample，不输出	sample_marginal_action
forward dynamics	action 作为 clean/low-noise condition	future 从 noise 迭代	sample_forward_dynamics
inverse dynamics	action 从 noise 迭代	next observation 作为 clean/low-noise condition	sample_inverse_dynamics
marginal video	action branch 固定高噪声；不输出 action	future 从 noise 迭代	sample_marginal_next_obs
joint	两个 branch 同时迭代	两个 branch 同时更新	sample_joint

这张表需要结合 DDIMScheduler.timesteps[0]（通常 descending schedule 的高噪声端）和 [-1]（低噪声端）解释；scheduler 版本、endpoint、clip 和 sample order 必须在本地运行时确认。mode 是 distribution slice，不代表所有 slice 都已校准。[1][2][3]

7. normalizer、checkpoint 与训练/评估

项目	DROID config	LIBERO config
RGB	exterior 2 + wrist 1	agentview + eye-in-hand 2
low-dim	cartesian 6 + gripper 1；可 normalize	template 默认无 low-dim
action	[10]	[7]
action normalize	normalize_action=True，min/max 线性到约 $[-1, 1]$	Robomimic template 没有 DROID normalizer 字段
image transform	dataset-specific	flip_rgb=True，wrapper 另做垂直 flip，需实测方向

DROID normalizer 使用 scale=(max-min)/2、offset=(max+min)/2；checkpoint 保存 action/lowdim normalizer；finetune 读取后替换 dataset normalizer，并将 image encoder LR 设为 base LR 的 0.05 倍。这些是 repo 行为，不是个人创新。[3]

checkpoint 保存 model、optimizer、scheduler、scaler、normalizers、step；README 只给 Google Drive folder，没有 hash/revision/file manifest。[3][4]

validation 记录 action/dynamics loss、marginal/inverse/joint action MSE、marginal/forward/joint image MSE；train_robomimic.py 的 maybe_collect_rollout 在 libero_90 上直接 return。故这些 proxy 与 LIBERO strict success 不可互换。[3]

8. LIBERO wrapper 与 strict-eval 边界

LIBEROEnvWrapper 在 reset 时用同一初始 observation 填满 history buffer，step 接收 action sequence 并在 success/done/truncation 时提前结束，success 调 env.check_success()，并把 agentview_rgb/eye_in_hand_rgb 映射到环境 key 后垂直翻转。reset_to_state 虽存在，当前 rollout 入

口使用 env.reset()，不证明 demo-state 初始化。num_rollouts=50 与 rollout_length=1000 只是 template defaults，不是官方 strict protocol。[3]

指标	证明	不能替代
marginal/inverse action MSE	某个 action conditional 的 sample proxy	policy execution
forward image MSE	给定 action 的 future latent proxy	contact/physics/closed loop
joint image/action MSE	joint sample 的近似质量	causal action validity
rollout success	固定 wrapper/task/seed 下结果	解释哪一分支有效

跨版本比较必须冻结 task set、episode、reset/init、camera、normalizer、horizon、replan、checkpoint SHA、environment 和 success predicate。

9. 失败诊断与证据链

症状	优先检查	不应直接结论
action MSE 降、0 success	normalization、axis、chunk、camera、reset	world model 没学到
video loss 好、action policy 差	action-free branch gradient、action coverage、loss balance	视频预训练必然帮助
forward MSE 好、contact fail	latent metric、friction、control frequency	dynamics 正确
inverse MSE 好、policy fail	多解、future state、time shift	inverse 等于 policy
joint 漂移	timestep endpoint、scheduler、shared-token shortcut	shared transformer 保证一致
finetune 不复现	checkpoint/normalizer/config manifest	论文结果不可复现

证据链必须分为 paper → repo → checkpoint → runtime → personal contribution：论文不能证明 repo default，README 不能证明 checkpoint，checkpoint link 不能证明 runtime success，上游 UWM 结构不能写成本人原创。

10. 面试答题骨架与当前边界

10.1 30 秒回答

“UWM 用共享 diffusion transformer 同时建模 action chunk 与 next-observation latent video，两个 modality 的 timestep 独立，因此可切换 policy、forward、inverse、marginal video 和 joint sampling。robot 数据是 (o, a, o')，action-free video 是 (o, o')，官网语义是把缺失 action 置 fully noised。代码上我会核对 action length、obs history、VAE latent、DDIM steps、normalizer、timestep endpoint 和 rollout wrapper；特别是 frozen action_mask 只改 timestep，当前 action MSE 未显式 mask。离线 MSE 只能证明 conditional proxy，不能替代 strict closed-loop success。”

10.2 尚未核实

- 已核实：arXiv v3 metadata、官方 website method、frozen repo commit/date、无 license tree entry、UWM model/config/dataset/normalizer/train/eval/wrapper。

- 尚未核实：full-text equations、Google Drive checkpoint hash/contents、用户本地 UWM run/log/rollout、strict LIBERO protocol、mask ablation、browser visual QA、oral drill。
- 不写入本页：UWM 74.4%、92.8%、WAM 45%/71.19% 等项目数字与任何个人贡献。
- 下一步：已完成 UWM Audit B 独立复核；下一 bounded slice 进入 `VLA-WM-015` knowledge + atomic card，Source Audit C（Cosmos Policy/WAM）继续后置。

11. 参考文献（References）

1. Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets”, arXiv 2504.02792v3, 2025. arXiv abstract；本地：%TEMP%/vol04-papers/uwm.html。
2. Unified World Models official project page. Website；本地：%TEMP%/vol04-papers/uwm_site.html。
3. WEIRDLabUW/unified-world-model, frozen commit b7d4067c8859775e5b08f681e76d538015cdd445. GitHub；重点文件：models/uwm/uwm.py、experiments/uwm/train.py、configs/model/uwm.yaml、environments/robomimic/wrappers.py。
4. UWM repository README at the same frozen commit. README。

Part 11 / 12 · `interview_materials/vla/sources/vol_04_world_model_foundations_claim_to_code.html` · SHA-256 `BB48AF7605688132DDD589109C8417DD0EBFE65F504A98CF5AB660F01E34BA7B`

返回本卷目录

Canonical Markdown SHA-256：`bb48af7605688132ddd589109c8417dd0ebfe65f504a98cf5ab660f01e34ba7b`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_world_model_foundations_claim_to_code.md`

VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 Foundations Claim-to-Code

审计日期：2026-07-16 状态：`drafted / upstream_paper_and_code_reviewed / independent_reviewed_slice / next_knowledge_batch` 覆盖题目：`VLA-WM-001..014`、`VLA-WM-019..021`
本页目的：把 world model 的输入输出合同、latent dynamics、one-step 与 rollout、RSSM、imagination actor-critic 以及评估边界固定成可审计的面试知识。UWM、Cosmos/WAM、LIBERO/RoboCasa 的项目数字不在本页作通用方法证据。

1. 结论先行

1. **World model 不是“能生成视频的模型”的同义词。**它是一个条件预测器：给定观测历史、动作以及必要的任务条件，预测未来 latent/observation、reward 和 termination/continuation。是否能支持 action consequence、长时 rollout 或 planning，取决于这些变量和训练/评估合同，而不是画面是否逼真。[1][2][3]
2. **早期 World Models 把视觉压缩和时序预测分开。**VAE 将图像压成 latent，MDN-RNN 建模 $p(z_{t+1} \mid a_t, z_t, h_t)$，小 controller 再读取 latent 与 recurrent state；论文甚至在 learned dream 中训练 controller，但这同时暴露了 model exploitation 风险。[1]
3. **PlaNet 的核心是 belief-state latent dynamics + online planning。**它同时使用 deterministic 与 stochastic transition，并用 latent overshooting 让多步 latent prediction 受约束；论文明确把模型误差、误差累积、多未来和 OOD 过度自信列为 planning 难点。[2]
4. **DreamerV3 的 RSSM 将 posterior observation update 与 prior action-conditioned imagination 分开。**论文公式中的 $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$、posterior $q_\phi(z_t \mid h_t, x_t)$、prior $p_\phi(\hat{z}_t \mid h_t)$、reward/continue/decoder heads，在 frozen repo 中分别对应 `RSSM.observe`、`RSSM.imagine`、`RSSM.loss` 与 `Agent.loss`。[3][4]
5. **Dreamer 的 actor/critic 训练不是把真实像素再喂一遍。**它从 replay 后验状态起点，在 prior dynamics 中生成 imagined latent trajectory，预测 reward/continue，再用 λ-return 更新 actor/critic；world model、actor、critic 的梯度边界由 `stop_gradient` 和不同 loss 路径共同决定。[3][4]

6. **one-step 指标下降不能证明 rollout 可用。**teacher-forced one-step 使用真实上一步 latent/observation；open-loop rollout 使用模型自己的上一步预测。误差可累积，且 action-insensitive、posterior collapse、背景 shortcut 会让 pixel/latent proxy 与 downstream success 脱钩。[2][3]
7. **reward、value、success classifier、termination predictor 不是一回事。**reward 是一步即时任务信号，value 是策略条件下的未来回报估计，success classifier 是评估或目标标签，continuation/termination 表示 episode 是否继续。把它们混成一个“好坏分数”会导致 planning 和审计错误。[3][4]
8. **随机性必须分层。**aleatoric uncertainty 是环境本身的多可能结果，epistemic uncertainty 是数据/模型知识不足；ensemble、posterior samples、disagreement 或 calibrated likelihood 只是估计手段，不能自动等价于真实风险。[2][3]
9. **“action-conditioned”仍不自动等于“可干预”。**如果动作只与背景变化相关、动作覆盖窄、数据存在 confounder，模型可能在观测预测上很好却不反映执行该动作的因果后果；需要 counterfactual/action-swap、action sensitivity 和 closed-loop tests。[2]
10. **DreamerV3 repo 的实现边界必须明说。**README 自称基于 DreamerV2 的 reimplementation，与 Google/DeepMind 无关；它是逐行实现证据，不应被描述为论文作者发布的官方代码，也不能由 repo 直接证明个人项目结果。[4]

2. 证据层级与冻结对象

层级	冻结对象	本页支持	本页不支持
[1] 论文	Ha & Schmidhuber, *World Models*, arXiv `1803.10122`，ar5iv HTML 已保存于 `%TEMP%\\vol04-papers\\worldmodels.html`	VAE/MDN-RNN/controller 分解、dream training、模型漏洞边界	现代 RSSM、机器人 action schema、本地项目
[2] 论文	Hafner et al., *Learning Latent Dynamics for Planning from Pixels*（PlaNet），arXiv `1811.04551`，ar5iv HTML 已保存于 `%TEMP%\\vol04-papers\\planet.html`	latent planning、stochastic+deterministic transition、latent overshooting、multi-step/OOD 难点、CEM planning	DreamerV3 的实现、VLA 项目指标
[3] 论文	Hafner et al., *Mastering Diverse Domains through World Models*（DreamerV3），arXiv `2301.04104`，ar5iv HTML 已保存于 `%TEMP%\\vol04-papers\\dreamer.html`	RSSM 合同、world-model/actor/critic 分工、imagined trajectories、λ-return、symlog/KL/free-bits 设计	frozen repo 的 exact defaults、用户项目结果
[4] 代码	`danijar/dreamerv3` frozen commit `e3f02248693a79dc8b0ebd62c93683888ddaccfe`	`rssm.py` observe/imagine/loss、`agent.py` loss graph、config scales/horizon、repo disclaimer	论文作者官方实现、checkpoint 兼容性、个人贡献
同源推断	[1]–[4] 联合分析	one-step/rollout gap、metric contract、最小测试矩阵	未运行的 rollout 成功、未见的本地数据与数字

许可证与可复现边界：GitHub metadata 将 frozen Dreamer repo 标为 MIT，但本页引用的代码语义仍需以 commit 为准；repo README 明确写 `reimplementation`，因此 license、作者归属、论文结果和本地使用应分别记录。论文 HTML 与 repo commit 已验证，论文 source tar、checkpoint、运行环境和浏览器视觉 QA 未作为已完成证据。

3. World model 的概率合同

3.1 直觉：在内部预测“采取动作后会发生什么”

普通视频预测只需回答“下一帧像什么”；用于控制的 world model 还要回答“如果我执行动作 a_t，未来状态、奖励和终止概率怎样变化”。它可以在 pixel、state 或 latent 空间运行，但必须明确 condition、预测 horizon、随机采样方式和输出单位。

专业地，令 o_t 是观测（observation），s_t 是隐藏/真实状态（state），a_t 是动作，r_t 是即时 reward，c_t 是 continuation/是否继续。一个最小 action-conditioned 合同是：

$$p_\theta(z_{t+1}, r_{t+1}, c_{t+1} \mid z_{\le t}, a_{\le t}, g),$$

其中 z_t 可以是观测压缩后的 latent，g 是任务/语言条件（若存在）。变量解释：p_θ 是模型学习的条件分布，不要求把每一帧都还原到 RGB；实现上必须能指出 latent transition、reward head、continue head 的输入输出。该表达是跨方法的合同框架，DreamerV3 的具体 factorization 见第 6 节；它不是某个项目的已验证接口。[2][3][4]

3.2 observation、state、latent、reward、termination 如何取舍

变量	作用	典型监督	缺失时风险
observation o_t/x_t	传感器实际看到的像素、proprio 或多模态输入	reconstruction / observation likelihood	latent 可能只学 reward shortcut
state/belief s_t	决策所需但可能不可直接观测的状态	filtering、latent consistency、任务监督	POMDP 下记忆不足
latent z_t, h_t	便于压缩、预测和 planning 的表示	posterior/prior、KL、multi-step consistency	表示错位或 collapse
reward r_t	一步任务信号	reward prediction	想象规划无法比较后果
continuation c_t / termination	episode 是否还在继续	Bernoulli head / environment flag	return bootstrapping、horizon 被算错
action a_t	干预变量	transition condition、action sensitivity test	只能做相关性预测

面试中先问“这个模型到底预测哪一个变量、在什么时间索引、使用真实还是模型历史”，比直接背“world model = dynamics model”更可靠。

3.3 与相邻模块的边界

- **policy**：给定当前信息选择动作 $\pi(a_t \mid s_t)$；world model 预测 action 后果，二者可联合训练但语义不同。
- **environment simulator**：由物理/规则引擎定义的真实交互对象；learned world model 是近似分布，可能在 OOD 或长 rollout 失真。
- **reward model**：只预测/定义 reward；它不必能重建观测或预测 action consequence。

- **value model**：预测给定 policy/return convention 下的未来累计回报；它不是 continuation，也不是物理动力学。
- **video generator**：若不接受 action 或不预测 reward/termination，就只能证明视觉先验；是否 actionable 必须用 intervention 与 closed-loop test 验证。

4. World Models：VAE、MDN-RNN 与 dream 中的 controller

4.1 论文方法的最短证据链

World Models 论文先用 VAE 将高维图像 frame 压缩为 latent vector，再用 MDN-RNN 预测下一个 latent 的概率分布，最后用小 controller 从 latent 与 RNN hidden state 产生动作。论文摘要和第 2 节明确指出：world model 训练是无监督的，controller 可以完全在模型自己的 hallucinated dream 中训练，再转回真实环境。[1]

MDN-RNN 的条件形式可以写为：

$$p_\theta(z_{t+1} \mid a_t, z_t, h_t) = \sum_{k=1}^{K} \pi_k(h_t, z_t, a_t)\, \mathcal{N}(z_{t+1}; \mu_k, \Sigma_k),$$

其中 h_t 是 recurrent hidden state，K 是 mixture components，π_k, μ_k, Σ_k 是 mixture weight、mean、covariance。直觉是让模型表达多个可能未来，而不是只输出平均 latent；实现映射是 MDN 输出分布参数后采样 z_{t+1}，不是对 RGB 像素做单一 MSE。[1]

4.2 dream training 的收益与漏洞

在 learned dream 里训练 controller 可以省去反复渲染真实环境、把训练信号放在紧凑 latent 空间，并使 controller 很小。[1] 但模型可能存在“漏洞”：policy 学会触发 world model 的分布外错误，使 dream reward 很高，真实环境却失败。这个现象是 model bias 与 model exploitation 的早期例子，不能用 dream return 替代 real rollout success。

最小审计问题：

1. dream 的初始 latent 来自真实观测还是纯模型生成?
2. rollout horizon、多次采样和 termination 如何处理?
3. policy 是否在真实数据分布之外行动?
4. dream reward 与真实 reward 是否在同一 contract、同一单位、同一 success predicate 下?

5. PlaNet：belief-state latent planning

5.1 为什么需要 deterministic + stochastic transition

PlaNet 从 pixels 学习 latent dynamics，并在线在 latent space 规划动作。论文摘要强调其 transition model 同时含 deterministic 和 stochastic components；这对 partial observability、contact dynamics 和多未来尤为重要。[2]

可用一个抽象 RSSM 写成：

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim q_\phi(z_t \mid h_t, o_t),$$

$$\hat{z}_t \sim p_\theta(\hat{z}_t \mid h_t), \qquad \hat{r}_t \sim p_\theta(r_t \mid h_t, z_t).$$

变量解释：h_t 保存动作与历史，z_t 表示当前观测提供的 stochastic belief，posterior q 使用真实观测，prior p 在 imagination 时只依赖历史与动作。实现映射：planning 期间不能偷偷读取未来真实 observation，否则不再是 open-loop latent planning。[2]

5.2 latent overshooting 的作用

单步 KL 只约束一步 prior/posterior；latent overshooting 将更远的 multi-step prior 与 posterior 也纳入变分目标，使模型对长期 latent transition 更一致。其目的不是保证 RGB 细节漂亮，而是提升 reward-relevant long-horizon prediction；PlaNet 论文明确将它描述为 generalized variational objective。[2]

抽象的 (k)-step 正则可写为：

$$\mathcal{L}^{(k)}_{\text{over}} = \mathbb{E}\left[\mathrm{KL}\left(q_\phi(z_{t+k} \mid o_{\le t+k}, a_{<t+k}) \,\middle\|\, p^{(k)}_\theta(z_{t+k} \mid h_t, a_{t:t+k-1})\right)\right].$$

这里 $p^{(k)}_\theta$ 是从当前 belief 连续执行 k 个动作得到的 prior；它是对 PlaNet 机制的形式化表达，变量、方向和是否加权必须按具体实现确认。不能把它误说成“多步 pixel MSE”。[2]

5.3 online planning 与 receding horizon

PlaNet 在每次决策时从当前 belief 初始化 action-sequence distribution，用 CEM 等方法采样、在 latent model 中 rollout、按 predicted reward 选择动作，并只执行第一步/短段后重新观测和规划。[2] 这是一种 receding-horizon contract：增加 planning compute 可能提升搜索，但会增加 latency；模型偏差越大，越应该缩短 open-loop execution 和增加真实观测校正。

6. DreamerV3 RSSM：论文合同到 frozen repo

6.1 论文中的完整 factorization

DreamerV3 论文给出 RSSM 的核心关系：

$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \quad z_t \sim q_\phi(z_t \mid h_t, x_t), \quad \hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t),$$

$$\hat{r}_t \sim p_\phi(\hat{r}_t \mid h_t, z_t), \quad \hat{c}_t \sim p_\phi(\hat{c}_t \mid h_t, z_t), \quad \hat{x}_t \sim p_\phi(\hat{x}_t \mid h_t, z_t).$$

其中 x_t 是 sensory input，z_t 是 stochastic representation，h_t 是 deterministic recurrent state，a_t 是 action，r_t 是 reward，c_t 是 continuation。posterior 使用观测进行 filtering；prior 不读当前观测，因而可用于 imagined rollout。[3]

6.2 `rssm.py` 的逐行映射

机制	frozen commit 证据	面试口径
state shape	`dreamerv3/rssm.py` `entry_space`：`deter=[B,4096]`、`stoch=[B,32,32]`（默认配置）	stochastic latent 是 grouped categorical representation；不要只说一个连续向量
posterior observe	`observe` / `_observe`：读取 `tokens, action, reset`，先 `_core` 再 `_logit('obslogit', ...)`，从 `_dist(logit).sample` 得到 stoch	真实序列用 observation posterior 更新 belief
prior imagine	`imagine`：由 policy 产生 action，调用 `_core` 与 `_prior(deter)`，采样 prior stoch	imagined sequence 不读未来 observation
dynamics loss	`loss`：`dyn = KL(stop_gradient(post) ‖ prior)`、`rep = KL(post ‖ stop_gradient(prior))`，可用 `free_nats` 截断	dyn/rep 两个方向表达不同梯度边界，不能只写“一个 KL”
reset	`_observe` 对 `reset` mask carry/action	episode 边界必须进入 world-model state contract
action normalization	`_core` 中按 stop-gradient 最大绝对值缩放 action	这是 repo 实现细节，不应泛化成所有 RSSM 的必然设计

6.3 KL balancing、free nats 与 collapse

DreamerV3 的 `rssm.py` 用两项 KL：

$$\mathcal{L}_{\text{dyn}} = \mathrm{KL}(\mathrm{sg}(q) \| p), \qquad \mathcal{L}_{\text{rep}} = \mathrm{KL}(q \| \mathrm{sg}(p)).$$

`sg` 是 stop-gradient；前一项主要推动 prior 追 posterior，后一项主要推动 posterior 与 prior 对齐但不让 prior 通过该项获得梯度。repo 还用 `free_nats` 做下限裁剪，避免极小 KL 过早压掉 stochastic information。具体 loss scales 来自 `configs.yaml`，不能仅凭论文摘要声称某个数值是 universal default。[3][4]

常见 failure：

- **posterior collapse**：$q(z_t \mid h_t, x_t)$ 忽略 observation，KL 接近零且重建/任务性能无改善；应检查 latent entropy、KL、observation ablation。

- **prior collapse / action ignoring**：prior 对 action 改变不敏感；做 action-swap、finite-difference 或 counterfactual rollout。
- **free-nats 过大**：可能让 KL 误差长期不回传，需看裁剪前后统计。
- **reset 泄漏**：episode 首帧 carry 未清空，导致跨 episode 信息污染。

7. DreamerV3 的 world-model loss、imagination 与 actor-critic

7.1 world-model loss 的组成

frozen `dreamerv3/agent.py` 的 `loss` 先编码 observation，再调用 `dyn.loss`，随后 decoder reconstruction、reward head 和 continuation head：

$$\mathcal{L}_{\text{wm}} = \mathcal{L}_{\text{dyn}} + \mathcal{L}_{\text{rep}} + \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_r\mathcal{L}_r + \lambda_c\mathcal{L}_c.$$

变量解释：$\mathcal{L}_{\text{rec}}$ 重建输入，$\mathcal{L}_r$ 预测 reward，$\mathcal{L}_c$ 预测 continuation，λ 由 `config.loss_scales` 展开；代码还可能为不同 observation key 复制 reconstruction scale。该式是对 repo loss dictionary 的结构化总结，不表示所有项目都使用相同 head 或权重。[3][4]

DreamerV3 论文还讨论 symlog prediction：decoder、reward predictor、critic 使用 symlog 形式以跨 reward scale 稳定训练。[3] 这是 DreamerV3 的设计，不应写成 world model 的必要组件。

7.2 imagined trajectory 的数据流

`Agent.loss` 的 imagination 流程是：

1. 从 replay sequence 的最后 $K = \min(\texttt{imag_last}, T)$ 个 posterior states 取 starts；
2. 用 `policyfn` 从当前 latent feature 采样 action；
3. `dyn.imagine(starts, policyfn, H, ...)` 通过 prior dynamics 生成 (H) 步 latent/action；
4. reward/continue/policy/value heads 作用于 imagined features；
5. `imag_loss` 用 λ-return、advantage 和 entropy 更新 actor/critic；
6. 可选 replay value loss 直接用真实 replay features 做 value regularization。

用张量合同表达：

$$s_0 \sim q_\phi(s_t \mid x_{\le t}, a_{<t}), \quad a_{1:H} \sim \pi_\theta(\cdot \mid s_{0:H-1}), \quad s_{1:H} \sim p_\phi(\cdot \mid s_{0:H-1}, a_{1:H}).$$

变量 $s_t = (h_t, z_t)$ 是 RSSM feature；前缀 s_0 来自真实 replay，之后的 $s_{1:H}$ 是模型想象。这样既利用真实数据校准起点，又避免每个 actor update 都与真实环境交互。[3][4]

7.3 actor、critic 与 λ-return

论文把 actor 写成 $a_t \sim \pi_\theta(a_t \mid s_t)$，critic 写成 $v_\psi(s_t) \approx \mathbb{E}_{p_\phi,\pi_\theta}[R_t]$。frozen repo 的 `imag_loss` 计算 continuation-weighted return、advantage、policy log-probability/entropy 与 value target；`lambda_return` 反向递推得到 bootstrap return。[3][4]

可用标准形式概括：

$$R_t^\lambda = r_t + \gamma c_t\big((1-\lambda)v_\psi(s_{t+1}) + \lambda R_{t+1}^\lambda\big).$$

这里 c_t 是 continuation mask，γ 是 discount，λ 控制 bootstrap 与多步 return 的混合。实现映射：repo 先从 `con` 与 `term/last` 生成 `weight`，再调用 `lambda_return`；具体 `disc` 还受 `contdisc` 与 `horizon` 配置影响。[4]

7.4 梯度边界与 model bias

Dreamer 的 actor/critic 依赖 model-generated states，因此 world model bias 会进入 imagined return。repo 通过 `sg`、`ac_grads`、`reward_grad`、`repval_grad` 等配置控制哪些梯度可回到 world model；不能笼统说“actor loss 反向更新整个 world model”。

如果 world model 在 OOD state 上高估 reward/value，actor 会主动寻找模型漏洞；典型防护包括：缩短 imagination horizon、增加真实数据覆盖、ensemble/disagreement penalty、uncertainty-aware value、receding-horizon replan，以及把 imagined proxy 与 real success 分开报告。[2][3][4]

8. one-step、teacher forcing、open-loop rollout 与 scheduled sampling

8.1 四种 protocol 必须拆开

Protocol	下一步输入	证明什么	不能证明
teacher-forced one-step	真实 $o_t/z_t + a_t$	局部条件预测能力	长时稳定、闭环成功
teacher-forced multi-step	每步仍注入真实 latent/observation	多步监督下的局部一致性	纯模型 rollout
open-loop rollout	第一步后使用模型自己的预测	误差累积与长期稳定性	真实环境可控性
closed-loop policy	真实环境重新观测并执行动作	任务成功与控制合同	world model 所有细节都正确

scheduled sampling 在训练中以一定概率把模型自己的预测替代真实历史，试图缩小 train/eval exposure bias；但它会改变训练分布、梯度路径和目标一致性，不能当作免费修复。面试回答应给出 schedule、替换粒度（observation/latent/action）和真实评估 protocol。[2][3]

8.2 误差为何会累积

若单步误差为 e_t，局部 Lipschitz 常数约为 L，粗略上可有：

$$\|\hat{s}_{t+k} - s_{t+k}\| \lesssim \sum_{i=0}^{k-1} L^{k-1-i} e_{t+i}.$$

这不是任一论文的 exact bound，而是解释 rollout compounding 的分析框架：当动力学放大误差（$L > 1$）或进入训练分布外区域，长 horizon 误差可能超线性增长。应通过真实 open-loop traces、latent disagreement 和 downstream task probes 验证，而不是从 one-step MSE 外推 success。[2]

9. uncertainty、actionability 与可查询 simulator 边界

9.1 aleatoric vs epistemic

- **Aleatoric**：同一 state/action 即使无限数据仍有多个结果，例如接触摩擦、遮挡后的不可见分支、随机环境；应输出条件分布或采样多模态结果。
- **Epistemic**：模型没见过该 state/action 或数据不足，随着覆盖增加理论上可降低；ensemble disagreement、dropout/posterior sampling、density/novelty score 是代理。

二者在 planning 中不同：aleatoric 可通过风险敏感期望/分位数处理；epistemic 更适合触发探索、保守 penalty 或 fallback。单个 Gaussian variance 无法自动区分两种来源。[2]

9.2 video model 什么时候 actionable

至少需要同时满足：

1. action 明确进入 transition condition，并做 action-swap/counterfactual 检验；
2. 预测变量含 reward、termination 或与控制目标有可验证关联；
3. open-loop 与 closed-loop 评估使用固定 schema、horizon、normalizer 和 reset；
4. 在分布外/失败轨迹上检查物理一致性与 action sensitivity。

否则最多称为“视觉预测器/视频生成器”。同一模型可以具备视频生成能力而没有可用 dynamics；不能用 FVD、PSNR 或画面主观质量直接证明 policy 成功。[1][2][3]

9.3 auxiliary representation vs queryable simulator

辅助表示：只要求 encoder/latent 对 action policy、reward 或 representation learning 有帮助；可不解码像素、可不支持任意 counterfactual action。

可查询 simulator：给定任意候选 action sequence，能在明确概率合同下预测未来 state/observation/reward/termination，且有 rollout horizon、uncertainty 和 reset 语义；planning 或 best-of-N 才能把它当作模拟器。

回答项目问题时，先写“本实验只用 world head 作为 auxiliary loss”还是“eval 真的查询 world model 选择 action”，否则很容易把 multi-task pretraining 夸成 model-based control。

10. metric boundary：proxy 不是 success

指标	主要证明	常见误读
PSNR/SSIM	像素重建局部相似	认为物理、动作、任务都正确
FVD/视频感知分数	视频分布/视觉质量（依实现而定）	认为 action consequence 正确
latent KL / reconstruction	表示与 dynamics 训练目标拟合	认为长 horizon 可规划
reward/value MSE	任务信号预测或价值拟合	认为 controller closed-loop success
action MSE	行为标签拟合	认为 rollout 不会漂移
open-loop rollout error	模型自回归稳定性	认为真实环境能执行
closed-loop success	固定 protocol 下的任务结果	不能单独解释 world model 哪个 head 正确

最小可比报告应同时记录 horizon、teacher forcing/open-loop、任务 split、reset、episode 数、seed、normalizer、checkpoint 和 success predicate。没有这些字段，数字只能作为方向性 context，不能声称“world model 更好”。

11. 从论文到代码的最小审计矩阵

Claim	论文证据	frozen repo 证据	仍未证明
RSSM posterior/prior 分离	DreamerV3 Eq. 3 / Figure 3	`rssm.py` `observe` vs `imagine`	用户项目是否复用此 RSSM
reward/continue/reconstruction heads	DreamerV3 World Model Learning	`agent.py` `rew`, `con`, `dec` 与 `loss`	head 的本地数据字段、权重和结果
latent imagination actor-critic	DreamerV3 Eq. 6–7	`Agent.loss` imagination + `imag_loss`	任何具体 VLA/WAM downstream gain
KL balancing/free nats	DreamerV3 训练目标与 appendix	`rssm.py` `dyn/rep/free_nats`; `configs.yaml` scales	不同配置的可比性
latent overshooting	PlaNet paper	DreamerV3 frozen repo README/代码未作为同一机制证据	DreamerV3 是否启用 PlaNet overshooting
dream controller	World Models paper	无需现代 repo 才成立	真实机器人可迁移性

12. 面试答题骨架与失败追问

12.1 30 秒主回答

“我会先把 world model 定义成 action-conditioned 的未来分布，而不是泛化成视频生成器。输入通常是 observation/history/action，输出至少要明确 latent/observation、reward 和 continuation。World Models 是 VAE+MDN-RNN 的早期分解；PlaNet 在 stochastic+deterministic latent belief 上在线规划，并用 latent overshooting 约束多步；DreamerV3 用 RSSM 的 posterior 从真实 observation 更新起点，再用 action-conditioned prior 做 imagination，reward/continue head 生成 return，actor/critic 在想象轨迹上更新。判断它是否有用要把 one-step、open-loop、closed-loop success 分开，并审计 action sensitivity、uncertainty、reset、normalizer 和模型漏洞。”

12.2 面试官继续追问时的机制点

- “为什么 one-step MSE 很低还会失败？”——真实历史 vs 自回归历史、error compounding、OOD、action ignoring。
- “Dreamer 的 posterior/prior 为什么分开？”——posterior 可用当前观测做 filtering；prior 才能在 imagination 不读未来观测。
- “reward head 和 value head 有何区别？”——一步信号 vs policy-conditioned cumulative return；训练/评估时间索引不同。
- “如何证明是 action-conditioned？”——action intervention/swap、finite difference、counterfactual rollout、real controller test。
- “DreamerV3 repo 是官方代码吗？”——不是；README 自称 reimplementation，论文事实和代码证据分开。

13. 当前证据边界与后续动作

- 已核实：World Models、PlaNet、DreamerV3 论文 HTML；DreamerV3 frozen commit 的 README、RSSM observe/imagine/loss、Agent world-model/imagination/replay-value loss、configs loss scales/RSSM/imagination horizon。
- 尚未核实：PlaNet source tar、Dreamer repo checkpoint 与 exact paper result reproduction、任何用户本地 world-model/WAM/UWM artifact、浏览器视觉 QA、真实口述。独立 frozen-source 事实复核已记录在 复核报告。
- 不写入本页：UWM/Cosmos 的 action/video/value slots、LIBERO/RoboCasa strict 数字、WAM/UWM 的本地 MSE/success、个人贡献边界；这些要到对应 Source Audit 与 VOL 07 项目证据后再写。
- 下一步：基于独立复核先生成 `VLA-WM-001..010` 知识页；之后再进入 Source Audit B（UWM）的完整审查，确认 video/action diffusion、license、checkpoint、mask/horizon 与 LIBERO eval contract。

14. 参考文献（References）

1. Ha & Schmidhuber, “World Models”, 2018. arXiv；本地论文快照：`%TEMP%\\vol04-papers\\worldmodels.html`。
2. Hafner et al., “Learning Latent Dynamics for Planning from Pixels” (PlaNet), 2019. arXiv；本地论文快照：`%TEMP%\\vol04-papers\\planet.html`。
3. Hafner et al., “Mastering Diverse Domains through World Models” (DreamerV3), 2023/2025. arXiv；本地论文快照：`%TEMP%\\vol04-papers\\dreamer.html`。
4. `danijar/dreamerv3`, frozen commit `e3f02248693a79dc8b0ebd62c93683888ddaccfe`. GitHub；重点文件：`rssm.py`、`agent.py`、`configs.yaml`。

Part 12 / 12 · `interview_materials/vla/sources/vol_04_world_model_foundations_independent_fact_review.html` · SHA-256 `EF09B0D8AC55D76C3C9EF7276DB8B93F4B27E0D37B1C8E49009CD52A4C1FE063`

返回本卷目录

Canonical Markdown SHA-256：`ef09b0d8ac55d76c3c9ef7276db8b93f4b27e0d37b1c8e49009cd52a4c1fe063`；生成自：`interview_materials\vla\sources\vol_04_world_model_foundations_independent_fact_review.md`

VOL 04 Source Audit A：World Model / PlaNet / DreamerV3 独立事实复核

状态：`reviewed_slice / WARN` 复核日期：2026-07-16 证据截止：2026-07-16 复核范围：`VLA-WM-001..014`、`VLA-WM-019..021` 的 source audit；下一知识批只开放 `VLA-WM-001..010`。结论先行：World Models、PlaNet、DreamerV3 的关键方法事实和 DreamerV3 frozen repo 的实现边界已独立核对；DreamerV3 repo 是 reimplementation，不应称作论文作者官方代码；PlaNet latent overshooting 不能外推为 DreamerV3 的实现事实。

问题（Question）

本记录回答：Source Audit A 中关于 world-model 概率合同、World Models、PlaNet、DreamerV3 RSSM、imagination actor-critic、KL/free-nats、one-step/open-loop 和 metric boundary 的主张，哪些可回到 primary paper 或 frozen official-code，哪些只能作为分析框架？

本次复核只读取 frozen paper/repository 内容，不执行 DreamerV3 训练、不下载 checkpoint、不声称复现论文结果，也不把通用机制写成本地项目贡献。

复核方法与证据等级

等级	含义	面试口径
`primary-checked`	直接核对论文 HTML/Ar5iv 的正文、公式或实验描述	“论文定义/报告……”
`official-code-checked`	直接读取 frozen repo 的 README、源码或配置	“该 commit 的实现……”
`framework`	根据论文/代码抽出的评估或诊断设计	“我会这样区分/测试……”
`warning`	事实方向正确，但来源是 reimplementation、配置依赖或缺实验 artifact	“在该来源/配置范围内……”
`blocked`	本地 checkpoint、运行日志、结果或 ownership 缺失	不升级为复现或个人成果

冻结对象：World Models `arXiv 1803.10122`、PlaNet `arXiv 1811.04551`、DreamerV3 `arXiv 2301.04104`，以及 `danijar/dreamerv3@e3f02248693a79dc8b0ebd62c93683888ddaccfe`。

1. World Models：VAE、MDN-RNN 与 controller 的分解已核对

World Models 论文把系统拆成视觉模型 V、记忆模型 M 和 controller C。VAE 把每帧图像压成 latent z_t；MDN-RNN 预测动作和历史条件下的未来 latent 分布；controller 读取 latent 与 recurrent state 产生动作。论文给出了 `env.reset → vae.encode → controller.action → env.step → rnn.forward` 的 rollout 逻辑。[1]

论文还明确展示了在模型生成的 dream 中训练/部署 controller，并把 decoder 用于可视化。因此 Audit A 中“world model 不等于视频生成器；是否可行动要看 action condition、状态变量和执行合同”的表述有 primary 支持；“dream controller 的收益与 model exploitation 风险需要单独审计”属于同源分析边界，不是本地实验结果。

2. PlaNet：deterministic/stochastic latent、overshooting 与 online planning 已核对

PlaNet 论文摘要和方法部分直接写明：latent dynamics 同时包含 deterministic 与 stochastic transition；提出 multi-step variational inference objective `latent overshooting`；利用 learned latent dynamics 在 latent space 进行 online planning，并采用 receding-horizon 方式每步重新规划。[2]

论文还讨论了 learned model planning 的风险，包括多个可能未来、分布外过度自信，以及模型误差造成的规划问题。因此 Audit A 中“latent overshooting 约束多步预测”和“不能从视觉质量直接推导 long-horizon planning 成功”是有 primary 依据的。

一个必须保留的边界是：PlaNet 的 overshooting 是 PlaNet 论文机制；当前 DreamerV3 frozen repo 的 `rssm.py` 显示 observe/imagine、dyn/rep KL 和 free-nats，但没有足够证据把它命名为 PlaNet overshooting。题库和知识页不得写成“DreamerV3 使用 PlaNet overshooting”。

3. DreamerV3 RSSM：论文因子分解与 frozen repo 路径已核对

DreamerV3 论文把 world model 写成 RSSM：序列模型由前一时刻 $(h_{t-1}, z_{t-1}, a_{t-1})$ 得到 h_t，posterior $q_\phi(z_t \mid h_t, x_t)$ 使用当前 observation，prior $p_\phi(z_t \mid h_t)$ 在 imagination 时不读取未来 observation；decoder、reward、continue 预测器作用在 latent feature 上。[3]

frozen repo 的 `dreamerv3/rssm.py` 与论文合同逐项对应：

论文/概念	frozen code evidence	复核结论
posterior filtering	`RSSM.observe`	`official-code-checked`
action-conditioned prior imagination	`RSSM.imagine`	`official-code-checked`
dynamics/representation KL	`RSSM.loss` 中 `dyn` / `rep`	`official-code-checked`
stop-gradient 方向	`sg(post)` 与 `sg(prior)` 两条 KL	`official-code-checked`
free nats	`free_nats=1.0` 与 `jnp.maximum`	`official-code-checked`
actor/critic imagination	`Agent.loss` 调用 `dyn.imagine`、lambda-return 与 policy/value loss	`official-code-checked`

DreamerV3 论文也直接描述 world model 预测 future representation/reward、actor 选择 action、critic 评估 outcome，并使用 imagined trajectories 学习 actor/critic。[3] 因而 Audit A 的 posterior/prior、imagination 和 actor-critic 主线可以保留。

4. DreamerV3 的 KL/free-nats 与 repo provenance 边界

论文给出 prediction、dynamics 和 representation loss，并说明 dynamics/representation loss 的 stop-gradient 方向、loss scale 与 free bits clipping；论文文字写的是低于 `1 nat` 时裁剪相关项。[3]

frozen `rssm.py` 采用 `free_nats: float = 1.0`，并分别计算 `KL[sg(post) || prior]` 与 `KL[post || sg(prior)]` 后用 `jnp.maximum` 截断；`configs.yaml` 另记录 `loss_scales`、RSSM 维度和 replay-value 相关配置。[4] 这支持机制映射，但具体训练结果仍依赖 config、数据、checkpoint 和运行环境。

README 的 Disclaimer 直接称该仓库是基于开源 DreamerV2 的 DreamerV3 reimplementation，并与 Google/DeepMind 无关。[4] 因此面试中应说“该 frozen repo 的 reimplementation code path”，不能说“DreamerV3 官方作者代码”或从 repo 推导个人项目结果。

5. one-step、open-loop、actionability 与 metric boundary

以下内容属于 `framework`，不是某个论文的单一 benchmark recipe：

1. teacher-forced one-step 使用真实历史，open-loop rollout 使用模型自己的前一步预测；两者必须分开报告。
2. action-conditioned 只说明 action 进入条件，不自动证明可干预；action-swap、magnitude sweep、cloned-state intervention 与 closed-loop trace 是合理的验证设计。
3. PSNR/SSIM/FVD、latent KL、reward/value loss、action loss、open-loop error 与 closed-loop success 处于不同证据层，不能跨层替代。
4. aleatoric/epistemic 是分析分类；ensemble、posterior sample 或 disagreement 只是估计手段，不能自动等价于真实风险。

这些框架与 PlaNet/DreamerV3 对 long-horizon model bias、future prediction 和 imagined control 的讨论一致，但本地尚无 calibration、action-swap、world-model checkpoint 或 rollout trace，不能写成“已验证”。

```
def audit_world_model_claim(claim, paper, repo, local_artifacts):
    # 1. Split the claim into method, implementation, metric and ownership atoms.
    atoms = split_claim_into_atoms(claim)

    # 2. Match each atom to primary paper and frozen-code evidence.
    upstream = [match_primary_or_code(atom, paper, repo) for atom in atoms]

    # 3. Require checkpoint/runtime artifacts before calling a local result reproduced.
    if local_artifacts.missing("checkpoint", "config", "episodes"):
        return phrase_as_upstream_or_framework(upstream)

    # 4. Only a continuous paper -> repo -> runtime -> ownership chain allows a strong claim.
    return build_evidence_bounded_answer(upstream, local_artifacts)
```

6. 题目覆盖与回写矩阵

范围	复核状态	允许动作
`VLA-WM-001..010`	`PASS WITH WARNING`	可进入下一批知识页；保留 paper/reimplementation/local boundary
`VLA-WM-011..014`	`PASS WITH WARNING`	继续留在 Source Audit A，后续单独生成知识批
`VLA-WM-019..021`	`PASS AS FRAMEWORK`	指标与 imagination 机制可教，但不写本地结果
DreamerV3 repo provenance	`PASS WITH WARNING`	不称官方作者代码
PlaNet overshooting → DreamerV3	`BLOCKED FOR STRONG CLAIM`	不做跨方法实现归因
用户 WAM/UWM 项目证据	`BLOCKED`	等 checkpoint、normalizer、run/episode artifact

本次没有发现需要改写 Audit A 核心技术事实的错误；新增的边界是把“PlaNet overshooting 不可外推到 DreamerV3”显式列为禁止升级项。下一知识批只开放 `VLA-WM-001..010`，不生成题卡。

复核结论与未解决项

Source Audit A 的关键 primary/official-code 事实已独立复核，review 仍保持 `WARN`，原因是：没有 DreamerV3 checkpoint/result reproduction、没有用户本地 world-model artifacts、浏览器 `file://` 视觉 QA 受策略阻断、也没有真实口述。知识页可以进入下一增量，但不能把模块状态推进到 `reviewed` 或 `drilled`。

参考文献（References）

1. Ha & Schmidhuber, “World Models”, arXiv `1803.10122`，Ar5iv HTML；arXiv record。
2. Hafner et al., “Learning Latent Dynamics for Planning from Pixels” (PlaNet), arXiv `1811.04551`，Ar5iv HTML；arXiv record。
3. Hafner et al., “Mastering Diverse Domains through World Models” (DreamerV3), arXiv `2301.04104`，HTML。
4. `danijar/dreamerv3@e3f02248693a79dc8b0ebd62c93683888ddaccfe`，README、`rssm.py`、`agent.py`、`configs.yaml`、`LICENSE`。
5. 本仓库 Source Audit A canonical Markdown、HTML 与 review。

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 05 · Engineering / Evaluation

训练工程、评测协议、复现、失败诊断与部署证据

本卷由 6 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 05 Engineering / Evaluation：数据、训练、部署与闭环诊断（Batch 1）

题卡

1. 02 · VOL 05 Engineering / Evaluation：核心面试题卡（Batch 1）

来源与事实复核

1. 03 · VOL 05 Batch 1：独立官方文档事实复核
2. 04 · VOL 05 Batch 2A：独立官方文档事实复核
3. 05 · VOL 05：Completion Audit / Independent Fact Review
4. 06 · VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结

Part 1 / 6 · `interview_materials/vla/knowledge/vol_05_engineering_eval.html` · SHA-256 `CA3156014A30FDC4B6A20E538CA077C783252A73AD1DC980E5830ED72CDA417A`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`ca3156014a30fdc4b6a20e538ca077c783252a73ad1dc980e5830ed72cda417a`；生成自：`interview_materials\vla\knowledge\vol_05_engineering_eval.md`

VOL 05 Engineering / Evaluation：数据、训练、部署与闭环诊断（Batch 1）

状态：`drafted / knowledge_10_of_44 / cards_10_of_44_unseen / review_warn` 报告日期：2026-07-17 证据截止：2026-07-17 当前覆盖：`VLA-ENG-001..008`、`VLA-EVAL-001..002`，10/44 个知识节点 当前目标：把数据合同、分布式训练、checkpoint lineage、policy serving、closed-loop protocol 和 first-failure debugging 连成可恢复的工程闭环。 项目边界：本文只使用五项目 inventory 中已冻结的机制与风险；项目数字不会因为进入知识页而升级证据等级。

问题（Question）

Batch 1 回答 10 个依赖关系明确的问题：

1. `VLA-ENG-001`：VLA 数据 schema 为什么必须同时描述 observation、action、时间和语义？
2. `VLA-ENG-002`：怎样做 split、dedupe 和 leakage audit，避免同场景或同图片跨集合？
3. `VLA-ENG-003`：DataLoader、预处理、GPU 计算和同步怎样共同决定吞吐？
4. `VLA-ENG-004`：DDP、FSDP 与 ZeRO 分别切分什么，如何选择？
5. `VLA-ENG-005`：mixed precision、gradient accumulation、clipping 和 effective batch 怎样交互？
6. `VLA-ENG-006`：一个可恢复、可审计的 checkpoint bundle 至少包含什么？
7. `VLA-ENG-007`：policy server 的 latency budget 如何影响控制频率、chunk 和 replan？
8. `VLA-ENG-008`：模型 action 输出到 controller 之间需要怎样的 deployment contract？
9. `VLA-EVAL-001`：closed-loop 评估为什么必须冻结 protocol tuple 和 artifact lineage？
10. `VLA-EVAL-002`：offline proxy、format、geometry、sim success 与真机 success 怎样组成 metric ladder，并用 first failure 调试？

这 10 个节点组成以下链路：

```
dataset schema + temporal/action semantics
  -> grouped split / dedupe / leakage audit
  -> dataloader / preprocessing / device pipeline
  -> DDP/FSDP/ZeRO + precision/accumulation
  -> checkpoint + optimizer + processor + data lineage
  -> policy server + action/controller contract
  -> frozen closed-loop protocol
  -> metric ladder + first-failure trace
  -> data / model / runtime / controller / evaluator owner
```

知识地图位置与边界

前置知识

- VOL 01：tensor shape、数值稳定性、坐标系、控制频率与统计估计。
- VOL 02：SFT/RL 数据合同、distribution shift、reward 与 offline/online 指标。
- VOL 03：action representation、normalization、chunk、replan 和 controller boundary。
- VOL 04：world-model proxy、strict protocol、Sim2Real 与 benchmark 可比性。
- VOL 07：五项目 claim–evidence inventory 提供真实工程 bad cases，但不替代通用方法来源。[6][7][8][9][10]

当前批次不做什么

1. 不把某个框架写成所有训练的唯一答案；DDP/FSDP/ZeRO 的选择依赖模型、网络、显存和保存需求。
2. 不把 reproducibility 写成“固定随机种子即可”；kernel、数据顺序、环境和外部 simulator 同样属于合同。[11]

3. 不把离线 loss、tool-call accuracy、format pass 或 world-model proxy 外推为机器人闭环成功。
4. 不展开 `VLA-ENG-009..022` 与 `VLA-EVAL-003..022`；它们属于后续 data-scale、observability、serving、safety、统计与 deployment batches。
5. 当前题卡只覆盖这 10 个知识节点；其余 34 个节点仍为 planned，所有练习状态以主索引为准。

VLA-ENG-001：先冻结数据 Schema，再谈训练

最小样本合同

一个 VLA 训练样本不只是 `image + text + action`。至少应明确：

$$x_i = (o_{t-h+1:t},\ g,\ s_t,\ a_{t:t+H-1},\ m,\ \tau,\ \mu,\ \sigma).$$

变量解释：(o) 是一个或多个相机/历史观测，(g) 是语言或任务条件，(s) 是可选 proprio/state，(a) 是动作窗口，(m) 是 padding/valid mask，(au) 是时间戳/控制频率，(mu,sigma) 表示动作归一化统计。实现上还要记录 dataset/source/episode/robot/camera/action-space/schema-version 等 metadata。

合同维度	必须回答	常见 silent bug
observation	camera key、分辨率、颜色空间、历史长度、时间对齐	wrist/base view 交换，RGB/BGR 混用，重复最后帧
action	shape、轴序、单位、绝对/增量、关节/EEF、gripper 语义	同为 7D/12D 但维度意义不同
time	observation/action timestamp、control Hz、window stride	图像和动作错一帧，训练看似收敛但执行滞后
normalization	per-dataset/per-robot stats、clip、denorm	checkpoint 配错 normalizer，动作系统性过冲
mask	padding、invalid step、missing modality	padding 被当真动作，aux loss 污染主任务
provenance	dataset version、episode/group key、transform SHA	无法复现 split 或追踪异常样本

实现前置检查

```
def validate_vla_row(row, schema):
    assert set(row["observations"]) == set(schema.camera_keys)
    assert row["actions"].shape == (schema.horizon, schema.action_dim)
    assert row["valid_mask"].shape == (schema.horizon,)
    assert row["action_space"] == schema.action_space
    assert row["axis_order"] == schema.axis_order
    assert row["unit"] == schema.unit
    assert row["normalizer_id"] == schema.normalizer_id
    assert monotonic(row["timestamps"])
```

项目映射：OpenPiVLA 的 `H=15,D=72` 与 formatter `limit=16`、RoboCasa official12 action order、UWM checkpoint/normalizer 都说明“shape 对上”不等于动作语义闭合。[8][10]

VLA-ENG-002：Split、Dedupe 与 Leakage 是分层问题

为什么随机按行切分不够

机器人数据高度相关：同一 episode 的相邻窗口、同一场景的多标注、同一图片的多个 box、同一路径的增强版本可能具有不同 row ID，却共享几乎全部信息。安全 split 应优先按最上层独立单位分组：

```
robot / site / scene / episode / image identity
  -> group key
  -> train / validation / test assignment
  -> row-level transforms inside the assigned split
```

若 $G(x_i)$ 是样本的 group key，则基本约束是：

$$G(\mathcal{D}_{train}) \cap G(\mathcal{D}_{eval}) = \varnothing.$$

这只是必要条件。还应检查 source-line、内容 fingerprint、image hash、prompt-answer-image key、near duplicate 和训练图片集合。OPD raw-holdout 同时冻结了 source-line、fingerprint、image+prompt+answer 与 train-image gates；它也保留同图不同合法标注，因此“重复图片”与“重复样本”不能混为一谈。[7]

Audit 输出

层级	推荐输出	解释
row	bad JSON、exact duplicate、schema violation	文件级完整性
group	episode/scene/image overlap	独立性主门槛
semantic	normalized prompt/action fingerprint	重写或增强后的近重复
media	image/video hash、near-duplicate embedding	媒体复用与裁剪风险
provenance	source file/line、transform/version	可追溯性

危险做法：先随机行切分，再在每个 split 中独立去重；这样同一 episode/图片的近邻可能已泄漏到两边。

VLA-ENG-003：吞吐是流水线最慢阶段的函数

Step 时间分解

一个训练 step 可粗略写成：

$$T_{step} \approx \max(T_{data}, T_{compute}) + T_{sync} + T_{misc}.$$

其中 T_{data} 包含存储读取、decode、augmentation、collate 和 host→device；$T_{compute}$ 包含 forward/backward/optimizer；T_{sync} 是梯度或参数通信；T_{misc} 包含 logging、checkpoint、eval callback。prefetch 只能隐藏可并行的数据时间，不能消除磁盘随机读、epoch boundary 或内存不足。

诊断顺序

1. DataLoader-only benchmark：不建模型，测 `next(loader)` wait 分布。
2. 单卡 no-sync GPU smoke：区分 compute 与数据。
3. 多卡 step timing：记录 data_wait、forward/backward、all-reduce、optimizer、checkpoint。
4. 改一个变量做 A/B：workers、prefetch、persistent、pin、preload、chunk size、sampler。
5. 同时报告冷启动时间、CPU 内存、GPU 利用率与稳态吞吐。

UWM/LIBERO 的 DataLoader-only A/B 中，finite/infinite 的 p95 wait 都约 13s，preload+infinite 后约 0.19s；这支持 zarr 小 chunk 随机读的诊断，但不能直接宣称 full GPU training 已同比加速。[10]

VLA-ENG-004：DDP、FSDP 与 ZeRO 切分的对象不同

方法	每卡保留	通信主路径	适合情况	主要风险
DDP	完整参数、梯度；optimizer state 通常由各 rank 独立持有	backward all-reduce gradient	模型能单卡装下、追求简单稳定	显存随模型增大，optimizer state 重复
ZeRO-1	参数/梯度完整，切 optimizer state	optimizer state partition	optimizer state 是主要压力	框架 checkpoint/恢复语义更复杂
ZeRO-2	参数完整，切 optimizer state + gradient	reduce-scatter/all-gather	模型参数可装下，梯度压力大	通信与 bucket 配置敏感
ZeRO-3	参数、梯度、optimizer state 都切分	layer-wise all-gather/reduce-scatter	单卡装不下完整模型	通信/保存/加载复杂，外部模型共存更难
FSDP	参数/梯度/optimizer state 按 shard policy	unit-level all-gather/reduce-scatter	PyTorch-native sharding、可配置 wrap	wrap policy、state-dict 类型和 tied weights

DDP 的核心语义是各 rank 复制模型、对梯度同步；FSDP/ZeRO-3 在执行层附近聚合分片参数。不能只按“哪个更省显存”选择，还要冻结：world size、shard/wrap policy、precision、activation checkpointing、offload、通信 bucket、save/load state dict 与 resume test。[1][2][4]

> 复核注：DDP 文档直接保证 replica、gradient all-reduce 与输入不自动 shard；“optimizer state 重复”是每进程 optimizer 的常见实现推论，不是 DDP 通信 API 单独保证的对象。详见 VOL 05 Batch 1 独立官方文档复核。

粗略显存账本

设参数量为 P，每参数权重、梯度、optimizer state 分别占 (b_w, b_g, b_o) bytes，未分片训练状态近似：

$$M_{state} \approx P(b_w + b_g + b_o).$$

实际显存还包括 activation、temporary buffer、KV/cache、teacher models、fragmentation 和 framework overhead；因此参数账本不能替代实测峰值。OPD 五 teacher `preloaded_zero3` 是“多个教师共存+分片”的项目例子，但不证明 ZeRO-3 对所有 VLA workload 最优。[7]

VLA-ENG-005：Mixed Precision、Accumulation 与 Clipping

Effective batch

若每卡 micro batch 为 B_μ，梯度累积步数为 A，world size 为 W，则全局 batch 近似：

$$B_{global} = B_\mu \times A \times W.$$

还必须说明这是 sample、sequence、frame 还是 token 数；多模态样本长度差异大时，同一个 sample batch 的实际 token/pixel/action workload 可能相差很大。

精度与稳定性合同

- FP16 通常需要 dynamic loss scaling 保护小梯度；BF16 exponent 范围更大，但并不保证所有算子稳定。[3]
- accumulation 前应按框架语义正确缩放 loss；scheduler/optimizer step 频率必须按 optimizer step 而非 micro step 解释。
- gradient clipping 应在 unscale 后、optimizer step 前执行；分片训练要使用框架提供的 global norm 语义。
- 日志至少记录 loss、grad norm、scale、overflow/skipped step、learning rate、effective batch、tokens/frames per step。

项目映射：OPD report 记录 grad norm、entropy、overlap 与 route loss；OpenPiVLA 的 eager/MACA 属于兼容性路径，不等于 mixed precision 自动稳定。[7][8]

VLA-ENG-006：Checkpoint 不是一个 `models.pt` 路径

可恢复 bundle

最小 checkpoint lineage 应绑定：

```
run_id + git/source SHA + config/CLI/env
  + base model / adapter / processor
  + model state + optimizer + scheduler + scaler
  + global step / epoch / sampler RNG / data cursor
  + dataset manifest + normalizer + schema
  + checkpoint hash + load-smoke stdout
  + eval harness/scorer/protocol + raw predictions/episodes
```

只保存模型权重可以做 inference candidate，却不能保证训练无缝 resume；只保存 optimizer 而缺 sampler/data cursor，恢复后的样本顺序和随机增强仍会变化。分布式 checkpoint 还要明确 full/sharded state dict、world-size reshard、rank ownership 和原子完成标记。[2][4][12]

Resume 验收

1. 在 step (k) 保存，独立进程加载；
2. 校验参数/processor/normalizer/config hash；
3. 对固定 batch 比较 pre-save 与 post-load 输出；

4. 继续至少一个 optimizer step，检查 finite loss/grad 与 step 递增；
5. 用同 eval harness 跑最小 smoke。

OpenPiVLA 的 remote path 无本地 checkpoint/load smoke、UWM 5-task final 无 raw checkpoint bundle，正说明“报告完成”与“lineage verified”是不同证据等级。[8][10]

VLA-ENG-007：Policy Server 的 Latency Budget

端到端延迟

$$T_{loop} = T_{sense} + T_{pre} + T_{queue} + T_{infer} + T_{network} + T_{decode} + T_{control}.$$

若每次请求输出 H 步、实际执行 $K \le H$ 步，控制周期为 Δt，策略最好满足：

$$T_{loop} < K\Delta t$$

才能在执行完当前子块前得到下一次决策。更严格的闭环还要预留 jitter、timeout 和安全 stop。提升 (K) 可以容忍更大推理延迟，却增加开环误差；减小 (K) 提高纠偏频率，却增加 server/网络/推理负载。

服务端必须记录

层	指标	失败模式
ingest	sensor timestamp、queue depth、drop rate	stale observation、拥塞
inference	batch size、model/processor version、p50/p95/p99	tail latency、OOM、动态 batching 抖动
action	chunk age、valid horizon、decode/denorm time	执行过期动作、维度/单位错误
controller	accepted/rejected、saturation、stop reason	控制器拒绝或裁剪模型动作
episode	request/action/observation correlation ID	无法定位 first failure

Harness 的 cancel/compensation、OpenPiVLA 的 action/controller roundtrip 缺口可直接作为 latency 与执行语义的项目追问。[8][9]

VLA-ENG-008：Action → Controller Deployment Contract

模型输出到控制器之间至少需要以下握手：

```
model output
  -> parser / format validation
  -> action tensor shape check
  -> denormalization
  -> axis/order/unit transform
  -> absolute/delta integration
  -> clipping / rate / workspace limits
  -> controller command
  -> acknowledgement / observation feedback
```

合同字段包括：`schema_version`、`model/checkpoint`、`normalizer_id`、`action_space`、`axis_order`、`unit`、`frame`、`horizon`、`execution_steps`、`control_hz`、`timestamp`、`valid_until`。任何 silent default 都可能让 offline prediction 看起来正确、closed-loop 却完全失败。

Roundtrip 测试

用 sentinel action 对每个维度赋唯一值：

$$a = [0.01, 0.02, \ldots, 0.01D].$$

依次通过 serialize→parse→denorm→frame/axis mapping→controller adapter，再检查每一维的位置、符号、单位和饱和。还要测试 NaN、缺维、额外字段、旧 schema、过期 timestamp 和取消。RoboCasa action order、OpenPiVLA formatter shape 冲突、Harness

schema-aware runtime 都是该节点的真实 hook。[8][9][10]

VLA-EVAL-001：Closed-Loop Protocol 是可重放 Tuple

“strict”不是形容词，而是一组可核对字段：

$$\Pi = (task, env, init, warmup, obs, action, normalizer, chunk, controller, horizon, success, seed, trials, checkpoint, runtime).$$

只有关键字段一致，成功率差异才主要能归因于模型；任务名相同不等于 protocol 相同。UWM fixed-pruned-init、UWM Table II OOD 和 repo reset-style 使用不同 init/seed/环境设置，所以 0.744、0.79、0.928 不能直接排成模型榜单。[5][10]

Run / Episode manifest

每个 run 保存：repo/config/checkpoint/processor/normalizer SHA、environment commit、task list、protocol tuple、launcher CLI、机器/依赖和 scorer。每个 episode 保存：episode ID、seed/init index、observation/action timestamps、模型输出、controller command、success/termination、视频/trace path 和 first failure。

成功率估计：

$$\hat{p} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[success_i].$$

若比较两个策略，优先在相同 init/seed 上做 paired eval，并同时报告分母、per-task、置信区间和 failure distribution；只报 macro average 会隐藏任务难度差异。

VLA-EVAL-002：Metric Ladder 与 First-Failure Debugging

指标层级

层级	例子	能支持	不能支持
training	loss、grad norm、entropy	优化是否运行、数值是否异常	策略能完成任务
offline action	MSE、NLL、token/format exact	demonstration distribution 上的拟合	closed-loop recovery
semantic/geometry	tool schema、box IoU、point Hit@K	接口/grounding 子能力	控制器/环境成功
world-model proxy	open-loop prediction、VLM score	未来预测/语义一致性	可执行因果与真实 success
simulator closed-loop	environment binary success	指定 simulator/protocol 下成功	真机泛化、不同 protocol
robot closed-loop	task success、safety/latency	指定硬件/场景的部署结果	所有地点/机器人泛化

First-failure trace

不要从最终失败反向猜一个故事；按时间找第一个合同被破坏的位置：

```
data / observation
  -> perception / grounding
  -> model decode
  -> parser / normalizer / action mapping
  -> network / queue / timeout
  -> controller acceptance
  -> environment transition
  -> success predicate / logging
```

第一处偏差决定主要 owner：数据/感知、模型、serving、controller、environment 或 evaluator。若第一处错误是 normalizer，后面的“抓取失败”只是后果；若 action trace 正确但 success predicate 错，训练模型没有意义。

最小诊断矩阵

1. 固定 checkpoint，变化 protocol：检测 eval confound。
2. 固定 protocol，变化 checkpoint：做真正模型比较。

3. 固定输入，重放 parser/denorm/controller：检测 deployment contract。
4. 固定动作，比较 simulator/robot transition：检测 environment gap。
5. 保留 raw sample/episode 与 paired diff：避免只看 aggregate。

OPD 的 point protocol mismatch、Harness 的 proxy/safety/browser 边界、UWM 的 reset/fixed/Table II 分层和 WAM 的 raw-artifact 缺口分别对应 ladder 中不同断点。[6][7][9][10]

Batch 1 复习表

ID	30 秒抓手	必须追问的证据
`VLA-ENG-001`	schema 不只写 shape，还写轴序、单位、时间、normalizer	schema/version + sentinel roundtrip
`VLA-ENG-002`	按 episode/scene/image group split，再做多层 dedupe	overlap/fingerprint/image audit
`VLA-ENG-003`	step 时间由 data、compute、sync、misc 共同决定	分阶段 timing + A/B
`VLA-ENG-004`	DDP 复制；FSDP/ZeRO 分片对象与保存语义不同	world/wrap/shard/state-dict
`VLA-ENG-005`	global batch=micro×accum×world；精度和 clipping 有顺序	scale/overflow/grad norm/step
`VLA-ENG-006`	checkpoint 是可恢复 lineage，不是路径	optimizer/scheduler/RNG/data cursor/load smoke
`VLA-ENG-007`	latency budget 决定 chunk/replan trade-off	p95/p99、queue、chunk age
`VLA-ENG-008`	action 到 controller 必须逐维可验证	normalizer/axis/unit/frame/expiry
`VLA-EVAL-001`	protocol 是 tuple；同任务不等于可比	init/warmup/horizon/success/seed/checkpoint
`VLA-EVAL-002`	metric 有层级；沿时间找 first failure	raw episode + paired diff

证据与真实性边界

类型	本批使用方式
官方文档/仓库	支持 DDP/FSDP/AMP/ZeRO/checkpoint 与 LIBERO protocol 的机制事实
本地 source audit	支持 UWM/LIBERO protocol、UWM architecture 和项目数字不可比性
项目 inventory	提供 schema、lineage、protocol、latency、proxy 和 ownership 的真实 bad case
推断	只用于提出诊断假设，必须给 paired test 或缺失 artifact

当前 review 保持 `WARN`：DDP/FSDP/ZeRO/AMP/DCP/Reproducibility 的官方机制已完成独立复核，但本地 workload artifacts、resume/performance smoke 和候选人口述仍缺失；HTML 生成或 source review 不等于 drilled。

References

1. PyTorch official documentation, `DistributedDataParallel`. Docs
2. PyTorch official documentation, `FullyShardedDataParallel`. Docs
3. PyTorch official documentation, Automatic Mixed Precision. Docs
4. Microsoft DeepSpeed official documentation, ZeRO. Docs
5. VOL 04 Benchmark / Evidence Source Audit：LIBERO/UWM/Cosmos protocol tuple 与可比性。
6. WAM Claim–Evidence Inventory：raw artifact 和 metric provenance 缺口。
7. OPD Claim–Evidence Inventory：data routing、raw-holdout、point protocol 与 checkpoint selection。
8. OpenPiVLA Claim–Evidence Inventory：checkpoint lineage、action representation 与 parser/controller 缺口。
9. Robot Harness Claim–Evidence Inventory：runtime、cancel/compensation、proxy/safety/browser 边界。
10. UWM / LIBERO Claim–Evidence Inventory：fixed-pruned-init/reset/Table II、normalizer、stall 与 RoboCasa boundary。
11. PyTorch official documentation, Reproducibility. Docs
12. PyTorch official documentation, Distributed Checkpoint. Docs
13. VOL 05 Batch 1 独立官方文档事实复核；版本化 PyTorch 2.13 与 DeepSpeed ZeRO tutorial 的逐条核对。

Part 2 / 6 · `interview_materials/vla/questions/vol_05_engineering_eval.html` · SHA-256 `C1EBFB0608AF37E47C9F71EEB592E2B2E108116AECA60DB8E61606FE5395CA05`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`c1ebfb0608af37e47c9f71eeb592e2b2e108116aeca60db8e61606fe5395ca05`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_05_engineering_eval.md`

VOL 05 Engineering / Evaluation：核心面试题卡（Batch 1）

状态：`drafted / cards_10_of_44 / review_warn` 更新日期：2026-07-17 当前题数：10 / 44 覆盖范围：`VLA-ENG-001..008`、`VLA-EVAL-001..002` 难度：L2 8 题 / L3 2 题 优先级：P0 10 题 练习状态：全部 `unseen` 使用方式：先闭卷说 30 秒版本，再展开机制与诊断；未真实口述前不得改为 pass/strong。 深度来源：VOL 05 Engineering / Evaluation Batch 1 知识页；项目 hook 只沿用五项目 inventory 的证据边界，不因进入题卡而升级。

A. 数据合同、切分与吞吐

▼ `L2` `P0` VLA-ENG-001 · 一个可训练、可部署的 VLA 数据 schema 至少要冻结哪些字段?

面试意图：判断你是否理解 VLA 样本是带时间、动作语义和来源的合同，而不是只检查 tensor shape。

30 秒回答：我会同时冻结 observation、语言/状态、action window、mask、timestamp/control Hz、normalizer，以及 robot/camera/source/schema version。action 还要写清轴序、单位、坐标系、绝对或增量语义；shape 相同不代表能被同一 controller 正确执行。

机制展开（约 2 分钟）：样本可写成 `history + goal + proprio -> action window`，但每项都要带语义：相机 key、颜色空间、历史对齐，动作的 horizon/维度/轴序/单位/frame，padding/invalid mask，normalizer 与 provenance。训练入口先做 schema assertion；部署前用逐维唯一的 sentinel action 跑 serialize→parse→denorm→frame/axis mapping→controller adapter。这样能把“模型学错”与字段交换、错帧、错单位或统计量错配分开。

诊断 / 证据：保存 schema JSON/version、dataset manifest、normalizer hash 与 roundtrip 输出；抽查 timestamp 单调性、camera/action key、mask 和每维 sentinel。只有 tensor shape 或 formatter pass 不能关闭语义风险。

项目挂钩：OpenPiVLA 的 `H=15,D=72` 与 formatter `limit=16` 冲突、RoboCasa official12 action order、UWM normalizer 缺口都可作为真实 bad case；不把它们说成已完成闭环修复。

追问树：为什么 schema version 必须进 checkpoint？normalizer 属于数据还是模型？padding mask 错会怎样？同为 7D action 为什么不能直接复用?

危险答案：说“shape 对上就能训练和部署”，或默认为所有机器人都使用同一轴序、单位和 gripper 语义。

来源：[1][8][10]　**练习状态**：unseen

▼ `L2` `P0` VLA-ENG-002 · VLA 数据应怎样做 split、dedupe 和 leakage audit?

面试意图：检查你能否区分 row、episode/scene/image group、语义指纹与媒体复用等不同泄漏层级。

30 秒回答：不能随机按行切。先按 robot/site/scene/episode/image 的最高独立单位构造 group key，再分 train/eval；之后审计 source-line、fingerprint、训练图片与 near duplicate。同图多标注不一定是重复样本，但跨 split 共享内容必须阻断。

机制展开（约 2 分钟）：相邻窗口、多标注和增强版本可能 row ID 不同却共享几乎全部信息，所以先保证 train/eval 的 group key 不相交，再在已冻结 split 上做 exact、semantic 与 media dedupe。审计至少输出坏行/schema、group overlap、normalized fingerprint、image/video hash、near duplicate 和 source provenance。去重规则还要保留合法的同图多 object/region 标注；先随机切行再各自去重，会把同一 episode 或图片的近邻留在两侧。

诊断 / 证据：输出每层 denominator、overlap count、样本 key 与 source line；对命中项人工抽查，并冻结 split manifest/transform SHA。零 exact duplicate 不等于零 near-duplicate 或零分布泄漏。

项目挂钩：OPD raw-holdout 的 source-line、fingerprint、image+prompt+answer 和 train-image gates 是可讲证据；同图多合法标注说明 unique image 少于样本数并不自动等于泄漏。

追问树：group key 选 episode 还是 scene？同图多标注如何处理？为什么 split 后去重仍可能泄漏？near duplicate 怎么定阈值？

危险答案：说“随机打乱后 8:2 就足够”，或看到重复图片路径就直接判定训练测试泄漏。

来源：[1][7]　练习状态：unseen

▼ `L2` `P0` **VLA-ENG-003 · 怎样定位 VLA 训练吞吐瓶颈，而不是盲目增加 DataLoader workers？**

面试意图：判断你能否把 data、compute、communication 和 callback 的时间拆开，用 A/B 证据定位瓶颈。

30 秒回答：先拆 step 的 data wait、compute、同步、optimizer 与 callback，记录分位数和 GPU 利用率。依次跑 DataLoader-only、单卡、多卡 timing，再一次只改 workers/prefetch/pin/preload/sampler。prefetch 只能隐藏可并行时间，修不了随机读或内存瓶颈。

机制展开（约 2 分钟）：稳态 step 可近似 `max(data, compute)+sync+misc`，但冷启动、epoch boundary 和 checkpoint spike 要另报。DataLoader-only 排除模型，单卡区分数据与 compute，多卡暴露 all-reduce/shard communication；每次 A/B 同时看 p50/p95 wait、CPU 内存、GPU active、samples/tokens/frames per second。workers 增加后若存储争用、decode 饱和或 page cache thrash，吞吐反而下降；preload 也可能用内存换速度。

诊断 / 证据：保留阶段 timing trace、硬件/存储、loader config、冷启动与稳态区间；只从 `next(loader)` 变快不能外推 full GPU training 同比例加速。

项目挂钩：UWM/LIBERO 的 DataLoader-only A/B 支持 zarr 小 chunk 随机读诊断与 preload 方向，但没有 full training trace，回答时必须保留这一层证据边界。

追问树：GPU 利用率低一定是 DataLoader 吗？为什么看 p95？prefetch 何时无效？怎样判断通信而非计算成为瓶颈？

危险答案：只说“把 `num_workers` 调大”，或把 DataLoader-only 加速直接写成端到端训练加速。

来源：[1][10]　练习状态：unseen

B. 分布式训练、数值稳定与恢复

▼ `L2` `P0` **VLA-ENG-004 · DDP、FSDP 与 ZeRO 分别切分什么，实际如何选择？**

面试意图：检查你是否理解复制、分片、通信和 checkpoint 语义，而不是只会说“ZeRO-3 最省显存”。

30 秒回答：DDP 每卡复制模型参数并 all-reduce 梯度，optimizer state 通常各 rank 独立持有；ZeRO-1/2/3 依次分片 optimizer、gradient、parameter；FSDP 按 wrap/shard policy 分片三者。选择还要看显存、网络、吞吐及恢复复杂度。

机制展开（约 2 分钟）：DDP 简单稳定，适合模型状态能放入单卡；ZeRO-1/2/3 依次切 optimizer、再切 gradient、再切 parameter；FSDP 是 PyTorch-native 的 unit-level all-gather/reduce-scatter。显存账本除参数/梯度/optimizer 外，还包括 activation、temporary buffer、teacher、fragmentation。方案冻结 world size、wrap/shard、precision、activation checkpointing/offload、bucket 与 state-dict 类型，并用 peak memory、step time 和独立 resume test 验收。

诊断 / 证据：先单卡测状态与 activation，再逐级启用 sharding；记录 per-rank peak、通信占比、OOM 点、save/load 时间和 world-size reshard。框架机制不等于当前 workload 的性能结论。

项目挂钩：OPD 五 teacher `preloaded_zero3` 可说明多模型共存的显存压力；它不证明 ZeRO-3 对所有 VLA workload 最优，也不替代完整 checkpoint bundle。

追问树：ZeRO-2 为什么参数仍可能放不下？FSDP wrap policy 影响什么？activation 显存为何没被参数分片解决？怎样保存可部署权重？

危险答案：把 FSDP 和 ZeRO-3 说成完全相同实现，或只按“最省显存”选择而不测通信和恢复。

来源：[1][2][4][7]　练习状态：unseen

▼ `L2` `P0` **VLA-ENG-005 · mixed precision、gradient accumulation、clipping 与 effective batch 怎样正确组合?**

面试意图：判断你能否说清 optimizer-step 语义、数值顺序和可观察信号，而不是只背一个 batch 公式。

30 秒回答：batch≈micro×accum×world，并说明按 sample/token/frame/action 计。micro step 只累积；完成后先 unscale，再算 global grad norm 与 clipping，最后 optimizer/scheduler step。FP16 另看 scale、overflow、skipped step。

机制展开（约 2 分钟）：accumulation 改变 optimizer 更新频率，scheduler、logging 和 checkpoint 的 step 定义必须一致；多模态 variable-length 样本即使 sample batch 相同，pixel/token/action workload 也不同。FP16 常靠 dynamic loss scaling，BF16 范围更大但不保证所有算子稳定。若在 unscale 前裁剪，阈值语义会错；分片训练要用框架的 global norm。增大 global batch 还可能需要重调学习率与 warmup，不能只为“等价单卡”。

诊断 / 证据：记录 micro/optimizer step、effective workload、loss scale、overflow/skipped、grad norm、clip ratio、LR 与 finite check；比较配置时按相同 optimizer-step 或样本预算对齐。

项目挂钩：OPD 的 grad norm、entropy、overlap 和 route loss 可作为训练可观察性 hook；OpenPiVLA 的 eager/MACA 是兼容路径，不代表 mixed precision 已自动稳定。

追问树：scheduler 按 micro step 会怎样？BF16 是否完全不需要检查溢出？为什么 clipping 要在 unscale 后？global batch 增大后 LR 是否必然线性放大?

危险答案：说“BF16 永远稳定”，或在每个 micro step 都执行 optimizer/scheduler，却仍声称做了梯度累积。

来源：[1][3][7][8]　练习状态：unseen

▼ `L3` `P0` **VLA-ENG-006 · 什么才算可恢复、可审计的 VLA checkpoint bundle?**

面试意图：考查你能否区分权重路径、推理候选、训练恢复与评估可追溯性，并给出实际验收步骤。

30 秒回答：checkpoint 不只是权重：需绑定源码/config、processor、optimizer/scheduler/scaler、step、RNG、data cursor、dataset/normalizer/schema 与 SHA。独立进程加载后，复现固定 batch 输出、续训一步，再跑同协议 eval smoke。

机制展开（约 2 分钟）：权重+processor+normalizer 可形成 inference candidate，但无 optimizer/scheduler/scaler 不能无缝续训；无 RNG、sampler 与 data cursor，样本顺序和增强会漂移。分布式保存还要说明 full/sharded state dict、rank ownership、world-size reshard 与原子完成。验收流程是在 step k 保存，独立进程校验 hash，对固定 batch 比较 pre/post-load 输出，继续一步检查 finite loss/grad/step，再用同一 harness 跑最小 eval smoke。

诊断 / 证据：保留 manifest、SHA、保存成功标记、load stdout、固定 batch diff、resume step 与 eval smoke；“远端有一个路径”或“报告写完成”都不是 checkpoint-verified。

项目挂钩：OpenPiVLA remote path 缺本地 checkpoint/load smoke，UWM 5-task final 缺 raw bundle；它们适合说明 lineage gap，不应在题卡中升级为已验证模型资产。

追问树：为什么只保存 RNG 仍不够？processor/normalizer 为什么要跟权重绑定？sharded checkpoint 如何跨 world size？怎样防止半写入 checkpoint 被加载?

危险答案：说“保存 `model.pt` 就能完全 resume”，或把文件存在、能下载与能重放评估视为同一证据等级。

来源：[1][2][8][10][12]　练习状态：unseen

C. Policy Serving 与动作执行

▼ `L2` `P0` **VLA-ENG-007 · policy server 的 latency budget 如何影响 action chunk 与 replan?**

面试意图：检查你能否把端到端 tail latency、控制周期和开闭环 trade-off 连起来，而不是只报模型推理耗时。

30 秒回答：预算要覆盖 sense、preprocess、queue、infer、network、decode 和 control，不是只有 GPU time。若每次预测 H 步、执行 K 步、周期为 Δt，最好让端到端延迟小于 `KΔt` 并留 jitter/timeout 余量。K 大能掩盖慢推理但更开环，K 小纠偏快却增加请求与算力压力。

机制展开（约 2 分钟）：server 要记录 sensor timestamp、queue depth、request/episode correlation ID、模型/processor 版本、p50/p95/p99、chunk age 与 valid horizon。动态 batching 可提高吞吐却增加排队；网络抖动或 stale observation 会让平均延迟正常但错过 deadline。执行当前子块时可异步请求下一块，但必须定义过期、乱序、取消、fallback 和安全 stop；不能用旧 action 填满空窗而无告警。

诊断 / 证据：分别压测 ingest、queue、inference、network、decode 和 controller，报告 deadline miss、drop、stale chunk 与尾延迟；对 K/replan 做 same-task A/B，同时看 success 与安全事件。

项目挂钩：Harness 的 cancel/compensation 与 OpenPiVLA 的 action/controller roundtrip 缺口可作为追问；当前证据不能证明真实机器人 latency 或 safety 已闭环。

追问树：为什么平均 latency 不够？动态 batching 的风险？K=H 有什么问题？timeout 后应该 stop、hold 还是 fallback?

危险答案：只报模型 forward 毫秒数就称达到控制频率，或认为增大 chunk 永远能解决延迟。

来源：[1][8][9]　**练习状态**：unseen

▼ `L3` `P0` **VLA-ENG-008 · 模型 action 到 controller 之间需要怎样的 deployment contract?**

面试意图：考查你能否把 parser、denormalization、坐标变换、安全限制与 acknowledgement 做成逐维可验证的接口。

30 秒回答：链路为 parse/shape→denorm→axis/unit/frame→absolute/delta→limits→controller→ack。请求带 schema、checkpoint、normalizer、轴序、单位、frame、H/K、Hz 和有效期；禁用 silent default，并以逐维 sentinel 做 roundtrip。

机制展开（约 2 分钟）：模型数值本身不具备可执行语义，adapter 必须把训练合同显式映射到 controller。用 `0.01,0.02,...` 的 sentinel action 跑 serialize→parse→denorm→坐标/轴变换→adapter，逐维检查位置、符号、单位与饱和；再测试 NaN、缺维、额外字段、旧 schema、过期时间、乱序和取消。controller 的 accepted/rejected、实际 command、clipping 与 stop reason 要回写同一 correlation ID，才能找 first failure。

诊断 / 证据：保存 contract schema、golden roundtrip、每维误差、异常输入结果、controller ack 与 observation feedback；offline action metric 不能代替这条执行链。

项目挂钩：RoboCasa official12 action order、OpenPiVLA formatter shape 冲突和 Harness schema-aware runtime 是可用 hook；不把 browser/sandbox tool success 说成真实 controller success。

追问树：为什么 clip 不能静默？绝对与增量动作如何测试？frame 变换错一轴会怎样？controller 拒绝后 episode 应如何记账?

危险答案：认为 parser 能输出浮点列表就代表部署完成，或让 normalizer、单位与坐标系使用未记录默认值。

来源：[1][8][9][10]　**练习状态**：unseen

D. Closed-loop 评估与 First-Failure

▼ `L2` `P0` **VLA-EVAL-001 · 怎样把 closed-loop 评估协议写成可重放、可比较的合同?**

面试意图：判断你是否会冻结模型、环境、初态、执行与判定全链路，而不是把“strict”当形容词。

30 秒回答：冻结 task/env、init/warmup、obs、action/normalizer、chunk、controller/Hz、horizon、success、seed/trials、checkpoint/runtime。manifest 绑代码/资产 SHA；episode record 留输出、command、终止与 first failure。

机制展开（约 2 分钟）：同一任务名不代表协议相同。比较模型时先 diff protocol tuple，关键字段一致后才优先归因 checkpoint；A/B 复用相同 task/init/seed 做 paired eval。每个 run 保存 repo/config/checkpoint/processor/normalizer/environment/scorer 版本和 launcher；每个 episode 是 immutable record，保留成功分母、invalid/timeout 规则与原始 trace。聚合时同时报总数、per-task、置信区间和 failure distribution，不能只留一个 macro average。

诊断 / 证据：先做 manifest diff 和缺失字段 gate，再核对 episode denominator、seed/init coverage、success predicate 与 raw trace；协议不一致时只报告不同 setting，不做模型排名。

项目挂钩：UWM fixed-pruned-init 的 0.744、Table II OOD 的 0.79、reset-style diagnostic 的 0.928 使用不同协议，必须分层叙述，不能直接比较。

追问树：为什么 warmup 与 horizon 都要冻结？paired eval 有什么价值？invalid episode 是否可删除？macro 与 micro 何时不同?

危险答案：说“都是 LIBERO、都是 50 rollouts，所以可以直接比较”，或只保存 aggregate success rate。

来源：[1][5][10]　练习状态：unseen

▼ L2 P0 **VLA-EVAL-002 · offline proxy 到真机 success 的 metric ladder 如何划分？闭环失败怎样找 first failure？**

面试意图：检查你能否限制不同指标的证据范围，并沿时间把失败路由到正确 owner。

30 秒回答：指标分 training、offline action、format/geometry、world proxy、sim/robot closed-loop；低层不能外推高层。失败时沿 obs→decode→parser/normalizer→network→controller→environment→predicate 找首个偏差，再定 owner。

机制展开（约 2 分钟）：loss/grad 证明优化在运行；MSE/NLL 证明 demonstration split 上拟合；format/IoU/Hit@K 证明接口或 grounding；world proxy 证明特定 scorer 下未来预测；sim/robot success 只在冻结协议和硬件范围内成立。调试时用同一 correlation ID 对齐 raw observation、prediction、denorm action、queue、controller ack、transition 与 scorer。第一处合同破坏决定数据/模型/serving/controller/environment/evaluator owner，后续失败只是传播结果。

诊断 / 证据：固定 checkpoint 改 protocol、固定 protocol 改 checkpoint、固定输入重放 parser/controller、固定动作比较 environment transition，并保存 raw episode 与 paired diff。

项目挂钩：OPD point protocol mismatch、Harness proxy/safety 边界、UWM 三种协议和 WAM raw-artifact 缺口分别位于 ladder 的不同层，不应合并成机器人成功结论。

追问树：tool accuracy 能证明什么？sim success 为什么不等于真机？normalizer 错时谁是 owner？action trace 正确但 success predicate 错该修哪里？

危险答案：看到 loss 或 proxy 变好就声称机器人能力提升，或从“抓取失败”直接反推 backbone 不够强。

来源：[1][5][6][7][9][10]　练习状态：unseen

参考来源

1. VOL 05 Engineering / Evaluation Batch 1 知识页；canonical Markdown。
2. PyTorch official documentation, FullyShardedDataParallel。
3. PyTorch official documentation, Automatic Mixed Precision。
4. Microsoft DeepSpeed official documentation, ZeRO。
5. VOL 04 Benchmark / Evidence Source Audit：LIBERO/UWM/Cosmos protocol tuple 与可比性。
6. WAM Claim–Evidence Inventory：raw artifact 和 metric provenance 缺口。
7. OPD Claim–Evidence Inventory：data routing、raw-holdout、point protocol 与 checkpoint selection。
8. OpenPiVLA Claim–Evidence Inventory：checkpoint lineage、action representation 与 parser/controller 缺口。
9. Robot Harness Claim–Evidence Inventory：runtime、cancel/compensation、proxy/safety/browser 边界。
10. UWM / LIBERO Claim–Evidence Inventory：fixed-pruned-init/reset/Table II、normalizer、stall 与 RoboCasa boundary。
11. PyTorch official documentation, DistributedDataParallel。
12. PyTorch official documentation, Distributed Checkpoint。

Part 3 / 6 · `interview_materials/vla/sources/vol_05_batch1_independent_fact_review.html` · SHA-256 `F0EF25BBB8C1D296B303B0ED47A3221D2EA63DF0D1829BD05879C2CF119353F9`

返回本卷目录

Canonical Markdown SHA-256：`f0ef25bbb8c1d296b303b0ed47a3221d2ea63df0d1829bd05879c2cf119353f9`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_05_batch1_independent_fact_review.md`

VOL 05 Batch 1：独立官方文档事实复核

状态：`reviewed_slice / WARN` 复核日期：2026-07-17 证据截止：2026-07-17 复核范围：`VLA-ENG-003..006` 的知识页、对应题卡，以及 DDP、FSDP、ZeRO、AMP、Distributed Checkpoint、Reproducibility 官方文档。 结论先行：分布式同步/分片、AMP unscale/clipping/accumulation、DCP load-time reshard 和“固定 seed 不保证跨版本/平台复现”均有官方文档支持；吞吐分解、checkpoint lineage、RNG/data cursor、项目数字与个人 ownership 仍是本仓库工程框架或本地证据边界。

问题（Question）

本记录回答：VOL 05 Batch 1 中哪些训练工程断言可以回到独立官方来源？哪些说法只是实现推断或面试设计框架？是否需要对题卡/知识页做最小修订？

本次复核只读取冻结版本的官方 HTML，不执行训练、分布式作业、checkpoint resume 或性能 benchmark；因此“官方机制已核对”不等于“本地 workload 已复现”。

复核方法与证据等级

等级	含义	允许的面试口径
`official-checked`	直接读取版本化官方文档的 API/机制原文	“PyTorch 2.13 文档说明……”
`official-tutorial-checked`	直接读取第一方 tutorial 的阶段定义或示例	“DeepSpeed 官方 tutorial 将……定义为……”
`framework`	本仓库的工程分解、验收顺序或诊断建议	“我会这样拆分/验收……”
`inference`	从每进程/每 rank 的实现语义推导出的结果	“通常意味着……但不是 API 保证”
`blocked`	需要本地运行、checkpoint、性能 trace 或 ownership 才能证明	不升级为本地结果

所有 PyTorch 来源固定到 `docs/2.13/` 路径，读取日期为 2026-07-17；页面在读取时的 `Last-Modified` 为 2026-07-16 或 2026-07-17。DeepSpeed 页面没有版本化路径，使用读取日期与响应摘要作为边界。

1. DDP：官方保证的是 replica 上的 gradient synchronization

PyTorch 2.13 的 `DistributedDataParallel` 文档明确说：DDP 在每个 model replica 之间同步梯度；它不会替用户切分或 shard 输入，输入切分应由 `DistributedSampler` 等上层逻辑负责。文档还写明参数不会在进程间持续 broadcast，而是对梯度做 all-reduce，并假设各进程的 optimizer 以相同方式修改参数；buffers（例如 BatchNorm 统计量）则由 rank 0 在每次迭代 broadcast。[1]

这直接支持题卡 `VLA-ENG-004` 的“DDP 复制模型、反向同步梯度、输入切分由用户负责”。但“每卡复制 optimizer state”是每进程各自构造 optimizer 的常见实现推论，不是 DDP 通信 API 本身保证的对象。因此题卡短答应说“通常各 rank 独立持有 optimizer state”，不要把它写成 DDP 内部的同步协议。

2. FSDP：sharding strategy 决定 all-gather、reduce-scatter 与 reshard 时机

PyTorch 2.13 FSDP 文档把 `FULL_SHARD` 与 `SHARD_GRAD_OP` 的差异写得很具体：两者都在 backward 后对 gradient 做同步/分片，并由每个 rank 本地更新 sharded optimizer state；`FULL_SHARD` 在 forward/backward 周围更积极地 reshard 参数，而 `SHARD_GRAD_OP` 在 forward 后保留 unsharded parameters，到 backward 结束后才重新分片。[2]

文档还把 all-gather/reduce-scatter 作为 FSDP process group 的核心 collective，并提示 sharding strategy 会在显存节省与通信开销之间取舍。[2] 因此知识页中“按 wrap/shard policy 分片、靠 unit-level all-gather/reduce-scatter 执行”的说法是 source-consistent；但具体 peak memory、step time、wrap policy 和 tied-weight 行为仍需 workload 级测试。

3. ZeRO：Stage 1/2/3 的分片对象已由官方 tutorial 直接定义

DeepSpeed 官方 ZeRO tutorial 将训练状态分为 weights、gradients、optimizer states，并定义：Stage 1 分片 optimizer states；Stage 2 在此基础上分片 reduced gradients；Stage 3 进一步分片 model parameters，并在 forward/backward 中自动收集和重新分区。[3]

这直接支持 `VLA-ENG-004` 的阶段表。它不支持“Stage 3 对所有 VLA workload 最优”，也不替代通信、offload、checkpoint 保存/加载和多 teacher 共存的实测。OPD 的 `preloaded_zero3` 只能作为显存压力的项目 hook，不能升级为通用性能结论。

4. AMP：unscale、clipping 与 accumulation 的顺序有直接官方依据

PyTorch AMP examples 将 `autocast` 与 `GradScaler` 作为常见 mixed-precision 组合，并明确指出：如果要做 gradient clipping，应先对 optimizer 调用 `scaler.unscale_(optimizer)`，否则 norm 或 maximum magnitude 仍处于 scale 后的数值，clip threshold 语义会错误。[4]

同一页对 gradient accumulation 给出更强的约束：effective batch 是 `batch_per_iter × iters_to_accumulate`（分布式时再乘进程数）；在一个 effective batch 内，梯度应保持 scaled，scale factor 保持不变；inf/NaN 检查、step skip 和 scale update 应在 effective-batch 粒度发生。[4]

因此 `VLA-ENG-005` 的核心顺序是 `backward → accumulate → unscale → global norm/clipping → optimizer.step/update`。scheduler 按 micro step 还是 optimizer step、variable-length 样本按 sample/token/frame/action 计，则是训练合同，需要由项目配置与日志明确，不能伪装成 AMP API 的默认答案。

```
def amp_accumulation_step(micro_batches, model, optimizer, scaler, clip_norm):
    # Keep gradients scaled and accumulate across the whole effective batch.
    for batch in micro_batches:
        with torch.autocast(device_type="cuda", dtype=torch.float16):
            loss = model.loss(batch) / len(micro_batches)
        scaler.scale(loss).backward()

    # Only now unscale, inspect/clip unscaled gradients, and update once.
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
```

代码是面试用的最小顺序示意，不代表每个 VLA trainer 的完整实现；分片 optimizer、多个 optimizer、梯度检查和 scheduler 仍需按框架合同补充。

5. Distributed Checkpoint：load-time reshard 是官方能力，lineage bundle 不是

PyTorch 2.13 Distributed Checkpoint 文档明确写明：DCP 支持从多个 rank 并行保存/加载，并支持 load-time reshard，因此 checkpoint 可以在一种 cluster topology 保存、另一种 topology 加载；其 checkpoint 通常由多个文件组成，而不是单一 `torch.save` 文件。[5]

这支持知识页 `VLA-ENG-006` 中“分布式 checkpoint 要明确 full/sharded state dict、world-size reshard、rank ownership”的机制提醒。但 `run_id + git SHA + config/env + processor + normalizer + optimizer/scheduler/scaler + RNG + sampler/data cursor + eval manifest` 是本仓库建议的 lineage bundle，不是 DCP 自动保存或验证的完整字段集合。必须把“DCP 能重分片加载”和“训练实验可审计、可无缝 resume”分开回答。

6. Reproducibility：固定 seed 不是充分条件

PyTorch 2.13 Reproducibility 文档直接警告：跨 PyTorch release、不同 commit、不同平台，以及 CPU/GPU 之间，即使使用相同 seed，也不保证完全复现；文档建议控制随机源、配置 deterministic 算法，但 deterministic 操作往往更慢。[6]

因此知识页中的“kernel、数据顺序、环境和 simulator 同样属于合同”是合理的工程扩展；“只固定随机种子即可复现”应继续列为危险答案。这个来源只证明 PyTorch 侧的可复现性边界，不证明任何 UWM/OpenPiVLA/Harness run 已复现。

7. 逐节点复核矩阵

节点	独立结果	可直接支持的内容	不可由该来源支持的内容
`VLA-ENG-003`	`FRAMEWORK / SOURCE-CONSISTENT`	data/compute/sync/misc 的 timing 分解与 A/B 诊断顺序	某个项目的实际吞吐提升比例

节点	独立结果	可直接支持的内容	不可由该来源支持的内容
`VLA-ENG-004` DDP	`PASS WITH WARNING`	replica、gradient all-reduce、输入不自动 shard；optimizer state 需标 inference	当前 workload 的显存/通信最优
`VLA-ENG-004` FSDP	`PASS`	sharding strategy、all-gather/reduce-scatter、reshard 时机	当前 wrap policy 的峰值显存与速度
`VLA-ENG-004` ZeRO	`PASS`	Stage 1/2/3 的分片对象	多 teacher、offload 与恢复的项目收益
`VLA-ENG-005`	`PASS WITH WARNING`	autocast/GradScaler、unscale 后 clipping、effective-batch accumulation	scheduler 选择、variable-length workload 与 BF16 项目稳定性
`VLA-ENG-006`	`PASS WITH WARNING`	DCP 并行保存/加载与 load-time reshard	lineage 完整、sampler cursor 恢复、固定 batch 输出相等
`VLA-ENG-013` 预览	`PASS WITH WARNING`	seed 不是跨 release/platform 的充分复现条件	本地训练已经 deterministic

8. 最小回写与真实性边界

本次发现一处需要最小回写的措辞风险：`VLA-ENG-004` 题卡短答把“DDP 每卡复制完整训练状态”说得像 DDP API 的直接保证。应改为“DDP 每卡复制模型参数，梯度 all-reduce；optimizer state 通常由各 rank 独立持有”，并在知识页补充“optimizer state 是每进程 optimizer 的实现推论”。其余 9 张 Batch 1 题卡不需要因本次官方复核改写核心机制。

项目 hook 继续沿用 evidence inventory：OpenPiVLA remote path、UWM raw checkpoint、OPD `preloaded_zero3`、Harness scorer 等只能作为追问入口；本次官方文档复核不升级任何个人贡献、项目数字、部署延迟或真机闭环结论。

复核结论

本批分布式训练/AMP/checkpoint 的核心机制已达到 `official-checked` 或 `official-tutorial-checked`；吞吐、lineage、resume 验收和项目结果继续保持 `framework` / `blocked`。总体 verdict 仍为 `WARN`，因为本地 runtime artifacts、跨 world-size resume smoke、性能 A/B、浏览器视觉 QA 和候选人口述尚未完成。

参考文献（References）

1. PyTorch 2.13, DistributedDataParallel，读取日期 2026-07-17；响应 SHA-256 `6895c721fe7e483fa473d30afba5e0666723c7a5e144078c329eb91ee383bdd4`。
2. PyTorch 2.13, FullyShardedDataParallel，读取日期 2026-07-17；响应 SHA-256 `d188ded37aec986272a6cefadb0d8eb1316030303c2069ab7b5abc71397e00b9`。
3. DeepSpeed official, ZeRO tutorial，读取日期 2026-07-17；响应 SHA-256 `32e2b242be5088e4e64d9cc2d03e553b4fdea7d0116c4e84eeee5eca282d653f`。
4. PyTorch 2.13, Automatic Mixed Precision examples，读取日期 2026-07-17；响应 SHA-256 `41b26df690e71e7a83d6ec1a62e47c776051978e319c6da31c355fa5adaa6a59`。
5. PyTorch 2.13, Distributed Checkpoint，读取日期 2026-07-17；响应 SHA-256 `c55275fd489a912f657fd528ea066bd1cfaf843a14889db04fb442b088d34b12`。
6. PyTorch 2.13, Reproducibility，读取日期 2026-07-17；响应 SHA-256 `cebd51bee8f3286d36db9a3ee2dba3ef7e1a46fd351269d59fc127ea8dac0966`。
7. 本仓库 VOL 05 Batch 1 知识页 与 10 张题卡。
8. 本仓库 项目 Evidence Manifest 与五项目 claim–evidence inventories。

Part 4 / 6 · `interview_materials/vla/sources/vol_05_batch2a_independent_fact_review.html` · SHA-256 `0EEEF7C84ED4B1BF87CC6E7476EC972BCF760472782638FB065AA7C4C157D8F6`

返回本卷目录

Canonical Markdown SHA-256：`0eeef7c84ed4b1bf87cc6e7476ec972bcf760472782638fb065aa7c4c157d8f6`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_05_batch2a_independent_fact_review.md`

VOL 05 Batch 2A：独立官方文档事实复核

状态：`source_reviewed / ENG-009..011 / WARN` 复核日期：2026-07-17 复核对象：Batch 2A 知识页与三张原子题卡 复核范围：DVC `.dvc` 元数据、PyTorch DataLoader / `pad_sequence` / Reproducibility、Hugging Face Datasets cache 与 Transformers collator。 结论边界：本报告核对官方 API/文档语义；不证明个人项目已完成 manifest diff、cache replay、dynamic-batch A/B 或 closed-loop 验收。

1. 复核问题与方法

本轮只回答三个问题：

1. `VLA-ENG-009` 中哪些 provenance 字段由 DVC 官方文档直接支持，哪些是本仓库额外设计？
2. `VLA-ENG-010` 中 DataLoader worker/cache/reproducibility 的表述是否准确？
3. `VLA-ENG-011` 中 `pad_sequence`、padding waste、mask 分离与 dynamic batching 的来源边界是什么？

复核方法：对官方页面直接 HTTP GET；移除 `script/style/nav/header/footer/svg`，合并正文空白后，记录 normalized main-text 长度和 SHA-256；再按“官方原文支持 / 工程推导 / 项目证据 / 未核实”逐条审查。完整 HTML 会包含请求时动态生成的资源标识，raw response SHA 在连续 GET 间可能变化，因此本报告不把它当稳定来源版本号。

2. 冻结来源账本

来源	冻结 URL / 版本	normalized text 长度 / SHA-256	可支持内容	不能支持内容
DVC `.dvc Files`	rolling official page	`8393` / `27cd7fc76b1f6fe720c835ad4c3b34bfd89549c464fddb032ebb1fee4e268b5a`	YAML 中的 `outs`、path、hash、`deps`、`wdir` 等元数据字段	远端对象已下载、本地可解码、与某 checkpoint 配套
PyTorch DataLoader	PyTorch 2.13	`47667` / `9032517582f5a20a811008e4f71a81e356d058380cfc985b552ef338b1cecf5f`	worker 生命周期、dataset/collate/worker init、IterableDataset replica、prefetch、persistent workers、pin memory	cache key schema、项目吞吐提升、端到端 determinism
PyTorch `pad_sequence`	PyTorch 2.13	`2405` / `b8fb4f7c4d334a2d65f957c5aa1387bc43f995dea64fe2999ced80db15ce928e`	pad 到最长长度、`TxBx*` / `BxTx*`、padding value/side	有效长度自动保存、mask 自动正确、dynamic batching 性能
PyTorch Reproducibility	PyTorch 2.13	`11230` / `5a19f15c86d56dbd0f9ea0572b86e960475328c99c36408cfb82d4fce90a03c5`	跨版本/平台不保证完全复现、Python/NumPy/torch seed、DataLoader worker reseed、deterministic trade-off	任意硬件/版本 bitwise identical、仅设一个 seed 即完全复现
HF Datasets cache management	Datasets v4.8.4	`5751` / `afaabe583e52ab68350addcd56f2fe90f746b52ae153b263143c63aa993cd61e`	Hub/Datasets 两类本地 cache、Arrow cache、`load_from_cache_file=False` 重算	自定义 VLA decode cache key 的全部字段
HF Datasets cache fingerprint	Datasets v4.8.4	`4695` / `bcedb14fe09d45e1cb43f58db96b00753e0a076992754d1f887ea6f42767cdfa`	初始 Arrow/file hash、前一 fingerprint 与最新 transform hash 的组合	decoder/version/dtype/shape/seed-policy 必然进入默认 fingerprint
HF Transformers Data Collator	Transformers v5.14.0	`17879` / `6250a91993214a7dee35eb553e110ddef230848a6d5561cff18c7fa65d44e945`	collator 形成 batch、padding、random masking、longest/max length/pad-to-multiple	VLA 的 `attention/valid/loss` 三 mask 自动分离

知识页初次冻结的 PyTorch DataLoader 与 `pad_sequence` 完整 HTML SHA 分别为 `b0d12f...`、`11c15d...`；复核发现 raw HTML 在连续 GET 间也会漂移，而 normalized main-text SHA 稳定。因此本轮把来源指纹统一改为规范化正文 SHA，并重新核对相关语义。Hugging Face 原先使用 `main`/无版本 URL，本轮改为可复查的稳定版本页。

3. `VLA-ENG-009`: Dataset / Version / Transform Manifest

3.1 直接支持的事实

DVC `.dvc` 文件是 YAML 元数据文件。官方示例的 `outs` 项包含内容 hash 与 `path`；schema 也列出 `deps`、`wdir`、`cmd`、`outs` 等字段。因此题卡说"pointer 可记录 path/hash/dependency/working-directory metadata"成立。

3.2 需要保留的边界

- "`.dvc` 文件存在不等于数据对象已经本地可读"是正确的可用性边界，但不是 DVC 页面逐字给出的保证；应表述为 metadata 与 object availability 的工程区分。
- `schema_version`、`split_manifest_sha`、`transform_graph_sha`、`normalizer_sha`、`sample_counts`、camera/episode lineage 是本仓库建议的 VLA manifest，不是 DVC 默认 schema。
- "关键字段不一致就阻断"是 fail-closed 工程策略；官方文档没有替项目决定哪些字段属于 fatal mismatch。

3.3 节点结论

`VLA-ENG-009 = PASS WITH WARNING`：DVC 元数据语义准确；完整 VLA manifest 仍需明确标记为 engineering contract，不能写成 DVC 的默认保证，也不能升级为项目已运行 manifest diff。

4. `VLA-ENG-010`: Decode / Cache / Determinism

4.1 DataLoader 语义核对

PyTorch 2.13 文档直接支持：

- 创建 DataLoader iterator 时，`num_workers>0` 会创建 worker process；dataset、`collate_fn`、`worker_init_fn` 被传给 worker，I/O、transform 与 collate 在 worker 中运行。
- IterableDataset 在多进程下每个 worker 持有 replica；若不分片，naive loading 可能重复数据。
- `prefetch_factor` 是每个 worker 提前加载的 batch 数；`persistent_workers=True` 会在一次数据集消费后保留 worker process 和 Dataset instance；`pin_memory=True` 会将可识别的 Tensor 放入 pinned memory。

因此知识页对 worker/prefetch/persistent/pin 的 API 描述成立。但"persistent worker 如果不重置随机状态会改变重跑顺序"是由生命周期语义推导出的风险，不是官方页保证必然发生。

4.2 Cache 与 reproducibility 边界

HF Datasets 官方 fingerprint 把初始 Arrow table/file hash 与后续 transform hash 结合；cache management 页面也允许通过 `load_from_cache_file=False` 重新执行 transform。这支持"cache 必须与数据状态和 transform 绑定"的方向。

本仓库的 VLA cache key：

$$k = H(\text{media content}, \text{decoder version}, \text{transform graph}, \text{dtype}, \text{shape}, \text{seed policy})$$

变量与实现边界：`media content` 与 `transform graph` 有 HF fingerprint 的相邻官方语义；decoder version、dtype、shape、seed policy 是面向 multimodal/VLA 的额外工程字段。公式不是 HF 默认实现，代码中必须自行构造 sidecar 和 invalidation policy。

PyTorch Reproducibility 页面明确说跨 release、commit、platform，甚至 CPU/GPU 之间都不保证完全复现；它建议控制 torch/Python/NumPy RNG，并用 DataLoader `worker_init_fn` 和 `generator` 处理 worker reseed。由此，题卡"prefetch/cache 不等于 determinism"成立。

4.3 节点结论

`VLA-ENG-010 = PASS WITH WARNING`：DataLoader 与 seed 边界有官方依据；VLA cache key、decode fallback 和 observability 字段是工程合同。UWM stall smoke 仍只能支持读取瓶颈线索，不能证明 full-training 加速或确定性。

5. `VLA-ENG-011`: Variable-Length / Dynamic Batching

5.1 Padding 语义与公式

PyTorch `pad_sequence` 明确把不同长度 Tensor pad 到 batch 最长长度；`batch_first=True` 时输出为 `B×T×*`，否则为 `T×B×*`。它只返回 padded Tensor，不附带每个样本的原始有效长度，因此题卡“不能把 padded 输出当有效长度信息”成立。

对长度为 `L_1,...,L_B`、统一 pad 到 `L_max` 的 batch：

$$w_{pad} = 1 - \frac{\sum_i L_i}{BL_{max}}.$$

变量解释：`B L_max` 是总 slot 数，`sum_i L_i` 是有效 slot 数，两者之差除以总 slot 即 padding fraction。该推导正确，但 `L` 可能代表 token、frame、action step 或 patch；实际 attention/decoder 计算成本不一定与 `L` 线性，因此不能把 `w_pad` 直接当 GPU FLOP 节省比例。

5.2 Dynamic batch 与 mask 边界

PyTorch DataLoader 文档说明 IterableDataset 的用户定义顺序允许 dynamic batch size；HF collator 文档支持 longest/max-length padding 和 padding/masking 类处理。这些来源不规定：

- 按 `(length, sample_id)` 排序；
- token/frame/action budget；
- oversize singleton 策略；
- `attention_mask`、`valid_mask`、`loss_mask` 的 VLA 三层合同；
- per-rank workload、梯度噪声或同步诊断。

这些是合理的工程设计，必须通过 matched fixed-batch vs dynamic-batch A/B 验证。不同 rank 的有效 token 数、loss denominator 和 step timing 应被记录，不能仅用 samples/s 宣称更快。

5.3 节点结论

`VLA-ENG-011 = PASS WITH WARNING`：`pad_sequence` shape/padding 与 `w_pad` 推导成立；bucket/budget/mask/per-rank 合同仍是 framework，不是 PyTorch/HF 默认保证。

6. Claim–Source 结论表

Claim	类型	结论	安全表达
`.dvc` 可保存 path/hash/deps/wdir 元数据	official	PASS	DVC 支持元数据追踪，不代表 object/runtime 可用
VLA manifest 必须含 schema/split/transform/normalizer/lineage	engineering contract	PASS WITH WARNING	这是可复现性设计，不是 DVC 默认字段
DataLoader 多 worker 会运行 dataset I/O/transform/collate	official	PASS	用 worker 语义解释并行边界
IterableDataset replica 未分片可能重复数据	official	PASS	使用 worker info/init 做 replica 配置
persistent workers 必然导致随机顺序变化	inference	REJECT AS ABSOLUTE	只说生命周期跨 epoch，RNG reset policy 必须显式验证
HF 默认 cache key 已包含 decoder/dtype/shape/seed	unsupported	REJECT	HF fingerprint 是相邻机制；VLA key 需自行实现
`pad_sequence` 返回有效长度和正确 mask	unsupported	REJECT	它只 pad/stack；length 与 mask 由上层维护
dynamic batching 必然提升吞吐或训练效果	unsupported	REJECT	用 matched A/B、token/frame throughput 与 per-rank workload 验证
三种 mask 应分离	engineering contract	PASS WITH WARNING	是 VLA loss/执行语义设计，不冒充框架默认行为

7. 对知识页与题卡的修订

本轮做以下最小修订：

1. 将官方页面指纹更新为可重复计算的 normalized main-text SHA，并记录规范化规则。
2. 将 Hugging Face source 从 rolling `main` 改为 Datasets v4.8.4 / Transformers v5.14.0；增加 `about_cache` fingerprint 来源。
3. 明确 `persistent_workers` 的 RNG 影响是需要验证的风险，不写成必然结果。
4. 明确 custom cache key、manifest schema、dynamic budget、三种 mask 是 engineering contract。

5. 不修改任何项目指标、checkpoint 结论、个人 ownership 或练习状态。

8. 总结与下一步

三个节点均达到 `PASS WITH WARNING`：官方 API 语义已复核，核心工程推导没有发现公式或 API 错误；WARN 来自四个未闭环项：rolling DVC 页面、custom cache/manifest/batching contract、缺项目 A/B artifact、缺真实口述与浏览器视觉 QA。

下一步不应直接扩展 `ENG-012..016`。优先级是：

1. 对 `VLA-ENG-009..011` 做一次一题一练；或
2. 若出现真实项目 artifact，补 manifest diff、cache replay、fixed-vs-dynamic batch A/B；或
3. 在来源准备充分后再进入 `ENG-012` 存储格式切片。

References

1. DVC, .dvc Files，读取日期 2026-07-17。
2. PyTorch 2.13, torch.utils.data，读取日期 2026-07-17。
3. PyTorch 2.13, pad_sequence，读取日期 2026-07-17。
4. PyTorch 2.13, Reproducibility，读取日期 2026-07-17。
5. Hugging Face Datasets v4.8.4, Cache management与Cache fingerprint，读取日期 2026-07-17。
6. Hugging Face Transformers v5.14.0, Data Collator，读取日期 2026-07-17。
7. 本仓库 Batch 2A canonical knowledge与Batch 2A atomic cards。

Part 5 / 6 · `interview_materials/vla/sources/vol_05_completion_independent_fact_review.html` · SHA-256 `643341487E2CA7D233D95DD4AE064B5B8776EBC90C8691BDF9B956910CDECB4B`

返回本卷目录

Canonical Markdown SHA-256：`643341487e2ca7d233d95dd4ae064b5b8776ebc90c8691bdf9b956910cdecb4b`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_05_completion_independent_fact_review.md`

VOL 05：Completion Audit / Independent Fact Review

状态：`completion_audited / 44-of-44 coverage / WARN` 复核日期：2026-07-19 复核对象：VOL 05 Engineering / Evaluation 的 44 个知识节点、对应题卡、来源冻结页与生成审查记录。 结论先行：本卷的 canonical knowledge/card/artifact coverage 已闭合；这只证明文件、链接和审查链完整，不证明候选人已经掌握，也不把五个项目的指标、复现或 ownership 升级为 verified。

1. 复核问题与方法

本轮只回答四个问题：

1. `VLA-ENG-001..022` 与 `VLA-EVAL-001..022` 是否各自有 canonical knowledge、HTML、review、question、question-list 和 question review?
2. 每个节点是否有独立来源入口：Batch 1/Batch 2A independent review，或节点级 source freeze?
3. canonical Markdown 的 SHA 是否与 HTML metadata、review JSON 一致？是否存在单 H1、ARIS academic v1、数学保护和本地链接等静态校验记录?
4. 44/44 coverage、`unseen` practice state、五项目 WARN 边界应如何表述，才能避免把“写完”说成“学会”或“做过”?

复核方法是只读盘点：按 node ID 建立 artifact matrix，解析 review JSON，重新计算 canonical Markdown SHA，并检查 HTML 是否嵌入该 SHA；同时阅读 Batch 1、Batch 2A 和 ENG/EVAL source freeze 的 review 内容。浏览器 `file://` 视觉检查继续受当前策略阻断，因此静态检查不被表述为视觉通过。

2. VOL 05 的组成与来源层级

切片	节点	来源入口	解释边界
Batch 1	`ENG-001..008`、`EVAL-001..002`（10）	Batch 1 independent fact review	官方 DDP/FSDP/ZeRO/AMP/DCP/Reproducibility 与上游项目协议已核对；本地吞吐、resume、checkpoint 和 ownership 仍是 framework/blocked
Batch 2A	`ENG-009..011`（3）	Batch 2A independent fact review	DVC、PyTorch DataLoader/ `pad_sequence` /Reproducibility、HF cache/collator 语义已核对；manifest/cache/dynamic-batch 是 engineering contract
Engineering	`ENG-012..022`（11）	每节点 source freeze	存储、复现、硬件、offload、teacher、observability、retry、config、权限、packaging、policy server；均保留 WARN，缺 workload 级 A/B 或 runtime 验收
Evaluation	`EVAL-003..022`（20）	每节点 source freeze	统计、protocol、raw bundle、release gate、OOD、latency、side effect、scorer bias、sim→real、ablation、selection、RCA、drift、ownership；均不替代真实项目 artifact

“来源已冻结”只表示主张可以回到指定版本/页面/本地审查入口；它不表示该来源证明了本人的运行结果。`paper/model card`、`repo/commit`、`config/launcher`、`checkpoint path`、`load smoke`、`summary` 和 `Git author` 各自只能覆盖证据链的一部分。

3. 44 节点 completion matrix

表中 `K/H/R` 分别表示 knowledge Markdown/HTML/review，`Q/L/Rq` 表示 question Markdown、question-list、question review。所有行均存在；`SHA*` 表示本轮重新核对通过，历史异常单列在第 4 节。

3.1 基础工程与 Batch 2A

ID	Knowledge	Question	Source gate	K/H/R + Q/L/Rq	Practice
`VLA-ENG-001`	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	`unseen`
`VLA-ENG-002`	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	`unseen`

ID	Knowledge	Question	Source gate	K/H/R + Q/L/Rq	Practice
VLA-ENG-003	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-004	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-005	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-006	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-007	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-008	Batch 1 K	Batch 1 Q	Batch 1 review	PASS	unseen
VLA-ENG-009	Batch 2A K	Batch 2A Q	Batch 2A review	PASS	unseen
VLA-ENG-010	Batch 2A K	Batch 2A Q	Batch 2A review	PASS	unseen
VLA-ENG-011	Batch 2A K	Batch 2A Q	Batch 2A review	PASS	unseen

3.2 ENG-012..022

ID	Knowledge	Question	Source freeze	K/H/R + Q/L/Rq	Practice
VLA-ENG-012	K	Q	source	PASS	unseen
VLA-ENG-013	K	Q	source	PASS	unseen
VLA-ENG-014	K	Q	source	PASS	unseen
VLA-ENG-015	K	Q	source	PASS	unseen
VLA-ENG-016	K	Q	source	PASS	unseen
VLA-ENG-017	K	Q	source	PASS	unseen
VLA-ENG-018	K	Q	source	PASS	unseen
VLA-ENG-019	K	Q	source	PASS	unseen
VLA-ENG-020	K	Q	source	PASS	unseen
VLA-ENG-021	K	Q	source	PASS	unseen
VLA-ENG-022	K	Q	source	PASS	unseen

3.3 EVAL-003..022

ID	Knowledge	Question	Source freeze	K/H/R + Q/L/Rq	Practice
VLA-EVAL-003	K	Q	source	PASS*	unseen
VLA-EVAL-004	K	Q	source	PASS	unseen
VLA-EVAL-005	K	Q	source	PASS	unseen
VLA-EVAL-006	K	Q	source	PASS	unseen
VLA-EVAL-007	K	Q	source	PASS	unseen
VLA-EVAL-008	K	Q	source	PASS	unseen
VLA-EVAL-009	K	Q	source	PASS	unseen
VLA-EVAL-010	K	Q	source	PASS	unseen
VLA-EVAL-011	K	Q	source	PASS	unseen
VLA-EVAL-012	K	Q	source	PASS	unseen
VLA-EVAL-013	K	Q	source	PASS	unseen
VLA-EVAL-014	K	Q	source	PASS	unseen
VLA-EVAL-015	K	Q	source	PASS	unseen
VLA-EVAL-016	K	Q	source	PASS	unseen
VLA-EVAL-017	K	Q	source	PASS	unseen
VLA-EVAL-018	K	Q	source	PASS	unseen
VLA-EVAL-019	K	Q	source	PASS	unseen
VLA-EVAL-020	K	Q	source	PASS	unseen
VLA-EVAL-021	K	Q	source	PASS	unseen

ID	Knowledge	Question	Source freeze	K/H/R + Q/L/Rq	Practice
VLA-EVAL-022	K	Q	source	PASS	unseen

PASS* 的含义见下一节：EVAL-003 的现有历史 review metadata 与当前 canonical Markdown SHA 不一致；本轮不改写历史 artifact，只把它作为审计 WARN 记录。

4. 完整性检查结果

检查	结果	证据与边界
节点覆盖	PASS	44 个唯一 node ID：ENG 22 + EVAL 22；knowledge 与 card 入口均可解析
文件链	PASS	每个节点都有 canonical Markdown、HTML、review JSON、question-list；共享 Batch 1/2A 页面由多个 node 显式复用
题卡唯一性	PASS	44 个覆盖 ID，跨卷 master index 统计总题卡 254，ID 唯一；所有 VOL 05 题卡正文保持 unseen
review JSON 可解析	PASS	VOL 05 节点 review 与 source review 均可 JSON parse；新增 audit 自身也将执行同一检查
Markdown→HTML SHA	PASS WITH WARNING	42 个单节点/共享知识与题卡链通过当前 SHA 检查；历史 VLA-EVAL-003 knowledge/source review 声明为旧 SHA，不能作为本轮新增错误修复
HTML 结构	PASS	既有新增页 review 记录 ARIS academic v1、单 H1、公式保护、sticky TOC、900px 响应式；本轮 audit 将按同一 renderer 生成
本地链接	PASS	已有全局 Markdown/HTML 相对链接缺失为 0；本页只引用已存在的 sources/knowledge/questions 文件
浏览器视觉 QA	BLOCKED	当前 in-app Browser 的 file:// 策略阻断；静态 HTML 检查不能替代桌面与窄屏视觉检查
口述掌握	PENDING	尚未发生真实口述训练；不创建 coaching_state.md，不把页面生成计入 mastery

4.1 历史 SHA 异常的处理

knowledge/vol_05_engineering_eval_eval003.review.json、sources/vol_05_eval003_success_rate_confidence_interval_source_freeze.review.json 及其 HTML metadata 使用旧 SHA。该异常已在既有仓库审计中记录；本轮 completion audit 只核对并报告，不重写历史 metadata，也不通过“修 SHA”掩盖 provenance 变化。后续若要修复，应单独建立 bounded metadata-repair slice，重新渲染并保留变更原因。

5. 五项目的共同证据边界

项目	当前可安全说	仍不能说
OPD	代码、数据路由、manifests、10k holdout、by-format/by-pool、point protocol 的局部 local evidence	有效权重、raw predictions、完整 lineage、ablation、exclusive ownership 或通用性能结论
Robot Harness	runtime/tool registry、SFT/GRPO data、reward、局部 eval summary	checkpoint/raw episodes、共同 scorer/n、完整 ownership、真实 effect-sink 或 recovery success
OpenPiVLA	formal trainer、JSON/causal-CE、recipe、smoke/report/diagnostic local-direct	97.4 之外的 formal checkpoint/load、raw bundle、lineage、ownership；archive summary 不等于复现
WAM / Cosmos Policy	public paper→repo→model-card→eval path 的上游审计	workspace 内完整 local raw run/checkpoint/config/log/metric/episode bundle 与个人贡献链
UWM / LIBERO	local scripts/report/protocol/normalizer/DataLoader smoke；186/250 为 user-confirmed remote summary	checkpoint binary、250 raw episodes、formal OOD/3-seed、fixed/reset paired evidence、ownership

共同规则：缺任一必需 edge 时，把 claim 降到最弱证据层；不要以 repo、path、summary、signature 或生成报告作者身份替代运行、raw result 和贡献 ledger。

6. 结论与下一 bounded slice

本卷完成条件“文件覆盖、来源入口、题卡入口、链接与静态审查记录”已满足：44/44。卷级 verdict 仍为 WARN，原因是：

- unseen 只表示尚未真实口述，不是掌握度；
- 五项目仍缺若干 checkpoint/raw/runtime/ownership 证据链；
- EVAL-003 历史 SHA 异常未在本轮修复；
- 浏览器视觉 QA 被环境策略阻断。

下一步应从以下一个 bounded slice 中选择一个，而不是再批量生成新卷：

1. 对跨卷高风险来源做一次独立事实复核（优先 VOL 03 Flow Matching / π0 与 VOL 04 的协议边界）；或
2. 选择一个项目，建立一小段 ownership ledger / claim-to-evidence reconciliation；或
3. 进行第一次真实口述 drill，完成后才创建 coaching state 并回写一题。

参考文献（References）

1. VLA Knowledge Map，VOL 05 计划、44/44 coverage 与后续切片入口。
2. Master Question Index，VOL 05 44 题与全局 254 题统计。
3. VOL 05 Batch 1 independent fact review。
4. VOL 05 Batch 2A independent fact review。
5. VOL 05 EVAL-022 evidence audit。

Part 6 / 6 · `interview_materials/vla/sources/vol_05_uwm_libero_robocasa_transfer_evidence_freeze.html` · SHA-256 `6FA38801E0205BF9F55ABE81F0C9CA267F6E33C0CDD088CD825DAA8760CC1627`

返回本卷目录

Canonical Markdown SHA-256：`6fa38801e0205bf9f55abe81f0c9ca267f6e33c0cdd088cd825daa8760cc1627`；生成自：`interview_materials\vla\sources\vol_05_uwm_libero_robocasa_transfer_evidence_freeze.md`

VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结

状态：`local_protocol_audited / user_confirmed_remote_final / robocasa_debugging / WARN` 复核日期：2026-07-21 证据来源：`C:\Users\mousongzhe\Project\wm` 与 `C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702`（外部工作区只读审计） Bounded slice：本页冻结 UWM 官方 LIBERO reproduction、RoboCasa-native transfer/actionheadfix 训练与闭环评估的协议、指标、修复和 provenance；不把用户确认远端摘要升级为本地 raw benchmark，不改变题卡 mastery 或 ownership。

1. 结论先行

这个项目最适合在面试中讲成“训练—评估协议工程与失败诊断”，而不是单纯报成功率：

1. UWM/LIBERO 路线已经有本地脚本、任务映射、`pruned_init`/reset 协议开关、5-step warmup、8-shard launcher 和 DataLoader-only smoke；
2. 用户确认的 strict fixed-init 结果为 `186/250=0.744`，但 JSON 明确标记为 `user-confirmed remote final`，本地没有 checkpoint、normalizer、250 条 episode JSONL 或 scorer，不能称 local raw reproduction；
3. 旧 `232/250=0.928` 使用 `env.seed(e)+env.reset()` 的 reset-style 协议，不能与 strict fixed-init 或 UWM Table II 直接比较；
4. RoboCasa actionheadfix 使 action-head 初始化、open-loop监督和trace诊断更可信；新增raw audit对一条30-epoch openpi-state-3view run重算：full h1/h2/h4/h5/h8为`2/54、1/54、0/54、1/54、1/54`，三组h5 input ablation为`2/54、2/54、1/54`，全部env errors为0；历史30/100-epoch结果属于相邻run，不能混算；
5. 严格 `convert_action` A/B 为 `0/6`、6 个环境错误（`expected 12, got 5`），说明 OpenPI-style dict 不能直接喂给当前 native RoboCasa env；GT bridge replay 的最大绝对差为 `0.0`，但模型闭环仍失败。

2. 证据等级与 provenance

层级	当前材料	能支持	不能支持
`local-direct`	本地脚本、repo patch、报告、commit、DataLoader smoke	task map、action/state shape、协议开关、修复逻辑、日志字段	远端权重内容或完整 episode 结果
`report-supported`	`uwm_robocasa_libero_reproduction_log_report.html`	Book-Caddy `42/50`、训练曲线片段、RoboCasa run ledger	本地可重算的 checkpoint/episode bundle
`user-confirmed-remote-final`	`final_strict_eval_user_confirmed_20260705.json`	5-task per-task summary 与协议说明	原始 250 episode、checkpoint hash、normalizer/scorer
`remote-snapshot-raw`	actionheadfix 30-epoch `train.out` + 8×54 rollout JSON	run completion、paired scenario set、success/env-error/contact recount	produced checkpoint binary/hash、env/scorer revision、videos、ownership
`working-tree-evidence`	`sync_manifest.json`、`sync_summary.md`	远端断联时最后状态、脚本 hash、dirty/未跟踪文件边界	将 recovery summary 变成 committed raw result
`local-inference`	action/contact/协议对照	对失败原因提出可检验假设	已确认的唯一 root cause
`unverified`	RoboCasa benchmark success、personal end-to-end ownership	仅列为待补证	任何强 C3 结果或原创架构声明

外部 Project 当前 HEAD 为 `6c524d9c6c6226075f100cdac395fef2fdc37671`（`Document RoboCasa actionheadfix diagnostics`）。工作树仍有：`sync_manifest.json`、`sync_summary.md` 修改，以及 `final_strict_eval_user_confirmed_20260705.json` 未跟踪。它们必须分别标记为 committed、working-tree、user-confirmed 和 raw-local，不可混为一个“完整复现实验”。

3. 官方 Baseline 与 UWM/LIBERO 结果对照

3.1 UWM/LIBERO 任务与协议

本地 `libero_official_common.py`、`uwm_libero_rollout_eval.py` 和 parallel launcher 固化了五个 LIBERO-10 任务：`book_caddy`、`soup_cheese`、`bowl_drawer`、`moka_moka`、`mug_mug`。项目 strict protocol 为：

字段	fixed-init 项目口径
initial state	LIBERO `*.pruned_init`，按官方顺序读取
warmup	5 个 zero-action steps
rollouts	50/task，5 task，共 250
rollout length	policy steps 600
launcher	8 shards / GPUs 0–7
success	environment `info["success"]` 二值信号
provenance	用户确认远端 summary；raw episode 未同步

SR（success rate）对 (N) 条 rollout 的定义为：

$$\mathrm{SR} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}[\mathrm{success}_i].$$

其中 (N) 是固定 protocol 下的 episode 数，(mathbf{1}[cdot]) 来自环境 success predicate。实现对应 `uwm_libero_rollout_eval.py` 的 per-episode summary 和 task-level aggregation；没有 episode JSONL 时只能信任 summary 的 provenance，不能在本地重新计算。

3.2 结果 reconciliation

口径	结果	证据等级	解释
Book-Caddy 单任务 report	`42/50=0.84`	`report-supported`	单任务复现实验，不是完整 LIBERO benchmark
UWM reset-style	`232/250=0.928`	`user-confirmed context / protocol-diagnostic`	`env.seed(e)+env.reset()`，随机 reset 分布
fixed-pruned-init strict	`186/250=0.744`	`user-confirmed-remote-final`	5-step warmup、50/task；本地无 raw bundle
UWM Table II reference	约 `0.79`	`upstream-verified`	OOD init、+0.03 object shift、背景移除、3 seeds

strict fixed-init per-task summary：

Task	Strict result	Assumed denominator
`libero_book_caddy`	`0.88`	44/50
`libero_soup_cheese`	`0.80`	40/50
`libero_bowl_drawer`	`0.76`	38/50
`libero_moka_moka`	`0.66`	33/50
`libero_mug_mug`	`0.62`	31/50
total	`0.744`	186/250

不能把 `0.744` 与 Table II 的 `0.79` 写成严格复现差距：两者的 initial-state distribution、object shift、background removal、seed 数和训练口径不同。`0.928→0.744` 的 `-18.4pp` 首先是 protocol sensitivity，不是同 checkpoint 的模型退化实验。

4. UWM Base I/O 与 LIBERO route

UWM 上游是共享 diffusion transformer，同时预测 action chunk 和未来观测/视频 latent，action/video 使用独立 diffusion timestep。项目 adapter 使用两路 RGB、7D action metadata 与 task-specific UWM wrapper；这属于 UWM 公有机制与本地接口迁移的组合，不应混成 Cosmos/WAM 的 action/world/value 三角色。

```
LIBERO-90 checkpoint
        |
        v
LIBERO-10 HDF5 -> compressed Zarr buffer -> UWM robomimic trainer
        |                                        |
        |                                        v
```

```
+-----------------------------> task checkpoint + normalizer
                                        |
                                        v
                                LIBERO fixed-init rollout evaluator
```

本地 evidence 支持的输入/输出边界是：两路 `128×128` RGB、proprio/state contract、7D action chunk，经过 UWM model builder 和 normalizer 后由 environment wrapper 执行；具体 action horizon、normalizer 文件和远端 checkpoint binary 仍需要 raw bundle 才能升级为 C3。

5. RoboCasa action-space 与 head 改造

RoboCasa native full action 是 12D：EEF 7D、base 4D、control 1D。actionheadfix 线将 UWM 的 7D EEF 输出与 RoboCasa 原生字段拆分、重排并加入 hierarchical decoder。可用的抽象映射为：

$$a^{\text{env}} = \text{concat}\left(a_7^{\text{eef}}, a_4^{\text{base}}, a_1^{\text{control}}\right),$$

其中 a_7^{eef} 是模型的连续末端动作，a_4^{base} 和 a_1^{control} 来自额外 head 或统计初始化。实现证据位于 `uwm_robocasa_train.py`、`uwm_robocasa_rollout.py` 与 `NativeRoboCasaDictActionWrapper`；它证明了接口设计和 trace 字段，不证明闭环成功。

actionheadfix 的训练侧改动：

改动	证据	作用	边界
partial decoder init	commit `a37a386`；`checkpoint_load_report`	从 `libero_90.pt` 复制 shape-compatible decoder，避免 12D head 全部从零学	不是完整 checkpoint identity
action stats bias init	同上	base/control bias 写入 normalized action space	统计量 provenance 仍需固定
predicted x0 MSE	训练脚本与 `teacher_forced_target_atomic_seen.json`	从 noise prediction 还原 clean action，再按真实 action 单位诊断	teacher-forced 不等于 rollout
group x0 metrics	eef/base/control/gripper 分组	暴露总 loss 掩盖的通道错误	不自动证明控制策略有效
auxiliary action override	rollout script	将 gripper/control auxiliary 参与最终 action 组装	可能引入执行时序副作用
h4 sharding	commit `1a03fe9`	54 scenario round-robin 分片后合并	sharding 不改变任务协议

checkpoint load smoke 的可核验摘要为：`matched_count=446`、`skipped_count=8`、`reset_action_head=true`、decoder partial init=true；报告同时记录早期 task embedding concat 会使 AdaLN condition dim 从 `3584` 变为 `3712`，随后改为 residual projection 以保持 trunk shape。不同 smoke 的 target-key 集合不同，`446` 不能和另一个 `196/258` smoke 直接比较。

6. RoboCasa Fine-tune 与 Eval Run Ledger

Run	数据/初始化	关键改动	训练证据	闭环结果	结论
early target pilot	`libero_90.pt` → target atomic	7D/12D 适配、task-id 等	report/README 有 run path	`0/3`、`0/3`、`1/3` task pilot	适配链可运行，非 benchmark
30 epoch task-id	target atomic seen	official12 + task-id，`exec_horizon=4`	checkpoint 与 run JSON 已记录	`1/54`，唯一成功为 `TurnOnElectricKettle`	仅 task-id 不足
100 epoch resume	30 epoch checkpoint 续训	继续拉 epoch	final summary：loss 与 checkpoint path	`0/54`，env errors `0`	更多 target epoch 未解决闭环
strict execution A/B	同 100 epoch checkpoint	direct vs strict `convert_action`	run-specific logs	direct `0/6`, errors `0`; strict `0/6`, errors `6`	OpenPI dict 不能直送 native env
Human300 pretrain	`libero_90.pt` → RoboCasa Human300	3 epoch，5271 steps，12D	action stats count `29,106,226`；loss `95.21→0.2265`；action `2.96→0.0411`；x0 `88.80→0.0285`	pretrain 是 offline 训练，不是 success	action head/分布可学习性改善
actionheadfix 30-epoch openpi-state-3view	`libero_90.pt` partial load → target atomic	30 epoch，3930 steps，12D action、task/prompt/state/right conditioning	printed stepo→3929 loss `3.3257→0.25385`；action `2.4300→0.07689`；raw `0.9986→0.03043`；`val_ratio=0`	paired 54-scenario sweep：h4 `0/54`，robot-env contact `45/54`，env errors `0`；其它setting `1-2/54`	训练fit未转化为稳定 closed-loop success；raw audit

6.1 Teacher-forced 诊断

在 256 action rows 上，报告记录：

指标	值
`x0_mse_total`	`0.0647`
`x0_mse_base`	`2.68e-6`
`x0_mse_control`	`4.47e-5`
`x0_mse_eef_pos`	`0.1489`
`x0_mse_gripper`	`0.3093`

这说明 base/control 通道在 teacher-forced/open-loop 分布上更接近标签，而 EEF position 与 gripper 仍是高误差通道；它不能直接推出 simulator success。

7. Online Rollout 与 0-success 诊断

实际评估由 `uwm_robocasa_rollout.py`、`remote_muxi_eval_h4_sharded.sh` 和 env wrapper 组成，核心路径可抽象为：

```
def rollout_one_scenario(env, policy, task, replan_steps=4):
    # 1. Reset task and collect camera/state/prompt observation.
    obs = env.reset(task)
    # 2. Build model input and sample a normalized action chunk.
    chunk = policy.sample(build_observation(obs, task))
    # 3. Denormalize, reorder to native 12D, and apply optional aux override.
    env_actions = action_dict_to_native_array(denormalize_and_parse(chunk))
    # 4. Execute only the first replan_steps actions, then query again.
    for action in env_actions[:replan_steps]:
        obs, reward, done, info = env.step(action)
        if done or info.get("success", False):
            break
    # 5. Persist trace/video/success/error fields for this scenario.
    return info
```

对新冻结的30-epoch openpi-state-3view run，full-h4 raw JSON 的直接症状是：`0/54`、`num_env_errors=0`、`45/54` rollout 出现至少一次 robot-env contact。此前报告中的 `27/54` 属于未冻结相同run identity的历史摘要，不能替代本页精确recount。八份54-scenario raw JSON已经提供完整config内denominator，但没有checkpoint binary、视频和immutable env/scorer revision；trace中的普通counter/link/hand接触仍不能等价为有效抓取、按键或开门，也不能宣布唯一根因。

7.1 convert_action A/B

路径	结果	解释
native direct	`0/6`，env errors `0`	12D array 可以执行，但没有成功
strict OpenPI-style `convert_action`	`0/6`，env errors `6`	每条在 step 0 报 `expected 12, got 5`
GT action bridge replay	`bridge_max_abs_diff_max=0.0`	dict→native 12D bridge 数值一致；不能证明模型动作正确

因此 execution bridge 是需要固定的协议字段；strict path 的 6 个错误不能和有效 rollout 的 0 success 混合计算。

8. DataLoader Stall 诊断

UWM/LIBERO 本地 `benchmark_libero_dataloader_stall.py` 只构建 DataLoader，不建模型、不占 GPU。报告记录的 A/B：

设置	p95 wait
finite，workers=8，prefetch=2	`13.862 s`
infinite sampler	`13.183 s`
preload + infinite	`0.194 s`
bs128 preload + infinite	`0.209 s`

这支持“zarr 小 chunk 随机读取造成周期性等待”的工程假设，但不等于完整 GPU training throughput、显存或最终训练时间已提升。要升级结论，仍需 30–100 step matched GPU A/B 并记录 `data_wait_sec/train_sec/total_sec/memory/throughput`。

9. 公有设计、本地证据与个人边界

类别	当前可安全说法	禁止升级
公有上游	UWM shared diffusion、LIBERO recipe、Table II protocol	把 UWM architecture 说成个人原创
本地迁移	我迁移/整理了 UWM-LIBERO launcher、RoboCasa action mapping、strict/reset protocol 和诊断脚本	“独立完成全部 upstream patch”
本地诊断	我用 DataLoader-only A/B 定位 zarr wait，用 trace 观察 contact/夹爪/control 失配	“已证明唯一 root cause”
用户确认结果	可以报告 `186/250`，并明确 `user-confirmed-remote-final`	“本地已复现 74.4%”
RoboCasa	可以讲 30/100 epoch、actionheadfix 和 A/B 失败路径	“完成 RoboCasa benchmark”

推荐 60 秒回答：

> UWM 项目我主要负责训练到闭环评估的协议工程。我在 C500/MACA 上迁移 LIBERO-90 到多个 LIBERO-10 task 的 finetune/rollout，固定 pruned-init、warmup、normalizer、horizon、rollout 数和 success predicate，并用 DataLoader-only A/B 定位 zarr 小 chunk 的周期性等待。LIBERO `186/250` 仍是用户确认summary；RoboCasa则有一条可重算的actionheadfix 30-epoch raw sweep：8卡3930 steps完成，同checkpoint的八组54-scenario rollout都没有env error，但只有0–2 success，full-h4为 `0/54` 且 `45/54` 有robot-env contact。因此我把它定位为train-to-closed-loop contract debugging，而不是benchmark成功。

10. 待补 P0 与恢复清单

1. 获取 strict run 的 checkpoint binary/hash、normalizer、exact config、init index、250 episode JSONL、失败视频和 scorer。
2. 用同一 checkpoint 做 reset-style vs fixed-pruned-init paired evaluation，记录每个 episode 的 seed/init/horizon。
3. 运行 30–100 step GPU matched DataLoader A/B，验证 preload/infinite 对真实训练吞吐的影响。
4. 对 soup/moka/mug 失败做 video/action trace/contact 分类，区分动作语义、空间定位、接触和长时序 recovery。
5. 对 RoboCasa wrapper bridge 重新跑 run-specific eval，明确 direct、strict convert_action、wrapper-convert 三种结果不混算。
6. 完成 `OWN-UWM-001`：逐文件标注上游、本人、协作、工具辅助、只运行、只审查和 operator/reviewer。

References

1. UWM/LIBERO 本地复现报告：`C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\reports\uwm_robocasa_libero_reproduction_log_report.html`，SHA-256 `F5B268F2E7108DF548309BBA869E0099F46A6F4D5CE8A31742D002570317DB62`。
2. RoboCasa transfer/actionheadfix 报告：`C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_transfer_report.html`，SHA-256 `D0ADC3C46D824242E5F8E4D7D1F787CA09810FAA108CA8C79C4E7110A1B00518`。
3. Strict recovery manifest：`C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\sync_manifest.json`，SHA-256 `9077C45992BF6852F5A8F4FA28ED3735FA590817D227A62D4179CCF239358D5F`。
4. User-confirmed final：`C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\final_strict_eval_user_confirmed_20260705.json`，SHA-256 `7616111AE7C7CA35CB10E8788ADCB1E8837D9B6F656B49553B6C7AFCCD586D11`。
5. UWM project inventory：UWM/LIBERO Claim–Evidence Inventory；P0 gate：UWM/LIBERO P0 Bundle Audit。
6. UWM upstream source audit：UWM video–action claim-to-code；benchmark protocol：Benchmark evidence claim-to-protocol。
7. UWM paper：Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets,” RSS 2025；LIBERO official benchmark documentation；RoboCasa benchmarking documentation。
8. UWM RoboCasa ActionHeadFix 30-Epoch Train-to-Closed-Loop P0 Audit。

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 06 · Perception / Grounding / Navigation

多视角感知、开放词汇定位、3D 表示、VLN 与安全导航

本卷由 43 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 06 Perception / Grounding / Navigation：Point、Box、Text 与坐标协议（PER-001）
2. 02 · VOL 06 Perception / Grounding / Navigation：2D → 3D Lift 与 Frame Contract（PER-002）
3. 03 · VOL 06 Perception / Grounding / Navigation：多视角、时序与输入 Schema（PER-003）
4. 04 · VOL 06 Perception / Grounding / Navigation：VLN 与 ObjectNav（PER-004）
5. 05 · VOL 06 Perception / Grounding / Navigation：Navigation Policy Architecture（PER-005）
6. 06 · VOL 06 Perception / Grounding / Navigation：Image Preprocess、Patch Token 与 Camera Identity（PER-006）
7. 07 · VOL 06 Perception / Grounding / Navigation：Open-vocabulary Grounding Pipeline（PER-007）
8. 08 · VOL 06 Perception / Grounding / Navigation：Point / Box / Mask / Keypoint / Affordance 表示选择（PER-008）
9. 09 · VOL 06 Perception / Grounding / Navigation：Grounding Matching、Negative 与 Loss Reduction（PER-009）
10. 10 · VOL 06 Perception / Grounding / Navigation：Grounding Metric、Strict Parser 与 Protocol Comparability（PER-010）
11. 11 · VOL 06 Perception / Grounding / Navigation：Pseudo-label、Teacher Noise、Confidence Calibration、Leakage 与 Curriculum（PER-011）
12. 12 · VOL 06 Perception / Grounding / Navigation：Segmentation、Depth、Keypoint 与 Manipulation Affordance（PER-012）
13. 13 · VOL 06 Perception / Grounding / Navigation：Point Cloud、Voxel、BEV、Occupancy 与 Sampling（PER-013）
14. 14 · VOL 06 Perception / Grounding / Navigation：3D Grounding、Spatial Relation 与 Open-vocabulary 3D Scene（PER-014）
15. 15 · VOL 06 Perception / Grounding / Navigation：Perception→Action、Reachability、Uncertainty 与 Failure Gate（PER-015）
16. 16 · VOL 06 Perception / Grounding / Navigation：Localization、Odometry、SLAM、Map Frame 与 Drift（PER-016）
17. 17 · VOL 06 PER-017：Belief、Memory、Frontier 与 Active Exploration
18. 18 · VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias
19. 19 · VOL 06 PER-019：Navigation Evaluation、Failure Taxonomy 与 Fail-Closed Protocol
20. 20 · VOL 06 PER-020：Robustness、Safety、Fallback 与风险门

题卡

1. 21 · VOL 06 Perception / Grounding / Navigation：PER-001 原子题卡
2. 22 · VOL 06 Perception / Grounding / Navigation：PER-002 原子题卡
3. 23 · VOL 06 Perception / Grounding / Navigation：PER-003 原子题卡
4. 24 · VOL 06 Perception / Grounding / Navigation：PER-004 原子题卡
5. 25 · VOL 06 Perception / Grounding / Navigation：PER-005 原子题卡
6. 26 · VOL 06 Perception / Grounding / Navigation：PER-006 原子题卡
7. 27 · VOL 06 Perception / Grounding / Navigation：PER-007 原子题卡
8. 28 · VOL 06 Perception / Grounding / Navigation：PER-008 原子题卡
9. 29 · VOL 06 Perception / Grounding / Navigation：PER-009 原子题卡
10. 30 · VOL 06 Perception / Grounding / Navigation：PER-010 原子题卡
11. 31 · VOL 06 Perception / Grounding / Navigation：PER-011 原子题卡
12. 32 · VOL 06 Perception / Grounding / Navigation：PER-012 原子题卡
13. 33 · VOL 06 Perception / Grounding / Navigation：PER-013 原子题卡
14. 34 · VOL 06 Perception / Grounding / Navigation：PER-014 原子题卡
15. 35 · VOL 06 Perception / Grounding / Navigation：PER-015 原子题卡
16. 36 · VOL 06 Perception / Grounding / Navigation：PER-016 原子题卡
17. 37 · VOL 06 Perception / Grounding / Navigation：PER-017 原子题卡
18. 38 · VOL 06 Perception / Grounding / Navigation：PER-018 原子题卡
19. 39 · VOL 06 Perception / Grounding / Navigation：PER-019 原子题卡
20. 40 · VOL 06 Perception / Grounding / Navigation：PER-020 原子题卡

来源与事实复核

1. 41 · VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias 独立事实复核
2. 42 · VOL 06 PER-020：Robustness、Safety、Fallback 与风险门独立事实复核
3. 43 · VOL 06 Perception / Grounding / Navigation：Coverage-Gap Audit

Part 1 / 43 · `interview_materials/vla/knowledge/vol_06_per001_grounding_coordinate.html` · SHA-256
`D9CF0164D45397BFCB941156F8A0663A34A3F67FCE67797E75C410C1643A17EA`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`d9cf0164d45397bfcb941156f8a0663a34a3f67fce67797e75c410c1643a17ea`；生成自：`interview_materials\vla\knowledge\vol_06_per001_grounding_coordinate.md`

VOL 06 Perception / Grounding / Navigation：Point、Box、Text 与坐标协议（PER-001）

状态：`drafted / VLA-PER-001 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：PER-001 Grounding / Coordinate Source Freeze。 项目入口：OPD Claim–Evidence Inventory。

问题（Question）

VLA 或 VLM grounding 面试不能只回答“模型找到了目标”。一个可执行的 grounding 输出必须同时说明：目标是什么、坐标在哪个空间、输出如何解析、怎样逆变换回原图，以及下游是否把它当作静态评估、规划输入还是控制 action。

本页回答五个问题：

1. point、box、text 三种 grounding 输出有什么不同?
2. `normalized cxcywh` 如何转换为 pixel `xyxy`，resize/letterbox 为什么会改变结果?
3. strict parser、lenient parser 和 protocol-fixed scorer 应怎样分层?
4. grounding score 为什么不能直接等价为 action success?
5. 如何把这些合同映射到 OPD 的五 expert、point protocol 和 raw-holdout 证据?

知识地图位置与前置依赖

```
VOL 01 坐标系 / 几何 / 图像尺寸
          ↓
PER-001 point-box-text contract
          ↓
PER-002 3D lift / camera calibration
          ↓
PER-003 multi-view fusion / temporal grounding
          ↓
VLA action head / planner / controller / closed-loop evaluation
```

前置知识是图像尺寸、坐标系、归一化和 parser 基础；后续节点将讨论深度、内外参、多视角融合和导航。Grounding DINO 和 RoboPoint 是公源机制参照，OPD 是项目证据 hook，不把项目数字当成通用算法结论。

1. 30 秒直觉：grounding 是接口合同，不是一个分数

grounding 的输出要像一个带 schema 的消息：`target`、`representation`、`coordinate_space`、`image_size`、`values`、`confidence`、`validity` 和 `parser_version`。同样四个数字，可能是 normalized `cxcywh`，也可能是 pixel `xyxy`；同样一串点，可能是部署要求的 `<point>...</point>`，也可能只是 lenient parser 接受的 Python 风格字符串。若这些字段不固定，IoU 或 Hit@threshold 的变化无法判断来自模型、预处理、解析还是坐标换算。

2. 三类输出合同

2.1 Box：目标区域

表示	例子	需要绑定	典型风险
normalized `cxcywh`	`(0.50, 0.40, 0.20, 0.10)`	原图宽高、中心/尺寸语义、范围 `[0,1]`	当成角点或使用错误宽高缩放
pixel `xyxy`	`(x_min,y_min,x_max,y_max)`	原点、边界是否 inclusive、rounding	x/y 交换、越界、IoU 改变
normalized `xyxy`	四个相对角点	normalization 轴、裁剪/填充规则	直接乘错 `W/H`
camera/world box	2D 区域加深度或平面假设	intrinsics、extrinsics、深度来源、frame ID	把 2D 区域中心当成 3D 抓取点

Grounding DINO frozen inference code 给出一个具体实现：模型输出 `pred_boxes` 后先乘 `[W,H,W,H]`，再用 `box_convert(..., in_fmt="cxcywh", out_fmt="xyxy")`。这说明代码路径，但不说明任何自定义模型已经遵守同一合同。

2.2 Point：位置或 affordance

point 可以是目标中心、抓取候选、导航 waypoint 或 RoboPoint 风格的 spatial affordance keypoint。至少要记录：

```
(x, y), coordinate_space, image_width, image_height,
ordering, confidence, valid, parser_version
```

`<point>[[x1,y1],[x2,y2]]</point>`、`[(x1, y1), ...]` 和 JSON `{"points": ...}` 是三个不同的接口。多点输出还要定义顺序：按置信度、从左到右、沿轨迹，还是无序集合。顺序不清时，几何误差可能看似相同，但下游动作会执行到不同的点。

2.3 Text：语言标签或 phrase grounding

文本输出不能只保存最终字符串。应区分原始 phrase、规范化 label、token span、confidence、与 box/point 的关联，以及 `no-object` / `ambiguous` 状态。Grounding DINO 的 phrase extraction 使用 tokenized caption 和 text threshold；因此 phrase 命中不等于目标绑定正确，也不等于 action head 已消费该结果。

3. 坐标变换：先声明假设，再写公式

令原图宽高为 (W, H)，模型输出 normalized `cxcywh=(c_x,c_y,w,h)`。当没有裁剪或 padding 时，一种明确的 pixel `xyxy` 转换是：

$$x_1 = W\left(c_x - \frac{w}{2}\right), \qquad y_1 = H\left(c_y - \frac{h}{2}\right), \qquad x_2 = W\left(c_x + \frac{w}{2}\right), \qquad y_2 = H\left(c_y + \frac{h}{2}\right).$$

这里的 (c_x, c_y, w, h) 是相对于原图的归一化量；公式只负责尺度和参数化，不负责 letterbox padding、crop offset、相机外参、深度或控制器 frame。代码实现必须保留 transform metadata，否则只能得到“看起来在图上”的框。

若预处理先把原图缩放为 (sW, sH)，并在左、上方向加入 padding (p_x, p_y)，则模型坐标到预处理图坐标的框为：

$$(x'_1, y'_1, x'_2, y'_2) = (sx_1 + p_x, ; sy_1 + p_y, ; sx_2 + p_x, ; sy_2 + p_y).$$

逆变换必须执行 $x = (x' - p_x)/s$，$y = (y' - p_y)/s$，并在最终 rounding 前裁剪到原图边界。若把 (p_x, p_y) 忽略，目标会随横纵比变化系统性偏移；这不是模型“空间理解变差”的充分证据。

4. 评估指标与证据阶梯

4.1 Box IoU

两个二维框 (A,B) 的交并比为：

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

IoU 只描述平面区域重叠；要报告阈值、样本数、空预测处理、坐标协议和 scorer 版本。它不包含动作单位、控制延迟、碰撞、接触稳定性或任务 success predicate。

4.2 Point Hit@threshold

point 指标应明确距离单位：像素距离、归一化距离还是相机/世界坐标距离。`Hit@50` 与 `Hit@100` 只有在图像尺寸、有效样本、parser mode 和距离定义都固定时才可比较。strict parser=0 可能是格式不匹配；lenient parser 的提升只能说明兼容格式的几何估计，不可自动替代部署协议。

4.3 Text / phrase score

text exact、token overlap、phrase grounding accuracy 和 box/point pairing 是不同指标。字符串正确但坐标错，或者坐标正确但标签格式不符，都会在不同 scorer 下产生相反结论。因此结果表必须按 representation 和 protocol 分栏，不能把所有输出平均成一个“grounding accuracy”。

5. 一个可审计的 grounding pipeline

```
def ground_and_validate(image, prompt, model, contract):
    # 1. Preserve the original image size and reversible preprocess metadata.
```

```
    preprocessed, transform = preprocess(image, contract.resize_policy)

    # 2. Run the model; output type is explicit, not inferred from four numbers.
    raw = model(preprocessed, prompt)
    parsed = parse_output(raw, contract.parser_version)

    # 3. Reject malformed or ambiguous outputs before any coordinate transform.
    validate_representation(parsed, contract.representation,
                            contract.coordinate_space, contract.value_range)

    # 4. Undo padding/crop/resize and clip only under the declared policy.
    original_space = inverse_transform(parsed, transform)
    original_space = clip_or_reject(original_space, contract.boundary_policy)

    # 5. Emit an auditable record for scorer and downstream planner/controller.
    return {
        "target": parsed.target,
        "representation": contract.representation,
        "coordinate_space": "original_pixel",
        "values": original_space.values,
        "confidence": parsed.confidence,
        "valid": parsed.valid,
        "parser_version": contract.parser_version,
        "transform_id": transform.id,
    }
```

伪代码的关键是先解析和验证，再做逆变换；不能看到四个浮点数就直接 reshape。真正进入 action head 前，还需增加深度/外参、frame ID、单位和 controller schema 检查。

6. 从 grounding 到 action：中间接口不能省略

```
grounding output
  → representation parser
  → original-image coordinate
  → camera/world lift (optional)
  → target/affordance selection
  → policy observation or planner goal
  → normalized action + denormalization
  → controller / safety predicate
```

这里至少有三个不同的成功事件：

1. **grounding valid**：格式、范围和坐标合同通过；
2. **geometric match**：IoU/Hit/距离达到 scorer 阈值；
3. **task success**：规划器、控制器和环境 predicate 均通过。

第 1、2 层通过不能推出第 3 层。尤其是 point 中心不一定是抓取点，box 不含深度和遮挡信息，text label 也不包含动作方向或接触力。

7. OPD 项目映射与真实性边界

OPD 证据	面试可说	不能升级为
五 expert mix：reasoning/object/region/spatial/point	数据构造显式分域，point/box/text 有不同 route	五个 expert 已分别独立最优或 route 已是 learned router
`target_expert` / `candidate_experts` / format-conflict metadata	路由监督和协议边界被写入样本	student 一定在推理时选择正确 expert
raw-holdout dedupe audit	可审计 image/fingerprint/source-line/prompt-answer-image leakage gate	静态 holdout 等价真实机器人泛化

OPD 证据	面试可说	不能升级为
original strict / protocol-fixed strict / lenient point scorer	能区分格式失败和几何能力	strict=0 直接等于模型不会 point grounding
by-format / by-pool checkpoint table	结果必须绑定 checkpoint、子集和 scorer，且最佳点可能不同	ckpt5000 是所有任务池全局最佳，或提升由 OPD 单独造成

项目材料支持的是数据路由、协议核对和评估边界；模型权重、完整 raw predictions、单因素 ablation 和逐文件 ownership 仍缺，题卡和口述应保持 `WARN/unseen`。

8. Trade-off 与高概率 bad case

决策	好处	风险 / 诊断
strict parser	部署协议稳定、失败可见	格式错误会掩盖几何能力；需 protocol-fixed 对照
lenient parser	兼容历史输出、适合定位 parser bug	可能接受错误 axis/unit，不能作为生产 scorer
normalized coordinates	跨分辨率方便	忘记 padding/crop 或 `W/H` 轴会产生系统偏移
pixel coordinates	直观、易画框	强依赖 image size、resize 和 rounding
point affordance	比 box 更接近抓取/导航候选	单点不含深度、姿态、遮挡与接触语义
多点候选	可表达路径/抓取备选	顺序、去重、置信度和下游选择必须固定

常见 bad case：把 `cxcywh` 当 `xyxy`；x/y 乘错图像尺寸；忽略 letterbox offset；用字符串命中代替 phrase grounding；strict=0 就断言能力为零；用 IoU/Hit 推断 controller success；把 OPD 的组合结果写成单一 OPD uplift；把 Git author 当作完整 ownership。

9. 面试回答检查表

回答 PER-001 时至少覆盖：

1. 表示：point、box、text 的输出字段和用途；
2. 坐标：原点、轴、范围、图像尺寸、resize/letterbox 逆变换；
3. 解析：strict/lenient/protocol-fixed、invalid/no-object 和 parser version；
4. 指标：IoU、Hit、text score 的单位、阈值、分母和 scorer；
5. 闭环：grounding valid → geometric match → task success 的三层差异；
6. 项目：OPD route/raw-holdout/point protocol 的可说与不可说边界。

10. 结论与下一步

Grounding 的核心不是“输出一个框或点”，而是把 representation、coordinate transform、parser、scorer 和 downstream interface 固定成可审计合同。Grounding DINO 的 `cxcywh → xyxy` 代码是具体实现样例，RoboPoint 说明 point affordance 的机器人语义，OPD 提供真实项目中的多 expert、格式冲突和 raw-holdout 边界。当前节点 verdict 为 `WARN`；对应 `VLA-PER-001` 题卡已同步并保持 `unseen`，下一步进入真实口述或补 parser/normalizer roundtrip，再进入 3D lift / multi-view 的后续训练。

References

1. PER-001 Grounding / Coordinate Source Freeze。
2. Grounding DINO，arXiv:2303.05499v1 与 official repository @856dde20。
3. RoboPoint，arXiv:2406.10721v1。
4. 本仓库 OPD Claim–Evidence Inventory。

Part 2 / 43 · `interview_materials/vla/knowledge/vol_06_per002_3d_lift_frame.html` · SHA-256
`2815ADD4306E1B6C9CA55F9B81A405B555FC3BF7AE2504B98F8F79CAC37A68C5`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`2815add4306e1b6c9ca55f9b81a405b555fc3bf7ae2504b98f8f79cac37a68c5`；生成自：`interview_materials\vla\knowledge\vol_06_per002_3d_lift_frame.md`

VOL 06 Perception / Grounding / Navigation：2D → 3D Lift 与 Frame Contract（PER-002）

状态：`drafted / VLA-PER-002 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：PER-002 3D Lift / Frame Source Freeze。 前置知识：PER-001 Grounding / Coordinate。

问题（Question）

图像上的 point 或 box 只是二维观测。要让规划器或控制器使用它，需要补齐深度、相机内参/畸变、frame 变换、时间戳、单位和不确定性。缺任一项时，“模型预测了一个 3D 抓取点”可能只是把二维中心乘了一个未经验证的常数。

本页回答：

1. pixel point 如何反投影到 camera optical frame?
2. depth、intrinsics、distortion、extrinsics 和 image size 如何绑定?
3. ROS 的 base/optical/map/odom convention 能否直接当作项目事实?
4. 时间同步、invalid depth 和不确定性怎样进入 planner/controller gate?
5. 什么证据才能把 projection math 升级为 3D execution 或闭环 success?

知识地图位置与前置依赖

```
PER-001 2D point/box/text + image coordinate
                    ↓
PER-002 depth + calibration + frame transform
                    ↓
PER-003 multi-view / temporal fusion
                    ↓
planner goal / action head / controller / closed-loop predicate
```

OpenCV 的 calibration/depth 文档和 ROS REP-103/105 是公源机制参照；OPD/WAM 仅用于项目真实性边界。当前材料没有用户项目的 calibration、TF、depth 或 3D target bundle，因此本页不声称个人完成了 3D lift。

1. 30 秒直觉：3D lift 是一条带时间戳的链

```
pixel (u,v)
  + depth Z + intrinsics/distortion
  → camera optical point p_c
  + timestamped extrinsic T_target←camera
  → target/world/base point p_t
  + uncertainty/validity gate
  → planner/controller input
```

二维坐标没有深度；深度没有 frame 就无法解释；frame 没有时间戳就可能把运动中的旧位姿应用到新图像。一个能画出投影 overlay 的 demo，只证明部分几何链路，不证明目标被正确抓取或导航。

2. Camera model：内参、畸变与 image-size contract

2.1 Pinhole intrinsics

相机内参矩阵通常写为：

```
K = [[fx,  0, cx],
     [ 0, fy, cy],
     [ 0,  0,  1]]
```

`fx,fy` 是像素尺度焦距，`cx,cy` 是主点。它们只对特定 image size、裁剪和 resize 语义有效；如果把 640×480 的主点直接用于 224×224 的 letterboxed tensor，必须先证明 transform 如何更新 `K`。

2.2 Distortion / rectification

现实镜头存在径向和切向畸变。point 位于原始畸变图像还是 rectified 图像，必须写在 schema 中。两种合法路径是：

1. 先对图像和 point 一起 `undistort/rectify`，再使用 rectified `K`；
2. 保留原始 point，使用带 distortion 的反投影函数。

不能对畸变 pixel 直接套无畸变 pinhole 公式，并把系统偏差归因给 grounding 模型。

2.3 Calibration evidence

一个可复查的 calibration record 至少包括：标定板/场景、视角数量、image size、`K`、distortion coefficients、reprojection error、工具版本、时间和 revision。OpenCV 教程支持这些概念和实现入口，但不证明任何本地相机已经通过标定门。

3. Depth：没有 Z 就没有唯一的 3D 点

3.1 深度来源对照

来源	适用前提	关键风险	需要记录
RGB-D aligned depth	RGB/depth 已配准且单位已知	hole、边缘混合、遮挡、时间偏移	depth unit、invalid code、registration revision
stereo disparity	baseline、左右相机模型和对应有效	远处误差放大、遮挡无视差	baseline、disparity scale、valid mask
multi-view triangulation	多视角 pose 和匹配可信	pose/匹配误差、动态物体	view IDs、pose timestamps、inlier rule
plane/table hypothesis	目标落在已知平面	目标不在平面、遮挡和倾斜	plane frame、residual、valid region

来源	适用前提	关键风险	需要记录
monocular depth	模型提供尺度或可校准深度	scale ambiguity、domain shift	model revision、scale calibration、uncertainty

RGB 图像本身只提供射线，不提供唯一的 `Z`。`0`、NaN、饱和、边缘混合和超出 range 的值必须进入 invalid 或 uncertainty 分支，不能静默当作真实深度。

4. 反投影与 frame transform

4.1 Pixel → camera optical frame

对 rectified pixel `(u,v)`、深度 `Z` 和内参 `K`，相机 optical frame 中的 pinhole 反投影为：

$$X_c = \frac{u - c_x}{f_x} Z, \qquad Y_c = \frac{v - c_y}{f_y} Z, \qquad Z_c = Z.$$

`u,v` 的原点、`Z` 的单位和 `K` 的 image-size revision 必须一致。若 point 来自 letterbox 图像，先逆 resize/padding 到 calibration image space，再采样 depth；否则 `u,v` 会带有系统偏移。

4.2 Camera → target/world frame

用齐次变换把 camera point 变到目标 frame：

$$\begin{bmatrix} p_t \\ 1 \end{bmatrix} = T_{t \leftarrow c}(\tau) \begin{bmatrix} p_c \\ 1 \end{bmatrix}, \qquad T_{t \leftarrow c} = \begin{bmatrix} R_{t \leftarrow c} & t_{t \leftarrow c} \\ 0 & 1 \end{bmatrix}.$$

`R` 是旋转，`t` 是平移，`τ` 是图像采集时刻或明确的插值时刻。下标要能回答“变换从哪一个 frame 到哪一个 frame”；只保存一个无标签 4×4 矩阵不够复现。

4.3 ROS convention 是参照，不是事实

REP-103 提供右手系、SI 单位和 body/optical frame 的常见约定；REP-105 讨论 `map`、`odom`、`base_link` 等移动平台 frame 语义。项目可以采用自己的 convention，但必须显式写轴方向、单位、frame ID 和变换方向，不能默认所有 VLA 项目都运行 ROS TF。

5. 时间同步与不确定性账本

至少保留以下时间戳：

```
t_image, t_depth, t_calibration, t_tf, t_robot_state, t_action
```

检查项包括：

- RGB 与 depth 是否属于同一 capture 或经过可解释配准；
- `T_target←camera` 是否在 `t_image` 插值，而不是使用未来 pose 或过期 pose；

- 机器人/目标运动期间，时间偏差造成的空间误差是否小于 freshness budget；
- depth 与 RGB 的 resolution、crop、registration 是否同源；
- action 发出时，3D target 是否仍有效。

不确定性账本可以按来源拆分：

来源	对 3D 结果的影响
pixel/parser error	深度越大，横向 `X/Y` 误差通常越大
depth noise	直接扰动 `Z`，并通过反投影影响 `X/Y`
intrinsics/distortion	系统性投影偏差
extrinsics/TF	整体旋转/平移偏差
timestamp skew	运动场景中的动态偏差
axis/unit/frame bug	可能产生大幅、非随机错误

uncertainty gate 可以在 planner 前拒绝 invalid/高风险目标，或把协方差/置信度作为 planner 输入；但 gate 的阈值和 fallback 仍需与任务安全合同绑定。

6. 可审计 3D-lift pipeline

```
def lift_grounding(point, rgb, depth, calib, tf_buffer, contract):
    # 1. Validate image/depth size, timestamp, units and invalid-depth policy.
    assert contract.image_size == calib.image_size
    z = sample_depth(depth, point, contract.depth_sampling)
    if not valid_depth(z, contract.depth_range):
        return Invalid(reason="depth", timestamp=rgb.timestamp)

    # 2. Move the point into rectified calibration pixels.
    uv = undo_resize_crop_letterbox(point, contract.transform)
    uv_rect = rectify_or_validate(uv, calib, contract.distortion_mode)

    # 3. Back-project to the camera optical frame.
    p_camera = backproject_pinhole(uv_rect, z, calib.K, calib.depth_unit)

    # 4. Interpolate the timestamped camera->target transform.
    T = tf_buffer.lookup(calib.camera_frame, contract.target_frame,
                         rgb.timestamp, contract.tf_policy)
    p_target = transform_point(T, p_camera)

    # 5. Apply uncertainty/safety gate and emit full provenance.
    uncertainty = propagate_uncertainty(point, z, calib, T)
    if not contract.gate(uncertainty, p_target):
        return Invalid(reason="uncertainty", timestamp=rgb.timestamp)
    return Target3D(point=p_target, frame=contract.target_frame,
                    timestamp=rgb.timestamp, uncertainty=uncertainty,
                    calib_revision=calib.revision, tf_revision=T.revision)
```

伪代码把 invalid depth、逆预处理、去畸变、反投影、时间戳 TF、uncertainty 和 provenance 作为同一条链；它是面试设计合同，不是用户项目已有实现。

7. 证据阶梯：从几何到闭环

层级	最小可审计证据	不能越级支持
projection math	固定 `K/distortion/image_size` 的 pixel→camera→pixel roundtrip	深度正确、外参正确
calibration	calibration inputs、reprojection error、版本、单位	运行时 TF 和同步正确

层级	最小可审计证据	不能越级支持
frame transform	`T_target←camera`、frame IDs、timestamp、逆变换 smoke	目标一定被抓取/导航
planner interface	3D target、uncertainty gate、planner input/output trace	controller 已执行成功
closed-loop episode	same protocol、controller trace、success predicate、failure video、完整分母	跨任务/真机泛化，除非协议一致

OPD 当前是 static point/box/text holdout，缺 depth/TF/3D planner trace；WAM bundle 有 camera/controller/horizon 字段，仍缺 calibration、depth、3D target 和 checkpoint verification。因此二者都不能作为 PER-002 的个人 3D 成果证明。

8. 项目映射与回答边界

OPD

可以讲：2D point/box/text 的协议、route metadata、strict/lenient scorer 和 raw-holdout leakage gate。不能讲：point 已自动 lift 到 base frame、五 expert 已解决深度、或 static holdout 证明机器人抓取。

WAM

manifest 中的 camera resolution、controller、open-loop 和 action shape 可用于说明 rollout provenance；它们不包含 calibration matrix、depth stream、TF tree、3D target 或真机 success。应把它们定位为 rollout interface evidence，而不是 3D perception evidence。

个人 ownership

如果没有 calibration script、TF config、roundtrip log、failure trace 或逐文件 ledger，只能说“理解并审计 3D lift 合同”，不能说“我完成了相机标定/外参标定/3D grasp pipeline”。

9. Trade-off 与高概率 bad case

选择	优点	风险
RGB-D	单帧可得 Z，链路短	配准、边缘 hole、单位和时间同步
stereo	不需要主动深度	视差、baseline、遮挡和远距离误差
plane hypothesis	桌面任务简单稳定	非平面目标和遮挡会系统偏差
monocular depth	硬件简单	scale/domain shift/uncertainty
static extrinsic	简单、低运行时成本	相机移动或机械臂 flex 后失效
timestamped TF	可处理运动	需要 buffer、插值和时钟同步

常见 bad case：用 2D box 中心当 3D grasp 点；把 RGB resolution 的 `K` 用到 resized tensor；畸变点直接套 pinhole；深度单位 mm/m 混淆；忽略 depth invalid code；camera→base 方向写反；使用最新 TF 而不是 capture-time TF；只看 overlay 不做 roundtrip；把 ROS frame convention 当作项目事实；把 planner trace 当作 controller success。

10. 面试回答检查表

回答 PER-002 至少覆盖：

1. 深度来源和 invalid policy；
2. `K/distortion/image_size` 与 pixel/rectified space；
3. (X_c, Y_c, Z_c) 反投影公式；
4. `T_target←camera(timestamp)` 的方向、frame ID、单位和 revision；
5. RGB/depth/TF/robot-state 时间同步；
6. uncertainty gate 与 planner/controller 分层；
7. OPD/WAM 当前没有 3D artifact 的证据边界。

11. 结论与下一步

3D lift 的关键不是记住一个反投影公式，而是把 depth、calibration、frame、timestamp、uncertainty 和 downstream predicate 绑定成可复查链路。OpenCV 和 ROS 文档支持机制与 convention；当前项目材料只能支持 2D grounding 或 rollout provenance，不能升级为个人 3D lift 或真机成果。节点保持 `WARN`，对应题卡已生成并保持 `unseen`；下一步进行 PER-001..004 真实口述或补一个本地 3D roundtrip artifact。

References

1. PER-002 3D Lift / Frame Source Freeze。
2. OpenCV，camera calibration 与 depth/stereo。
3. ROS REP-103 与 REP-105。
4. PER-001 Grounding / Coordinate Knowledge。
5. OPD Claim–Evidence Inventory 与 WAM Artifact Manifest。

Part 3 / 43 · `interview_materials/vla/knowledge/vol_06_per003_multiview_temporal.html` · SHA-256 `2AF9B56C69C9A8470646EF29A3445D450AA9DE6C56DC3EA858AD1822B3D3C21E`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`2af9b56c69c9a8470646ef29a3445d450aa9de6c56dc3ea858ad1822b3d3c21e`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per003_multiview_temporal.md`

VOL 06 Perception / Grounding / Navigation：多视角、时序与输入 Schema（PER-003）

状态：`drafted / VLA-PER-003 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：PER-003 Multi-view / Temporal Source Freeze。 前置知识：PER-001 Grounding、PER-002 3D Lift。

问题（Question）

多相机、多帧输入经常被简化为“把图片 concat 后喂给模型”。这会隐藏四个关键问题：模型是否知道每张图来自哪台相机、每个像素属于哪个 frame、历史图像和动作是否时间对齐、缺失相机是否被零值误认为真实观测。

本页回答：

1. per-view schema 为什么不能只有 image tensor?
2. camera identity、frame identity、timestamp 与 presence mask 各自解决什么问题?
3. early/token/late fusion 和 temporal queue 的 trade-off 是什么?
4. 多视角预测为什么必须先变到 common frame 再融合?
5. 如何把 view/time schema 接到 grounding、action chunk 和 closed-loop evaluation?

知识地图位置与前置依赖

```
PER-001 point/box/text contract
          ↓
PER-002 depth/calibration/frame contract
          ↓
PER-003 multi-view/temporal schema + fusion
          ↓
PER-004 navigation / VLN / ObjectNav
          ↓
action chunk / planner / controller / closed-loop evaluation
```

Open X-Embodiment 和 WM-017 提供 heterogeneous schema 参照，Octo 提供 observation-group/readout/attention 的公开实现参照；这些来源不等于用户项目已经完成多视角融合。OPD/WAM 仅作为 evidence-boundary hook。

1. 30 秒直觉：一张图必须带身份证和时间戳

一个可用的 view/time item 不应只是 `image[T,C,H,W]`，而应像一条记录：

```
image + camera_id + frame_id + K/T revision
      + timestamp + valid/presence + transform_id
```

`camera_id` 防止 workspace/wrist 或 left/right 顺序交换；`frame_id` 防止不同坐标系直接比较；`timestamp` 防止旧图配新 pose 或新图配旧 action；`presence/valid` 防止缺失相机被零填充伪装成真实黑图。shape 正确只说明张量能进模型，不说明语义正确。

2. Per-view schema：把几何和时间作为数据字段

2.1 最小字段

字段	作用	缺失时的风险
`image`	经过声明颜色/resize policy 的像素	预处理与 calibration space 不一致
`camera_id`	稳定逻辑身份，如 workspace/wrist	camera order 交换仍能通过 shape check
`frame_id`	optical/body/world frame 标识	不同 pixel/3D frame 被直接平均
`intrinsics_id`	`K/distortion/image_size` revision	point/box 逆投影使用错误标定
`extrinsics_id`	`T_target←camera` revision	视角无法变换到共同 frame
`timestamp`	capture time	history/action 物理时间错位
`valid/presence`	camera/depth/frame 是否有效	zero shortcut、loss/scorer 污染
`transform_id`	crop/resize/letterbox metadata	输出坐标无法逆变换

Open X 的 RLDS episode/step container 能统一数据容器，但不代表每个子数据集拥有相同相机、action semantics 或 horizon。WM-017 的 schema freeze 将 camera、state、action、timestamp、mask 和 normalizer 明确拆开；这比“所有数据都有 image 字段”更强。

2.2 Camera 与 time embedding

在 token-level fusion 中，可以把每个 view/time 特征写成：

$$z_{i,t} = f_\theta(x_{i,t}) + e_i^{cam} + e_t^{time} + e_{i,t}^{frame},$$

其中 `i` 是 camera identity，`t` 是物理时间或经过离散化的时间位置，`e_frame` 可编码 view 的几何/embodiment 条件。公式只是设计模板；代码还必须把 `valid/presence` 和 transform metadata 传到 attention/pooling，而不是只加 embedding。

3. Temporal identity：index 不等于物理时间

历史窗口应保留真实时间或可解释的 control-step mapping：

```
history = [(camera_id, frame_id, t_i, valid_i, image_i), ...]
action_target = (t_action, horizon, control_hz, repeat, replan_policy)
```

至少区分：

- `history_len`：帧数；
- `stride`：采样间隔；
- `control_hz`：控制频率；
- `action_repeat`：一条 action 保持多久；
- `observation_age`：当前 action 使用的观测距采集时间多久；
- `replan_policy`：何时重新 grounding 或刷新 chunk。

frame drop、重复 terminal frame、异步 camera 和 action repeat 都可能让相同的数组长度对应不同物理时长。若历史帧用于训练，必须说明 causal/ordered mask，避免 future frame leakage；若历史帧用于在线推理，还要把 stale observation 纳入 action freshness gate。

4. Presence 与 validity：mask 不是可选装饰

缺失相机、无效 depth、掉帧或不同步 frame 应显式记录：

```
presence[i,t] ∈ {0,1}
valid_depth[i,t] ∈ {0,1}
timestamp_ok[i,t] ∈ {0,1}
```

如果只把缺失 view 置零而不提供 mask，模型可能学习“全零就是某个相机缺失模式”，而不是根据可用视角完成 grounding。mask 必须在以下位置保持一致：

1. token pooling/attention：无效 view 不应拥有与真实 view 相同的权重；
2. loss：缺失 target 或 padding 不应进入监督分母；
3. scorer：无效预测应有明确 reject/skip 规则；
4. planner/controller：高风险或 stale target 应进入 invalid/safe 分支。

5. Fusion 位置与 trade-off

方式	数据流	优点	风险
early pixel/feature concat	view features 在早期拼接	交互早、实现直观	resolution、order、显存和几何对齐复杂
token-level attention	每个 view/time 保留 token，用 camera/time embedding 和 mask	可表达跨视角/时序依赖	attention mask、缺失 view 和计算量需要审计
late prediction fusion	各 view 独立 grounding，再合并 box/point/score	单视角模块可复用	对象关联、重复检测、置信度校准难
temporal queue/chunk	重用历史特征、减少重复编码	降低部分推理成本	stale action、frame drop、replan 边界
view dropout/masking	训练时模拟缺失视角	可能提升缺失鲁棒性	没有 matched ablation 就不能声称提升

Octo 的 observation groups、task prefix、readout token 和历史 attention 是一种公开实现参照：它说明 view/time 信息可以通过 token/group/mask 组织，但不证明所有 VLA 都使用相同融合方式，更不证明用户项目已经复现该行为。

6. 多视角几何一致性：先 common frame，再融合

如果不同相机分别输出一个 point/box，不能直接对 pixel `(x,y)` 求平均，因为每个 image plane 的原点、焦距、畸变和视角不同。正确的概念链是：

```
view_i pixel
  → (K_i, distortion_i) → camera_i ray/3D candidate
  → T_common←camera_i(timestamp_i)
  → common-frame candidate + uncertainty
  → object association / fuse / reject
```

融合前应检查：

- `K_i/distortion_i/image_size` 是否是同一 revision；
- extrinsic 是否静态，或是否需要按 timestamp 查询；
- timestamp skew 是否在运动误差预算内；
- confidence 是否跨视角校准；
- 遮挡、重复检测和 object identity 如何匹配；
- depth/plane hypothesis 是否一致。

只在二维做 late box voting 而没有 frame/association contract，仍是二维 heuristic，不应称为 3D multi-view fusion。

7. 可审计的 view/time pipeline

```
def fuse_views_and_history(samples, model, contract, tf_buffer):
    # 1. Validate identity, shape, timestamps and reversible transforms.
    records = validate_view_time_manifest(samples, contract.schema_version)

    # 2. Preprocess each view independently; retain transform metadata.
    tokens = []
    for rec in records:
        if not rec.presence or not rec.timestamp_ok:
            continue
        image, transform = preprocess(rec.image, rec.resize_policy)
        z = model.encode(image)
        tokens.append(add_camera_time_frame_embedding(
            z, rec.camera_id, rec.timestamp, rec.frame_id, transform))

    # 3. Fuse only through the declared attention/missingness contract.
    fused = masked_view_time_attention(tokens, contract.presence_mask,
                                       contract.causal_mask)

    # 4. Convert per-view predictions to a common frame before association.
    candidates = []
    for pred in model.ground(fused):
        candidate = lift_and_transform(pred, tf_buffer, pred.timestamp,
                                       contract.common_frame)
        if candidate.valid:
            candidates.append(candidate)
    target = associate_fuse_or_reject(candidates, contract.association)

    # 5. Emit observation age and provenance for planner/action freshness.
    return attach_age_uncertainty_and_revision(target, records, contract)
```

伪代码的关键是先验证 manifest，再独立预处理各视角，保留 mask/transform，最后在 common frame 做 association；它是设计合同，不是 OPD/WAM 已有实现。

8. Grounding、action chunk 与 observation age

多视角模型输出一个“正确”的 target，也可能因为 observation 太旧而执行错误 action。因此 planner/policy 接口应至少绑定：

```
target_frame / target_timestamp / observation_age
valid / uncertainty / source_views
horizon / control_hz / action_repeat / replan_policy
```

如果 `observation_age` 超过 freshness budget，应重新 grounding、缩短执行窗口、进入 safe hold 或拒绝 action。action chunk 的长度不能只用数组 `H` 表示，还要换算为物理时间 `H / control_hz`，并与 replan policy 一起评估。

9. 证据阶梯与项目边界

层级	最小证据	不能越级支持
schema	per-view manifest、camera order、timestamp、presence mask	view fusion 正确
geometry	per-view calibration/extrinsic、common-frame transform、roundtrip	temporal consistency
fusion	matched multi-view target、association/uncertainty、view ablation	action/controller success
temporal	frame-drop/jitter/replay、observation age、causal mask audit	long-horizon task success
closed-loop	same task/init/seed/controller、episode denominator、failure trace	跨 embodiment/真机泛化

OPD 当前是 static point/box/text holdout，WAM bundle 只有有限 rollout provenance；二者都没有 multi-view calibration、sync、view ablation 或 temporal replay artifact。不能把“多路 camera 字段存在”写成完成了多视角模型。

10. 项目映射与常见 bad case

OPD

可用于说明 point/box/text route、strict/lenient scorer、raw-holdout 和格式冲突。它不能证明多相机 fusion、temporal consistency 或机器人闭环。

WAM

manifest 的 camera resolution、controller、horizon 和 rollout files 可说明接口 provenance；它没有 per-camera calibration、同步记录、view association、temporal replay 或 3D target，因此不能说完成多视角/时序 grounding。

Bad cases

- workspace/wrist 或 left/right camera order 交换，但 shape 仍正确；

- 不同 image size 使用同一个 `K`，或 resize 后忘记更新 transform；
- 把多个 pixel frame 直接平均；
- 只用数组 index 表示时间，忽略 frame drop/action repeat；
- 缺失 camera 置零却不提供 presence mask；
- 将旧观察的 target 沿用到新 action chunk；
- 用 view dropout 后的单次指标声称鲁棒性提升；
- 将 Octo/Open X 的公开 schema 直接写成个人项目实现。

11. 面试回答检查表

回答 PER-003 至少覆盖：

1. per-view schema：camera/frame identity、K/T revision、timestamp、presence/valid、transform；
2. temporal schema：history、stride、control Hz、repeat、replan、observation age；
3. fusion：early/token/late/queue 的取舍；
4. common-frame association，而不是 pixel 直接平均；
5. causal/missingness mask 与 action/loss/scorer 的一致性；
6. view/geometry/fusion/temporal/closed-loop 五层证据；
7. OPD/WAM 当前没有 multi-view/temporal artifact 的边界。

12. 结论与下一步

多视角/时序 grounding 的难点不是输入图片数量，而是 identity、geometry、time、mask 和 fusion contract 是否保持一致。Open X/WM-017 支持 heterogeneous schema 审计，Octo 提供 observation group/readout/attention 参照；它们不能替代用户项目的 per-view calibration、sync、ablation 和 replay evidence。PER-003 保持 `WARN`，对应题卡已生成并保持 `unseen`；下一步进行 PER-001..004 真实口述或补一个本地 multi-view artifact。

References

1. PER-003 Multi-view / Temporal Source Freeze。
2. Open X-Embodiment frozen README。
3. Octo architecture/execution source audit。
4. WM-017 heterogeneous schema source freeze。
5. PER-001 Grounding / Coordinate Knowledge 与 PER-002 3D Lift / Frame Knowledge。
6. OPD Claim–Evidence Inventory 与 WAM Artifact Manifest。

Part 4 / 43 · `interview_materials/vla/knowledge/vol_06_per004_vln_objectnav.html` · SHA-256
`8D69223D9B9B998C7194F095F5A64B1454F7B54A5A603E4C4BB04CFC7FDABFFD`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`8d69223d9b9b998c7194f095f5a64b1454f7b54a5a603e4c4bb04cfc7fdabffd`；生成自：`interview_materials\vla\knowledge\vol_06_per004_vln_objectnav.md`

VOL 06 Perception / Grounding / Navigation：VLN 与 ObjectNav（PER-004）

状态：`drafted / VLA-PER-004 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：PER-004 VLN / ObjectNav Source Freeze。 前置知识：PER-001 Grounding、PER-002 3D Lift、PER-003 Multi-view / Temporal。

问题（Question）

VLN、ObjectNav 和 REVERIE 都把视觉、语言和移动动作放在同一条链上，但它们不是同一个任务。面试回答不能从“输入一张图和一句话，输出动作”直接跳到模型结构；应先冻结 episode 与 goal contract，再解释 observation/action/stop，最后说明 success、SPL 和 simulator-to-real 证据。

本页回答：

1. VLN、ObjectNav、REVERIE 的目标与监督有什么不同?
2. 一个可复现的 episode、observation、action、goal schema 应包含哪些字段?
3. Habitat-Lab 的 `stop`、距离度量、ObjectGoal/view-point 如何进入 success?
4. SR、SPL、TL、NE、OS 分别测什么，为什么不能脱离 protocol 比较?
5. 如何把任务协议接到 VLA grounding、planner、controller 和项目真实性边界?

知识地图位置与前置依赖

```
PER-001 point/box/text grounding
          ↓
PER-002 depth / calibration / frame
          ↓
PER-003 multi-view / temporal schema
          ↓
PER-004 VLN / ObjectNav task + evaluation contract
          ↓
planner / action chunk / controller / closed-loop evidence
```

Habitat-Lab 是任务与评估框架，Habitat-Sim 负责场景、传感器和导航/物理模拟；VLN-CE 将 R2R/RxR 指令迁移到连续环境。R2R、REVERIE 和 ObjectNav Revisited 是任务/评估语境的原始论文。它们提供公开协议参照，不等于用户项目已经运行过这些 benchmark。

1. 30 秒直觉：先定义“到哪里”，再讨论“怎么走”

导航回答的最小闭环是：

```
episode(scene + start pose + goal/instruction + split)
  → observation(rgb/depth/semantic + pose + language + valid)
  → action(discrete/continuous + stop)
  → simulator transition(navmesh + collision + controller)
  → success predicate + path metrics
  → protocol-scoped report
```

同一个 `stop` 动作，在 PointNav、ObjectNav、R2R 和 REVERIE 中可能依赖不同目标字段、距离定义或 object-grounding 条件。没有 episode denominator、split、scene revision、action space、success threshold 和实际轨迹，单独的“成功率”不具备可复核性。

2. 任务 taxonomy：输入相似，目标合同不同

任务	goal / instruction	典型 observation	action / termination	success 的核心问题
ObjectNav	目标类别或 object ID；`ObjectGoal` 可带 room 与 `view_points`	RGB/Depth/Semantic、object-goal、pose/GPS/Compass	移动/转向或连续速度 + `stop`	停止时是否在 protocol 指定的目标点或 view-point 距离内
R2R/RxR VLN	自然语言 route instruction、scene、start pose、split、参考路径	RGB/RGB-D、pose、语言 token；连续环境还记录轨迹	nav-graph action 或连续前进/转角/俯仰 + `stop`	是否到达目标附近，以及走过的路径是否高效
REVERIE	instruction + referring expression 指向具体物体	视觉、语言、pose、候选 object/region	导航动作 + object selection/grounding	是否到达目标区域与是否选中正确物体，通常应分层报告

三个常见混淆：

- ObjectNav 的“椅子”是类别 goal，不等同于 REVERIE 的某个 referring expression 实例。
- R2R 的参考路线用于 instruction-following 与路径评估，不等同于 agent 必须逐点复现的 controller trace。
- grounding hit、进入物体附近和 task success 是不同证据层；point/box/text scorer 不能自动变成移动成功。

3. Episode contract：把 reset 和评估分母冻结

3.1 最小字段

```
episode_id
scene_id / scene_dataset_config
start_position: [x, y, z]
start_rotation: [qx, qy, qz, qw]
goals: position/radius 或 ObjectGoal + view_points
instruction / reference_expression (VLN/REVERIE)
shortest_paths (若数据协议提供)
split / seed / dataset_revision
```

Habitat-Lab 的 `NavigationEpisode` 以 episode/scene ID、起点位姿和 goals 组织导航任务；ObjectNav 数据集进一步保存 category mappings、`ObjectGoal`、`ObjectViewLocation` 和可选 shortest paths。字段存在只说明 schema 可解析，不说明该 episode 已经被完整执行。

3.2 Reset 与数据泄漏

reset 时至少要记录：

- scene asset 与 scene-dataset revision；
- start pose、agent embodiment、navmesh 和 collision/controller 配置；
- language/object goal parser 的版本；
- split（seen/unseen 等）、seed 和 episode ID；
- 是否加载 reference path、teacher action 或 oracle 信息。

如果把 shortest path、未来轨迹或 terminal observation 误放进 policy observation，就会产生 future leakage。teacher-forced action accuracy 也不能当作 closed-loop success。

3.3 Goal 与 view-point

`ObjectGoal` 的字段可能包括 `object_id`、`object_category`、room 信息和 `view_points`。因此至少要区分：

```
target point        → 到几何点的距离
target view-point   → 到可见目标的候选位置集合的距离
category goal       → 目标类别是否满足
instance/object ID  → 是否找到指定实例
```

若 config 在 `POINT` 和 `VIEW_POINTS` 之间切换，或将 semantic category 与 instance ID 混用，SR/SPL 就不再是同一任务的数字。

4. Observation contract：合法 tensor 不等于语义正确

4.1 视觉、语言与状态

建议将每个 observation 记录为带 provenance 的结构，而不是裸字典：

```
rgb/depth/semantic
  + sensor_uuid / resolution / intrinsics / noise policy
pose / gps / compass
  + frame_id / timestamp / calibration revision
instruction / reference expression
  + language / token_ids / attention_mask
valid / presence / observation_age
  + missing-sensor and terminal-observation policy
```

PER-003 的 camera identity、timestamp、presence mask 在这里成为导航输入合同；如果 RGB 与 depth 对调、语言 padding 未 mask、pose frame 错位或重复 terminal frame，shape 检查仍可能通过，但 policy 学到的是错误关联。

4.2 连续环境中的时间合同

对每个动作 a_t，应保留：

```
observation_timestamp
action_timestamp
control_hz / action_repeat
```

```
controller_delay
observation_age
replan_or_stop_policy
```

动作 chunk 的长度 (H) 不能只看数组维度；若控制频率为 (f)，近似执行时长是 (H/f)，还要加 action repeat、通信和 controller delay。旧 observation 上计算的目标可能在机器人移动后已经 stale，应触发 re-grounding、缩短 horizon、safe hold 或 action reject。

5. Action 与 stop：从任务动作到 controller

5.1 离散与连续 action

常见 action 层级如下：

层级	示例	必须冻结的字段
nav-graph edge	next viewpoint / shortest-path edge	graph、edge semantics、oracle 是否可见
fixed-step simulator	`move_forward`、`turn_left/right`、look up/down	步长、转角、碰撞和 action repeat
velocity control	linear/angular velocity	velocity range、time step、滑移/碰撞策略
waypoint/controller	target pose 或 local waypoint	frame、controller、latency、tracking tolerance

VLN-CE README 的 challenge 示例使用固定 turn angle、step size 和 RGB-D resolution；这是一个特定 benchmark contract，不是所有 VLN 或真实机器人系统的默认值。

5.2 `stop` 不是“模型结束了”

Habitat-Lab 的 `StopAction` 会设置 task 的 `is_stop_called`。随后 `Success` 还要依赖距离度量与 success distance；只在目标附近经过但没有调用 stop，按该实现可能仍是失败。相反，某些 benchmark 的成功谓词还包括 object ID、region/IoU、视野或引用表达式匹配，不能用一个通用的“距离小于阈值”替代全部任务。

5.3 最小动作执行日志

```
episode_id, step_id
observation_timestamp, action_timestamp
action_type, action_parameters, repeat
pose_before, pose_after, collision, controller_status
stop_called, terminal, failure_reason
```

没有 action trace，就无法判断失败来自 grounding、规划、controller、碰撞、延迟还是终止条件。

6. Success、SR、SPL 与配套指标

6.1 Success predicate 与 SR

先为每个 episode 定义 $S_i \in \{0, 1\}$。一个 Habitat 风格的导航例子是：

```
S_i = 1{stop_called_i == true
        and distance_to_goal_i < success_distance}
```

实际任务还可能要求 object category/ID、view-point 可见性、region IoU 或 language reference match。Success Rate 为：

$$SR = \frac{1}{N}\sum_{i=1}^{N} S_i.$$

这里的 (N) 必须是明确的 evaluated episodes；过滤掉失败、重复 episode 或不同 split 后再报 SR，会改变指标含义。

6.2 SPL

在 shortest path L_i、实际轨迹长度 P_i 和 success predicate 都冻结时：

$$SPL = \frac{1}{N}\sum_{i=1}^{N} S_i \frac{L_i}{\max(P_i, L_i)}.$$

解释：成功但绕路，L_i/P_i 变小；失败 episode 的 $S_i = 0$。实现仍需注明：

- L_i 是 navmesh geodesic、graph path、view-point 集合最小值还是其他参考；
- P_i 是否包含 stop 前最后一步、controller 微步和重复 action；
- 是否按 episode、任务、seed 或 scene 聚合；
- success threshold、episode filtering、终止/超时规则。

6.3 TL、NE、OS 与“高分陷阱”

指标	解释	不能单独证明
TL	Trajectory Length	到达正确目标
NE	final navigation error	agent 是否正确 stop、是否高效
OS	Oracle Success：轨迹中曾到达	agent 的 stopping policy 正确
SR	episode-level success ratio	路径效率或跨 split 泛化
SPL	success × path efficiency	真机稳定性、碰撞安全或操作成功

例如 OS 高而 SR 低，可能表示 agent 经过目标但不会停止；SR 高而 SPL 低，可能表示到达但路径绕行；NE 低但 SR 低，可能是 stop predicate 或阈值问题。面试时先拆 failure taxonomy，再谈模型 uplift。

7. 一个可审计的 evaluator 伪代码

```
def evaluate_episode(episode, trace, protocol):
    # 1. Verify denominator and identity before computing a metric.
    assert trace.episode_id == episode.episode_id
    assert trace.scene_revision == protocol.scene_revision
    assert trace.split == protocol.split

    # 2. Reconstruct the executed path, not the reference shortest path.
    path = reconstruct_path(trace.poses, trace.controller_steps)
```

```
    path_len = geodesic_or_declared_length(path, protocol.path_metric)

    # 3. Compute task-specific goal distance/predicate.
    goal_distance = distance_to_goal(
        trace.final_pose, episode.goals,
        mode=protocol.distance_to,  # POINT or VIEW_POINTS, for example
    )
    stop_ok = trace.stop_called and trace.terminal_reason == "stop"
    success = protocol.success_predicate(
        episode=episode,
        final_pose=trace.final_pose,
        goal_distance=goal_distance,
        stop_ok=stop_ok,
        object_selection=trace.object_selection,
    )

    # 4. Only then compute SR/SPL and retain all provenance.
    shortest_len = shortest_reference_length(episode, protocol)
    spl = float(success) * shortest_len / max(path_len, shortest_len)
    return {
        "success": int(success),
        "spl": spl,
        "tl": path_len,
        "ne": goal_distance,
        "protocol": protocol.id,
        "failure_reason": classify_failure(trace, success),
    }
```

关键设计不是公式本身，而是 `protocol.success_predicate`、`distance_to`、`path_metric`、`scene_revision` 和 episode identity 不能被 evaluator 隐式猜测。所有失败样本和 denominator 都应保留，便于 paired comparison 与置信区间分析。

8. Habitat-Sim / Habitat-Lab 的 simulator 边界

Habitat-Sim 的 `PathFinder` 以 navmesh 近似可通行区域，可查询 geodesic path、`isNavigable`、island 和 try-step；navmesh 由 agent height、radius、最大爬升、坡度等约束影响。Habitat-Lab 则在上层组织 task、dataset、sensor、action 和 measures。

这意味着：

1. shortest geodesic path 是 simulator/navmesh 参考，不是 controller 实际轨迹；
2. 改变 agent radius、navmesh 参数、scene asset 或碰撞/滑移策略，可能改变 success 与 SPL；
3. simulator RGB-D、reset 和同步条件不自动等于真机相机、底盘和时间系统；
4. VLN-CE README 中的 baseline 数字只能在相同 repo、数据、预处理、硬件与 evaluator 语境下解释。

9. Grounding → navigation → control 的失败树

```
任务失败
├─ goal/语言解析
│  ├─ 类别、实例、room、reference expression 混淆
│  └─ instruction token / language mask / split 错误
├─ perception / grounding
│  ├─ point/box 坐标或 frame 错误
│  ├─ 多视角未做 common-frame association
│  └─ stale observation / invalid sensor 未被 mask
├─ planning
│  ├─ navmesh island 无路径
│  ├─ stop/replan/horizon policy 不合适
│  └─ shortest path 与 actual path 混淆
├─ controller / simulator
│  ├─ action repeat、速度、转角或 latency 不匹配
│  ├─ collision/sliding 与 protocol 不同
│  └─ terminal / timeout / stop 记录错误
└─ evaluation / evidence
   ├─ denominator、split、scene revision 不一致
```

```
    ├─ SR/SPL 聚合或 shortest path 定义改变
    └─ simulator proxy 被写成真机成功
```

这棵树适合把“模型没到目标”拆成可验证的故障，而不是把所有失败归因于 backbone 或语言理解。

10. 与个人项目的真实连接

项目	可回答的连接点	当前不能声称
OPD / VLM grounding	point/box/text route、坐标变换、strict/lenient scorer 可作为导航前的目标 grounding 讨论	没有 VLN/ObjectNav episode、navmesh trajectory、SPL 或真实移动成功证据
Robot Harness	tool schema、timeout/cancel/compensation 可用于讨论导航服务的接口和失败恢复	没有 Habitat task success、R2R/RxR 指令跟随或底盘控制 benchmark
OpenPiVLA	observation/action packaging、latency、chunk/replan contract 可迁移到导航 policy server	没有导航 split、episode denominator 或 SR/SPL artifact
WAM / Cosmos	simulator rollout provenance、controller/horizon 和 metric boundary 可用于比较 simulator evidence 层	没有 ObjectNav/VLN 结果、3D navigation 或真机闭环
UWM / LIBERO	reset、任务协议、closed-loop eval 与 evidence ladder 可迁移到 benchmark 设计	没有语言导航目标或 ObjectNav success

当前项目证据只支持这些“接口与审计方法”的连接，不支持把现有项目包装成 VLN/ObjectNav 经验。

11. 面试回答检查表

回答 PER-004 至少覆盖：

1. 先区分 VLN、ObjectNav、REVERIE 的 goal/instruction/success contract；
2. 说清 episode 的 scene、start pose、goal、split、seed、revision 和 denominator；
3. 说明 RGB-D、pose、language、timestamp、mask 与 observation age；
4. 区分 nav-graph、fixed-step、velocity、waypoint/controller action 和 `stop`；
5. 给出 success predicate、SR、SPL，并说明 shortest path 与 actual path 的区别；
6. 用 failure tree 分离 grounding、planning、controller、simulator 和 evaluator 问题；
7. 明确 simulator、offline proxy、simulated closed-loop、real closed-loop 的证据阶梯；
8. 把 OPD/Harness/WAM/UWM 的项目挂钩限定在真实 artifact 支持的范围内。

12. 结论与下一步

VLN/ObjectNav 的难点不是把语言接到 policy，而是让 goal、observation、action、stop、simulator 和 evaluator 在同一个 protocol 中闭合。Habitat-Lab/Habitat-Sim/VLN-CE 提供可审计的公开实现参照；R2R、REVERIE、ObjectNav Revisited 与 SPL 论文提供任务和指标语境。当前本地项目没有 VLN/ObjectNav run bundle，因此本页保持 `WARN`，不生成个人导航结果。

对应的 `VLA-PER-004` 原子题卡已生成并保持 `unseen`；下一 bounded slice 进行 PER-001..004 的真实口述训练，或补本地导航 artifact，不把公开 baseline 写成个人结果。

References

1. PER-004 VLN / ObjectNav Source Freeze。
2. Habitat-Lab frozen README、ObjectNav task、navigation measures。
3. Habitat-Sim frozen README 与 PathFinder。
4. VLN-CE frozen README。
5. R2R、REVERIE、ObjectNav Revisited、SPL。
6. PER-001 Grounding、PER-002 3D Lift、PER-003 Multi-view / Temporal。

Part 5 / 43 · `interview_materials/vla/knowledge/vol_06_per005_navigation_policy_architecture.html` · SHA-256 `AA5E724E690E750C2D42DC8B094488F82A2CA1BB9AEE06E60F386ED9B921BBBB`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`aa5e724e690e750c2d42dc8b094488f82a2ca1bb9aee06e60f386ed9b921bbbb`；生成自：`interview_materials\vla\knowledge\vol_06_per005_navigation_policy_architecture.md`

VOL 06 Perception / Grounding / Navigation：Navigation Policy Architecture（PER-005）

状态：`drafted / VLA-PER-005 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：PER-005 Navigation Policy Architecture Source Freeze。 前置知识：PER-003 Multi-view / Temporal、PER-004 VLN / ObjectNav。

问题（Question）

一个导航 policy 可以直接从视觉/语言历史回归动作，也可以维护 semantic map、预测 waypoint，再交给 planner/controller。两者的差别不只是“端到端 vs 模块化”，还涉及记忆、partial observability、长时域探索、frame、action age、stop/replan 和延迟预算。

本页回答：

1. map-free、semantic map-based、waypoint 和 hybrid policy 的接口与 trade-off 是什么?
2. semantic exploration 如何把观测、地图、目标类别先验和 frontier/subgoal 连成闭环?
3. waypoint / subgoal 传给 planner/controller 时必须携带哪些 frame、unit、uncertainty 和 freshness 字段?
4. 如何拆分 capture、perception、map/planner、policy、controller 和 actuation latency?
5. 怎样通过 ablation、episode trace 和项目证据，避免把公开机制写成个人导航结果?

知识地图位置与前置依赖

```
PER-001 point/box/text grounding
          ↓
PER-002 depth / calibration / frame
          ↓
PER-003 multi-view / temporal schema
          ↓
PER-004 task / episode / success / SPL contract
          ↓
PER-005 policy architecture / exploration / waypoint / controller
          ↓
closed-loop evaluation / sim2real / VOL 07 project claims
```

GOSE、VLN-CE waypoint、Habitat-Lab/Sims 的公开实现和论文是机制参照。当前五项目的 PER-004 navigation artifact gate 为 `0_local_direct_navigation_bundles`，因此本页讲架构与审计方法，不宣称个人已实现或提升导航。

1. 30 秒直觉：记忆在哪里，动作在哪里落地

所有架构都要回答三件事：

```
我记住什么？ (history / map / semantic prior)
我输出什么？ (low-level action / waypoint / subgoal)
谁负责最后执行？ (learned policy / planner / controller)
```

map-free policy 把记忆压进 recurrent/transformer state，直接输出动作或局部 waypoint；map-based policy 显式维护空间/语义记忆，再挑选探索目标；waypoint policy 把长 horizon 拆成中间目标；hybrid policy 将学习模块与几何 planner/controller 分开。没有明确的 frame、stop、replan 和 latency，架构图无法说明真实闭环行为。

2. 架构 taxonomy

架构	典型数据流	优点	主要风险
map-free / reactive	image + language + pose/history → action 或 local waypoint	端到端、运行时简单、部署组件少	partial observability、长时域探索、hidden-state drift、stale observation；要记录 history、action age 与 replan
semantic map-based	perception + pose → occupancy/semantic map → goal/frontier score → planner → action	可积累历史、支持长时域探索、子目标较可解释	mapping/calibration/pose error、map stale、显存和延迟；要保存 map revision、更新轨迹、planner config、ablation
waypoint policy	language/visual state → next waypoint/subgoal → local controller	将长序列动作压缩为中间接口，适合连续环境	waypoint frame/scale、tracking error、stop/replan 与 controller latency；要有 waypoint-to-action trace
hybrid learned + classical	learned semantic proposal → classical planner/controller	可将语义理解和几何/安全约束分层调试	接口 mismatch、planner latency、目标抖动、model/planner 归因混淆；需要 matched component ablation

这些是可组合的设计家族，不是互斥模型名称。例如 map-free encoder 可以输出 waypoint，再交给 controller；map-based explorer 也可以使用 learned goal scoring。面试中先画清接口，再讨论模型优劣。

3. Map-free policy：简单不等于没有记忆

3.1 状态与 partial observability

真实导航通常只能看到局部 RGB-D/语义和有限历史，因此 policy 近似：

$$a_t = \pi_\theta(o_{t-k:t},\ l,\ s_t),$$

其中 o 是视觉/传感器 observation，l 是 instruction/goal，s_t 可以是 pose/proprio/history state。即使没有显式 map，Transformer cache、RNN hidden state 或 stacked frames 仍是记忆；需要冻结 history length、stride、mask、timestamp 和 reset 行为。

3.2 常见 failure

- 目标暂时不可见时，policy 没有探索先验，容易原地抖动或重复路径；

- long horizon 中 hidden state 与实际 pose 漂移，局部动作仍合法但全局方向错误；
- inference latency 或 action repeat 变化，使训练时的 action age 与部署时不一致；
- 只看 imitation/action loss，未记录 stop、replan、collision 和 episode success。

map-free 并不自动意味着低延迟：视觉 encoder、语言 cross-attention、policy server queue 和 controller 仍要计入 latency ledger。

4. Semantic map-based exploration

4.1 最小闭环

GOSE 的公开机制可抽象为：

```
RGB-D + pose
  → local geometry / semantic observation
  → episodic semantic map
  → target-category prior + frontier/object scoring
  → exploration subgoal
  → planner/controller
  → execute, update map, repeat
```

地图不是把过去图片简单拼起来，而是带有坐标、时间、语义和有效性合同的状态：

```
map_revision
cell/frame/metric resolution
pose source + timestamp + uncertainty
occupancy / obstacle / semantic evidence
last_update / confidence / decay policy
candidate object / frontier / subgoal
```

4.2 Category prior 与探索边界

目标类别先验可以改变候选 frontier 或 subgoal 的排序，但不能伪装成已观测到目标：

```
score(frontier) = geometry + semantic_prior + uncertainty/value
```

若 prior 只来自公开论文或训练数据，它支持的是探索机制解释；只有在项目中保存了 map snapshot、候选分数、执行轨迹和 matched ablation，才可讨论个人实现或 uplift。

4.3 Map-based failure

失败层	例子	需要的诊断
geometry	depth/pose/calibration 错、地图坐标漂移	roundtrip、pose error、map revision
semantic	object label 错、类别先验 shortcut、遮挡	per-class confusion、candidate evidence、view trace
exploration	frontier 重复、目标排序错误、无 fallback	subgoal score、visited/replan trace
planner	navmesh island 无路径、路径过长	planner status、geodesic/actual path
controller	waypoint tracking、collision、latency	action trace、tracking error、controller log

5. Waypoint / subgoal interface

5.1 中间目标 schema

waypoint 不能只是 `(x, y)` 或一个未注明单位的 pose；至少保留：

```
subgoal_id / source_views / instruction_span
frame_id / position-or-heading / unit
created_at / observation_timestamp / observation_age
uncertainty / valid / stop_allowed
replan_policy / horizon / controller_budget
```

同一 `(x, y)` 在 agent、camera、map、world frame 中含义不同；resize、map origin、heading convention 和 unit 变化会让 shape 正确的 waypoint 语义错误。`valid=false`、age 超预算或 uncertainty 超阈值时，应重新 grounding、replan、safe hold 或 reject，不能静默执行。

5.2 Waypoint 与 controller 的边界

waypoint policy 优化的是中间目标预测，不等于 controller 已经完成跟踪。至少要分别记录：

```
predicted waypoint
→ planner/local path
→ controller command
→ pose_after / tracking error / collision
→ stop or replan reason
```

如果只报告 waypoint prediction accuracy，而没有实际 action trace、controller frequency、tracking tolerance 和 stop predicate，不能称为 closed-loop success。

6. Planner、controller 与 stop/replan

6.1 重新规划触发

常见 trigger 包括：

- 固定时间或执行步数；
- tracking error、collision 或 controller timeout；
- map revision、目标重新出现或 semantic confidence 变化；
- observation age、uncertainty 或 action deadline 超阈值；
- stop predicate 满足或发现无可行路径。

每次 replan 应有 `reason` 和 `source_timestamp`，否则无法判断动作抖动来自模型、地图还是调度策略。

6.2 Stop 是独立协议

PER-004 已冻结：Habitat-Lab 的 `StopAction` 设置 `is_stop_called`，success 还依赖 task-specific distance/predicate。PER-005 进一步要求 policy、planner 和 controller 共享 stop contract：

```
stop_allowed = valid_goal
               and distance/view/object predicate satisfied
               and observation_age <= budget
```

如果 policy 预测“停”，controller 仍在执行旧 chunk，或 evaluator 没记录 terminal reason，模型与指标就不是同一个系统。

7. Latency ledger：平均 FPS 不够

一个闭环周期至少拆分：

```
T_capture + T_transfer + T_encode/ground + T_map/planner
  + T_policy + T_controller + T_actuation
```

若 observation 在 t_o 捕获、action 在 t_a 生成，则：

$$age_{action} = t_a - t_o + T_{queue} + T_{communication} + T_{execution\ delay}.$$

对窄空间、动态目标或短 stop threshold，平均 latency 不能替代 p50/p95、queue wait、replan count、stale-action rate 和 per-episode trace。action chunk 长度 (H) 还需结合 control-hz、repeat 和 replan cadence 换算成物理执行时间。

8. 可审计的 policy loop 伪代码

```
def navigation_step(obs, memory, protocol, planner, controller):
    # 1. Validate identity, timestamp, frame, unit and freshness.
    rec = validate_observation(obs, protocol.schema)
    if not rec.valid or rec.age > protocol.max_observation_age:
        return safe_hold(reason="stale_or_invalid_observation")

    # 2. Update explicit map only when this architecture declares one.
    if protocol.architecture in {"semantic_map", "hybrid"}:
        memory = update_semantic_map(memory, rec, protocol.map_contract)
        candidate = score_frontiers_or_objects(
            memory, rec.goal, protocol.exploration_contract
        )
    else:
        candidate = None

    # 3. Predict low-level action or a typed waypoint/subgoal.
    proposal = policy_or_waypoint_model(rec, memory, candidate)
    proposal = validate_subgoal(proposal, protocol.subgoal_contract)

    if protocol.stop_predicate(proposal, rec, memory):
        return stop(reason="predicate_satisfied", source=rec.timestamp)
    if not proposal.valid or proposal.uncertainty > protocol.max_uncertainty:
        return safe_hold(reason="invalid_subgoal")

    # 4. Planner/controller remain explicit evidence boundaries.
    local_path = planner.plan(proposal, protocol.planner_contract)
    command = controller.track(local_path, protocol.controller_contract)
    return command, memory, {
        "observation_age": rec.age,
        "replan_reason": proposal.replan_reason,
        "latency_breakdown": collect_latency_trace(),
    }
```

伪代码的关键不是某个模型名称，而是把架构选择、map 更新、waypoint validation、stop、safe hold、planner、controller 和 latency trace 分开。没有这些边界，端到端模型的一个输出无法解释失败归因。

9. 实验与 evidence ladder

9.1 必须固定的比较因素

因素	例子	不固定时的风险
policy family	map-free / map-based / waypoint / hybrid	把组件变化当作 backbone uplift
memory	history length、map size、semantic prior	长时域优势无法归因
planner/controller	same planner、controller、control-hz	policy 与执行器混淆
timing	queue、latency、repeat、replan	stale action 伪装成模型失败
task protocol	scene、split、goal、stop、denominator	SR/SPL 不可比

9.2 最小 ablation

map-based claim 至少应比较：去 map、去 semantic prior、固定 planner/controller；waypoint claim 至少应比较：direct action、waypoint + same controller、matched horizon/replan。每个分支保存 episode-level result、failure type、latency 和 raw trace，而不是只保留最终均值。

9.3 证据阶梯

```
schema/interface
  → map/subgoal mechanism
  → planner/controller trace
  → simulated closed-loop SR/SPL + denominator
  → matched sim-real / real navigation safety log
```

当前 PER-004 navigation artifact gate 只达到“公开机制 + 项目接口边界”；OPD/Harness/OpenPiVLA/WAM/UWM 没有 map/subgoal/action bundle，不能跨越到个人 navigation result。

10. 项目映射与真实性边界

项目	可以连接的机制	当前不能声称
OPD	point/box/text grounding 可作为目标候选或 semantic observation 前端	没有 semantic map、frontier、waypoint、planner 或 ObjectNav success
Robot Harness	semantic-navigation tool schema、scope、timeout/cancel/compensation 可作为高层服务接口	没有低层 controller、navmesh trajectory、VLN/ObjectNav episode 或真机导航
OpenPiVLA	action packaging、normalizer、chunk、latency 可迁移到 waypoint/policy server	没有导航 scene/split/stop/SR/SPL artifact
WAM / Cosmos	rollout provenance、controller/horizon、metric ladder 可用于 simulator evidence 讨论	没有 map-based exploration、waypoint 或 navigation result
UWM / LIBERO	reset、closed-loop protocol、failure/evidence ladder 可迁移到评估设计	没有语言导航或 ObjectGoal 结果

安全口径是：可以解释如何设计导航 policy 和审计 planner/controller 接口，但当前项目材料不支持“我实现了 GOSE/waypoint/VLN/ObjectNav 并取得某个结果”。

11. 面试回答检查表

回答 PER-005 至少覆盖：

1. 先画出 observation/memory → policy/map/waypoint → planner/controller → stop/replan 的接口；
2. 比较 map-free、map-based、waypoint、hybrid 的记忆、长时域和 failure trade-off；
3. 说明 semantic map 的坐标、pose、revision、semantic evidence、frontier/subgoal 和 stale policy；
4. 给出 waypoint 的 frame/unit/uncertainty/age/valid/replan schema；
5. 拆 capture、perception、map/planner、policy、controller、actuation latency；
6. 用 matched ablation 和 episode trace 区分 model、map、planner、controller、protocol effect；
7. 把 OPD/Harness/OpenPiVLA/WAM/UWM 的连接限定在现有 evidence gate 内。

12. 结论与下一步

导航 policy architecture 的核心不是选择一个“更先进”的家族，而是把记忆、空间、动作接口、controller、stop、replan 和 timing 变成可审计合同。GOSE、VLN-CE waypoint 与 Habitat-Sim 提供公开机制参照；当前本地项目仍没有 map/subgoal/action/episode bundle，因此本页保持 `WARN`。

对应的 `VLA-PER-005` 原子题卡已同步并保持 `unseen`；下一 bounded slice 进行 PER-001..005 的真实口述训练，或补本地导航 artifact。

References

1. PER-005 Navigation Policy Architecture Source Freeze。
2. GOSE、ObjectNav Revisited。
3. VLN-CE paper、Waypoint Models、VLN-CE frozen README。
4. Habitat-Lab navigation measures 与 Habitat-Sim PathFinder。
5. R2R 与 REVERIE。
6. PER-003 Multi-view / Temporal 与 PER-004 VLN / ObjectNav。

Part 6 / 43 · `interview_materials/vla/knowledge/vol_06_per006_image_preprocess_token_camera_contract.html` · SHA-256 `FD923E666474A599E73A013FBD5F80615B708CF027A77BEFFB79A1D4DF30BE99`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`fd923e666474a599e73a013fbd5f80615b708cf027a77beffb79a1d4df30be99`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per006_image_preprocess_token_camera_contract.md`

VOL 06 Perception / Grounding / Navigation：Image Preprocess、Patch Token 与 Camera Identity（PER-006）

状态：`drafted / VLA-PER-006 / source_synced / card_synced / WARN` 报告日期：2026-07-19 证据截止：2026-07-19 来源入口：PER-006 Source Freeze。 题卡入口：PER-006 Atomic Card。 前置知识：PER-001 Grounding / Coordinate、PER-003 Multi-view / Temporal。

问题（Question）

一张机器人相机图像进入 Qwen3-VL 之前发生了什么？面试中如何从 resize/crop/letterbox、pixel range 与 normalization，一直讲到 patch grid、image tokens、camera identity 和 point/box 坐标逆变换？为什么本地 OPD 与 OpenPiVLA 不能只用一句“都调用 AutoProcessor”概括？

本页回答：

1. 原图、处理图、patch grid、image token 是哪四个坐标/表示空间？
2. Qwen3-VL 的 `patch_size=16`、`merge_size=2` 如何决定 factor32 和 token 数？
3. direct resize、crop、letterbox 的坐标映射有什么不同？
4. `min_pixels=max_pixels` 为什么仍不保证固定尺寸或固定 token 数？
5. camera identity、image order、frame 和 timestamp 为什么不由视觉 token 自动提供？
6. 如何解释 OPD double-resize 与 OpenPiVLA image lineage，而不把静态风险说成性能结论？

知识地图位置与前置依赖

```
raw image + camera/frame/time metadata
  -> PER-006 preprocess / pixel / patch / token contract
  -> PER-001 point/box/text inverse mapping
  -> PER-002 2D-to-3D / calibration / TF
  -> PER-003 multi-view / temporal fusion
  -> PER-007 open-vocabulary grounding pipeline
```

PER-006 不重复 VOL 01 的 Transformer 基础，也不重复 VOL 03 的 VLA action head；它负责把“图像如何成为 visual tokens”变成可复算、可逆映射、可绑定 camera identity 的工程合同。

1. 30 秒直觉：图像不是一个 tensor 名字

面试中最容易犯的错是从“输入一张 RGB 图”直接跳到“模型输出 point/box/action”。中间至少有四层：

层	典型对象	必须知道
原图空间	`I_raw∈R^{H×W×3}`	camera/frame、timestamp、原始宽高、颜色/位深
处理图空间	`I_proc∈R^{H'×W'×3}`	resize/crop/pad、插值、pixel range、mean/std
patch grid	`(T_g,H_g,W_g)`	patch size、temporal patch、grid 顺序
语言侧 image tokens	`N_img` 个 `<`	image_pad	> `	spatial merge、placeholder 顺序、与图像数量一致性

如果只保存 `pixel_values`，却不保存 `original_size`、`processed_size`、`image_grid_thw`、camera identity 和 transform 参数，就无法回答：

- point/box 坐标到底相对哪张图；
- 多个 image placeholders 分别属于哪台相机；
- token 数为何随宽高比变化；
- 训练与评估是否真的使用同一条 processor path。

2. Transform chain：先画数据流，再谈模型

冻结的 Qwen3-VL image-only 官方路径可抽象为：

```
path / URL / base64 / PIL
  -> decode + RGB
  -> smart_resize
  -> rescale + normalize
  -> temporal duplicate for a static image
  -> patchify
  -> image_grid_thw
  -> spatial merge
  -> image placeholder tokens
```

Qwen frozen `qwen-vl-utils` 代码使用保持宽高比的 smart resize 并直接调用 PIL resize；没有 crop、letterbox 或 padding-offset 分支。Qwen3 官方两阶段示例在 utils 已 resize 后给 processor 传 `do_resize=False`，避免重复 resize。

这不代表所有 VLM 都不使用 crop/letterbox；它只说明当前冻结的 Qwen path 不能被口述成 letterbox。工程回答必须以实际 processor/code revision 为准。

3. Smart resize：pixel budget 不是固定分辨率

3.1 三个约束

设原图为 $H \times W$，patch size 为 p，spatial merge 为 m，整除因子：

$$F = pm.$$

Qwen3-VL-8B 的 checkpoint contract 为 $p = 16, m = 2$，所以：

$$F = 16 \times 2 = 32.$$

smart resize 尝试满足：

1. $H' \bmod F = 0,\ W' \bmod F = 0$；
2. $H'W'$ 落在 `min_pixels/max_pixels` band；
3. 宽高比尽量接近原图。

变量映射：H', W' 是 processor 实际输入尺寸；F 对应 `patch_size*merge_size`；pixel band 来自 checkpoint processor config 或调用覆盖值。

3.2 为什么 min=max 也不精确

算法先把高宽 round 到 factor 倍数；若像素积过大，按 $\sqrt{HW/P_{\max}}$ 缩小并 floor；若过小，按 $\sqrt{P_{\min}/HW}$ 放大并 ceil。离散 factor 与 ceil/floor 意味着：

$$P_{\min} = P_{\max} = P \quad \not\Rightarrow \quad H'W' = P.$$

直觉：对某个宽高比，可能不存在同时满足“两个维度都是32倍数、保持近似宽高比、像素积刚好等于50176”的整数解。`min=max` 是一个强 pixel-budget 请求，不是 fixed `H'×W'`。

3.3 复杂度含义

视觉 token 数大致随处理后像素积增长；因此 pixel budget 同时影响：

- 细粒度 grounding 信息；
- `image_grid_thw` 与上下文长度；
- vision encoder 计算/显存；
- 文本可用 token budget与batch shape波动。

不能只说“分辨率越大越好”：还要说明目标物大小、宽高比、memory/latency 和训练评估一致性。

4. Patch、grid 与 image tokens

4.1 从处理图到 grid

对静态图像，Qwen processor 会为 temporal patch 做帧复制，最终通常得到 $T_g = 1$。空间 grid 为：

$$H_g = \frac{H'}{p}, \qquad W_g = \frac{W'}{p}.$$

`image_grid_thw=(T_g,H_g,W_g)` 不是原图尺寸；它描述 patchify 后的三维 grid。代码中 `pixel_values` 以 patch rows 组织，model 根据 grid 恢复位置与分组。

4.2 Spatial merge 后的 token 数

Qwen3 processor 展开语言侧 image placeholders 的公式是：

$$N_{\text{img}} = \frac{T_g H_g W_g}{m^2}.$$

其中 m^2 表示每个 `m×m` spatial block 合并为一个语言侧 image token。对静态图、$p = 16, m = 2$：

$$N_{\text{img}} = \frac{H'W'}{16^2 \cdot 2^2} = \frac{H'W'}{1024}.$$

例：$H' = 192, W' = 288$，则 $N_{\text{img}} = 192 \times 288 / 1024 = 54$。

实现映射：

```
image processor -> pixel_values + image_grid_thw
Qwen3VLProcessor -> prod(image_grid_thw) / merge_size**2
chat text -> repeated <|image_pad|> placeholders
model -> checks placeholder count matches image features
```

因此“图像 token 数”不能用原始分辨率或一个固定常数猜测，必须读取实际 `image_grid_thw`。

5. Pixel range、rescale 与 normalization

冻结的 `Qwen/Qwen3-VL-8B-Instruct@0c351d...` processor artifact 声明：

$$\mu = [0.5, 0.5, 0.5], \qquad \sigma = [0.5, 0.5, 0.5].$$

若输入 $x \in [0, 255]$，processor 先做 $x_s = x/255$，再：

$$x_{\text{norm}} = \frac{x_s - \mu}{\sigma} = 2\frac{x}{255} - 1.$$

变量映射：`do_rescale` 控制除255，`do_normalize` 控制减 mean/除 std。若上游已输出0–1，却仍让 processor 再除255，数值会被压到约 $[0, 1/255]$ 后再 normalize；这不是“小的数值差异”，而是 input contract 错误。

最小检查：

检查	正确问题
dtype/range	uint8 0–255，还是 float 0–1?
processor kwargs	`do_rescale/do_normalize/do_resize` 实际值是什么?
config	mean/std、patch、merge、pixel band 来自哪个 revision?
parity	train/eval 是否同 processor、同 kwargs、同 input range?

6. Direct resize、crop 与 letterbox 的坐标映射

PER-006 必须解释三种不同 transform；不能看到“输入尺寸一样”就认为坐标语义一样。

6.1 Direct resize

若原图 (W, H) 直接缩放到 (W', H')：

$$x' = s_x x, \quad y' = s_y y, \qquad s_x = \frac{W'}{W}, \quad s_y = \frac{H'}{H}.$$

逆映射：

$$x = \frac{x'}{s_x}, \qquad y = \frac{y'}{s_y}.$$

若 $s_x \neq s_y$，形状会发生各向异性变形。Qwen smart resize试图近似保持宽高比，但整数 rounding 仍可能让比例略变；因此逆映射应使用实际 s_x, s_y，不能只存一个 scale。

6.2 Letterbox

若使用统一 scale s 后在左右/上下 padding p_x, p_y：

$$x' = sx + p_x, \qquad y' = sy + p_y.$$

逆映射：

$$x = \frac{x' - p_x}{s}, \qquad y = \frac{y' - p_y}{s}.$$

不保存 padding offset 会让 point/box 系统性平移。当前冻结的 Qwen path没有 letterbox，因此不能人为补一个不存在的 padding inverse。

6.3 Crop + resize

若先从原图裁剪左上角 (c_x, c_y)，再缩放：

$$x' = s_x(x - c_x), \qquad y' = s_y(y - c_y).$$

逆映射必须恢复 crop origin：

$$x = \frac{x'}{s_x} + c_x, \qquad y = \frac{y'}{s_y} + c_y.$$

box 应分别变换两个角点，再 clip/validate；不能只变 center 或只变 width/height。无论哪种 transform，都要记录原图尺寸、处理图尺寸、scale、crop、pad 和坐标 convention。

7. OPD double-resize：如何讲风险，不讲伪因果

7.1 静态调用链

本地 OPD `expert_sft.py` 与 raw-holdout eval 的调用形态是：

```
message image + min_pixels=max_pixels=50176
  -> qwen_vl_utils.process_vision_info(messages)
     # omitted image_patch_size => default 14, factor 28
  -> processor(images=...)
     # omitted do_resize=False
     # Qwen3 artifact patch 16, merge 2 => factor 32
```

这是直接可见的 contract mismatch：第一阶段按 factor28，第二阶段按 factor32。它不自动等于“训练错了”或“指标下降”，因为还缺 matched preprocessing A/B。

7.2 Source-derived probe

原始 H×W	stage1 factor28	stage2 factor32	最终 tokens
`480×640`	`168×252`	`192×288`	54
`720×1280`	`168×280`	`192×320`	60
`1000×1000`	`224×224`	`224×224`	49

面试安全表述：

> 我通过冻结版本与确定性 probe 定位了 factor28→factor32 的双阶段 resize 风险，并提出 train/eval parity、grid manifest 与 coordinate round-trip gate；当前没有 matched A/B，所以不把它说成已确认的性能根因。

7.3 修复验证顺序

正确顺序不是“改一行直接报提升”，而是：

1. pin `qwen-vl-utils`、Transformers、model/processor config；

2. Qwen3 path 显式传 `image_patch_size=16`；
3. 二选一：utils resize + processor `do_resize=False`，或只让 processor resize；
4. 保存多个 aspect ratio 的 original/resized/grid/token；
5. 运行 point/box sentinel round-trip；
6. 固定训练/评估同一 policy；
7. 最后才做 matched metric A/B。

8. OpenPiVLA：三条 image lineage 不能混

路径	Image behavior	能证明	不能证明
formal MindCube	原始 RGB PIL 列表交给 `AutoProcessor`；默认 `min=max=200704`	VLM QA 分支存在 image processor path	DROID action 或 robot VLA 是 image-conditioned
formal DROID action	当前 sample 返回 `images=[]`，user text含 task/state/action JSON监督	action-text causal path	图像驱动 DROID action training
public-data dry-run adapter	`_read_image` 强制 `224×224`、除255、CHW；dry-run collator	smoke/dry-run tensor path	formal Qwen processor、真实 checkpoint 或 rollout lineage

同一原图的 formal MindCube source-derived probe：

原始 H×W	processed H×W（factor32，min=max200704）	tokens
`480×640`	`384×512`	192
`720×1280`	`320×576`	180
`1000×1000`	`448×448`	196

这些数字只说明 pixel/token budget 不同，不能比较 OPD 与 OpenPiVLA 模型效果；dataset、任务、checkpoint、loss 与运行 lineage 不同。

9. Camera identity 与 image order

9.1 Token不等于camera semantic

`<|image_pad|>` 只占据序列位置；它不会自动知道“这是wrist camera”还是“这是workspace camera”。多图输入至少要绑定：

```
sample_id / image_id
camera_id / frame_id
capture_timestamp
original_h,w / processed_h,w
transform revision / image_grid_thw
placeholder_index / image_index
valid / presence / uncertainty
```

formal collator 将batch内 image list 依次 flatten；如果message placeholder顺序、flatten顺序和camera metadata没有共同 manifest，即使shape正确也可能语义错位。

9.2 与 PER-003 的边界

PER-003 负责多视角/时序 fusion schema；PER-006 负责单张/多张图进入 processor 时的 identity/order。两者连接点是：

```
PER-003 per-view item
  -> PER-006 transform + grid + placeholder
  -> model tokens
  -> output tied back to the same camera/frame/time
```

若缺 `camera_id/frame/timestamp`，后续3D lift、common-frame fusion和controller mapping都无法可靠归因。

10. 最小可复现 manifest

类别	最小字段
provenance	dataset/sample/image ID、source path/hash、split、processor/model revision
camera	camera ID、optical frame、intrinsics/extrinsics revision、timestamp
raw image	original H/W、mode、dtype、range、orientation
transform	resize policy、requested/actual H/W、scale x/y、crop origin、pad offsets、interpolation
normalization	rescale flag/factor、normalize flag、mean/std
tokenization	patch/temporal/merge、`image_grid_thw`、image token count、placeholder index
inverse	point/box convention、processed→raw inverse、raw→camera/robot frame gate
validity	decode/resize/finite/range/order checks、failure reason

一个summary中写“image_size=224”远远不够：它没有说明是否square stretch、smart resize、crop还是letterbox，也没有说明坐标与camera。

11. 可执行审计伪代码

```
def audit_vlm_image(sample, processor, contract):
    # 1. Freeze provenance and camera identity before image decoding.
    rec = {
        "sample_id": sample.id,
        "image_id": sample.image_id,
        "camera_id": sample.camera_id,
        "frame_id": sample.frame_id,
        "timestamp": sample.timestamp,
        "processor_revision": contract.processor_revision,
    }

    # 2. Record raw image semantics; never infer them after transforms.
    image = decode_rgb(sample.image)
    rec["original_hw"] = image.shape[:2]
    rec["raw_dtype_range"] = inspect_dtype_range(image)

    # 3. Apply exactly one declared resize owner.
    if contract.resize_owner == "qwen_vl_utils":
        image, transform = qwen_resize(
            image,
            patch_size=contract.patch_size,   # Qwen3: 16
            min_pixels=contract.min_pixels,
            max_pixels=contract.max_pixels,
        )
        processor_kwargs = {"do_resize": False}
    elif contract.resize_owner == "processor":
        transform = {"kind": "processor_owned"}
        processor_kwargs = {}
    else:
        raise ValueError("ambiguous resize owner")
```

```
    # 4. Processor output must expose actual grid/token count.
    batch = processor(images=[image], return_tensors="pt", **processor_kwargs)
    grid = batch["image_grid_thw"][0]
    token_count = int(grid.prod()) // (contract.merge_size ** 2)
    rec.update({
        "transform": transform,
        "processed_grid_thw": grid.tolist(),
        "image_token_count": token_count,
        "pixel_values_shape": list(batch["pixel_values"].shape),
    })

    # 5. Round-trip a sentinel point/box and bind it to camera/frame.
    assert roundtrip_coordinate(sample.sentinel, transform, tolerance=contract.tol)
    assert rec["camera_id"] is not None and rec["frame_id"] is not None
    return rec
```

伪代码的核心是“resize owner唯一、actual grid可见、coordinate可逆、camera identity显式”。它不是本地已实现证明；只有保存真实manifest和round-trip日志后才能升级。

12. Failure taxonomy

失败层	症状	最小诊断
decode/color	RGB/BGR、EXIF orientation、alpha/background错误	raw hash、mode、orientation probe
transform	double resize、square stretch、误以为letterbox	original/actual H/W、owner、scale/pad/crop
numeric	0–1再次/255、mean/std错、dtype溢出	min/max/mean/std before/after
grid/token	patch/merge不匹配、placeholder数错	processor class/config、grid、token count
coordinate	point/box无法回原图、rounding偏移	sentinel round-trip、per-axis error
camera/order	wrist/workspace互换、batch flatten错序	camera ID、placeholder/image index
evidence	smoke path冒充formal、public机制冒充个人实现	lineage、checkpoint/config/run bundle

13. 方案比较与选择

选择	优点	风险	适用 gate
utils负责resize，processor不resize	message级pixel policy清晰、避免二次shape变化	必须传正确patch size并关闭processor resize	记录utils/processor版本与actual grid
processor独占resize	单一owner、配置随checkpoint	message级自定义尺寸需正确传递；本地checkpoint config必须pin	hash preprocessor config、train/eval parity
fixed square stretch	batch shape稳定、实现简单	破坏宽高比和grounding几何	只在任务接受且有matched验证时使用
dynamic aspect-preserving	保留几何、token按内容变化	batch显存与token数波动	pixel budget、bucket、OOM与latency统计
letterbox	保持宽高比且fixed canvas	必须保存pad/inverse，模型会看到padding	sentinel inverse + padding mask/ablation
crop	高分辨率局部细节	目标可能被裁掉、坐标需恢复crop origin	crop coverage、target visibility、inverse

没有通用最优解；面试中应根据grounding精度、目标大小、宽高比分布、memory/latency和坐标可逆性选择。

14. 项目映射与真实性边界

项目	可以安全连接	当前不能声称
OPD	Qwen3-VL processor调用、50176 pixel budget、factor28→32静态风险、point/box protocol	mismatch已修复、导致某个metric下降、修复带来uplift
OpenPiVLA	MindCube单次processor、DROID action无图像、dry-run square224三条lineage	formal DROID是image-conditioned VLA、dry-run等于formal
WAM/UWM	camera/schema/provenance字段可迁移到manifest设计	已使用同一Qwenprocessor或同一token contract
Robot Harness	工具输入可引用camera/frame/time schema	已运行视觉导航或验证image token mapping

安全回答遵循：

```
public upstream contract
  -> local static code evidence
  -> deterministic probe
  -> missing runtime/roundtrip/A-B evidence
  -> bounded conclusion
```

15. 面试回答检查表

回答 PER-006 至少覆盖：

1. 先画 raw image→processed image→grid→tokens 四层；
2. 给出 $F = pm$ 与 $N_{\text{img}} = T_g H_g W_g / m^2$；
3. 解释pixel budget不是fixed shape，`min=max`仍受factor和rounding影响；
4. 区分direct resize、crop、letterbox和各自inverse；
5. 说明pixel range/rescale/normalize与train/eval parity；
6. 说明camera ID/order不是token自动提供；
7. 用OPD double-resize和OpenPiVLA三条lineage做项目hook；
8. 明确静态风险不等于性能因果，补证需要manifest、round-trip和matched A/B。

16. 结论与下一步

PER-006 的核心不是背“Qwen3 patch size是16”，而是把图像输入变成一条可复算合同：谁resize、实际shape、pixel范围、grid/token数、坐标inverse、camera/order和lineage都必须显式。当前source、knowledge与card已闭合，但OPD double-resize性能影响、OpenPiVLA image-conditioned DROID、camera/round-trip manifest仍未闭合，因此保持`WARN/unseen`。

`VLA-PER-007..009` source/knowledge/card已同步且保持`unseen`。下一bounded slice只做PER-010 metric/parser source freeze，不生成knowledge/card或修改其他mastery。

References

1. PER-006 Image Preprocess / Token / Camera Source Freeze。

2. Vision Transformer v2。
3. Qwen3-VL frozen README 与 qwen-vl-utils vision_process.py。
4. Transformers 4.57.1 Qwen2VLImageProcessorFast 与 Qwen3VLProcessor。
5. Qwen3-VL-8B preprocessor artifact。
6. PER-001 Grounding / Coordinate 与 PER-003 Multi-view / Temporal。

Part 7 / 43 · `interview_materials/vla/knowledge/vol_06_per007_open_vocabulary_grounding_pipeline.html` · SHA-256 `F058B5E2AFB127554FD8F6574C03C70446FC7909D963A3C33270E8FF534D7541`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`f058b5e2afb127554fd8f6574c03c70446fc7909d963a3c33270e8ff534d7541`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per007_open_vocabulary_grounding_pipeline.md`

VOL 06 Perception / Grounding / Navigation：Open-vocabulary Grounding Pipeline（PER-007）

状态：`drafted / VLA-PER-007 / source_synced / card_synced / WARN` 报告日期：2026-07-19 证据截止：2026-07-19 来源入口：PER-007 Source Freeze。 前置知识：PER-006 Image Preprocess / Token / Camera、PER-001 Point / Box / Text Contract。 原子题卡：VLA-PER-007（`unseen`）。

问题（Question）

输入图像和目标文本后，open-vocabulary detector/grounder 如何从跨模态特征得到 region、score 和 phrase？`box_threshold`、`text_threshold`、NMS分别控制什么？为什么同样输出box，本地OPD生成式grounding不能被口述成Grounding DINO式显式detector？

本页回答：

1. Grounding DINO的feature enhancer、language-guided query selection与cross-modality decoder如何组成pipeline？
2. `Q×L` query–token logits的两个轴分别是什么？
3. query gate与phrase-token gate为什么是两个不同阈值？
4. phrase如何从token positions重建，prompt标点/顺序/截断为什么会改变结果？
5. frozen helper没有NMS意味着什么、不意味着什么？
6. 如何设计typed manifest、伪代码、failure taxonomy与项目真实性边界？

知识地图位置与边界

```
PER-006 image/pixel/token/camera contract
  -> PER-007 open-vocabulary detector/grounder pipeline
  -> PER-008 output representation choice
  -> PER-009 training matching/loss
  -> PER-010 metric/parser comparability
  -> PER-015 perception-to-action gate

PER-001 point/box/text + coordinate contract
  <-> PER-007 phrase/region binding + threshold/NMS contract
```

PER-007只负责inference pipeline和可观测接口：prompt如何进入、query如何与text token匹配、哪些query/phrase被筛选、box如何后处理。训练loss属于PER-009，最终metric/scorer属于PER-010，point/box/mask/text表示选择属于PER-008。

1. 30秒直觉：不是“文本分类+框”

显式open-vocabulary grounding的中间对象不是一个class ID，而是一组object queries与文本tokens之间的score matrix。每个query同时有一个normalized box；先用query的最大token score筛掉低置信region，再用另一个阈值决定该region绑定哪些tokens并重建phrase。最后才做坐标转换和可选NMS。

因此一个完整回答至少说清四件事：

1. prompt/token contract；
2. `query×token` score semantics；
3. box gate与phrase gate；
4. geometry/NMS/output manifest。

只说“模型输入文字，输出框”无法解释空phrase、复合phrase、重复框、阈值变化、token截断或坐标错位。

2. Pipeline总览

Grounding DINO论文把language–vision fusion概念上分成三个阶段：

阶段	直觉	专业表述	主要可观测量
Feature enhancer	先让图像和文本上下文互相增强	multi-scale visual features与encoded text进行cross-modal feature enhancement	visual/text feature shape、mask、revision
Language-guided query selection	从图像候选中选与文本更相关的queries	text-conditioned encoder proposals初始化object queries	proposal/query index、initial reference
Cross-modality decoder	query在解码时继续读取图像和文本	decoder updates query features并预测query-token logits与boxes	decoder query、`pred_logits`、`pred_boxes`

冻结的数据流可画为：

```
raw image + raw prompt
  -> PER-006 image transform + normalized prompt/tokenization
  -> visual features + encoded text
  -> feature enhancer
  -> language-guided query selection
  -> cross-modality decoder
  -> pred_logits [B,Q,L_max] + pred_boxes [B,Q,4]
  -> sigmoid + query threshold
  -> token threshold + phrase decode
  -> cxcywh-to-xyxy + PER-006 inverse
  -> optional explicit NMS
  -> typed grounded-region manifest
```

这里`B`是batch size，`Q`是object query数，`L_max`是文本token轴上限。`L_max=256`不表示“最多256个类别”。

3. Prompt与token contract

3.1 Raw prompt不等于model prompt

冻结GroundingDINO helper会：

1. `lower()`；
2. `strip()`；
3. 若末尾没有句点则补`.`。

Transformers 4.57.1 processor对candidate-label list执行小写并用“`.`”连接，最后补句点。例如：

```
["red mug", "metal spoon"]
  -> "red mug. metal spoon."
```

这些不是无关紧要的字符串清洗。句点是phrase boundary的一部分；label顺序、同义词、复合表达、标点和tokenizer都会改变token positions，进而改变query–token score与phrase decode。

3.2 256是token-axis上限

冻结model对text tensors截断到`max_text_len=256`。需要保存：

字段	为什么需要
`prompt_raw`	还原用户/任务真实输入
`prompt_normalized`	复算实际caption
`prompt_hash`	防止实验之间prompt悄然变化
tokenizer/config revision	解释subword与special token变化
`input_ids/attention_mask`	复算phrase token positions
truncation flag / original token length	识别后半labels静默丢失

长label list的风险不是“类别超过256”，而是tokenization后超过256 positions；一个多词label会占多个subword tokens。

4. Query–token matching

4.1 Tensor contract

冻结输出：

Tensor	Shape	语义
query feature `Z`	`[B,Q,D]`	decoder中每个region query的embedding
text feature `E`	`[B,L,D]`	每个有效text token的embedding
score/logit `S`	`[B,Q,L_max]`	每个query对每个token的匹配logit
box `C`	`[B,Q,4]`	normalized `cxcywh`

`ContrastiveEmbed`对query $z_q \in R^D$ 与token $e_t \in R^D$ 做点积：

$$s_{q,t} = z_q^\top e_t.$$

变量解释：q是object query，t是文本token，D是embedding dimension。直觉是每个region query对每个token分别打分；代码映射是`x @ y.transpose(-1,-2)`。

padding token被mask为`-inf`，随后score tensor padding到`L_max=256`。这防止padding位置被phrase gate选中。

4.2 Logit、sigmoid与calibration

官方inference使用：

$$p_{q,t} = \sigma(s_{q,t}) = \frac{1}{1 + \exp(-s_{q,t})}.$$

$p_{q,t}$是同一frozen模型/prompt下的token-wise sigmoid值。它没有因“看起来在0到1之间”就自动成为跨prompt、跨checkpoint或跨类别可比较的calibrated probability。

实现与表述边界：

- 可以说：frozen helper用sigmoid后的最大token score筛query；
- 不能直接说：0.7在所有prompts上都代表70%正确率；
- 若要比较threshold，必须固定model、processor、prompt normalization、image transform和evaluation set。

5. 双阈值：region gate与phrase gate

5.1 Query / box gate

每个query的region score定义为：

$$r_q = \max_t p_{q,t}.$$

保留规则：

$$\text{keep}(q) = \mathbb{1}[r_q > \tau_{\text{box}}].$$

τ_box控制哪些queries进入结果集。代码使用严格大于号；score等于threshold时会被丢弃。

5.2 Phrase-token gate

对已保留query：

$$M_{q,t} = \mathbb{1}[p_{q,t} > \tau_{\text{text}}].$$

$M_{q,t}$决定哪些token IDs送入phrase decoder。τ_text主要改变phrase组成，不一定改变region数量。

5.3 两个阈值不能合并

调整	直接影响	不一定直接影响
提高 `τ_box`	保留query数减少，可能提高precision/降低recall	已保留query的phrase token集合
降低 `τ_box`	更多regions进入，可能带来冗余/false positives	phrase是否完整
提高 `τ_text`	phrase更短、可能变空或碎片化	region是否保留
降低 `τ_text`	phrase可能包含更多修饰词/相邻tokens	region数量

一个重要edge case：若 `r_q>τ_box`，但所有token都不满足 `p_{q,t}>τ_text`，box仍被保留而phrase为空。manifest必须支持 `invalid_reason="empty_phrase"`，不能凭最近label强行补值。

6. Deterministic threshold lab

人工probability matrix：

Query	token probabilities	`τ_box=0.35`	`τ_text=0.25`
`q0`	`[0.90,0.20,0.81,0.01]`	keep	`t0,t2`
`q1`	`[0.34,0.33,0.01,0.01]`	drop	不进入phrase gate
`q2`	`[0.60,0.24,0.26,0.01]`	keep	`t0,t2`

改变一个变量：

1. `τ_text:0.25→0.30`：`q0/q2`仍保留，`q2` phrase由`t0,t2`变成`t0`；
2. `τ_box:0.35→0.65`：只保留`q0`；
3. query max恰为0.35：严格`>`判断为drop。

这只验证逻辑，不代表真实模型分布。面试中不要把synthetic probe说成calibration experiment。

7. Phrase reconstruction

7.1 从token mask到phrase

冻结路径用`input_ids`和boolean token mask恢复phrase。需要区分：

- token index；
- tokenizer subword；
- decoded span；
- normalized label；
- canonical downstream category。

同一个label可能被拆成多个subwords；一个query也可能对多个相邻/不相邻tokens高分。直接对最终字符串做exact match会丢失这些诊断信息。

7.2 标点与复合phrase

句点既是prompt规范化结果，也是candidate label boundary。冻结official helper的`remove_combined`分支会借助`[CLS]/[SEP]/.` token positions把phrase限制到对应区间；默认分支则从所有positive tokens恢复phrase。

因此manifest应同时保存`phrase_token_indices`和`phrase_text`。只保存decoded string无法判断错误来自matching还是decode。

7.3 Transformers API drift

Transformers 4.57.1的`GroundingDinoProcessor.post_process_grounded_object_detection`：

1. 从`outputs.input_ids`或显式`input_ids`恢复phrase；
2. `text_labels` argument在frozen code中存在但不参与绑定；
3. result同时返回`text_labels`和`labels`，当前两者都是decoded strings；
4. code包含deprecation/migration warning，未来`labels`可能变成integer IDs。

工程上必须为output key和value type做golden test，不能跨版本只靠字段名。

8. Box geometry与NMS boundary

8.1 Normalized `cxcywh` 到pixel `xyxy`

对target image width W、height H：

$$x_1 = W(c_x - w/2), \quad y_1 = H(c_y - h/2),$$

$$x_2 = W(c_x + w/2), \quad y_2 = H(c_y + h/2).$$

变量解释：(c_x, c_y, w, h)是normalized center/size；(x_1, y_1, x_2, y_2)是pixel corners。代码映射是先乘`[W,H,W,H]`，再`cxcywh→xyxy`。`target_sizes`在Transformers中按`(height,width)`传入。

该公式只恢复到指定target image space。若模型看到的是processed image，还要调用PER-006记录的crop/pad/scale inverse回到raw image。

8.2 Frozen helper没有NMS

冻结`inference.py`做threshold、phrase extraction和box conversion，没有NMS call。这意味着：

- 不能把`box_threshold`叫作NMS threshold；
- 不能假设输出已经做class-wise或class-agnostic suppression；

- 也不能反推模型一定产生重复框。

如果部署增加NMS，它是一个新的pipeline stage，必须保存：

字段	原因
pre-NMS indices/boxes/scores/phrases	复算被删候选
IoU threshold	控制suppression强度
phrase grouping policy	“red mug”和“mug”是否互相suppression
class-agnostic flag	决定跨phrase竞争
sort/tie-break/max detections	保证确定性
post-NMS selected indices	绑定后续region/action
coordinate space	保证IoU在同一空间计算

NMS可以缓解重复region，但可能错误删除相邻同类物体；是否需要、如何分组必须由任务和matched evaluation决定。

9. Typed grounded-region manifest

推荐把每个region记录为：

层	必需字段
sample/image	`sample_id,image_id,camera_id,frame_id,timestamp`
prompt	`prompt_raw,prompt_normalized,prompt_hash,input_ids`
revision	`model,checkpoint,processor,tokenizer,code` hashes
raw output	`query_index,query_token_logits,pred_box_cxcywh_norm`
threshold	`score,score_semantics,box_threshold,text_threshold`
phrase	`phrase_token_indices,phrase_text,canonical_label`
geometry	processed/raw `xyxy`、transform revision
NMS	policy revision、pre/post indices
validity	`valid,invalid_reason,ambiguous`

`score_semantics`必须显式写`max_token_sigmoid`；否则它可能被误解成class softmax、cosine similarity、sequence likelihood或calibrated confidence。

10. 可审计伪代码

```
def ground_one(sample, candidate_labels, cfg):
    # 1) 继承PER-006图像/camera contract, 不能丢original->processed变换
    image, image_meta = preprocess_image(sample.image, cfg.image_processor)

    # 2) 保存raw与normalized prompt; 句点、顺序和tokenizer都属于合同
    prompt_raw = candidate_labels
    prompt = normalize_candidate_labels(candidate_labels)
    inputs = processor(image=image, text=prompt, return_tensors=True)

    # 3) 保存raw QxL logits与normalized cxcywh, 便于重放threshold
    outputs = model(**inputs)
```

```
    probs = sigmoid(outputs.pred_logits)
    boxes = outputs.pred_boxes

    pre_nms = []
    for q in range(boxes.shape[1]):
        score = probs[0, q].max()

        # 4) query/box gate使用严格大于号
        if not (score > cfg.box_threshold):
            continue

        # 5) phrase gate独立于query gate; 允许empty phrase并标无效原因
        token_idx = where(probs[0, q] > cfg.text_threshold)
        phrase = decode_phrase(inputs.input_ids[0], token_idx)
        invalid_reason = "empty_phrase" if len(token_idx) == 0 else None

        # 6) box先到processed xyxy, 再按PER-006 manifest逆变换到raw image
        xyxy_processed = cxcywh_norm_to_xyxy(boxes[0, q], image_meta.processed_size)
        xyxy_raw = inverse_transform(xyxy_processed, image_meta.transform)

        pre_nms.append(make_region_record(
            query_index=q,
            score=score,
            phrase_token_indices=token_idx,
            phrase_text=phrase,
            box_processed=xyxy_processed,
            box_raw=xyxy_raw,
            invalid_reason=invalid_reason,
        ))

    # 7) frozen official helper没有NMS; 只有显式配置时才运行并记录selected indices
    regions = apply_nms_with_trace(pre_nms, cfg.nms) if cfg.nms.enabled else pre_nms

    # 8) 输出raw revision/threshold/geometry/NMS manifest, 不只返回boxes
    return write_grounding_manifest(sample, inputs, outputs, regions, cfg, image_meta)
```

伪代码的关键不是复刻某个库API，而是保留可重放的raw outputs、两个threshold、phrase tokens、坐标inverse和NMS trace。

11. 显式detector与生成式grounding

维度	Grounding DINO式显式detector	Qwen/OPD式生成式grounding
核心输出	`Q×L` logits + `Q×4` boxes	autoregressive text/token sequence
region候选	object queries显式存在	坐标作为生成内容出现
phrase绑定	token score mask + decode	prompt/answer grammar中的tag/text
置信度	max-token sigmoid等显式score	通常需token likelihood或另建scorer，不能直接等价
threshold	box/text两个后处理阈值	generation config、parser、format/geometry gate
NMS	frozen helper无；可由下游显式添加	通常无region set NMS，除非另建解析后候选集
主要debug artifact	logits、query/token heatmap、pre/post threshold boxes	generated text、token logprobs、parser trace、tag/coordinate validity

“open-vocabulary”描述任务能力，不要求所有系统都采用同一种网络。生成式VLM可以完成referring/coordinate输出，但不能借用显式detector的score、threshold和NMS语义。

12. 本地OPD项目边界

冻结本地证据链：

```
annotation/data pool
  -> clip/filter box + normalize to 0..1000
  -> construct tagged <box>/<point>/text SFT rows
  -> expert mix + route metadata
```

```
    -> Qwen3-VL causal SFT
    -> model.generate
    -> tag/coordinate parser
    -> IoU / distance / text scorer
```

当前四个直接文件和repo静态搜索支持：

- 有生成式point/box/text数据、训练和评估路径；
- 有prompt、tag、`0..1000`坐标、parser/scorer与route metadata；
- 没有在可搜索源码中发现显式GroundingDINO/NMS调用。

当前不能声称：

- OPD部署了GroundingDINO、region proposal或query–token detector；
- route metadata就是learned detector/router效果；
- 生成式IoU/point/static text指标等于机器人执行成功；
- absence search证明远端/历史/未归档环境绝对没有detector；
- Git metadata或本次审计自动证明候选人个人贡献。

面试安全表达：

> 公源上我能解释并审计Grounding DINO式显式query–token pipeline；本地OPD证据对应生成式tagged-coordinate pipeline。我会把两者按output tensor、threshold、parser和debug artifact分开，不把公开detector机制归到项目，也不从静态grounding跳到机器人成功。

13. Failure taxonomy

层	症状	常见根因	首个检查
prompt	同义词/顺序变化导致输出漂移	normalization、label boundary、tokenization	raw/normalized prompt + input IDs
truncation	后半labels完全不响应	超过`max_text_len`	original/effective token length
matching	region绑定错误token	feature/query/text alignment	query-token heatmap + token mask
query gate	漏检或候选过多	`τ_box`不匹配分布	pre-threshold max-score histogram
phrase gate	空phrase/碎片/复合词	`τ_text`或token boundary	positive token indices + decode
geometry	框整体偏移/拉伸	target size、cxcywh/xyxy、inverse	processed/raw round-trip
duplicate	相邻/重复regions不稳定	无NMS或NMS grouping错误	pre/post NMS trace
API	label key/type突变	Transformers版本迁移	golden output schema test
project	把生成式路径说成detector	pipeline类型与证据混淆	model output contract + local imports

诊断顺序遵循“先合同、后模型”：prompt/token → raw logits/boxes → two-threshold → phrase decode → geometry → NMS → metric/action。

14. Threshold与evaluation设计

PER-007只规定如何保留raw evidence，最终metric协议由PER-010展开。最小matched sweep应：

1. 固定model/checkpoint、processor、prompt、image transform、split与seed；
2. 缓存raw logits/boxes/input IDs，不为每个threshold重跑模型；
3. 对`τ_box`与`τ_text`做二维grid，但分开观察region count和phrase integrity；
4. NMS off/on作为独立factor，固定IoU/grouping policy；
5. 保存per-sample prediction，而不是只报aggregate；
6. 明确空phrase、no-object、ambiguous和invalid boxes的分母处理；
7. 不从offline detector结果推断controller/robot success。

若同时改变prompt wording、threshold与NMS，不能把结果变化归因给其中一个因素。

15. 实现与面试检查表

回答PER-007至少覆盖：

1. 画image/text→features→queries→`Q×L` logits+boxes→双阈值→phrase/geometry/NMS；
2. 说明`L_max=256`是token positions，不是classes；
3. 写出query–token dot product和sigmoid；
4. 分开`τ_box`与`τ_text`；
5. 说明frozen helper无NMS，但不保证无重复框；
6. 保存raw logits、input IDs、query index、phrase tokens、processed/raw boxes与revision；
7. 解释Transformers`labels/text_labels`版本风险；
8. 对比显式detector与OPD生成式grounding；
9. 明确offline grounding不等于action/robot success。

常见危险答案：

- “每个query只对应一个class”；
- “256就是最多256类”；
- “box threshold就是NMS threshold”；
- “text threshold只影响置信度，不影响phrase”；
- “HF的text_labels参数直接决定输出label”；
- “Grounding DINO官方默认已经做NMS”；
- “OPD有box输出，所以项目用了Grounding DINO”。

16. 结论与下一步

PER-007的核心是把open-vocabulary grounding从一句“文本找框”展开为可复算pipeline：prompt/token、query–token logits、query gate、phrase gate、geometry、optional NMS和typed manifest必须逐层显式。当前source/knowledge/card已闭合且题卡保持`unseen`；本地detector checkpoint/raw logits/threshold calibration/NMS trace和closed-loop evidence仍缺，因此保持`WARN`。

`VLA-PER-008..009` source/knowledge/card已同步且保持`unseen`。下一bounded slice只做PER-010 metric/parser source freeze，不生成knowledge/card，不创建`coaching_state.md`。

References

1. PER-007 Open-vocabulary Grounding Pipeline Source Freeze。
2. Grounding DINO v5。
3. IDEA-Research/GroundingDINO@856dde20。
4. Transformers 4.57.1 Grounding DINO。
5. PER-006 Image Preprocess / Token / Camera。
6. PER-001 Point / Box / Text Grounding。
7. OPD Claim–Evidence Inventory。

Part 8 / 43 · `interview_materials/vla/knowledge/vol_06_per008_representation_choice.html` · SHA-256 `5ACBDCDEE6C46476C8847F6DF31DB09891C1B2EAB3B3E3033053325830ED7DAE`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`5acbdcdee6c46476c8847f6df31db09891c1b2eab3b3e3033053325830ed7dae`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per008_representation_choice.md`

VOL 06 Perception / Grounding / Navigation：Point / Box / Mask / Keypoint / Affordance 表示选择（PER-008）

状态：`drafted / VLA-PER-008 / source_synced / card_synced / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：PER-008 Source Freeze。 前置知识：PER-001 Point / Box / Text Contract、PER-006 Image Preprocess / Token / Camera、PER-007 Open-vocabulary Grounding。 原子题卡：VLA-PER-008（`unseen`）。

问题（Question）

机器人收到“把杯子挂到架子上”“在空位放下物体”或“拉开抽屉”后，perception 应输出 point、box、mask、semantic keypoint 还是 action-conditioned affordance？更密集的输出是否一定更好？本地数据里有一个叫`affordance`的字段，是否就能说系统学习了可执行动作？

本页给出的答案不是五类表示的排行榜，而是一条可审计选择链：

```
task + downstream consumer + image/camera/time contract
  -> consumer真正读取的最小字段
  -> minimum-sufficient representation
  -> uncertainty / multimodality / validity
  -> 有损转换及其provenance
  -> parser / metric / controller gate
  -> evidence-bounded project claim
```

知识地图位置与边界

```
PER-001 point/box/text schema + coordinate/parser basics
  -> PER-006 image/camera/time lineage
  -> PER-007 text-conditioned region proposal
  -> PER-008 representation choice and conversion boundary
  -> PER-009 matching/loss
  -> PER-010 metric/protocol
  -> PER-012 concrete segmentation/keypoint/affordance models
  -> PER-015 perception-to-action gate

PER-002 owns 2D-to-3D lift and frame transforms.
```

PER-008只回答“下游需要什么信息、应选择什么输出合同、转换会丢什么”。它不重讲投影、训练loss、最终metric、具体模型recipe或controller feasibility。这样可以避免把“选了affordance表示”偷换成“已经闭环执行成功”。

1. 30秒直觉：表示不是越密越好

最短可用回答是：

> 我先看downstream consumer需要哪些字段。只要ROI或粗tracking，box通常够；要轮廓、占用或精细重叠才需要mask；只问一个或多个候选位置可以用point set；类别级几何角色需要带semantic ID与frame的keypoints；只有当primitive、tool、state、方向和success semantics进入合同后，才能叫action-conditioned affordance。密度不等于语义，也不等于可执行性。

五个容易混淆的反例：

1. 一个pixel坐标不知道自己是`handle`还是`rim`；
2. 一个box不知道杯柄轮廓或中间空洞；
3. 一个SAM mask不知道“这是要拉的把手”；
4. 一个名为`affordance_pointing`的point target不包含push/pull方向；
5. 一个actionability score也不自动等于特定机器人上的已校准成功率。

2. 先从downstream consumer反推字段

2.1 “最小充分”不是“最简单”

表示应当同时满足两个条件：

- 充分：consumer所需的extent、contour、semantic role、frame或action condition都存在；
- 最小：不为未被消费的dense字段支付标注、存储、推理、同步与评估成本。

一个可审计的工程目标可写为：

$$r^* = \arg\min_{r \in \mathcal{R}} \left(C_{\text{label}}(r) + C_{\text{infer}}(r) + \lambda\, \mathbb{E}[L_{\text{down}} \mid r] + \mu\, C_{\text{convert}}(r) + \nu\, C_{\text{stale}}(r) \right).$$

变量解释：r是候选representation；C_{label}和C_{infer}分别是标注与推理成本；L_{down}是consumer上的任务损失；C_{convert}记录格式/坐标转换及信息损失；C_{stale}表示输出随时间陈旧的风险；λ, μ, ν是部署权重。

直觉：更复杂的表示只有在降低downstream failure足以覆盖额外成本时才值得选。实现映射：这些量来自labeling ledger、latency/memory log、conversion manifest、timestamp trace与matched downstream ablation。证据边界：这是从冻结source合同推导出的decision rubric，不是RoboPoint、SAM、kPAM或Where2Act共同提出的训练目标。

2.2 Required-fields矩阵

Downstream consumer	真正读取的字段	起始表示	为什么不是别的表示
proposal / ROI crop / 粗tracking	instance extent、phrase/ID、score	box	point无extent；mask可能是无必要的dense成本

Downstream consumer	真正读取的字段	起始表示	为什么不是别的表示
look-at / free-space / 粗contact候选	一个或多个位置、valid radius、score	point set	box/mask提供的形状若不被使用就是冗余
contour / overlap / occupancy预处理	pixel membership、instance、quality	mask	box把背景、空洞与细长结构混入
类别级几何约束	semantic role、visibility、2D/3D frame、covariance	semantic keypoints	裸point没有role；mask未必提供可迁移的几何锚点
primitive-conditioned interaction	support geometry、primitive、tool、state、direction、horizon、success semantics	affordance field/points	任何纯几何表示都缺action condition

这张表只给default starting contract。若ROI consumer后来需要碰撞边界，box可能升级为mask；若mask最后只被压成中心点，反而应审计是否能直接训练point output。

3. 五类表示的严格合同

3.1 Point set：回答“哪里”，不回答“是什么角色”

一个最小point set不应只存坐标，而应至少冻结：

```
points / coordinate_space / count policy / ordering
point_scores + score_semantics / valid_radius
multimodal_group / empty-set policy / image-camera-time lineage
```

它适合低带宽位置建议，也天然允许输出多个mode。RoboPoint的公开README以多个`[(x,y),...]` normalized image points作为输出合同；本地OPD主要使用`0..1000` tagged point。两者的坐标space、parser与metric不同，不能只因都长得像二元组就混用。

point的主要信息缺口是extent、surface normal、semantic role、action precondition与reachability。单点“落在目标上”也不等于末端执行器能以安全姿态到达。

3.2 Box：回答“粗范围”，不回答“精细形状”

box需要声明`xyxy`或`cxcywh`、pixel或normalized、corner inclusive/exclusive、clip规则、instance/phrase binding、score与NMS revision。它适合proposal、crop与association，因为四个数就能表示粗范围。

代价是矩形会包含背景、孔洞与其他实例；对细长、旋转或部分遮挡物体尤其不紧致。box score、mask quality与affordance success是三种不同语义，不能放在同一阈值表里直接比较。

3.3 Mask：回答“哪些像素属于region”，不回答“应该怎么做”

binary或soft mask需要保存shape、encoding、threshold、component/instance、quality semantics、source prompt和transform lineage。SAM的`SamPredictor`公开合同支持point/box/mask prompt，并返回原图尺寸的一个或多个masks、predicted mask quality与低分辨率logits。

这证明promptable segmentation I/O，不证明mask自带language label、instance permanence、depth、contact normal或robot actionability。mask适合轮廓、遮挡、surface/occupancy预处理，但会引入dense storage、边界噪声、component ambiguity与temporal flicker。

3.4 Semantic keypoints：回答“哪个几何角色在哪里”

一个可用keypoint set可写为：

$$K = \{(s_i, p_i, v_i, \Sigma_i)\}_{i=1}^{N}.$$

变量解释：s_i是semantic ID，如`handle_center`或`rim_left`；p_i是2D或3D坐标；v_i是visibility/validity；Σ_i是位置不确定性；N是定义或检测到的anchor数。

直觉：坐标相同而s_i不同，controller中的几何constraint就可能完全不同。实现映射：schema要把`semantic_ids, coords, visibility, covariance, instance_id, category, frame`绑定在同一record。证据边界：kPAM支持semantic 3D keypoints进入几何cost/constraint与planning factorization；本地OPD没有由所读文件证明这条pipeline。

3.5 Action-conditioned affordance：回答“在什么条件下哪里可如何交互”

affordance不应只写成一个heatmap名称。抽象合同可记为：

$$A(p \mid \pi, g, s, d, h).$$

变量解释：p是候选位置或support geometry；π是action primitive，如push/pull；g是tool或gripper；s是object/environment state；d是方向或orientation；h是时间horizon；A的数值必须另行声明是ranking score、likelihood还是校准后的成功概率。

直觉：同一pixel对push和pull、不同gripper、门开与关、不同碰撞上下文可以有不同可行性。实现映射：manifest必须显式存`primitive, tool_or_gripper, state_precondition, direction_or_orientation, horizon, success_score_semantics`。Where2Act支持按action primitive预测per-point actionability、orientation proposal与critic success likelihood；它不支持把分数迁移为任意机器人上的通用成功率。

4. Point、Keypoint与Affordance为何不能按坐标形式合并

4.1 三层语义阶梯

```
point:       xy/xyz + coordinate contract
keypoint:    point + semantic ID + visibility + frame/covariance
affordance:  support geometry + primitive/tool/state/direction/horizon + score semantics
```

这不是“point < keypoint < affordance”的通用性能排名，而是字段合同逐步增加。一个affordance可以最终以top-k points输出，但每个point仍必须携带action condition；一个semantic keypoint也可以是2D point，但semantic ID不能丢。

4.2 两个确定性反例

相同坐标	Record A	Record B	裸坐标丢掉什么
`(200,300)`	`mug_handle`	`mug_rim`	semantic role与不同几何constraint
`(500,500)`	`push / +x`	`pull / -x`	primitive、direction与不同执行假设

因此“模型输出了point”只能证明位置字段；是否是keypoint或affordance，必须看坐标外的typed fields、监督目标、head、parser与下游消费方式。

5. Box与Mask的信息量、成本与不可逆性

5.1 Mask到box可计算但有损

对非空binary mask M，half-open axis-aligned box为：

$$B(M) = \left(\min_{M(x,y)=1} x,\ \min_{M(x,y)=1} y,\ \max_{M(x,y)=1} x+1,\ \max_{M(x,y)=1} y+1 \right).$$

变量解释：$M(x,y)=1$表示foreground membership；$B(M)$只保存四个坐标极值。直觉：contour、hole、component与area都会丢失。实现映射：可用`where(mask)`求min/max并在conversion manifest中记录`mask_to_xyxy_half_open`；这不是可逆codec。

确定性5×5 probe：3×3实心mask和只保留四角的mask都得到`(1,1,4,4)`，面积却分别为9和4。由此可知`box -> filled rectangle`是新增假设，不是恢复原mask。

5.2 什么时候box比mask更好

如果consumer只做crop、proposal ranking或粗association，mask的额外像素可能没有收益，反而增加label、memory、latency与temporal instability。只有consumer真的读取contour、overlap、thin structure、hole或occupancy时，mask的额外信息才成为必要字段。

5.3 什么时候两者都不够

box和mask都主要描述视觉region。它们没有自动提供3D frame、surface normal、force closure、action direction或reachability。因此“用了mask就更适合抓取”仍需depth/geometry与PER-015 action gate证据。

6. 转换图：可计算不代表等价

转换	操作	信息损失或新增假设	必需provenance
mask → box	foreground极值	contour、hole、component、area	mask hash、threshold、corner convention
keypoints → box	visible points极值	object extent未知，可能只包局部anchors	semantic IDs、visibility filter
affordance field → top-k points	score排序与空间抑制	dense landscape、校准与被截mode	primitive、score semantics、k、spacing
box → mask	segmentation model或矩形填充	新prediction或强矩形假设	model/prompt revision或fill policy

转换	操作	信息损失或新增假设	必需provenance
point → keypoint	不能只靠坐标	必须新增semantic identity	ontology/relabel source
point → affordance	不能只靠坐标	必须新增action conditions	primitive/tool/state/direction source

每次转换都应生成新artifact stage，而不是覆盖原输出。最小记录包括input hash、converter revision、parameters、output hash、declared loss与validity。

7. Uncertainty、Multimodality与Validity

7.1 表示不同，不确定性也不同

表示	至少保留的不确定性	常见错误压缩
point set	per-point score、mode group、valid radius、empty set	只取均值点，落到所有mode之外
box	proposal score、coordinate uncertainty、duplicate grouping	NMS后只留一个框且不保留pre-NMS trace
mask	multimask candidates、quality semantics、soft logits、components	强制二值化并删掉替代mask
keypoints	visibility、covariance、missing/occluded state	把遮挡点填0并当成真实坐标
affordance	score semantics、orientation proposals、state/primitive validity	把ranking score口述成成功概率

RoboPoint的多个points与SAM的multimask都提醒：歧义是输出合同的一部分。强行压成单点或单mask会把模型不确定性变成下游的隐式错误。

7.2 Fail-closed validity

一个representation record至少区分：

- `valid=true`：schema、坐标、lineage与必需语义字段完整；
- `valid=false`：记录明确`invalid_reason`，禁止controller静默消费；
- empty but valid：模型明确预测“无候选”，不等于parser失败；
- missing/unverified：artifact或字段没有被归档，不能填默认值冒充证据。

8. Typed Representation Manifest

所有类型先继承PER-006的image/camera/time lineage：

```
{
  "sample_id": "...",
  "image_id": "...",
  "camera_id": "...",
  "frame_id": "...",
  "timestamp": 0.0,
  "representation_type": "point_set | box | mask | semantic_keypoints | affordance",
  "coordinate_space": "original_pixel | processed_pixel | normalized_0_1 | normalized_0_1000 | world",
  "transform_revision": "...",
  "prompt_hash": "...",
  "model_processor_code_revision": "...",
  "score": 0.0,
  "score_semantics": "...",
  "uncertainty": {},
  "valid": true,
  "invalid_reason": null,
```

```
  "payload": {}
}
```

类型特有payload：

类型	最小字段
point set	`points, point_scores, valid_radius, ordering, multimodal_group`
box	`format, coords, phrase, instance_id, clipped, nms_revision`
mask	`shape, encoding, threshold, component_ids, quality_semantics, source_prompt`
semantic keypoints	`semantic_ids, coords, visibility, covariance, instance_id, category, frame`
affordance	`support_geometry, primitive, tool_or_gripper, state_precondition, direction_or_orientation, horizon, success_score_semantics`

只保存`payload`会让相同的两个数字在pixel、normalized、semantic anchor与action point之间静默漂移。只保存一个`score`又会混淆objectness、mask quality、point hit与action success。

9. 可审计选择算法

```
def choose_representation(task, consumer, evidence):
    # 1. Freeze fields that the downstream consumer actually reads.
    need = consumer.required_fields(task)

    # 2. Select the minimum representation whose native contract is sufficient.
    if need.action_primitive or need.contact_orientation:
        rep = "affordance"
    elif need.semantic_anchor or need.geometric_constraint:
        rep = "semantic_keypoints"
    elif need.contour or need.pixel_occupancy:
        rep = "mask"
    elif need.extent or need.roi:
        rep = "box"
    else:
        rep = "point_set"

    # 3. Fail closed if local schema/parser/evaluator cannot consume it.
    required = ["schema", "parser", "lineage", "validity"]
    if not all(evidence.has(rep, item) for item in required):
        return invalid("representation_not_supported")

    # 4. Preserve raw multimodal output and every conversion stage.
    record = build_typed_manifest(rep, task, consumer, evidence)
    record["conversions"] = freeze_conversion_chain(evidence.conversions)

    # 5. Action consumption remains gated by PER-015 evidence.
    record["controller_ready"] = False
    return record
```

这段伪代码是工程gate，不是某个official API。它要求先冻结consumer fields，再检查本地schema/parser/lineage，而不是看到一个模型能输出mask就默认升级整个系统。`controller_ready=false`也提醒表示选择与可执行性是两个节点。

10. RoboPoint、SAM、kPAM与Where2Act接口对照

方法	输入条件	原生输出合同	能支持的口述	不能偷换成
Grounding DINO（PER-007）	image + text tokens	query-token scores + boxes + phrase	open-vocabulary proposal/ROI	mask、semantic keypoint或action feasibility

方法	输入条件	原生输出合同	能支持的口述	不能偷换成
RoboPoint	image + language instruction	multiple normalized image points	spatial relation/free-space/action-point suggestion	3D semantic keypoints或已执行contact
SAM	image + point/box/mask prompt	one/multiple masks + quality + low-res logits	promptable region/contour extraction	language semantics或robot affordance
kPAM	segmented object + semantic keypoint detector	semantic 3D keypoints进入geometric costs/constraints	category-level manipulation geometry	任意2D point list已具备planning合同
Where2Act	articulated geometry + action primitive	per-point actionability + orientation proposals + critic score	primitive-conditioned interaction proposal	任意robot/tool上的校准成功率

这五个接口的input、supervision、output、metric与下游都不同，不能引用各论文数字做横向“表示冠军”比较。正确比较应在同一任务、数据、compute、consumer与metric下做matched ablation。

11. 本地OPD：Affordance-labelled Point的真实性边界

冻结本地 `muxi` HEAD为 `756e27e16adcfb3ccbfe98c2920dda42095b127f`。五个直接读取文件支持的链路是：

```
ShareRobot affordance {x,y} / trajectory or point records
  -> normalize or scale to point coordinates
  -> "point to actionable location" prompt
  -> tagged point/box/text SFT target
  -> autoregressive generation
  -> point/box/text parser and static scorer
```

具体证据与边界：

1. `opd_project/scripts/build_prompt_pool.py` 把 `affordance.x/y` 缩放到 `0..1000`，输出单个 `gt_points` 并标为 `affordance_pointing`；没有primitive、direction、gripper、state或success head。
2. `point/point_data_only.py` 最终仍生成point answer；prompt里的“actionable”不会自动改变representation type。
3. `point/build_expert_mixes_seed0.py` 提取 `<box>` 与 `<point>`；`point/eval_qwen_vl_raw_holdout.py` 按point/box/text做format与static geometry/text scoring。
4. 选定五个文件中没有直接证明 `keypoint`、SAM、mask encoding、dense actionability head或controller episode；静态未发现也不代表仓库其他位置或历史上绝对不存在。

面试安全表述：

> 我的本地证据支持affordance-labelled point supervision，以及point/box/text生成与静态评估。这里“affordance”描述数据问题，“point”描述输出表示；当前证据不支持dense affordance map、semantic-keypoint/SAM pipeline、action-conditioned success calibration或机器人执行闭环。

本地关键证据hash：

文件	SHA-256
`build_expert_mixes_seed0.py`	`60DFDF46FDAC8EB9DBBE38A13298D6D71819EB7AD13F6B0D00A86B87032EABD0`
`eval_qwen_vl_raw_holdout.py`	`29574ADCE53B254BC33ED39C370642707F570000F7F3FB8F037DBD1F5E0D206B`

文件	SHA-256
`build_clean_grounding_pool.py`	`A7A5906E11EA687C6FD0DEC40F3D30922A91B538EB98C740230227F9FF943087`
`opd_project/scripts/build_prompt_pool.py`	`293C776BA03E28DAC0029F985EEFFDC20036D24A289F3D74B1F761D1C7629315`
`point/point_data_only.py`	`F650CB1F1B4A37566E11FA30C47A855DD59B740525887998 39DF28F71EEC5787`

12. Matched Representation Evaluation设计

PER-008只规定比较设计，metric公式和threshold细节留给PER-010。一个可信A/B至少固定：

1. 相同task/split/seed与原始输入；
2. 相同image transform、camera/time lineage；
3. 尽可能匹配backbone、training budget与data volume；
4. 每种表示使用正确的native parser，而不是把所有输出先转成box；
5. 同一downstream consumer与controller gate；
6. 同时报告表示自身质量、conversion failure、latency/memory与downstream outcome；
7. 保存per-sample raw outputs、invalid/empty/multimodal cases；
8. 禁止用不同论文的headline metric直接推断表示优劣。

建议实验表：

Factor	固定项	变化项	观察项
representation native	task/data/backbone budget	point/box/mask/keypoint/affordance head	native validity、latency、storage
conversion	同一raw output	direct native vs converted	information loss、invalid rate
consumer	同一prediction bundle	ROI/contact/geometry consumer	downstream failure taxonomy
uncertainty	同一candidate set	top-1 vs multimodal retention	mode miss、calibration、safety reject

如果同时换dataset、model、metric与controller，就无法把结果归因给representation。

13. Failure Taxonomy与诊断顺序

失败层	症状	常见根因	首个检查
task sufficiency	point无法支持contour，box无法支持contact	consumer字段没冻结	required-fields table
semantic collapse	把裸point叫keypoint/affordance	semantic/action fields缺失	typed payload
geometry	2D/3D、pixel/normalized、raw/processed混用	coordinate/frame lineage缺失	transform + round-trip
ambiguity	单点或单mask压掉多解	top-1过早	raw candidate/multimask trace
conversion	box填充成mask后当真值	把prediction当inverse	conversion provenance
temporal	mask/keypoint已陈旧仍执行	timestamp/instance association缺失	observation age
score	quality/objectness/action success混比	score semantics缺失	score definition + calibration split
parser	empty prediction被当成no-object	format failure与valid empty混淆	raw text/output + parser status
project claim	affordance-named数据被说成机器人能力	supervision、head、controller证据混淆	claim-evidence chain

推荐诊断顺序：consumer fields → native schema → image/coordinate/time lineage → raw uncertainty → conversion trace → parser golden test → matched metric → reachability/controller → robot episode。先从接口查起，比直接换模型更可解释。

14. 面试回答模板、追问与危险答案

14.1 两分钟结构化回答

1. 先说“按downstream minimum-sufficient fields选，不按密度排”；
2. 用一句话区分point、box、mask、keypoint、affordance；
3. 给出`mask -> box`有损和`point != keypoint/affordance`两个反例；
4. 提到uncertainty、multimodality与typed manifest；
5. 用RoboPoint/SAM/kPAM/Where2Act做接口对照；
6. 落到OPD：affordance-labelled point supervision，不夸成dense/controller能力；
7. 说明最终要做matched downstream evaluation与PER-015 gate。

14.2 高频追问

追问	回答抓手
为什么不用mask统一所有任务?	extra density不等于consumer收益；成本、temporal flicker与语义仍缺
一个point什么时候能叫keypoint?	有稳定semantic ID、instance/category、visibility与2D/3D frame
一个point什么时候能叫affordance point?	绑定primitive/tool/state/direction/horizon与score semantics
SAM的quality能当抓取成功率吗?	不能；它是mask输出侧quality，不是controller success calibration
box转mask是否可逆?	不可逆；分割模型会产生新prediction，矩形填充是新假设
多模态候选怎么处理?	保留candidate groups/scores/validity，在consumer端显式选择或拒绝
如何证明某表示更好?	matched task/data/compute/consumer/metric，报告native与downstream结果

14.3 危险答案

- “mask信息最多，所以任何任务都最好”；
- “keypoint就是point的别名”；
- “affordance就是可交互点坐标”；
- “SAM输出mask，所以它理解了物体语义”；
- “Where2Act score就是任意机器人成功率”；
- “box内部填满就恢复了mask”；
- “OPD训练了affordance，所以已经能做机器人action”；
- “不同论文各自metric更高就能说明表示更优”。

15. 项目补证清单

若要把当前`WARN`升级，优先归档：

1. 真实local mask、semantic-keypoint或dense affordance schema/head/parser（若存在）；
2. primitive、tool/gripper、state、direction、horizon与score-calibration字段；
3. raw multimodal outputs与conversion manifests；
4. matched representation ablation，包含latency/memory/invalid与downstream outcome；
5. perception-to-action reachability、controller reject与episode bundle；
6. Where2Act相关复用若发生时的实际license审查，因为冻结commit只有README中的MIT声明而没有LICENSE文件。

这些补证没有归档前，最强可信结论仍是“表示合同和本地point supervision已审计”，不是“系统已选择并验证最优affordance表示”。

16. 结论与下一步

PER-008的核心是把representation choice变成接口工程：point回答位置，box回答粗范围，mask回答pixel membership，semantic keypoint增加几何角色，action-conditioned affordance再增加primitive/tool/state/direction/horizon与score semantics。`mask -> box`可算但有损，`box -> mask`不可逆；同样的point坐标也不能恢复semantic或action fields。

当前source、knowledge与card已同步；本地OPD的证据边界已明确为affordance-labelled point supervision和point/box/text静态pipeline。由于缺少local mask/keypoint/dense-affordance head、matched representation ablation、success calibration与controller/robot bundle，本节点保持`WARN`，题卡保持`unseen`且尚无真实口述记录。

`VLA-PER-009` source/knowledge/card已同步并保持`unseen`。下一bounded slice只做PER-010 metric/parser source freeze；不生成knowledge/card，不创建`coaching_state.md`，不改变260张题卡的mastery。

References

1. PER-008 Representation Choice Source Freeze。
2. RoboPoint v1 与 wentaoyuan/RoboPoint@206cb1e。
3. Segment Anything v1 与 facebookresearch/segment-anything@dca509f。
4. kPAM v2 与 weigao95/kPAM@7214936。
5. Where2Act v2 与 daerduoCarey/where2act@1daf1a2。
6. PER-001 Grounding / Coordinate、PER-006 Image / Token / Camera、PER-007 Open-vocabulary Grounding。
7. OPD Claim–Evidence Inventory。

Part 9 / 43 · `interview_materials/vla/knowledge/vol_06_per009_grounding_matching_negative_loss.html` · SHA-256 `619D683E638DA847DC9BCC66FF396DEFABE9D9FB4E1399C2062BE8FD92D458D0`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`619d683e638da847dc9bcc66ff396defabe9d9fb4e1399c2062be8fd92d458d0`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per009_grounding_matching_negative_loss.md`

VOL 06 Perception / Grounding / Navigation：Grounding Matching、Negative 与 Loss Reduction（PER-009）

状态：`drafted / VLA-PER-009 / source_synced / card_synced / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：PER-009 Source Freeze。 前置知识：PER-001 Point / Box / Text Contract、PER-006 Image / Token / Camera、PER-007 Open-vocabulary Grounding、PER-008 Representation Choice。 原子题卡：VLA-PER-009（`unseen`）。

问题（Question）

Grounding detector训练时，Hungarian matching到底匹配什么？matching cost和最终training loss有什么区别？未匹配query、非目标token、padding、negative image与hard negative分别如何进入目标？focal、L1、GIoU、auxiliary loss又如何reduction？最后，本地OPD的assistant-token CE、routed top-1 CE和online teacher-KL能不能讲成Grounding DINO式detector loss？

本页用一条可审计链回答：

```
prediction/target schema + positive token map
  -> pairwise cost matrix
  -> Hungarian one-to-one assignment
  -> matched/unmatched token targets
  -> classification + localization components
  -> token/object/sample/batch/worker reduction
  -> auxiliary/encoder branches + diagnostics
  -> project-specific objective and evidence boundary
```

知识地图位置与边界

```
PER-001 schema / coordinate / parser
  -> PER-006 image-token-camera-time lineage
  -> PER-007 open-vocabulary inference dataflow
  -> PER-008 output representation choice
  -> PER-009 training matching / negative / loss / reduction
  -> PER-010 metric / parser / protocol comparability
  -> PER-011 pseudo-label / teacher noise
  -> PER-015 perception-to-action gate
```

PER-009只负责训练目标。它不把loss下降解释成IoU/PCK/Recall提升，不评价teacher是否可靠，也不从offline objective推出controller或robot success。VOL 01已讲CE、KL、focal等通用数学；本页新增grounding

tensor、assignment、mask、distributed denominator和本地代码映射。

1. 30秒直觉：先分配责任，再计算梯度

最短可用回答是：

> Set-prediction detector先构造每个query和每个GT之间的class/token、L1与GIoU cost，再用Hungarian算法做一对一assignment；这个cost只决定“谁负责谁”，不是最终反向传播的loss。assignment之后，matched query有token与box监督，unmatched query通常仍作为classification negative，但没有box target。padding必须被mask掉，不能当negative。最终loss的组件、权重和reduction必须按paper revision与runtime config冻结。本地OPD所读代码是生成式CE/KL lineage，没有query set、Hungarian或L1/GIoU head，所以不能口述成detector criterion。

面试中最容易失分的四个混淆：

1. 把Hungarian cost当作可直接求梯度的total loss；
2. 把padding、unmatched query、wrong token与negative image都叫“负样本”；
3. 只背 `1/5/2` 或 `2/5/2`，却不说matching/final与版本位置；
4. 看见本地有grounding数据与loss曲线，就声称训练了Grounding DINO或grounding质量提升。

2. Detector训练数据流与Tensor合同

2.1 最小schema

以冻结的Transformers 4.57.1 Grounding DINO bridge为例：

对象	Shape / 语义	必须冻结的约束
`logits`	`[B,Q,L]`，query×text-token logits	`L` 是当前prompt token positions，不是固定class数
`pred_boxes`	`[B,Q,4]`，normalized `cxcywh`	GIoU前转换为corner format
`labels[b].class_labels`	`[M_b]`	索引当前sample的label/phrase map
`labels[b].boxes`	`[M_b,4]`，normalized `cxcywh`	sample间GT数量可变
`label_maps[b]`	`[C_b,L]`	class/phrase映射到一个或多个token positions
`text_mask`	`[B,Q,L]` bool	只让有效attention positions进入classification loss
`indices[b]`	`(query_idx,target_idx)`	Hungarian一对一assignment
`num_boxes`	scalar	跨worker平均GT数并至少为1

2.2 Positive token map为什么重要

Open-vocabulary grounding不一定把类别轴做成固定的 `C` 个object classes，而是让query与text tokens对齐。一个phrase可能覆盖多个subword token，因此target需要 `phrase/class -> token positions` 的映射。

冻结的first-party GroundingDINO代码从character spans构造固定宽度positive map，并在phrase内部做token归一化；Transformers 4.57.1则按delimiter与special-token group构造label map。两条map construction没有golden test时不能当成bitwise或语义完全等价。

3. Matching Cost与Final Training Loss必须分开

3.1 Pairwise cost与Hungarian assignment

令query (i)与GT (j)的pairwise cost为：

$$C_{ij} = \lambda_{\text{cls}} C_{ij}^{\text{token}} + \lambda_1 \|\hat{b}_i - b_j\|_1 + \lambda_G \big(-\operatorname{GIoU}(\hat{b}_i, b_j)\big), \qquad \hat{\sigma} = \arg\min_{\sigma} \sum_j C_{\sigma(j),j}.$$

变量解释：$\hat{b}_i$和b_j是predicted/target normalized boxes；C_{ij}^{token}是query对目标phrase tokens聚合后的classification cost；$\lambda_{\text{cls}}, \lambda_1, \lambda_G$是matching阶段权重；$\hat{\sigma}$是离散的一对一assignment。直觉上，先比较所有“query负责GT”的候选代价，再选全局最小组合。实现映射：Transformers bridge计算focal-style token cost、`torch.cdist(..., p=1)`和negative GIoU，再调用SciPy `linear_sum_assignment`。证据边界：该式概括version-pinned paper/implementation bridge，不证明本地OPD存在matcher。

一个2×2 probe足以说明“全局”而非逐行贪心：对`[[0.2,1.3],[1.2,0.1]]`，assignment是`q0→t0,q1→t1`，总cost为`0.3`；交换后为`2.5`。这个probe只验证assignment逻辑，不验证任何模型预测正确。

3.2 为什么cost不是loss

比较项	Matching cost	Final training loss
作用	选择离散assignment	对选定targets形成梯度
输入	所有query×GT pair	matched pairs + classification negatives
权重	matcher coefficients	criterion coefficients
形式	可为排序友好的近似cost	按实现定义的可微loss
输出	integer index pairs	scalar与per-component loss dict

即便两边都写class/L1/GIoU，它们也可以有不同系数和具体公式。auxiliary decoder层在冻结bridge里还会重新matching，不能默认所有层复用final layer的assignment。

4. Negative Taxonomy：至少拆成五类

4.1 Query-token target matrix

冻结bridge先建立全零`target_classes_onehot`，再只在matched query位置写入目标phrase的token map：

```
matched query + target phrase tokens -> positive target 1
matched query + other valid tokens  -> negative target 0
unmatched query + all valid tokens  -> negative target 0
padding/out-of-attention positions  -> masked out; no loss
```

因此该实现中的background并不是DETR categorical classifier末尾一个独立`no-object`类，而是“对所有有效text tokens均为0”的binary target。

4.2 五类概念不能合并

类型	在冻结实现中的语义	是否进入loss	不应偷换成
matched positive token	query对应GT phrase token	正focal target	整个phrase只有一个固定class ID
matched query的其他有效token	wrong/non-target token	负focal target	dataset negative image
unmatched query	所有有效token target为0	classification negative；无box target	padding或显式no-object class
padding/special masked position	无有效监督语义	由 `text_mask` 或 `-100` 排除	easy negative
dataset/hard negative	负图、wrong prompt、挖掘样本等recipe	取决于数据构造	focal loss自动提供的能力

focal loss可以降低easy-negative dominance，但不会自动构造negative image、选择hard negatives或修正错误标签。负样本的“来源”与loss的“加权方式”是两个不同设计层。

5. Focal Loss：处理不平衡，不负责采样

对binary target，令p_t为正确标签的模型概率：

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log p_t.$$

变量解释：α_t平衡positive/negative类别；γ控制well-classified example的down-weighting；冻结的GroundingDINO/HF相关路径使用 `alpha=0.25`、`gamma=2`。直觉上，$p_t \to 1$时，modulating factor $(1-p_t)^{\gamma}$快速变小。实现映射：先做 `binary_cross_entropy_with_logits(reduction="none")`，再乘modulating factor与alpha，最后按冻结reduction聚合。证据边界：该式不证明概率校准、negative coverage或最佳超参数。

确定性probe中，对negative target取$\alpha_t = 0.75, \gamma = 2$：$p_t = 0.99$时loss约 `7.54e-7`，$p_t = 0.5$时约 `0.129965`，hard/easy约 `1.72e5` 倍。这只说明easy negative被强烈降权，不说明hard-negative mining已经发生。

6. L1与GIoU：只监督Matched Box Pairs

assignment之后，box supervision可写为：

$$\mathcal{L}_{\text{box}} = \frac{1}{N_{\text{gt}}} \sum_{(i,j)\in\hat{\sigma}} \Big[\lambda_1 \|\hat{b}_i - b_j\|_1 + \lambda_G \big(1 - \mathrm{GIoU}(\hat{b}_i, b_j)\big) \Big].$$

变量解释：$\hat{\sigma}$是matching pairs；N_{gt}是跨worker平均后且至少为1的GT box数；L1作用在normalized `cxcywh`，GIoU前转corner format。直觉上，L1直接约束坐标，GIoU补充overlap与relative geometry，特别是在boxes不重叠时仍能提供区分。实现映射：冻结HF base loss分别做elementwise L1求和和 `1-diag(generalized_box_iou)` 求和，再除 `num_boxes`。证据边界：training GIoU loss不是PER-010的evaluation IoU protocol。

unmatched query没有box target，因此不会进入L1/GIoU；但它仍可在classification tensor里产生负监督。这正是“no box loss”不等于“no gradient”。

7. Reduction顺序属于Objective合同

7.1 Detector classification与box denominator

冻结HF GroundingDINO classification bridge对`text_mask`后的valid query-token loss求sum，再除`num_boxes`。因此：

- unmatched-query negatives进入numerator；
- padding不进入numerator；
- denominator是GT box count，不是valid token数、query数或batch size；
- prompt长度或query数改变时，negative数量及loss scale可能改变。

base criterion先统计local GT count；distributed/Accelerate初始化时跨rank reduce，再除world size并`clamp(min=1)`。它近似"每worker平均GT数"，不是global token mean。复现时必须保存world size、per-rank targets、gradient accumulation与empty-target batch行为。

7.2 Weighted total、auxiliary与encoder分支

Transformers 4.57.1的version-specific总loss可概括为：

$$\mathcal{L} = 2\mathcal{L}_{\text{ce}} + \lambda_{\text{bbox}}\mathcal{L}_1 + \lambda_{\text{giou}}\mathcal{L}_{\text{giou}} + \sum_{l \in \text{aux}} \mathcal{L}^{(l)} + \mathcal{L}_{\text{enc}}.$$

变量解释：$\mathcal{L}^{(l)}$是intermediate decoder层的loss；$\mathcal{L}_{\text{enc}}$只在two-stage路径出现；该bridge默认`bbox_loss_coefficient=5`、`giou_loss_coefficient=2`，final classification coefficient为2。直觉上，auxiliary深监督让中间层直接收到matching/class/box梯度。实现映射：`auxiliary_loss`、`two_stage`与weight dictionary由该版本config/loss bridge控制。证据边界：这些值不是所有checkpoint或其他版本的runtime truth。

`cardinality_error`可以出现在loss dictionary中用于logging，但冻结Grounding DINO weight dictionary并未给它优化权重。看到loss dict中的key，必须继续检查它是否乘权后进入total loss。

8. Grounding DINO权重冲突：版本门比背数字重要

Grounding DINO v5正文与附录Table 8在classification matching/final权重上相反：

来源位置	Matching class/L1/GIoU	Final class/L1/GIoU
v5正文3.5与实验段叙述	`2 / 5 / 2`	`1 / 5 / 2`
v5附录Table 8	`1 / 5 / 2`	`2 / 5 / 2`
Transformers 4.57.1 bridge	`1 / 5 / 2`	`2 / 5 / 2`

安全口径是：论文确认token focal classification、L1/GIoU、bipartite matching及auxiliary structure；具体系数必须绑定paper位置、implementation revision、resolved config和runtime weight dictionary。冻结的first-party GroundingDINO commit没有公开完整matcher/criterion主体，不能靠该repo消除冲突。

面试官若追问“到底哪个对”，可以回答：

> 对我冻结的HF 4.57.1实现，matching是`1/5/2`、final是`2/5/2`，与paper Table 8一致；但paper正文给了相反的classification系数。因此我不会把任一组说成跨版本唯一官方值，会查看实际config和loss code。

9. 本地OPD三条Objective Lineage

9.1 Assistant-token SFT

`point_sft.py`分别tokenize full conversation和prompt-only conversation，用`prompt_len`把`labels[:prompt_len]`设为`-100`，collator也用`-100`补齐labels。直接证据只支持：prompt/padding不进入model loss，assistant response tokens被监督。

它把实际CE交给loaded model与Transformers `Trainer`，所以wrapper本身没有冻结具体model class内部shift/reduction。`SafeTrainer`还会在encode/filter失败、non-finite loss或training exception时产生zero-loss fallback step并写bad-batch log。aggregate loss必须与zero-loss/dummy/retry事件账本一起看。

9.2 Routed object/region top-1 CE

`train_opd_top1_vl.py`只在原assistant supervised positions写object/region teacher top-1 token ID，其余位置为`-100`。sample-level objective为：

$$L_b = w_{b,o}\,\mathrm{mean}_{t\in T_b}\,\mathrm{CE}(z_{b,t}, y^o_{b,t}) + w_{b,r}\,\mathrm{mean}_{t\in T_b}\,\mathrm{CE}(z_{b,t}, y^r_{b,t}), \qquad L = \mathrm{mean}_b\, L_b.$$

变量解释：T_b是sample b未被`-100`屏蔽的response positions；$w_{b,o}, w_{b,r}$由object/region/both route决定，`both`为`.5/.5`；z是student logits，y^o, y^r是teacher top-1 token IDs。直觉上，每个样本先对自己的有效tokens求mean，再按route混合，最后样本等权。实现映射：代码以`F.cross_entropy(..., reduction="mean")`得到sample CE，再做`sample_loss.mean()`。证据边界：这不是full-distribution distillation，也没有detector query/box matching。

确定性reduction probe：一个`both`样本的object/region token-mean CE为`2/4`，sample loss为`3`；另一个object样本loss为`8`，batch mean为`5.5`。把所有tokens拼接后直接global mean通常不会得到`5.5`。

9.3 Online full-vocabulary teacher KL

`train_opd_online_vl.py`由student或teacher生成response，再在同一generated sequence上计算teacher/student full-vocabulary logits：

$$q_{b,t} = \mathrm{softmax}\big(z^T_{b,t} + \beta\,\mathrm{stopgrad}(z^S_{b,t})\big), \qquad L_b = \frac{1}{|T_b|}\sum_{t\in T_b}\mathrm{KL}(q_{b,t}\|p^S_{b,t}), \qquad L = \mathrm{mean}_b(w_b L_b).$$

变量解释：z^T, z^S是teacher/student logits；β是veto schedule coefficient，0时target退化为teacher distribution；p^S是student softmax；T_b是full response或prefix positions；w_b是route weight。直觉上，它让

student拟合整个vocabulary distribution，而非只拟合teacher argmax token。实现映射：代码先逐token KL，再以`token_mask.sum(...).clamp_min(1)`做sample mean，最后`(sample_loss*weights).mean()`。证据边界：KL下降不证明teacher正确、parser改善或robot成功；teacher noise属于PER-011。

online路径还要求microbatch内route一致；ZeRO-3预载teacher时，各rank也必须走一致route，否则teacher调用和collective sequence可能失配。

10. 为什么三条本地Loss都不是Hungarian Detector Loss

能力/字段	显式Grounding detector	本地SFT	本地top-1 CE	本地online KL
query set `[B,Q,*]`	有	无直接证据	无	无
Hungarian assignment	有	无	无	无
unmatched-query negative	有	无	无	无
box L1/GIoU head	有	无	无	无
response token mask	非主要接口	`-100`	`-100`	prefix/full mask
teacher target	非必要	gold assistant token	teacher top-1 token	full-vocab distribution
reduction unit	query-token/GT boxes	model内部未完全冻结	token→sample→route→batch	token→sample→route→batch

“grounding数据 + 生成式token loss”可以训练模型输出point/box/text，但不能因此把其architecture/objective重命名为set-prediction detector。面试中先说objective family，再说target、mask与reduction，能够避免术语漂移。

11. 2500行Metrics应该怎样解释

冻结CSV共2500行，从raw rows重算后与summary JSON一致：

Window	loss	opd_loss	top-k overlap	grad_norm	opd_loss_tokens
first 100 mean	0.107026	0.091948	0.610231	8.164954	35.02
last 100 mean	0.029667	0.029326	0.828569	1.841578	33.29
last row / step 2500	0.0288	0.020672	0.806510	0.671648	24.0

这支持“归档optimization/logging trace内部存在下降趋势，summary算术可复算”。它不支持：

- 某一设计是唯一原因；
- checkpoint已经不可变绑定到这些rows；
- same-protocol holdout parser或grounding metric提升；
- teacher target更正确；
- robot rollout或closed-loop success提升。

更可信的证据链应为`step -> resolved config/command -> immutable checkpoint -> raw holdout prediction -> parser/metric protocol -> robot episode`。缺任一层时，只在已闭合层陈述结论。

12. Typed Loss Manifest

只记录一个scalar `loss`无法复现objective。建议最小记录：

```
{
  "sample_ids": [],
  "model_checkpoint_revision": "...",
  "code_config_processor_revision": "...",
  "objective_family": "hungarian_detector | assistant_ce | routed_top1_ce | online_teacher_kl",
  "prediction_schema": {},
  "target_schema": {},
  "positive_map_revision": "...",
  "matching_costs": {},
  "loss_coefficients": {},
  "negative_semantics": {},
  "masks": {"text": null, "prompt": null, "padding": null, "token": null},
  "reduction_order": ["element", "token_or_object", "sample", "batch", "worker"],
  "num_targets_per_rank": [],
  "auxiliary_encoder_branches": {},
  "zero_loss_dummy_retry_counts": {},
  "raw_loss_dict": {},
  "valid": true,
  "invalid_reason": null
}
```

这个manifest把“什么被监督、哪些位置被mask、怎么归一化、哪些分支计权、是否有fallback”从口头假设变成运行证据。对于本地OPD，还应保存route、response/prefix长度和teacher/student checkpoint；对于detector，还要保存positive map与per-rank GT counts。

13. 可审计训练Step伪代码

```
def detector_training_step(batch, model, matcher, criterion):
    # 1. Freeze prediction/target schema and valid text positions.
    outputs = model(batch["images"], batch["text_tokens"])
    targets = build_box_and_token_targets(batch)
    text_mask = batch["text_mask"]

    # 2. Build pairwise class/L1/GIoU costs only for assignment.
    cost = matcher.pairwise_cost(outputs, targets, text_mask)
    indices = matcher.hungarian(cost)

    # 3. Compute final differentiable losses after assignment.
    loss_dict = criterion(outputs, targets, indices, text_mask)

    # 4. Apply runtime weight_dict; logging-only keys get no gradient weight.
    total = sum(loss_dict[k] * criterion.weight_dict[k]
                for k in loss_dict if k in criterion.weight_dict)

    # 5. Persist denominators, masks, assignments and fallback events.
    save_loss_manifest(batch, outputs, targets, indices, loss_dict, total)
    return total
```

这段是工程伪代码，不是某个repository的可直接调用API。关键检查点是matching与criterion分层、`weight_dict`决定哪些key进入total、manifest保存真实denominator与mask。

14. Failure Taxonomy与诊断顺序

失败层	症状	常见根因	首个检查
target/map	phrase对错token，特殊字符下异常	char span/tokenizer/map revision错	raw tokens + positive map golden case
matching	assignment频繁跳、duplicate责任	cost尺度/格式/系数错	component cost matrix + box format
negative	loss被大量easy negatives主导	Q/L变化、mask漏、target全零异常	positive/negative/masked counts
box	GIoU/L1异常或NaN	`cxcywh/xyxy`、归一化、degenerate box	pre/post conversion tensors

失败层	症状	常见根因	首个检查
reduction	world size或batch变化时scale漂移	denominator/reduction顺序错	numerator + per-rank num_boxes
auxiliary	total loss多出或少掉层	weight dict/two-stage flag不一致	raw loss dict + resolved config
local SFT	loss虚低、有效token骤降	`-100` mask或zero-loss fallback	valid-token与fallback ledger
local route	object/region loss不可比	route混批、sample/global mean混淆	per-sample route + token counts
evidence	loss下降却metric不升	checkpoint/parser/protocol未绑定	immutable eval bundle

推荐顺序：schema与token map → raw target/mask counts → cost components → assignment probe → box format → numerator/denominator → auxiliary weight dict → zero-loss/route events → immutable checkpoint → PER-010 holdout protocol。先查合同，通常比先调learning rate更有解释力。

15. 面试回答模板、追问与危险答案

15.1 两分钟结构化回答

1. 用一句话分开matching cost与final loss；
2. 报出 `[B,Q,L]` token-axis logits、`[B,Q,4]` boxes和positive map；
3. 解释matched/unmatched/non-target/padding/dataset negative；
4. 说明focal、L1/GIoU及GT-box distributed normalization；
5. 主动指出paper正文/Table 8权重冲突与runtime config门；
6. 落到本地OPD三条CE/KL lineage，不夸成Hungarian detector；
7. 最后说loss曲线只证明optimization trace，metric/robot证据仍需绑定。

15.2 高频追问

追问	回答抓手
Hungarian matching可反向传播吗?	assignment是离散责任分配；梯度来自assignment后的可微loss
unmatched query完全没loss吗?	无box target，但通常仍有全零token classification target
padding算negative吗?	不算；由 `text_mask` 或 `labels=-100` 排除
focal loss等于hard-negative mining吗?	不等于；一个是loss reweighting，一个是数据/样本选择
为什么除GT boxes而不是tokens?	这是冻结criterion的scale合同；必须和valid-token numerator一起理解
aux层复用final matching吗?	冻结HF bridge会对每个aux output重新matching
`cardinality_error` 是否优化?	该bridge只logging，未进weight dict
Grounding DINO权重是多少?	先问matching还是final、paper位置与版本；HF 4.57.1与Table 8一致
OPD top-1和online KL差别?	argmax token CE vs full-vocabulary distribution KL，mask/reduction也需分别冻结
loss下降能证明grounding提升吗?	不能；要绑定checkpoint、raw prediction和同protocol metric

15.3 危险答案

- “Hungarian cost就是训练总loss”；
- “未匹配query和padding都是negative”；

- “用了focal loss就做了hard-negative mining”；
- “所有loss都按batch mean”；
- “loss dict里出现的项都会反向传播”；
- “Grounding DINO官方权重固定就是某一组数字”；
- “OPD用了point/box数据，所以loss就是Grounding DINO loss”；
- “2500步loss下降证明holdout和机器人效果提升”。

16. Project Truth Boundary与补证顺序

项目口径	当前可安全表述	当前不能声称
Public detector	能解释Grounding DINO paper与HF 4.57.1 matching/focal/L1/GIoU/reduction	本地部署或复现了Grounding DINO training
OPD SFT	prompt/padding以 `-100` mask，assistant tokens被监督	wrapper证明了exact model-internal CE reduction
OPD top-1	object/region teacher top-1 CE、route weights、sample→batch reduction	Hungarian、box head或full-distribution KL
OPD online	generated-response full-vocab KL、prefix mask、route/rank invariants	teacher可靠、holdout/robot uplift
Metrics	2500 rows与summary算术一致，optimization trace可复算	checkpoint/eval绑定、因果改进或candidate ownership

补证优先级：

1. exact command、resolved config、model class与immutable checkpoint；
2. batch target/mask/route manifest与raw per-component loss；
3. numerator、denominator、world size及zero-loss/dummy/retry ledger；
4. per-step或interval checkpoint到metrics的绑定；
5. same-protocol holdout raw predictions、strict parser与PER-010 metrics；
6. matched ablation和error taxonomy；
7. 最后才连接controller/robot result与candidate ownership。

17. 结论与下一步

PER-009的核心是把“责任分配”和“梯度目标”分开：Hungarian matcher在query×GT pairwise cost上选一对一assignment；final criterion再对matched token/box和unmatched token negatives计算focal、L1、GIoU及auxiliary/encoder loss。padding通过mask排除，dataset/hard negative需要单独recipe，reduction顺序与distributed denominator必须视为objective的一部分。

本地OPD的直接证据支持assistant-token SFT、routed teacher top-1 CE和generated-response full-vocabulary KL三条lineage，但没有显式query set、Hungarian matcher、unmatched-query negatives或L1/GIoU head。2500行metrics只闭合optimization trace算术，未闭合checkpoint→holdout→robot因果链。因此本节点保持 `WARN`。

当前PER-009 source、knowledge与atomic card已同步；全局260张题卡仍全部 `unseen`。下一bounded slice只做 `VLA-PER-010` metric/parser source freeze；不生成knowledge/card，不创建 `coaching_state.md`，不改

变任何mastery。

References

1. PER-009 Grounding Matching / Negative / Loss Source Freeze。
2. Grounding DINO v5。
3. DETR v3。
4. Focal Loss v2。
5. Generalized IoU v2。
6. IDEA-Research/GroundingDINO@856dde20。
7. Transformers 4.57.1 Grounding DINO loss 与 object-detection loss。
8. PER-007 Open-vocabulary Grounding 与 PER-008 Representation Choice。
9. OPD Claim–Evidence Inventory。

Part 10 / 43 · `interview_materials/vla/knowledge/vol_06_per010_grounding_metric_parser_protocol.html` · SHA-256 `5ECF8C5D984007B5C9B27F162D6ABCC69F34CF1DCCD402FD62182AC62E7C098A`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`5ecf8c5d984007b5c9b27f162d6abcc69f34cf1dccd402fd62182ac62e7c098a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per010_grounding_metric_parser_protocol.md`

VOL 06 Perception / Grounding / Navigation：Grounding Metric、Strict Parser 与 Protocol Comparability（PER-010）

状态：`drafted / VLA-PER-010 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-001` point/box/text schema、`VLA-PER-008` representation choice；后续：`VLA-PER-011` pseudo-label/noise、`VLA-PER-015` perception→action gate。 来源入口：PER-010 Grounding Metric / Parser / Protocol Source Freeze。本页只做已冻结证据的教学转译；atomic card已在独立切片同步并保持`unseen`，真实口述状态仍未开始。

问题（Question）

面试官问“你的grounding准确率是多少”时，不能只报一个数字。需要先回答：模型输出是否符合格式、parser如何工作、point/box/mask/keypoint分别用什么metric、invalid与empty是否进入分母、multiple predictions怎样matching，以及两次实验的split/checkpoint/prompt/parser/scorer是否完全可比。

本节点的核心不是多背几个指标名，而是建立一条不会偷换口径的评估链：

```
raw output
  -> strict parser + typed validity
  -> representation-specific matching
  -> per-sample metric
  -> invalid/empty denominator
  -> aggregation and strata
  -> immutable protocol identity
  -> project claim boundary
```

知识地图位置与边界

相邻节点	该节点负责	PER-010负责
PER-001	point/box/text输出schema、坐标范围与basic parser	strict/lenient双轨、invalid taxonomy和metric denominator
PER-006	resize/crop/letterbox、camera与transform lineage	将这些字段写进protocol identity，不重讲图像变换
PER-007	open-vocabulary inference、threshold/NMS、phrase与box输出	scorer怎样读取、match和aggregate输出
PER-008	point/box/mask/keypoint/affordance表示选择	不同表示为什么不能共用同一metric名称
PER-009	training matcher、negative、loss与reduction	evaluation metric；不把training cost/loss当最终指标

相邻节点	该节点负责	PER-010负责
PER-011	teacher noise、calibration、leakage与curriculum	只冻结gold/split/scorer身份，不评价teacher可靠性
PER-015	reachability、uncertainty与action failure gate	不从static grounding metric推出robot success
VOL 05	通用置信区间、paired test与评估工程	grounding特有的parser、matching、representation语义

1. 30秒直觉：先问“这个数字是怎么产生的”

一个可靠的30秒回答是：

> Grounding评估不是“从文本里抓到坐标就算分”。我先冻结expected representation和strict parser，把tag、syntax、coordinate、geometry四层状态分开；再按box、keypoint或point-set选择IoU、PCK、mask-hit或fixed-distance metric，并预注册invalid/empty分母。最后用dataset、sample IDs、checkpoint、prompt、decode、parser/scorer revision和aggregation生成protocol ID。任何字段不同，数字默认不直接可比。

这段回答主动封住四种常见偷换：格式失败被lenient parser隐藏、metric同名但公式不同、invalid被悄悄drop、不同split/checkpoint被说成单因素提升。

2. 评估栈：四层成功不能压成一个 `valid`

2.1 四层状态

层	核心问题	典型失败	是否等于几何错误
tag / format	是否使用要求的 `<point>` 或 `<box>` 协议	missing、mixed、extra tag	不一定
syntax / parse	完整字符串是否匹配grammar	trailing prose、wrong arity、ambiguous match	不一定
coordinate	数值是否finite、在约定范围、坐标系一致	NaN、越界、未反归一化	不一定
geometry	box是否正面积、集合是否非空、位置是否正确	degenerate box、empty、large error	是最终几何层

同一个输出可能“坐标几何正确但格式失败”，也可能“tag正确但坐标越界”。如果只保存 `valid=false`，后续无法判断是instruction following、parser compatibility、preprocess还是vision localization问题。

2.2 Typed record

```
{
  "sample_id": "...",
  "expected_representation": "point | box | text",
  "raw_output": "...",
  "parser_revision": "sha256:...",
  "parse_mode": "strict | lenient_diagnostic",
  "tag_status": "pass | missing | mixed | extra",
  "syntax_status": "pass | no_match | ambiguous | trailing_content",
  "coordinate_status": "pass | out_of_range | non_finite | wrong_arity",
  "geometry_status": "pass | degenerate | empty",
  "parsed_prediction": null,
  "invalid_reason": null
}
```

工程上必须保存 `raw_output`，否则parser升级后无法在同一generation上做paired reconciliation，只能重新推理并同时引入decode或runtime差异。

3. Strict Parser 与 Lenient Diagnostic 必须双轨

3.1 Strict的含义

strict parser应对完整字符串做`fullmatch`，而不是在任意prose里搜索数字。示意grammar为：

```
# Point and box outputs must occupy the complete response.
point       = "[", number, ",", number, "]" ;
point_out   = "<point>", "[", point, {",", point}, "]", "</point>" ;
box_out     = "<box>", "[", number, ",", number, ",", number, ",", number, "]", "</box>" ;
```

grammar通过后还要检查finite、0..1000范围、box的$x_2 > x_1, y_2 > y_1$、空集合、重复tag和mixed representation。strict score衡量“模型能否按部署合同输出”。

3.2 Lenient的用途

lenient parser只回答诊断问题：格式错时是否仍存在可恢复几何。它可用于构造如下transition matrix：

Strict	Lenient geometry	解释
pass	pass	格式与几何都正确
fail	pass	可能是instruction following或parser mismatch
pass	fail	tag/grammar通过但几何无效
fail	fail	格式与几何均失败，仍需看invalid reason

不能只发布lenient数字，也不能用它覆盖历史strict结果。正确做法是发布新的parser revision和protocol ID，并保留old/new paired结果。

4. 本地OPD Parser到底做了什么

冻结的`eval_qwen_vl_raw_holdout.py`不是纯strict parser，而是“tag gate + lenient geometry extraction + format-specific scoring”：

1. gold只含`<point>`时路由point，只含`<box>`时路由box；两者都有变成`mixed_grounding`，但没有专门scoring branch，最终落到text；
2. `parse_points`从任意文本搜索`[x,y]`，不要求坐标位于`<point>`内；
3. `parse_box`依次尝试flat tagged box、tag内部前四个数字、文本中的前两个point pairs；
4. `format_pass`另行要求对应tag与坐标合法，因此几何可能有分但格式失败；
5. text branch无条件`format_pass=true`，使mixed grounding的结构错误被隐藏。

确定性probe：

Raw output	可抽取point	`format_pass`
`<point>[[100,200]]</point>`	是	true
`[[100,200]]`	是	false
`answer is [100,200]`	是	false
`[(100,200)]`	否	false

这也说明早期prompt若要求`[(x,y)]`，不能简单说“strict为0只是缺tag”：当前regex连圆括号pair都无法恢复。需要先在raw outputs上做版本化parser reconciliation。

5. Box：IoU、Mean IoU 与 Acc@IoU

对corner boxes A, B，axis-aligned IoU为：

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

变量解释：$|A \cap B|$是intersection area，分母是union area；实现映射到本地`box_iou`的intersection clamp、area与union计算。直觉上它是单对框的连续overlap，不含dataset aggregation，也不判断输出格式。

阈值准确率为：

$$\mathrm{Acc}@\mathrm{IoU}(\tau) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\mathrm{IoU}(\hat{b}_i, b_i) \geq \tau\right].$$

变量解释：τ是overlap threshold，N必须明确是全部box样本还是valid-parsed subset。本地`Acc@0.3/0.5/0.75`将invalid prediction记为false并除全部box pool；`iou_mean`却只平均存在finite IoU的样本，所以两者分母不同。

边界probe：identical boxes的IoU为1，disjoint为0，touching edge也为0。`Acc@0.5=0.52`不代表mean IoU为0.52；改变`>=`、坐标逆变换、box order、parser或invalid policy都会改数字。

6. Referring-expression Recall@k：先冻结Overlap函数

one-expression/one-target场景常写为：

$$R@k(\tau) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\max_{j \leq k} O(\hat{b}_{i,j}, b_i) \geq \tau\right].$$

变量解释：k是取score排序后的top-k boxes，O必须明确是IoU、GIoU或其他overlap，N是expression数量。直觉上它问“top-k里是否至少有一个命中”，不是检测任务完整的precision–recall curve。

6.1 MDETR的revision trap

MDETR v3论文描述IoU-based Recall@k；固定commit `ea09acc...`的`RefExpEvaluator`实际调用`generalized_box_iou`，以0.5阈值判定，并打印`Precision @ 1,5,10`。下面的反例会翻转结论：

```
A = [0, 0, 2, 2]
B = [0.34, 0.34, 2.34, 2.34]
```

```
IoU  = 0.5254366562  -> pass at 0.5
GIoU = 0.4832129730  -> fail at 0.5
```

面试中最有价值的不是背“MDETR用Recall@1”，而是主动说：paper table、code revision、overlap function和打印名称必须绑定，否则同名scalar可能不是同一protocol。

7. COCO AP / AR 为什么不能和RefExp R@k互换

冻结COCO API detection defaults包含：

维度	默认合同
IoU thresholds	0.50到0.95，步长0.05，共10个
recall positions	0到1，步长0.01，共101个
max detections	1、10、100
precision tensor	`[T,R,K,A,M]`
recall tensor	`[T,K,A,M]`
matching	score排序后按image/category/threshold匹配，处理ignore/crowd/area
absent category	保存为`-1`，summary时排除

COCO AP汇总precision–recall envelope，AR也绑定IoU grid、area、category与`maxDets`。RefExp R@k通常对每个expression做一次binary success。两者的统计单元、ranking、matching和denominator都不同；都含“recall”不代表可比较。

8. Point Metrics：PCK、Mask-hit 与 Fixed-distance Hit

8.1 PCK需要correspondence与normalization

常见normalized keypoint hit可写为：

$$h_{n,k}(\tau) = \mathbf{1}\left[\left\|\frac{\hat{p}_{n,k} - p_{n,k}}{s_n}\right\|_2 < \tau\right], \qquad m_{n,k} = 1.$$

变量解释：$\hat{p}_{n,k}, p_{n,k}$是有语义对应的pred/GT keypoint，s_n是bbox、head或H/W normalization factor，$m_{n,k}$是visibility mask。冻结MMPose实现逐维除`norm_factor`、忽略invisible target，并先对每个keypoint type求accuracy再做macro average。直觉上PCK比较的是“相对于对象尺度是否足够近”，不是固定坐标距离。

普通grounding point set若没有keypoint identity，就不能直接套PCK；必须先冻结nearest-neighbor、Hungarian或semantic correspondence。

8.2 RoboPoint mask-hit

RoboPoint v1的spatial-affordance accuracy可写为：

$$\text{MaskHit} = \frac{1}{|P|}\sum_{p\in P}\mathbf{1}[p \in M].$$

变量解释：P是预测点集合，M是target mask。它是prediction precision-like度量：一个位于mask内的点就可能得到高分，却不保证覆盖全部target modes，也不等于PCK或GT recall。

8.3 本地fixed-coordinate any-pair Hit

本地OPD定义：

$$d_{\min} = \min_{p\in P, g\in G}\|p-g\|_2, \qquad \text{Hit @r} = \mathbf{1}[d_{\min} \le r], \quad r \in \{50, 100\}.$$

变量解释：P, G是0..1000规范坐标中的pred/GT point sets；没有image diagonal、bbox或object-size normalization。它只问“是否存在至少一对点足够近”，因此应完整称为`fixed-coordinate any-pair Hit@50/100`，不要简称PCK。

9. Multi-point：一个最近点不能代表集合质量

本地还计算两个directed nearest-neighbor mean：

$$d_{P\to G} = \frac{1}{|P|}\sum_{p\in P}\min_{g\in G}\|p-g\|_2, \qquad d_{G\to P} = \frac{1}{|G|}\sum_{g\in G}\min_{p\in P}\|g-p\|_2.$$

变量解释：$d_{P\to G}$更接近prediction purity，$d_{G\to P}$更接近GT coverage；两者都允许many-to-one，duplicate不会增加coverage。实现映射到本地两个nearest-neighbor list mean。

对`P={(0,0),(1000,1000)}`、`G={(1,1),(900,900)}`，最小距离为1.414，所以Hit@50/100均通过；但两个directed means约71.418。应联合报告point counts、count difference、两个directed means与task-specific coverage；若任务要求一一对应，再引入带unmatched penalty的bipartite matching。

10. Invalid、Empty 与 Denominator 是Metric的一部分

10.1 先写policy，再跑scorer

情况	推荐strict主口径	本地当前行为
缺tag但坐标可抽	format fail；几何另作diagnostic	format false，仍可能算geometry
out-of-range point	coordinate fail	coord false，但非空时仍可能算distance
invalid/degenerate box	bool hit=false，numeric undefined	Acc fields false，IoU/center为`None`
empty prediction	explicit empty fail	Hit false，numeric distance为`None`
mixed point+box	单独schema或拒绝	落入text，format被记true

情况	推荐strict主口径	本地当前行为
duplicate points	预注册dedup或precision penalty	保留并影响count与pred→gold mean

10.2 分母反例

本地`Metrics.summary`对bool字段除全部pool样本数，对numeric字段只除finite value count。一个invalid box加一个perfect box会得到：

```
format_pass = 0.5
Acc@IoU0.5 = 0.5
iou_mean    = 1.0
```

这三个数都符合当前代码，但`iou_mean=1.0`若不附`valid_iou_count=1/2`就会误导。minimum report必须给每个field的numerator、denominator与invalid reason counts。

11. Aggregation：Micro、Macro 与Episode不是一回事

Aggregation unit	含义	风险
sample micro	每条prompt/sample等权	多prompt image可能权重过大
image	同图多个prompt先聚合	需要定义图内规则
category/task macro	各类或各representation等权	小slice方差大，需附count/CI
keypoint type macro	每类keypoint先跨instance聚合	不等于visible joints micro average
robot episode	一次closed-loop任务为单位	static image metric无法替代

本地10k holdout由6,100 box、1,400 point、2,500 text组成。一个overall average容易被pool比例主导，所以至少按representation、object size、point count、prompt template、source dataset、validity reason和checkpoint分层。

12. Protocol Identity：什么时候两个数字可比较

最小protocol manifest应包含：

```
{
  "dataset_revision": "...",
  "split_and_sample_ids_sha256": "...",
  "image_and_gold_sha256": "...",
  "checkpoint_sha256": "...",
  "processor_prompt_template": "...",
  "decode_config": {},
  "expected_representation": "...",
  "coordinate_and_image_transform": {},
  "parser_revision_and_mode": "...",
  "matching_and_overlap": {},
  "thresholds": [],
  "invalid_empty_duplicate_policy": {},
  "denominators": {},
  "aggregation_and_strata": {},
  "scorer_runtime_revision": "..."
}
```

canonical serialization后计算SHA-256得到protocol ID。判断规则：

差异	结论
manifest相同，只改变seed	可做paired或multi-seed比较
同raw predictions，发布strict parser v2并同时跑v1/v2	条件可比；可隔离parser effect
IoU改GIoU、threshold或 `>=` 改 `>`	不直接可比
invalid由false改为drop	不直接可比，denominator变了
fixed Hit改PCK或mask-hit	不可比，问题定义变了
split、prompt、checkpoint、transform同时改变	不能做单因素归因
static holdout改robot success	不可比，evaluation unit与闭环语义不同

13. 可审计Scoring伪代码

```
def evaluate_row(row, protocol):
    # 1. Keep raw output and freeze the expected representation from the gold schema.
    raw = row["raw_output"]
    expected = protocol.expected_representation(row["gold"])

    # 2. Run the full-match strict parser and preserve typed failure reasons.
    strict = protocol.strict_parser.parse(raw, expected)

    # 3. Run lenient recovery only as a separate diagnostic channel.
    diagnostic = protocol.lenient_parser.parse(raw, expected)

    # 4. Apply the pre-registered invalid/empty policy before aggregation.
    if not strict.is_valid:
        metric = protocol.invalid_policy.score_failure(expected, strict.reason)
    else:
        # 5. Match predictions to targets using the representation-specific rule.
        matching = protocol.matcher.match(strict.value, row["gold"])
        metric = protocol.metric.compute(matching)

    # 6. Persist per-field denominator inclusion; never save only a rounded scalar.
    return {
        "sample_id": row["sample_id"],
        "raw_output": raw,
        "strict_status": strict.to_dict(),
        "lenient_diagnostic": diagnostic.to_dict(),
        "per_sample_metric": metric.values,
        "included_in_denominators": metric.denominator_flags,
        "protocol_id": protocol.sha256,
    }
```

这是评估合同伪代码，不是本地scorer的直接API。关键是raw preservation、strict/lenient分流、invalid policy先于summary、matching依representation选择，以及每个field保存denominator inclusion。

14. 本地OPD三口径Reconciliation

现有inventory记录：raw holdout 10,000条、8,251 unique images，box/point/text分别6,100/1,400/2,500；fingerprint、source-line、image+prompt+answer重复为0，与train image交集为0。OPD5000 checkpoint 5000的已记录数字包括Box Acc@0.5 `0.5238`、Point Hit@50/100 `0.7493/0.8729`、Text exact `0.8724`，但不同pool最佳checkpoint不同。

这些数字只支持冻结inventory中的static holdout语境。下一次评估必须分三栏：

1. **original strict**：原prompt、原parser、原scorer，保留历史可复现性；
2. **protocol-fixed strict**：修复prompt/parser合同，发布新protocol ID；
3. **lenient geometry diagnostic**：同raw outputs上做恢复分析，不替换主指标。

最可信的证据顺序是：归档raw predictions与checkpoint/generation config→同raw output跑parser v1/v2→输出paired transition matrix→必要时再固定decode重新generation。`strict=0`不等于几何能力为0，lenient较高也不能抹去部署格式失败。

15. Failure Taxonomy 与诊断顺序

失败层	症状	常见根因	首个检查
schema	point/box/text路由异常	gold tag或mixed representation	expected-format counts + raw gold
parser	strict低、lenient高	prompt/tag/grammar mismatch	raw output + parser golden tests
coordinate	大量越界或退化box	transform/normalization/order错	pre/post image transform与range
matching	point-set score异常乐观	any-pair minimum、duplicates	pairwise distance matrix + counts
metric	paper与code数字对不上	IoU/GIoU、threshold或name drift	exact scorer function/revision
denominator	mean很高但valid很少	numeric drop、invalid exclusion	numerator/denominator per field
aggregation	overall升而point降	pool比例或micro/macro变化	representation-stratified table
protocol	“改了一个点”但多项变化	split/checkpoint/prompt/decode漂移	two protocol manifests diff
project claim	static metric被说成robot成功	evaluation unit越界	episode/controller evidence gate

推荐顺序：schema→raw parser cases→coordinate transform→pairwise matching→metric function→denominator→aggregation→protocol diff→project claim。先查合同，通常比先调threshold更能定位根因。

16. 面试回答模板、追问与危险答案

16.1 两分钟结构化回答

1. 先说parser是metric的一部分，并拆tag/syntax/coordinate/geometry；
2. 给出strict主口径与lenient diagnostic双轨；
3. 按representation说明IoU/Acc、RefExp R@k、PCK、mask-hit、fixed Hit的不同问题；
4. 主动说明invalid/empty、matching与field-specific denominator；
5. 用protocol manifest判断两次结果是否可比；
6. 落到本地OPD code：mixed fallback、regex、any-pair hit和numeric denominator；
7. 最后把static holdout与robot success、因果提升、ownership分开。

16.2 高频追问

追问	回答抓手
为什么parser也算metric protocol?	raw→typed prediction决定哪些样本与几何进入scorer
格式错但坐标对，算不算正确?	strict主指标失败；lenient另报diagnostic，不静默替换
Mean IoU与Acc@0.5区别?	连续overlap mean vs thresholded sample success，分母也可能不同
Recall@1一定是IoU@0.5吗?	不一定；要问overlap、threshold、ranking、GT unit与code revision

追问	回答抓手
PCK与Hit@50一样吗?	不一样；PCK有normalizer、visibility与correspondence
多点只看min distance有什么问题?	一个近邻pair可通过，无法反映purity/coverage/duplicates
invalid prediction要不要drop?	预注册；部署主口径通常应失败计入，numeric可undefined但必须附count
overall metric为什么危险?	box/point/text pool比例与aggregation unit可能掩盖slice回退
何时能说新模型提升?	protocol manifest相同或差异被隔离，并绑定raw predictions/checkpoint
static grounding高能否说明机器人成功?	不能；还缺reachability、controller、闭环episode与failure semantics

16.3 危险答案

- “能从文本里抓到数字就算格式正确”；
- “IoU、Acc@0.5、Recall@1和AP都是定位准确率，差不多”；
- “Hit@50就是PCK@50”；
- “invalid样本删掉即可，不影响比较”；
- “同一指标名就能横向比较”；
- “MDETR官方就是固定用IoU/GIoU中的某一个”，却不绑定paper/code revision；
- “point Hit提高，所以机器人操作成功率提高”；
- “我实现了严格评估器”，但本地代码仍有mixed→text和lenient fallback。

17. Project Truth Boundary 与两个项目Hook

项目Hook	当前可安全表述	当前不能声称
OPD direct evidence	能逐函数解释point/box/text routing、tag gate、lenient parse、IoU/Hit、mixed fallback与field-specific denominator；inventory记录10k holdout构成和限定metrics	scorer是标准PCK/RefCOCO evaluator；raw predictions/checkpoint hashes已闭合；OPD单因素因果；候选人ownership
Robot Harness / downstream interface	grounding metric可作为perception diagnostic；进入controller前仍需PER-015 reachability/action gate与episode protocol	static holdout数字等于closed-loop manipulation/navigation success，或已有local-direct navigation bundle

补证优先级：raw predictions→sample/image/gold hashes→parser golden tests→strict parser v2→old/new paired matrix→fixed generation rerun→representation-stratified report→candidate ownership→最后连接controller/episode证据。

18. 结论与下一步

PER-010的核心是：metric从raw output进入parser时已经开始。strict与lenient必须双轨，tag/syntax/coordinate/geometry必须typed；IoU、Acc、RefExp R@k、COCO AP/AR、PCK、mask-hit和fixed-distance Hit回答不同问题；invalid、empty、matching、denominator和aggregation都属于protocol，而不是报告阶段的可选细节。

本地OPD可以安全讲清parser/scorer的真实代码、10k holdout inventory和三口径reconciliation方案，但不能把当前fixed-distance Hit叫PCK，不能把lenient geometry覆盖strict failure，也不能从static holdout推出

robot success或个人贡献。由于raw predictions、checkpoint hashes、strict parser v2 paired rerun和ownership仍缺，本节点保持`WARN`。

当前PER-010 source、knowledge与atomic card已同步；全局261张题卡全部保持`unseen`。下一bounded slice只冻结`VLA-PER-011` pseudo-label/teacher-noise source；不生成其knowledge/card，不创建`coaching_state.md`，不改变任何既有题卡状态。

References

1. PER-010 Grounding Metric / Parser / Protocol Source Freeze。
2. MDETR v3 与 mdetr@ea09acc RefExp evaluator。
3. COCO API@8c9bcc cocoeval.py。
4. RoboPoint v1。
5. MPII Human Pose v1 与 MMPose@759b39 keypoint_eval.py。
6. PER-001 Point / Box / Text Grounding。
7. PER-008 Representation Choice 与 PER-009 Matching / Negative / Loss。
8. OPD Claim–Evidence Inventory。
9. VOL 06 Coverage-Gap Audit。
10. PER-010 Atomic Card。

Part 11 / 43 · `interview_materials/vla/knowledge/vol_06_per011_pseudo_label_teacher_noise_calibration_leakage.html` · SHA-256 `7F0512C52BF9B0528679150FB19994211EB60AFBC2D894500AED2B2488C44777`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`7f0512c52bf9b0528679150fb19994211eb60afbc2d894500aed2b2488c44777`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per011_pseudo_label_teacher_noise_calibration_leakage.md`

VOL 06 Perception / Grounding / Navigation：Pseudo-label、Teacher Noise、Confidence Calibration、Leakage 与 Curriculum（PER-011）

状态：`drafted / VLA-PER-011 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-009` matching/loss、`VLA-PER-010` metric/parser、VOL 02 imitation learning；后续：`VLA-PER-012` grounding methods、`VLA-PER-015` perception→action gate。 来源入口：PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Source Freeze。本页只做冻结证据的教学转译；atomic card已在独立切片同步并保持`unseen`，真实口述状态仍未开始。

问题（Question）

面试官问“你们用teacher做pseudo-label，怎么保证标签可靠、没有数据泄漏”时，不能只回答“设0.95阈值并做了去重”。需要讲清楚：teacher在什么输入和decode协议下产生hard/soft target；confidence是什么单位；是否在独立有标签validation上校准；threshold后的coverage、accepted error和class/route bias怎样变化；每轮teacher/student怎样版本化；row、image、fingerprint、semantic、teacher-training和model-selection leakage分别怎样检查。

本节点建立的主链是：

```
candidate provenance
  -> teacher checkpoint + inference protocol
  -> typed hard / soft target
  -> confidence + calibration revision
  -> accept / reject / reweight
  -> coverage + accepted risk + distribution shift
  -> curriculum / iteration manifest
  -> leakage gates
  -> same-protocol student evaluation
  -> project truth boundary
```

知识地图位置与边界

相邻节点	该节点负责	PER-011负责
VOL 02 IL	behavior cloning、DAgger、covariate shift	perception teacher生成的数据与可靠性合同
PER-006	image resize/crop、camera/time lineage	lineage怎样写入pseudo record与split gate
PER-009	top-1 CE、full-vocabulary KL、mask与reduction	target由谁产生、为何可信、怎样筛选和迭代

相邻节点	该节点负责	PER-011负责
PER-010	strict parser、metric、denominator、protocol comparability	calibration split、accepted-set quality与leakage层级
PER-012	具体grounding/VLM方法谱系	teacher/data机制，不展开模型百科
PER-015	uncertainty到reachability/action/robot gate	不从static pseudo-label或holdout分数推出robot success
VOL 05	通用CI、paired test、评估工程	selective risk、teacher lineage和pseudo-feedback leakage

1. 30秒直觉：Pseudo-label是一条有血缘的记录，不是一列Label

一个可靠的30秒回答是：

> 我先把pseudo-label定义成带sample/image fingerprint、teacher checkpoint、prompt/decode、target type、confidence语义、calibration revision、acceptance decision和round的typed record。高confidence只是teacher自认为确定，不等于正确；所以threshold必须和coverage、独立gold audit上的accepted error以及class/route分布一起报告。temperature scaling只在独立有标签validation上拟合，不能把generation temperature或entropy logging叫校准。泄漏也分row、image、content、semantic、teacher training和model selection多层，任一身份未知都应fail closed。

这段回答主动封住四种常见偷换：

1. “teacher输出”被默认等同于gold；
2. “0.95 confidence”被默认等同于95%正确率；
3. “path overlap为0”被默认等同于完全无泄漏；
4. “超参随step变化”被默认等同于有效curriculum。

2. Typed Pseudo-label Record：可重放是可靠性的前提

2.1 最小Schema

```
{
  "sample_id": "...",
  "source_id": "...",
  "image_id": "...",
  "content_fingerprint": "sha256:...",
  "split": "train_unlabeled | calibration | validation | holdout",
  "teacher_revision": "checkpoint-sha256:...",
  "teacher_training_manifest": "manifest-sha256:...",
  "prompt_revision": "sha256:...",
  "decode_mode": "teacher_forced | free_generation",
  "target_type": "hard_token | soft_distribution | point | box | text",
  "target": "...",
  "confidence_type": "max_probability | margin | sequence_probability | none",
  "raw_confidence": null,
  "calibration_revision": null,
  "acceptance": {"decision": "accept | reject", "threshold": null, "weight": 1.0},
  "route": "object | region | both | general",
  "round": 0
}
```

工程直觉是：任何会改变target、选择集合或风险解释的字段，都必须成为artifact identity的一部分。只缓存label ID而不缓存teacher和confidence，会导致后续无法回答“哪个teacher在什么阈值下接收了多少样本”。

2.2 三个不可混淆的轴

轴	选项A	选项B	为什么不同
inference	teacher-forced	free generation	前者看到gold response prefix/位置；后者自行生成完整结构，exposure与格式失败不同
target	hard argmax	soft distribution	hard target丢弃非top-1概率，无法恢复teacher uncertainty
score state	raw confidence	calibrated confidence	后者还绑定有标签fit split、方法、参数和revision

对grounding还要区分token correctness、sequence exactness、strict parser validity和parsed geometry correctness。单token概率高，不代表整个point/box sequence格式正确或几何正确。

3. Teacher Target 与Noise Taxonomy

设teacher logits为$z_T(x)$，温度为$T > 0$，soft target写为：

$$q_T^{(T)}(c \mid x) = \mathrm{softmax}\left(\frac{z_T(x)}{T}\right)_c, \qquad \tilde{y} = \arg\max_c q_T(c \mid x).$$

变量解释：x是输入，c是token/class，$q_T^{(T)}$是温度化分布，$\tilde{y}$是hard target。直觉上，soft target保留teacher对其他类别的相对判断，hard target只留下冠军。实现映射：PER-011 source显示本地top-1 cache只保存argmax IDs；online trainer才计算full-vocabulary KL，二者不能被统称为同一种pseudo-label。

Noise层	典型失败	最小诊断	该诊断不能证明
semantic	指错对象、关系判断错	labeled gold accuracy、accepted precision	robot可执行性
representation	point/box/text不足以下游使用	representation-specific gold	action reachability
format/parser	tag缺失、坐标越界、空结构	strict validity与invalid reasons	语义正确
exposure	teacher-forced依赖gold prefix	paired free-generation eval	所有domain shift
domain	新相机、场景或语言分布外	per-domain risk–coverage	未观测OOD
class/route	easy/common类被过度接收	accepted histogram与per-group risk	分布已公平
correlated	teacher/student共享系统性错误	gold audit、diverse teachers	完全独立错误
temporal	旧cache被新round误消费	round/checkpoint manifest	内容无泄漏

4. Hard、Soft、Teacher-forced 与Free-generation怎样选

方案	保留信息	优点	主要风险	需要记录
hard token	argmax ID	存储与训练简单	丢失uncertainty，错误被当one-hot	teacher、位置、mask、ID、confidence
soft token	vocabulary distribution	可表达相似类别与暗知识	存储/计算高，temperature敏感	logits/probs、T、vocab、mask
teacher-forced cache	gold response位置的teacher target	与监督token对齐稳定	exposure bias；不是自由生成	prefix/labels、supervised positions

方案	保留信息	优点	主要风险	需要记录
free-generation structure	teacher自己生成point/box/text	更接近部署输出	格式/sequence error与decode随机性	prompt、decode、raw output、parser

选择原则不是“soft永远优于hard”，而是目标与风险对齐：若要蒸馏token分布，可用soft KL；若要构建可部署的结构化pseudo dataset，必须自由生成、保存raw output并走strict parser。二者可以共同存在，但应是不同artifact和metric。

5. FixMatch式Confidence Gate：Threshold只定义选择

5.1 Hard Pseudo-label与Mask

令x_i^w, x_i^s分别为同一无标签样本的weak和strong view，teacher在weak view上产生hard target：

$$\tilde{y}_i = \arg\max_c p_T(c \mid x_i^w), \qquad m_i = \mathbf{1}\left[\max_c p_T(c \mid x_i^w) \geq \tau\right].$$

变量解释：τ是threshold，m_i是accept mask。直觉上，模型只学习teacher最有把握的样本。实现映射：冻结FixMatch代码对weak logits取argmax与max probability，再用默认0.95阈值形成mask；但代码没有承诺0.95的样本就有95%正确率。

5.2 Masked Loss与Reduction Trap

$$L_u = \frac{1}{B}\sum_{i=1}^{B} m_i \ \mathrm{CE}(p_S(\cdot \mid x_i^s), \tilde{y}_i), \qquad \rho_\tau = \frac{1}{B}\sum_{i=1}^{B} m_i.$$

变量解释：B是全部unlabeled batch size，L_u是pseudo loss，ρ_τ是coverage。直觉上，FixMatch实现的`mean(loss * mask)`仍除以总batch，而不是accepted count；coverage下降会同时缩小有效loss scale。实现映射：复现时必须同时冻结threshold、pseudo-loss weight、batch组成和reduction。

确定性反例：confidence为$[0.96, 0.94]$，$\tau = 0.95$，per-sample CE为$[0.2, 0.8]$。mask是$[1, 0]$，total-batch mean为0.1，而accepted-only mean为0.2。只说“用了masked CE”不足以复现objective。

6. Quality Contract：Coverage、Accepted Error与Risk–Coverage

6.1 两个最小量

$$\mathrm{coverage}(\tau) = \Pr(m = 1), \qquad \eta_\tau = \Pr(\tilde{y} \neq y \mid m = 1).$$

变量解释：coverage是候选池中被接收的比例，η_τ是accepted-set error，需要独立gold labels。直觉上，threshold解决的是“收多少”，gold audit才回答“收进来的有多错”。实现映射：没有gold audit时可以报告score分布与coverage，但不能报告accepted precision或声称teacher可靠。

6.2 为什么高Threshold仍可能错

若teacher对错误class给0.99，$\tau = 0.95$和0.98都会接收它。threshold只能截断teacher自己的score，无法修复overconfidence、domain shift或系统性偏差。

最低报告表应包含：

Threshold	Candidate N	Accepted N	Coverage	Gold-audit N	Accepted error	Strict validity	Class/route drift
0.90	…	…	…	…	…	…	…
0.95	…	…	…	…	…	…	…
0.99	…	…	…	…	…	…	…

对grounding还要分token/sequence/parser/geometry risk；只用token-level max probability做threshold，不能直接代表结构化输出risk。

7. Confidence Calibration：Score怎样变成可解释概率

7.1 Temperature Scaling

固定模型参数，在独立有标签calibration set $\mathcal{V}$上拟合标量$T > 0$：

$$T^* = \arg\min_{T>0} \sum_{i \in \mathcal{V}} -\log \operatorname{softmax}\left(\frac{z_i}{T}\right)_{y_i}.$$

变量解释：z_i是样本logits，y_i是真实label，T^*是validation NLL最优温度。直觉上，它不改argmax class，而是调整分布尖锐程度，使score更接近经验正确率。实现映射：必须保存fit split、logits/checkpoint、optimizer或搜索方法和T^*；generation `temperature/top_p` 控制采样，不是这条校准过程。

7.2 ECE有完整协议

把样本按confidence分到bins B_b，top-label Expected Calibration Error写为：

$$\mathrm{ECE} = \sum_{b=1}^{M} \frac{|B_b|}{N} |\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|.$$

变量解释：M是bin数，N是样本数，acc/conf是每个bin的经验准确率与平均confidence。直觉上，它测量“说有多确定”与“实际多常正确”的差距。实现映射：需冻结bin edges、equal-width/equal-mass、top-label/classwise、sample unit和correctness定义；TorchMetrics的top-label分类实现不能自动定义VLA sequence或point/box正确性。

四样本$(0.55, 0), (0.60, 1), (0.85, 1), (0.90, 0)$分成两bin时，ECE为0.225；改变binning数值会改变。因此ECE只能在同一协议下比较，并应和risk–coverage一起看。

7.3 Calibration Set不能被回收进训练

若用于拟合T或threshold的labels又进入student train，之后还把同一split当独立评估，就产生data reuse。建议把`calibration`、`model_selection_validation`和`sealed_holdout`作为不同ACL角色；任何角色转换都要生成新protocol revision。

8. Selection Bias与Route/Class Balance

设g表示class、route、source或domain，threshold后的训练分布为：

$$p_\tau(x, y, g) = p(x, y, g \mid m = 1) \propto p(m = 1 \mid x, y, g)\, p(x, y, g).$$

变量解释：$p(m = 1 \mid x, y, g)$是某组样本被接收的概率。直觉上，如果common object比rare relation更容易拿高confidence，accepted set会越来越偏向easy/common，而不是原candidate pool的随机子集。实现映射：每次filter后都要保存candidate/accepted/rejected的route、class、source、domain histogram和per-group risk。

常见修正与边界：

手段	解决什么	新风险
per-group threshold	不同组score尺度不同	需要每组gold calibration；小组方差高
quota / balancing	避免rare group被清空	可能保留低质量样本
reliability weight	让低可靠target贡献更小	权重本身也要校准
active relabeling	把困难样本交给human/gold	成本与抽样偏差
diverse teacher agreement	降低单teacher偶然错误	teacher错误可能相关

平衡predicted class不等于平衡真实class，更不等于平衡场景难度或route；必须以gold audit和downstream结果复核。

9. Noisy Student：Filter、Balance与Iteration都是协议

冻结Noisy Student代码所表达的最小循环是：

```
labeled data -> train teacher_r
candidate unlabeled -> teacher_r inference -> pseudo_manifest_r
pseudo_manifest_r -> threshold filter + class/route balance
labeled + accepted pseudo -> noisy student_r
student_r same-protocol eval -> promote or reject -> teacher_(r+1)
```

每轮需要版本化：

Manifest字段	防止的问题
round与teacher checkpoint	旧cache被新round误消费
candidate manifest/hash	候选池悄悄变化
prompt/decode/inference revision	target变化无法解释

Manifest字段	防止的问题
calibration与threshold	acceptance policy不可重放
accepted/rejected counts	有效数据量不透明
route/class/source histogram	selection bias被掩盖
student config/checkpoint	teacher晋升身份不清
evaluation protocol与decision	用不同口径晋升teacher

teacher晋升不能只看training loss；应预注册same-protocol validation metric、risk–coverage和failure gates。若换了split、parser、prompt或checkpoint pool，不能把分数差称为单因素teacher迭代收益。

10. Curriculum：不是所有Schedule都叫课程学习

一个可验证curriculum至少包含：

1. **ordering signal**：confidence、difficulty、agreement、round或human audit；
2. **schedule**：何时放宽threshold、加入新domain/route或更新teacher；
3. **intervention**：样本以什么顺序和权重进入哪个objective；
4. **counterfactual**：random order、fixed threshold或fixed teacher基线；
5. **promotion gate**：什么证据允许进入下一轮。

示例manifest：

```
{
  "round": 2,
  "teacher_revision": "sha256:...",
  "calibration_revision": "sha256:...",
  "thresholds": {"object": 0.97, "region": 0.93},
  "accepted_by_route": {"object": 21000, "region": 18000, "both": 3000},
  "gold_audit": {"n": 800, "accepted_error": 0.06},
  "student_revision": "sha256:...",
  "evaluation_protocol": "sha256:...",
  "promotion_decision": "accept"
}
```

本地`veto_beta`随step线性变化，只证明detached student logits混入teacher logits的比例变化。没有easy-to-hard样本顺序、gold-calibrated ordering和fixed-β对照时，安全名称是`target-logit mixing schedule`，不能直接叫“校准curriculum已验证有效”。

11. Leakage Taxonomy：从同一Row到Teacher上游数据

11.1 十层检查

层级	典型污染	最低Gate	仍然未知
row	同一JSON row重复	canonical row hash	文本改写
source	holdout source混入train	source ACL/blocklist	跨source复制
image path/ID	同一路径或ID跨split	normalized key join	alias path、重编码
content fingerprint	相同bytes/内容重复	byte/normalized-image SHA	crop、resize、近重复

层级	典型污染	最低Gate	仍然未知
perceptual duplicate	不同bytes的近似图	pHash/embedding NN audit	threshold误差
semantic	同场景、模板或语义泄漏	entity/template/cluster audit	难以穷尽的同源关系
teacher training	teacher上游见过holdout	dataset/checkpoint lineage	黑盒预训练数据
preprocessing	split前用全数据拟合统计量	train-only fit manifest	processor隐藏状态
model selection	反复看test挑模型/阈值	validation + sealed test	人工观察后的隐性决策
pseudo feedback	holdout prediction进入下一轮train	immutable ACL与artifact lineage	缓存/人工复制

11.2 Alias-path反例

`sourceA/images/001.jpg`和`sourceB/copied/cup.jpg`若内容相同，`hash(source + path)`不同，可能进入不同split；content hash或perceptual fingerprint才会暴露它。因此“image-path overlap=0”只能证明同一个key没重复，不能证明无内容泄漏。

11.3 Fail-closed原则

以下任一情况应拒绝生成或消费pseudo-label：

- split未知或sample命中holdout ACL；
- 缺content fingerprint；
- teacher checkpoint/training lineage未知；
- calibration split与train/holdout角色冲突；
- cache round和teacher revision不匹配；
- duplicate check失败或检查程序没有成功退出。

目录名含`high_quality`或`no_eval_holdout`只是字符串，不是审计证据。证据必须是manifest、join结果、hash、脚本revision与退出状态。

12. Fail-closed Pipeline与伪代码

```
def build_pseudo_manifest(candidate, teacher, calibration, acl, policy):
    # 1. Freeze all identities before inference; unknown lineage is a hard failure.
    assert candidate.manifest_sha256
    assert teacher.checkpoint_sha256 and teacher.training_manifest_sha256
    assert calibration.fit_split == "calibration"
    assert calibration.fit_split not in {"train_unlabeled", "holdout"}

    accepted, rejected, seen = [], [], set()
    for sample in candidate.rows:
        # 2. Enforce sample- and content-level holdout isolation.
        assert sample.sample_id not in acl.holdout_sample_ids
        assert sample.content_fingerprint not in acl.holdout_fingerprints
        assert sample.content_fingerprint not in seen
        seen.add(sample.content_fingerprint)

        # 3. Generate under a frozen prompt/decode protocol and keep raw output.
        pred = teacher.infer(sample, protocol=policy.inference_protocol)

        # 4. Transform raw score with a validation-fitted calibration revision.
        conf = calibration.transform(pred.raw_confidence)
        threshold = policy.threshold[(sample.route, sample.source_id)]
        record = typed_record(sample, teacher, pred, conf, calibration, policy)

        # 5. Reject invalid structure before confidence selection.
        if not pred.strict_valid:
            record.reject_reason = "strict_invalid"
```

```
            rejected.append(record)
        elif conf < threshold:
            record.reject_reason = "below_threshold"
            rejected.append(record)
        else:
            accepted.append(record)

    # 6. Report coverage, gold-audited risk and distribution shift, then freeze hashes.
    report_coverage_risk_balance(accepted, rejected, policy.gold_audit)
    return freeze_manifest(accepted, rejected, teacher, calibration, acl, policy)
```

实现审查顺序应固定为：identity/ACL → raw generation → strict validity → calibration → threshold → balance → gold audit → immutable manifest。若先filter后丢弃raw output，就无法区分teacher语义错、格式错和parser错。

13. 本地OPD：四条链必须分开口述

13.1 Mix Builder：人工Route，不是Confidence Gate

`build_opd_mix_v1.py` 按source/bucket指定 `teacher=obj/reg/both`，目标比例35% object、35% region、30% general，样本不足时回采。它没有读取teacher confidence、entropy或calibration，也没有在该层做fingerprint dedup。

安全表述：

> 这一层做的是数据配比和teacher route assignment，不是基于校准置信度的pseudo-label筛选。目录名里的high-quality/no-holdout需要另用manifest和重叠检查验证。

13.2 Top-1 Cache：Teacher-forced Token Distillation

`prepare_opd_top1_cache.py` 让teacher进入 `eval()` 和 `inference_mode()`，在真实assistant response的supervised token positions上取logits argmax，保存 `obj_top1_ids/reg_top1_ids` 及count。

```
image + prompt + gold response positions
  -> teacher-forced token logits
  -> per-position argmax IDs
  -> cached hard distillation targets
```

它不是 `image + prompt -> teacher自由生成point/box -> parser`。cache也没有confidence、margin、entropy、temperature、threshold或accept/reject字段，因此不能事后重算coverage与accepted error。

13.3 Top-1 Trainer：固定Route权重

`train_opd_top1_vl.py` 直接消费缓存IDs； `both` route固定0.5/0.5。它支持“hard teacher targets进入训练”的代码事实，不支持“target经过校准过滤”或“权重反映teacher可靠性”。hard CE公式属于PER-009，本页只解释target quality evidence。

13.4 Online KL / Veto：Logging不是Quality Gate

`train_opd_online_vl.py` 冻结teacher、使用 `eval()`、可选teacher/student rollout并计算full-vocabulary KL；teacher entropy与top-k overlap被记录，但未据此拒绝或降权target。sampling temperature/top-p控制rollout采样，不是calibration。

若target logits为：

$$z' = z_T + \beta z_S^{\text{detach}},$$

变量解释：z_T是teacher logits，z_S^{detach}是不回传梯度的student logits，β是`veto_beta`。改变β会改变soft target，但没有gold y就不能判断它更正确。实现映射：安全名称是target-logit mixing，不是confidence calibration或已验证curriculum。

14. 本地Raw Holdout：强证据与剩余未知

`eval_raw_holdout_v1/summary.json`报告：10,000 rows、8,251 unique images、10,000 unique fingerprints、reported fingerprint duplicate=0；10,000 rows均`image_seen_in_train=false`，blocklist覆盖source keys、fingerprints和713,545 train images。

可以说	不能说
当前summary对应manifest通过了记录中的row/image/fingerprint可见gate	teacher checkpoint从未见过相同上游数据
blocklist和fingerprint检查是比路径名更强的证据	semantic/near-duplicate完全为零
static holdout支持格式/几何诊断	static grounding等同robot success
可见train-image overlap被报告为false	test从未参与prompt/parser/checkpoint选择

安全结论应分层写：`row/source/image/fingerprint checked`；`near-duplicate/semantic/teacher lineage unknown`；`model-selection process not sealed`。不能压成一个`no_leakage=true`。

15. 诊断Playbook：Student没提升时先查哪里

症状	首查证据	常见根因	下一Probe
accepted N突然下降	threshold、calibration revision、score histogram	温度/teacher/domain变化	固定candidate做paired score diff
overall涨、rare route跌	accepted-by-route、per-group risk	selection bias	per-route threshold/quota matched A/B
train loss低、free generation差	teacher-forced vs free mode	exposure/format error	同checkpoint paired free-generation eval
confidence高但gold错	reliability bins、risk–coverage	overconfidence/OOD	labeled audit + temperature scaling
新round结果不可复现	teacher/cache/manifest hash	revision混用	fail round identity check
holdout异常高	overlap、teacher lineage、selection log	content/semantic/model-selection leakage	near-dup audit + sealed rerun
entropy下降但效果不升	gold risk与strict validity	score更尖锐而非更正确	paired correctness/calibration audit
veto schedule有变化无收益	β日志、fixed-β baseline	target mixing无有效机制	matched ablation

推荐诊断顺序：先检查identity和split，再检查target mode与strict validity，然后检查calibration/risk–coverage，最后才讨论student architecture或优化器。数据身份不稳定时，调模型无法产生可信因果结论。

16. 面试回答模板、追问与危险答案

16.1 两分钟结构化回答

> 我把teacher supervision拆成生成、选择、训练和评估四段。生成段冻结teacher checkpoint、prompt/decode和teacher-forced/free-generation模式，并保存typed target和raw output。选择段不把confidence当正确率，而是在独立有标签calibration set上拟合temperature，按threshold报告coverage、accepted error和class/route分布；FixMatch式masked loss还要说明分母是总batch。迭代段记录每轮teacher、candidate、filter/balance、student和promotion manifest。泄漏段分别检查row、source、image/path、content/near-duplicate、semantic、teacher training和model selection。对本地OPD，我只能说有人工route mix、teacher-forced top-1 token cache、online KL/veto和10k holdout gate；cache没有confidence，entropy只是logging，所以不能说伪标签已校准，也不能从static holdout推出robot成功率。

16.2 高频追问

1. 为什么0.95 confidence不代表95%正确率?
2. FixMatch的masked loss到底除accepted count还是总batch?
3. temperature scaling和generation temperature有什么区别?
4. ECE为什么不能脱离binning、sample unit和correctness定义?
5. sequence/point/box pseudo-label的confidence怎么定义?
6. threshold升高为什么可能让long-tail更差?
7. class balance为什么不等于真实分布无偏?
8. Noisy Student每轮必须冻结哪些identity?
9. 什么条件下一个schedule才配称curriculum?
10. path overlap=0为什么仍可能image leakage?
11. teacher预训练污染和student split污染怎样区分?
12. 为什么本地top-1 cache不是自由生成pseudo-label dataset?
13. entropy下降为何不能证明校准变好?
14. 10k holdout当前证明了哪几层、没证明哪几层?

16.3 危险答案与修正

危险答案	问题	安全修正
“0.95以上就是高质量标签”	confidence≠correctness	报coverage、gold accepted error与分层risk
“调temperature做了校准”	可能只是sampling temperature	说明独立validation拟合T与held-out检验
“entropy越来越低，teacher更准”	可能只是更尖锐/过度自信	配对gold accuracy与calibration
“image path不重叠，所以没泄漏”	alias/content/semantic仍可能重叠	加content/perceptual/semantic audit
“top-1 cache是teacher生成标签”	隐藏teacher-forced exposure	说清监督token位置argmax
“veto beta是curriculum”	只有schedule，没有ordering/baseline/effect	称target-logit mixing schedule
“holdout无重复，结果绝对可信”	teacher/model-selection污染未封	分层陈述checked与unknown

危险答案	问题	安全修正
“静态grounding提升就能上机器人”	perception→action gate缺失	留给PER-015和closed-loop证据

17. Project Truth Boundary与项目Hook

证据层	当前可以说	当前不能说	补强动作
public mechanism	FixMatch threshold/mask；Noisy Student filter/balance/iteration；calibration与curriculum合同	已在OPD等价复现并有效	配置、日志与matched result
local code	mix route、path-hash split、top-1 cache、hard trainer、online KL/veto存在	全部由候选人原创/亲自实现	git blame/PR/周报/本人确认
target semantics	cache是teacher-forced token argmax	自由生成point/box pseudo dataset	raw generation + parser manifest
confidence	online记录entropy/top-k overlap	已做gating/calibration	labeled calibration、T^*、risk–coverage
leakage	10k summary报告image/fingerprint gate	semantic/teacher/model-selection完全无污染	near-dup、teacher lineage、sealed test
outcome	static raw holdout summary存在	pseudo-label因果提升或robot success	exact raw/checkpoint、ablation、robot trials

17.1 OPD Hook

OPD是本节点的主要项目hook，但答案必须按“公开机制→本地代码事实→缺失证据”三层展开。最有区分度的点不是声称teacher pipeline先进，而是准确发现：top-1 cache缺confidence字段、teacher-forced与free-generation不同、online entropy没有成为selection gate、visible leakage gate仍留teacher lineage未知。

17.2 Robot Harness Hook

Robot Harness只能作为下游消费与安全门提醒：即使static perception pseudo-label可靠，仍需PER-015的reachability、uncertainty、controller和closed-loop failure gate。本节点没有证据把OPD holdout结果映射成robot成功率。

18. 白板Checklist与练习标准

面试前能在白板上独立写出：

- typed pseudo-label record的十个关键identity；
- hard/soft与teacher-forced/free-generation四象限；
- threshold mask、masked loss、coverage和accepted error公式；
- temperature scaling与ECE，并说明sample unit/binning；
- selection后的条件分布与route/class drift；
- Noisy Student round manifest和promotion gate；
- curriculum的ordering/schedule/intervention/baseline；
- 十层leakage taxonomy和alias-path反例；
- 本地OPD四条链及每条claim boundary；

- 一段两分钟回答和至少六个追问。

本页`drafted`不等于掌握。只有候选人真实口述、追问、纠错并记录状态后，题卡才可从`unseen`推进；本轮不创建`coaching_state.md`。

19. 结论与下一步

Pseudo-label系统的核心不是选一个threshold，而是建立可重放的teacher/data/selection/iteration身份，并用独立gold证据测量risk–coverage。Calibration解决score解释，balance处理selection distribution，curriculum规定有对照的呈现顺序，leakage gate保护评估独立性；四者不能互相替代。

本地OPD当前最准确的结论是：存在人工route mix、teacher-forced top-1 token cache、hard-target/online KL训练代码与10k raw holdout的可见image/fingerprint gate；但尚无confidence-aware cache、独立calibration、accepted-set gold audit、teacher lineage、sealed model selection或robot结果。

下一bounded slice只冻结`VLA-PER-012` segmentation/depth/keypoint/affordance source；不生成knowledge/card，不创建`coaching_state.md`，不改变任何既有mastery。

References

1. PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Source Freeze：固定论文、官方代码、本地OPD脚本、SHA、deterministic probes与truth boundary。
2. Sohn et al., FixMatch v2；official implementation@d4985a1。
3. Xie et al., Noisy Student v4；official implementation@aac52bf。
4. Guo et al., On Calibration of Modern Neural Networks v2。
5. TorchMetrics Calibration Error@37a8080。
6. Bengio et al., Curriculum Learning，ICML 2009。
7. Kapoor & Narayanan, Leakage and the Reproducibility Crisis in ML-based Science v1。
8. PER-009 Grounding Matching / Negative / Loss 与 PER-010 Grounding Metric / Parser / Protocol。
9. OPD Claim–Evidence Inventory：本地mix/cache/train/eval事实与ownership/robot边界。

Part 12 / 43 · `interview_materials/vla/knowledge/vol_06_per012_segmentation_depth_keypoint_affordance.html` · SHA-256 `92EAC7125AA585A6DC2DB83F1E78DE2AE5E13352D38E835AA1CFEE21E2782320`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`92eac7125aa585a6dc2db83f1e78de2ae5e13352d38e835aa1cfee21e2782320`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per012_segmentation_depth_keypoint_affordance.md`

VOL 06 Perception / Grounding / Navigation：Segmentation、Depth、Keypoint 与 Manipulation Affordance（PER-012）

状态：`drafted / VLA-PER-012 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-002` 2D→3D lift、`VLA-PER-008` representation choice、`VLA-PER-010` metric/parser；后续：`VLA-PER-013/014` 3D representation/grounding、`VLA-PER-015` perception→action gate。 来源入口：PER-012 Segmentation / Depth / Keypoint / Affordance Source Freeze；对应自测：PER-012 Atomic Card。本页只做冻结证据的教学转译；题卡已生成并保持 `unseen`，真实口述状态仍未开始。

问题（Question）

面试官问“如何把图像中的物体变成机器人可用的操作目标”时，不能只列SAM、Depth Anything或Where2Act模型名。需要讲清：segmentation task到底是semantic、instance、panoptic还是promptable；depth是relative还是metric；mask/depth怎样对齐原图、相机和时间；keypoint为何必须有semantic identity和3D frame；affordance为何绑定primitive、orientation和state；以及这些perception proposal为什么仍需planner、IK、collision和controller gate。

本节点建立的主链是：

```
RGB + camera/preprocess identity
  -> task-typed segmentation
  -> relative or metric depth + validity
  -> registered mask-conditioned 3D surface
  -> semantic keypoint + visibility/covariance/frame
  -> primitive-conditioned point/orientation proposal
  -> reachability/IK/collision/controller gate
  -> episode trace and success predicate
```

知识地图位置与边界

相邻节点	该节点负责	PER-012负责
PER-001	point/box/text schema与coordinate protocol	dense mask/depth/keypoint/affordance方法产生链
PER-002	pinhole lift、intrinsics、depth validity、TF/time	depth model输出怎样满足metric lift前置条件
PER-006	generic resize/crop/letterbox与camera lineage	SAM/Mask2Former/Depth Anything specific preprocess/postprocess

相邻节点	该节点负责	PER-012负责
PER-008	为什么选择point/box/mask/keypoint/affordance	五种代表方法的architecture/output/composition
PER-009	Hungarian、negative、loss/reduction	只解释Mask2Former mask criterion，不重讲通用loss课
PER-010	metric/denominator/protocol comparability	方法输出的task-specific metric字段
PER-011	pseudo-label/calibration/leakage	Depth Anything teacher/pseudo-data只作来源，不重讲可靠性
PER-013/014	point cloud/voxel/occupancy与3D grounding	本页止于depth surface/keypoint/affordance接口
PER-015	reachability、collision、uncertainty、controller	本页不从proposal推出可执行或robot success

1. 30秒直觉：先冻结Task与Unit，再谈模型

一个可靠的30秒回答是：

> 我先冻结输出合同：semantic、instance、panoptic和promptable mask不等价；relative depth不能当米制深度。mask和metric depth对齐原图、内参及时间后才能反投影。操作层的keypoint必须带semantic ID、visibility和3D frame，affordance还绑定point、orientation、primitive与state。即使proposal高分，也必须经过reachability、IK、collision和controller；因此单模块mIoU、depth图或actionability都不能直接代表robot success。

这段回答封住五种常见偷换：

1. 一个`mask`字段被默认同时具备类别、实例和时间身份；
2. relative depth可视化被默认当真实米制距离；
3. 无名point被默认当semantic keypoint；
4. “actionable”文本被默认当动作条件化affordance；
5. perception score被默认当closed-loop success。

2. Typed Dense Perception Record

2.1 最小Schema

```
{
  "sample_id": "...",
  "image_id": "...",
  "camera_id": "...",
  "timestamp": "...",
  "model_revision": "checkpoint-sha256:...",
  "preprocess_revision": "sha256:...",
  "task": "semantic | instance | panoptic | promptable | relative_depth | metric_depth | semantic_keypoint | affordance",
  "prompt_or_primitive": null,
  "payload": "mask | depth | keypoints | action_proposals",
  "coordinate_space": "original_pixel | resized_pixel | camera_3d | target_3d",
  "unit": "class_id | instance_id | relative | meter | normalized",
  "score_semantics": "class | mask_quality | depth_uncertainty | visibility | success_likelihood",
  "validity_mask": null,
  "frame_transform_revision": null,
  "age_ms": 0
}
```

2.2 Shape相同不等于Semantics相同

同一个`H×W`数组可能是class ID、instance ID、binary mask、relative depth或meter depth；同一个`[x,y]`可能是无名point、`mug_handle` semantic keypoint或`pull` contact proposal。只有payload而没有task、unit、frame和score semantics，接口一定会静默漂移。

3. Segmentation四类Task

Task	预测单元	输出	两个同类物体	Stuff	操作系统常见用途
semantic	pixel→class	class score/map	合并	支持	区域语义、scene layout
instance	object instance	mask+class+score	分开	通常非主目标	选定具体目标、crop/tracking seed
panoptic	non-overlap segment	segment ID+category/isthing	thing分开	全覆盖	完整scene decomposition
promptable	prompt-conditioned region	one/multiple binary masks+quality	取决于prompt	可返回	language/point/box驱动轮廓

两个mug在semantic map里都可标成`mug`，但操作“拿左边那个”需要instance identity；panoptic还需处理thing/stuff与segment merge；SAM prompt mask可能边界很好，却没有category label或跨帧instance ID。

3.1 Task先于Metric

问“分割准确率多少”时，先反问：semantic mIoU、instance mask AP、panoptic PQ还是prompt-conditioned mask IoU？dataset、class set、thing/stuff、candidate policy、threshold、matching和denominator不同，数字不能横比。

4. Mask2Former：Query→Class+Mask

4.1 Pipeline

```
image -> normalized/padded batch
  -> backbone + pixel decoder
  -> transformer queries with masked attention
  -> pred_logits [B,Q,C+1] + pred_masks [B,Q,H',W']
  -> Hungarian responsibility
  -> class + sigmoid-mask + Dice supervision
  -> semantic / instance / panoptic postprocess
```

固定代码允许同一架构打开三类inference分支，但并不意味着输出合同统一。semantic输出是class per pixel；instance输出保留ranked masks/instances；panoptic输出是non-overlap segment IDs与`category_id/isthing`。

4.2 Criterion：Assignment不是Loss本身

$$\mathcal{L}_{\text{seg}} = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{mask}}\mathcal{L}_{\text{sigmoid}} + \lambda_{\text{dice}}\mathcal{L}_{\text{dice}}.$$

变量解释：classification监督matched query类别；sigmoid mask CE监督采样pixel；Dice关注区域overlap；λ来自runtime weight dictionary。直觉上，Hungarian先确定query负责哪个GT，再计算可微loss。实现映射：固定Mask2Former criterion明确调用sigmoid CE与Dice；它与OPD assistant-token CE完全不同。

4.3 Semantic Inference

$$P_{c,h,w} = \sum_{q=1}^{Q} \mathrm{softmax}(l_q)_c \, \sigma(m_q(h,w)).$$

变量解释：l_q是query class logits，m_q是mask logits，Q是query数量。每个query用“属于哪个类×覆盖哪些pixel”对semantic map投票。instance/panoptic还要冻结top-k、object threshold、overlap threshold、metadata与resize/postprocess顺序。

5. SAM：Promptable Mask Candidate

5.1 输入输出合同

```
original image -> image embedding
point(+/- labels) OR box OR mask_input
  -> prompt encoder + mask decoder
  -> C original-resolution masks
  -> C predicted quality scores
  -> C x 256 x 256 low-resolution logits
```

`multimask_output=true`意味着一个ambiguous click可返回多个合理candidate。因此SAM integration至少冻结：

- prompt坐标在original还是resized image；
- positive/negative point label；
- box corner convention；
- multimask选择policy；
- mask threshold、component与hole policy；
- category/instance/language binding；
- temporal ID、observation age和resegment条件。

5.2 Predicted Quality不是Robot Success

SAM score用于候选mask质量排序，不自动等于语义正确、接触面安全或机器人成功概率。若用它触发动作，需在独立task validation上验证calibration与risk–coverage，并继续过PER-015的reachability/collision/controller gate。

6. Depth四种合同

类型	Unit/Ambiguity	可否直接Lift	必须记录
relative monocular	near/far结构，无meter contract	否	checkpoint、alignment policy、range
metric monocular	训练域/range定义的metric output	有条件	unit、max depth、dataset、validity
RGB-D/stereo sensor	设备unit、invalid code与noise	有条件	scale、intrinsics、registration、timestamp

类型	Unit/Ambiguity	可否直接Lift	必须记录
aligned relative	reference拟合scale/shift	仅fit域内	fit split、scale/shift、residual

若用reference将relative prediction对齐：

$$\hat{d}_m(u,v) = s\,\hat{d}_r(u,v) + t.$$

变量解释：s, t是从合法reference拟合的scale/shift，$\hat{d}_r$是relative prediction，$\hat{d}_m$只是aligned metric candidate。直觉上，排序正确不等于距离正确。实现映射：Depth Anything relative code不会自动执行该对齐，必须保存fit split、reference和residual。

7. Depth Anything V2代码合同

7.1 Relative Path

固定代码为DINOv2 intermediate features→DPT fusion head→ReLU dense depth。`infer_image`默认518；保持aspect ratio、lower-bound resize、保证14的倍数、ImageNet normalize，最后双线性插值回original $H \times W$。README把主模型称为relative depth models，因此“输出shape与原图相同”不等于“单位为米”。

7.2 Metric Path

固定metric path使用单独代码与checkpoint：head末端Sigmoid，forward乘`max_depth`，默认20.0。对同一raw head output 0.8，`max_depth=20/80`会得到16/64；所以checkpoint family、训练dataset、range与unit必须进入artifact identity。把relative checkpoint塞入metric code或手动乘常数不构成metric calibration。

7.3 Preprocess与Postprocess Trap

把prediction resize回原图只恢复array shape，不会自动修复：

- relative scale/shift；
- resized image与camera intrinsics关系；
- RGB-depth registration；
- thin structure的双线性混合；
- transparent/reflective surface；
- invalid/uncertainty mask；
- temporal staleness。

7.4 License也属于Artifact

固定README声明Small model为Apache-2.0，Base/Large/Giant models为CC-BY-NC-4.0；code LICENSE为Apache-2.0。部署或分发必须分别冻结source、checkpoint hash与checkpoint license。

8. Mask+Metric Depth→3D Surface

8.1 Pixel Backprojection

对original pixel (u, v)、metric depth d与intrinsics K：

$$X_c = d\,K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}.$$

变量解释：X_c是camera-frame 3D point，d必须与该original pixel、camera、timestamp和intrinsics匹配。直觉上，depth unit或pixel transform错一点，3D位置会沿ray整体漂移。实现映射：Depth Anything output还需unit/range/validity，随后才进入PER-002的TF与target frame。

8.2 Mask-conditioned Surface

$$\mathcal{P}_M = \left\{ d(u,v) K^{-1} [u, v, 1]^\top \mid M(u,v) = 1,\ d(u,v) \text{ valid} \right\}.$$

变量解释：M是选定instance/prompt mask，只有valid depth pixel进入surface。边界pixel若混入background depth会产生飞点；应记录erode policy、boundary band、robust statistic、outlier removal与保留比例。

8.3 Weakest-link反例

若mask完全正确，但depth整体scale×2，则所有3D surface point沿ray距离×2；2D IoU仍可满分，机器人接触点却完全错误。这说明“segmentation高分”不能替代3D和action evaluation。

9. kPAM：Semantic Keypoint是可解释Task State

9.1 四段Factorization

```
instance segmentation
  -> semantic 3D keypoint detection
  -> optimization-based robot action planning
  -> local dense-geometry execution
```

kPAM不是把物体压成若干无名点，而是用`mug_handle`、`rim`等semantic anchors表达category-level task geometry。相同坐标若semantic ID不同，规划含义就不同。

9.2 Planning Contract

$$T^* = \arg\min_{T} \sum_{j} w_j c_j(T k_j) \quad \text{s.t.} \quad g_r(T k_{1:J}) \le 0.$$

变量解释：$k_j \in \mathbb{R}^3$是semantic keypoint，T是candidate transform，c_j是几何cost，g_r是关系/可行性constraint，w_j可结合visibility/covariance。实现映射：具体detector、private dependency、planner和execution需逐revision审计；公式只表达已冻结的几何合同。

9.3 最小Record

```
{
  "semantic_id": "mug_handle",
  "instance_id": "...",
  "category": "mug",
  "coordinates": [0.1, -0.2, 0.7],
  "frame": "camera | base | object",
  "visibility": true,
  "covariance": [[0.0, 0.0, 0.0]],
  "timestamp": "..."
}
```

只有`[[x,y]]`无法支持category-level 3D planning。

10. Where2Act：Affordance是条件成功分布

10.1 Model Components

固定3D model包含：

- PointNet++ point feature；
- ActionScore：每点actionability；
- Actor：给定point feature提出6D orientation/action；
- Critic：评估point+orientation interaction success；
- simulation interaction与online sampling，面向articulated object的push/pull。

10.2 Score Semantics

$$A(p, R \mid \pi, s, e) = \Pr(\text{success} \mid p, R, \pi, s, e).$$

变量解释：p是contact point，R是orientation，π是primitive，s是object/environment state，e是训练/eval protocol。直觉上，同一点对push与pull、不同gripper或door state的actionability可相反。实现映射：固定score不自动跨robot/primitive/domain校准。

10.3 Proposal不是Controller Action

从affordance proposal到执行仍缺：3D frame、tool/end-effector transform、IK、reachability、collision、force/trajectory、controller rate、replan与recovery。PER-012止于proposal；PER-015负责action gate。

11. 端到端Typed Composition

```
language / point / box prompt
  -> SAM candidate masks OR Mask2Former task-specific segments
  -> explicit instance/category selection
  -> metric Depth Anything checkpoint OR registered sensor depth
  -> valid mask-conditioned 3D surface
  -> semantic keypoint detector + covariance/frame
  -> primitive-conditioned contact/orientation proposals
  -> planner/IK/collision/controller
  -> episode success/failure trace
```

每个artifact都要保存upstream ID：mask必须能追到prompt/model，surface能追到mask/depth/camera，keypoint能追到surface/semantic model，affordance能追到primitive/state，action能追到planner/controller revision。否则无法定位first divergence。

12. Objective与Metric对照

输出	训练/预测单元	代表Metric	不等价于
semantic mask	class per pixel/query mixture	mIoU/pixel accuracy	instance AP、PQ、robot contact
instance mask	ranked instance masks	mask AP/AR	semantic mIoU、tracking
panoptic	non-overlap segments	PQ/SQ/RQ	arbitrary overlap proposals
promptable mask	prompt-conditioned candidate	mask IoU/quality	semantic/task success
relative depth	aligned/rank structure	aligned AbsRel/δ1，绑定alignment	meter error
metric depth	valid metric pixels	AbsRel/RMSE/δ threshold	3D/robot success
semantic keypoint	named visible anchors	PCK/3D distance，绑定frame	naked point hit
affordance	point+orientation+primitive	interaction success/ranking	objectness、reachability

任何数字都继承PER-010 protocol identity：dataset/split、checkpoint、preprocess、task、postprocess、matching、valid denominator、threshold、aggregation和sample unit。

13. Failure Taxonomy与诊断顺序

Failure	表面症状	首查字段	最小Probe
task mismatch	同类对象被合并	semantic/instance/panoptic	two-mug case
prompt ambiguity	SAM候选不稳定	prompt+multimask policy	same-click candidates
boundary leakage	surface出现飞点	mask edge/depth discontinuity	erode vs raw surface
depth scale	方向对、距离错	relative/metric/checkpoint/range	known-distance plane
registration	RGB对、depth偏	camera IDs/extrinsics/timestamp	RGB-depth edge overlay
keypoint swap	rim/handle互换	semantic ID/category mapping	same-coordinate labels
keypoint uncertainty	planner抖动	visibility/covariance	occlusion sweep
primitive mismatch	pull点用于push	primitive/state/model ID	same-point push/pull
frame/time	proposal正确、执行偏移	TF/timestamp/age	roundtrip+latency trace
controller reject	高score但不可达	IK/collision/tool limits	planner rejection reasons

推荐顺序：task/schema→pre/postprocess→model/checkpoint→depth unit/validity→camera/frame/time→semantic identity→primitive/state→planner/controller→episode metric。不要在depth unit未知时先调IK。

14. Fail-closed Pseudocode

```
def propose_action(image, camera, prompt, primitive, models, policy):
    # 1. Every model and transform must have an immutable identity.
```

```
    assert models.segmenter.sha256 and models.depth.sha256
    assert camera.intrinsics_sha256 and camera.timestamp is not None

    # 2. Keep segmentation task semantics; never collapse all outputs into "mask".
    segments = models.segmenter.predict(image, prompt=prompt)
    target = select_segment(segments, task=policy.segmentation_task)
    if target is None or target.score < policy.mask_threshold:
        return reject("no_valid_segment")

    # 3. 3D lift requires metric depth, valid pixels and matching original-image geometry.
    depth = models.depth.predict(image)
    assert depth.unit == "meter" and depth.checkpoint_family == "metric"
    valid = target.mask & depth.validity_mask
    if valid.sum() < policy.min_surface_pixels:
        return reject("insufficient_metric_surface")

    # 4. Backproject and transform at the same camera timestamp.
    surface = backproject(valid, depth.values, camera.intrinsics)
    surface = transform_at_timestamp(surface, camera.tf_revision, camera.timestamp)

    # 5. Semantic keypoints keep identity, visibility and covariance.
    keypoints = models.keypoint.predict(target, surface)
    if any((not k.visible) or k.covariance > policy.max_cov for k in keypoints):
        return reject("uncertain_semantic_keypoint")

    # 6. Affordance proposals are conditioned on primitive and current state.
    proposals = models.affordance.propose(surface, primitive, camera.robot_state)

    # 7. Perception scores never bypass reachability, IK, collision or controller gates.
    feasible = planner_filter(proposals, keypoints, camera.robot_state)
    return feasible if feasible else reject("no_feasible_action")
```

这段是目标合同，不是本地实现事实。真实系统还需checkpoint/license、depth uncertainty、mask candidate policy、tool geometry、planner/IK/controller revision与episode logger。

15. 本地OPD：名字是Affordance，输出仍是Point

15.1 真实链

```
ShareRobot affordance {x,y} / trajectory points
  -> scale to 0..1000
  -> prompt: actionable affordance location
  -> target: <point>[[x,y], ...]</point>
  -> point/box/text parser and static metric
```

15.2 五文件Audit

1. `build_prompt_pool.py` 把 `affordance.x/y` 变成单个 `gt_points` 与 `task_type=affordance_pointing`；无primitive/orientation/state/success head。
2. `point_data_only.py` 最终调用 `point_answer(scaled)`；prompt语义不改变representation。
3. `build_expert_mixes_seed0.py` 与 `eval_qwen_vl_raw_holdout.py` 只有point/box/text route/parser/metric。
4. 五个固定文件对 `keypoint/segment_anything/sam_model/mask_rle/polygon/<mask>/depth` 均为0匹配。

静态zero-match只覆盖选定路径，不排除远端、历史或未归档artifact。但在证据出现前，安全结论是“affordance-labelled point supervision”，不是dense perception stack。

16. Project Truth Boundary

项目	当前可以挂钩	当前不能说	补强Artifact
OPD	affordance-labelled point、point/box parser/static holdout	mask/depth/keypoint/dense affordance/robot result	model/checkpoint/raw dense outputs、3D/episode bundle
WAM	可讨论future frame是否保存contact geometry	已训练seg/depth auxiliary或其提升	local head/config/loss curve/matched eval
Robot Harness	typed perception result可进入tool/scope/cancel gate	已运行视觉模型或真实3D controller	perception server、controller trace、episodes
OpenPiVLA	end-to-end image→action可比较显式dense intermediate	已集成SAM/depth/keypoint	code/checkpoint/ablation/rollout
UWM	future observation可作为geometry diagnostic	learned affordance map或real transfer	raw predictions/checkpoints/episodes

公共方法只能证明机制可解释，不证明已被个人项目集成。Git文件存在也不能替代候选人逐文件ownership确认。

17. 面试回答模板、追问与危险答案

17.1 两分钟结构化回答

> 我把操作感知拆成task-typed segmentation、metric geometry、semantic anchors和action-conditioned proposal。Mask2Former可统一semantic/instance/panoptic architecture，但各分支postprocess与metric不同；SAM给prompt-conditioned mask candidates和quality，不自带类别或robot-success校准。Depth Anything V2的relative路径是ReLU dense map，metric路径是独立Sigmoid×max-depth代码和checkpoint，resize回原图不等于恢复米制几何。只有metric depth与mask、内参、时间对齐后才能形成3D surface。kPAM式keypoint还要semantic ID、visibility、frame；Where2Act式affordance绑定point、orientation、primitive和state。最后必须过IK、collision和controller。本地OPD目前只是affordance-labelled point target，不能称完整感知栈。

17.2 高频追问

1. semantic、instance、panoptic和promptable mask分别解决什么？
2. Mask2Former为什么能用同一架构做三种task，但metric仍不同？
3. Hungarian assignment与mask loss怎样分工？
4. SAM multimask该怎样选？predicted quality代表什么？
5. relative depth为何不能直接反投影？
6. `max_depth`改变会怎样影响metric output？
7. resize回原图为何不等于几何对齐？
8. mask边界为何容易产生3D飞点？
9. semantic keypoint比普通point多哪些字段？
10. kPAM如何把keypoint变成planning cost/constraint？
11. Where2Act的ActionScore、Actor和Critic各负责什么？
12. affordance为何必须绑定primitive和state？
13. proposal高分为何仍不能执行？
14. OPD的affordance point和Where2Act有什么本质差别？

17.3 危险答案与修正

危险答案	问题	安全修正
“有mask就知道是哪个物体”	prompt mask未必有class/instance ID	冻结task与binding
“SAM quality高就能抓”	score未对robot success校准	加task validation和action gate
“Depth Anything输出就是米”	relative/metric路径混用	绑定metric checkpoint、range和unit
“resize回原图就对齐相机”	array shape≠camera registration	对齐intrinsics/extrinsics/time
“一个point就是keypoint”	缺semantic ID/frame/visibility	用typed keypoint record
“actionable point就是affordance model”	缺primitive/orientation/state/success	明确point supervision边界
“affordance score高就执行”	无IK/collision/controller	fail-closed planner gate
“我实现了完整OPD操作感知”	selected code只有point/box/text	只陈述直接文件证据与ownership缺口

18. 白板Checklist与练习标准

能够不看材料完成：

- segmentation四类task与输出/metric；
- Mask2Former class+mask query、assignment和三项loss；
- SAM prompt/multimask/quality边界；
- relative/metric/sensor/aligned depth四类合同；
- Depth Anything V2 relative与metric代码差异；
- mask+metric depth反投影与边界飞点；
- semantic keypoint record和kPAM planning公式；
- Where2Act ActionScore/Actor/Critic与条件score；
- 完整typed composition和failure诊断顺序；
- OPD point-only真实性边界；
- 两分钟回答与至少六个追问。

`drafted`不等于掌握。只有真实口述、追问与纠错后才能改变题卡状态；本轮不创建`coaching_state.md`。

19. 结论与下一步

Dense perception对机器人有价值的前提不是“模型更强”，而是task、unit、frame、time、score和downstream gate全都可追溯。Segmentation提供区域，metric depth提供几何，semantic keypoint提供可解释task state，primitive-conditioned affordance提供候选交互；任何一层语义或坐标漂移都会让后续模块失效。

本地OPD当前最准确的结论仍是affordance-labelled point supervision和point/box/text静态评估；没有证据支持SAM/Mask2Former、relative/metric depth、semantic keypoint、Where2Act式head或robot闭环。

`VLA-PER-012` source/knowledge/card已在独立bounded slices同步，题卡保持`unseen`。下一bounded slice只冻结`VLA-PER-013` point-cloud/voxel/BEV/occupancy representation source；不生成knowledge/card，不创建`coaching_state.md`，不改变任何mastery。

References

1. PER-012 Segmentation / Depth / Keypoint / Affordance Source Freeze：固定论文/代码revision、SHA、license、本地OPD audit与deterministic probes。
2. Cheng et al., Mask2Former v3；official repository@9b0651c。
3. Kirillov et al., Segment Anything v1；official repository@dca509f。
4. Yang et al., Depth Anything V2 v2；official repository@a561b84。
5. Gao & Tedrake, kPAM v2；official repository@7214936。
6. Mo et al., Where2Act v2；official repository@1daf1a2。
7. PER-002 3D Lift / Frame、PER-008 Representation Choice、PER-010 Metric / Parser 与 PER-011 Teacher Noise / Calibration / Leakage。
8. OPD Claim–Evidence Inventory。

Part 13 / 43 · `interview_materials/vla/knowledge/vol_06_per013_pointcloud_voxel_bev_occupancy.html` · SHA-256 `E8097EAA42EA9EABB34BD5B0B730C1AA0241A8E8DA0AB974BFFE92E110168A79`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`e8097eaa42ea9eabb34bd5b0b730c1aa0241a8e8da0ab974bffe92e110168a79`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per013_pointcloud_voxel_bev_occupancy.md`

VOL 06 Perception / Grounding / Navigation：Point Cloud、Voxel、BEV、Occupancy 与 Sampling（PER-013）

状态：`drafted / VLA-PER-013 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-002` 2D→3D lift、`VLA-PER-003` multi-view/time、`VLA-PER-012` dense perception；后续：`VLA-PER-014` 3D semantic grounding、`VLA-PER-015` perception→action gate。 来源入口：PER-013 Point Cloud / Voxel / BEV / Occupancy Source Freeze；对应自测：PER-013 Atomic Card。本页只把冻结证据转成可教学知识；题卡已生成并保持 `unseen`，不创建训练状态，也不改变任何 mastery。

问题（Question）

面试官问“点云、voxel、BEV 和 occupancy 怎样选，sampling 怎样影响结果”时，不能只回答“点云精细、voxel 规则、BEV 高效”。需要从 downstream query 反推表示：连续坐标、局部几何、高度、空闲空间、unknown、语义身份分别要保留什么；frame、origin、range、resolution、axis、time 怎样定义同一个 tensor；FPS、ball query、voxel cap 和 aggregation 怎样改变模型身份；稀疏计算为何不是免费午餐；以及本地项目到底只有地图/PCD 接口，还是已有可复现的 3D encoder 与机器人闭环。

本节点建立的主链是：

```
metric 3D samples + frame/time/schema
  -> point-set sampling and grouping
  -> voxel quantization and aggregation
  -> sparse 3D coordinates + features
  -> pillar or height-compressed BEV
  -> explicit / octree / implicit occupancy
  -> PER-014 semantic 3D grounding
  -> PER-015 reachability / collision / controller gate
```

知识地图位置与边界

相邻节点	该节点负责	PER-013负责
PER-002	pixel→camera/base 3D、intrinsics、TF	已有 metric 3D sample 如何编码与量化
PER-003	multi-view/time identity 与 fusion	representation artifact 保留 source/time lineage
PER-008	point/box/mask/keypoint/affordance 的选择	point/voxel/BEV/occupancy 内部合同与转换损失

相邻节点	该节点负责	PER-013负责
PER-012	segmentation、metric depth、surface、keypoint、affordance	surface 之后的 3D representation
PER-014	3D semantic/open-vocabulary grounding	本页不产生 language-region binding 或 3D box
PER-015	reachability、IK、collision、controller	本页不从 occupancy score 推出可执行动作
PER-016	localization、SLAM 与 map drift	OctoMap 只作 representation exemplar

1. 30秒直觉：表示不是形状，而是物理合同

一个可靠的 30 秒回答是：

> 我先从 downstream query 选表示。Point cloud 保留连续坐标，但邻域不规则；PointNet++ 用 FPS、ball query/kNN 和对称 pooling 建 local hierarchy。Voxel 把坐标按 origin、range、axis、voxel size 做 floor 量化，换来规则邻域，同时引入边界跳变、碰撞和 cap/drop。Sparse tensor 只计算 active coordinates，但 coordinate map、kernel map 和索引也有成本。Pillar/BEV 把高度变成 point feature 或 channel，并不天然丢掉全部 z；多传感器 BEV 还必须校准到同一物理平面。Occupancy 要区分 dense/sparse grid、概率 octree 和 implicit function，unknown 不能当 free。最终 occupancy 或 BEV 仍不是 semantic grounding、collision safety 或 robot success。

这段回答先封住六个偷换：

1. 同一个 `[i,j,k]` 被默认在不同 frame/origin 下表示同一位置；
2. FPS 被默认无随机性、无起点依赖且总优于 random；
3. MeanVFE 被默认与 learned VFE 信息等价；
4. “sparse”被默认总是更快、更省显存；
5. BEV 被默认完全删除高度或自动完成传感器对齐；
6. occupancy probability 被默认等于可碰撞、可导航或可执行。

2. Typed 3D Representation Manifest

2.1 最小 Schema

```
{
  "sample_id": "...",
  "source_ids": ["camera/depth/lidar/frame ids"],
  "timestamp_range": ["t0", "t1"],
  "coordinate_frame": "camera | base | map",
  "axis_convention": "x-forward/y-left/z-up",
  "unit": "meter",
  "representation": "points | voxels | pillars | bev | occupancy_grid | octree | implicit_occupancy",
  "spatial_range": [[-1, 1], [-1, 1], [0, 2]],
  "origin": [0, 0, 0],
  "resolution_or_voxel_size": [0.02, 0.02, 0.02],
  "point_feature_schema": ["x", "y", "z", "intensity"],
  "sampling": {"method": "fps", "seed": 0, "budget": 4096},
  "aggregation": "max | mean | vfe | sparse_conv",
  "empty_unknown_policy": "unknown != free",
  "model_revision": "sha256:...",
  "validity_and_age": {"valid_ratio": 0.0, "age_ms": 0}
}
```

2.2 为什么 Manifest 属于模型身份

只有 `tensor.shape`，没有 frame、origin、resolution、range 与 axis，就不知道同一个 index 对应哪个物理位置；只有 point count，没有 sampling seed、input order 与 feature schema，就无法复现输入。对于 BEV，还必须记录 z aggregation、meters-per-cell 与 rotation augmentation；对于 occupancy，还必须记录 observed/unknown/free/occupied 的编码、阈值与更新时间。

推荐把 representation hash 计算在“原始来源 ID + frame/time + quantization/sampling config + model revision”之上。这样离线评估、线上日志和失败样本才能追到同一物理合同。

3. Representation 选择总览

表示	保留	主要损失/成本	适合回答	不能自动回答
unordered points	连续坐标、per-point feature	不规则邻域、sampling、带宽	surface/local geometry query	free space、语义、可执行性
dense voxel	规则 3D 邻域	$O(N_xN_yN_z)$ memory、量化	dense 3D feature/collision grid	unknown 语义、动作安全
sparse voxel	active coordinate + feature	index/kernel map、active-pattern drift	稀疏 3D backbone	一定更快、更省
pillar	xy cell + 聚合点特征	z spatial bin 折叠、collision	快速 2D backbone/BEV	所有高度关系
height-compressed BEV	z slice→channel	2D convolution 的归纳偏置	shared planar query	自动 calibration
probabilistic octree	multi-resolution log odds	sensor model/staleness	ray-consistent map	learned surface/semantics
implicit occupancy	continuous query function	query/threshold/calibration	reconstruction/surface	sensor-ray free space、collision guarantee

选择原则不是“谁最先进”，而是“下游要问什么”。接触点需要局部曲面和法向；导航需要 robot footprint 下的 free/occupied/unknown；语言 grounding 需要实例与语义；控制还需要时间、uncertainty、reachability 与 dynamics。一个 feature map 不会因为名字叫 BEV 就同时满足这些合同。

4. Point Set：Permutation Invariance 不等于 Pipeline Invariance

点集可写为：

$$\mathcal{P} = \{(\mathbf{x}_i, \mathbf{f}_i)\}_{i=1}^{N}, \qquad \mathbf{x}_i \in \mathbb{R}^3.$$

变量解释：$\mathbf{x}_i$ 是连续坐标，$\mathbf{f}_i$ 可含 RGB、intensity、normal、timestamp 或 learned feature。直觉：点的排列本身不应改变几何语义，所以 PointNet 用共享 MLP 与 symmetric reduction；但 pipeline 仍会受 input order、采样起点、邻域截断、浮点排序与 tie-break 影响。实现映射：固定 PointNet++ 代码的 set abstraction 先 sampling，再 grouping，再 local PointNet，而不是对全体点一次 max pool。

因此应区分：

- **函数层 permutation invariant**：同一无序集合经理想对称聚合，结果不依赖排列；

- 数据管线不一定 **invariant**：random/FPS 初始化、voxel cap、邻域前 K 个、duplicate handling 都可能依赖顺序；
- 物理坐标不等于语义：点仍需 frame、unit、timestamp 与 source lineage。

5. PointNet++ Set Abstraction

一层 set abstraction 可抽象为：

$$\mathbf{h}_j = \mathrm{POOL}_{i \in \mathcal{N}(\mathbf{c}_j)} \, \phi([\mathbf{x}_i - \mathbf{c}_j, \mathbf{f}_i]).$$

变量解释：$\mathbf{c}_j$ 是 sampled centroid；$\mathcal{N}(\mathbf{c}_j)$ 是 ball query 或 kNN 邻域；ϕ 是 shared PointNet/MLP；POOL 通常是 max 等 symmetric reduction。直觉：先用 centroid 覆盖空间，再在 metric-local neighborhood 中学习相对几何。实现映射：固定 PointNet++ SSG 代码明确使用 512 个 centroid、radius 0.2、K=32，再用 128、radius 0.4、K=64；这些超参共同定义 receptive field。

不能省略的模型身份包括：

1. input order 与 FPS 初始点/seed；
2. centroid 数量、radius、K；
3. ball query 或 kNN；
4. 空邻域、重复 neighbor 与不足 K 的 padding；
5. relative coordinate normalization；
6. point feature schema、pooling 和 multi-scale/density policy。

6. Sampling：Coverage、Density 与 Bias

6.1 FPS

Farthest Point Sampling 的一步是：

$$i_t = \arg\max_i \min_{s<t} \|\mathbf{x}_i - \mathbf{x}_{i_s}\|_2.$$

变量解释：i_t 是第 t 个新点，已选集合为 $\{i_s\}_{s<t}$。直觉：每次选择离已有集合最远的点，倾向扩大 coverage。实现映射：固定 PointNet++ 实现维护每点到已选集合的最小距离并反复 argmax；第一个 centroid 与 tie-break 会影响最终集合。

FPS 不是“无偏代表性采样”。它对孤立点、outlier 和坐标尺度敏感，精确实现近似 $O(NM)$；在非均匀密度下 coverage 改善也不等于任务 accuracy 改善。

6.2 Ball Query 与 kNN

邻域	固定量	会漂移的量	适用直觉	风险
ball query	metric radius r	neighbor count	保留物理尺度	稀疏区可能空、稠密区被 K 截断
kNN	neighbor count K	metric radius	tensor shape 稳定	密度变化使 receptive field 尺度漂移

多尺度 grouping 能缓解 density variation，但增加计算，也不会自动消除传感器距离、遮挡和采样模式带来的 domain shift。

6.3 必报诊断

至少报告 points kept/dropped、coverage radius、nearest-neighbor density strata、每邻域有效点分布、empty/duplicate ratio、sampling seed 与 run variance。只报告 “4096 points” 不足以复现。

7. Voxelization：规则邻域来自离散量化

对 origin $\mathbf{o}$ 与 voxel size $\mathbf{v}$：

$$\mathbf{q}_i = \left\lfloor \frac{\mathbf{x}_i - \mathbf{o}}{\mathbf{v}} \right\rfloor .$$

变量解释：$\mathbf{q}_i$ 是 integer voxel coordinate；$\mathbf{o}$ 决定 grid 相位；$\mathbf{v}$ 决定各轴 resolution。直觉：把连续点映射到规则 cell，便于 3D convolution，却牺牲 cell 内精确位置。实现映射：MinkowskiEngine 量化路径会处理负坐标 floor 并生成 unique/inverse map；OpenPCDet 配置同时固定 `POINT_CLOUD_RANGE`、`VOXEL_SIZE`、`MAX_POINTS_PER_VOXEL` 和 train/test max voxels。

7.1 四个常见边界

1. **origin/range/axis**：同一世界点在不同原点或轴顺序下 index 不同；
2. **floor/round**：负坐标与边界点归属会改变；
3. **boundary flip**：$x = 0.99$ 与 1.01 在 $v = 1, o = 0$ 时落入 0/1；
4. **cap/drop**：每 voxel 最大点数和全局最大 voxel 数会丢弃数据，且可能受顺序/seed 影响。

训练与部署若只对齐 voxel size、没有对齐 range/origin/axis/floor，得到的是不同模型输入。

8. Aggregation：MeanVFE 与 Learned VFE 不等价

固定 OpenPCDet `MeanVFE` 可写为：

$$\mathbf{f}_v = \frac{1}{\max(n_v, 1)} \sum_{i \in v} \mathbf{f}_i .$$

变量解释：n_v 是 voxel 中的 valid point count，$\mathbf{f}_i$ 是被聚合的 point feature。直觉：mean 是简单稳定的 permutation-invariant baseline，但会把不同集合压成同一向量。实现映射：集合 $[0, 2]$ 与 $[1, 1]$ 的 mean 都是 1；若不保留 count、local offset、variance 或 higher moments，二者不可区分。

VoxelNet 的 stacked VFE 会编码 point-wise feature 与 locally aggregated feature，并且 fixed-T voxel 内可能 random sample；PointPillars 也有 point-wise decorated features。它们不能因为最后都得到一个 cell feature 就统称为 “mean pooling”。

面试中应主动说出三种 alias：

- quantization collision：不同连续位置落入同 cell；
- aggregation collision：不同 cell 内集合得到同 feature；
- truncation collision：cap 后不同原始集合变成相同保留子集。

9. Sparse Tensor 与 Sparse Convolution

Sparse tensor 由 active coordinates $\mathcal{C}$ 与 features $\mathcal{F}$ 组成，batch index 也是 coordinate 的一部分。一般化 sparse convolution 可抽象为：

$$\mathbf{y_u} = \sum_{\mathbf{k} \in \mathcal{K}(\mathbf{u}, \mathcal{C}_{\text{in}})} \mathbf{W_k} \mathbf{x_{u+k}}.$$

变量解释：$\mathbf{u}$ 是 output coordinate；$\mathcal{K}$ 只保留 kernel offset 与 active input coordinate 的交集；$\mathbf{W_k}$ 是对应 kernel weight。直觉：避免在大量 empty cell 上做 dense multiply，但要先维护 coordinate 与 kernel mapping。实现映射：固定 OpenPCDet backbone 同时出现 `SubMConv3d`、`SparseConv3d` 与 `SparseInverseConv3d`，它们的 output coordinate set 与 stride 行为不同。

9.1 三类算子不可合并

算子	active pattern	典型作用	风险
SubMConv	通常只在原 active sites 输出	保持稀疏形状的局部特征	信息传播受 active topology 限制
SparseConv	stride/kernel 可产生新 output sites	下采样与扩大 receptive field	active count 与 coordinate map 改变
InverseConv	用对应 mapping 恢复分辨率	decoder/upsampling	必须绑定正确 indice/key

9.2 “Sparse 一定更快”为什么错

总成本还包括 coordinate dedup、hash/map、kernel map、index gather/scatter、allocator、feature width、batch fragmentation 与硬件支持。active ratio 很低且空间大时通常有利；场景很小、feature 很窄或 active pattern 不友好时，索引开销可能主导。应在同硬件、同 batch、同精度下报告 latency p50/p95、peak memory、active count 与 dense/sparse parity。

10. Pillar：折叠 z 的 Spatial Bin，不必删除 z Feature

PointPillars 把整个 vertical column 内的点组织成 pillar，用 PointNet 编码后 scatter 到 2D pseudo-image。若 z range 为 $[-3, 1]$，z voxel size 为 4，则 $z = -2.5$ 和 0.5 的 spatial z index 都是 0。

这意味着：

- z 不再作为多层 spatial grid index；

- per-point z、相对 pillar center 的 z、height statistics 仍可进入 feature；
- 聚合以后能否保留细粒度高度取决于 feature decoration、encoder 与 cap；
- 不能回答“pillar 完全不含高度”，也不能回答“pillar 无损保存 3D”。

固定 PointPillars/OpenPCDet 配置中的 range、voxel size、max points、scatter shape 与 backbone 都属于 artifact identity。

11. Height Compression 与 BEV

OpenPCDet 的一条 HeightCompression 路径将：

$$[N, C, D, H, W] \longrightarrow [N, C \cdot D, H, W].$$

变量解释：D 个 height slice 被 reshape 到 channel，而不是先做数值平均；H, W 是 BEV 平面。直觉：2D backbone 获得更高效率，同时高度只以 channel pattern 存在。实现映射：固定 `height_compression.py` 直接把 dense spatial feature 的 C, D 合并。

BEV artifact 至少要绑定：

- frame 与 x/y axis；
- origin、range、meters-per-cell；
- z range 与 aggregation/compression；
- ego/map pose 与 timestamp；
- augmentation matrix；
- invalid/unknown/padding policy。

二维数组相同，不代表物理平面相同。

12. BEVFusion：共享平面不是自动对齐

BEVFusion 的关键思想是 camera 与 LiDAR feature 先进入统一 BEV，再在 shared planar coordinates 上融合。camera 分支需要 view transform 与 BEV pooling，LiDAR 分支需要 voxelization，之后才是 fuser/head。

最小 calibration contract 包含：

```
camera intrinsics/extrinsics
LiDAR-to-ego transform
ego/map pose
image and point-cloud augmentation matrices
depth/view-transform bins
BEV origin/range/resolution
sensor timestamp, validity and age
```

把两张同 shape feature map 按 array index 相加，不构成 calibrated fusion。固定论文/代码的 nuScenes、Waymo、KITTI 结果也不能直接外推为桌面 manipulation、VLA 或本地 Robot Harness 的收益；必须有目标域 config、checkpoint、calibration、matched ablation 与 same-protocol result。

13. Occupancy：至少三种不同合同

类型	表示	unknown/free 关系	典型用途	主要风险
dense/sparse explicit grid	cell→state/probability/logit	必须显式编码	collision map、scene completion	cubic memory、unobserved 误当 free
probabilistic octree	allocated node + log odds	未分配通常 unknown	multi-resolution mapping/raycast	sensor model、clamp、staleness
implicit occupancy	$o_\theta(\mathbf{p}, \mathbf{z})$	classifier 语义，不自带 ray-free	reconstruction/surface query	query cost、threshold、calibration

“Occupancy model”若没有说是哪一种，答案仍不完整。

14. OctoMap：Observed Free 与 Unknown 必须分开

概率 octree 的 log-odds update 可抽象为：

$$L_t(n) = \mathrm{clip}(L_{t-1}(n) + \lambda(z_t, n), L_{\min}, L_{\max}).$$

变量解释：$L_t(n)$ 是 node n 的 occupancy log odds；$\lambda(z_t, n)$ 根据 sensor ray 中的 free cell 或 endpoint hit 使用不同增量；上下界防止置信度无限累积。直觉：free 是“被射线穿过的观测证据”，unknown 是“没有足够观测/未分配”，二者不能合并。实现映射：固定 OctoMap code 先计算 `free_cells` 与 `occupied_cells`，再分别以 miss/hit 更新 node。

Octree resolution、hit/miss probability、occupancy threshold、clamping、max range、ray origin、time decay 与 dynamic-object policy 都会改变地图语义。Source freeze 只用固定 official code 支撑这一机制，没有可靠冻结 OctoMap 论文 PDF，也没有本地 OctoMap integration。

15. Occupancy Networks：连续 Query 与 Surface Threshold

Implicit occupancy 可写为：

$$o_\theta : \mathbb{R}^3 \times \mathcal{Z} \to [0, 1], \qquad \mathcal{S}_\tau = \{\mathbf{p} \mid o_\theta(\mathbf{p}, \mathbf{z}) = \tau\}.$$

变量解释：$\mathbf{p}$ 是任意 3D query point，$\mathbf{z}$ 是 condition/latent，τ 是 surface decision threshold。直觉：网络没有存储一张固定分辨率网格，而是按坐标查询 inside/outside probability；surface 是某个等值面。实现映射：固定 occupancy-networks generation 默认 probability threshold 0.5，对应 logit 0；阈值改为 0.7 时 logit 约 0.8473，抽取出的 surface 会改变。

MISE 能自适应细化 query 以减少 dense evaluation，却不会自动提供：

- sensor-ray observed free space；
- unknown 与 unobserved policy；
- dynamic occupancy；

- calibrated collision probability；
- robot footprint、swept volume 或控制安全。

因此 implicit occupancy score 不能直接作为 collision gate。

16. Memory 与 Compute Scaling

Dense grid feature memory 的下界是：

$$M_{\text{dense}} = N_x N_y N_z \, C \, b.$$

变量解释：N_x, N_y, N_z 是 cell 数，C 是 feature channels，b 是每个 feature 的 bytes。直觉：range 每轴扩大 2 倍，dense 3D cell 数最多扩大 8 倍；resolution 每轴减半同样会带来 8 倍 cell。实现映射：$200^3 \times 64 \times 4 = 2{,}048{,}000{,}000$ bytes，约 1.907 GiB；若 active ratio 为 1%，仅 active feature 约 19.53 MiB。

后一个数字不包括 coordinates、kernel maps、allocator、activations、gradients 与 optimizer state，所以不能据此宣称“节省 99% 总显存”。报告表示效率时至少需要：

类别	必报字段
geometry	range、resolution、active count/ratio、point count
loss	quantization residual、collision/cap/drop、unknown coverage
runtime	hardware、batch、precision、warmup、p50/p95 latency
memory	feature-only estimate、measured peak、training/inference
task	same split/protocol 下的 downstream metric

17. Conversion Loss 与 Downstream Sufficiency

```
metric points
  -> sample/group                # sparse region may be dropped
  -> quantize + aggregate        # boundary, collision, cap, alias
  -> sparse 3D features          # coordinate/kernel contract
  -> pillar / height compression # z loss differs by path
  -> BEV fusion                  # calibrated common frame required
  -> occupancy map/function      # unknown/free/threshold semantics
  -> PER-014 semantic grounding  # object/language identity
  -> PER-015 action gate         # reachability/collision/controller
```

每次转换都应保存 input artifact ID、config/hash、output schema 和 loss diagnostics。建议用 round-trip probe：从 point→voxel index→voxel center，检查 quantization residual；从 sensor ray→occupancy，检查 observed free 与 endpoint occupied；从 3D point→BEV cell→physical center，检查 frame/origin/resolution。

17.1 Metric 边界

输出	可报告	不等价于
sampled points	coverage radius、retention、density strata	unbiased sampling、task accuracy
voxel/pillar	active count、collision/cap/drop、residual	3D detection 或 action success
sparse feature	active pattern、latency、memory、dense parity	总是更快、semantic correctness
BEV	round-trip、cell coverage、sensor validity	calibrated fusion、navigation success
explicit occupancy	observed-cell IoU、free error、unknown coverage	implicit surface 或 collision safety
implicit occupancy	occupancy IoU、Chamfer/F-score、query cost	ray-consistent free space、dynamic map

所有数字都要绑定 dataset/split、frame/range/resolution、sampling seed、aggregation、threshold、observed denominator、hardware 与 code/checkpoint revision。

18. Deterministic Probes：先验证合同，再跑大模型

Probe	固定输入	结果	只支持
FPS coverage	`[0,1,2,10]`，start=0，budget=2	FPS `[0,3]` radius=2；fixed random `[0,1]` radius=9	该例 coverage，不证明普遍 accuracy
voxel boundary	x=0.99/1.01，v=1，origin=0	index 0/1	0.02 平移可改离散 tensor
mean alias	`[0,2]` vs `[1,1]`	mean 均为 1	mean 需要 count/offset 等补充
dense/sparse estimate	$200^3, C = 64$, float32	dense 1.907 GiB；1% feature 19.53 MiB	不含 sparse overhead
pillar z collapse	range [-3,1]，dz=4	z=-2.5/0.5 均 index 0	spatial bin 相同，非 feature 全丢
implicit threshold	probability 0.5/0.7	logit 0/0.8473	threshold 属于 surface identity
local pixel/world	pixel (100,200)，固定 origin/res	world (-48.92593,35.5881134)	map transform 可复算
local PCD header	固定 `world.pcd`	54,856 点，六个 fields	asset/schema，不是 encoder
local path census	86 个 high-signal tracked 文本	78 third_party、8 data/docs/bridge/tests	vendored code 不等于个人实现

这些 probe 应进入 CI 或 artifact audit；它们验证 contract，不替代真实 dataset 与 robot evaluation。

19. Failure Taxonomy 与诊断顺序

Failure	表面症状	首查字段	最小 Probe
frame/origin drift	整图平移/旋转	frame、axis、origin、TF	physical round-trip
voxel boundary	小位姿变化导致 feature 跳变	floor、resolution、augmentation	0.99/1.01
sampling miss	稀疏小物体消失	seed、budget、density strata	coverage sweep
voxel cap/drop	稠密表面被截断	max points/voxels、order	shuffled-input parity
aggregation alias	不同局部几何同 feature	mean/max/VFE、count	`[0,2]` vs `[1,1]`
sparse pattern explosion	memory/latency 突增	operator、active sites、stride	layerwise active trace
pillar height alias	上下结构混淆	z feature/bin/compression	two-height probe
BEV misalignment	camera/LiDAR ghosting	calibration/augmentation/time	landmark overlay
unknown→free	未观测区可规划	occupancy encoding/ray update	occluded-cell test

Failure	表面症状	首查字段	最小 Probe
stale occupancy	动态障碍残留或消失	timestamp/decay/update	moving-object trace
implicit threshold drift	surface thickness 改变	probability/logit threshold	threshold sweep
controller rejection	occupancy 好但动作失败	footprint/IK/collision/controller	rejection reasons

推荐顺序：

```
schema/frame/unit/time
  -> sampling and range
  -> quantization/cap/aggregation
  -> sparse active pattern
  -> BEV calibration
  -> occupancy unknown/threshold/staleness
  -> semantic grounding
  -> planner/controller
  -> episode metric
```

不要在 frame 或 unknown policy 未知时先调网络深度。

20. Fail-closed Pseudocode

```
def encode_3d(points, manifest, cfg):
    # 1. Geometry without an explicit physical identity is rejected.
    require(manifest.coordinate_frame)
    require(manifest.unit == "meter")
    require(manifest.timestamp_range and manifest.source_ids)
    require(cfg.origin and cfg.spatial_range and cfg.voxel_size)

    # 2. Sampling is part of the artifact identity.
    sampled, sample_idx = sample_points(
        points,
        method=cfg.sampling_method,
        budget=cfg.point_budget,
        seed=cfg.seed,
    )

    # 3. Quantization reports every loss mechanism.
    coords = floor((sampled.xyz - cfg.origin) / cfg.voxel_size)
    valid = inside_range(coords, cfg.grid_shape)
    unique, inverse, counts = unique_coordinates(coords[valid])
    audit = report_collision_cap_drop(
        input_count=len(points),
        sampled_count=len(sampled),
        valid_mask=valid,
        unique=unique,
        counts=counts,
    )

    # 4. Representation semantics remain explicit.
    if cfg.representation == "sparse_voxel":
        features = aggregate(sampled.features[valid], inverse, cfg.aggregation)
        return SparseTensor(unique, features), audit

    if cfg.representation == "pillar_bev":
        require(cfg.bev_frame and cfg.meters_per_cell)
        pillars = aggregate_by_xy(sampled[valid], cfg)
        return scatter_to_bev(pillars, cfg.bev_frame), audit

    if cfg.representation == "occupancy":
        # An empty tensor cell is never silently interpreted as observed free.
        require(manifest.sensor_origin and cfg.unknown_policy)
        return update_occupancy(
            sampled,
            sensor_origin=manifest.sensor_origin,
            timestamp=manifest.timestamp_range[1],
        ), audit

    raise ValueError("unfrozen representation")
```

这是面试与目标实现合同，不是本地项目已有代码。真实系统还需 sensor model、calibration、TF/time、dynamic-object policy、uncertainty、footprint/swept volume、memory/latency 与 downstream episode logger。

21. Robot Harness：直接证据映射

本地 `agent@3f4c8b3` 有一个必须单独陈述的 direct exception：

Artifact	直接支持	不能支持
`occupancy2d_map.json/png`	2D tri-state map；0 occupied、127 unknown、255 free；约 0.02m resolution、origin/range	3D occupancy、OctoMap、地图 accuracy
`world.pcd`	54,856 点；`x y z intensity timestamp ring` schema	个人生成、online stream、PointNet/voxel inference
`ros2_bridge.py`	occupancy/pointcloud path、existence ready flag、URL 暴露	PCD 解析、voxelization、sparse convolution
scenario 文档	2D occupancy、PCD 引用、heightmap/A* 与 WebUI 设计说明	运行时等价、个人 ownership、机器人收益

其中 PCD SHA-256 为 `494A53320AF2EC10C4D1AADD17CE0F7D943820BA34536C23871BB5CA711F0B42`；bridge SHA-256 为 `8291D64DE01AAC2EFDF52409008EFD7AE18C5FEC0BA7B9D4677F6636F8287A48`。86 个 high-signal tracked 文本中 78 个位于 `third_party/`；依赖或 vendored code 的存在不等于候选人实现。

最安全的项目表述是：

> 我核对到 Harness 中有三态 2D occupancy 资产、带时间戳与 ring 字段的 PCD 文件，以及把路径/URL 暴露给前端的 bridge；这说明数据接口和仿真资产存在。当前没有证据表明第一方路径解析 PCD、做 voxel/BEV/OctoMap 或运行 learned occupancy，所以我不会把它讲成自研 3D perception stack。

22. 五项目 Project Truth Boundary

项目	当前可以讲	当前不能说	若要升级需补
Robot Harness	2D tri-state occupancy、PCD asset/schema、path/URL bridge	自研 PointNet/voxel/BEV/OctoMap、online 3D、个人 ownership	generator、ingest/voxel code、ROS bag、TF/time、planner trace
OPD	2D 标注点集合可转 box	3D point cloud、voxel/occupancy	z/frame/time/schema、3D checkpoint/eval
OpenPiVLA	image/language/action 可与显式 3D intermediate 比较	已集成 3D encoder 或其带来 97.4	module/config/checkpoint/ablation/episodes
WAM	future image 可作为 geometry diagnostic 候选	learned occupancy/BEV head	local head/loss/config/raw prediction
UWM	image/action/video world model 与 LIBERO protocol	point/voxel/occupancy 模型或 3D 泛化	model diff、3D data、checkpoint、rollout

OPD 代码中的 “point cloud(s)” 只指 2D 标注点集合经 min/max x/y 加 margin 转 box 或 prompt 文本，没有 z、frame、PCD、voxel、BEV 或 occupancy tensor。公共论文说明机制，项目文件说明资产；二者都不能替代 local integration、matched result 与候选人 ownership。

23. 面试回答模板、追问与危险答案

23.1 两分钟结构化回答

> 我会先从 downstream query 反推表示，并冻结 frame、axis、origin、range、resolution、time 和 feature schema。Point cloud 保留连续坐标，PointNet++ 用 FPS 选覆盖中心，再用 ball query 或 kNN 建邻域，local PointNet 和 symmetric pooling 得到 hierarchy；采样起点、radius、K 和 density policy 都属于模型身份。Voxel 用 floor((x-origin)/voxel_size) 量化，规则化邻域，但会有边界跳变、同 cell 碰撞、max-points/max-voxels drop；MeanVFE 与 learned VFE 不能混称。Sparse convolution 只计算 active coordinates，但 coordinate/kernel map 有成本，SubM、stride sparse 和 inverse conv 的 active pattern不同。Pillar 折叠 z 的 spatial bin，z 仍可能作为 point feature；另一种 BEV 会把 D reshape 到 channel。多传感器 BEV 必须绑定 calibration 和 timestamp。Occupancy 还要区分 explicit grid、OctoMap log-odds 与 implicit function，unknown 不等于 free，surface threshold 不等于 collision safety。本地 Harness 只有 2D map、PCD asset 和 bridge 的直接证据，不能声称我实现了这些 3D 模型。

23.2 高频追问

1. PointNet 为什么 permutation invariant，为什么完整 pipeline 仍可能依赖输入顺序?
2. FPS 的目标是什么? 起点、outlier 和复杂度有什么影响?
3. ball query 与 kNN 在 density shift 下分别怎么变?
4. voxel coordinate 为什么要用 origin、range、axis 和 floor 一起定义?
5. hard voxelization 与 dynamic voxelization 的主要边界是什么?
6. `[0,2]` 与 `[1,1]` 为什么暴露 MeanVFE alias?
7. SubMConv、SparseConv 和 InverseConv 的 output sites 有何差异?
8. 什么场景下 sparse convolution 未必比 dense 快?
9. PointPillars 是否完全丢失 z?
10. HeightCompression 把 D 放进 channel 有什么利弊?
11. BEVFusion 为什么仍需要完整 calibration?
12. empty voxel、unknown 和 observed free 有何不同?
13. OctoMap 如何利用 ray 的 miss/hit evidence?
14. implicit occupancy 的 0.5 threshold 改变会怎样?
15. occupancy IoU 为什么不等于 collision safety?
16. 你会怎样用 deterministic probes 检查 representation regression?
17. 本地 Robot Harness 到底支持哪些 3D 相关 claim?

23.3 危险答案与修正

危险答案	问题	安全修正
“PointNet 对点顺序完全不敏感”	sampling/cap/tie-break 可依赖顺序	区分 symmetric function 与 pipeline
“FPS 总比 random 好”	只保证某种 coverage 倾向	报 task、seed、coverage 与 matched result

危险答案	问题	安全修正
“voxel size 一样输入就一样”	origin/range/axis/floor 仍可不同	冻结完整 quantization contract
“MeanVFE 就是 VoxelNet”	learned VFE 与 mean 机制不同	指明具体 encoder/revision
“Sparse 一定节省 99% 显存”	忽略坐标/kernel/allocator	测 measured peak 和 latency
“Pillar 把高度全丢了”	z 可能仍在 point feature	区分 spatial z bin 与 feature
“BEV 天然对齐多传感器”	shared shape 不等于 shared frame	绑定 calibration/augmentation/time
“没有点就是 free”	无点可能是 occluded/unobserved	unknown 与 observed free 分离
“occupancy 0.9 就可以执行”	缺 footprint、uncertainty、controller	继续过 PER-015 gate
“Harness 已有 3D perception”	只有 asset/path direct evidence	只说 2D map、PCD schema、bridge

24. 白板 Checklist 与练习标准

能够不看材料完成：

- 画出 point→sample/group→voxel→sparse→pillar/BEV→occupancy 主链；
- 写出 typed 3D manifest 的 frame/origin/range/resolution/time；
- 推导 PointNet++ set abstraction 并解释 FPS、ball query、kNN；
- 写出 voxel floor 公式并演示 boundary flip；
- 用 `[0,2]` 与 `[1,1]` 解释 aggregation alias；
- 比较 SubM/Sparse/Inverse convolution 与 active pattern；
- 解释 pillar z bin 与 height-compression channel 的区别；
- 列出 BEVFusion 的 calibration contract；
- 比较 explicit grid、OctoMap 和 implicit occupancy；
- 算出 $200^3 \times 64 \times 4$ 的 memory；
- 走完十二类 failure 的诊断顺序；
- 说清 Robot Harness 与其余四项目的真实性边界；
- 完成两分钟回答并接住至少八个追问。

`drafted` 不等于掌握。只有真实口述、追问、评分与纠错后才能改变题卡状态；对应题卡已生成并保持 `unseen`，`coaching_state.md` 也不应创建。

25. 结论与下一步

Point、voxel、sparse tensor、pillar/BEV 与 occupancy 不是几种可随意替换的 tensor shape，而是不同的信息保留、物理坐标、unknown 语义与计算合同。可靠答案必须同时讲清 representation identity、conversion loss、runtime cost、metric boundary 与 downstream sufficiency。

本地项目目前最强的直接映射是 Robot Harness 的 2D 三态 occupancy 资产、54,856 点 PCD schema 和 path/URL bridge；没有证据支持自研 PointNet++、voxel encoder、BEVFusion、OctoMap、learned occupancy、online 3D perception 或其 robot benefit。

`VLA-PER-013` source、knowledge 与 card 已在独立 bounded slices 同步，题卡保持 `unseen/WARN`。后续 `VLA-PER-014` source与knowledge也已同步；下一 bounded slice只生成一张PER-014 atomic card并保持unseen，不启动PER-015，不创建 `coaching_state.md`，不改变任何 mastery。

References

1. PER-013 Point Cloud / Voxel / BEV / Occupancy Source Freeze：固定六篇 primary papers、六个 official repositories、九个 deterministic probes 与五项目 audit。
2. Qi et al., PointNet++ v1；official code@4292663。
3. Zhou & Tuzel, VoxelNet v1。
4. Lang et al., PointPillars v2。
5. Choy et al., Minkowski ConvNet v4；MinkowskiEngine@02fc608。
6. Liu et al., BEVFusion v3；official code@326653d。
7. Mescheder et al., Occupancy Networks v2；official code@406f794。
8. OpenPCDet@233f849 与 OctoMap@f012f5f。
9. PER-002 3D Lift / Frame、PER-003 Multi-view / Temporal、PER-012 Segmentation / Depth / Keypoint / Affordance。
10. Robot Harness Claim–Evidence Inventory 与 source freeze 中固定的 `agent@3f4c8b3` direct artifacts。

Part 14 / 43 · `interview_materials/vla/knowledge/vol_06_per014_3d_grounding_spatial_relation_open_vocabulary.html` · SHA-256 `E3622ADE1C28EF874D82535C1373C89A97454F04A89AF6382279F4DAE65AFA04`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`e3622ade1c28ef874d82535c1373c89a97454f04a89af6382279f4dae65afa04`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per014_3d_grounding_spatial_relation_open_vocabulary.md`

VOL 06 Perception / Grounding / Navigation：3D Grounding、Spatial Relation 与 Open-vocabulary 3D Scene（PER-014）

状态：`drafted / VLA-PER-014 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-007` 2D open-vocabulary grounding、`VLA-PER-013` point/voxel/BEV/occupancy representation；后续：`VLA-PER-015` reachability、IK、collision 与 controller gate。 来源入口：PER-014 3D Grounding / Spatial Relation / Open-vocabulary Source Freeze；对应自测：PER-014 Atomic Card。本页只把已冻结证据转成可教学、可诊断、可面试回答的知识；题卡已生成并保持 `unseen`，不创建训练状态，也不改变任何 mastery。

问题（Question）

面试官问“3D grounding 怎么做，spatial relation 与 open-vocabulary 3D scene 怎么落地”时，最危险的回答是把所有方法概括成“把 CLIP 特征投到点云上，再做相似度”。一个可靠回答必须先区分输出：是在 raw scene 中定位 3D box、在已分割对象中选 candidate、给每个点赋开放词表语义、返回 3D instance mask、查询连续 language field，还是查询增量 semantic map；再冻结 candidate、frame、relation arity、prompt、negative、normalization、threshold 与 metric；最后说明这些 semantic target 为什么还不是可执行机器人动作。

本节点的主链是：

```
PER-007 2D region-text semantics
        +
PER-013 metric 3D representation
        |
        v
candidate / point / mask / field / map identity
        -> language score and spatial relation
        -> output-specific metric and failure diagnosis
        -> semantic target with uncertainty
        |
        v
PER-015 reachability / IK / collision / controller gate
```

知识地图位置与边界

相邻节点	该节点负责	PER-014负责
PER-007	2D detector/segmenter、region-text score、prompt/threshold基础	把语言绑定到3D candidate、point、mask、field或map

相邻节点	该节点负责	PER-014负责
PER-013	point/voxel/BEV/occupancy的frame、量化、sampling、aggregation	3D entity identity、开放词表feature与spatial relation
PER-002/003	2D→3D lift、pose/frame/time与multi-view lineage	消费这些几何合同，不声称semantic score能修复错误pose
PER-016	localization、odometry、SLAM与map drift	本页只验证map/query接口，不负责闭合SLAM
PER-015	reachability、IK、collision、controller	本页输出semantic target，不从相似度直接推出action

PER-014不重复PointNet++、voxel、BEV或occupancy内部结构，也不把论文中的公有机制写成个人项目实现。

1. 30秒直觉：先问输出原子，再问模型

一个可靠的30秒回答是：

> 3D grounding不是单一任务。我先写清输出原子和候选来源：ScanRefer从场景proposal里选并回归3D box；ReferIt3D在已分割对象里做top-1 listener；OpenScene给每个3D点开放词表feature；OpenMask3D先有class-agnostic instance masks，再用多视图CLIP查询；LERF是per-scene多尺度连续language field；ConceptFusion是pose/depth约束下的可查询3D feature map。然后我冻结frame、relation arity、encoder/prompt/negative、normalization、threshold和metric。target不在候选集合时准确率上界就是0；point semantic也不会自动变成instance identity。最后这些输出仍要经过reachability、IK、collision和controller gate，不能把cosine或IoU直接叫robot success。

这段回答先封住八个偷换：

1. 3D box、candidate ID、point score、instance mask、field heatmap与map point被混成一种输出；
2. raw-scene detector与已分割对象listener被混成同一任务；
3. per-point semantic class被当作稳定instance identity；
4. target不在candidate set仍假设matching head可以恢复；
5. left/right不记录observer/world/object reference frame；
6. dot、cosine、softmax probability与relative relevance被统称confidence；
7. query heatmap被当作SLAM正确、可达或可抓；
8. public paper、local asset或Git身份被升级为个人实现与收益。

2. 六类输出原子

任务	输入前提	最小输出	典型方法	第一瓶颈	对应指标
3D referring localization	raw/encoded scene + utterance，可含proposal generator	3D box	ScanRefer	proposal recall、box geometry	Acc@3D-IoU 0.25/0.5、oracle recall
candidate-object identification	已分割3D objects + utterance	candidate ID / top-1	ReferIt3D	segmentation、distractor、relation frame	top-1、easy/hard、view-dependent split
open-vocabulary point semantics	points + language-aligned features	per-point label/score	OpenScene	view coverage、distillation、prompt	per-class IoU、mIoU/mAcc/allAcc

任务	输入前提	最小输出	典型方法	第一瓶颈	对应指标
open-vocabulary 3D instance	class-agnostic masks + posed views	mask ID/class/score	OpenMask3D	mask proposal、visibility、view crop	AP/AP50/AP25、proposal oracle
continuous language field	posed images + per-scene optimized field	continuous relevance / heatmap	LERF	scale、negative calibration、per-scene fit	localization/relevancy protocol
queryable semantic map	RGB-D + intrinsics + pose + features	scored map points/regions	ConceptFusion	pose/depth drift、fusion、threshold	retrieval/localization/map protocol

“Open-vocabulary”只说明query/label空间不是一个训练时固定小词表，不说明模型理解所有概念，不说明无需prompt与threshold，也不说明输出具有instance、metric geometry或robot affordance。

3. Typed 3D Grounding Manifest

3.1 最小 Schema

```
{
  "sample_id": "scene/query pair",
  "scene_artifact": "point | voxel | mesh | radiance_field | point_map",
  "scene_revision": "sha256:...",
  "query_text": "the chair left of the table",
  "entity_candidates": {
    "source": "detector | segmented_objects | masks | points | continuous_field",
    "revision": "sha256:...",
    "count": 12,
    "target_present": true
  },
  "output_type": "3d_box | candidate_id | point_scores | instance_mask | field_heatmap | map_points",
  "coordinate_frame": "camera | base | map | scene",
  "axis_unit": "x-forward/y-left/z-up, meter",
  "pose_depth_intrinsics_revision": "sha256:...",
  "relation": {
    "predicate": "left_of",
    "arity": 2,
    "reference_frame": "observer | world | anchor_object",
    "entity_ids": ["target", "anchor"],
    "threshold_meter": null
  },
  "encoder": "name/checkpoint/revision",
  "prompt_template": "a photo of {}",
  "negative_prompts": ["object", "things", "stuff", "texture"],
  "feature_normalization": "l2",
  "score": "cosine | dot | relative_softmax | learned_logit",
  "temperature_threshold_topk": [1.0, 0.5, 1],
  "metric_protocol": "top1 | Acc@IoU | semantic_mIoU | instance_AP",
  "metric_revision": "sha256:...",
  "timestamp_and_age": ["t", "age_ms"]
}
```

3.2 为什么它属于结果身份

同一句“left chair”在observer yaw改变后可指向另一对象；同一组embedding用dot或cosine会得到不同ranking；同一mask用top-1 view或多视图mean会得到不同score；同一预测box在IoU 0.25与0.5上会得到不同结论。因此scene、candidate、relation、encoder/prompt、score和metric都必须进入manifest/hash。

只保存一张彩色heatmap不能回答：候选是否含target、颜色是否scene-level min-max、query是否用了negative prompt、3D位置是否来自正确pose、阈值如何选、metric按point还是instance聚合。

4. 通用 Pipeline：Geometry、Entity、Language、Decision

```
scene geometry + frame/time/revision
  -> entity atomization
       proposals | segmented objects | points | masks | field samples | map points
  -> language encoding and prompt contract
  -> entity-language score
  -> optional spatial-relation filtering/reasoning
  -> calibrated decision: top-k / threshold / unknown
  -> output-specific metric and raw trace
  -> PER-015 action feasibility
```

四层不能颠倒：

1. **Geometry**：point或map位置是否在正确frame、unit、pose与timestamp；
2. **Entity**：什么是一个候选，target是否存在，mask是否split/merge；
3. **Language**：encoder、prompt、negative、normalization与relation语义；
4. **Decision**：score、temperature、threshold、top-k、unknown与metric。

先出现的错误会污染所有下游。一个semantic score很高的点若被错误pose写进墙内，仍是错误3D target；一个完美listener若candidate set漏掉目标，仍无法成功。

5. Candidate Grounding 与硬上界

对候选集合$\mathcal{C} = \{1, \dots, M\}$，通用language-conditioned ranking可写为：

$$s_j = g_\theta([\mathbf{p}_j, \mathbf{e}_q]), \qquad \mathcal{L}_{\text{ref}} = -\sum_{j=1}^{M} y_j \log \operatorname{softmax}(s)_j.$$

变量解释：$\mathbf{p}_j$是第j个proposal/object feature，$\mathbf{e}_q$是query embedding，s_j是matching logit，y_j是target分布。直觉：语言只在已有候选中排序。实现映射：ScanRefer固定代码实际对proposal-language feature输出`cluster_ref`并用`SoftmaxRankingLoss`；ReferIt3D固定训练脚本对candidate logits使用cross entropy。上式是统一教学表达，不宣称两份代码逐行相同。

若GT target不在$\mathcal{C}$中，则该sample的最大top-1为0。应同时报告：

- target-present/proposal recall；
- oracle candidate/box质量；
- 给定candidate的listener accuracy；
- end-to-end metric。

只报最后一个数会把“候选漏检”和“语言匹配错误”混在一起。

6. ScanRefer：Raw Scene → Proposal → 3D Box

ScanRefer从场景点云与自然语言出发，使用VoteNet-style 3D object proposals，再把proposal feature与language feature拼接，屏蔽invalid objectness，输出每个proposal的reference score。其输出原子是3D box，不只是candidate class。

6.1 三条loss/metric链

链	作用	典型证据	不能替代
objectness + vote/box	生成与回归3D proposals	`refnet.py`、`loss_helper.py`	language matching
reference ranking	从valid proposals中选query target	`match_module.py`、`SoftmaxRankingLoss`	box geometry质量
language auxiliary	预测object category等辅助信号	`lang_scores` CE	end-to-end grounding

3D box IoU为：

$$\mathrm{IoU}_{3D}(B_p, B_g) = \frac{\mathrm{Vol}(B_p \cap B_g)}{\mathrm{Vol}(B_p \cup B_g)}.$$

变量解释：B_p和B_g分别是预测与GT box。两个unit cube仅在x方向错开0.5时，intersection为0.5、union为1.5，IoU是1/3：通过0.25但失败0.5。实现映射：固定`eval_helper.py`同时维护reference selection与box-overlap评估；`ref_acc`与`Acc@IoU`不可互换。

6.2 面试中主动补的诊断

先查proposal recall与oracle IoU，再查objectness mask、candidate score、unique/multiple split，最后查box regression。如果候选index选对但IoU 0.5失败，问题更接近geometry；如果oracle candidate已很好但top-1错，才优先查language/distractor。

7. ReferIt3D：已分割对象中的Neural Listener

ReferIt3D假设scene已经被分成对象集合。固定模型用共享object encoder提取每个对象的point/color feature，编码utterance，再用DGCNN构建对象图，并按配置在graph前、后或两处融合language，最终为每个candidate产生logit。

其listener loss为：

$$\mathcal{L}_{\text{listener}} = -\log \frac{\exp s_{j^*}}{\sum_{j \in \mathcal{C}} \exp s_j}.$$

变量解释：j^*是被指代对象，$\mathcal{C}$是当前scene的候选对象集合。直觉：对象已经存在，模型学习用类别、属性、几何关系和distractor信息选一个。实现映射：固定`referit3d_net.py`输出`logits`，`train_referit3d.py`用CE；object-class和language auxiliary logits不能被误说成detector head。

7.1 ScanRefer与ReferIt3D不可合并

维度	ScanRefer	ReferIt3D
candidate来源	模型从scene生成3D proposals	数据已提供segmented objects
输出	3D referring box	candidate object ID/top-1
主要上界	proposal recall + box quality	object segmentation/candidate presence
核心metric	Acc@3D-IoU、ref selection	top-1 accuracy及distractor split

维度	ScanRefer	ReferIt3D
适合追问	proposal miss还是matching错	relation/distractor还是object feature错

说“ReferIt3D可以直接检测任意3D物体”忽略了它的candidate assumption；说“ScanRefer只做分类”忽略了proposal与box链。

8. Spatial Relation：Reference Frame 与Arity

令R把reference frame的轴映射到世界坐标，target a相对anchor b的left relation可写为：

$$r_{\text{left}}(a, b; R) = \mathbf{1}\left[\left(R^{\top}(\mathbf{p}_a - \mathbf{p}_b)\right)_x < 0\right].$$

变量解释：$\mathbf{p}_a, \mathbf{p}_b$是两个entity的位置，$R$可来自observer、world或anchor-object frame。直觉：先把相对位移转进声明的参考系，再判断x轴符号。实现映射：Sr3D relation生成与ReferIt3D listener能提供relation/distractor训练入口，但实际artifact仍必须保存frame、anchor IDs、axis和tolerance。

同一对点在yaw 0°时可为left，在yaw 180°时变成right。没有R的predicate不可复算。

8.1 Relation最小合同

Relation	最小元数	必须冻结	典型shortcut
left/right/front/behind	2	target、anchor、observer/world/object frame、axis	image-left冒充map-left
above/below	2	gravity/up axis、centroid/box/support定义	pixel-y冒充metric height
near/far	2	metric、unit、threshold或rank policy	scene min-max冒充固定距离
inside/on/supporting	2	geometry primitive、tolerance/contact policy	bbox overlap冒充support/contact
between	3	target、两个anchors、segment/corridor tolerance	只给一个anchor
closest/farthest	set-valued	candidate set、distance metric、tie rule	改候选仍比较accuracy

8.2 View-dependent 与View-independent

left/right/front/behind常随observer变化；above/below通常绑定gravity；inside/support更依赖物体几何。评估应按relation family与view dependence分层，否则模型可以靠绝对位置、类别共现或anchor频率获得表面accuracy。

9. Open-vocabulary Score：相似不等于置信

对entity feature $\mathbf{f}_j$与text feature $\mathbf{t}_q$，cosine为：

$$s_{j,q} = \frac{\mathbf{f}_j^{\top}\mathbf{t}_q}{\|\mathbf{f}_j\|_2 \, \|\mathbf{t}_q\|_2}.$$

变量解释：cosine只比较方向；raw dot还受feature norm影响。直觉：`[10,0]`和`[1,0]`对query `[1,0]`的dot分别是10和1，但cosine都为1。实现映射：OpenMask3D、OpenScene evaluation与ConceptFusion query路径都显式或隐式依赖feature normalization；是否normalize属于协议身份。

对positive s^+与negative s^-，relative probability可写为：

$$p^+ = \frac{\exp(s^+/\tau)}{\exp(s^+/\tau) + \exp(s^-/\tau)}.$$

变量解释：τ是temperature。若positive=.55、negative=.60、$\tau = 1$，则$p^+ = 0.4875026$，不通过0.5。实现映射：固定LERF encoder将positive与`object/things/stuff/texture` negatives两两softmax，并选择最保守comparison；raw cosine不能被叫作同一个relative confidence。

必须冻结：encoder/checkpoint、tokenizer、prompt template/ensemble、大小写与复数、negative set、L2 normalization、temperature、candidate set、top-k、threshold、unknown policy与scene-level min-max。

10. OpenScene：把2D开放词表Feature带进3D

OpenScene有两条紧密相关路径：把2D language-aligned pixel feature按几何对应融合到3D points；或训练3D sparse network去蒸馏这些fused features。固定代码的cosine distillation为：

$$\mathcal{L}_{\text{distill}} = 1 - \cos\left(\mathbf{f}_i^{2D}, \mathbf{f}_i^{3D}\right).$$

变量解释：$\mathbf{f}_i^{2D}$是投影到第i个3D点的2D teacher feature，$\mathbf{f}_i^{3D}$是3D network输出。直觉：让3D feature保留视觉-语言embedding方向。实现映射：固定`distill.py`对mask后的feature调用`CosineSimilarity`；`evaluate.py`再与text features计算score并报告semantic mIoU/mAcc/allAcc。

10.1 Per-point semantic的能力边界

能回答：哪些points与“chair”相似，或按label set给point分类。不能自动回答：两个chair中哪个是目标、哪些points属于同一个实例、目标box在哪里、哪个抓取点可达。无2D view coverage的point应显式unknown或使用冻结fallback，不能静默当background。

11. Semantic 与Instance 是两条轴

若两个chair的per-point class feature相同，语义预测可以完全正确，但“左边那把chair”的instance ID仍未知。实例化至少需要mask partition、connected/graph structure、tracking或proposal identity。

情况	semantic结果	instance结果	诊断
两个chair被同一mask合并	chair点可能全对	instance merge	mask proposal/partition错
一个chair被分成多个mask	class可都为chair	instance split	mask consistency错
target points无teacher coverage	unknown/低质	target可能漏掉	view/pose/occlusion错
class正确但left chair选错	semantic对	relation/identity错	frame、anchor或instance graph错

因此semantic mIoU不能替代instance AP或referring top-1。

12. OpenMask3D：Mask Proposal先于文本查询

OpenMask3D先生成class-agnostic 3D masks；再为每个mask选择top-k可见视图，把3D mask投影点作为SAM辅助，生成多尺度crops，提取normalized CLIP features并跨view聚合。文本query只负责给已有mask排序/分类。

12.1 Proposal bottleneck

若mask generator把target漏掉，最大query recall为0；若两个objects被merge，文本score不能无中生有地拆分；若一个object被split，多个高分mask会造成duplicate。应报告class-agnostic proposal oracle与open-vocabulary classification两层结果。

12.2 View aggregation

一个可审计的加权聚合为：

$$\bar{\mathbf{f}}_m = \frac{\sum_{v \in \mathcal{V}_m} w_{m,v}\, \mathbf{f}_{m,v}}{\sum_{v \in \mathcal{V}_m} w_{m,v} + \epsilon}.$$

变量解释：$\mathcal{V}_m$是mask m的selected views，$w_{m,v}$可由visibility、crop quality或固定平均得到。直觉：只让真正看到对象的视图贡献feature。实现映射：固定OpenMask3D代码选择top-k visible views，对多尺度image feature normalize后取mean；上式是诊断性推广，不声称fixed code已有learned weight。

`[1,0]`与`[-1,0]`均值为零，之后cosine undefined；加入orthogonal view也会让query score从1变成约0.7071。应保存per-view embedding/score、visibility、crop、occlusion和norm，而不是只保存mean。

13. LERF：Per-scene Continuous Language Field

LERF将multi-scale CLIP/DINO supervision嵌入per-scene radiance field。沿ray r、scale s的rendered feature可写为：

$$\hat{\mathbf{f}}(r, s) = \sum_i w_i(r) F_\theta(\mathbf{x}_i, s).$$

变量解释：$F_\theta(\mathbf{x}_i, s)$是在3D sample与physical scale处的language feature，$w_i(r)$是volume-rendering weight。直觉：通过posed images学习一个连续、可按尺度查询的3D feature field。实现映射：固定LERF代码在多个scale上求relevance，positive query与固定negatives比较；输出是field relevance，不自动给稳定object ID或box。

13.1 论文目标与fixed code边界

当前固定repo对CLIP output使用weight 0.1、delta 1.25的Huber loss，对DINO使用MSE。面试中可以讲“学习language-aligned continuous field”，但不能说“代码逐行就是负余弦loss”。per-scene optimization也意味着它不是一个无需scene fitting的通用3D detector。

13.2 Scale是query identity

“mug”和“handle”的适合尺度不同。LERF跨scale搜索relevance，因此scale range、sample count、selection rule和negative set属于结果身份。最大relevance heatmap仍需阈值、component/peak policy才能变成discrete target。

14. ConceptFusion：增量可查询Semantic Map

ConceptFusion把global image feature与local region feature结合成pixel-aligned representation，再借助depth、intrinsics与pose把像素提升/关联到3D map points，累积多帧feature。抽象更新为：

$$\mathbf{f}_{k,t} = \frac{\bar{c}_k \mathbf{f}_{k,t-1} + \alpha \mathbf{f}_{u,v,t}}{\bar{c}_k + \alpha}, \qquad \bar{c}_k \leftarrow \bar{c}_k + \alpha.$$

变量解释：$\mathbf{f}_{k,t}$是map point k在时刻t的feature，$\mathbf{f}_{u,v,t}$是当前pixel feature，α是correspondence/confidence weight，$\bar{c}_k$是累计权重。直觉：同一3D位置从多帧获得semantic evidence。实现映射：固定example使用RGB-D、intrinsics、pose与GradSLAM/Pointclouds融合feature；具体邻域、merge与weight以固定revision为准。

固定text-query example对text和map embeddings做L2 normalize、计算cosine，再用top-k或scene-level min-max threshold可视化。min-max结果是相对当前scene的ranking，不是跨scene calibrated probability。

14.1 Map query不证明SLAM正确

如果pose或depth错误，正确语义feature会被写进错误3D位置；多帧平均还可能把动态对象拖成ghost。必须同时记录pose revision、depth validity、association distance、update count、age与dynamic-object policy。semantic query成功不能反证geometry、localization或map consistency正确。

15. Field、Map、Mask、Box怎样选

下游问题	最小充分原子	推荐方向	不必要/不足
“那张椅子的3D框”	proposal + box geometry	ScanRefer式box grounding	只有point heatmap不足
“已分割对象中是哪一个”	candidate IDs + graph	ReferIt3D式listener	重跑detector可能多引入误差
“场景哪些点像fire extinguisher”	per-point semantic feature	OpenScene式point query	不需要先承诺instance
“返回完整对象mask”	class-agnostic masks + query	OpenMask3D式instance query	point class不等于instance
“在连续场中找某概念峰值”	scale-conditioned field	LERF式field	需per-scene fit与peak policy
“跨帧累计并查询语义地图”	pose/depth-aligned map	ConceptFusion式map	需SLAM/age/dynamic policy

选择原则是downstream minimum sufficiency、候选来源与runtime budget，而不是把六个方法串成一个越长越好的pipeline。

16. Metric 与协议不可互换

层	报告内容	不能替代
proposal	target-present recall、oracle IoU、proposal count	listener top-1
referring box	Acc@IoU 0.25/0.5、unique/multiple	`ref_acc`、semantic mIoU
candidate listener	top-1、distractor/easy-hard/view split	detector recall/box IoU
point semantics	per-class IoU、mIoU/mAcc/allAcc、seen/unseen	instance AP
instance semantics	AP/AP50/AP25、mask oracle	point mIoU
field/map query	localization/retrieval、threshold calibration、pose quality	robot success

还要冻结denominator：按query、scene、object、point还是class平均；是否去掉no-target或unknown；threshold是在validation上选还是test后调；是否对同一scene的多个queries视作独立样本。不同protocol的数字不能直接排榜。

17. 九个Deterministic Probes

Probe	固定输入	结果	诊断含义
3D box IoU	unit cubes，x错开0.5	IoU=`1/3`；过0.25、不过0.5	threshold属于metric identity
dot vs cosine	q=`[1,0]`；f=`[10,0]`，`[1,0]`	dot `[10,1]`；cos `[1,1]`	normalization改变ranking/scale
candidate bottleneck	GT 7；candidates `[2,4,5]`	最大top-1=`0`	先查target-present
relation frame	同两点；yaw 0/180°	left翻为right	predicate必须绑定frame
semantic vs instance	两个同class chair	class相同；instance ID不同	semantic不自动给identity
view cancellation	`[1,0]`，`[-1,0]` mean	`[0,0]`，cos undefined	aggregation需zero-norm gate
view selection	top view `[1,0]`，再加orthogonal view	score 1→`0.70710678`	top-k/view policy改变score
negative calibration	positive .55，negative .60	$p^+ = 0.4875026$	raw score不是relative pass
prompt identity	q1 scores `[.8,.6]`；q2 `[.6,.8]`	argmax 1→2	prompt/embedding属于协议

这些probe证明机制边界，不是ScanNet benchmark复现，也不支持本地项目收益。

18. First-divergence Failure Diagnosis

失败症状	第一检查层	关键字段	最小probe
query落在错误物理位置	geometry	frame/axis/unit/pose/depth/time	landmark reprojection
target完全无分数	candidate/coverage	target-present、view coverage	oracle candidate
正确类别但选错同类物体	instance/relation	mask ID、anchor、frame	two-chair relation test
box index正确但IoU失败	geometry/box	center/size/orientation	threshold sweep
mask merge/split	proposal	mask count、overlap、component	oracle mask AP

失败症状	第一检查层	关键字段	最小probe
多视图后分数下降	aggregation	per-view score/norm/visibility	top-1 vs mean
prompt微调后ranking翻转	language	template/tokenizer/negative	prompt matrix
scene间阈值失效	calibration	temperature/min-max/unknown	held-out calibration
map里出现semantic ghost	fusion/time	pose/age/dynamic policy	moving-object trace
semantic target不可执行	action gate	reachability/IK/collision/controller	PER-015 rejection log

推荐顺序：

```
scene revision / frame / pose / time
  -> candidate source and target-present
  -> point/mask/instance identity
  -> relation arity and reference frame
  -> encoder / prompt / negative / normalization
  -> score / threshold / unknown
  -> output-specific metric
  -> reachability / controller / episode
```

不要在candidate recall未知时先调prompt，也不要在pose错误时先换language encoder。

19. Fail-closed Inference Pseudocode

```
def grounded_3d_query(scene, query, manifest):
    # 1. Semantic scoring cannot repair an unidentified physical scene.
    require(scene.revision, scene.frame, scene.axis_unit)
    require(manifest.pose_depth_intrinsics_revision)

    # 2. Freeze the query interface before comparing scores or runs.
    require(manifest.encoder, manifest.prompt_template)
    require(manifest.feature_normalization, manifest.score)
    require(manifest.temperature_threshold_topk)

    # 3. Candidate atoms must match the requested output type.
    candidates = build_candidates(scene, manifest.entity_candidates)
    audit_candidate_count_and_target_presence(candidates)
    if manifest.entity_candidates.target_present is False:
        return {"status": "proposal_miss", "executable": False}

    # 4. Spatial predicates require explicit arity, anchors and reference frame.
    validate_relation_arity(query.relation)
    require(query.relation.reference_frame)

    # 5. Preserve raw per-candidate and per-view evidence before aggregation.
    raw = score_candidates(candidates, query, manifest)
    audit_view_coverage_zero_norm_and_unknown(raw)

    # 6. Apply the frozen decision and an output-specific metric protocol.
    target = apply_frozen_threshold_or_topk(
        raw, manifest.temperature_threshold_topk
    )
    target["metric_protocol"] = manifest.metric_protocol

    # 7. This node returns a semantic target, never an executable action.
    target["executable"] = False
    target["next_gate"] = "VLA-PER-015"
    return target
```

这是面试与目标实现合同，不是本地项目已有代码。真实系统还需local candidate generator、pose/depth lineage、model/checkpoint、query service、raw predictions、calibration set、latency/memory和controller integration。

20. OPD：2D Image Relation 不是3D Relation

固定`muxi@756e27e...`直接证据：

Artifact	直接支持	不能支持
`build_expert_mixes_seed0_v2.py`	`spatial_rel_expert`来自Visual Genome 2D relationship；prompt明确写“in the image”	observer/world 3D frame、3D object graph、3D box/mask
`point/point_data_only.py`	2D annotation points经x/y几何转box；输出可为0–1000 point/box或boolean	z、meter、pose、3D instance、language field

安全项目表述：

> 我构造过2D image spatial-relation和point/box监督，重点是数据路由、输出格式与解析边界。它没有3D metric frame、entity graph或semantic map，所以我不会把这段经历说成3D grounding；如果扩展，会先定义typed 3D entities、frame/pose、relation arity，再做candidate oracle和raw query评估。

21. Robot Harness：PCD存在不等于Semantic Map

固定direct evidence显示：2D occupancy JSON/PNG编码0 occupied、127 unknown、255 free；`world.pcd`有54,856 points，字段为`x y z intensity timestamp ring`；bridge暴露occupancy/pointcloud path与URL。

PCD header没有semantic label、instance ID、language embedding、query score或relation edge。它支持“存在3D点云资产与接口路径”，不支持“运行OpenScene/OpenMask3D/ScanRefer/ReferIt3D/LERF/ConceptFusion”“可以按语言查地图”或“semantic grounding提升了planner/robot success”。

安全项目表述：

> Harness当前可见的是三态2D occupancy、非语义PCD schema和path/URL bridge。我会把它当作未来semantic map的geometry/interface入口，但在没有semantic field、query server、pose lineage、raw episode和ownership确认前，不声称已经实现语言可查询3D地图。

22. 五项目 Project Truth Boundary

项目	当前可讲	当前不能说	升级所需artifact
OPD	2D Visual Genome relation、point/box监督、image-frame prompt	3D relation frame、3D grounding、semantic map	typed 3D entities/relations、pose、head/checkpoint/raw eval
Robot Harness	2D tri-state occupancy、non-semantic PCD、bridge/interface	language-queryable 3D map、instance grounding、robot benefit、ownership	semantic/instance fields、query server、pose lineage、episodes
OpenPiVLA	image/language/action policy，可讨论显式3D中间表示trade-off	已集成3D grounding或其导致97.4	module/config/checkpoint、matched ablation/rollouts
WAM	future-image/world-model可作为geometry/semantic consistency诊断候选	LERF/OpenScene式field或semantic-map收益	3D head/loss/data、query artifacts、matched metric
UWM	image/action/video world model与LIBERO protocol	3D semantic map、spatial listener、3D泛化	model diff、3D data、checkpoint、raw query/rollout

面试表述按`public upstream → local inventory → local-direct file → same-source inference → missing evidence → candidate ownership`分层。public mechanism说明“我理解什么”，local direct artifact才说明“项目里确实有什么”，两者都不能单独证明“我实现并验证”。

23. 两分钟结构化回答

> 我会先定义3D grounding的输出原子。ScanRefer是raw scene中先做3D proposal，再用语言给proposal排序并评估box IoU；ReferIt3D假设对象已分割，只在candidate中做top-1，所以二者的proposal上界和metric不同。OpenScene把2D语言对齐feature融合或蒸馏到3D points，适合point semantics，但两个同类对象仍缺instance identity。OpenMask3D先生成class-agnostic masks，选择可见视图、用SAM crop和多尺度CLIP得到per-mask embedding，所以mask proposal、pose和view aggregation是硬门。LERF是per-scene、multi-scale连续language field；ConceptFusion则在RGB-D、intrinsics和pose约束下增量融合queryable point map。对spatial relation，我会记录target、anchor、arity、observer/world/object frame、axis和metric threshold；between是三元关系，left/right会随observer旋转。开放词表score还必须冻结encoder、prompt、negative、L2 normalization、temperature与threshold，cosine不是校准概率。评估按proposal recall、Acc@3D-IoU、listener top-1、semantic mIoU、instance AP和field/map localization分层。最后这些都只输出semantic target，还要过reachability、IK、collision与controller。本地OPD只支持2D image relation，Harness只有非语义PCD/occupancy接口，因此不能讲成我已经实现3D semantic grounding。

24. 高频追问

1. ScanRefer与ReferIt3D的candidate assumption有什么根本差异?
2. 为什么proposal recall是end-to-end grounding硬上界?
3. `ref_acc`与`Acc@IoU 0.5`为什么可能不一致?
4. ReferIt3D为什么不是3D detector?
5. Sr3D与Nr3D分别适合诊断什么?
6. left/right到底绑定camera、observer、world还是object frame?
7. between为什么至少需要三个entity?
8. point semantic正确为什么仍可能选错“左边那把chair”?
9. OpenScene的2D fusion与3D distillation有何关系?
10. 无view coverage的3D points该怎么处理?
11. OpenMask3D的mask proposal、SAM crop和CLIP各负责什么?
12. 为什么多视图mean可能降低而不是提高score?
13. LERF为什么需要scale，为什么是per-scene field?
14. LERF fixed code的CLIP/DINO loss是什么?
15. ConceptFusion的map query为什么不能证明SLAM正确?
16. dot、cosine和relative softmax分别是什么量?
17. negative prompt为什么会改变“confidence”?
18. semantic mIoU、instance AP与referring top-1为什么不可互换?

19. 怎样拆分geometry、candidate、language和decision错误?

20. 本地五项目现在最多能支持哪一级3D claim?

25. 危险答案与修正

危险答案	问题	安全修正
“3D grounding就是CLIP投点云”	忽略六类输出与candidate identity	先定义box/ID/point/mask/field/map
“ReferIt3D能从raw scene检测物体”	输入已是segmented objects	称为candidate listener
“target漏检还能靠语言找回来”	候选集合没有目标	报proposal oracle/target-present
“point mIoU高就能区分两个chair”	semantic不等于instance	加mask/instance/graph identity
“left就是x更小”	x轴依赖reference frame	冻结observer/world/object frame
“between是二元关系”	缺第二个anchor	target + two anchors + tolerance
“cosine 0.8就是80%置信度”	cosine不是校准概率	报normalization、negative、temperature
“多视图平均一定更稳”	cancellation/低质view可降score	保存per-view与zero-norm诊断
“LERF直接输出object box”	输出连续relevance	另需peak/component/instance policy
“ConceptFusion能查询说明SLAM对”	feature可被写到错误位置	单独评估pose/depth/map geometry
“semantic target可以直接执行”	缺reachability/IK/collision	进入PER-015 gate
“Harness已有semantic 3D map”	PCD无semantic/query字段	只说非语义PCD与bridge

26. 白板 Checklist 与练习标准

能够不看材料完成:

- 画出geometry→entity→language→decision→action gate主链;
- 列出box、candidate ID、point、mask、field、map六种输出;
- 写出typed grounding manifest中的candidate/frame/relation/prompt/metric字段;
- 推导candidate CE并解释proposal hard upper bound;
- 用unit-cube probe算出3D IoU 1/3;
- 比较ScanRefer与ReferIt3D的输入、输出和metric;
- 写出reference-frame left relation并解释yaw flip;
- 说明between的三元合同与closest的candidate-set合同;
- 区分dot、cosine和positive-vs-negative relative probability;
- 解释OpenScene的per-point semantics与instance缺口;
- 画出OpenMask3D mask→view→SAM crop→CLIP→query链;
- 解释LERF scale-conditioned field与fixed code loss;
- 解释ConceptFusion增量map update与pose drift;
- 按first-divergence顺序诊断十类失败;
- 说清OPD、Robot Harness与其余三项目的真实性边界;
- 完成两分钟回答并接住至少十个追问。

`drafted`不等于掌握。只有真实口述、追问、评分与纠错后才能改变题卡状态；PER-014题卡已生成并保持`unseen`，`coaching_state.md`也不应创建。

27. 结论与下一步

可靠的3D grounding答案不是背六个模型名字，而是建立一套类型系统：scene geometry是否可信、entity如何产生、target是否在candidate中、relation参考系与元数是否完整、language score是否冻结并校准、metric是否与输出原子匹配。ScanRefer、ReferIt3D、OpenScene、OpenMask3D、LERF与ConceptFusion分别占据这套类型系统中的不同位置，不能用一个“CLIP+点云”概括。

本地项目当前最强边界是：OPD有2D image spatial-relation与point/box监督；Robot Harness有2D tri-state occupancy、non-semantic PCD与bridge；OpenPiVLA/WAM/UWM只有image/language/action或video/world-model证据。没有local 3D semantic head/map/checkpoint、raw query、proposal oracle、same-protocol metric、robot episode或ownership闭环。

`VLA-PER-014` source、knowledge与card已在独立bounded slices同步；265张题卡继续全部`unseen`。下一bounded slice只冻结`VLA-PER-015` perception→action/reachability/uncertainty/failure-gate source；不生成其knowledge/card，不创建`coaching_state.md`，不改变任何mastery。

References

1. PER-014 3D Grounding / Spatial Relation / Open-vocabulary Source Freeze：六篇完整primary PDF、六个official repo、36个文件SHA、九个probe与五项目audit。
2. Chen et al., ScanRefer v3；official code@9d74830。
3. Achlioptas et al., ReferIt3D, ECCV 2020 official PDF；official code eccv@725a5d3。
4. Peng et al., OpenScene v2；official code@0f369bc。
5. Takmaz et al., OpenMask3D v2；official code@3bc3fc5。
6. Kerr et al., LERF v1；official code@db08d57。
7. Jatavallabhula et al., ConceptFusion v3；official code@4457c1f。
8. PER-007 Open-vocabulary Grounding 与 PER-013 Point/Voxel/BEV/Occupancy。
9. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 15 / 43 · `interview_materials/vla/knowledge/vol_06_per015_perception_action_reachability_uncertainty_failure_gate.html` · SHA-256 `2682AB3EB045EE47E2900BABEA7985B8F8498257C8BBB0185A39F43ED99222E0`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`2682ab3eb045ee47e2900babea7985b8f8498257c8bbb0185a39f43ed99222e0`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per015_perception_action_reachability_uncertainty_failure_gate.md`

VOL 06 Perception / Grounding / Navigation：Perception→Action、Reachability、Uncertainty 与 Failure Gate（PER-015）

状态：`drafted / VLA-PER-015 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-014` 3D semantic target、`VLA-PER-002` frame/lift、VOL 03 action/controller contract；后续：`VLA-PER-016` localization/SLAM drift、`VLA-PER-019` navigation evaluation/failure。 来源入口：PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Source Freeze。本页只把已冻结证据转成可教学、可诊断、可面试回答的知识；对应一张原子题卡已在独立bounded slice同步并保持`unseen`，不创建训练状态，也不改变任何mastery。

问题（Question）

面试官问“VLA已经找到了目标，怎样把它变成机器人能安全执行的动作”时，最危险的回答是“把3D坐标送给IK，再做碰撞检测”。可靠回答必须先把semantic target转成带frame、unit、timestamp、tool和controller合同的action target，再按workspace、IK residual、joint limit、singularity、self/environment collision、clearance、trajectory dynamics、uncertainty和controller precheck逐层fail closed；任何一层未知都不能默认为pass，最终通过也只得到controller-ready candidate，而不是robot success。

本节点主链：

```
PER-014 semantic target / pose / mask
  -> identity: frame / unit / timestamp / revision
  -> action target: tool / approach / gripper / controller
  -> workspace / IK / joint bounds / singularity
  -> state + edge collision / robust clearance
  -> trajectory timing / controller precheck
  -> calibrated uncertainty / abstention / fallback
  -> controller-ready candidate
  -> execution feedback (not task success yet)
```

知识地图位置与边界

相邻节点	该节点负责	PER-015负责
PER-014	3D entity、relation、semantic target、score、metric	判断semantic target能否成为controller-ready candidate
PER-002/003	depth、calibration、frame、multi-view/time lineage	消费并校验身份；不修复错误TF、pose或时间同步

相邻节点	该节点负责	PER-015负责
VOL 03	action token/chunk、normalization、policy head与controller接口	不训练head；验证输出shape/semantics/order/unit/time
PER-016	localization、odometry、SLAM、map-frame drift	不做SLAM；只拒绝stale或revision不一致的输入
PER-019	episode、collision/stop、success与统计评估	不从单个静态candidate推出benchmark/robot success

PER-015是接口与可行性节点，不是完整机器人安全认证。接触力、硬件急停、functional safety、真实人机环境法规仍在本页范围之外。

1. 30秒直觉：目标不是动作，通过不是成功

一个可靠的30秒回答是：

> 感知输出只是semantic target，我先补齐frame、unit、timestamp、tool和controller schema，把它变成typed action target。然后用workspace做cheap reject，IK返回后重算FK residual，检查joint limit和Jacobian最小奇异值；再对goal state和整条edge做self/environment collision与uncertainty-inflated clearance；几何路径还要通过velocity、acceleration、current-state和controller joint/order/unit/time检查。语义、几何或执行不确定性超过校准风险预算时就ask、re-perceive、replan或stop，不能拿top-1硬执行。所有gate通过只得到controller-ready candidate，执行反馈和任务成功要另算。

这段回答先封住十个偷换：

1. semantic target被当成controller action；
2. 点在粗workspace里被当成指定姿态可达；
3. IK solver返回数组被当成有效解；
4. 数学IK存在被当成joint-limit-aware IK存在；
5. goal无碰撞被当成整条path无碰撞；
6. nominal clearance被当成有感知误差时仍安全；
7. geometric path被当成满足velocity/acceleration的trajectory；
8. action shape相同被当成axis/unit/semantics相同；
9. 未校准confidence被当成逐样本安全概率；
10. candidate accepted被当成robot/task success。

2. Typed Perception→Action Manifest

2.1 最小 Schema

```
{
  "candidate_id": "scene/query/proposal revision",
  "semantic_source": {
    "artifact_sha256": "...",
    "query": "pick the red mug",
    "target_id": "mug-17",
    "score_type": "cosine | logit | calibrated_set",
    "raw_score_and_threshold": [0.73, 0.60]
  },
  "target": {
    "type": "point | pose | grasp | pick_place | trajectory",
```

```
    "frame": "camera | base | world | map",
    "axis_unit": "x-forward/y-left/z-up, meter/radian",
    "pose_or_waypoints": [],
    "tool_frame": "tcp",
    "timestamp_ns": 0,
    "age_deadline_ms": [0, 100],
    "covariance": []
  },
  "robot_contract": {
    "robot_model_sha256": "...",
    "planning_group": "arm",
    "end_effector": "gripper",
    "joint_order": [],
    "joint_position_velocity_acceleration_limits": [],
    "controller_name": "joint_trajectory_controller",
    "controller_axis_unit_rate": "..., rad, 100Hz"
  },
  "scene_contract": {
    "planning_scene_sha256": "...",
    "collision_geometry_revision": "...",
    "allowed_collision_matrix_sha256": "...",
    "padding_and_scale": [],
    "motion_validation_resolution": 0.0
  },
  "risk_contract": {
    "calibration_split_revision": "...",
    "risk_budget_alpha": 0.05,
    "uncertainty_inflation": [],
    "allowed_outcomes": ["accept", "replan", "ask_for_help", "stop", "reject"]
  }
}
```

2.2 为什么Manifest属于结果身份

同一个3D pose在`camera`与`base` frame中数值和语义不同；同一个7D向量可能是Cartesian delta，也可能是joint position；同一条path在不同collision mesh、allowed-collision matrix、padding或motion resolution下可从pass变reject。缺少这些revision时，结果不可复算，也无法解释失败来自perception、planning还是controller。

3. 九层Fail-closed Gate

总gate可写为：

$$G = G_{\text{identity}} \wedge G_{\text{workspace}} \wedge G_{\text{IK}} \wedge G_{\text{bounds}} \wedge G_{\text{singularity}} \wedge G_{\text{collision}} \wedge G_{\text{trajectory}} \wedge G_{\text{uncertainty}} \wedge G_{\text{controller}}.$$

变量解释：G_k是第k层布尔gate，但实现中必须保留`pass/fail/not_checked`三态与evidence；`not_checked`在fail-closed模式下不能执行。直觉是整条链是串联系统，任一层未知都不能靠后层“平均掉”。实现映射：source freeze中的typed manifest和reason taxonomy分别给每个gate绑定input revision、threshold与recovery action。

Gate	核心问题	主要输入	失败去向
identity	frame/unit/time/revision齐全吗	target manifest	reject / ask
workspace	粗几何必要条件满足吗	robot/tool geometry	reject / base reposition
IK	位姿残差在阈值内吗	target pose、solver	alternate branch / reject
bounds	q在joint limits内吗	robot model	branch / reject
singularity	conditioning可接受吗	Jacobian	change pose / replan
collision	state与edge无碰撞吗	scene、ACM、resolution	replan / reject

Gate	核心问题	主要输入	失败去向
trajectory	continuity与动态约束满足吗	waypoints、dt、limits	re-time / replan
uncertainty	校准风险预算满足吗	score/covariance/calibration	ask / re-perceive
controller	shape/semantics/order/unit/state/status对吗	controller contract	stop / reject

4. Semantic Target → Action Target

PER-014可给出entity ID、box/mask、3D point或pose与semantic score。PER-015需要把它转成任务原子：

```
semantic_target = {
  entity_id, pose_or_region, frame, semantic_score, timestamp
}

action_target = {
  tool_pose, pregrasp_pose, approach_direction, gripper_command,
  robot_model, planning_group, joint/controller contract
}
```

Transporter Networks与CLIPort提供清晰范例：从heightmap上预测pick，再以pick-conditioned correlation预测place；CLIPort加入language-conditioned semantic/spatial streams。它们说明perception如何输出pick/place proposal，但不自动提供：

- tool-center-point transform；
- grasp width、surface normal、force closure；
- joint-limit-aware IK branch；
- self/environment collision；
- velocity/acceleration timestamps；
- real controller status。

所以模型输出`pick/place pose`时，面试回答应主动说“proposal进入feasibility gate”，而不是“模型直接控制机械臂”。

5. Workspace：Cheap Reject，不是最终证明

5.1 二连杆可达环

对link length l_1, l_2和目标位置$\mathbf{p}$，二维二连杆的位置可达条件为：

$$|l_1 - l_2| \le \|\mathbf{p}\|_2 \le l_1 + l_2.$$

变量解释：$\|\mathbf{p}\|_2$是目标到base的半径；内半径来自两杆相折，外半径来自两杆伸直。直觉是先用极便宜的几何条件拒绝明显不可能目标。实现映射：deterministic probe固定$l_1 = 1, l_2 = 0.7, r = 1.8$，因上界为1.7而直接`OUTSIDE_WORKSPACE`。

5.2 四级Reachability

层	新增约束	为什么上一层不够
geometry workspace	link length、粗joint range	不含orientation/tool
task workspace	tool、approach、support surface	不含current state/path
current-state reachability	当前q、branch、obstacle、path	不含model/sensor uncertainty
robust reachability	covariance、tracking error、inflation	nominal pass可能变reject

一个点在workspace内，指定orientation仍可能无解；换gripper、base pose或URDF后，旧reachability map也失效。

6. IK：返回值之后还要验证Residual

6.1 SE(3) Pose Error

对目标位姿T^*与候选关节$\mathbf{q}$的forward kinematics $T(\mathbf{q})$，body-frame误差可写为：

$$\mathbf{e}(\mathbf{q}) = \mathrm{se3ToVec}\left(\log\left(T(\mathbf{q})^{-1}T^*\right)\right).$$

变量解释：$\mathbf{e} \in \mathbb{R}^6$含rotation与translation误差；$T(\mathbf{q})$来自FK。直觉是solver说“成功”后，用独立FK重新计算“实际到了哪里”。实现映射：Modern Robotics固定`IKinBody/IKinSpace`分别以`eomg`和`ev`终止；工程gate也应分开记录角度与位置tolerance，不能只看一个return code。

6.2 Newton–Raphson 与Pseudoinverse

数值IK的一步可写成：

$$\Delta\mathbf{q} = J(\mathbf{q})^{\dagger}\mathbf{e}, \qquad \mathbf{q}_{k+1} = \mathbf{q}_k + \Delta\mathbf{q}.$$

变量解释：$J^{\dagger}$是Jacobian pseudoinverse，$\mathbf{e}$是task-space error。直觉是用局部线性近似把末端误差映射到关节修正。实现映射：Modern Robotics教学代码直接迭代该结构；README明确代码优先可读性而非robustness，所以production系统还需bounds、timeout、branch、collision与conditioning。

常见失败：

1. 初值落入错误branch或局部极小；
2. translation通过但rotation超阈值；
3. solver timeout却留下“看起来合理”的最后iterate；
4. q被clamp到limit后未重算residual；
5. 单位或frame错误导致“稳定收敛到错误目标”。

7. Joint Limit 与TRAC-IK

数学IK解存在，不代表受限IK解存在。最小合同是：

$$\mathbf{q}_{\min} \le \mathbf{q} \le \mathbf{q}_{\max}.$$

变量解释：上下界来自冻结robot model；continuous joint与bounded revolute joint要分开处理。直觉是“末端能到”还要问“关节是否允许这样到”。实现映射：TRAC-IK固定代码显式接收`q_min/q_max`；KDL-TL会处理越界与随机重启，NLOpt路径设置lower/upper bounds。

TRAC-IK并发运行KDL-style路径与SQP/NLOpt路径，可改善joint-limit附近的求解率，但官方README明确：

- solver不使用mesh判断self-collision；
- IK solution不保证environment collision-free；
- 多解排序不等于轨迹branch连续；
- timeout/success统计不等于具体部署task success。

deterministic probe中$\mathbf{q} = [\pi/2, 0]$满足纯几何目标，但q1 limit为`[-1,1]`，因此必须`JOINT_LIMIT`拒绝，不能clamp后直接执行。

8. Singularity、Conditioning 与Manipulability

对Jacobian做SVD：

$$J(\mathbf{q}) = U\Sigma V^{\top}, \qquad \sigma_{\min}(J) < \tau_{\sigma} \Rightarrow \text{near singular}.$$

变量解释：$\sigma_{\min}$是最小奇异值，τ_{σ}是协议阈值。直觉是某个末端运动方向几乎无法由关节产生，微小末端速度会要求巨大关节速度。实现映射：MoveIt `RobotState::getJacobian`或Modern Robotics Jacobian可生成矩阵；gate必须记录joint scaling、task dimensions和阈值。

可选的Yoshikawa manipulability为：

$$w(\mathbf{q}) = \sqrt{\det(JJ^{\top})} = \prod_i \sigma_i.$$

变量解释：w越小通常越接近rank loss。直觉是末端局部速度椭球体积缩小。实现映射：它适合排序多个IK candidates，不应取代hard bounds、collision或velocity check；rotation与translation单位混合时，数值需要scale，否则不同实现不可直接比较。

二维2R在完全伸直$\mathbf{q} = [0, 0]$时，deterministic probe得到$\sigma_{\min} = 0$，因此即使目标位置可达也要change pose或replan。

9. Collision：State Validity 与Motion Validity

9.1 Goal State无碰撞仍不够

要区分：

```
state validity  : q 本身是否满足bounds/collision/constraints
motion validity : q_a -> q_b 的整条edge是否有效
path validity   : 所有states与edges是否有效
```

RRT-Connect论文把free configuration space作为搜索域；OMPL固定`RRTConnect.cpp`在扩展时调用`isValid/checkMotion`。因此“RRT-Connect生成collision-free path”的前提是集成方提供了正确的state validity、motion validator、geometry与resolution。

9.2 MoveIt PlanningScene

MoveIt固定代码中：

- `RobotState::satisfiesBounds`检查joint bounds；
- `RobotState::setFromIK`调用kinematics solver；
- `PlanningScene::checkCollision/isStateColliding`处理collision；
- `PlanningScene::isStateValid`组合collision与constraints；
- trajectory相关路径遍历waypoints。

这些API存在不等于部署正确。还要冻结URDF/SRDF、attached object、tool geometry、allowed-collision matrix、padding、octomap/scene revision和current state。

9.3 Discrete Motion Validation

OMPL `DiscreteMotionValidator`依据`validSegmentCount`插值检查edge。步长过粗会穿过薄障碍；步长极细会增加开销。deterministic segment/AABB probe中端点都在box外，但segment穿过box，因此只查端点会产生false pass。

10. Clearance 与Uncertainty Inflation

robust clearance gate可写为：

$$d_{\min}(\mathcal{R}(\mathbf{q}), \mathcal{O}) > r_{\text{robot}} + m_{\text{model}} + m_{\text{uncertainty}}.$$

变量解释：$\mathcal{R}(\mathbf{q})$是机器人几何，$\mathcal{O}$是障碍，m_{model}覆盖mesh/标定误差，$m_{\text{uncertainty}}$覆盖target/obstacle covariance。直觉是不能用“零误差的薄模型”贴障碍执行。实现映射：可通过collision padding、scale或显式inflated geometry实现，但必须把膨胀量与scene revision写入manifest。

deterministic probe中path位于y=0.25：nominal margin 0.10时不碰撞，uncertainty inflation增至0.20后碰撞，结论从pass翻为reject。这证明covariance不是只写日志的附加字段，而会改变动作决策。

常见错误：

- 对target加covariance，却不对obstacle/robot model加；
- 只在goal pose算clearance，不检查swept path；

- 把visual bbox边界当physical collision mesh；
- 感知分数低时只降速度，却不增加margin或abstain。

11. Planning 与RRT-Connect

RRT-Connect维护从start与goal生长的两棵树，`EXTEND`向sample推进，`CONNECT`贪心尝试连接另一棵树。它适合高维single-query planning，但面试中必须补四点：

1. sampling planner是probabilistically complete类思想，不承诺有限时间一定找到解；
2. 找不到路径不一定证明路径不存在，可能是timeout/resolution/sampler问题；
3. 找到几何path不代表满足dynamics或controller limits；
4. shortcut/smoothing后必须重新做motion validation。

OMPL把具体robot collision checker留给集成层，这是清晰的责任边界：planner负责搜索，MoveIt PlanningScene等负责physical validity。

12. Trajectory：Geometry、Continuity 与Time

几何waypoints进入controller前至少满足：

$$\mathbf{q}_{\min} \le \mathbf{q}_t \le \mathbf{q}_{\max}, \qquad |\dot{\mathbf{q}}_t| \le \dot{\mathbf{q}}_{\max}, \qquad |\ddot{\mathbf{q}}_t| \le \ddot{\mathbf{q}}_{\max}.$$

变量解释：$\mathbf{q}_t, \dot{\mathbf{q}}_t, \ddot{\mathbf{q}}_t$是位置、速度与加速度；上限来自joint/controller contract。直觉是空间中平滑的线，若时间给得太短仍不可执行。实现映射：MoveIt固定Time-Optimal Trajectory Generation接收velocity/acceleration limits并生成timestamps；deterministic probe固定q `[0,.5,1.5]`、dt `.1`，得到vmax 10、amax 50，必须`DYNAMIC_LIMIT`。

还要检查：

- branch continuity：相邻IK解不能跳到另一branch；
- unwrap：continuous joint不能错误跨2π；
- jerk与controller interpolation；
- start-state tolerance；
- partial execution后的replan起点；
- attached object和scene在trajectory期间是否变化。

13. Controller Contract：Shape相同仍可能语义错

维度	必须冻结	典型静默错误
shape	`H×D`、valid dims、mask、joint count	72D被截成16D
semantics	absolute/delta/velocity、joint/cartesian	delta当absolute
order	xyz/rpy/gripper或joint names	roll与gripper错位

维度	必须冻结	典型静默错误
unit	m/mm、rad/deg、normalized/raw	数量级错
time	Hz、dt、timestamp、chunk H、replan K	stale action继续执行
state	current q与start tolerance	轨迹从错误状态起步
status	ready/abort/timeout/cancel/result	failure被包装为success

deterministic schema probe期望7D `xyz_rpy_grip/m_rad`，却收到12D `dual_arm_plus_base/normalized`，必须`CONTROLLER_SCHEMA`拒绝。安全实现不能为了“能跑”而silently slice/pad。

MoveIt `TrajectoryExecutionManager`在执行前验证current joint state、trajectory joint集合和起点容差，并保留controller timeout/status。VLA policy server的成功返回只说明网络调用成功，不说明controller接受或完成了动作。

14. Action Chunk 与Staleness

若policy一次预测horizon H，系统每次只执行前K步再replan，则第j步action的age近似为：

$$\text{age}(a_j) = t_{\text{execute},j} - t_{\text{observe}}.$$

变量解释：t_{observe}是生成chunk所依据观测时间，$t_{\text{execute},j}$是第j步实际执行时刻。直觉是chunk越靠后，依赖的世界状态越旧。实现映射：manifest应记录observation timestamp、queue、H/K/Hz和deadline；PER-015 probe中target age 240ms超过100ms deadline，直接`TARGET_STALE`。

每次replan前要重新核验：

- scene revision和attached object；
- current robot state；
- 剩余target/action age；
- controller queue是否已cancel/flush；
- 新轨迹与已执行prefix的continuity。

15. 三类Uncertainty不能混成一个Confidence

类型	来源	可见信号	合适动作
semantic	多个对象/指令解释	candidate set、class/LLM score	ask / clarify
geometric	depth/pose/calibration/shape	covariance、view disagreement	re-perceive / inflate / replan
execution	latency/tracking/contact/controller	state error、timeout、status	slow/stop/recover

semantic cosine高不能证明geometry准确；geometry covariance小不能证明controller ready；controller online也不能修复选错对象。first-divergence diagnosis必须从最早失败层开始。

一个实用risk contract至少包含：

- calibration split与revision；
- target error definition；
- desired coverage/risk budget；
- uncertainty到margin或abstention的映射；
- distribution-shift detector；
- `accept/replan/ask/stop/reject`状态机。

16. KnowNo 与Conformal Prediction

16.1 Calibration

对calibration样本(x_i, y_i)，以正确option的score $f(x_i)_{y_i}$构造nonconformity：

$$s_i = 1 - f(x_i)_{y_i}, \qquad \hat{q} = \text{Quantile}_{\lceil (N+1)(1-\epsilon) \rceil / N} \{s_i\}_{i=1}^{N}.$$

变量解释：N是calibration size，ϵ是目标error level，$\hat{q}$是higher quantile。直觉是用历史正确答案的“低置信程度”确定test threshold。实现映射：KnowNo官方MobileManipulation/Tabletop notebooks用`np.quantile(..., method='higher')`计算该阈值。

test prediction set为：

$$C(x) = \{y : f(x)_y \geq 1 - \hat{q}\}.$$

变量解释：$C(x)$保留所有超过校准threshold的options。直觉是当多个解释都合理时，不强迫模型假装只知道一个。实现映射：KnowNo notebook在set非singleton时触发human help；mobile版本还把“none of the above” option进入set视为help。

16.2 保证边界

在exchangeability等条件下，conformal prediction提供marginal coverage目标，而不是：

- 每一个单样本都有相同条件成功率；
- collision-free或硬件安全保证；
- distribution shift下阈值永久有效；
- 错误label、漏候选或parser bug免疫；
- help之后human一定选择正确。

因此PER-015可以借鉴“集合预测→abstain”的结构，但low-level collision/trajectory仍要独立hard gate。

17. Abstention、Fallback 与Recovery状态机

推荐状态：

```
ACCEPT       all checked gates pass; return candidate only
REPLAN       target valid, current path/clearance/dynamics fail
ASK          semantic ambiguity or calibrated set non-singleton
REPERCEIVE   target stale or geometry uncertainty too large
STOP         controller/status/state unknown or execution unsafe
REJECT       schema/workspace/IK/bounds hard failure
```

fallback不是默认动作，而是受限状态转移：

```
semantic ambiguous -> ask / re-perceive
workspace or IK fail -> alternate proposal / branch / base reposition
collision fail -> update scene and replan
trajectory fail -> re-time or shorten horizon
controller mismatch -> reject; never silently slice/pad
execution timeout -> cancel -> verify stop -> read state -> recover
```

每条transition必须有retry上限、timeout、cancel acknowledgement和fresh-state check。否则“自动恢复”可能变成无限replan或执行旧轨迹。

18. 十个Deterministic Probes

Probe	固定输入	结果	教学结论
workspace annulus	$l_1 = 1, l_2 = .7, r = 1.8$	range `[.3,1.7]`，reject	cheap必要条件
IK residual	target `[1.3,.2]`，q `[.1,-.1]`	residual `.407507m>.01`	return后重算FK
joint limit	q `[π/2,0]`，q1 `[-1,1]`	reject	几何解不等于受限解
singularity	2R q `[0,0]`	$\sigma_{\min} = 0$	可达不等于well-conditioned
segment/AABB	edge穿过box	collision true	端点pass不够
clearance inflation	nominal `.10`，inflated `.20`	pass翻reject	covariance改变决策
dynamics	q `[0,.5,1.5]`，dt `.1`	vmax 10、amax 50	path不等于trajectory
abstention	risk upper `.12`，alpha `.05`	ask	风险超预算不取top-1
stale target	age 240ms，deadline 100ms	reject	时间属于身份
controller schema	expected 7D，got 12D	reject	dim/order/unit fail closed

10个probe的deterministic oracle结果为`10/10 expected decisions matched`。它们只验证合同逻辑，不是MoveIt、TRAC-IK、CLIPort或KnowNo benchmark复现。

19. First-divergence Failure Diagnosis

表面症状	第一检查层	关键证据	常见误诊
抓错对象	semantic identity	candidate/raw scores/query	调IK参数
pose方向错	frame/tool transform	TF、axis、tcp	加planning time
IK timeout	workspace/IK	target pose、seed、residual	直接称模型差
q贴limit	bounds/branch	q min/max、alternate solutions	clamp后执行
末端抖动	singularity/controller	sigma、qdot、rate	只平滑视觉
起终点都安全却碰撞	motion validity	edge samples/resolution	只查goal

表面症状	第一检查层	关键证据	常见误诊
靠障碍偶发失败	clearance/uncertainty	covariance、padding	只降速度
simulator能跑真机不能	controller schema	order/unit/rate/driver	归因Sim2Real大词
chunk后段发散	staleness/replan	H/K/Hz/age/state	只改action loss
policy server返回但不动	execution precheck	controller status/joints/start	报推理成功
help率异常高	calibration/option bias	split、qhat、set size	降threshold硬执行
单次success但总体差	evaluation downstream	episode denominator/failure codes	把gate pass当success

诊断顺序：

```
identity/time
 -> semantic target
 -> workspace
 -> IK residual/bounds/singularity
 -> state/edge collision + clearance
 -> timing/controller schema/status
 -> calibrated uncertainty
 -> execution/contact/task outcome
```

20. Fail-closed Pseudocode

```
def perception_to_action_gate(semantic_target, manifest, robot, scene, controller):
    # 1. Identity first: never guess frame, unit, timestamp, or revision.
    require_fields(semantic_target, ["frame", "axis_unit", "timestamp_ns", "artifact_sha256"])
    require_fields(manifest, ["robot_contract", "scene_contract", "risk_contract"])
    if age_ms(semantic_target) > manifest.target.age_deadline_ms:
        return {"status": "reperceive", "reason": "TARGET_STALE"}

    # 2. Convert entity/pose into a typed tool/action target.
    candidate = build_action_target(semantic_target, manifest)
    if not controller.schema_matches(candidate):
        return {"status": "reject", "reason": "CONTROLLER_SCHEMA"}

    # 3. Cheap workspace reject before expensive IK.
    if not robot.workspace_contains(candidate.tool_pose):
        return {"status": "reject", "reason": "OUTSIDE_WORKSPACE"}

    # 4. Verify every IK result independently with FK, bounds, and SVD.
    valid_q = []
    for q in robot.solve_ik(candidate.tool_pose, timeout=manifest.ik_timeout):
        if pose_residual(robot.fk(q), candidate.tool_pose) > manifest.ik_tolerance:
            continue
        if not robot.satisfies_bounds(q):
            continue
        if min_singular_value(robot.jacobian(q)) < manifest.sigma_min:
            continue
        valid_q.append(q)
    if not valid_q:
        return {"status": "reject", "reason": "IK_NO_VALID_SOLUTION"}

    # 5. Check state and every edge with one frozen planning scene.
    paths = plan_from_current_state(valid_q, scene, manifest.motion_resolution)
    paths = [p for p in paths if collision_free_path(p, scene)]
    paths = [p for p in paths if robust_clearance(p, scene, candidate.covariance)]
    if not paths:
        return {"status": "replan", "reason": "COLLISION_OR_CLEARANCE"}

    # 6. Convert geometry to a timed trajectory and validate current controller state.
    timed = time_parameterize(paths, robot.velocity_limits, robot.acceleration_limits)
    if not timed:
        return {"status": "replan", "reason": "DYNAMIC_LIMIT"}
    if not controller.precheck(timed, current_state=robot.current_state()):
        return {"status": "stop", "reason": "EXECUTION_PRECHECK"}

    # 7. Unknown or over-budget risk is not a pass.
    decision = calibrated_risk_decision(candidate, timed, manifest.risk_contract)
    if decision == "ask":
        return {"status": "ask_for_help", "reason": "UNCERTAIN"}
```

```
    if decision != "accept":
        return {"status": "reject", "reason": "RISK_BUDGET"}

    # 8. Return a candidate only; execution feedback owns robot/task success.
    return {
        "status": "controller_ready_candidate",
        "trajectory": timed[0],
        "checked_revisions": manifest.revisions(),
        "robot_success": None
    }
```

伪代码是面试与目标系统合同，不是五个本地项目已有实现。真实系统还需并发scene update、force/contact gate、cancel acknowledgement、hardware stop与episode logger。

21. Public Method → Code Mapping

机制	固定实现	面试可讲	不能反推
numerical IK	Modern Robotics `IKinBody/IKinSpace`	residual/Jacobian/pseudoinverse	production robustness
bounded IK	TRAC-IK KDL-TL + NLOpt	q bounds、dual solver、timeout	collision-free path
planning scene	MoveIt RobotState/PlanningScene	bounds、IK、collision、constraints	URDF/scene一定正确
sampling planner	OMPL RRTConnect/MotionValidator	state vs edge validity	finite-time必有解
trajectory/execution	MoveIt TOTG/ExecutionManager	limits、timestamps、precheck/status	task success
pick/place proposal	Transporter/CLIPort	perception→action atom	reachable/control-ready
calibrated abstention	KnowNo notebooks	higher quantile、prediction set、help	low-level safety证书

这张表建立公开机制的source-to-code桥，不证明本地项目用了固定commit，也不证明候选人ownership。

22. OPD 与Robot Harness：最容易被过度包装的两类Hook

22.1 OPD

OPD direct evidence支持2D point/box/relation supervision与semantic-nav box数据构造。项目中的`fallback`命中是system prompt、model path或text cleaning fallback，不是physical recovery。

安全表述：

> OPD可以为gate提供2D semantic target或grounding score，但当前没有robot/tool model、metric 3D action target、IK/collision trace或controller gate。我会把它讲成perception upstream，不讲成已闭环执行。

22.2 Robot Harness

Robot Harness固定文件支持tool scope、锁、cancel、poll/timeout、best-effort compensation，以及2D occupancy/PCD/state bridge。map/state fallback不是fresh sensing；第三方SenseBot底盘controller目录也不自动证明当前runtime integration、manipulator scope或候选人ownership。

安全表述：

> Harness的强项是上层tool/runtime约束和取消补偿，不是机械臂IK与mesh collision。tool sandbox通过只能说明调用合同更干净，不能替代低层controller、接触和真实机器人成功。

23. OpenPiVLA、WAM、UWM 的Action Interface边界

23.1 OpenPiVLA

本地formal trainer把action window序列化为assistant JSON并做full-chat causal CE；formal package仍缺generation/parser/controller。official-checkpoint ALOHA/LIBERO diagnostics是平行lineage。deprecated MuJoCo adapter文件自身明确缺official embodiment、controller、IK、workspace和contact semantics。

因此可讲action schema与diagnostic，不能说formal Qwen已经通过PER-015 gate。

23.2 WAM

WAM diagnostic可见7D→12D环境action拼接、dim mismatch hard error、OSC_POSE字段与有限action trace。它们适合检查shape/order/normalizer/controller边界；episode仍失败，缺robot model、planning scene、IK/collision/TOTG报告，不能归因reachability或collision收益。

23.3 UWM

UWM/LIBERO固定脚本有`action_len=16/action_dim=7`、warmup与env.step；RoboCasa GT replay明确定位为action-order/controller probe，不是demo success oracle。74.4%等协议结果不能反向证明通用feasibility gate。

24. 五项目 Project Truth Boundary

项目	当前可以讲	当前不能说	升级所需artifact
OPD	2D semantic target/box/point可作为上游输入	已转成reachable manipulation	typed target、robot/tool、IK/collision trace
Robot Harness	tool scope、timeout/cancel/compensation、map bridge	sandbox等于物理执行安全	controller/trajectory/collision logs、ownership
OpenPiVLA	action serialization与official baseline diagnostics	formal Qwen已闭环parser/controller/IK	strict parser、axis/unit/norm、roundtrip、episodes
WAM	OSC_POSE/action trace与shape诊断	已定位reachability/collision根因	scene/model/gate reports、paired rerun
UWM	7D/16-step LIBERO与RoboCasa action-order probe	结果证明通用gate	checkpoint/norm/controller、raw gate logs

表述顺序必须是：

```
public upstream mechanism
 -> local inventory
 -> local-direct file
 -> diagnostic inference
 -> missing evidence
 -> candidate ownership
```

Git author、目录名、official example或一个可运行脚本都不能替代逐文件ownership与matched result。

25. 两分钟结构化回答

> 我会把perception→action拆成typed interface和九层fail-closed gate。首先semantic target必须带entity、frame、unit、timestamp、score与artifact revision，再结合tool frame、approach、gripper和controller schema形成action target。先用workspace做cheap reject，但它只是必要条件；IK返回后我会独立FK重算SE(3) residual，检查joint limits和Jacobian最小奇异值。TRAC-IK能通过KDL与NLOpt双路径改善joint-limit场景，但官方边界是不做mesh self-collision。接下来区分state和motion validity：MoveIt PlanningScene负责robot/scene collision与constraints，OMPL RRT-Connect依赖外部isValid/checkMotion，resolution、ACM、padding和scene revision都是结果身份。nominal path还要用pose/collision uncertainty膨胀clearance。几何path通过后，TOTG或等价模块生成timestamps并检查velocity、acceleration；controller侧再核验H×D、absolute/delta/velocity、axis、unit、Hz、current state、timeout和status，绝不silent slice/pad。uncertainty要拆成semantic、geometric和execution三类；像KnowNo可用calibration quantile形成prediction set，非singleton就ask，但这只是marginal coverage，不替代碰撞安全。最后输出是controller-ready candidate，execution feedback和task success另算。本地五项目目前只有perception/action/interface或diagnostic hook，没有完整通用gate，所以我会明确哪些是上游方法、哪些是local direct evidence、哪些仍缺。

26. 高频追问

1. semantic target与action target字段有什么本质差别?
2. 为什么frame/unit/timestamp属于结果身份而不是日志字段?
3. workspace内为什么仍可能IK无解?
4. IK solver返回success后为何还要独立算FK residual?
5. rotation与translation tolerance该怎样分开?
6. pseudoinverse IK在joint limit附近为什么容易失败?
7. TRAC-IK两条solver路径分别解决什么，仍缺什么?
8. $\sigma_{\min}$与manipulability各有什么优缺点?
9. rotation/translation单位为什么影响condition number?
10. goal state collision-free为什么不代表path collision-free?
11. OMPL为什么不自动知道机器人collision geometry?
12. motion validation resolution太粗和太细分别有什么问题?
13. attached object与allowed collision matrix为什么会改变结果?
14. clearance inflation怎样由pose/depth uncertainty决定?
15. RRT-Connect timeout能否证明无路径?
16. geometric path为什么还要time parameterization?
17. 7D action shape一致时仍有哪些semantic mismatch?
18. action chunk的H、K、Hz和staleness怎样关联?
19. semantic/geometric/execution uncertainty为何不能用一个confidence?
20. KnowNo的marginal coverage为什么不是collision safety?
21. prediction set为空、单例、多例时分别怎样处理?

22. ask-for-help与replan的触发层有什么区别?

23. cancel发出后为什么还要ack与fresh-state check?

24. 怎样用first-divergence定位“policy server返回但机器人不动”?

25. 本地五项目哪一层有direct evidence，哪一层仍缺?

27. 危险答案与修正

危险答案	问题	安全修正
“检测到3D点就能执行”	缺tool/action/controller合同	先构造typed action target
“点在workspace里就是reachable”	orientation/joints/path未检查	workspace只做cheap reject
“IK返回了数组就是解”	residual/bounds/singularity未知	独立FK与SVD验证
“TRAC-IK会处理碰撞”	官方明确不做mesh self-collision	IK后单独PlanningScene
“goal不碰撞就行”	edge可能穿障碍	state+motion validity
“RRT-Connect找不到就是无解”	sampling/timeout/resolution影响	只能报本次protocol失败
“clearance大于0就安全”	model/sensor/tracking有误差	加model+uncertainty margin
“path平滑就能执行”	没有timestamps与limits	time parameterization
“都是7D所以接口一致”	semantics/order/unit可不同	schema roundtrip
“policy server成功就是机器人成功”	只是RPC返回	查controller status与episode
“confidence 0.9就是90%安全”	score未校准且安全层不同	报calibration/risk–coverage
“conformal保证每个样本”	通常是marginal coverage	明确exchangeability与边界
“不确定就执行慢一点”	ambiguity可能选错对象/路径	ask/re-perceive/replan/stop
“fallback就是默认动作”	unknown被强行执行	有界状态机、fail closed
“Harness已有physical safety gate”	tool sandbox不含物理闭环	只说上层runtime约束
“WAM action trace证明IK没问题”	trace无robot/scene/gate	只作shape/controller诊断

28. 白板 Checklist 与练习标准

能够不看材料完成:

- 画出semantic target→action target→九层gate→execution主链;
- 写出typed manifest的semantic/target/robot/scene/risk五组字段;
- 解释 `pass/fail/not_checked` 为何需要三态;
- 推导2R annulus并完成workspace probe;
- 写出SE(3) residual与pseudoinverse IK update;
- 区分solver return、residual、bounds与branch;
- 写出Jacobian SVD并解释$\sigma_{\min}$;
- 解释TRAC-IK能力与self-collision边界;
- 区分state、motion和path validity;
- 画出MoveIt PlanningScene与OMPL planner责任边界;

- 解释resolution、ACM、attached object与scene revision；
- 推导robust clearance inequality；
- 写出q/qdot/qddot约束与time parameterization；
- 列出controller shape/semantics/order/unit/time/state/status；
- 解释action chunk staleness与replan；
- 区分三类uncertainty；
- 推导conformal qhat与prediction set；
- 画出accept/replan/ask/reperceive/stop/reject状态机；
- 走完first-divergence failure diagnosis；
- 复述10个deterministic probes；
- 说清五项目truth boundary；
- 完成两分钟回答并接住至少十二个追问。

`drafted`不等于掌握。PER-015一张L3/P0题卡已生成但仍为`unseen`；只有真实口述、追问、评分、纠错后才能改变状态。当前266张题卡全部`unseen`，`coaching_state.md`不应创建。

29. 结论与下一步

可靠的perception→action答案不是背“IK、RRT、MoveIt”三个名词，而是建立可审计类型系统：semantic target是否有物理身份，action target是否匹配robot/tool/controller，workspace是否只做cheap reject，IK是否通过residual/bounds/singularity，state与edge是否在冻结scene中无碰撞，uncertainty inflation后是否有clearance，path是否能被time-parameterize，controller current state与schema/status是否一致，风险超过预算时是否abstain。

Modern Robotics、TRAC-IK、MoveIt、OMPL、Transporter/CLIPort和KnowNo分别支撑这条链中的不同原子，不能互相替代；尤其IK不负责mesh collision，planner不自带robot validity，pick/place proposal不是trajectory，conformal coverage不是硬件安全。

本地五项目当前最强的是perception/action/interface或diagnostic evidence：OPD为2D grounding upstream；Robot Harness为tool/runtime与map bridge；OpenPiVLA为action serialization及official diagnostics；WAM/UWM为action/controller protocol probes。没有统一typed gate、robot/tool/scene immutable bundle、IK/collision/TOTG raw report、calibrated risk–coverage、controller execution trace、matched robot episodes或ownership闭环。

`VLA-PER-015` source、knowledge与一张atomic card已在三个独立bounded slices同步；题卡保持`unseen`。下一bounded slice只冻结`VLA-PER-016` localization/odometry/SLAM/map-frame/drift source；不生成其knowledge/card，不创建`coaching_state.md`，不改变任何mastery。

References

1. PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Source Freeze：六份完整PDF、六个official repo/35文件、10个probe与五项目direct audit。
2. Lynch & Park, Modern Robotics；official code@cbd41fd。

3. Kuffner & LaValle, RRT-Connect；OMPL official code@eb3baca。
4. Beeson & Ames, TRAC-IK；official source@57057ce。
5. Zeng et al., Transporter Networks v2。
6. Shridhar et al., CLIPort v1；official code@2be5c47。
7. Ren et al., KnowNo v2；official code@95e3a1d。
8. MoveIt 2 official code@2fb8492。
9. PER-014 3D Grounding Knowledge 与 VOL 03 Action Knowledge。
10. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 16 / 43 · `interview_materials/vla/knowledge/vol_06_per016_localization_odometry_slam_map_frame_drift.html` · SHA-256 `ECBDDB8BD222774181DE3D569C24BC53E6545E3FF387C8660F5138777EBCA775`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`ecbddb8bd222774181de3d569c24bc53e6545e3ff387c8660f5138777ebca775`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per016_localization_odometry_slam_map_frame_drift.md`

VOL 06 Perception / Grounding / Navigation：Localization、Odometry、SLAM、Map Frame 与 Drift（PER-016）

状态：`drafted / VLA-PER-016 / source_synced / card_synced / WARN` 更新日期：2026-07-20 前置：`VLA-PER-002` calibration/frame、`VLA-PER-003` multi-view/time、`VLA-PER-005` navigation policy；后续：`VLA-PER-017` belief/memory/exploration、`VLA-PER-019` navigation evaluation/failure。 来源入口：PER-016 Localization / Odometry / SLAM / Map-frame / Drift Source Freeze。本页只把已冻结的六份primary paper、六个official repo、REP-105、10个deterministic probes与五项目local audit转成可教学、可诊断、可面试口述的知识；一张 `unseen` atomic card已在后续独立bounded slice同步，不创建训练状态，也不改变任何mastery。

问题（Question）

面试官问“机器人怎样知道自己在哪里，odometry、localization和SLAM有什么差别，loop closure怎样修正drift，为什么要区分map与odom frame”时，合格回答不能停在“前端提特征、后端做优化”。必须先冻结sensor、clock、calibration、estimator、frame、map与evaluation身份，再解释local连续状态怎样由propagation/update得到、global loop或relocalization怎样改变 `map→odom`、错误约束为什么会污染全图，以及ATE/RPE的alignment怎样改变结论。

本节点主链：

```
sensor packets + calibration + clock
  -> frontend association / scan match / keyframe
  -> local propagation + filter/window update
  -> continuous odom->base + covariance + tracking state
  -> place candidate
  -> geometric verification + relative transform
  -> robust pose graph / map merge
  -> revisioned map->odom + map state
  -> planner/action consumer freshness gate
  -> timestamp association + SE(3)/Sim(3) evaluation
```

知识地图位置与边界

相邻节点	相邻节点负责	PER-016负责
PER-002	intrinsics/extrinsics、depth、TF方向与pixel→3D基础	state、covariance与map/odom语义如何随时间更新
PER-003	camera/frame/time/presence与multi-view schema	packet order、clock、state history与estimator freshness

相邻节点	相邻节点负责	PER-016负责
PER-005	map-free/map-based、waypoint、planner/controller	planner消费的pose/map revision是否可信
PER-013/014	point/voxel/occupancy与3D semantic query	geometry/semantic map绑定哪个trajectory与revision
PER-015	reachability、collision、trajectory与controller gate	global correction或lost state怎样让旧action target失效
PER-017	belief、frontier、memory与active exploration	不在本页设计POMDP belief或探索策略
PER-019	episode success、collision/stop与统计	不把ATE好或tracking状态直接写成task success

PER-016是state/map contract节点，不是传感器标定教程、导航policy综述，也不是“任意SLAM系统选型排名”。公开论文与代码只支撑机制；本地项目是否集成、候选人是否实现、真实机器人是否成功必须由独立artifact证明。

1. 30秒直觉：局部连续与全局一致不是同一个承诺

一个可靠的30秒回答是：

> 我先区分odometry、localization和SLAM：odometry累积相邻运动，强调连续但会漂；localization在一个有UUID和revision的已知地图里估pose；SLAM同时估trajectory与map。工程上通常发布连续的 `odom→base` 给局部控制，再让loop closure或relocalization更新 `map→odom`，所以global pose可能跳但局部速度不能被伪造。loop candidate必须经过几何验证、relative transform和robust graph residual gate，外观相似不能直接加边。评估时ATE看global trajectory，RPE看local drift，并冻结timestamp association、SE(3)/Sim(3) alignment、单位与lost denominator；特别是Sim(3)会吸收monocular scale error。本地项目当前只有map/PCD/interface资产，没有可核SLAM runtime，所以我只讲接口边界，不讲已部署结果。

这段回答先封住八个偷换：

1. 有pose字段等于已定位；
2. odometry连续等于长期准确；
3. global map pose更准所以可直接做高频控制；
4. place candidate等于loop constraint；
5. loop closure一定改善地图；
6. relocalization、loop closure与map merge是同一事件；
7. ATE低等于尺度、可用性与map质量都好；
8. 有PCD、occupancy或ROS目录等于实现了SLAM。

2. 五个概念必须分开

概念	典型输入	主要输出	map关系	典型失败
tracking	当前sensor与local reference/map	current pose/state、tracking status	消费local reference	blur、textureless、dynamic、association错误
odometry	连续camera/IMU/LiDAR/encoder	relative motion、continuous local trajectory	可无global map	累积translation/yaw/scale drift

概念	典型输入	主要输出	map关系	典型失败
localization	sensor + fixed/revisioned map	map-frame pose、covariance、status	消费已知map	wrong map、alias、kidnapped robot
mapping	pose estimate + observations	occupancy/landmark/surface map	生成或更新map	pose error被写入map
SLAM	sensor stream	trajectory + map + graph/status	同时估计	gauge、wrong association、false loop

专业表达不是“SLAM包含定位和建图”一句话，而是明确估计变量与条件：

- odometry主要约束短时间相对运动，误差通常随路径累积；
- localization假定map身份已知，核心风险包括initial pose、map mismatch和data association；
- SLAM把pose与map变量共同放进filter、sliding window、bundle adjustment或pose graph；
- tracking是运行状态/局部过程，不能代替系统类别；
- mapping若消费错误pose，会生成内部一致但物理错误的地图。

“定位有效”至少要绑定`map_uuid/revision`、sensor/calibration SHA、estimator config/checkpoint、timestamp、tracking state与covariance。裸`x,y,yaw`没有可审计身份。

3. Typed Localization / SLAM Manifest

```
{
  "sensor_contract": {
    "camera_imu_lidar_encoder_ids": [],
    "intrinsics_extrinsics_sha256": "...",
    "axis_unit": "REP-103-like or explicit custom",
    "timestamp_clock_sync": "...",
    "noise_bias_rolling_shutter_model": "..."
  },
  "estimator_contract": {
    "family": "VO | VIO | LiDAR odom | localization | SLAM",
    "paper_repo_commit_config_checkpoint": [],
    "initialization_and_scale": "...",
    "frontend_backend_window_marginalization": "..."
  },
  "frame_contract": {
    "map_odom_base_sensor": [],
    "transform_direction": "T_target_source",
    "map_to_odom_authority": "localizer_or_slam",
    "controller_local_frame": "odom"
  },
  "state_contract": {
    "timestamp_ns": 0,
    "pose_velocity_bias_covariance": [],
    "tracking_state": "initializing | tracking | recently_lost | lost | relocalized",
    "state_revision": "..."
  },
  "map_contract": {
    "map_uuid_revision_sha256": "...",
    "keyframe_landmark_submap_pose_graph_revision": "...",
    "loop_constraints_and_rejections": [],
    "map_update_sequence": 0
  },
  "evaluation_contract": {
    "dataset_split_sequence": "...",
    "association_and_alignment": "timestamp + SE3 | Sim3",
    "ate_rpe_drift_lost_relocal_loop_metrics": [],
    "failure_denominator": "..."
  }
}
```

这个manifest的面试价值是把“算法名”转成可失败的合同。比如同一组pose数值若没有map revision，就不知道global correction前后是否混用；同一ATE若不知道Sim(3)还是SE(3)，就不知道scale error是否被alignment消掉；同一“VIO”若没有clock、noise与extrinsic身份，也无法把漂移归因到模型。

4. REP-105 Frame Tree：Map负责Global，Odom负责Local

典型组合是：

$$T_{\text{base}}^{\text{map}}(t) = T_{\text{odom}}^{\text{map}}(t) T_{\text{base}}^{\text{odom}}(t).$$

变量解释：`T^A_B`把B-frame量变到A-frame；`T^{map}_{odom}`由global localizer/SLAM维护，`T^{odom}_{base}`由local odometry维护。直觉是controller消费连续的local motion，而global correction只改变两个参考系的关系。实现映射：每个consumer记录TF timestamp、direction、map revision和state revision；若PER-015的action target跨越一次global correction，必须重新投影并重做feasibility gate。

4.1 为什么不能只保留一个world frame

- 若把loop correction直接写进高频local pose，controller可能把位置跳变解释成巨大速度；
- 若永远只保留continuous odom，长期drift又会让global goal与map障碍错位；
- 分层不是“复制两份pose”，而是把不同连续性承诺显式化；
- ROS REP-105只是约定，不证明本地TF tree、authority或timestamp正确。

4.2 Correction jump的固定反例

如果global pose从`10m`修正到`8.6m`，同一时刻local odom只前进`0.1m`，则map-frame delta为`-1.4m`而真实local motion仍约`0.1m`。局部控制、速度估计与motion compensation应消费local连续量；global planner则需要接收新map revision并重新验证旧目标。

5. State Estimation：Propagation、Update 与Covariance

常见VIO状态可抽象为：

$$\mathbf{x}_t = \left(R_t, \mathbf{p}_t, \mathbf{v}_t, \mathbf{b}_t^g, \mathbf{b}_t^a\right), \qquad P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q_t.$$

变量解释：`R,p,v`分别是orientation、position、velocity；`b^g,b^a`是gyro与accelerometer bias；`P`是state covariance；`F,Q`是linearized transition与process noise。直觉是IMU能高频propagate，但noise和bias会持续扩大不确定性。实现映射：OpenVINS `Propagator.cpp`对应propagation，`UpdaterMSCKF.cpp/UpdaterSLAM.cpp`对应不同feature update；这些文件存在不代表任何本地项目的noise、clock与calibration正确。

一个线性化measurement update可写为：

$$K_t = P_t^- H_t^\top \left(H_t P_t^- H_t^\top + R_t\right)^{-1}, \qquad P_t^+ = (I - K_t H_t) P_t^-.$$

变量解释：`H_t`把state perturbation映射到measurement residual，`R_t`是measurement noise，`K_t`是Kalman gain。直觉是prediction uncertainty与measurement uncertainty共同决定信任比例。实现映射：若

inlier、innovation、noise model或timestamp错误，covariance数值即使变小也不代表更可信。

固定covariance probe：`P0=.01,Q=.04`传播10步得到prior `.41`；以`R=.09`更新时`K=.82`、posterior约`.0738`。它只验证合同算术，不是OpenVINS benchmark。

6. IMU Preintegration不是“做平均”

VINS-Mono与Kimera把两个keyframes之间的高频IMU sample压成relative-motion factor。preintegration需要：

- camera与IMU各自timestamp和clock offset；
- gyro/accelerometer bias与noise density；
- gravity direction与unit；
- camera–IMU extrinsic；
- bias linearization point与repropagation策略；
- packet dropout、out-of-order与saturation处理。

它的目标是避免每次nonlinear iteration都从头积分所有raw IMU，而不是丢掉sensor identity。若bias或extrinsic改变却继续复用旧preintegrated factor，会把系统误差写进backend。

7. Front-end、Local Estimator、Back-end 与Global Layer

```
sensor packets
  -> frontend: feature/flow/scan association, motion filter, keyframe
  -> local estimator: propagation + filter/window update
  -> backend: residual factors, robust objective, marginalization
  -> global layer: place recognition, geometric verification, pose graph/map merge
  -> typed map/odom state + covariance + tracking status
```

层	主要职责	必须保留的证据	常见误判
frontend	detect/track/scan-match、keyframe、association	inliers、residual、blur/dynamic、candidate IDs	点多等于pose对
local estimator	current pose/velocity/bias/covariance、local map/window	innovation、conditioning、lost status	covariance小等于准确
backend	nonlinear objective、factors、marginal prior	cost、factor residual、solver status	cost下降等于合同正确
global layer	place、loop/relocal/merge、global graph	geometry inliers、relative transform、graph delta	candidate等于constraint

前端、后端与global layer是责任分解，不是固定线程数。DROID-SLAM把learned update与dense BA耦合，OpenVINS是modular EKF，Cartographer按local submap与global constraint组织；仍需逐层保留输入输出与failure state。

8. Robust Objective、Gauge 与可观性

一般估计目标可写成：

$$\hat{\mathcal{X}} = \arg\min_{\mathcal{X}} \sum_{i \in \mathcal{F}} \rho_i \left(\| \mathbf{r}_i(\mathcal{X}; \mathbf{z}_i) \|^2_{\Sigma_i^{-1}} \right).$$

变量解释：`\mathcal X`包含poses、velocity、bias、landmarks或depth；`\mathcal F`是visual/IMU/scan/loop factors；`\rho_i`是robust loss；`\Sigma_i`是measurement covariance。直觉是多类约束共同决定state，outlier不能与正常measurement等权。实现映射：VINS sliding window、Cartographer scan/pose graph、DROID dense BA和Kimera factor graph的state与residual不同，不能只说“都做BA”。

8.1 Gauge freedom

没有global anchor时，把整条trajectory与map共同平移或旋转，relative residual可能不变；monocular还常有scale ambiguity。这意味着：

- optimizer需要fix gauge或引入prior；
- evaluation alignment是在处理坐标自由度，不是美化步骤；
- Sim(3)可吸收scale，SE(3)不能；
- 加IMU能改善metric scale与gravity约束，但bias、excitation、time/extrinsic错误仍会污染可观性；
- “VIO有IMU所以绝不漂”是错误结论。

9. Keyframe、Sliding Window 与Marginalization

保留全部frame会让state、memory与optimization不断增长。常见做法是选keyframe、保留fixed-lag window，并把旧变量的影响压成prior。

主要取舍：

- window太短会丢失长期约束，太长会增加latency和memory；
- keyframe阈值影响feature baseline、graph density、loop candidate和metric；
- marginalization把旧信息保留下来，但也可能把错误association固化进prior；
- 线性化点变化会带来consistency风险；
- filter与optimization不是“一个快、一个准”的普遍二选一，必须在同sensor、同预算、同协议下比较。

面试时可把VINS-Mono `estimator.cpp`、Kimera `VioBackend.cpp`与OpenVINS clone/state management作为三种组织方式，但不能反推本地系统选择或效果。

10. Loop Closure：四道门，不是一次相似度检索

```
place candidate
  -> temporal/map exclusion
  -> descriptor or learned match
  -> geometric verification + relative pose
  -> robust residual / inlier / graph-consistency gate
  -> pose graph optimization
  -> map-to-odom correction + new map revision
```

pose graph可写为：

$$\{\hat{T}_i\} = \arg\min_{\{T_i\}} \sum_{(i,j)\in\mathcal{E}} \rho\left(\left\|\mathrm{Log}\left(Z_{ij}^{-1}T_i^{-1}T_j\right)\right\|^2_{\Omega_{ij}}\right).$$

变量解释：`Z_{ij}`是odometry或loop relative-pose measurement，`\Omega_{ij}`是information matrix，`T_i`是node pose。直觉是让所有relative constraints一致，同时降低outlier边的破坏力。实现映射：VINS `pose_graph.cpp`维护drift transform并做4-DoF optimize；ORB-SLAM3 `LoopClosing.cc`区分loop与merge；Cartographer构建node/submap constraints；Kimera论文用robust PGO处理outlier。

固定false-loop probe：odometry chain要求`x_2=2`，外观candidate却要求`x_2=0`，loop residual为`2>.5`时必须拒绝。若系统为了"闭环成功率"直接加入该边，optimizer可能把全图拉成错误但低cost的形状。

11. Relocalization、Loop Closure 与Map Merge

事件	触发	主要输出	不可偷换
relocalization	tracking lost / known map	恢复current pose、tracking state变化	不一定新增loop或改全图
loop closure	current与history同地点	verified loop constraint、global correction	candidate不是constraint
map merge	current map与另一map/session重叠	inter-map transform、map fusion/revision	不是拼接point clouds

ORB-SLAM3 Atlas允许tracking lost后新建map，并尝试跨map relocalization/merge；VINS-Mono提供relocalization与4-DoF pose graph。这些公开设计可用于比较state machine，但不能说本地Robot Harness已有相同Atlas、thread或recovery机制。

12. Drift Taxonomy：先找第一处分叉

Drift / failure	常见来源	第一检查	不能直接归因
translation drift	incremental noise、poor geometry、mismatch	RPE、inliers、motion excitation	"缺global BA"
yaw drift	gyro bias、弱heading约束	bias、gravity、loop evidence	"视觉模型差"
scale drift	monocular gauge、弱excitation	sensor mode、SE3/Sim3	"单位换算"
vertical drift	bias、gravity、ground assumption	gravity、axis、bias	"地图高度错"
time drift	clock skew、delay、interpolation	timestamps、queue age	"随机抖动"
map corruption	false loop、dynamic、wrong map	loop residual、graph delta、map UUID	"optimizer失败"
tracking loss	blur、occlusion、textureless、dynamic	frontend status、relocal log	"直接多加keyframe"
global jump	loop/relocal correction	map→odom event、consumer frame	"机器人高速移动"

IMU改善某些短期约束与可观性，但不会修复错误clock/extrinsic；loop closure改善长期一致性，但false loop可能比不闭环更坏。回答要讲"哪个error source由哪个measurement约束"，而不是堆sensor名称。

13. Learned SLAM：学习模块仍受几何与协议约束

DROID-SLAM在dynamic frame graph上用recurrent update operator预测correspondence update，再调用differentiable dense BA更新pose和inverse depth。它说明learned front/back-end coupling可以成立，但仍需

要：

- camera intrinsics与input preprocessing；
- checkpoint、training domain与sensor mode；
- frame graph、keyframe与memory threshold；
- GPU memory、latency与termination；
- lost/failure status与trajectory filling；
- 同association/alignment协议的ATE/RPE。

因此“learned”不等于不需要geometry，“end-to-end”不等于没有graph/state，“benchmark好”也不等于目标机器人上无distribution shift。

14. ATE、RPE 与Alignment Protocol

Absolute Trajectory Error在timestamp association与alignment后可写为：

$$\mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\mathrm{trans}\left(T_{\mathrm{align}}\hat{T}_i(T_i^*)^{-1}\right)\right\|_2^2}.$$

变量解释：`\hat T_i`是estimate，`T_i^*`是ground truth，`T_{align}`来自SE(3)或Sim(3) alignment。直觉是测global trajectory形状/位置误差。实现映射：必须保存association tolerance、alignment family、单位、采样频率、轨迹截断与lost段处理。

Relative Pose Error可写为：

$$E_i^{\Delta} = \left((T_i^*)^{-1}T_{i+\Delta}^*\right)^{-1}\left(\hat{T}_i^{-1}\hat{T}_{i+\Delta}\right).$$

变量解释：`\Delta`按frame、time或distance定义；RPE关注local incremental error，较少受global offset影响。实现映射：GT step为`1`、estimate step为`1.1`时translation RPE RMSE为`.1`，但必须说明delta和reduction。

Metric	回答什么	必须冻结	不能推出
ATE	global trajectory一致性	association、SE3/Sim3、unit、lost truncation	local control稳定
RPE	local incremental drift	delta、translation/rotation reduction	map全局正确
drift %	路程归一化长期误差	segment length、route、loop policy	任意路径泛化
lost/relocal	可用性与恢复	lost定义、timeout、denominator	pose精度
loop precision/recall	place/constraint质量	candidate/verified、GT loop定义	map geometry质量
map quality	occupancy/mesh/landmark质量	pose source、coverage、resolution、threshold	policy success

14.1 Alignment反例

- GT `[0,1,2]`、estimate `[5,6,7]`的raw RMSE为`5`；translation alignment后可变成`0`；
- estimate `[0,1.2,2.4]`在SE(3)下RMSE约`.1633`，Sim(3)用scale `.8333`后可变成`0`。

因此alignment不是脚注。对monocular结果用Sim(3)合理处理scale gauge时，必须同时承认“metric scale没有被该分数验证”。

15. 六类系统怎样比较

系统	核心组织	强项式回答	必须保留的边界
ORB-SLAM3	tracking/local mapping/loop & merge/Atlas	visual-inertial multi-map、relocal/merge	sensor/setup与local deployment
VINS-Mono	IMU preintegration + sliding-window VIO + 4DoF pose graph	bias/scale、marginalization、relocal	calibration/time/excitation
Cartographer	local scan/submap + global constraints	local continuity与global loop分层	LiDAR/map config与robot TF
DROID-SLAM	recurrent update + dynamic graph + dense BA	learned correspondence与geometry coupling	checkpoint/domain/latency/global behavior
OpenVINS	modular on-manifold EKF + MSCKF/SLAM updates	state/covariance与filter模块化	不含完整loop-closing global SLAM
Kimera	stereo/IMU frontend + factor backend + loop/semantic stack	metric-semantic modular pipeline	VIO repo不等于全Kimera stack

安全比较顺序是`input sensor → state/map → local/global architecture → failure/status → compute → metric protocol`。不要做“哪个最好”的无条件排名。

16. 十个Deterministic Probes

Probe	固定输入	结果	教学结论
map/odom composition	`map→odom=-2m`，`odom→base=10m`	`map→base=8m`	frame chain必须显式
SE(2) roundtrip	yaw 90°、t `[2,3]`、p `[1,0]`	q `[2,4]`，inverse error `6.12e-17`	transform方向可测
covariance	Po `.01`、Q `.04`、10步、R `.09`	prior `.41`，K `.82`，post `.0738`	无update uncertainty增长
ATE alignment	GT `[0,1,2]`，est `[5,6,7]`	raw 5，aligned 0	alignment属于身份
RPE	GT step 1，est step 1.1	RMSE `.1`	local drift与offset分开
false loop	chain x2=2，loop要求x2=0	residual 2>.5，reject	candidate不能直接加边
scale protocol	est `[0,1.2,2.4]`	SE3 `.1633`，Sim3后0	Sim3隐藏scale error
correction jump	map pose `10→8.6`，odom delta `.1`	map `-1.4` vs odom `.1`	controller避免global jump
timestamp	`[0,.1,.09]`	non-monotonic reject	order/clock属于输入身份
local fallback	`runtimeReady=false/softwareOnly=true`	localization gate false	map asset不是live localization

10个probe得到`10/10 expected decisions matched`。它们验证frame、state、metric与failure合同，不是ORB-SLAM3、VINS、Cartographer、DROID、OpenVINS或Kimera benchmark复现。

17. First-divergence Failure Diagnosis

表面症状	第一检查层	关键证据	常见误诊
pose整体翻转/旋转	frame direction/axis	T convention、REP/custom axes	调loop threshold
开始即漂	initialization/scale/gravity	excitation、bias、sensor mode	只怪地图
匀速累积误差	frontend/local estimator	RPE、bias、inliers	直接加global BA
某时刻突然跳	map→odom correction	loop/relocal event、graph delta	当真实高速运动
闭环后全图扭曲	false loop/weight	geometry inliers、residual	说optimizer随机坏了
tracking反复lost	sensor/frontend	blur、texture、dynamic、timestamp	只多加keyframe
ATE很好但尺度错	alignment	Sim3/SE3、estimated scale	报metric scale正确
map可见但机器人不动	localization/runtime boundary	runtimeReady、pose age、frame	报SLAM成功
同轨迹不同分数	evaluation protocol	association/alignment/truncation	报模型回归
重启后位置错误	map/session identity	UUID、load path、relocal log	归因random seed

诊断顺序：

```
sensor identity / packet order / clock
  -> calibration / frame direction
  -> initialization / scale / gravity
  -> frontend association
  -> local state / covariance / marginal prior
  -> loop geometric gate
  -> graph correction / map revision
  -> ATE/RPE protocol
  -> planner / controller consumer
```

18. Fail-closed Localization Pipeline

```
def update_localization(packet, manifest, estimator, map_bundle):
    # 1. Identity first: never guess sensor, frame, time, or calibration.
    require(packet, ["sensor_id", "timestamp_ns", "frame_id", "calibration_sha256"])
    require(manifest, ["estimator_revision", "frame_contract", "evaluation_contract"])
    if packet.timestamp_ns <= estimator.last_timestamp_ns:
        return {"status": "reject", "reason": "NON_MONOTONIC_TIME"}

    # 2. Propagate local state and covariance with the frozen sensor model.
    predicted = estimator.propagate(packet.imu, manifest.noise_bias_model)
    if not predicted.finite or predicted.covariance_exceeds(manifest.local_risk_budget):
        return {"status": "lost", "reason": "PROPAGATION_UNCERTAIN"}

    # 3. Frontend measurements may be absent or outliers; status stays explicit.
    measurements = estimator.frontend(packet)
    local = estimator.local_update(predicted, measurements)
    if not local.valid:
        relocalized = estimator.relocalize(packet, map_bundle)
        if not relocalized.geometrically_verified:
            return {"status": "lost", "reason": "RELOCALIZATION_FAILED"}
        local = relocalized.state

    # 4. A visual/place candidate is not a graph edge until geometry passes.
    candidate = estimator.detect_loop(local, map_bundle)
    if candidate is not None:
        verified = estimator.verify_loop(candidate, manifest.loop_gate)
        if verified.accepted:
            optimized = estimator.optimize_pose_graph(verified.constraint)
            if optimized.max_residual > manifest.graph_residual_limit:
                return {"status": "reject_loop", "reason": "GRAPH_INCONSISTENT"}
            map_bundle = optimized.new_map_revision

    # 5. Publish local continuity and global correction separately.
    return {
        "status": "tracking",
        "odom_to_base": local.odom_to_base,
        "map_to_odom": estimator.map_to_odom(map_bundle),
        "covariance": local.covariance,
```

```
        "map_revision": map_bundle.revision,
        "robot_success": None,
    }
```

伪代码保留`initializing/tracking/lost/relocalized/reject_loop`语义与map revision。它是面试中的目标合同，不是五项目现有实现。生产系统还要处理queue/backpressure、sensor dropout、concurrent map update、TF cache、watchdog、kidnapped robot、controller stop与immutable evaluation bundle。

19. Public Method → Code Mapping

机制	fixed official code	面试可讲	不能反推
tracking/relocal/map atlas	ORB-SLAM3 Tracking/LoopClosing/Atlas	lost/relocal、loop vs merge、multi-map	local deployment
sliding-window VIO	VINS estimator/IntegrationBase	preintegration、marginalization、bias/scale	arbitrary sensor robustness
drift/pose graph	VINS pose_graph	world↔VIO transform、4DoF optimize	false loop自动无害
local/global LiDAR SLAM	Cartographer local builder/constraint/pose graph	submap、scan match、global loop	visual/semantic mapping
learned dense BA	DROID droid/factor graph/backend	recurrent update + dense BA	checkpoint/latency/generalization
modular EKF	OpenVINS manager/propagator/updaters	state/covariance、MSCKF vs SLAM feature	full global SLAM
VIO + loop candidate	Kimera frontend/backend/LCD	stereo+IMU、factor graph、loop detection	VIO repo等于完整Kimera stack

代码映射用于回答“机制落到哪里”，不是ownership证据。固定commit、license和文件SHA已经在source freeze中记录；论文、dataset、driver、map、weights与第三方依赖仍有独立license和provenance边界。

20. Robot Harness：Map Asset 与Live Localization的边界

Robot Harness的local-direct证据包括：

- `occupancy2d_map.json`：0/127/255语义、0.02m resolution、origin与conversion参数；
- `world.pcd`：PCD v0.7，含`x y z intensity timestamp ring`，共54,856点；
- `reloc_map/poses.txt`：timestamp与pose/quaternion-like八列资产；
- `ros2_bridge.py`：map/PCD/reloc路径、pixel↔world与state fallback；
- simulator fixture：`localization_mode/map_id/valid`字段；
- fallback targeted test：`1 passed`，并明确`runtimeReady=false/runtimeRealApiCalls=false`。

这些证据能支持“系统有地图资产schema与software-only bridge”，不能支持“实现了SLAM、loop closure或可靠relocalization”。缺失项包括sensor bag、clock/calibration lineage、estimator config/commit、live TF、pose/covariance stream、keyframe/pose graph、loop accept/reject、map UUID lineage和ATE/RPE trajectory bundle。

安全表述：

> Robot Harness里我能核验的是occupancy、PCD、reloc pose资产以及pixel/world和state bridge；fallback测试明确不是live runtime。没有estimator和trajectory证据前，我不会把这些资产包装成自研SLAM或真实定位结果。

21. 其余四项目的Localization Hook

21.1 OPD

OPD支持2D point/box/relation supervision与static holdout，可作为semantic observation upstream；没有sensor trajectory、3D map、localizer status或SLAM graph。不能说“做了semantic SLAM”。

21.2 OpenPiVLA

OpenPiVLA支持image/language/action训练与sim diagnostic。policy消费state或image不证明state来自可审计localizer；至少需要frame/pose lineage、localizer status、map revision与matched rollout。

21.3 WAM

WAM有video/world/action diagnostic，可讨论prediction对pose error的敏感性；world model latent不是机器人pose graph，也不能从video prediction推出定位改善。

21.4 UWM / LIBERO

UWM/LIBERO使用simulator state/action protocol；sim ground-truth state不是estimator output。若要研究localization robustness，应显式加入noisy observation estimator并保存raw estimated/GT trajectory与protocol。

22. 五项目 Project Truth Boundary

项目	当前可以讲	当前不能说	最小补强artifact
OPD	2D grounding可作semantic observation	已做3D localization/semantic SLAM	sensor/pose/map manifest、trajectory-query绑定
Robot Harness	map asset schema、pixel/world bridge、software-only state	已部署SLAM、loop或relocalization	sensor bag、config、TF/graph、ATE/RPE、live logs
OpenPiVLA	policy侧image/state/action接口诊断	policy结果证明localization正确	pose lineage、status、map revision、matched rollout
WAM	video/world/action trace与pose-error sensitivity假设	world model等于SLAM或提升定位	camera trajectory、estimator/map revision、paired eval
UWM	simulator state/action protocol	simulator GT等于估计器输出	noisy estimator、raw estimated/GT trajectory

回答顺序必须是：

```
public paper mechanism
  -> fixed official code
  -> local canonical inventory
  -> local-direct asset/interface
```

```
   -> missing estimator/runtime/result
   -> candidate ownership confirmation
```

Git author、目录名、ROS package、地图文件和一个green fallback test都不能替代个人实现与matched result。

23. 两分钟结构化回答

> 我会先把问题拆成odometry、localization和SLAM。Odometry主要从相邻sensor估relative motion，给连续local trajectory但误差累积；localization在一个有UUID/revision的已知map里估pose；SLAM共同估trajectory与map。系统身份不能只有算法名，还要固定camera/IMU/LiDAR、clock、intrinsic/extrinsic SHA、noise/bias、estimator commit/config/checkpoint、frame direction、map revision与tracking state。状态侧，IMU propagation更新pose、velocity、bias和covariance，视觉或scan measurement再update；VINS/Kimera常用preintegration与sliding window，OpenVINS是modular EKF，但filter和optimization不能脱离同协议排名。工程上按REP-105把连续 `odom→base` 与可能跳变的 `map→odom` 分开，loop或relocalization更新global relation，局部controller不应把jump当真实速度。Loop closure也不是检索到相似图就加边，而是candidate、geometry、relative transform、robust graph residual和new map revision五步；relocalization恢复current pose，map merge对齐不同session，它们不是同一事件。误差要按translation/yaw/scale/time/false-loop/lost分类，从sensor与clock开始找第一处分叉。评估时ATE看global trajectory、RPE看local drift，必须报告timestamp association、SE(3)/Sim(3)、delta、unit与lost denominator，因为Sim(3)会隐藏monocular scale error。公开ORB-SLAM3、VINS、Cartographer、DROID、OpenVINS和Kimera只支撑这些机制。本地五项目目前没有完整SLAM runtime；Robot Harness只有map/PCD/reloc assets和software-only bridge，所以我会明确讲接口证据，不声称自研SLAM或真实定位指标。

24. 高频追问

1. tracking、odometry、localization、mapping和SLAM的state分别是什么?
2. 为什么有pose字段不能证明localization有效?
3. `map→odom→base` 每条边由谁维护?
4. map pose为什么允许跳，odom为什么允许漂?
5. controller直接消费map-frame pose有什么风险?
6. IMU propagation为什么会让covariance增长?
7. measurement update中 `Q` 与 `R` 各表示什么?
8. IMU preintegration为什么依赖bias linearization point?
9. frontend inlier很多为什么仍可能pose错?
10. backend cost下降为什么不证明frame/time合同正确?
11. filter与sliding-window optimization怎样公平比较?
12. gauge freedom是什么，为什么需要anchor或alignment?
13. monocular scale ambiguity怎样影响metric claim?
14. keyframe阈值怎样影响compute与loop?
15. marginalization为什么可能引入inconsistency?

16. loop candidate到graph edge之间有哪些门?
17. false loop为什么可能比不闭环更糟?
18. relocalization、loop closure与map merge怎样区分?
19. pose graph中的`Z_ij`与`Omega_ij`是什么?
20. DROID-SLAM既learned又为什么仍有BA与graph?
21. ATE与RPE分别对global offset和local drift多敏感?
22. SE(3)与Sim(3) alignment何时使用，隐藏什么?
23. lost/relocalization rate的denominator怎样定义?
24. map quality与trajectory quality为什么不能用一个分数?
25. Robot Harness的PCD与reloc poses为什么不是SLAM证据?
26. 怎样把localization revision传播给planner与PER-015 action gate?

25. 危险答案与修正

危险答案	问题	安全修正
“SLAM就是定位加建图”	没有state/map/protocol	先区分输入、估计变量与map假设
“里程计一直有pose所以可靠”	连续不等于无漂移	报RPE、covariance与lost state
“map frame一定比odom准”	global correction可跳	global一致与local连续分工
“VIO有IMU就有metric scale且不漂”	bias/excitation/time仍影响	报sensor mode与observability
“前端点多说明tracking好”	association可系统性错误	查geometry residual与state innovation
“optimizer cost降了所以对”	错误模型也可低cost	查frame/time/scale与held-out residual
“检索相似地点就能闭环”	缺geometry与relative pose	candidate后做verify与graph gate
“loop closure一定修正drift”	false loop会毁图	报precision、residual与reject log
“relocalization就是loop closure”	trigger/output不同	分开current pose恢复与global edge
“拼两张点云就是map merge”	缺inter-map transform与revision	保存verified transform与map lineage
“ATE低说明定位各方面都好”	不含local drift/lost/map quality	配套RPE、availability和map metric
“Sim3对齐只是坐标换一下”	scale error被吸收	明确scale与SE3对照
“DROID端到端不需要几何”	仍有frame graph与dense BA	讲learned update与geometry coupling
“有world.pcd说明做了SLAM”	资产无producer lineage	要sensor/config/graph/trajectory
“fallback test过了说明定位上线”	明确softwareOnly	只说offline asset bridge可用
“Git里有代码就是我实现的”	ownership未确认	分开upstream、local file与个人决策

26. 白板 Checklist 与练习标准

能够不看材料完成:

- 用表格区分tracking/odometry/localization/mapping/SLAM;
- 写出sensor/estimator/frame/state/map/evaluation六组manifest;
- 推导`map→odom→base`组合并解释authority;

- 用correction-jump反例说明local controller选择；
- 写出VIO state、covariance propagation和measurement update；
- 解释preintegration的clock/bias/extrinsic依赖；
- 画出frontend/local/backend/global分层；
- 写出robust residual objective并解释gauge；
- 说明keyframe/window/marginalization取舍；
- 写出loop candidate→geometry→graph→revision链；
- 区分relocalization、loop closure与map merge；
- 写出pose-graph residual与information matrix；
- 复述八类drift与first-divergence顺序；
- 说明DROID learned update与dense BA边界；
- 写出ATE、RPE与SE3/Sim3协议；
- 复算10个deterministic probes；
- 走通fail-closed pseudocode；
- 比较六类公开系统而不做无条件排名；
- 说清五项目truth boundary；
- 完成两分钟回答并接住至少十二个追问。

`drafted`不等于掌握。PER-016一张L3/P1题卡已在后续独立card slice生成并保持`unseen`；当前267张题卡全部`unseen`。只有真实口述、追问、评分、纠错后，相关状态才可能变化。

27. 结论与下一步

可靠的localization/SLAM回答不是堆“特征、BA、回环”名词，而是建立三条可审计链：第一条是sensor/clock/calibration到state/covariance的local estimation链；第二条是candidate、geometry、relative pose、robust graph到map revision的global correction链；第三条是timestamp association、SE(3)/Sim(3)、ATE/RPE与failure denominator的evaluation链。

`odom`承诺local continuity但允许drift，`map`追求global consistency但允许correction jump；loop closure、relocalization和map merge有不同trigger与output；ATE、RPE、scale、lost与map quality也不能合并。ORB-SLAM3、VINS-Mono、Cartographer、DROID-SLAM、OpenVINS与Kimera分别支撑不同机制，不能反推本地部署或candidate ownership。

本地最接近本节点的是Robot Harness的occupancy、PCD、reloc poses与software-only state bridge，但没有producer estimator、live TF、pose graph、loop log或trajectory evaluation。OPD、OpenPiVLA、WAM与UWM也只有上游/interface hook。因此所有项目叙述继续保持`WARN`和evidence boundary。

`VLA-PER-016` source、knowledge与一张atomic card已在三个独立bounded slices同步；题卡保持`unseen`，不创建`coaching_state.md`、不改变任何mastery。下一bounded slice只冻结`VLA-PER-017` belief/memory/frontier/active-exploration source；不生成其knowledge/card。

References

1. PER-016 Localization / Odometry / SLAM / Map-frame / Drift Source Freeze：六份完整PDF、六个official repo/36文件、REP-105、10个probe与五项目local audit。
2. Campos et al., ORB-SLAM3 v2；official code@4452a3c。
3. Qin et al., VINS-Mono v1；official code@90dabb5。
4. Hess et al., Real-Time Loop Closure in 2D LIDAR SLAM；Cartographer code@877157a。
5. Teed & Deng, DROID-SLAM v2；official code@2dfd39f。
6. Geneva et al., OpenVINS ICRA 2020；official code@6948812。
7. Rosinol et al., Kimera v3；Kimera-VIO code@ce8c59b。
8. ROS REP-105, Coordinate Frames for Mobile Platforms@11ca24a。
9. PER-002 3D Lift / Frame Knowledge、PER-003 Multi-view / Temporal Knowledge、PER-005 Navigation Policy Knowledge 与 PER-015 Perception→Action Gate Knowledge。
10. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 17 / 43 · `interview_materials/vla/knowledge/vol_06_per017_belief_memory_frontier_active_exploration.html` · SHA-256 `29C8B3D4A55DC6C403162486DB04245F302FF3E5B91C8DFC3BBE8B2809B8CED5`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`29c8b3d4a55dc6c403162486db04245f302ff3e5b91c8dfc3bbe8b2809b8ced5`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per017_belief_memory_frontier_active_exploration.md`

VOL 06 PER-017：Belief、Memory、Frontier 与 Active Exploration

状态：`knowledge_drafted / VLA-PER-017 / source_frozen / card_synced / project_truth_bounded / WARN` 更新日期：2026-07-20 权威来源：PER-017 Source Freeze 学习边界：本页把已冻结的 POMDP、memory、frontier、active exploration 与 evaluation contract 转化为可学习讲义；下游单张题卡已在独立 bounded slice 同步且保持 `unseen`，不改变 mastery，不进入 PER-018 的 BC/DAgger/RL 训练，也不把公开方法写成本地项目成果。

问题（Question）

在 partial observation 的导航中，机器人如何用 belief 和不同形态的 memory 汇总历史，怎样从 occupancy map 提取可达 frontier，怎样权衡 information gain、目标概率、coverage、路程和风险，以及怎样诊断 revisit、stuck、unreachable frontier 和 stale map memory?

1. 学习出口

学完本页，面试中应能完成六件事：

1. 用 POMDP 语言严格区分 hidden state、observation、action、history、belief 与 policy。
2. 解释 recurrent latent、explicit spatial map、episodic history 三种 memory 的信息、优缺点和 reset/invalidation 条件。
3. 写出 Bayes belief update，并说明 zero-likelihood、underflow、model mismatch 时为什么必须 fail closed。
4. 从 `known-free / unknown / occupied / predicted-free` 地图语义出发，讲清 frontier extraction、聚类、可达性和 risk gate。
5. 把 information gain、goal probability、coverage/novelty、travel cost 与 risk 拆成可审计的 utility，而不是只说“探索未知区域”。
6. 给出 coverage/visitation/revisit/frontier-reach 的评估合同，并守住五项目“只有接口、没有 exploration runtime”的真实性边界。

2. 一分钟心智模型

```
hidden environment state s_t
        ↓ 只产生局部 observation o_t
Bayes / learned belief update + typed memory write
        ↓
belief + recurrent latent + spatial map + episodic events
        ↓
known-free / unknown boundary → frontier components
        ↓
reachability + freshness + risk hard gates
        ↓
IG + goal probability + coverage + novelty - distance - risk
        ↓
choose frontier/viewpoint → bounded plan prefix → observe again
        ↓
progress/revisit/stuck/map-revision watchdog → replan or stop
```

最重要的顺序是：**先确认信息状态可信，再产生探索候选；先做 hard gate，再做 soft ranking；执行有限前缀后重新观测，而不是一次规划到底。**

3. POMDP：为什么不能只看当前帧

部分可观测马尔可夫决策过程（Partially Observable Markov Decision Process, POMDP）可写为：

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, \mathcal{O}, Z, R, \gamma \rangle, \qquad b_t(s) = P(s_t = s \mid o_{1:t}, a_{1:t-1}).$$

变量解释：`S` 是真实但不可完全读取的环境状态；`A` 是动作；`T` 是状态转移；`O` 是观测空间；`Z` 是 observation likelihood；`R` 是 reward；`γ` 是折扣；`b_t` 是给定历史后的 state posterior。直觉上，当前图像可能看不到门后、被遮挡目标或累计定位误差，因此策略需要一个 information state。实现映射：belief record 必须绑定 `episode_id`、observation/action schema、transition/observation model revision、timestamp 与 normalization；只有一个 RNN hidden tensor 不能自动满足这些语义。

量	回答的问题	导航例子	常见误写
hidden state `s_t`	世界真实是什么	真实 pose、完整障碍、目标位置	把 simulator GT 当部署输入
observation `o_t`	此刻传感器看到了什么	RGB-D、IMU、局部 collision	把一帧图像当完整 state
history `h_t`	过去发生过什么	observation/action/result 序列	不记录 episode/time/revision
belief `b_t`	基于模型和历史，现在相信什么	pose/object/map posterior	把任意 feature 称为概率 belief
policy `π`	根据 information state 做什么	选 frontier、waypoint 或 stop	belief update 与 action head 混为一层

4. Bayes Belief Update 与 Zero-likelihood Gate

离散 belief update 分为 prediction、correction 与 normalization：

$$\bar{b}_t(s') = \sum_s T(s' \mid s, a_{t-1}) b_{t-1}(s), \qquad \tilde{b}_t(s') = Z(o_t \mid s', a_{t-1}) \bar{b}_t(s'), \qquad b_t(s') = \frac{\tilde{b}_t(s')}{\sum_x \tilde{b}_t(x)}.$$

变量解释：`\bar b_t` 是动作后的 prior，`\tilde b_t` 是乘 observation likelihood 后的未归一化 posterior，分母是 evidence/normalizer。直觉上，动作模型先把可能状态向前传播，传感器再排除与观测不一致的状

态。实现映射：source probe 使用 prior `[0.6,0.4]`、likelihood `[0.2,0.8]`，得到 posterior `[0.272727,0.727273]`；代码必须检查 finite、sum>阈值和 model/schema revision。

4.1 为什么不能把全零 likelihood 变成均匀分布

若 `\sum_x\tilde b_t(x)=0`，可能原因包括：observation model 覆盖不全、单位/坐标错误、数值下溢、错误 episode、传感器异常或真实状态不在 hypothesis set。静默改成均匀分布会把模型错误伪装成“高不确定性”，随后策略仍可能输出危险动作。

正确路径是：

- 返回 `invalid_belief`，保留 last-safe belief 但标记 stale；
- 切到 log-domain 或扩大 hypothesis set，仅在数学原因明确时恢复；
- 请求新 observation、relocalization 或 safe-stop；
- 记录 normalizer、model revision、packet identity 与 rejected reason。

4.2 Log-domain 实现

对小概率连乘，可用 log-sum-exp：

$$\log b_t(s') = \log Z(o_t \mid s', a_{t-1}) + \mathrm{LSE}_s \left(\log T(s' \mid s, a_{t-1}) + log b_{t-1}(s)\right) - \log \eta_t.$$

变量解释：`LSE` 是 log-sum-exp，`η_t` 是归一化常数。直觉是把概率乘法变成 log 空间加法，避免浮点下溢。实现映射：需要对 impossible transition 使用 `-inf`、对 all-`-inf` row 单独 gate，并在 normalize 后验证 `logsumexp(log_b)=0`；本地五项目没有该实现证据，因此这里只是标准工程合同。

5. Memory 不是一个模块名

5.1 Recurrent latent memory

$$h_t = f_\theta(h_{t-1}, e(o_t), e(a_{t-1})).$$

变量解释：`h_t` 是 GRU/LSTM/Transformer cache 等 recurrent state，`e(·)` 是 observation/action encoder。直觉是把长历史压成固定维向量。实现映射：EmbodiedQA official code 使用 action/visual history 与 LSTM hidden state；但若没有 hidden reset、sequence truncation、rollout ablation 和 calibration，不能说 `h_t` 等价于显式 belief 或地图。

优点：紧凑、端到端可学习、可表达难以手工设计的历史特征。缺点：语义不透明、容量有限、容易 episode leakage、难以做 map revision invalidation，也未必表达多模态 uncertainty。

5.2 Explicit spatial memory

空间地图至少应拆成：

Layer	语义	必要身份	典型 consumer
observed occupancy	直接观测的 free/occupied	sensor/model/time/map revision	collision gate、planner
predicted occupancy	anticipation 的 free/occupied probability	checkpoint/calibration/domain	soft utility、risk estimate
known/explored	是否真正进入 sensor visibility	visibility model/cell size	frontier、coverage metric
semantic	category/object/affordance score	class vocabulary/checkpoint	ObjectGoal utility
visited	agent真正到达或观测过的位置	pose/frame/time quantization	novelty、revisit/stuck
uncertainty	pose/map/semantic confidence	estimator/calibration revision	IG、hard risk gate

最危险的混淆是：`unknown`、padding、outside-map、传感器不可见、概率0.5 与 anticipation 的 predicted-free 被编码成同一个数。经典 frontier 只能从 **observed known-free** 一侧产生；predicted-free 最多在额外 risk budget 下辅助排序或扩展规划。

5.3 Episodic / history memory

episodic memory 记录“何时、何地、看到了什么、做了什么、结果怎样”，最小字段为：

```
{
  "episode_id": "...",
  "event_id": "...",
  "timestamp_ns": 0,
  "pose_frame_revision": ["map", "map_revision"],
  "observation_ref": "content-addressed artifact",
  "action_and_result": {},
  "belief_or_map_delta_ref": "...",
  "confidence_and_failure_reason": {},
  "producer_model_revision": "..."
}
```

文本对话压缩也属于 history management，但只有当记录绑定 navigation observation、pose/map revision、action/result 与 episode lifecycle 时，才可升级为 embodied episodic memory。

6. Typed Exploration State Manifest

```
{
  "episode_contract": {
    "episode_id_seed_scene_start_goal": [],
    "reset_timestamp_ns": 0,
    "step_time_path_energy_budget": [],
    "termination_and_stop_contract": "..."
  },
  "belief_contract": {
    "state_hypothesis_schema": "...",
    "transition_observation_model_revision": [],
    "normalizer_log_evidence_calibration": [],
    "validity": "valid | stale | zero_likelihood | divergent"
  },
  "memory_contract": {
    "recurrent_hidden_revision_and_reset": [],
    "spatial_map_uuid_revision_layers": [],
    "episodic_event_store_revision": [],
    "capacity_compaction_and_invalidation": "..."
  },
  "frontier_contract": {
    "observed_free_unknown_occupied_predicted_free_legend": {},
    "connectivity_inflation_component_min_size": [],
    "frontier_id_map_revision_age": [],
    "reachable_viewpoint_and_distance_field_revision": []
  },
  "utility_contract": {
    "information_gain_goal_coverage_novelty": [],
    "travel_time_energy_collision_localization_risk": [],
    "hard_reject_thresholds_before_soft_weights": []
```

```
  },
  "evaluation_contract": {
    "gt_visibility_explorable_mask_cell_size": [],
    "coverage_visitation_revisit_frontier_reach": [],
    "failure_denominator_confidence_interval": []
  }
}
```

这个 manifest 的价值不是“字段多”，而是阻止跨 episode、跨 map revision、跨 checkpoint 和跨 metric protocol 的静默拼接。任何 frontier score 若缺 `map_revision + distance_field_revision + observation age`，都不能安全复用。

7. Memory 生命周期与 Invalidation

事件	recurrent latent	spatial map	episodic history	frontier / cache
episode reset	强制清零或新 sequence token	新 map/session 或显式复用门	新 episode namespace	全量失效
map revision update	通常重编码相关 context	按 delta 更新并保留 revision	旧 event 保留但坐标锚点标 stale	重抽取 frontier、重算 distance
global localization jump	清除 pose-sensitive hidden 或重对齐	更新 frame transform	记录 correction event	旧 goal/viewpoint失效
sensor/calibration change	hidden revision不兼容	重投影或新 layer	新 producer revision	visibility/IG cache失效
semantic checkpoint change	semantic hidden不兼容	semantic layer新 revision	保留旧预测但不可直接融合	goal probability重算
clock discontinuity	sequence断开	observation age不可用	新 time segment	stale-age gate失败

不要删除历史来伪装一致性：旧 episodic event 可以保留作为审计证据，但 consumer 必须看到它的 revision 已 stale。空间 memory 的“可变当前状态”和 episodic memory 的“不可变事件记录”应分开。

8. Frontier 的严格定义

对 occupancy map `M` 与邻域 `N(c)`：

$$\mathcal{F}(M) = \{c \mid M(c) = \text{known-free},\ \exists c' \in \mathcal{N}(c) : M(c') = \text{unknown}\}.$$

变量解释：frontier cell `c` 本身位于已观测安全一侧，至少有一个邻居仍是 unknown。直觉是机器人站在“已知地板边缘”向未知看，而不是把未知格直接设成 goal。实现映射：source 的4-neighbor probe 在固定网格上得到 `[(1,2),(2,2)]`；代码必须冻结4/8 connectivity、unknown legend、map bounds 与 obstacle inflation。

8.1 Frontier extraction pipeline

```
def extract_reachable_frontiers(map_state, pose, contract):
    # 1. Keep observed-free separate from anticipated/predicted-free.
    observed_free = map_state.observed_free
    unknown = map_state.unknown

    # 2. Inflate occupied cells by the robot footprint and safety margin.
```

```
    traversible = inflate_and_subtract_obstacles(
        observed_free,
        map_state.observed_occupied,
        contract.robot_radius_and_margin,
    )

    # 3. A frontier lies on the traversible side adjacent to unknown cells.
    frontier_mask = traversible & dilate(unknown, connectivity=contract.connectivity)

    # 4. Cluster components; reject tiny/noisy/stale components.
    components = connected_components(frontier_mask)
    components = [c for c in components if c.area >= contract.min_component_area]

    # 5. Project each component to a reachable free viewpoint, not an unknown centroid.
    reachable = []
    for component in components:
        view = select_reachable_viewpoint(component, traversible, pose)
        if view is not None and view.map_revision == map_state.revision:
            reachable.append(view)
    return reachable
```

代码注释对应五道 gate：map semantics、footprint inflation、boundary extraction、component denoise、reachable view projection。Yamauchi 支持 classic frontier 定义；PONI 的 potential policy可以使用 unexplored/distance maps，但 fixed code 的 `needs_frontier_maps=False`，不能把二者说成同一实现。

9. Information Gain：探索什么不确定性

$$H(b_t) = -\sum_s b_t(s) \log b_t(s), \qquad \mathrm{IG}(a) = H(b_t) - \mathbb{E}_{o \sim P(o|b_t,a)}[H(b_{t+1})].$$

变量解释：`H` 是 entropy，`IG(a)` 是执行 action 后预期减少的 uncertainty。直觉是一个 viewpoint 的价值来自“预计能排除多少假设”，不是它离未知区域有多近。实现映射：source probe 的均匀二状态 prior、准确率 0.9对称 observation model 得到 `0.531004 bits`；实际系统还要说明 entropy 属于 pose、occupancy、object location 还是 task feature，它们不能直接相加。

常见近似：

- 对 occupancy cells 求独立 Bernoulli entropy，但要承认忽略 cell correlation；
- 用 ensemble/dropout disagreement 近似 epistemic uncertainty，但不等于 calibrated posterior；
- 用预期新增可见 cell 数近似 coverage gain，它不是严格 IG；
- 用 downstream reconstruction/recognition improvement 作为 task utility，不能改名成 navigation success。

10. 综合 Exploration Utility：先 Gate，后 Rank

通过 hard gate 后，可用综合 utility：

$$U(f) = w_I \,\mathrm{IG}(f) + w_g p_{\mathrm{goal}}(f) + w_c \Delta C(f) + w_n N(f) - w_d d(f) - w_e E(f) - w_r \,\mathrm{Risk}(f).$$

变量解释：`f` 是 frontier/viewpoint；`ΔC` 是预期 coverage gain；`N` 是 novelty；`d` 是 path/time cost；`E` 是 energy；`Risk` 汇总 collision、localization、predicted-free 与 uncertainty。直觉是“信息多、接近任务、覆盖新区域、代价低且风险可控”的候选更好。实现映射：这是本页对多篇来源的 工程综合，不是 PONI、

Active Neural SLAM 或任何单篇论文的逐字公式；配置必须记录每项单位、归一化、weight 与 hard threshold。

10.1 为什么 risk 不能只做负权重

如果一个候选超过 collision/localization/safety hard limit，它应被 reject；否则极高的 IG 或 goal probability 可能在加权和里“买掉”安全约束。正确结构是：

```
schema/revision valid
  AND localization/map valid
  AND reachable in inflated observed-free
  AND risk <= hard_limit
  AND not stale/blacklisted
      → only then compute soft utility and argmax
```

10.2 目标导向与任务无关探索

目标	主要 score	适合	典型偏差
coverage	新增可见区域	建图、无任务巡检	看得多但不解决任务
information gain	uncertainty reduction	localization/map/object消歧	observation model错会产生伪 IG
semantic goal	object/room potential	ObjectGoal	detector/commonsense prior bias
downstream task	reconstruction/recognition/pose proxy	active perception	proxy好不等于导航成功

Exploring Exploration 的核心启示是：不同 exploration objective 对 downstream task 的排序可能不同；应先冻结最终任务，再选择 intrinsic objective。

11. 公开方法如何放进同一张图

方法	memory / information state	高层探索决策	本页应记住的边界
classic frontier	occupancy known/unknown map	去 free/unknown 边界	定义清晰，但依赖地图与可达性
Active Neural SLAM	learned map + explored/visited + pose	global goal，local planner执行	learned mapper不等于 calibrated belief
Occupancy Anticipation	observed + anticipated occupancy memory	加快 area seen / navigation	predicted-free不能自动进入安全层
SemExp	episodic semantic map	goal-conditioned long-term goal	detector bias会污染 memory/utility
PONI	semantic map + area/object potential + distance	potential peak/global goal	potential policy不等于 classic frontier extractor
EmbodiedQA navigator	recurrent visual/action history	LSTM planner/controller	recurrent hidden不等于 explicit map/belief

方法选择不是“learned 比 classic 高级”：如果 map semantics、revision 和 reachability可审计，classic frontier很强；如果目标语义重要，需要 semantic map/potential；如果 observation model与分布可靠，可加入 anticipation/IG；如果真实部署缺少可靠不确定性，宁可保守 fail closed。

12. Revisit、Stuck 与 Unreachable Frontier

revisit 不应一刀切。loop recovery、换视角确认目标、从死路返回都可能是合理 revisit；问题是“重复是否带来新信息或任务进展”。

一种可测试的 novelty 衰减为：

$$N(c;n_c)=\frac{1}{\sqrt{1+n_c}},\qquad \text{progress}_t=d_{\text{plan}}(x_{t-k},g)-d_{\text{plan}}(x_t,g).$$

变量解释：`n_c` 是 cell/view 的有效访问次数，`progress_t` 是过去 `k` 步规划距离减少量。直觉是访问越多 novelty 越低；若动作持续执行但 goal distance 不下降，则可能 stuck。实现映射：source probe 在 `n=0,1,4` 时得到 `1,0.707107,0.447214`；部署还需 cell/view quantization、最小 revisit 间隔、collision 与 odometry confidence。

状态	判据组合	动作
useful revisit	新 view、IG/goal score上升、progress恢复	允许，记录收益原因
oscillation	action n-gram重复、两 frontier交替、progress≈0	cooldown、切 candidate、局部恢复
physical stuck	collision/actuation timeout、pose近似不变	safe recovery或stop
unreachable	inflated graph disconnected、planner fail	blacklist component+revision
stale target	frontier/map/distance revision不一致	invalidate并重抽取
no valid frontier	全部被 gate、地图封闭或传感器失败	explicit terminal/relocalize，不随机撞

13. Coverage、Visitation、Revisit 与效率

唯一 coverage ratio 为：

$$C_t=\frac{|\mathcal{V}_t\cap\mathcal{E}|}{|\mathcal{E}|},\qquad \mathcal{V}_t=\bigcup_{\tau\le t}\text{VisibleCells}(o_\tau).$$

变量解释：`E` 是 ground-truth explorable cells，`V_t` 是截至预算 `t` 真正进入 sensor visibility 的 cell union。直觉是只计算“看见过的唯一可探索区域”，不是走过的 path footprint。实现映射：OccupancyAnticipation official metrics 将 explored/free/occupied cell count 乘 `map_scale²` 得到 area seen；若要 coverage ratio，还必须冻结 GT denominator、visibility model、cell size 和 noise setting。

revisit ratio 可写为：

$$R_{\text{visit}}=\frac{\sum_{t=1}^{T}\mathbb{1}[n(q(x_t))>0]}{T},$$

变量解释：`q(x_t)` 把 pose/view quantize 到协议单元，分子统计访问已出现单元的 step。直觉是衡量轨迹有多少重复，但它不能区分必要返回和无效振荡。实现映射：必须冻结 quantization resolution、时间间隔、

view orientation、teleport/reset 和 denominator；最好再报告 `useful_revisit` 标签或 revisit 后的 IG/progress。

指标	最小协议身份	回答什么
area seen / coverage	GT mask、visibility、cell size、budget	看了多少唯一空间
visitation	reference pose/object/view、阈值、stop	覆盖了哪些任务相关观察位
revisit ratio	quantization、间隔、必要性标签	重复程度及是否有效
frontier accept/reach	accepted/rejected/reached denominator	候选生成与执行是否一致
coverage AUC	step/time/path/energy axis	同预算下探索效率
collision/stuck/no-frontier	failure definition和episode denominator	安全与恢复性

平均值不能隐藏失败：应保留每 episode curve、seed/scene/start、timeout、zero-likelihood、lost-localization、no-frontier 与 stuck reason，并报告 seed dispersion 或 confidence interval。

14. 十个可手算 / 可编码 Probe

#	Probe	Expected	面试意义
1	两状态 Bayes update	`[.272727,.727273]`	会算 predict/correct/normalize
2	likelihood全0	fail closed	不伪造 posterior
3	binary IG	`.531004 bits`	会算 expected entropy reduction
4	frontier grid	`[(1,2),(2,2)]`	frontier 在 known-free 一侧
5	障碍隔断 frontier	unreachable	语义候选还需 graph gate
6	综合 utility 两候选	candidate A	分项和单位可复算
7	revisit count `0/1/4`	`1/.707107/.447214`	novelty单调下降
8	map revision `7/8`	只保留 current `8`	stale anchor失效
9	episode reset hidden `[3,-2]`	`[0,0]`	防止 leakage
10	`runtimeReady=false` 或 trace缺失	local gate false	interface/map不等于 runtime

source freeze 已实际执行并得到 `10/10 expected decisions matched`。这些 probe 用于验证概念合同，不是论文复现，不允许被写成本地项目 benchmark。

15. First-divergence 诊断

固定顺序：

```
episode/sensor/time
  → localization/map revision
  → observed/predicted/unknown semantics
  → belief update/normalizer
  → memory write/reset/invalidation
  → frontier extraction/component
  → inflation/reachability/freshness/risk
  → utility components and weights
  → planner/controller/progress watchdog
  → metric visibility/denominator
```

症状	first divergence	首查证据	常见误诊
belief NaN或突然均匀	likelihood/normalizer/model mismatch	log evidence、schema、dtype、packet	“模型不确定所以正常”
frontier在墙后	map legend/inflation/reachability	observed-free mask、footprint、path graph	planner不够强
反复去同一区域	visited/revisit/cooldown未持久化	frontier ID、visit count、action/pose trace	加随机噪声即可
reset后成绩异常好	hidden/map跨episode泄漏	reset log、episode namespace	memory能力强
coverage高但目标失败	semantic goal score/visibility mismatch	semantic FP/FN、goal map、view coverage	继续增加coverage reward
map更新后走旧目标	stale revision/distance cache	map/frontier/goal revisions	controller延迟
无 frontier	真正探索完成或全候选被gate	rejection reasons、map bounds、sensor state	随机走一步试试

16. Fail-closed Active Exploration Pipeline

```
def active_exploration_step(packet, state, models, planner, contract):
    # 1. Validate episode, clock, sensor schema, localization and map revision.
    if not contract.packet_valid(packet, state):
        return safe_stop("invalid_packet_or_revision")

    # 2. Update belief; zero/NaN evidence is an explicit invalid state.
    belief = models.update_belief(state.belief, packet)
    if not belief.finite or belief.normalizer <= contract.min_evidence:
        return relocalize_or_stop("invalid_belief")

    # 3. Write recurrent, spatial and episodic memory with immutable identities.
    state.memory.write(packet, belief)

    # 4. Build frontiers only from observed known-free adjacent to unknown.
    frontier_views = extract_reachable_frontiers(
        state.spatial_map,
        packet.pose,
        contract.frontier,
    )

    # 5. Apply hard freshness/reachability/risk gates before any weighted score.
    valid = [
        f for f in frontier_views
        if state.fresh(f)
        and not state.cooldown_or_blacklisted(f)
        and contract.risk(f, state) <= contract.hard_risk_limit
    ]
    if not valid:
        return recover_or_stop("no_valid_frontier")

    # 6. Rank valid candidates using auditable utility components.
    goal = max(valid, key=lambda f: component_utility(f, belief, state, contract))
    plan = planner.plan(packet.pose, goal, state.spatial_map.inflated_observed_free)
    if not plan.reachable:
        state.blacklist(goal, packet.map_revision, reason="unreachable")
        return recover_or_stop("unreachable_frontier")

    # 7. Execute a bounded prefix, then re-observe and replan on any revision/progress event.
    return execute_prefix(plan, watchdogs=["progress", "collision", "revision"])
```

这段伪代码体现面试回答的主线：belief valid → typed memory → frontier → hard gate → utility → bounded execution → watchdog。它是目标架构，不是五项目现有代码。

17. 本地五项目真实性边界

项目	能安全挂钩	不能升级	面试中的安全说法
OPD	2D grounding/region score可作为 semantic observation 候选	active ObjectNav、frontier runtime、navigation coverage uplift	“我能解释 grounding 如何进入 semantic map，但项目没有该闭环证据”

项目	能安全挂钩	不能升级	面试中的安全说法
Robot Harness	message history、navigation tool安全合同、occupancy/map bridge	learned belief、spatial/episodic navigation memory、active exploration	“当前证据是 runtime接口，不是模型记忆或探索算法”
OpenPiVLA	image/state/action history与chunk contract	navigation belief/frontier policy	“temporal context 不等于 POMDP belief”
WAM	future/video prediction可讨论 observation model 候选	calibrated posterior 或 exploration gain	“world prediction 要经 calibration和active A/B 才能支持该claim”
UWM	simulator state/action可构造 partial-observation实验	GT state等于部署belief、DataLoader memory等于agent memory	“当前只支持协议设计，不支持在线探索结果”

Robot Harness `history.py` 的 source-freeze 定向测试为 `2 failed, 2 passed, 2 warnings`，因此连 message compaction 都要带当前失败边界。项目回答顺序固定为：`公开定义 → fixed paper/code → 本地直接证据 → inference → missing artifact → 个人ownership`。

18. 两档面试回答

18.1 约30秒回答

partial observation 下我不会把当前帧当 state，而是维护 belief 和 typed memory。belief 用 transition 与 observation likelihood 做 Bayes update；normalizer 为0就停止或重定位。空间图要区分 observed free、unknown、occupied 和 predicted free，frontier 是 free 邻接 unknown 的边界。候选先过 revision、可达性和风险硬门，再按信息增益、目标概率、coverage/novelty减去路程和风险排序；执行有限前缀后根据 revisit、stuck 和 map revision 重规划。

18.2 约2分钟机制回答

我会先把系统写成 POMDP：真实 state 不可直接读取，策略消费的是由 observation/action history 得到的 belief。memory 分三层：RNN hidden 压缩历史，explicit map 保存 occupancy/semantic/visited/uncertainty，episodic store 保存带 episode、时间、pose/map revision 的 observation-action-result。Bayes update 必须检查 evidence；zero-likelihood、NaN 或 model revision 不匹配要 fail closed。frontier 只从 inflated observed-free 与 unknown 的边界抽取，component centroid还要投影到真实可达 viewpoint。通过 freshness、reachability、collision/localization risk 等硬门后，才用 IG、goal probability、预期coverage、novelty、path/time/energy cost 做 soft ranking。执行时只跑 plan prefix，并监控 progress、collision、两点振荡、revisit收益与 map revision；旧 frontier、distance field 和坐标锚点必须失效。评估至少冻结 GT visibility、cell size、预算和 denominator，分开报 coverage、visitation、revisit、frontier reach、collision/stuck。当前五项目只有接口或上游观测证据，没有本地 active-exploration runtime，所以我不会声称个人导航指标。

19. 高频追问与答题抓手

追问	回答抓手
RNN hidden 为什么不一定是 belief?	没有概率语义、model identity、normalization、calibration与多模态保证
explicit map 与 belief 谁更好?	不是二选一；map可解释，belief表达不确定性，recurrent latent压缩非结构信息

追问	回答抓手
frontier 为什么在 free 一侧?	robot goal必须是当前已知可站立位置，unknown只是预期可观测区域
predicted-free 能否直接规划?	只能在独立 risk/calibration门下使用，不能冒充 observed-free
coverage 与 IG 有什么区别?	coverage计新增可见空间，IG计 posterior entropy预期下降
revisit 一定不好吗?	不；关键看是否带来新信息、恢复进展或必要返回
no frontier 怎么办?	区分探索完成、map/sensor错误、全部候选被gate，显式stop/relocalize
map revision 后哪些状态失效?	frontier ID、goal/viewpoint、distance cache、visited坐标锚点、pose-sensitive hidden
如何证明 exploration 好?	frozen budget 下 coverage/AUC、task utility、failure denominator、matched A/B 与 episode trace
你的项目实现了吗?	当前没有 direct runtime/episode证据，只能讲接口与设计迁移条件

20. 常见错误

1. 把 simulator GT state 写成部署 belief。
2. 把 action/image history tensor 直接称为长期记忆。
3. 把 unknown、unseen、概率0.5、padding和 predicted-free 混成一个 map value。
4. 把 frontier target 放在 unknown 或 obstacle cell 中。
5. 用高 information gain 抵消 collision risk hard limit。
6. 把所有 revisit 都惩罚，导致无法绕障或回到关键观察位。
7. map revision 后继续用旧 frontier/distance cache。
8. 用预测 map 的 explored channel同时做策略输入和唯一评估真值。
9. 只报最终 coverage，不报 budget curve、failure denominator和scene/seed分布。
10. 把 official repo、字段名、map asset 或本地 history 管理说成个人 active-exploration成果。

21. 本页证据边界与下一步

本页已把 source freeze 转成可学习知识：POMDP/belief、zero-likelihood、三类memory、typed manifest、frontier extraction、IG/utility、revisit/stuck/unreachable、metric contract、10 probes、诊断树、伪代码、两档回答和五项目truth boundary均已覆盖。

仍为 `WARN`：没有独立 fact-review、真实口述、PER-017 atomic card、本地 calibrated belief、typed exploration trace、matched A/B、checkpoint/episode bundle 或候选人 ownership闭环。页面生成不改变 mastery。

下游 VLA-PER-017 Atomic Card 已在独立 bounded slice 同步，短答、机制答、诊断、追问与五项目边界保持 `unseen/WARN`。下一 bounded slice 只冻结 `VLA-PER-018` navigation BC/DAgger/RL/exposure-bias source；不生成其 knowledge/card，不创建 coaching state。

References

1. PER-017 Source Freeze：八份论文、六个 fixed official repo/36文件、10 probes 与五项目证据边界。

2. Kaelbling, Littman & Cassandra, Planning and Acting in Partially Observable Stochastic Domains, Artificial Intelligence, 1998.
3. Yamauchi, A Frontier-Based Approach for Autonomous Exploration, CIRA, 1997.
4. Chaplot et al., Learning to Explore using Active Neural SLAM; official code.
5. Ramakrishnan et al., Occupancy Anticipation for Efficient Exploration and Navigation; official code.
6. Chaplot et al., Object Goal Navigation using Goal-Oriented Semantic Exploration; official code.
7. Ramakrishnan et al., PONI; official code.
8. Ramakrishnan, Jayaraman & Grauman, An Exploration of Embodied Visual Exploration; official code.
9. Das et al., Embodied Question Answering; official code.
10. PER-005 Navigation Policy Architecture、PER-016 Localization / SLAM 与 PER-004 Navigation Artifact Gate.
11. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories.

Part 18 / 43 · `interview_materials/vla/knowledge/vol_06_per018_navigation_bc_dagger_rl_exposure_bias.html` · SHA-256 `28C489B8AA79C7C19B8C6709B8AF394C909DC620DC1CF144F5193D75AD1C45E1`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`28c489b8aa79c7c19b8c6709b8af394c909dc620dc1cf144f5193d75ad1c45e1`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per018_navigation_bc_dagger_rl_exposure_bias.md`

VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias

状态：`knowledge_drafted / VLA-PER-018 / source_frozen / independent_fact_reviewed / project_truth_bounded / card_synced / WARN` 更新日期：2026-07-20 权威来源：PER-018 Source Freeze 学习边界：本页把已冻结的 navigation behavior cloning、teacher/student forcing、scheduled sampling、DAgger、GAE/PPO/DD-PPO 与 train/eval identity 转成面试讲义；本轮不生成题卡、不改变 268 张题卡的 mastery、不创建 coaching state。

问题（Question）

VLN、ObjectNav 或 PointGoal 中，离线 behavior cloning（BC）、teacher forcing、student forcing、scheduled sampling、DAgger 与 PPO 到底改变了什么状态分布、监督信号和优化目标？怎样把 oracle、reward、rollout freshness、STOP、termination/truncation、checkpoint 与 metric denominator 冻结成一个可审计的训练—评估合同，并避免把 recovery 数据、文件名或上游代码误写成个人 DAgger/PPO 成果？

1. 学习出口

完成本页后，面试中应能做到：

1. 用数据分布和 update object 区分 BC、teacher forcing、student forcing、scheduled sampling、DAgger、PPO 与 DD-PPO。
2. 写出带 action mask、sample weight 和安全分母的 BC 目标，解释为什么 train CE 下降不等于闭环导航成功。
3. 说明 exposure bias 的 history、state、termination 三层含义，并指出 scheduled sampling 与 DAgger 的身份差异。
4. 从 visited state、oracle relabel、immutable aggregate、β_i lineage 四个 gate 判断一个流程是否真的叫 DAgger。
5. 写出 GAE、PPO clip 和完整 value/entropy 目标，区分 true termination 与 time-limit truncation 的 bootstrap。
6. 设计一份 typed navigation training manifest，把 task、split、oracle、policy、rollout、reward、update、result 和 ownership 绑定起来。

7. 按 first-divergence 顺序诊断“loss 变好但 rollout 变差”“reward 上升但 SR 不升”“PPO ratio 异常”等问题。
8. 对 OpenPiVLA、Robot Harness、OPD、UWM、WAM 给出不越界的项目回答。

2. 一分钟心智模型

```
fixed expert trajectories
        ↓
BC / teacher forcing: learn labels on expert-state distribution
        ↓  errors change the next state
student rollout + current-state oracle query
        ↓
DAgger: immutable aggregate rounds, β schedule, retrain
        ↓ or a separate route
fresh policy rollout + reward + old log-prob + value
        ↓
GAE returns → PPO/DD-PPO update → checkpoint lineage
        ↓
matched split / STOP / termination / denominator evaluation
```

主线只有一句话：**先确认训练时策略访问了哪些 state，再确认每个 state 的 target 或 feedback 从哪里来，最后确认 update 与评估 protocol 是否匹配。** 一个叫 `dagger` 的目录、一个叫 `online` 的脚本或一次闭环 evaluation 都不能替代这三层身份。

3. BC 与 Teacher Forcing：在 expert 分布上学动作

行为克隆（Behavior Cloning, BC）把 expert trajectory 当作固定监督数据。令 D_E 为 expert 轨迹集合，$m_{b,t}$ 为有效 action mask，$w_{b,t}$ 为 sample/inflection weight，π_θ 为策略，离散动作的 masked negative log-likelihood 为：

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{\sum_{b,t} m_{b,t} w_{b,t} \log \pi_\theta(a^*_{b,t} \mid o^E_{b,t}, h^E_{b,t}, c_b)}{\sum_{b,t} m_{b,t} w_{b,t} + \varepsilon}.$$

变量解释：a^* 是 expert/oracle action，o^E, h^E 来自 expert trajectory，c 是 instruction 或 goal，m 排除 padding/无效 action，w 可在 action inflection 处加权，ε 只用于防止数值除零。直觉是“在已经走过的正确轨迹上模仿下一步”；实现上应对应 R2R 的 cross-entropy target、VLN-CE 的 oracle action/inflection weight 和 fail-closed empty-mask gate。若有效分母为零、oracle action 为 `-1` 或 STOP 语义不明，应停止该 batch，而不是返回一个伪造的 0 loss。

3.1 Teacher forcing 与 covariate shift

teacher forcing 在第 t 步执行 expert action，再读取下一 observation。因此训练访问分布接近 d_{π^*}，而部署时错误动作会使 state 分布转向 d_{π_θ}。单步 action accuracy 或 CE 只能回答“在 expert state 上是否像”，不能回答“犯错后能否恢复”。

机制	下一 state 由谁产生	target	主要优点	主要风险
offline BC	数据集已固定	expert action	便宜、可复现	不覆盖策略犯错后的 state
teacher forcing	expert action	下一步 expert action	label 稳定、训练简单	history/state exposure bias
student forcing	learner action	当前 state 的 target	可暴露恢复状态	需要环境与正确 current-state target

4. Exposure Bias、Student Forcing 与 Scheduled Sampling

exposure bias 至少有三层：

1. **history exposure**：训练输入 previous action/token 是 ground truth，部署输入是 model output。
2. **state exposure**：错误动作改变 pose、view 或 collision 状态，后续 observation 脱离 expert distribution。
3. **termination exposure**：错误 STOP、loop、timeout 或 collision 改变 episode 长度与 metric denominator。

scheduled sampling 以概率 ϵ_i 选择 ground-truth previous input，否则把模型输出喂回下一步：

$$a_{t-1}^{\text{in}} = \begin{cases} a_{t-1}^{*}, & z_t < \epsilon_i, \\ \hat{a}_{t-1} \sim \pi_\theta, & z_t \geq \epsilon_i, \end{cases} \qquad z_t \sim \text{Uniform}(0,1).$$

变量解释：$\epsilon_i = 1$ 是完整 teacher forcing，$\epsilon_i = 0$ 是 always sampling/student forcing，z_t 是随机门。它改变的是序列输入和访问历史的 curriculum；它本身没有要求在每个 learner-visited state 查询 external oracle，也没有跨轮 immutable aggregate。面试中应明确说：**scheduled sampling 可以缓解 history mismatch，但不自动满足 DAgger 的 current-state expert-label contract。**

student forcing 若由 learner action 推进环境，并在当前 pose 重新计算 oracle target，可以成为 online imitation 的组成部分；若只是把 model token 喂回去，仍在旧 expert path 上套错位 label，则不能声称拥有 current-state guarantee。

5. DAgger：四道身份门

DAgger 的第 i 轮混合策略和 aggregate dataset 可写为：

$$\pi_i = \beta_i \pi^* + (1-\beta_i)\hat{\pi}_i, \qquad D \leftarrow D \cup \{(s, \pi^*(s)) : s \sim d_{\pi_i}\}.$$

变量解释：π^* 是 expert/oracle，$\hat{\pi}_i$ 是当前 learner，d_{π_i} 是这一轮实际访问的 state 分布，β_i 记录 expert/learner 混合比例。直觉是“让 learner 真正走到会犯错的地方，再让同一个 oracle 在这些地方重新标注”，而不是把旧 demonstration 再复制一次。VLN-CE source freeze 记录的实现使用 $\beta = p^{\text{data_it}}$，`p=.75`、10 轮、每轮 nominal 5,000 trajectories、4 个 IL epochs，并带 oracle action 与 inflection weight。

一个可审计的 DAgger round 必须同时通过：

Gate	必须有的证据	缺失时的结论
policy rollout	trajectory 的 policy/checkpoint hash 与声明的 current/mixture policy 一致	不能确认访问分布
oracle query	同一 task/map/oracle revision 在 visited state 给出 label	只有 self-training 或 recovery BC 候选
aggregation	round、episode、invalid/skipped、duplicate 可追溯，样本不可变地追加	不能确认 aggregate dataset
schedule	β_i、round id、训练 config 与 checkpoint lineage	不能重建 curriculum

“收集失败轨迹再微调”可能是 recovery BC；“student rollout”可能是 self-training/distillation；只有四道 gate 都满足，才安全地叫 canonical DAgger。

6. GAE、PPO 与 DD-PPO：从 action label 转向 return

令 r_t 为 reward，$V_\phi(s_t)$ 为 value，d_t^{term} 只表示真正 terminal。TD residual 与 GAE 为：

$$\delta_t = r_t + \gamma(1 - d_t^{\text{term}})V_\phi(s_{t+1}) - V_\phi(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda(1 - d_t^{\text{term}})\hat{A}_{t+1}.$$

变量解释：γ 是 discount，λ 控制 bias/variance，$\hat{A}_t$ 是 advantage。直觉是把未来多个 TD error 按 $\gamma\lambda$ 逆时间累积；实现要保存 reward、value、mask、return，并按 episode 边界反向递推。time-limit truncation 通常仍需 bootstrap，真正 terminal 才把 next value 切掉；把两者都压成一个 `done` 会污染 return。独立复核确认固定 Habitat-Lab `0fb6f43` 的 trainer 本身仍是 `dones → not_done_masks`，所以这里是目标工程合同，不是该pinned code已完成双字段分离。

PPO 使用采样时的 old policy 与更新中 new policy 的概率比：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, \qquad L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t\hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right].$$

变量解释：r_t 是 policy ratio，ϵ 是 clip 范围，$\hat{A}_t$ 来自 GAE。直觉是允许有益的小更新，同时限制 ratio 把旧 rollout 推得太远。Habitat/VLN-CE code mapping 还包含 value loss、entropy、auxiliary loss、advantage normalization 和 gradient statistics；PPO clip 公式本身不是完整 trainer。若 behavior log-prob 不是采样该 action 的 θ_{old}，或 rollout 在多次更新后仍被叫作 fresh on-policy，ratio 就失去身份。

典型完整目标可写成：

$$\mathcal{L}_{\text{total}} = -L^{\text{CLIP}} + c_v\mathcal{L}_{\text{value}} - c_H\mathcal{H}(\pi_\theta) + \mathcal{L}_{\text{aux}}.$$

变量解释：c_v 与 c_H 是 value/entropy 权重，$\mathcal{H}$ 是策略熵，$\mathcal{L}_{\text{aux}}$ 代表实现中的辅助项。面试时应把 reward、value target、entropy、mask、denominator 和 clip/ratio 统计一起讲，而不是用一个 surrogate equation 代替整个训练器。

DD-PPO 的关键不是“用了多 GPU”，而是多 worker 采集 fresh rollout、同步更新、无参数服务器的去中心化训练，以及 straggler preemption。官方 PointGoal 配置中的 `2.5e9` frames、clip `.2`、$\gamma = .99$、GAE $\lambda = .95$ 是 frozen public recipe，不是本地运行规模。

7. Navigation Training Manifest：把身份写进数据

任何 `BC / DAgger / PPO improved navigation` claim 至少绑定以下层级：

Layer	必备字段	缺失时不能回答
task/data	task、dataset revision/license、scene split、episode IDs、observation/action/STOP schema、frame/unit、horizon	学的是什么、是否泄漏
expert/oracle	oracle type、shortest-path/navmesh revision、query availability、invalid-action rule	label 是否可信、是否 DAgger
policy	architecture/config、parent checkpoint/hash、normalizer/tokenizer、memory reset、action distribution	loss/result 属于哪个模型
rollout	collector SHA、policy SHA、behavior log-prob、env/controller revision、seed、timestamp、termination/truncation	是否 fresh/on-policy
reward/value	reward components/scale、success/STOP predicate、discount、bootstrap rule、value checkpoint	是否真在优化 RL objective
update	algorithm、loss mask/denominator、optimizer、grad norm、round 或 PPO epoch/minibatch、KL/clip stats	更新了什么
result	checkpoint SHA、split、episode denominator、SR/SPL/nDTW/collision、raw trace、failure taxonomy	指标能否比较
ownership	upstream/local diff、author/reviewer、run operator、artifact custodian	谁真正做了什么

推荐 immutable lineage：

```
task_manifest_sha + dataset/env revision + oracle/reward revision
    → collector policy_sha / behavior_policy_sha
    → round_or_rollout_id + raw trajectories
    → update_config_sha + loss/ratio/return logs
    → checkpoint_sha + eval_protocol_sha
    → per_episode_results + summary + ownership ledger
```

8. Training ↔ Evaluation Protocol

训练和评估要冻结同一套身份，但不能把训练输入当作评估真值：

Contract	需要明确的字段	典型错误
split	train、val-seen、val-unseen、test；early-stop 规则	合并 seen/unseen 报平均
episode	instruction/goal、scene、start/goal、seed、horizon	不报 timeout/lost denominator
observation	RGB/depth/GPS/compass/proprio、resize、normalization、frame、age	训练看 GT，部署看传感器
action	discrete/continuous、axis/unit、STOP、collision/no-op、repeat/controller	action 顺序或单位错位
rollout	checkpoint、memory reset、determinism、replan/chunk、termination/truncation	旧 memory 或 stale policy
reward	distance-progress、success、slack、collision/stop 组件及尺度	reward 上升直接等同 SR
result	SR/SPL/nDTW、episode denominator、raw trace、failure taxonomy	只报最终一个百分比

R2R teacher/student forcing 的结论只在其图导航、split 和 scorer 下成立；VLN-CE DAgger 的超参只支持其 continuous environment 与 oracle/config；DD-PPO 的 PointGoal 结果不能外推到 VLN、ObjectNav 或真实机器人。

9. 方法比较：面试一张表讲清楚

Method	state distribution	target/signal	aggregation	最小 runtime	常见误写
offline BC	fixed expert states	expert action	fixed dataset	dataset + mask + loss + checkpoint	action loss 低 = 闭环好
teacher forcing	expert action 产生后续 state	next expert action	可固定	ordered trajectory	没有 exposure bias
student forcing	learner action 产生后续 state	current-state target	视实现	env rollout + target rule	model rollout自动等于 DAgger
scheduled sampling	expert/model previous input 混合	sequence label	通常不是跨轮 aggregate	ϵ schedule + trace	等同 no-regret DAgger
DAgger	mixture/current policy states	visited-state oracle action	是	round collector + oracle + immutable aggregate	recovery demos = DAgger
PPO	fresh current/old-policy rollout	reward/return/advantage	rollout buffer	reward + old logp + value + GAE + update	有 reward 就是 PPO
DD-PPO	多 worker fresh rollout	同 PPO + synchronized gradient	distributed rollout	worker/step/preemption ledger	GPU 多 = 复现论文规模

10. First-Divergence Diagnosis

诊断顺序固定为：

```
task/split/schema
  → expert/oracle/reward identity
  → policy/checkpoint/normalizer
  → collector state/action/STOP
  → label/log-prob/value/return
  → mask/denominator
  → optimizer/grad/ratio
  → checkpoint save/load
  → eval protocol/result
```

症状	首个分叉假设	首查 artifact	不应先做
BC loss 低、rollout 立刻偏航	expert/model state distribution 不同	pose/action trace、teacher/student coverage	只继续降 train loss
DAgger 越迭代越差	oracle/map revision、β、aggregate 污染	round manifest、invalid/skipped、duplicate	无条件拼全部旧 LMDB
student forcing 不收敛	current-state target 错位或 learner 太弱	executed action、next state、recomputed target 三联	直接改名 scheduled sampling
PPO reward 升、SR 不升	reward hacking 或 STOP mismatch	reward components、success predicate、raw episode	把 reward 当 SPL
PPO ratio/clip 异常	stale rollout 或 old logp 错	policy hash、collector/update timestamps、ratio histogram	只调 clip epsilon
value loss 爆炸	return/terminal/bootstrap 错	reward、terminated/truncated、Vnext、GAE trace	先换大 backbone
DD-PPO 不可复现	worker/preemption/denominator 不一致	worker ledger、sync fraction、seed	只报 GPU 数和 wall time
val-seen 好、val-unseen 差	scene/instruction overfit	split manifest、duplicate/leakage、per-scene failure	合并两个 split

11. Fail-Closed Navigation Training Pseudocode

```
def train_navigation(run, manifests, policy, expert=None, value=None):
    # 1. Freeze task/data/env/policy identity before reading a sample or rollout.
    assert manifests.task_sha and manifests.action_schema_sha
    assert manifests.policy_parent_sha and manifests.split_sha

    if run.algorithm == "bc":
        # 2. BC consumes immutable expert trajectories; empty masks are invalid.
        batch = manifests.expert_dataset.next_batch()
        assert batch.oracle_revision == manifests.oracle_revision
        if batch.valid_action_mask.sum() == 0:
            return stop("empty_bc_denominator")
        return update_masked_action_nll(policy, batch)

    if run.algorithm == "dagger":
        # 3. Roll out the declared mixture/current policy and label visited states.
        beta = run.p ** run.round_id
        trajectory = collect_mixture_rollout(policy, expert, beta, manifests.env)
        if trajectory.policy_sha != policy.sha or not trajectory.all_states_labeled:
            return stop("invalid_dagger_round")
        manifests.aggregate.append_immutable(run.round_id, trajectory)
        return update_on_aggregate(policy, manifests.aggregate)

    if run.algorithm == "ppo":
        # 4. PPO requires fresh behavior log-probs, reward, terminal masks and value.
        rollout = collect_fresh_rollout(policy, value, manifests.env)
        if rollout.behavior_policy_sha != policy.sha:
            return stop("stale_behavior_policy")
        if not rollout.has_reward_logp_value_terminated_truncated:
            return stop("incomplete_ppo_rollout")
        advantages, returns = compute_gae_fail_closed(rollout)
        return ppo_update_with_ratio_stats(policy, value, rollout, advantages, returns)

    return stop("unknown_algorithm_identity")
```

这段代码是 identity/gate 目标，不是五个本地项目已有的统一 trainer。生产实现还要处理并发 collector、oracle 限流、dataset transaction、distributed worker failure、NaN rollback、atomic checkpoint、evaluation canary 和安全 STOP。

12. 十个 Deterministic Probes

Probe	固定输入	预期结果	面试要点
masked BC NLL	`p=.8` 有效、`.5` 被 mask	`.2231435513`	denominator 只含有效 label
empty mask	mask `[0,0]`	`gate=false`	不返回伪 0 loss
exposure shift	expert 首步 `+1`、model 首步 `-1`	visited state 不同	单个错误改变后续分布
scheduled endpoints	`ε=1/0`	teacher/model input	schedule identity 可测
DAgger beta	`p=.75`，round `1..4`	`.75/.5625/.421875/.316406`	记录 round 与 policy
nominal aggregate	10×5,000 trajectories	50,000 nominal	仍需扣 invalid/duplicate/early stop
inflection weight	actions `0,0,1,1,2`，coef `3.2`	`1,1,3.2,1,3.2`	只在 change 位置加权
GAE reverse	`δ=[1,1,1]`，$\gamma = .99, \lambda = .95$	`2.82504,1.9405,1`	逆时间递推
PPO clip	`(r,A)=(1.3,1)/(.7,-1)`，$\epsilon = .2$	`1.2/-0.8`	正负 advantage 方向不同
terminal/truncation	`r=1,γ=.99,Vnext=10`	`1 / 10.9`	true terminal 与 timeout 分开

Source freeze 中的 deterministic probes 已得到 `10/10 expected decisions matched`。它们是合同和数值反例，不是论文 benchmark、训练稳定性或本地项目结果。

13. Public Method → Code Mapping

Mechanism	固定来源/代码	可以直接讲	仍需单独核实
BC/teacher forcing	R2R paper + Matterport R2R	CE target、teacher executed action、test no-cheat gate	continuous regression/local checkpoint
student forcing	R2R paper/code	policy action 进入 next state、current target	oracle correctness、local run
scheduled sampling	Bengio et al.	ϵ 切换 previous truth/model output	navigation effectiveness
DAgger	Ross + VLN-CE	aggregate、$\beta = p^i$、5k×10、oracle action、inflection weight	local collector/oracle/LMDB lineage
PPO/GAE	PPO/GAE + Habitat-Lab	ratio/clip、returns/GAE、value/entropy、grad stats	reward validity、fresh rollout、local result
DD-PPO	DD-PPO + Habitat-Lab	distributed sync、preemption、PointGoal config	local worker/run/2.5B-scale reproduction
IL+RL hybrid	R2R-EnvDrop	teacher CE、sample action、distance reward、value/entropy	与 PPO/DAgger 的身份不能混写

14. 五个项目的 Truth Boundary

项目	可以安全讲	当前不能说	最小补强 artifact
OpenPiVLA	offline action-to-text supervision、recipe/mask/serialization 冲突与 rollout lineage 缺口	已做 navigation BC/DAgger/PPO、formal checkpoint 闭环提升	continuous action target/parser/controller、collector/oracle/reward、checkpoint+episode bundle
Robot Harness	tool trajectory SFT、schema-aware reward 与 proxy eval	PPO/GRPO 已在线提升机器人导航成功率	actual trainer revision、fresh effect-sink rollout、old logp/returns 或 GRPO fields、robot episodes
OPD	teacher generation/student CE 与 route-conditioned distillation	“online”就等于 env on-policy、DAgger 或 RL	environment state/action、current rollout、expert relabel、round aggregate、navigation eval
UWM	offline finetune → separate rollout eval、reset/seed/protocol 工程	evaluation episode 被用于 DAgger/RL 更新、0.744 证明 online RL	immutable train dataset/checkpoint、online collector/oracle/reward/update logs、matched eval
WAM	action trace、controller/horizon 字段与失败诊断	trained manifest 证明 BC/RL improvement	loadable matched checkpoints、training objective/log、完整 denominator 与 ownership

被 ignored/third-party checkout 命中的 `StreamVLN/streamvln_dagger.py` 等关键词不能计入候选人 ownership；Git author、文件名、训练 loss 或 closed-loop evaluation 也不能替代 runtime、oracle/reward、update 和 result 证据。

安全回答顺序是：`public definition → fixed paper/repo → algorithm identity gates → local inventory → local-direct exception → missing runtime/result → candidate ownership`。

15. 两档面试回答

15.1 约 30 秒回答

BC 在 expert-state distribution 上用 action label 训练，teacher forcing 继续执行 expert action，所以 train loss 低不代表策略犯错后能恢复。student forcing 让 learner action 改变后续 state；scheduled sampling 主要缓解 previous-input mismatch。DAgger 更严格：current/mixture policy rollout、visited-state oracle

relabel、immutable aggregate 和 β_i lineage 四道门都要有。PPO 则不吃 expert action label，而是用 fresh rollout 的 reward、old log-prob、value 和 GAE advantage 做 clipped policy update；time-limit truncation 还要 bootstrap。我的项目目前没有 navigation oracle/DAgger collector 或 fresh PPO runtime，因此不会把 offline recipe 或上游代码说成个人闭环成果。

15.2 约 2 分钟机制回答

我会先按 state distribution 组织答案。BC/teacher forcing 访问的是 expert trajectory，目标是 expert action 的 masked likelihood；一个动作出错，部署时的 pose、observation 和后续 STOP 就可能脱离训练分布，这就是 exposure bias。scheduled sampling 逐步把 previous ground truth 换成 model output，但它不自动拥有 current-state oracle。student forcing 如果真的在当前环境 state 重新查询 oracle，可以成为 online imitation 的组成部分。DAgger 的身份更严格：第 (i) 轮用 β_i 混合 expert 与 learner，记录 learner 实际访问的 states，在这些 states 上查询同一 revision 的 expert，再把不可变 round 追加到 aggregate dataset，最后重训。PPO 是另一条路线：采集与 old policy 对齐的 fresh rollout，区分 terminated 和 truncated，计算 GAE、return、ratio/clip，同时记录 value、entropy、KL、grad 和 checkpoint lineage；DD-PPO 再加入同步多 worker 与 straggler 处理。评估时必须绑定 split、STOP、horizon、controller、seed、episode denominator 和 raw trace。对我的五个项目，我能把 offline action/text supervision、tool reward schema、teacher distillation 或 separate rollout eval 映射到这些概念，但当前没有完整 navigation DAgger/PPO runtime 和 matched result，所以会明确说“可迁移的设计边界”，不声称已经实现。

16. 高频追问与答题抓手

追问	回答抓手
student forcing 等于 DAgger 吗?	不一定；必须有 learner-visited state 的同源 oracle label 与 aggregate lineage。
scheduled sampling 为什么不保证 no-regret?	它主要改变 previous input curriculum，没有 DAgger 的跨轮 oracle aggregation contract。
BC loss 下降为何 SR 不升?	state distribution、STOP、action unit、horizon 或 metric denominator 可能已变。
PPO 为什么需要 old log-prob?	ratio 要比较同一 action 在 old/new policy 下的概率；stale logp 会破坏 surrogate。
truncation 和 termination 有什么区别?	timeout 不是任务完成失败，通常仍 bootstrap；true terminal 才切 next value。
reward 上升为什么不等于 SPL 上升?	reward 可能被 shaping/STOP/collision hack，SPL 需要独立路径和成功协议。
DD-PPO 的关键是什么?	fresh rollout、同步更新、worker/preemption/step ledger，不是 GPU 数量。
如何判断 recovery 数据是不是 DAgger?	查 collector policy、visited-state oracle、round manifest、immutable append、β schedule。
你在项目里实现了吗?	当前证据只支持 offline/interface/upstream 邻接，不支持个人 navigation DAgger/PPO ownership。

17. 常见错误

1. 把文件名带 `dagger` 或 `online` 当成 algorithm identity。
2. 把离线 recovery demonstrations 或 student rollout 自动升级为 DAgger。
3. 用单步 CE/动作 loss 推断闭环 SR、SPL 或真实机器人泛化。
4. 把 reward、advantage、PPO clip 混成一个公式，漏掉 value、entropy、mask 与 denominator。
5. 把 timeout、collision、STOP 和 true terminal 都编码成同一个 `done`。
6. 用 stale rollout 或错误 old log-prob 计算 PPO ratio。

7. 把官方 repo 的超大 frame 数、配置或第三方 checkout 写成个人运行规模。
8. 把 ignored upstream navigation tree、Git author 或自动生成 manifest 写成个人贡献。
9. 合并 val-seen 与 val-unseen，或隐藏 early termination/invalid action 的分母。
10. 没有 task/oracle/policy/collector/checkpoint hash 就声称“可复现”。

18. 本页证据边界与下一步

本页已把 PER-018 source freeze 转成可学习知识：BC/teacher forcing、exposure bias、student forcing、scheduled sampling、DAgger 四门、GAE/PPO/DD-PPO、typed manifest、train/eval contract、方法比较、first-divergence、fail-closed 伪代码、10 probes、public-code mapping、五项目 truth boundary、两档口述回答和追问抓手均已覆盖。

独立 fact-review 已重新命中7/7 primary PDF、10/10代表official files、10/10 probes与8/8 local-direct SHA，并补充pinned Habitat done-mask版本边界。仍为 `WARN`：没有真实口述、本地 navigation expert/oracle、current-policy collector、immutable DAgger rounds、fresh PPO rollout、behavior log-prob/value/return、matched checkpoint/eval episode、跨 seed 结果或逐文件 ownership 闭环。页面生成与事实复核不改变 mastery。

下游 VLA-PER-018 Atomic Card 已在独立 bounded slice 同步并保持 `unseen/WARN`；Independent Fact Review 已完成。下一 bounded slice 优先进行一题真实口述；不启动 PER-019、不创建 coaching state。

References

1. Ross, Gordon & Bagnell, DAgger, AISTATS 2011；固定版本 `1011.0686v3`。
2. Bengio et al., Scheduled Sampling, NeurIPS 2015；固定版本 `1506.03099v3`。
3. Anderson et al., R2R, CVPR 2018；Matterport3DSimulator。
4. Krantz et al., VLN-CE；official code。
5. Wijmans et al., DD-PPO, ICLR 2020；Habitat-Lab。
6. Schulman et al., PPO, 2017；GAE, ICLR 2016。
7. Tan, Yu & Bansal, R2R-EnvDrop；official code。
8. PER-018 Source Freeze 与五项目 Claim–Evidence Inventories。
9. PER-018 Independent Fact Review：7 PDF/10 official files/10 probes/8 local files的独立再获取与reconciliation。

Part 19 / 43 · `interview_materials/vla/knowledge/vol_06_per019_navigation_evaluation_failure.html` · SHA-256 `8B6C39320986EB16A63882FADAD8AADC1E76800164A7A7C795C86F92051A4CEA`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`8b6c39320986eb16a63882fadad8aadc1e76800164a7a7c795c86f92051a4cea`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per019_navigation_evaluation_failure.md`

VOL 06 PER-019：Navigation Evaluation、Failure Taxonomy 与 Fail-Closed Protocol

状态：`knowledge_drafted / VLA-PER-019 / source_frozen / project_truth_bounded / card_pending / WARN` 更新日期：2026-07-21 权威来源：PER-019 Navigation Evaluation / Failure Source Freeze 学习边界：本页把 `Success`、`DistanceToGoal`、SPL、SoftSPL、NE、OS、TL、split、episode denominator、统计报告与 failure taxonomy 转成可面试、可审计的知识；不生成题卡，不修改 274 张题卡的 mastery，不创建 `coaching_state.md`。

问题（Question）

在 VLN、ObjectNav 与 PointGoal 评估中，怎样保证 `success`、`SPL`、`SoftSPL`、`NE`、`OS`、`TL` 与 failure taxonomy 共享同一 episode、split、stop 和 denominator？当本地没有完整 navigation bundle 时，如何 fail closed，而不是从一张 summary、训练 loss 或项目文件名推断导航能力？

1. 学习出口

完成本页后，面试中应能做到：

1. 解释为什么 success 不是“曾经到过附近”，而是 stop、距离模式和阈值共同定义的协议谓词。
2. 从 `POINT` / `VIEW_POINTS`、Euclidean step length、geodesic distance 三层区分距离语义。
3. 写出 SPL，并说明参考最短距离、实际路径、success 必须来自同一 episode 与同一 scorer。
4. 区分 binary success、SoftSPL 的连续软成功项、NE、OS、TL 与 summary 平均值；不把不同 metric namespace 混报。
5. 设计包含 task、split、simulator、trajectory、metric、statistics、failure 和 ownership 的 typed evaluation manifest。
6. 说明 `EVAL.EPISODE_COUNT == -1`、invalid/duplicate row、missing episode、early stop 与 CI 样本单位如何改变 estimand。
7. 按 `symptom → first verified divergence → root cause → recovery/terminal outcome` 诊断导航失败；没有 raw trace 时保留 `unknown`。
8. 对 OpenPiVLA、Robot Harness、OPD、WAM、UWM/LIBERO 给出不越界的项目回答。

2. 一分钟心智模型

```
task / dataset / split / episode manifest
                ↓
simulator + navmesh + distance mode + success threshold
                ↓
ordered poses + action + STOP + termination/truncation trace
                ↓
per-episode scorer: Success / Distance / SPL / SoftSPL / NE / OS / TL
                ↓
denominator manifest + CI/statistics + failure taxonomy
                ↓
aggregate report + checkpoint/scorer/ownership lineage
```

核心句子是：**先确认评估了哪些 episode、使用什么几何和 stop 语义，再确认每一行 trajectory 能否重算指标，最后才讨论平均值和根因。** 论文数字、leaderboard 行、项目 summary 或训练 loss 都不能替代这条 lineage。

3. Success：位置接近不等于协议成功

Habitat-Lab 固定 revision 的 `Success` measure 依赖 `DistanceToGoal`，并要求 agent 调用 STOP，同时满足距离阈值。对 episode (i)，可用下式表达其协议谓词：

$$s_i = \mathbf{1}\left[\text{stop}_i = 1 \ \wedge \ d_i^{\text{goal}} < \tau_i^{\text{success}}\right].$$

这里 s_i 是 episode-level success，stop_i 是 scorer 记录的 STOP 事件，d_i^{goal} 是指定距离模式下的目标距离，τ_i^{success} 是 success threshold。直觉是“到过附近”仍可能没有完成任务；实现映射是 `Success.update_metric` 读取 STOP 状态并依赖 `DistanceToGoal`，而不是从最后一帧的欧氏距离直接猜测。该语义来自固定 Habitat-Lab `nav.py`，不能自动推广到任意本地 evaluator。

面试追问时要先问四件事：STOP 是显式 action 还是 episode 截断？距离是 position 还是 viewpoint？阈值是否随 task/config 变化？最后一行是自然终止、timeout 还是 scorer 丢行？任一项缺失，都只能说“候选 success symptom”，不能说 protocol-confirmed success。

4. Distance：POINT、VIEW_POINTS 与 geodesic contract

4.1 两种 goal 解释

Habitat `DistanceToGoal` 根据 `distance_to` 选择不同语义：

模式	目标几何	评估含义	常见误写
`POINT`	到 goal position 的 simulator geodesic distance	位置接近	用直线欧氏距离代替 navmesh 路径
`VIEW_POINTS`	到一个或多个 goal viewpoint 的 simulator geodesic distance	需要抵达可观察目标的视点集合	把 viewpoint 当成任意 goal center

对地图上的位置 (x) 和 goal 集合 (G)，可把几何距离抽象为：

$$d_{\text{geo}}(x, G) = \min_{g \in G} \text{ShortestPath}_{\text{navmesh}}(x, g).$$

x 是 simulator pose，G 是 position 或 viewpoint 集合，$\text{ShortestPath}_{\text{navmesh}}$ 是受场景 navmesh 约束的最短路径。公式表达的是 geodesic contract 的直觉，不替代具体 simulator API。实现上必须记录 `distance_mode`、scene/navmesh revision 与 goal viewpoint 列表；否则相同坐标可能对应不同的 success 前置量。

4.2 轨迹长度不是 action count

Habitat SPL probe 显示实际路径长度累计相邻 simulator positions 的 Euclidean step distance：

$$L_i = \sum_{t=1}^{T_i} \|p_{i,t} - p_{i,t-1}\|_2.$$

$p_{i,t}$ 是 episode i 在第 t 个记录点的位置，T_i 是 scorer 看到的轨迹长度。直觉是同样执行 20 个 action，如果 controller step size、停顿、碰撞或重复动作不同，物理路径长度可能不同。实现映射是 trajectory trace 中的 ordered poses；不能用 action 数、policy query 数或 wall-clock 秒数替代 L_i。

5. SPL、SoftSPL 与 metric namespace

5.1 SPL

令 $L_i^\star$ 为同一 episode、同一 scene/navmesh 与同一 goal 语义下的参考最短距离，L_i 为实际路径长度，s_i 为同一 scorer 的 binary success，则 success-weighted path length 为：

$$\text{SPL}_i = s_i \cdot \frac{L_i^\star}{\max(L_i^\star, L_i)}.$$

变量 $L_i^\star, L_i, s_i$ 必须来自同一 episode lineage；max 惩罚绕路，$s_i = 0$ 时失败 episode 的 SPL 为 0。代码映射对应 `SPL.reset_metric` 保存 start/end distance、`update_metric` 累计路径并读取 `Success`；零距离 episode、缺失路径和重复 episode 需要 scorer 的显式 guard，不能静默除零。

episode-level 汇总通常是：

$$\overline{\text{SPL}} = \frac{1}{N_{\text{eval}}} \sum_{i \in \mathcal{E}_{\text{eval}}} \text{SPL}_i, \qquad \text{SR} = \frac{1}{N_{\text{eval}}} \sum_{i \in \mathcal{E}_{\text{eval}}} s_i.$$

这里 $\mathcal{E}_{\text{eval}}$ 是经过 split、inclusion/exclusion 与 invalid-row policy 确认的 episode 集合，$N_{\text{eval}} = |\mathcal{E}_{\text{eval}}|$。实现上必须保存 episode ID set 和 denominator manifest；只汇总成功样本会把 SR/SPL 的 estimand 改成条件均值。

5.2 SoftSPL：连续软成功，不是另一种 binary SR

固定 Habitat source probe 只保证 SoftSPL 使用连续的 soft-success 项替换 binary success，再乘 path-length penalty。为避免把未逐行复述的实现写成强 claim，使用下面的教学抽象：

$$\mathrm{SoftSPL}_i = q_i^{\mathrm{soft}} \cdot \frac{L_i^\star}{\max(L_i^\star, L_i)}, \qquad q_i^{\mathrm{soft}} \in [0,1].$$

q_i^{soft} 表示官方实现计算的连续接近度；它不是 s_i，也不能在没有固定 scorer 代码时自行指定成某个距离线性函数。直觉是允许“接近目标但未满足 STOP 阈值”的 episode 得到诊断性分数；实现映射是读取 pinned `SoftSPL` 的实际 soft-success 定义，并同时记录 distance mode、threshold、path length 与 denominator。面试中应说“SoftSPL 是同一 path penalty 下的连续 soft-success 指标”，不要把它当成 leaderboard SPL 或 binary success。

5.3 NE、OS、TL：名称相同也要绑定协议

指标	常见语义	必须绑定	不能直接推出
`NE`	终点到目标的导航误差	final pose、goal mode、geodesic scorer、episode set	success 或 recovery root cause
`OS`	是否曾到达目标区域或是否满足任务的 oracle-success 条件	task-specific scorer、是否要求 STOP、时间范围	已正确终止或 leaderboard SR
`TL`	trajectory length，可能是 steps 或 path length	unit、controller step、timeout policy	efficient execution，除非与 SPL 协议一致

`NE/OS/TL` 的缩写在不同 benchmark 中可能有细节差异；因此回答时先给 task/revision/scorer，再给数值。不得把 VLN-CE 的指标表、Habitat 的 measure 名称或项目 JSON 字段直接当成跨 benchmark 同义词。

6. Split、denominator 与统计报告

6.1 评估对象是一个集合，不是一个平均数

建议把 episode inclusion 写成显式集合：

$$\mathcal{E}_{\mathrm{eval}} = \{e \in \mathcal{E}_{\mathrm{requested}} : \mathrm{split}(e) = \sigma,\ e \notin \mathcal{E}_{\mathrm{excluded}},\ \mathrm{row}(e) \text{ is valid}\}.$$

σ 是 `train`、`val_seen`、`val_unseen` 或 `test_challenge` 等 split，$\mathcal{E}_{\mathrm{excluded}}$ 记录显式过滤项。直觉是 denominator 先于平均值；实现上要保存 requested IDs、included IDs、excluded IDs、missing IDs、duplicate keys 与过滤原因。

VLN-CE README 明确区分 train/val_seen/val_unseen/test-challenge，并说明 `EVAL.EPISODE_COUNT == -1` 才评估全部 episode。若只跑前 (k) 个、只保留 successful rows、过滤 invalid episode 却不写入 manifest，得到的是另一个 estimand，不能复述为同一 leaderboard 结果。

6.2 Success rate 的不确定性

VOL 05 EVAL-003 已冻结 episode-level Bernoulli 与 Wilson/Clopper–Pearson 边界。对 $k = \sum_i s_i$、$n = N_{\text{eval}}$，Wilson 区间的 teaching form 可写为：

$$\hat{p} = \frac{k}{n}, \qquad \mathrm{CI}_{\text{Wilson}} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}.$$

$\hat{p}$ 是 episode-level SR，z 是置信水平对应的正态分位数。变量 k, n 必须来自同一 denominator manifest；zero/all-success、小样本、task/scene/seed clustering 与 optional stopping 应按 EVAL-003 规则处理。这个统计公式只说明报告不确定性，不证明 scorer、policy 或项目 ownership。

7. Typed Navigation Evaluation Manifest

任何“navigation score / improved navigation / failure root cause”回答至少需要以下字段：

Layer	最低字段	缺失时的 fail-closed 结论
task	task family、dataset revision/license、scene、episode ID、goal type、action/STOP schema、frame/unit	不知道评估的任务与几何语义
split	split、scene split、episode inclusion/exclusion、episode count	不能比较或复述公开数字
simulator	Habitat/VLN-CE commit、scene/navmesh、distance mode、threshold、controller/action step	不能确认 scorer 与环境一致
trajectory	ordered poses、action、timestamp/step、STOP、termination/truncation、invalid action	不能重算 path length、stop 或 failure
metrics	SR、SPL/SoftSPL、NE、OS、TL、collision/timeout、denominator、missing policy	summary-only 不是完整评估
statistics	episode-level rows、task/scene/seed clustering、CI method、zero/all-success handling	不报告聚合不确定性
failure	first divergence、taxonomy label、evidence pointer、recovery/terminal outcome	只能说 symptom，不能说 confirmed root cause
ownership	code/config/data/scorer author、operator、reviewer、artifact custodian	upstream code/Git author 不能成为个人 ownership

推荐 immutable lineage：

```
task_manifest_sha + split_manifest_sha + simulator/scorer_revision
    -> episode_ids + trajectory_schema + controller_revision
    -> raw_episode_trace + termination_reason
    -> per_episode_metrics + denominator_manifest
    -> aggregate/CI_report + failure_taxonomy
    -> checkpoint_sha + ownership_ledger
```

这条链的作用不是增加文档字段，而是让每个数值都能追溯到 episode、scorer、代码 revision 和责任边界。当前仓库没有五个项目的完整 local-direct navigation bundle，所以该 manifest 是面试设计合同，不是现有项目已实现的统一 evaluator。

8. Failure Taxonomy：从 symptom 到 first divergence

诊断顺序固定为：

```
protocol/data
  -> policy/decision
  -> geometry/localization
  -> control/execution
  -> scorer/statistics
  -> unknown
```

类别	典型症状	首查 artifact	不能直接推出
protocol/data	split 泄漏、episode 缺失、goal/viewpoint 不一致、scene/navmesh mismatch	split manifest、episode ID set、revision SHA	模型能力退化
policy/decision	invalid action、early STOP、loop、memory reset	raw action、parser error、policy/memory trace	controller 或 scorer 有 bug
geometry/localization	frame mismatch、drift、goal distance 异常	frame IDs、pose transform、navmesh、relocalization log	平均 NE 已确认根因
control/execution	collision、controller reject、clip、timeout/horizon	controller status、step trace、termination reason	timeout 等于 collision
scorer/statistics	denominator、duplicate、missing row、metric mode、CI 单位错误	denominator manifest、per-episode rows、scorer revision	policy 真失败
unknown	只有 summary、视频或时间顺序	请求 raw trace；保留 `unknown`	用最接近的故事替代证据

一个完整根因表必须写成 `symptom → first verified divergence → root cause → recovery/terminal outcome`。例如“val-unseen SPL 下降”只是 symptom；只有确认 split manifest、scene revision、policy hash、trajectory first divergence 后，才可以把 root cause 收窄到 geometry、policy 或 scorer。

9. Fail-Closed 评估伪代码

```
def evaluate_navigation(manifest, env, policy, scorer):
    # 1. Freeze task/split/simulator/scorer identity before running an episode.
    assert manifest.task_sha and manifest.split_sha
    assert manifest.simulator_sha and manifest.scorer_sha
    expected_ids = manifest.expected_episode_ids()

    rows = []
    for episode_id in expected_ids:
        # 2. Reset the declared scene, goal mode, controller and STOP schema.
        trace = rollout_one_episode(env, policy, episode_id, manifest)
        if trace.policy_sha != manifest.policy_sha:
            return stop("policy_lineage_mismatch")
        if not trace.has_ordered_pose_action_stop_termination:
            return stop("incomplete_episode_trace")

        # 3. Score one complete episode; do not silently drop invalid rows.
        row = scorer.score(trace, manifest.metric_config)
        if row.episode_id != episode_id or row.duplicate_key:
            return stop("duplicate_or_misaligned_episode")
        rows.append(row)

    # 4. Build the denominator from expected/included/excluded IDs explicitly.
    denominator = build_denominator_manifest(expected_ids, rows, manifest)
    if denominator.missing_ids or denominator.invalid_ids:
        return stop("non_recomputable_denominator")

    # 5. Aggregate SR/SPL/NE/OS/TL and attach CI plus failure evidence pointers.
    report = aggregate_with_ci(rows, denominator, manifest.statistics_config)
    return report
```

这段代码是评估身份和安全 gate 的教学目标，不是五个项目已有的统一 scorer。尤其不能用 `except: continue` 隐藏 invalid episode，也不能从 successful rows 反推 denominator。若无法得到 expected/included/excluded IDs 与 raw trace，输出应是 `not_recomputable` 或 `unknown`。

10. Deterministic Probes：把 source 语义变成面试检查

Probe	固定观察/输入	预期结论	面试要点
P19-01	`Success.update_metric` 检查 STOP 与 `distance < success_distance`	success 需要 stop + threshold	位置接近不等于完成
P19-02	`Success` 依赖 `DistanceToGoal`	measure identity 绑定	不能换成任意 distance
P19-03	`POINT` 调用 geodesic distance 到 goal position	POINT 不是欧氏距离	记录 navmesh revision
P19-04	`VIEW_POINTS` 调用 geodesic distance 到 viewpoints	viewpoint 集合改变前置量	不混 goal center
P19-05	`SPL.reset_metric` 保存 start/end distance 并依赖 Success	shortest/success 同 episode	不能拼不同 scorer
P19-06	`SPL.update_metric` 累加相邻 pose Euclidean step	path length 不是 action count	需要 ordered poses
P19-07	`success * shortest / max(shortest, actual)`	失败 SPL 为 0，绕路降分	不把 TL 当 SPL
P19-08	SoftSPL 使用连续 soft-success × path penalty	SoftSPL ≠ binary SPL	需固定 soft-success 实现
P19-09	README 列出 split 与 TL/NE/OS/SR/SPL	namespace 进入 protocol tuple	不能跨 split 平均
P19-10	`EVAL.EPISODE_COUNT == -1` 才全量评估	子集/过滤会改变 estimand	denominator 先于平均数

这些 probe 是 source-text contract 和手算检查，不是 simulator runtime、benchmark reproduction、训练稳定性或个人项目结果。

11. 方法比较：面试一张表讲清楚

对象	输入/状态	输出或信号	最小可审计证据	常见误写
Success	final pose + STOP + goal mode	binary s_i	stop flag、distance measure、threshold	到过附近就成功
SPL	success + shortest + actual path	success-weighted efficiency	shortest、ordered poses、same episode	action count 就是路径
SoftSPL	soft success + path penalty	连续进度诊断	pinned scorer 的 q_i^{soft}	直接当 SR/leaderboard SPL
NE	final pose + goal geometry	final navigation error	final pose、geodesic scorer	NE 低就一定 stop 成功
OS	task-specific oracle success	progress/any-time signal	exact scorer predicate、time window	所有 benchmark 的 OS 同义
TL	steps 或 physical path length	trajectory cost	unit、controller step、termination	TL 低就是高效成功
failure taxonomy	raw trace + first divergence	diagnosis label	trace pointer、revision、recovery outcome	summary 能证明根因

12. 五项目真实性边界

项目	可以安全连接的局部 hook	当前不能说	若要升级需要
OpenPiVLA	image/language/action schema、normalizer/parser/controller contract	已完成 VLN/ObjectNav evaluator、SPL/NE 或 navigation rollout	local split/scorer、episode trace、checkpoint 与 ownership

项目	可以安全连接的局部 hook	当前不能说	若要升级需要
Robot Harness	semantic-navigation tool schema、2D tri-state occupancy、PCD/map interface	个人 navigation policy、真实 robot episode、SPL/OS/collision safety	navigation collector、统一 scorer、raw episodes
OPD	point/box/text route、strict/lenient scorer、grounding evidence	closed-loop navigation、3D route、导航 success uplift	environment state/action、trajectory、matched eval
WAM	camera/controller/provenance、simulator diagnostic	navigation benchmark、移动平台或 navigation metric	navigation task bundle、scorer、ownership
UWM/LIBERO	simulated manipulation protocol、loader/launcher diagnosis	VLN/ObjectNav、mobile navigation、navigation generalization	navigation dataset、episode rows、matched protocol

安全回答顺序是：`public definition → pinned paper/repo → protocol identity → local inventory → local-direct exception → missing artifact → ownership`。当前仓库的结论是“没有个人导航结果”，不是“项目导航失败”。

13. 两档面试回答

13.1 约 30 秒回答

我会先把 success 定义成协议谓词：必须在指定 `POINT` 或 `VIEW_POINTS` 距离模式下满足 threshold，并且 agent 真正调用 STOP。SPL 再用同一 episode 的参考最短距离、ordered pose 累计路径和 success 计算；SoftSPL 是连续 soft-success 加 path penalty，不能当 binary SR。NE、OS、TL 也必须带 task、split、scorer、unit 和 denominator。评估前我会固定 expected/included/excluded episode IDs、simulator/scorer revision 和 raw trace；缺任何一层就 fail closed。对我的项目，目前只有 schema/interface/upstream hook，没有完整 navigation bundle，所以不会把 summary 或训练 loss说成个人导航结果。

13.2 约 2 分钟机制回答

我先问评估对象是谁：task、split、scene、episode ID、goal mode、STOP schema 和 scorer revision。Habitat 的 Success 依赖 DistanceToGoal，并要求 stop 加 threshold；POINT 和 VIEW_POINTS 走不同 goal 几何，距离是 simulator geodesic，不是直接欧氏距离。SPL 用同一 episode 的 shortest path、实际 pose path 和 binary success，SoftSPL 则把 binary success 换成官方实现定义的连续 soft-success，再乘路径惩罚。NE、OS、TL 不能脱离 benchmark 的 exact predicate、unit 和时间窗口。然后我检查 denominator：VLN-CE 的 `EVAL.EPISODE_COUNT=-1` 才是全量，任何只评估成功行、过滤 invalid、丢失 episode 或 duplicate row 都会改变 estimand。最后按 protocol/data、policy/decision、geometry/localization、control/execution、scorer/statistics、unknown 做 first-divergence 诊断；没有 raw trace，只保留 symptom 或 unknown。我的五个项目目前缺少统一 navigation scorer、split manifest、逐 episode trace 和 ownership，因此我会把这些当成面试中的评估合同和补证据计划，而不是声称已有导航 benchmark 结果。

14. 高频追问与答题抓手

追问	回答抓手
到过目标附近但没 STOP，算 success 吗?	按固定 Habitat measure 不算；success 同时需要 STOP 与 threshold。
SPL 为什么不能用 action 数计算?	Habitat probe 累计相邻 pose 的 Euclidean step distance，action count 没有物理单位。

追问	回答抓手
POINT 和 VIEW_POINTS 的差别?	goal position 与 viewpoint 集合不同，都会影响 geodesic distance 和 success 前置量。
SoftSPL 是不是更宽松的 SR?	不是；它是连续 soft-success × path penalty，必须带 scorer 定义，不能替换 binary SR。
`EVAL.EPISODE_COUNT=-1` 为什么重要?	它表达全量 episode；子集、过滤和 early cutoff 会改变 denominator/estimand。
只有平均 NE 能找到根因吗?	不能；先要 per-episode trace、first divergence、frame/scorer/revision。
timeout 算 collision 吗?	不应默认相同；termination reason、controller status 和 horizon 必须分开。
论文 SPL 和 leaderboard SPL 不同怎么办?	报告 split、revision、hardware/build、scorer 和 protocol tuple，不直接比较裸数字。
你的项目做过 navigation evaluator 吗?	当前没有 local-direct bundle；只能讲 schema/interface/upstream 机制和补证据路径。

15. 本页证据边界与下一步

本页已把 PER-019 source freeze 转成可学习知识：success/STOP、POINT/VIEW_POINTS、geodesic distance、SPL、SoftSPL 的实现边界、NE/OS/TL namespace、split/denominator、Wilson CI、typed manifest、failure taxonomy、fail-closed 伪代码、10 个 deterministic probes、五项目 truth boundary、两档口述回答与追问抓手均已覆盖。

直接支持来自 PER-019 Source Freeze、Anderson SPL、Krantz VLN-CE、pinned Habitat-Lab `nav.py`、pinned VLN-CE README、VOL 05 EVAL-003 与 PER-004。SoftSPL 的 q_i^{soft} 在本文中明确标为教学抽象；不能把未逐行复核的具体 soft-success 函数写成本地实现事实。

当前仍为 `WARN`：没有五个项目的 local-direct navigation episode bundle、统一 scorer、split manifest、per-episode trace、matched checkpoint/eval 或 ownership confirmation；没有真实口述，所以不创建 coaching state、不生成题卡、不改变 mastery。下一 bounded slice 才考虑生成 `VLA-PER-019` atomic card；PER-020 source gate 与 SYS-VLA-004/ PER-018 真实口述仍是独立队列。

References

1. Anderson et al., On Evaluation of Embodied Navigation Agents, arXiv `1807.06757v3`。
2. Krantz et al., Beyond the NavGraph: VLN in Continuous Environments, arXiv `2004.02857v2`。
3. Habitat-Lab `nav.py`，commit `0fb6f43ffe806a8088a171b036336c093bcf604e`。
4. VLN-CE `README.md`，commit `729d141b2ee10628061ada74dd3a5b9f70faeba5`。
5. VOL 05 EVAL-003 statistical contract。
6. VOL 06 PER-004 VLN/ObjectNav source freeze。
7. VOL 06 PER-019 Source Freeze。

Part 20 / 43 · `interview_materials/vla/knowledge/vol_06_per020_robustness_safety_fallback.html` · SHA-256 `412F737102741E0B826667021F3BB4B835FC36492E3A2685984AA17EA6A2BD55`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`412f737102741e0b826667021f3bb4b835fc36492e3a2685984aa17ea6a2bd55`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_06_per020_robustness_safety_fallback.md`

VOL 06 PER-020：Robustness、Safety、Fallback 与风险门

状态：`knowledge_drafted / VLA-PER-020 / source_frozen / project_truth_bounded / card_synced / WARN` 更新日期：2026-07-21 权威来源：PER-020 Robustness / Safety Source Freeze 学习边界：本页把 corruption protocol、dataset shift uncertainty、risk–coverage、hard constraint、风险门、chunk queue flush、fallback、recovery 与 E-stop 转成可面试、可审计的知识；题卡已在独立切片同步并保持 `unseen`，不修改 276 张题卡的 mastery，不创建 `coaching_state.md`。

问题（Question）

VLA 面对视觉腐蚀、传感器 dropout、OOD 场景或执行风险时，怎样区分 robustness score、uncertainty calibration、hard constraint、risk gate、fallback、recovery 与 E-stop？如何让每个结论都能回到 corruption family、severity、split、seed、denominator、checkpoint 和 raw trace，而不是把高 confidence 或模拟器 cost 误写成安全保证？

1. 学习出口

完成本页后，面试中应能做到：

1. 解释为什么 robustness 不是一个裸分数，而是 corruption family、severity、split、seed、metric、denominator 的协议元组。
2. 区分预生成 corruption 与在线随机注入，说明 severity 表、RNG、dtype、clip 与 artifact SHA 对复现的影响。
3. 说明 clean accuracy、OOD confidence、calibration、risk–coverage、false-safe 和 false-alarm 的边界。
4. 设计从 observation validity 到 policy proposal、risk gate、bounded prefix、queue flush、recovery、fallback、E-stop 的执行状态机。
5. 写出 chunked VLA 风险事件后的 fail-closed 伪代码，并说明 stale action 为什么必须被丢弃。
6. 用 visual/sensor、representation/model、geometry/time、dynamics/control、protocol 和 safety orchestration 组织 failure diagnosis。
7. 对 OpenPiVLA、Robot Harness、OPD、WAM/Cosmos、UWM/LIBERO 给出不越界的项目回答。
8. 明确哪些只是公开机制、哪些是本地证据、哪些是推断、哪些仍未核实。

2. 一分钟心智模型

```
fixed task / split / clean-shift pair
                ↓
corruption family + severity + seed + generated artifact
                ↓
policy output + uncertainty + validity bits
                ↓
risk gate: accept / reject
                ↓
bounded action prefix or flush queued chunk
                ↓
recover → fallback → E-stop (latched terminal state)
                ↓
episode trace + cost/risk/coverage + failure taxonomy
```

主线只有一句话：**先冻结输入与评估协议，再判断模型是否知道自己不确定，最后把不确定性连接到有日志、有终止语义的控制动作。** `randomize`、`confidence`、`cost` 或 `safety` 这些字段名本身，都不能替代这三层证据。

3. Robustness：先冻结 shifted input，再谈分数

3.1 Corruption family 与 severity

Hendrycks & Dietterich 的 ImageNet-C/P 语境把常见扰动按 family 组织，并为每个 family 设定离散 severity。它的启发不是“所有 corruption 都能统一排序”，而是要求比较时写清楚：

字段	面试中的含义	缺失时的结论
family	Gaussian noise、shot noise、brightness、JPEG 等具体扰动	不知道变了哪一种输入因素
severity	family-specific 的 1–5 参数档位	不能把不同 family 的同号 severity 当作同一物理量
source mode	预生成样本或在线随机生成	不能判断 deterministic 与 RNG 语义
seed/RNG	NumPy 或其他随机数状态	不能重现随机 corruption
clean/shifted pair	同一任务样本的 clean 与 shifted 对照	不能把模型差异归因于 shift
denominator	有效 episode、图像或样本集合	平均分可能改变 estimand

官方 `corruptions.py` 为 family 提供 severity 参数表；Gaussian、shot、impulse、speckle 等路径使用随机数，并涉及归一化与 clip。因此在线注入必须记录 RNG、dtype、clip 和输出 artifact；预生成 JPEG corruption 更适合做可重复的评估输入，但不等于真实传感器噪声。

3.2 一个可审计的 shifted metric

对 corruption family c、severity s，令 $\mathcal{E}_{c,s}$ 为固定的有效 episode 集合，$x_{e,c,s}$ 为按协议生成的输入，m 为任务 metric，则：

$$M_{c,s}(\theta) = \frac{1}{|\mathcal{E}_{c,s}|} \sum_{e \in \mathcal{E}_{c,s}} m(\pi_\theta, x_{e,c,s}).$$

变量解释：θ 是固定 checkpoint，c, s 指定 corruption family 与 severity，m 可以是任务 success、动作误差或 collision-related metric。直觉是“先固定被评估的样本集合，再计算 shift 下的策略分数”；实现上对应 clean/shifted pair、episode ID、checkpoint SHA 和 denominator manifest。若 clean 与 shifted 使用不同 episode 集合，$M_{c,s} - M_{\text{clean}}$ 就不再是纯粹的 robustness effect。

3.3 汇总分数不能隐藏 family

当需要汇总多个 family 时，可以报告 normalized degradation 或 mean corruption error，但必须保留 family-level rows。平均值适合做导航指标，不适合隐藏某一类传感器故障。面试回答应同时给 clean metric、每个 family/severity 的 shifted metric、worst-case 或分位数、失败样本与 denominator、seed/CI 或 episode-level variability。

4. Dataset shift 下的 uncertainty 与 calibration

4.1 Confidence 不是 risk

Ovadia et al. 研究的是 dataset shift 下 predictive uncertainty 的行为。迁移到 VLA 时，关键问题不是“模型有没有一个 confidence 字段”，而是 confidence 是否随 shift 仍有排序能力、接受的动作是否真的比拒答更安全、高 confidence 错误是否形成 false-safe、低 confidence 但实际安全是否造成 false-alarm，以及 calibration 是否在 shifted split 上重新测量。

因此 clean accuracy 或平均 confidence 都不能单独证明 OOD safety。至少要同时报告 shifted calibration、selective prediction 的 risk–coverage 和拒答样本的处理结果。

4.2 Risk–coverage

令 `accept=1` 表示风险门允许执行，C 为 coverage，ℓ 为动作或任务损失，则：

$$\text{Risk}(C) = \mathbb{E}\big[\ell(\hat{y}, y) \mid \text{accept} = 1\big], \qquad C = \Pr(\text{accept} = 1).$$

变量解释：$\hat{y}$ 可以是动作、目标 grounding 或任务结果，y 是对应真值，ℓ 可以是动作违规、collision、goal failure 或任务损失。直觉是 coverage 越低，系统只执行更有把握的样本；实现上必须保留被拒答的样本、转交 fallback 的原因和最终 outcome，不能只对成功执行的样本求平均。

4.3 Uncertainty 的三层输入

层	可观测量	不能直接推出
observation validity	missing frame、depth invalid、timestamp age、camera mask	模型已经知道语义风险
policy uncertainty	ensemble/variance、distribution entropy、calibrated score	confidence 高就一定安全
outcome risk	collision、goal failure、constraint cost、fallback outcome	一次成功就证明 calibration

5. Safety：把风险连接到执行状态机

5.1 从 proposal 到 E-stop

一个 fail-closed 执行器至少应区分以下状态：

```
observation_valid
  -> policy_proposal
  -> uncertainty/risk_gate
       -> accept: apply bounded action prefix
       -> reject: stop applying queued chunk
                -> recover: reobserve / stop / retreat / retry
                -> fallback: safe controller or human handoff
                -> E-stop: cut actuation and latch terminal state
```

每次状态转移必须记录 `event_id`、时间戳、observation/action/checkpoint SHA、gate reason、queue length、termination reason 与 operator/ownership。`fallback` 是控制权转移，不是换一个 prompt；`recovery` 是有次数上限、重新观测和升级条件的状态，不是隐含 retry。

5.2 Hard constraint 与风险门

Safety Gym 的公开语境把 hazard constraint 与 cost 放在环境接口中，并要求显式配置约束开关；`info` 中的 `cost_hazards` 与聚合 `cost` 只能说明该模拟器记录了什么，不能直接说明真实机器人安全。

面试中可把风险门抽象为：

$$g(o_t, \hat{a}_t, u_t) = \mathbf{1}[\mathrm{valid}(o_t) \wedge \mathrm{risk}(u_t) \le \tau \wedge \mathrm{constraint}(\hat{a}_t) = 1].$$

变量解释：o_t 是观测，$\hat{a}_t$ 是策略提议，u_t 是不确定性或风险估计，τ 是门限。直觉是“只有输入有效、风险不超过阈值、动作满足硬约束才允许 apply”；实现上要绑定 validity bit、risk estimator、constraint checker、gate reason 和日志。这个公式是教学抽象，不能被写成五个项目已经拥有的安全控制器。

5.3 Chunked VLA 的 stale action

VLA 往往一次产生动作 chunk，再执行前 K 个动作。若在第 t 步出现风险事件 $r_t = 1$，继续执行原 chunk 会把已经过期的决策施加到新状态。最低安全语义是先停止 apply，再 flush queue：

$$\mathrm{execute}(a_t) = \begin{cases} a_t, & r_t = 0 \wedge \mathrm{valid}(o_t), \\ \mathrm{safe_prefix}(o_t), & r_t = 1 \wedge \mathrm{recoverable}(o_t), \\ \mathrm{EStop}, & r_t = 1 \wedge \neg\,\mathrm{recoverable}(o_t). \end{cases}$$

变量解释：r_t 是风险事件，`safe_prefix` 是经过控制器和约束检查的短动作前缀，`recoverable` 是明确的状态判定。直觉是“风险发生后，旧队列不再拥有默认执行权”；实现上必须记录 action age、queue length、flush event、controller state 和 E-stop latch。公式只定义执行合同，不证明任意项目满足硬件安全。

6. Robustness、Safety 与 Recovery 的最小 manifest

Layer	必填字段	失败时的回答
input	camera/sensor name、resolution、dtype、normalization、timestamp	不知道扰动作用在哪个输入
corruption	family、severity、parameter table、online/offline、RNG、artifact SHA	不能复现 shifted input

Layer	必填字段	失败时的回答
data	dataset revision、scene/object split、episode ID、clean/shifted pair	不能比较 clean 与 shift
policy	checkpoint SHA、pre/post-process、action head、chunk horizon	分数无法绑定模型
risk	uncertainty estimator、threshold、calibration split、accept/reject rule	不能解释 gate
safety	hard constraint、controller、safe prefix、fallback、E-stop latch	不能把 cost 当 safety
metric	clean/shifted score、risk–coverage、cost、denominator	平均分不是完整结论
statistics	seed、episode rows、CI、missing/invalid policy	不能判断方差或丢样本
ownership	code/config/data/scorer author、operator、reviewer、artifact custodian	upstream 不是个人贡献

推荐 immutable lineage：

```
task/split manifest SHA + corruption/config SHA
    -> generated clean/shifted artifact + RNG state
    -> policy checkpoint + risk/calibration revision
    -> action proposal + gate/queue/flush trace
    -> per-episode metric + cost + failure label
    -> aggregate report + ownership ledger
```

这条链的目的不是增加字段，而是让一个风险结论能回到输入、模型、门控、执行、结果和责任边界。

7. Fail-Closed 伪代码

```
def safe_chunk_step(manifest, obs, action_chunk, risk_estimator, controller):
    # 1. Reject invalid observations before trusting model uncertainty.
    if not obs.valid or obs.timestamp_age > manifest.max_observation_age:
        flush_queue(reason="invalid_observation")
        return fallback_or_estop("observation_invalid")

    # 2. Evaluate risk and hard constraints on the current state.
    risk = risk_estimator.score(obs, action_chunk)
    if risk > manifest.risk_threshold:
        flush_queue(reason="risk_gate")
        if controller.recoverable(obs, risk):
            return controller.safe_prefix(obs)
        return estop_latched("unrecoverable_risk")

    # 3. Apply only a bounded prefix; never assume a stale chunk is valid forever.
    prefix = action_chunk[:manifest.replan_steps]
    if not controller.constraint_ok(obs, prefix):
        flush_queue(reason="constraint_violation")
        return fallback_or_estop("hard_constraint")

    # 4. Record lineage and execute; the next observation can trigger a new gate.
    log_event(manifest, obs, prefix, risk, queue_length=len(action_chunk))
    return controller.apply(prefix)
```

这段伪代码是风险门的教学目标，不是五个本地项目已有的统一安全 runtime。真实实现还要处理并发 action broker、controller acknowledgement、sensor watchdog、hardware E-stop、human handoff、retry budget、atomic logs 和 replay。

8. Failure Taxonomy：从 symptom 到 first divergence

诊断顺序固定为：

```
input validity
  -> corruption/shift identity
  -> representation/model calibration
```

```
    -> geometry/time synchronization
    -> dynamics/control execution
    -> risk orchestration
    -> unknown
```

类别	典型症状	首查 artifact	不能直接推出
visual/sensor	遮挡、曝光、深度空洞、dropout、timestamp stale	validity bit、sensor trace、clean/shifted pair	模型语义理解退化
representation/model	OOD embedding、confidence miscalibration、action collapse	calibration bin、risk–coverage、checkpoint	一定是传感器问题
geometry/time	frame、标定、延迟、单位、轨迹同步错误	frame IDs、timestamps、transform log	policy 已经失效
dynamics/control	saturation、collision、controller reject、stale chunk	controller status、action age、queue trace	risk estimator 错
task/protocol	split、severity、layout randomization、denominator mismatch	manifest、config、episode set	模型鲁棒性差
safety orchestration	gate 未触发、fallback 不可用、E-stop 未 latch	event log、state transition、hardware audit	仅靠最终 success 能解释
unknown	只有 summary、视频或时间顺序	请求 raw trace；保留 `unknown`	用最接近的故事替代证据

最小故障注入矩阵：

注入	应观察	面试结论
camera severity 1→5	clean/shifted pair、family、seed、risk–coverage	只能谈视觉 shift 协议
depth dropout / invalid timestamp	last valid frame、gate reason、queue flush	不能只看最终成功率
high-confidence wrong action	calibration bin、false-safe、fallback outcome	高 confidence 不等于安全
hazard proximity	constraint flag、cost_hazards、safe prefix、E-stop latch	Safety Gym cost 不等于硬件安全
stale action chunk	queue length、action age、flush event、controller state	需要风险后的执行语义

完整根因表必须写成 `symptom → first verified divergence → root cause → recovery/terminal outcome`。没有 first divergence，最多只能报告 symptom 或 unknown。

9. 方法比较：面试一张表讲清楚

对象	主要作用	最小可审计证据	常见误写
corruption benchmark	测量输入 shift 下的性能变化	family、severity、pair、seed、denominator	一个平均分代表所有鲁棒性
calibration	检查不确定性与真实风险的关系	shifted bins、reliability/risk curve、拒答结果	confidence 就是 calibrated risk
hard constraint	禁止不可接受动作或状态	constraint checker、单位、controller response	reward/cost 自动等于约束
risk gate	决定 accept/reject/transfer	threshold、gate reason、policy version	阈值没有 protocol
fallback	转移控制权到安全 controller/人	entry/exit、max retry、owner、trace	换 prompt 就是 fallback
recovery	重新观测、短动作、重试或退避	budget、backoff、state reset、outcome	retry 不需要上限
E-stop	切断执行并锁存终止状态	hardware event、latch、operator、reset policy	simulator stop 就是 physical E-stop

10. 五项目真实性边界

项目	可以安全连接的局部 hook	当前不能说	若要升级需要
OpenPiVLA	image/action normalizer、parser、controller contract、shifted-input eval design	已有 OOD robustness、calibration、zero-accident safety	clean/shifted bundle、risk log、checkpoint、matched eval、ownership
Robot Harness	tool schema、地图/点云输入、约束与日志字段设计	Safety Gym/真实机器人 cost、fallback 成功率或 E-stop ownership	local risk/fallback runtime、逐 step trace、hardware audit
OPD	grounding confidence、negative/ambiguous case、risk–coverage 设计	grounding robustness uplift 或部署安全保证	shifted holdout、calibration report、failure trace、owner
WAM / Cosmos	simulator diagnostics、visual shift taxonomy、失败 trace	上游 randomization 或公开 rollout 是个人 sim-to-real/安全结果	fixed artifact、paired shift eval、controller/safety bundle
UWM / LIBERO	fixed-init/reset protocol、strict denominator、failure taxonomy	真实机器人 OOD 泛化、碰撞率或安全 controller 结果	matched episode rows、risk/cost trace、hardware or simulator gate

回答顺序是：`public mechanism → fixed source → local evidence → inference → unverified → missing artifact → ownership`。当前结论是“有可迁移的协议与接口 hook”，不是“已有个人安全结果”。

11. 两档面试回答

11.1 约 30 秒回答

我会先冻结 robustness protocol：corruption family、severity、clean/shifted pair、split、seed、checkpoint 和 denominator 都要固定。预生成 corruption 与在线随机注入不能混为一谈，severity 也不是跨 family 的统一物理量。然后在 shifted split 上检查 calibration 和 risk–coverage，不能只看 confidence。执行时把 observation validity、risk gate、hard constraint、chunk queue flush、recovery、fallback 和 E-stop 分层；风险事件出现后先停止旧 chunk，只有明确 recoverable 才执行 safe prefix。我的五个项目目前没有完整 local corruption bundle、逐 step risk log 或硬件 E-stop audit，所以我会讲机制和补证据路径，不把 Safety Gym 或上游 randomization 说成个人安全结果。

11.2 约 2 分钟机制回答

我会把问题拆成 measurement、uncertainty 和 execution 三层。Measurement 层先固定 task、split、episode IDs、输入传感器、corruption family/severity、RNG、checkpoint 和 denominator；clean 与 shifted 必须是可配对、可复现的输入，不能只报一个平均分。ImageNet-C 的 severity 是 family-specific 参数档位，Gaussian 等 corruption 的在线生成还涉及随机数、归一化和 clip。Uncertainty 层不把最大 softmax 当安全证据，而是在 shift 下看 calibration、risk–coverage、false-safe 和 false-alarm，保留被拒答和转 fallback 的样本。Execution 层把风险门接到控制权：观测无效或风险超阈值时 flush 排队 chunk；如果能恢复，就重新观测或执行受约束的短前缀；不能恢复则转 safe controller、人接管或锁存 E-stop。每次转移都记录 gate reason、action age、queue length、controller response 和 outcome。对我的项目，我可以把 normalizer/parser/controller contract、tool schema、grounding confidence 或 fixed-init denominator 作为局部 hook，但当前没有统一 shifted-input bundle、逐步 risk trace 和 hardware audit，因此不会声称已经完成 OOD 泛化或真实安全闭环。

12. 高频追问与答题抓手

追问	回答抓手
severity=5 一定比另一个 family 的 severity=3 更严重吗?	不一定；severity 是 family-specific 参数索引，不能当统一物理单位。
预生成 JPEG 与在线 Gaussian noise 能直接比较吗?	不能直接比较；source mode、RNG、dtype、clip 和 sensor semantics 不同。
confidence 高但动作错，说明什么?	可能是 shifted miscalibration，需查 risk–coverage、false-safe 和 calibration bin。
risk gate 触发后能继续执行旧 chunk 吗?	默认不能；先停止 apply、flush queue，再决定 recovery/fallback/E-stop。
Safety Gym 的 cost=0 是否代表真实安全?	不代表；它只是在固定 simulator/config 下的 cost 记录。
randomize_layout=False 与 True 是同一泛化评估吗?	不是，布局分布不同，必须分别报告。
fallback 和 retry 有什么区别?	fallback 是控制权转移，retry 是有预算和状态条件的 recovery。
没有 raw trace 能确认 collision 根因吗?	不能；只保留 symptom/unknown，先找 first verified divergence。
你的项目做过 robustness/safety 吗?	当前只有接口、协议或诊断 hook，没有可复算 shifted bundle、逐 step risk log 或硬件 E-stop ownership。

13. 本页证据边界与下一步

本页已把 PER-020 source freeze 转成可学习知识：corruption identity、severity/RNG、clean-shift metric、dataset-shift calibration、risk–coverage、risk gate、hard constraint、queue flush、fallback/recovery/E-stop、typed manifest、fail-closed 伪代码、故障注入思路、五项目 truth boundary、两档口述回答与追问抓手均已覆盖。

直接支持来自 PER-020 Source Freeze、Hendrycks & Dietterich、Ovadia et al.、Hendrycks robustness README/`corruptions.py`、OpenAI Safety Gym README/`engine.py`、VOL 04 WM-031 与 WM-038。风险门和 chunk 执行公式是教学抽象；它们不证明任何本地项目已经有安全 controller 或 E-stop 审计。

当前仍为 `WARN`：没有五个项目的 local-direct shifted-input bundle、逐 step risk log、hardware E-stop audit、matched robustness panel 或 ownership confirmation；没有真实口述，所以不创建 coaching state、不改变 mastery。下游 VLA-PER-020 Atomic Card 已同步并保持 `unseen/WARN`；下一 bounded slice 选择一题真实口述或独立 fact review。

References

1. Hendrycks & Dietterich，Benchmarking Neural Network Robustness to Common Corruptions and Perturbations，arXiv `1903.12261`。
2. Ovadia et al.，Can You Trust Your Model's Uncertainty?，arXiv `1906.02530v2`。
3. Hendrycks robustness `README.md` 与 `corruptions.py`。
4. OpenAI Safety Gym `README.md` 与 `engine.py`。
5. VOL 04 WM-031 reality-gap taxonomy 与 VOL 04 WM-038 uncertainty/safety/fallback。
6. VOL 06 PER-003 multiview/temporal source freeze 与 VOL 06 PER-015 perception-action gate source freeze。

Part 21 / 43 · `interview_materials/vla/questions/vol_06_per001.html` · SHA-256
`67D43068A1FD7AE2DDE0A61D0924D6CFA5022B233029EC1DA193BAA5C8B6D471`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`67d43068a1fd7ae2dde0a61d0924d6cfa5022b233029ec1da193baa5c8b6d471`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per001.md`

VOL 06 Perception / Grounding / Navigation：PER-001 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`VLA-PER-001` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-001 Grounding / Coordinate Knowledge；来源边界见 Source Freeze。

Point / Box / Text Grounding 与坐标协议

▼ `L3` `P0` **VLA-PER-001 · 为什么 grounding 面试必须先讲 coordinate contract，而不能只报 IoU 或 point hit?**

面试意图：判断你能否把表示、预处理、解析、评估和下游 action 接口串起来，并识别 strict=0 可能是协议失败而非能力为零。

30 秒回答：point/box/text 不是同一输出：要固定坐标空间、图像尺寸、参数化、parser 和 scorer。`cxcywh` 需按 `W,H` 转 `xyxy`，还要逆 letterbox；strict/lenient 结果不能混报，IoU/Hit 也不等于 action success。

机制展开（约 2 分钟）：先记录表示、空间、尺寸、顺序和 parser。normalized `cxcywh` 先乘 `[W,H,W,H]` 再转 `xyxy`；有 crop/letterbox 时还要减 padding、除 scale。strict 检查部署格式，protocol-fixed 统一 prompt，lenient 只作兼容诊断。IoU/Hit 只说明二维几何，后面仍有 parser、深度/外参、planner、action normalization 和 controller predicate。OPD 的五 expert/point/box/text route 是项目证据，不能推出 learned router 或单独 uplift。

诊断 / 证据：固定 prompt、image size、transform、parser 和 scorer；用 sentinel box/point 做 round-trip，分别测 original strict、protocol-fixed strict、lenient geometry，记录有效样本、距离单位和每个失败阶段。若 strict=0，先检查 tag、括号、坐标范围、x/y 轴、`W/H` 缩放和 padding offset，再判断模型几何能力；同时保存 raw output、transform metadata 和 downstream action trace。

项目挂钩：OPD 的 raw-holdout、point protocol reconciliation、format-conflict metadata 和 by-format/by-pool checkpoint 表可作为真实例子；安全表述是“我把 grounding 结果按协议和证据层拆开”，不把静态 holdout 写成机器人闭环成功。

追问树：`cxcywh` 与 `xyxy` 怎样互转？为什么 letterbox 会造成系统偏移？point 中心能否直接当抓取点？strict=0 是否说明模型不会 grounding？IoU 高但任务失败可能在哪一层？lenient parser 何时可以使用？

易错点：把四个数默认当 `xyxy`；x/y 乘错 `W/H`；忽略 crop/padding；把 phrase 命中当目标绑定；把 strict=0 当能力为零；把 IoU/Hit 当 action success；把 OPD 组合结果写成单一 OPD uplift。

危险答案：说“坐标归一化后任何分辨率都天然一致”“IoU 高就能抓到目标”“lenient 分数更高所以模型更强”“point 就是 3D grasp pose”或“OPD 的 10k holdout 证明真机泛化”。

来源：[1][2][3][4]　**练习状态**：unseen

参考来源

1. PER-001 Grounding / Coordinate Knowledge：表示合同、变换公式、评估阶梯、pipeline 与 OPD 边界。
2. PER-001 Grounding / Coordinate Source Freeze：Grounding DINO、RoboPoint 和 protocol 指纹。
3. Grounding DINO official inference.py：`cxcywh → xyxy` 后处理实现。
4. OPD Claim–Evidence Inventory：point/box/text route、strict/lenient/protocol-fixed scorer 与项目 ownership 边界。

Part 22 / 43 · `interview_materials/vla/questions/vol_06_per002.html` · SHA-256
`3B12BF46F4DF25440AB34AC39AD7CF78DBFE96AB04B3BD9993611F7688F4B073`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`3b12bf46f4df25440ab34ac39ad7cf78dbfe96ab04b3bd9993611f7688f4b073`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per002.md`

VOL 06 Perception / Grounding / Navigation：PER-002 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`VLA-PER-002` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-002 3D Lift / Frame Knowledge；来源边界见 Source Freeze。

2D Grounding 如何变成可信的 3D Target

▼ `L3` `P0` **VLA-PER-002 · 一个 grounding point 怎样才能安全地进入 planner 或 controller?**

面试意图：判断你能否把深度、标定、frame、时间同步和不确定性组成可审计链路，而不是把二维中心直接当抓取点。

30 秒回答：先固定 rectified pixel、depth 单位和 `K/distortion`，用反投影得到 camera point，再用带时间戳的 `T_target←camera` 变到目标 frame；做 invalid/uncertainty gate 后才交给 planner。2D holdout 不等于 3D 或闭环成功。

机制展开（约 2 分钟）：深度可来自 RGB-D、stereo、triangulation 或平面假设，记录单位、invalid 和 registration。对 rectified `(u,v,Z)`，`X=(u-cx)Z/fx`、`Y=(v-cy)Z/fy`、`Zc=Z`；再用 `T_target←camera(τ)` 变换，绑定 frame、timestamp、calibration revision。crop/letterbox 要先逆变换；depth 无效、TF 不同步或 uncertainty 超阈值就返回 invalid/safe branch。OpenCV/ROS 只支持机制；OPD 是 2D holdout，WAM 只有 camera/controller 字段，不能声称个人完成 3D lift。

诊断 / 证据：固定 `K/distortion/image_size`、depth unit、frame IDs 和 timestamp，使用 known geometry 做 pixel→camera→pixel roundtrip；再做 camera↔target 逆变换 smoke、静态/运动同步对照、invalid-depth 注入和 uncertainty gate。保存 calibration/reprojection error、TF revision、depth valid mask、planner input/output 与 controller trace；缺任一项就停在 projection/calibration 层。

项目挂钩：OPD 可用于说明 point/box/text 协议与静态 scorer；WAM manifest 可说明 camera、controller、horizon 的 rollout provenance。安全表达是“我能审计 3D lift 合同和缺口”，不把这些材料升级为抓取、导航或真机成功。

追问树：为什么 RGB 图像没有唯一深度？畸变 pixel 能否直接套 pinhole？`T_target←camera` 方向怎样核对？为什么要用 capture-time TF？depth 单位错会怎样？ROS REP-103 是项目事实吗？overlay 看起来对能否证明 controller 正确？

易错点：把 2D 中心当 3D 点；`K` 与 resize image size 不匹配；忽略畸变、padding、depth invalid 或 mm/m；camera→base 方向写反；使用最新而非 capture-time TF；只做 overlay 不做 roundtrip。

危险答案：说“有 RGB 就能恢复深度”“4×4 外参矩阵不需要 frame/时间戳”“ROS 轴序默认适用于所有项目”“3D 点生成就等于抓取成功”或“OPD/WAM 的静态/rollout证据证明 3D 泛化”。

来源：[1][2][3][4]　**练习状态**：unseen

参考来源

1. PER-002 3D Lift / Frame Knowledge：内参、深度、反投影、TF、同步、不确定性与证据阶梯。
2. PER-002 3D Lift / Frame Source Freeze：OpenCV、ROS REP-103/105 与项目边界。
3. OpenCV calibration/depth tutorials：camera matrix、distortion、stereo/depth 机制。
4. ROS REP-103/105：单位、坐标和移动平台 frame convention。

Part 23 / 43 · `interview_materials/vla/questions/vol_06_per003.html` · SHA-256
`61775EA5A25238B0642B44B652B15C7B6F725F4E97B7928F4921812644148C6F`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`61775ea5a25238b0642b44b652b15c7b6f725f4e97b7928f4921812644148c6f`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per003.md`

VOL 06 Perception / Grounding / Navigation：PER-003 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`VLA-PER-003` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-003 Multi-view / Temporal Knowledge；来源边界见 Source Freeze。

多视角 / 时序 Grounding 为什么必须绑定 Schema

▼ `L3` `P0` **VLA-PER-003 · 多相机、多帧输入为什么不能只是把图片 concat 后喂给模型?**

面试意图：判断你能否处理 camera identity、frame、timestamp、presence mask、common-frame fusion 和 action freshness，而不是只谈“更多视觉信息”。

30 秒回答：view/time 都要带 `camera_id、frame_id、K/T revision、timestamp、presence/valid、transform`。先独立预处理，再按 mask fusion；多视角预测先变到 common frame 再关联，history 绑定 control-hz、replan 和 observation age。

机制展开（约 2 分钟）：workspace/wrist 顺序错了，shape 仍可能正确但语义全错；缺失相机不能只置零，要有 presence mask。历史帧用真实 timestamp/stride，不能把 index 当物理时间；frame drop、action repeat 和 replan 会改变 observation age。各 view 输出先经过 `K/distortion` 与 `T_common←camera(timestamp)` 变到共同 frame，再 association/fusion，不能直接平均 pixel。Open X/WM-017/Octo 是机制参照；OPD 是静态 holdout，WAM 是有限 rollout provenance，不能声称项目已有多视角时序闭环。

诊断 / 证据：冻结 per-view manifest、camera order、image size、K/T revision、timestamps、valid mask 和 transform；做 camera swap、掉帧/jitter、missing-view、common-frame roundtrip、view ablation、causal-mask 和 observation-age replay。记录每视角 raw output、association、uncertainty、replan/action trace；缺 calibration/sync/replay 就停在 schema 或 geometry 层。

项目挂钩：OPD 可讲 point/box/text route、strict scorer 和静态 raw-holdout；WAM manifest 可讲 camera resolution、controller、horizon 和 rollout provenance。安全表达是“我能审计多视角/时序接口缺口”，不把多路 camera 字段写成 fusion 或真机收益。

追问树：为什么不同 pixel frame 不能平均？camera_id 和 frame_id 有什么区别？缺 view 为什么不能只补零？history stride 如何换成物理时间？怎样防 future leakage？observation age 超阈值怎么办？Octo 的 observation group 能证明你的项目用了同样 fusion 吗？

易错点：交换 workspace/wrist 顺序；不同分辨率复用同一 K；忽略 timestamp/TF；zero padding 无 presence mask；index 当时间；不做 common-frame association；用 view dropout 单次结果宣称鲁棒性；把静态 holdout 当闭环。

危险答案：说“相机越多一定越准”“concat 后模型自动知道视角”“缺失图置零即可”“历史帧天然没有泄漏”“把不同 camera 的坐标直接平均”或“OPD/WAM 已证明 multi-view temporal fusion 提升真机成功”。

来源：[1][2][3][4]　练习状态：unseen

参考来源

1. PER-003 Multi-view / Temporal Knowledge：per-view schema、fusion、common-frame、mask 与 observation-age。
2. PER-003 Multi-view / Temporal Source Freeze：Open X、Octo、WM-017 与 OPD/WAM 边界。
3. Octo architecture/execution source audit：observation groups、task/readout token 与 attention 参照。
4. WM-017 Heterogeneous Schema Source Freeze：camera/state/action/timestamp/mask/normalizer 的跨 embodiment 合同。

Part 24 / 43 · `interview_materials/vla/questions/vol_06_per004.html` · SHA-256 `5EF81D434993D048FF00F4F7AB2C676D52C73A2911111BA0307786BCEFAAA856`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`5ef81d434993d048ff00f4f7ab2c676d52c73a2911111ba0307786bcefaaa856`；生成自：`interview_materials\vla\questions\vol_06_per004.md`

VOL 06 Perception / Grounding / Navigation：PER-004 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`VLA-PER-004` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-004 VLN / ObjectNav Knowledge；来源边界见 PER-004 Source Freeze。本题不把现有项目证据升级为导航结果。

如何设计可复现的 VLN / ObjectNav 评估协议

▼ `L3` `P0` **VLA-PER-004 · VLN、ObjectNav 和 REVERIE 的 success/SPL 协议怎样拆开，为什么不能只报一个成功率？**

面试意图：判断你能否先冻结任务合同，再解释 stop、success、路径效率和 simulator-to-real 边界，而不是把不同导航 benchmark 混成一个分数。

30 秒回答：先分任务：ObjectNav 是类别/实例与 view-point，VLN 是语言路线，REVERIE 评估 grounding。冻结 scene、起点、split、action/stop、predicate、分母；SR 按 episode success，SPL 依赖 shortest 与实际轨迹。不能横比，更不能把 simulator 分数写成真机结果。

机制展开（约 2 分钟）：先冻结 episode 的 scene、起点、goal/instruction、split 和分母；observation 记录 RGB-D、pose、timestamp/valid，action 记录类型与 repeat。Habitat-Lab 的 `stop` 设置 `is_stop_called`，success 要按 POINT/VIEW_POINTS 距离和 predicate 判断。SR 是 episode success 平均；SPL 还用 shortest path 与轨迹长度。保留 TL/NE/OS 和 failure trace，才能区分 grounding、规划、controller。不同 action space、scene 或版本不能横比；不能写成导航结果。

诊断 / 证据：建立逐 episode manifest，保存 scene/dataset/navmesh revision、start pose、goal/view-point、language/object parser、sensor/action config、stop reason、pose/action trace、success predicate、shortest path、actual path、seed 和 denominator。用固定 protocol 做 reset/seed replay；对 POINT↔VIEW_POINTS、action repeat、controller latency、navmesh 和 stop threshold 做 matched sensitivity；输出 SR/SPL/TL/NE/OS 与失败分类。若只有论文表格、单个 rollout 或 simulator summary，没有 episode trace 和分母，就保持 `WARN`。

项目挂钩：OPD 可连接 point/box/text grounding、坐标和 scorer，Robot Harness 可连接导航工具的 timeout/cancel/compensation；OpenPiVLA 可连接 observation/action packaging 与 chunk/replan；WAM/UWM 可连接 simulator provenance 和 metric ladder。安全表达是“我能把 grounding、导航接口和评估证据拆层”，不把这些项目写成已有 VLN/ObjectNav benchmark 或真机闭环。

追问树：ObjectGoal 的 view-point 与目标点有什么区别？为什么经过目标但没 stop 可能仍失败？SPL 的 L_i 和 P_i 怎么得到？OS 高而 SR 低说明什么？如何防 reference path 泄漏？navmesh 改变会影响哪些指标？R2R 图导航和 VLN-CE 连续动作能否直接比较？

易错点：混淆类别与实例；把 REVERIE grounding 当 ObjectNav success；把 shortest path 当实际轨迹；忽略 stop、timeout、repeat、latency、scene revision 或 denominator；跨 protocol 平均 SR/SPL；把 simulator/offline proxy 写成真机结果。

危险答案：说“所有导航只要 final distance 小就成功”“SPL 高就说明真机可靠”“ObjectNav、VLN、REVERIE 可以用同一 success”“论文 baseline 就是我的实验”或“OPD/WAM 已证明导航闭环提升”。

来源：[1][2][3][4][5]　练习状态：unseen

参考来源

1. PER-004 VLN / ObjectNav Knowledge：任务 taxonomy、episode/observation/action/stop、success/SPL、失败树与项目边界。
2. PER-004 VLN / ObjectNav Source Freeze：Habitat-Lab/Habitat-Sim/VLN-CE pinned source、R2R、REVERIE、ObjectNav Revisited 与 SPL 入口。
3. Habitat-Lab navigation measures 与 ObjectNav task。
4. VLN-CE frozen README。
5. R2R、REVERIE、ObjectNav Revisited、SPL。

Part 25 / 43 · `interview_materials/vla/questions/vol_06_per005.html` · SHA-256
`C5098EC9CA0F017314F685FEF677CEF0D71C80C7278615BD2C7495EE4301E3A2`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`c5098ec9ca0f017314f685fef677cef0d71c80c7278615bd2c7495ee4301e3a2`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per005.md`

VOL 06 Perception / Grounding / Navigation：PER-005 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`VLA-PER-005` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-005 Navigation Policy Architecture Knowledge；来源边界见 PER-005 Source Freeze。

导航 policy 的记忆、子目标与执行边界

▼ `L3` `P0` **VLA-PER-005 · map-free、map-based、waypoint policy 如何比较？planner/controller 接口和 latency 要冻结哪些字段？**

面试意图：判断你能否把 policy family、显式记忆、中间目标、planner/controller、stop/replan 和时延放进同一可审计闭环，而不是只说端到端或模块化。

30 秒回答：先按接口比较：map-free 直接出 action；map-based 维护语义地图并选 subgoal；waypoint 预测中间目标交给 controller。取舍在探索、partial observability、延迟和可解释性；都要冻结 frame/unit、uncertainty、age、stop/replan。公开结果不能写成个人导航。

机制展开（约 2 分钟）：接口分三层：map-free(history→action)、map-based(map→subgoal)、waypoint(subgoal→controller)。subgoal 冻结 frame/unit、timestamp/age、uncertainty、valid、stop/replan。延迟记各阶段及 p95/stale-action。比较固定 planner/controller、hz、scene/split、stop/分母，做去 map、去 semantic prior、direct-action ablation；无 map/action trace 和 episode bundle，只讲机制，不称导航结果。

诊断 / 证据：为每个 episode 保存 observation/action schema、map revision、pose/semantic update、subgoal frame/unit/age/uncertainty、planner status、controller command、stop/replan reason 和 latency breakdown。做 map-free↔map-based、direct-action↔waypoint、去 semantic prior 的 matched ablation，并固定 scene、split、seed、controller、control-hz、action repeat 和 denominator。用 stale-action、tracking error、collision、replan 次数和 terminal reason 分开归因；若只有论文表格、接口草图或单次 rollout，证据停在机制层。

项目挂钩：OPD 可连接 point/box/text grounding 作为目标候选前端；Robot Harness 可连接 semantic-navigation 高层接口、timeout/cancel/compensation；OpenPiVLA 可连接 observation/action packaging、chunk 与 latency；WAM/UWM 可连接 rollout provenance 和 simulator/evaluation ladder。安全口径是“我能设计并审计这些接口”，不声称五个项目已有 map-based、waypoint、VLN/ObjectNav 或真机导航结果。

追问树：map-free 没有显式地图时记忆在哪里？semantic prior 如何避免伪装成已观测目标？waypoint 的 frame/unit 错了会怎样？为什么 action age 比平均 FPS 更关键？controller 固定后如何归因 policy uplift？stop 与 replan 谁负责？没有 episode bundle 时你能声称什么？

易错点：把 waypoint prediction 当 closed-loop success；把 map revision、pose、unit 或 timestamp 丢掉；只报平均 latency；改变 planner/controller 却归因 policy；把论文 baseline、远端路径或高层工具接口写成个人导航结果。

危险答案：说“map-based 一定优于端到端”“waypoint 就等于 controller 已执行”“平均 FPS 足够说明实时”“只要 final distance 小就成功”或“OpenPiVLA/WAM/UWM 已证明我的导航闭环”。

来源：[1][2][3][4][5]　练习状态：unseen

参考来源

1. PER-005 Navigation Policy Architecture Knowledge：policy taxonomy、semantic exploration、waypoint contract、planner/controller、latency 与 evidence ladder。
2. PER-005 Navigation Policy Architecture Source Freeze：GOSE、VLN-CE waypoint、Habitat-Lab/SIM 与项目边界。
3. GOSE、ObjectNav Revisited。
4. VLN-CE、Waypoint Models。
5. Habitat-Lab navigation measures 与 Habitat-Sim PathFinder。

Part 26 / 43 · `interview_materials/vla/questions/vol_06_per006.html` · SHA-256
`54D2B95C1048E0BD102C7CA3CF5860A571B7A8B9208935820E3DA90E3104EA22`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`54d2b95c1048e0bd102c7ca3cf5860a571b7a8b9208935820e3da90e3104ea22`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per006.md`

VOL 06 Perception / Grounding / Navigation：PER-006 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-19 覆盖范围：`VLA-PER-006` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-006 Image Preprocess / Token / Camera Knowledge；来源边界见 PER-006 Source Freeze。

图像预处理、视觉 token 与 camera identity 合同

▼ `L3` `P0` **VLA-PER-006 · 图像进入 Qwen3-VL 前，如何审计 preprocess、patch token、坐标逆变换与 camera identity?**

面试意图：判断你能否把“调用 AutoProcessor”展开为可复算、可逆、可绑定相机语义的输入合同，并把静态代码风险与实测性能因果分开。

30 秒回答：先分原图、处理图、patch grid、image token四层，冻结resize、数值范围、归一化与坐标逆变换。Qwen3-VL的factor为16×2=32，token数由grid/merge决定；相机顺序另存manifest。OPD的factor28→32只是静态风险，无matched A/B不称性能根因。

机制展开（约 2 分钟）：逐样本记录原图尺寸、resize/crop/pad、处理尺寸、pixel range、mean/std、`image_grid_thw`、placeholder 顺序与 camera/frame/time。Qwen3 静态图满足 $N_{img} = H'W'/1024$，但 pixel budget 相等不代表固定 shape。point/box 必须按实际 direct resize、crop 或 letterbox 参数逆映射。OPD 先用 utils 默认 factor28、再进 factor32 processor；应先统一 resize ownership、做 grid/token 与坐标 round-trip，再固定 train/eval 跑 matched A/B。

诊断 / 证据：保存 `sample_id/image_id/camera_id/frame_id/timestamp`、原始与处理宽高、transform revision、scale/crop/pad、input range、processor/config hash、`image_grid_thw`、token count、placeholder/image index 和 valid mask。用多宽高比 sentinel 复算 $N_{img} = T_g H_g W_g / m^2$，再做 point/box 原图→处理图→原图 round-trip。只有 shape/token trace 能证明合同差异；要归因指标变化，还需相同数据、checkpoint、seed、budget、train/eval policy 的单变量 A/B。

项目挂钩：OPD 可安全讲“冻结版本后定位 factor28→32 双阶段 resize 风险，并设计 manifest、round-trip 与 matched A/B gate”，不能说已修复或带来 uplift。OpenPiVLA 要分 formal MindCube 原始 PIL、formal DROID action 的 `images=[]`、dry-run square-224 三条 lineage；不能用 dry-run 或 MindCube 为 image-conditioned DROID 补证。WAM/UWM/Harness 只连接 camera/schema 设计，不声称共享 Qwen processor 或已验证视觉导航。

追问树：为什么 `min_pixels=max_pixels` 仍不是固定分辨率？ `image_grid_thw` 如何变成 placeholder 数？direct resize、crop、letterbox 的 inverse 各缺什么参数？多图顺序 shape 正确为何仍可能语义错位？pixel range 重复 rescale 会怎样？如何设计 double-resize 的 matched A/B？

易错点：把 pixel budget 当固定 shape；把 patch 数直接当 merge 后 token 数；默认 Qwen path 是 letterbox；只保存 `pixel_values` 不保存 inverse/camera manifest；把多图 list 顺序当 camera identity；把静态 mismatch 直接归因指标下降。

危险答案：说“AutoProcessor 会自动处理所有坐标和相机语义”“Qwen3 永远固定 224×224”“patch16 就是一行16个 token”“两次 resize 一定导致效果下降”或“OpenPiVLA 的 DROID 已由图像条件训练”。

来源：[1][2][3][4][5]　**练习状态**：unseen

参考来源

1. PER-006 Image Preprocess / Token / Camera Knowledge：四层表示、smart resize、patch/grid/token、坐标 inverse、camera manifest 与项目 truth boundary。
2. PER-006 Image Preprocess / Token / Camera Source Freeze：冻结 Qwen3-VL、Transformers、qwen-vl-utils 与本地 OPD/OpenPiVLA 调用链。
3. Qwen3-VL frozen README 与 qwen-vl-utils vision_process.py。
4. Transformers 4.57.1 Qwen3VLProcessor 与 Qwen3-VL modeling。
5. Qwen3-VL-8B preprocessor artifact。

Part 27 / 43 · `interview_materials/vla/questions/vol_06_per007.html` · SHA-256
`BF0A4EC1A54ADB14B010C61E9C39BB66C1F0195FFDBDEAFF7F2D71C185221634`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`bf0a4ec1a54adb14b010c61e9c39bb66c1f0195ffdbdeaff7f2d71c185221634`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per007.md`

VOL 06 Perception / Grounding / Navigation：PER-007 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-19 覆盖范围：`VLA-PER-007` 难度：L2 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-007 Open-vocabulary Grounding Pipeline Knowledge；来源边界见 PER-007 Source Freeze。

Open-vocabulary grounding、双阈值与生成式边界

▼ `L2` `P0` **VLA-PER-007 · Grounding DINO 式 open-vocabulary grounding 如何从文本得到带 phrase 的框？box/text threshold 与 NMS 分别做什么？**

面试意图：判断你能否把“文本找框”展开为 prompt/token、query–token matching、双阈值、phrase、geometry 与 optional NMS 的可复算 pipeline，并区分显式 detector 与本地生成式 grounding。

30 秒回答：Grounding DINO输出query对文本token的`Q×L` logits和normalized box。先按query最大sigmoid分数过box threshold，再按text threshold选token重建phrase；坐标逆变换后才可选NMS。OPD是生成式坐标路径，不能借用显式detector的score语义。

机制展开（约 2 分钟）：feature enhancer→query selection→cross-modal decoder输出`[B,Q,L_max]` logits和`[B,Q,4]` normalized `cxcywh`。$s_{q,t} = z_q^\top e_t$经sigmoid；$\max_t p_{q,t} > \tau_{box}$保留query，$p_{q,t} > \tau_{text}$选token解码phrase，均用严格大于。`L_max=256`是token位置。box转processed `xyxy`后按PER-006 inverse回原图。官方helper无NMS；下游若添加，需冻结IoU、phrase grouping和pre/post indices。

诊断 / 证据：先保存 raw/normalized prompt、tokenizer 与 `input_ids`，再保存 raw `Q×L` logits、padding mask、query index、normalized box、两个 threshold、phrase token indices/text、processed/raw `xyxy`、transform revision 与 NMS trace。按 prompt/token→matching→query gate→phrase gate→geometry→NMS 顺序定位；threshold sweep 固定模型、processor、prompt、图像变换、split 与 seed，缓存 raw output 后重放。静态 probe 只证明严格 `>` 和两道门的逻辑，不证明 calibration、precision/recall 或机器人成功。

项目挂钩：本地 OPD 证据支持 annotation→tagged `<box>/<point>/text` SFT→Qwen3-VL `generate`→parser/scorer 的生成式 grounding，可讲 prompt/schema/parser、坐标与 evidence manifest；当前静态搜索没有显式 GroundingDINO/NMS 路径。安全表述是“能审计并对比两类 pipeline”，不能说项目部署了 Grounding DINO、route metadata 是 learned detector/router、静态 IoU 等于闭环成功，或用 Git/本次审计自动证明个人贡献。

追问树：`Q` 和 `L` 两个轴是什么？为什么 256 不是类别数？score 等于 threshold 时保留吗？提高 `t_text` 为什么可能让 phrase 变空却不删 box？ `text_labels` 参数一定决定输出 label 吗？官方 helper 没有 NMS 意味着什么？phrase-aware NMS 如何分组？怎样做 matched threshold sweep？OPD 与显式 detector 的 raw artifact 有什么不同？

易错点：把 query 当 class ID；把 max-token sigmoid 当跨 prompt 已校准概率；合并 box/text threshold；把 box threshold 叫 NMS threshold；只存 decoded phrase 不存 token indices；传错 `(height,width)` 或丢失 preprocess inverse；把 frozen helper 无 NMS 说成不会有重复框；把 OPD 的 box 文本输出说成显式 detector。

危险答案：说“Grounding DINO 最多支持 256 类”“每个 query 只匹配一个 class”“text threshold 只改置信度”“官方默认已经做 NMS”“HF 的 `text_labels` 直接决定 phrase”，或“OPD 能输出 box，所以项目使用了 Grounding DINO并证明机器人成功”。

来源：[1][2][3][4][5][6]　**练习状态**：unseen

参考来源

1. PER-007 Open-vocabulary Grounding Pipeline Knowledge：三阶段 pipeline、query–token math、双阈值、phrase、geometry/NMS、manifest 与项目边界。
2. PER-007 Open-vocabulary Grounding Pipeline Source Freeze：Grounding DINO v5、官方实现、Transformers 4.57.1 与本地 OPD 证据冻结。
3. Grounding DINO v5。
4. IDEA-Research/GroundingDINO@856dde20。
5. Transformers 4.57.1 Grounding DINO。
6. OPD Claim–Evidence Inventory。

Part 28 / 43 · `interview_materials/vla/questions/vol_06_per008.html` · SHA-256
`70B489D16EA77D1CAF6DE438194A60328E0348C2627E7E2BF91333243A8CCF56`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`70b489d16ea77d1caf6de438194a60328e0348c2627e7e2bf91333243a8ccf56`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per008.md`

VOL 06 Perception / Grounding / Navigation：PER-008 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-008` 难度：L2 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-008 Representation Choice Knowledge；来源边界见 PER-008 Source Freeze。

Point / Box / Mask / Keypoint / Affordance 的最小充分选择

▼ `L2` `P0` **VLA-PER-008 · 机器人视觉什么时候该输出 point、box、mask、semantic keypoint 或 affordance？为什么更密不一定更好，OPD 的 affordance point 能说到哪一步？**

面试意图：判断你能否从downstream consumer反推最小充分表示，区分几何密度、semantic role与action condition，并守住本地OPD项目证据边界。

30 秒回答：我按下游最小充分字段选：定位用point，ROI用box，轮廓/占用用mask，类别级几何角色用带ID与frame的keypoints；绑定动作原语、工具、状态、方向和分数语义才叫affordance。mask→box有损，坐标不能还原语义。OPD仅证明带affordance语义的point监督，不证明dense head或机器人成功。

机制展开（约 2 分钟）：先看consumer需要范围、轮廓、语义锚点还是动作条件。point存坐标/mode；box存format/instance；mask存shape/threshold；keypoint增加semantic ID、visibility和frame；affordance再绑定primitive、tool/state、direction、horizon和score语义。`mask→box` 只留四个极值，`box→mask` 产生新prediction；同一坐标可对应不同role或push/pull。输出继承image/camera/time lineage，保留uncertainty、validity和conversion provenance，仍需PER-015 controller gate。

诊断 / 证据：按consumer required-fields→native schema→image/coordinate/time lineage→raw multimodal candidates→conversion trace→parser golden test→matched metric→controller gate检查。matched A/B固定task、data、backbone budget、transform、consumer与protocol，分别报告native质量、invalid/empty、latency/memory、conversion loss和downstream outcome；不能拿不同论文headline metric比较表示优劣。若score含义不清，先区分objectness、mask quality、point hit、ranking score和校准成功概率。

项目挂钩：本地 `build_prompt_pool.py` 把ShareRobot `affordance.{x,y}` 缩放为单个 `0..1000` point，`point_data_only.py` 最终也生成point answer；训练/评估证据是tagged point/box/text generation与static parser/scorer。因此可讲“affordance-labelled point supervision”，不可讲SAM/keypoint pipeline、dense actionability map、primitive/direction success calibration、controller feasibility、robot success

或个人ownership。若未来补到真实head/schema，也要先归档revision、raw output、matched ablation与episode bundle。

追问树：为什么不用mask统一所有任务？point何时能叫semantic keypoint？affordance point最少还缺哪些字段？SAM mask quality能当抓取成功率吗？mask与box能否互相无损转换？多解point/multimask如何保留？如何设计matched representation ablation？Where2Act score能否跨robot比较？OPD字段名为什么不能证明闭环能力？

易错点：按输出密度排性能；把point/keypoint/affordance当同义词；把box填满称为恢复mask；只存payload不存representation/coordinate/score semantics；过早压掉多模态候选；把SAM mask当语义/动作理解；把offline表示质量升级为controller success；用不同任务与metric横比论文数字。

危险答案：说“mask信息最多所以任何任务都最好”“keypoint就是point”“affordance就是可交互坐标”“SAM quality就是抓取成功率”“box可以无损恢复mask”“Where2Act分数是任意机器人成功概率”，或“OPD有affordance字段，所以我实现了dense affordance和机器人闭环”。

来源：[1][2][3][4][5][6][7]　练习状态：unseen

参考来源

1. PER-008 Representation Choice Knowledge：minimum-sufficient选择、五类typed合同、不可逆转换、matched evaluation与项目边界。
2. PER-008 Representation Choice Source Freeze：RoboPoint、SAM、kPAM、Where2Act与五个本地OPD文件的冻结证据。
3. RoboPoint v1 与 official repository。
4. Segment Anything v1 与 official repository。
5. kPAM v2 与 official repository。
6. Where2Act v2 与 official repository。
7. OPD Claim–Evidence Inventory。

Part 29 / 43 · `interview_materials/vla/questions/vol_06_per009.html` · SHA-256
`F426BB17C26ED349E13B83923B6B547EB0BBE8F6F77C27EA525A68230B0F4FAD`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`f426bb17c26ed349e13b83923b6b547eb0bbe8f6f77c27ea525a68230b0f4fad`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per009.md`

VOL 06 Perception / Grounding / Navigation：PER-009 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-009` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-009 Grounding Matching / Negative / Loss Knowledge；来源边界见 PER-009 Source Freeze。

Grounding Matching、Negative 与 Loss Reduction

▼ `L3` `P1` **VLA-PER-009 · Grounding模型训练时，Hungarian matching、negative、focal/L1/GIoU和reduction如何配合？为什么本地OPD不能说成detector loss？**

面试意图：判断你能否分开matching cost与final loss，解释token-axis negative、mask和distributed reduction，并把公源detector机制与本地生成式CE/KL objective放在正确证据边界内。

30 秒回答：Hungarian用token、L1、GIoU cost做一对一分配；cost只定query–GT责任，梯度来自分配后的loss。unmatched query仍是分类负例，padding须mask，权重/reduction须绑定版本。本地OPD是assistant CE、top-1 CE和online KL，没有matcher或box head。

机制展开（约 2 分钟）：logits为`[B,Q,L]`，类别轴是text token。matcher在`Q×M` cost上分配；matched query写phrase map并回归box，unmatched query对有效token全为0，padding由`text_mask`排除。分类loss对valid query-token求和后除跨worker平均GT数；box算L1/GIoU，aux层重匹配。paper正文与Table 8权重冲突，须冻结版本/config；cardinality只logging。本地SFT用`-100`屏蔽prompt/padding；top-1 CE与online KL按token→sample→route→batch聚合，均无query set或box head。

诊断 / 证据：按prediction/target schema与positive map→positive/negative/masked counts→class/L1/GIoU component costs→Hungarian assignment probe→`cxcywh/xyxy`转换→numerator/`num_boxes` denominator→aux/encoder `weight_dict`→world size与empty-target batch检查。若本地loss异常，再查有效response token、route一致性、zero-loss/dummy/retry ledger和teacher/student checkpoint。任何“质量提升”都要继续绑定immutable checkpoint、raw holdout prediction、strict parser与同protocol metric；2500行optimization trace本身不够。

项目挂钩：本地`point_sft.py`只直接证明assistant-token supervision及prompt/padding `-100` mask；`train_opd_top1_vl.py`是object/region teacher top-1 token CE；`train_opd_online_vl.py`是generated-response full-vocabulary KL。三条路径均未读到detector query set、Hungarian matcher、unmatched-query negatives或L1/GIoU head。归档CSV的2500行与summary可复算，只能讲loss/overlap/grad的optimization trend，不能讲checkpoint-bound holdout、teacher正确性、robot uplift或个人ownership。

追问树：matching cost为什么不是training loss？Hungarian assignment能否直接反向传播？unmatched query有没有梯度？padding和negative有什么区别？focal loss等于hard-negative mining吗？为何classification numerator除GT box数？aux层是否复用final matching？`cardinality_error`会不会优化？Grounding DINO权重究竟是哪组？top-1 CE与full-vocab KL怎样区分？loss下降如何补成可信grounding提升？

易错点：把cost matrix当total loss；把unmatched query、非目标token、padding和negative image混为一谈；认为focal自动完成hard-negative mining；默认所有loss按batch mean；看到loss dict key就认为它反向传播；忽略prompt长度、query数和world size对scale的影响；不绑定revision就背固定权重；把生成式coordinate supervision称为set-prediction detector。

危险答案：说“Hungarian cost就是最终loss”“padding也是negative”“未匹配query完全没有梯度”“用了focal就是做了hard-negative mining”“Grounding DINO永远固定使用某一组权重”“OPD有point/box数据所以用了Grounding DINO loss”，或“2500步loss下降证明holdout与机器人成功率提升”。

来源：[1][2][3][4][5][6][7][8]　练习状态：unseen

参考来源

1. PER-009 Grounding Matching / Negative / Loss Knowledge：tensor schema、matching/final loss、negative、reduction、本地objective lineage与面试边界。
2. PER-009 Source Freeze：primary/official/local revision、hash、权重冲突、metrics复算与deterministic probes。
3. Grounding DINO v5。
4. DETR v3。
5. Focal Loss v2。
6. Generalized IoU v2。
7. Transformers 4.57.1 Grounding DINO loss 与 object-detection loss。
8. OPD Claim–Evidence Inventory。

Part 30 / 43 · `interview_materials/vla/questions/vol_06_per010.html` · SHA-256
`A6A780CBC7545B957CEA6B06BFD6FFB67073E850E111C1B395DB71F587E997C4`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`a6a780cbc7545b957cea6b06bfd6ffb67073e850e111c1b395db71f587e997c4`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per010.md`

VOL 06 Perception / Grounding / Navigation：PER-010 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-010` 难度：L2 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-010 Grounding Metric / Parser / Protocol Knowledge；来源边界见 PER-010 Source Freeze。

Grounding Metric、Strict Parser 与 Protocol Comparability

▼ `L2` `P0` **VLA-PER-010 · Grounding评估时，strict parser、IoU/PCK/Recall和protocol comparability该怎么讲？为什么本地OPD的Hit@50/100不能直接叫PCK?**

面试意图：判断你能否把parser、validity、matching、metric、denominator和aggregation连成可复现合同，识别同名指标的口径漂移，并守住本地OPD静态评估的项目证据边界。

30 秒回答：我把parser纳入metric：拆格式/语法/坐标/几何，strict主评、lenient仅诊断；再按表示选IoU、R@k、PCK或mask-hit，固定invalid与分母。跨实验还要对齐数据、checkpoint、prompt、scorer和聚合。OPD Hit@50/100只是在规范坐标中找最近点对，没有PCK的尺度归一化、可见性和点对应，不能混称。

机制展开（约 2 分钟）：输出先按预期表示完整匹配，分开记录格式、语法、坐标、几何；宽松恢复另栏。IoU是单框连续重叠，Acc@τ是阈值样本率；RefExp R@k还绑定top-k排序、重叠函数和GT单元，COCO AP/AR另含阈值网格、匹配、面积和maxDets。PCK需点对应、尺度、阈值、可见性；mask-hit以预测点为分母。本地Hit只取任意预测—GT最小距离，一个近邻即可通过。若invalid的布尔项记false而数值项跳过，会出现Acc=.5但IoU均值=1，因此每字段须报分子/分母。最后把数据、checkpoint、prompt/decode、transform、parser/scorer、matching、threshold和aggregation写入protocol ID。

诊断 / 证据：依次查expected-format与raw gold→strict/lenient golden cases→transform与坐标范围→pairwise box/point matrix→overlap/matching函数和threshold→invalid/empty/duplicate policy→每字段numerator/denominator→micro/macro与slice→两份protocol manifest diff→checkpoint/raw prediction绑定。确定性probe应覆盖identical/disjoint/touching IoU、tagged/bare/prose/圆括号point、任意点对Hit与双向mean、invalid+perfect box分母，以及MDETR IoU=.525/GIoU=.483的0.5判定翻转。

项目挂钩：本地`eval_qwen_vl_raw_holdout.py`会全局抓`[x,y]`，box还可从tag内数字或前两个point pairs恢复；format gate与geometry score分离，`mixed_grounding`却落入text，bool rate除全pool而numeric mean跳`None`。inventory记录10k静态holdout与限定pool metrics，但raw predictions、exact

checkpoint hashes和parser-v2 paired rerun未归档。因此只能讲original-strict / protocol-fixed-strict / lenient-diagnostic三口径方案，不能把Hit叫标准PCK、把静态分数升级为robot success、单因素因果或个人ownership。

追问树：格式错但坐标对算不算正确？Mean IoU和Acc@0.5有什么区别？Recall@1一定用IoU吗？MDETR论文与代码为什么会翻转判定？COCO AR与RefExp R@k能否比较？PCK最少要冻结哪些字段？mask-hit偏precision还是recall？多点min distance为何乐观？invalid该drop还是记false？同raw output换parser怎样公平比较？static grounding高能否说明robot success？

易错点：把regex抽取当strict parse；用一个 `valid` 吞掉四层状态；混用IoU、Acc、R@k、AP/AR；把fixed Hit叫PCK；忽略point correspondence与normalizer；只报numeric mean不报valid count；把invalid静默drop；只凭指标名横比；整体平均掩盖box/point/text slice；把parser修复、checkpoint和prompt同时变化说成单因素提升。

危险答案：说“能抓到坐标就算格式正确”“IoU、Recall@1和AP都是定位准确率，直接比较即可”“Hit@50就是PCK@50”“invalid删掉不影响结果”“MDETR官方固定用某个overlap，不必绑定revision”“OPD point分数高证明机器人操作成功”，或“我实现了严格scorer”，却不提mixed→text、lenient fallback和字段分母差异。

来源：[1][2][3][4][5][6][7]　练习状态：unseen

参考来源

1. PER-010 Grounding Metric / Parser / Protocol Knowledge：四层validity、metric family、protocol identity、伪代码、failure taxonomy与项目边界。
2. PER-010 Source Freeze：MDETR、COCO API、RoboPoint、MMPose、本地evaluator/inventory与deterministic probes。
3. MDETR v3 与 fixed RefExp evaluator。
4. COCO API cocoeval.py。
5. RoboPoint v1。
6. MMPose keypoint_eval.py。
7. OPD Claim–Evidence Inventory。

Part 31 / 43 · `interview_materials/vla/questions/vol_06_per011.html` · SHA-256
`1E2507CF9E58AE0FD19C565ED52BDC8158E1FD40069EF2208EBB8867487F0403`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`1e2507cf9e58ae0fd19c565ed52bdc8158e1fd40069ef2208ebb8867487f0403`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per011.md`

VOL 06 Perception / Grounding / Navigation：PER-011 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-011` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Knowledge；来源边界见 PER-011 Source Freeze。

Pseudo-label、Teacher Noise、Calibration、Curriculum 与 Leakage

▼ `L3` `P0` **VLA-PER-011 · 用teacher做pseudo-label时，confidence threshold、calibration、selection bias、curriculum和leakage怎么讲？本地OPD现在到底证明了什么？**

面试意图：判断你能否把teacher target当作带血缘的数据artifact，区分confidence、correctness与calibration，解释threshold后的risk/coverage和分布偏移，并守住本地OPD teacher-forced cache、静态holdout与robot/ownership之间的真实性边界。

30 秒回答：我把伪标签绑定样本指纹、teacher版本、推理协议、target类型、置信度/校准、接收策略和轮次。置信度不等于正确率，阈值须同时报coverage、独立gold上的accepted error及route分布；校准温度只能在独立validation拟合。泄漏还要查内容、语义、teacher训练和模型选择。OPD cache无置信度，不能声称已校准筛选。

机制展开（约 2 分钟）：hard取argmax，soft保留分布；teacher-forced在真实response位置读logits，不等于自由生成。FixMatch以max-prob≥τ成mask且loss除总batch，故阈值须配risk–coverage、gold error和route drift。Temperature scaling在独立标注集拟合T，采样温度与entropy日志不是校准。Noisy Student每轮冻结teacher、候选池、filter/balance、student及晋升协议；curriculum还需排序、schedule和固定/随机对照。泄漏从row/path扩展到内容近重复、语义、teacher预训练及test调参，未知身份应fail closed。

诊断 / 证据：先diff candidate/teacher/prompt/decode/calibration/round manifest→核验teacher-forced或free-generation与raw output→strict validity→threshold前后score、coverage、accepted/rejected count→gold-audit error与reliability bins→class/route/source/domain drift→cache/checkpoint identity→row/source/path/content/perceptual/semantic overlap→teacher training lineage与sealed model-selection log。确定性probe应覆盖0.96/0.94在τ=.95的mask与总batch reduction、0.99错误仍被接收、ECE换bin变化、同内容不同path穿透split、无confidence cache不可重算coverage、改变veto-beta只改target而不证明correctness。

项目挂钩：`build_opd_mix_v1.py`只按source/bucket做35/35/30 mix和obj/reg/both route，不是confidence filter；`prepare_opd_top1_cache.py`在真实assistant supervised positions取teacher argmax，只存top-1 IDs/count；`train_opd_top1_vl.py`直接消费ID，both固定0.5/0.5；online脚本记录entropy/top-k overlap并做veto-beta target mixing，但没有gating或validation-fitted calibration。10k summary报告8,251 unique images、10k unique fingerprints和train-image blocklist，是可见image/fingerprint gate；仍缺near-duplicate/semantic、teacher lineage、sealed selection、exact raw/checkpoint与robot证据。

追问树：为什么0.95不代表95%正确？FixMatch masked loss除什么？accepted error没有gold能算吗？sampling temperature和temperature scaling有何区别？ECE为何绑定binning与sample unit？threshold升高为何伤long-tail？class balance为何仍可能错？Noisy Student每轮冻结哪些identity？什么schedule才叫curriculum？path overlap=0为何仍可能泄漏？teacher预训练污染如何查？top-1 cache为何不是自由生成pseudo dataset？entropy下降能否证明更准？static holdout能否推出robot success？

易错点：把confidence当correctness；只报threshold不报coverage/risk；accepted-only mean与总batch mean混用；用train或test拟合calibration；把generation temperature叫校准；只平衡predicted class；旧cache与新teacher混用；把线性超参称为curriculum；把path hash当content/semantic dedup；把静态holdout、代码存在或Git身份升级为因果、robot结果或个人ownership。

危险答案：说“0.95以上就是高质量伪标签”“entropy下降说明teacher更准”“调temperature就是做校准”“path不重叠所以绝无泄漏”“top-1 cache是teacher自由生成标签”“veto-beta schedule证明curriculum有效”“10k fingerprint唯一说明teacher没见过测试集”，或“我设计并验证了完整OPD伪标签系统”，却不提confidence字段、teacher lineage、gold audit、sealed selection和候选人逐文件确认缺失。

来源：[1][2][3][4][5][6][7]　练习状态：unseen

参考来源

1. PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Knowledge：typed record、risk–coverage、calibration、selection/curriculum、leakage taxonomy、诊断与项目边界。
2. PER-011 Source Freeze：固定论文/代码revision、SHA、本地OPD映射与deterministic probes。
3. FixMatch v2 与 official implementation。
4. Noisy Student v4 与 official implementation。
5. Guo et al., On Calibration of Modern Neural Networks v2。
6. Bengio et al., Curriculum Learning；Kapoor & Narayanan, Leakage and the Reproducibility Crisis in ML-based Science v1。
7. OPD Claim–Evidence Inventory。

Part 32 / 43 · `interview_materials/vla/questions/vol_06_per012.html` · SHA-256
`82305A7C18397E0CCEE925207AC122B4B7EAA182E6C4A02188746278563DB2BC`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`82305a7c18397e0ccee925207ac122b4b7eaa182e6c4a02188746278563db2bc`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per012.md`

VOL 06 Perception / Grounding / Navigation：PER-012 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-012` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-012 Segmentation / Depth / Keypoint / Affordance Knowledge；来源边界见 PER-012 Source Freeze。

Segmentation、Depth、Semantic Keypoint 与 Manipulation Affordance

▼ `L3` `P1` **VLA-PER-012 · 如何把segmentation、depth、keypoint与affordance串成机器人可用的操作目标？为什么单模块高分不等于robot success？**

面试意图：判断你能否先冻结segmentation/depth/keypoint/affordance的task、unit、frame与score semantics，再把2D输出组合成3D proposal并经过planner/controller gate；同时识别public method、本地point supervision和真实机器人闭环之间的证据边界。

30 秒回答：先冻结task/unit：四类mask不等价，relative depth不能当米。mask与metric depth对齐原图、内参和时间才能lift；keypoint需语义ID、frame与不确定性，affordance绑定点、方向、primitive和state，最后过IK、碰撞与控制门。OPD证明affordance-labelled point监督。

机制展开（约 2 分钟）：Mask2Former的class+mask需按semantic/instance/panoptic分别后处理；SAM的prompt、多候选和quality没有类别、跨帧身份或robot校准。Depth Anything V2 relative路径是ReLU图，metric路径是独立checkpoint与Sigmoid×max_depth，resize只恢复shape。只有米制depth与mask、原像素、内参、配准和时间一致才能lift。kPAM keypoint带语义ID、visibility、covariance与frame；Where2Act proposal绑定点、方向、primitive和state，最终过reachability、IK、collision与controller。

诊断 / 证据：按first-divergence检查task/schema→prompt/candidate policy→model/checkpoint/license→pre/postprocess→depth relative/metric、unit/range/validity→RGB-depth registration、intrinsics、TF/time→semantic keypoint identity/visibility/covariance→affordance primitive/state/orientation→planner rejection与episode trace。确定性probe应覆盖two-mug semantic/instance差异、同click的SAM multimask、relative scale×2、metric `max_depth=20/80`、resize后仅shape恢复、同坐标不同keypoint语义、同点push/pull score变化，以及正确mask+错误depth导致3D整体漂移。

项目挂钩：本地OPD的ShareRobot `affordance{x,y}` 被缩放为0..1000并写成 `<point>[[x,y]]</point>`；prompt虽写actionable location，训练与评估仍走point/box/text target、parser和static metric。选定五个文件没有显式SAM/mask polygon/depth/keypoint实现命中，这只支持“affordance-labelled point

supervision”，不排除远端或未归档路径，也不证明dense affordance、metric 3D、planner/controller、robot result或个人ownership。WAM、OpenPiVLA、UWM与Harness只能作为待补artifact的接口对照。

追问树：semantic、instance、panoptic、promptable mask如何区分？Mask2Former为何统一架构却不能统一metric？SAM quality代表什么、不代表什么？relative depth为何不能直接反投影？metric `max_depth` 怎样绑定checkpoint？resize回原图为何不等于camera registration？mask边界为何产生飞点？semantic keypoint比普通point多哪些字段？kPAM怎样把keypoint转成cost/constraint？Where2Act的ActionScore、Actor、Critic分别做什么？affordance为何绑定primitive与state？proposal高分为何还要IK/collision/controller？OPD point target和Where2Act有何本质差别？

易错点：把任意mask当category+instance；把SAM quality当robot-success confidence；把relative depth或手乘常数叫米制；把output resize当intrinsics/registration恢复；忽略invalid depth、边界飞点、frame和staleness；把无名point叫semantic keypoint；把actionable文本叫primitive-conditioned affordance；用2D IoU替代3D/episode metric；把公开模型、代码零匹配或Git身份升级为本地集成、效果或个人贡献。

危险答案：说“有mask就知道抓哪个”“Depth Anything输出就是米”“resize回原图就对齐相机”“一个point就是keypoint”“SAM/affordance分数高就直接执行”“OPD已经实现SAM+depth+kPAM+Where2Act完整栈”，却不交代task、checkpoint、unit、camera/frame/time、primitive、planner/controller、raw artifact与ownership缺口。

来源：[1][2][3][4][5][6][7][8]　练习状态：unseen

参考来源

1. PER-012 Segmentation / Depth / Keypoint / Affordance Knowledge：typed composition、方法合同、诊断顺序、项目边界与面试模板。
2. PER-012 Source Freeze：固定paper/code revision、SHA、license、本地OPD audit与deterministic probes。
3. Cheng et al., Mask2Former v3；official repository。
4. Kirillov et al., Segment Anything v1；official repository。
5. Yang et al., Depth Anything V2 v2；official repository。
6. Gao & Tedrake, kPAM v2；official repository。
7. Mo et al., Where2Act v2；official repository。
8. OPD Claim–Evidence Inventory。

Part 33 / 43 · `interview_materials/vla/questions/vol_06_per013.html` · SHA-256 `A8D294736A6CC433E8DA2ACA0680A7273455D19BB9AAD0368D7D24635966D8D1`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`a8d294736a6cc433e8da2aca0680a7273455d19bb9aad0368d7d24635966d8d1`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per013.md`

VOL 06 Perception / Grounding / Navigation：PER-013 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-013` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-013 Point Cloud / Voxel / BEV / Occupancy Knowledge；来源边界见 PER-013 Source Freeze。

Point Cloud、Voxel、BEV、Occupancy 与 Sampling

▼ `L3` `P1` **VLA-PER-013 · 点云、voxel、BEV和occupancy怎样选？sampling、量化与unknown语义如何影响下游？**

面试意图：判断你能否从downstream query反推3D representation，说明PointNet++ sampling/grouping、voxel quantization/VFE、sparse convolution、pillar/BEV和三类occupancy的机制与损失；同时冻结frame/time/config、metric与本地项目证据边界。

30 秒回答：按下游选表示，冻结frame/range/resolution/time。PointNet++以FPS、ball/kNN建层次；voxel量化有边界、碰撞、截断和聚合alias。Sparse未必更快；BEV需说明z与校准。occupancy分grid、octree和implicit；unknown不等于free，更不等于碰撞安全或robot success。

机制展开（约 2 分钟）：点集函数可排列不变，但FPS起点、radius/K和voxel cap仍属模型身份。q=floor((x-o)/v)由frame、origin、range、axis定义；MeanVFE使[0,2]与[1,1]碰撞。Sparse tensor含坐标/特征，SubM、stride sparse和inverse conv的active pattern不同，索引也有成本。Pillar折叠z bin但可保留z feature；HeightCompression把D放入channel。BEV需标定、pose和time。grid、OctoMap hit/miss与implicit threshold不可混用；unknown、free、occupied分开，再进grounding与controller。

诊断 / 证据：按first-divergence检查source/frame/unit/time→sampling seed/start/budget与density→origin/range/axis/voxel size/floor→collision、max-points/max-voxels与aggregation→layerwise active pattern、latency和peak memory→pillar z feature或height channel→BEV calibration/augmentation/time→occupancy observed/unknown/free/occupied、threshold与staleness→semantic grounding→planner rejection与episode trace。确定性probe至少覆盖FPS `[0,1,2,10]`、voxel 0.99/1.01 boundary、MeanVFE `[0,2]`/`[1,1]` alias、`200^3×64×float32` memory、pillar z collapse和implicit threshold 0.5/0.7。

项目挂钩：本地Robot Harness有2D occupancy PNG/JSON，编码0 occupied、127 unknown、255 free；固定`world.pcd`有54,856点和`x y z intensity timestamp ring`字段，bridge只暴露occupancy/pointcloud path、existence flag与URL。它们支持地图资产、PCD schema和接口事实，不支

持候选人生成地图、在线读取PCD、实现PointNet++/voxel/sparse/BEV/OctoMap/learned occupancy、获得representation uplift或真实机器人结果。86个高信号tracked文本中78个位于`third_party/`；vendored code与Git身份也不能替代逐文件ownership确认。

追问树：PointNet为何permutation-invariant而pipeline仍可能依赖顺序？FPS起点、outlier与复杂度怎样影响结果？ball query和kNN在density shift下分别漂移什么？voxel为何必须绑定origin/range/axis/floor？hard与dynamic voxelization有何差异？MeanVFE和learned VFE为何不等价？SubM、Sparse、InverseConv怎样改变active sites？何时sparse未必更快？pillar是否完全丢z？HeightCompression与pillar有何区别？BEVFusion为何仍需calibration？empty、unknown与observed free如何区分？OctoMap怎样利用ray miss/hit？implicit threshold为何属于surface identity？occupancy IoU为何不等于collision safety？本地Harness究竟支持哪些claim？

易错点：把permutation invariance扩大成整个pipeline无序敏感；说FPS总优于random；只报voxel size不报frame/origin/range/axis；把MeanVFE叫VoxelNet；忽略boundary、cap/drop与aggregation alias；声称sparse必然节省99%显存；说pillar完全删除z；把同shape BEV当已校准；把空cell当free；把implicit occupancy score当collision probability；用coverage、IoU或Chamfer替代semantic、navigation和episode metric；把公共论文、third-party code、PCD资产或Git身份升级为个人3D系统。

危险答案：说“点云直接丢进PointNet就行”“FPS一定最好”“voxel size相同输入就相同”“sparse一定更快更省”“BEV天然对齐相机和LiDAR”“没有点就是free”“occupancy 0.9即可执行”“Harness已经实现完整3D perception”，却不交代sampling、frame/origin/range/resolution、aggregation、active pattern、calibration/time、unknown policy、controller gate、raw artifact与ownership缺口。

来源：[1][2][3][4][5][6][7][8][9][10]　练习状态：unseen

参考来源

1. PER-013 Point Cloud / Voxel / BEV / Occupancy Knowledge：typed manifest、representation机制、诊断顺序、项目边界与面试模板。
2. PER-013 Source Freeze：固定paper/code revision、SHA、license、九个deterministic probes与五项目audit。
3. Qi et al., PointNet++ v1；official repository。
4. Zhou & Tuzel, VoxelNet v1；Lang et al., PointPillars v2。
5. Choy et al., Minkowski ConvNet v4；MinkowskiEngine。
6. Liu et al., BEVFusion v3；official repository。
7. Mescheder et al., Occupancy Networks v2；official repository。
8. OpenPCDet@233f849。
9. OctoMap@f012f5f。
10. Robot Harness Claim–Evidence Inventory 与source freeze中固定的本地map/PCD/bridge artifacts。

Part 34 / 43 · `interview_materials/vla/questions/vol_06_per014.html` · SHA-256
`54CC722A10E5CB2916048AC5B94439D28695A16ED763729A53C6AE3A1B53448D`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`54cc722a10e5cb2916048ac5b94439d28695a16ed763729a53c6ae3a1b53448d`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per014.md`

VOL 06 Perception / Grounding / Navigation：PER-014 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-014` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-014 3D Grounding / Spatial Relation / Open-vocabulary Knowledge；来源边界见 PER-014 Source Freeze。

3D Grounding、Spatial Relation 与 Open-vocabulary 3D Scene

▼ `L3` `P1` **VLA-PER-014 · 3D grounding有哪些输出？candidate、spatial relation、open-vocabulary score和metric该怎样冻结？**

面试意图：判断你能否先区分3D box、candidate ID、point semantics、instance mask、continuous field与semantic map六类输出，再说明proposal hard upper bound、relation frame/arity、prompt/negative/score/metric identity、first-divergence diagnosis与项目真实性边界。

30 秒回答：先定输出：3D框、候选ID、点语义、实例mask、连续场或语义地图；再冻结候选来源/目标覆盖、坐标时间、关系元数与参考系、prompt/negative/归一化、阈值和指标。目标不在候选中时top-1上界为0，点语义不等于实例。语义目标还要过可达性、IK、碰撞与控制器，不能把cosine或IoU叫机器人成功。

机制展开（约 2 分钟）：ScanRefer先做3D proposal，再做语言排序与框IoU；ReferIt3D只在已分割对象中做top-1。OpenScene给点语言特征但无实例；OpenMask3D先有class-agnostic mask，再经视图、SAM crop和CLIP查询。LERF是逐场景多尺度连续场；ConceptFusion依赖RGB-D/内外参/pose融合语义地图。left/right须绑observer/world/object frame，between是三元关系；dot、cosine与正负relative probability不可混。指标按proposal recall、Acc@IoU、top-1、mIoU、AP、场/地图定位分层，输出再交PER-015。

诊断 / 证据：按first-divergence检查scene revision/frame/axis/unit/pose/depth/time→candidate source/count/target-present与proposal oracle→point/mask/instance的merge、split与view coverage→relation target/anchors/arity/reference frame/tolerance→encoder/checkpoint/prompt/tokenizer/negative/L2 normalization→dot/cosine/temperature/threshold/top-k/unknown→与输出匹配的metric与denominator→reachability/IK/collision/controller rejection。确定性probe至少保留unit-cube x错0.5得到IoU=1/3、GT 7不在`[2,4,5]`中top-1上界0、yaw 0/180°使left翻转、同类双chair的semantic/instance差异、多视图`[1,0]+[-1,0]`零范数、positive .55对negative .60得到relative probability 0.4875。

项目挂钩：本地OPD的`spatial_rel_expert`来自Visual Genome 2D relationship，prompt明确写“in the image”；point路径只有x/y与0–1000 `<point>/<box>`或boolean，不含z、meter、3D frame或instance graph。Robot Harness只有0/127/255三态2D occupancy、54,856点且字段为`x y z intensity timestamp ring`的non-semantic PCD，以及暴露path/URL的bridge；没有semantic label、instance ID、language embedding/query service。OpenPiVLA/WAM/UWM只有image/language/action或video/world-model证据。因此只能把这些项目讲成2D supervision、geometry/interface或未来扩展入口，不能说候选人已实现ScanRefer/OpenScene/OpenMask3D/LERF/ConceptFusion、获得3D grounding uplift或robot success；Git身份也不能替代ownership确认。

追问树：ScanRefer和ReferIt3D的candidate assumption有什么根本差异？proposal recall为何是硬上界？`ref_acc`为何不等于`Acc@IoU 0.5`？point mIoU为何不等于instance AP？两个同类chair怎样暴露identity缺口？OpenMask3D的mask proposal、view selection、SAM crop和CLIP各负责什么？多视图mean为何可能降低score？LERF为何是per-scene且需要scale？ConceptFusion为何依赖pose/depth且query成功不能证明SLAM正确？left/right绑定哪个frame？between为何是三元关系？closest为何依赖candidate set？dot、cosine、relative probability有何差异？negative prompt与temperature为何属于协议？scene-level min-max为何不是跨scene calibration？你会怎样拆分geometry、candidate、relation、language与action failure？本地五项目当前最多支持哪一级claim？

易错点：把六类输出统称“3D grounding”；把ReferIt3D说成raw-scene detector；忽略target-present/proposal oracle；把point semantic当instance；把image-left当map-left；把between写成二元；把cosine 0.8叫80%概率；把多视图平均当必然增益；把field heatmap当box/object ID；用semantic mIoU替代instance AP或robot success；用query heatmap反证SLAM；把OPD 2D relation、Harness PCD/occupancy asset、公开代码或Git身份升级为个人3D系统。

危险答案：说“把CLIP特征投到点云就完成3D grounding”“语言能找回proposal漏掉的目标”“left就是x小”“point mIoU高就能区分实例”“多视图平均一定更稳”“cosine就是confidence”“semantic map可直接发动作”“Harness已经有语言可查询3D地图”，却不交代output atom、candidate source/target-present、frame/pose/time、relation arity/anchors、prompt/negative/normalization、threshold/metric、PER-015 gate、raw artifact与ownership缺口。

来源：[1][2][3][4][5][6][7][8][9]　练习状态：unseen

参考来源

1. PER-014 3D Grounding / Spatial Relation / Open-vocabulary Knowledge：六类输出、typed manifest、candidate上界、relation、score、metric、diagnostic、面试回答与项目边界。
2. PER-014 Source Freeze：六篇完整primary PDF、六个official repositories、36个文件SHA、九个deterministic probes与五项目audit。
3. Chen et al., ScanRefer v3；official repository。
4. Achlioptas et al., ReferIt3D, ECCV 2020；official repository。
5. Peng et al., OpenScene v2；official repository。
6. Takmaz et al., OpenMask3D v2；official repository。
7. Kerr et al., LERF v1；official repository。

8. Jatavallabhula et al., ConceptFusion v3；official repository。

9. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 35 / 43 · `interview_materials/vla/questions/vol_06_per015.html` · SHA-256
`3CD57AF4CF1AEE2B31051EE43A16C51E115C6D87EC7B63847A5A880843610BE6`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`3cd57af4cf1aee2b31051ee43a16c51e115c6d87ec7b63847a5a880843610be6`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per015.md`

VOL 06 Perception / Grounding / Navigation：PER-015 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-015` 难度：L3 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Knowledge；来源边界见 PER-015 Source Freeze。

Perception→Action、Reachability、Uncertainty 与 Failure Gate

▼ `L3` `P0` **VLA-PER-015 · VLA从semantic target到controller-ready action要过哪些gate？可达性、不确定性和失败怎样处理？**

面试意图：判断你能否把semantic target转成typed action target，区分workspace、IK、bounds、singularity、state/motion collision、trajectory、controller与uncertainty的责任边界，并在项目追问中坚持“gate通过不等于robot success”。

30 秒回答：先把语义目标补成带坐标系、单位、时间、工具与机器人/场景/控制器版本的动作目标；再查工作空间、IK残差、关节限位、奇异性、状态/路径碰撞、误差膨胀安全间距、速度加速度与控制器字段。任一层未知都拒绝执行，风险超预算就重感知、重规划、求助或停止；全通过也只是控制器候选，不是机器人成功。

机制展开（约 2 分钟）：工作空间只做粗拒绝；IK返回后以FK重算SE(3)位置/姿态残差，再验关节界与Jacobian最小奇异值。MoveIt PlanningScene负责状态/碰撞语义，OMPL和RRT-Connect依赖外部状态与边验证，因此终点无碰撞不等于整条路径安全；场景、ACM、膨胀量和离散分辨率都属结果身份。几何路径还要定时并检查速度、加速度、当前状态、关节顺序/单位/执行状态与动作块过期。不确定性分语义、几何、执行；KnowNo式conformal set只给校准假设下的边际覆盖，非单元素时求助，不能当物理安全证明。

诊断 / 证据：按first divergence检查frame/unit/time/revision与target age→semantic candidate/score→workspace→IK solver return之后的FK residual、q bounds与σmin→同一scene revision下的goal state、每条edge、ACM/padding/resolution与uncertainty-inflated clearance→q/qdot/qddot、timestamps、branch continuity与chunk staleness→controller shape/semantics/order/unit/current-state/status→calibration split、qhat、prediction-set size→execution/contact/task outcome。确定性probe至少保留2R半径1.8超出上界1.7、IK residual 0.407507m超0.01、q1=π/2超`[-1,1]`、伸直位形σmin=0、端点安全但edge穿AABB、clearance膨胀

0.10→0.20使pass翻reject、dt=0.1得到vmax=10/amax=50、risk upper 0.12>α=0.05触发ask、age 240ms>100ms和expected 7D/got 12D拒绝。

项目挂钩：OPD只能提供2D point/box/relation与grounding score上游，不能推出metric 3D target、IK或collision。Robot Harness有tool scope、lock、timeout/cancel、best-effort compensation与map/PCD bridge，但tool sandbox不是机械臂物理安全。OpenPiVLA有action JSON/full-chat CE及official checkpoint diagnostics，formal generation/parser/controller仍缺；WAM只有7D→12D、OSC_POSE与失败trace诊断；UWM只有7D/16-step LIBERO和RoboCasa action-order/controller probe。五项目都没有统一robot/tool/scene manifest、IK/collision/TOTG raw report、calibrated risk–coverage、controller execution trace、matched robot episode或已确认ownership，因此只能讲interface/diagnostic hook，不能说已部署通用gate或获得robot uplift。

追问树：semantic target与action target字段差在哪？为什么frame/unit/time/revision属于结果身份？workspace内为何仍可能IK无解？solver success后为何重算FK residual？位置与姿态tolerance怎样分开？TRAC-IK改善什么又不负责什么？σmin和manipulability怎样用？goal无碰撞为何不够？state validity与motion validity有何区别？OMPL为何不自带robot collision geometry？resolution、ACM、attached object与padding怎样改变结论？uncertainty inflation如何让pass翻reject？几何path为何还不是trajectory？action chunk何时stale？controller shape相同为何仍可能语义错？conformal prediction保证什么、不保证什么？ask、re-perceive、replan、re-time、reject、stop怎样区分？怎样定位first-divergence？五项目分别最多支持哪一级claim？全部gate通过后为什么仍不能报success？

易错点：把3D点直接送IK；把workspace内当pose可达；只看solver return不重算residual；clamp关节后不复验；把TRAC-IK说成带mesh collision；把goal collision-free当整条edge安全；不冻结scene/ACM/resolution；只给target加covariance却不膨胀clearance；把几何path当动态可执行轨迹；silent slice/pad action；把未校准confidence叫逐样本安全概率；把conformal marginal coverage叫硬件认证；把fallback写成默认动作；把server/tool返回或单次sim success当robot success；用公开机制、目录或Git身份替代本地runtime与ownership。

危险答案：说“找到目标后做IK和RRT就能执行”“MoveIt会自动保证安全”“confidence低就慢一点”“7D向量shape一致即可”“controller返回success就是任务成功”“Harness已有physical safety gate”“WAM action trace证明reachability改善”，却不交代typed manifest、residual/bounds/singularity、state/edge collision、robust clearance、timing/controller contract、calibration/abstention、fresh-state/cancel acknowledgement、raw episode与ownership缺口。

来源：[1][2][3][4][5][6][7][8][9]　练习状态：unseen

参考来源

1. PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Knowledge：typed manifest、九层gate、10个probe、first-divergence diagnosis、两档回答与五项目truth boundary。
2. PER-015 Source Freeze：六份完整primary PDF、六个official repositories/35文件、10个deterministic probes与11个local-direct文件审计。
3. Lynch & Park, Modern Robotics；official code@cbd41fd。
4. Kuffner & LaValle, RRT-Connect；OMPL official code@eb3baca。

5. Beeson & Ames, TRAC-IK；official source@57057ce。
6. Ren et al., KnowNo v2；official code@95e3a1d。
7. MoveIt 2 official code@2fb8492。
8. Shridhar et al., CLIPort v1；official code@2be5c47。
9. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 36 / 43 · `interview_materials/vla/questions/vol_06_per016.html` · SHA-256 `768ED52EC5DA08D3BBA1C3825381C5A1653EA30FEDCF07987F74DC927798D4F7`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`768ed52ec5da08d3bba1c3825381c5a1653ea30fedcf07987f74dc927798d4f7`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per016.md`

VOL 06 Perception / Grounding / Navigation：PER-016 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-016` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-016 Localization / Odometry / SLAM / Map-frame / Drift Knowledge；来源边界见 PER-016 Source Freeze。

Localization、Odometry、SLAM、Map Frame 与 Drift

▼ `L3` `P1` **VLA-PER-016 · 机器人怎样定位自己？Odometry、Localization和SLAM如何区分，map/odom、loop closure、drift与ATE/RPE怎样讲?**

面试意图：判断你能否从sensor/clock/calibration身份讲到state/covariance、local/global frame、verified loop与map revision，再用ATE/RPE/alignment解释评估，并在项目追问中坚持“map资产不等于SLAM runtime”。

30 秒回答：Odometry积累相邻运动，连续但会漂；localization在有UUID和版本的已知地图里估位姿；SLAM共同估轨迹与地图。控制用连续`odom→base`，回环或重定位更新跳变的`map→odom`。回环候选须过几何验证、相对位姿和图残差门。ATE看全局轨迹，RPE看局部漂移，并冻结时间关联、SE3/Sim3、单位与丢失段；Sim3会隐藏尺度误差。

机制展开（约 2 分钟）：先固定sensor、时钟、内外参、噪声、estimator/config/checkpoint和map revision。IMU传播pose/velocity/bias与covariance，视觉或scan更新；VINS/Kimera用preintegration与滑窗，OpenVINS是EKF。前端关联、局部估计、后端因子和全局层分责。Loop按候选→几何→relative transform→robust pose graph→新map版本，false loop会毁图；relocalization恢复当前pose，map merge对齐不同session。ATE/RPE要报告alignment、delta和failure denominator，不能用一个分数代表可用性、尺度和地图质量。

诊断 / 证据：按first divergence检查sensor ID/packet order/clock→calibration与frame方向→initialization/scale/gravity→frontend inliers/geometry residual→local state、bias、covariance、conditioning与marginal prior→loop candidate、geometric inliers、relative transform、graph residual与map revision→timestamp association、SE3/Sim3、RPE delta、unit与lost denominator→planner/controller是否消费stale或jumped pose。确定性probe至少保留`map→odom=-2m`与`odom→base=10m`合成8m、SE2 roundtrip误差`6.12e-17`、P0=.01/Q=.04十步得到prior .41且update后.0738、ATE raw 5对齐后0、RPE .1、false-loop residual 2>.5拒绝、scale 1.2经Sim3归零、map delta

-1.4而odom delta .1、非单调时间拒绝和`runtimeReady=false/softwareOnly=true`不通过localization gate。

项目挂钩：Robot Harness有0/127/255三态occupancy、0.02m resolution、54,856点`x y z intensity timestamp ring` PCD、`reloc_map/poses.txt`、pixel/world与state bridge；fallback targeted test为`1 passed`，但明确`runtimeReady=false/runtimeRealApiCalls=false`，没有sensor bag、clock/calibration lineage、estimator config/checkpoint、live TF、pose/covariance、keyframe/pose graph、loop accept/reject或ATE/RPE bundle。OPD只有2D point/box/relation上游；OpenPiVLA只有image/state/action接口；WAM只有video/world/action诊断；UWM用sim ground-truth state。五项目都不能升级为候选人已实现SLAM、定位提升或真实robot result，Git身份、ROS目录与map资产也不能替代runtime和ownership。

追问树：tracking、odometry、localization、mapping与SLAM各估什么？有`x,y,yaw`为何不等于定位有效？`map→odom→base`各由谁维护？为什么map可跳而odom可漂？controller直接吃map pose有什么风险？IMU传播为何增大covariance？preintegration依赖哪些clock/bias/extrinsic合同？前端点多为何仍可能错？cost下降为何不证明frame/time正确？filter与sliding window怎样公平比较？gauge和monocular scale ambiguity是什么？keyframe/window/marginalization怎样取舍？loop candidate到graph edge有哪些门？false loop为何比不闭环更坏？relocalization、loop closure与map merge怎样区分？pose graph中的`Z_ij/Ω_ij`是什么？DROID learned为何仍有graph与BA？ATE与RPE分别回答什么？SE3与Sim3隐藏什么？lost/relocal denominator怎样定？map quality与trajectory quality为何分开？Robot Harness的PCD、reloc poses与green fallback test最多支持什么claim？

易错点：把tracking/odometry/localization/SLAM混成pose输出；把连续当准确、global当平滑；让controller消费global correction jump；说IMU自动消除scale/drift；把preintegration说成平均；用inlier多或cost低代替frame/time验证；把place similarity直接加图边；把relocalization等同loop、点云拼接等同map merge；无条件声称filter快、optimization准；只报ATE不报alignment、RPE、lost；用Sim3后ATE证明metric scale；把DROID说成无geometry；用PCD、occupancy、ROS路径、fallback test或Git身份证明本地SLAM与ownership。

危险答案：说“SLAM就是前端提特征、后端BA”“map比odom准所以控制都用map”“有IMU就不漂”“检索到相似地点就闭环”“ATE低就是定位好”“Sim3只是换坐标”“DROID端到端不需要几何”“Harness已经有PCD和reloc map，所以做了SLAM”，却不交代sensor/clock/calibration、state/covariance、frame authority、loop geometry/graph residual、map revision、alignment/delta/denominator、live estimator artifact与候选人ownership。

来源：[1][2][3][4][5][6][7][8][9]　练习状态：unseen

参考来源

1. PER-016 Localization / Odometry / SLAM / Map-frame / Drift Knowledge：typed manifest、map/odom、state/covariance、loop/relocal/map merge、drift、ATE/RPE、10 probes、两档回答与五项目truth boundary。
2. PER-016 Source Freeze：六份完整primary PDF、六个official repositories/36文件、REP-105、10个deterministic probes与Robot Harness local-direct audit。

3. Campos et al., ORB-SLAM3 v2；official code@4452a3c。

4. Qin et al., VINS-Mono v1；official code@90dabb5。

5. Hess et al., Real-Time Loop Closure in 2D LIDAR SLAM；Cartographer code@877157a。

6. Teed & Deng, DROID-SLAM v2；official code@2dfd39f。

7. Geneva et al., OpenVINS ICRA 2020；official code@6948812。

8. Rosinol et al., Kimera v3；Kimera-VIO code@ce8c59b。

9. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 37 / 43 · `interview_materials/vla/questions/vol_06_per017.html` · SHA-256
`C2D6CB22037599D33BF0680685568DAB5EDD929A65BA828AAB9A8236F279337C`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`c2d6cb22037599d33bf0680685568dab5edd929a65ba828aab9a8236f279337c`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per017.md`

VOL 06 Perception / Grounding / Navigation：PER-017 原子题卡

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-017` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-017 Belief / Memory / Frontier / Active Exploration Knowledge；来源边界见 PER-017 Source Freeze。

Belief、Memory、Frontier 与 Active Exploration

▼ `L3` `P1` **VLA-PER-017 · Partial observation下怎样维护belief和memory，并用frontier、information gain、coverage、revisit与stuck完成active exploration？**

面试意图：判断你能否把“记住历史、探索未知”拆成有身份和失效条件的 belief/memory 状态、known-free/unknown frontier、硬门与软效用、短执行与重规划、可复算指标，再在项目追问中坚持“history/map/interface 不等于 active-exploration runtime”。

30 秒回答：用transition/likelihood更新belief，分母0就停或重定位；memory分hidden、map、event。frontier是free邻接unknown，先过revision、可达、风险门，再按IG、目标概率、coverage减路程与风险排序；执行短前缀，用revisit、stuck、map revision触发重规划。

机制展开（约 2 分钟）：先冻结episode、传感器/时钟、模型和地图版本。belief按预测→校正→归一化更新，分母0/NaN就停，不伪造后验。memory分hidden、版本绑定map、时间戳event；reset、重定位或schema变化时清空/失效。frontier只取膨胀后free邻接unknown，经新鲜度、可达、风险硬门，再以IG、目标概率、coverage/novelty减路程/风险排序。执行短plan；进展停滞、碰撞、振荡、revisit或map revision触发重规划。评估冻结visibility/cell/budget/denominator；五项目只有接口/资产，无exploration runtime/result。

诊断 / 证据：按first divergence检查episode/sensor/clock/model/map身份→predict/correct/normalizer与数值有限性→hidden reset、spatial anchor revision、event timestamp→occupancy legend、inflation、free/unknown边界→frontier component、viewpoint、path reachability、risk/freshness gate→IG/goal/coverage/novelty/cost分项→plan prefix、progress、collision、oscillation、revisit/cooldown与map revision→visibility/cell/budget/denominator。固定probe至少保留Bayes `[.6,.4]×[.2,.8]→[.272727,.727273]`、全零likelihood拒绝、binary `.5`/accuracy `.9`得IG `.531004 bits`、frontier为`[(1,2),(2,2)]`、隔墙候选不可达、utility选A、visit `0/1/4`的novelty为`1/.707107/.447214`、revision 7/8只留e1、hidden `[3,-2]` reset为0、`runtimeReady=false`或缺trace均不通过；10/10只证明合同，不是benchmark或项目结果。

项目挂钩：Robot Harness有semantic duplicate turn compaction、导航tool contract、occupancy/PCD/reloc map路径与pixel/world bridge，但定向history test仍为`2 failed, 2 passed, 2 warnings`，且fallback明确`runtimeReady=false`；没有typed belief/frontier/visited trace、planner/controller log或matched episode。OPD的`target_coverage/pred_coverage`是2D region overlap；OpenPiVLA的action history/chunk只是policy temporal context；WAM的video/world prediction不是calibrated belief；UWM的simulator ground-truth state和DataLoader memory也不是部署memory。五项目都不能升级为已实现frontier exploration、coverage提升或真实机器人结果；Git身份、字段名和map资产也不能替代机制、运行证据与候选人ownership。

追问树：POMDP的state、observation、history、belief和policy怎样区分？Bayes predict与correct各依赖什么模型？normalizer为0、underflow或NaN为何不能填均匀分布？belief、recurrent hidden、spatial map、episodic event各保存什么？episode reset、relocalization、map revision、model/schema变化分别失效什么？为什么frontier位于known-free一侧而不是unknown一侧？predicted-free为何不能直接当observed-free？inflation、viewpoint与path reachability怎样进入硬门？frontier cluster大小能否等同information gain？IG的expectation对什么 observation 求？goal probability、coverage、novelty、distance和risk怎样避免量纲支配？何时必须hard-risk gate而不是负权重？为什么只执行plan prefix？怎样区分暂时stuck、系统振荡与真正unreachable？revisit是失败还是必要行为？coverage、area seen、visitation、revisit rate与frontier reach rate的分母怎样冻结？reset泄漏为何会虚高？Robot Harness的history、map和tool字段最多支持什么claim？

易错点：把history当belief、action chunk当长期memory、world-model latent当calibrated posterior；漏掉transition predict或用全零likelihood生成均匀后验；让hidden/map跨episode泄漏；把unknown或predicted-free直接当安全可走区域；只按frontier大小排序，不做revision、可达和风险门；把PONI potential说成经典frontier或通用IG；用随机动作掩盖no-valid-frontier；只报coverage，不冻结visibility/cell/budget/denominator；把revisit一律判错；用字段名、map文件、fallback test、公开repo或Git身份证明本地exploration runtime与个人贡献。

危险答案：说“belief就是把历史喂给LSTM”“frontier就是未知格子”“信息增益就是frontier越大越好”“coverage高说明导航好”“卡住就随机走”“有occupancy map和history所以项目做了active exploration”，却不交代episode/sensor/model/map身份、predict/correct/normalize与zero-likelihood、memory reset/invalidation、known-free/unknown语义、reachability/risk硬门、utility分项、短执行/重规划、metric denominator、matched runtime artifact和候选人ownership。

来源：[1][2][3][4][5][6][7][8][9][10]　练习状态：unseen

参考来源

1. PER-017 Belief / Memory / Frontier / Active Exploration Knowledge：POMDP/Bayes、三类memory、typed manifest、frontier/IG/utility、coverage/revisit、10 probes、诊断、两档回答与五项目truth boundary。
2. PER-017 Source Freeze：八份primary paper、六个fixed official repositories/36文件、10个deterministic probes、五份inventory与Robot Harness local-direct audit。
3. Kaelbling, Littman & Cassandra, Planning and Acting in Partially Observable Stochastic Domains, Artificial Intelligence, 1998。

4. Yamauchi, A Frontier-Based Approach for Autonomous Exploration, CIRA, 1997。
5. Chaplot et al., Learning to Explore using Active Neural SLAM, ICLR, 2020； official code@8876e1e。
6. Ramakrishnan et al., Occupancy Anticipation for Efficient Exploration and Navigation, ECCV, 2020； official code@aea6a2c。
7. Chaplot et al., Object Goal Navigation using Goal-Oriented Semantic Exploration, NeurIPS, 2020； official code@5d76902。
8. Ramakrishnan et al., PONI, CVPR, 2022； official code@30682c2。
9. Ramakrishnan, Jayaraman & Grauman, An Exploration of Embodied Visual Exploration； official code@09d3f9b； Das et al., Embodied Question Answering； official code@306fd6e。
10. OPD、OpenPiVLA、Robot Harness、WAM 与 UWM Claim–Evidence Inventories。

Part 38 / 43 · `interview_materials/vla/questions/vol_06_per018.html` · SHA-256
`B4354BE0E2CEC44178C793CB48F975E96EAC46A186BC6881954DAD47C069596C`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`b4354be0e2cec44178c793cb48f975e96eac46a186bc6881954dad47c069596c`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per018.md`

VOL 06 Perception / Grounding / Navigation：PER-018 原子题卡

状态：`drafted / card_1_of_1 / source_independently_reviewed / WARN` 更新日期：2026-07-20 覆盖范围：`VLA-PER-018` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 前置知识：PER-018 Navigation BC / DAgger / RL / Exposure Bias Knowledge；来源边界见 PER-018 Source Freeze。

Navigation BC、DAgger、PPO 与 Exposure Bias

▼ `L3` `P1` **VLA-PER-018 · VLN/ObjectNav 中 BC、Teacher/Student Forcing、Scheduled Sampling、DAgger 与 PPO 如何区分？怎样判断项目是否真的实现了 DAgger/PPO？**

面试意图：判断你能否从 state distribution、target/feedback、update object 和 train/eval protocol 四层区分 imitation/RL，并在项目追问中守住 oracle、reward、rollout freshness、STOP、termination/truncation、checkpoint 与 ownership 边界。

30 秒回答：BC在expert分布学action；student forcing改后续state；scheduled sampling混合previous input。DAgger需oracle、immutable aggregate和β；PPO需fresh rollout、reward、old log-prob、GAE。无runtime不宣称实现。

机制展开（约 2 分钟）：按state distribution区分：BC/teacher forcing在expert轨迹做masked likelihood；student forcing用learner action推进环境；scheduled sampling只改previous input。DAgger每轮用β混合策略，在visited state查oracle并追加immutable aggregate。PPO需fresh rollout、reward、old log-prob、value/GAE，再做ratio/clip更新。评估绑定split、STOP、denominator；项目仅有offline/tool/teacher/separate eval证据，不能升级成个人DAgger/PPO。

诊断 / 证据：先查 task/split/observation/action/STOP schema→expert/oracle或reward revision→policy/checkpoint/normalizer→collector policy hash、visited state和next state→oracle label或behavior log-prob/value/return→mask与denominator→ratio/clip/grad/KL→checkpoint save/load→matched eval protocol。BC loss低但rollout偏航优先查 state-distribution mismatch；DAgger变差查β、oracle/map revision、invalid/skipped/duplicate和aggregate污染；student forcing不收敛查current-state target是否错位；reward升而SR不升查 reward hacking/STOP；ratio异常查 stale rollout/old log-prob；value爆炸查 terminated/truncated bootstrap。固定Habitat `0fb6f43` 仍是单一 `dones→masks`，双字段是目标合同。probe至少保留empty-mask拒绝、β `.75/.5625/.421875/.316406`、GAE `2.82504/1.9405/1`、PPO clip `1.2/-0.8`、terminal/truncation `1/10.9`；10/10只证明合同，不是benchmark。

项目挂钩：OpenPiVLA 的 action-to-text/full-chat CE、LoRA recipe 与 serialization/rollout lineage只能支持 offline supervision；Robot Harness 有 SFT trajectory、schema-aware reward 和 proxy eval，不能替代 fresh physical rollout/PPO；OPD 是 teacher generation/student CE/distillation，不是 env-state DAgger；UWM 是 offline finetune 后 separate rollout evaluation，不是 evaluation-to-training DAgger/RL；WAM 只有 CoffeePressButton failed traces 与 controller/horizon字段，没有完整 trainer/checkpoint/denominator。`muxi/embodied/` 中 ignored/third-party 的 DAgger/PPO 关键词、Git author、文件名、训练 loss 或 closed-loop eval 都不能证明候选人 ownership。

追问树：student forcing何时才接近DAgger？scheduled sampling为什么不是no-regret DAgger？BC action loss下降为何不等于SR/SPL提升？DAgger round最小immutable字段是什么？oracle label如何绑定map/navmesh revision？PPO old log-prob为何必须来自采样policy？ratio与clip异常如何区分stale rollout、advantage符号和mask错误？terminated与truncated如何影响GAE？reward升而SR不升如何查STOP、collision与denominator？DD-PPO的关键是多GPU还是fresh同步rollout？如何给五个项目各写一句不越界回答？

易错点：把 student rollout、recovery demos 或名为 `dagger` 的目录当成 DAgger；把 scheduled sampling当作oracle relabel；只背 PPO clip而漏 value/entropy/reward/return/log-prob；把 timeout和true terminal都写成done；把 reward当SPL；把官方 repo、第三方 checkout、Git author或字段名写成个人贡献；把 separate eval episode倒推成online training data；合并seen/unseen或隐藏无效动作/early stop分母。

危险答案：说“有失败轨迹就能做DAgger”“模型自己滚动就是on-policy”“PPO就是reward加clip”“train loss降了所以导航成功”“项目里有online/ppo/dagger脚本所以我实现了”，却不能给出 visited-state oracle、aggregate round、fresh behavior log-prob/value/return、termination/truncation、checkpoint/eval protocol 和 ownership证据。

来源：[1][2][3][4][5][6][7][8][9]　练习状态：unseen

参考来源

1. PER-018 Navigation BC / DAgger / RL / Exposure Bias Knowledge：BC、forcing、exposure bias、DAgger四门、GAE/PPO/DD-PPO、typed manifest、诊断、10 probes、两档回答与五项目边界。
2. PER-018 Source Freeze：7份固定版本 primary PDF、4个 official repositories、27个代码文件、10个 deterministic probes 与本地五项目 direct audit。
3. Ross, Gordon & Bagnell, DAgger, AISTATS 2011；固定版本 `1011.0686v3`。
4. Bengio et al., Scheduled Sampling, NeurIPS 2015；固定版本 `1506.03099v3`。
5. Anderson et al., R2R, CVPR 2018；Matterport3DSimulator。
6. Krantz et al., VLN-CE；official code。
7. Schulman et al., PPO 与 GAE；Wijmans et al., DD-PPO。
8. OpenPiVLA、Robot Harness、OPD、UWM 与 WAM Claim–Evidence Inventories。
9. PER-018 Independent Fact Review：7 PDF、10 official files、10 probes、8 local files与Habitat done-mask版本边界复核。

Part 39 / 43 · `interview_materials/vla/questions/vol_06_per019.html` · SHA-256
`7C9B5F2110A8E04DB308A811EDC1346C5F54C81299626E35F1651DED18F70046`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`7c9b5f2110a8e04db308a811edc1346c5f54c81299626e35f1651ded18f70046`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per019.md`

VOL 06 Perception / Grounding / Navigation：PER-019 原子题卡

状态：`drafted / card_1_of_1 / source_frozen / knowledge_synced / WARN` 更新日期：2026-07-21 覆盖范围：`VLA-PER-019` 难度：L2 1 题 优先级：P0 1 题 练习状态：`unseen` 前置知识：PER-019 Navigation Evaluation / Failure Knowledge；来源边界见 PER-019 Source Freeze。

Navigation Evaluation、Denominator 与 Failure Taxonomy

▼ `L2` `P0` **VLA-PER-019 · VLN/ObjectNav/PointGoal 中怎样统一 Success、SPL、SoftSPL、NE、OS、TL、split、denominator 与 failure taxonomy？没有完整 navigation bundle 时如何 fail closed？**

面试意图：判断你能否把导航指标讲成同一套 task/split/episode/scorer 协议，并在缺少 raw trace 时拒绝从 summary、loss 或项目文件名推断个人导航结果。

30 秒回答：先锁 task/split/episode、scorer、STOP、分母。Success=STOP+geodesic阈值；SPL 用同 episode 的 shortest path、pose path、success；SoftSPL 是连续项，不是 SR。NE/OS/TL 要带 unit；无 raw trace 不把 summary/loss 当导航结果。

机制展开（约 2 分钟）：先锁 episode IDs、split、navmesh、scorer SHA；确认 Success=STOP+threshold 与 POINT/VIEW_POINTS geodesic。SPL 需 shortest distance、pose path、同一 success；SoftSPL 只按 pinned scorer。NE/OS/TL 绑定 predicate、unit、termination、分母。诊断按 protocol→policy→geometry→control→scorer→unknown，root cause 要 trace；invalid/duplicate/filter 会改分母。五项目无完整 bundle，只讲接口与缺口。

诊断 / 证据：先查 task/split/episode ID set→simulator/navmesh/distance mode/threshold→ordered pose/action/STOP/termination trace→per-episode Success/SPL/NE/OS/TL→included/excluded/missing/duplicate denominator→CI method→first-divergence taxonomy→checkpoint/scorer/ownership。`EVAL.EPISODE_COUNT == -1` 才代表全量评估；只保留成功行、过滤 invalid、混合 seen/unseen 或把 timeout 当 collision 都会改变结论。没有这些 artifact 时保留 `not_recomputable` 或 `unknown`。

项目挂钩：OpenPiVLA 只能连接 image/language/action schema 与 parser/controller contract；Robot Harness 只能连接 2D occupancy、PCD/map interface；OPD 只能连接 point/box/text scorer；WAM、UWM/LIBERO 只能连接 simulator/provenance 或 manipulation protocol。它们目前都不能证明个人 VLN/ObjectNav evaluator、SPL/NE/OS、navigation rollout、collision safety 或 ownership。

追问树：到过目标附近但没 STOP 算 success 吗？POINT 与 VIEW_POINTS 怎样影响距离？SPL 为什么不能用 action count？SoftSPL 能否当 SR？`EVAL.EPISODE_COUNT=-1` 为什么重要？论文 SPL 与 leaderboard SPL 不一致如何解释？NE 下降能否确认根因？timeout 与 collision 如何区分？缺 episode 或 duplicate row 如何处理？如何给五个项目各写一句不越界回答？

易错点：把“到过附近”当 Success；把欧氏距离当 geodesic；把 action 数当 path length；把 SoftSPL 当 binary SR；只报成功样本的均值；把 summary、训练 loss、视频或字段名当作 root cause/ownership；把 timeout、invalid action、collision 和 scorer 丢行混成同一种 failure。

危险答案：说“项目 SPL 很高所以导航成功”“SoftSPL 就是更宽松的 SR”“NE 降了说明模型修好了”“没有 raw episode 也能从 loss 推断闭环能力”，却不能给出 split、episode denominator、scorer revision、STOP、trajectory trace 与 first divergence。

来源：[1][2][3][4][5][6]　练习状态：unseen

参考来源

1. PER-019 Navigation Evaluation / Failure Knowledge：Success/STOP、POINT/VIEW_POINTS、geodesic distance、SPL/SoftSPL、denominator、failure taxonomy、fail-closed 伪代码与五项目边界。
2. PER-019 Source Freeze：2 个 primary papers、2 个 pinned official sources、10/10 deterministic probes 与 claim boundary。
3. Anderson et al., On Evaluation of Embodied Navigation Agents，SPL 与 shortest-path 评估语境。
4. Krantz et al., Beyond the NavGraph: VLN in Continuous Environments，VLN-CE continuous environment 与 metric namespace。
5. Habitat-Lab `nav.py`，Success/DistanceToGoal/SPL/SoftSPL 实现入口。
6. VLN-CE `README.md`，split、episode count、leaderboard/build protocol 边界。

Part 40 / 43 · `interview_materials/vla/questions/vol_06_per020.html` · SHA-256 `CBD0F08B1ECB85390699E71A48785769EB41EFB9ADEA033737F33FF5150345A8`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`cbd0f08b1ecb85390699e71a48785769eb41efb9adea033737f33ff5150345a8`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_06_per020.md`

VOL 06 Perception / Grounding / Navigation：PER-020 原子题卡

状态：`drafted / card_1_of_1 / source_frozen / knowledge_synced / WARN` 更新日期：2026-07-21 覆盖范围：`VLA-PER-020` 难度：L3 1 题 优先级：P1 1 题 练习状态：`unseen` 短答限制：≤180字；机制展开限制：≤330字 前置知识：PER-020 Robustness / Safety Knowledge；来源边界见 PER-020 Source Freeze。

Robustness、Risk Gate、Fallback 与 E-stop

▼ `L3` `P1` **VLA-PER-020 · VLA 面对 corruption、sensor dropout、dataset shift 或风险动作时，怎样证明 robustness、uncertainty、risk gate、fallback 与 E-stop 是同一套可复现协议？**

面试意图：判断你能否把 shifted-input measurement、uncertainty calibration 和 chunked action safety 连接起来，并在项目追问中区分公开机制、局部接口、推断与未核实的个人安全结果。

30 秒回答：先锁 corruption/severity、denominator；shift 看 calibration 与 risk-coverage。风险门检查观测、硬约束、不确定性；触发即 flush 旧 chunk，按 recovery/fallback 处理，无法恢复才 E-stop。缺 raw shift、risk log、硬件审计时，不声称个人安全结果。

机制展开（约 2 分钟）：冻结 family/severity、RNG、shift set、denominator；不同 family 同号 severity 不横比。shifted split 看 calibration、risk-coverage、false-safe/false-alarm，保留拒答。执行层串起 validity→risk gate→hard constraint；风险后 flush chunk，按 recoverable 选择 reobserve、fallback 或 E-stop。五项目只有 schema/confidence hook；缺 raw shift、risk trace、matched eval、ownership，不声称安全提升。

诊断 / 证据：先查 sensor validity、timestamp、clean/shifted pair、family/severity/RNG→policy checkpoint、uncertainty/calibration bin→risk threshold/gate reason→action age、queue length、flush event→controller constraint/fallback/E-stop response→per-episode metric、cost、denominator→first-divergence taxonomy→ownership。camera severity sweep只能证明输入 shift协议；Safety Gym `cost_hazards` 只能证明模拟器记录了cost。没有逐步trace时保留 `unknown/not_recomputable`。

项目挂钩：OpenPiVLA可谈image/action normalizer、parser、controller contract与shifted eval design；Robot Harness可谈tool schema、地图/点云和日志字段；OPD可谈confidence、negative case、risk-coverage；WAM/Cosmos可谈simulator diagnostics与visual shift taxonomy；UWM/LIBERO可谈fixed-init/reset、strict denominator。五者目前都不能证明个人OOD robustness、calibration uplift、fallback成功率、真实collision safety或E-stop ownership。

追问树：severity=5一定比另一个family的3严重吗？预生成JPEG和在线Gaussian能直接比较吗？confidence高但动作错是什么风险？风险门触发后为什么必须flush chunk？fallback与retry区别？Safety Gym cost=0能否代表真实安全？randomize_layout=False/True是否同一泛化协议？没有raw trace能否确认collision根因？如何给五个项目各写一句不越界回答？

易错点：把severity跨family当物理量；把clean accuracy或confidence当OOD safety；把阈值叫calibration却不报告risk-coverage；风险后继续执行旧chunk；把retry当fallback；把Safety Gym cost、randomization、simulator rollout或上游代码写成个人安全结果；把Git author、字段名或summary当ownership。

危险答案：说“confidence高所以动作安全”“Safety Gym cost为0所以真实机器人安全”“风险门触发后执行完当前chunk再说”“有randomization就证明sim2real”“项目已经有robustness提升”，却不能给出family/severity、shifted denominator、calibration、queue flush、fallback/E-stop trace与个人证据。

来源：[1][2][3][4][5][6]　练习状态：unseen

参考来源

1. PER-020 Robustness / Safety Knowledge：corruption protocol、risk–coverage、risk gate、queue flush、fallback/recovery/E-stop、五项目边界与两档回答。
2. PER-020 Source Freeze：2 个 primary papers、4 个 official code/docs、10/10 probes 与 claim boundary。
3. Hendrycks & Dietterich，Benchmarking Neural Network Robustness to Common Corruptions and Perturbations。
4. Ovadia et al.，Can You Trust Your Model's Uncertainty?。
5. Hendrycks robustness `corruptions.py` 与 OpenAI Safety Gym `engine.py`。
6. VOL 04 WM-038 uncertainty/safety/fallback source freeze。

Part 41 / 43 · `interview_materials/vla/sources/vol_06_per018_navigation_bc_dagger_rl_exposure_bias_independent_fact_review.html` · SHA-256 `21EA6D806ED42B09279569043E76A33E0CF04733B2ACF3E77241ED18FDD83AB6`

返回本卷目录 · 对应知识页 · 对应题卡

Canonical Markdown SHA-256：`21ea6d806ed42b09279569043e76a33e0cf04733b2acf3e77241ed18fdd83ab6`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_06_per018_navigation_bc_dagger_rl_exposure_bias_independent_fact_review.md`

VOL 06 PER-018：Navigation BC、DAgger、RL 与 Exposure Bias 独立事实复核

状态：`independent_fact_review / VLA-PER-018 / paper_code_refetched / source_knowledge_card_reconciled / PASS_WITH_WARNING` 审查日期：2026-07-20 证据截止：2026-07-20 范围：独立重新下载并哈希7份primary PDF，重新抓取并哈希4个fixed official repositories中的10个代表文件，重跑10个deterministic probes，复核source→knowledge→card与五项目truth boundary；不生成新题卡、不改变269张题卡的 `unseen` 状态、不启动PER-019、不创建coaching state。

问题（Question）

PER-018 source、knowledge 与 atomic card 对 BC、teacher/student forcing、scheduled sampling、DAgger、GAE/PPO/DD-PPO、termination/truncation 和项目真实性边界的表述，是否能被重新获取的primary paper、fixed official code 与当前local-direct artifacts逐项支持？哪些内容是paper-exact、code-exact、工程综合或仍未验证？

1. 结论先行

1. **7/7 primary PDFs byte-identical**：重新下载的版本、bytes与SHA全部命中source freeze，合计96页；DAgger aggregate、Scheduled Sampling previous-input mismatch、R2R teacher/student forcing、VLN-CE `0.75^n/5,000/4 epochs`、DD-PPO同步去中心化与preemption、PPO ratio/clip、多epoch minibatch、GAE的$\gamma\lambda$ advantage均可直接定位。
2. **10/10代表official files byte-identical**：VLN-CE 3个、Habitat-Lab 5个、R2R-EnvDrop 1个、Matterport R2R 1个文件全部命中source freeze SHA。VLN-CE code确实执行 `p ** data_it`、expert/policy action mix、LMDB append和oracle-action inflection weighting；Habitat code确实保存action log-prob、计算GAE、PPO ratio/clip、value/entropy与grad statistics。
3. **R2R语境中的student forcing可以称online DAgger，但不能无条件泛化**：论文p7明确写“equivalent to an online version of DAgger / always sampling”，前提是target始终从agent当前pose到goal的shortest path重新计算。knowledge/card把general case继续要求current-state oracle label是正确收紧，不与论文冲突。
4. **Scheduled Sampling与DAgger仍须分开**：前者直接支持true/generated previous token curriculum；它不提供navigation oracle、immutable aggregate或no-regret身份。把 `ε=0` 叫always sampling是paper-

exact，把任意always-sampling流程叫canonical DAgger则不成立。

5. **termination/truncation分离是本仓库的工程合同，不是pinned Habitat路径的code-exact事实**：固定commit `0fb6f43` 的 `ppo_trainer.py` 读取单一 `dones` 并生成 `not_done_masks`，`rollout_storage.py` 只消费 `masks`；该路径没有显式 `terminated/truncated` 字段。知识页说"真正terminal切bootstrap、time-limit truncation通常保留bootstrap"是合理目标，但必须标为engineering requirement，不能说该官方代码已经实现。
6. **PPO完整loss是实现相关的**：PPO论文直接支持ratio、clipped surrogate和多epoch minibatch；pinned Habitat实现还包含clipped value、entropy、aux loss、可选VER importance coefficients与adaptive entropy。知识页的 `-Lclip + value - entropy + aux` 是教学综合，不是该commit逐项完全展开式。
7. **10/10 probes独立复算通过**；它们只验证identity、mask、schedule、GAE、clip和bootstrap反例，不是训练稳定性、论文benchmark或个人项目结果。
8. **五项目边界再次成立**：8个local-direct files重新哈希命中；定向关键词审查只发现UWM的separate rollout eval、OPD teacher rollout和Harness reward spec邻接事实，没有local navigation DAgger collector、visited-state oracle aggregate、fresh PPO behavior log-prob/value/return或matched result。ignored/third-party代码仍不能计入候选人ownership。

本页 verdict 为 `PASS WITH WARNING`：public mechanism与source→knowledge→card对齐通过；pinned Habitat terminated/truncated差异、local runtime、ownership与真实口述仍为warning/pending。

2. Primary PDF 再获取与完整性

Source	再获取版本	Pages / bytes	SHA-256	独立定位到的核心事实
DAgger	`1011.0686v3`	9 / 551,229	`12B032CFD451B8ACE4B18145DBE24382D2CBE2EDA6B562331BC86799BBAAFD5E`	Algorithm 3.1：mixture policy、visited states、expert actions、aggregate、retrain
Scheduled Sampling	`1506.03099v3`	9 / 283,137	`9769DF7150C7686237A2E709FA4FC004BFA03636A8329C9C1FD5C60163CFB00C`	train用true previous token、inference用generated token、ϵ_i curriculum与always sampling
R2R	`1711.07280v3`	13 / 9,720,283	`B7589865EE573D4753EE9E25BDC494CBBB199E6EC17D0A349091AED0FD86C420`	p7 teacher/student forcing；target从current pose到goal shortest path；online DAgger限定语境
VLN-CE	`2004.02857v2`	18 / 10,596,423	`8C62871FFB3D266F38EE2FEA944583645F74DBE10AC85213AD536486C18CB1A3`	$\beta = 0.75^n$、每阶段5,000 trajectories、4 IL epochs、inflection weighting
DD-PPO	`1911.00357v2`	21 / 3,578,072	`CB9A5B9C603A6EDDD95F6A8D17E946C46681506FD51302371FE67EE208AF6F2D`	distributed、decentralized、synchronous、straggler preemption与2.5B-step experiment
PPO	`1707.06347v2`	12 / 2,923,532	`E78FEADADBDBB0B601B3C2BCC81404722CD431A489B307545F9B7BEA1E8C4F5B`	old/new ratio、clipped surrogate、多epoch minibatch update
GAE	`1506.02438v6`	14 / 1,798,117	`5AA736CCF3517A1F36D0CA8146982E69BEAF9A2BDCCAC2AE324F780743BC045E`	TD residual、$\sum_l (\gamma\lambda)^l \delta_{t+l}$、bias/variance trade-off

7个文件均重新从明确version URL获取；byte SHA与source freeze完全一致。PDF文本重新抽取96/96页；代表性公式与算法段可定位。此前150 DPI视觉检查证明可读，本轮不重复生成第二套图片，避免把重复渲染冒充额外事实证据。

3. Official Code 独立抽样

Repo / file	Fixed commit	再取SHA	复核结果
VLN-CE `dagger_trainer.py`	`729d141`	`924BE549FE25C31AE3E85AF95DE5A94142543233858 83A1093EDC830497C0AD7`	`beta=p**data_it`；random gate选expert/policy action；LMDB从existing entries追加
VLN-CE `recollection_dataset.py`	`729d141`	`61042AADE2CD06AB70D6B8F1B4FD0D19613FC3E0E70 D2B6282632650B2EE3DC6`	prev/oracle action；oracle-action change位置生成inflection weights
VLN-CE `seq2seq_da.yaml`	`729d141`	`9376FA4E112D1844C5066E0E8FD6E4D07DD585EE590 E3ECFFCB1FA2E1D300D44`	iterations=10、update_size=5000、p=.75、epochs=4
Habitat `ppo.py`	`0fb6f43`	`639063D9ADC1AA8D2BFDD0054958177EDD362D90ED4 3CD2BE05C74A2436C3ECF`	log-prob ratio、clip、value、entropy、aux、grad/ratio stats
Habitat `rollout_storage.py`	`0fb6f43`	`914DCA269E640992B5B0343B5811B24110DCCF073B9 2AE7097F551388DF351B0`	rewards/value/action_log_probs/masks；reverse GAE/returns
Habitat `ppo_trainer.py`	`0fb6f43`	`3D092753BF3B0CA6273E0213316C9A66653EED8D519 C9DF60F2104CA883FA169`	env `dones` → `not_done_masks`；未显式拆terminated/truncated
Habitat `ddppo.py`	`0fb6f43`	`20C91A45BA4F7F857A0EB938A3A404A33EE88F288BF 45F41C1A17F3361EC0AFF`	distributed mean/variance、DDP wrapper、继承PPO
Habitat `ddppo_pointnav.yaml`	`0fb6f43`	`B8B443ACE3E48556E500260606B96390963A84ECDB1 E23253918348BF9B3077A`	2.5e9、clip=.2、GAE、γ=.99、τ=.95、sync_frac=.6
R2R-EnvDrop `agent.py`	`6f938ab`	`13710C80CA462E68379E24CF0D347B8C1934ECD1085 450A835C7C69C4CB2F565`	teacher/sample/argmax、CE、distance reward、value/entropy、policy gradient混合
Matterport R2R `agent.py`	`589d091`	`CCE3492EDB2AD0A36CDAF750022052420F3615953DC 2DD0CBA0DA26DD9C42F7C`	teacher/sample/argmax、CE、test时禁止teacher cheat

10/10文件的bytes与source freeze记录一致。这个抽样覆盖关键机制路径，但不替代27文件的source ledger，也不证明任何本地复现或运行规模。

4. BC、Forcing 与 Scheduled Sampling 复核

BC教学目标写为：

$$\mathcal{L}_{\mathrm{BC}}(heta) = -\frac{\sum_{b,t} m_{b,t} w_{b,t} \log \pi_\theta(a^*_{b,t} \mid o^E_{b,t}, h^E_{b,t}, c_b)}{\sum_{b,t} m_{b,t} w_{b,t} + \varepsilon}.$$

变量与映射：a^*是expert action，m是valid-action mask，w可表达inflection/sample weighting；R2R official code直接支持cross-entropy target，VLN-CE dataset支持oracle action与inflection weight。此式是对离散action trainer的安全教学综合，不是R2R论文逐字公式。empty denominator必须在公式外先gate；ε不能把空batch变成有效更新。

Scheduled Sampling的核心门为：

$$a^{\mathrm{in}}_{t-1} = \begin{cases} a^*_{t-1}, & z_t < \epsilon_i, \\ \hat{a}_{t-1}, & z_t \geq \epsilon_i. \end{cases}$$

变量与映射：ϵ_i控制true/generated previous input比例；paper直接支持从fully guided到mostly generated的curriculum。它没有定义navigation shortest-path oracle、round dataset或cross-round aggregation，因此knowledge/card说“Scheduled Sampling不自动等于DAgger”通过。

R2R的特殊点是target a_t^*始终根据agent当前pose重算。论文因此把student forcing称为online DAgger / always sampling。安全口述应说：“在R2R这个current-pose shortest-path oracle语境下等价；离开该语境必须重新检查oracle relabel和aggregation。”

5. DAgger Identity 与 VLN-CE 映射

$$\pi_i = \beta_i \pi^* + (1-\beta_i)\hat{\pi}_i, \qquad D \leftarrow D \cup \{(s, \pi^*(s)) : s \sim d_{\pi_i}\}.$$

变量与映射：π^*为expert，$\hat{\pi}_i$为learner，d_{π_i}为实际visited-state distribution。DAgger Algorithm 3.1直接包含mixture、visited state expert action、dataset aggregate与retrain；VLN-CE code把`p ** data_it`映射到β_i，用`torch.where(rand < beta, expert, policy)`推进环境，并将trajectory追加到LMDB。

独立复核后的四门不变：

Gate	Paper/code evidence	不能用什么替代
rollout identity	mixture/current policy实际产生visited states	文件名、旧demo、静态sample
oracle identity	visited state上的同revision expert action	原expert path错位label、self label
aggregation identity	round append、episode/invalid/skipped/duplicate可追溯	无lineage拼接、recovery数据别名
schedule/checkpoint	round、β_i、policy SHA、train config	“online”字样或最终loss

VLN-CE的10×5,000是nominal config规模；paper还写6–10次collection取决于model/convergence。知识页把50,000标为nominal并要求扣除invalid/duplicate/early stop是正确边界，不能把配置乘法当实际有效trajectory denominator。

6. GAE、PPO 与 termination/truncation 差异

GAE的paper-exact主体为：

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V.$$

变量与映射：δ_t^V是TD residual，λ控制多步bias/variance；Habitat storage用reverse recursion和`masks[step+1]`实现有限rollout版本。GAE paper讨论terminal absorbing state，但没有现代Gymnasium式terminated/truncated双字段。

PPO paper-exact主体为：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, \qquad L^{\text{CLIP}} = \mathbb{E}_t\left[\min\left(r_t\hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right].$$

变量与映射：old log-prob必须与采样action的behavior policy对齐；Habitat code以`exp(new_logp-buffer_logp)`求ratio，再计算clipped action loss、value loss、entropy、aux和grad statistics。knowledge页的total objective是教学层综合；pinned code还含可选VER importance coefficients、clipped value与adaptive entropy，不能把简式说成该commit逐行等价。

6.1 新确认的 pinned-code warning

`ppo_trainer.py`只接收env返回的`dones`，再构造：

```
# Fixed Habitat-Lab path: one done flag becomes the bootstrap mask.
not_done_masks = torch.tensor([[not done] for done in dones])

# Rollout storage uses the same mask in TD and GAE recursion.
delta = reward + gamma * next_value * next_mask - value
gae = delta + gamma * tau * gae * next_mask
```

代码注释与结论：该路径没有显式`terminated`与`truncated`字段，因此不能证明time-limit truncation被正确bootstrap。PER-018中“true terminal切bootstrap、time-limit truncation通常保留bootstrap”保留为**engineering contract / failure gate**；若项目采用这个pinned path，必须另查env wrapper是否在进入`dones`前处理time-limit信息。

7. DD-PPO 与分布式边界

DD-PPO paper直接支持distributed、decentralized、synchronous、no-stale与straggler preemption。pinned code把DDP wrapper/跨worker mean-variance放在`ddppo.py`，PPO update放在父类，preemption orchestration位于trainer而不是这个短mix-in文件。配置的`sync_frac=.6`和`total_num_steps=2.5e9`是public recipe；不能由“本地多GPU”反推完成论文规模。

工程上仍应记录每worker collected steps、preemption、episode termination与global denominator；这是可复现性要求，不是DD-PPO论文保证每个本地系统自动拥有的artifact。

8. 10个 Deterministic Probes 独立重算

Probe	独立结果	Verdict
masked BC NLL	`-log(.8)=.2231435513142097`	PASS
empty mask	`gate=false`，明确拒绝	PASS
exposure shift	expert `+1`与model `-1`访问不同state	PASS
scheduled endpoints	`ε=1→teacher`，`ε=0→model`	PASS
DAgger beta	`.75,.5625,.421875,.31640625`	PASS
nominal aggregate	`10×5000=50000`，只作nominal	PASS
inflection	`0,0,1,1,2→1,1,3.2,1,3.2`	PASS

Probe	独立结果	Verdict
GAE reverse	`2.82504025,1.9405,1`	PASS
PPO clip	`(1.3,+1)→1.2`；`(.7,-1)→-.8`	PASS
terminal/truncation counterexample	`1` 与 `1+.99×10=10.9`	PASS

结果为`10/10`。第10项是目标合同反例，不是pinned Habitat code已经拆分双字段的证明。

9. Source → Knowledge → Card Claim Reconciliation

Claim cluster	Source	Knowledge	Card	Verdict / correction
BC/teacher forcing	expert distribution、CE、current state/action	masked NLL、covariate shift	30秒/机制答保留	PASS；empty denominator须公式外gate
student forcing	R2R current-pose oracle与explicit online-DAgger表述	general case增加oracle gate	追问保留scope	PASS；R2R限定等价，非全局同义词
scheduled sampling	true/generated previous input curriculum	与DAgger分开	简答保留	PASS
DAgger	Algorithm 3.1 + VLN-CE paper/code/config	四门、manifest、nominal denominator	oracle/aggregate/β	PASS
GAE/PPO	paper公式 + Habitat storage/update	reward/value/logp/ratio/clip/entropy	fresh runtime门	PASS WITH WARNING；total loss为教学综合
terminated/truncated	engineering requirement + counterexample	明确双字段与bootstrap	diagnosis追问	PASS WITH WARNING；pinned trainer未显式实现
DD-PPO	paper sync/decentralized/preemption + code/config	worker/denominator contract	“不等于多GPU”	PASS
project mapping	inventories + 8 local files	五项目truth table	五项目安全回答	PASS WITH WARNING；local runtime/ownership缺失

本轮没有发现必须重写30秒答案的错误。面试时新增一句版本限定最稳妥：“terminated/truncated分离是我要求系统补齐的合同，固定Habitat路径本身仍只有done mask。”

10. Local-direct 再哈希与 Truth Boundary

Local artifact	SHA重新命中	直接支持	仍不能升级
OpenPiVLA `formal_train.py`	`51B67D...CC6F94`	action JSON/full-chat CE、LoRA trainer	navigation DAgger/PPO、formal closed-loop result
Harness `prepare_sft_data.py`	`AF75E0...85589`	SFT trajectory/split	physical navigation BC
Harness `prepare_grpo_data.py`	`EAA413...28250`	reward schema/proxy weights	trainer、fresh rollout、PPO/online GRPO、robot success
UWM finetune launcher	`E5B6DC...9D92`	offline finetune recipe	oracle/reward update、checkpoint content
UWM rollout eval	`05E912...F3943`	separate env rollout/eval loop	eval episode进入online update
OPD root trainer	`BF9120...D8D9`	teacher generation、student CE/distillation	env-state DAgger、navigation RL

Local artifact	SHA重新命中	直接支持	仍不能升级
WAM base manifest	`63A518...33A86`	matched protocol字段、`success=false`	trainer、checkpoint binary、denominator
WAM trained manifest	`809AE4...55CC`	trained-side失败trace引用	BC/RL uplift或checkpoint因果

六个代码/launcher文件的定向关键词复核只发现：UWM `rollout_eval` 执行separate evaluation；OPD使用teacher rollout/logits；Harness生成reward spec。没有`old_log_prob/compute_returns/GAE/PPO`训练链，也没有navigation visited-state oracle aggregate。WAM两份manifest仍是single-task failed diagnostic。结论仍为“当前检查范围内未绑定”，不是对未同步远端或历史分支的绝对不存在声明。

11. 审查状态与下一步

本次独立再获取、哈希、paper/code定位、10 probes、local rehash与source→knowledge→card reconciliation均完成。PER-018 public facts可从`WARN`提升为`independent_fact_reviewed / PASS WITH WARNING`，但题卡仍保持`unseen`，整个节点仍不能升级为drilled/done。

仍开放：

- pinned Habitat path的terminated/truncated处理需要项目级wrapper或新实现证据；
- local navigation oracle/DAgger rounds或fresh PPO rollout/checkpoint/eval bundle缺失；
- 五项目逐文件候选人ownership待本人确认；
- 没有真实口述，不创建coaching state。

下一 bounded slice优先进行一题真实口述；若用户暂不练习，可再选择VOL 04 Audit C/D独立复核，不启动PER-019。

References

1. PER-018 Source Freeze。
2. PER-018 Knowledge。
3. PER-018 Atomic Card。
4. Ross, Gordon & Bagnell, DAgger, AISTATS 2011。
5. Bengio et al., Scheduled Sampling, NeurIPS 2015。
6. Anderson et al., R2R, CVPR 2018。
7. Krantz et al., VLN-CE；official code@729d141。
8. Schulman et al., PPO 与 GAE。
9. Wijmans et al., DD-PPO；Habitat-Lab@0fb6f43。
10. R2R-EnvDrop@6f938ab 与 Matterport3DSimulator@589d091。
11. OpenPiVLA、Robot Harness、OPD、UWM 与 WAM Claim–Evidence Inventories。

Part 42 / 43 · `interview_materials/vla/sources/vol_06_per020_robustness_safety_fallback_independent_fact_review.html` · SHA-256 `B656495E28AEB09144207BEB1D67C4DC27F631BD27EBACAB8EBE07FF07281D38`

返回本卷目录 · 对应知识页 · 对应题卡

Canonical Markdown SHA-256：`b656495e28aeb09144207beb1d67c4dc27f631bd27ebacab8ebe07ff07281d38`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_06_per020_robustness_safety_fallback_independent_fact_review.md`

VOL 06 PER-020：Robustness、Safety、Fallback 与风险门独立事实复核

状态：`independent_fact_review / VLA-PER-020 / paper_code_refetched / source_knowledge_card_reconciled / PASS_WITH_WARNING` 审查日期：2026-07-21 证据截止：2026-07-21 范围：重新获取 2 份 primary PDF、4 个 fixed official files，重跑 bounded source probes，并复核 PER-020 source→knowledge→card；不生成新题卡、不改变 276 张题卡的 `unseen` 状态、不创建 `coaching_state.md`。

问题（Question）

PER-020 对 corruption family/severity、dataset shift uncertainty、risk–coverage、Safety Gym cost、queue flush、fallback/recovery/E-stop 的表述，是否能被重新获取的 primary paper、fixed official code 与当前仓库证据逐项支持？哪些内容仍只能作为工程合同或面试回答，不能升级为个人项目安全/鲁棒性结果？

1. 结论先行

1. **6/6 外部文件重新获取成功并命中 source freeze SHA**：Hendrycks & Dietterich PDF 1,080,004 bytes / 16 pages；Ovadia et al. PDF 5,887,184 bytes / 25 pages；robustness README、`corruptions.py`、Safety Gym README、`engine.py` 均与冻结记录 byte-identical。
2. **公开机制边界成立**：README 同时区分 ImageNet-C 与 ImageNet-P；`corruptions.py` 保留 severity 1–5 的 family-specific 参数与随机/clip 行为；Safety Gym `engine.py` 暴露 `constrain_hazards`、`cost_hazards`、`cost` 与 `randomize_layout` 语义。这些只支持 benchmark/environment 机制，不支持本地 VLA 已跑过对应协议。
3. **Ovadia 的可安全转译结论是“confidence ≠ shifted risk”**：应报告 calibration、selective risk–coverage 与 false-safe/false-alarm；不能把 clean confidence 或单一平均分当作安全保证。
4. **queue flush / fallback / recovery / E-stop 仍是 cross-volume WM-038 的工程合同**：本轮没有发现任何五项目 raw step trace、硬件审计或 ownership ledger，不能把该合同说成项目已经实现。
5. **source→knowledge→card 八个 claim cluster 对齐**：corruption/severity、shifted calibration、risk–coverage、hard constraint、chunk flush、fallback/recovery/E-stop 与五项目 truth boundary 在三层页面中语义一致；题卡仍 `unseen`。
6. 本页 verdict 为 `PASS WITH WARNING`：外部事实与下游映射通过；本地 robustness/safety runtime、真实机器人 cost、零事故、fallback 成功率和候选人 ownership 仍为 `unverified`。

2. 外部文件再获取与 SHA

Source	Fixed revision / URL	Bytes / pages	Re-fetched SHA-256	结果
Hendrycks & Dietterich	arXiv 1903.12261	1,080,004 / 16	`c9df5483aa331dd99808350608a795c4bd9b84a7ee50396748c7d258903ea582`	PASS
Ovadia et al.	arXiv 1906.02530v2	5,887,184 / 25	`1610d04d7f69d10751f0e9ff1db9fbd8b3fdf5306798879016192eb2dbf17a6f`	PASS
robustness README	commit `8190fe329f5f072a06e3a2aea02bb5dda69aed9f`	7,354	`718d37a53a84615961dcdf519602cfaf1467b192bb691f8e880fbadf9e92de0d`	PASS
`corruptions.py`	same commit	14,857	`fb962115845e0a7ceaf5a6e3cda17f4aab07c66e3c367c219cafc7032c4a43fb`	PASS
Safety Gym README	commit `f31042f2f9ee61b9034dd6a416955972911544f5`	11,348	`2728e304d7b8483ce910d60c4880782aeb111c5fa127614b0a1d3f8bfd4653b5`	PASS
Safety Gym `engine.py`	same commit	72,367	`9d52a3206648938408c05473eb48f12a78230938ea5c4b9ca878afaab087693c`	PASS

PDF 页数由独立 `pypdf` 读取；文件 bytes 与冻结记录完全一致。重新获取证明 source identity，不证明任何训练、仿真或硬件运行。

3. 独立 source probes

Probe	重新观察	支持的窄结论
P20-R1	robustness README 含 `ImageNet-C`	README 明确静态 common-corruption 评估入口
P20-R2	robustness README 含 `ImageNet-P`	README 同时覆盖 temporal perturbation 语境
P20-R3	`corruptions.py` 含 40 处 `severity` 参数引用	severity 是实现参数，不是跨 family 的统一物理量
P20-R4	`corruptions.py` 的 Gaussian/shot/impulse/speckle 路径使用随机与 clip/归一化	在线注入必须记录 RNG、dtype、clip 与输出策略
P20-R5	Safety Gym `engine.py` 含 `constrain_hazards`	未启用 constraint 配置不能声称测过 hazard constraint
P20-R6	`engine.py` 含 `cost_hazards` 与聚合 `cost`	safety 记录应保留逐 step/episode cost 与 denominator
P20-R7	`engine.py` 含 `randomize_layout`	固定布局与随机布局是不同 protocol，不能混报泛化
P20-R8	Ovadia PDF 可重新读取 uncertainty/calibration/dataset-shift 章节	confidence 不能直接替代 shifted risk
P20-R9	WM-038 source freeze SHA `017dbe706784...eabbb3c` 命中本地文件	queue flush、short-prefix、fallback/E-stop 是本仓库工程合同
P20-R10	PER-020 source/knowledge/card 的 stable ID 均为 `VLA-PER-020`	downstream identity 与题卡映射一致，不新增题卡

结果：**10/10 probes 通过**。P20-R1–R7 是 source-text probes，P20-R8 是 paper retrieval/section probe，P20-R9–R10 是 local identity probes；它们都不是 benchmark runtime 或硬件安全证据。

4. Claim boundary：论文、官方代码与工程合同

4.1 Corruption 与 severity

Hendrycks & Dietterich 支持 common corruptions/perturbations 与 ImageNet-C/P 的 benchmark 语境；官方 `corruptions.py` 的 severity 参数是每个 family 自己定义的离散等级。安全的面试说法是：“我会固定 family、severity、clean/shifted pair、split、seed、checkpoint 和 denominator，再比较同一 protocol 下的指标。”不能说 severity 5 的 Gaussian 一定比 severity 3 的 JPEG 更严重，也不能把预生成 JPEG 与在线随机 Gaussian 当作同一个实验。

4.2 Uncertainty 与 selective risk

Ovadia 支持 dataset shift 下对 predictive uncertainty、calibration 和 risk 行为进行评估。风险–coverage 的教学表达为：

$$\mathrm{Risk}(C) = \mathbb{E}[\ell(\hat{y}, y) \mid \mathrm{accept} = 1], \qquad C = \Pr(\mathrm{accept} = 1).$$

`accept` 由 risk gate 决定，`ell` 可是任务错误、约束违规或失败损失。公式只定义 selective evaluation，不证明模型已校准；拒答/fallback 样本必须保留，否则会制造 false-safe。

4.3 Safety Gym 与真实安全的边界

Safety Gym README/`engine.py` 支持 constrained task、hazard cost 与布局随机化配置。它不支持以下升级：

- 使用 Safety Gym 就等于真实机器人 safety；
- `cost=0` 就等于 zero collision 或 zero accident；
- `randomize_layout=True` 就等于 sim-to-real 泛化；
- 没有逐 episode/step cost、seed、布局与 checkpoint，就能复算个人结果。

4.4 Queue flush、fallback、recovery 与 E-stop

WM-038 的工程合同把执行语义写成：风险门触发后停止继续 apply，清空过期 chunk，再选择 re-observe、safe controller、人工接管或 E-stop。该合同解决的是“系统应该如何 fail-closed”，不是“项目已经安全”。若没有 action-age、queue length、flush event、gate reason、controller response 与 E-stop latch 的 raw trace，结论必须保持 `unknown/not_recomputable`。

5. Source → Knowledge → Card reconciliation

Claim cluster	Source evidence	Knowledge page	Atomic card	Verdict
family/severity/RNG	Hendrycks paper + `corruptions.py`	§3 protocol/manifest	30秒答先锁 family/severity/denominator	PASS
clean vs shifted	ImageNet-C/P and fixed pair semantics	shifted metric + denominator	clean score 不替代 shifted score	PASS
confidence/calibration	Ovadia paper	calibration、risk–coverage、false-safe	confidence 不等于 safety	PASS

Claim cluster	Source evidence	Knowledge page	Atomic card	Verdict
hard constraint/risk gate	Safety Gym contract + WM-038	validity→risk→constraint state machine	gate reason 与 evidence gate	PASS
stale chunk	WM-038	queue flush / action age	风险后必须 flush	PASS
fallback/recovery/E-stop	WM-038 engineering boundary	recoverable vs terminal state	fallback≠retry；不可恢复才 E-stop	PASS WITH WARNING
five projects	local inventories and absence checks	interface hooks only	不声称 personal OOD/safety result	PASS WITH WARNING
practice state	local card/review metadata	card remains unseen	no mastery/coaching update	PASS

本轮没有发现需要改写 PER-020 30 秒答案或机制展开的事实错误；新增的最稳妥限定仍是：“Safety Gym、randomization 和公开 queue/fallback 机制是协议参照，不是我的个人安全结果。”

6. 五项目真实性复核

项目	当前可说	本轮仍不能说
OpenPiVLA	image/action normalizer、parser、controller contract、shifted-eval 设计	已完成 OOD robustness、calibration uplift、zero-accident 或 E-stop ownership
Robot Harness	tool schema、地图/点云输入、约束与日志字段设计	Safety Gym cost、fallback success rate、真实硬件 safety
OPD	grounding confidence、negative case、risk–coverage 设计	grounding robustness uplift 或 deployed safety guarantee
WAM / Cosmos	simulator diagnostics、visual-shift taxonomy、失败 trace	上游 randomization/rollout 是个人 sim-to-real 或安全结果
UWM / LIBERO	fixed-init/reset、strict denominator、failure taxonomy	真实机器人 OOD 泛化、碰撞率或安全控制器结果

当前检查的 local-direct 入口仍只支持 `public upstream mechanism → local evidence/interface → inference → unverified` 四层边界。没有新发现可把任何项目升级为已完成安全闭环。

7. 静态结构与状态检查

- PER-020 source、knowledge、card canonical Markdown SHA 与各自 review JSON 已存在且一致；本页 source SHA 为 `独立复核文件写入后生成`。
- 题卡 stable ID 仍只有 `VLA-PER-020`；global card count 不变，`unseen=276`。
- 本轮不创建、不修改 `interview_materials/vla/practice/coaching_state.md`。
- 新 HTML 必须由 academic renderer 生成，并执行 one-H1、heading hierarchy、MathJax balance、`.formula em = 0`、contained overflow、local-link 与 SHA 检查。
- 浏览器 connector 当前不可用，因此只能报告静态 ARIS/responsive 检查；不把缺失截图写成 browser PASS。

8. 审查状态与下一步

本轮的一个 bounded primary deliverable 是 PER-020 independent fact-review 三联件。其退出条件已满足：6/6 外部文件 SHA 命中、10/10 probes 通过、source→knowledge→card 对齐、五项目边界未升级、mastery 未变化。

仍开放：

1. 本地 shifted-input bundle、逐步 risk log、matched robustness panel、硬件 E-stop audit 与 candidate ownership ledger；
2. 一题真实 PER-020 口述，之后才允许更新 mastery 或创建 coaching state；
3. 若用户暂不练习，下一切片选择 VOL 04 Audit C/D 独立复核，不启动新卷或批量生成。

References

1. PER-020 Source Freeze。
2. PER-020 Knowledge。
3. PER-020 Atomic Card。
4. Hendrycks & Dietterich，Benchmarking Neural Network Robustness to Common Corruptions and Perturbations。
5. Ovadia et al.，Can You Trust Your Model's Uncertainty?。
6. Hendrycks robustness `README.md` 与 `corruptions.py`。
7. OpenAI Safety Gym `README.md` 与 `engine.py`。
8. VOL 04 WM-031 reality-gap taxonomy 与 VOL 04 WM-038 uncertainty/safety/fallback。

Part 43 / 43 · `interview_materials/vla/sources/vol_06_perception_grounding_navigation_coverage_gap_audit.html` · SHA-256 `E51749E5B199B94DA24D237CB1CBF3C5650B6F99AAAF234150BB3E8DF95DAC21`

返回本卷目录

Canonical Markdown SHA-256：`e51749e5b199b94da24d237cb1cbf3c5650b6f99aaaf234150bb3e8df95dac21`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_06_perception_grounding_navigation_coverage_gap_audit.md`

VOL 06 Perception / Grounding / Navigation：Coverage-Gap Audit

审计日期：2026-07-20 状态：`coverage_contract_frozen / planned_20 / sources_19 / knowledge_19 / cards_19 / per018_independent_fact_reviewed / per019_triplet_synced / missing_sources_1 / missing_knowledge_1 / missing_cards_1 / WARN` 审计对象：VOL 06 已有 source/knowledge/card、全局 Knowledge Map、Repository Completion Audit、GitHub skeleton guidance、五项目 evidence inventory 与导航 artifact gate。 核心边界：本报告冻结整卷范围与退出条件；不把 planned/source/knowledge-only ID 计入题库，不生成占位卡，不把公开 perception/navigation 机制写成个人项目成果。

问题（Question）

现有 `VLA-PER-001..005` 是否足以支持“VOL 06 Perception / Grounding / Navigation 已完成”？如果不足，缺失的是哪些不可合并的知识节点、来源门、项目证据门和真实训练门?

1. 结论先行

1. 二十个节点覆盖 image/token contract 与 grounding representation → 3D lift → multi-view/time → VLN/ObjectNav protocol → navigation architecture → open-vocabulary grounding → matching/loss → metric/parser → pseudo-label/noise/calibration/leakage → segmentation/depth/keypoint/affordance → point/voxel/BEV/occupancy → 3D grounding/spatial relation/open-vocabulary 3D scene → perception/action feasibility → localization/odometry/SLAM/map-frame/drift → belief/memory/frontier/active-exploration → navigation BC/DAgger/RL/exposure bias → navigation evaluation/failure → robustness/safety主链；`PER-001..020`已有source/knowledge/card，二十张卡均为`unseen/WARN`。
2. image preprocessing/token contract由PER-006独立承载；open-vocabulary inference由PER-007承载；PER-008..018均已按独立source→knowledge→card切片同步。PER-018独立事实复核重新命中7/7 PDF、10/10 official files、10/10 probes、8/8 local files，并确认R2R student-forcing的限定语境与pinned Habitat done-mask版本差异；完整评估与robustness/safety仍没有独立节点。
3. VOL 06 整卷冻结为 **20 个原子节点**：当前source为`20/20=100%`，knowledge为`20/20=100%`，card为`20/20=100%`；PER-001..020三联件同步，二十张题卡保持`unseen`。

4. 五项目中有 interface/provenance hook，但 PER-004 Navigation Artifact Gate 重算为 `0` 个 local-direct VLN/ObjectNav episode bundle；因此不得产生个人导航成功率、SPL 或真实机器人 claim。
5. 覆盖完成、事实审查、项目证据、HTML 生命周期和口述 mastery 分开计数。即使未来达到 20/20 页面覆盖，没有真实回答、visual QA 或项目 artifact 时也不能写成 `drilled/done`。

2. 本次审计依据

依据	实际支持	不能支持
GitHub Source Scout	WinstonJQ 骨架规则：主索引、分卷、Markdown canonical、折叠题卡、review、L1/L2/L3、题目前置清单	具体 VLA/perception 技术事实或题目频次
VLA Knowledge Map	VOL 06 范围为 image/token、point/box/text、3D geometry、multi-view、VLN/ObjectNav、grounding quality	现有五页已经覆盖所有原子问题
Repository Completion Audit	当前全局276 cards；VOL 06有20个完整节点，整卷退出条件仍未满足	planned ID、source或knowledge自动成为已生成题卡
PER-001 Source Freeze	point/box/text、parser、坐标逆变换、OPD truth boundary	3D、action success 或个人导航结果
PER-002 Source Freeze	calibration/depth/back-projection/frame/TF 合同	本地 3D round-trip 或 controller success
PER-003 Source Freeze	camera/frame/time/presence 与 fusion schema	本地 multi-camera calibration、sync 或 ablation
PER-004 Source Freeze	VLN/ObjectNav/REVERIE task、episode、stop、SR/SPL 协议	用户复现 benchmark 或真实导航
PER-005 Source Freeze	map-free/map-based/waypoint/planner/controller taxonomy	用户实现、matched ablation 或 ownership
PER-008 Source Freeze / Knowledge / Card	point/box/mask/semantic-keypoint/affordance语义、选择、转换损失与OPD边界	local dense affordance、matched ablation或robot result
PER-009 Source Freeze / Knowledge / Card	Hungarian/token negatives/focal/L1/GIoU/reduction、本地CE/top1/KL、typed manifest、面试诊断与2500行metrics边界	first-party criterion、checkpoint/holdout/robot绑定
PER-010 Source Freeze / Knowledge / Card	strict/lenient parser、IoU/Acc、RefExp/COCO、PCK/mask-hit/fixed-distance Hit、denominator、protocol identity、面试诊断与项目边界	raw prediction reconciliation、parser v2 rerun与robot结果
PER-012 Source Freeze / Knowledge / Card	segmentation task、relative/metric depth、3D surface、semantic keypoint、primitive-conditioned affordance、typed composition、atomic answer与OPD point-only边界	local dense checkpoint、metric-depth/camera bundle、planner/controller episode或robot success
PER-013 Source Freeze / Knowledge / Card	PointNet++/VoxelNet/PointPillars/Minkowski/BEVFusion/occupancy contracts、九个probe、教学诊断、atomic answer与Harness 2D-map/PCD boundary	local learned 3D encoder、matched benefit、robot result或ownership
PER-014 Source Freeze / Knowledge / Card	ScanRefer/ReferIt3D/OpenScene/OpenMask3D/LERF/ConceptFusion、六类output、typed manifest、relation、score/metric、diagnostic、155/335字符atomic answer与五项目真实性边界	local 3D semantic head/map/checkpoint/raw query/robot result或ownership
PER-015 Source Freeze / Knowledge / Card	Modern Robotics/MoveIt/OMPL/TRAC-IK/CLIPort/KnowNo、typed action manifest、九层gate、10个probe、first-divergence diagnosis、141/257字符atomic answer与五项目direct audit	本地完整gate/runtime、robot success或ownership
PER-016 Source Freeze / Knowledge / Card	ORB-SLAM3/VINS-Mono/Cartographer/DROID-SLAM/OpenVINS/Kimera、REP-105、typed manifest、state/covariance、pose graph、drift、ATE/RPE、10个probe、179/350字符回答与Harness map/PCD真实性边界	本地SLAM estimator/runtime、trajectory bundle、robot localization结果或ownership
PER-017 Source Freeze / Knowledge / Card	POMDP/Bayes/zero-likelihood、三类memory、typed manifest、known-free/unknown frontier、IG/utility、revisit/stuck、metric contract、10 probes、172/315字符回答、诊断与五项目direct audit	local exploration runtime/episode、checkpoint/result或ownership

依据	实际支持	不能支持
PER-018 Source Freeze / Knowledge / Card / Independent Review	BC/forcing/DAgger/GAE/PPO-DDPPO、7 PDFs/96页、10 official files rehash、10 probes、8 local files、两档回答、诊断与单张unseen题卡	local navigation DAgger/PPO runtime、matched episode/result、ownership与真实口述
PER-019 Source Freeze	stop-gated success、POINT/VIEW_POINTS geodesic、SPL/SoftSPL、split/denominator、failure taxonomy、10 source probes与五项目truth boundary	local navigation bundle、统一scorer、per-episode trace、个人导航结果或ownership
PER-020 Source Freeze	corruption family/severity、clean-shift protocol、dataset-shift uncertainty、risk-coverage、queue flush、fallback/recovery/E-stop、10 source probes与五项目truth boundary	local shifted-input bundle、逐step risk log、hardware E-stop audit、个人OOD或安全结果
OPD Claim–Evidence Inventory	local point/box/text route、scorer 与 leakage 边界	3D grounding、closed-loop manipulation 或 navigation success

3. 当前资产盘点

层	已有	目标	覆盖	仍开放的门
generated question cards	20	20	100%	`PER-001..020` 已生成且unseen
source freeze	20	20	100%	PER-001..019与PER-020 source已冻结；PER-020 knowledge/card仍后置
knowledge nodes	20	20	100%	PER-001..020已生成；全部对应题卡保持unseen
question card	20	20	100%	PER-001..020均有单张原子卡；仍未进行真实口述
oral drill	0	最少 8	0%	二十张已有卡仍全部 `unseen`
local-direct navigation bundle	0	不强制有正结果，但必须保持 truth boundary	—	episode/scene/start/goal/action/denominator/failure trace 缺失
browser desktop/narrow QA	0	当前卷页面均需验收	0%	`file://` 策略阻断；静态检查不能替代视觉 QA

4. 整卷 Stable-ID 合同

权威题目清单为 VOL 06 Phase 0 Question List。规划 20 个节点，按 12 个 P0、8 个 P1 分层；以下只给出聚类，不重复清单全文。

学习簇	IDs	现有 / 规划	核心缺口
输入与 2D grounding	`001,006..012`	8 complete / 8	独立fact review、local dense artifact与真实drill仍开放
3D、多视角与动作接口	`002,003,013..015`	5 complete / 5	独立fact review、本地physical-gate runtime与真实drill仍开放
导航任务、地图与训练	`004,005,016..018`	5 triplets / 5	学习簇覆盖完成；独立fact review与真实drill仍开放
评估、鲁棒性与安全	`019,020`	2 source + 2 knowledge + 2 cards / 2 nodes	PER-019/020 source+knowledge+card均同步；真实口述仍缺

依赖主链：

```
# Planned IDs are registry entries, not generated mastery.
PER-006 image/token
  -> PER-007 open-vocab grounding
  -> PER-008 representation choice
  -> PER-009 loss/matching -> PER-010 metric/parser -> PER-011 pseudo-label/noise
  -> PER-012 segmentation/depth/keypoint/affordance

PER-002 3D lift + PER-003 multi-view
  -> PER-013 3D representation -> PER-014 3D grounding
  -> PER-015 perception-to-action gate

PER-004 task contract + PER-005 policy architecture + PER-016 SLAM
  -> PER-017 memory/exploration -> PER-018 IL/RL training
  -> PER-019 evaluation/failure -> PER-020 robustness/safety
```

5. Source Queue 与冻结边界

下表是后续逐节点 source freeze 的检索队列，不表示已经核验完成。只有读取 primary paper、official code/docs 或本地 artifact 并记录 revision 后，才能进入知识页。

IDs	待冻结的来源类型	本仓库可复用入口	当前状态
006	VLM/ViT 官方 image processor、resize/crop/patch/position contract	GitHub Source Scout 中的 Qwen3-VL frozen repo；VOL 01/03 vision-token pages	`source + knowledge + card synchronized / unseen`
007	Grounding/open-vocabulary detection primary paper与official code；matching、two-threshold、phrase与NMS boundary	PER-001 Grounding DINO；PER-006 image contract；OPD inventory	`source + knowledge + card synchronized / unseen`
008	point/box/mask/keypoint/affordance primary papers与official output contracts	PER-001/007；RoboPoint/SAM/kPAM/Where2Act；OPD local point evidence	`source + knowledge + card / unseen`
009	grounding loss/matching primary/official sources	PER-007/008 source gates；GroundingDINO/DETR/HF；OPD train artifacts	`source + knowledge + card synchronized / unseen`
010	grounding metric/parser primary/official sources	PER-001；PER-007/008 source gates；OPD scorer inventory	`source + knowledge + card synchronized / unseen`
011	noisy/pseudo-label、confidence calibration、curriculum与data leakage primary/official sources	FixMatch/Noisy Student/calibration/leakage来源、OPD mix/cache/online/holdout	`source+knowledge+card synced / unseen`
012	segmentation/depth/keypoint/affordance concrete methods	Mask2Former/SAM/Depth Anything V2/kPAM/Where2Act、OPD point-only audit	`source+knowledge+card synced / unseen`
013	point-cloud/voxel/BEV/occupancy representation与sampling	PointNet++/VoxelNet/PointPillars/Minkowski/BEVFusion/Occupancy Networks/OctoMap、Harness map/PCD	`source+knowledge+card / unseen`
014	3D grounding、spatial relation、open-vocabulary 3D scene primary/official sources	ScanRefer/ReferIt3D/OpenScene/OpenMask3D/LERF/ConceptFusion；PER-007/013；五项目inventories与direct exceptions	`source + knowledge + card synchronized / unseen`
015	reachability/uncertainty primary sources	PER-002/003/013/014、Modern Robotics/MoveIt/OMPL/TRAC-IK/CLIPort/KnowNo、VOL 03 action contract	`source+knowledge+card synced / unseen`
016	SLAM/odometry/map-frame/drift primary paper、official code与REP-105 frame contract	ORB-SLAM3/VINS-Mono/Cartographer/DROID-SLAM/OpenVINS/Kimera；Harness map/PCD/reloc assets	`source + knowledge + card synced / unseen`

IDs	待冻结的来源类型	本仓库可复用入口	当前状态
017	POMDP belief、active exploration/memory、frontier、coverage/revisit primary + official code	Kaelbling/Yamauchi/Neural-SLAM/OccupancyAnticipation/SemExp/PONI/Exploring Exploration/EmbodiedQA；PER-004/005/016	source + knowledge + card synced / unseen
018	VLN IL/RL、teacher forcing与exposure bias primary + official benchmark code	DAgger/Scheduled Sampling/R2R/VLN-CE/DD-PPO/PPO/GAE；VLN-CE/Habitat/R2R official code；PER-004/005与VOL 02	source + knowledge + card synchronized / unseen
019	official navigation measures、SPL/benchmark protocol、statistical reporting	PER-004、VOL 05 evaluation、PER-004 artifact gate	source + knowledge + card synchronized / unseen
020	sensor/domain randomization、uncertainty/safety/fallback primary + simulator docs	VOL 04 Sim2Real Audit D、VOL 05 failure taxonomy、Hendrycks/Ovadia/Safety Gym	source + knowledge synchronized / card pending

6. 项目挂钩与真实性边界

项目	可用 hook	当前不能说
OPD	point/box/text route、affordance-labelled point supervision、Visual Genome 2D image spatial-relation数据与strict/protocol-fixed/lenient scorer	3D relation frame、3D semantic grounding、closed-loop grasp/navigation、单因素 uplift 或未确认 ownership
Robot Harness	semantic-navigation tool schema、2D tri-state occupancy、non-semantic PCD与tool-call/eval interface	language-queryable 3D map、personal VLN/ObjectNav policy、physical navigation episode 或 success/SPL
OpenPiVLA	image/language/action schema、normalization/controller contract 风险	使用 grounding/SLAM/navigation module，或 formal rollout 已闭环
WAM	camera/controller/provenance 与有限 paired diagnostic	multi-view fusion、3D target、navigation success 或真机 Sim2Real
UWM / LIBERO	simulated manipulation protocol与 camera/action schema	VLN/ObjectNav、mobile navigation、跨协议真实泛化

面试回答必须按 `public upstream mechanism → local interface/evidence → inference → unverified` 分层。Git author、official example、字段名或工具描述不能自动成为个人贡献。

7. VOL 06 退出条件

7.1 Coverage complete

1. `VLA-PER-001..020` 全部有唯一 source/knowledge/card 三联件，卷级 question list 与索引同步。
2. 每张卡包含 stable ID、难度/优先级、短答、机制展开、来源、项目挂钩、易错点、追问与练习状态。
3. P0/P1 节点均有 primary/official source revision；discovery list 或跨卷引用不能单独支撑强技术答案。

7.2 Reviewed

1. 20 个节点的 canonical/HTML/review SHA、单 H1、ID、锚点、公式、表格和本地链接通过。
2. 四个学习簇各完成至少一个独立 fact-review slice；高风险 metric、3D frame、navigation stop/success 与 project truth boundary 必须覆盖。
3. Project hook 只引用 evidence inventory/ownership ledger；缺本地 bundle 时明确说“没有直接证据”。

7.3 Drilled

1. 至少完成 8 个真实口述 drill，四个学习簇各至少 1 题，其中至少 2 题带项目追问。
2. 只有真实回答后才创建/更新 coaching state；状态变化必须有回答、反馈、日期和 next review。
3. 覆盖完成或 HTML 生成不改变 `unseen/weak/pass/strong/stale`。

7.4 Visual / evidence boundary

1. 支持的本地页面表面恢复后，对卷级入口与代表性原子页做 desktop/narrow QA；当前 `BLOCKED` 不能写成 PASS。
2. 不强制虚构正向导航结果；如果没有 local-direct bundle，退出时应保留“无个人导航结果”的真实边界。

8. 缺口优先级与下一切片

优先级	缺口	原因	下一动作
P0-1	真实口述与 navigation artifact	二十张卡均 unseen，local-direct bundle 仍为0	先做一题真实口述或收集 immutable episode bundle

下一 bounded slice：只做一题真实口述或独立 fact review；不修改任何mastery。local-direct navigation/robustness bundle继续保留为证据队列。

参考入口（References）

1. VOL 06 Phase 0 Question List
2. GitHub Source Scout
3. VLA Knowledge Map
4. Repository Completion Audit
5. PER-001 Source Freeze
6. PER-002 Source Freeze
7. PER-003 Source Freeze
8. PER-004 Source Freeze
9. PER-005 Source Freeze
10. PER-004 Navigation Artifact Gate
11. OPD Claim–Evidence Inventory
12. PER-007 Open-vocabulary Grounding Pipeline Source Freeze
13. PER-007 Open-vocabulary Grounding Pipeline Knowledge
14. PER-007 Open-vocabulary Grounding Atomic Card
15. PER-008 Representation Choice Source Freeze
16. PER-008 Representation Choice Knowledge
17. PER-008 Representation Choice Atomic Card
18. PER-009 Grounding Matching / Negative / Loss Source Freeze

19. PER-009 Grounding Matching / Negative / Loss Knowledge
20. PER-009 Grounding Matching / Negative / Loss Atomic Card
21. PER-010 Grounding Metric / Parser / Protocol Source Freeze
22. PER-010 Grounding Metric / Parser / Protocol Knowledge
23. PER-010 Grounding Metric / Parser / Protocol Atomic Card
24. PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Source Freeze
25. PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Knowledge
26. PER-011 Pseudo-label / Teacher Noise / Calibration / Leakage Atomic Card
27. PER-012 Segmentation / Depth / Keypoint / Affordance Source Freeze
28. PER-012 Segmentation / Depth / Keypoint / Affordance Knowledge
29. PER-012 Segmentation / Depth / Keypoint / Affordance Atomic Card
30. PER-013 Point Cloud / Voxel / BEV / Occupancy Source Freeze
31. PER-013 Point Cloud / Voxel / BEV / Occupancy Knowledge
32. PER-013 Point Cloud / Voxel / BEV / Occupancy Atomic Card
33. PER-014 3D Grounding / Spatial Relation / Open-vocabulary 3D Scene Source Freeze
34. PER-014 3D Grounding / Spatial Relation / Open-vocabulary 3D Scene Knowledge
35. PER-014 3D Grounding / Spatial Relation / Open-vocabulary 3D Scene Atomic Card
36. PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Source Freeze
37. PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Knowledge
38. PER-015 Perception→Action / Reachability / Uncertainty / Failure Gate Atomic Card
39. PER-016 Localization / Odometry / SLAM / Map-frame / Drift Source Freeze
40. PER-016 Localization / Odometry / SLAM / Map-frame / Drift Knowledge
41. PER-016 Localization / Odometry / SLAM / Map-frame / Drift Atomic Card
42. PER-017 Belief / Memory / Frontier / Active Exploration Source Freeze
43. PER-017 Belief / Memory / Frontier / Active Exploration Knowledge
44. PER-017 Belief / Memory / Frontier / Active Exploration Atomic Card
45. PER-018 Navigation BC / DAgger / RL / Exposure Bias Source Freeze

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 07 · Personal Project Deep Dives

五项目实现证据、真实性边界、失败链与面试口述

本卷由 49 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

来源与事实复核

1. 01 · VOL 07：五项目证据、Ownership 与面试叙事 Reconciliation
2. 22 · UWM libero_90.pt 本地 Immutable Checkpoint Manifest Gate
3. 23 · VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结
4. 30 · OpenPiVLA Action JSON Parser / Round-trip Artifact Gate
5. 31 · OpenPiVLA ALOHA Action-Level Eval：Checkpoint Load / Serve / L1 Gate
6. 32 · OpenPiVLA 本地 Artifact Gate：JSON/CE 动作表示与 H/D shape contract
7. 33 · OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate
8. 34 · OpenPiVLA / LIBERO Matched Failure Probe Readiness Gate
9. 35 · OpenPiVLA Official Checkpoint Immutable Manifest Gate
10. 36 · OpenPiVLA 本地 Runtime Rollout Artifact Gate：ALOHA 与 LIBERO

口述与练习

1. 02 · 五项目串联：从实现证据反推的 VLA 项目总述口述稿
2. 03 · PROJ-WAM-001/002 从项目实现反推的证据约束口述稿
3. 16 · PROJ-UWM-001..004 从项目实现反推的证据约束口述稿
4. 24 · PROJ-OPENPIVLA-001 + Batch 2 从项目实现反推的证据约束口述稿
5. 37 · PROJ-HARNESS-001..008 从项目实现与 P0 Audit 反推的证据约束口述稿
6. 44 · PROJ-OPD-001..009 从项目实现反推的证据约束口述稿

题卡

1. 04 · PROJ-WAM：Cosmos Policy / WAM 项目原子题卡（Training Failure + Diagnostic + Ownership Batch 1）
2. 17 · VOL 07 UWM / LIBERO 项目原子题卡（Protocol Boundary Batch 1 + Ownership）
3. 25 · PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 1）
4. 26 · PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 2）
5. 38 · PROJ-HARNESS：Robot Harness 项目原子题卡（Direct-Evidence Batch 1）
6. 45 · PROJ-OPD：OPD / VLM Grounding 项目原子题卡（Direct-Evidence Batch 1 + Ownership）

项目证据

1. 05 · Cosmos / WAM Claim–Evidence Inventory（WAM-01）
2. 06 · Cosmos / WAM Ownership Ledger：多目标训练、RoboCasa 评测与诊断边界
3. 07 · WM-019 WAM P0 Bundle Audit：CoffeePressButton
4. 08 · Cosmos Policy full10k trained checkpoint：RoboCasa 24×5 P0 评测证据审计
5. 09 · Cosmos Policy RoboCasa Official-HF Baseline：P0 评测证据审计
6. 10 · Cosmos Policy TurnOffMicrowave 本地训练失败：P0 First-Divergence 审计
7. 11 · Cosmos Policy / WAM Checkpoint Save-Load Gate：P0 证据审计
8. 12 · WM-019 WAM Diagnostic Artifact Manifest（CoffeePressButton）
9. 13 · WM-019 WAM Action-Trace Paired Diagnostic（CoffeePressButton）
10. 14 · WM-019 WAM Future-Image Metric Paired Diagnostic（CoffeePressButton）
11. 15 · WM-019 WAM Matched-Checkpoint Protocol Bridge（iter_000010000）
12. 18 · UWM / LIBERO Claim–Evidence Inventory（UWM-01）
13. 19 · UWM / LIBERO Ownership Ledger：协议迁移、数据管线与结果边界
14. 20 · UWM / LIBERO P0 Bundle Audit：Protocol / Data Pipeline / Remote Result
15. 21 · UWM RoboCasa ActionHeadFix 30 Epoch：Train-to-Closed-Loop P0 证据审计
16. 27 · OpenPiVLA Claim–Evidence Inventory（OPV-01）
17. 28 · OpenPiVLA Ownership Ledger：个人贡献、文件归属与面试口径
18. 29 · OpenPiVLA P0 Bundle Audit：Formal Trainer / Action Route / Eval Lineage
19. 39 · Robot Harness Claim–Evidence Inventory（RH-01）
20. 40 · Robot Harness Ownership Ledger：Owner 决策、Codex 实现与结果归属
21. 41 · Robot Harness P0 Bundle Audit：Runtime / Post-training / Eval
22. 42 · Robot Harness 五/八模型 × 十套件聚合证据门
23. 43 · Robot Harness 四模型共同支持集 Raw Bundle Manifest
24. 46 · OPD / VLM Grounding Claim–Evidence Inventory（OPD-01）
25. 47 · OPD / VLM Grounding Ownership Ledger：数据、路由、训练与结果归属
26. 48 · OPD / VLM Grounding P0 Bundle Audit：Data Route / Distillation / Holdout
27. 49 · OPD Training Objective Lineage Gate：top1 CE、Full-Vocabulary KL 与 Rollout Source

Part 1 / 49 · `interview_materials/vla/sources/vol_07_project_evidence_reconciliation.html` · SHA-256 `CE54FFACAE691069894B64398D5F4E1C5CF5CF1143F9255F1C4B1EE0609C069A`

返回本卷目录

Canonical Markdown SHA-256：`ce54ffacae691069894b64398d5f4e1c5cf5cf1143f9255f1c4b1ee0609c069a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_07_project_evidence_reconciliation.md`

VOL 07：五项目证据、Ownership 与面试叙事 Reconciliation

状态：`project_evidence_reconciliation / VOL-07 / inventories_5_of_5 / ownership_ledgers_5_of_5 / p0_audits_5_of_5 / cards_37_unseen / WARN` 审查日期：2026-07-21 范围：只汇总五个项目现有的 claim–evidence inventory、ownership ledger、诊断 artifact 与项目题卡；本轮补入 UWM RoboCasa ActionHeadFix 30-epoch P0 audit（此前已纳入 Cosmos full10k trained 24×5 P0 audit），但不改变 280 张题卡的 `unseen` 状态、不创建 `coaching_state.md`，也不把 summary、Git 作者或上游机制升级为个人贡献。

问题（Question）

五个项目目前分别有不同强度的代码、配置、报告、远端引用和诊断 artifact。怎样把它们统一成一套可面试复述的证据合同：每个 claim 的来源是什么、候选人能说到哪一层、还缺什么 P0 artifact、如何回答高压追问?

1. 结论先行

1. **五个项目的 claim inventory 与 ownership ledger 均已存在**，但它们解决的是“登记与边界”，不是“所有项目结果已证实”。
2. **项目题卡共 37 张**：OPD 9、OpenPiVLA 12、Robot Harness 8、WAM 4、UWM/LIBERO 4；全部保持 `unseen`。
3. **证据强度不对称**：OpenPiVLA 的 local-direct 代码/recipe 最完整；Robot Harness 有较多工程与 raw partial bundle；OPD 有静态数据/评估链；UWM 有 user-confirmed remote final 但无 raw checkpoint/episode bundle；WAM 只有有限诊断 artifact，历史强结果已安全降级。
4. **所有项目都必须使用四层叙事**：`public upstream mechanism → local evidence/interface → inference → unverified`。其中 inference 不能被口述成 measured result，unverified 不能被包装成 ownership。
5. **五项目 P0 audit coverage 已完成 5/5，但 P0 closure 尚未完成**：WAM、OpenPiVLA、Robot Harness、OPD、UWM/LIBERO 均有独立 P0 审计页；这些页面确认了现有证据和缺口，不等于 checkpoint/raw/scorer/ownership 已闭合。VOL 07 仍缺候选人逐文件 ownership 确认、P0 immutable bundle、真实口述和项目级 matched evaluation，因此 verdict 继续为 `WARN`。

2. 统一证据合同

2.1 Claim 的四层标签

层级	可以说什么	不可以说什么	最小证据
`public_upstream`	论文、官方 repo 或官方配置定义的机制	把机制说成自己设计或自己的结果	固定版本 URL/commit + 可定位文件
`local_evidence`	本地代码、配置、日志、manifest、trace 实际显示的内容	从文件名、Git author 或 summary 推断完整 ownership	文件路径、SHA、字段、运行/生成关系
`inference`	基于局部证据提出的诊断或设计解释	把假设写成 root cause、uplift 或 benchmark	明确假设、反例、待验证实验
`unverified`	当前缺失或冲突的 checkpoint、raw bundle、角色确认	在面试中用“应该/大概/项目里有”填空	明确缺口、下一步 artifact

2.2 叙事最小单元

每个项目的 2 分钟回答都按以下顺序组织：

1. **问题与约束**：任务、输入输出、时间/算力/安全约束；
2. **个人边界**：我直接改动/验证/决策的文件或接口；
3. **机制**：数据、模型、loss、评估或部署环节的因果链；
4. **证据**：可复核 artifact、协议、分母和失败样本；
5. **反例与下一步**：当前不能证明什么，以及若继续工作要补的最小实验。

没有第 4 步的 raw evidence 时，应主动降级为“设计/接口/诊断经验”，不要补造数字。

3. 五项目 Reconciliation Matrix

项目	Inventory / ownership	当前可核验的 local evidence	当前安全叙事	Po 缺口	题卡覆盖
OpenPiVLA / OpenPI / LIBERO / MACA	claim inventory；ownership ledger；Po audit	formal trainer、Qwen3-VL + PEFT LoRA、JSON/causal-CE、DROID/MindCube recipe、smoke/report/diagnostic artifacts；部分 bundle 为 remote-referenced 或 archive-summary-only	可以讲数据/格式/训练器/解析器的实现边界，以及如何做 formal-vs-smoke 分层	immutable checkpoint、raw benchmark denominator、archive→formal lineage、processor/normalizer/controller round-trip、逐文件 ownership	`PROJ-OPENPIVLA-001..012`，12 unseen
Robot Harness	claim inventory；ownership ledger；Po audit；raw manifest	tool schema、PydanticAI/runtime、mutex/cancel/compensation、SFT/GRPO-style 路由、reward/constraint 设计；raw recovery 28/40，common support 6/10	可以讲问题定义、约束取舍、数据/reward/eval 设计与 Codex 协作边界；不能把 proxy 100% 当物理成功	余下 raw files、checkpoint identity、matched scorer/holdout、cancel/fallback trace、角色与实现分工确认	`PROJ-HARNESS-001..008`，8 unseen
OPD / VLM Grounding	claim inventory；ownership ledger；Po audit	tagged point/box/text SFT→generation→parser/scorer、teacher/route、raw-holdout 10k 与 checkpoint-5000 邻接证据；静态 processor/geometry 风险	可以讲数据路由、teacher/student、parser/scorer 和 protocol 选择；组合收益不能归因于单一 OPD 因素	weights/optimizer/RNG、统一 ablation、raw prediction bundle、point protocol fixed scorer、账号身份与逐文件 ownership	`PROJ-OPD-001..009`，9 unseen
Cosmos Policy / WAM / RoboCasa	claim inventory；ownership ledger；Po audit；artifact manifest；full10k trained eval；official-HF baseline；save-load gate	CoffeePressButton episode-0 base/trained manifest、action trace、未来帧诊断、DeepSpeed save gate、full10k trained 24×5 raw/trusted summary、official-HF baseline audit；不同 run family 分层	可以讲 slot/mask/loss、失败 trace、保存门、eval harness 与 raw/trusted denominator；不能把 official `84/118` 归为本地训练，或恢复历史 uplift/root-cause 强 claim	binary checkpoint、fresh reload、完整 episode/metric denominator、matched scorer、训练/rollout lineage、candidate ownership	`PROJ-WAM-001..004`，4 unseen
UWM / LIBERO	claim inventory；ownership ledger；Po audit；ActionHeadFix 30-epoch audit	LIBERO route、fixed-pruned-init/reset-style 协议、DataLoader stall smoke；RoboCasa一条8卡3930-step run与8×54 raw rollout已冻结，full-h4 `0/54` /contact `45/54`；`186/250` 仍为user-confirmed remote final	可以讲 protocol 对齐、数据/normalizer/评估口径、train→closed-loop 断层和 failure diagnosis；strict/reset/official与相邻RoboCasa run不能混比	produced checkpoint binary/fresh reload、LIBERO 250 episodes、RoboCasa env/scorer revision/videos、matched repeat、角色确认	`PROJ-UWM-001..004`，4 unseen

4. 项目间共性风险

4.1 结果分母与 protocol

任何成功率、准确率、SPL、point hit 或 proxy 分数都必须绑定：

`metric = (checkpoint, dataset/scene split, init/reset, seed, horizon, scorer, denominator, invalid policy)`。

只要其中一项来自另一条实验链，回答就应改成“不可直接比较”。尤其是：

- UWM `186/250` 不等于官方 Table II 或真机成功；
- OpenPiVLA `97.4%` 缺 raw denominator 时不计算 CI；
- OPD 10k 是 metric-specific holdout unit，不是 robot rollout success；
- Robot Harness proxy 100% 不是 physical safety 或 browser-complete；
- WAM trace geometry 不是训练 uplift。

4.2 Ownership 证据

以下材料不能单独证明个人贡献：Git author、文件名、目录位置、远端 summary、上游 repo、自动生成报告、协作工具默认提交者。安全回答应拆成：

```
我能直接指向的改动/决策
→ 我实际验证过的输入输出或 trace
→ 我根据证据做出的推断
→ 仍需本人/团队确认的 ownership 或结果
```

4.3 失败诊断顺序

五个项目统一采用 `symptom → first verified divergence → root-cause hypothesis → recovery/next experiment`。如果没有 raw trace，只能停在 `unknown / not recomputable`，不能用时间先后、日志摘要或“训练完成”替代 first divergence。

5. 面试高压追问入口

追问	安全回答骨架
“你具体改了哪几个文件？”	先给可定位的 local-direct 文件与接口，再说明协作/上游/远端部分；不能用 Git author 代替 ownership
“这个数字是你跑的吗？”	说清 local-direct、user-confirmed、remote-referenced、archive-summary-only 四类 provenance；缺 raw bundle 就不称为可复现本地结果
“为什么不是数据配比导致的？”	给 matched ablation 设计、固定 checkpoint/seed/denominator；当前没有就承认 inference
“失败根因是什么？”	指出 first verified divergence；没有 trace 就保留 unknown，并给最小补证实验
“你如何证明上线安全？”	说 schema/constraint/logging/fallback 设计与实际 safety audit 的差别；不把 simulator/proxy/上游机制当真机安全
“如果重做一次会先做什么？”	选择一个 P0 immutable artifact：checkpoint hash、raw episode JSONL、matched scorer、seed/init manifest 或 ownership confirmation

6. 面试回答模板（不含虚构数字）

6.1 30 秒总述

> 我把项目回答拆成问题约束、我直接负责的接口或决策、机制链和可复核证据四层。现在五个项目都有 claim inventory 和 ownership ledger，但证据强度不同：OpenPiVLA 偏代码/recipe，Harness 偏工程与 partial raw bundle，OPD 偏静态数据和 scorer，UWM 有 protocol-limited remote result，WAM 主要是 diagnostic artifact。没有 checkpoint、raw episode、matched denominator 或逐文件确认的地方，我会明确说是 inference 或 unverified，不把上游机制和 summary 写成个人结果。

6.2 2 分钟展开

> 先固定任务输入输出、split/init/seed/horizon/scorer，再定位我实际改动或验证的文件。然后沿 data→model/loss→parser/controller→evaluation trace 解释机制；结果只引用带 checkpoint、分母和 raw artifact 的记录。遇到失败，我先找 first divergence，再区分数据、表示、几何/时间、模型、控制和协议问题。当前缺少 immutable bundle 的项目，我会把答案收敛到接口设计、诊断方法和下一步补证，而不是声称 uplift、sim-to-real 或安全闭环已经完成。

7. 退出条件与下一步

本切片完成了五项目的统一 reconciliation，但 G09 仍未完成。退出条件还包括：

1. 候选人逐文件 ownership 确认；
2. 每个高风险项目至少一个 P0 immutable bundle；
3. 一题真实项目口述与追问记录；
4. 对关键数字建立 checkpoint/protocol/denominator 对齐；
5. 不把本页 coverage 当作 mastery。

下一 bounded slice：优先进行一题项目口述（建议 WAM 或 OpenPiVLA），或在用户提供新 artifact 后只审计一个 P0 bundle；不批量新增项目题卡。

References

1. OpenPiVLA Claim–Evidence Inventory 与 Ownership Ledger。
2. Robot Harness Claim–Evidence Inventory、Ownership Ledger 与 Raw Bundle Manifest。
3. OPD Claim–Evidence Inventory 与 Ownership Ledger。
4. WAM Claim–Evidence Inventory、Ownership Ledger 与 WM-019 Artifact Manifest。
5. UWM/LIBERO Claim–Evidence Inventory、Ownership Ledger 与 P0 Audit。
6. 五项目 P0 Artifact Intake Contract。
7. VOL 04 Audit C Cosmos/WAM 与 Audit D Sim2Real。

Part 2 / 49 · `interview_materials/vla/practice/project_portfolio_oral_model_answer.html` · SHA-256 `1423BA25C37FC767C5DA105D321398B0FCA2FCF8B038FFEBC31C3E2A05D4706C`

返回本卷目录

Canonical Markdown SHA-256: `1423ba25c37fc767c5da105d321398b0fca2fcf8b038ffebc31c3e2a05d4706c`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\practice\project_portfolio_oral_model_answer.md`

五项目串联：从实现证据反推的 VLA 项目总述口述稿

状态：`implementation_derived_portfolio_model_answer / evidence_bounded / not_candidate_transcript / unseen` 更新日期：2026-07-22 适用场景：面试开场“介绍一下你做过的 VLA/具身项目”、项目选型、跨项目追问

1. 30 秒总述

我的项目经历可以按一条 VLA 工程链来讲，而不是五个互不相关的 demo。Cosmos Policy/WAM 侧重训练入口、在线文本条件和 RoboCasa 闭环评测；UWM/LIBERO 侧重数据窗口、task-specific checkpoint、normalizer 与 episode 分母；OpenPiVLA 侧重多模态 causal fine-tune、LoRA 和 action formatter 的 shape/lineage 风险；Robot Harness 侧重 robot scope、mutex、cancel、compensation 以及 schema-aware reward；OPD 侧重多 expert 数据合同、路由蒸馏和 holdout 评测。我的共同工作方式是先冻结输入输出和评测协议，再从日志、raw artifact 和 first-divergence 定位问题。现有材料能证明实现接入、运行审计和失败诊断，但不能把不同 checkpoint、scorer 或 denominator 拼成跨项目的统一提升。

2. 2 分钟主线

我会把这五个项目看成从 policy 到 evaluation 再到执行协议的五个切面。 第一条是 Cosmos Policy/WAM。我接入了 official trainer/model/dataset 路径和 online Reason1 conditioning，处理 placeholder 与 lazy success metadata，另外把 local DCP checkpoint loader 接回 RoboCasa 官方闭环。训练上有一条单卡 full10k run，日志到 step 10000、loss= `0.0543` 并记录了保存成功；另一条 8 卡、global batch 128、 `9907` -step 的 1-epoch gate 则先遇到 `cycle(dataloader)` 引起的 CPU cgroup OOM，修复后的 5-step smoke 通过，但正式 run 又在 DeepSpeed `state_dict(destination=...)` 保存接口失败，所以我不会说 1 epoch gate 已完成。评测固定 24 个任务、每任务 5 个 episode、chunk 32、执行前 16 步；full10k 的 trusted 结果是 `0/118` ，同一 harness 的 official-HF path 是 `84/118` ，这只能缩小排查范围，不能写成训练 uplift。 第二条是 UWM/LIBERO。我从 HDF5 转 Zarr 的窗口、 `process_batch` 、DDP trainer 和 epoch-aligned queue 追到 evaluator。这里最容易讲错的是把五个任务说成一个 multi-task checkpoint；实现和日志显示它们使用 task-specific checkpoints，且 evaluator 只读取 `ckpt["model"]` ，没有读取 `action_normalizer` 。用户确认的 `186/250` 是 summary-level 结果，本地缺 250 条 raw episode 和 checkpoint/hash，因此我把 strict/reset、completed denominator 和 DataLoader stall 分开讲，不把 `232/250` 或 `186/250` 直接解释成模型改进。 第三条是 OpenPiVLA。当前可核验的 formal path 是 Qwen3-VL + PEFT LoRA + `model(**batch)` 的 full-chat causal cross-entropy；不是 assistant-only CE，也不是已经闭合的 flow-matching loss。代码里 `H=15,D=72` 与 formatter `limit=16` 形成 lossy representation 风险，所以我会优先讲 token/action round-trip、padding-only label mask 和 checkpoint/parser/normalizer/controller lineage，而不会把 `97.4%` 、official `96.85%` 与 formal 100k runs 拼成一条因果曲线。 第四条是 Robot Harness。它解决的是高层工具调用如何绑定到 robot scope 和 request lifecycle：runtime 管 scope、mutex、cancel、timeout、compensation，训练侧把 SFT/GRPO 数据和 schema-aware reward 分开。sandbox 100% 只是结构化协议 proxy；Po raw primary JSONL 是 `28/40` ，exact common support 是 `6/10` ，本地可 hash/load 的 checkpoint binary 是 `0/6` ，所以 `37.8→95.6` 不能当作严格 GRPO uplift，也不能当作 physical success。 第五条是 OPD。它围绕五 expert 数据合同、image-hash split、seen/unseen/boundary/format-conflict、coldstart 到 OPD5000、route teacher 和 full-vocabulary KL 展开。实现上必须区分根目录 legacy teacher-rollout/top1-CE trainer 与 `point/` formal trainer；后者才支持 coldstart teacher rollout、OPD5000 student rollout和 $D_{KL}(q_\beta \| p_S)$。route metadata 不是 learned router；跨 split image-hash 交集为 0 也不等于 eval 内没有重复图片，更不等于语义泛化。缺 SFT-only、single-teacher、no-synthetic 和 raw prediction 对照时，我只把结果讲成 protocol-fixed descriptive evidence。 因此这五个项目共同体现的不是“我发明了五个模型”，而是我能把模型输入输出、数据转换、训练合同、checkpoint、controller 和 scorer 串起来，并在失败时报告 first divergence 和证据缺口。

3. 项目选择：面试官问什么时优先讲哪一段

面试官问题	首选项目	口述切入点	不要越界
训练为什么 OOM、保存为什么失败?	WAM	`cycle(dataloader)` 、no-cycle smoke、 `state_dict` first divergence	不说 1 epoch 已完成
action/normalizer/checkpoint 如何对齐?	UWM/LIBERO	task-specific checkpoint、 `ckpt["model"]` 、strict/reset、completed denominator	不把 summary 当 raw benchmark

面试官问题	首选项目	口述切入点	不要越界
VLM 到 action token 的训练目标是什么?	OpenPiVLA	Qwen3-VL、LoRA、full-chat causal CE、`H=15,D=72` 对 formatter 的风险	不把 archived flow 当 formal path
如何保证机器人工具调用可取消且不越权?	Robot Harness	scope → mutex → dispatch/poll → cancel/compensation	不把 sandbox 分数当物理安全
多 expert 数据如何路由和蒸馏?	OPD	数据合同、route teacher、full-vocabulary KL、holdout split	不把 route metadata 叫 learned router
你如何证明结果可信?	五项目横向	protocol tuple、raw artifact、checkpoint lineage、scorer、denominator	不跨项目拼 uplift

4. 共用的实现审计框架

我会按下面六步回答任何一个项目的“你具体怎么做的”:

1. **冻结输入输出**: 观测帧、文本/任务条件、action shape、axis/frame/unit、chunk 与执行 prefix。
2. **冻结训练合同**: 数据来源与 split、target、mask、normalizer、checkpoint 保存格式和 resume 语义。
3. **冻结运行身份**: 代码 revision、配置、模型/Tokenizer/统计量 revision、环境与 seed。
4. **冻结评测协议**: task、init、episode、success predicate、invalid policy、scorer backend 和 denominator。
5. **寻找 first divergence**: 先判断是数据、模型 forward、保存/加载、parser/denormalizer、controller 还是 scorer 出现第一处不一致。
6. **分级结论**: 把事实、实现推断、候选人 ownership 和仍未验证项分开，不用一个 summary 数字替代整条证据链。

这套框架对应的闭环可以写成:

> observation/text → policy output → parse/denormalize → controller/environment → episode artifact → scorer + denominator

其中每个箭头都需要有可复核的 schema 或日志；只看到 loss、视频或 summary，不能自动推出后面的 closed-loop success。

5. 横向真实性边界

结论类型	当前可说	当前不能说
项目范围	“我开展了训练/评测复现、协议对齐、运行审计和失败诊断”	“五个模型都是我独立原创”
训练结果	“某条 run 到达某 step，并留下相应日志/保存记录”	“有可加载 immutable checkpoint”——除非 binary/hash/reload 已闭合
指标	“按某个固定 protocol 重算得到 raw/trusted 结果”	“不同 n、scorer、checkpoint 的数字构成 uplift”
工程能力	“我能定位 first divergence 并设计下一步 gate”	“一个 smoke 通过就代表长跑、保存和闭环都成功”
机器人能力	“完成 simulator/offline/interface/proxy 层验证”	“已完成 physical success、Sim2Real 或 real canary”
ownership	“按 ledger 说明接入、运行、审计、定位和复核边界”	“Git author、Codex 生成或 upstream 代码自动等于个人原创”

6. 可能的压迫式追问

“五个项目里哪个最能证明你有 ownership?”

> 我不会用一个项目的 Git author 或生成代码量回答。WAM 最能展示训练/评测协议迁移和 first-divergence 诊断，UWM 最能展示 action/checkpoint/denominator 审计，Harness 最能展示 runtime contract 和安全 proxy，OPD 最能展示数据合同与蒸馏评估。真正的 ownership 需要逐文件、逐命令、逐 artifact 说明我是亲自修改、协作实现、工具辅助、只审查还是只运行；当前若没有候选人逐项确认，我会使用“开展、接入、运行、审计、定位”这些保守动词。

“你做了这么多项目，为什么没有一个结果能直接证明 uplift?”

> 因为我把可比性当成结果的一部分。WAM 的 full10k 与 8-GPU gate 不是同一 run；UWM 的 summary 缺 raw episode 和 normalizer usage；OpenPiVLA 的 formal causal-CE 与 archived flow/benchmark 不是同一 lineage；Harness 的 scorer、n 和 checkpoint 不完全一致；OPD 缺少关键 ablation。当前最可信的能力信号是把这些断点找出来，而不是用不可比的百分比制造单一结论。下一步如果要证明 uplift，我会先冻结 matched protocol tuple、checkpoint hash、raw per-episode predictions 和 paired rerun。

“如果只能留一个项目讲，你怎么选?”

> 如果岗位偏训练与评测，我选 WAM，因为它同时覆盖训练入口、OOM、checkpoint save、RoboCasa 闭环和失败诊断；如果岗位偏数据/蒸馏，我选 OPD；如果岗位偏机器人系统和安全，我选 Harness；如果岗位偏 action representation，我选 UWM 或 OpenPiVLA。选择依据是问题与岗位的匹配，不是哪个项目的百分比更大。

7. 面试前的最小补强清单

1. 为每个项目补一个 run-specific immutable bundle：配置、代码 revision、checkpoint SHA、stats/tokenizer、eval command、raw episode JSONL 和 scorer 版本。
2. 为每个项目确认个人 ownership：亲自修改、协作修改、工具辅助、仅审查、仅运行五类至少各有一条证据。
3. 只为一个最重要的结论做 matched paired rerun；不要同时声称五个项目都有 uplift。
4. 练习同一项目的 30 秒、2 分钟和 10 分钟版本；回答中始终保留“结果、失败、边界、下一步”四个位置。

8. 证据入口

- WAM implementation-derived oral answer
- UWM/LIBERO implementation-derived oral answer
- OpenPiVLA implementation-derived oral answer
- Robot Harness implementation-derived oral answer
- OPD implementation-derived oral answer
- 首轮口述队列
- 通用口述 capture template

> 本页是从项目实现、日志和 evidence inventory 反推的示范口述，不是候选人真实 transcript；所有项目题卡仍保持 `unseen`，不得据此创建或更新 `coaching_state.md`。

Part 3 / 49 · `interview_materials/vla/practice/wam_oral_model_answer.html` · SHA-256 `3695686BE4E219401F93E176F24A14AC0E83C6F6221560E16F93FE644A0CBF40`

返回本卷目录

Canonical Markdown SHA-256: `3695686be4e219401f93e176f24a14ac0e83c6f6221560e16f93fe644a0cbf40`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\practice\wam_oral_model_answer.md`

PROJ-WAM-001/002 从项目实现反推的证据约束口述稿

状态：`implementation_derived_model_answer / PROJ-WAM-001+002 / candidate_self_report_bounded / not_candidate_transcript / unseen` 更新日期：2026-07-22 题目：如何介绍 Cosmos Policy / WAM 项目，并解释 1 epoch gate、两类训练系统失败、CoffeePressButton 诊断证据与个人 ownership?

1. 项目定位

这不是把所有 Cosmos/WAM 日志拼成“一个 1 epoch 结果”的故事。当前需要明确区分两个 run family：第一条是按 Predict2 + T5 + RoboCasa 合同搭建的 8 卡、global batch 128、`9907` steps 单 epoch gate；它先暴露 dataloader 导致的 CPU cgroup OOM，修复后又在 `state_dict(destination=...)` 保存接口处失败。第二条是独立的单卡 GPU7 full10k repro：日志记录 step `10000` loss=`0.0543`、远端 checkpoint 保存成功、`EXIT:0`，随后进入 24 task × 5 episode 的 RoboCasa eval。前者不能称完成的 1 epoch checkpoint，后者可以称独立 full10k 训练与评测链路，但 checkpoint binary/hash、fresh-process reload 与训练带来的 uplift 仍未闭合。

项目分型：VLA/WAM 训练复现 + RoboCasa 闭环评测适配 + 失败诊断。目标岗位：VLA / embodied-AI 算法与训练评测工程。当前最强信号：能把模型训练、checkpoint、normalizer、controller、episode scorer 与 raw artifact 连成协议链；当前短板：逐文件 ownership、immutable checkpoint 与修复后 paired rerun 尚未闭合。回答结构固定为：实现链路 → 评测合同 → bad case → 验证与边界，而不是从 paper architecture 开始背诵。

2. 30 秒版本

我做的是 Cosmos Policy 在 RoboCasa 上的训练与评测复现：训练侧接入 online Reason1 并完成一条单卡 10k-step run，日志记录 loss=`0.0543`、checkpoint saved 和 `EXIT:0`；评测侧把 local DCP 接回官方闭环，固定 chunk 32、执行前 16 步和 5-step denoising。24×5 eval 的 raw 是 `2/120`，但两条都是 length=1 artifact，trusted 为 `0/118`；同一 harness 的 official-HF path 为 `84/118`。所以我强调协议对齐和失败诊断，不宣称 uplift；另一条 8 卡 1-epoch gate 在 OOM 与保存接口处失败，也不和 full10k 合并。

3. 2–3 分钟版本

这个项目的目标是把 Cosmos Policy 的 WAM 训练从“能启动”推进到“训练、保存、加载、闭环评测都能按协议验收”。我先保留官方 experiment、trainer、model 和 RoboCasa dataset 路径，在训练入口里启用 online Reason1 文本编码；由于在线编码不需要巨大的预计算 embedding 字典，我放了只在模型重算前被消费的 placeholder，同时补齐 lazy success-rollout-as-demo 路径里缺失的 `success=True` metadata。单卡 full10k recipe 使用 batch 1、chunk 32、demo/rollout 各 0.5、`p_world_model=0.5`，原始日志在 step 10000 记录 loss=`0.0543`、保存 `iter_000010000`、`Done with training` 和 `EXIT:0`。评测时我没有重写 RoboCasa 主循环，而是只替换 local DCP model loader，并给 action/future/value inference 加 BF16 autocast。链路仍是官方的 create env→prepare observation→get action→env.step→save rollout video；同时旁路记录每次 `32×7` action chunk 的 min/max/mean/std 和 first row，日志故障不影响 eval。协议固定为 24 个任务、每任务 5 个 episode、chunk 32、每次执行前 16 步、action denoising 5 步、固定 seed。full10k summary 的 raw 是 `2/120`，但两个 success 都发生在 length=1，按预注册的同一 invalid policy 排除后是 `0/118`；同一 harness 下 official-HF path 是 raw `86/120`、trusted `84/118`。这说明环境、controller、success predicate 和 rollout 保存链不是天然全坏，但不能据此断言唯一根因；排查优先级转向 checkpoint/config、Reason1 embedding、dataset statistics、action unnormalization 与训练退化。 我还单独搭了 8-GPU、每卡 batch 16、global batch 128、`9907` steps 的 1-epoch gate。它先在 step 1903 以后出现 rank `-9` / `EXIT=247`，结合 cgroup 记录定位 `itertools.cycle(dataloader)` 缓存 batch 的 CPU 内存增长；改成普通 iterator 并在 `StopIteration` 重建后，8 卡 5-step smoke `EXIT=0`。但正式 run 到 step 1500 保存时，DeepSpeed 调用标准 `state_dict(destination, prefix, keep_vars)`，Cosmos override 不兼容并以 `EXIT=1` 结束，所以这条 gate 没有可用 checkpoint。面试中我会把个人价值讲成训练/评测协议迁移、可观测性和 first-divergence 诊断；不会把 upstream architecture 说成原创，也不会把 full10k 与失败的 1-epoch gate 拼成一个成功结果。

3.1 从实现读出的训练与评测链路

```
RoboCasa demos / rollout episodes / command
  -> official Cosmos dataset + online Reason1 conditioning
  -> action/world sample routing + masked training_step
  -> full10k DCP checkpoint path
  -> local DCP loader + official RoboCasa eval loop
  -> 32x7 action chunk -> execute first 16 -> controller/env step
  -> per-episode success/video + action-debug JSONL
  -> raw/trusted recount + bad-case diagnosis
```

训练适配中真正改变的是文本条件与 lazy-data 边界，不是 Cosmos Policy 的 backbone：`enable_official_online_text_encoder()` 把 `compute_online` 打开，`OnlineTextEmbeddingPlaceholder` 避免强依赖预计算 embedding，`repair_success_rollout_demo_metadata()` 修补 success rollout 被当作 demo 时缺失的 metadata。评测适配中真正改变的是 checkpoint loader、process compatibility、BF16 autocast 与旁路 action logging；环境创建、observation preparation、官方 `get_action`、controller step、success predicate 和视频保存仍走 upstream eval loop。

3.2 Implementation anchors：文件、SHA 与能证明的内容

Project 相对路径	SHA-256	能证明	不能证明
`wm/launch_cosmos_policy_online_text_formal.sh`	`8C67A97288341A1875FC46C95AB3AD77F05516F7DEA273BA823CE1B5FB093806`	single-GPU、10k、batch 1、dataset mix、base/tokenizer 与保存配置	实际运行成功或个人独立编写
`wm/cosmos_policy_train_with_consolidated_online_text.py`	`FBE879A3AD6E3E77B258AE971480338CF41D6EC3378E29C9DE05209CF9D38DF8`	online Reason1、placeholder、lazy success metadata repair 与 official trainer path	每个函数的个人 authorship
`report/cosmos_wam_repro/cosmos_policy_wam_repro_full10k_20260619.log`	`403604E8A1C2F77041E5E1ED43B80AE0BDD2630911D8DB1645B6FB6D0F0443EC`	step 10000 loss= `0.0543`、checkpoint path、`Done with training`、`EXIT:0`	本地 checkpoint binary/hash 与 reload parity
`report/cosmos_wam_repro/loss_breakdown_full10k.jsonl`	`B6CEB4459C8CB5A00AADF3AE7F00F440DC359111B143FE44D5AF21CCB639BDDE`	full10k 逐步训练诊断记录存在	held-out performance 或 closed-loop uplift
`wm/cosmos_policy_robocasa_official_eval_wrapper.py`	`45CB2D44A547C645D15AE890DCFF023569CF126FC20E85A88B7160AED2CF5E42`	local DCP loader、official loop、BF16 autocast、action-debug schema	official scorer 等价或个人模型效果
`wm/launch_cosmos_policy_robocasa_eval_20260620.sh`	`AFD68C7405EA1465D6E1D2B2AFDB387D6FF9085FDD3C75B729AF3E9FDB054DB0`	checkpoint/stats/embedding/task/seed/open-loop/denoise 命令合同	episode 结果本身
`wm/run_cosmos_policy_robocasa_eval_5ep_tasks_20260620.sh`	`C94DB66F67920CBABA0B3FBBE492967F85DE5080F7F25C774A8D579967B77DEC`	24 task × 5 episode sweep 与 per-task status	24 个任务都由同一 immutable checkpoint 内容运行
`report/cosmos_policy_eval/remote_artifacts/eval_summary_conservative.json`	`C9538A2E418BCBEDD88B8DCFB308F0E1AF428DBA2912A959DEA946872A3247BE`	full10k raw `2/120`、artifact=2、trusted `0/118`	benchmark uplift、唯一 root cause
`report/cosmos_policy_eval/baseline_remote_artifacts/eval_summary_officialhf_baseline.json`	`7B211B9D6F20BC95986A77C3CC901814AB6A705065016D7F68B67548BA6CACE9`	official-HF path raw `86/120`、artifact=2、trusted `84/118` 的本地重算材料	official checkpoint ownership 或与 full10k 的严格 matched causal comparison
`report/cosmos_policy_eval/.../CoffeePressButton...smoke-1x1.log`	`B5FF9DB2129EBC0B41CA2D0C11911C94753B5400A8BBACFFBAB1F3109CB2938F`	1 episode、`success=false`、`0/1`、`EXIT:0`、视频保存	多 episode success 或 checkpoint uplift
`report/cosmos_policy_eval/.../CoffeePressButton...smoke-1x1.jsonl`	`6357CD83CB4A40B184B42721110A7FADD4693DED75E68685FD6C2B6949557018`	19 次 query、每次 `32×7` action chunk、有限值与 checkpoint path	action-label loss、7D→12D controller roundtrip
`report/cosmos_policy_eval/muxi_deepspeed_1epoch_20260623/run_contract.json`	`9CAAC2937AC14AAA69DF2EBFD3C7603CB92B34254879BD1CD599C9FE73749D7A`	8 卡、per-GPU 16、global batch 128、`9907` steps、5-frame condition	1 epoch 已完成
`.../nocycle_fix_20260624/...no_cycle_20260624.py`	`FD5F96C4C9004A5FBFE35B80504C034B83FEDEC0431C47EAD91D958A379DE07D`	iterator 修复实现快照	matched long-run 内存因果已完全闭合
`.../nocycle_fix_20260624/smoke5_launch.log`	`76378669F19CCC87E9418241A7EB17AFD0854BE704362CA30EFF5491BE66D94E`	8 卡 no-cycle 5-step smoke，exit 0	长跑、save/load 或 eval 成功
`.../ds_8gpu_bs16_1epoch_nocycle_save1500.../launch.log`	`8FEC262805242AFC3FCD7846A3DDB1043860E17BF5DEB008369D7CD9F67061C3`	step 1500 `state_dict(destination=...)` TypeError、exit 1	checkpoint 已生成

`...` 只为表格可读性缩短公共前缀，所有原文件均位于 `C:\Users\mousongzhe\Project`；本页没有修改该 Project 工作树。

3.3 Truth Boundary 与 ownership 用词

口述内容	当前证据等级	推荐第一人称	禁止升级
Cosmos fine-tune 与 eval 项目范围	候选人已 self-report；代码与日志部分支持	“我开展了训练/评测复现并按协议验收”	“所有代码均由我独立编写”
online Reason1、metadata repair、DCP eval wrapper	implementation-local；逐函数角色仍未确认	“这条实现中我采用/接入了……”	未确认前不说“我原创设计”
full10k 训练终点	log-backed / checkpoint-path-only	“该独立 run 到 10k 并记录保存成功”	“本地 immutable checkpoint 已验证”
8-GPU 1-epoch gate	run-contract + failure-log-backed	“我搭建并排查了 1-epoch gate”	“我完成了 1 epoch”
full10k `0/118` 与 official-HF `84/118`	remote-artifact-backed / checkpoint-unverified	“我按同一 invalid policy 重算并用于诊断”	“训练导致下降 71.19 个百分点”
Cosmos slot/mask/routing architecture	upstream-verified	“我按 upstream contract 对齐”	“我设计了 WAM architecture”

3.4 面向 VLA 岗位的能力映射

岗位能力	当前实现证据	强度	面试策略
多模态训练理解	online Reason1、demo/rollout mix、world-policy routing 与 loss diagnostics	强	讲输入、mask、route 与训练 target，不背抽象名词
大模型训练工程	single-GPU full10k、8-GPU gate、cgroup OOM、DeepSpeed save first divergence	中强	重点讲 gate 与验证覆盖面
机器人闭环评测	official RoboCasa loop、chunk/replan、controller、raw/trusted recount	强	讲协议 tuple 与 invalid rule
数据与 action contract	dataset statistics、normalizer、7D action、32→16 execution	中强	主动承认 7D→12D roundtrip 仍缺 bundle
结果可信度	有 raw logs/summary/action trace，但 checkpoint identity 与 paired rerun 未闭合	中	不用 loss 或单 episode替代 success

3.5 Production Gap、补强状态与 SP/SSP 材料就绪度

Gap	大厂追问	当前状态	下一证据
checkpoint lineage	跑 eval 的 binary 到底是哪一个?	可补做	remote content hash、local manifest、fresh-process reload parity
matched comparison	official-HF 与 full10k 是否只差权重?	可补做	config/stats/T5/scorer/seed 全字段 join key
action semantics	`32×7` 如何变成 controller `12D`?	部分已做	normalizer roundtrip、7D→12D adapter trace、executed prefix
root cause	0-success 是 conditioning、stats 还是退化?	部分已做	同 checkpoint、同 init 的 patch→paired rerun
ownership	哪些函数是本人/协作/工具?	待确认	per-file role ledger 与 review/operator 记录

当前分数是**材料就绪度**，不是候选人 mastery：业务真实性 `2/5`、贡献边界 `2.5/5`、技术深度 `4/5`、数据理解 `3.5/5`、评估可信度 `4/5`、工程闭环 `3/5`、生产差距认知 `4.5/5`、抗追问材料 `4/5`、VLA 岗位匹配 `4.5/5`、ownership `2.5/5`，合计 **34.5/50，强可投材料档**。SP 阻断项是 checkpoint lineage、逐文件 ownership 和修复后 paired rerun；在真实口述前不把这一分数记入 coaching state。

补强顺序：`半天` 完成 per-file role ledger 与三遍口述计时（`可补做`）；`1 天` 完成 save→destroy→fresh-load manifest（`可补做`）；`3 天` 冻结 full10k/official-HF 的 matched protocol tuple（`部分已做`）；`1 周` 完成至少一个候选根因的同 init paired rerun（`可补做`）。当前简历安全表达是“开展 Cosmos Policy / RoboCasa 训练与评测复现，完成协议对齐、运行审计与失败诊断”；只有前三个 gate 闭合后才可升级为“负责实现/优化”，只有 paired result bundle 到位后才可写具体效果。

4. CoffeePressButton：另一层诊断证据

> 除训练 gate 外，项目还有一组 CoffeePressButton diagnostic bundle：base/trained manifest、两侧各 300 步的 action trace、失败视频、单样本 future-image PSNR/SSIM proxy 和 checkpoint path bridge。它能验证文件 provenance、trace shape/finite、逐步动作差异和失败样例，但不能证明这次 1 epoch run 有 checkpoint，也不能给出 benchmark success。policy 侧 7D 到 environment 侧 12D 的 MAE/RMSE 是 trace geometry，不是 action-label loss；PSNR/SSIM 是 offline proxy，不是 closed-loop metric；路径字符串相同也不是 binary hash。两层材料可以共同支持“我做了协议审计和失败诊断”，不能拼接为“训练后效果提升”。

5. Evidence Contract

Claim	证据	等级	当前口径
8 卡、每卡 16、global batch 128、`max_steps=9907`	1 Epoch Evidence Freeze 的 run contract	`local-direct / C1`	可讲训练合同与完成 gate
数据集运行时统计	同页 `dataset_runtime_summary.json` 冻结值	`local-direct / C1`	可讲 1,199 demo、1,291 rollout episodes、343,635 steps、242 commands
step 1903 后 cgroup CPU OOM	launch/status log、`EXIT=247`、rank `-9` 与 cgroup 诊断	`local-direct + local-inference / C2`	可讲定位过程；不要称已完成修复闭环
`cycle(dataloader)` 缓存 batch 与 no-cycle smoke	根因报告；8 卡 5-step、`SAVE_EVERY=0`、`EXIT=0`	`local-report / C2`	可讲迭代器假设被 smoke 支持
step 1500 保存签名失败	DeepSpeed/Cosmos error stack；空的 `step_001500`	`local-direct / C2`	可讲保存接口 first divergence；不能称 checkpoint
step 1900 loss `0.2945`	save-2k 中断前 metric	`local-direct / diagnostic-only`	只能称中间快照，不能称 final/best/epoch loss
完整 1 epoch、可加载 checkpoint、本轮 eval	当前不存在	`C3 / unverified`	禁止宣称
full10k step 10000 loss `0.0543`、远端 checkpoint saved、CoffeePressButton `0/1`	独立 `cosmos_wam_repro` run；日志/report 可复核	`local-report / partial`	可作为独立训练与最小 eval smoke 证据；不能当作 8-GPU 1 epoch gate 的结果或完整 benchmark

Claim	证据	等级	当前口径
official-HF baseline raw `86/120`、trusted `84/118`	RoboCasa Baseline Po Audit：24 task logs、24/24 exit 0、2692 action-debug rows	`remote-artifact-backed / checkpoint-unverified`	可讲运行与协议审计；禁止归属于 1 epoch/full10k 训练或个人模型 uplift
CoffeePressButton trace/video/proxy	WM-019 Artifact Manifest 与 paired diagnostics	`local-direct / diagnostic-only`	可讲评测诊断，不讲 uplift
个人 ownership	WAM Ownership Ledger	`candidate-confirmation-pending`	只能按本人确认的文件、函数、命令和验收动作回答

当前 C1/C2 可讲：run contract、数据统计、OOM 分析、no-cycle 修改与 smoke、保存接口定位、full10k 的 step-10000 训练与 `0/1` smoke、诊断 bundle 分层。当前 C3 禁止讲：8-GPU gate 的完整 1 epoch、该 gate 的 final loss、任一 run 的本地 checkpoint binary/hash、完整 suite success、官方 baseline ownership 或训练带来的 uplift。

6. 关键问题三档回答

“你到底有没有完成 1 epoch？”

危险回答：完成了，已经跑到 1900 step，loss 也降到 `0.2945`。

可过回答：没有完整跑完，但训练主体基本成功，后来只是保存出了问题。

强回答：

> 对 8-GPU、global batch 128 的 `9907`-step gate，答案是没有：完成定义是达到 `9907` steps、保存并在新进程加载 checkpoint、再跑固定协议 eval；它在 step 1903 和 step 1500 分别卡在 OOM 与保存接口。另有一条独立的单卡 full10k repro 确实记录了 step 10000、loss=`0.0543`、远端 checkpoint saved、`EXIT:0`，并做了 CoffeePressButton `0/1` smoke，但它不是前一条 gate 的 continuation，也没有本地 binary/hash 与 load parity。因此我能认领两条运行链路的实现/排障边界，不能把 full10k 数字包装成 8-GPU 1 epoch 的最终结果。

需要补的证据：完整 `9907`-step log、checkpoint 文件清单与 SHA-256、新进程 load smoke、run-specific eval bundle。

“step 1900 loss 0.2945 能不能当最终结果？”

危险回答：可以，这是训练最后记录到的 loss，所以就是 final loss。

可过回答：它只是中间 loss，但可以大致代表这一轮效果。

强回答：

> 不能。它来自 save-2k run 的中断前快照，距离 `9907`-step gate 很远，且没有与它绑定的可加载 checkpoint。它最多证明当时优化过程有一个可观测值；不能叫 epoch loss、best loss，更不能外推 closed-loop success。要成为结果，至少要绑定 step、checkpoint identity、验证集或 rollout protocol 和完整 denominator。

需要补的证据：同 step checkpoint、loss 定义与聚合口径、完整训练曲线、固定协议 eval。

“OOM 修复算不算训练成功？”

危险回答：算，5-step smoke 已经 `EXIT=0`，说明 1 epoch 没问题。

可过回答：算修掉了一个 bug，但训练还没有最终完成。

强回答：

> 它只算“迭代器修复通过了最小验证”。`cycle(dataloader)` 会保存已经 yield 的 batch 以便之后回放，对大 tensor batch 的长跑会持续占用 CPU 内存；改为普通 iterator 并在 `StopIteration` 时重建后，8 卡 5-step smoke `EXIT=0`，支持根因假设。但 smoke 关闭了保存、步数极短，不能覆盖长时内存、checkpoint 或 `9907`-step 完成条件。后续正式 run 在保存接口失败，正说明一个 gate 通过不等于训练系统成功。

需要补的证据：matched long-run RSS/cgroup memory A/B、完整 gate、save/load 与 eval。

“为什么没有 checkpoint？”

危险回答：目录已经生成了，checkpoint 应该只是还没同步回来。

可过回答：保存时接口报错，所以 checkpoint 不完整。

强回答：

> 对 8-GPU 1 epoch gate，DeepSpeed 在 step 1500 调用模型的标准 `state_dict(destination, prefix, keep_vars)` 接口，而 Cosmos 的 override 不接受这些关键字，调用在实际写出可用权重前失败；目录存在只证明保存流程开始过。独立 full10k 日志另有 `Saved checkpoint to ...iter_000010000`，但那是远端 path reference，不等于我已在本地拿到 binary 或完成新进程 load parity。修复 gate 仍应让 override 兼容标准签名，然后做 1-step train→save→销毁进程→新进程 load；检查文件非空、参数 key/shape、global step、optimizer/scheduler state 和固定输入前向一致性，再重跑长任务。

需要补的证据：兼容层 diff、save/load smoke log、checkpoint manifest/hash 和 reload forward parity。

“full10k loss=0.0543 能不能作为最终结果？”

危险回答：可以，0.0543 说明模型已经学会了，RoboCasa 只是偶然失败。

可过回答：可以作为 full10k 训练结果，但闭环只有一个失败样本。

强回答：

> 我会把它限定为独立 full10k run 的 step-10000 training metric。原始日志有 checkpoint saved、`Done with training` 和 `EXIT:0`，report 也有 action/world/value 的 loss breakdown；但 eval 只有 CoffeePressButton 1 episode 的 `0/1`，没有完整 denominator、同 checkpoint 的 raw episode bundle、本地 binary/hash 或 base paired control。因此它能证明训练和最小 eval smoke 链路跑通，不能证明 benchmark 成功或“loss 下降导致 uplift”，更不能替代 8-GPU 1 epoch gate 的 save/load 闭环。

需要补的证据：full10k checkpoint immutable manifest、同 checkpoint 新进程 load parity、固定 task/trial/scorer 的 raw episode JSONL，以及 base/paired control。

“84/118 是不是你的 1 epoch 结果？”

危险回答：是，1 epoch 后 RoboCasa 达到了 84/118。

可过回答：不是同一条 run，是官方结果，但可以当作我的 baseline。

强回答：

> 不是。它是我按 official-HF checkpoint path 跑并审计的一条独立 24×5 RoboCasa baseline eval：raw logs 重算为 `86/120`；其中 CoffeeServeMug 两个 length=1 success 按当前 artifact policy 排除后是 `84/118=71.19%`。24 条 formal status 都是 `exit_code=0`，24 个 action-debug 文件共 2692 行、shape 都是 `32×7` 且 finite。可以认领的是评测运行、协议固定和日志重算；不能把官方权重训练归到个人，也不能把 71.19% 写成 1 epoch/full10k checkpoint 的成绩或 uplift。当前 bundle 仍缺官方 checkpoint binary/hash、完整 episode JSONL 与 scorer/version，所以我会同时报告 raw/trusted 两个口径和排除规则。

需要补的证据：official checkpoint/stats/T5 的 immutable revision/hash、完整 120 episode JSONL、scorer/invalid policy version、operator/reviewer，以及任何本地训练 checkpoint 与该 baseline 的 matched join key。

“为什么 fixed batch 能输出负 gripper，TurnOffMicrowave 还是 0/1？”

危险回答：训练集没有负样本，或者 action 第 7 维符号反了。

可过回答：可能是 gripper collapse，但需要更多 debug。

强回答：

> 我把它拆成 fixed-batch gate、closed-loop 和 same-input probe 三层。local negative-only 1k checkpoint 在固定 `32×7` batch 上的 sampled first gripper 是 `-0.8277`，trajectory final clean 是 `-0.8275`，slot-5 roundtrip 与 normalization 也通过；所以基本 action path 可学。进入 TurnOffMicrowave 后，1 episode 是 `0/1`、跑满 500 steps，32 次 query 的第 7 维全部为正，整体 min/max/mean 是 `+0.0646/+0.5614/+0.2205`。为找 first divergence，我对齐同一个 initial observation 的三路相机、proprio、T5 和 data-batch hash。official-HF first gripper 是 `-1.0156`，local 是 `+0.1429`；local 换 8 个 seed，first 仍在 `+0.1052..+0.2217`，所以不是单次随机采样。数据侧也不是完全没负向监督：52 个 HDF5、11230 states 的 top-24 eval-like neighbors 中 first-action negative rate 是 `37.5%`。更关键的是旧 debug sampler 没显式传 official 的 `num_conditional_frames=5`：缺省条件下同一 HDF5 batch 可全负，补齐 5 frames 后转为全正。corrected single-batch overfit 仍能把 fixed clean/sample 拉到约 `-1.02`，但 exact eval first 仍 `+0.316`，negative-only strong-reg 也没有修复。因此我会说

> “conditioning contract 差异和 exact-observation transfer gap 已被证据支持，唯一 root cause 与修复效果还没确认”，不会把它包装成已修复的 gripper bug 或 benchmark 成绩。

需要补的证据：显式 5-frame matched training patch、immutable checkpoint/load parity、同 state 的 patch 前后 query trace，以及多 episode 的 scorer/controller/denominator bundle。

“base_load_summary 有 715 个 key，为什么还不能说 checkpoint 可加载？”

危险回答：因为远端文件还没同步回来，应该已经保存了。

可过回答：base model 能加载，但训练 checkpoint 的保存失败了。

强回答：

> 我把 gate 拆成 base load、短训练、长跑、save、fresh-process reload 和 matched eval。13 个 base-load summary 都指向同一个 HF cache snapshot，`715` keys、missing/unexpected 都是 `0`；fixed-optional/bs8 1-step 和 no-cycle 5-step smoke 也有 `exit=0`。但这些只证明上游 base 初始化和短路径通过，不证明训练产物存在。9907-step save2k 线以 rank `-9` / `EXIT=247` 结束，no-cycle save1500 线在 DeepSpeed 调用 `state_dict(destination=..., prefix=..., keep_vars=...)` 时 TypeError/`EXIT=1`。归档 65 个文件里 checkpoint-like binary 数量是 `0`，也没有 save→销毁进程→fresh load 的 parity artifact，所以我不能把 base-load summary 说成训练 checkpoint 的 load proof。

需要补的证据：`state_dict(destination=None, prefix='', keep_vars=False)` compatibility patch、真实 1-step save/load smoke、checkpoint shard/optimizer/scheduler/global-step manifest 与 SHA，以及固定输入 reload forward parity。

7. 面试官追问树

Root：你做的 1 epoch finetune 结果是什么？

- 若回答“完成了”：要求给出 `9907`-step 终点、checkpoint SHA、reload log 与 run-specific eval；任一缺失即降级为未完成 gate。
- 若回答“到 step 1903”：继续问 `EXIT=247` 与 rank `-9` 如何解释，证据为何指向 cgroup CPU OOM 而不是 GPU OOM。
- 若回答“修复了 OOM”：继续问 `itertools.cycle(dataloader)` 为什么会缓存 batch，5-step smoke 覆盖了什么、没覆盖什么。
- 若回答“保存失败”：继续问 `state_dict(destination=...)` 兼容层如何设计，为什么空目录不是 checkpoint。
- 若回答“full10k loss 0.0543”：要求给出 run name、checkpoint identity、CoffeePressButton denominator 与 eval artifact，禁止把它和 9907-step gate 或官方 baseline 做无 key join。
- 若回答“准备重跑”：继续问 save/load smoke 的最小验收项，以及怎样防止花费 8 卡长跑后才暴露保存错误。
- 若回答“84/118”：要求指出它的 run、checkpoint、协议和 operator，禁止与本轮中间 loss 拼接。
- 若回答“715 keys 说明 checkpoint 没问题”：追问它是 HF base snapshot 还是 produced checkpoint，是否有 save manifest、fresh-process reload 和 matched eval；没有就降级为 base-load evidence。
- 最后追 ownership：谁设计合同、谁改 iterator、谁运行远端 8 卡、谁定位 error stack、谁 review；按 `candidate / operator / reviewer / collaborator / tool / upstream` 分开。

8. 高频技术追问答法

“`cycle(dataloader)` 为什么会缓存 batch？”

`itertools.cycle` 为了在源 iterator 耗尽后重复输出，会把首轮迭代得到的每个对象保存在内部缓存。对 DataLoader 而言，这些对象可能包含大 tensor、worker 返回结构或 pinned-memory 引用；如果一个 epoch 很长，缓存会随已见 batch 数增长。无限训练应重建 iterator，而不是用 `cycle` 保存整轮 batch。

“怎么验证是 cgroup OOM，不是 GPU OOM？”

看证据链而不是只看进程被杀：是否有 CUDA allocator/OOM traceback；主机或容器的 `memory.current`、`memory.events`、`oom_kill` 是否增长；launcher 的 `EXIT=247` 与 rank `-9` 是否对应 SIGKILL；GPU 显存是否稳定；no-cycle matched smoke/长跑的 CPU RSS 是否停止线性增长。当前材料支持 cgroup CPU OOM 诊断，但完整 matched 长跑 A/B 仍可补强。

“`EXIT=247` 和 rank `-9` 怎么解释？”

rank `-9` 表示该子进程被 SIGKILL，常见于外部 OOM killer；顶层 `247` 是 launcher 汇总后的退出码，不能单独当根因。需要把它与 cgroup memory event、无 Python/CUDA OOM 栈、其他 ranks 被连带终止和时间线对齐。

“保存接口应该怎么修？”

让 Cosmos 的 override 兼容 PyTorch 标准签名 `state_dict(destination=None, prefix='', keep_vars=False)`，并把参数正确传给父类/实际 state dict 生成逻辑；不要只吞掉参数。还要覆盖 DeepSpeed 所需的 optimizer、scheduler、global step 和分片元数据。这个方案目前是下一步，不应说成已实现。

“怎样设计 save/load smoke？”

用真实 DeepSpeed 配置和最小 batch 跑 1 step，在计划保存路径写 checkpoint；进程完全退出后从空的新进程加载，核对文件清单与 hash、参数 key/shape、global step、optimizer/scheduler state，并在固定随机种子和固定输入上比较 reload 前后输出。多卡时还要检查所有 rank barrier 与 shard completeness。

9. PROJ-WAM-002：哪些部分能说是我做的？

9.1 30 秒 ownership 版本

> 我能认领的是 Cosmos Policy / RoboCasa 的训练评测复现、协议对齐和失败诊断：我能沿 full10k、8-GPU gate、DCP eval、action trace 与 raw/trusted summary 解释验收链。latent slot、mask、routing 属于 upstream，不是我的原创；六个关键 WAM 文件只保留 `c4469eb` initial snapshot，没有增量 diff，因此我不凭 Git author 声称所有函数独立编写。面试中我用“开展、接入、运行、审计、定位”，并明确协作、工具和 operator/reviewer 边界仍待逐项确认。

9.2 2–3 分钟 ownership 版本

> 我会把贡献拆成四层。第一层是 upstream：Cosmos Policy 在统一 latent sequence 里组织 action、future、value，并通过 condition/loss mask 和 sample routing 切换训练角色；这些机制来自 NVlabs paper/repo，我只按合同理解和对齐，不说是我设计的。 第二层是本地训练与评测实现。项目里的训练入口启用了 online Reason1，处理 placeholder embedding 和 lazy success-rollout metadata；评测 wrapper 把 local DCP checkpoint 接回官方 RoboCasa loop，固定 chunk 32、执行前 16 步、5-step action denoising，并旁路写 action-debug。根据我已明确的项目范围，我可以说自己开展了 Cosmos fine-tune、eval 和协议对齐；但六个关键 WAM 文件在 Git 中都只来自 `c4469eb` initial snapshot，没有 pre-snapshot commit 或 before/after diff，因此更安全的说法是“我的项目实现采用/接入了这些路径，我负责把它们运行、核对并用于诊断”，而不是“每个函数都由我从零独立编写”。 第三层是运行与故障闭环。我能讲清 full10k 日志到 step 10000、8-GPU `9907`-step gate 的 cgroup OOM/no-cycle smoke、step 1500 `state_dict(destination=...)` first divergence，以及 24×5 eval 的 raw/trusted 重算。这里也要把 role 拆开：代码存在不等于我就是原作者，远端 path 存在不等于我就是 operator，报告存在不等于我就是 reviewer；当前缺 shell history、review record 和逐文件工具/协作者说明，所以我认领的是项目级训练评测与证据审计，不认领尚未闭合的逐文件 authorship。 第四层是结果归属。full10k 的 `0/118`、official-HF path 的 `84/118`、CoffeePressButton 失败 trace 和 offline proxy 都可以作为诊断材料，但 official checkpoint 不是我训练的，8-GPU gate 也没有完成，历史 MSE/uplift 更没有完整 provenance。我的强项是把实现、运行、metric 和 unknown 分开，并给出下一步 save/load、matched protocol、7D→12D roundtrip 和 paired rerun；我的边界是暂不宣称 architecture 原创、完整 1 epoch、benchmark uplift 或唯一 root cause。

9.3 从实现与 Git 能得到的 role matrix

层级	当前证据	能说的第一人称	不能说
Upstream architecture	frozen paper/repo：latent slots、mask、routing、sampler	“我按 upstream contract 理解并对齐”	“我设计了 WAM architecture”
项目范围	候选人 self-report：Cosmos 1-epoch fine-tune、eval、协议对齐	“我开展了训练、评测与协议核对”	self-report 自动证明逐文件 authorship
WAM 训练入口	`c4469eb` snapshot + current full-SHA code	“项目实现接入 online Reason1、placeholder 与 metadata repair；我能解释并运行这条链”	“我从零独立编写全部函数”
RoboCasa eval wrapper	`c4469eb` snapshot + DCP loader/action-debug/eval logs	“我把 local checkpoint 接到官方闭环并审计 action/episode artifact”	“官方 evaluator、controller 或 scorer 是我实现的”
full10k / 8-GPU runs	run contract、launch log、summary、failure stack	“我搭建/运行/排查这些 run family”是项目级 self-report 口径	没有 operator record 时声称每次远端命令均由本人执行
raw/trusted recount	两份 summary 与 audit code/artifact	“我按相同 invalid policy 复核并用于诊断”	把 official `84/118` 写成本人模型成绩

层级	当前证据	能说的第一人称	不能说
`cv` evidence pages	当前面试仓库的 Codex/工具辅助审查产物	“我用这些材料准备和复核口述”	把面试准备文档当成原项目个人贡献

Git 的可核验事实是：`wm/cosmos_policy_train_with_consolidated_online_text.py`、训练 launcher、RoboCasa eval wrapper、eval launcher、24-task sweep 和报告生成器均只显示 `c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`（`Initial project snapshot`，author identity 为 `mousongzhe`）；当前 HEAD 对这些路径没有后续增量 history。author identity 是 lineage 线索，不是 authorship、operator 或 reviewer 的充分证据。

9.4 危险 / 可过 / 强回答

危险回答：

> Cosmos Policy 的 action/world/value architecture 是我设计的，代码都是我写的；我完成了 1 epoch，并用自己的模型跑到了 `84/118`。

问题：同时混淆 upstream architecture、snapshot author、失败的 8-GPU gate、official checkpoint 和本地 full10k。

可过回答：

> 我主要做 Cosmos Policy 的训练和 RoboCasa 评测复现，architecture 是上游的。我做了训练适配、跑实验并分析失败，但部分代码和运行角色需要再核对。

优点：没有抢占 upstream；缺点：没有文件、函数、run 与验收证据，仍像泛泛“参与”。

强回答：

> 我能确认的项目边界是训练评测复现与协议诊断，不是 architecture 原创。训练侧我沿 `launch_cosmos_policy_online_text_formal.sh` 和 online-text entry 解释 Reason1、dataset mix 与 full10k；评测侧我沿 local DCP loader、24-task sweep、action-debug 和 raw/trusted summary 解释 checkpoint→action→controller→episode 链。8-GPU gate 的 no-cycle smoke 与 `state_dict` failure 也能用日志复核。个人逐文件 ownership 仍有一个诚实缺口：这些 WAM 文件只保留 initial snapshot，缺 before/after diff，operator/reviewer/tool 记录也不完整。因此我说“我开展、接入、运行并审计了这条复现链”，不说“所有函数独立编写”；官方 `84/118` 只作为同 harness 诊断对照，也不写成我的模型结果。

需要补的证据：per-file role ledger、pre-snapshot diff 或设计记录、远端 command/operator record、review/验收记录，以及工具/协作者边界。

9.5 Ownership 追问树与最小表达合同

```
“哪些是你做的？”
  -> 先报项目级职责：训练 / 评测 / 协议 / 诊断
  -> 再给一个训练文件 + 一个评测文件 + 一个运行 artifact
  -> 每个 claim 说明 action、evidence、verification、boundary

“这个函数是你写的吗？”
  -> 有 pre-snapshot diff / review：按真实角色回答
  -> 只有 initial snapshot：说实现路径与自己如何使用，authorship 保持 pending

“远端 8 卡是谁跑的？”
  -> 有 shell history/operator record：报 operator 与 reviewer
  -> 没有：只说项目 run artifact 可复核，不推断执行者

“84/118 是你的成绩吗？”
  -> 明确是 official-HF checkpoint path 的 eval 对照
  -> 个人贡献是 harness 运行/审计与 denominator 重算，不是权重训练
```

口述时每个 ownership claim 应满足：`我做的动作 + 文件/函数/命令 + 如何验收 + upstream/协作/工具边界`。缺任一项就降级为“项目实现包含 / 我参与使用或审查”，而不是用 Git author 或目录位置补齐。

10. 练习边界

- 这是 `PROJ-WAM-001/002` 的 model answer，不是候选人的真实 transcript；两张题卡继续保持 `unseen`。
- “我搭建/定位/修改/运行”必须替换为候选人可确认的文件、函数、命令、operator/reviewer 和验收 artifact。
- 上游 Predict2/WAM 机制、官方 HF baseline、远端路径与 Git author 不能自动升级为个人 ownership。

- 新增 checkpoint 或 eval 证据时，先更新 evidence freeze、inventory 与 ownership ledger，再升级回答。
- 真实口述前不创建 `coaching_state.md`。

References

1. Cosmos Policy 1 Epoch Evidence Freeze。
2. WAM 1 epoch / full10k Run Reconciliation。
3. Cosmos Policy RoboCasa Official-HF Baseline P0 Audit。
4. Cosmos Policy / WAM Claim-to-Code Audit。
5. WAM Claim–Evidence Inventory。
6. WAM Ownership Ledger。
7. WM-019 Artifact Manifest。
8. WM-019 Action-Trace Diagnostic。
9. WM-019 Future-Metric Diagnostic。
10. WM-019 Matched-Checkpoint Bridge。
11. TurnOffMicrowave trained-failure P0 audit。
12. Checkpoint save-load gate P0 audit。

Part 4 / 49 · `interview_materials/vla/questions/proj_wam.html` · SHA-256 `90B5AFB60FD57D59580DB4276D3FD2A959EC458DC51A1ADF9359F2C39153E142`

返回本卷目录

Canonical Markdown SHA-256: `90b5afb60fd57d59580db4276d3fd2a959ec458dc51a1adf9359f2c39153e142`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_wam.md`

PROJ-WAM：Cosmos Policy / WAM 项目原子题卡（Training Failure + Diagnostic + Ownership Batch 1）

状态：`drafted / cards_4_of_4 / WARN` 更新日期：2026-07-21 当前题数：4 / 4 覆盖范围：`PROJ-WAM-001..004` 难度：L3 4 题 优先级：P0 4 题 练习状态：`unseen` 前置证据：Cosmos Policy 1 Epoch Evidence Freeze、WAM Claim–Evidence Inventory、WAM Ownership Ledger、WM-019 Artifact Manifest。

A. 1 epoch gate 与两类训练系统失败

▼ `L3` `P0` **PROJ-WAM-001 · 你是否完成了 Cosmos Policy 1 epoch finetune？如何把两次中断讲成可信项目经历？**

面试意图：判断你是否有明确的训练完成门槛，能否从 exit code、资源证据和 error stack 定位 first divergence，并避免把中间 loss、空目录或其他 run 的 baseline 拼成结果。

30 秒回答：`9907`-step、8卡/batch128 gate未完成1 epoch：step1903因`cycle(dataloader)`缓存触发cgroup CPU OOM；改iterator后5-step smoke通过，但step1500又因DeepSpeed/Cosmos `state_dict`签名不兼容保存失败，因而无checkpoint/eval。

机制展开（约 2 分钟）：完成定义是 `9907` steps→保存→新进程加载→同协议 eval。save-2k run 在 step 1903 后 `EXIT=247`、rank `-9`，无 GPU OOM 栈且 cgroup 证据指向 CPU memory；`cycle` 会缓存已 yield batch。改为普通 iterator、耗尽后重建，8 卡 5-step smoke `EXIT=0`。正式 run 到 step 1500 时，DeepSpeed 传 `destination/prefix/keep_vars`，Cosmos override 不接收，目录无可用权重。因此只认领合同、根因和 smoke，不认领 epoch 结果。

诊断 / 证据：先展示 `run_contract.json` 的 8×16、global batch 128 和 `max_steps=9907`，再把两条 run 分开：step 1900 loss `0.2945` 是中断前快照；`step_001500` 是空目录，不是 checkpoint。下一步让 `state_dict` 兼容并正确转发 PyTorch 标准签名，做 1-step train→save→退出→新进程 load smoke，核对参数 key/shape、optimizer/scheduler、global step、文件 SHA 和固定输入 forward parity；通过后才重跑长任务。

项目挂钩：可讲 C1/C2 的 run contract、数据统计、cgroup OOM 分析、no-cycle smoke 和保存接口定位。CoffeePressButton manifest、300-step trace、失败视频、PSNR/SSIM proxy 是另一层诊断 bundle；它们不能补出本轮 checkpoint 或 success。C3 禁止讲完整 1 epoch、final loss、closed-loop success、uplift 与官方 baseline ownership。

追问树：`cycle(dataloader)` 为什么缓存 batch？如何证明 cgroup CPU OOM 而非 GPU OOM？`EXIT=247` 与 rank `-9` 是什么关系？为什么 5-step smoke 不等于训练成功？`state_dict(destination=...)` 兼容层如何设计？save/load smoke 验收什么？`84/118` 为什么不是本轮结果？

危险答案：说“step 1900 就算完成 1 epoch”“loss `0.2945` 是 final”“5-step smoke 证明训练成功”“有 `step_001500` 目录就是 checkpoint”或“`84/118` 是我的 1 epoch eval”。

来源：[1][2][3]　练习状态：unseen

B. Checkpoint save/load gate 与 base-load 边界

▼ `L3` `P0` **PROJ-WAM-003 · base_load_summary 有 715 个 key，为什么仍不能说 checkpoint 可加载？请给出 save/load gate 的最低验收协议。**

面试意图：判断你是否区分 base model 初始化、短训练 smoke、长跑、checkpoint save、fresh-process reload 与同 checkpoint eval，能否把“key 对齐”与“训练产物可复现”拆开。

30 秒回答：715-key base-load 与1/5-step smoke均通过，但save2k为rank `-9` / `EXIT=247`，save1500为`state_dict(destination=...)` TypeError/`EXIT=1`；归档无checkpoint binary，所以不能称checkpoint可加载。

机制展开（约 2 分钟）：完成门是 `base load → smoke → 9907 steps → save → 销毁进程 → fresh load → fixed-input parity → eval`。715 keys 只代表 HF base snapshot；smoke 没覆盖保存。save2k 是长跑资源线，save1500 是 DeepSpeed 传标准 `destination/prefix/keep_vars` 而 Cosmos override 不接收的 API first divergence。修复后应在配置下跑 1 step save，退出并新进程 load，核对 shard、optimizer/scheduler、global step、文件 SHA 与固定 batch 输出，再重跑 formal gate。

诊断 / 证据：展示 Checkpoint Save-Load Gate Po Audit 的 65-file inventory、13 份 base-load summary、`smoke5_exit_code=0`、save2k `EXIT=247`、save1500 TypeError 与 checkpoint-like binary `0`。不要把 HF cache snapshot、`step_001500` 目录、early loss 或 official `84/118` 当成 produced checkpoint 证据。

项目挂钩：可讲你如何定义验收门、拆分 run family、定位保存接口和设计 fresh-process parity；不能讲“1 epoch 已完成”“checkpoint 已保存”“loss 下降带来 closed-loop uplift”或从远端路径推断 ownership。

追问树：715 keys 检查的对象到底是什么？为什么 5-step smoke 不覆盖 save？`EXIT=247` 与 `EXIT=1` 是否同一个根因？`state_dict` 兼容层哪些参数必须转发？为什么必须销毁进程再 load？除了模型权重，还要校验哪些 DeepSpeed 状态？

危险答案：说“missing=0 就是 checkpoint 可加载”“有 step 目录就是权重”“5-step smoke 证明 epoch 成功”“把 save2k OOM 和 save1500 TypeError 合成一个错误”或“官方 baseline 就是我的训练结果”。

来源：[1][3][7]　练习状态：unseen

C. 上游、运行角色、诊断与个人 ownership

▼ `L3` `P0` **PROJ-WAM-002 · Cosmos/WAM 哪些部分能说是你做的？如何区分 upstream、训练排障、远端运行和个人 ownership？**

面试意图：判断你能否把上游机制、Project 快照、训练系统改动、operator/reviewer、诊断 artifact 和结果归属拆开，而不是把 Git author、远端目录或报告当作全部个人贡献。

30 秒回答：Predict2/WAM architecture是upstream；本地能核验8卡`9907`合同、两类失败和CoffeePressButton诊断。个人贡献必须绑定文件/函数/命令/验收和角色；Git author、远端路径、官方`84/118`都不能自动归我。

机制展开（约 2 分钟）：分五层：上游 latent slot/mask/routing；Project 的训练、动作与 rollout 快照；step 1903 cgroup OOM、no-cycle smoke 和 step 1500 保存签名错误；CoffeePressButton manifest/trace/video/proxy；最后才是个人角色。前三层能证明项目与运行发生过，不能证明逐行原创；诊断 bundle 也不证明 benchmark。现场只认领本人能解释的 diff、命令和验收，远端 8 卡运行、review、协作与工具辅助分别记录。

诊断 / 证据：用 ownership ledger 逐项回答：谁设定 8×16/`9907` 合同，谁修改 iterator，谁发起远端 run，谁读取 cgroup/launcher 证据，谁定位 `state_dict` stack，谁验收 smoke。升级为“我负责并取得训练结果”还需要 candidate-confirmed diff、operator/reviewer 记录、完整 `9907`-step log、immutable checkpoint/normalizer/config 和 protocol-complete episode/scorer bundle。

项目挂钩：当前安全表达是“参与或负责训练协议迁移、系统失败定位和证据审计”，具体动词由候选人逐文件确认。CoffeePressButton 的 7D→12D trace geometry、offline visual proxy 与 path bridge可讲诊断层级；不能把 policy RMSE 当 action-label loss、把单 episode `success=false` 当 benchmark、把 path equality 当 binary verification。

追问树：哪个函数是你亲手改的？`cycle` 修复由谁提出、实现和验收？谁运行远端 8 卡？保存接口方案是否已落地？谁是 reviewer？CoffeePressButton 的 checkpoint binary 在哪？Codex/协作者做了什么？若没有完整 checkpoint，为什么简历能写 finetune？

危险答案：说“Git author 证明所有代码是我写的”“WAM architecture 是我设计的”“OOM 修了就等于 1 epoch 成功”“官方 `84/118` 是我的结果”或“远端目录存在就等于 checkpoint 归我”。

来源：[1][4][5][6]　练习状态：unseen

D. Eval run provenance 与 raw / trusted denominator

▼ `L3` `P0` **PROJ-WAM-004 · Cosmos Policy eval 中 full10k 的 raw 2/120、trusted 0/118 与 official-HF 的 raw 86/120、trusted 84/118 如何解释？为什么不能说本地训练达到 71.19%？**

面试意图：判断你能否按 checkpoint lineage、run family、scorer 与 invalid policy 管理结果，而不是把同一评估环境下的数字错误 join 成个人模型 uplift。

30 秒回答：独立full10k trained run为raw `2/120`、trusted `0/118`；official-HF另一checkpoint为raw `86/120`、trusted `84/118=71.19%`。我能讲harness和重算，不能把官方成绩写成本地训练；8-GPU `9907` 仍未完成。

机制展开（约 2 分钟）：先冻结 join key：checkpoint/revision、stats/T5、task/init、seed、denoise/chunk、controller、predicate、invalid policy 和 denominator。full10k 是单卡 batch 1 的独立训练线，不能与 8-GPU `9907` gate 合并；其 24×5 只有两个 length=1 immediate success，所以报告 raw `2/120` 与 trusted `0/118`。official-HF 使用另一模型 path，重算为 `86/120` 和 `84/118`；缺 matched checkpoint、scorer 与 raw episode bundle时不能计算 uplift。

诊断 / 证据：full10k `eval_summary_conservative.json` 明确记录 `raw_successes=2`、artifact=2、trusted=0、eligible=118；official baseline audit 从 24 个 task log 重算 `86/120`，排除同类两个 CoffeeServeMug length=1 后为 `84/118`，并验证 24 个 action-debug 文件共 2692 行、shape `32×7`、finite。`length<5` 是当前本地审计规则，不应冒充 RoboCasa 官方统一定义；checkpoint binary/hash、完整 episode JSONL 和 scorer version 仍缺。

项目挂钩：可讲我如何把 trained、official baseline、8-GPU gate、1k ablation 与 TurnOffMicrowave 单 episode分开管理，复算 raw/trusted denominator 并保留 invalid policy；不可说“本地训练达到 71.19%”“full10k 比 1 epoch 提升”“官方 baseline 是我的模型成绩”。

追问树：为什么 denominator 从 120 变 118？排除 invalid episode 后为什么不是 `84/120`？ `exit_code=0` 是否等于 success？如何证明两个 eval 使用相同 scorer？checkpoint path 为什么不足以做 paired comparison？若要报告 uplift，最小 matched bundle 是什么？

危险答案：说“71.19% 是我的 full10k 成绩”“两个 length=1 success 证明 trained model 有能力”“raw 与 trusted 选更高的报”“同为 24×5 就能直接比”或“官方 checkpoint path 等于我持有并验证了权重”。

来源：[1][3][8][9][10]　**练习状态**：unseen

参考来源

1. Cosmos Policy 1 Epoch Evidence Freeze：8 卡 run contract、`9907`-step gate、step 1903 cgroup OOM、no-cycle smoke、step 1500 保存签名失败与结果边界。
2. Cosmos Policy / WAM Claim-to-Code Audit：upstream latent slot、mask/routing 与公开 contract。
3. PROJ-WAM-001 Evidence-Bounded Model Answer：五个高压问题的危险/可过/强三档回答与 save/load smoke。
4. Cosmos / WAM Claim–Evidence Inventory：claim 等级、C0–C3 证据门与结果升级条件。
5. Cosmos / WAM Ownership Ledger：per-file/per-decision ownership、operator/reviewer 与工具边界。
6. WM-019 Artifact Manifest、Action-Trace Diagnostic、Future-Metric Diagnostic 与 Matched-Checkpoint Bridge：CoffeePressButton 诊断证据层。
7. Checkpoint Save-Load Gate P0 Audit：13 份 base-load summary、短 smoke、formal save2k/save1500 first divergence 与 checkpoint/reload 边界。
8. Cosmos Policy RoboCasa Official-HF Baseline P0 Audit：official path 下 raw `86/120`、trusted `84/118`、24×5 status、action-debug integrity 与 ownership 边界。
9. WAM 1 epoch / full10k Run Reconciliation：8-GPU `9907` gate、独立单卡 full10k 与最小 eval 的 run-family 分离。
10. Cosmos Policy full10k trained eval P0 audit：trained `iter_000010000` path 下24×5、raw `2/120`、local-policy trusted `0/118`、4066行 action-debug 与 checkpoint/scorer边界。

Part 5 / 49 · `interview_materials/vla/evidence/projects/wam_claim_evidence_inventory.html` · SHA-256 `AE34D4453524037BCD55A892C5BA2BF49DCCD581D859EA6776AC7690F82A07A2`

返回本卷目录

Canonical Markdown SHA-256: `ae34d4453524037bcd55a892c5ba2bf49dccd581d859ea6776ac7690f82a07a2`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wam_claim_evidence_inventory.md`

Cosmos / WAM Claim–Evidence Inventory（WAM-01）

状态：`drafted / local_claim_audit / WARN / resume_safe_downgraded / diagnostic_artifacts_partial / ownership_ledger_added` 审查日期：2026-07-21 证据截止：2026-07-21 项目范围：Cosmos Policy / WAM 多目标后训练、RoboCasa official-checkpoint eval 与本地 0-success 诊断 目标岗位：VLA / World Model / Robot Learning 算法与训练评估方向 结论先行：上游 Cosmos Policy 的 action/future/value slot、routing、loss mask 与 RoboCasa protocol 已有 frozen paper/repo 支持；2026-07-17 对 `cv` 工作区的扫描未找到 WAM 训练/评测 raw bundle，因此历史本地数字先保持 `local-claim-only`。2026-07-18 在外部 `C:\Users\mousongzhe\Project\wm\outputs` 发现一组 CoffeePressButton diagnostic artifact；2026-07-21 又冻结了 8 卡 1 epoch gate 的运行合同与失败链、独立 full10k trained `iter_000010000` 的 RoboCasa 24×5 remote-artifact bundle（raw `2/120`、当前本地 policy trusted `0/118`，两个 CoffeeServeMug length=1 artifact success 被排除）、official-HF RoboCasa baseline 的 24×5 remote-artifact bundle（raw `86/120`、trusted `84/118`）、TurnOffMicrowave local 1k checkpoint 的 fixed-batch→closed-loop→same-input first-divergence 审计，以及 DeepSpeed base-load/short-smoke/formal-save gate 审计。新证据支持“trained-run 与 official-HF 评测链路、一个本地 0/1 失败链和 save/load first-divergence 可审计”，但不支持本地 checkpoint binary/hash、训练 uplift、官方 scorer 等价、唯一 root cause 或 ownership。2026-07-17 已完成安全降级：optimized resume 的 WAM 段不再包含 MSE、task success、official-HF reproduction 或 local 0-success 数字；投递 PDF 已从新 HTML 重建并完成 A4 单页视觉/文本检查。投递 release blocker 已关闭，但项目证据仍为 WARN，不能升级为 verified/reviewed。

问题（Question）

这份清单回答四个问题：当前 WAM 简历中哪些内容有上游事实支撑，哪些只是本地自述；`0.09718→0.05349`、`84/118`、`0.90–0.95 / 0.75` 与 `0 success` 分别属于哪个证据层；哪些表述现在能安全口述；要补什么 artifact 才能把 claim 从“知道/参与”升级为“负责/优化/结果影响”。

本页不是新的项目成绩报告，也不把已有 HTML 相互引用当作独立证据。它把简历、项目页、旧版简历和 frozen upstream audit 当作不同来源，显式记录一致、冲突和缺失。[1][2][3][4][5][6][7]

1. 范围、标签与判定规则

1.1 Evidence labels

标签	定义	本页如何使用
`upstream-verified`	primary paper、官方 frozen repo/model card 可直接支持	只支持 Cosmos Policy 公有机制与官方协议，不支持个人运行
`local-direct`	当前可读取 code/config/log/checkpoint/eval/episode artifact	`cv` 工作区没有 WAM checkpoint；2026-07-18 在外部 `C:\Users\mousongzhe\Project\wm\outputs` 发现一组 manifest/trace/video diagnostic bundle，已单独登记，checkpoint 仍未验证
`local-claim-only`	内容只出现在简历、项目页或重构计划	可作为待核线索，不能作为结果证明
`inference`	从同源证据推得，但来源没有直接声明	必须写成假设或分析框架
`conflict`	两个来源的 checkpoint、protocol 或结论无法同时对齐	在 provenance 澄清前禁止升级
`ownership-unverified`	上游设计已核实，但缺 local diff/设计记录证明个人贡献	只能说“梳理/对齐/复现尝试”，不能说“设计/提出”

1.2 Claim levels

等级	允许的表达	最低证据
C0	了解、阅读、参与讨论	可解释的项目文档或口述
C1	负责模块、实现、迁移	local code/diff、配置、命令或日志

等级	允许的表达	最低证据
C2	设计、优化、定位根因	对照实验、bad case、设计记录、修复前后证据
C3	提升 X%、达到成功率、完成上线/正式训练	完整 metric protocol、checkpoint、raw summary、可重放记录与 ownership

强度不是按措辞感觉决定，而是按 `claim + protocol + artifact + ownership` 四项共同决定。缺任一项时，C2/C3 必须降级。[8]

2. Role Fit 与岗位能力映射

本项目与 VLA/World Model 岗位的匹配点是：多目标训练合同、动作/未来/value 的信息流、RoboCasa closed-loop eval、normalization/controller boundary 和 0-success failure attribution。当前最强的是“能解释上游机制和应如何审计”，最弱的是“本地运行和个人改动没有 raw artifacts”。

岗位能力项	当前材料	强度	缺口	当前策略
WAM/VLA architecture	frozen Cosmos audit 已核对 latent slot、condition/loss mask、direct/planning paths	中强（上游）	local tensor trace/diff 缺失	前置为机制理解，不包装成原创
Multi-objective training	项目页描述 action/world/value routing 与 loss weighting	中（叙事）	exact local config、mask smoke、gradient trace 缺失	只说“梳理/对齐”，不说“设计并证明有效”
Training/debugging	项目页记录 gripper、high-sigma、5-frame、OOM、state_dict 假设链	中低	日志、patch、rerun 与因果闭环缺失	说“形成诊断假设”，不说“定位根因”
Offline evaluation	MSE 0.09718→0.05349 出现在三份自述材料	低	split、metric code、prediction/label artifact 缺失	从投递稿移除，待核后恢复
Closed-loop evaluation	full10k trained 24×5 audit：raw 2/120、local-policy trusted 0/118；official-HF 24×5 audit：raw 86/120、trusted 84/118	中低	checkpoint revision/hash、完整 episode JSONL、scorer/version、matched trained-vs-baseline run 缺失	可讲两条 eval harness 与日志重算；不写成本地训练结果 uplift 或官方 scorer 等价
Scientific ablation	已提出 action-only/+world/+value/all	弱	没有执行结果	标为可补做，不写成已完成
Ownership	项目页称“我负责复现/诊断”	低	local diff、run ledger、人工贡献边界缺失	需要候选人逐项确认

没有提供具体 JD，因此这里做的是目标岗位能力映射，不伪造公司面试流程或岗位关键词。

3. Frozen inputs 与当前工作区检索

Source	SHA-256	能支持	不能支持
optimized 简历 [1]	`8e9c029d40163c83e9c689e22d326bca23efaf248f046d3fa40b402bd4b470a1`	当前安全降级后的三条 WAM 表述	任一本地数字或完成态 ownership
optimized 投递 PDF [11]	`65b1dcd34cd6e357c6264effcf4b2683dea66508c6bc58315762b5e4c03ac74a`	WAM 安全降级与 OpenPiVLA 安全重写已同步到可投递 A4 单页 PDF	项目 raw evidence
WAM 项目页 [2]	`251cbcac43a1acb2052e7866e2d5a9a6f71c01fd07ef0cdb0035d66f5ebc61a5`	evidence-gated 叙事、失败假设、artifact queue	artifact 实际存在或结果已复现
resume restructure plan [3]	`e31ccb40381dc6abe9fd6b9622ab48b9a6cfc8fe71039f6d97db8722450e43e6`	当前安全口径与 evidence queue	checkpoint/episode 归属
research resume [4]	`9a8828ae0b6317c107ab16ec3d106cad8ba944854469e7919a7ba0c1342b643f`	full10k/loss/sample-count 历史表述	与 MACA 1-epoch 分支的关系
engineering resume [5]	`2aa3f0dae040ad5fb6bff32d68b5f4fff574bf4b2be80abc93d9f29b96f78732`	同上历史表述	可加载 checkpoint 与正式 eval
Cosmos source audit [6]	`7f620f4ded9766f71bb75b0a927c168b8e5dec1875257dc94e2c8f35fdfd686f`	frozen upstream mechanism、repo/model revision、protocol boundary	用户本地训练数字与 ownership
Benchmark/evidence audit [7]	`58a5636e953f44c2aa27e3acf65c621460186daf3732b916eac70e6ffab2de6a`	paper→repo→checkpoint→runtime 证据合同	当前缺失的 local trained runtime artifacts
Official-HF baseline P0 audit [11]	`39E59144D6CE36135A09314294C5E399BC1F2D686A55486C919DA5F5A63972B8`（canonical audit）	24×5 logs、raw/trusted recount、protocol tuple、action-debug integrity	checkpoint binary/hash、完整 episode JSONL、candidate ownership
TurnOffMicrowave trained-failure P0 audit [12]	`80789C958CBDBBEF9FEDEDBA13D6BAFBECAB64FAE1A45E5AFF0C3470FFF57CAB`（canonical audit）	local 1k fixed-batch gate、`0/1` closed-loop、5-frame counterfactual、same-input first divergence、state mining 与 corrected overfit	immutable checkpoint binary/hash、matched patch→rerun、多 episode scorer、candidate ownership
Checkpoint save-load gate P0 audit	`D5A4DB639C3C0C152D55F3F9C6F6BEDD8E18408F306036A28DA5154BAB3D9AAC`	13 base-load summaries、715-key clean load、1/5-step smoke、9907-step save2k resource line、save1500 `state_dict(destination=...)` first	produced checkpoint、fresh-process reload parity、matched eval、patch ownership

Source	SHA-256	能支持	不能支持
[13]	（canonical audit）	divergence、0 checkpoint-like binaries	
Full10k trained eval Po audit [16]	`DB83888B1279E548EBCA6426009448936 6905C632C5CC4F9E44F834BC12D66CF`（canonical audit）	full10k `iter_000010000` remote path 下24×5 status/log/summary 对齐，raw `2/120`、local-policy trusted `0/118`、24 action-debug/4066 lines/32×7 finite	checkpoint binary/hash、fresh-process reload、official scorer equivalence、完整 episode JSONL、matched uplift、candidate ownership

2026-07-17 对 `C:\Users\mousongzhe\cv` 做文件名与全文检索时，确实未找到这些外部 Project artifacts；这个历史结论只覆盖当时的 `cv` 工作区。2026-07-21 对 `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts` 做了独立只读审计：full10k trained run 有 24 条 `*-5ep-alltasks-*` logs、24 个 action-debug JSONL、24/24 formal `exit_code=0`，raw `2/120`；两个 `CoffeeServeMug` length=1 success 按当前本地 policy 排除后为 `0/118`。同一目录另有 25 个 command manifest、14 个本地同步 media 和 169-entry light archive；仍没有 `.pt/.ckpt/.safetensors`，不能代替完整 120 episode media/JSONL。official-HF baseline 保持独立 bundle 与 `86/120` raw、`84/118` trusted 口径。

安全降级前的 optimized resume、项目页与 restructure plan SHA 分别为 `58c004d85b60...`、`1de96b9cc2b5...`、`96cb4f8c3168...`；它们保留在本 review 的变更历史中，用于证明本次处理确实移除了投递稿的未核验 C3 表述，而不是重写证据来源。

4. Claim reconciliation matrix

ID	当前 claim	出处	等级	当前证据	Verdict	处理动作
`WAM-CL-001`	复现 RoboCasa WAM 1-epoch 后训练	optimized/project/旧简历 + save-load gate Po audit	C1/C2	8-GPU run contract、base-load、smoke、formal save2k/save1500 logs 可复核；长训与 checkpoint gate 失败	`LOCAL-ARTIFACT-BACKED / GATE-FAILED`	可说“开展并审计复现 gate”；不能说完成 1 epoch 或有可加载 checkpoint
`WAM-CL-002`	设计 action/world/value 三目标、mask 互斥和 slot routing	optimized/project	C2	上游机制已核实；无 local diff	`ownership-unverified`	改为“按 upstream contract 梳理/对齐”；不得把 slot/mask 称个人原创
`WAM-CL-003`	held-out policy MSE `0.09718→0.05349`，下降约 45%	pre-downgrade resume/project/plan	C3	无 split、metric script、pred/label、checkpoint	`C3-BLOCKED / REMOVED`	已从当前可投递稿移除；四类 artifacts 齐全后才可恢复
`WAM-CL-004`	short-horizon 任务 `0.90-0.95`、pick/place `0.75`	pre-downgrade resume/plan	C3	没有 task ID、trial、checkpoint、summary；历史项目页同时记录 local 0-success	`CONFLICT / REMOVED`	已从当前投递稿移除；先确认是否属于 official checkpoint、其他 run 或误写
`WAM-CL-005`	official HF baseline 24×5 raw `86/120`、trusted `84/118=71.19%`	project page + official-HF Po audit	C2/C3	24 task logs/status/action-debug 可复核；checkpoint 只有 remote path，2 个 artifact success 的 exclusion 规则是本地审计口径	`REMOTE-ARTIFACT-BACKED / CHECKPOINT-UNVERIFIED`	可说“运行并审计 official-HF eval harness”；不能说个人训练成绩或 uplift；补 revision/hash、完整 episode JSONL 与 ownership 后再升级
`WAM-CL-006`	本地训练 checkpoint TurnOffMicrowave closed-loop `0/1`	project page + TurnOffMicrowave Po audit	C2	1 episode、500 steps、32 queries、action-debug 与 same-input summaries 可复核；checkpoint 仍 remote-referenced	`LOCAL-ARTIFACT-BACKED / CHECKPOINT-UNVERIFIED`	可讲最小失败复现与 first divergence；不能写 benchmark success rate 或本地 checkpoint hash
`WAM-CL-007`	failure first divergence 收窄到 5-frame contract 与 exact-observation transfer gap	TurnOffMicrowave Po audit	C2	fixed batch sampled `-0.8277`、official/local exact first `-1.0156/+0.1429`、best-of-8 全正、top-24 negative `37.5%`；没有 matched fix→rerun	`PARTIALLY-CONFIRMED / ROOT-CAUSE-OPEN`	说“已确认 conditioning 差异与 transfer gap”，不说唯一 root cause 或已修复
`WAM-CL-008`	在 MACA/C500 修复 dataloader cycle OOM 与 DeepSpeed state_dict	project page + 1 epoch freeze + save-load gate Po audit	C1/C2	no-cycle 5-step `exit=0`、save2k rank -9/ `EXIT=247`、save1500 `state_dict(destination=...)` TypeError/ `EXIT=1` 可复核；patch diff/修复后 parity 仍缺	`LOCAL-ARTIFACT-BACKED / PATCH-UNVERIFIED`	可讲故障边界与验证设计；不能说修复已完成
`WAM-CL-009`	full10k checkpoint 保存成功，iter 10000 loss=0.0543，sample counts 4950/2519/2531	research/engineering resume	C3	只在旧版简历；项目页又说 C500 1-epoch 无可加载 final checkpoint	`RUN-PROVENANCE-OPEN`	区分 hardware、run ID、base checkpoint、保存格式；不能合并为同一训练线
`WAM-CL-010`	“我负责”官方路径复现、routing 与失败诊断	project page	C1/C2	无 ownership ledger 或 local diff	`ownership-unverified`	候选人按“亲手/协作/仅理解/工具辅助”逐项确认
`WAM-CL-011`	full10k trained checkpoint path 下 RoboCasa 24×5 raw `2/120`、local-policy trusted `0/118`	Full10k trained eval Po audit	C2	status/log/summary/action-debug 对齐；checkpoint 只有远端 path，`length<5` exclusion 是本地审计 policy，完整 episode JSONL/scorer/ownership 缺失	`REMOTE-ARTIFACT-BACKED / CHECKPOINT-UNVERIFIED`	可说“运行并审计 full10k trained eval harness”；不能写官方 benchmark、checkpoint ownership 或相对 official-HF 的 uplift

5. 三个关键冲突

5.1 `0.90-0.95 / 0.75` 与 local 0-success

两组表述不一定逻辑上绝对矛盾：成功率可能来自 official checkpoint、另一 local checkpoint、不同任务子集或不同协议。但当前材料没有给出身份元组：

```
(run_id, checkpoint_sha, model_role, task_id, init_set, trial_count,
 horizon, controller, success_predicate, summary_path)
```

因此现在无法判断它们能否并存。2026-07-17 已把成功率从投递稿移除；在身份元组和 raw episode records 完整前，不恢复该表述。

5.2 full10k eval 与 C500 final checkpoint 缺失

旧版 research/engineering resume 说 `full10k checkpoint 保存成功`；新增 remote-artifact audit 证明该独立 run 的 `iter_000010000` path 进入了完整 24×5 eval，raw `2/120`、当前本地 policy trusted `0/118`。项目页的高压回答说 MACA/C500 1-epoch 主线“当前没有可加载 final checkpoint”。这仍是两条不同硬件或不同阶段的 run：full10k 的 path、训练/eval logs 和 8-GPU save1500 failure 不能互相替代，且 full10k checkpoint binary/hash、fresh reload 和完整 scorer bundle 仍未闭合。面试中不得把 full10k 的 `0/118` 直接与 official-HF `84/118` 做 uplift。

5.3 “三个 head”与 upstream slot/mask contract

frozen source audit 显示 Cosmos Policy 更接近在统一 latent video sequence 中注入/抽取 proprio、action、future 与 value，并用 condition/loss mask 和 sample routing 切换任务；把它简化成独立的 action/world/value 三个 head 适合初级直觉，但不能当作实现事实。[6] 面试安全说法是“共享 diffusion backbone，隔离的是 latent slots、condition/loss mask 和 sample route”。

6. Truth boundary 与安全口径

内容	当前状态	简历	面试	证据到位后
阅读并解释 Cosmos Policy action/future/value contract	`upstream-verified`	可写为技术栈/理解，不作项目成果	可以机制级解释	无需升级为个人原创
开展 local WAM 复现	`local-claim-only`	仅在候选人确认后用“开展/尝试复现”	说明当前材料缺 raw bundle	code/config/log/checkpoint 到位后升 C1
MSE 45%	`C3-BLOCKED / REMOVED`	已从当前简历移除	不主动报数	split/script/prediction/checkpoint 齐全后恢复
RoboCasa 0.90–0.95 / 0.75	`CONFLICT / REMOVED`	已从当前简历移除	不报数	checkpoint/task/trials/episodes 完整后重新判定
official HF 71.19%	`remote-artifact-backed / checkpoint-unverified`	不写成本人模型成绩	可说按官方 checkpoint path 跑过并重算 eval；必须说明 86/120 vs 84/118 口径和缺失 local binary/hash	revision/hash + 完整 episode JSONL + scorer/version + ownership 后才可写“复现官方 checkpoint”
full10k trained 24×5 `2/120` raw、`0/118` local-policy trusted	`remote-artifact-backed / checkpoint-unverified`	可写“审计 full10k trained eval harness”，不写官方 benchmark	说明两条 length=1 artifact、local invalid policy、24×5 protocol 与 remote checkpoint path；不把结果写成官方 scorer 或 uplift	checkpoint binary/hash + fresh reload + versioned scorer + complete episode JSONL + matched baseline + ownership
local 0-success 与 root cause	`claim-only / inference`	不写性能结果	可以讲诊断框架，原因用“候选/待确认”	修复前后 paired rerun 后写 confirmed/partially confirmed
上游 routing/mask	`upstream-verified`	不写“我设计”	说“我按 upstream contract 审计/对齐”	local substantive diff 到位后再描述个人修改

当前仍不能升级成强结果项目的原因

1. 最强的四组数字都没有 raw artifact。
2. 历史 optimized resume 与项目页对本地 closed-loop 结果没有统一 provenance；当前简历已降级，但事实缺口仍未关闭。
3. full10k 与 MACA/C500 训练线没有 run ledger，可能被面试官一问即穿。
4. upstream architecture 与 personal contribution 没有 diff/ownership 分界。
5. 现有诊断是合理假设链，但没有修复后的因果对照。

当前保守表达

仅在候选人确认“确实亲手做过该复现”后，当前可用的最低风险版本是；optimized resume 已按这一边界落地：

> 基于 Cosmos Policy / RoboCasa 公开 recipe 开展 WAM 后训练复现与失败诊断，重点梳理 action/future/value latent slot、condition/loss mask、action normalization 与 closed-loop eval contract；当前成果定位为复现与诊断，不宣称本地策略成功率提升。

这句话是 C0/C1 边界表达，仍需候选人确认 ownership；它刻意不包含 45% MSE、0.90–0.95、0.75、71.19% 和已确认 root cause。

7. P0 artifact manifest

Priority	Artifact	最低字段	关闭哪些 claim	当前状态
P0	`wam_run_manifest.json`	run ID、hardware、repo commit、base/model checkpoint SHA、config SHA、dataset/stats SHA、command、start/end/status	CL-001/009	`MISSING`
P0	local code/config diff	changed files、diff、为什么改、author/tool boundary	CL-002/008/010	`MISSING`
P0	MSE eval bundle	split manifest、metric code SHA、baseline/prediction/label、per-dim/aggregate result	CL-003	`MISSING`
P0	task-success provenance	checkpoint、task IDs、init、trial count、horizon、controller、success predicate、episode records	CL-004/011	`PARTIAL / full10k remote-artifact-backed; binary/hash, scorer version and complete episode records missing`
P0	full10k trained eval bundle	24×5 status/log/summary/action-debug、checkpoint/stats/embedding identity、raw/trusted recount、run-family join key	CL-011	`PARTIAL / REMOTE-ARTIFACT-BACKED`; current local policy excludes two length=1 artifacts; checkpoint/reload/scorer/ownership remain open
P0	official-HF eval bundle	HF repo+revision、config/stats SHA、24×5 episode records、raw/trusted exclusion reason、summary/videos	CL-005	`PARTIAL / REMOTE-ARTIFACT-BACKED`
P0	local 0-success bundle	local checkpoint load log、same-protocol episodes、action/gripper hist、first-divergence trace	CL-006/007	`PARTIAL / TURN-OFF-MICROWAVE ARTIFACT-BACKED`; immutable checkpoint、matched fix→rerun、multi-episode denominator 与 ownership 仍缺
P0	checkpoint save-load gate bundle	base-load summaries、run contracts、short-smoke exit/metrics、formal save logs、checkpoint file manifest、fresh-process reload	CL-001/008/009	`PARTIAL / BASE-LOAD-BACKED`; current archive has 0 produced checkpoint-like binaries and no reload parity
P1	training curves	total/action/world/value diagnostic scalars、step、smoothing rule	CL-001/009	`MISSING`
P1	debug patch bundle	OOM/state_dict before log、patch/diff、rerun log、checkpoint load smoke	CL-008	`MISSING`
P1	ownership ledger	亲手实现、协作、只阅读、Codex/工具生成、上游未改五类	CL-002/010	`MISSING`

Evidence promotion pseudocode

```
def promote_claim(claim, bundle):
    # 1. A metric claim must bind to one immutable run and checkpoint.
    require(bundle.run_id and bundle.checkpoint_sha and bundle.config_sha)

    # 2. Evaluation must expose the complete protocol and raw episode records.
    require(bundle.protocol and bundle.episode_records and bundle.metric_code_sha)

    # 3. Personal ownership must be separated from frozen upstream code.
    require(bundle.ownership_ledger and bundle.local_diff)

    # 4. Only then may a C2/C3 claim enter the resume or strong oral answer.
    return "verified" if all_checks_pass(bundle) else "claim-only"
```

该伪代码表达的是证据门，不是用户项目中已经存在的实现。

8. 必须回答的六个问题

1. `0.90–0.95 / 0.75` 分别对应哪些 RoboCasa task、哪个 checkpoint、每项多少 trial、何种 init/horizon/controller?
2. `full10k checkpoint` 与 MACA/C500 “无可加载 final checkpoint”是否是两条 run？各自的硬件、base checkpoint、run ID 和保存路径是什么?
3. `0.09718→0.05349` 的 held-out split 怎样生成，MSE 对哪些 action steps/dimensions 求平均，prediction/label 文件在哪里?
4. `84/118` 被排除的 2 条 episode 是什么异常? `86/120`、`84/118` 的 raw summary、视频与 model revision 在哪里?
5. 哪些 WAM code/config 是你亲手改的，哪些来自 upstream，哪些由 Codex/同事/脚本完成?
6. gripper/high-sigma/conditioning/DeepSpeed 中，哪些只是假设，哪些有修复后的 paired rerun 能确认因果?

9. Preliminary SP/SSP project scorecard

这是“项目证据就绪度”而不是候选人能力评分；由于 raw artifacts 缺失，置信度为 `Medium-Low`。

维度	分数（5）	依据
岗位相关性	4.0	WAM、multi-objective、RoboCasa eval 与 VLA 岗位高度相关
机制深度	3.5	source audit 已达到 slot/mask/sampler/protocol 层，但项目页仍有 three-head 简化
工程闭环	2.0	有诊断 bundle 与叙事，但缺完整 run bundle 与可加载 checkpoint
指标可信度	1.0	四组本地/复现数字均无 raw artifact，且一组存在 provenance 冲突
Ownership 清晰度	1.5	有自述，缺 local diff 与贡献分类
抗追问能力	2.5	边界意识较好，但数字和 run 身份会被高压追问击穿
合计	**14.5 / 30**	投递稿已安全降级，但当前仍不具备强结果项目的 raw evidence

强结果升级阻断项：full10k/C500 run provenance 未拆开；现有 local direct 仅覆盖一组 CoffeePressButton diagnostic bundle，checkpoint/metric/denominator/ownership 仍未闭合。原 `WAM-CL-004` 投递 blocker 已通过安全降级关闭。

9.5 Artifact Discovery Addendum（2026-07-18）

本 addendum 不回溯改写 2026-07-17 的 `cv` 工作区扫描，而是记录一次外部 Project 目录的只读发现。详细文件、SHA、manifest 字段和缺口见 WM-019 WAM Diagnostic Artifact Manifest；同一输入的结构/逐步差异计算见 WM-019 WAM Action-Trace Paired Diagnostic，离线 future-image PSNR/SSIM 对照见 WM-019 WAM Future-Metric Paired Diagnostic，两侧 `iter_000010000` 的 exact checkpoint-path bridge 见 WM-019 Matched-Checkpoint Protocol Bridge。

发现的代表性文件包括 CoffeePressButton base/trained diagnostic bundle、full10k trained 的 `eval_summary_conservative.json` / `formal_5ep_status.tsv` /24 条 task logs/24 个 action-debug JSONL、official-HF baseline 的 `eval_summary_officialhf_baseline.json` / `formal_5ep_status.tsv` /24 条 task logs/24 个 action-debug JSONL，以及 TurnOffMicrowave local 1k 的 fixed-batch、query-probe、state-mining 与 `0/1` eval summaries。四类证据必须分开：CoffeePressButton 是单 episode 失败诊断，full10k trained 是远端 `iter_000010000` path 下的 24×5 eval（raw `2/120`、local-policy trusted `0/118`），official-HF 是另一官方 checkpoint path 下的 24×5 eval，TurnOffMicrowave 是 local checkpoint 的一条可复核失败链。full10k、official-HF 与 TurnOffMicrowave 的 checkpoint、stats 与 T5 仍含 remote path reference；没有 local binary/hash，selected/local videos 也不能替代完整 episode bundle。

因此：

- `WAM-CL-001` 可新增一条 `local-direct diagnostic / checkpoint-unverified` 证据线，但不能升级为完成 WAM 复现；
- `WAM-CL-005` 可从 `local-claim-only` 提升为 `remote-artifact-backed / checkpoint-unverified`，但不进入个人模型成绩或 uplift；
- `WAM-CL-011` 可登记为 full10k trained 24×5 的 `remote-artifact-backed / checkpoint-unverified`，raw/trusted 两套口径必须同时保留，不能与 official-HF 结果直接做因果 join；
- `WAM-CL-003..004/008..009` 的历史 MSE、task-success、uplift 与 full10k/C500 join 仍保持 `REMOVED`、`local-claim-only` 或 `RUN-PROVENANCE-OPEN`；`WAM-CL-006/007` 已有一条 local artifact-backed 的 TurnOffMicrowave 失败/first-divergence 证据线，但仍不升级为 benchmark 或 fixed root cause；
- action trace 和 episode-0 `success=false` 只能支撑接口/失败诊断，不能构成 benchmark denominator、模型比较或 0-success 结论；
- base/trained trace 的 `MAE=0.3100783611`、`RMSE=0.3906446459` 只是 JSONL 几何差异，不能替代 target-based action loss、checkpoint causal comparison 或 success uplift；
- `iter_000005000` 与 `iter_000010000` 的单 sample future-only PSNR 为 `8.5813` 与 `8.2140`、SSIM 为 `0.04527` 与 `0.04286`；它们是 offline visual proxy，样本数为 1，不能替代 action/simulator/real success。
- ownership 仍需要逐文件 ledger，不能由 artifact 所在目录或 Git author 推断。

10. 分阶段补强计划

时间	动作	完成标准	状态
半天	回答六个问题；补充 WAM diagnostic manifest；建立 run/metric/ownership 三张 manifest；先从投递稿降级冲突数字	每个数字都有唯一 run ID 或明确删除	`部分完成 / P0 仍待补`
1 天	收集 config、command、checkpoint/load log、MSE script、official/local eval summary	P0 artifact 表至少关闭 4 项	`待补做`
3 天	在同 split 重算 repeat/model MSE；做 official checkpoint 同协议重放；生成 per-episode ledger	数值可由命令重算，异常 episode 有原因	`待补做`
1 周	在可加载 local checkpoint 上做 action-only/+world/+value/all 或最小 paired ablation，并记录 closed-loop failure trace	可以区分表示收益、接口错误和 closed-loop 失败	`待补做`

任何补强只有在 artifacts 实际生成后才能改为“已做”。

11. 下一切片

2026-07-21 已完成五个独立 bounded slices：Cosmos 1 epoch evidence freeze 冻结 9907-step gate、step-1900 cgroup-OOM 与 step-1500 `state_dict` 保存失败；full10k trained eval Po audit 审计远端 `iter_000010000` path 下24×5 raw/trusted 与4066行 action-debug；official-HF RoboCasa baseline Po audit 重算 24×5 raw/trusted 结果并冻结 action-debug integrity；TurnOffMicrowave trained-failure Po audit 把 local `0/1` 收窄到 fixed-batch 可学但 official 5-frame exact-observation 不迁移；checkpoint save-load gate Po audit 将 base-load、short smoke、resource failure 与 save API first divergence 分开。五者提升了 WAM 的训练/评测/诊断/保存门证据等级，但不改变 local trained checkpoint hash、1 epoch completion、paired uplift、唯一 root cause 和 ownership gates。

优先级最高的下一步是二选一：

1. 先补齐 WAM diagnostic manifest 与 full10k trained eval Po audit 的 checkpoint/hash、fresh reload、完整 denominator、versioned scorer 和 metric script，再决定是否能升级 `WAM-CL-003..004/006/011`；official-HF `WAM-CL-005` 的 24×5 remote-artifact audit 已完成，但仍需 revision/hash、完整 episode JSONL 与 ownership；或
2. 在 WAM 保持 WARN 的同时，OpenPiVLA 已完成 direct audit 与安全简历重写；后续按 direct-evidence-bounded 题卡批次推进，WAM raw bundle 仍独立排队。

WAM optimized resume 安全降级已完成；raw evidence gate 未关闭前仍不生成 WAM 强回答卡，不创建 mastery/coaching state，也不把 G09 推进到 reviewed。

参考文献（References）

1. optimized resume，当前投递稿，WAM 三条 bullet。
2. Cosmos / WAM 项目面试页，现有叙事、诊断与 artifact queue。
3. resume restructure plan，WAM 数字与谨慎边界。
4. research resume，历史 full10k/loss/sample-count 表述。
5. engineering resume，历史 full10k/loss/sample-count 表述。
6. VOL 04 Cosmos Policy / WAM frozen source audit；canonical Markdown，review JSON。
7. VOL 04 Benchmark / Evidence audit；canonical Markdown，review JSON。
8. ProjectProof evidence contract：本轮使用的 C0–C3 规则；该规则只定义证据门，不是用户项目事实。
9. Kim et al., Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning, 2026。
10. Cosmos 1 epoch evidence freeze；canonical Markdown，review JSON。
11. Cosmos official-HF RoboCasa baseline Po audit；canonical Markdown，review JSON。
12. NVlabs, `cosmos-policy@18a2acc`，Apache-2.0 code；model weights 另受 model-card license 约束。
13. optimized resume PDF，由当前 optimized HTML 重新生成；PDF SHA-256 `65b1dcd34cd6e357c6264effcf4b2683dea66508c6bc58315762b5e4c03ac74a`，A4 单页，旧 WAM 数字文本检索为 0。
14. TurnOffMicrowave trained-failure Po audit；canonical Markdown，review JSON。
15. Checkpoint save-load gate Po audit；canonical Markdown，review JSON。
16. Cosmos full10k trained eval Po audit；canonical Markdown，review JSON。

Part 6 / 49 · `interview_materials/vla/evidence/projects/wam_ownership_ledger.html` · SHA-256 `931A7FAE9C6D224EAC292D63EEFFDFA374504318F8181E6F610F15D25F7C0D07`

返回本卷目录

Canonical Markdown SHA-256: `931a7fae9c6d224eac292d63eeffdfa374504318f8181e6f610f15d25f7c0d07`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wam_ownership_ledger.md`

Cosmos / WAM Ownership Ledger：多目标训练、RoboCasa 评测与诊断边界

状态：`ownership_audited / diagnostic_artifacts_partial / candidate_confirmation_pending / WARN` 审查日期：2026-07-19 证据截止：2026-07-19 项目范围：`C:\Users\mousongzhe\Project` 中的 Cosmos Policy / WAM 训练适配、RoboCasa rollout、Cosmos/WAM diagnostics 与当前 CoffeePressButton paired artifact。结论先行：Cosmos Policy 的 latent slot、condition/loss mask、policy/world/value routing、32→16 action queue 和 RoboCasa controller 合同由 frozen upstream source 支持；本地 Project snapshot、训练脚本、报告和一组 CoffeePressButton base/trained action trace 可直接读取，但 checkpoint binary、metric denominator、完整 episode suite、local diff 与候选人逐项角色仍未闭合。当前可以讲“训练/评测协议迁移与失败诊断”，不能把 WAM architecture、历史 MSE/成功率或单 episode trace 说成个人原创、benchmark uplift 或已验证 checkpoint 结果。

0. 候选人项目范围自述（self-report，不等于 artifact/ownership 证明）

2026-07-21，候选人在本线程明确给出以下项目范围自述：

> “我复现了 Cosmos Policy 的 1 epoch fine-tune，做了 Cosmos Policy 的 eval，做了 UWM 在 LIBERO 的 fine-tune 和 eval，做了 UWM 在 RoboCasa 的尝试，协议对齐和训练评估；具体的指标与日志应该在对应项目下有落档。”

这条自述用于确定面试准备的项目覆盖面和 artifact 搜索优先级；它不自动证明具体代码 ownership、checkpoint lineage、指标分母、运行者或结果归因。当前安全映射如下：

自述范围	当前证据层	现在可用的安全动词	仍不能直接说
Cosmos Policy 1 epoch fine-tune	`candidate-self-report` + 本地训练 gate / save-load audit 部分支持	“开展/尝试复现 1-epoch 训练链，并审计训练 gate”	已完成正式 1 epoch、拥有可加载 checkpoint、取得训练 uplift
Cosmos Policy eval	`candidate-self-report` + RoboCasa harness/diagnostic artifacts 部分支持	“运行/审计 Cosmos Policy eval harness”	个人模型 benchmark 成绩、官方 scorer 等价、相对 baseline 提升
UWM / LIBERO fine-tune 与 eval	`candidate-self-report` + UWM/LIBERO protocol inventory	“开展 LIBERO 适配、训练与评估对齐”	strict/reset 结果归因、checkpoint ownership、250 episodes 全量闭合
UWM / RoboCasa 尝试	`candidate-self-report` + RoboCasa actionheadfix / transfer diagnostics	“尝试 RoboCasa 适配并做协议/失败诊断”	已解决 root cause、完整成功率或稳定迁移结果
协议对齐与训练评估	`candidate-self-report` + 多项目 protocol/evidence pages	“对齐输入输出、normalizer、controller、评估协议并复核日志”	所有实现由候选人独立完成，或所有指标已经 result-owned

在候选人逐项回答第 6 节 ownership questions 前，这五行保持 `candidate-self-report / role-pending`，不迁移为 `role-confirmed` 或 `result-owned`。

1. Ownership 证据合同

强主张“我负责并完成了 X”至少需要同时满足：文件/commit identity、可读取 artifact、设计或修改过程、运行/验收证据、个人/协作者/工具边界。Git author、项目报告、远端路径或一条 summary JSON 均不能单独证明个人 ownership。

状态	含义	当前可用动词
`upstream-verified`	frozen Cosmos paper/repo/model card 支持的公开机制或协议	“上游定义/实现”
`snapshot-local`	Project Git snapshot 中存在脚本、配置、报告或输出	“本地快照包含”
`artifact-local-partial`	可读取 manifest/trace/video，但 checkpoint 或 protocol bundle 不完整	“复核到诊断 artifact”
`report-supported`	只有项目报告/历史材料支持	“材料记录/项目描述”
`role-confirmed`	候选人逐项确认亲手、协作、工具、运行或审查角色	“我修改/我运行/我验收”
`result-owned`	checkpoint、normalizer、protocol、episode、scorer 与角色全部闭合	“我完成并得到……”

2. Git、上游与工作树冻结事实

检查	结果	能证明	不能证明
Project HEAD	`6c524d9c6c6226075f100cdac395fef2fdc37671`，`Document RoboCasa actionheadfix diagnostics`	当前提交包含 WAM/UWM 诊断报告和 loss summary	每个文件由候选人独立设计、编写或运行
Project initial snapshot	`c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`	初始快照与 WAM 目录边界	pre-snapshot 来源、协作者和工具链
Cosmos/WAM training line	`wm/cosmos_policy_*` scripts、`cosmos_policy_deepspeed_train.py`、RoboCasa dataset/rollout scripts	本地存在训练/推理适配入口与参数合同	正式 1-epoch、full10k 或 checkpoint ownership
RoboCasa evolution	`796374c` → `57bf4ea` → `a37a386` → `2a2912d` → `1a03fe` → `8eaddf` → `6c524d9`	actionheadfix、single-task gate、sharded eval、loss reports 经多次提交演化	每次修改由谁决定、谁在 remote 执行、谁验收
Current worktree	`Project` 当前有 recovery 文件修改；WAM outputs 在外部目录	working-tree 状态和外部 diagnostic 文件存在	dirty recovery 或远端路径不等于可复算 raw run
Frozen upstream	`NVlabs/cosmos-policy@18a2accadf4e7a3531e56754102af5a24d2316da`；论文 arXiv `2601.16163v1`	latent slot、mask、routing、direct/planning、RoboCasa 32→16 合同	用户本地改动、结果、ownership 或 license 可自动继承

3. Per-file / per-decision ownership matrix

ID	文件 / 决策	当前可核验事实	Git / artifact 事实	当前 role verdict	安全口径	升级所需证据
`WAM-OWN-001`	Project initial snapshot 与 WAM 目录来源	`c4469eb` 是根快照；包含 `wm/cosmos_policy_*` 与 RoboCasa 适配目录	一次性 snapshot；无 pre-snapshot history	`snapshot-local / role-pending`	“项目快照包含 Cosmos/WAM 训练与评测材料”	pre-snapshot 来源、贡献记录、候选人确认
`WAM-OWN-002`	Cosmos Policy upstream mechanism	paper/repo 支持 unified latent video sequence、action/future/value slot、sample route、condition/loss mask	frozen commit `18a2acc…`；上游 Apache-2.0；model card 另有 NSCLv1	`upstream-verified / upstream-only`	“我按 upstream contract 梳理/对齐”，不说原创 architecture	永远不升级为个人原创；只绑定本地 substantive diff
`WAM-OWN-003`	`wm/cosmos_policy_deepspeed_train.py` 训练适配	参数包含 chunk=32、action multiplier、world/value sampling、normalization、DeepSpeed 与 dataset contract；代码是 proxy/objective-compatible harness	Project snapshot 可读；未闭合 full model run	`snapshot-local / implementation-role-pending`	“本地有 DeepSpeed 训练适配与目标合同”	before/after diff、运行命令、effective config、load/checkpoint log
`WAM-OWN-004`	RoboCasa dataset / action schema	`uwm_robocasa_lerobot_dataset.py`、`uwm_robocasa_dataset.py`、tasksets 固定图像/proprio/action/normalizer 与 7D→12D controller 边界	多次 `796374c` → `6c524d9` 提交	`snapshot-local / decision-owner-pending`	“项目适配了 RoboCasa 数据与动作接口”	数据 manifest、schema roundtrip、候选人具体改动和 review
`WAM-OWN-005`	action rollout launcher	`run_cosmos_policy_robocasa_action_rollout_now.sh`、`cosmos_policy_robocasa_action_rollout_video.py` 提供 32 chunk、16 open-loop、5 denoise 与 episode output	入口脚本和日志可读取；远端 checkpoint path	`process-supported / operator-pending`	“项目提供可审计的 rollout launcher”	remote shell history、runtime env、完整 episode manifest、运行者
`WAM-OWN-006`	official checkpoint / HF baseline eval path	wrapper、task list、controller、success predicate 与 24×5/50×3 协议在材料中可定位	上游 model card 数字不是本地运行；本地 historical 84/118 仍 claim-only	`upstream+report-supported / result-owner-blocked`	“我建立了 harness 验证路径”，不报历史分数为个人结果	model revision、checkpoint/config/stats hash、raw episodes、summary/videos
`WAM-OWN-007`	training report / loss summary	`uwm_robocasa_transfer_report.html`、`human300_*` summary 与图表记录 action/aux/lr/memory 线索	`6c524d9` 加入报告与 PNG/CSV/JSON；不是独立 raw checkpoint	`report-supported / analysis-role-pending`	“报告记录训练诊断线索”	run manifest、metric code、原始 logs、checkpoint lineage、审查角色
`WAM-OWN-008`	WAM historical MSE claim `0.09718→0.05349`	只出现在旧简历/项目叙事，缺 split、prediction/label、metric script	inventory 判定 `C3-BLOCKED / REMOVED`	`local-claim-only / result-blocked`	当前不报数；只讲应如何建立 held-out metric bundle	split manifest、metric SHA、pred/label、checkpoint/config/stats SHA
`WAM-OWN-009`	historical success `0.90–0.95 / 0.75` 与 `84/118`	历史材料中的 task/denominator/checkpoint 未闭合，且与 local 0-success 叙事存在 provenance conflict	inventory 判定 `CONFLICT / REMOVED`	`claim-only / protocol-blocked`	不恢复到简历或强回答	task IDs、trial/init/seed/horizon/controller、raw episodes、invalid rule、model revision
`WAM-OWN-010`	local 0-success / root-cause chain	项目页提出 gripper、high-sigma、5-frame conditioning、normalizer/slot roundtrip、OOM/state_dict 等候选原因	未发现完整 fix→paired rerun 因果链	`inference / diagnosis-pending`	“我列出并排序了诊断假设”，不说已定位根因	failure trace、patch、unit test、修复后同协议 rerun

ID	文件 / 决策	当前可核验事实	Git / artifact 事实	当前 role verdict	安全口径	升级所需证据
WAM-OWN-011	CoffeePressButton base/trained manifests	两个 manifest SHA：base 63A518…33A86、trained 809AE4…55CC；同 task/episode/seed/env_seed/max_steps/open-loop/denoise；均 success=false	C:\Users\mousongzhe\Project\wm\outputs 可读；checkpoint 字段是远端路径	artifact-local-partial / checkpoint-unverified	“复核到一组 paired diagnostic manifest”	checkpoint binary/immutable remote hash、processor/normalizer/config SHA、完整 suite
WAM-OWN-012	CoffeePressButton action trace 与 video	base trace SHA C6A362…88525D、trained trace SHA 9C6A4F…82CF2D；300×7 policy、300×12 env、无 NaN/Inf；trained video SHA 591763…2877	action trace MAE 0.3100783611、RMSE 0.3906446459；两边 episode 0 均失败	local-direct / diagnostic-only	“trace 几何差异与失败字段可复核”，不称 uplift/MSE/benchmark	target/pred metric、controller roundtrip、多 episode denominator、same-checkpoint paired run
WAM-OWN-013	iter_00001000 0 matched path bridge 与个人总边界	offline future metric 与 trained rollout 逐字引用同一 checkpoint path；path string SHA 6CAE0B…14E833	这是路径 fingerprint，不是 binary SHA；offline n=1，输入/seed/horizon 与 rollout 不同	local-direct / protocol-limited / candidate-confirmation-pending	“我能把 offline 与 rollout 绑定到同一路径，但不能做相关/因果结论；个人 ownership 仍按前 12 行确认”	binary hash/load smoke、matched protocol、candidate role ledger、reviewer/operator 记录

4. Claim → ownership reconciliation

面试/简历 claim	当前证据合同	安全处理
“我设计了 action/world/value 三目标与 slot routing”	上游机制已核实，本地缺 substantive diff 与 ownership	改为“按 upstream contract 梳理/对齐”；确认本地改动后再描述修改
“我完成了 Cosmos/WAM 后训练”	有训练适配入口与报告，但缺 immutable run/checkpoint/metric chain	说“参与/开展训练适配与评测诊断”，不说完成正式训练结果
“MSE 下降约 45%”	split、metric code、prediction/label、checkpoint 缺失	当前移除；补齐四类 artifact 后重新判定
“RoboCasa 达到 0.90–0.95 / 0.75 或 84/118”	task/seed/trial/checkpoint/episode provenance 未闭合且存在冲突	禁止恢复；先建立 protocol-complete raw bundle
“trained action trace 比 base 更好/更差”	只有单 episode trace 差异，checkpoint binary 未验证，两边 success 都 false	只说 trace-level diagnostic，不做模型效果结论
“我定位了 0-success 根因”	目前是候选假设链，无修复前后 paired rerun	说“形成诊断矩阵/优先级”，原因用“待确认”
“Git author 证明这些代码都是我写的”	多次提交和一次性 snapshot 只能证明提交身份	必须按文件、commit、运行者和工具边界回答
“iter_00010000 的 offline metric 解释了 rollout 失败”	只证明 path string 相同；n=1 且 episode/seed/horizon 不同	只说 checkpoint-reference bridge，不做相关或因果结论

5. Production gap 与最小补证据

Gap	面试官追问	最小补证据	当前表达
checkpoint lineage	base/trained 的权重和 normalizer 是什么?	binary/remote content hash、processor、stats/config SHA、load smoke	checkpoint-unverified
metric contract	MSE 对哪些 action steps/dims 求平均?	split manifest、metric script SHA、pred/label、valid denominator	C3-blocked
episode denominator	84/118 为什么删 2 条?	预注册 invalid rule、逐 episode JSONL、视频与 scorer	conflict
action semantics	7D policy 如何变成 12D controller action?	schema、normalizer roundtrip、controller config/hash、timestamped trace	diagnostic pending
root cause	gripper/high-sigma/5-frame 谁被验证了?	failure trace、patch、unit test、同协议 paired rerun	inference
ownership	哪个函数是你亲手写/改的?	before/after diff、review note、候选人逐项确认	role-pending
offline→online bridge	PSNR/SSIM 能否预测 success?	matched task/init/seed、同 checkpoint、多样本 paired bundle	当前只能说 path bridge
collaboration/tool boundary	Codex/同事/远端运行者分别做了什么?	contribution ledger、shell history、operator/reviewer 记录	不推断

6. 候选人确认表

真实口述或材料核对时，请逐项回答“亲手 / 协作 / 工具辅助 / 只审查 / 只运行 / 不确定”:

1. c4469eb 初始 snapshot 中的 WAM/Cosmos 文件来自哪里? 哪些是在 796374c 之后由你修改?
2. cosmos_policy_deepspeed_train.py 的 proxy/objective-compatible 训练适配由谁设计、编写、运行和验收?
3. RoboCasa dataset、7D→12D action、normalizer、camera 与 taskset contract 中哪些字段是你的决策?
4. action rollout launcher 与 official/HF baseline harness 谁编写、谁在 remote 执行、谁检查 success predicate?
5. historical MSE、84/118、0.90–0.95/0.75、full10k 分别属于哪条 run? checkpoint、split、denominator 和运行者是什么?
6. CoffeePressButton base/trained manifest、action trace、video 是谁生成、收集、审查和保存的?
7. gripper、high-sigma、5-frame conditioning、normalizer/slot roundtrip、OOM/state_dict 中哪些有修复后的 paired rerun?

8. 哪些代码来自 NVlabs upstream、哪些来自协作者或工具，哪些 substantive diff 确实由你亲手完成?

在候选人真实回答前，不创建 `coaching_state.md`，不把任何行升级为 `role-confirmed` 或 `result-owned`。

7. 两分钟叙事骨架（当前安全版）

> Cosmos/WAM 这条线我会拆成上游机制、训练适配、评测协议和诊断四层。上游 Cosmos Policy 把 action、future、value 放进统一 latent sequence，通过 condition/loss mask 和 sample routing 切换 policy/world/value 角色；这部分属于上游，不是我的原创。我的项目材料与 Project snapshot 可以核验 DeepSpeed 训练适配、RoboCasa 数据/动作接口、32→16 open-loop rollout 和多轮 actionheadfix 诊断。现在最强的本地证据是一组 CoffeePressButton episode 0 的 base/trained manifest、300×7 action trace、失败视频和逐步差异计算；但两侧 checkpoint 仍是远端路径，只有 path fingerprint，不能证明 binary 内容或 uplift。历史 MSE 和成功率没有完整 split、denominator、episode、checkpoint 链，已从安全简历口径移除。我的个人贡献需要按 ledger 的脚本、提交、命令和验收记录逐项确认，因此当前我会把它讲成协议迁移与失败诊断，而不是已验证的 benchmark 提升。

8. 面试前 6 题

1. 为什么 Cosmos Policy 不能简单描述成 video backbone 后面接 action/world/value 三个独立 head?
2. `mask=1` 在 Cosmos dataset/training contract 中表示什么？它和 attention mask 有何区别?
3. 为什么 RoboCasa 的 7D policy action 会变成 12D controller action，且只执行 32-step chunk 的前 16 步?
4. 单 episode base/trained action MAE/RMSE 能证明 checkpoint uplift 吗？缺哪些证据?
5. 为什么 offline future-image metric 与 rollout success 不能直接做相关或因果解释?
6. 请从 `cosmos_policy_deepspeed_train.py` 或 action rollout launcher 的一个函数开始，说明哪些是你亲手、协作、工具辅助、只审查或只运行的。

参考文献（References）

1. Cosmos/WAM Claim–Evidence Inventory：历史 claim、冲突、artifact gate 与安全口径。
2. WM-019 WAM Diagnostic Artifact Manifest：CoffeePressButton manifest、trace、video 和 checkpoint 缺口。
3. WM-019 WAM Action-Trace Paired Diagnostic：300×7/300×12 trace integrity 与 MAE/RMSE 边界。
4. WM-019 Matched-Checkpoint Protocol Bridge：iter_000010000 path bridge 与 n=1/协议差异。
5. Cosmos Policy / WAM Source Audit：frozen upstream mechanism、slot/mask/routing 与 RoboCasa contract。
6. Cosmos/WAM Project Page：项目叙事与历史问题树；不替代 raw evidence。

9. Ownership 审计伪代码

```
def accept_wam_claim(claim, ledger):
    # Upstream mechanism is not personal ownership.
    if claim.is_upstream and not ledger.local_diff(claim):
        return "upstream_only"
    # A metric requires protocol, checkpoint and raw outputs, not a report number.
    if claim.level >= "C2" and not ledger.protocol_checkpoint_raw_chain(claim):
        return "downgrade_to_claim_only"
    # Personal result language waits for explicit role and verification evidence.
    if not ledger.role_confirmed(claim) or not ledger.verification(claim):
        return "candidate_confirmation_pending"
    return "claim_allowed_with_exact_scope"
```

这段伪代码是本仓库的 ownership gate，不是 Cosmos/WAM 官方实现。

Part 7 / 49 · `interview_materials/vla/evidence/projects/wm019_wam_p0_bundle_audit.html` · SHA-256 `C8716750BC7213FE920E7D1A8FBA1BBB8185B0AC2C78B0814931106EDA77029D`

返回本卷目录

Canonical Markdown SHA-256: `c8716750bc7213fe920e7d1a8fba1bbb8185b0ac2c78b0814931106eda77029d`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wm019_wam_p0_bundle_audit.md`

WM-019 WAM P0 Bundle Audit: CoffeePressButton

状态: `p0_audited / partial / checkpoint-unverified / ownership-pending / WARN` 审计日期: 2026-07-21 范围: 只审计仓库已登记的 CoffeePressButton diagnostic bundle、paired action trace 和 future-image proxy; 不新增 benchmark 结果, 不把 episode 0 升级成成功率, 也不确认个人 ownership。

1. 审计结论

该 bundle 已经满足"存在可读取诊断 artifact"的最低门槛, 但没有满足 `ELIGIBLE` 的 P0 结果门槛。可以复核 base/trained 两份 manifest、300×7 action trace、失败视频、rollout log 以及一组单样本 future-image PSNR/SSIM; 两个 manifest 的 checkpoint 仍是远端路径, 本地没有 binary/hash, metric scorer、完整 denominator、run lineage 和候选人角色也没有闭合。

因此本页的 promotion 结果是:

```
P0 status = PARTIAL
eligible metric claims = 0
eligible ownership claims = 0
allowed = diagnostic/interface evidence
forbidden = uplift, benchmark success rate, checkpoint-verified comparison, personal authorship
```

2. Bundle identity

side	task / episode	seed	env_seed	horizon	open-loop	denoise	manifest	action trace
base	CoffeePressButton / 0	195	0	300	16	5	`63A518...3A86`	`C6A362...85D`
trained	CoffeePressButton / 0	195	0	300	16	5	`809AE4...5CC`	`9C6A4F...2D`

两侧均记录 `OSC_POSE`、`control_delta=true`、三路 camera 和 `success=false`。它们是同一 episode protocol 下的 paired diagnostic, 但 checkpoint binary 不在当前工作区。

3. P0 field gate

P0 字段	当前证据	状态	不能推断
checkpoint identity	manifest 中有 base/trained 远端 path	`PARTIAL`	本地可加载、binary 相同/不同的完整 hash
parent/run lineage	输出目录、log 与 path bridge 有线索	`PARTIAL`	训练命令、parent checkpoint、完整 run DAG
code/config/data SHA	manifest/config 字段和既有项目 snapshot	`PARTIAL`	本次 rollout 使用的所有文件内容与版本
protocol tuple	task、episode、seed、env_seed、horizon、controller、camera、open-loop、denoise	`PRESENT`	多 episode、multi-seed 或跨任务性能
raw action output	base/trained `actions.jsonl`, 各 300 行	`PRESENT`	action label loss、target fit 或 controller correctness
raw video	两条失败视频, trained video 有 alternate output root	`PRESENT`	video 差异的 checkpoint 因果归因
scorer / denominator	trace integrity 与 paired MAE/RMSE 脚本结果	`PARTIAL`	action MSE、success rate、完整 suite aggregate
invalid policy	JSONL 中 finite/row/t-range 检查	`PARTIAL`	benchmark 级 invalid handling contract
operator / reviewer	未登记候选人、远端 operator 和 reviewer	`MISSING`	"我运行/我验收/我负责结果"
ownership diff	ledger 已列候选问题, 但没有候选人确认	`PENDING`	个人原创 architecture 或逐文件实现

4. Deterministic checks already supported

- base/trained 均为 300 行, `t=0..299`;
- `policy_action7` 均为 `300×7`, `env_action` 均为 `300×12`;

- 两侧 policy action 均未发现 NaN/Inf；
- 两侧 `success_after_step=True` 次数均为 0，reward sum 均为 0.0；
- paired policy trace 的 MAE=`0.3100783611`、RMSE=`0.3906446459`；这只是几何差异，不是 action-label loss；
- future-image proxy 只有单样本、12 个 future frames，PSNR/SSIM 不能与 action rollout 或 success 合并解释。

这些检查证明文件结构和诊断计算可复核，不证明 checkpoint 内容、动作语义、控制器闭环或模型优劣。

5. 面试可用与禁用表述

可以说：

> “我能从一组 CoffeePressButton diagnostic bundle 复核 base/trained 的 rollout manifest、action trace、controller/camera/horizon 字段和失败视频；我还检查了 trace 的 shape、finite 和逐步差异。由于 checkpoint 仍是远端 path，缺少完整 scorer、denominator 和 ownership，所以我把它定位为接口与失败诊断证据，不把它说成 benchmark uplift。”

不能说：

1. 本地已经 load/hash 验证 base 或 trained checkpoint；
2. trained 比 base 成功率更高/更低；
3. episode 0 失败等于 WAM 0-success；
4. `0.3100783611` 是 action MSE 或训练 loss；
5. 上游 Cosmos slot/mask/routing 是个人原创；
6. 文件存在或 Git author 自动等于本人实现和运行。

6. 关闭 P0 的最小动作

按优先级只补一项即可继续：

1. 为 base/trained 提供不可变 checkpoint hash、processor/normalizer/config SHA；
2. 为同一 protocol 收集完整 episode manifest、success predicate 和 denominator；
3. 提供 prediction/label 与 metric script，使 action error 可重算；
4. 候选人回答 `OWN-WAM-001`，明确设计、实现、协作、工具、运行和审查角色。

在上述字段闭合前，本 bundle 保持 `PARTIAL/WARN`，不进入简历强 claim，也不改变 `PROJ-WAM-001` 或通用题卡 mastery。

References

1. WM-019 WAM Artifact Manifest
2. WM-019 Action-Trace Paired Diagnostic
3. WM-019 Future-Image Metric Paired Diagnostic
4. WAM Ownership Ledger
5. 五项目 P0 Artifact Intake Contract

Part 8 / 49 · `interview_materials/vla/evidence/projects/cosmos_policy_full10k_trained_eval_p0_audit.html` · SHA-256 `DB83888B1279E548EBCA64260094489366905C632C5CC4F9E44F834BC12D66CF`

返回本卷目录

Canonical Markdown SHA-256：`db83888b1279e548ebca64260094489366905c632c5cc4f9e44f834bc12d66cf`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\cosmos_policy_full10k_trained_eval_p0_audit.md`

Cosmos Policy full10k trained checkpoint：RoboCasa 24×5 P0 评测证据审计

状态：`p0_audited / trained-run-remote-artifact-backed / checkpoint-unverified / scorer-policy-local / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计 `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts` 中 run id 为 `full10k-20260620-5ep-alltasks` 的 24 个 formal task；不把它与 8-GPU `max_steps=9907` gate、Cosmos official-HF baseline、TurnOffMicrowave local 1k failure 或 CoffeePressButton 1×1 smoke 合并为同一 run。

1. 问题（Question）

这份审计回答四个问题：full10k 训练 checkpoint path 是否实际进入了完整 24×5 RoboCasa eval；status、task log、summary 与 action-debug 是否互相对齐；raw `2/120` 与 trusted `0/118` 如何在面试中准确解释；以及哪些结果仍不能升级为 checkpoint ownership、benchmark 或训练 uplift。

2. 结论先行

当前 bundle 足够证明：一条独立的 full10k trained-run eval harness 已完成 24 个 task、每 task 5 个 episode；`formal_5ep_status.tsv` 的 24 行全部 `exit_code=0`；24 个 task log 与 conservative summary 的 task set、episode 数和 success 数一致；action-debug 的 24 个 formal JSONL 共 4066 行，全部可解析、`32×7`、finite，并且都指向同一个远端 `iter_000010000` checkpoint path。

独立重算得到：

```
raw successes       = 2 / 120 = 1.6667%
artifact successes  = 2       (CoffeeServeMug episode 1/2, each length=1)
eligible episodes   = 118
trusted successes   = 0 / 118 = 0%
conservative rate   = 0 / 120 = 0%
```

这支持的面试口径是：“我把独立 full10k trained checkpoint path 接入了固定的 RoboCasa 24×5 eval，完成了日志、status、summary 和 action-debug 的交叉审计；原始日志有 2 条 CoffeeServeMug 的 length=1 immediate success，按当前本地审计规则排除后 trusted 是 `0/118`。”

它不支持：本地持有或 hash 了 checkpoint binary；证明 full10k 训练带来 benchmark uplift；把 `0/118` 写成 RoboCasa 官方统一 scorer 的结果；把 full10k 与 official-HF `84/118` 做因果比较；或从 launcher、远端路径、Git author 推导个人独立 ownership。

3. P0 promotion 结论

```
P0 status = PARTIAL
eligible trained-run eval claims = PRESENT / remote-artifact-backed
eligible raw/trusted recount = PRESENT / local-policy-bounded
eligible local checkpoint claims = 0
eligible matched-uplift claims = 0
eligible ownership claims = 0
```

P0 字段	当前证据	状态	安全解释
task coverage	`formal_5ep_status.tsv` 24 行、24 unique tasks、每 task 5 trials	`PRESENT`	可称 full10k trained-run 的 24×5 formal eval
process completion	24/24 formal rows `exit_code=0`	`PRESENT`	进程正常退出，不等于策略成功
task-log recount	24 logs、120 episodes、2 successes	`PRESENT / independently recomputed`	可报告 raw `2/120`

Po 字段	当前证据	状态	安全解释
summary alignment	conservative JSON 24 rows 与 status task set 完全一致	`PRESENT`	summary 不是 checkpoint ownership 证据
artifact policy	CoffeeServeMug 两条 `SUCCESS:length=1` 被排除	`LOCAL POLICY`	只能称当前审计规则，不称官方通用定义
trusted recount	118 eligible episodes、0 trusted successes	`PRESENT / protocol-bounded`	可报告 `0/118`，须同时说明 denominator 规则
checkpoint identity	4066 行 action-debug 都指向 `/data/msz/wm/outputs/cosmos_policy_wam_repro_full10k_20260619/.../iter_000010000`	`REMOTE-REFERENCED`	path 不是本地 binary、revision 或 hash
action-debug integrity	24 files、4066 lines、`32×7`、`float64`、无 malformed/non-finite	`PRESENT`	结构健康，不证明动作语义或 success 原因
media completeness	本地同步 14 个 media；archive 169 entries	`PARTIAL`	不能当作 120 episode 完整 media bundle
scorer/version	log 有 SUCCESS/FAILURE/length；`length<5` exclusion 是本地 policy	`PARTIAL`	仍缺 immutable scorer/version 与 raw episode JSONL
ownership	launcher、wrapper、远端 operator/reviewer 未逐文件确认	`PENDING`	不从路径或报告作者推导“我负责”

4. Frozen inputs 与 SHA-256

文件	SHA-256	用途
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\formal_5ep_status.tsv`	`7FCA2C1C57EEAD560D602A6F3CFAF6D0BE7C21F747AEF8F567662C66B2BEC55C`	24 条 formal launcher/status 记录
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\eval_summary_conservative.json`	`C9538A2E418BCBEDD88B8DCFB308F0E1AF428DBA2912A959DEA946872A3247BE`	conservative task/episode summary
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\eval_summary_conservative.csv`	`676062F1318ADF3CC7DE4188AE575878BA1A779EFBE7214CA49FE6F2341D06BB`	task-level CSV summary
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\dataset_statistics.json`	`BFEC3286EF23AADF19B42C33937118C540073C44F668BA012D9E8B773AA77E41`	action/proprio normalization source
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\dataset_stats_contract.json`	`D41BC62760B937939067958D4FAAE78F08CFD0D823C4BD74F9A10F7265B8EA8A`	action dim 7、proprio dim 9 contract
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\cosmos_policy_eval_light_20260620.tgz`	`853E2434EF48AA774FAB59C13EED663AA85364956AB3E3F50A76CEF4B2B715C3`	169-entry light archive

目录审计计数：`logs=25`、`stdout=25`、`action_debug=25`、`manifests=75`；其中各类 25/25 中有 1 份 CoffeePressButton smoke，24 份才进入本页 formal 主表。archive 为 `8,403,550` bytes、169 entries；本地同步 media 计数为 14，不能替代完整 120 episode 视频或 JSONL。

5. Protocol tuple

25 个 command manifest 共享同一运行合同；本页只取其中 24 个 `5ep-alltasks` formal manifest。

字段	冻结值
host / GPU	`iv-yej2jeh2bkay8n70cofj` / `CUDA_VISIBLE_DEVICES=7`
seed	`195`
checkpoint path	`/data/msz/wm/outputs/cosmos_policy_wam_repro_full10k_20260619/cosmos_policy/cosmos_v2_finetune/cosmos_policy_wam_repro_full10k_20260619/checkpoints/iter_000010000`
stats / embeddings	`/data/msz/wm/cosmos_policy_stats_only/dataset_statistics.json` / `/data/msz/wm/outputs/cosmos_policy_robocasa_eval_20260620/reason1_eval_embeddings.pkl`
launcher / wrapper	`launch_cosmos_policy_robocasa_official_eval.sh` / `cosmos_policy_robocasa_official_eval_wrapper.py`
eval config	`cosmos_predict2p5_2b_480p_robocasa_50_demos_per_task__inference_no_s3`
observation	2 third-person + 1 wrist image；proprio enabled
normalization / denoise	image flip + JPEG；normalize proprio；unnormalize action；action denoise 5 steps；shift 5
action execution	`chunk_size=32`；`num_open_loop_steps=16`；action dim 7
policy selection	deterministic；best-of-N queries 1；no parallel inference
robot/controller	`PandaMobile` / `OSC_POSE`
task split / layout	object split `B`；`((1,1),(2,2),(4,4),(6,9),(7,10))`

字段	冻结值
formal trials	5 per task; `deterministic_reset=false`

这组字段支持“协议固定/对齐”的叙事，不构成模型权重内容、训练有效性或个人 ownership 证据。

6. 独立重算与 task-level result

6.1 Recompute gates

检查	结果
status rows / unique tasks / trial sum	`24 / 24 / 120`
status exit code	`24/24 = 0`
summary rows / task-set match	`24 / true`
formal task logs / task-set match	`24 / true`
log episodes / successes	`120 / 2`
action-debug files / lines	`24 / 4066`
malformed JSON / bad shape / non-finite / task mismatch	`0 / 0 / 0 / 0`
action shape / dtype / chunk / open-loop	`32×7 / float64 / 32 / 16`

6.2 Task-level summary

task	raw success / 5	artifact excluded	eligible	trusted success
CloseDoubleDoor	0	0	5	0
CloseDrawer	0	0	5	0
CloseSingleDoor	0	0	5	0
CoffeePressButton	0	0	5	0
CoffeeServeMug	2	2	3	0
CoffeeSetupMug	0	0	5	0
OpenDoubleDoor	0	0	5	0
OpenDrawer	0	0	5	0
OpenSingleDoor	0	0	5	0
PnPCabToCounter	0	0	5	0
PnPCounterToCab	0	0	5	0
PnPCounterToMicrowave	0	0	5	0
PnPCounterToSink	0	0	5	0
PnPCounterToStove	0	0	5	0
PnPMicrowaveToCounter	0	0	5	0
PnPSinkToCounter	0	0	5	0
PnPStoveToCounter	0	0	5	0
TurnOffMicrowave	0	0	5	0
TurnOffSinkFaucet	0	0	5	0
TurnOffStove	0	0	5	0
TurnOnMicrowave	0	0	5	0
TurnOnSinkFaucet	0	0	5	0
TurnOnStove	0	0	5	0
TurnSinkSpout	0	0	5	0
合计	**2 / 120**	**2**	**118**	**0 / 118**

6.3 Artifact-success boundary

两条 raw success 都来自 `CoffeeServeMug` episode 1/2，日志标记为 `SUCCESS:length=1`。本地 conservative summary 将它们标为 `immediate_success_artifact`，所以 trusted 分母从 120 变为 118，trusted successes 为 0。`length<5` 是本地审计 policy；在 scorer/version、termination reason 与完整 episode JSONL 未冻结前，不能把它写成 RoboCasa 官方统一 invalid 定义。

7. Action-debug integrity

24 个 formal action-debug JSONL 共 4066 行，逐行 JSON 解析无 malformed；`action.shape=[32,7]`、`dtype=float64`、`chunk_size=32`、`num_open_loop_steps=16` 保持一致；对 action summary 与 `first_row` 做 finite 检查无 `NaN/Inf`；task name 与文件名无 mismatch。单文件行数范围为 95–315。

这只能证明 inference 输出的记录结构可读、形状和数值有限，不能证明 action normalization 的语义正确、控制器执行正确、策略学到了任务，或某个失败由单一模块造成。

8. 与相邻 run family 的分离

run family	checkpoint / 运行线	本页关系
full10k trained eval（本页）	远端 `iter_000010000`；单卡 GPU7；24×5；raw `2/120`、trusted `0/118`	本页主证据
official-HF baseline	`nvidia/Cosmos-Policy-RoboCasa-Predict2-2B`；24×5；raw `86/120`、trusted `84/118`	外部 baseline；不能当本地成绩
8-GPU 1 epoch gate	`max_steps=9907`；save-2k cgroup OOM；save1500 `state_dict(destination=...)` first divergence	未完成训练/save gate；不能用本页 eval 替代
TurnOffMicrowave local 1k	local negative-only 1k；single-task `0/1` failure 与 same-input probe	诊断 run；不并入本页分母
CoffeePressButton smoke	1×1 smoke	本地目录中的代表性 smoke；不进入 24×5 主表

没有共同的 checkpoint binary/hash、config/data/stats immutable manifest、scorer version 与 episode-level paired join，因此不能计算 full10k 相对 official-HF 的 uplift，也不能把 `2/120` 或 `0/118` 与其它 run 族拼成一条训练曲线。

9. 面试可用与禁用表述

30 秒回答

> 我把 full10k trained run、official-HF baseline 和 8 卡 1 epoch gate 分开管理。对 full10k 的远端 `iter_000010000` path，我跑了 RoboCasa 24 个 task、每 task 5 个 episode，24 条 formal status 全部正常退出；从 task logs 独立重算是 raw `2/120`。两条 success 都是 CoffeeServeMug 的 length=1 immediate artifact，按当前本地 policy 排除后是 trusted `0/118`。我还核了 24 个 action-debug JSONL，4066 行全部 `32×7` 且 finite。这个结果证明评测链和审计闭环，不证明 benchmark uplift、官方 scorer 等价或本地 checkpoint binary ownership。

高压追问：为什么不是直接报告 2/120？

> `2/120` 是 raw log 口径，我不会隐藏它。因为两条成功都在 length=1，conservative summary 把它们标成 immediate artifact；如果采用这份本地审计 policy，eligible denominator 是 118，trusted success 是 `0/118`。这不是 RoboCasa 官方统一定义，所以我同时保留 raw 和 trusted，并把 scorer/version 与完整 episode JSONL 列为 P0 缺口。

高压追问：full10k 比 official-HF 差多少？

> 目前不能给因果差值。两者 checkpoint identity、revision、raw episode bundle、scorer/invalid policy 和 ownership 还没有完全 matched；能说的是本页 full10k trained-run audit 为 raw `2/120`、local trusted `0/118`，official-HF audit 为 raw `86/120`、local trusted `84/118`。这两组数字必须标注 run family，不能把后者写成本地模型成绩。

禁用表述

不安全说法	原因	安全替代
“full10k 在 RoboCasa 是 0%”	`0/118` 是本地 invalid policy，不是官方 scorer	“按当前本地 policy，trusted 为 `0/118`；raw 为 `2/120`。”
“我的模型达到 71.19%”	`84/118` 属于 official-HF path	“我运行并审计了 official-HF baseline，不把它归因于训练模型。”
“训练提升/下降了 X%”	没有 matched paired eval	“当前只有跨 run 的分布对账，不能宣称 uplift。”
“checkpoint 已保存并验证”	当前只有远端 path，无 binary/hash/reload parity	“action-debug 与 manifest 引用了远端 path，checkpoint gate 仍未闭合。”

10. 关闭 P0 的最小动作

1. 回传 `iter_000010000` checkpoint binary 或 immutable revision/ETag/SHA，并做 fresh-process load parity。
2. 冻结训练 config、代码 revision、dataset/statistics/embedding SHA 与 run manifest，确认 full10k 训练日志和 eval 的 lineage。
3. 保存 120 个 episode 的原始 JSONL（termination、length、success predicate、error）并给 `length<5` policy 一个版本化 scorer；保留 raw 与 trusted 两套结果。
4. 完整同步 episode media 或至少提供每条 episode 的可核验 media locator；当前 14 个 local media 不是完整 bundle。
5. 逐文件确认 launcher、wrapper、协议 patch、operator 与 reviewer 的候选人角色。
6. 若要写 uplift，用同一 checkpoint lineage、stats、seed/init、controller、task/layout、scorer 和 24×5 denominator 做 paired eval；在此之前保持 `WARN`。

References

1. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\formal_5ep_status.tsv`。
2. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\eval_summary_conservative.json` 与 `eval_summary_conservative.csv`。
3. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\logs\ENV_EVAL-*.txt`（24 formal task logs；另有 1 份 smoke）。
4. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\action_debug\*.jsonl`（24 formal action-debug files）。
5. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\remote_artifacts\manifests\*_command.txt`（24 formal + 1 smoke）。
6. Cosmos Policy RoboCasa Official-HF Baseline P0 Audit。
7. Cosmos / WAM 1 epoch–full10k Run Reconciliation。
8. WAM Claim–Evidence Inventory。
9. VOL 07 五项目证据 Reconciliation。

Part 9 / 49 · `interview_materials/vla/evidence/projects/cosmos_policy_robocasa_baseline_p0_audit.html` · SHA-256 `39E59144D6CE36135A09314294C5E399BC1F2D686A55486C919DA5F5A63972B8`

返回本卷目录

Canonical Markdown SHA-256: `39e59144d6ce36135a09314294c5e399bc1f2d686a55486c919da5f5a63972b8`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\cosmos_policy_robocasa_baseline_p0_audit.md`

Cosmos Policy RoboCasa Official-HF Baseline：P0 评测证据审计

状态：`p0_audited / remote-artifact-backed / checkpoint-unverified / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计 `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts` 的 official-HF RoboCasa 5-episode-per-task 评测；不把官方 checkpoint 结果写成本地训练 checkpoint 的 uplift，也不改变题卡 `unseen`。

1. 结论先行

这批材料已经足够证明一条完整的 **official-HF inference/eval harness** 运行链：24 个 task、每 task 5 个 episode，所有 24 条 formal status 行的 `exit_code=0`；25 个日志中另有一个 CoffeePressButton 1×1 smoke，不计入 24×5 主表。独立从 24 个 raw task log 重算得到：

```
raw = 86 / 120 = 71.6667%
artifact-successes excluded = 2
trusted = 84 / 118 = 71.1864%
task logs = 24
action-debug files = 24, total JSONL lines = 2692, non-finite bundles = 0
```

这支持的面试口径是：“我运行并审计了 Cosmos Policy 官方 HF RoboCasa baseline 的 24×5 eval，固定了 checkpoint/model-card path、stats/T5、5-step action denoise、chunk/open-loop、seed、controller、layout 与 success predicate，并重算了 raw/trusted denominator。”

它仍不支持：本地保存并 hash 官方 checkpoint、把 `84/118` 归因于个人训练、与 1 epoch/full10k checkpoint 做 uplift、或把 24×5 official-HF baseline 当成个人模型成绩。

2. P0 promotion 结论

```
P0 status = PARTIAL
eligible protocol/eval claims = PRESENT
eligible local checkpoint claims = 0
eligible causal-uplift claims = 0
eligible ownership claims = 0
```

P0 字段	当前证据	状态	安全解释
task coverage	24 条 `formal_5ep_status.tsv` 行，task 均 5 trials	`PRESENT`	可称 24×5 formal eval
process completion	24/24 `exit_code=0`	`PRESENT`	运行过程正常退出，不等于模型成功
raw success	24 个 task log 的 `Total successes` 合计 86/120	`PRESENT / independently recomputed`	可报告 raw 口径
invalid/artifact policy	2 个 CoffeeServeMug success 的 length=1 被排除	`PRESENT`	trusted denominator=118；需明确这是本地审计规则
trusted success	84/118	`PRESENT / protocol-bounded`	不与论文或本地训练 run 自动可比
checkpoint identity	每条 log 为 `nvidia/Cosmos-Policy-RoboCasa-Predict2-2B` path；bundle 内无 `.pt/.pth/.safetensors/.bin/.ckpt`	`REMOTE-REFERENCED`	path 不是本地 binary/hash
model/statistics revision	stats/T5 path 与 ckpt path 固定字符串	`PARTIAL`	未形成不可变 revision+SHA manifest
raw episode media	24 task logs、selected rollout manifest 与拷回的视频索引	`PARTIAL`	selected videos 不是完整 120 episode media bundle
scorer / success predicate	log 中有 success/length 与 task aggregate	`PARTIAL`	独立 scorer commit、invalid policy version、环境版本仍缺
ownership	logs/launcher/wrapper 有来源，但没有候选人逐文件确认	`PENDING`	不能从路径或 Git author 推导个人贡献

3. 原始输入与 SHA-256

文件	SHA-256	用途
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\eval_summary_officialhf_baseline.json`	`7B211B9D6F20BC95986A77C3CC901814AB6A705065016D7F68B67548BA6CACE9`	结构化 totals/task summary/selected video 索引
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\eval_summary_officialhf_baseline.csv`	`4751FDDBFE73CA237220C171A6C8D560069D31948643DF05820426FCA7A12B63`	task-level CSV 摘要
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\formal_5ep_status.tsv`	`8B3BAEE9029340387AE803E48152C301C0F6C679461FCF132AF5E6B5E612124E`	24 条 formal launcher/status 记录
`C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\selected_rollouts_manifest.json`	`126F006B952E988512BF88631BC01A8ACF46F38AA54119E6479C3BED9419C566`	8 个精选 rollout/failure 视频索引，不是完整 episode bundle

目录级只读计数：`logs=25`（24×5 + 1×1 smoke）、`action_debug=25`（主表 24）、`stdout=25`、`manifests=75`（每 task 3 类 manifest 文件）。bundle 内没有本地 checkpoint binary。

4. Protocol tuple

24 条主表日志的首行配置保持一致：

字段	冻结值
model/checkpoint path	`nvidia/Cosmos-Policy-RoboCasa-Predict2-2B`
experiment	`cosmos_predict2_2b_480p_robocasa_50_demos_per_task__inference`
stats / T5	`robocasa_dataset_statistics.json` / `robocasa_t5_embeddings.pkl`
cameras / proprio	2 third-person + 1 wrist；proprio enabled
image / normalization	flip images、JPEG compression、normalize proprio、unnormalize actions
denoising	action 5 steps；future/value 1 step
action execution	`chunk_size=32`、`num_open_loop_steps=16`、action dim 7
selection	deterministic、`num_queries_best_of_n=1`、no parallel inference
robot/controller	`PandaMobile` / `OSC_POSE`
task split/layout	`obj_instance_split=B`；`((1,1),(2,2),(4,4),(6,9),(7,10))`
trials/seed	5 per task；logs use seed 195

这些字段支持“协议对齐”叙事；它们不构成模型权重内容或个人训练 ownership 证据。

5. 独立重算结果

5.1 Task-level trusted table

task	raw successes / 5	artifact successes excluded	trusted denominator	trusted rate
CloseDoubleDoor	5	0	5	100%
CloseDrawer	5	0	5	100%
CloseSingleDoor	5	0	5	100%
CoffeePressButton	5	0	5	100%
CoffeeServeMug	3	2	3	33.33%
CoffeeSetupMug	2	0	5	40%
OpenDoubleDoor	2	0	5	40%
OpenDrawer	4	0	5	80%
OpenSingleDoor	5	0	5	100%
PnPCabToCounter	1	0	5	20%
PnPCounterToCab	4	0	5	80%
PnPCounterToMicrowave	2	0	5	40%
PnPCounterToSink	3	0	5	60%

task	raw successes / 5	artifact successes excluded	trusted denominator	trusted rate
PnPCounterToStove	3	0	5	60%
PnPMicrowaveToCounter	4	0	5	80%
PnPSinkToCounter	3	0	5	60%
PnPStoveToCounter	2	0	5	40%
TurnOffMicrowave	5	0	5	100%
TurnOffSinkFaucet	5	0	5	100%
TurnOffStove	0	0	5	0%
TurnOnMicrowave	5	0	5	100%
TurnOnSinkFaucet	5	0	5	100%
TurnOnStove	3	0	5	60%
TurnSinkSpout	5	0	5	100%
合计	**86 / 120**	**2**	**118**	**84 / 118 = 71.1864%**

5.2 Artifact-success exclusion

被排除的两个 success 都来自 `CoffeeServeMug`：episode 1 与 episode 2 的 log length 均为 `1`。它们计入 raw `86/120`，但不进入 trusted `84/118`。这里的 `length<5` 是当前本地审计规则，不应在没有 scorer/version 记录时写成 RoboCasa 官方通用定义。

5.3 Action-debug integrity

对 24 个 `*-5ep-alltasks-*.jsonl` 独立解析：共 `2692` 行；每行 action shape 均为 `32×7`；数值摘要与 `first_row` 中未发现 `NaN/Inf`。这说明 action-debug 文件结构和 finite 检查可复核，但不等于 action 语义正确或 episode success 的因果解释。

6. 面试可用口径

30 秒版本

> 我先把 official-HF baseline 和自己的训练 run 分开。基于固定的 Cosmos Policy RoboCasa inference contract，我跑了 24 个 task、每 task 5 个 episode；24 条 formal status 全部正常退出。按 raw log 独立重算是 `86/120`，其中 CoffeeServeMug 有两个 length=1 的 artifact success，按审计规则排除后是 `84/118=71.19%`。我还核了 24 个 action-debug JSONL，2692 行都是 `32×7` 且 finite。这个结果证明我跑通并审计了 eval harness，不是我训练 checkpoint 的 uplift；官方 checkpoint 在当前 bundle 中仍只有远端 path，没有本地 binary/hash。

高压追问：`84/118` 是不是你的模型成绩?

强回答：

> 它是 official-HF checkpoint path 下的 baseline eval summary，不是我 1 epoch 或 full10k 训练 checkpoint 的结果。`86/120` 与 `84/118` 的差异来自本地审计里对两个 length=1 success artifact 的 trusted denominator 处理；我会同时报告两个口径并解释排除规则。可以认领的是 launcher、协议固定、日志重算和 action-debug integrity audit；不能认领官方权重训练、结果 uplift 或 checkpoint ownership。要比较本地训练模型，必须拿到同一 checkpoint lineage、同一 scorer/invalid policy、同一 24×5 denominator 和 paired raw episode bundle。

高压追问：为什么不能直接拿 71.19% 和论文数字比?

因为仍需确认论文/模型卡的 checkpoint revision、task/layout/init/seed、success scorer、artifact policy 和 trial denominator 完全一致。当前页只冻结本地 remote-artifact eval 的 protocol tuple，不宣称论文 strict reproduction。

7. 关闭 Po 的最小动作

1. 保存 official checkpoint、stats、T5 embedding 的 immutable revision/ETag/SHA manifest，或保留可审计的远端 revision identity。
2. 把 `length<5` invalid/artifact policy 固定成带版本的 scorer contract，并保留原始 episode JSONL。
3. 将 120 个 episode 的 raw success/termination/error manifest 与选定视频索引对齐；当前 8 个 selected videos 不能替代完整 bundle。
4. 逐文件确认候选人负责的 launcher、wrapper、protocol patch、运行者与 reviewer；不要从远端路径或 Git author 推导 ownership。
5. 若要写 uplift，必须与本地训练 checkpoint 做 matched 24×5 paired eval；当前没有这样的因果 join。

在这些字段闭合前，本页保持 `PARTIAL/WARN`：允许讲评测协议、日志重算与失败分布，不允许讲个人模型达到 `71.19%` 或“训练提升了 baseline”。

References

1. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\eval_summary_officialhf_baseline.json`。
2. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\formal_5ep_status.tsv`。
3. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\logs\ENV_EVAL-*-officialhf-predict2-20260620-5ep-alltasks-*.txt`（24 条 task logs）。
4. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\baseline_remote_artifacts\action_debug\*-officialhf-predict2-20260620-5ep-alltasks-*.jsonl`（24 个 action-debug JSONL）。
5. Cosmos / WAM 1 epoch–full10k Run Reconciliation。
6. WAM Claim–Evidence Inventory。

Part 10 / 49 · `interview_materials/vla/evidence/projects/cosmos_policy_turnoffmicrowave_trained_failure_p0_audit.html` · SHA-256 `80789C958CBDBBEF9FEDEDBA13D6BAFBECAB64FAE1A45E5AFF0C3470FFF57CAB`

返回本卷目录

Canonical Markdown SHA-256：`80789c958cbdbbef9fededba13d6bafbecab64fae1a45e5aff0c3470fff57cab`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\cosmos_policy_turnoffmicrowave_trained_failure_p0_audit.md`

Cosmos Policy TurnOffMicrowave 本地训练失败：P0 First-Divergence 审计

状态：`p0_audited / local-artifact-backed / checkpoint-remote-referenced / root-cause-partial / ownership-pending / WARN` 审计日期：2026-07-21 证据截止：2026-06-23 范围：只审计 local negative-only 1k checkpoint 在 TurnOffMicrowave 上的 fixed-batch gate、`0/1` closed-loop、同输入 official-vs-local query probe、5-frame conditioning 对照、eval-state mining 与 corrected single-batch overfit；不把该 checkpoint 与 8-GPU 1-epoch gate、独立 full10k run 或 official-HF `84/118` 合并。

1. 结论先行

这批本地归档已经把失败定位到一个更窄、但仍未完全闭合的边界：**训练链路和单个负向 batch 可学习，local checkpoint 也能在缺省 debug conditioning 下采出负向夹爪；但一旦对齐 official 的 5-frame conditioning，输出在同一 HDF5 batch 和 exact eval observation 上都转为小正值。** 因此不能把失败简化成“源数据没有负样本”“第 7 维符号或 normalization 错了”“action slot 没梯度”，也不能把 `num_conditional_frames` 的缺失本身直接称为唯一 root cause。

关键证据链如下：

```
negative-only 1k fixed batch
  sampled first gripper = -0.827738
  sampled negative fraction = 100%
  trajectory final clean first = -0.827517
              |
              | same remote-referenced checkpoint
              v
TurnOffMicrowave closed-loop
  success = 0/1, episode length = 500
  32 queries, each action chunk = 32x7
  gripper min/max/mean = +0.064589 / +0.561399 / +0.220511
  negative fraction below -0.5 = 0
              |
              | exact same initial observation hashes
              v
official-HF first gripper = -1.015586
local first gripper       = +0.142931
best-of-8 local first range = +0.105245 .. +0.221671
all eight candidates: no gripper value below -0.5
```

目前最安全的技术结论是：**first divergence 出现在 official 5-frame conditioning 下从 observation-conditioned sampler 到 7D action chunk 的映射，而不是 action tensor 的 shape、slot-5 roundtrip、stats 路径或单 batch 可学习性。** 这仍是 `partially confirmed` 的边界，不是完整因果闭环；缺少 immutable local checkpoint binary/hash、matched patch→rerun、更多 task/episode、完整 scorer/version 与候选人 ownership。

2. 问题（Question）

为什么 negative-only 1k checkpoint 在固定负向 batch 上能产生 `< -0.5` 的夹爪动作，到了 TurnOffMicrowave closed-loop 却是 `0/1` 且 32 次 query 全正？失败是负样本缺失、动作接口错误、采样随机性、5-frame conditioning、还是训练 observation 到 exact eval observation 的迁移问题？现有证据能确认到哪一层，面试中又应怎样避免过度归因？

3. P0 promotion 结论

```
P0 status = PARTIAL
eligible local failure claims = PRESENT
eligible first-divergence claims = PRESENT
eligible root-cause claims = PARTIAL
eligible fixed claims = 0
eligible causal-uplift claims = 0
eligible ownership claims = 0
```

Po 字段	当前证据	状态	安全解释
local checkpoint identity	所有 summary 指向同一个远端 `iter_000001000` path	`REMOTE-REFERENCED`	可绑定 run name/path；没有本地 binary/full SHA 或新进程 load parity
fixed-batch learnability	sampled first `-0.827738`，trajectory final clean `-0.827517`	`PRESENT`	证明该 batch 和链路可输出负向，不证明 eval-domain transfer
closed-loop symptom	TurnOffMicrowave `0/1`，length `500`，32 次 query 全正	`PRESENT / ONE-EPISODE`	可讲最小失败复现；不是 benchmark success rate
same-input comparison	official first `-1.015586`，local `+0.142931`，observation/T5/proprio hashes 对齐	`PRESENT`	可讲 same-state first divergence；两个模型的 denoise steps 不同，结果是诊断而非公平性能排名
sampling variance	local best-of-8 first gripper `0.105245-0.221671`，无 `<-0.5`	`PRESENT`	排除该 state 上“单次倒霉 seed”；不能概括全部 states
source negative support	52 个 HDF5、11230 states；top-24 GT first-action negative rate `37.5%`	`PRESENT`	排除“相似成功数据完全没有负向 first action”
5-frame conditioning effect	缺省 debug path 为全负；补齐 official 5 frames 后为全正	`PRESENT / COUNTERFACTUAL`	证明 conditioning contract 会改变输出；不能单独证明模型为何没有学会该条件分布
corrected overfit transfer	ordinary loss 能拟合 fixed batch，但 exact eval first 仍 `+0.315938`	`PRESENT`	证明 parameter/update/action path 可学；暴露 observation-conditioned transfer gap
repaired final result	没有 patch 后的 matched multi-episode success rerun	`MISSING`	不得称 root cause fixed 或成功率恢复
ownership	run/operator/code author/reviewer 未由候选人逐项确认	`PENDING`	路径、日志和 Git author 不自动等于个人贡献

4. Frozen inputs 与 SHA-256

本地证据	SHA-256	本页用途
`negative_only_gate_iter1000_summary.json`	`87E46164CA21BC302298505E08CF8D15492CBAA16BD75FD30126D7B4B31E9039`	训练配置、fixed-batch gate、trajectory final clean 与 run exits
`eval_action_debug_summary.json`	`7BA5101928FDF089404E033A9C915BEFD678D1D140338A726AE6D1E27531F1C0`	`0/1` closed-loop、500 steps、32 次 query 的 gripper 统计
`query_probe_comparison_summary.json`	`C73FF213B6F584BEF73975BA07E621E26BEEDB294F7C5BF656E80207268C280E`	同一 initial observation 的 official-vs-local first action
`best_of_8_query_probe_summary.json`	`1EA9C2A74B569DDD6811D6DAD41A27C438D17DDEB5FC27E2DDBBF71A5B0EAC49`	8 个 denoising seeds 的 local query 分布
`eval_state_mining_summary.json`	`0A989EB0A59947156A42A6853785D826F277D0EF52F08F8E92FD3CDCAAB65726`	52-file/11230-state mining、top-24 标签与早期 overfit 摘要
`eval_vs_hdf5_query_compare.json`	`3548EB7ECECFB5689091E596CCA164A5A3268405009D104D7FB0E06D23A6B52D`	HDF5 nearest state、conditioning-frame counterfactual、exact queries
ordinary-loss overfit summary	`E8F10210D1542F5B52BF6CFB527F97F93946967268A3CA3BDBCE628B676EEE9D`	corrected fixed-batch learnability 与 exact-eval non-transfer
negative-only strong-reg overfit summary	`2FF4545E75B83BAA83BC39AFDECCE64106FD01842EF485396B627D4E6E85C407`	strong auxiliary 的 clean/sample/exact-eval 三层对照

上表冻结的是 Windows 本地归档文件。checkpoint 本身仍只由远端 path 引用：

```
/data/msz/wm/outputs/cosmos_policy_predict2_official_repro_20260621/
imaginaire_runs/cosmos_policy_repro/predict2_robocasa_official_t5/
train1k_full1199_sampler_gripper_aux_w20_sig200_first8_negonly_neg64/
checkpoints/iter_000001000
```

本页没有把该 path 当作 immutable checkpoint hash，也没有独立复算远端权重内容。

5. Run 总账与相邻 Delta

Run / probe	相较上一阶段的改造	数据/输入	Checkpoint / steps	输出产物	结果	能证明 / 不能证明
`negonly_1k_gate`	full1199、action-only、balanced sampler、first8 negative-only aux	fixed negative/positive batch，`32×7` action	local `iter_000001000`，train 1000，sample 35 steps	gate/chain/trajectory JSON	sampled first `-0.8277`，gate pass	能输出负向；不能证明 closed-loop
`turnoff_1ep_smoke`	从 fixed batch 进入 RoboCasa online rollout	TurnOffMicrowave initial state，5-frame official contract	同一 local path	eval log、JSONL、2 视频、summary	`0/1`，length 500，32 queries 全正	closed-loop 症状；单 episode 不是 benchmark
`exact_query_official`	截断到 initial observation 单次 query	相同 camera/proprio/T5 hashes	official HF，5 denoise steps	comparison JSON	first gripper `-1.0156`	official 能在该 state 输出负向；不是本地训练对照成绩
`exact_query_local`	checkpoint 替换为 local，denoise 35 steps	同上	local 1k	comparison JSON	first gripper `+0.1429`	first divergence；不同 denoise steps 不支持性能排名
`best_of_8_local`	seeds 195–202	同一 exact eval state	local 1k	summary JSON	first `0.1052-0.2217`；global min `-0.0372`	排除单 seed；不排除其他 state 能负向

Run / probe	相较上一阶段的改造	数据/输入	Checkpoint / steps	输出产物	结果	能证明 / 不能证明
`eval_state_mining`	从 52 HDF5 搜索 eval-like success states	11230 states，top-24	无新训练	neighbors/mining JSON	top-24 first negative `37.5%`；nearest GT first `-1`	相似数据存在负向；不是 exact observation 等价
`conditioning_counterfactual`	只补 `num_conditional_frames=5`	同一 nearest HDF5 batch	local 1k	compare JSON	缺省为全负；5 frames 后全正	conditioning contract 是高影响变量；不是唯一 root cause
`corrected_overfit_none`	official 5-frame eval-like contract，ordinary loss 单 batch 1000 steps	nearest negative + positive samples	in-memory overfit	summary/JSONL	fixed clean `-1.0154`、sampled `-1.0259`；exact eval `+0.3159`	单 batch可学但不迁移 exact observation
`corrected_overfit_negonly`	加 sampler negative-only strong-reg	同上	in-memory overfit 1000 steps	summary/JSONL	clean `-0.9001`、fixed sample `+0.5002`、exact eval `+0.3279`	aux 有梯度但未修复 sampler/eval transfer

以上 run 只构成诊断序列，不是递增性能曲线。相邻阶段的 Δ 主要落在 observation contract、sampler seed 与 in-memory objective；checkpoint、task set、denoise steps、episode denominator 并非全部 matched，因此禁止把数值差写成 uplift。

6. 动作、采样与评测合同

每次 policy query 产生一个动作块 $A_q \in \mathbb{R}^{H\times D}$，当前归档中 $H=32$、$D=7$，第 7 维是 gripper。令 $g_{q,h}=A_q[h,6]$，本页使用的负向比例为：

$$r_q^- = \frac{1}{H}\sum_{h=0}^{H-1}\mathbf{1}[g_{q,h} < -0.5].$$

变量解释：q 是 policy query，h 是 chunk 内 action index，阈值 `-0.5` 来自这些本地 gate/summary 的判定合同。实现映射：`eval_action_debug_summary.json` 对 32 个 query 分别统计 chunk 维度和 gripper 分布；closed-loop 中所有 $r_q^-=0$。这是本地证据阈值，不应扩写成 RoboCasa 官方统一成功条件。

同一 initial observation 的 first-action gripper 差异可写为：

$$\Delta_g = g_{0,0}^{\text{local}} - g_{0,0}^{\text{official}} = 0.142931 - (-1.015586) = 1.158517.$$

变量解释：两侧共享 task label、三路 camera、raw/normalized proprio、T5 embedding 与 constructed data batch 的短 SHA；official 使用 5-step action denoise，local 使用 35-step。直觉上这是第一个可观测 action divergence，不是训练提升或同超参模型质量差。实现映射：数字来自 `query_probe_comparison_summary.json`，因此只支持该 state、该 seed 和各自记录的 sampler 配置。

conditioning 对照把 sampler 写成：

$$\hat{A} = f_\theta(o, c, z; C), \qquad C \in \{1, 5\},$$

其中 o 是图像与 proprio observation，c 是 T5 task condition，z 是 denoising noise，C 是 conditional-frame count。缺省 debug path（实质为 $C=1$）在 nearest HDF5 batch 上 first gripper 为 `-0.961349`、全 chunk 负向率 `1.0`；对齐 official $C=5$ 后 first gripper 为 `+0.129789`、负向率 `0.0`。实现映射：两项来自 `eval_vs_hdf5_query_compare.json` 的 `hybrid_sampling`；它证明 C 改变了模型条件分布，却没有证明“把 C 改回 1”能修复 official closed-loop。

7. Online Rollout 与 First-Divergence 定位

```
def audit_turnoffmicrowave_rollout(env, policy, checkpoint, task):
    # 1. Reset one TurnOffMicrowave episode and retain exact initial observation hashes.
    obs = env.reset(task=task, seed=195)

    # 2. Build the official Predict2 contract: three cameras, proprio, T5 and 5 conditional frames.
    model_batch = build_predict2_batch(obs, task, num_conditional_frames=5)

    # 3. Query a 32x7 action chunk and record the seventh gripper dimension before execution.
    action_chunk = policy.sample(model_batch, checkpoint=checkpoint, denoise_steps=35)
    record_query(action_chunk, shape=(32, 7), gripper_index=6)

    # 4. Execute only the configured open-loop prefix, then replan until success or horizon 500.
```

```
    for action in action_chunk[:16]:
        obs, _, done, info = env.step(action)
        if done or info.get("success", False):
            return info

    # 5. Repeat queries and preserve JSONL/video/episode length for first-divergence analysis.
    return continue_until_horizon(env, policy, max_steps=500)
```

这段伪代码重建归档中的 control flow，不声称函数名逐字存在于项目。直接证据是 `num_open_loop_steps=16`、chunk `32×7`、5-frame slot contract、seed 195、500-step episode 与 action-debug JSONL；环境 success predicate 的具体源码 revision 仍未冻结。

7.1 Closed-loop 症状

`eval_action_debug_summary.json` 记录：1 个有效 episode、success `0/1`、length `500`、32 次 action query。把全部 query 的第 7 维汇总后，最小值 `+0.064589`、最大值 `+0.561399`、query mean 的均值 `+0.220511`，所有 query 的 `<-0.5` 比例都是 `0`。first-row gripper 甚至只落在 `+0.131084..+0.145603`。

这说明"第一个动作恰好为正"不足以解释失败：在整个 episode 的记录 query 中，没有一次进入本地 gate 定义的负向区间。但它仍然只有 1 episode；不能把 `0/1` 写成任务族 success rate。

7.2 Same-input official-vs-local probe

comparison summary 冻结了 primary/secondary/wrist image、raw proprio、constructed video batch、T5 embedding 与 normalized proprio 的短 SHA。官方 checkpoint 在同一 initial observation 上 first gripper 为 `-1.015586`；local 1k 为 `+0.142931`。local 的 best-of-8 seeds 195–202 全部为正，first range `+0.105245..+0.221671`，所有 chunk 元素的最小值也只有 `-0.037203`。

因此，在这个 state 上可以排除"local 只是碰到一个倒霉 denoising seed"以及"只有 best-of-N value selection 选错候选"。不能排除的是：其他 states、其他 checkpoint step、不同训练 distribution 或 matched denoise setting 下可能出现不同结果。

8. 负样本、5-frame 与 Exact-Observation Transfer

8.1 不是简单的"没有负样本"

state mining 扫描 52 个 HDF5、11230 个 states。按图像/proprio 相似度选出的 top-24 中，GT first gripper `<-0.5` 的比例为 `37.5%`；最近邻 first gripper 是 `-1.0`，其 chunk 同时包含负/正动作。这证明 eval-like success data 中存在关闭夹爪方向的监督信号。

同时，full1199 stats 中 gripper action 的 min/max 是 `-1/+1`，full1199 与 subset8 stats 的最大绝对差为 `0.0`。因此"源 gripper 第 7 维只有正值"与"eval 误用了另一套 subset stats"都被现有 structured summaries 反驳。

8.2 历史 debug sampler 漏了关键 conditioning 字段

早期 fixed-batch/debug path 没有显式传入 `num_conditional_frames`。在 nearest HDF5 batch 上，它可以得到全负 chunk；一旦设置为 official contract 的 5 frames，同一 checkpoint 的 chunk 变为全正。这不是无关紧要的参数变化，而是改变 observation information flow 的输入合同差异。

但安全结论不是"5 frames 是 bug"。official checkpoint 在相同 5-frame eval contract 上能输出 `-1.0156`；真正的问题更接近 local model 没有把训练监督迁移到 official 5-frame conditioned exact observation。要确认根因，仍需 matched training/eval：训练期显式固定相同 $C = 5$，保存 immutable checkpoint，再对同一 state 与多 episode 做 patch 前后 rerun。

8.3 Corrected overfit 揭示 transfer gap

在对齐 5-frame 的 eval-like overfit 中，ordinary action loss 训练 1000 steps 后：

层级	ordinary loss	negative-only strong-reg	解释
fixed-batch clean first	`-1.015420`	`-0.900114`	两种目标都能把 clean prediction 拉负
fixed-batch sampled first	`-1.025936`	`+0.500155`	ordinary loss 在该 seed/batch 可过 sampler；strong-reg 反而未过
exact eval first	`+0.315938`	`+0.327874`	两者都没有迁移到 exact eval observation
exact eval negative frac	`0`	`0`	没有形成 `<-0.5` action

ordinary loss 的 final base loss 是 `0.267922`；strong-reg 的 base/aux/total 是 `1.138504 / 0.039185 / 1.922204`，第 7 维 target gradient L2 为 `7.924779`。这说明 auxiliary 不是"写了但没有梯度"，却也没有保证 full sampler 或 exact observation 的正确条件映射。

本页不画跨 run loss curve：这些 summary 只有少量 step snapshots，且目标函数不同，把它们连接成平滑曲线会制造可比性。可复核的做法是保留 step-level JSONL，并把上述终点作为诊断表，而不是伪造训练趋势。

9. 0-Success 假设审计

假设	支持/反驳证据	当前动作	再评估结果	状态
源数据没有负向 gripper	52 HDF5/11230 states；top-24 first negative `37.5%`；stats min `-1`	mining + stats 对照	找到 eval-like negative windows	`REJECTED`
第 7 维 action shape/order 已断	action chunk 始终 `32×7`；gripper index 6；slot-5 roundtrip error `1.19e-7`	chain trace	fixed batch 可输出 `-0.8277`	`REJECTED FOR BASIC PATH`
normalization/stats path 错配	full1199 vs subset8 stats max diff `0.0`；dataset/manual normalize error `5.96e-8`	stats/roundtrip audit	basic mismatch 未发现	`REJECTED FOR ARCHIVED CASE`
单个 denoising seed 倒霉	best-of-8 全部 first 为正，无 `<-0.5`	seeds 195–202	仍全正	`REJECTED FOR THIS STATE`
debug sampler 与 official eval 输入合同不同	缺省 conditional frames 时全负；5 frames 后全正	补齐 `num_conditional_frames=5`	counterfactual 稳定复现	`CONFIRMED CONTRACT DIFFERENCE`
训练链路/参数完全不可学	ordinary overfit clean/sample 达 `-1.015/-1.026`	corrected 1000-step single-batch overfit	fixed batch 学会	`REJECTED`
exact eval observation 没有被模型正确条件化	fixed batch 学会，但 exact eval 仍 `+0.316`；official 同 state 为 `-1.016`	same-input + overfit	gap 仍存在	`PARTIALLY CONFIRMED`
negative-only strong-reg 已修复	clean 为负但 fixed sample `+0.500`、exact eval `+0.328`	1000-step corrected overfit	未修复	`REJECTED`
gripper 是 `0/1` 的唯一原因	当前只有 action/debug 与单 episode；未冻结完整 state/video causal trace	需要 patch→rerun	尚无成功恢复	`OPEN`

这里的“rejected”都带作用域：例如 basic action path 通过，不代表 controller axis semantics、environment success predicate 或所有 episode 都已核验。

10. 面试可用口径

30 秒版本

> 我把 local 1k checkpoint 的 fixed-batch gate 和 closed-loop 结果分开。它在负向 fixed batch 上确实能采出 first gripper `-0.8277`，但 TurnOffMicrowave 1-episode eval 是 `0/1`，500 steps 的 32 次 query 第 7 维全为正。对同一个 initial observation，official-HF first gripper 是 `-1.0156`，local 是 `+0.1429`，best-of-8 local 也没有 `<-0.5`。进一步发现早期 debug sampler 没带 official 的 5-frame conditioning：补齐后同一 HDF5 batch 会从全负转成全正。corrected overfit 说明单 batch 和参数链路可学，但 exact eval observation 仍不迁移，所以我把结论定为 observation-conditioned sampler transfer gap，尚不称唯一 root cause 或已修复。

2 分钟版本

> 我先固定身份：这是 full1199 上 negative-only auxiliary 的 local 1k checkpoint，不是 8-GPU 1-epoch gate、独立 full10k，也不是 official-HF `84/118`。固定 batch 的动作 shape 是 `32×7`，slot-5 roundtrip 和 normalization 都通过，35-step sampler first gripper 能到 `-0.8277`，所以基础链路不是完全学不会。真正进入 TurnOffMicrowave closed-loop 后，1 episode 跑满 500 steps 仍是 `0/1`；32 次 query 的 gripper min 都在正区间，整体最小值是 `+0.0646`。为了找 first divergence，我截取同一个 initial observation，对齐三路图像、proprio、T5 和 batch hash。official-HF 输出 `-1.0156`，local 输出 `+0.1429`；local 换 8 个 seed，first 仍在 `0.1052-0.2217`。数据侧也不是完全没负样本：52 个 HDF5、11230 states 的 top-24 eval-like neighbors 中有 `37.5%` GT first action 为负。最关键的 protocol bug 是历史 debug sampler 缺 `num_conditional_frames`；缺省时同一 HDF5 batch 可全负，按 official 的 5 frames 后转成全正。但我不把它直接叫唯一根因，因为 official 模型在 5-frame contract 下工作正常，而且 corrected single-batch overfit 虽然能把 fixed clean/sample 拉到约 `-1.02`，exact eval first 仍是 `+0.316`。所以已确认的是 conditioning contract 差异和 exact-observation transfer gap；未确认的是哪种训练数据/条件建模改动能恢复 multi-episode success。

高压追问：所以是因为训练集负样本太少吗？

强回答：

> 不能这样归因。数据不平衡可能是背景风险，但现有证据已经找到 eval-like negative windows：top-24 first-action negative rate 是 `37.5%`，nearest GT first 是 `-1`；ordinary single-batch overfit 也能把 fixed sampled first 拉到 `-1.026`。失败发生在 5-frame conditioned exact eval observation 上，所以“没有负样本”不足以解释。下一步应做 matched (C=5) 训练、immutable checkpoint、同 state probe 和多 episode rerun，而不是只加负样本权重。

高压追问：你定位并修复了 root cause 吗？

强回答：

> 我定位了两个可复核边界：历史 debug sampler 与 official eval 的 5-frame contract 不一致；local checkpoint 从 fixed batch 到 exact eval observation 存在 first-action gripper divergence。还没有修复后的 matched success rerun，所以我不会说 root cause 已修复。negative-only strong-reg 的 corrected overfit 也失败了，这反而说明单纯加强夹爪 auxiliary 不是充分解。

11. 公源、本地证据与 Ownership 边界

层级	本页如何使用	不能越界到
公有上游	Cosmos Policy Predict2/RoboCasa 的 5-frame、action chunk 与 official checkpoint path	个人提出模型架构、个人训练 official 权重
本地证据	JSON summaries、logs、JSONL、视频、mining/overfit outputs	checkpoint binary 已 hash、本地结果可严格复现
本地推断	first divergence 位于 5-frame observation-conditioned sampler transfer	唯一 root cause、gripper 必然导致所有 success failure
未核实	operator、code author、reviewer、checkpoint lineage、scorer revision、controller semantics	个人主导、修复已上线、benchmark uplift

候选人可以认领哪些具体脚本、运行、分析和结论，仍需 ownership ledger 的人工确认；本页不从 Windows 用户名、远端路径或文件作者自动推断。

12. 关闭 P0 的最小动作

1. 把 local `iter_000001000` checkpoint、config、stats、T5 与代码 revision 形成 immutable manifest，并在新进程做 load/query parity。
2. 用同一 training/eval contract 显式固定 `num_conditional_frames=5`；保留 patch、训练日志和 step-level sampler diagnostics。
3. 对 patch 前后 checkpoint 跑 matched exact-state query：同 task、初始 state、三路图像/proprio/T5 hash、denoise steps、seed set 和 output transform。
4. 把 TurnOffMicrowave 从 1 episode 扩到预注册 seeds/episodes，冻结 success predicate、controller、horizon、JSONL 与视频，并报告 denominator。
5. 对 first divergence 做逐层 trace：condition tensor、slot mask、action latent、denoising trajectory、denormalized 7D action 与 controller input；不要只看 final gripper histogram。
6. 由候选人逐文件确认 run operator、脚本/patch author、分析者和 reviewer；工具生成与人工判断分开记录。

在这些字段闭合前，本页保持 `PARTIAL/WARN`。允许讲“我复现了一个可审计的 0/1 failure，并把 first divergence 收窄到 official 5-frame conditioned exact-observation transfer”；不允许讲“我修好了 Cosmos Policy 的 gripper root cause”、训练提升或本地 checkpoint benchmark 成绩。

References

1. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\negative_only_aux_gate_20260622\train1k_full1199_sampler_gripper_aux_w20_sig200_first8_negonly_neg64\negative_only_gate_iter1000_summary.json`。
2. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\negative_only_aux_gate_20260622\eval_1k_TurnOffMicrowave_1ep_cudaenv\eval_action_debug_summary.json`。
3. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\query_probe_20260622\query_probe_comparison_summary.json`。
4. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\query_probe_20260622\best_of_8_query_probe_summary.json`。
5. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\eval_state_mining_20260622\eval_state_mining_summary.json`。
6. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\eval_state_compare_20260622\eval_vs_hdf5_query_compare.json`。
7. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\eval_state_mining_20260622\eval_like_overfit_none_steps1000_w0_sig200\single_batch_overfit_none_summary.json`。
8. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\eval_state_mining_20260622\eval_like_overfit_sampler_negonly_strong_reg_steps1000_w20_sig200\single_batch_overfit_sampler_negonly_strong_reg_summary.json`。
9. Cosmos Policy RoboCasa Official-HF Baseline P0 Audit。
10. Cosmos / WAM Claim–Evidence Inventory。
11. Cosmos / WAM 口述 Model Answer。

Part 11 / 49 · `interview_materials/vla/evidence/projects/cosmos_policy_wam_checkpoint_save_load_gate_p0_audit.html` · SHA-256 `D5A4DB639C3C0C152D55F3F9C6F6BEDD8E18408F306036A28DA5154BAB3D9AAC`

返回本卷目录

Canonical Markdown SHA-256：`d5a4db639c3c0c152d55f3f9c6f6bedd8e18408f306036a28da5154bab3d9aac`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\cosmos_policy_wam_checkpoint_save_load_gate_p0_audit.md`

Cosmos Policy / WAM Checkpoint Save-Load Gate：P0 证据审计

状态：`p0_audited / base-load-backed / train-save-gate-failed / checkpoint-binary-missing / ownership-pending / WARN` 审计日期：2026-07-21 证据截止：2026-06-25 范围：只审计 `muxi_deepspeed_1epoch_20260623` 归档中的 base-model load、1-step/5-step smoke、9907-step save2k gate 与 no-cycle save1500 gate；不把 smoke 的成功、base-load 的完整 key 对齐或空的 output directory写成“训练完成”或“checkpoint 可加载”。

1. 结论先行

本轮把上一轮“保存接口失败”的叙述升级成了可复核的 **save/load gate ledger**：

1. 13 个 `base_load_summary.json` 都指向同一个 HF cache snapshot，报告 `715` keys、`missing_count=0`、`unexpected_count=0`。这证明 base model 初始化/加载契约在这些进程中通过了 key-level check；它不是训练后 checkpoint 的 load parity。
2. `batch_size=1` 的 8-GPU fixed-optional 1-step smoke、`batch_size=8` 的 1-step smoke 和 no-cycle 5-step smoke 都有 `exit=0`；它们只覆盖短步训练与初始化，不覆盖保存、重启、完整 epoch 或 eval。
3. 独立 `9907`-step、8-GPU、global batch 128 的 save2k run 在 step 1903 附近出现 rank `-9`，launcher 记录 `EXIT=247`；这是长跑/资源 gate 的失败线，不是 save/load 成功线。
4. no-cycle、同为 `9907` steps 的 save1500 run 进入保存点后明确失败：DeepSpeed 调用 `module.state_dict(destination=..., prefix=..., keep_vars=...)`，Cosmos `DiffusionModel.state_dict()` 不接受 `destination`，launcher `EXIT=1`。这条 first divergence 比“目录存在”更强，但仍没有产生可加载权重。
5. `remote_outputs_20260629` 目录共 `65` 个文件、`18` 个 run contract、`13` 个 base-load summary、`9` 个 exit-code 文件，**0 个 checkpoint-like binary**（`.pt/.pth/.ckpt/.safetensors/.bin` 或 checkpoint 文件名）。因此当前 P0 状态是 `PARTIAL`：base load 通过，短 smoke 通过，长训与 save gate 未闭合，checkpoint/save-load parity 缺失。

面试最安全的说法是：

> 我把“模型能初始化”“1-step/5-step smoke 能跑”“长跑资源 gate”“保存接口”拆开验收。base HF snapshot 的 715 个 key 在 13 条 load summary 中都无 missing/unexpected；短 smoke 有 exit 0。但 9907-step save2k 线在长跑中以 rank -9/EXIT=247 退出，no-cycle save1500 线又在 DeepSpeed 标准 `state_dict` 参数签名处退出。归档中没有训练产出的 checkpoint binary 或新进程 reload parity，所以我能认领的是 gate 分层和 first divergence 审计，不能说 1 epoch 已完成或 checkpoint 可加载。

2. 问题（Question）

为什么不能把 `base_load_summary`、1-step/5-step smoke 或一个已经创建的 `step_001500` 目录当作 checkpoint 完成？哪一个 run 证明了长跑资源问题，哪一个 run 证明了保存 API 问题？当前证据能把“初始化、训练、保存、重启加载、评测”五个 gate 关闭到哪一层？

3. P0 promotion 结论

```
P0 status = PARTIAL
base-model initialization = PRESENT
short training smoke = PRESENT
long-run 1-epoch completion = 0
checkpoint save = 0
new-process load parity = 0
same-checkpoint eval = 0
ownership = PENDING
```

Gate	证据	状态	能说什么	不能说什么
base model load	13 summaries；715 keys；missing/unexpected=0	`PRESENT`	HF snapshot 在这些进程中 key-level load 通过	不能说本地训练 checkpoint 已保存/可重载
1-step smoke	fixed-optional 与 bs8 记录 `exit=0`	`PRESENT / SMOKE`	初始化、一个 training step 和分布式启动可运行	不能覆盖 checkpoint 或长时资源
no-cycle 5-step	`smoke5_exit_code.txt=0`，5 条 metrics	`PRESENT / SMOKE`	iterator 修复下短运行可启动	不能证明 9907 steps、save 或 eval
9907-step save2k	run contract + launcher `rank=-9` / `EXIT=247`	`FAILED`	长跑在保存前后出现外部 kill/资源失败线	不能把中间 loss 当 epoch 结果
9907-step no-cycle save1500	launch log TypeError at `state_dict(destination=...)`，`EXIT=1`	`FAILED / FIRST DIVERGENCE`	保存 API 的具体错误点可复核	不能说 `step_001500` 目录就是权重
checkpoint binary	65-file archive 中 checkpoint-like count `0`	`MISSING`	只能保留 remote output path/日志	不能给 checkpoint SHA 或 reload parity
new-process reload	没有 load-after-save command/result	`MISSING`	只能把它列为恢复动作	不能说参数、optimizer、global step 已恢复
matched eval	没有同一 produced checkpoint 的 eval bundle	`MISSING`	不升级为 trained-model benchmark	不能与 official-HF `84/118` join
ownership	operator/patch author/reviewer 未人工确认	`PENDING`	只能讲 evidence-backed run boundary	不能从远端路径推导个人贡献

4. Frozen inputs 与 SHA-256

证据	SHA-256	用途
`base_load_summary.json`（13 份内容相同）	`197419EDBEB136DC54F8CC746C5E889FC03FA37A2C0CDA077D4F43441DE81CB9`	load path、HF cache snapshot、715 keys、missing/unexpected
save2k `run_contract.json`	`A8D8EB2FE76F2B9EA80DB23C6DCCE006D6B981B68396BEA16E31A5EF9F852BC8`	8 GPU、per-GPU batch 16、9907 steps、save every 2000、5 frames
save2k `launch.log`	`F416050867C71ECC6BA5A7C67AFC67028B04BEC7A1E9DC4F4956BEA5531616C2`	base load、长跑退出、rank -9 与 `EXIT=247`
no-cycle save1500 `run_contract.json`	`201B4EEA49DCC5B53F6785B10D1DB3697E286C046A3FA2E060CA063CC7465DF2`	同一 9907-step contract、save every 1500
no-cycle save1500 `launch.log`	`8FEC262805242AFC3FCD7846A3DDB1043860E17BF5DEB008369D7CD9F67061C3`	DeepSpeed `state_dict(destination=...)` TypeError、`EXIT=1`
no-cycle smoke5 `run_contract.json`	`1D0BFE1F1125FC78FA224DA4449E7C60E22161B29345FA0D5F27830AD4E11B18`	8 GPU、batch16、5 steps、save disabled、5-frame contract
no-cycle smoke5 metrics	`5CBD9BBCCE2B3EF171A35F022F84C8320248033F2C0502FCD3251EDFB7D0C50A`	5 step-level scalar records
no-cycle smoke5 exit code	`9A271F2A916B0B6EE6CECB2426F0B3206EF074578BE55D9BC94F6F3FE3AB86AA`	内容为 `0`
formal1420 metrics snapshot	`EDC4799C7494ADE72B9A93D692472B2C1BD3712ECB5C5AD92597CB6540511D31`	13 条 early scalar records；不是完整曲线
parent `metrics.jsonl`	`C9B8B081EA3FF1329804DD6923FC5ED3AA3EA8FA36BF7E0248DF050B51608C28`	save2k 线仅保留 step100 单条 metrics
1-epoch evidence freeze	`7C9C4EA2B317A00DF72B1CB29457B1ACD112066C01AA25BC6E750F8D0DE91AA6`	前一切片的 cgroup-OOM / step1500 边界
WAM full10k reconciliation	`70DB0DD0271FD3A90E9DF0D011AA30A52CBFB8E6131773CFCC36C128DC122706`	与独立 full10k run family 分离

原始归档根目录：

```
C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623
```

其中 `remote_outputs_20260629` 只包含日志、JSON、contract 和脚本快照；本页没有把远端 `/data/msz/wm/outputs/...` 路径当作本地权重证据。

5. 官方与本地训练合同

5 条主要 run contract 均采用 Cosmos Predict2 / RoboCasa route：

字段	save2k / save1500 formal	no-cycle smoke5	证据边界
experiment	`cosmos_predict2_2b_480p_robocasa_50_demos_per_task`	同上	contract 直接证据
data_dir	`success_only_from_all_episodes`	同上	path reference，不等于本地数据 hash
rollout_data_dir	`all_episodes`	同上	path reference
T5	`success_only_from_all_episodes/t5_embeddings.pkl`	同上	未冻结 binary/hash
model class	`CosmosPolicyVideo2WorldModel`	同上	contract 直接证据
dataset class	`RoboCasaDataset`	同上	contract 直接证据
batch / world size	16 per GPU / 8	16 per GPU / 8	global batch 128；不是结果本身

字段	save2k / save1500 formal	no-cycle smoke5	证据边界
max steps	9907	5	smoke 与 formal gate 不可混合
save interval	2000 或 1500	0	smoke 禁止把不保存当作 save 通过
p_world_model	0.5	0.5	objective-side contract
action loss multiplier	1	1	objective-side contract
conditional frames	`[5,5]`	`[5,5]`	与上轮 TurnOffMicrowave 诊断的 conditioning 边界一致

base model load summary 的 `local_path` 是：

```
/data/msz/wm/hf_cache/hub/models--nvidia--Cosmos-Predict2-2B-Video2World/
snapshots/f50c09f5d8ab133a90cac3f4886a6471e9ba3f18/model-480p-16fps.pt
```

这表示进程从 HF cache snapshot 读入上游 base model；它不是 `iter_001500` 或 `iter_010000` 的训练产物。

6. 模型输入输出与保存边界

从 contract 可直接确认的模型接口是：`RoboCasaDataset → model.training_step(data_batch, iteration) → CosmosPolicyVideo2WorldModel`。数据 batch 包含 RoboCasa images/proprio/T5 conditioning 与 action/future/value route，但该归档没有单独的 head-shape manifest，因此不把“action/world/value 三个独立 head”写成实现事实；更保守的解释是共享模型内的路由、slot 和 loss contract。

保存前至少需要同时满足模型参数、优化器/调度器状态、global step 与 DeepSpeed 分片元数据的可写入；本轮只看到 DeepSpeed 进入 `_save_checkpoint → module_state_dict → module.state_dict(...)`，没有看到 save/load 完成记录。

对一个 run r，本页把验收状态写成：

$$G_r = (L_r, T_r, S_r, R_r, E_r),$$

其中 L 是 base-load key check，T 是 training-step smoke，S 是 checkpoint save，R 是新进程 reload parity，E 是同 checkpoint eval。当前 smoke run 的状态近似为 $(1, 1, 0, 0, 0)$，formal save2k/save1500 run 分别是 $(1, ?, 0, 0, 0)$ 与 $(1, 1, 0, 0, 0)$。变量与状态来自 `base_load_summary`、`run_contract`、`launch.log` 和缺失的 checkpoint/reload artifact；直觉是“初始化通过”不能跨越保存、重启和评测三个 gate。

7. Run 总账

Run ID	目的/变化	配置	结果	结论
`ds_base_load_smoke_1548`	只验证上游 base load	1 process、max steps 0	715 keys，missing/unexpected 0	load gate 通过；无训练/保存
`ds_train_smoke_1556`	1-step train smoke	1-step、fp route	base load summary 通过	短训练启动；无 save/load
`ds_8gpu_1step_fixedoptional_1728`	修复 optional metrics/config 的 8-GPU 1-step smoke	batch1/GPU、world8、max1	`exit=0`	分布式短 smoke 通过
`ds_8gpu_bs8_1step_1738`	batch8 1-step 对照	batch8/GPU、world8、max1	`exit=0`	batch 变体短 smoke 通过
`ds_8gpu_bs16_nocycle_smoke5_1400`	iterator no-cycle 最小 smoke	batch16/GPU、world8、max5、save0	`exit=0`；5 metrics	iterator 修复短 smoke 通过
`ds_8gpu_bs16_1epoch_save2k_2040`	formal 9907-step gate，save every 2000	batch16/GPU、world8、p_world=.5	rank -9、`EXIT=247`；仅 step100 metrics	长跑资源/外部 kill 线失败
`ds_8gpu_bs16_1epoch_nocycle_save1500_1420`	no-cycle 后重跑 formal gate	batch16/GPU、world8、save every1500	保存时 TypeError，`EXIT=1`	save API first divergence
`formal1420_metrics_snapshot`	对 no-cycle formal run 的 early snapshot	13 scalar records	step13 loss `2.6662`；随后仍在 save gate失败	只能作为片段，不是完整 loss curve

8. 训练 Loss 曲线与可比性

本轮没有完整可绘制的 formal loss curve：

- parent save2k `metrics.jsonl` 只有 1 条 step100 记录，loss `0.609638`；
- no-cycle smoke5 有 5 条记录，loss 从 `2.911592` 到 `2.995014`，但只有 5 steps；

- formal1420 snapshot 有 13 条 early records，step13 loss `2.666178`，随后保存 gate 失败。

因此不能把三条线连接成一条“训练收敛曲线”，也不能用 step100 的 `mse_loss=0.053420` 叫作 epoch MSE。保存的标量可以用于诊断字段覆盖：例如 step100 记录 `world_model_sample_frac=0.1953`、`value_function_sample_frac=0.3281`、GT gripper negative fraction `0.6465`；它们说明 loss logging 在早期可用，不说明最终策略效果。

9. 保存失败与资源失败的因果分析

9.1 save2k：长跑/外部 kill 线

save2k contract 的 `max_steps=9907`、`batch_size_per_gpu=16`、`world_size=8` 与 `save_every_steps=2000`。launch log 最终记录一个 rank 以 `-9` 退出，DeepSpeed launcher 汇总 `EXIT=247`；前一审计中的 cgroup memory 证据把 step1903 的异常归入 CPU cgroup OOM 候选。当前页只把它称为 **resource/launcher failure line**，因为本 bundle 没有完整 cgroup event 文件、最终 RSS 曲线或 checkpoint artifact。

9.2 no-cycle save1500：保存 API first divergence

no-cycle run 的 5-step smoke 通过后，formal save1500 继续到保存点。错误栈核心是：

```
DeepSpeed engine._save_checkpoint
  -> module_state_dict
  -> self.module.state_dict(destination=destination, prefix=prefix, keep_vars=keep_vars)
TypeError: DiffusionModel.state_dict() got an unexpected keyword argument 'destination'
EXIT=1
```

这条证据比“step_001500 目录存在”更强，因为它说明程序在实际生成可用 module state dict 前已经失败。但没有 patch diff、修复后 rerun、非空权重清单或 load parity，所以根因只确认到 API compatibility boundary，修复状态仍是 `OPEN`。

9.3 为什么两个 failure 不能合并

save2k 的 first failure 是长跑/外部 kill 线；save1500 的 first failure 是 state-dict signature 线；两者的 save interval、日期、输出目录不同。把 `EXIT=247`、`EXIT=1`、某个中间 loss 和一个远端 full10k checkpoint 拼成“同一条 1 epoch 训练”会丢失 run identity。

10. 假设—证据—修复审计

假设/问题	证据	修复/下一动作	当前状态
base model 没有正确加载	13 summaries，715 keys，missing/unexpected=0	保留 snapshot identity；继续做 produced-checkpoint reload	`REJECTED FOR BASE LOAD`
1-step/5-step 分布式路径完全不可运行	fixedoptional/bs8/smoke5 exit0	将 smoke 与 formal gate 分开记录	`REJECTED FOR SHORT SMOKE`
no-cycle 修复已完成全训练	smoke5 exit0，但 formal save1500 exit1	先完成 save/load smoke 再跑长训	`REJECTED`
save2k 是保存 API 失败	save2k log 最终是 rank -9/EXIT247；没有 state_dict TypeError 作为 first error	单独保留 resource failure line	`NOT THE SAME FAILURE`
save1500 只是远端同步延迟	launch log 有明确 TypeError，archive 无 checkpoint binary	修复 `state_dict` 标准签名并做新进程 reload	`REJECTED`
`step_001500` 目录等于 checkpoint	archive checkpoint-like count=0；错误发生在 module state dict	生成 file manifest/hash、参数 key/shape、global step	`REJECTED`
early loss `0.0534` 是 epoch-level result	parent metrics 只有 step100 一条	记录 step/aggregation，补完整 curve	`REJECTED`
base load 通过即可跑 eval	没有 produced checkpoint、reload 或 matched eval	完成 save→destroy→load→fixed-input parity	`REJECTED`

11. 恢复所需的最小 save/load smoke

```
def verify_checkpoint_gate(model, engine, fixed_batch, output_dir):
    # 1. Run one real DeepSpeed training step under the target config.
    train_step(engine, fixed_batch)

    # 2. Save model, optimizer, scheduler and global-step metadata through the real API.
    tag = "step_000001"
    engine.save_checkpoint(output_dir, tag=tag)

    # 3. End the process so reload is not satisfied by in-memory parameters.
    destroy_process_group_and_exit()

    # 4. Start a fresh process, load the exact manifest and assert shard completeness.
    fresh_engine = build_engine_from_same_contract()
    load_result = fresh_engine.load_checkpoint(output_dir, tag=tag)
    assert load_result is not None
```

```
    assert parameter_keys_and_shapes_match(fresh_engine, output_dir, tag)

    # 5. Compare a fixed forward/loss before and after reload, including global step.
    reloaded = fresh_engine.forward(fixed_batch)
    assert close_enough(reloaded, reference_output)
    assert fresh_engine.global_steps == 1
    return "save_load_parity_pass"
```

这段伪代码是验收协议，不是归档中已经成功运行的脚本。`state_dict(destination=None, prefix='', keep_vars=False)` 兼容层、DeepSpeed shard/optimizer/scheduler metadata、文件 SHA、固定输入输出 parity 和新进程边界都必须在真实 run 中落档后，才能把 `S=R=1` 写入 gate state。

12. 公源、本地证据与 Ownership 边界

层级	本页支持	不支持
公有上游	PyTorch/DeepSpeed 标准 state-dict 调用形态；Cosmos Predict2/RoboCasa contract	个人设计了 Cosmos 架构
本地证据	base load summary、run contract、metrics、launch logs、exit codes	训练 checkpoint binary/hash、reload parity
本地推断	save2k 与 save1500 是两类 first divergence；gate 必须拆分	资源失败的唯一 root cause、修复后必然成功
未核实	candidate/operator/reviewer、patch author、远端权重归属	“我完成了 1 epoch”“我修复了 DeepSpeed”

13. 面试可用口径

30 秒版本

> 我把 checkpoint gate 拆成 base load、短 smoke、长跑、save、reload、eval 六层。13 个 base-load summary 都是 715 keys 且 missing/unexpected 为 0；8-GPU 1-step 和 no-cycle 5-step smoke 也能 exit 0。但 9907-step save2k 线在长跑中以 rank -9/EXIT=247 退出，no-cycle save1500 线又在 DeepSpeed 调用 `state_dict(destination=...)` 时 TypeError/EXIT=1。归档里 65 个文件没有任何 checkpoint-like binary，所以我能说初始化和 failure boundary 可审计，不能说 1 epoch 完成或 checkpoint 已可加载。

高压追问：目录里有 `step_001500`，为什么不是 checkpoint？

> 因为保存流程在生成 module state dict 时就失败了：DeepSpeed 传入 PyTorch 标准的 `destination/prefix/keep_vars`，Cosmos 的 override 不接受 `destination`。目录存在只说明 save routine 开始，不能证明 module weights、optimizer/scheduler、global step 或分片元数据写完。下一步必须做真实 1-step save，销毁进程，新进程 reload，核对文件清单/hash、key/shape、global step 和固定输入 forward parity。

高压追问：为什么 5-step smoke 通过还不够？

> 因为 smoke 的 save interval 是 0，且只有 5 steps；它只验证 no-cycle iterator、模型初始化、一个短训练路径和进程退出。正式 gate 需要 9907 steps、定期保存、最后保存、跨进程 reload 和同协议 eval。这个项目里 smoke 通过后，formal run 仍在 save1500 处暴露 state-dict 签名错误，正说明短 smoke 不是完整验收。

14. 下一步与关闭条件

1. 给 `DiffusionModel.state_dict` compatibility layer 补标准签名，并记录实际 diff、author 和 review boundary；不要只吞掉关键字参数。
2. 在 1-step/真实 DeepSpeed config 下完成 save→process exit→fresh load→fixed-input parity，生成 checkpoint file manifest 与 SHA-256。
3. 重新跑 save1500 formal gate，保留每个 rank 的 status、checkpoint shard、optimizer/scheduler/global-step 元数据。
4. 只有产生可加载 checkpoint 后，才做同 checkpoint 的 TurnOffMicrowave/RoboCasa eval；不要拿 official-HF `84/118` 或 full10k remote path 做无 key join。
5. 由候选人确认 iterator patch、state-dict patch、运行者、日志审计者与 reviewer 的角色；工具/协作者/上游代码分开登记。

在这些条件闭合前，本页保持 `PARTIAL/WARN`。它可以用于回答“如何定义 save/load gate、如何定位 first divergence”，不能用于宣称 1 epoch 完成、训练 uplift 或 checkpoint ownership。

References

1. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623\remote_outputs_20260629\`（65-file archive；run contracts、base-load summaries、metrics、launch logs）。
2. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623\remote_outputs_20260629\ds_8gpu_bs16_1epoch_save2k_20260623_2040\launch.log`。
3. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623\remote_outputs_20260629\ds_8gpu_bs16_1epoch_nocycle_save1500_20260624_1420\launch.log`。
4. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623\remote_outputs_20260629\ds_8gpu_bs16_nocycle_smoke5_20260624_1400\run_contract.json` 与 `base_load_summary.json`。
5. `C:\Users\mousongzhe\Project\report\cosmos_policy_eval\muxi_deepspeed_1epoch_20260623\nocycle_fix_20260624\smoke5_exit_code.txt`、`smoke5_metrics.jsonl`。
6. Cosmos 1 Epoch Evidence Freeze。
7. WAM 1 epoch / full10k Run Reconciliation。
8. WAM Claim–Evidence Inventory。

Part 12 / 49 · `interview_materials/vla/evidence/projects/wm019_wam_artifact_manifest.html` · SHA-256 `1AB8709201E2E75F580CB0AA0F8465DEFB546D63B2AAB70A1752239A7DA8F697`

返回本卷目录

Canonical Markdown SHA-256: `1ab8709201e2e75f580cb0aa0f8465defb546d63b2aab70a1752239a7da8f697`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wm019_wam_artifact_manifest.md`

WM-019 WAM Diagnostic Artifact Manifest（CoffeePressButton）

状态: `manifested / local-artifacts-partial / checkpoint-unverified / WARN` 盘点日期: 2026-07-18 证据截止: 2026-07-18 范围: 只登记一组 `CoffeePressButton` WAM action-rollout diagnostic bundle 的本地文件、manifest 字段和缺口; 不把它升级为完整 benchmark、训练结果、个人 uplift 或 checkpoint-verified run。

1. 结论先行

当前工作区外部项目目录存在一组可直接读取的 WAM diagnostic artifact: base/trained 两个 rollout `manifest.json`、两个 `actions.jsonl`、base/trained raw video 与共同 rollout log。两个 manifest 都记录 `task=CoffeePressButton`、`episode_idx=0`、`seed=195`、`env_seed=0`、`max_steps=300`、`num_open_loop_steps=16`、`num_denoising_steps_action=5`，并记录 `success=false`。

但 manifest 中的 base checkpoint 与 trained checkpoint 都是 `/data/msz/...` 路径; 在当前 Windows 工作区没有对应 checkpoint binary。trained raw video 已在同一 Project 下的 `cosmos_policy_robocasa_action_rollout_20260605` 目录找到，其 SHA 与 manifest 记录的远端文件名相符，但 output root 与 action-trace目录分离。因此本 manifest 的证据等级是 `local-direct artifact bundle + checkpoint-unverified`，不是完整可复算的 model comparison。

2. Bundle identity

```
project_root = C:\Users\mousongzhe\Project\wm\outputs
task         = CoffeePressButton
episode_idx  = 0
seed         = 195
env_seed     = 0
max_steps    = 300
open_loop    = 16
denoise      = 5
controller   = OSC_POSE / control_delta=true
cameras      = robot0_agentview_left, robot0_agentview_right, robot0_eye_in_hand
```

Manifest 还记录三路 camera 为 `224×224`、PandaMobile、`obj_instance_split=B` 和单个 `[1,1]` layout/style id。action JSONL 的前 3 条记录包含 `policy_action7`、扩展后的 `env_action`、reward、done 和 `success_after_step` 字段; 这支持 action/step trace 的存在，不支持任务整体成功率。

3. Immutable file inventory

role	local path	bytes	SHA-256	当前可支持
base manifest	`C:\Users\mousongzhe\Project\wm\outputs\cosmos_policy_fourway_robot_eval_20260605\action_base\CoffeePressButton--action_rollout--2026_06_05-23_59_36\manifest.json`	3901	`63A518674F061D1C14A880805FF9C23DAC2425C9D7B34CC4C75FD3CEDBD33A86`	base checkpoint 引用、task/seed/horizon/controller/camera 字段、`success=false`
base action trace	同一目录 `actions.jsonl`	126118	`C6A3623B26B97F7ED88AA13AE675C8315DC509D3D867CBB6E417CAF51F88525D`	per-step `policy_action7/env_action/reward/done/success_after_step`
base raw video	同一目录 `2026_06_05-23_58_51--episode=0--success=False--task=press_the_button_on_the_coffee_machine_to_serve_coffee.mp4`	419966	`D469A8B2CB3686C07C888B637009EE06D8DA10F2B2AEA2A5F3B186AD2E62F388`	一条失败 episode 的本地视频文件
trained manifest	`C:\Users\mousongzhe\Project\wm\outputs\cosmos_policy_fourway_robot_eval_20260605\action_trained\CoffeePressButton--action_rollout--2026_06_05-22_00_19\manifest.json`	3918	`809AE4F879C9421F954C7BFC26C70005B94387412449039F9FA8B6E479EF55CC`	trained checkpoint 引用、同一 task/protocol 字段、`success=false`
trained action	同一目录 `actions.jsonl`	128770	`9C6A4F00DA8107EB0C1FDDDC642D1986CD510D92F7F6E3A68ADE8`	trained per-step action trace

role	local path	bytes	SHA-256	当前可支持
trace			`34E8B82CF2D`	
trained raw video	`C:\Users\mousongzhe\Project\wm\outputs\cosmos_policy_robocasa_action_rollout_20260605\CoffeePressButton--action_rollout--2026_06_05-22_00_19\2026_06_05-21_59_38--episode=0--success=False--task=press_the_button_on_the_coffee_machine_to_serve_coffee.mp4`	39810303	`59176353B1E069C6F3A3A20BDF13EE6A9FFD969537C5501B685293D87802877E`	另一输出 root 的同名 trained 失败视频；需保留 root reconciliation
rollout log	`C:\Users\mousongzhe\Project\wm\outputs\cosmos_fourway_robot_eval_20260605_pack\logs\cosmos_action_rollout_trained_20260605.log`	157555	`1944896104A97BD9F867E3AD39F53CCB407226E0E942E56AD5545B0AB2A27143`	运行时模块、`NUM_ACTIONS_CHUNK=32`、`ACTION_DIM=7` 等日志线索

4. Checkpoint 与 provenance gate

字段	base	trained	结论
manifest checkpoint	`/data/msz/models/cosmos_predict2p5_2b_base/base/pre-trained/308eb96c-c4c0-4a06-9cc1-103a43beff28/model_ema_bf16.pt`	`/data/msz/wm/outputs/cosmos_policy_online_text_formal_20260605/cosmos_policy/cosmos_v2_finetune/cosmos_policy_online_text_formal_20260605/checkpoints/iter_000010000`	两者均是远端引用
local binary	未找到	未找到	不能做独立 load/hash/fixed-batch replay
config/statistics	manifest 的 `config` 与 `dataset_stats_path` 字段存在	同左	只支持引用关系，不支持文件内容同源
output video	base raw video 存在	trained raw video 在另一 Project output root 找到，SHA 已登记	video 文件齐，但 output-root provenance 需写入 run manifest

因此不能把 base/trained 的 action trace 差异写成“checkpoint 导致的因果变化”，也不能把 episode 0 的 `success=false` 外推成 WAM 任务成功率或 0-success benchmark。

5. WM-019 metric mapping

WM-019 层级	本 bundle 有什么	缺什么	当前 verdict
offline target fit	无 prediction/label metric script	split、target、mask、reduction、valid count	`MISSING`
teacher-forced one-step	manifest 有 protocol/horizon 字段	per-offset target error、checkpoint binary	`MISSING`
open-loop	action trace、300-step episode、16-step open-loop 字段、base/trained raw video	per-offset scorer、action-sensitivity probe、root reconciliation	`PARTIAL / WARN`
simulator closed loop	base/trained manifest 各有 episode 0 `success=false`	多 episode denominator、同 checkpoint 可复算、完整 environment version	`DIAGNOSTIC ONLY`
real closed loop	无 real hardware artifact	controller/camera/latency/安全 trace	`MISSING`

6. 允许的面试口径

可以说：

> “我能从一个 CoffeePressButton 诊断 bundle 复核到 base/trained 的 rollout manifest、action trace、controller/camera/horizon 字段和两条失败视频；但 checkpoint 仍是远端路径，输出 root 与多 episode denominator 还需 reconciliation，所以我把它作为接口与失败诊断证据，不把它说成完整 benchmark 或模型 uplift。”

不能说：

1. “本地已经加载并验证了 base/trained checkpoint。”
2. “trained 比 base 成功率提升/下降。”
3. “episode 0 失败证明 WAM 是 0-success。”
4. “action trace 存在就等于 action MSE、proxy score 或 closed-loop success 已闭合。”

7. 下一步最小补件

1. 为 base/trained 各补 checkpoint binary 或不可变远端 hash、processor/normalizer/config SHA。
2. 把 trained 视频的 alternate output root 写入 immutable run manifest，并为同一 protocol 收集完整 episode manifest，而非只保留 episode 0。
3. 添加 prediction/label 与 metric script，报告 per-offset/action-sensitivity，而不是只保存 action JSONL。
4. 建立 run/ownership tuple：`(run_id, checkpoint_sha, config_sha, task, init, seed, controller, predicate, summary_path, owner)`。

当前结论保持 `WARN`；本 manifest 只关闭“本地完全没有任何 WAM rollout artifact”的旧疑问，不关闭 checkpoint、metric、denominator、ownership 或 uplift gate。

参考来源

1. Cosmos/WAM Claim–Evidence Inventory。
2. WM-019 Project Metric Artifact Gate。
3. WM-019 Metric Source Freeze。
4. Benchmark / Evidence Audit E。

Part 13 / 49 · `interview_materials/vla/evidence/projects/wm019_wam_action_trace_paired_diagnostic.html` · SHA-256 `97276C418F26E3EDD6F922FD3BF5A4494456ECA3FCA46A7B3882CC7AD71627E6`

返回本卷目录

Canonical Markdown SHA-256：`97276c418f26e3edd6f922fd3bf5a4494456eca3fca46a7b3882cc7ad71627e6`；生成自：`interview_materials\vla\evidence\projects\wm019_wam_action_trace_paired_diagnostic.md`

WM-019 WAM Action-Trace Paired Diagnostic（CoffeePressButton）

状态：`paired_diagnostic / local-direct-trace-math / checkpoint-unverified / WARN` 计算日期：2026-07-18 证据截止：2026-07-18 范围：只比较同一 CoffeePressButton episode 0、相同 seed/env_seed 和相同 300-step protocol 下的 base/trained `actions.jsonl`；不把 trace 差异解释为 checkpoint 因果、success uplift 或 benchmark 结果。

1. 目的与边界

WM-019 要求把 action MSE、open-loop trace 与 downstream success 分层。本切片只回答三个低层问题：

1. base/trained action trace 是否逐步对齐、shape 是否一致、是否有非有限值?
2. 两条 trace 的逐步差异有多大?
3. 两条 trace 的 `success_after_step`、reward 和 episode 长度是否支持更高层结论?

它不回答：哪个 checkpoint 更好、为什么不同、是否提升成功率、是否代表 WAM 全部任务。

2. 输入 identity

side	manifest SHA	action trace SHA	protocol
base	`63A518674F061D1C14A880805FF9C23DAC2425C9D7B34CC4C75FD3CEDBD33A86`	`C6A3623B26B97F7ED88AA13AE675C8315DC509D3D867CBB6E417CAF51F88525D`	CoffeePressButton、episode 0、seed 195、env_seed 0、300 steps、open-loop 16、denoise 5
trained	`809AE4F879C9421F954C7BFC26C70005B94387412449039F9FA8B6E479EF55CC`	`9C6A4F00DA8107EB0C1FDDDC642D1986CD510D92F7F6E3A68ADE834E8B82CF2D`	同上；checkpoint 字段仍为远端路径

输入文件均来自 WM-019 WAM Artifact Manifest。本报告没有读取或修改远端 checkpoint。

3. Trace integrity

check	base	trained	结论
rows	300	300	一致
`t` range	0..299	0..299	逐步对齐
`policy_action7` shape	300×7	300×7	一致
`env_action` shape	300×12	300×12	一致
non-finite policy values	0	0	未发现 NaN/Inf
`success_after_step=True`	0	0	两条 trace 都未触发成功字段
reward sum	0.0	0.0	不能作为更高层 success 证明

shape 与有限性检查只证明 JSONL 的结构可读；不证明动作语义、controller 映射或 checkpoint 内容正确。

4. Paired difference

对齐同一 `t` 后，令 $a_t^{(b)}$ 和 $a_t^{(t)}$ 分别为 base/trained 的 7 维 `policy_action7`，计算：

$$\mathrm{MAE} = \frac{1}{Td}\sum_{t=1}^{T}\sum_{j=1}^{d}|a_{t,j}^{(b)} - a_{t,j}^{(t)}|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{Td}\sum_{t=1}^{T}\sum_{j=1}^{d}(a_{t,j}^{(b)} - a_{t,j}^{(t)})^2}.$$

这里 (T=300,d=7)。它们只是两份 action trace 的几何差异，不是 action-label loss，也不等于成功率。

measure	value
policy MAE	`0.3100783611`
policy RMSE	`0.3906446459`
per-dimension MAE	`[0.38234915, 0.31824048, 0.27147196, 0.38302800, 0.30951413, 0.24404278, 0.26190202]`
mean policy norm, base	`0.6584048120`
mean policy norm, trained	`0.6782378918`

逐步差异明显，但由于两个 checkpoint binary 不在本地、没有 prediction target/metric script，也没有成功 episode，不能把差异归因于某个训练改动，更不能说 trained “更好”或“更差”。

5. Range diagnostic

dimension	base min	base max	trained min	trained max
0	0.074116	0.389210	-0.569726	0.206362
1	0.064427	0.395674	-0.485705	0.294611
2	0.081652	0.409145	-0.454376	0.420165
3	0.050104	0.426199	-0.686172	0.402813
4	0.027087	0.366351	-0.622367	0.335208
5	-0.017900	0.354746	-0.501100	0.373214
6	-0.049619	0.515565	-0.428721	0.371992

范围差异可以作为 controller/action-normalizer 排查入口，但不能单凭 range 断言 high-sigma、gripper、normalizer 或动作顺序是根因。需要 action schema、normalizer、checkpoint 和 paired rerun 才能继续归因。

6. WM-019 ladder placement

层级	本切片能支持什么	仍不能支持什么
trace integrity	两份 JSONL 的 row/t/shape/finite/success 字段可审计	checkpoint 可加载、动作语义正确
open-loop diagnostic	300-step trace 的逐步差异与范围差异	per-offset prediction error、action sensitivity causal test
simulator success	episode 0 两边 `success=false` 字段	denominator、完整 suite success、模型比较
project claim	“有一组可复核的 action-rollout diagnostic bundle”	WAM MSE、uplift、0-success benchmark、个人原创 architecture

7. 下一最小补件

1. 补 base/trained checkpoint 或 immutable remote hash，并绑定 processor/normalizer/config SHA。
2. 把 alternate trained-video output root 绑定进 run manifest，并建立多 episode manifest 和 success predicate/denominator。
3. 用同一 target/schema 计算 per-offset prediction error、action sensitivity 与 controller round-trip。
4. 只有 paired rerun 显示固定 protocol 下的 success 差异，才讨论 model effect；当前不更新简历数字。

参考来源

1. WAM Artifact Manifest。
2. WAM Claim–Evidence Inventory。
3. WM-019 Project Metric Artifact Gate。
4. WM-019 Metric Source Freeze。

Part 14 / 49 · `interview_materials/vla/evidence/projects/wm019_wam_future_metric_paired_diagnostic.html` · SHA-256 `DC58EE08006776333D0D2BF130DDEBA8896AFBC754E276B9F67D6CFE04CECE2E`

返回本卷目录

Canonical Markdown SHA-256：`dc58ee08006776333d0d2bf130ddeba8896afbc754e276b9f67d6cfe04cece2e`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wm019_wam_future_metric_paired_diagnostic.md`

WM-019 WAM Future-Image Metric Paired Diagnostic（CoffeePressButton）

状态：`paired_metric_diagnostic / offline-future-image / checkpoint-unverified / WARN` 计算日期：2026-07-18 证据截止：2026-07-18 范围：只比较同一离线 episode/prompt、同一 sample indices、同一 seed=7 下 `iter_000005000` 与 `iter_000010000` 的 future-image PSNR/SSIM JSON；不把视觉指标升级为 action validity、simulator success 或 real success。

1. 输入与协议

字段	值
manifest	`cosmos_policy_eval_like_dreamdojo_20260605/manifest.json`
manifest SHA-256	`8C4872F711897F718621C7AE8A6DF49EEDDE759CBECEDE530A3D82ABA819EC71`
prompt	`press the button on the coffee machine to serve coffee`
episode source	`/data/msz/wm/wm_data/.../ep=16--success=True--regen_demo.hdf5`（远端引用）
sample count	1
future frame count	12
sample indices	`[11230,11235,11241,11246,11252,11258,11263,11269,11275,11280,11286,11292]`
future indices	`[32,37,43,48,54,60,65,71,77,82,88,94]`
chunk / steps	`32 / 4`
seed	`7`
metric implementation	SSIM 标记为 `skimage`；LPIPS 为 `null`

两个 metrics JSON 的 checkpoint 字段均为 `/data/msz/.../iter_000005000` 或 `/data/msz/.../iter_000010000` 远端路径，本地没有对应 binary/hash。

2. Immutable metric inputs

label	metrics JSON	SHA-256	summary SHA-256
`iter_000005000`	`iter_000005000/0000_metrics.json`	`0FD008BBAD68A9FDD273A7E1DC7659F0F6B66927194671AC112FA5DEDAF72C18`	`9BDC8A438153DB2ED02BAC77DDF5964A7A3BA241BDEEB526D7993507D9CDDDAE`
`iter_000010000`	`iter_000010000/0000_metrics.json`	`C9AB7F04AFD9C9057CC548BF0176285E001BA778BCD37DC4FB7D857E29BAEC9D`	`805112AB530C63DD0293E918731C3721707EC508B9239D7EE3F2DB935FC868A1`

这是一组离线 future-image prediction metrics，不是上一份 seed=195、300-step action-rollout trace；两种协议必须分开记录。

3. Metric comparison

metric	iter 5000	iter 10000	delta (10000−5000)	unit/notes
future-only PSNR	8.5813418454	8.2140051576	-0.3673366878	12 future frames、单 sample
future-only SSIM	0.0452700028	0.0428580477	-0.0024119550	`skimage`、单 sample
with-condition PSNR	15.5366232419	15.1975432224	-0.3390800195	13 frames，含 condition frame
with-condition SSIM	0.1187107718	0.1164843517	-0.0022264201	`skimage`，含 condition frame
LPIPS	null	null	—	没有 LPIPS 结果

PSNR 依赖像素误差，常见形式为：

$$\mathrm{PSNR} = 10 \log_{10} \frac{MAX_I^2}{\mathrm{MSE}(I, \hat{I})}.$$

本表的 delta 只表示该离线 sample、该 scorer 和该 checkpoint 引用下的视觉指标差异；它不能证明长时序 action consequence 或策略优劣。

4. Evidence interpretation

可以支持：

- 两个 checkpoint label 在同一离线采样协议下都有可读的 PSNR/SSIM JSON；
- `iter_000010000` 在该单 sample 的 future-only 与 condition-inclusive 指标都低于 `iter_000005000`；
- metric 输出记录了 frame count、future-only/condition-inclusive 分层和 scorer implementation。

不能支持：

- “训练到 10000 steps 后视觉质量下降是 checkpoint 的因果结论”：本地没有 checkpoint binary、训练 lineage 或重复 samples；
- “PSNR/SSIM 下降说明 action policy 变差”：没有 action target、controller、closed-loop episode；
- “CoffeePressButton success 下降”：离线 episode source 与 rollout episode protocol 不同，且这里没有 simulator success predicate；
- “WAM benchmark 或 real robot 泛化”：样本数为 1，远端 episode source，缺 denominator 与 multi-seed。

5. WM-019 ladder placement

层级	当前证据	仍缺
teacher-forced/offline future-image proxy	12 future frames、PSNR/SSIM、两个 checkpoint label	checkpoint binary/hash、重复 sample、per-offset scoring
open-loop action rollout	另有 seed=195 action trace，但不与本页协议合并	target-based per-offset error、action sensitivity
simulator closed loop	本页没有 success predicate	多 episode denominator、controller/reset/init、checkpoint
real closed loop	没有 real hardware artifact	全部 hardware trace/safety fields

6. 下一最小补件

1. 绑定本地或不可变远端 checkpoint hash、processor/normalizer/config revision。
2. 对同一 checkpoint 收集多个 episode/sample 和 fixed-init/seed manifest。
3. 将 future-image metric 与 action trace、simulator predicate 做 matched checkpoint paired validation。
4. 保持 PSNR/SSIM 为 proxy，不写入 WAM success/uplift 或个人简历结果。

参考来源

1. WAM Artifact Manifest。
2. WAM Action-Trace Paired Diagnostic。
3. WM-019 Project Metric Artifact Gate。
4. WM-019 Metric Source Freeze。

Part 15 / 49 · `interview_materials/vla/evidence/projects/wm019_wam_matched_checkpoint_protocol_bridge.html` · SHA-256 `648F4D3180BF99299DFB5C6E826592BF861B372416DC4BBCEA5854DAD263CCE3`

返回本卷目录

Canonical Markdown SHA-256：`648f4d3180bf99299dfb5c6e826592bf861b372416dc4bbcea5854dad263cce3`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\wm019_wam_matched_checkpoint_protocol_bridge.md`

WM-019 WAM Matched-Checkpoint Protocol Bridge（iter_000010000）

状态：`matched-checkpoint-path / offline-to-rollout bridge / binary-unverified / WARN` 审查日期：2026-07-18 证据截止：2026-07-18 范围：只证明离线 future-image metric 与 trained CoffeePressButton rollout 引用了完全相同的 checkpoint path；不把 path equality 当 checkpoint hash，不把离线视觉指标与 rollout success 直接相关或因果化。

1. Identity 结论

离线 `iter_000010000/0000_metrics.json` 的 `checkpoint_path` 与 trained action-rollout `manifest.json` 的 `checkpoint` 字段逐字节相同：

```
/data/msz/wm/outputs/cosmos_policy_online_text_formal_20260605/
cosmos_policy/cosmos_v2_finetune/
cosmos_policy_online_text_formal_20260605/checkpoints/iter_000010000
```

UTF-8 path string SHA-256 为：

```
6CAE0B6BAA4378F25A0AF92720303922B709F7B8A21994684E6F77B6ED14E833
```

这是路径字符串 fingerprint，不是 checkpoint binary SHA。当前 Windows 工作区没有该 checkpoint binary，不能独立验证权重内容、processor、normalizer 或 config。

2. 冻结输入

role	local artifact	SHA-256
offline protocol manifest	`cosmos_policy_eval_like_dreamdojo_20260605/manifest.json`	`8C4872F711897F718621C7AE8A6DF49EEDDE759CBECEDE530A3D82ABA819EC71`
offline iter-10000 metrics	`iter_000010000/0000_metrics.json`	`C9AB7F04AFD9C9057CC548BF0176285E001BA778BCD37DC4FB7D857E29BAEC9D`
trained rollout manifest	`cosmos_policy_robocasa_action_rollout_20260605/.../manifest.json`	`809AE4F879C9421F954C7BFC26C70005B94387412449039F9FA8B6E479EF55CC`
trained action trace	paired rollout `actions.jsonl`	`9C6A4F00DA8107EB0C1FDDDC642D1986CD510D92F7F6E3A68ADE834E8B82CF2D`
trained raw video	paired rollout failure video	`59176353B1E069C6F3A3A20BDF13EE6A9FFD969537C5501B685293D87802877E`

3. Protocol bridge

field	offline future-image metric	trained action rollout	是否相同
checkpoint path	`.../iter_000010000`	`.../iter_000010000`	exact match
task semantics	prompt：press the button on the coffee machine to serve coffee	task=`CoffeePressButton`，description 同 prompt	semantic match
input episode	successful regen-demo HDF5，relative indices 0..62	RoboCasa env episode 0	不同
seed	7	policy seed 195 / env seed 0	不同
horizon	chunk 32；4 denoising steps；12 future frames	300 env steps；open-loop 16；5 action denoising steps	不同
output	future-image PSNR/SSIM	action trace、raw video、success predicate	不同证据层
denominator	sample_count=1	episode_count=1	都过小且单位不同

这张表完成的是 checkpoint-reference bridge，不是 protocol equivalence。offline sample 与 rollout episode 不同，不能做逐样本 proxy-success correlation。

4. 两侧观测

Offline future-image side

- `sample_count=1`，`future_frame_count=12`；
- future-only `PSNR=8.2140051576`、`SSIM=0.0428580477`；
- with-condition `PSNR=15.1975432224`、`SSIM=0.1164843517`；
- SSIM implementation 标记为 `skimage`，LPIPS 为 `null`。

Action-rollout side

- 300 条 action rows，`policy_action7` shape `300×7`，`env_action` shape `300×12`；
- 无非有限 policy value；
- `max_steps=300`、`num_open_loop_steps=16`、`num_denoising_steps_action=5`；
- episode-level `success=false`，逐 step `success_after_step=True` 次数为 0；
- trained raw video 已本地哈希，但 checkpoint 仍未本地验证。

5. 能说与不能说

可以说：

> “离线 future-image metric 和一条 CoffeePressButton rollout 确实引用同一个 `iter_000010000` checkpoint path；我能分别复核 metrics JSON、rollout manifest、action trace 和视频。但离线与 rollout 的 episode、seed、horizon 和统计单位不同，所以我只把它当 checkpoint-reference bridge，不把 PSNR/SSIM 与 success 做相关或因果结论。”

不能说：

1. “checkpoint 已被本地 hash/加载验证”——当前只有路径字符串相同。
2. “PSNR/SSIM 低导致 rollout 失败”——n=1、输入不同、无 matched sample。
3. “iter 10000 比 iter 5000 成功率更差”——iter 5000 没有同协议 rollout denominator。
4. “episode 0 失败证明 WAM 0-success”——没有完整任务/seed/init ledger。

6. 下一步最小闭环

1. 获取 `iter_000010000` checkpoint binary 或 immutable remote content hash，并绑定 processor/normalizer/config SHA。
2. 对相同 task/init/seed，保存 future metric、action trace、controller、predicate 和 episode outcome。
3. 对 `iter_000005000` 补同协议 rollout，或对 `iter_000010000` 扩展多 sample/multi-seed；再计算 proxy-success correlation。
4. 在完成 matched protocol 前，WAM 历史 MSE/success/uplift 继续保持 `WARN/REMOVED`。

参考来源

1. WAM Future-Metric Paired Diagnostic。
2. WAM Action-Trace Paired Diagnostic。
3. WAM Artifact Manifest。
4. WM-019 Project Metric Artifact Gate。

Part 16 / 49 · `interview_materials/vla/practice/uwm_oral_model_answer.html` · SHA-256 `56AF65AB74FE2FC655125FEC43058F6B0A4795E988DCA261F371E5E06D8B3BFE`

返回本卷目录

Canonical Markdown SHA-256：`56af65ab74fe2fc655125fec43058f6b0a4795e988dca261f371e5e06d8b3bfe`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\practice\uwm_oral_model_answer.md`

PROJ-UWM-001..004 从项目实现反推的证据约束口述稿

状态：`implementation_derived_model_answer / evidence_bounded / not_candidate_transcript / unseen` 更新日期：2026-07-22 题目：如何介绍 UWM/LIBERO 项目，并解释训练适配、strict/reset 协议、DataLoader stall、ownership 与 RoboCasa 闭环失败?

1. 项目定位：讲协议工程，不讲架构原创

这个项目最可信的定位不是“我提出了 UWM 新架构”，而是：在 C500/MACA 环境中开展 UWM 的 LIBERO-90→LIBERO-10 task-specific fine-tune 与闭环评估，对齐数据、训练和 rollout 协议；再把这些合同迁到 RoboCasa，定位 action/state/prompt/normalizer/execution 不一致导致的 train-to-closed-loop 断层。

回答顺序固定为：目标与职责 → 数据/模型/训练链路 → 评估 tuple → 一个 bad case → 结果证据等级 → ownership 与未闭合项。不要先报 `0.744`，也不要把低训练 loss 说成闭环性能提升。

2. 30 秒版本

我做的是 UWM 从 fine-tune 到闭环评估的协议工程。在 LIBERO 路线，我把五个 LIBERO-10 task 分别接入两路 RGB、7D action、16-step chunk 的训练与 rollout，区分 `env.seed+reset` 和 fixed `pruned_init` 两套评估，并用 DataLoader-only A/B 定位 Zarr 随机读取等待。用户确认的 fixed-init 结果是 `186/250`，但本地没有对应五个 checkpoint 和 250 条 episode raw bundle；RoboCasa actionheadfix 则是训练完成但闭环只有 `0-2/54` 的失败链。所以我讲协议对齐、工程诊断和结果边界，不称 UWM 架构原创或 benchmark 完成。

3. 2–3 分钟主叙事

项目最初是把 UWM 的 `libero_90.pt` 迁到 RoboCasa，但 action order、state/prompt、normalizer 和 eval harness 没有形成闭环，所以我先回到 UWM/LIBERO 路线建立可解释的训练—评估基线。 数据侧，项目把五个 LIBERO-10 HDF5 task 各自转成 compressed Zarr buffer。每个训练 window 长 19：前 2 帧作为当前两路 RGB 观测，中间对齐 16 个 7D action，最后 2 帧作为 future observation target。LIBERO adapter 明确关闭 low-dim 和 language 输入；这些是当前 route 的接口，不代表 UWM 在所有数据上都只有这两种输入。 训练侧，从 `libero_90.pt` strict-load task-specific 模型，8 卡 DDP 训练；image encoder 使用 base learning rate 的 `0.05` 倍，其余参数使用 base learning rate。这里不是一个共享五任务 checkpoint：Book-Caddy 指向单独的 6000-step run，其余四个 task 指向 bs64 epoch-aligned run。所谓 aligned 是把官方 `20k steps × 36 × 2 GPUs` 换算为本地 `64 × 8 GPUs` 下相近的 nominal epoch 数，得到约 `2728–2814` 个 optimizer steps，不是证明优化轨迹与官方一致。 评估侧，`uwm_reset` 用 `env.seed(e)+env.reset()`；`libero_fixed_init` 按 `.pruned_init` index reset，并先执行 5 个 zero-action warmup。每个 task 50 条 episode，被拆成 8 个 shard；每条记录保存 seed、init index、success 和 policy-action count，task/global summary 用 completed episodes 作分母。用户确认的 fixed-init summary 是 `44/50、40/50、38/50、33/50、31/50`，合计 `186/250=0.744`；旧 reset-style 是 `232/250=0.928`。两者只能说明 protocol sensitivity，论文 Table II 还有 OOD init、object shift、背景和多 seed 差异，不能横比。 实现复核还有三个边界。第一，LIBERO evaluator 只读取 `ckpt["model"]`，没有读取 action normalizer；base checkpoint 中 normalizer 也是 `None`，因此不能说这条路径“加载并固定了 target normalizer”。第二，五个 task 使用 task-specific checkpoint，不能讲成一次 multi-task fine-tune。第三，聚合器虽然记录 expected=50，却没有 assert completed=50；当前 `186/250` 依赖用户确认的五个 50-rollout summary，本地 raw episode 不在。 工程 bad case 是周期性 DataLoader wait。finite sampler 的 p95 约 `13.862s`，只换 infinite sampler 仍约 `13.183s`，preload+infinite 后约 `0.194s`。这支持随机 Zarr 读取是主要等待来源，但还不是 GPU 训练吞吐提升。RoboCasa 侧也保持失败结论：actionheadfix 30-epoch run 的 action loss 从 `2.4300` 降到 `0.07689`，而 full-h4 仍 `0/54`、`45/54` 仅发生 robot-env contact。我的结论是训练拟合没有转化为任务成功，下一步要追 expert→x0→sampled→converted→executed action 的 first divergence。

4. LIBERO-only 90 秒版本

LIBERO 这条线不是训练一个五任务统一策略，而是把五个 LIBERO-10 task 分别从 `libero_90.pt` 继续训练。数据是两路 `128×128` RGB 和 7D action；window 长 19，模型用 2 帧历史、预测 16-step action chunk 和 2 帧未来观测。模型配置是 ResNet observation encoder、768

hidden、12 层、12 heads、24 registers、100-step diffusion training schedule 和 10-step inference schedule；这是上游 UWM 架构在本地 adapter 中的实例化，不是我的原创。 训练适配的关键不是直接照搬 20k optimizer steps。官方口径是 per-device batch 36、2 卡；本地是 64、8 卡，所以脚本按每个 task 的 sequence 数换算 nominal epochs，remaining four 得到约 2.7k–2.8k steps；Book-Caddy 仍来自独立 6000-step run。评估再区分 reset-style 和 fixed-pruned-init：后者 5-step zero-action warmup、50 rollouts/task、8 shard。用户确认 fixed-init 为 `186/250`，但 checkpoint/hash/250 episodes 没同步，因此只作为 user-confirmed summary。实现上 evaluator 只 strict-load model weights，没有加载 normalizer；聚合器也没有强制 completed 等于 expected。这两点是我面试中会主动说明的证据边界。

5. 从实现讲清数据、训练与评估

5.1 数据 window 如何形成

`libero_official_common.py` 固定五个 task、两路 RGB `agentview_rgb` / `eye_in_hand_rgb` 与 7D action。`prepare_libero_buffers.py` 检查每个 task 50 个 episode，将 HDF5 写入 compressed Zarr，并在该 route 中设置 `flip_rgb=True`。

`process_batch` 对长度 19 的 window 做以下切分：

```
# obs_horizon=2, action_horizon=16, seq_len=19
action_start = 1
action_end = 17
curr_obs = obs[:, :2]       # 两帧当前观测
actions = action[:, 1:17]   # 16×7 action chunk
next_obs = obs[:, 17:]      # 两帧 future observation target
```

口述时要强调：该 LIBERO route 的 `use_low_dim=False`、`use_language=False`；RoboCasa route 的 state/prompt/right-view 是另一套输入合同，不能倒灌进 LIBERO 结果解释。

5.2 模型与训练入口

本地 evaluator 实例化的接口为：2-frame observation、`action_len=16`、`action_dim=7`、ResNet encoder、768 embedding、latent patch `[2,4,4]`、12 layers/12 heads、24 registers、100 diffusion train steps 和 10 inference steps。训练 `train_one_step` 同时把 `curr_obs`、`next_obs` 和 action chunk 送入 UWM；shared action/video diffusion 机制属于上游。

`train_robomimic.py` 在 pretrain load 后把 image encoder 参数单独分组为 `0.05×base_lr`，其余 trainable 参数使用 base lr，再包 DDP。`save_every=999999` 不代表不保存：`maybe_save_checkpoint` 仍在 final step 写 `models.pt`。

5.3 为什么本地不是简单跑 20k steps

脚本先计算每个 task 在官方与本地 world-size/batch 下的 steps per epoch，再按 nominal epoch 数换算：

$$N_t^{\text{local}} = \left\lceil N^{\text{official}} \frac{S_t^{\text{local}}}{S_t^{\text{official}}} \right\rceil, \qquad N^{\text{official}} = 20000.$$

变量解释：`S` 是脚本用 per-rank sample 数和 per-device batch 计算的 steps per epoch；`t` 表示 task。直觉上，这保留的是近似训练 epoch 数，不是 optimizer update 数、样本顺序或数值轨迹。实现映射是 `remote_muxi_libero_remaining4_bs64_aligned_queue.sh::steps_per_epoch/task_steps`。

Task	train sequences	official steps/epoch	local steps/epoch	aligned optimizer steps	实际 checkpoint route
Book-Caddy	8243	114	16	2808（公式值）	严格评估 launcher 实际指向独立 6000-step run
Soup-Cheese	12102	168	23	2739	remaining4 bs64 aligned run
Bowl-Drawer	11104	154	21	2728	remaining4 bs64 aligned run
Moka-Moka	18957	263	37	2814	remaining4 bs64 aligned run
Mug-Mug	11542	160	22	2750	remaining4 bs64 aligned run

因此安全表达是“按 task sequence 数做 epoch-aligned step conversion，并使用 task-specific checkpoints”，不是“完全复现官方 20k-step training”。

5.4 两套 rollout protocol

字段	`uwm_reset`	`libero_fixed_init`
reset	`env.seed(seed); env.reset()`	`reset_to_state(pruned_init[index])`
init index	无	`(init_start+episode) % len(init_states)`
warmup	0	5 个 shape `(1,7)` zero actions
rollout horizon	必须看 launcher；evaluator default 1000	strict launcher 显式传 600，并将 warmup 加到 env max length
success	episode 终止时 `bool(info.get("success", False))`	同左
artifact	shard `episodes.jsonl/summary.json`	同左，另有 init path/count

8 shard 对 50 条 episode 的分配是 `7,7,6,6,6,6,6,6`；seed 与 init-start 用相同 offset，目标是覆盖 0–49 且不重叠。并行发生在同一 task 的 shard 内，五个 task 仍按 launcher 顺序处理。

5.5 结果分母与聚合风险

代码中的 task/global success rate 都按实际写入的 episode 数计算：

$$\widehat{p} = \frac{\sum_{i=1}^{N_{\text{completed}}} \mathbf{1}[s_i = 1]}{N_{\text{completed}}}.$$

变量解释：`s_i` 是 episode 最终 `info["success"]`；分母是 `num_rollouts_completed`，不是配置的 `num_rollouts_expected`。直觉上，如果一个 shard 没写出 episode，聚合结果仍可能生成但分母变小。实现映射是 5-task parallel launcher 的 per-task/global Python aggregator；当前缺少 `completed==expected` 的 fail-closed assertion。

用户确认的 fixed-init per-task summary为：

Task	Successes / assumed 50	Rate
Book-Caddy	44/50	0.88
Soup-Cheese	40/50	0.80
Bowl-Drawer	38/50	0.76
Moka-Moka	33/50	0.66
Mug-Mug	31/50	0.62
合计	186/250	0.744

这是 `user-confirmed-remote-final`，不是本地由 250 行 JSONL 复算出的 C3 artifact。

6. RoboCasa actionheadfix：90 秒专项版本

RoboCasa 的接口与 LIBERO 不同：旧 base 是 7D action、两路视觉且无 state/language；target route 使用 reordered 12D action，并加入 task、prompt、16D state 和 right-view。actionheadfix 用 shape-compatible partial load：478 个 target keys 中 446 matched，7D→12D action encoder/decoder 的 8 个 key 重置，其余新增条件模块按 missing-init 处理。 30-epoch run 在 8 卡上完成 3930 steps，打印 total/action/raw-action loss 从 `3.3257/2.4300/0.9986` 降到 `0.25385/0.07689/0.03043`，但 `val_ratio=0`。同 checkpoint 和 normalizer 的八组 54-scenario rollout 都没有 env error，full h1/h2/h4/h5/h8 只有 `2/1/0/1/1` success，no-prompt/no-right/no-state h5 为 `2/2/1`；成功只集中在 Kettle 或 Microwave。h4 虽有 `45/54` robot-env contact，仍是 `0/54` task success。 我把 execution error 与策略失败分开：native direct `0/6`、env errors 0，是有效但失败的 rollout；strict OpenPI-style `convert_action` `0/6`、env errors 6，全部在 step 0 因 `expected 12, got 5` 退出，不能计成六条模型失败。安全结论是“接口错误被定位，训练 fit 到闭环成功仍未闭合”，不是“actionheadfix 修复成功”。

7. Run ledger：哪些结果能放在同一张表

Run / 阶段	Checkpoint / schedule	Protocol	Denominator	结果	可说什么
Book-Caddy early	task-specific 6000-step `models.pt`	reset-style parallel rollout	report 50	`42/50=0.84`	单任务 report-supported；不等于五任务结果
Five-task reset-style	Book 6000 + remaining4 aligned task checkpoints	`env.seed+reset`	user-confirmed 250	`232/250=0.928`	protocol diagnostic，不是 uplift

Run / 阶段	Checkpoint / schedule	Protocol	Denominator	结果	可说什么
Five-task fixed-init	同一 launcher 的 task-specific checkpoint mapping	`pruned_init` + 5 warmup + horizon 600	user-confirmed assumed 50/task	`186/250=0.744`	项目 fixed-init summary；raw bundle缺
Paper Table II	upstream recipe / 3 seeds	OOD init + object shift + background changes	paper protocol	约 `0.79`	只作上游上下文，不与本地直接减法归因
RoboCasa actionheadfix	remote 30-epoch / 3930-step run	18 tasks×3 scenarios，multiple horizons/inputs	raw 54 per setting	`0-2/54`	train→closed-loop failure，不称 benchmark success

8. Evidence Contract

Claim	证据	等级	当前能否强说
五 task、两路 RGB、7D action	`libero_official_common.py`	`local-direct / C1`	可讲 adapter contract
2 obs +16 actions +2 future obs	dataset `seq_len=19` + `process_batch`	`local-direct / C2`	可现场解释 tensor slicing
ResNet/768/12-layer/24-register/16×7/10-step inference	evaluator `build_model`	`local-direct / upstream-instance`	可讲实例化，不说个人架构原创
image encoder `0.05×lr`、DDP、final save	trainer patch	`snapshot-local / C1-C2`	可讲代码路径；个人函数 ownership待确认
task-specific/mixed schedule checkpoints	5-task launcher checkpoint mapping	`local-direct / C2`	必须纠正 multi-task checkpoint 说法
epoch-aligned约2728–2814 steps	bs64 aligned queue arithmetic	`local-direct / C2`	可讲 nominal epoch conversion，不说同优化轨迹
fixed-init/warmup/8-shard/50 rollout	evaluator + parallel launcher	`local-direct / C2`	可讲协议与设计取舍
evaluator不加载normalizer	`build_model` 只读取 `ckpt["model"]`	`local-direct / negative code evidence`	应主动说明；不能说 target normalizer 被固定
`186/250=0.744`	user-confirmed remote JSON	`user-confirmed / raw-missing`	只能带 provenance
`232/250=0.928`	reset-style report context	`protocol-diagnostic`	不能讲模型 uplift
DataLoader p95 `13.862→0.194s`	DataLoader-only benchmark/report	`smoke-bounded / C2`	可讲 wait first-divergence，不报 GPU uplift
Actionheadfix h4 `0/54`、contact `45/54`	ActionHeadFix Po Audit	`remote-snapshot-raw / checkpoint-unverified`	可讲 run-specific failure，不称 benchmark完成
个人 ownership	UWM Ownership Ledger + 用户范围自述	`self-report / role-confirmation-pending`	可说做了 fine-tune/eval/protocol alignment；逐函数仍保守

9. 关键问题三档回答

9.1 “0.744 能写成我复现了官方 UWM 吗？”

危险回答：可以，我严格复现了官方 UWM，成功率 74.4%，只比官方低 4.6pp。

可过回答：这是远端 fixed-init 结果，但与论文还有一些设置差异。

强回答：

> 我不会写成严格复现。`186/250` 是用户确认的项目 fixed-pruned-init summary，本地缺五个 target checkpoint、250 条 episode、scorer 和完整 manifest；论文 Table II 使用 OOD init、object shift、背景调整和多个 finetune seeds。它可以证明我做过协议对齐和结果 reconciliation，但当前不是与论文完全同口径的 C3 benchmark。

9.2 “0.928 更高，为什么不报它？”

危险回答：因为我们的模型比官方更强，超过了 13.8pp。

可过回答：reset-style 更容易，所以不作为正式结果。

强回答：

> `0.928` 和 `0.744` 的模型权重映射大体来自同一 task-specific run family，但 init distribution 明确不同，旧 run 的 exact horizon/raw manifest 也没有完整同步。它首先是 protocol sensitivity diagnostic。只有同 checkpoint、同 horizon、同 seed/init mapping、同 scorer 的 paired episodes，才能把差异归因到 reset protocol；更不能用它和论文 0.79 做模型 uplift。

9.3 “你到底有没有用 normalizer？”

危险回答：用了，所有 checkpoint 都绑定了 action normalizer。

可过回答：checkpoint里可能有，但现在缺 hash。

强回答：

> 对 LIBERO 这条 evaluator，我从代码只能确认 `torch.load` 后 strict-load `ckpt["model"]`，没有读取 `action_normalizer`；本地 base `libero_90.pt` 的 normalizer 字段也是 `None`。因此我会把“normalizer presence/usage”登记为协议字段，但不会说这条 rollout 加载了 target normalizer。RoboCasa route 有独立 normalizer 证据，不能和 LIBERO 混用。

9.4 “五个任务是不是一个 multi-task checkpoint？”

危险回答：是，我训练了一个覆盖五任务的 UWM 模型。

可过回答：五个任务分别训练，但一起评估。

强回答：

> 是 task-specific fine-tune。launcher 对 Book-Caddy 映射独立 6000-step checkpoint，对其余四个任务映射各自的 bs64 epoch-aligned checkpoint。五任务只在结果聚合层汇总，模型权重不是一个共享 multi-task binary。

9.5 “186/250 的分母可靠吗？”

危险回答：脚本配置50条，所以一定是250条。

可过回答：用户确认每个任务50条，但本地没有raw。

强回答：

> 当前数字依赖用户确认的五个 50-rollout summary。代码层 aggregate 用 `num_rollouts_completed` 作分母，虽然保存 expected=50，但没有 fail-closed assert；如果 shard 缺失，仍可能生成缩小分母的 summary。所以升级到 C3 前必须拿到五个 task summary、250 行 episode JSONL，并校验 seed/init 0–49 无重缺。

9.6 “DataLoader stall 已经解决了吗？”

危险回答：解决了，训练速度提升了七十多倍。

可过回答：DataLoader-only wait 明显下降，但完整训练还需验证。

强回答：

> 我定位了 DataLoader wait 的 first divergence：finite 和 infinite-only 的 p95 都在约 13 秒，preload+infinite 降到 0.194 秒，说明主要矛盾是 Zarr 随机读取而不是单纯 epoch iterator 重建。但没有 matched GPU samples/s、显存、loss parity 和长期稳定性，因此只能说数据等待 smoke 改善，不能报端到端训练 uplift。

10. PROJ-UWM-002：哪些部分能说是我做的

10.1 30 秒 ownership 版本

> 用户范围自述已经确认我做过 UWM/LIBERO fine-tune、eval，以及 UWM/RoboCasa 的协议对齐和训练评估。因此我可以说“我开展、运行、对齐、审计并定位了这些链路”。但 `0341c50` 是一次性 recovery snapshot，Git author 不能证明每个函数独立编写；UWM shared diffusion 属于 upstream，`186/250` 仍是 user-confirmed summary。现场我会落到一个具体 launcher/evaluator 解释输入、协议分支、artifact 和限制，不把整仓库认领为原创。

10.2 Role matrix

层	当前可核事实	安全第一人称	禁止自动升级
UWM architecture	upstream shared action/video diffusion	“我基于 UWM 做适配”	“我设计了 UWM”
LIBERO fine-tune	用户自述 + local launcher/patch	“我开展/运行了 task-specific fine-tune”	“所有 trainer 函数由我独立编写”

层	当前可核事实	安全第一人称	禁止自动升级
eval protocol	用户自述 + reset/fixed code	“我对齐并审计了评估协议”	“0.744 是本地 raw verified benchmark”
DataLoader	local benchmark/patch	“我定位了 data-wait first divergence”	“我证明了完整训练吞吐提升”
RoboCasa	用户自述 + raw run audit	“我尝试迁移并做失败诊断”	“RoboCasa benchmark 已完成”
result operation	remote path + user-confirmed JSON	“我记录并核对了结果边界”	“我拥有 checkpoint/operator/reviewer 全链路”
Codex/interview pages	当前 `cv` 仓库产物	“我用材料复盘口述”	把生成页算作原项目工程贡献

10.3 Ownership 追问树

```
你具体做了什么?
  ├ 训练: 指出一个 launcher/config override, 解释输入、checkpoint 和输出。
  ├ 评估: 解释 reset_to_state、warmup、8-shard、success 与 denominator。
  ├ 数据: 解释 seq_len=19、preload/infinite A/B 和验证范围。
  ├ 结果: 区分 user-confirmed、local-direct、remote-snapshot-raw。
  └ 边界: 说明 upstream、协作/工具、远端 operator 和个人动作。
```

最小表达合同：先说“我做过的范围”，再说“实现里能证明的机制”，最后说“不能从快照证明的 ownership”。

11. P9 面试就绪度

11.1 Role Fit 与 JD Mapping

VLA 岗能力	项目证据	强度	面试策略
多模态/action接口	两路RGB、2-frame obs、16×7 action、future obs	强	现场画 tensor timeline
分布式训练迁移	8卡DDP、lr分组、epoch-aligned steps、final checkpoint	中强	解释为何不能照搬20k steps
Closed-loop eval	reset/fixed init、warmup、shard、success predicate	强	先报协议tuple再报数字
数据工程	HDF5→Zarr、preload/infinite benchmark	中强	只报DataLoader-only范围
跨benchmark迁移	LIBERO 7D→RoboCasa 12D/state/prompt/view	中	以失败链展示诊断能力
Ownership/结果可信度	self-report+snapshot+user-confirmed/raw audit	中弱	主动降级强结果，绑定证据等级

11.2 Truth Boundary

- 做过范围：UWM/LIBERO fine-tune 与 eval；UWM/RoboCasa 尝试；协议对齐与训练评估。
- 本地实现直接支持：task/data/model interface、trainer/evaluator/launcher、stall smoke、RoboCasa raw sweep。
- 只能 **user-confirmed**：LIBERO fixed-init `186/250` 及每任务结果。
- 不能写：UWM architecture 个人原创、一个共享五任务 checkpoint、LIBERO evaluator 已加载 normalizer、Table II 严格复现、RoboCasa 性能修复完成。

11.3 Production Gap

缺口	面试官会问	最小补件
Target checkpoints	五个 `models.pt` 是什么hash?	task→checkpoint SHA、step、fresh-load
Denominator	真有250条吗?	merged JSONL + expected/completed assertion
Protocol parity	reset/fixed是否只差init?	same-checkpoint paired run + exact horizon/env revision
Normalizer	哪条路径用了哪个统计?	presence/absence manifest；不得假设
GPU throughput	preload是否端到端有效?	30–100 step matched GPU A/B
Failure taxonomy	soup/mug为何低?	video/action/state/terminal trace
Ownership	谁写、谁跑、谁review?	per-file role + shell/run/review record

11.4 SP/SSP 证据就绪度

当前材料就绪度为 `20.6/30`：岗位相关性 `4.5`、机制深度 `4.4`、工程闭环 `3.8`、指标可信度 `2.6`、ownership `2.1`、抗追问 `3.2`。这不是候选人能力分；提升来自实现合同更清晰，checkpoint/raw/operator 缺口仍是一票否决项。

11.5 补强计划与简历三档

时间	动作	状态
半天	口述一个 task 的 data→train→eval；确认具体亲手/协作/工具角色	可补做
1天	为 aggregator 加 completed==expected、seed/init uniqueness gate；整理现有 raw	可补做
3天	同 checkpoint paired reset/fixed eval；冻结env/scorer/horizon	可补做
1周	获取五checkpoint与250 episodes；做GPU stall matched A/B和failure taxonomy	可补做

- **当前保守版**：开展 UWM/LIBERO task-specific fine-tune 与闭环评估，对齐 fixed-init/reset 协议，定位 Zarr/DataLoader wait，并分析 RoboCasa 跨接口训练—闭环失败。
- **补 ownership 后标准版**：负责指定 launcher/evaluator/loader 适配与协议验收，以文件、命令和 artifact 说明个人边界。
- **C3 后偏强版**：只有补齐 checkpoint、episode、scorer、paired eval 与 GPU A/B 后，才能写可复算结果和端到端工程收益。

12. 练习边界

- 这是从实现反推的 model answer，不是候选人的真实 transcript。
- `PROJ-UWM-001..004` 继续保持 `unseen`；不创建 `coaching_state.md`。
- 真实口述时先答一题，不把本页四题自动计为已练习。
- `0.744` 标记 `user-confirmed-remote-final`；RoboCasa actionheadfix 标记 `remote-snapshot-raw / checkpoint-unverified`。
- 任何“我修改了某函数”的表述仍需本人按 ownership ledger 确认。

13. Implementation Anchors 与 References

13.1 现场可解释的 anchors

Anchor	Locator	能证明	不能证明
A1	`scripts/libero_official_common.py:17-47`	五task与HDF5映射	选择者个人ownership
A2	`scripts/libero_official_common.py:41-48`	两路RGB与7D action	所有UWM数据schema
A3	`scripts/prepare_libero_buffers.py:61-99`	50 episode检查、seq_len19、flip_rgb、Zarr	训练实际完成
A4	`repo_patch/experiments/uwm/train.py:20-31`	2 obs/16 action/2 future切分	上游模型原创
A5	`scripts/uwm_libero_rollout_eval.py:35-70`	模型接口与diffusion实例化	本地设计了UWM architecture
A6	`repo_patch/experiments/uwm/train_robomimic.py:102-141`	pretrain strict-load与0.05×encoder lr	远端effective config完整一致
A7	`scripts/remote_muxi_libero_remaining4_bs64_aligned_queue.sh:46-99`	epoch-aligned换算	完全等价官方优化轨迹
A8	`scripts/remote_muxi_libero_rollout_eval_5tasks_parallel.sh:10-17`	mixed task-specific checkpoint与strict defaults	checkpoint binary存在于本地
A9	`scripts/uwm_libero_rollout_eval.py:113-150`	reset/fixed分支、warmup、sample、success	论文Table II同协议
A10	`scripts/uwm_libero_rollout_eval.py:169-195`	checkpoint/data prefix与episode summary	250条raw已同步
A11	`scripts/remote_muxi_libero_rollout_eval_5tasks_parallel.sh:139-165`	8-shard episode分配	shard全部成功
A12	`scripts/remote_muxi_libero_rollout_eval_5tasks_parallel.sh:217-262`	completed denominator与global aggregation	expected==completed fail-closed
A13	`repo_patch/datasets/robomimic/dataset.py:153-172`	optional Zarr preload	GPU吞吐提升
A14	`repo_patch/datasets/utils/loader.py:23-62`	infinite distributed sampler	唯一root cause
A15	`scripts/benchmark_libero_dataloader_stall.py:64-139`	DataLoader-only timing schema	full training parity

13.2 References

1. UWM/LIBERO Claim–Evidence Inventory。
2. UWM/LIBERO P0 Bundle Audit。
3. UWM Ownership Ledger。
4. UWM Base Checkpoint Immutable Gate。
5. VOL 04 Benchmark Evidence Audit。
6. UWM Video/Action Source Audit。
7. UWM/LIBERO/RoboCasa Transfer Evidence Freeze。
8. UWM RoboCasa ActionHeadFix 30-Epoch P0 Audit。

9. PROJ-UWM-001..004 Atomic Cards。

Part 17 / 49 · `interview_materials/vla/questions/proj_uwm.html` · SHA-256 `0348CDFD78C0276BDBCA4DD9D021FD85E05355E137C09A72BFF217C1BC99F2C6`

返回本卷目录

Canonical Markdown SHA-256: `0348cdfd78c0276bdbca4dd9d021fd85e05355e137c09a72bff217c1bc99f2c6`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_uwm.md`

VOL 07 UWM / LIBERO 项目原子题卡（Protocol Boundary Batch 1 + Ownership）

状态: `drafted / cards_4_of_4 / WARN` 更新日期: 2026-07-21 当前题数: 4 / 4 覆盖范围: `PROJ-UWM-001..004` 难度: L3 4 题 优先级: P0 4 题 练习状态: `unseen` 前置证据: UWM / LIBERO Claim–Evidence Inventory; 协议来源见 VOL 04 benchmark evidence audit; RoboCasa 训练/0-success 边界见 UWM/LIBERO/RoboCasa transfer evidence freeze。

评估协议冲突如何安全回答

▼ `L3` `P0` **PROJ-UWM-001 · UWM/LIBERO 中 0.744、0.928 和论文约 0.79 三个结果如何解释，为什么不能直接比较?**

面试意图: 判断你能否固定评估 tuple、区分本地证据与远端确认结果，并在结果冲突时避免把协议差异包装成模型 uplift 或官方复现。

30 秒回答: 先冻结 tuple: 0.744 是 fixed-init 用户确认 summary，0.928 是 reset-style，论文约0.79 是 OOD/shift/多seed; 三者不可横比。五 task 用 task-specific checkpoint，evaluator 只读 model，raw episode 仍缺。

机制展开（约 2 分钟）: 先把 init/seed/warmup/horizon/normalizer presence/checkpoint、predicate、expected/completed denominator 写入 manifest。186/250 是 fixed-pruned-init 的 5×50 结果，缺 raw JSONL/权重; 0.928 用 env.seed+reset，协议更容易; 论文约0.79 用 OOD init、shift、去背景、3 seeds。三者不可横比，0.928→0.744 只说明 protocol sensitivity。要归因模型，需同 checkpoint paired init、episode trace、scorer、ownership; 只能讲 report/user-confirmed evidence，不称 benchmark。

诊断 / 证据: 逐 episode 保存 task、init index、seed、warmup、horizon、camera/state/action schema、normalizer presence/usage、checkpoint hash、success predicate、terminal reason 和 expected/completed denominator。先做同 checkpoint 的 reset-style↔fixed-pruned-init paired run，再按 task/失败类型分层; 若要对齐论文，还要复现 OOD initialization、object shift、背景设置与多 seed，并冻结 scorer 版本。当前本地有 launcher、task mapping、协议开关、report 和用户确认 summary，但缺 250 条 raw episode、task-specific checkpoint bundle、normalizer usage manifest 与 ownership ledger，保持 `WARN`。

项目挂钩: 可讲我负责把 LIBERO-90→LIBERO-10 的 task-specific UWM finetune/rollout 路径、pruned-init、warmup、rollout 数和 success predicate 固定下来，并定位 zarr 小 chunk 导致的 DataLoader stall。要主动说明 LIBERO evaluator 只 strict-load `ckpt["model"]`，没有把 normalizer 当作已加载证据; RoboCasa 可讲 actionheadfix 的 partial decoder init、12D contract 与 raw sweep: 一条30-epoch run的full-h4为 `0/54`、robot-env contact `45/54`，其它setting也只有 `1-2/54`，所以只称训练/诊断推进，不称性能修复。历史 `27/54` 来自未对齐run摘要，不能混用。安全口径是"我负责协议迁移、诊断和验收"，不把 UWM shared diffusion architecture、远端 checkpoint 或结果写成个人原创/严格复现。

追问树: 为什么 reset-style 更高? `0.744 vs 0.79` 能否说只差 4.6pp? 如何证明 checkpoint 相同? DataLoader stall 如何定位? actionheadfix 为什么 loss 改善仍 `0/54`? teacher-forced x0 MSE 与 closed-loop success 差在哪里? strict `convert_action` 的 `0/6` 为什么不能算六条模型失败?

易错点: 混用 strict/fixed/reset 口径; 把用户确认 JSON 当本地 raw artifact; 把五个 task 说成一个 multi-task checkpoint; 把 evaluator 未读取的 normalizer 说成已固定; 把 Table II 论文均值当同协议 baseline; 忽略 init、seed、warmup、horizon、success predicate 或 expected/completed denominator; 把 DataLoader smoke 写成完整训练吞吐提升。

危险答案: 说"0.928 证明模型最好""0.744 只比官方低 4.6pp""loss 从 95 降到 0.22 所以 actionheadfix 已成功""strict convert_action 0/6 都是模型失败""Git author 证明全部代码是我写的"或"RoboCasa 已完成 benchmark"。

来源: [1][2][3][4] 练习状态: unseen

Ownership 与结果归属

▼ `L3` `P0` **PROJ-UWM-002 · UWM/LIBERO 哪些部分能说是你做的？如何区分上游、Git 快照、远端结果和个人 ownership？**

面试意图：判断你能否把迁移、协议、诊断、运行和结果归属拆开，而不是把一次性 snapshot 或用户确认 summary 认领为全部个人工作。

30 秒回答：我把贡献拆成训练迁移、评估协议、数据管线和失败诊断。源码/patch/report 可核验，但 Git 提交不等于逐文件原创；UWM architecture 属于上游，`186/250` 是用户确认的远端 summary，checkpoint/normalizer/episodes 未闭合，ownership 需按 ledger 保守表述。

机制展开（约 2 分钟）：`0341c50` 一次加入 official repro，`a37a386/2a2912d/1a03fe9/8eaddf2` 记录 actionheadfix、single-task gate、h4 sharding 与 aux 参数修复；可讲 evaluator、launcher、loader patch 和训练诊断的文件事实，但 commit 仍不等于逐文件原创。再固定 fixed-pruned-init、warmup、normalizer、horizon、scorer，不能混入 reset-style。最后说明 recovery manifest 有未提交修改，strict JSON 是 user-confirmed remote final，不是本地 raw verified；现场解释一个函数并划清亲手、协作、工具和运行边界。

诊断 / 证据：UWM Ownership Ledger 的 13 行矩阵；检查 Git path history、blob/SHA、工作树 status、sync manifest、user-confirmed JSON 和 upstream audit。未口述前保持 `candidate_confirmation_pending / unseen`，不创建 coaching state。

项目挂钩：可以可信地讲协议迁移、strict/reset reconciliation、zarr/DataLoader first failure 和 RoboCasa contract debugging，同时明确不把 UWM shared diffusion、远端 checkpoint 或 0.744 写成个人原创/严格复现。

追问树：哪个函数是你亲手改的？谁运行 remote？actionheadfix 的 partial init/x0 metric/h4 launcher 各由谁设计、实现和验收？0.744 的 checkpoint/normalizer/episodes 在哪？preload 是否提升 GPU throughput？RoboCasa 0/54 的 raw trace 在哪？

危险答案：说“Git author 证明全部 patch 是我写的”“0.744 是我的本地 benchmark”“UWM architecture 是我设计的”或“有 summary 就等于有 checkpoint”。

来源：[1][5][6] 练习状态：unseen

数据管线停顿与证据边界

▼ `L3` `P0` **PROJ-UWM-003 · zarr DataLoader 的周期性长停是怎么定位的？为什么 preload+infinite smoke 不能直接写成训练吞吐提升？**

面试意图：判断你能否把数据等待、GPU 计算、DDP 同步拆开，用最小 A/B 找 first divergence，并区分 DataLoader-only 诊断与真实 GPU 训练收益。

30 秒回答：拆开 `data_wait/train/total` 后，DataLoader-only A/B 的 `bs64 finite` p95 wait 从 `13.862s` 降到 preload+infinite 的 `0.194s`，支持 zarr 小 chunk 是主要等待来源；没有 GPU 吞吐、显存和 loss parity，不能称端到端提速。

机制展开（约 2 分钟）：用 $T_{step} \approx \max(T_{GPU}, T_{data}) + T_{sync}$ 分层，固定数据和加载配置，记录 `data_wait_sec/train_sec/total_sec`。`bs64` finite 复现 5 次超过 10 秒的长停；只开 infinite sampler 仍约 `13.183s`，而 preload zarr 后降到 `0.194s`，说明主要等待在随机 chunk 读取。下一步做 30–100 step matched GPU A/B，比较 samples/sec、显存、三段耗时、loss 和 checkpoint，确认没有改变数据顺序或目标。

诊断 / 证据：local report、`benchmark_libero_dataloader_stall.py` 和 dataset/loader patch 给出四组 smoke 数字与开关：`UWM_INFINITE_TRAIN_SAMPLER`、`UWM_ROBOMIMIC_PRELOAD`、workers/prefetch/pin、`UWM_STEP_TIMING_EVERY`。现有证据不包含正式 GPU matched A/B、长期吞吐、显存曲线或收敛 parity，因此根因表述应是“证据支持的主要等待来源”，不是已证明唯一 root cause。

项目挂钩：可讲 UWM LIBERO-90→LIBERO-10 迁移中，训练出现周期性停顿时如何先隔离数据管线，再决定是否预载；这与 0.744/0.928 评估协议问题是两条证据链，不能用 DataLoader smoke 补出 benchmark uplift，也不能把 patch 或 Git author 自动升级为个人独立 ownership。

追问树：为什么 infinite sampler 没解决长停？preload 的内存代价如何测？如何保证 A/B 数据顺序与 normalizer 一致？`T_data` 与 DDP `T_sync` 怎么区分？为什么不能用 `median sec/it` 证明吞吐提升？下一步 GPU smoke 的最小样本量和验收字段是什么？

易错点：把 `p95 wait` 当 samples/sec；把 DataLoader-only 结果当 GPU 训练结果；只跑 preload 没有 baseline；忽略 zarr chunk、epoch 边界、workers/prefetch 和 batch 改变；把数据等待改善写成模型 loss 或 closed-loop success 提升。

危险答案：说“preload 后训练吞吐提升已证明”“infinite sampler 解决了根因”“13.862→0.194 就是端到端加速”“DataLoader smoke 证明模型收敛更好”或“这个 patch 一定是我独立原创”。

来源：[1][4][6] 练习状态：unseen

训练拟合、Open-loop 与 Closed-loop 的断层

▼ `L3` `P0` **PROJ-UWM-004 · actionheadfix 的 loss 和 teacher-forced x0 都改善了，为什么 RoboCasa h4 仍是 0/54？你如何定位 first divergence?**

面试意图：判断你能否把优化器收敛、open-loop action fit、execution validity、关键目标接触和任务成功分层，而不是用低 loss 宣称闭环修复。

30 秒回答：训练 loss 从 `2.4300` 降到 `0.07689`，但无 validation；同 checkpoint/normalizer 下 full h4 仍 `0/54`，仅 `0-2/54` 成功，env errors=0且 `45/54` 有contact；first divergence 在 fit 到闭环之间，不能称修复。

机制展开（约 2 分钟）：base-load preflight是478 target keys中446 matched，7D→12D action encoder/decoder的8个key重置，task/prompt/state/right-view等新增模块保持missing-init；训练用12D reordered action、16D state和三视角。八份raw JSON共享同一54-scenario集合、checkpoint和normalizer，full h1/h2/h4/h5/h8为 `2/1/0/1/1` success，no-prompt/no-right/no-state h5为 `2/2/1`。差值被一两个Kettle/Microwave episode主导，不能做ablation因果；普通接触也不等于命中任务谓词。

诊断 / 证据：ActionHeadFix Po Audit 独立解析8×54 records：每task 3 trials、scenario/seed set完全相同、success field与record recount一致、env errors全0。full-h4 `45/54` 有robot-env contact但 `0/54` success；checkpoint只剩remote `models.pt` path，本地有normalizer hash但没有binary/fresh-load。历史30/100 epoch、strict/direct A/B与Human300属于相邻run family，只作旁证不混分母。

项目挂钩：可讲从 LIBERO 7D action 迁到 RoboCasa native 12D 时，如何做 partial init、分组 x0 gate、execution bridge 和 contact taxonomy；不可把 UWM 架构、低训练 loss、任意接触或 GT bridge parity 说成个人原创、闭环成功或唯一 root cause。

追问树：为什么没有validation仍能说训练完成？teacher-forced与rollout分布差在哪里？ `45/54` contact为什么不等于成功？为什么432 records不是432个独立benchmark trial？no-prompt `2/54` 能否说prompt有害？checkpoint path为什么不等于binary验证？single-task overfit gate应验收什么？

易错点：把训练 loss、x0 MSE、任意 contact、valid rollout 和 success 混成一个指标；把 step-0 env error 计为模型失败；只比较 epoch，不固定 checkpoint、normalizer、horizon、aux 与 init；从 GT bridge parity 推导模型 action 正确。

危险答案：说“loss降到0.0769所以修复成功”“0/54只是环境太难”“45/54接触率证明策略有效”“no-prompt多一个success证明prompt有害”“432 records可以合并成总成功率”或“远端models.pt path等于本地checkpoint已验证”。

来源：[1][3][6][7]　练习状态：unseen

参考来源

1. UWM / LIBERO Claim–Evidence Inventory：claim 分层、strict/reset 协议、数据管线诊断与 ownership 边界。
2. VOL 04 benchmark evidence audit：LIBERO 默认、UWM Table II OOD 与项目 fixed-init 的不可比性。
3. UWM video/action claim-to-code audit：shared diffusion、action/video target、normalizer 与上游/本地边界。
4. UWM/LIBERO project page：项目叙事、报告结果和评估协议上下文。
5. UWM Ownership Ledger：Git history、13 行 ownership matrix、工作树与 user-confirmed result boundary。
6. UWM/LIBERO/RoboCasa Transfer Evidence Freeze：actionheadfix、Human300/target/x0 指标、30/100 epoch、h4 `0/54` 与 convert_action A/B。
7. UWM RoboCasa ActionHeadFix 30-Epoch Po Audit：8卡3930-step训练、12D normalizer、8×54 raw rollouts、full-h4 `0/54` /contact `45/54` 与checkpoint/ownership边界。

Part 18 / 49 · `interview_materials/vla/evidence/projects/uwm_libero_claim_evidence_inventory.html` · SHA-256 `48616E5E88E87AA70D196983021C7D627AB6FE3B4ABFD8FAFF3AAAE436B7494D`

返回本卷目录

Canonical Markdown SHA-256：`48616e5e88e87aa70d196983021c7d627ab6fe3b4abfd8faff3aaae436b7494d`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\uwm_libero_claim_evidence_inventory.md`

UWM / LIBERO Claim–Evidence Inventory（UWM-01）

状态：`drafted / local_direct_plus_user_confirmed_final / ownership_audited / WARN` 审查日期：2026-07-21 证据截止：2026-07-21 项目范围：UWM unified world model、LIBERO-90 → LIBERO-10 finetune、fixed-pruned-init rollout、reset-style 对照、DataLoader/zarr stall debugging、RoboCasa 转向边界 项目仓库：`C:\Users\mousongzhe\Project`，当前 HEAD：`6c524d9c6c6226075f100cdac395fef2fdc37671` 远端主机：`root@100.77.83.126`；当前本地证据包含一条 RoboCasa actionheadfix 30-epoch run 的八份 54-scenario raw rollout JSON，以及 project copy 中可验证的 UWM base `libero_90.pt` binary；仍不包含 produced target checkpoint/hash 或 LIBERO 250-episode raw bundle

结论先行

本轮审计可以把 UWM/LIBERO 的训练入口、动作/观测形状、评估脚本、协议开关、数据管线诊断和 RoboCasa 转向记录提升到 `local-direct` 或 `report-supported`。本地 project snapshot 的 commit `0341c50` 添加了 LIBERO official repro recovery artifacts：5 个 LIBERO-10 task 映射、LIBERO-90 checkpoint 入口、8 卡训练/评估 launcher、`pruned_init` helper、5-step zero-action warmup、50 rollouts/task、episode summary 聚合和 zarr DataLoader smoke。UWM/LIBERO/RoboCasa transfer evidence freeze 冻结了 actionheadfix、30/100 epoch、convert_action A/B 和 provenance；新增 ActionHeadFix 30-Epoch Train-to-Closed-Loop Po Audit 又从 remote snapshot 独立重算了同一 30-epoch run 的 8×54 raw rollouts：训练进程完成，但 full h1/h2/h4/h5/h8 仅 `2/54、1/54、0/54、1/54、1/54`，三组 h5 input ablation 为 `2/54、2/54、1/54`，全部 env errors 为 0。UWM 的共享 diffusion transformer、action/video 两类输出与 LIBERO 公开 recipe仍由 VOL 04 source audits 分层核对。

关键结果必须按协议分开：

- Book-Caddy 单任务 `42/50=0.84` 是 project report 记录的单任务结果，不是完整 LIBERO benchmark；本地没有对应 checkpoint/raw episode bundle。
- 5-task `186/250=0.744` 来自用户在远端完成后确认的 JSON，本地只保存了 user-confirmed summary；它对应项目的 fixed-pruned-init / 5-step warmup / 50-task rollout 口径。
- 旧的 reset-style `232/250=0.928` 使用 `env.seed(e)+env.reset()`，只能作为 protocol-sensitivity diagnostic，不能作为正式 benchmark 成绩。
- UWM Table II 的论文均值约 `0.79` 使用 OOD initializations、+0.03 object-init shift、移除背景物体和 3 个 finetune seeds；现有 source audit 判定本地 fixed-pruned-init 结果与 Table II 不可直接比较。因此不能把“74.4% 只低官方 4.6pp”当作严格复现结论，只能作为同任务名下的方向性上下文。

当前 verdict 保持 `WARN`：LIBERO 最终 5-task 数字仍不是本地 raw artifact；RoboCasa actionheadfix 虽有八份完整 denominator 的 raw JSON，且 base `libero_90.pt` 已有独立 SHA，但 produced target checkpoint/hash、base hash 与旧 load report 的 shared run identity、fresh reload、immutable env/scorer revision 和 episode videos 仍缺。UWM ownership ledger 已建立，但逐文件候选人角色、远端运行者和工具/协作边界仍 pending。RoboCasa-native 方向能证明训练—闭环失败断层，不等于可靠 benchmark success。项目最可信的面试价值是评估协议控制、训练-闭环链路、数据管线诊断和失败分析，而不是宣称超过官方结果。

1. 证据标签与门槛

标签	含义	UWM 例子
`local-direct`	当前工作区可直接读取源码、配置、脚本、report 或静态 artifact	rollout protocol 开关、task mapping、DataLoader preload/infinite sampler、RoboCasa 8×54 raw rollout JSON、UWM base binary/hash
`upstream-verified`	frozen UWM/LIBERO source audit 对论文/官方 repo 的机制或协议核对	UWM 双模态 diffusion、LIBERO `pruned_init` /wrapper 语义、Table II OOD 设定
`report-supported`	committed report 有结论，但底层远端 raw artifact 未同步	Book-Caddy 42/50、训练 loss、远端 checkpoint path

标签	含义	UWM 例子
`user-confirmed-remote-final`	用户在远端完成后提供的最终 summary，本地保留 JSON，但无法由本地 raw episode 复算	5-task 186/250=0.744
`protocol-limited`	数字存在，但 init、warmup、seed、horizon、normalizer 或 scorer 不同，不能横比	fixed-pruned-init vs UWM Table II vs reset-style
`local-inference`	由代码/失败语义推断，缺 paired experiment	soup/mug 低分与接触/恢复分布缺口的解释
`ownership-partial`	Git author 或设计记录提供参与线索，缺逐文件 ownership ledger	`mousongzhe` commits、用户指令中的路线转向
`unverified`	缺 raw checkpoint、episode、视频、复算命令或无法证明	RoboCasa benchmark success、完整 5-task run lineage

面试回答门槛：C0=理解 UWM/LIBERO 概念；C1=能指向 local script/report；C2=能解释协议取舍、stall failure 和结果冲突；C3=有 checkpoint/normalizer/init/episode/scorer/ownership 的 immutable bundle。当前机制与协议达到 C1/C2，强数字多数是 report-supported 或 user-confirmed，尚未形成 C3。

2. Claim reconciliation

ID	当前 claim	证据	Verdict	面试处理
`UWM-CL-001`	项目主线从 RoboCasa-native 调试转向 UWM 官方 LIBERO reproduction	project report、commit `0341c50`、sync summary	`local-direct / report-supported`	可讲路线转向与问题边界；RoboCasa 不得写成已完成 benchmark
`UWM-CL-002`	UWM LIBERO recipe 从 LIBERO-90 checkpoint finetune 到 5 个 LIBERO-10 task	train launcher、task mapping、report、UWM source audit、base `libero_90.pt` immutable gate	`local-direct / upstream-verified recipe / base-binary-verified`	可讲训练-评估闭环与 base 文件 hash；不能把 base hash 当作 target fine-tune checkpoint 或完整结果证据
`UWM-CL-003`	LIBERO route 使用两路 RGB、action chunk 与 UWM shared diffusion model	rollout script、repo patch、UWM source audit	`local-direct / upstream-verified`	可解释 action/video unified modeling；不要混成 Cosmos/WAM 的 value head
`UWM-CL-004`	评估必须固定 init、warmup、normalizer、checkpoint、horizon、rollout 数和 success predicate	rollout script、source audit、project report	`local-direct / protocol-critical`	用可重放 tuple 回答；任务名相同不代表协议相同
`UWM-CL-005`	Book-Caddy 单任务得到 42/50=0.84	committed report、远端路径记录	`report-supported / single-task-only`	只能说单任务复现实验；不能说完整 LIBERO success
`UWM-CL-006`	5-task fixed-pruned-init 结果为 186/250=0.744	user-confirmed final JSON、sync summary	`user-confirmed-remote-final / protocol-limited`	绑定 fixed-pruned-init/5 warmup/50-task；不能称 Table II strict aligned
`UWM-CL-007`	reset-style 232/250=0.928 高于 strict 0.744	user-confirmed JSON、source audit	`protocol-diagnostic`	只能说明评估分布敏感；不能当模型 uplift 或正式成绩
`UWM-CL-008`	74.4% 与 UWM Table II 0.79 只差 4.6pp	project page/report + benchmark source audit	`NON_COMPARABLE`	说明任务同名但 OOD init、seed、背景和训练口径不同；不写“接近官方/复现官方”
`UWM-CL-009`	soup_cheese 是相对官方 gap 最大任务，moka/mug 绝对成功率较低	user-confirmed per-task table、project report	`report-supported / local-inference`	可提出接触与长时序 recovery 假设；必须说仍需失败视频/trace 验证
`UWM-CL-010`	zarr 小 chunk 随机读造成周期性 DataLoader stall，preload+infinite smoke 后 p95 wait 约 13.86s→0.19s	local benchmark script、report SVG/table、repo patch	`local-direct / smoke-bounded`	可讲数据管线定位；不能承诺 full GPU training throughput 已提升
`UWM-CL-011`	normalizer/action order、task conditioning、eval output reuse 是 RoboCasa 方向的主要风险	project report、action/state/eval debug sections	`local-direct / inference`	讲成排查项和修复方向；RoboCasa success 仍 unverified
`UWM-CL-012`	所有 UWM/RoboCasa 实现与结果均为个人独立完成	Git author、项目页；已有 ownership ledger，但候选人逐项确认未完成	`ownership-partial / unverified`	说“我负责协议、迁移、诊断和验收的部分”；不把上游 UWM 结构或 Codex/协作实现写成个人原创
`UWM-CL-013`	actionheadfix 30-epoch run 完成训练，但同 checkpoint 的八组 54-scenario rollout 只有 0–2 success；full-h4 `0/54` 且 contact `45/54`	ActionHeadFix P0 audit + 8 raw JSON	`remote-snapshot-raw / checkpoint-unverified`	可讲 train→closed-loop 断层和 run-specific denominator；不能称 RoboCasa benchmark、ablation uplift 或 checkpoint closure

3. Source inventory（冻结输入）

Source	类型	SHA-256	能支持	不能支持
`C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\reports\uwm_robocasa_libero_reproduction_log_report.html`	local report	`F5B268F2E7108DF548309BBA869E0099F46A6F4D5CE8A31742D002570317DB62`	路线转向、UWM/LIBERO task、Book-Caddy 42/50、strict/reset 叙事、stall smoke、RoboCasa 风险	远端 checkpoint/raw episode、完整 5-task 可复算结果
`scripts/libero_official_common.py`	local code	`DE9DB94113554DB50B151900647E6D32BFFB9649C7F4D1D97EA0D65A8A937E2B`	五 task、HDF5/BDDL/pruned-init/buffer path、RGB/action shape metadata	远端文件是否与当前本地版本一致之外的训练结果
`scripts/uwm_libero_rollout_eval.py`	local code	`05E912C2096EFF4BFE8D061E4C334517B03DE80847793D83025A2AA3E18F3943`	`uwm_reset` vs `libero_fixed_init`、5-step warmup、seed/init、checkpoint prefix、success summary	raw remote episodes、target fine-tune binary、Table II OOD environment
`scripts/remote_muxi_libero_rollout_eval_5tasks_parallel.sh`	local launcher	`C61C7E6C5E187A640205D8F6EF609E32FE515AFE4B3687D1F47D8AF17932EE32`	5 task/8 shard/50 rollout/600 step launcher、manifest fields、run-specific eval root	该 launcher 实际远端完成的全部输出
`scripts/remote_muxi_libero_finetune_5tasks_ddp.sh`	local launcher	`E5B6DCB03FD6612B996AB9DFD3227CB2426E0A147F21FACEE383BBF56EC29D92`	LIBERO-90 checkpoint、5 task、DDP、batch/step、buffer path、checkpoint output contract	远端 checkpoint contents、loss/episode raw log
`scripts/benchmark_libero_dataloader_stall.py`	local code	`47A2A545E5FC6B0D6E540839BDD02D7C4C054B8B5A9D36D90B05B356C36B6B36`	finite/infinite/preload DataLoader-only benchmark schema	full GPU training speedup
`repo_patch/experiments/uwm/train_robomimic.py`	local patch	`842302703CB09A75A34611CD6B289D18C0268B710BCC2C0F9D0138192E2640BD`	pretrain load、num_steps、rollout hooks、step timing	upstream patch deployment state beyond sync report
`repo_patch/datasets/utils/loader.py`	local patch	`FC8E43D8D852794157B490ED7E00A68978803D5C9FE30CF6569CD640423E3551`	infinite sampler、workers/prefetch/pin controls	formal training throughput result
`repo_patch/datasets/robomimic/dataset.py`	local patch	`002065D77A59C81E9CD84445EA58AE3939FDA84681B7CCC79E38F182FF8B5B0E`	optional zarr preload and data slicing	memory cost/speed at production scale
`report/uwm_libero_strict_eval_recovery_20260702/sync_manifest.json`	local sync manifest	`9077C45992BF6852F5A8F4FA28ED3735FA590817D227A62D4179CCF239358D5F`	last remote state, protocol settings, script hashes, upstream revisions, caveat that final is superseded	final raw remote summary; partial run is not final
`report/uwm_libero_strict_eval_recovery_20260702/sync_summary.md`	local recovery report	`103E2E1EEA40DE384941081C78050E922D3914E54435A819DD921F54A68C1AB4`	user-confirmed final context, strict/reset distinction, recovery boundary	local independent reproduction
`report/.../final_strict_eval_user_confirmed_20260705.json`	user-confirmed remote final	`7616111AE7C7CA35CB10E8788ADCB1E8837D9B6F656B49553B6C7AFCCD586D11`	per-task 0.88/0.80/0.76/0.66/0.62, 186/250, reset comparison and interpretation	raw episode IDs/videos/checkpoint/normalizer hash
`interview_materials/vla/sources/vol_04_benchmark_evidence_claim_to_protocol.md`	source audit	`58A5636E953F44C2AA27E3ACF65C621460186DAF3732B916EAC70E6FFAB2DE6A`	LIBERO defaults、UWM Table II OOD、repo reset、project fixed-init 的不可比性	完整 paper source tar 或本地 user checkpoint
`interview_materials/vla/sources/vol_04_uwm_video_action_claim_to_code.md`	source audit	`B031B1BBF8326CBFD03FFB21AC69DB070FAC2CBFF723A16F3545D378E1354B60`	UWM shared diffusion、独立 timestep、action shape/normalizer、upstream/local boundary	本地 UWM run success、mask ablation、checkpoint contents
`interview_materials/05_uwm_libero_eval_protocol.html`	project page	`89C9958EE0129C99D0CD77867E0E337E53FCF787BB62CC55EDCE9BB6BB93BF80`	resume/project narrative、74.4/92.8 claims and interview framing	independently verified final run
`evidence/projects/uwm_robocasa_actionheadfix_30epoch_train_eval_p0_audit.md`	canonical Po audit	`41B9CDDDB95163A369719E007EA11AD3A7776E79F0901F80DCACD34FAA83F117`	3930-step completion、18-task×3-scenario paired sweep、8 raw JSON recount、contact/success boundary	target checkpoint binary/hash（base `libero_90.pt` 已单独冻结）、official benchmark、ablation causality、candidate ownership

4. 训练与模型边界

4.1 UWM/LIBERO route

本地项目报告和 source audit 支持的最小链路是：

```
LIBERO-90 checkpoint
  -> LIBERO-10 HDF5 + compressed Zarr buffer
  -> UWM train_robomimic.py / DDP finetune
  -> task checkpoint + action normalizer
  -> LIBERO environment rollout
  -> binary success summary / episode trace
```

LIBERO route 的本地 task map 是 `book_caddy`、`soup_cheese`、`bowl_drawer`、`moka_moka`、`mug_mug` 五个任务。`libero_official_common.py` 将两路 RGB 设为 `[128,128,3]`，LIBERO action metadata 为 `[7]`；rollout script 的 model builder 设 `action_len=16`、`action_dim=7`。这证明的是当前 LIBERO adapter 的接口合同，不应泛化为所有 UWM 数据集固定 7D/16-step。

UWM 的上游机制由 source audit 记录为共享 diffusion transformer，同时建模 action chunk 与 next-observation latent video，action/video 使用独立 diffusion timestep；它不是 Cosmos/WAM 的 action/world/value 三头。项目个人贡献边界应落在环境/数据/训练脚本迁移、协议固定、诊断和验收，而不是把上游 UWM 结构说成原创。

4.2 RoboCasa 转向边界

原路线尝试 UWM `libero_90.pt` → RoboCasa target finetune，并遇到 official12 action order、state/proprio、task prompt、AdaLN conditioning、normalizer reuse、eval output reuse 等问题。actionheadfix 30-epoch raw audit 已证明一条具体链：8卡3930 steps完成、训练打印 loss下降、同一 remote `models.pt` /normalizer 进入八组54-scenario rollout且env errors为0，但每组仅0–2 success；full-h4 `0/54`、robot-env contact `45/54`。这把问题收窄到 train/open-loop→closed-loop transfer，不等于 benchmark success，也不能把接触当任务完成。

5. 评估协议 reconciliation

口径	init	warmup	rollouts	结果	证据等级	可否与论文 Table II 比
UWM repo reset-style	`env.seed(e)+env.reset()`	无项目 fixed warmup	50/task	232/250=0.928	report/user-confirmed context	否
项目 fixed-pruned-init	LIBERO `.pruned_init`	5 zero-action	50/task	186/250=0.744	user-confirmed remote final	不能直接比
UWM Table II v3	OOD initializations，+0.03 object init shift，移除背景	paper-specific	50 OOD init × 3 seeds	avg≈0.79	upstream-verified source audit	论文内部可比
LIBERO frozen default	fixed pruned states	5 zero7 default contract	default suite-specific	protocol reference	upstream-verified source audit	只与同 config 比

因此，“strict”在本项目内只能作为固定项目 protocol 的名称；它不自动等于 UWM paper Table II strict aligned。`0.928→0.744` 的 `-18.4pp` 主要是协议分布变化的诊断，不能当成模型权重退化；`0.744 vs 0.79` 的 `-4.6pp` 也不能在不同 OOD/init/seed 口径下解释成真实模型差距。

6. 结果与失败分析边界

最终 user-confirmed per-task 表为：`book_caddy=0.88`、`soup_cheese=0.80`、`bowl_drawer=0.76`、`moka_moka=0.66`、`mug_mug=0.62`。可以把它作为项目材料中的“用户确认的 fixed-pruned-init 结果”，但不能升级为本地 raw verified。`soup_cheese` 相对论文列表值差距最大，`moka/mug` 绝对成功率较低；接触精度、对象放置和长时序 recovery 是合理的 failure hypotheses，但当前缺失败视频、action trace、per-episode JSON 和 paired protocol ablation，因此这些仍是 `local-inference`。

推荐的失败分析顺序：

1. 先核对 checkpoint、action normalizer、camera keys、init index、warmup、horizon、success predicate 和 run-specific output root；
2. 再把失败按 grasp/contact、placement、action scale、long-horizon recovery、OOD initialization 分类；
3. 最后才决定补 recovery data、contact-sensitive task 或 world-rollout perturbation。

RoboCasa actionheadfix 的 run-specific raw sweep 必须单列：八个 setting 使用相同的 54 个 scenario/task/seed、同一 checkpoint path 与 normalizer，full h1/h2/h4/h5/h8 为 `2/54、1/54、0/54、1/54、1/54`，no-prompt/no-right/no-state h5 为 `2/54、2/54、1/54`。成功只出现在 TurnOnElectricKettle 或 TurnOnMicrowave；0–2 个 success 的差异不足以证明 prompt、state 或 right-view 的因果效果。

7. DataLoader stall 诊断

项目 report 记录的 DataLoader-only smoke 是一个相对强的 local-direct 工程证据：`bs64 finite workers8 pf2` 的 p95 wait 为 13.862s；只开 infinite sampler 仍为 13.183s；`preload+infinite` 降到 0.194s；`bs128 preload+infinite` 为 0.209s。该结果支持“zarr 小 chunk 随机读是主要数据等待来源”的诊断，但不等于完整 GPU 训练的吞吐/显存提升。代码开关包括 `UWM_INFINITE_TRAIN_SAMPLER`、`UWM_ROBOMIMIC_PRELOAD`、DataLoader workers/prefetch/pin 和 `UWM_STEP_TIMING_EVERY`；正式训练仍需 30–100 step GPU smoke 验证。

8. 个人贡献与 ownership 边界

可安全表述为：参与/负责在 C500/MACA 上迁移 UWM/LIBERO 训练与 rollout 脚本，固定项目级评估 tuple，定位 zarr/DataLoader 周期性停顿，整理 strict/reset 结果和失败分析，并据此调整项目叙事。不能自动表述为：个人原创 UWM architecture、独立完成全部 upstream repo patch、已经验证 5-task raw result、完成 RoboCasa benchmark 或单独证明 74.4% 与 Table II 的可比性。Git commits (`c4469eb`、`0341c50`、`6c524d9`) 只提供提交署名线索；逐文件 personal/collaboration/tool-assisted ledger 仍待补。

9. 面试安全表述

推荐 60 秒版本：

> UWM 这个项目我主要做的是训练到闭环评估的协议工程。我在 C500/MACA 上迁移 LIBERO-90 到多个 LIBERO-10 task 的 UWM finetune/rollout 路径，固定 checkpoint、normalizer、pruned-init、warmup、rollout 数和 success predicate，并排查 zarr 小 chunk 导致的 DataLoader 周期性长停。LIBERO 的 `186/250` 仍只是用户确认 summary，不能与 Table II 直接比较；RoboCasa 则有一条可重算的 actionheadfix 30-epoch raw sweep：8卡训练完成、同 checkpoint 的八组54-scenario rollout都无环境错误，但只有0–2 success，full-h4是 `0/54`。所以我把它定位为 train-to-closed-loop transfer debugging，不宣称完成 benchmark。

高压追问：

- “为什么 0.744 还值得讲？”——因为它是用户确认的固定项目协议结果，能暴露任务级失败和评估差异；我不会把它叫 Table II 复现。
- “0.928 是否说明模型很好？”——不能，random reset 更容易且协议不同，只能说明 protocol sensitivity。
- “你怎么证明 stall 是 zarr？”——DataLoader-only A/B 中 preload+infinite 将 p95 wait 从约 13.86s 降到约 0.19s；full GPU throughput 仍需独立 smoke。
- “哪些是你原创的？”——我按 ownership ledger 区分迁移、协议、诊断和协作实现；UWM shared diffusion architecture 属于上游机制。

10. P0/P1 artifact queue

P0

1. 获取 5-task strict run 的 target checkpoint、normalizer、exact config、init index、250 episode JSONL、失败视频和 run-specific manifest；base `libero_90.pt` 的本地 hash 已单独冻结，但尚未与该 run shared-ID 绑定。
2. 明确项目 fixed-pruned-init protocol 与 UWM Table II OOD protocol 的差异；如要声称 reproduction，必须补 OOD environment patch、3 seeds 和同 scorer。
3. 对 reset-style 与 fixed-pruned-init 做同 checkpoint paired run，记录每 episode seed/init/step/horizon。
4. 逐文件 ownership ledger 已建立；下一步补候选人确认，标注 upstream、个人、协作、工具辅助、只运行和仅读审计。
5. 对 RoboCasa actionheadfix 回传 `models.pt` hash/fresh-load parity、完整 train JSONL、env/scorer revision 与54条 episode video locator；当前 raw denominator 与 base hash 已有，但 target checkpoint和可重放环境仍未闭合。

P1

1. 对 soup/moka/mug 失败做 video/action trace 分类，并验证 contact/placement/recovery 假设。
2. 做 30–100 step GPU smoke，比较 preload/infinite 与 baseline 的 `data_wait_sec/train_sec/total_sec`、显存和吞吐。
3. 对 actionheadfix 的相同54 scenario做多 checkpoint seed repeat，报告 paired interval；沿 full-h4 matched failure 建立 expert→x0→sampled→converted→executed action 的 first-divergence trace。

11. References

- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\reports\uwm_robocasa_libero_reproduction_log_report.html`
- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\libero_official_common.py`
- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\uwm_libero_rollout_eval.py`
- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\remote_muxi_libero_rollout_eval_5tasks_parallel.sh`
- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\remote_muxi_libero_finetune_5tasks_ddp.sh`
- `C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\scripts\benchmark_libero_dataloader_stall.py`
- `C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\sync_manifest.json`
- `C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\sync_summary.md`
- `C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\final_strict_eval_user_confirmed_20260705.json`

- `C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_benchmark_evidence_claim_to_protocol.md`
- `C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_04_uwm_video_action_claim_to_code.md`
- `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\uwm_libero_ownership_ledger.md`
- UWM upstream revision recorded in sync manifest: `b7d4067c8859775e5b08f681e76d538015cdd445`
- LIBERO upstream revision recorded in sync manifest: `8f1084e3132a39270c3a13ebe37270a43ece2a01`
- UWM/LIBERO/RoboCasa Transfer Evidence Freeze；canonical Markdown，review JSON。
- UWM RoboCasa ActionHeadFix 30-Epoch Train-to-Closed-Loop P0 Audit；canonical Markdown，review JSON。
- UWM `libero_90.pt` 本地 immutable checkpoint manifest gate；canonical Markdown，review JSON。

Part 19 / 49 · `interview_materials/vla/evidence/projects/uwm_libero_ownership_ledger.html` · SHA-256 `8BB8E60A91111D1A15E4101A49A4E31D42750EFCC92F5BA154DFAF98D4789E2F`

返回本卷目录

Canonical Markdown SHA-256：`8bb8e60a91111d1a15e4101a49a4e31d42750efcc92f5ba154dfaf98d4789e2f`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\uwm_libero_ownership_ledger.md`

UWM / LIBERO Ownership Ledger：协议迁移、数据管线与结果边界

状态：`ownership_audited / coarse_git_supported / candidate_confirmation_pending / WARN` 审查日期：2026-07-21 证据截止：2026-07-21 项目范围：`C:\Users\mousongzhe\Project` 中的 UWM/LIBERO official repro、RoboCasa transfer、DataLoader stall diagnosis、strict/reset recovery 与用户确认结果。 结论先行：本地可以核验 LIBERO task mapping、finetune/rollout launcher、`pruned_init` /warmup 协议、DataLoader-only smoke，以及RoboCasa actionheadfix一条30-epoch run的8×54 raw rollout；但 project snapshot 是一次性快照，RoboCasa 路径经历多次同步。当前 `mousongzhe` 提交证明的是候选账号提交了这些 artifact，不证明每个函数由候选人独立设计、编写、运行或验收。 LIBERO `186/250` 仍是用户确认summary；RoboCasa raw denominator虽已存在，produced checkpoint binary/hash、env/scorer/videos与个人角色仍未闭合。

0. 候选人项目范围自述（self-report，不等于 artifact/ownership 证明）

2026-07-21，候选人在本线程明确表示：

> “做了 UWM 在 LIBERO 的 fine-tune 和 eval，做了 UWM 在 RoboCasa 的尝试，协议对齐和训练评估。”

这条自述用于把 UWM 两条线纳入面试准备和 artifact 搜索范围；它不自动证明具体代码 ownership、checkpoint lineage、严格/重置协议下的结果归因，或 RoboCasa 的 root cause。当前安全映射如下：

自述范围	当前证据层	现在可用的安全动词	仍不能直接说
UWM / LIBERO fine-tune	`candidate-self-report` + local launcher/protocol inventory	“开展 LIBERO 适配与 fine-tune 训练链对齐”	独立设计 UWM architecture、拥有可加载 checkpoint、结果已 result-owned
UWM / LIBERO eval	`candidate-self-report` + strict/reset/Table II protocol evidence	“运行/审计 LIBERO 评估协议”	把 strict、reset-style、Table II 结果混成一个分数，或把用户确认数字写成 raw 本地结果
UWM / RoboCasa 尝试、协议对齐与训练评估	`candidate-self-report` + actionheadfix/raw rollout 与诊断部分支持	“尝试 RoboCasa 迁移、对齐 action/normalizer/controller/eval contract 并做训练诊断”	已定位 root cause、完整成功率、checkpoint/env/scorer ownership

在候选人逐项回答本页 ownership questions 前，这三行保持 `candidate-self-report / role-confirmation-pending`，不迁移为 `role-confirmed` 或 `result-owned`。

1. 问题（Question）

面试官问“UWM/LIBERO 哪些是你做的？”时，要拆开：

1. UWM shared diffusion 与 LIBERO benchmark 的上游机制；
2. 本地 snapshot 中的迁移脚本、patch、配置、报告和诊断 artifact；
3. 候选人亲手设计、修改、运行、审查或验收的动作；
4. 上游 repo、远端运行者、协作者和工具辅助的边界；
5. checkpoint、normalizer、episode trace、success summary 与论文比较的归属。

本页不是把项目页中的“负责/迁移”自动认证为个人贡献。没有候选人逐项确认或过程证据的行保持 `role-confirmation-pending`。

2. Role Fit 与 JD 能力映射

岗位能力	本地可核证据	Ownership 强度	当前策略
VLA/VLM 训练迁移	LIBERO-90→LIBERO-10 finetune launcher、DDP patch、task map、checkpoint path contract	local-direct；个人动作待确认	可讲训练/评估接口迁移，不说原创 UWM architecture

岗位能力	本地可核证据	Ownership 强度	当前策略
Policy rollout / benchmark protocol	fixed-pruned-init、5-step warmup、50 rollouts/task、success summary	local-direct / user-confirmed final	先绑定 init/seed/normalizer/horizon/scorer，再说数字
数据管线工程	Zarr/HDF5 buffer、infinite sampler、preload、workers/prefetch/pin 与 stall benchmark	local-direct；full GPU A/B 缺	可讲 DataLoader diagnosis，不承诺训练吞吐 uplift
跨 benchmark 适配	RoboCasa action/state/prompt/normalizer/eval patches、30-epoch train log与8×54 raw rollout	local-direct / remote-snapshot-raw	讲train→closed-loop failure，不写RoboCasa benchmark success
结果与可比性	0.744、0.928、Table II≈0.79 的 protocol reconciliation	user-confirmed/report-supported；C3 未闭合	不把 0.744 写成本地 raw verified 或严格论文复现

3. Ownership 证据合同

3.1 五层 AND-gate

强主张“我负责并完成了 X”至少需要：

$$G(c) = I(c) \wedge A(c) \wedge D(c) \wedge V(c) \wedge O(c).$$

`I` 是 file/commit/run identity，`A` 是 artifact，`D` 是设计与修改过程，`V` 是运行/验收，`O` 是个人、上游、协作者和工具边界。commit author、项目 report、远端 path 和一个 summary JSON 都只能覆盖部分层级。

3.2 状态词汇

状态	含义	可用动词
`upstream-verified`	frozen UWM/LIBERO source audit 支持的公开机制/协议	“上游定义/论文使用”
`snapshot-local`	本地 Git snapshot 中存在文件或报告	“本地快照包含”
`git-coarse`	候选账号提交，但没有逐文件历史	“候选账号提交了快照”
`self-attested`	简历/项目页自述本人负责	“候选人自述负责”
`role-confirmed`	候选人逐项确认亲手/协作/工具/运行角色	“我负责/我修改”
`result-owned`	checkpoint、normalizer、protocol、episodes、scorer 与角色闭合	“我完成并得到……”

4. Git、工作树与 run lineage 冻结事实

4.1 Project history

检查	结果	能证明	不能证明
当前 HEAD	`6c524d9c6c6226075f100cdac395fef2fdc37671`，2026-07-02；author/committer `mousongzhe`	当前 committed snapshot 的提交身份	每个文件由候选人独立完成
初始 Project snapshot	`c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`，root commit	初始项目材料一次进入 Git；UWM official repro 当时尚不存在	初始文件来源、协作者和生成顺序
UWM official repro recovery	`0341c5016a63c4160a9027e5c333a33ce880bd1d`；21 files / 3,784 additions	18 个 `uwm_libero_official_repro` 路径和 recovery report 在候选账号提交中进入	远端训练/rollout 是否由候选人执行，逐行 ownership
RoboCasa transfer evolution	`796374c`、`57bf4e`、`a37a386`、`2a2912d`、`1a03fe`、`8eaddf`、`6c524d9`	路径经历多次候选账号提交和诊断迭代	每次修改的设计者、运行者、reviewer

4.2 Current worktree boundary

`Project` 当前不是 clean worktree：`report/uwm_libero_strict_eval_recovery_20260702/sync_manifest.json` 与 `sync_summary.md` 有未提交修改，`final_strict_eval_user_confirmed_20260705.json` 是未跟踪文件。这个状态能证明“当前工作区保存了 recovery/user-confirmed material”，不能证明它们已经进入 HEAD，也不能把用户确认 JSON 变成本地 raw episode bundle。

4.3 Result artifact boundary

当前没有本地 5-task checkpoint、normalizer hash、250 条 episode JSONL、失败视频、逐样本 scorer 或完整 remote log。因此：

- `local-direct`：脚本、patch、静态报告、DataLoader-only smoke 和 Git blob；

- `report-supported`：项目 report 中的 Book-Caddy 42/50、路径和诊断叙事；
- `user-confirmed-remote-final`：用户确认的 186/250 及 per-task summary；
- `protocol-limited`：fixed-pruned-init、reset-style 与 Table II OOD 之间的数字；
- `remote-snapshot-raw`：RoboCasa actionheadfix同一run的train log、normalizer与8×54 raw rollout；
- `result-owned`：当前未达到，不能说候选人拥有 0.744 的 checkpoint/result。

5. Per-file / per-decision ownership matrix

ID	文件 / 决策	Local artifact 事实	Git / run 事实	当前 role verdict	当前安全口径	升级所需证据
`UWM-OWN-001`	Project initial snapshot 与 UWM/RoboCasa 路径来源	`c4469eb` 是 root snapshot；当时 `uwm_libero_official_repro` 不存在，RoboCasa transfer 有 50 个初始路径	author `mousongzhe`；无 pre-snapshot history	`snapshot-local / git-coarse / role-pending`	“项目快照包含 UWM/RoboCasa 相关材料”	pre-snapshot 来源、协作者/工具记录、候选人逐项确认
`UWM-OWN-002`	`scripts/libero_official_common.py` task map / shape contract	五 task、HDF5/BDDL/pruned-init/buffer、RGB `[128,128,3]`、LIBERO action `[7]`；SHA `DE9D…7E2B`	87 行在 `0341c50` 一次进入	`local-direct / decision-owner-pending`	“本地 adapter 固定了 task 与 shape 合同”	为什么选这些字段、原始 upstream diff、运行命令、review
`UWM-OWN-003`	`scripts/uwm_libero_rollout_eval.py` strict/reset evaluator	`uwm_reset` 与 `libero_fixed_init`、5-step warmup、seed/init/checkpoint/horizon/summary；SHA `05E9…F3943`	211 行在 `0341c50` 一次进入	`local-direct / process-role-pending`	“本地实现了可审计 rollout protocol”	亲手函数、same-checkpoint paired run、episode trace、scorer tests
`UWM-OWN-004`	5-task rollout launcher	8 shard、50 rollouts/task、600-step/manifest/output-root contract；SHA `C61C…EE32`	270 行在 `0341c50` 一次进入	`local-direct / operator-pending`	“项目提供 5-task rollout launcher”	remote shell history、运行者、完整 output manifest、失败重试记录
`UWM-OWN-005`	5-task finetune launcher + DDP path	LIBERO-90 checkpoint、5 task、DDP、batch/steps、output checkpoint contract；SHA `E5B6…9D92`	101 行在 `0341c50` 一次进入	`local-direct / role-pending`	“本地组织了 LIBERO finetune path”	base checkpoint/normalizer、effective config、train log、候选人改动点
`UWM-OWN-006`	`prepare_libero_buffers.py` 与 dataset buffer route	HDF5/BDDL → compressed Zarr buffer 的准备脚本；SHA `2517…4E8A`	106 行在 `0341c50` 一次进入	`local-direct / data-owner-pending`	“项目建立了 LIBERO buffer preparation path”	数据来源、过滤/切分决策、运行输出、hash manifest
`UWM-OWN-007`	DataLoader stall benchmark	finite/infinite/preload A/B；p95 wait 13.862s→0.194s；SHA `47A2…6B36`	141 行在 `0341c50`；DataLoader-only，不是 GPU training	`process-supported / verification-role-pending`	“我定位了数据等待路径的候选原因”需候选人确认	benchmark command、raw timing、memory/throughput matched GPU smoke
`UWM-OWN-008`	`repo_patch/datasets/utils/loader.py` + sampler	infinite sampler、workers/prefetch/pin；loader SHA `FC8E…3551`，sampler SHA `6B45…DA95`	patch 在 `0341c50`；无 upstream before/after provenance	`snapshot-local / implementation-role-pending`	“本地 patch 包含 preload/infinite loader 选项”	upstream diff、亲手/协作/工具边界、unit test、full run验证
`UWM-OWN-009`	`repo_patch/experiments/uwm/train_robomimic.py` 与 `train.py`	pretrain load、num_steps、rollout hooks、timing；train_robomimic SHA `8423…40BD`	228 行 patch 在 `0341c50`，无逐函数历史	`snapshot-local / role-pending`	“项目 patch 扩展了训练与 rollout hooks”	patch 来源、代码 review、实际执行命令、checkpoint load
`UWM-OWN-010`	`sync_manifest.json` / `sync_summary.md` recovery	记录 remote host、script hashes、strict/reset 差异；当前文件已被工作树修改	committed blob 与 current SHA 不同；final JSON 未跟踪	`working-tree / report-editor-pending`	“当前工作树保存了 recovery note”	谁修改、SSH/raw output、manifest commit、reviewer确认
`UWM-OWN-011`	`final_strict_eval_user_confirmed_20260705.json`	user-confirmed per-task 0.88/0.80/0.76/0.66/0.62，186/250=0.744；当前 SHA `7616…6D11`	未跟踪文件；result source 明确为 user-confirmed remote final	`user-confirmed-remote-final / result-owner-blocked`	“用户确认的 fixed-pruned-init summary”	checkpoint/normalizer/init/episode/scorer bundle、运行/审查角色
`UWM-OWN-012`	RoboCasa transfer patch/actionhead diagnostics	train/rollout code、30-epoch train log、12D normalizer与8×54 raw JSON；full-h4 `0/54` /contact `45/54`；train SHA `EFC5…1FFE`	`796374c` → `6c524d9` 迭代；remote_sync cache非committed result bundle	`remote-snapshot-raw / checkpoint-unverified / role-pending`	“参与跨 benchmark contract 调试并审计 train→closed-loop断层”	upstream diff、produced checkpoint SHA/fresh-load、env/scorer/videos、候选人/远端运行者确认
`UWM-OWN-013`	UWM shared diffusion / LIBERO benchmark method	source audits 支持 shared action/video diffusion、独立 timestep、LIBERO protocol 与 Table II OOD	upstream revisions `b7d4067…` / `8f1084e…`；不属于本地原创	`upstream-verified / upstream-only`	“基于/对照 UWM 与 LIBERO 上游方法”	永远不升级为个人原创；只确认适配、迁移或复现工作

6. Claim → ownership reconciliation

面试/简历 claim	当前证据合同	安全处理
“我迁移并完成 UWM/LIBERO 训练闭环”	scripts/patch/report direct；远端 train/checkpoint/operator/role 未闭合	候选确认前用“项目包含/参与迁移”；确认后绑定具体文件和命令
“我取得 LIBERO 5-task 74.4%”	0.744 是 user-confirmed remote final，无本地 checkpoint/episodes	说“用户确认的 fixed-pruned-init summary”，不说 local raw verified

面试/简历 claim	当前证据合同	安全处理
“74.4% 复现官方 0.79”	init/shift/background/seeds/scorer 不同，inventory 判定 NON_COMPARABLE	禁止“严格复现/只差4.6pp”作为强结论
“0.928 说明模型成功率更高”	reset-style 与 fixed-pruned-init 协议不同	只说 protocol sensitivity diagnostic
“我解决了 DataLoader 性能问题”	DataLoader-only p95 A/B 有直接证据；GPU throughput/memory 未验证	说“定位并验证 DataLoader-only wait 降低”，不报训练吞吐 uplift
“RoboCasa benchmark 已完成”	有一条8×54 raw success bundle但只有项目协议、无produced checkpoint/env-scorer/videos	保持 `protocol-bounded`，讲跨benchmark train→closed-loop failure
“UWM architecture 是我设计的”	source audit 指向 upstream shared diffusion mechanism	明确上游；个人贡献只落在迁移、协议、诊断和验收
“Git author 证明所有代码是我写的”	一次性 snapshot + 多次同步；无逐文件 provenance	用 13 行 ledger 区分 snapshot、self-attested、role pending 和 upstream

7. Production Gap：抗追问最小补证据

Gap	面试官追问	最小补证据	当前表达
快照来源	`c4469eb` 之前代码从哪来?	pre-snapshot source/patch/contributor record	`git-coarse`
trainer ownership	哪个函数是你亲手改的?	before/after diff、review note、现场解释	`role-pending`
checkpoint/normalizer	0.744 用的权重和 normalizer 是什么?	binary/config/normalizer hash + load smoke	`user-confirmed-remote-final`
episode evidence	186/250 的 250 个 episode 在哪?	per-episode JSONL/video/terminal reason/scorer	`raw bundle missing`
protocol comparison	为什么不和 0.79 直接比?	exact init/seed/shift/background/scorer manifest	`NON_COMPARABLE`
DataLoader diagnosis	preload 是否提高完整训练吞吐?	30–100 step GPU matched A/B、memory、data_wait/train/total	`smoke-bounded`
RoboCasa result	actionheadfix 后 success 多少?	8×54 raw JSON已存在；仍需checkpoint SHA/fresh-load、env/scorer/videos和repeat	`raw-rollout-backed / result-owner-blocked`
tool/collaboration	哪些由协作者或 AI 辅助?	contribution ledger、review记录、运行者名单	不推断 Codex-primary

8. Question Tree 与回答分层

```
你个人做了什么?
  ├ 训练: LIBERO-90→LIBERO-10 的 launcher、patch、normalizer 和 checkpoint contract 哪些是你改的?
  ├ 评估: fixed-pruned-init、reset-style、Table II OOD 的 tuple 如何冻结?
  ├ 数据: zarr 小 chunk 的 first failure、preload/infinite 的验证范围是什么?
  ├ 结果: 0.744 的 checkpoint、250 episodes、scorer 和运行者在哪?
  └ 归属: UWM architecture、上游 patch、协作者、工具和你自己的动作如何分开?
```

危险回答：说“我把 UWM 完整复现了，74.4% 就是我的 benchmark 结果，Git author 证明所有 patch 都是我写的”。

可过回答：我把贡献拆成迁移、协议、数据管线和诊断。LIBERO 186/250仍是用户确认summary；RoboCasa一条30-epoch run已有8×54 raw rollout，可讲full-h4 `0/54`和contact `45/54`的失败断层，但checkpoint/env-scorer/ownership仍未闭合。UWM architecture属于上游，我的具体ownership按文件、命令、远端运行和验收记录回答。

强回答（补证后才可用）：我能从一个具体 launcher/patch 说明输入、改动、失败、验证和输出 manifest，并给出同 checkpoint paired protocol、normalizer hash、episode JSONL 和 scorer；对不能闭合的部分主动降级为 report-supported 或 user-confirmed。

9. SP/SSP 证据就绪度

维度	分数（5）	依据
岗位相关性	4.5	VLA 训练迁移、闭环 rollout、benchmark protocol 和数据管线高度相关
机制深度	4.0	shared diffusion、task map、init/warmup、normalizer、stall diagnosis 可解释
工程闭环	3.7	scripts/patch/report与RoboCasa 8×54 raw齐；produced checkpoint/load/env-scorer/videos仍缺
指标可信度	2.5	LIBERO 0.744 user-confirmed；RoboCasa raw denominator可复算但协议/权重未闭合
Ownership 清晰度	2.0	13-row ledger 建立；逐文件、工具、运行者确认 pending
抗追问能力	3.0	protocol boundary 强；结果和个人实现仍有断点
合计	**19.7 / 30**	证据就绪度，不是候选人能力评分；项目保持 WARN

10. 候选人确认表

请逐项回答“亲手 / 协作 / 工具辅助 / 只审查 / 只运行 / 不确定”：

1. `c4469eb` 初始 snapshot 中的 UWM/RoboCasa 文件来自哪里？ `0341c50` 的 official repro 由谁设计、实现和 review？
2. `libero_official_common.py` 的五 task、shape、pruned-init 和 buffer 选择是谁决定的？
3. `uwm_libero_rollout_eval.py` 中 strict/reset、warmup、seed/init、horizon 和 summary 逻辑哪些由你修改？
4. finetune/rollout launcher 与 `train_robomimic.py` patch 谁编写、谁在 remote 执行、谁验收 checkpoint？
5. DataLoader stall benchmark 与 loader/sampler patch 的 first failure、修改和 matched validation 是什么？
6. 0.744 的 checkpoint、normalizer、250 episodes 和 scorer 由谁生成、保管、审查？
7. RoboCasa actionheadfix、dataset、rollout 和 loss report 中哪些是你亲手、协作或工具辅助？
8. UWM shared diffusion architecture 与 LIBERO Table II 哪些明确属于上游，你的个人贡献如何界定？

在真实回答发生前，不创建 `coaching_state.md`，不把任一行升级为 `role-confirmed` 或 `result-owned`。

11. 分阶段补强计划

阶段	动作	完成标准	状态
半天	完成 8 项 ownership drill；每个关键脚本选一个现场解释点	13 行均有 role/source/verification	待候选人确认
1 天	将 committed sync manifest、工作树 recovery 和 user-confirmed JSON 标成不同 lineage	final result 不再依赖单个 summary/path	待补做
3 天	获取 checkpoint/normalizer/init/episode/scorer，做 load smoke 与 same-checkpoint paired protocol	0.744 至少形成可复算 C3 bundle	待补做
1 周	做 30–100 step GPU DataLoader matched A/B，并为 RoboCasa 建 action/state/normalizer/eval manifest	stall 诊断与跨 benchmark 迁移均有可验证闭环	待补做

12. Output Pack

12.1 简历 bullet 三档

- 当前保守版：参与 UWM/LIBERO 训练与 rollout 迁移、固定 strict/reset 评估协议、定位 Zarr/DataLoader stall，并整理用户确认的 fixed-pruned-init 结果及不可比性边界。
- 候选确认后标准版：负责指定 UWM/LIBERO launcher、协议开关、数据管线诊断和评估验收；以文件/commit/命令说明个人、协作与上游边界。
- 补齐 C3 后偏强版：在 checkpoint、normalizer、episode、scorer 和 paired protocol 完整后，给出可复算的 LIBERO 结果与 GPU stall A/B；当前禁止提前使用。

12.2 两分钟叙事骨架

> UWM 项目我主要做的是训练到闭环评估的协议工程：LIBERO侧固定task map、pruned-init、warmup、normalizer和success tuple，并用DataLoader-only smoke定位zarr等待；`186/250` 仍是用户确认summary，不能与Table II直接横比。RoboCasa侧有一条actionheadfix 30-epoch raw sweep：8卡训练完成，同checkpoint的8×54 rollout都无env error但只有0–2 success，full-h4 `0/54` 且contact `45/54`，所以定位为train→closed-loop failure。UWM架构属于上游；我的具体贡献按脚本、命令、远端运行和验收记录逐项确认。

12.3 面试前 6 题

1. fixed-pruned-init、reset-style、UWM Table II OOD 三种评估 tuple 分别是什么？
2. 为什么 `0.928→0.744` 主要应解释为 protocol gap，而不是模型权重退化？
3. `libero_official_common.py`、rollout evaluator 与 finetune launcher 如何串成一条 lineage？
4. Zarr preload/infinite 为什么能降低 DataLoader-only p95 wait？怎样证明 GPU training 也受益？
5. RoboCasa action/state/prompt/normalizer/eval contract 的 first-failure 如何定位？
6. 哪些是 UWM/LIBERO 上游，哪些文件/决策是你亲手、协作、工具辅助、只运行或只审查？

13. Ownership 审计伪代码

```
def accept_uwm_claim(claim, ledger):
    if not ledger.identity_and_artifact(claim):
        return "reject_strong_claim"
    if not ledger.role_confirmed(claim):
        return "use_snapshot_or_user_confirmed_wording"
    if claim.level >= "C2" and not ledger.process_and_verification(claim):
```

```
        return "downgrade_to_migrated_or_reviewed"
    if claim.level == "C3" and not ledger.checkpoint_protocol_episode_chain(claim):
        return "report_or_user_summary_only"
    return "claim_allowed_with_exact_scope"
```

这段伪代码是本仓库真实性 gate，不是 UWM 官方实现。

参考文献（References）

1. UWM / LIBERO Claim–Evidence Inventory：12 条 claim、strict/reset、DataLoader 和 RoboCasa 边界。
2. UWM 项目题卡：当前 `PROJ-UWM-001` protocol-boundary card；本 ledger 将新增 ownership card。
3. Evidence Manifest：五项目 evidence 与 ownership 状态总索引。
4. VLA Knowledge Map：VOL 07 项目深挖与题目 ID 空间。
5. Local Git repository `C:\Users\mousongzhe\Project`，HEAD `6c524d9c6c6226075f100cdac395fef2fdc37671`；official repro `0341c5016a63c4160a9027e5c333a33ce880bd1d`；RoboCasa evolution `796374c` → `6c524d9`。
6. `wm/uwm_libero_official_repro/scripts/uwm_libero_rollout_eval.py` SHA-256 `05E912C2096EFF4BFE8D061E4C334517B03DE80847793D83025A2AA3E18F3943` 与 `scripts/libero_official_common.py` SHA-256 `DE9DB94113554DB50B151900647E6D32BFFB9649C7F4D1D97EA0D65A8A937E2B`。
7. UWM benchmark evidence audit 与 UWM video/action audit：上游/本地机制和 Table II 协议边界。
8. Optimized Resume 与 UWM 项目面试页：候选人自述和投递口径，未经本 ledger 自动认证。
9. UWM RoboCasa ActionHeadFix 30-Epoch P0 Audit：run-specific raw rollout与checkpoint/ownership边界。

Part 20 / 49 · `interview_materials/vla/evidence/projects/uwm_libero_p0_bundle_audit.html` · SHA-256 `AE430E44AF3A3B59CF444850D782D06FFD0A147ACA5F8FCB46616C0C9A81E034`

返回本卷目录

Canonical Markdown SHA-256: `ae430e44af3a3b59cf444850d782d06ffd0a147aca5f8fcb46616c0c9a81e034`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\uwm_libero_p0_bundle_audit.md`

UWM / LIBERO Po Bundle Audit：Protocol / Data Pipeline / Remote Result

状态: `p0_audited / libero-user-confirmed / robocasa-raw-rollout-backed / checkpoint-unverified / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计 UWM/LIBERO local scripts/reports/protocols、DataLoader-only smoke、user-confirmed remote summary 和 RoboCasa debugging boundary；不把远端 summary 升级为本地可复算 benchmark。

1. 结论

UWM/LIBERO 的训练入口、task map、fixed-pruned-init/5-step warmup、normalizer/action shape、rollout evaluator、DataLoader stall benchmark 和 RoboCasa patch 均有 local-direct 或 report-supported 入口。用户确认的 fixed-pruned-init `186/250=0.744` 仍只有 per-task summary，本地没有对应 checkpoint、250 episode JSONL、失败视频或 scorer。RoboCasa 证据则已显著增强：ActionHeadFix 30-Epoch Train-to-Closed-Loop Po Audit 冻结了一条8卡3930-step run、12D normalizer和八份54-scenario raw rollout JSON；full h1/h2/h4/h5/h8为 `2/54、1/54、0/54、1/54、1/54`，三组h5 input ablation为 `2/54、2/54、1/54`，全部env errors为0。这证明训练—评测失败链可重算，仍不是 RoboCasa benchmark success。

Po 仍未闭合：LIBERO结果 provenance、论文 Table II comparability、same-checkpoint reset paired run、GPU matched A/B，以及 RoboCasa produced checkpoint/env-scorer revision/videos和个人 ownership 都保持未确认。

```
P0 status = PARTIAL / USER-CONFIRMED-REMOTE-FINAL
eligible local benchmark claims = 0
eligible result ownership claims = 0
allowed = protocol engineering, DataLoader-only diagnosis, RoboCasa raw-rollout failure taxonomy
forbidden = local 0.744 reproduction, Table II 0.79 comparison, RoboCasa success, UWM architecture authorship
```

2. Po gate matrix

Po 字段	当前证据	状态	安全解释
task / rollout protocol	five LIBERO-10 task map、50 rollouts/task、600-step、5-step warmup、success summary	`PRESENT`	可讲 protocol tuple
checkpoint / normalizer	launcher/path contract、report 引用	`MISSING`	远端 path 不是 local weight/hash
fixed-pruned-init result	user-confirmed JSON：186/250、per-task summary	`USER-CONFIRMED-REMOTE`	只能说用户确认的项目 summary
reset-style comparison	user-confirmed 232/250	`PROTOCOL-DIAGNOSTIC`	不当作模型 uplift 或正式成绩
Table II comparison	paper OOD init、+0.03 shift、background removal、3 seeds	`NON-COMPARABLE`	不能说只低 4.6pp 或复现论文
episode evidence	无 250 episode JSONL、视频、terminal reason	`MISSING`	不可本地复算 success
scorer / init manifest	脚本和报告说明字段	`PARTIAL`	exact init index/normalizer/scorer hash 缺
DataLoader diagnosis	finite/infinite/preload A/B；p95 wait 13.862s→0.194s	`SMOKE-BOUNDED`	可讲 DataLoader-only wait，不报 full GPU uplift
RoboCasa actionheadfix train/eval	30-epoch train log/summary、12D normalizer、8×54 raw rollout JSON	`RAW-ROLLOUT-BACKED / CHECKPOINT-UNVERIFIED`	讲 train→closed-loop failure；不称 benchmark success
ownership	13-row ledger、snapshot/working-tree/recovery 分层	`PENDING`	需逐文件、运行者、审查者确认

3. 已核验的协议边界

- 项目 fixed-pruned-init `186/250=0.744` 与 reset-style `232/250=0.928` 使用不同初始化协议；
- UWM Table II 约 `0.79` 使用 OOD initialization、+0.03 object-init shift、背景移除和 3 个 seed；
- 因此 `0.744 vs 0.79` 不能解释为模型差距，`0.928→0.744` 不能解释为权重退化；

- DataLoader-only smoke 中 preload+infinite 将 p95 wait 从约 13.86s 降至约 0.19s，但没有 matched GPU training throughput/memory 证据；
- RoboCasa 已有一条项目协议下的8×54 raw success bundle，但 produced checkpoint binary、env/scorer revision、videos和benchmark comparability仍缺；只能称 contract debugging 与 failure audit。

4. 面试可用与禁用表述

可以说：

> “我主要做的是训练到闭环评估的协议工程：固定 LIBERO task map、pruned-init、warmup、normalizer、horizon、rollout 数和 success predicate，并用 DataLoader-only A/B 定位 zarr 小 chunk 的周期性等待。项目材料里有用户确认的 fixed-pruned-init 186/250 summary，但 checkpoint、250 episode、scorer 不在本地，所以我不称它为 local raw reproduction；reset-style 和 UWM Table II 也按协议分开。RoboCasa 仍是跨 benchmark contract 调试。”

RoboCasa 追问时补充：actionheadfix 的一条30-epoch run已经有八份54-scenario raw JSON，能说full-h4 `0/54`、contact `45/54`与0 env errors；不能说checkpoint binary已验证、接触等于成功或input ablation产生因果提升。

不能说：

1. 本地已复现 LIBERO 0.744；
2. 0.744 与 Table II 0.79 严格可比；
3. reset-style 0.928 证明模型更好；
4. DataLoader-only wait 降低等于 GPU training throughput 提升；
5. RoboCasa benchmark 已完成；
6. UWM shared diffusion architecture 是个人原创。

5. 关闭 P0 的最小动作

1. 获取 5-task strict run 的 checkpoint、normalizer、exact config、init index、250 episode JSONL、失败视频和 run manifest；
2. 对同 checkpoint 做 reset-style vs fixed-pruned-init paired run，记录每 episode seed/init/scorer；
3. 做 30–100 step GPU matched DataLoader A/B，保存 data_wait/train/total/memory/throughput；
4. 候选人回答 `OWN-UWM-001`，明确 launcher、patch、远端运行、诊断和工具协作边界。
5. 对 actionheadfix run 回传 `models.pt` SHA/fresh-load parity、完整 train JSONL、RoboCasa/env/scorer revision和episode video locator。

在这些字段闭合前，UWM/LIBERO 保持 `PARTIAL/WARN`，不改变 `PROJ-UWM-*` 或通用题卡 mastery。

References

1. UWM/LIBERO Claim–Evidence Inventory
2. UWM/LIBERO Ownership Ledger
3. 五项目 P0 Artifact Intake Contract
4. UWM/LIBERO/RoboCasa Transfer Evidence Freeze
5. UWM RoboCasa ActionHeadFix 30-Epoch P0 Audit

Part 21 / 49 · `interview_materials/vla/evidence/projects/uwm_robocasa_actionheadfix_30epoch_train_eval_p0_audit.html` · SHA-256 `41B9CDDDB95163A369719E007EA11AD3A7776E79F0901F80DCACD34FAA83F117`

返回本卷目录

Canonical Markdown SHA-256：`41b9cdddb95163a369719e007ea11ad3a7776e79f0901f80dcacd34faa83f117`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\uwm_robocasa_actionheadfix_30epoch_train_eval_p0_audit.md`

UWM RoboCasa ActionHeadFix 30 Epoch：Train-to-Closed-Loop P0 证据审计

状态：`p0_audited / remote-snapshot-raw-rollouts / checkpoint-path-only / protocol-bounded / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计 `target_atomic_seen_official12_task_30epoch_muxi_20260628_openpi_state_3view_phase_30ep` 这一条 30-epoch run、对应的八组 54-scenario rollout 与同阶段 preflight；不把 Human300、100-epoch resume、其它 taskcondfix/state-weighted run、LIBERO `186/250` 或不同 action bridge 合并进主结果。

1. 问题（Question）

这条 UWM→RoboCasa actionheadfix 运行线是否真的完成了训练、保存引用和 closed-loop evaluation？训练端的 loss 下降能否转化为任务成功？`exec_horizon`、prompt/state/right-view ablation 的八组 rollout 有什么可复核结论？哪些字段仍不足以称为 RoboCasa benchmark、checkpoint closure 或个人独立成果？

2. 结论先行

当前 remote snapshot 已经超过 report-only：它保留了训练命令与 `train.out`、run summary、DeepSpeed effective config、12D normalizer、三类 preflight、八份原始 rollout JSON 和八份 eval log。独立复核可以证明：

1. 8 卡 MetaX C500 的 30-epoch run 完成 `3930` steps；8/8 process 成功退出，未发现 traceback 或 CUDA OOM；
2. 训练使用 18 个 atomic-seen task、9126 episodes、133457 sequences、12D reordered action、16D state、三视角、task/prompt/state conditioning；
3. `train.out` 中 41 个打印 metric point 从 step 0 的 loss/action/raw-action `3.3257/2.4300/0.9986` 到 step 3929 的 `0.25385/0.07689/0.03043`；但 `val_ratio=0`、`eval_every=0`，这些只是训练 batch 指标，不是 held-out generalization；
4. 八个 rollout JSON 都包含相同的 54 个 `(scenario, task, seed)`，覆盖 18 task×3 trials；全部使用同一远端 `models.pt` path、同一 normalizer path、`official12 + convert_action`，共 432 条配置内 record，环境错误为 0；
5. closed-loop success 极低且不稳定：full input 的 h1/h2/h4/h5/h8 分别为 `2/54、1/54、0/54、1/54、1/54`；h5 ablation 的 no-prompt/no-right/no-state 为 `2/54、2/54、1/54`；所有 success 只出现在 TurnOnElectricKettle 或 TurnOnMicrowave；
6. full-h4 有 `45/54` episode 记录 robot-env contact，但 success 为 `0/54`，说明“产生接触”不等于“完成任务”。

因此，安全结论是：**训练链、12D action contract 和 54-scenario eval harness 可以审计；训练 loss 明显下降，但没有转化为稳定的 closed-loop task success。** 当前不能宣称 actionheadfix 已解决 RoboCasa transfer，也不能从 1–2 个 success 推导 prompt/right-view ablation 的因果收益。

3. P0 promotion 结论

```
P0 status = PARTIAL
eligible training-completion claim = PRESENT / remote-snapshot-backed
eligible raw-rollout recount = PRESENT / 8 x 54 records
eligible checkpoint-binary claim = 0
eligible benchmark claim = 0
eligible ablation-uplift claim = 0
eligible ownership claim = 0
```

P0 字段	当前证据	状态	安全解释
run identity	单一 run name、output/checkpoint/normalizer path 一致	`PRESENT`	可以按 run family 讲，不与其它 30/100 epoch 混用
training completion	3930 steps、30 epochs、8/8 process exit success	`PRESENT`	可说训练进程完成
training metrics	`train.out` 有 41 个打印 point；derived verifier 声称读取 3930 records	`PARTIAL`	直接可重算 41 点；完整 `train.jsonl` 未同步
validation	`val_ratio=0`、`eval_every=0`	`ABSENT BY CONFIG`	训练 loss 不能包装成 held-out metric

Po 字段	当前证据	状态	安全解释
dataset contract	18 tasks、9126 episodes、133457 sequences、seq 19、stride 16、12D action	`PRESENT`	支持输入输出和 sampling contract
base load	446 matched、8 action-head keys skipped、32 missing-after-partial-load、0 unexpected	`PRESENT / RESET-HEAD`	证明迁移策略，不证明新 head 已学好
produced checkpoint	summary 与 rollout 指向同一 `checkpoints/models.pt`	`REMOTE-REFERENCED`	当前 snapshot 中 checkpoint-like binary 为 0
normalizer	本地 12D normalizer JSON 与 rollout path 对齐	`PRESENT`	可 hash；第 11 个重排维度 std=0 需保持 contract 解释
raw rollout	8 JSON×54 records，task/seed/scenario 可逐条复算	`PRESENT`	每个 config 的 denominator 完整
environment health	八组均 `num_env_errors=0`	`PRESENT`	失败不是 env exception，但仍可能是 protocol/model failure
scorer	JSON 有 runtime `success` 与 per-task aggregation	`PARTIAL`	缺 immutable RoboCasa/env/scorer revision 与 episode videos
ownership	Project commit、脚本和 Codex-derived verifier 存在	`PENDING`	不从路径、Git author 或工具生成 summary 推导个人独立实现

4. Frozen inputs 与 SHA-256

4.1 Run、preflight 与 aggregate files

文件	SHA-256	用途
`logs/.../train.out`	`4C56A7181A912156C482D7982B7568FBAB1FE597DC25702B318D615BD3588B80`	训练命令、打印 metrics、run summary、8-rank exit
`runs/.../train_summary.json`	`88EDF9D28C7EF5D40316A7C71DE0C8E932492257A50383397C497974568C8E93`	steps/epochs/dataset/checkpoint path/memory/load report
`runs/.../ds_config_effective.json`	`2BF8216F04391CE4323FC79AE74AAE9728E8B13F8D4057F281734A0A4DA54642`	global batch 1024、micro batch 128、ZeRO 0
`runs/.../artifacts/dataset_ready.json`	`B9AF4A9118F83F466DACD1CBE0AE7F56A37BA0AB932E50BB992392A0571CB38B`	rank-0 dataset-ready marker
`runs/.../artifacts/target_atomic_seen_action_normalizer.json`	`27D2E8E24B3FDD43A841597D0CF3CF9CF322829FB28A26B663012338B1E48500`	12D action normalizer
`sweep_summary.json`	`F4426789264BD5165363AD3F416B574C4C4ADB463D434C7C2182D428B835371F`	八组 rollout aggregate
`sweep_summary_codex_verified.json`	`3AD967659D8208C2B67F0AE421E29353317BF067792878AEA14EF883B853BFC6`	tool-derived training/rollout reconciliation；不替代 raw files
`checkpoint_smoke.json`	`BC95D7EEFC162D7A0BEE432424FF80B982B05562A5D6755A44EC8528B99C39F4`	base UWM partial-load / reset-head preflight
`dataset_smoke.json`	`63A6DF7EE65C59560F2A58B2E73B13C8B52FBA91837FD1E3A7DE8B2D6681E681`	三视角、state/action shape 与 task sample smoke
`save_smoke.json`	`5132415BA944C4DB7BC0E8FA992B537675FC73EB6DC51E598340A5073FCD9442`	独立 10-step save smoke；不是本页 30-epoch binary

本地代码快照：`uwm_robocasa_train.py` SHA `EFC53BB652302183F56B6F1BB9ACEB7ABDE31D01D953720EA1C05BB605A91FFE`；`uwm_robocasa_rollout.py` SHA `E43D3751F8D079DAC54C902B5B222A13242FF5A9CE0F3975FB00D630DA53775E`；evidence collector SHA `A0651BC6E1569A48D48F723770C86EC4FEF0290D62F89F7D32ACD5684856CC67`。Project HEAD 为 `6c524d9c6c6226075f100cdac395fef2fdc37671`，但 `remote_sync` 是本地缓存，commit 与路径都不能自动证明 candidate ownership。

4.2 Raw rollout files

setting	raw JSON SHA-256	bytes
full_h1	`7FECC56D94A5BEBB44E78944015351F5E4B8EA7EFC684306204595B9F5EF5E44`	197952
full_h2	`31310DB9852B4FF49F393C8066CFC71C713C639BEA57C23FCE38F561D1C4D2E7`	200613
full_h4	`7284F7F3E713F1987A74041EA51B48B8EB8E7510A7E296CE695FBE1EA89B287E`	200242
full_h5	`0CE505AA8D820B9F59432CC90C54869D691525FC726A183DAB234A6B827A0156`	193888
full_h8	`7E9A6257D8993397E167EB6E2716A6C6558BFE334CA7CE3EDC627D7D0810E458`	196528
no_prompt_h5	`A6CBE854FA7E3A06671EA148B82AE46AF702FBB5BCEF28130F7D58F5D4741B77`	193775

setting	raw JSON SHA-256	bytes
no_right_h5	`F4A8D28921ABB82430D88BE85B1DA8BB706C2D9C470583A72C45CC3EBBDA2557`	194197
no_state_h5	`933DBFDBA2DC1D67C6BE7E9FA18226DDE17BA80086C42511959EA2224B7BB740`	198679

5. Preflight 与训练合同

5.1 Base-load / reset-head contract

base checkpoint 为远端 `libero_90.pt`、checkpoint step 100000。preflight 显示 `target_key_count=478`、`matched_count=446`、`skipped_count=8`、`unexpected=0`；被跳过的 8 个 key 是 7D→12D action encoder/decoder 的 weight/bias，且 `reset_action_head=true`。新增 task/prompt/state/right-view projection、action head 和 gripper phase head 共 32 个 missing-after-partial-load key，normalizer 也不从 base checkpoint 继承。

这说明 actionheadfix 的基本迁移逻辑是：保留能 shape-match 的 UWM trunk，重置 12D action I/O 与新增 conditioning 模块。它不证明这些新参数经过 30 epoch 后产生了稳健策略。

5.2 Dataset / observation / action contract

字段	值
dataset	LeRobot `target_atomic_seen`
tasks / episodes / sequences	18 / 9126 / 133457
seq_len / stride	19 / 16
images	agentview + right-view + eye-in-hand，4 frames，224×224
state	16D：EEF relative pos/rot、base pos/rot、gripper qpos
action	12D，indices `5,6,7,8,9,10,11,0,1,2,3,4`
condition	task ID + text prompt + state + right view
weighted loss	EEF-pos 2.0、EEF-rot 1.0、gripper 3.0、base 1.5、control 1.0
train batch	8 GPUs × micro-batch 128 = global batch 1024
save policy	epoch 30，limit 1，remote `models.pt` path

normalizer 的一个重排维度 `std=0`、scale=`1e-6`；这与源数据该维度恒为 0 相符，但它是重要的 action-contract 风险，不能在 adapter 或 action-order 改动时静默复用。

6. 训练完成与 metric 边界

`train.out` 可独立解析出 41 条打印 metric point：step 0、每 100 step 和 step 3929。首末值为：

metric	step 0	step 3929
total loss	3.325663	0.253850
action loss	2.430044	0.076886
raw action loss	0.998558	0.030433
epoch	0.0076	30.0

日志末尾 8 个 rank 都是 `exits successfully`，没有 traceback 或 CUDA OOM。`sweep_summary_codex_verified.json` 另称读取了 3930 个 training record，并给出 tail-100 average；但原始 `runs/.../logs/train.jsonl` 没有同步到当前 snapshot，因此本页不把 tail-100 值升级为独立重算结果。

更关键的是，训练命令设置 `val_ratio=0`、`eval_every=0`。所以这张表只支持“optimizer 在训练分布上显著降低 loss”，不支持 held-out action accuracy、OOD generalization 或 closed-loop improvement。

7. Closed-loop protocol tuple

八组 raw JSON 的共同 tuple 为：

```
task_set=atomic_seen
split=target
```

```
tasks=18
scenario_sampling=round_robin
scenarios=54 (3/task)
scenario/seed set identical across all 8 settings
action_mode=official12
adapter_mode=official12
action_execution_mode=convert_action
checkpoint=<same remote models.pt path>
normalizer=<same local-hashable 12D normalizer path>
env_errors=0
```

full settings只改变 `exec_horizon ∈ {1,2,4,5,8}`；三组 h5 ablation 分别屏蔽 prompt、right-view 或 state input。因为同一组 54 scenario 被重复评估，所以 432 records 不是 432 个独立 benchmark trial，不能把各 setting 的 success 简单相加。

8. 八组 raw rollout 独立重算

setting	horizon	success / 54	env errors	robot-env contact episodes	成功 task / seed
full_h1	1	2	0	42	TurnOnElectricKettle / 15, 33
full_h2	2	1	0	47	TurnOnElectricKettle / 33
full_h4	4	0	0	45	—
full_h5	5	1	0	43	TurnOnMicrowave / 34
full_h8	8	1	0	45	TurnOnElectricKettle / 51
no_prompt_h5	5	2	0	46	TurnOnElectricKettle / 15, 33
no_right_h5	5	2	0	45	TurnOnElectricKettle / 15, 33
no_state_h5	5	1	0	43	TurnOnElectricKettle / 15

每份 raw JSON 都满足：54 records、18 unique tasks、每 task 3 records、54 unique seeds；field `num_successes` 与逐 record recount 完全一致；per-task aggregation 与 micro/macro summary 一致；八份 JSON 的 scenario/task/seed set 完全相同。

8.1 为什么不能声称 ablation 结论

full_h5 是 `1/54`，no-prompt/no-right 是 `2/54`，差值只有一个 episode，而且成功集中在单一 TurnOnElectricKettle task。缺多 seed repeat、checkpoint repeat 和置信区间时，不能说“去掉 prompt/right view 提升性能”，也不能据此证明这些输入无用。它只告诉我们：当前 success 稀疏，ablation signal 被单任务和单 episode 主导。

8.2 为什么 contact 不是 success

full_h4 的 45 个 episode 至少一次记录到 robot-env contact，但 54 个 episode全部失败。contact pair 可能是手、link、柜门、台面等几何接触；它没有包含任务谓词、目标物体状态或有效抓取/按键语义。因此接触率只能作为“策略是否靠近交互区域”的诊断 proxy，不能替代 RoboCasa success。

9. Train → Closed-loop first divergence

当前最稳健的链路判断是：

```
base partial load passes
  -> 12D/action-state-view dataset smoke passes
  -> 30-epoch 8-GPU training exits cleanly
  -> training action/raw-action loss decreases
  -> same checkpoint and normalizer load into 8 rollout settings
  -> env execution has 0 exceptions
  -> task success remains 0–2 / 54 and concentrates in 1–2 tasks
```

这将 first divergence 收窄到“训练目标/teacher-forced action fit 到 long-horizon closed-loop control”之间，但不能唯一归因于 action head。仍可能包括：action semantics/normalizer 的细微错位、diffusion sampling误差、exec horizon、observation/state distribution shift、controller dynamics、gripper phase、recovery 缺失与 runtime scorer sensitivity。

10. Run-family 与 ownership 边界

相邻证据	与本页关系	不允许的 join
Human300 pretrain / target continuation	另一 curriculum/run family	不把其 loss 或 checkpoint 当成本页 base/结果
100-epoch resume	另一 checkpoint 与训练阶段	不用 `0/54` 或其它结果替代本页 30 epoch sweep

相邻证据	与本页关系	不允许的 join
state_prompt_weighted strict/direct A/B	另一 run 和 action bridge	不把 `expected 12, got 5` 错误混入本页零 env-error denominator
LIBERO fixed-init `186/250`	user-confirmed remote summary、不同 benchmark	不与 RoboCasa 54-scenario success 比较
`sweep_summary_codex_verified.json`	工具生成 reconciliation	不把工具生成文件当 candidate ownership 或 raw source

当前能认领到项目层的是：这条训练/评测运行发生过，run tuple 与失败断层可复核。个人“亲手实现/设计/运行/验收”的具体边界仍需用户按脚本、command、remote operator、analysis 和 reviewer 逐项确认。

11. 面试可用口径

30 秒版本

> 我在 UWM→RoboCasa 迁移里做过一条 actionheadfix 30-epoch run 的训练—评测审计。它把 LIBERO 7D action head 重置成 RoboCasa official12，并加入 task、prompt、16D state 和第三视角条件；8 卡跑完 3930 steps，训练 action loss 从打印点的 2.43 降到 0.0769。之后我固定同一 checkpoint、normalizer 和 18 task×3 scenario，扫了 h1/2/4/5/8 以及三组输入 ablation。八组都没有环境异常，但每组只有 0–2/54 success，h4 是 0/54；而 h4 有 45/54 episode 出现接触，说明接触和训练 loss 都没有转化为稳定任务完成。我把它定位为 train-to-closed-loop transfer failure，不宣称完成 RoboCasa benchmark。

高压追问：训练 loss 降这么多，为什么 rollout 还是几乎 0？

> 因为训练侧没有 validation，loss 是 teacher-forced 训练分布指标；closed-loop 会累积 observation shift、sampling error 和 controller/action-contract误差。当前证据已经排除了“训练进程没跑完”和“env 全部 crash”，但还没有排除 normalizer/action semantics、exec horizon、gripper phase和 recovery。正确下一步是固定一个 failure scenario，对 expert action、model x0、sampled 12D action、adapter 后 action和环境状态做逐步 first-divergence，而不是继续只压训练 loss。

高压追问：no-prompt 比 full-h5 高，是不是 prompt 有害？

> 不能。`2/54` 对 `1/54` 只差一个 episode，成功集中在 TurnOnElectricKettle，且只有一个 checkpoint/run。它可以作为下一轮 matched repeat 的假设，不能作为 prompt causality 或 ablation uplift。

禁用表述

1. “actionheadfix 已解决 RoboCasa transfer”；
2. “30 epoch 模型在 RoboCasa 达到 X% benchmark”——当前只是 18-task×3-scenario 项目协议；
3. “训练 loss 下降证明泛化提升”——没有 validation；
4. “prompt/right view 有害”——成功数过少且无 repeat；
5. “checkpoint 已在本地验证”——只有远端 path，本地没有 binary/hash/fresh reload；
6. “所有代码和实验都是我独立完成”——ownership 尚未确认。

12. 关闭 P0 的最小动作

1. 回传或登记 30-epoch `models.pt` 的 immutable SHA/ETag，做 fresh-process load 与 fixed-input parity；当前 snapshot checkpoint-like binary 为 0。
2. 同步完整 `train.jsonl`，独立重算 3930 training records；增加 held-out split 或固定 teacher-forced eval，避免只看训练点。
3. 冻结 RoboCasa、MuJoCo、controller、success predicate 与 env revision；保存 54 episode 的视频或可审计 locator。
4. 对相同 54 scenario、多个 checkpoint seed 重复 h5/full/no-prompt/no-right/no-state，报告 paired difference 与置信区间。
5. 选择 full-h4 的 matched failure case，记录 expert/model-x0/sampled/converted/executed action、state、contact 和 task predicate 的逐步 first-divergence。
6. 逐文件确认候选人对 train/rollout/dataset/evidence collector、远端运行和结果审查的 ownership；工具生成 summary 单列。

在这些字段闭合前，本页保持 `PARTIAL/WARN`：允许讲完整训练—评测失败链和协议工程，不允许讲 RoboCasa benchmark 已完成或训练取得稳定成功。

References

1. `C:\Users\mousongzhe\Project\remote_sync\muxi_20260629\uwm_robocasa_transfer_muxi\logs\target_atomic_seen_official12_task_30epoch_muxi_20260628_openpi_state_3view_phase_30ep\train.out`。
2. `C:\Users\mousongzhe\Project\remote_sync\muxi_20260629\uwm_robocasa_transfer_muxi\runs\target_atomic_seen_official12_task_30epoch_muxi_20260628_openpi_state_3view_phase_30ep\` 下的 summary/config/normalizer。
3. `C:\Users\mousongzhe\Project\remote_sync\muxi_20260629\uwm_robocasa_transfer_muxi\logs\rollout_sweeps_atomic_seen_target\target_atomic_seen_official12_task_30epoch_muxi_20260628_openpi_state_3view_phase_30ep\` 下八份 raw rollout JSON 与 eval log。
4. `C:\Users\mousongzhe\Project\remote_sync\muxi_20260629\uwm_robocasa_transfer_muxi\diagnostics\openpi_state_3view_phase_20260628\` 下的 checkpoint/dataset/save smoke。
5. UWM / LIBERO Claim–Evidence Inventory。
6. UWM / LIBERO Po Bundle Audit。
7. UWM / LIBERO / RoboCasa Transfer Evidence Freeze。
8. UWM Project Cards。

Part 22 / 49 · `interview_materials/vla/sources/uwm_libero90_checkpoint_immutable_manifest_gate.html` · SHA-256 `A89C15D72D4504253E332052AD6A0B1A8F87D0DD26FB3201235359B2C7BFE554`

返回本卷目录

Canonical Markdown SHA-256: `a89c15d72d4504253e332052ad6a0b1a8f87d0dd26fb3201235359b2c7bfe554`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\sources\uwm_libero90_checkpoint_immutable_manifest_gate.md`

UWM `libero_90.pt` 本地 Immutable Checkpoint Manifest Gate

状态: `local-binary-verified / base-checkpoint-only / fine-tune-lineage-open / WARN` 审查日期: 2026-07-21; 证据截止: 2026-07-21 范围: `C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer` 中 UWM 官方 LIBERO base checkpoint 与加载脚本。 结论先行: 本次在用户提供的 project copy 中发现并重新哈希了 `libero_90.pt` 本体; 它是一个可读取的 4.04 GB PyTorch archive, SHA-256 为 `E43C562E1D5145662006877AAC0FB30C3C5D57302AEFC16524DA2D862D694602`。归档内部可静态确认顶层包含 `model`、`optimizer`、`scaler`、`scheduler`、`action_normalizer`、`lowdim_normalizer` 与 `step=100000` 等字段, 且 `action_normalizer` / `lowdim_normalizer` 的值为 `None`。这关闭了"base binary 只有远端路径引用"的旧缺口, 但不证明 RoboCasa 微调后的 `models.pt`、normalizer、fresh-process reload、评估 scorer 或个人 ownership。

1. Question 与证据边界

本页只回答四个可审计问题:

1. 当前 project copy 是否真的包含 UWM `libero_90.pt` binary?
2. 文件的字节数、SHA-256、容器结构和 checkpoint step 是什么?
3. 它能支持"加载了哪个 base checkpoint"的哪一级面试表述?
4. 哪些仍不能从 base binary 推断到 RoboCasa fine-tune 或结果 claim?

本页把四层证据分开:

层级	本页材料	允许的结论
`local-direct`	`.pt` 文件、zip member inventory、`data.pkl` 静态 opcode、下载脚本与 README	文件存在、hash、容器字段、上游路径/用途说明
`upstream/recipe`	`download_uwm_libero90_checkpoint.sh`、UWM transfer README	该文件被项目当作官方 UWM LIBERO base 输入; 下载来源与命名合同
`local-inference`	`models_100001`、`step=100000` 与字段结构的共同解释	这是可用于迁移初始化的训练 checkpoint, 而非只含裸模型权重的 state dict
`unverified`	RoboCasa `models.pt`、normalizer JSON、fresh reload、episode scorer、ownership	不能把 base checkpoint 的 hash 当作 fine-tune 或 benchmark 证据

2. Immutable manifest 合同

对 checkpoint artifact (c), 最小可复核记录定义为:

$$M(c) = \big(\text{path},\ \text{bytes},\ \text{sha256},\ \text{container},\ \text{logical_step},\ \text{role}\big).$$

变量解释: `path` 是 project copy 内的相对路径, `bytes` 是文件长度, `sha256` 用于检测替换, `container` 是 PyTorch archive 的成员摘要, `logical_step` 是 checkpoint 内记录的训练步, `role` 区分 base、fine-tune、eval-only 等身份。直觉上, 只有这些字段一起冻结, 面试中才可以回答"server/loader 实际拿到哪一个文件", 而不是只复述一个字符串路径。此处的 `logical_step` 与 `role=official UWM base` 仍需和上游发布记录一同解释, 不能单独证明训练 provenance。

```
def verify_checkpoint(path, expected_sha256):
    # 1. 只读计算文件 hash; 不覆盖、不重新保存、不改变用户 project copy。
    digest = sha256_file(path)
    assert digest == expected_sha256

    # 2. 只列出 PyTorch zip members 与 data.pkl; 不执行 pickle 中的任意对象构造。
    members = list_zip_members(path)
    metadata = disassemble_pickle_without_unpickling(path, "models_100001/data.pkl")

    # 3. 把 base binary、fine-tune checkpoint 和 normalizer 分开登记。
    return {"sha256": digest, "members": members, "metadata": metadata}
```

3. 本地文件 manifest

根目录：`C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer`。

ID	相对路径	字节数	SHA-256	角色	状态
C01	`_downloads/libero_90.pt`	`4,044,929,076`	`E43C562E1D5145662006877AAC0FB30C3C5D57302AEFC16524DA2D862D694602`	UWM LIBERO base checkpoint binary	`local-direct`
C02	`_downloads/libero_90_chunk7.bin`	`268,435,456`	`13B10DFCEF838A3AF30EAF2BF21CE7A306BB31A9B62AD5EA63A60BFA8427596D`	下载过程中的独立 chunk artifact；不宣称可重组为 C01	`local-direct / role-open`
C03	`download_uwm_libero90_checkpoint.sh`	`697`	`0C18903D55BB25E1EAFE8731B7770A5BBCCF9E4B5178985CC3DE0362B21F8DCE`	下载与软链接合同	`local-direct`
C04	`README.md`	`6,087`	`E05378FC0E1718B073635EFB5159B9FF7DBB0EFB50C817FE7C2F35E1E57C63D0`	项目用途、checkpoint smoke 与 hash 文字记录	`local-direct`

文件时间只作为辅助字段：C01 最后修改时间为 `2026-06-22 11:07:14 UTC`。时间戳不能替代 hash，也不能证明该文件被某一次 RoboCasa rollout 实际加载。

3.1 Project-wide negative scan

为避免“只看了一个已知目录”造成假阴性，本次还在 `C:\Users\mousongzhe\project` 全树按扩展名扫描 `.pt/.pth/.ckpt/.safetensors/.bin`。结果只有 C01 与 C02 两个文件：

扫描结果	数量	解释
`libero_90.pt`	1	UWM base binary，C01
`libero_90_chunk7.bin`	1	独立下载 chunk，C02；未证明可重组
`models.pt` / `checkpoint_step_*.pt`	0	当前 project copy 中没有可直接冻结的 RoboCasa target checkpoint
`.safetensors` / `.ckpt` / `.pth`	0	未发现其他常见 checkpoint 格式

```
# 只读负向检索：不把日志中的字符串或目录名误当作 binary。
Get-ChildItem $project -Recurse -File |
  Where-Object { $_.Extension -in '.pt','.pth','.ckpt','.safetensors','.bin' }
```

这个负向结果只对当前 Windows project copy 有效；它不能排除远端 `/data/msz/...` 或其他未挂载目录中存在 target checkpoint。因此安全结论是“本地 target binary 尚未归档”，不是“target checkpoint 从未生成”。

4. Container 与静态 metadata 复核

`libero_90.pt` 以 PyTorch zip archive 开头（magic bytes `PK\x03\x04`），共 `1070` 个 member，唯一顶层目录为 `models_100001/`。本次没有调用 `torch.load` 去构造完整 4 GB tensor；而是读取 `data.pkl` 并用 `pickletools` 做不执行对象的 opcode inspection。这样能确认字段名与 storage metadata，同时避免把静态检查误写成模型 forward 或 fresh-load 证明。

检查	结果	可支持的结论
archive header	`PK\x03\x04`	文件是 zip-based PyTorch serialization
member count	`1070`	归档含大量 tensor storage，不是空壳路径
top-level prefix	`models_100001/`	文件内部目录名与 step 命名一致
`data.pkl` bytes	`182,415`	metadata pickle 可独立读取
tensor-key strings	`453`（unique `452`）	约 452 个带 weight/bias/running-stat 名称的 tensor key 字符串；不等同于 loader report 的 model-key count
optimizer / scaler / scheduler	字段字符串存在	归档不仅有 model 字段；是否按某次 run 恢复仍需 loader log
`last_epoch`	`100001`	scheduler metadata 中存在该值
`step`	`100000`	顶层末尾 metadata 记录该逻辑 step
normalizer fields	`action_normalizer=None`、`lowdim_normalizer=None`	base binary 不提供可直接继承的 target normalizer

这里的“字段存在”来自 pickle opcode，不是把所有 tensor 载入 CPU 后的运行时对象 dump；因此它是 metadata-level evidence。项目 README 另记录了一次 checkpoint smoke：`454/454` 个 shape-compatible model keys 被加载，并按设计跳过两个 normalizer 字段。[1]

[2]

5. Base checkpoint 与 RoboCasa fine-tune 的身份分离

当前身份元组应写成：

```
base = (
  role="official UWM LIBERO base",
  path="_downloads/libero_90.pt",
  sha256="E43C...694602",
  logical_step=100000,
  action_normalizer=None,
  lowdim_normalizer=None
)

target_finetune = (
  role="RoboCasa target fine-tune",
  checkpoint_path="not frozen in this gate",
  normalizer_path="run-specific JSON, not frozen here",
  scorer="not frozen here"
)
```

因此，base binary 的 SHA 可以证明“这份本地 UWM 初始化文件没有被无声替换”，但不能证明：

- RoboCasa `models.pt` 是由它初始化并成功保存的；
- `--reset-action-head`、7D→12D action adaptation 或 task/prompt/state/right-view head 采用了哪个 exact config；
- fine-tune 后的 normalizer 与 checkpoint 属于同一 run；
- rollout server 在 fresh process 中重新加载了同一 binary；
- `0-2/54` 或 `0/54` 等闭环结果来自可复放且同一 scorer 的 checkpoint；
- 上游 UWM architecture、迁移 patch 或完整 run 是候选人独立原创。

6. 与既有 evidence gate 的 reconciliation

旧的 `vol_05_local_artifact_gate_eng004_006` 以当时可见的 report snapshot 为依据，曾把 `libero_90.pt` 标成“只有远端路径引用”。本次新证据位于用户明确提供的 `project` copy，且包含 C01 binary；因此该旧结论应改写为：

Claim	修订前	当前修订	仍未闭合
base checkpoint binary	remote-path-only	`local-direct`，C01 hash 可复算	上游发布 revision、下载来源与 loader run UUID
base model key load	report reference	README + loader code 支持 `454/454` smoke claim	一次带 stdout/summary/hash 的 fresh rerun
target fine-tune checkpoint	unverified	仍 `unverified`	`models.pt` binary/hash、normalizer、config、fresh reload
closed-loop benchmark	unverified	仍 `unverified`	episode JSONL、scorer/version、denominator、matched protocol
ownership	unverified	仍 `unverified`	逐文件个人/协作/工具辅助确认

这不是把 base checkpoint 误升级为 RoboCasa 结果；它只修正了“本地是否有 base binary”这一项。

7. 面试安全口径

30 秒回答

“我会把 base checkpoint 和 fine-tune checkpoint 分开验。当前 project copy 里确实有 UWM 的 `libero_90.pt`，文件大小是 4,044,929,076 bytes，SHA-256 是 `E43C562E...694602`；我还静态检查了 PyTorch archive，确认它内部是 `models_100001`、逻辑 step 100000，并且 action/lowdim normalizer 在 base 中是空值。项目 README 和 loader report 记录过 shape-compatible smoke，但这只能证明 base 初始化文件和加载合同，不能证明 RoboCasa 的 `models.pt`、normalizer、fresh-process reload 或 benchmark scorer。后者必须用 run-specific checkpoint hash、effective argv、episode bundle 和 ownership 再闭合。”

高频追问

1. **为什么 `summary.json` 的 checkpoint 字符串不够？** 因为字符串没有绑定文件 hash、加载进程、normalizer、代码 revision 和 run UUID；它可能只是传入参数或上游路径。
2. **base hash 是否证明微调成功？** 不证明。它只冻结初始化文件；微调后的 binary 和 target normalizer 是另一条 lineage。
3. **为什么不直接 `torch.load` 作为验证？** 4 GB 文件的静态 manifest 足以先验证 identity；完整 load 还需要记录 device、代码 revision、shape-match report 和 fresh-process output parity。

4. **你能说“我训练了这个 checkpoint”吗？** 只有在 run manifest、保存产物、日志、加载复现与 ownership 逐项闭合后才能用更强措辞；当前安全说法是“按 UWM base checkpoint 开展迁移/训练尝试并审计加载合同”。

8. Gate 状态与下一步

Gate	状态	下一最小补件
C01 base binary existence/hash	`PASS`	保留 SHA 与 byte count；不移动原文件
C01 archive metadata	`PASS WITH WARNING`	如需更强结论，保存一次 trusted loader 的 CPU shape report
base→RoboCasa fine-tune lineage	`WARN`	`models.pt`、exact config、normalizer 与 shared run ID
fine-tune fresh-load parity	`UNVERIFIED`	新进程 reload + parameter/schema parity
closed-loop scorer/denominator	`UNVERIFIED`	episode JSONL、scorer revision、task/init/seed tuple
ownership	`PENDING`	候选人逐文件标注亲手/协作/工具辅助/只运行/只审查

本页不新增题卡，不改变任何 `unseen` 状态，也不把 C01 的 base hash 写成 RoboCasa 成绩。现有 `PROJ-UWM-*` 与 RoboCasa ActionHeadFix 题卡继续保持 `unseen`。

参考文献（References）

1. 本地项目 README：`C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer\README.md`（C04；含 base hash、checkpoint smoke 口径与运行合同）。
2. 本地加载实现：`C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer\uwm_robocasa_train.py`（`torch_load_checkpoint`、`load_shape_compatible`、`checkpoint_smoke`）。
3. 本地下载合同：`C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer\download_uwm_libero90_checkpoint.sh`（C03）。
4. 本地 base binary：`C:\Users\mousongzhe\project\wm\uwm_robocasa_transfer\_downloads\libero_90.pt`（C01；SHA-256 由本次只读扫描重算）。
5. UWM/LIBERO/RoboCasa transfer evidence freeze。
6. UWM/LIBERO project claim inventory。
7. UWM RoboCasa ActionHeadFix P0 audit。

Part 23 / 49 · `interview_materials/vla/sources/vol_05_uwm_libero_robocasa_transfer_evidence_freeze.html` · SHA-256 `6FA38801E0205BF9F55ABE81F0C9CA267F6E33C0CDD088CD825DAA8760CC1627`

返回本卷目录

Canonical Markdown SHA-256：`6fa38801e0205bf9f55abe81f0c9ca267f6e33c0cdd088cd825daa8760cc1627`；生成自：`interview_materials\vla\sources\vol_05_uwm_libero_robocasa_transfer_evidence_freeze.md`

VOL 05 UWM / LIBERO / RoboCasa Transfer：证据冻结

状态：`local_protocol_audited / user_confirmed_remote_final / robocasa_debugging / WARN` 复核日期：2026-07-21 证据来源：`C:\Users\mousongzhe\Project\wm` 与 `C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702`（外部工作区只读审计） Bounded slice：本页冻结 UWM 官方 LIBERO reproduction、RoboCasa-native transfer/actionheadfix 训练与闭环评估的协议、指标、修复和 provenance；不把用户确认远端摘要升级为本地 raw benchmark，不改变题卡 mastery 或 ownership。

1. 结论先行

这个项目最适合在面试中讲成“训练—评估协议工程与失败诊断”，而不是单纯报成功率：

1. UWM/LIBERO 路线已经有本地脚本、任务映射、`pruned_init`/reset 协议开关、5-step warmup、8-shard launcher 和 DataLoader-only smoke；
2. 用户确认的 strict fixed-init 结果为 `186/250=0.744`，但 JSON 明确标记为 `user-confirmed remote final`，本地没有 checkpoint、normalizer、250 条 episode JSONL 或 scorer，不能称 local raw reproduction；
3. 旧 `232/250=0.928` 使用 `env.seed(e)+env.reset()` 的 reset-style 协议，不能与 strict fixed-init 或 UWM Table II 直接比较；
4. RoboCasa actionheadfix 使 action-head 初始化、open-loop监督和trace诊断更可信；新增raw audit对一条30-epoch openpi-state-3view run重算：full h1/h2/h4/h5/h8为`2/54、1/54、0/54、1/54、1/54`，三组h5 input ablation为`2/54、2/54、1/54`，全部env errors为0；历史30/100-epoch结果属于相邻run，不能混算；
5. 严格 `convert_action` A/B 为 `0/6`、6 个环境错误（`expected 12, got 5`），说明 OpenPI-style dict 不能直接喂给当前 native RoboCasa env；GT bridge replay 的最大绝对差为 `0.0`，但模型闭环仍失败。

2. 证据等级与 provenance

层级	当前材料	能支持	不能支持
`local-direct`	本地脚本、repo patch、报告、commit、DataLoader smoke	task map、action/state shape、协议开关、修复逻辑、日志字段	远端权重内容或完整 episode 结果
`report-supported`	`uwm_robocasa_libero_reproduction_log_report.html`	Book-Caddy `42/50`、训练曲线片段、RoboCasa run ledger	本地可重算的 checkpoint/episode bundle
`user-confirmed-remote-final`	`final_strict_eval_user_confirmed_20260705.json`	5-task per-task summary 与协议说明	原始 250 episode、checkpoint hash、normalizer/scorer
`remote-snapshot-raw`	actionheadfix 30-epoch `train.out` + 8×54 rollout JSON	run completion、paired scenario set、success/env-error/contact recount	produced checkpoint binary/hash、env/scorer revision、videos、ownership
`working-tree-evidence`	`sync_manifest.json`、`sync_summary.md`	远端断联时最后状态、脚本 hash、dirty/未跟踪文件边界	将 recovery summary 变成 committed raw result
`local-inference`	action/contact/协议对照	对失败原因提出可检验假设	已确认的唯一 root cause
`unverified`	RoboCasa benchmark success、personal end-to-end ownership	仅列为待补证	任何强 C3 结果或原创架构声明

外部 Project 当前 HEAD 为 `6c524d9c6c6226075f100cdac395fef2fdc37671`（`Document RoboCasa actionheadfix diagnostics`）。工作树仍有：`sync_manifest.json`、`sync_summary.md` 修改，以及 `final_strict_eval_user_confirmed_20260705.json` 未跟踪。它们必须分别标记为 committed、working-tree、user-confirmed 和 raw-local，不可混为一个“完整复现实验”。

3. 官方 Baseline 与 UWM/LIBERO 结果对照

3.1 UWM/LIBERO 任务与协议

本地 `libero_official_common.py`、`uwm_libero_rollout_eval.py` 和 parallel launcher 固化了五个 LIBERO-10 任务：`book_caddy`、`soup_cheese`、`bowl_drawer`、`moka_moka`、`mug_mug`。项目 strict protocol 为：

字段	fixed-init 项目口径
initial state	LIBERO `*.pruned_init`，按官方顺序读取
warmup	5 个 zero-action steps
rollouts	50/task，5 task，共 250
rollout length	policy steps 600
launcher	8 shards / GPUs 0–7
success	environment `info["success"]` 二值信号
provenance	用户确认远端 summary；raw episode 未同步

SR（success rate）对 (N) 条 rollout 的定义为：

$$\mathrm{SR} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}[\mathrm{success}_i].$$

其中 (N) 是固定 protocol 下的 episode 数，(mathbf{1}[cdot]) 来自环境 success predicate。实现对应 `uwm_libero_rollout_eval.py` 的 per-episode summary 和 task-level aggregation；没有 episode JSONL 时只能信任 summary 的 provenance，不能在本地重新计算。

3.2 结果 reconciliation

口径	结果	证据等级	解释
Book-Caddy 单任务 report	`42/50=0.84`	`report-supported`	单任务复现实验，不是完整 LIBERO benchmark
UWM reset-style	`232/250=0.928`	`user-confirmed context / protocol-diagnostic`	`env.seed(e)+env.reset()`，随机 reset 分布
fixed-pruned-init strict	`186/250=0.744`	`user-confirmed-remote-final`	5-step warmup、50/task；本地无 raw bundle
UWM Table II reference	约 `0.79`	`upstream-verified`	OOD init、+0.03 object shift、背景移除、3 seeds

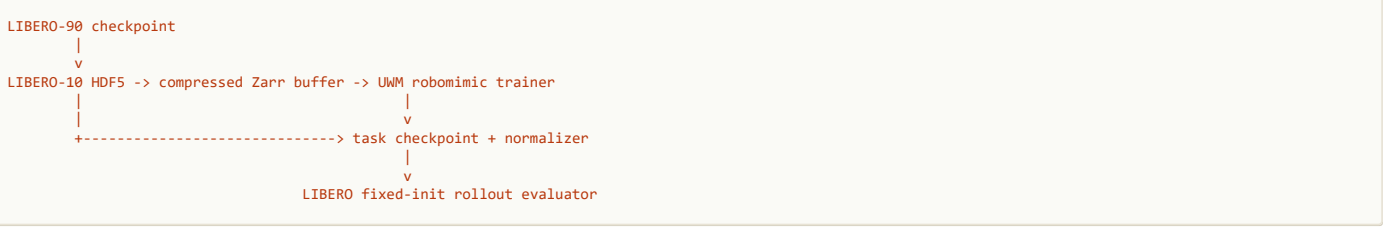
strict fixed-init per-task summary：

Task	Strict result	Assumed denominator
`libero_book_caddy`	`0.88`	44/50
`libero_soup_cheese`	`0.80`	40/50
`libero_bowl_drawer`	`0.76`	38/50
`libero_moka_moka`	`0.66`	33/50
`libero_mug_mug`	`0.62`	31/50
total	`0.744`	186/250

不能把 `0.744` 与 Table II 的 `0.79` 写成严格复现差距：两者的 initial-state distribution、object shift、background removal、seed 数和训练口径不同。`0.928→0.744` 的 `-18.4pp` 首先是 protocol sensitivity，不是同 checkpoint 的模型退化实验。

4. UWM Base I/O 与 LIBERO route

UWM 上游是共享 diffusion transformer，同时预测 action chunk 和未来观测/视频 latent，action/video 使用独立 diffusion timestep。项目 adapter 使用两路 RGB、7D action metadata 与 task-specific UWM wrapper；这属于 UWM 公有机制与本地接口迁移的组合，不应混成 Cosmos/WAM 的 action/world/value 三角色。



本地 evidence 支持的输入/输出边界是：两路 `128×128` RGB、proprio/state contract、7D action chunk，经过 UWM model builder 和 normalizer 后由 environment wrapper 执行；具体 action horizon、normalizer 文件和远端 checkpoint binary 仍需要 raw bundle 才能升级为 C3。

5. RoboCasa action-space 与 head 改造

RoboCasa native full action 是 12D：EEF 7D、base 4D、control 1D。actionheadfix 线将 UWM 的 7D EEF 输出与 RoboCasa 原生字段拆分、重排并加入 hierarchical decoder。可用的抽象映射为：

$$a^{\text{env}} = \text{concat}\left(a_7^{\text{eef}}, a_4^{\text{base}}, a_1^{\text{control}}\right),$$

其中 a_7^{eef} 是模型的连续末端动作，a_4^{base} 和 a_1^{control} 来自额外 head 或统计初始化。实现证据位于 `uwm_robocasa_train.py`、`uwm_robocasa_rollout.py` 与 `NativeRoboCasaDictActionWrapper`；它证明了接口设计和 trace 字段，不证明闭环成功。

actionheadfix 的训练侧改动：

改动	证据	作用	边界
partial decoder init	commit `a37a386`；`checkpoint_load_report`	从 `libero_90.pt` 复制 shape-compatible decoder，避免 12D head 全部从零学	不是完整 checkpoint identity
action stats bias init	同上	base/control bias 写入 normalized action space	统计量 provenance 仍需固定
predicted x0 MSE	训练脚本与 `teacher_forced_target_atomic_seen.json`	从 noise prediction 还原 clean action，再按真实 action 单位诊断	teacher-forced 不等于 rollout
group x0 metrics	eef/base/control/gripper 分组	暴露总 loss 掩盖的通道错误	不自动证明控制策略有效
auxiliary action override	rollout script	将 gripper/control auxiliary 参与最终 action 组装	可能引入执行时序副作用
h4 sharding	commit `1a03fe9`	54 scenario round-robin 分片后合并	sharding 不改变任务协议

checkpoint load smoke 的可核验摘要为：`matched_count=446`、`skipped_count=8`、`reset_action_head=true`、decoder partial init=true；报告同时记录早期 task embedding concat 会使 AdaLN condition dim 从 `3584` 变为 `3712`，随后改为 residual projection 以保持 trunk shape。不同 smoke 的 target-key 集合不同，`446` 不能和另一个 `196/258` smoke 直接比较。

6. RoboCasa Fine-tune 与 Eval Run Ledger

Run	数据/初始化	关键改动	训练证据	闭环结果	结论
early target pilot	`libero_90.pt` → target atomic	7D/12D 适配、task-id 等	report/README 有 run path	`0/3`、`0/3`、`1/3` task pilot	适配链可运行，非 benchmark
30 epoch task-id	target atomic seen	official12 + task-id，`exec_horizon=4`	checkpoint 与 run JSON 已记录	`1/54`，唯一成功为 `TurnOnElectricKettle`	仅 task-id 不足
100 epoch resume	30 epoch checkpoint 续训	继续拉 epoch	final summary：loss 与 checkpoint path	`0/54`，env errors `0`	更多 target epoch 未解决闭环
strict execution A/B	同 100 epoch checkpoint	direct vs strict `convert_action`	run-specific logs	direct `0/6`, errors `0`; strict `0/6`, errors `6`	OpenPI dict 不能直送 native env
Human300 pretrain	`libero_90.pt` → RoboCasa Human300	3 epoch，5271 steps，12D	action stats count `29,106,226`；loss `95.21→0.2265`；action `2.96→0.0411`；x0 `88.80→0.0285`	pretrain 是 offline 训练，不是 success	action head/分布可学习性改善
actionheadfix 30-epoch openpi-state-3view	`libero_90.pt` partial load → target atomic	30 epoch，3930 steps，12D action、task/prompt/state/right conditioning	printed step0→3929 loss `3.3257→0.25385`；action `2.4300→0.07689`；raw `0.9986→0.03043`；`val_ratio=0`	paired 54-scenario sweep：h4 `0/54`，robot-env contact `45/54`，env errors `0`；其它setting `1–2/54`	训练fit未转化为稳定 closed-loop success；raw audit

6.1 Teacher-forced 诊断

在 256 action rows 上，报告记录：

指标	值
`x0_mse_total`	`0.0647`
`x0_mse_base`	`2.68e-6`
`x0_mse_control`	`4.47e-5`
`x0_mse_eef_pos`	`0.1489`

指标	值
`x0_mse_gripper`	`0.3093`

这说明 base/control 通道在 teacher-forced/open-loop 分布上更接近标签，而 EEF position 与 gripper 仍是高误差通道；它不能直接推出 simulator success。

7. Online Rollout 与 0-success 诊断

实际评估由 `uwm_robocasa_rollout.py`、`remote_muxi_eval_h4_sharded.sh` 和 env wrapper 组成，核心路径可抽象为：

```
def rollout_one_scenario(env, policy, task, replan_steps=4):
    # 1. Reset task and collect camera/state/prompt observation.
    obs = env.reset(task)
    # 2. Build model input and sample a normalized action chunk.
    chunk = policy.sample(build_observation(obs, task))
    # 3. Denormalize, reorder to native 12D, and apply optional aux override.
    env_actions = action_dict_to_native_array(denormalize_and_parse(chunk))
    # 4. Execute only the first replan_steps actions, then query again.
    for action in env_actions[:replan_steps]:
        obs, reward, done, info = env.step(action)
        if done or info.get("success", False):
            break
    # 5. Persist trace/video/success/error fields for this scenario.
    return info
```

对新冻结的30-epoch openpi-state-3view run，full-h4 raw JSON 的直接症状是：`0/54`、`num_env_errors=0`、`45/54` rollout 出现至少一次 robot-env contact。此前报告中的 `27/54` 属于未冻结相同run identity的历史摘要，不能替代本页精确recount。八份54-scenario raw JSON 已经提供完整config内denominator，但没有checkpoint binary、视频和immutable env/scorer revision；trace中的普通counter/link/hand 接触仍不能等价为有效抓取、按键或开门，也不能宣布唯一根因。

7.1 convert_action A/B

路径	结果	解释
native direct	`0/6`，env errors `0`	12D array 可以执行，但没有成功
strict OpenPI-style `convert_action`	`0/6`，env errors `6`	每条在 step 0 报 `expected 12, got 5`
GT action bridge replay	`bridge_max_abs_diff_max=0.0`	dict→native 12D bridge 数值一致；不能证明模型动作正确

因此 execution bridge 是需要固定的协议字段；strict path 的 6 个错误不能和有效 rollout 的 0 success 混合计算。

8. DataLoader Stall 诊断

UWM/LIBERO 本地 `benchmark_libero_dataloader_stall.py` 只构建 DataLoader，不建模型、不占 GPU。报告记录的 A/B：

设置	p95 wait
finite，workers=8，prefetch=2	`13.862 s`
infinite sampler	`13.183 s`
preload + infinite	`0.194 s`
bs128 preload + infinite	`0.209 s`

这支持“zarr 小 chunk 随机读取造成周期性等待”的工程假设，但不等于完整 GPU training throughput、显存或最终训练时间已提升。要升级结论，仍需 30–100 step matched GPU A/B 并记录 `data_wait_sec/train_sec/total_sec/memory/throughput`。

9. 公有设计、本地证据与个人边界

类别	当前可安全说法	禁止升级
公有上游	UWM shared diffusion、LIBERO recipe、Table II protocol	把 UWM architecture 说成个人原创
本地迁移	我迁移/整理了 UWM-LIBERO launcher、RoboCasa action mapping、strict/reset protocol 和诊断脚本	“独立完成全部 upstream patch”
本地诊断	我用 DataLoader-only A/B 定位 zarr wait，用 trace 观察 contact/夹爪/control 失配	“已证明唯一 root cause”
用户确认结果	可以报告 `186/250`，并明确 `user-confirmed-remote-final`	“本地已复现 74.4%”
RoboCasa	可以讲 30/100 epoch、actionheadfix 和 A/B 失败路径	“完成 RoboCasa benchmark”

推荐 60 秒回答：

> UWM 项目我主要负责训练到闭环评估的协议工程。我在 C500/MACA 上迁移 LIBERO-90 到多个 LIBERO-10 task 的 finetune/rollout，固定 pruned-init、warmup、normalizer、horizon、rollout 数和 success predicate，并用 DataLoader-only A/B 定位 zarr 小 chunk 的周期性等待。LIBERO `186/250` 仍是用户确认summary；RoboCasa则有一条可重算的actionheadfix 30-epoch raw sweep：8卡3930 steps完成，同checkpoint的八组54-scenario rollout都没有env error，但只有0–2 success，full-h4为 `0/54` 且 `45/54` 有robot-env contact。因此我把它定位为train-to-closed-loop contract debugging，而不是benchmark成功。

10. 待补 P0 与恢复清单

1. 获取 strict run 的 checkpoint binary/hash、normalizer、exact config、init index、250 episode JSONL、失败视频和 scorer。
2. 用同一 checkpoint 做 reset-style vs fixed-pruned-init paired evaluation，记录每个 episode 的 seed/init/horizon。
3. 运行 30–100 step GPU matched DataLoader A/B，验证 preload/infinite 对真实训练吞吐的影响。
4. 对 soup/moka/mug 失败做 video/action trace/contact 分类，区分动作语义、空间定位、接触和长时序 recovery。
5. 对 RoboCasa wrapper bridge 重新跑 run-specific eval，明确 direct、strict convert_action、wrapper-convert 三种结果不混算。
6. 完成 `OWN-UWM-001`：逐文件标注上游、本人、协作、工具辅助、只运行、只审查和 operator/reviewer。

References

1. UWM/LIBERO 本地复现报告：`C:\Users\mousongzhe\Project\wm\uwm_libero_official_repro\reports\uwm_robocasa_libero_reproduction_log_report.html`，SHA-256 `F5B268F2E7108DF548309BBA869E0099F46A6F4D5CE8A31742D002570317DB62`。
2. RoboCasa transfer/actionheadfix 报告：`C:\Users\mousongzhe\Project\wm\uwm_robocasa_transfer\uwm_robocasa_transfer_report.html`，SHA-256 `D0ADC3C46D824242E5F8E4D7D1F787CA09810FAA108CA8C79C4E7110A1B00518`。
3. Strict recovery manifest：`C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\sync_manifest.json`，SHA-256 `9077C45992BF6852F5A8F4FA28ED3735FA590817D227A62D4179CCF239358D5F`。
4. User-confirmed final：`C:\Users\mousongzhe\Project\report\uwm_libero_strict_eval_recovery_20260702\final_strict_eval_user_confirmed_20260705.json`，SHA-256 `7616111AE7C7CA35CB10E8788ADCB1E8837D9B6F656B49553B6C7AFCCD586D11`。
5. UWM project inventory：UWM/LIBERO Claim–Evidence Inventory；P0 gate：UWM/LIBERO P0 Bundle Audit。
6. UWM upstream source audit：UWM video–action claim-to-code；benchmark protocol：Benchmark evidence claim-to-protocol。
7. UWM paper：Zhu et al., “Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets,” RSS 2025；LIBERO official benchmark documentation；RoboCasa benchmarking documentation。
8. UWM RoboCasa ActionHeadFix 30-Epoch Train-to-Closed-Loop P0 Audit。

Part 24 / 49 · `interview_materials/vla/practice/openpivla_oral_model_answer.html` · SHA-256 `9A238E196843A505DCC28F09D96DA5794144B215594946DE5A67E53E5697453D`

返回本卷目录

Canonical Markdown SHA-256: `9a238e196843a505dcc28f09d96da5794144b215594946de5a67e53e5697453d`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\practice\openpivla_oral_model_answer.md`

PROJ-OPENPIVLA-001 + Batch 2 从项目实现反推的证据约束口述稿

状态: `implementation_derived_model_answer / evidence_bounded / not_candidate_transcript / unseen` 更新日期: 2026-07-22 题目: 从代码证据看，formal trainer 的模型、参数更新、稳定性选择、shape contract 与 checkpoint lineage 是什么?

1. 30 秒版本

这条 formal path 使用 Qwen3-VL 做条件生成模型，通过 PEFT LoRA 只更新 adapter 参数。训练循环把 batch 送进 `model(**batch)`，用 `outputs.loss` 做反向传播和梯度累积；collator 复制 `input_ids` 作为 labels，只把 padding 位置设为 `-100`。所以当前代码证明的是 full-chat causal CE trainer，包括 prompt、wrapper 和 assistant content 的非 padding token，不是 assistant-only CE，也不是原始 π0 的连续 flow-matching loss。代码和 report 能证明 trainer path，不能单独证明 checkpoint 效果或 benchmark 结果。

这段话只描述当前可读代码；“我实现/我修改”必须由候选人用 ownership ledger 逐文件确认。

2. 2 分钟版本

我先区分模型、可训练参数和监督目标。模型侧是 Qwen3-VL conditional-generation backbone，LoRA 通过 PEFT 注入指定 target modules，所以 optimizer 更新的是 LoRA adapter 以及配置中允许训练的参数，而不是默认意义上的全参数 Qwen。具体 trainable/total 参数量、target modules 和保存方式要以当前 config、启动脚本和 load smoke 为准。 数据侧，formal package 会把 DROID action window 或 MindCube QA 组织成 chat-style input。collator 先复制 `input_ids` 生成 labels，再依据 attention/padding mask 将 padding 位置改为 `-100`；没有看到 assistant-span-only 的 label mask。因此非 padding 的 user prompt、chat wrapper 和 assistant JSON/答案都会进入 causal cross-entropy。这个结论来自 collator 的实现，不应被描述成“只学习 action token”。 训练侧是标准 `model(**batch) → outputs.loss → backward → optimizer/gradient accumulation → checkpoint save` 路径。它不是连续 action expert 的 flow-matching objective：这里的 action 被序列化为文本/JSON 后进入语言模型 CE，表示、tokenization、parser 和执行端必须单独审计。尤其 recipe 里的 `fast_tokenizer` 字段不能证明实际调用了 FAST 的 DCT/BPE processor。 最后我会把证据分层：formal_train.py、recipes/config、smoke 和 committed report 可以证明 trainer/data contract；远端 log、checkpoint、load smoke、generation→strict parser→normalizer→controller round-trip 和 raw benchmark bundle 仍需要单独核验。因此我会说“formal trainer path 已实现/已审计”，不会直接说“模型效果已验证”或“已复现官方 97.4%”。

3. 证据映射

结论	证据	当前强度
Qwen3-VL + PEFT LoRA	OpenPiVLA Claim–Evidence Inventory 与 formal trainer 记录	`local-direct / ownership-open`
`model(**batch)` 与 `outputs.loss`	formal trainer / committed report	`local-direct / trainer-path`
labels 只屏蔽 padding	`Qwen3VLCollator` 代码审计与题卡	`local-direct / full-sequence-CE`
JSON/causal-CE 与 FAST 分离	OpenPiVLA JSON/CE Shape Gate	`PASS WITH WARNING / representation boundary`
checkpoint/benchmark 尚未闭环	inventory 的 remote-referenced、archive-summary-only 与 ownership ledger	`WARN`

4. 高频追问答法

“为什么不是 assistant-only CE? ”

因为当前 collator 的 mask 规则只把 padding 位置变成 `-100`，没有根据 assistant span 或 response token 区间生成额外 mask；所以 prompt、wrapper 和 assistant content 的非 padding token 都参与 CE。若要改成 assistant-only，需要明确 span parser、token offset、empty-response 和 batch reduction 的测试。

“LoRA checkpoint 如何证明可加载？”

需要 base model revision、adapter/config SHA、保存目录或 immutable artifact、load command 和 stdout smoke；仅有 `save` 调用、report 行或远端路径不能证明当前环境可加载。

“为什么这不等于 flow matching？”

flow matching 直接在连续 action/noise path 上学习向量场或速度目标；当前 formal route 把 action window 序列化为 assistant JSON/text，再用语言模型 causal CE。两者的 target、mask、decode 和执行接口不同，loss 数值不能直接比较。

“recipe 里有 `fast_tokenizer`，是不是已经使用 FAST？”

不能这样推断。要沿 call graph 找到 FAST processor、DCT/量化/BPE、decode/IDCT 和 round-trip；当前审计显示 local JSON/CE route 与 upstream FAST 机制分开。

“代码里有 trainer，能否说 97.4% 已复现？”

不能。97.4% 是 archive-summary-only，official 96.85 是另一条 frozen upstream baseline；缺 checkpoint lineage、trial/seed/init、harness、raw episodes 和 metric recomputation，不能做 uplift 或同源结论。

5. Batch 2：稳定性、证据阶梯与动作合同

5.1 30 秒项目版本

> 我把 OpenPiVLA formal path 拆成三层：代码直接证明 Qwen3-VL+LoRA、`eager` attention 和 `outputs.loss` 的 full-chat causal CE；public smoke 只证明公开数据的 batch/schema 与 tiny step；formal report 记录三条 `complete @ 100000`，但 raw log、checkpoint hash 和 load smoke 尚未闭合。更关键的是 recipe 的 `H=15,D=72` 与 formatter 每步 `limit=16` 冲突，action parser/normalizer/controller round-trip 不存在，所以不能说 formal Qwen 已闭环。

5.2 2 分钟版本

> 我会先讲稳定性决策。历史 co-train 记录出现过 Qwen3-VL flash-attention/position-id 失败，当前 `formal_train.py` 的 CLI 默认 `--attn-implementation eager`，并把它传给 `Qwen3VLForConditionalGeneration.from_pretrained`。这能证明当前路径选择了一个可诊断的兼容实现，不能证明 eager 比 flash 更快、更准或带来 benchmark uplift；要证明性能差异，必须做同硬件、同 seed、同数据和同 batch 的 profiler A/B。然后分证据等级。三份 public smoke summary 标记 `private_data_deployed=false`，只能证明公开 roots、`(B,H,D)` shape、padding 和 tiny optimizer step；formal report 是本地报告对三条 `complete @ 100000` 和远端路径的记录，属于 report-supported/remote-referenced；当前缺 immutable raw log、checkpoint/processor SHA、load stdout、generation/parser 和 raw eval bundle。因此“报告记录完成”不等于“当前环境拥有可加载最终模型”。 最后讲动作合同。recipe/smoke 的目标是 `H=15,D=72`，但 formal JSON formatter 每步最多保留 16 个数，sentinel probe 证明 `15×72` 中有 840/1080 个位置被丢弃，tail-repeat、empty/ragged/NaN 也没有严格校验。Python `json.dumps→loads` 只能还原 `15×16` 前缀；formal package 没有 Qwen generation、strict parser、normalizer 或 controller adapter。故我的可信项目贡献是建立训练/证据审计并定位 representation/lineage 风险，而不是宣称完成 72D action deployment。

5.3 Batch 2 Evidence Contract

Claim	当前证据	强度	安全口径
formal 默认 `eager`	`formal_train.py` CLI 与 Qwen loader call	`local-direct / C1`	可讲兼容性选择；不能讲性能 uplift
public smoke 通过	三份 `*_smoke_summary.json`，`private_data_deployed=false`	`local-direct / C1`	可讲 schema、shape、tiny step；不能替代私有 full run
三条 `complete @ 100000`	formal reproduction report	`report-supported / C1`	可讲报告记录；不能称本地 verified checkpoint
raw formal checkpoint/load	当前缺 binary、SHA、stdout smoke	`unverified / C3`	禁止宣称可加载、可 rollout
`H=15,D=72`	recipe + public smoke shape	`config/smoke / C1`	只能说目标 shape，不等于语义闭环
`limit=16` lossy formatter	local source + deterministic probe	`local-direct / C2`	可讲 72→16 schema loss 与 bad case
formal parser/controller	冻结 package 未找到 generation/parser/normalizer/controller call	`negative evidence / C2`	可讲缺口定位，不说已修复
archived `flow_matching_loss`	recipe historical metadata	`archive-summary-only`	不与 current `outputs.loss` 合并
97.4% / official 96.85	archive/upstream references	`non-comparable`	不写成 current formal uplift

当前可以讲 C1/C2：eager call graph、smoke/formal/raw 证据阶梯、shape conflict、deterministic probe、lineage manifest 设计。当前 C3 禁止讲：正式 checkpoint 已验证、72D controller-ready action、formal Qwen closed-loop success、eager 带来的性能收益或 97.4% uplift。

5.4 四个 Batch 2 高压追问

“eager 是不是比 flash 更好？”

危险回答：是，eager 更稳定也更快，所以最终效果更好。

可过回答：eager 是为了绕过 flash 的兼容问题，但我没有做性能对比。

强回答：

> 我只把 eager 认定为兼容性路径：代码确实把它传入 Qwen loader，历史记录也有 flash/position-id 失败，但没有同硬件、同 seed、同数据的 profiler、显存、step-time 或成功率 A/B。因此我不会说 eager 更快或更准；要验证，只能固定其他变量跑 eager/flash smoke 和 matched training comparison。

“smoke 和 report 都说完成，为什么还不能说有最终模型？”

危险回答：report 已经写了 complete，所以 checkpoint 肯定存在。

可过回答：smoke 和 report 的证据等级不同，raw checkpoint 还要补。

强回答：

> smoke 的 `private_data_deployed=false`，只验证公开数据的 schema、batch、padding 和 tiny step；report 是本地对历史/远端运行的转述，路径仍是 remote-referenced。要升级为 checkpoint lineage verified，必须拿到 raw log、checkpoint/adapter/processor SHA、新进程 load stdout 和最小 eval manifest。`complete @ 100000` 只是 steps_reached，不是可加载权重证明。

“`H=15,D=72` 不是已经写在 recipe 里了吗？”

危险回答：是，所以模型已经输出 15×72 的真实控制量。

可过回答：recipe 是目标 shape，但 formatter 只保留 16 维，语义还没闭合。

强回答：

> recipe/smoke 证明的是目标或 tensor shape，不证明 formatter、parser、normalizer、axis/unit 和 controller 共用同一合同。当前 formatter 每步最多写 16 个数，probe 证明相对 `15×72` 丢掉 840 个标量，还存在 tail-repeat、ragged 和 NaN 风险。JSON codec 只能还原保留前缀；在严格 parser、逐维 round-trip 和 controller trace 出现前，不能称 72D action 已验证。

“历史日志有 flow matching loss，能否说 current trainer 做了 flow matching？”

危险回答：能，loss 名字已经说明模型在做 flow matching。

可过回答：那是历史 metadata，current code 目前看到的是 `outputs.loss`。

强回答：

> 我会把两条证据分开：`recipes.py` 中的 `flow_matching_loss` 是 archived historical metadata；current formal trainer 明确执行的是 chat-template token labels 进入 Hugging Face `outputs.loss` 的 causal CE。没有同一代码、checkpoint、action head 和 parser lineage，就不能把它们称为同一个 loss 或连续修复。要重建 flow-matching 结论，需要直接看到连续 action/noise target、训练 call graph 和可加载权重。

5.5 Batch 2 追问树与 lineage 最小合同

```
“三条 formal run 已完成”
  -> 追问 raw log、checkpoint/processor SHA、load stdout、eval manifest
  -> 缺任一项：降级为 report-supported

“模型预测 15×72 action”
  -> 追问 formatter 输出、strict parser、axis/unit、normalizer、controller trace
```

```
  -> 出现 limit=16: 定位 840/1080 dropped positions

"eager 带来收益"
  -> 追问同硬件/seed/data/batch 的 flash A/B profiler
  -> 没有对照: 只保留 compatibility claim

"我负责这次修复"
  -> 追问具体文件、函数、diff、operator、reviewer 和验收命令
  -> 只有 report/author: ownership 保持 pending
```

最小 lineage manifest 必须绑定：`run_id`、trainer/config/recipe SHA、base model revision、LoRA adapter、checkpoint/processor SHA、data manifest、raw log、load smoke、eval harness 与 episode manifest。只记录 `steps_reached=100000` 不足以证明权重存在或结果可复算。

6. 练习边界

- 这是示范回答，不是候选人 transcript；不要把第一人称事实自动认领为个人贡献。
- 真实回答应至少指出一个具体函数/文件、label mask 规则、`H/D` contract 缺口和一个证据缺口。
- 真实口述前，`PROJ-OPENPIVLA-001` 保持 `unseen`，不创建 `coaching_state.md`。

References

1. OpenPiVLA Claim–Evidence Inventory。
2. OpenPiVLA Ownership Ledger。
3. OpenPiVLA JSON/CE + H/D Shape Gate。
4. OpenPiVLA Action JSON Parser / Round-trip Gate。
5. PROJ-OPENPIVLA-001..008 Atomic Cards。
6. OpenPiVLA Batch 2 Evidence Freeze。
7. PROJ-OPENPIVLA-009..012 Atomic Cards。

Part 25 / 49 · `interview_materials/vla/questions/proj_openpivla.html` · SHA-256 `86B0818702D5E62F117BDF626106A6B71E6B667BDF49B7FCEFEE5668A2EB7B88`

返回本卷目录

Canonical Markdown SHA-256：`86b0818702d5e62f117bdf626106a6b71e6b667bdf49b7fcefee5668a2eb7b88`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_openpivla.md`

PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 1）

状态：`drafted / cards_8_of_8 / WARN` 更新日期：2026-07-21 当前题数：8 / 8 覆盖范围：`PROJ-OPENPIVLA-001..008` 难度：L2 4 题 / L3 4 题 优先级：P0 6 题 / P1 2 题 练习状态：全部 `unseen` 使用方式：先闭卷回答 30 秒，再展开代码证据与缺口；未真实口述前不得改为 `pass/strong`。

A. Formal trainer 与动作监督

▼ `L2` `P0` PROJ-OPENPIVLA-001 · 从代码证据看，formal trainer 的模型、参数更新和 loss 是什么？

面试意图：确认你能从源码说明“实现了什么”，又不把实现路径升级成 benchmark 结果。

30 秒回答：`formal_train.py` 用 Qwen3-VL + LoRA；collator 复制全部 `input_ids`，只屏蔽 padding，所以优化 full-chat causal CE，不是 assistant-only CE。代码证明 trainer，不证明 checkpoint 效果。

机制展开（约 2 分钟）：模型加载、LoRA 配置、`model(**batch)`、梯度累积和 checkpoint 保存都在同一 formal trainer 中；labels 由 `input_ids` 克隆并仅把 attention padding 置为 `-100`，没有 assistant-span mask。因此 user prompt、chat wrapper 与 assistant JSON/QA 的非 padding token 都进入 CE；这不是原始 Pi0 action expert 的连续 flow-matching loss。面试时还要把“代码可读”与“checkpoint 可加载/指标可复算”分开。

诊断 / 证据：打印 model class、LoRA target modules、trainable/total params；用一个 batch 检查 `input_ids`、`labels`、padding mask 和 `outputs.loss`；再用 immutable checkpoint 做 load smoke。当前前两层有本地源码/报告，最后一层仍缺 formal checkpoint。

项目挂钩：可把个人贡献落在 trainer、collator、recipe 和稳定性路径，但须另做 ownership ledger；不能只凭 Git author 认领所有历史代码。

追问树：为什么不用全参训练？`-100` 如何影响 CE？LoRA checkpoint 如何和 base model 合并？为什么这不等于 flow matching？

危险答案：说“LoRA trainer 就是原始 OpenPI action expert 的等价实现”，或说代码里有 trainer 就证明 97.4% 已被该模型复现。

来源：[1][2][3][4][19] 练习状态：unseen

▼ `L2` `P0` PROJ-OPENPIVLA-002 · local action route 如何把连续 action window 变成 Qwen supervision？

面试意图：检查你是否能讲清 formatter、chat template、label mask 和执行端缺口。

30 秒回答：DROID 读取未来动作窗口，按 horizon 截断或补齐，序列化为 `{"actions": [...]}` assistant 文本；Qwen processor/chat template 转成 token，collator 屏蔽 padding label。这是 JSON/causal-CE 路线，不是已证明的 FAST DCT/BPE。

机制展开（约 2 分钟）：`DroidActionTextDataset` 按 episode 收集 action，每行最多保留 16 个数并重复尾行，再用 `json.dumps` 生成 assistant content。collator 复制全部 `input_ids`，只屏蔽 padding。72D probe 只能恢复 `15×16` prefix；formal package 没有 Qwen generation/action parser，official WebSocket eval 又属于另一 checkpoint lineage。因此这里只证明训练文本构造，不证明 formal Qwen closed-loop rollout。

诊断 / 证据：现有 sentinel 已确认 72→16 丢 840/1080 positions、tail-repeat 与 empty/NaN schema 风险；collator probe 确认只有 padding 被 mask。下一步必须用 immutable Qwen checkpoint 做 tokenizer→generate→strict parser→normalizer→controller round-trip。若只看 loss 下降，不能证明动作单位、轴序和夹爪语义正确。

项目挂钩：简历可写“assistant JSON content + full-chat causal-CE formal path，并定位 lossy serializer/parser gap”，不能写“复用了 FAST tokenizer”“assistant-only loss”或“已证明 action expert 等价”。

追问树：为什么用 JSON？数值精度怎么控制？parser 在哪里？D=72 每一维是什么？

危险答案：看到 recipe 里的 `fast_tokenizer` 字段，就说 local action 已经过 DCT/BPE；或把 JSON 字符串直接称为机器人控制量。

来源：[1][2][5][6][19] 练习状态：unseen

▼ L3 P0 **PROJ-OPENPIVLA-003 · 为什么不能把 `fast_tokenizer` 字段说成 local FAST DCT/BPE 实现?**

面试意图：检查你能否用 call graph 而不是字段名判断真实实现。

30 秒回答：`recipes.py` 有 `fast_tokenizer` 字段，但 trainer 调用 Qwen processor、JSON assistant text 和 `outputs.loss`；未见 FAST processor、DCT/BPE 或 roundtrip。因此 local route 只能称 JSON/causal-CE。

机制展开（约 2 分钟）：证据审计要从 recipe 字段追到运行时调用：配置声明不等于执行。当前 call graph 是 DROID parquet → action JSON → chat template → Qwen processor → causal CE；FAST 的公开 call graph 则包含 normalization、DCT、量化/BPE、decode/IDCT。两条路径都能把动作纳入序列模型，但 token 语义、损失和反解析合同不同，不能在简历或口述中合并。

诊断 / 证据：静态检查是否 import/call frozen FAST processor；用一个 action window 做两条 encode/decode roundtrip；比较 token length、重建误差和 parser 输出。当前 local FAST call/roundtrip 证据缺失，应保持 `representation-conflict` 风险。

项目挂钩：这道题是 OpenPiVLA 的高频防穿帮题；主动区分 upstream FAST、历史 action expert 和本地 Qwen formal path，反而能体现证据意识。

追问树：如果后续接入 FAST，哪些文件和测试必须变化？JSON/CE 为什么可能更容易先跑通？两条 loss 能否比较数值大小?

危险答案：说“配置里写 fast_tokenizer，所以已经实现 FAST”，或说两条 loss 只差 tokenizer。

来源：[2][3][7][8] 练习状态：unseen

▼ L2 P0 **PROJ-OPENPIVLA-004 · VLA-only、VLM-only、co-train 三类 recipe 的数据流区别是什么?**

面试意图：确认你能把 run mode 映射到数据集、采样和训练目标，而不是只背三个名字。

30 秒回答：VLA-only 只启用 DROID action-text dataset；VLM-only 只启用 MindCube QA；co-train 同时构造两者并由 `AlternatingDataset` 交替采样。三类 recipe/config 和 public smoke shape 可直接核验，但不自动证明私有数据结果或某个模式更优。

机制展开（约 2 分钟）：`build_dataset` 根据 recipe mode 选择 DROID/MindCube iterable dataset，并设置各自权重；DROID 样本含机器人动作 JSON 目标，MindCube 样本含视觉语言答案，co-train 将它们送进同一 Qwen causal CE。配置中的 private roots 与 public smoke roots 要分开：smoke 只验证 schema、batch shape 和 tiny optimizer step，不能替代历史私有语料或 benchmark。

诊断 / 证据：打印每个 mode 的 dataset source、sample count、mixture schedule、batch fields 和 labels mask；对 co-train 做单源/双源比例消融。若缺完整 run ledger，不能从 loss 绝对值推断三类模式的性能排序。

项目挂钩：可讲 recipe 设计和数据 contract；“三条都完成并保存可加载 checkpoint”属于 report-supported/remote-referenced，需单独补证据。

追问树：co-train 如何防止 QA 淹没 action？private/public 数据差异？为什么保留 VLM-only?

危险答案：把 public smoke 当作私有 DROID/MindCube 的等价替代，或把三种 mode 当作三种模型架构创新。

来源：[2][3][4][9] 练习状态：unseen

B. Formal completion、归档结果与评估边界

▼ L3 P1 **PROJ-OPENPIVLA-005 · 三条 100k formal run 应该如何分层陈述?**

面试意图：训练你区分源码、报告、远端日志和 checkpoint load smoke 四种证据强度。

30 秒回答：scripts/config/report 是 direct；report 记三条 `complete @ 100000`，属 report-supported；远端 log/checkpoint 为路径引用，属 remote-referenced。缺 raw log、checkpoint SHA、load smoke，不能称 checkpoint 已验证。

机制展开（约 2 分钟）：先列证据链：recipe/config → launch command → formal log → checkpoint hash → load smoke → eval manifest。当前已有前两项和 committed report，后两项只保留 `/data/msz/openpivla/...` 路径引用。报告还说明 formal JSON/CE/LoRA 与原 flow/action expert 不等价，因此“跑到 100k”只能说明该 formal execution path 被记录为完成，不能说明复现了原项目性能。

诊断 / 证据：为每个 run 建 immutable manifest，记录脚本 SHA、config SHA、steps reached、checkpoint SHA、base model、LoRA adapter、load command 和 stdout；再用同一 checkpoint 做 eval smoke。当前 gate 未关闭。

项目挂钩：简历只写“搭建/迁移 formal training path”；面试可说“报告记录 completion，raw checkpoint 待核”，不要说“我已加载三条最终模型并验证 benchmark”。

追问树：为什么 report-supported 不等于 verified？保存点如何生成？第三条 batch 参数为何需要看远端脚本？

危险答案：只凭报告中的 `complete` 行就声称三条 checkpoint 已加载、可 rollout、可复现。

来源：[3][4][10][11]　**练习状态**：unseen

▼ L3 P0 PROJ-OPENPIVLA-006 · `224_h10_40000` 的 97.4% 能否与 official 96.85 比较?

面试意图：检查你是否会在数字很诱人时主动冻结 protocol tuple。

30 秒回答：不能下“高于 official”的结论。97.4% 只有 committed report 汇总，属 `archive-summary-only`；official 96.85 是 frozen README 事实，但 checkpoint、trial/seed、init state、harness、controller、raw episodes 尚未对齐。

机制展开（约 2 分钟）：可比 benchmark 至少要绑定 suite/task version、checkpoint/base model、image preprocessing、normalizer、H/K、controller、trial/seed/init state、success predicate、metric code 和 aggregation。当前报告确实有四套件数字，但没有 raw summary、episodes 和 archive→formal lineage；因此要把“报告里出现的数字”“官方 baseline”“统计可比性”分成三层，不能用 0.55pp 差值做 uplift 结论。

诊断 / 证据：收集 archive checkpoint SHA、每 suite/task/seed episode records、eval script/config、norm stats 和 metric recomputation；用 protocol diff 表逐字段对齐，再考虑多 seed 或置信区间。

项目挂钩：这是简历与项目页的高压边界题；安全回答不是回避数字，而是说明数字目前处于什么证据等级、下一步怎样升级。

追问树：official 96.85 的 suite order 和 trials？97.4 是哪条模型链？单点差是否有统计意义？

危险答案：说“97.4 比 96.85 高，所以我超过了官方”，或把归档表格自动归因给 current Qwen formal。

来源：[3][12][13][14]　**练习状态**：unseen

▼ L2 P0 PROJ-OPENPIVLA-007 · local ALOHA/LIBERO/replay artifacts 分别证明什么?

面试意图：防止把 official baseline rollout、诊断视频和 formal Qwen benchmark 混成一个结果。

30 秒回答：ALOHA `pi0_aloha_sim` 为 `2/3`，多视角 `1/2`；LIBERO `pi05_libero` task-0 为 `0/1`。server log 确认 Orbax restore、norm stats 与 client connect；replay/L1 仅诊断，不证明 formal Qwen 或 97.4。

机制展开（约 2 分钟）：ALOHA 3 集 `2/3`、六相机 2 集 `1/2`；LIBERO task 0 的 model/no-op 各 70 步、均 `0.0` reward/false success。server log 记录 Orbax 从 `pi05_libero/params` 恢复约 6.2 GiB、17 秒完成，从 checkpoint assets 加载 norm stats，并在 8399 接受 localhost client；script 固定 image/wrist/state/prompt 与 action chunk 前 `K×7D`。仍缺逐文件 SHA、exact argv/run UUID、独立 scorer 和 formal Qwen lineage。

诊断 / 证据：summary 字段只描述结果，不能证明 server 真正加载了什么；checkpoint 绑定必须看 restore log。ALOHA LeRobot action eval 的 4×50×14 mean L1 `0.033071` 只作离线 action-level diagnostic。当前 load/serve/connect 已闭合到 `PASS WITH WARNING`，下一步生成 params/norm-stats 逐文件 SHA，并保存 shared run ID、完整 argv、request trace 与 scorer revision。

项目挂钩：可讲“我用 restore/normalizer/connect log 审计部署闭环”；在 ownership 未确认前仍不把脚本实现或 official checkpoint 训练归到个人名下。

追问树：为什么 client summary 的 checkpoint 字段不够？如何证明 client 连的是同一 server？0-success 排除了哪些原因、还没排除哪些?

危险答案：把 replay video 当作 Bridge/Fractal 任务成功，或把 official `pi05_libero` rollout 说成自己的 Qwen 模型结果。

来源：[3][10][11][15][20][21][22]　**练习状态**：unseen

▼ L3 P1 PROJ-OPENPIVLA-008 · 如何回答 OpenPiVLA 的个人 ownership?

面试意图：区分 Git authorship、代码事实、个人贡献和工具/协作边界。

30 秒回答：`c4469eb` 只证明候选账号提交了包含 360 个 OpenPiVLA paths 的初始快照。13-row ledger 已区分上游、本地文件、验证和待确认角色；没有逐项回答前，不能把 Git author 当作独立原创证明。

机制展开（约 2 分钟）：Git 审计发现 OpenPiVLA 的 360 paths 一次进入 root snapshot，关键文件没有更早 diff；frozen OpenPI tree 又没有 `vla_qwen3_repro`，只能证明它是相对该上游的本地新增路径，不能证明由谁独立编写。ledger 把 trainer、JSON/collator、recipe、smoke、MACA、eval、report 与 upstream 分成 13 行，并要求逐项填写亲手、协作、工具辅助或上游。设计/修复还需 before/after、决策记录和运行验收。

诊断 / 证据：ledger 结构已完成，当前 gate 是候选人回答 8 个角色问题，并为关键函数补 pre-snapshot 来源、diff、命令或 review；所有题卡仍 unseen。

项目挂钩：未确认前说“项目快照包含 Qwen/recipe/eval contract”；确认后只认领 ledger 中自己的模块，不说“整个 OpenPiVLA 都是我写的”。

追问树：哪一段代码是你亲手改的？为什么选择 JSON/CE/eager？工具生成代码如何审查？没有 commit diff 怎么证明贡献？

危险答案：把 Git author 等同于所有代码的独立原创证明，或把上游 FAST/OpenPI 贡献算成自己的算法创新。

来源：[3][16][17][18]　**练习状态**：unseen

参考来源

1. 本地 `C:\Users\mousongzhe\Project\openpivla_deployment\repro_train\openpi_src\src\openpi\training\vla_qwen3_repro\formal_train.py`：Qwen3-VL、LoRA、JSON dataset、collator 和训练循环。
2. 本地 `C:\Users\mousongzhe\Project\openpivla_deployment\repro_train\openpi_src\src\openpi\training\vla_qwen3_repro\recipes.py`：三类 recipe、shape 字段与 formal 计划。
3. 本地 OpenPiVLA Claim–Evidence Inventory：local-direct / report-supported / archive-summary-only / missing 分层。
4. 本地 `C:\Users\mousongzhe\Project\openpivla_deployment\repro_train\openpi_src\src\openpi\training\vla_qwen3_repro\data.py`：public smoke data contract 与 batch shape。
5. 本地 `formal_train.py` 中的 `DroidActionTextDataset`：action window → compact JSON assistant text。
6. 本地 `formal_train.py` 中的 `Qwen3VLCollator`：chat template、processor 和 padding label mask。
7. Physical Intelligence, FAST：上游 DCT/量化/BPE 机制；不证明 local Qwen route 使用 FAST。
8. 本地 formal report：三条 report-supported formal run 与 `224_h10_40000` archive summary。
9. 本地三份 `repro_train/configs/*.yaml` 与 `run_formal_*.sh`：recipe/config/launch evidence。
10. 本地 DEPLOYMENT.md：remote-referenced logs/checkpoints、ALOHA/LIBERO/replay 边界。
11. 本地 official LIBERO launcher：official policy server/client eval harness。
12. Frozen OpenPI `examples/libero/README.md`：official `pi05_libero` 96.85 baseline；见 inventory frozen source SHA。
13. 本地 `224_h10_40000` report summary：97.4% archive-summary-only；不含 raw episode bundle。
14. 本地 `openpivla_deployment` rollout artifacts：ALOHA/LIBERO true rollout、replay-only、action adapter 和 L1 diagnostics。
15. 本地 `Project` Git commit `c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`：粗粒度 authorship signal，不等于 ownership ledger。
16. 本地 OpenPiVLA 项目页：安全口述、项目 hook 与高压边界。
17. 本地 resume restructure plan：当前安全简历口径和禁写规则。
18. 本地 OpenPiVLA Ownership Ledger：13 个文件/决策、Git/upstream 边界与 8 项候选人确认问题。
19. 本地 OpenPiVLA Action JSON Parser / Round-trip Artifact Gate：72→16 loss、full-sequence CE、parser absence 与 official WebSocket parallel lineage。
20. 本地 OpenPiVLA Runtime Rollout Artifact Gate：冻结 ALOHA/LIBERO summary SHA、true-physics rollout、视频/轨迹清单与 fine-tune/benchmark/ownership 边界。
21. 本地 OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate：冻结 Orbax restore、checkpoint norm stats、8399 listen/localhost connect 与 hashed rollout contract；binary manifest、exact argv/run UUID 和 private fine-tune lineage 仍未闭合。
22. 本地 OpenPiVLA ALOHA Action-Level Eval Load Gate：冻结 `pi0_aloha_sim` restore/norm-stats/8020 connection 与 4×50×14 offline action L1；不支持 task success 或 fine-tune claim。

Part 26 / 49 · `interview_materials/vla/questions/proj_openpivla_batch2.html` · SHA-256 `BE95B4F4561F971B926D4CE3C2A2732BB31561A11A14B4D05398AB9B529A7135`

返回本卷目录

Canonical Markdown SHA-256: `be95b4f4561f971b926d4ce3c2a2732bb31561a11a14b4d05398ab9b529a7135`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_openpivla_batch2.md`

PROJ-OPENPIVLA：OpenPiVLA 项目原子题卡（Direct-Evidence Batch 2）

状态：`drafted / cards_4_of_4 / WARN` 更新日期：2026-07-17 当前题数：4 / 4 覆盖范围：`PROJ-OPENPIVLA-009..012` 难度：L2 2 题 / L3 2 题 优先级：P0 3 题 / P1 1 题 练习状态：全部 `unseen` 前置证据：Batch 2 evidence freeze；未闭合的 checkpoint、parser、axis/unit 和 ownership gate 不得升级为强结论。

A. 稳定性与证据阶梯

▼ `L2` `P0` PROJ-OPENPIVLA-009 · 为什么 local formal trainer 默认使用 eager attention?

面试意图：判断你能否把硬件/依赖兼容性处理讲成可验证的稳定性决策，而不是性能神话。

30 秒回答：历史 co-train 记录指向 Qwen3-VL flash-attention/position-id 失败；current formal CLI 默认 eager，并把它传给 Qwen loader。它证明兼容路径可执行，不证明吞吐或 benchmark 提升。

机制展开（约 2 分钟）：`formal_train.py` 的 `--attn-implementation` 默认值是 `eager`，随后传入 `Qwen3VLForConditionalGeneration.from_pretrained`。训练仍走 processor/chat template、padding label `-100`、`outputs.loss` 和 LoRA 更新。eager 的证据是“避开已记录失败点并让路径可诊断”；没有同 recipe、同硬件的 flash/eager profiler、显存、step time 或结果对照，不能说它更快、更准或带来 uplift。

诊断 / 证据：固定 recipe、seed、数据和 batch，分别跑 eager/flash smoke；记录 import、显存、step time、OOM/position-id traceback、loss 是否可比，并保存完整 stdout。当前只有 eager call graph 和报告转述，flash 对照与 profiler 缺失。

项目挂钩：可讲我在 MACA/C500 迁移中优先关闭已知 attention 崩溃面，再建立可复查的训练路径；不要把硬件适配写成算法创新。

追问树：为什么不直接修 flash？eager 会牺牲什么？如何证明不是数据变化造成稳定？同一 checkpoint 的输出是否应一致?

危险答案：说“eager 一定比 flash 更稳定/更快”，或把 `complete @ 100000` 直接归因于 eager 带来的 benchmark 提升。

来源：[1][2][3] **练习状态**：unseen

▼ `L2` `P0` PROJ-OPENPIVLA-010 · smoke summary、formal report 和 raw log/checkpoint 的证据等级有什么区别?

面试意图：检查你能否把“工程链路跑通”“报告记录完成”和“权重可复核”拆成三件事。

30 秒回答：smoke JSON 只证明公开数据的 schema、shape 和 tiny step；formal report 记录三条 `complete @ 100000` 及远端路径；raw log、checkpoint、load smoke 不在本地，不能称模型已验证。

机制展开（约 2 分钟）：smoke summaries 标记 `private_data_deployed=false`，给出 public roots、batch shapes 和 tiny optimizer step，属于 local-direct。formal report 是本地 HTML 对历史/远端运行的转述，属于 report-supported；`/data/msz/...` 只是 remote-referenced 路径。只有能读取 immutable raw log、checkpoint/processor hash、load stdout、eval script 和 episode manifest，run 可从“报告记录”升级为 `checkpoint lineage verified`。

诊断 / 证据：每条 run 建 manifest，校验 trainer/config/base-model/data/checkpoint/log/eval 的 SHA；执行同一 checkpoint 的 load smoke，再重算一个最小 eval。当前 smoke 与 report 有，raw formal log、checkpoint binary 和 load smoke 仍缺。

项目挂钩：简历和面试可说“搭建 public smoke 与 formal training path”；应主动说明 completion 是 report-supported，不能说三条最终模型已加载并完成 benchmark。

追问树：为什么 smoke loss 不能代表 full run？报告里的 checkpoint path 算不算 checkpoint？怎样防止日志与权重错配?

危险答案：把 `tiny_train_step` 当作私有数据训练结果，或只凭 report 的 `complete` 行声称可 rollout。

来源：[1][4][5] 练习状态：unseen

B. Lineage 与动作 shape

▼ `L3` `P1` **PROJ-OPENPIVLA-011 · 如何设计一份可审计的 OpenPiVLA checkpoint lineage manifest?**

面试意图：看你是否能把训练、数据、权重和评估连接成可恢复链路，而不是只记一个 run id。

30 秒回答：lineage manifest 要把 run_id、trainer/config SHA、base model、adapter、checkpoint/processor hash、data manifest、raw log、load smoke 和 eval harness 绑成一条链；当前只是字段设计，hash 与 load evidence 仍缺。

机制展开（约 2 分钟）：最小链路是 `recipe/config → launch script → trainer → data manifest → base model/LoRA → checkpoint + processor → raw log → load smoke → eval harness → episode manifest`。每项记录 SHA、版本、路径、时间，并区分 public smoke、remote-referenced、archive-only。`complete @ 100000` 只填 `steps_reached`；缺 checkpoint/processor hash 或 load stdout，manifest 保持 `lineage-open`，不能把路径当权重证明。

诊断 / 证据：为三条 run 生成 manifest，先检查脚本/配置/源代码 SHA，再核对 checkpoint 目录、adapter config、processor 文件、log step 和 eval 输入是否一一对应；最后运行只读 load smoke。当前模板已写入 Batch 2 审计，但真实 checkpoint/processor/raw log 字段仍为 missing。

项目挂钩：这是我把训练迁移从“能跑”提升到“能复查”的工程抓手；个人 ownership 仍需另做 per-file ledger，不能由 manifest 自动推断。

追问树：为什么 processor 也要 hash？base model 没有 binary 怎么记录？如何处理远端不可访问的 artifact？

危险答案：只保存 `run_id` 和最终 step，或把 manifest 模板写成已经完成的 checkpoint 证据。

来源：[1][3][6][7] 练习状态：unseen

▼ `L3` `P0` **PROJ-OPENPIVLA-012 · `H=15,D=72` 与 formal formatter 的 `limit=16` 冲突时，应该怎样解释?**

面试意图：检查你能否发现 shape 看似一致、语义却未闭合的动作表示风险。

30 秒回答：recipe 是 `H=15,D=72`，formatter 每步只保留 16 个数；probe 丢 840/1080 positions，formal package 又没有 action parser。Python JSON 只恢复 `15×16` 前缀，不是 controller round-trip。

机制展开（约 2 分钟）：令 $\mathbf{A} \in \mathbb{R}^{H \times D}$，其中 `H=15,D=72`。formal formatter 只把每行前 16 个数写入 JSON，再重复尾行补满 horizon；`dumps→loads` 只能恢复 lossy prefix。empty window/row 还会产生 `15×1/15×0`，NaN 未被拒绝。没有 strict parser、valid mask、axis/unit、normalizer 与 controller trace，shape 和执行语义都未闭合。

诊断 / 证据：已用 72D sentinel 冻结 retained/dropped positions、tail-repeat、empty/ragged/NaN 与 collator label mask；还要记录 immutable Qwen generation、strict parser、normalizer/denormalizer、axis/unit 和 controller 输入，做逐维 round-trip。项目中 7D/4-step official WebSocket eval 使用 `pi0_aloha_sim` checkpoint，是平行 lineage，不能替 formal Qwen 补证据。

项目挂钩：安全表达是“我用 probe 量化了 72→16 的 77.78% schema loss，并确认 formal text route 缺 parser/controller；当前这是可信 bad case，不是已修复上线结果”。

追问树：为什么 smoke 要 pad/repeat？`limit=16` 是临时占位还是正式设计？如何判断截断丢的是哪类关节？loss 下降能否证明 shape 正确?

危险答案：说“配置写 72 所以模型已经预测 72 维控制量”，或把 public smoke 的重复动作当成真实 DROID 未来窗口。

来源：[1][2][4][6][9] 练习状态：unseen

参考来源

1. OpenPiVLA Batch 2 evidence freeze：eager、evidence ladder、lineage、`H=15,D=72` / `limit=16` 冲突的冻结审计。
2. 本地 `C:\Users\mousongzhe\Project\openpivla_deployment\repro_train\openpi_src\src\openpi\training\vla_qwen3_repro\formal_train.py`：eager 参数、JSON formatter、Qwen `outputs.loss`，SHA `51b67d3e04496e68076bf6461506141ebcddf9a82e6c30af1fdc0f1f2ccc6f94`。

3. 本地 `C:\Users\mousongzhe\Project\openpivla_deployment\artifacts\vla_formal_reproduction_report_20260617.html`：report-supported formal completion 与 attention 边界，SHA `d74126a3896f68ad44b75c7a88f1b7b9a90cb55df0c6c6beb90a5748e9e8e074`。
4. 本地 `artifacts/vla_train_repro/*_smoke_summary.json`：public smoke shape、tiny train step 与 `private_data_deployed=false`。
5. 本地三份 `repro_train/configs/*.yaml` 与 `recipes.py`：run mode、shape 字段、历史 metadata，SHA `397d1b384a2bd93d9dc81b7003e6ab7bfac3fc458ea04008197c3f503f9d780e`。
6. 本地 `data.py`：public smoke 的 `(H,D)` padding/repeat 逻辑，SHA `32c6aa76929d941c4832d2eb4c2550527d3e607e31c2a9f5507cc76eeedbc2ec`。
7. OpenPiVLA Claim–Evidence Inventory：项目 claim、证据等级与 ownership 边界。
8. OpenPiVLA 项目页：安全口述口径与项目 hook。
9. OpenPiVLA Action JSON Parser / Round-trip Artifact Gate：deterministic probes、full-sequence CE 与 parallel official-eval lineage。

Part 27 / 49 · `interview_materials/vla/evidence/projects/openpivla_claim_evidence_inventory.html` · SHA-256 `13023578D1B7343083BEB588C7F8CE7B1BDC267E2BC834204D309106E8EA132A`

返回本卷目录

Canonical Markdown SHA-256：`13023578d1b7343083beb588c7f8ce7b1bdc267e2bc834204d309106e8ea132a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\openpivla_claim_evidence_inventory.md`

OpenPiVLA Claim–Evidence Inventory（OPV-01）

状态：`drafted / local_direct_audit / ownership_ledger_drafted / WARN / resume_safely_rewritten` 审查日期：2026-07-19 证据截止：2026-07-19 项目范围：OpenPI/OpenPiVLA、归档 LIBERO 97.4%、Qwen3-VL action supervision、MACA/C500 formal runs 与 rollout contract 目标岗位：VLA / Robot Policy / Multimodal Training / Embodied Evaluation 结论先行：frozen OpenPI/FAST source 能支持 π0/π0.5/FAST 的公开 action-modeling、normalization、H/K 与 policy-serving 机制，也直接确认 official `pi05_libero` 平均 96.85。真实项目证据位于 `C:\Users\mousongzhe\Project\openpivla_deployment`：当前可直接读取 formal trainer、recipe/config、启动脚本、smoke summaries、部署记录、诊断 rollout artifact 与一份 committed formal report；远端 formal log/checkpoint 只被报告和部署记录引用，当前未在本地镜像中取得。`224_h10_40000` 的 97.4% 有 committed report 的汇总表支持，但仍缺 checkpoint hash、raw summary、trial/seed/protocol 与 episode records，因此只能标为 `archive-summary-only`。这些证据足以把 Qwen JSON/causal-CE 路径、三类 recipe 和训练工程闭环从纯自述升为 `local-direct` / `report-supported`，但不足以证明 FAST DCT/BPE 等价、97.4 与 formal checkpoint 同源、97.4 高于 official 96.85，或个人 ownership。逐文件 ownership ledger 已建立，但候选人尚未确认亲手/协作/工具辅助边界；optimized resume、PDF 与 OpenPiVLA 项目页已经完成安全重写，项目证据继续保持 `WARN`。

2026-07-19 bounded artifact gate：直接执行/读取 `formal_train.py` 的 formatter 与三个 public smoke summaries，确认 `H=15,D=72` recipe/batch contract 与每行 `limit=16` 的 JSON serializer 同时存在；AST probe 得到 `15×16`、tail-repeat 输出。该切片把 `OPV-CL-002` 升为 local-direct with warning，把 `OPV-CL-003` 固定为 representation conflict，把 `OPV-CL-004` 固定为 semantics-open；没有升级 checkpoint、benchmark 或 ownership。

2026-07-19 parser/round-trip gate：formal package AST 中没有 `generate/decode/action-output json.loads`，唯一 `json.loads` 是 MindCube 输入 JSONL；72D sentinel 经 formatter/JSON 只保留 `15×16=240` 个标量，相对 `15×72=1080` 少 840（77.78%）。collator 只屏蔽 padding，故 loss 覆盖 full chat 非 padding token，而不是 assistant-token-only。项目中的 7D/4-step WebSocket eval 使用 official `pi0_aloha_sim` checkpoint，不能替 formal Qwen parser/rollout 补 lineage。`OPV-CL-002/004/010` 已据此降级/澄清，ownership 与 mastery 不变。

问题（Question）

本页回答：`97.4%` 是哪个 checkpoint、哪套 protocol 的结果；它是否能与官方 `96.85` 直接比较；归档 `224_h10_40000` 与当前 Qwen3-VL/MACA formal runs 是不是同一模型链；assistant-side JSON/token 与 FAST DCT/BPE 是什么关系；哪些属于 upstream，哪些能归入个人贡献；要补什么 artifact 才能形成能投、能讲、能抗追问的 OpenPiVLA 项目。

1. Evidence labels 与判定规则

标签	含义	OpenPiVLA 中的例子
`upstream-verified`	frozen paper/official repo 直接支持	OpenPI policy construction、norm stats、LIBERO client、FAST DCT/BPE、official 96.85
`local-direct`	当前可读取的本地 code/config/script/report/smoke/eval/episode artifact；不等于 checkpoint 或 benchmark raw bundle 齐全	`C:\Users\mousongzhe\Project\openpivla_deployment` 下的 formal trainer、recipes、smoke summaries、部署报告与 rollout diagnostics
`remote-referenced`	本地文档给出远端 log/checkpoint 路径，但本地没有对应 immutable 文件	`/data/msz/openpivla/logs/vla_qwen3_repro/*/formal_train.log` 与 `/data/msz/openpivla/outputs/vla_qwen3_repro/<run>/checkpoint-*`
`archive-summary-only`	committed report 中有结果汇总，但缺少可重算 raw bundle	`224_h10_40000` 四套件 97.4% 汇总表
`local-claim-only`	只出现在简历、项目页或计划	未落到代码/报告的个人 ownership 与高于 official 的比较
`run-lineage-open`	archive、formal train、eval checkpoint 的继承关系不清	`224_h10_40000` vs Qwen3-VL/MACA runs

标签	含义	OpenPiVLA 中的例子
`representation-conflict`	FAST 与 local JSON/token route 被讲成同一实现	DCT/BPE upstream vs assistant-side serialization
`ownership-unverified`	上游事实有来源，但个人 diff/职责无直接证据	Qwen 接入、MACA migration、trainer/collator/recipes

Claim 强度沿用 C0–C3：C0 可由项目文档支撑“了解/参与”；C1 的“实现/迁移”需要 local code/config/log；C2 的“设计/优化/定位”需要 diff、对照和 bad case；C3 的指标/完成态需要 immutable checkpoint、完整 protocol、raw result 与 ownership。[1][3][7][8]

2. Role Fit 与岗位能力映射

岗位能力项	当前材料	强度	缺口	当前策略
VLA action representation	source audits 已核对 FAST、flow、chunk、normalize/denormalize；formal code 明确把 DROID action window 序列化为 JSON assistant text	强（上游 + local-direct）	local route 没有 FAST processor/roundtrip 证据	分开讲 upstream FAST 与 local JSON/CE
Multimodal model adaptation	formal code 直接加载 `Qwen3VLForConditionalGeneration` 并注入 PEFT LoRA；报告记录 eager attention 与 6.42M trainable params	强（local-direct）	缺 checkpoint binary/load smoke 与 per-file ownership	可讲“formal trainer path 已实现”，不写已验证模型效果
Data contract	recipes/config 与 public smoke 明确 `H=15`、`D=72`；Batch 2 复核发现 current formal JSON formatter 对 action/state 使用 `limit=16`	中（local-direct + representation-conflict）	parser、valid dims、axis/unit、私有数据 revision 与完整 sampling 仍缺	必须同时报告 config/smoke shape 与 formal formatter 宽度冲突，不能只报 `(15,72)`
Distributed/hardware migration	formal report 记录三条 100k completion，scripts/config 与 C500/MACA deployment 记录可读	中高（report-supported + remote-referenced）	完整 log/checkpoint/load smoke 不在本地	说“报告支持的远端完成态，raw artifact 待补”
Closed-loop evaluation	ALOHA 3-episode、LIBERO 单任务 true rollout、Bridge/Fractal replay/adapter diagnostics 有 local-direct artifact；official eval script 可读	中（local-direct diagnostics）	formal Qwen checkpoint 的四套件 raw eval 仍缺	只讲诊断/官方 baseline rollout，不把它们绑定 97.4
Experimental reasoning	224/h10/40k 与 resolution/horizon/pretrain 分析	低	单点最优，缺 factorial/multi-seed	只能称 hypothesis，不作因果结论
Ownership	Project Git initial snapshot author 为 `mousongzhe`；逐文件 ledger已建立	中低（Git-supported）	360 个 OpenPiVLA paths 一次进入 root snapshot；候选人尚未确认亲手/协作/工具边界	只能说“提交中包含这些实现”；按 ledger 逐项确认后再升级动词

没有具体 JD，因此本页只做 VLA/robot-policy 通用能力映射，不编造公司面试流程。

3. Frozen inputs 与官方 baseline 核验

Source	SHA-256 / revision	能支持	不能支持
optimized resume [1]	`5374d4b6f6617394381bb0c17e81c298ccef34bf6c3fc329e6e6bf04d01f1d4c`	已安全移除 OpenPiVLA 未闭合 benchmark 数字，保留 direct engineering claims	raw provenance
optimized PDF [2]	`65b1dcd34cd6e357c6264effcf4b2683dea66508c6bc58315762b5e4c03ac74a`	A4 单页、与安全重写后的 HTML 一致	项目 raw C3 证据
OpenPiVLA 项目页 [3]	`ca24762864c998c6fe79812a1bc2aada8e3895773e64fd9c98a86a40464e0059`	已对齐 archive-summary-only、local JSON/CE 与 formal report-supported 口径	raw checkpoint/episode 实际存在性
resume plan [4]	`d9ae41dc5d6d8b3601ac4184ba68b9e5b6b8cd4b1b452ec42172768f1d9248af`	当前安全简历口径与禁写边界	checkpoint 与 protocol
research resume [5]	`9a8828ae0b6317c107ab16ec3d106cad8ba944854469e7919a7ba0c1342b643f`	action JSON、H/D、四套件历史表述	raw code/eval
engineering resume [6]	`2aa3f0dae040ad5fb6bff32d68b5f4fff574bf4b2be80abc93d9f29b96f78732`	trainer/collator/rollout 历史表述	raw code/eval
π0/FAST/OpenPI audit [7]	`ded60a239f8edbf2171096c89fa5fd85e41e51efbbc260df9c188ae57c9fb829`	frozen OpenPI/FAST action path、H/K/norm 边界	本地 97.4/Qwen route/ownership
architecture/execution audit [8]	`bef8ebb6540644cdbab76a92adaa71ba97523c17e04ce19632b71d54b11db809`	model→decode→controller 与 audit contract	本地 checkpoint/axis/unit/latency
VOL 03 source ledger [9]	`c77e1cc6ee161114ab7c19f0d95ee93bfcaf87ea10c07eccd1212360e0877455`	明确 archived 97.4 ≠ current formal result	任一 local result
frozen OpenPI root README [10]	`openpi@15a9616a`，raw SHA `567fc57766bd736664f50126cab8145af4071ca16cc5e238f2b11e369169c3b5`	official checkpoint/config 入口	96.85 table 的完整行

Source	SHA-256 / revision	能支持	不能支持
frozen OpenPI LIBERO README [11]	raw SHA `a2caaf5459c4b01837b73aa93a6f69e8cdf10385b39c34eba36afa85f83fafe9`	π0.5@30k: spatial 98.8、object 98.2、goal 98.0、LIBERO-10 92.4、average 96.85	本地协议可比性
Project Git snapshot [12]	repo `C:\Users\mousongzhe\Project`, commit `c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`, author `mousongzhe`	OpenPiVLA deployment source/artifact snapshot and coarse authorship	per-file ownership、remote checkpoint provenance
formal trainer [13]	SHA `51b67d3e04496e68076bf6461506141ebcddf9a82e6c30af1fdc0f1f2ccc6f94`	HF Qwen3-VL + PEFT LoRA、JSON action text、causal CE、save plan	FAST equivalence、actual formal checkpoint load
formal report [14]	SHA `d74126a3896f68ad44b75c7a88f1b7b9a90cb55df0c6c6beb90a5748e9e8e074`	run table、loss curves、100k report-supported completion、archived 97.4 summary and limitations	raw formal logs/checkpoints、raw 97.4 eval bundle

2026-07-17 重新审计 `C:\Users\mousongzhe\Project` 后，发现上述项目页引用的真实目录已提交到 `c4469eb`。因此当前证据不是"local direct = 0"，而是分层如下：

Evidence tier	已发现内容	仍不能证明
`local-direct`	`DEPLOYMENT.md`、`repro_train/README.md`、三份 YAML、`formal_train.py` / `data.py` / `recipes.py`、三条 `run_formal_*.sh`、三个 smoke summary、formal report、official LIBERO eval launcher、ALOHA/LIBERO/diagnostic rollout artifact	这些文件本身不包含三条 formal checkpoint binary、完整远端 log、97.4 raw episodes 或 per-suite summary
`remote-referenced`	formal report 与 deployment log 指向 `/data/msz/openpivla/logs/vla_qwen3_repro/*/formal_train.log`、`/data/msz/openpivla/outputs/vla_qwen3_repro/<run>/checkpoint-*`	当前没有 immutable remote export、checkpoint SHA 或 load smoke 可复核
`archive-summary-only`	formal report 的 `224_h10_40000` 表格：spatial 0.98、object 0.995、goal 0.99、libero_10 0.93、average 97.4	model class、checkpoint、trial/seed/init protocol、raw episode records、与 formal run 的 lineage

这轮审计只升级证据等级，不升级简历强度；投递稿和项目页仍需下一切片重写。

4. Claim reconciliation matrix

ID	当前 claim	等级	当前证据	Verdict	处理动作
`OPV-CL-001`	基于 OpenPI 改造 Qwen3-VL-2B + LoRA VLA 路径	C1/C2	`formal_train.py` 直接加载 `Qwen3VLForConditionalGeneration`、`LoraConfig` / `get_peft_model`；三份 recipe/config 与 formal report 可读	`local-direct / ownership-open`	可说"formal trainer path 已实现"；不能把可读源码等同于本人独立设计或已加载最终模型
`OPV-CL-002`	local route 将 continuous action window 编成 assistant JSON content	C2	`DroidActionTextDataset` 生成 `{"actions": ...}`；collator 只屏蔽 padding，prompt/wrapper/assistant 全部非 padding token 进入 CE；formatter probe 复核 `15×16`	`local-direct / full-sequence-CE / PASS WITH WARNING`	还缺 generation/parser/controller；不得称 assistant-token-only CE 或连续 action loss
`OPV-CL-003`	local route 复用了 FAST DCT/BPE	C2	recipe 保留 `fast_tokenizer` 字段，但 formal trainer 未调用 FAST processor；report 明确 JSON/CE 与 FAST 分离	`REPRESENTATION-CONFLICT`	禁止把 local JSON 路径称为 FAST；FAST 只作 upstream 对照
`OPV-CL-004`	DROID `H=15/D=72` 与 MindCube QA 统一训练	C1/C2	recipes/config 与 smoke 报 `(B,15,72)`；current formatter 每步 `limit=16`；72D probe 丢 840/1080 positions，empty/NaN/ragged cases 无 schema gate	`LOSSY-SERIALIZATION / PARSER-MISSING / semantics-open`	不能声称 current formal path 保留 72 维；先实现 strict parser、axis/unit、normalizer 与 controller round-trip
`OPV-CL-005`	VLA-only、VLM-only、co-train 三类 recipe	C1/C2	`recipes.py`、三份 YAML、三条 launch script、smoke summaries、formal report run table	`local-direct + report-supported`	recipe 定义可核；三条远端完成态仍需 immutable logs/checkpoints/load smoke
`OPV-CL-006`	在 MACA/C500 完成三条 100k-step formal runs 与多 checkpoint 保存	C3	committed formal report 记录三条 `complete @ 100000`、保存点与远端路径；deployment/launch script 可读	`REPORT-SUPPORTED / REMOTE-ARTIFACT-PENDING`	不写"本地 checkpoint 可加载"；补三条 log、checkpoint SHA、load smoke 后才升 verified C3
`OPV-CL-007`	archived `224_h10_40000` 四套件 99.0/99.5/98.0/93.0，平均 97.4	C3	committed formal report 有该汇总表	`ARCHIVE-SUMMARY-ONLY`	可说"归档报告汇总为 97.4%"；不能声称 raw eval、checkpoint 或 formal lineage 已闭合
`OPV-CL-008`	local 97.4 高于 official OpenPI 96.85	C3	official 96.85 已核实；local side 与 protocol comparability 未核	`COMPARISON-FAIL`	不写"高于"；需 same-suite/trial/seed/harness 与 uncertainty
`OPV-CL-009`	224/h10/40k、resolution/horizon/pretrain 导致更稳策略	C2/C3	单点最优自述；无 controlled ablation	`INFERENCE`	改成"候选解释"；不得作因果归因

ID	当前 claim	等级	当前证据	Verdict	处理动作
`OPV-CL-010`	formal Qwen runs 通过 LIBERO/ALOHA rollout 验证 controller 语义	C2/C3	official WebSocket eval 直接返回 numeric actions，A01 为 `pi0_aloha_sim`，`D=7,H=4`；formal Qwen package 无 generation/parser	`PARALLEL-OFFICIAL-LINEAGE / FORMAL-QWEN-MISSING`	可讲 official-checkpoint eval diagnostics；不能绑定 formal Qwen、JSON route 或 97.4
`OPV-CL-011`	archived 97.4 与 formal Qwen/MACA runs 构成同一项目结果链	C3	source ledger 明确二者不能直接等同	`RUN-LINEAGE-OPEN`	建 `archive→base→formal→eval` checkpoint lineage；当前分开讲
`OPV-CL-012`	Qwen 接入、MACA migration、trainer/collator/recipes 是个人核心贡献	C1/C2	相关文件在 Project `c4469eb` 初始 snapshot；逐文件 ledger 已将 13 个文件/决策分层	`GIT-SUPPORTED / LEDGER-DRAFTED / ROLE-CONFIRMATION-PENDING`	Git authorship 不是个人贡献证明；候选人需确认亲手/协作/工具边界并补 diff

5. 三个高风险断点

5.1 archived 97.4 ≠ current formal runs

项目页已经承认 97.4 是 strongest archived eval evidence，而 formal Qwen/MACA 训练只证明训练 contract 可恢复，不能直接称为 97.4 checkpoint 的来源。[3][9] 但 optimized resume 连续排列两组 bullet，面试官容易追问：“97.4 是哪条 100k run 的哪个 checkpoint？”当前没有 lineage 可以回答。

最小 lineage 必须是：

```
archive_run_id -> model_class/base_checkpoint -> local_diff/config
-> train_run_id/checkpoint_sha -> norm_stats_sha
-> eval_run_id/suite/trials/seeds -> episode_records/summary
```

5.2 97.4 与 official 96.85 的比较没有闭合

official 96.85 是 frozen OpenPI LIBERO README 的 π0.5@30k 表格事实。[11] 本地 97.4 现在有 committed report summary，但没有 raw eval bundle。即使汇总数字真实，也要对齐 suite order、task version、trials、seeds、init states、checkpoint、image preprocessing、H/K、controller、success predicate 与 aggregation。单点差 0.55pp 也不能自动称 statistically meaningful uplift。

5.3 FAST 与 assistant JSON/token 不能混写

FAST 是 normalization→DCT→quantization/BPE→token→decode/IDCT 的公开路径；项目页又描述 local Qwen LoRA 将 action window 序列化为 assistant-side JSON/token。除非 local formatter 明确调用 frozen FAST processor 并有 roundtrip test，这两者只能是“上游机制对照”与“本地候选表示”两条线，不能说 local route 完整复用了 FAST。[3][7]

6. Truth boundary 与当前安全口径

内容	当前状态	简历	面试	证据到位后
OpenPI/FAST 机制理解	`upstream-verified`	可作技术栈，不作个人成果	可机制级回答	不升级为原创
Qwen3-VL/LoRA local route	`local-direct / ownership-open`	可写“实现 formal trainer 路径/梳理 contract”，不写模型效果	指向真实源码；说明 adapter/CE/eager 与 upstream 的差异	per-file ownership + checkpoint/load smoke 后可升 C1/C2
assistant action JSON content	`local-direct / full-sequence-CE`	可写“将 action window 序列化为 assistant JSON content，并在 full-chat causal CE 中训练”	与 FAST DCT/BPE、assistant-only mask 明确分开	generation/parser/controller robustness 后升更强
three formal 100k runs	`report-supported / remote-artifact-pending`	暂不写“本人完成且 checkpoint 可用”	可说报告记录三条 `complete @ 100000`，raw logs/checkpoints 待核	immutable manifests + load smoke 后恢复 C3
archived 97.4	`archive-summary-only`	暂不放投递强结果	可说“committed archive report 汇总 97.4%，raw eval pending”	exact checkpoint/eval bundle 后恢复
high vs 96.85	`COMPARISON-FAIL`	不写“高于”	只说明 official baseline 已核实	same-protocol paired evidence 后再判断
resolution/horizon/pretrain 影响	`INFERENCE`	不写因果	说“待验证假设”	controlled ablation 后升级
personal ownership	`ownership-unverified`	用保守动词	逐项说明边界	diff/ledger 到位后写具体模块

当前仍不能升级为 verified C3 的原因

1. 97.4 的 checkpoint、trial/seed、summary 与 episodes 全缺。
2. three 100k formal runs 有 launch scripts 与 committed report，但缺 immutable formal logs、checkpoint SHA 和 load smoke。
3. archived eval 与 formal Qwen/MACA 没有 checkpoint lineage。
4. local JSON/token 已有源码证据，且与 upstream FAST 实现不同；项目页已分开表述，但 parser/roundtrip 仍待补。
5. ownership ledger 已把个人、历史项目、上游和工具辅助拆成 13 行，但候选人角色确认与 pre-snapshot diff 仍缺。

当前保守表达

在候选人确认自己确实参与该复现后，当前可用：

> 在 OpenPI 工程框架上实现 Qwen3-VL + PEFT LoRA formal trainer，将 DROID action window 序列化为 assistant JSON content，并在 full-chat causal LM CE 中与 MindCube QA 共同训练；用 vla-only、vlm-only、co-train 三类 recipe 组织 MACA/C500 训练与部署诊断。审计确认这不等价于 FAST DCT/BPE，且 `D=72` 经 `limit=16` 后为 lossy serialization，formal package 尚无 generation/parser/controller。committed report 的 100k/97.4 仍缺 raw checkpoint、protocol、episode bundle 和 archive→formal lineage，不能写“高于 official 96.85”或 formal checkpoint rollout 已验证。

7. P0 artifact manifest

Priority	Artifact	最低字段	关闭 claim	当前状态
P0	archived eval bundle	`224_h10_40000` checkpoint SHA、model class、config、norm stats、suite/task/trial/seed、summary、episodes/videos	CL-007/008/009	`PARTIAL`（committed report summary；raw bundle missing）
P0	checkpoint lineage	archive/base/formal/eval run ID、parent SHA、hardware、date、status	CL-006/007/011	`MISSING`
P0	local model/diff bundle	Qwen model class、forward/loss、LoRA targets、changed files、startup/load log	CL-001/012	`PARTIAL`（源码/config direct；diff/ownership/load smoke missing）
P0	action representation bundle	formatter、FAST processor revision if used、JSON schema、labels/loss mask、decode/parser/roundtrip tests	CL-002/003	`PARTIAL`（JSON/CE direct；FAST/parser/roundtrip missing）
P0	data contract bundle	DROID/MindCube revision、sample schema、H/D valid dims、axis/unit、mixture/sampling、padding mask	CL-004/005	`PARTIAL`（recipe/shape direct；semantic/provenance fields missing）
P0	formal run ledger	three commands/config SHAs、logs、steps reached、checkpoint hashes、load smoke	CL-005/006	`PARTIAL`（scripts/report direct；logs/checkpoints/load smoke remote-pending）
P0	rollout bundle	policy server/client revision、H/K/Hz、preprocess、normalizer、controller、episodes、success predicate	CL-010	`PARTIAL`（official baseline + diagnostics direct；formal Qwen rollout missing）
P1	comparison/ablation	official-vs-local protocol diff、resolution×H×checkpoint factorial、multi-seed CI	CL-008/009	`MISSING`
P1	ownership ledger	亲手、协作、历史归档、上游、Codex/工具辅助	CL-012	`PARTIAL`（13-row ledger drafted；candidate confirmation / pre-snapshot diff missing）

```
def accept_openpivla_result(bundle):
    # Bind the metric to one immutable model lineage, not a project title.
    require(bundle.checkpoint_sha and bundle.parent_checkpoint_sha)

    # Freeze data/action semantics and the complete closed-loop protocol.
    require(bundle.norm_stats_sha and bundle.action_schema and bundle.rollout_protocol)

    # Keep raw episode records and metric code so the score can be recomputed.
    require(bundle.episode_records and bundle.summary and bundle.metric_code_sha)

    # Separate upstream, archive, personal and tool-assisted contributions.
    require(bundle.ownership_ledger and bundle.local_diff)
    return "verified" if all_checks_pass(bundle) else "claim-only"
```

这是证据门伪代码，不是项目现有实现。

8. 必须回答的六个问题

1. `224_h10_40000` 的 model class、base checkpoint、checkpoint SHA、norm stats、eval summary 与 episode 路径是什么？
2. 97.4% 的每 suite trial 数、seed、init state、task version、H/K、controller 和 success predicate是什么？

3. archived 97.4% 与三条 Qwen3-VL/MACA 100k formal runs 是什么继承关系；哪条 formal checkpoint 做过四套件 eval?
4. local action route 是 FAST DCT/BPE、普通离散 token、JSON 文本，还是它们的组合；formatter、loss mask 和 decode/parser 在哪里?
5. DROID `H=15/D=72` 的 72 维哪些有效，axis/unit 是什么，如何映射到 LIBERO/ALOHA controller?
6. `RunRecipe`、collator、trainer、MACA migration、eval scripts 中哪些是你亲手改，哪些来自 archive/upstream，哪些由工具或协作者完成?

9. Preliminary SP/SSP project scorecard

这是项目证据就绪度，不是候选人能力评分；置信度为 `Medium`。local-direct 工程证据提高了工程闭环判断，但 C3 指标与 ownership 仍未闭合。

维度	分数（5）	依据
岗位相关性	4.5	VLA action、multimodal training、LIBERO eval 与目标岗位直接相关
机制深度	4.0	上游 audit 已覆盖 FAST/flow/H-D-K/norm/controller
工程闭环	3.5	Project snapshot 有 trainer/recipe/script/report/smoke/rollout diagnostics；formal raw logs/checkpoints 仍 remote-pending
指标可信度	1.5	official 96.85 可核；97.4 仅 committed report summary；formal 100k 为 report-supported
Ownership 清晰度	2.0	per-file ledger 已建立，但没有新增个人角色确认或过程证据，因此不加分
抗追问能力	3.0	能讲 JSON/CE、LoRA、recipe、eager 与 deployment；lineage/FAST/protocol 仍是高风险分叉
合计	**18.5 / 30**	代码与工程证据已出现，C3 结果与 ownership 仍是主要短板

一票否决项：current resume/PDF 仍含未核验 97.4 与“高于官方”；archive/formal lineage 未闭合；formal Qwen checkpoint/load smoke 与 raw benchmark bundle 缺失。

10. 分阶段补强计划

时间	动作	完成标准	状态
半天	用已发现源码/报告回答 representation、recipe、formal-run 状态；逐项确认 ownership ledger；先降级投递稿	13 个 ledger 行有亲手/协作/工具/上游角色，强数字保持唯一 lineage	`ledger drafted / candidate confirmation pending`
1 天	收集远端 model/config/diff、formatter/loss/parser、三条 formal run logs/checkpoints/load smoke	P0 manifest 至少关闭 4 项	待补做
3 天	从 raw episodes 重算四套件分数；冻结 official/local protocol diff	97.4 可由命令和 records 重算	待补做
1 周	做 resolution×H×checkpoint 最小 ablation、多 seed 与 action roundtrip/controller replay	因果与可比性不再依赖单点叙事	待补做

11. 下一切片

安全重写、Batch 1/2 的 12 张题卡、13-row ownership ledger 与 parser/round-trip negative-evidence gate 已完成。ledger 证明 custom path 不在 frozen upstream tree，并记录 root snapshot 的粗粒度 Git 边界；它没有替候选人完成角色确认。下一步优先做 ownership confirmation drill，或补 immutable formal checkpoint/generation/parser bundle；97.4、formal lineage 与 official comparison 继续作为高压待核项。

参考文献（References）

1. optimized resume，已安全重写 OpenPiVLA bullet。
2. optimized resume PDF，A4 单页安全重写派生版本，SHA `65b1dcd34cd6e357c6264effcf4b2683dea66508c6bc58315762b5e4c03ac74a`。
3. OpenPiVLA 项目面试页，archived/formal/Qwen/ownership claim 入口。
4. resume restructure plan，当前 OpenPiVLA 简历口径。
5. research resume，历史 action representation 与指标表述。
6. engineering resume，历史 trainer/collator 与 rollout 表述。
7. VOL 03 π0/FAST/OpenPI source audit；canonical Markdown，review JSON。
8. VOL 03 Architecture / Execution audit；canonical Markdown，review JSON。

9. VOL 03 source ledger，记录 archived 97.4 与 current formal run 边界。
10. Physical Intelligence, `openpi@15a9616a` README，Apache-2.0 code，访问 2026-07-17。
11. Physical Intelligence, `openpi@15a9616a` LIBERO README，lines 66–71 给出 official checkpoint 和 96.85 table，访问 2026-07-17。
12. Pertsch et al., FAST: Efficient Action Tokenization for Vision-Language-Action Models, 2025。
13. 本地 formal trainer：`C:\Users\mousongzhe\Project\openpivla_deployment\repro_train\openpi_src\src\openpi\training\vla_qwen3_repro\formal_train.py`，SHA `51b67d3e04496e68076bf6461506141ebcddf9a82e6c30af1fdc0f1f2ccc6f94`。
14. 本地 formal report：`C:\Users\mousongzhe\Project\openpivla_deployment\artifacts\vla_formal_reproduction_report_20260617.html`，SHA `d74126a3896f68ad44b75c7a88f1b7b9a90cb55df0c6c6beb90a5748e9e8e074`。
15. 本地部署记录：`C:\Users\mousongzhe\Project\openpivla_deployment\DEPLOYMENT.md`，SHA `c0a3dd0c0da8c1c0d36023046d5337900a71680b413657bc9b1062eef2eea14e`。
16. 本地 official LIBERO launcher：`C:\Users\mousongzhe\Project\openpivla_run_libero_official_eval.sh`，SHA `0bc9186f10b6a91ede65d933f1d99dab09827b2285ee1a437a96148dc35e13a3`。
17. OpenPiVLA Batch 2 evidence freeze；canonical Markdown，review JSON。
18. OpenPiVLA Ownership Ledger；canonical Markdown，review JSON。
19. OpenPiVLA JSON/CE + H/D Shape Contract Artifact Gate；canonical Markdown，review JSON。
20. OpenPiVLA Action JSON Parser / Round-trip Artifact Gate；canonical Markdown，review JSON。

Part 28 / 49 · `interview_materials/vla/evidence/projects/openpivla_ownership_ledger.html` · SHA-256 `564CF577B1DD41D98FCD8AE568A3BF68EF9A47AF4CA37CACF150CD37866A4851`

返回本卷目录

Canonical Markdown SHA-256：`564cf577b1dd41d98fcd8ae568a3bf68ef9a47af4ca37cacf150cd37866a4851`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\openpivla_ownership_ledger.md`

OpenPiVLA Ownership Ledger：个人贡献、文件归属与面试口径

状态：`ownership_audited / coarse_git_supported / candidate_confirmation_pending / WARN` 审查日期：2026-07-19 证据截止：2026-07-19 项目范围：`C:\Users\mousongzhe\Project\openpivla_deployment`、OpenPI frozen upstream、当前简历和 OpenPiVLA 项目题卡。 结论先行：本地 Qwen3-VL/LoRA trainer、JSON action route、recipes、smoke、deployment/eval scripts 与报告均有可读取的 local-direct 证据；但它们一次性进入一个 1326-path 的初始仓库快照，OpenPiVLA 自身没有更早的逐文件 Git 历史。因此当前可以证明“本地快照包含一套区别于 frozen upstream 的实现”，不能仅凭 commit author 证明每个文件由候选人独立设计、编写和运行。个人 ownership 仍需候选人逐项确认，并补 before/after diff、设计记录、运行日志或协作说明。

2026-07-19 evidence update：`OpenPiVLA JSON/CE + H/D Shape Contract Artifact Gate` 已直接复核 `formal_train.py` 的 `limit=16` serializer、`H=15,D=72` recipe/smoke 与 AST formatter probe。它强化了 `OPV-OWN-002/003` 的技术事实与风险发现，但不改变个人 role-confirmation-pending、checkpoint lineage 或 benchmark ownership 状态。

2026-07-19 parser/round-trip update：`Action JSON Parser / Round-trip Gate` 证明 Python JSON 只可逆恢复 `15×16` retained prefix、相对 `15×72` 丢 840 positions；formal package 没有 action-output parser，collator 是 full-sequence CE。`pi0_aloha_sim` 的 7D/4-step official WebSocket artifact 是平行 lineage。技术风险已复核，但谁设计/发现/修复仍由候选人确认。

1. 问题（Question）

面试官问“OpenPiVLA 哪部分是你做的？”时，应该怎样把以下五件事分开：

1. OpenPI / FAST 的公开上游设计；
2. 本地仓库中确实存在的代码、配置、脚本和报告；
3. 候选人亲手实现、修改、调试或验收的动作；
4. 协作者、历史归档或工具辅助产生的部分；
5. benchmark 数字、checkpoint 和闭环结果的归属。

本页是 ownership ledger，不是重新认领项目，也不是对 97.4% 的结果升级。没有候选人确认或独立过程证据的条目一律保持 `role-confirmation-pending`。

2. Role Fit 与 JD 能力映射

当前没有具体 JD，因此只映射通用 VLA / Robot Policy / Multimodal Training / Embodied Evaluation 岗位。

岗位能力	本地可核证据	Ownership 强度	当前策略
多模态训练适配	Qwen3-VL loader、PEFT LoRA、chat template、causal CE、adapter save path	文件事实强；个人角色待确认	面试先解释输入、label、loss、保存语义，再说明个人动作
VLA action representation	action window → compact JSON；`H=15,D=72` 与 formatter `limit=16` 冲突	文件事实强；设计决策 ownership 未证实	用“采用/实现该路线”替代“发明新 action tokenizer”
数据与 recipe 工程	VLA-only、VLM-only、co-train recipes/configs/scripts；public smoke manifests	local-direct；full-data 与个人决策部分未闭合	可讲 schema、mixture、public/private 边界，不把 smoke 当 formal result
国产算力迁移	MACA/C500 runtime、eager path、remote layout、formal report	report-supported；raw remote run 与个人操作记录不足	可讲迁移问题和验收设计，不写硬件带来算法提升
Policy serving / evaluation	ALOHA、LIBERO、LeRobot action eval scripts 与 artifacts	local-direct；主要绑定 official checkpoint	可讲 eval engineering，不绑定 formal Qwen 或 97.4%
实验与结果	formal report、smoke summaries、archive table	summary/report 层；C3 与 ownership 均未闭合	禁止“本人训练并超过官方”

3. Ownership 证据合同

3.1 五层 AND-gate

一个“我负责并完成了 X”的强主张，至少需要五类证据共同成立：

$$G(c) = I(c) \wedge A(c) \wedge D(c) \wedge V(c) \wedge O(c).$$

变量解释：`I` 是 identity，证明审查的是哪个文件、commit、run 或 checkpoint；`A` 是 artifact，证明实现或结果实体存在；`D` 是 decision/process provenance，证明为什么这样做、由谁决定和如何修改；`V` 是 verification，证明候选实现被运行或验收；`O` 是 ownership boundary，证明个人、协作者、工具和上游各承担什么。直觉上，Git author 只覆盖 `I` 的一部分，代码存在只覆盖 `A`，两者都不能替代 `D/V/O`。实现映射是 `git log/diff`、源码、设计记录、命令日志、review/协作记录与候选人确认表。

3.2 状态词汇

状态	含义	可用动词
`upstream-verified`	frozen paper/repo 直接支持的公开设计	“上游实现/论文提出”
`local-snapshot`	本地 commit 中存在文件或 artifact	“本地快照包含”
`git-coarse`	commit author/committer 是候选人，但无细粒度历史	“由候选账号提交快照”
`self-attested`	简历/口述声明本人做过，尚无独立过程证据	“候选人自述负责”
`role-confirmed`	候选人逐项确认亲手/协作/工具边界	“我负责/我实现”
`process-supported`	有 before/after diff、设计记录、命令或 review 佐证	“我设计并完成/定位并修复”
`result-owned`	checkpoint、protocol、raw result 和个人角色连续闭合	“我完成并得到……”

4. Git 与 upstream 冻结事实

4.1 本地仓库

检查	结果	能证明	不能证明
Project commit	`c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`，2026-06-28，author/committer `mousongzhe`	候选账号提交了初始快照	快照内每个文件均为其独立原创
commit 范围	root commit 一次加入 1326 paths；其中 OpenPiVLA 360 paths	OpenPiVLA 作为大快照的一部分进入 Git	生成顺序、协作者、工具或历史来源
numstat	OpenPiVLA text paths 合计约 66,757 additions；binary paths 不计行数	快照规模与审查范围	工作量归属或代码质量
path history	`git log --all -- openpivla_deployment` 只有上述 1 条 commit	当前 Git 没有更早/后续 OpenPiVLA diff	文件在进入 Git 前如何形成
key-file history	`formal_train.py`、`data.py`、`recipes.py`、`DEPLOYMENT.md` 的 `--follow` 均只有 root commit	关键文件与快照绑定	哪些行是手写、移植、生成或协作
current worktree	OpenPiVLA 路径无未提交修改；Project 的 dirty state 位于 UWM 报告文件	本轮审查未混入 OpenPiVLA 未提交变化	remote 主机是否有更新 artifact

4.2 Frozen OpenPI 对照

Physical Intelligence/OpenPI commit `15a9616a00943ada6c20a0f158e3adb39df2ccac` 的 GitHub tree 在 2026-07-19 返回 177 项、`truncated=false`；其中没有 `vla_qwen3_repro` 或 `train_vla_qwen3*` 路径。这个事实支持“本地训练模块不是该 frozen upstream tree 中的原文件”，但不支持“因此必然由候选人独立原创”。本地实现仍可能来自历史私有分支、协作者、工具辅助或重新组织的代码，必须由 role ledger 继续区分。

5. Per-file / per-decision ownership matrix

ID	文件 / 决策	Local artifact 事实	Git / upstream 事实	当前 role verdict	当前安全口径	升级所需证据
`OPV-OWN-001`	`formal_train.py`：Qwen3-VL + LoRA trainer	608 行；加载 `Qwen3VLForConditionalGeneration`、注入 LoRA、计算 `outputs.loss`、保存 adapter/processor；SHA `51b67d...ccc6f94`	blob `aafd6e...`；仅 root commit；frozen upstream 无该路径	`local-snapshot / git-coarse / role-pending`	“项目快照包含一条 Qwen3-VL+LoRA formal training path”	候选人确认具体函数；pre-snapshot diff；运行命令和 review
`OPV-OWN-`	action JSON dataset / collator	`DroidActionTextDataset` 构造 `{"actions":...}`；Qwen chat template；只有	位于同一 608 行文件；无逐函数历史	`local-direct / full-sequence-CE /`	“本地路线采用 assistant JSON content + full-chat	为什么选 JSON、为何没有 assistant mask、

ID	文件 / 决策	Local artifact 事实	Git / upstream 事实	当前 role verdict	当前安全口径	升级所需证据
002		padding label 为 -100，其余 full chat token 监督		decision-owner-pending	causal CE，不等价于 FAST”	parser/round-trip、候选人修改点
OPV-OWN-003	H=15,D=72 与 limit=16	recipe/smoke 输出 (B,15,72)；formatter 每步最多 16 数；probe 确认丢 840/1080 positions，parser missing	冲突与 negative evidence 可直接复核	technical-finding-verified / parser-missing / fixer-pending	“审计发现 lossy shape contract 尚未闭合”	谁发现、何时、是否在别处修复、strict parser 与 controller mapping
OPV-OWN-004	recipes.py + YAML + launch scripts	三类 mode、数据组、100k 计划、batch/save/eager 参数可读	recipes blob 89a076...；三组文件都只在 root commit	local-snapshot / role-pending	“本地工程组织了三类可审计 recipe”	逐文件亲手/协作确认；历史 shell/config diff；run manifest
OPV-OWN-005	data.py / public smoke	288 行；public robot/VLM roots、shape padding/repeat、manifest 与 batch；三份 smoke summary 可读	blob ed7b80...；root commit；smoke 明确 private_data_deployed=false	local-direct / verification-local / role-pending	“public smoke 验证 schema、shape 和 tiny step”	smoke command/stdout、谁运行、private data manifest、同 formatter parity
OPV-OWN-006	runner.py / model_probe.py	runtime、base-model probe、tiny train step、public/full plan 分层	blobs 878184... / 3d87d1...；root commit	local-snapshot / role-pending	“项目提供分层 smoke/probe 工具”	候选人实现范围、failure log、工具辅助与 review 记录
OPV-OWN-007	eager / MACA formal path	CLI 默认 eager；deployment/report 记录 MACA Torch 与历史 flash-attention failure	code + report 直接；full formal logs/checkpoints 仅 remote-referenced	engineering-choice-supported / decision-owner-pending	“在 MACA 上采用 eager 作为兼容路径”	候选人决策确认、eager/flash A/B、raw traceback、remote log/export
OPV-OWN-008	formal 100k runs / checkpoint saving	report 记录三条 complete @ 100000；代码可保存 adapter/processor	remote log/checkpoint 不在本地；path 不是 bytes	report-supported / result-owner-blocked	“报告记录三条 formal completion”	三条 raw log、checkpoint SHA、load smoke、run operator 与 reviewer
OPV-OWN-009	LIBERO / ALOHA rollout scripts	322 行 LIBERO true rollout、265 行 ALOHA rollout；episode/video/summary artifacts 可读	blobs 57a976... / a29f98...；root commit；使用 official checkpoints	local-direct / execution-role-pending	“项目搭建 official-checkpoint rollout diagnostics”	谁编写/运行、命令、env commit、formal Qwen checkpoint parity
OPV-OWN-010	LeRobot action eval / dashboards	457 行 server eval；timeout、future-window、L1/action norm、summary/dashboard	blob 5dcefd...；artifacts 在 root snapshot	local-direct / execution-role-pending	“项目提供 offline action-interface diagnostics”	运行者/作者确认、raw request trace、metric reducer tests
OPV-OWN-011	DEPLOYMENT.md / remote operations	543 行记录 remote layout、stop boundary、checkpoint/log 路径和 simulator classification	blob b358c4...；root commit；文档不是 remote state proof	documentation-direct / operator-pending	“部署记录定义了环境和证据边界”	shell history、sync manifest、remote file hashes、候选人操作范围
OPV-OWN-012	formal report / archived 97.4	1439 行报告；含 formal run table、loss curves、224_h10_40000 97.4 summary	blob bd0fbe...；raw checkpoint/protocol/episodes 缺失	report-supported / C3-and-owner-blocked	“committed report 汇总 archived 97.4%”	raw eval bundle、reducer、checkpoint lineage、运行/分析/审查角色
OPV-OWN-013	OpenPI / FAST 方法	frozen upstream 直接支持 policy、normalizer、FAST DCT/BPE、official LIBERO baseline	public upstream；本地 custom path不在 frozen tree	upstream-only	“我基于/对照 OpenPI 与 FAST”	永远不升级为个人原创；只确认适配或复现部分

6. Claim → ownership reconciliation

面试/简历 claim	Claim 等级	当前 Evidence Contract	当前处理
“实现 Qwen3-VL + PEFT LoRA formal trainer”	C1	code exists + custom path absent upstream + coarse Git；缺个人逐项确认/diff	候选人确认 OPV-OWN-001/002 后可保留；否则降为“参与搭建”
“设计 assistant-side JSON action representation”	C2	code fact成立；设计动机、alternative、owner、roundtrip 缺	当前改为“采用/实现 JSON action route”
“组织三类 recipe”	C1/C2	recipes/config/scripts direct；个人决策和 formal raw artifacts不全	候选人确认后可写“负责组织”；不写三条模型均验证
“完成 MACA/C500 迁移”	C2	eager/runtime/deployment/report支持；raw remote run和operator history缺	写“迁移训练路径/处理兼容性”；不要写性能 uplift
“完成三条 100k 训练”	C3	report-supported + remote-referenced	不作为独立 verified 结果；面试主动说明 raw artifact pending
“取得 LIBERO 97.4%，高于官方”	C3	archive summary；protocol/lineage/raw/ownership 均缺	禁写“高于官方”；只可陈述报告中存在该归档汇总
“完成 LIBERO/ALOHA rollout 验证”	C2/C3	local artifacts存在，但使用 official checkpoints	可讲 eval engineering；不能绑定 formal Qwen/97.4
“OpenPiVLA 核心方法是我原创”	C2/C3	与 frozen OpenPI/FAST 冲突	明确拒绝；贡献应落在适配、contract、诊断和证据工程

7. Production Gap：从快照到可抗追问 ownership

当前做法	大厂面试官会追问	缺口等级	最小补证据	安全表达
一次性 initial snapshot	哪个文件是你改的？相对上游 diff 在哪？	P0	导出 pre-snapshot 来源、逐文件 diff/patch、角色确认	“本地快照包含；个人改动以 ledger 为准”
文档引用 remote checkpoint	checkpoint 能加载吗？和 log 对得上吗？	P0	checkpoint/processor SHA、load stdout、run manifest	“报告记录完成，权重待核”
JSON action route	为什么这样设计？为何 full-sequence CE？如何 decode/controller round-trip？	P0	schema、assistant span、strict parser、sentinel、axis/unit/normalizer	“本地采用 JSON/full-chat CE，parser/controller 闭环仍缺”
official checkpoint diagnostics	是你训练的模型吗？	P0	formal Qwen checkpoint → same harness manifest	“只证明 eval pipeline，不证明 formal 模型效果”
report 97.4	trial/seed/init/protocol/raw episodes 呢？	P0	raw eval bundle与冻结 reducer	“archive-summary-only”
未记录协作者/工具	哪段由 AI/同事生成？你如何审查？	P0	contribution ledger、review notes、现场解释	“按亲手/协作/工具辅助逐项说明”
只有最终代码	遇到什么 bad case，怎么定位？	P1	failure→hypothesis→diff→rerun timeline	“当前代码证明结果形态，不证明问题解决过程”

8. Question Tree 与回答分层

8.1 主追问树

```
你个人做了什么?
  ├ 回答 trainer
  │    ├ 哪些 class/function 亲手写?
  │    ├ 相对 frozen OpenPI 的 diff 在哪?
  │    └ 为什么 JSON/CE, 不用 FAST/action expert?
  ├ 回答 MACA 迁移
  │    ├ 原始 traceback 是什么?
  │    ├ 为什么选 eager? 做过 A/B 吗?
  │    └ 哪条 checkpoint 真正加载过?
  ├ 回答评估
  │    ├ official checkpoint 还是 formal Qwen?
  │    ├ H/K、normalizer、controller、success predicate是什么?
  │    └ 97.4 的 raw episodes 在哪?
  └ 回答“全部负责”
       ├ 初始快照为何没有逐文件历史?
       ├ 协作者和工具辅助边界是什么?
       └ 现场选一个函数解释输入、输出、失败和验收
```

8.2 高压题：这个项目到底哪部分是你写的？

- 危险回答：整个 OpenPiVLA 都是我写的；Git author 是我，所以这些代码全是我的。
- 可过回答：我负责本地 Qwen/LoRA formal path、recipe 和评估适配；OpenPI/FAST 是上游。当前 Git 是一次性快照，逐文件历史不完整，所以我会按文件说明亲手、协作和工具辅助边界。
- 强回答（仅在候选人确认后使用）：我亲手负责 `formal_train.py` 中的 Qwen/LoRA load、JSON action dataset/collator 和 adapter save，负责 `recipes.py` 与三类 launcher 的 run contract，并在 MACA 上处理 eager fallback；rollout scripts 中我负责的是某几个明确函数和验收。OpenPI policy/FAST 不属于我的原创，97.4 也只是 archive summary。可以现场从一个 sample 进入 formatter、labels、`outputs.loss` 和 checkpoint save 逐行解释，同时指出 `D=72` 与 `limit=16` 尚未闭合。

强回答不是自动批准稿；候选人没有逐项确认前，只能使用“可过回答”。

9. SP/SSP 证据就绪度评分

这不是候选人能力评分，而是 OpenPiVLA 项目当前能否抗大厂追问的证据评分。

维度	分数（5）	依据
岗位相关性	4.5	VLA action、multimodal training、policy serving 和 LIBERO 高相关
机制深度	4.0	能解释 Qwen/LoRA、JSON/CE、FAST 边界、H/D/K 与 controller contract
工程闭环	3.5	trainer/recipe/smoke/deployment/eval artifact 齐；formal raw checkpoint 与 lineage 缺
指标可信度	1.5	97.4 仍是 archive-summary-only；official 96.85 不可直接比较
Ownership 清晰度	2.0	本页完成逐文件分类，但个人动作、协作和工具边界尚未由候选人确认
抗追问能力	3.0	技术路径可讲；一旦追问 diff、运行者、checkpoint 与 raw result 仍有断点
合计	**18.5 / 30**	ownership 结构已建立，但没有新增独立个人贡献证据，不人为加分

一票否决项：把 upstream 当原创；把 root commit author 当全部文件作者；把 official checkpoint rollout 绑定 formal Qwen；把 report/path 当 checkpoint；把 archive 97.4 写成本人超过官方。

10. 候选人确认表

下一次口述或材料核对时，只需逐项回答“亲手 / 协作 / 工具辅助 / 上游 / 不确定”，并给出一条可验证细节：

1. `formal_train.py` 的哪些函数由你从零写、改写或审查？原始来源在哪里？
2. JSON action representation 是谁决定的？为什么设 `limit=16`？是否有 parser？
3. `recipes.py` 的三条 recipe 是你根据哪些历史日志恢复的？哪些字段只是归档转述？
4. 三条 `run_formal_*.sh` 是谁编写并在 remote 执行的？你是否亲自查看过最终 checkpoint？
5. eager fallback 是你定位并决定的吗？原始 flash-attention traceback 和验证命令是什么？
6. ALOHA、LIBERO、LeRobot 三类 eval scripts 中哪些由你编写、修改或只负责运行？
7. formal report 与 dashboard 是手写、脚本生成、AI 辅助还是协作产物？你审查了哪些结论？
8. 参与该项目的其他人、AI 工具或历史代码分别贡献了什么？

这些回答没有发生前，不创建 coaching state，也不把本页状态升级为 `role-confirmed`。

11. 分阶段补强计划

时间	动作	完成标准	状态
半天	完成上述 8 项角色确认；每个关键文件选择 1 个可现场解释的函数	13 个 ledger 行都有 `role` 和 `source`	待候选人确认
1 天	找回 pre-snapshot 来源，生成 custom path 相对上游/历史分支的 diff；记录工具/协作者	Qwen/JSON/recipe/eval 的 authored/modified/reviewed 边界可复查	待补做
3 天	导出三条 formal log/checkpoint manifest，并做 load smoke；补 action roundtrip	C1/C2 和 run ownership 不再只靠快照/报告	待补做
1 周	补 formal Qwen same-harness eval、raw episodes、protocol diff 和失败复盘	C3 才有升级可能；不保证得到 uplift	待补做

12. Output Pack：当前可用与补证后表达

12.1 简历 bullet 三档

- **保守版（当前外部证据即可支持）**：参与 OpenPI/OpenPiVLA 训练复现，搭建 Qwen3-VL + PEFT LoRA formal training path、VLA/VLM/co-train recipes 与 public smoke/eval diagnostics；明确 local JSON/causal-CE 与 FAST/action expert 的实现边界。
- **标准版（候选人逐文件确认后）**：负责 Qwen3-VL + LoRA trainer、assistant-side JSON action supervision 与三类训练 recipe 的实现和 MACA 兼容性适配，并通过 public smoke、checkpoint 保存合同与 official-checkpoint rollout diagnostics 验证训练/评估接口。
- **偏强版（必须补 diff、checkpoint/load、formal eval 后）**：设计并落地可追溯的 Qwen-based VLA training/eval pipeline，闭合数据、动作表示、checkpoint、policy serving 与 LIBERO raw evaluation lineage；具体结果只使用同协议可重算数据。当前禁止写成已完成态。

12.2 两分钟项目叙事骨架

> 这个项目的上游基础是 OpenPI 和 FAST，我不会把 policy/action tokenizer 的公开方法算成个人原创。项目本地恢复出一条可审查的 formal path：Qwen3-VL + PEFT LoRA，机器人 action window 写入 assistant JSON content，当前 collator 实际优化 full-chat causal CE，并组织 VLA-only、VLM-only、co-train 三类 recipe；在 MACA/C500 环境里用 eager、public smoke、adapter/processor 保存和 deployment diagnostics 检查 contract。 当前最重要的技术发现是三条边界：FAST DCT/BPE 与 local JSON text 不能混写；`H=15,D=72` 经 `limit=16` 丢失 77.78% schema positions 且没有 formal parser/controller；7D/4-step official WebSocket eval 也不是 formal Qwen lineage。项目报告的 100k/97.4 仍缺 checkpoint、raw episodes、protocol 与 lineage，所以我不声称超过 official 96.85。个人贡献只按候选人确认的 trainer、recipe、兼容性与评估脚本逐项说明。

叙事最后一句必须依据候选人确认表改写；未确认前不要把“项目本地做的是”自动替换为“我独立完成”。

12.3 面试前 6 题

1. 从 DROID row 到 Qwen `outputs.loss` 的完整数据流是什么？
2. JSON/CE 路线与 FAST DCT/BPE 的目标、token 和 decode 差异是什么？

3. `H=15,D=72` 与 `limit=16` 为什么是高危 bug，而不是普通配置差异?
4. eager fallback 解决了什么已知失败，又牺牲了什么？怎样做 matched A/B?
5. report-supported 100k、checkpoint verified、benchmark verified 三者差在哪?
6. 请逐文件说明你的亲手、协作、工具辅助与上游边界。

13. Ownership 审计伪代码

```
def accept_personal_claim(claim, ledger):
    # 1. File/run identity must be frozen; a path without bytes is insufficient.
    if not ledger.identity_and_artifact(claim):
        return "reject_strong_claim"

    # 2. A snapshot author does not prove who designed or wrote each file.
    if not ledger.role_confirmed(claim):
        return "use_participated_or_project_contains"

    # 3. Strong design/fix language requires decision history and verification.
    if claim.level >= "C2" and not ledger.process_and_test_supported(claim):
        return "downgrade_to_implemented_or_adapted"

    # 4. Numeric result ownership requires checkpoint, protocol and raw outputs.
    if claim.level == "C3" and not ledger.raw_result_chain_complete(claim):
        return "report_summary_only"

    return "claim_allowed_with_exact_scope"
```

该伪代码是本仓库的面试真实性 gate，不是 OpenPI 的官方实现。它把本页 `identity/artifact/decision/verification/ownership` 五层合同映射成可执行的审查顺序。

参考文献（References）

1. OpenPiVLA Claim–Evidence Inventory，12 条 claim 与 evidence tier。
2. OpenPiVLA Batch 2 Evidence Freeze，eager、smoke/formal/raw ladder、lineage 与 shape conflict。
3. VOL 05 EVAL-022 Evidence Audit，Git identity、artifact integrity 与 ownership 非替代关系。
4. OpenPiVLA Batch 1 Cards 与 Batch 2 Cards，12 张题卡，全部 unseen。
5. Local Project Git: `C:\Users\mousongzhe\Project`，commit `c4469eb8ec4bb879bdb7d12cf9a303c8264a41ea`。
6. Local formal trainer: `openpivla_deployment/repro_train/openpi_src/src/openpi/training/vla_qwen3_repro/formal_train.py`，SHA-256 `51b67d3e04496e68076bf6461506141ebcddf9a82e6c30af1fdc0f1f2ccc6f94`。
7. Local recipes/data: `recipes.py` SHA-256 `397d1b384a2bd93d9dc81b7003e6ab7bfac3fc458ea04008197c3f503f9d780e`；`data.py` SHA-256 `32c6aa76929d941c4832d2eb4c2550527d3e607e31c2a9f5507cc76eeedbc2ec`。
8. Local deployment record: `openpivla_deployment/DEPLOYMENT.md`，SHA-256 `c0a3dd0c0da8c1c0d36023046d5337900a71680b413657bc9b1062eef2eea14e`。
9. Local formal report: `openpivla_deployment/artifacts/vla_formal_reproduction_report_20260617.html`，SHA-256 `d74126a3896f68ad44b75c7a88f1b7b9a90cb55df0c6c6beb90a5748e9e8e074`。
10. Physical Intelligence, OpenPI commit API，resolved commit `15a9616a00943ada6c20a0f158e3adb39df2ccac`；tree checked 2026-07-19。
11. Physical Intelligence, FAST，upstream action tokenization method；不证明 local formal route 使用 FAST。
12. Optimized Resume 与 OpenPiVLA 项目面试页，当前候选人自述与安全投递口径。
13. OpenPiVLA Action JSON Parser / Round-trip Artifact Gate，72→16、full-sequence CE、parser absence 与 official-eval parallel lineage。

Part 29 / 49 · `interview_materials/vla/evidence/projects/openpivla_p0_bundle_audit.html` · SHA-256 `01FF0256E5E2197586FF28754FE0956B5EAFF9CDF94CA384904699617F9DBC2A`

返回本卷目录

Canonical Markdown SHA-256: `01ff0256e5e2197586ff28754fe0956b5eaff9cdf94ca384904699617f9dbc2a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\openpivla_p0_bundle_audit.md`

OpenPiVLA P0 Bundle Audit：Formal Trainer / Action Route / Eval Lineage

状态：`p0_audited / partial / checkpoint-pending / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计现有 OpenPiVLA local-direct code/config/report/smoke/diagnostic evidence；不把 report summary、official checkpoint rollout 或 Git author 升级为个人 benchmark 结果。

1. 结论

OpenPiVLA 已具备较强的工程接口证据：Qwen3-VL + PEFT LoRA formal trainer、三类 recipe、JSON action dataset/collator、smoke summaries、部署记录、official-checkpoint rollout diagnostics 和 committed formal report 均可定位。但 P0 结果链仍是 `PARTIAL`：formal checkpoint binary/hash、三条 formal run raw logs、97.4 archive 的 trial/seed/protocol/episodes、action parser/controller round-trip、pre-snapshot diff 和候选人 ownership 未闭合。

```
P0 status = PARTIAL
eligible benchmark claims = 0
eligible ownership claims = 0
allowed = trainer/data/loss/recipe/interface evidence
forbidden = “97.4 verified”, “higher than official 96.85”, “FAST-equivalent”, “personally trained end-to-end”
```

2. P0 gate matrix

P0 字段	当前证据	状态	面试边界
model/trainer identity	`formal_train.py` 加载 `Qwen3VLForConditionalGeneration`、LoRA、`outputs.loss`、adapter save	`PRESENT`	可解释 trainer path；不等于最终 checkpoint 已加载
code/config lineage	trainer、data、recipes、YAML、launch scripts、smoke/report SHA 已登记	`PARTIAL`	可讲接口与 recipe；pre-snapshot diff 仍缺
action representation	assistant JSON content、`H=15,D=72` recipe/smoke、formatter `limit=16`	`PARTIAL / CONFLICT`	必须说明 72D→15×16 lossy serialization；不能称 FAST DCT/BPE
labels/loss mask	collator 只屏蔽 padding，full-chat causal CE 可复核	`PRESENT WITH WARNING`	不得说 assistant-only CE 或 continuous action loss
parser/controller round-trip	formal package 无 action-output parser；official 7D/4-step WebSocket 是平行 lineage	`MISSING`	不能说 formal Qwen 已完成闭环控制
formal checkpoint	report/path 有 remote references，无本地 binary/hash/load smoke	`MISSING`	97.4 保持 archive-summary-only
formal run ledger	三条 launch scripts、report run table、smoke summaries	`PARTIAL`	completion 记录不能替代 raw logs/checkpoint
rollout/eval protocol	official LIBERO/ALOHA diagnostics 与 scripts 可读	`PARTIAL`	不可绑定 formal Qwen 或 97.4
raw benchmark bundle	`224_h10_40000` summary 有，raw episodes/trials/seed/init/denominator 缺	`MISSING`	不写 verified benchmark 或高于 official
ownership	13-row ledger、root snapshot author 线索	`PENDING`	Git author 不等于逐文件个人实现

3. Deterministic technical checks

- `H=15,D=72` recipe/smoke shape 与 formatter `limit=16` 同时存在；
- 72D sentinel 经过 JSON formatter 只保留 `15×16=240` 个标量，相对 `15×72=1080` 少 840；
- collator 的 mask 只排除 padding，full-chat 非 padding token 进入 CE；
- formal package 中没有 generation/decode/action-output `json.loads` parser；唯一 `json.loads` 是 MindCube 输入；
- official `pi0_aloha_sim` rollout 为 7D/4-step，不能补 formal Qwen 的 parser/controller lineage。

这些检查支持“当前工程路径的形状、mask 和缺口”，不支持模型效果或个人 ownership。

4. 安全口径

可以说：

> “我能从本地 formal trainer、recipe、collator 和 smoke/report 解释 Qwen3-VL + LoRA 的数据到 loss 路径，也能指出 action JSON 的 `H=15,D=72` 与 `limit=16` 冲突，以及 formal package 缺少 parser/controller round-trip。报告里的 97.4 仍是 archive summary，checkpoint、raw episodes、protocol 和 ownership 没有闭合，所以我不把它说成高于官方 96.85 的已验证结果。”

不能说：

1. 本地已经加载并验证三条 100k formal checkpoint；
2. 97.4 是本人可复算、协议完整的 LIBERO 结果；
3. local JSON action route 等价于 FAST DCT/BPE；
4. official-checkpoint ALOHA/LIBERO rollout 就是 formal Qwen 的效果；
5. root commit author 自动证明所有文件由本人独立完成。

5. 关闭 P0 的最小动作

1. 导出 formal checkpoint/processor/normalizer/config 的 immutable hash 与 load smoke；
2. 为 `224_h10_40000` 提供 trial/seed/init/task/version、raw episodes、success predicate 和 reducer；
3. 实现并测试 strict JSON parser、axis/unit/normalizer/controller round-trip；
4. 提供 pre-snapshot diff、运行命令、review/协作记录，回答 `OWN-OPV-001`。

在这些字段闭合前，OpenPiVLA 保持 `PARTIAL/WARN`，不改变 `PROJ-OPENPIVLA-*` 或通用题卡 mastery。

References

1. OpenPiVLA Claim–Evidence Inventory
2. OpenPiVLA Ownership Ledger
3. OpenPiVLA JSON/CE + H/D Shape Contract Gate
4. OpenPiVLA Action JSON Parser / Round-trip Gate
5. 五项目 P0 Artifact Intake Contract

Part 30 / 49 · `interview_materials/vla/sources/openpivla_action_json_parser_roundtrip_artifact_gate.html` · SHA-256 `AA5FCBD39FA14D8C6E1246F4FAF36AB3977829287F2CD4882110EB832E6CED26`

返回本卷目录

Canonical Markdown SHA-256：`aa5fcbd39fa14d8c6e1246f4faf36ab3977829287f2cd4882110eb832e6ced26`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_action_json_parser_roundtrip_artifact_gate.md`

OpenPiVLA Action JSON Parser / Round-trip Artifact Gate

状态：`local_direct_negative_evidence / retained_prefix_roundtrip_pass / controller_roundtrip_missing / WARN` 审查日期：2026-07-19 本地范围：`C:\Users\mousongzhe\Project\openpivla_deployment` 中 `vla_qwen3_repro` formal package、official OpenPI WebSocket eval script 与冻结 summary。结论先行：当前 Qwen formal route 只实现 **action window → assistant JSON training text**，没有找到 Qwen generation、action JSON output parser、shape/finiteness validator、normalizer 或 controller adapter。Python `json.dumps→json.loads` 能无损恢复被保留的 `15×16` 前缀，但相对 recipe 的 `15×72` 已丢失 840/1080 个标量（77.78%），因此这不是 controller round-trip。另一个 7D/4-step WebSocket artifact 使用 official OpenPI checkpoint，是平行 lineage，不能替 Qwen JSON route 补证据。

问题（Question）

`H=15,D=72` 的 action window 经 `limit=16` JSON formatter 后，是否存在可验证的 tokenizer/generation/parser/normalizer/controller round-trip？如果没有，当前代码到底证明了什么，哪些项目表述必须降级？

1. 项目分型与 Role Fit

字段	当前判断
项目类型	VLA training/evaluation recovery + multimodal causal-LM action representation
目标岗位	VLA / Robot Policy / Multimodal Training / Embodied Evaluation
当前最像	有 formal training source 与多条 official-checkpoint eval diagnostics，但 Qwen text-action execution contract 未闭合的研究工程项目
安全包装	“实现并审计 JSON action-supervision training path，定位 shape/loss-mask/parser lineage 风险”
禁止包装	“完成了 72D Qwen action decoder、FAST 等价实现或 formal Qwen closed-loop deployment”

本切片只关闭 parser/round-trip 证据门，不替候选人确认 ownership，也不把 local negative evidence 伪装成已修复功能。

2. Evidence freeze

ID	本地对象	直接支持	SHA-256
Lo1	`...\vla_qwen3_repro\formal_train.py`	formatter、JSON target、collator labels、Qwen/LoRA train loop；formal package 内没有 generation/parser call	`51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94`
Lo2	`...\vla_qwen3_repro\recipes.py`	`H=15,D=72` recipe intent	`397D1B384A2BD93D9DC81B7003E6AB7BFAC3FC458EA04008197C3F503F9D780E`
Lo3	`...\vla_qwen3_repro\runner.py`	public smoke runner；没有 formal Qwen action generation/parser	`F0908B9F824A0AAFD99F170F0FEB7F72E931F88EBC60594D9F5636E2539C13C5`
Lo4	`scripts\eval_lerobot_action_server.py`	official `WebsocketClientPolicy` 返回 numeric `response["actions"]`；按交集裁剪后算 L1	`B1B7FB9B7A86355B9C1843CDE971B38FD26A2DAB1F1855012867 1F79F1AB4F39`
Lo5	`scripts\export_bridge_fractal_rollout_actions.py`	official OpenPI numeric action chunk、`:7` slice、replan steps	`B8222F99F0ED08939E3B8D52DE30B9386729D3C43B3B3E6D1223AE4357EC3BC8`
Ao1	`bridge_fractal_eval_real_v2/.../widowx_spoon_on_towel/summary.json`	checkpoint=`pi0_aloha_sim`、numeric action `D=7,H=4`、4 samples	`7238B0238315852730A559AF0E01678448E4507E76F7BBCCCE608C8D426CD4E1`

这些文件位于项目外本地目录，本轮只读，没有修改项目源码或生成新的项目 artifact。项目目录本身没有可用 `.git` metadata；ownership 继续依赖现有粗粒度 snapshot ledger 与候选人确认。

3. 四层合同不能混写

层	当前实现	当前 verdict
training target	DROID rows 经 `limit=16`、round(5)、tail-repeat 后写成 assistant JSON content	`local-direct / lossy`
training objective	`labels=input_ids.clone()`，只把 attention padding 设为 `-100`	`full-sequence causal CE`，不是 assistant-token-only CE
formal Qwen inference	未找到 `model.generate`、`decode/batch_decode` 或 action-output `json.loads`	`missing`
robot execution	未找到 Qwen JSON schema validator、72D reconstruction、axis/unit、normalizer、controller adapter	`missing`

formal package 中唯一的 `json.loads` 位于 `MindCubeDataset._load_rows`，读取 VLM QA 的输入 JSONL；它不是 action prediction parser。这个负面检索只证明冻结 package 中未出现相应 call，不证明其他未归档机器或分支绝对不存在 parser。

4. 数学合同：保留前缀不等于 72D round-trip

对非空、每行至少 16 维的典型输入，令 action window 为：

$$A \in \mathbb{R}^{H \times D}, \qquad H = 15, \quad D = 72.$$

变量解释：H 是未来 action 行数，D 是 recipe/smoke 中的每行宽度。实现映射：`recipes.py` 给出 `action_horizon=15, action_dim=72`；该式是 schema intent，不是 serializer 输出事实。

当前 formatter 对每行做：

$$S(A)_{h,:} = \operatorname{round}\big(A_{h,0:\min(D,16)}, 5\big).$$

变量解释：S 是 `_format_action_sequence`，只保留前 16 个标量并四舍五入到 5 位；短 window 还会重复最后一行。实现含义：即使 JSON parser 完全正确，也只能恢复 $S(A)$，不能逆推出被删除的 $A_{:,16:72}$。

若用 Python 自带 JSON codec 记为 $J^{-1}(J(\cdot))$，本轮 probe 只得到：

$$J^{-1}(J(S(A))) = S(A) \neq A.$$

变量解释：等号左侧表示保留前缀的 Python object round-trip；右侧不等式来自 shape 和信息丢失，而不是数值误差。实现边界：这没有经过 Qwen tokenizer、model generation、schema validator、denormalizer 或 controller。

5. Deterministic probes

本轮从真实 AST 提取 `_to_flat_float_list`、`_format_action_sequence` 与 `Qwen3VLCollator`，不导入完整训练栈；所有 probes 只在内存中运行。

Probe	结果	判定
两行 72D sentinel，`H=15`	JSON parse 后 shape= `15×16`	horizon 保留，72D 未保留
serialized scalars	240；相对 `15×72=1080` 少 840	77.78% schema positions 未序列化
retained prefix equality	`True`	只证明 Python JSON codec 对已保留数值可逆
tail repeat	第 3–15 行共 13 行重复第 2 行	没有 valid length/mask，真实重复与 padding 不可区分
empty window	`15×1` 的 `[0.0]`	output width 不固定为 16/72
one empty row	`15×0`	可产生 ragged/zero-width action rows
NaN row	输出 `{"actions":[[NaN],[NaN]]}`	Python 可读回非有限值，但这是非标准 JSON，且没有 finite check
collator labels	`[11,12,13,14,-100]`	仅 padding 被屏蔽；四个非 padding token 全部监督
formal package AST calls	`generate/decode/batch_decode=0`；`loads=1` at MindCube input	没有 Qwen action output parser 证据

因此当前问题不是“parser 精度还没测”，而是 **parser contract 尚未出现在冻结 formal path 中**。

6. Loss-mask 事实纠正

旧材料把该 route 简写成“assistant token CE”。冻结代码实际是：

```
# Chat template contains user prompt, wrappers and assistant JSON content.
batch = processor(text=full_chat_text, padding=True, ...)

# Only attention padding is ignored; prompt/wrapper tokens remain labels.
labels = batch["input_ids"].clone()
labels[batch["attention_mask"] == 0] = -100
loss = model(**batch).loss
```

其 token-level objective 更准确地写为：

$$\mathcal{L}_{\text{full-seq}} = \frac{1}{N} \sum_{t:m_t=1} -\log p_\theta(x_t \mid x_{<t}),$$

变量解释：x_t 覆盖 chat template 的全部非 padding token，m_t 只是 attention mask，$N = \sum_t m_t$。实现映射：代码没有 assistant-span mask，HF causal LM 在内部做 next-token shift。它仍包含 action JSON 学习信号，但 loss 也包含可预测 prompt/wrapper；因此 loss 下降不能单独证明 action token 或 controller 行为改善。

安全术语应从“assistant-token-only CE”改为：**“assistant JSON content embedded in a full-chat causal-LM objective”**。

7. 为什么 official WebSocket eval 不能补 Qwen parser

`eval_lerobot_action_server.py` 的实际链路是：

```
LeRobot obs
  -> official OpenPI WebsocketClientPolicy
  -> response["actions"] numeric array
  -> min(gt_h, pred_h, requested_h) × min(gt_d, pred_d)
  -> offline L1 diagnostic
```

A01 明确记录 checkpoint `/.../pi0_aloha_sim`、`action_dim=7`、`action_horizon=4`。这证明 official checkpoint/server 的 numeric I/O 路径可以返回动作并形成离线 L1 artifact；它不经过 Qwen JSON generation，也不使用 `H=15,D=72` formal recipe。

此外，eval script 用 `min` 裁剪 horizon/dim 后比较，没有 assert 预测 shape 等于目标 shape；因此即使该 official diagnostic 状态为 `ok`，也不能反向证明另一路 Qwen contract 严格匹配。

8. 缺失 parser 的最小生产合同

以下是 **建议补做**，不是当前项目已有实现：

```
def parse_action_json(text, *, expected_h=15, expected_d=72):
    # 1. Use a strict JSON decoder; reject NaN/Infinity and extra prose.
    obj = strict_json_loads(text)

    # 2. Require exactly one actions field and a dense numeric matrix.
    actions = require_dense_finite_matrix(obj, key="actions")

    # 3. Reject shape mismatch instead of silently slicing/padding.
    assert actions.shape == (expected_h, expected_d)

    # 4. Apply the frozen normalizer/axis/unit contract.
    env_actions = denormalize_and_map_axes(actions)

    # 5. Enforce controller limits and record a round-trip trace.
    return controller_safety_check(env_actions)
```

最低测试矩阵：valid sentinel、missing key、extra prose、ragged rows、empty rows、NaN/Infinity、wrong H、wrong D、axis permutation、unit mismatch、normalizer mismatch、gripper bounds、tail validity mask、JSON→array→controller→logged-command 逐维对账。

9. Evidence Contract 与 claim 更新

Claim	证据结论	当前安全表达	禁止升级
`OPV-CL-002` JSON/CE route	formatter/target/labels direct；parser missing	“action window 被写入 assistant JSON content，并在 full-chat causal CE 中训练”	assistant-token-only loss、controller-ready decoder
`OPV-CL-003` FAST reuse	formal package 无 FAST processor call	“FAST 仅作 upstream 对照；local route 是 JSON text”	FAST 等价、DCT/BPE round-trip

Claim	证据结论	当前安全表达	禁止升级
`OPV-CL-004` H/D contract	`15×72` intent → `15×16` serialized prefix；840 scalars missing	“定位到 lossy shape contract，尚未修复/闭环”	72D action semantics 已保留
`OPV-CL-010` rollout	official numeric WebSocket eval direct；Qwen lineage absent	“做过 official-checkpoint eval diagnostics”	formal Qwen closed-loop rollout
Ownership	技术 finding 可复核；decision/fixer role 未确认	“仓库/项目中发现并记录风险”	“我独立定位并修复”

10. Production Gap、Question Tree 与局部评分

Production Gap

当前做法	生产追问	缺口	可补证据
`json.dumps` training target	模型输出由谁解析?	generation/parser missing	immutable checkpoint + generation samples + parser source
`limit=16`	72D 中被删的是哪些轴?	schema/axis ownership missing	field registry + sentinel axis test
repeat-last padding	哪些行有效?	valid mask/length missing	explicit `valid_horizon` + loss/execution mask
padding-only label mask	action signal占 loss 多少?	assistant-span mask/metric missing	token-span audit + masked-vs-full CE ablation
official WebSocket L1	是否验证 formal Qwen?	checkpoint lineage mismatch	same formal checkpoint/server/parser/controller bundle

Question Tree

```
为什么说 round-trip 没闭合?
  -> 如果只说 json.loads 成功: 追问 72→16 的 840 个标量去哪了?
  -> 如果拿 official WebSocket eval 作证: 追问 checkpoint/config 是否属于 Qwen formal lineage?
  -> 如果说 loss 能下降: 追问 prompt/wrapper token 是否也进入 CE?
  -> 如果声称已修复: 要求 parser source、strict schema、sentinel trace 与 controller log。
```

局部 SP/SSP 评分（只评 action-contract 证据）

维度	当前分数 / 5	原因
技术洞察	4	能定位 loss mask、shape loss 与 parallel lineage
可复现证据	3	source hash + deterministic probe 齐全，但没有 formal checkpoint/generation
生产闭环	1	parser/normalizer/controller contract 缺失
Ownership	1	candidate role/fixer 未确认
面试防线	4	可明确说清“做了什么、没做什么、如何补”

局部结论：这是一个可信的高价值 bad-case/审计故事，但还不是“Qwen VLA 部署成功”的项目结果。

11. Upgrade Plan 与 Output Pack

时间	动作	证据升级	状态
半天	冻结 JSON schema、assistant span、axis/unit registry；补 strict parser 单测	CL-002/004 从 negative evidence 到 parser contract	可补做
1 天	用 immutable Qwen checkpoint 生成 valid/invalid samples，跑 tokenizer→generate→parser	inference contract 可复现	可补做
3 天	接 normalizer/controller dry-run，记录 sentinel 与 safety rejection trace	controller round-trip 可审计	可补做
1 周	matched full-seq vs assistant-mask、16D vs intended schema ablation + closed-loop eval	训练目标与效果可比较	可补做

30 秒回答

“我复核后不会把这条路线说成已经完成 action round-trip。formal code 只把每步 action 截到 16 个数并写成 assistant JSON；Python JSON codec 能还原这个 `15×16` 前缀，但相对 recipe 的 `15×72` 少了 840 个标量，而且 package 里没有 Qwen generate、action parser、normalizer 或 controller adapter。collator 也只屏蔽 padding，所以是 full-chat causal CE，不是 assistant-token-only CE。项目里的 7D/4-step WebSocket eval 用的是 official OpenPI checkpoint，是另一条 lineage。当前可信贡献是实现/审计训练路径并定位 contract 风险，不能说 formal Qwen 已闭环部署。”

简历安全表达

- 当前可写：`审计 Qwen3-VL JSON action-supervision path，定位 72→16 lossy serialization、full-sequence CE mask 与 official/Qwen rollout lineage 分叉，并建立 parser/controller 补证矩阵。`
- 补做后可写：`实现 strict action JSON parser 与 tokenizer→generation→denormalizer→controller sentinel round-trip。`
- 当前不建议写：`完成 72D Qwen action decoder并在机器人闭环验证。`

12. 风险与下一步

1. 本报告没有 immutable formal Qwen checkpoint，因此没有运行真实 generation；parser absence 是 frozen source/package 结论。
2. Python 对 `NaN` 的默认行为不是严格 JSON 合规；真正接口必须显式拒绝非有限值。
3. official OpenPI offline L1 使用 shape 交集，不能作为严格 I/O parity 或 closed-loop success。
4. candidate 尚未确认谁设计 `limit=16`、谁发现问题、是否曾在别处修复；ownership 保持 PENDING。
5. 下一 bounded slice 优先做 OpenPiVLA ownership confirmation drill，或冻结一个 formal checkpoint/generation bundle；不新增项目题卡，不改变 12 张卡的 `unseen` 状态。

参考文献（References）

1. Local `formal_train.py`、`recipes.py`、`runner.py`，路径与 SHA 见 Evidence freeze。
2. Local `eval_lerobot_action_server.py`、`export_bridge_fractal_rollout_actions.py` 与 A01 summary，路径与 SHA 见 Evidence freeze。
3. 前一阶段 JSON/CE + H/D Shape Contract Gate。
4. OpenPiVLA Claim–Evidence Inventory。
5. OpenPiVLA Ownership Ledger。
6. OpenPiVLA Batch 1 Cards 与 Batch 2 Cards。
7. FAST / OpenPI upstream independent review，只用于 upstream 方法与 local route 边界。

Part 31 / 49 · `interview_materials/vla/sources/openpivla_aloha_action_eval_load_gate.html` · SHA-256 `AE425706E5D0B5507C99AED0176380431D5B87551EE0F9F787C0E4766E44F7A2`

返回本卷目录

Canonical Markdown SHA-256：`ae425706e5d0b5507c99aed0176380431d5b87551ee0f9f787c0e4766e44f7a2`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_aloha_action_eval_load_gate.md`

OpenPiVLA ALOHA Action-Level Eval：Checkpoint Load / Serve / L1 Gate

状态：`local-direct / log-hash-frozen / official-action-eval / task-success-not-proven / WARN` 审查日期：2026-07-21 范围：`C:\Users\mousongzhe\project\openpivla_deployment\artifacts\lerobot_action_eval_aloha_sample_real_v2` 与对应 scripts。结论先行：ALOHA LeRobot action-level eval 的 server log 直接确认官方 `pi0_aloha_sim` checkpoint 被 Orbax 恢复约 6.0 GiB、17.31 秒完成，norm stats 从 checkpoint assets 加载，server 在 8020 监听并接受 client；summary 冻结 4 个样本、每个 50 步、14D action 的 mean L1 `0.0330713950`、std `0.0205169544`。这是 action-vector 对齐/部署 harness 证据，不是 task-level success、fine-tune 成绩或泛化 benchmark。

1. Evidence freeze

ID	本地对象	SHA-256	直接支持
A01	`artifacts\lerobot_action_eval_aloha_sample_real_v2\serve.log`	`F02D0E4D08D6879C2F080B3DA905B4EA11A73B7510B7D8867785FFD1384A0868`	`pi0_aloha_sim` restore、6.0 GiB、17.31 s、norm stats、8020 listen、localhost connection
A02	`artifacts\lerobot_action_eval_aloha_sample_real_v2\summary.json`	`D367885AFB621469C81CB885C53FD36A87F02F06B8F4ED9B854F32BDB6663BAC`	official WebsocketClientPolicy、dataset/config/checkpoint 字段、4×50、14D、L1 统计
A03	`scripts\eval_lerobot_action_server.py`	`B1B7FB9B7A86355B9C1843CDE971B38FD26A2DAB1F18550128671F79F1AB4F39`	parquet/action window、image/state request、client infer、L1 计算与 summary writer
A04	`scripts\run_lerobot_action_eval.sh`	`61DFE972B20722CF08B3FA139D68B29B6C1B52286671A7705000AE3738A8BF60`	config/checkpoint/port/action-horizon 的 launch 参数与 server/client 顺序
D01	`DEPLOYMENT.md`	`C0A3DD0C0DA8C1C0D36023046D5337900A71680B413657BC9B1062EEF2EEA14E`	deployment provenance、official checkpoint 与 action-eval 边界

本页不复制远端权重、parquet 或图片，只冻结本地日志、summary 与脚本 hash。

2. Server load / client connection

`A01` 的关键顺序是：

```
Restoring checkpoint from .../checkpoints/pi0_aloha_sim/params
total bytes: 6.0 GiB
Finished restoring checkpoint in 17.31 seconds
Loaded norm stats from .../checkpoints/pi0_aloha_sim/assets/lerobot/aloha_sim_transfer_cube_human
server listening on 0.0.0.0:8020
Connection from ('127.0.0.1', 40058) opened
Connection ... closed
```

这条日志比 summary 里的 checkpoint 字符串更强，因为它记录了 restore 的执行事件、读取体量、normalizer 路径与 WebSocket 生命周期。它与 LIBERO Load–Serve–Client Lineage Gate 形成两个 task-specific official checkpoint load 样本：`pi0_aloha_sim` 与 `pi05_libero`。

但 A01 仍不等价于 immutable binary manifest：没有 `params` 目录逐文件 SHA、object-store revision、processor/tokenizer hash 或 shared run UUID。

3. Action-level eval contract

`A03` / `A04` 支持的流水线是：

```
LeRobot parquet row
  -> state/action/image fields
  -> WebsocketClientPolicy(host=127.0.0.1, port=8020)
  -> predicted action chunk
  -> align with GT action window
  -> per-sample L1 + norm statistics
  -> summary.json / dashboard / plots
```

本地脚本中可直接核对的字段与边界：

项目	证据	边界
dataset	`lerobot/aloha_sim_transfer_cube_human` 的 sample copy	只取本地 sample parquet，不等于全量 benchmark
checkpoint/config	`pi0_aloha_sim` 与 official checkpoint path	server restore direct；binary SHA 缺失
sample count	4	四个采样点来自 episode 0 的 row 0/50/100/150
action window	horizon 50	这是离线 action comparison，不是环境执行 50 步
action dimension	14	summary 与 script compare shape direct；不推断 task controller semantics
image/state request	`observation.images.top`、state key，wrist placeholders 按脚本构造	数据字段/预处理可复核；完整 camera calibration 未闭合
metric	per-sample absolute error 的 mean L1	不等价 reward/success 或 closed-loop success

L1 的实现语义可以写成：

$$\mathrm{L1}_i = \frac{1}{HD}\sum_{h=1}^{H}\sum_{d=1}^{D}\left|\hat{a}_{i,h,d} - a_{i,h,d}\right|$$

其中 (i) 是 sample，(H=50) 是 action horizon，(D=14) 是比较维度，(hat a) 是 server 返回动作，(a) 是 parquet 中的 GT action window。直觉上，它只回答“预测 action vector 与离线标签在数值上有多近”；实现映射来自 `A03` 的 `compare_horizon`、`compare_dim` 与逐元素 absolute error。它不包含环境状态转移、接触、终止或 success predicate。

4. Frozen result and comparability

sample	row/frame	L1	predicted norm	GT norm
0	0 / 0	`0.0678215474`	`2.0785110`	`2.1135752`
1	50 / 50	`0.0227967594`	`2.18176198`	`2.21180510`
2	100 / 100	`0.0148706269`	`2.14331412`	`2.14696836`
3	150 / 150	`0.0267966483`	`2.04372430`	`2.08739448`
aggregate	4 samples	mean `0.0330713950`, std `0.0205169544`	—	—

这组数字可以安全表述为：

> “我通过官方 OpenPI WebSocket server 对 ALOHA LeRobot sample 做了 4 个 action-window 对齐检查，每个窗口 50 步、14 维，mean L1 是 0.03307；同时保留了 server restore、norm stats 与 client connection log。”

不能表述为：

- ALOHA task success rate 或官方 benchmark score；
- 训练后模型的结果；
- 4 个 sample 代表完整数据集泛化；
- L1 较低就证明 controller/axis/unit/normalizer 全部正确；
- 与 LIBERO `0/1` 或 ALOHA true-physics `2/3` 直接做 uplift 比较。

5. Pseudocode mapping

```
def offline_action_eval(parquet, policy_client, row_indices, horizon):
    # 1. Read one LeRobot row and its future GT action window.
    for row_index in row_indices:
        row = parquet[row_index]
        gt = stack_future_actions(parquet, row_index, horizon)

        # 2. Build the official OpenPI observation request from image/state fields.
        request = build_openpi_observation(row)

        # 3. Query the WebSocket policy server for a predicted action chunk.
        pred = policy_client.infer(request)["actions"]

        # 4. Compare only the common horizon and dimensions; do not step a simulator.
        pred_cmp, gt_cmp = align(pred, gt, horizon=horizon)
        l1 = abs(pred_cmp - gt_cmp).mean()
```

```
        save_sample_metrics(row_index, l1, pred_cmp, gt_cmp)

    # 5. Aggregate mean/std and write summary.json and plots.
    return write_action_eval_summary()
```

此伪代码只抽象 `A03` 的离线 action comparison；它明确没有 `env.step`、controller、reward 或 success scorer。

6. Interview boundary

30 秒回答

“除了 true-physics rollout，我还把官方 `pi0_aloha_sim` 接到 LeRobot action-eval harness，先验证 server load、normalizer、WebSocket client 和 action parsing 是否闭环。4 个 sample、每个 50 步、14D action 的 mean L1 是 0.03307，std 是 0.02052。它只能说明离线 action-vector 对齐和部署链路可运行，不是 task success，也不是我训练出的 checkpoint；要升级到 benchmark，还要补完整 task matrix、controller、success scorer 和 immutable checkpoint manifest。”

高频追问

1. **为什么 L1 不是成功率？** 因为没有环境状态转移和终止条件，只比较预测 chunk 与 parquet GT。
2. **为什么 summary checkpoint 字段不够？** 因为 A01 的 server restore 才证明真正加载了 `pi0_aloha_sim/params`。
3. **14D 是什么？** 是该 action comparison 的共同 tensor 维度；本页不把它解释成某个真实 controller 的 axis/unit contract。
4. **能否与 LIBERO 0-success 对比？** 不能直接比较；一个是离线 L1、一个是 task-level physics rollout，协议层级不同。
5. **下一步是什么？** 固定 action axis/order/unit、normalizer、controller 和 success scorer，再做 matched closed-loop probe。

7. 公源 / 本地 / 未核实边界

类型	当前内容
public / official-code	OpenPI WebSocket policy server/client、官方 `pi0_aloha_sim` config/checkpoint convention
local evidence	A01–A04 的 hash、4×50×14 action comparison、L1/norm summary
local inference	server restore + summary + launch script 共同支持一次 action-eval pipeline 的运行链
unverified	checkpoint binary SHA、processor/tokenizer revision、完整数据集覆盖、controller semantics、个人 ownership、fine-tune lineage

8. 下一步

1. 把 `pi0_aloha_sim` 与 `pi05_libero` 的 params/assets 生成逐文件 manifest，并绑定 OpenPI commit、processor/tokenizer revision。
2. 记录 server/client exact argv、UTC timestamp、shared run UUID 与 request IDs。
3. 用相同 action contract 做 matched LIBERO diagnostic probe，再连接 task-native controller/scorer。
4. 本页已作为诊断来源加入 `PROJ-OPENPIVLA-007`；题卡仍保持 `unseen`。

参考文献（References）

1. Local action-eval server log（A01）。
2. Local action-eval summary（A02）。
3. Local evaluator script（A03）与 launch script（A04）。
4. Local deployment record（D01）。
5. OpenPiVLA Runtime Rollout Artifact Gate。
6. OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate。

Part 32 / 49 · `interview_materials/vla/sources/openpivla_formal_json_ce_shape_contract_artifact_gate.html` · SHA-256 `1901574866450FC4345809C5C09D9160E60D3750CE65B5CE37EF3577E512DC62`

返回本卷目录

Canonical Markdown SHA-256：`1901574866450fc4345809c5c09d9160e60d3750ce65b5ce37ef3577e512dc62`；生成自：`interview_materials\vla\sources\openpivla_formal_json_ce_shape_contract_artifact_gate.md`

OpenPiVLA 本地 Artifact Gate：JSON/CE 动作表示与 H/D shape contract

状态：`local_direct_audit / formatter_probe_passed / semantics_open / WARN` 审查日期：2026-07-19 范围：`C:\Users\mousongzhe\Project\openpivla_deployment` 中 Qwen3-VL formal trainer、recipe、public smoke summary 与 action formatter；只关闭一个本地证据切片，不宣称 checkpoint、benchmark 或个人 ownership 已闭合。 结论先行：本地 formal trainer 的可核验实现是 **assistant JSON content + full-chat Qwen causal LM CE**，不是 assistant-token-only loss，也不是 frozen FAST DCT/BPE processor或连续 flow/action-expert loss。recipe/smoke 的模型合同是 `H=15,D=72`，但 JSON formatter 对每个 action/state row 使用 `limit=16`；实际 probe 得到 `15×16`。后续 parser/round-trip gate 进一步确认 840/1080 个 schema positions 未序列化、formal package 中没有 action-output parser，axis/unit/decoder/controller 继续 `semantics-open`。

1. Evidence freeze

ID	本地对象	直接核对	SHA-256 / 状态
Lo1	`...\vla_qwen3_repro\formal_train.py`	`_format_action_sequence`、`DroidActionTextDataset`、Qwen chat template、padding labels、`Qwen3VLForConditionalGeneration`、LoRA、`outputs.loss`	`51B67D3E04496E68076BF6461506141EBCDDF9A82E6C30AF1FDC0F1F2CCC6F94` / `local-direct`
Lo2	`...\vla_qwen3_repro\recipes.py`	`action_dim=72`、`proprio_dim=72`、`action_horizon=15`、`fast_tokenizer` 字段、三种 run recipe	`397D1B384A2BD93D9DC81B7003E6AB7BFAC3FC458EA04008197C3F503F9D780E` / `local-direct`
Lo3	`...\vla_qwen3_repro\data.py`	public smoke dataset/batch adapter 与 action padding helper	`32C6AA76929D941C4832D2EB4C2550527D3E607E31C2A9F5507CC76EEEDBC2EC` / `local-direct`
So1	`artifacts\vla_train_repro\20260305_122516-akpdp2to_smoke_summary.json`	VLA-only public smoke、`[2,15,72]`、tiny loss/grad、private data false	`local-direct`
So2	`artifacts\vla_train_repro\20260310_125200-c6665s5x_smoke_summary.json`	co-train public smoke、robot/VLM batch shape、tiny loss/grad、private data false	`local-direct`
So3	`artifacts\vla_train_repro\20260311_114400-nodm49pk_smoke_summary.json`	VLM-only public smoke、private data false	`local-direct`
Ro1	`C:\Users\mousongzhe\Project\openpivla_deployment\artifacts\vla_formal_reproduction_report_20260617.html`	报告中 100k/97.4 等汇总与历史路线描述	`report-supported`；不作为本页 formatter 事实唯一来源

本页的 `Project` 路径是用户工作区外的本地项目目录；它没有被复制进本仓库，也没有改写其代码。smoke JSON 的 `private_data_deployed=false` 明确限制了“公源接口 smoke”与“私有 formal run”之间的推断。

2. 实际数据流：DROID row → JSON assistant → causal CE

`formal_train.py` 的 DROID path 逐步做：

1. 从 parquet 读取 `action`、`observation.state`、episode 与 language instruction；
2. 连续 action window 按 `action_horizon` 截取；
3. 每一行通过 `_to_flat_float_list(..., limit=16)`，然后 `_format_action_sequence` 在不足 horizon 时重复最后一行；
4. 生成紧凑 `{"actions": ...}` assistant 文本；
5. 通过 Qwen chat template/tokenizer；
6. collator 只把 `attention_mask==0` 的 token label 置为 `-100`；user prompt、chat wrapper 与 assistant JSON 的全部非 padding token 进入 HF causal LM `outputs.loss`。

抽象为：

$$\text{action window} \xrightarrow{\text{flatten/limit=16 + repeat}} Y \in \mathbb{R}^{H\times 16} \xrightarrow{\text{JSON + tokenizer}} y_{1:L} \xrightarrow{\text{causal LM}} \mathcal{L}_{\text{CE}}(y).$$

变量与边界：$H = 15$ 是 recipe 的窗口长度；`16` 是当前 formatter 的每行上限；L 是 JSON/token 文本长度，不等于 $H \times D$。`action_dim=72` 出现在 recipe/smoke batch contract，但当前 `formal_train.py` 的 assistant serializer 没有把 72 维 row 原样写入 JSON。除非存在 decoder/parser/axis mapping 证据，不能把 `(15,72)` 直接当作 controller-ready action。

3. 直接代码证据与 formatter probe

关键片段（来源为 L01）：

```
def _format_action_sequence(values, horizon):
    # Each serialized row is flattened and truncated to at most 16 numbers.
    seq = [_to_flat_float_list(v, limit=16) for v in values]
    if not seq:
        seq = [[0.0]]
    # Tail padding repeats the last available row, not a valid-length mask.
    while len(seq) < horizon:
        seq.append(seq[-1])
    return seq[:horizon]

# The dataset writes the result as assistant-side compact JSON.
answer = json.dumps({"actions": action_seq}, separators=(",", ":"))
```

本轮用 AST 提取上述两个真实函数（没有导入完整训练栈），输入两行 72D synthetic rows、`horizon=15`：

Probe	结果	含义
formatter output length	`15` rows	recipe horizon 被使用
every serialized row width	`16`	`limit=16` 截断真实生效
short window tail	第 3–15 行重复第 2 行	padding 是 repeat，不是 mask
first row `[0..71]`	输出前 16 个值 `[0..15]`	72D row 未原样序列化

probe 输出摘要：`horizon=15`，`row_widths=[16]`，`tail_repeat=True`。这验证的是 formatter 语义，不证明 tokenizer round-trip、robot axis/unit 或 learned checkpoint。

4. H/D/serialized-width 冲突的正确解释

层	当前事实	能说什么	不能说什么
recipe/model	`action_horizon=15`, `action_dim=72`, `proprio_dim=72`	formal batch/schema 目标是 `H=15,D=72`	不能证明 JSON 每行包含 72 个可执行量
smoke batch	robot `actions=[2,15,72]`, state `[2,72]`	public smoke tensor contract 通过	不能证明 private DROID/formal checkpoint 同源
formal serializer	每行 `limit=16`, assistant JSON	当前 local representation 是 15 行 × 最多 16 数值	不能称为 15×72 的完整 action decode
controller boundary	本切片未发现 parser/axis/unit/normalizer round-trip	semantics-open、需补 artifact	不能声称 controller-ready 或 rollout success

因此 `H=15,D=72` 与 `limit=16` 不是简单“超参数不同”：前者是 batch/model contract，后者改变了 serialized action semantics。面试中应主动指出冲突，并把“谁发现、是否修复、如何验证”作为 ownership 追问，而不是用 `shape=(15,72)` 掩盖它。

5. FAST 边界：字段名不是 processor 调用

`ModelRecipe` 保留 `fast_tokenizer="physical-intelligence/fast"`、`slowfast=1` 与 `fast_only=True` 等字段；但本次直接读取的 `formal_train.py` action path 使用 `json.dumps`、Qwen chat template 与 causal LM CE，没有调用 frozen FAST processor 的 DCT/quantization/BPE/inverse pipeline。

安全分类：

- **upstream-verified**：FAST 的 DCT/quantization/BPE 方法与 OpenPI/FAST frozen source；
- **local-direct**：本地 formal JSON/CE formatter；
- **representation-conflict**：仅凭 recipe 字段把 local JSON 宣称为 FAST；
- 待补：formatter→tokenizer→decode→72D/action-controller round-trip。

6. Smoke evidence 与 结果边界

三个 smoke summary 都记录 Qwen3-VL recipe、public roots、batch shape 与 tiny train step；其中 VLA-only 和 co-train 的 robot batch 是 `[2,15,72]`，co-train 另有 VLM batch，且三份 summary 都写 `private_data_deployed=false`。这些文件支持：

1. 代码路径能够在 public smoke 设置下构造 batch；
2. tiny train step 能返回 loss/grad norm；
3. 三种 recipe 的 mode/config 具有可读的分层合同。

它们不支持：

- 远端 formal checkpoint 已在本地存在或可加载；
- archived `97.4%` 与这些 smoke/formal run 同一 lineage；
- private DROID/MindCube 数据已进入 smoke；
- JSON representation 已通过 robot controller 或 closed-loop success。

7. 对项目 claim 的回写

Claim	当前 verdict	面试安全口径	升级所需 artifact
`OPV-CL-002` assistant JSON + causal CE	`local-direct / PASS WITH WARNING`	“formal trainer 把 action window 序列化为 assistant JSON content，但当前 objective 覆盖 full chat 非 padding token；不是 assistant-only 或连续 flow loss”	parser、token/JSON round-trip、assistant-span mask test
`OPV-CL-003` 复用 FAST DCT/BPE	`REPRESENTATION-CONFLICT`	“recipe 有 FAST 字段，但当前直接审计到的是 JSON/CE；不能把两者混写”	frozen processor call、revision、round-trip
`OPV-CL-004` H=15/D=72 action contract	`local-direct / SEMANTICS-OPEN`	“smoke tensor 是 `[B,15,72]`，但 formatter 每行 limit=16；语义闭环仍待修”	decoder、axis/unit、normalizer、controller trace
`OPV-CL-005` 三种 recipe	`local-direct`	“三类 recipe 与 public smoke 可读；不能据此证明三条 private formal completion”	immutable logs/checkpoints/load smoke
`OPV-CL-007` archived 97.4	`archive-summary-only`	“报告有归档汇总；不是本页 local formatter/smoke 的结果证明”	checkpoint/protocol/raw episodes

30 秒项目回答

“我会把这条 local route 讲成 Qwen3-VL 的 JSON action training path：代码把 action window 截成 $H = 15$，序列化为 assistant JSON content，但 collator 只屏蔽 padding，所以实际是 full-chat causal CE。它与 FAST 的 DCT/BPE 不是同一个实现。recipe/smoke 写 $D = 72$，formatter 每行只取 16 个数；后续 probe 又确认没有 formal Qwen parser/controller round-trip，所以不能说已经形成 72 维 controller contract。”

8. 风险与下一步

1. 已关闭：local formal source 的 JSON/CE 表示、`H=15,D=72` recipe/smoke、`limit=16` serializer 与 tail-repeat probe。
2. 仍开放：72D action 的 decode、axis order、unit/frame、normalizer、controller 与 safety contract。
3. 仍开放：FAST 字段是否只是 recipe metadata，还是另有未读取的 processor path；需要 call-site/revision/round-trip 证据。
4. 仍开放：checkpoint/load/eval lineage 与个人 ownership；smoke summary 不升级 C3。
5. 下一 **bounded slice**：parser/round-trip negative-evidence gate 已完成；后续优先做 ownership confirmation，或补 immutable formal checkpoint/generation bundle；不新增题卡。

参考文献（References）

1. Local `formal_train.py`、`data.py`、`recipes.py`，路径与 SHA 见本页 Evidence freeze。
2. Local smoke summaries `vla_train_repro/*_smoke_summary.json`，本页 S01–S03。
3. OpenPiVLA Claim–Evidence Inventory，用于 claim tier 与真实性边界。
4. OpenPiVLA Ownership Ledger，用于个人贡献边界。
5. FAST upstream audit，用于 DCT/BPE 与 local JSON/CE 的对照边界。
6. OpenPiVLA Action JSON Parser / Round-trip Artifact Gate，用于 72→16 loss、full-sequence CE、parser absence 与 official-eval lineage 边界。

Part 33 / 49 · `interview_materials/vla/sources/openpivla_libero_checkpoint_load_lineage_gate.html` · SHA-256 `02B6D0A5ECC380C992B92EBC03E7BBE6EEAF47E30B9A22020B3F7A43F8C4648A`

返回本卷目录

Canonical Markdown SHA-256: `02b6d0a5ecc380c992b92ebc03e7bbe6eeaf47e30b9a22020b3f7a43f8c4648a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_libero_checkpoint_load_lineage_gate.md`

OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate

状态：`local-direct / log-hash-frozen / checkpoint-load-confirmed / binary-unverified / WARN` 审查日期：2026-07-21 范围：`C:\Users\mousongzhe\project\openpivla_deployment` 中 `libero_franka_true_rollout_v1` 的 server/client log、rollout script 与部署记录。结论先行：本地 `serve.log` 直接证明 **Orbax 从官方 `pi05_libero/params` 路径恢复了约 6.2 GiB 参数、从 checkpoint assets 加载 normalization stats、启动 WebSocket server 并接受 localhost client 连接**；同目录 `client.log` 与 `summary.json` 记录了随后完成的 LIBERO task-0 70-step model/no-op rollout。该证据关闭了“是否真的 load 并 serve 官方 checkpoint”的主要疑问，但仍没有 checkpoint 文件级 manifest/SHA、精确启动命令、processor/tokenizer revision、run UUID 或私有 LoRA checkpoint，因此不能升级为 fine-tune lineage、完整可重复 benchmark 或个人 ownership。

1. Question 与证据目标

上一份 Runtime Rollout Artifact Gate 已冻结 ALOHA `2/3`、多视角 `1/2` 与 LIBERO task-0 `0/1`，但当时只有 summary 中的 checkpoint 字段，尚未直接读取 checkpoint restore 和 server/client 连接日志。本页只回答四个问题：

1. server 是否真正从 `pi05_libero` checkpoint 恢复参数；
2. normalization stats 是否随相同 checkpoint lineage 被加载；
3. server 是否监听、client 是否连接、rollout 是否产生结果；
4. 这组日志仍缺哪些内容，因而不能支持哪些强 claim。

本页不重新运行远端训练或评估，不复制权重，也不把部署日志改写成候选人的算法贡献。

2. Immutable evidence freeze

ID	本地对象	SHA-256	直接支持
Lo 1	`artifacts\libero_franka_true_rollout_v1\serve.log`	`FF0B39D7B87D604D86486A0088E7CF7FD2D3EA99CB151B99FD5D9B9E60AACD2A`	Orbax version、checkpoint restore path/bytes/time、norm stats path、listen port、client connection
Lo 2	`artifacts\libero_franka_true_rollout_v1\client.log`	`BC1B7FE6E54735B4623062D151D47667BD418F4F7A46B6CC471EC1F961869572`	LIBERO task/config/checkpoint summary、70-step model/no-op run、reward/success/result filenames
Co 1	`scripts\run_libero_true_rollout.py`	`1C7F74DE83FD373A27FEDC70D863D5F0BF164B4512ED63E1D76552A0F22C3016`	observation schema、state transform、image preprocessing、action slice/replan、environment step 与 summary writer
Do 1	`DEPLOYMENT.md`	`C0A3DD0C0DA8C1C0D36023046D5337900A71680B413657BC9B1062EEF2EEA14E`	remote directory、OpenPI recorded commit `c23745b`、official checkpoint inventory、环境与结果边界

`summary.json` 已在 Runtime Rollout Artifact Gate 中冻结为 `341E77110D75E50B601A078923F51EF8FA39AAA045FA491843A1003B0320E1DB`。本页新增冻结的关键对象是 `serve.log`、`client.log` 与 rollout script。所有路径均指向用户本地项目目录，未把权重或运行产物复制进面试仓库。

3. Server-side checkpoint load chain

`L01` 的顺序证据为：

```
Loading model...
orbax-checkpoint version: 0.11.13
Restoring checkpoint from .../checkpoints/pi05_libero/params
read total bytes: 6.2 GiB
Finished restoring checkpoint in 17.00 seconds
Norm stats not found in official source assets path, skipping
Loaded norm stats from .../checkpoints/pi05_libero/assets/physical-intelligence/libero
server listening on 0.0.0.0:8399
Connection from ('127.0.0.1', 46490) opened
```

这比 summary 的字符串字段更强：它记录了 Orbax restore 的执行、读取体量和完成事件，也把 normalization stats 定位到同一 checkpoint 根目录的 assets。可审计的运行身份可写成：

$$I_{\text{serve}} = (P_{\text{params}}, P_{\text{norm}}, V_{\text{orbax}}, B_{\text{read}}, T_{\text{restore}}, H, p)$$

变量解释：`P_params` 是 `/pi05_libero/params`，`P_norm` 是 checkpoint 内的 `assets/physical-intelligence/libero`，`V_orbax=0.11.13`，`B_read≈6.2 GiB`，`T_restore=17.00 s`，`H` 是日志中的容器 host，`p=8399`。直觉上，这个 tuple 绑定的是“这次 server 进程加载了什么并监听在哪里”；它来自本地 log 的直接证据，而不是从 summary 反推。

仍不能把它称为 immutable checkpoint manifest：本地没有 `params` 目录的逐文件列表、总 SHA/Merkle root、对象存储 revision 或下载 manifest。路径一致和 restore success 能证明可加载性，不能证明远端目录之后绝不会被替换。

4. Client 与 rollout contract

`C01` 说明 client 不是只发一个空请求，而是按以下 contract 构造 policy observation：

字段	来源与变换	当前证据边界
`observation/image`	`agentview_image` 双轴翻转，再 resize-with-pad 和 uint8 转换	script direct；本次 invocation 的原始图像 hash 未保存
`observation/wrist_image`	`robot0_eye_in_hand_image`，同样翻转、resize、uint8	script direct
`observation/state`	eef position + quaternion-to-axis-angle + gripper qpos	script direct；axis/unit 的独立 round-trip 未做
`prompt`	LIBERO task language	script direct；summary 固定了 task-0 文本
`actions`	`client.infer(...)["actions"]`，取前 `replan_steps` 行、前 7 维入队	script direct；server-side action tensor trace 未落档

脚本默认 `resize_size=224`、`replan_steps=5`、`seed=7`，但 `client.log` 没有打印完整 `argv`，所以这些只能说明 **代码默认值**，不能冒充本次 run 的精确 effective config。summary 直接支持本次执行为 task 0、70 steps、model/no-op 都是 `0.0 reward / false success`。

```
def observed_rollout_contract(obs, task_text, client, replan_steps):
    # 1. 从 LIBERO 观测构造 agent/wrist image、8D state 与语言提示。
    request = build_libero_request(obs, task_text)

    # 2. WebSocket client 请求 action chunk；本地日志证明 server 已接受连接。
    chunk = client.infer(request)["actions"]

    # 3. 只执行 chunk 的前 replan_steps × 7D，再重新请求。
    for action in chunk[:replan_steps, :7]:
        obs, reward, done, info = env.step(action.tolist())
        if done:
            break
```

伪代码是 `C01` 的紧缩映射。它不补造本次 invocation 未打印的 `replan_steps`，也不声称 client 侧的 `--checkpoint` 字段控制了 server；真正的 checkpoint 绑定来自 `L01` 的 server restore path。

5. Same-run binding 强度

完整的可重复 run identity 至少需要：

$$I_{\text{run}} = (R_{\text{code}}, M_{\text{checkpoint}}, N_{\text{stats}}, A_{\text{argv}}, O_{\text{request}}, E_{\text{env}}, S_{\text{scorer}}, Y_{\text{raw}})$$

变量解释：`R_code` 是代码 revision，`M_checkpoint` 是不可变 checkpoint manifest，`N_stats` 是 normalization asset，`A_argv` 是精确启动参数，`O_request` 是 observation/action contract，`E_env` 是环境和 init state，`S_scorer` 是成功判定，`Y_raw` 是逐步输出。当前 bundle 对 `N_stats`、部分 `O_request`、任务级 `E_env` 和 summary `Y_raw` 有直接证据；对 `M_checkpoint` 只有 path+restore、对 `R_code` 只有部署记录中的 commit、对 `A_argv` 和 server-side raw request/response 不完整。

Gate	当前状态	证据	限制
checkpoint load	`PASS WITH WARNING`	Orbax restore 6.2 GiB、17 s、明确 params path	无逐文件 SHA / immutable manifest
normalizer load	`PASS WITH WARNING`	从 checkpoint assets 明确加载	stats 内容与 SHA 未冻结
server ready	`PASS`	监听 `0.0.0.0:8399`	无 readiness probe latency/SLO
client connection	`PASS WITH WARNING`	localhost connection opened；同目录 client log 有结果	无 shared run UUID、无带时间戳 request ID

Gate	当前状态	证据	限制
observation/action contract	`PASS WITH WARNING`	本地 script hash 与字段/切片逻辑	invocation argv 与 server response trace 缺失
success scorer	`WARN`	script 以 `done` 聚合 success	无独立 scorer/version；`last_info={}`
private fine-tune lineage	`UNVERIFIED`	无	official `pi05_libero` 不能替代 LoRA/Qwen checkpoint
ownership	`PENDING`	无候选人口述和 pre-snapshot diff	不能使用个人端到端 ownership 口径

6. 面试安全口径

30 秒回答

“我没有把 summary 里的 checkpoint 字符串当作已经加载的证明，而是继续核对了 server/client log。server 侧 Orbax 0.11.13 确实从官方 `pi05_libero/params` 恢复了约 6.2 GiB 参数，17 秒完成，又从 checkpoint assets 加载了 LIBERO normalization stats，随后在 8399 端口接受了 localhost client。client 脚本把 agent view、wrist view、eef/gripper state 和 task prompt 发给 policy server，并执行返回 action chunk 的 7D 前缀。这个闭环能证明官方 checkpoint 的 load、serve 和 rollout 路径，但没有 checkpoint 文件级 SHA、精确 argv 和私有 LoRA lineage，所以我不会把它说成自己的 fine-tune 成绩；本次 task-0 结果也仍是 0 reward、false success。”

高频追问

1. **如何证明 client 连接的是这次 server？** 同目录 server log 记录 localhost connection，client log 紧接着保存任务 summary；这是强运行证据，但没有 shared run UUID/request ID，所以仍标 `PASS WITH WARNING`。
2. **为什么 summary 的 checkpoint 字段不够？** client script 只是把 `--checkpoint` 文本写入 summary；真正控制模型的是 server 进程，因此必须看 server restore log。
3. **normalizer 是否确认？** 确认了加载路径和成功事件；未冻结 stats 文件 hash/内容，故不能主张完整 immutable bundle。
4. **这能否证明私有 fine-tune？** 不能。日志加载的是官方 `pi05_libero`，与 Qwen3-VL LoRA formal trainer 是不同 lineage。
5. **0 success 的根因是什么？** 仍未知。action axis/unit、controller、camera preprocess、replan 参数和 scorer 需要 matched probe；本页只关闭 load/serve 疑问。

7. 下一步

1. 在远端只读生成 checkpoint manifest：逐文件 path/size/SHA、norm stats SHA、OpenPI commit、processor/tokenizer revision。
2. 启动时保存 server/client 完整 argv、effective config、UTC timestamp、run UUID，并让每个 request/summary 携带同一 ID。
3. 做一组 matched LIBERO diagnostic probe，固定 observation keys、resize/flip、state axis/unit、7D action order、replan interval、controller 与 success scorer。
4. 本页证据已映射到 `PROJ-OPENPIVLA-007`；该卡仍保持 `unseen`。下一切片优先补 immutable manifest，或执行 matched LIBERO failure probe。

参考文献（References）

1. Local server log：`C:\Users\mousongzhe\project\openpivla_deployment\artifacts\libero_franka_true_rollout_v1\serve.log`（L01）。
2. Local client log：同目录 `client.log`（L02）。
3. Local rollout implementation：`C:\Users\mousongzhe\project\openpivla_deployment\scripts\run_libero_true_rollout.py`（C01）。
4. Local deployment record：`C:\Users\mousongzhe\project\openpivla_deployment\DEPLOYMENT.md`（D01）。
5. OpenPiVLA Runtime Rollout Artifact Gate。
6. OpenPiVLA formal JSON/CE artifact gate，用于区分 private formal trainer 与 official deployment lineage。

Part 34 / 49 · `interview_materials/vla/sources/openpivla_libero_matched_failure_probe_readiness_gate.html` · SHA-256 `8DCBEF96245990714683859C4ADA6C3DCA2F3140CAABFB8117C94E32C6EA09A5`

返回本卷目录

Canonical Markdown SHA-256: `8dcbef96245990714683859c4ada6c3dca2f3140caabfb8117c94e32c6ea09a5`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_libero_matched_failure_probe_readiness_gate.md`

OpenPiVLA / LIBERO Matched Failure Probe Readiness Gate

状态：`local-summary-video-timeline-consistent / probe-not-rerun / raw-trace-missing / WARN` 审查日期：2026-07-22 范围：`C:\Users\mousongzhe\Project\openpivla_deployment\artifacts\libero_franka_true_rollout_v1` 的 summary、server/client log、rollout script 与已生成视频/指标图。结论先行：本轮只使用本地文件，对现有 LIBERO model/no-op bundle 做了可重算的 preflight 与视频时间线审计：9/9 summary/file 检查通过，两条视频均为 71 帧、10 FPS、7.1 秒；前 10 帧共享 dummy warm-up，之后 model branch 出现非零变化 action、机械臂向碗区运动，画面与 state/action norm 明显偏离 no-op，但 70 步内 reward/success 始终为 0，未完成抓取—放置。这足以排除“全程只执行 dummy / 仿真完全没动”，仍不足以把失败单因归到 preprocess、axis/unit、normalizer、controller、scorer 或 policy。

1. Question 与边界

前两页已经冻结了官方 `pi05_libero` 的 load/serve/client lineage 与本地 checkpoint 文件缺口。本页只回答：

1. 已归档的 model/no-op rollout 是否在同一个 task/episode/horizon 上可重算；
2. 哪些 observation/action/termination contract 可以从本地 script 直接恢复；
3. 还缺哪些原始 trace，才能执行一个真正的 matched failure probe。

“matched”在本页中不是“两个 summary 看起来相似”，而是至少固定以下 tuple：

$$P_{mathrmmatch} = (T, I, S, O, A, N, R, C, G)$$

其中 `T` 是 task suite/id，`I` 是 init-state/episode，`S` 是 seed，`O` 是 observation keys 与图像变换，`A` 是 action order/dimension/unit，`N` 是 normalizer/checkpoint assets，`R` 是 replan/warm-up，`C` 是 controller/environment step，`G` 是 success scorer。变量与代码映射：`T/I/S` 来自 `run_libero_true_rollout.py` 的 benchmark/task/init-state/seed 入口，`O/A/R` 来自 `_policy_action` 与 `deque` 逻辑，`C/G` 来自 `_run_episode` 的 `env.step` 与 `done` 聚合；当前 bundle 只完整落档了部分 tuple，未满足全部 matched 条件。

本页不连接或改动远端，不复制 checkpoint，不把本地视频后验观察当作失败根因，也不更新题卡 mastery。

2. Immutable evidence freeze

ID	本地对象	SHA-256	直接支持
So1	`artifacts/libero_franka_true_rollout_v1/summary.json`	`341E77110D75E50B601A078923F51EF8FA39AAA045FA491843A1003B0320E1DB`	task tuple、70-step runs、reward/success、norm summary、artifact names
So2	`artifacts/libero_franka_true_rollout_v1/serve.log`	`FF0B39D7B87D604D86486A0088E7CF7FD2D3EA99CB151B99FD5D9B9E60AACD2A`	Orbax restore、checkpoint path、norm-stats load、8399 listen/connection
So3	`artifacts/libero_franka_true_rollout_v1/client.log`	`BC1B7FE6E54735B4623062D151D47667BD418F4F7A46B6CC471EC1F961869572`	client-side summary/log warnings；没有完整 argv
Co1	`scripts/run_libero_true_rollout.py`	`1C7F74DE83FD373A27FEDC70D863D5F0BF164B4512ED63E1D76552A0F22C3016`	observation transform、state/action contract、episode loop、scorer wiring
Vo1	`artifacts/libero_franka_true_rollout_v1/zero_reference/zero_reference_rollout.mp4`	`27BFAB393ECF92E3092D768A6294759287D6F779527EEEB2CE6160F26E7063FA`	no-op visual rollout artifact
Vo2	`artifacts/libero_franka_true_rollout_v1/model_policy/model_policy_rollout.mp4`	`B8C14912B1202085581286B05949D8957A5AA77E040AE52AB91FFC0F67A60ED6`	model visual rollout artifact
Vo3	`artifacts/libero_franka_true_rollout_v1/zero_reference_vs_model.mp4`	`226953734221483E03B509C03AE997B98C256722B9281A78A39C86D226D2ECE1`	paired visualization，不等于 paired causal experiment

ID	本地对象	SHA-256	直接支持
P01	`artifacts/libero_franka_true_rollout_v1/model_policy/model_policy_metrics.png`	`D5EDB3193BCD2257771304CA97CB226A55CA0567267A9CC209DAC810D7C7CC28`	model reward/action-norm/state-norm/success 曲线
P02	`artifacts/libero_franka_true_rollout_v1/zero_reference/zero_reference_metrics.png`	`0214FC58F66A3CEFFB9A4B2F652369F76F9C3620E96ED4EF8D06FE5443ED4020`	dummy-reference 同口径曲线

3. Local preflight audit

使用 Python 标准库读取 `S01`，按脚本相对目录规则检查结果引用；没有修改项目文件。审计结果如下：

Check	Result	Evidence
`status == ok`	PASS	`summary.json`
task tuple	PASS	`libero_spatial / task_id=0 / episode_idx=0`
run labels	PASS	exactly `zero_reference` and `model_policy`
matched horizon	PASS	both `steps_executed=70`
reward consistency	PASS	both `total_reward=0.0` and `mean_reward=0.0`
termination boundary	PASS	both `success=false`, `done=false`
finite scalar metrics	PASS	action norm/state norm fields finite
success-rate recomputation	PASS	`success_rate=0.0` equals mean of run success flags
referenced artifacts	PASS	per-run video/plot and comparison video paths exist

这 9/9 只证明归档 summary 与本地文件引用自洽。它没有检查视频帧内的真实动作数组，也没有重新调用 policy server。

3.1 Local video timeline audit

本轮在临时目录中只读解码 V01/V02，没有修改 `C:\Users\mousongzhe\Project`。两条视频都得到 71 帧、10 FPS、7.1 秒。帧画面由 72px banner、256×256 agent view 和 256×256 wrist view 构成；对每个 camera c，以 frame 0 为基准计算 8-bit RGB mean absolute difference：

$$D_c(t) = \frac{1}{3HW} \sum_{u=1}^{H} \sum_{v=1}^{W} \sum_{k=1}^{3} |I_{t,c}(u,v,k) - I_{0,c}(u,v,k)|.$$

$D_c(t)$ 只量化像素变化，不是状态距离、物体位姿或任务进度。它的价值是把“视频看起来动了”变成可复算的 local visual proxy。

Frame / time	model agent MAD	model wrist MAD	dummy agent MAD	dummy wrist MAD
10 / 1.0s	5.403	17.071	5.369	17.099
20 / 2.0s	6.976	36.943	5.377	17.228
30 / 3.0s	10.454	45.608	5.385	17.268
40 / 4.0s	10.028	42.909	5.394	17.260
50 / 5.0s	9.932	51.265	5.393	17.274
60 / 6.0s	9.572	48.840	5.397	17.280
70 / 7.0s	9.685	45.524	5.392	17.282

时间线与脚本互相印证：

1. frame 0–10 两侧 overlay 都是 `[0,0,0,0,0,0,-1]`；C01 也定义 `LIBERO_DUMMY_ACTION` 为该向量。它不是七维全零，最后一维固定为 −1，所以“zero reference”应称 dummy/no-motion arm baseline，而不是 GT 或数学零向量。
2. frame 20 以后，model overlay 出现连续变化的 7D action；agent/wrist MAD 明显高于 dummy，P01 的 action norm 约在 1.0–1.53 间变化，state norm 从约 3.37 降到 3.27 后小幅回升。由此可说 policy action 已进入 env 并诱导出不同 trajectory。
3. 抽样帧显示机械臂从初始位姿向左侧碗区移动，并在后半程停留于碗附近；画面没有显示目标碗被稳定抓起并放到盘子上。P01/S01 同时保持 70 步 reward=0、success=false、done=false。
4. 这些观察排除“model 分支全程仍在执行 dummy”与“仿真完全静止”，但不能证明 action order/unit 正确、接触发生、抓取力有效、task predicate 正确，或某个单一模块导致失败。视频 overlay 只有两位小数，不能替代 raw response/env-action trace。

4. What the local script fixes

4.1 Observation contract

`C01` 的 `_policy_action` 直接固定：

Field	Local implementation	Remaining uncertainty
`observation/image`	`agentview_image[::-1, ::-1]`，`resize_with_pad`，`convert_to_uint8`	本次 invocation 的原始图像 hash 与实际 resize 参数未写入 summary
`observation/wrist_image`	`robot0_eye_in_hand_image[::-1, ::-1]`，同样的 resize/uint8	同上
`observation/state`	eef position + quaternion-to-axis-angle + flattened gripper qpos	axis convention/unit 的 round-trip 未做；逐步 state 未落档
`prompt`	`str(task_description)`	server 收到的最终文本未落档

4.2 Action and execution contract

脚本将 `client.infer(element)["actions"]` 转成 `float32`，在空 `action_plan` 时取 `action_chunk[:replan_steps, :7]`，再通过 `env.step(action.tolist())` 执行。默认 `replan_steps=5`、`resize_size=224`、`seed=7`、`max_steps=80`、`wait_steps=10` 写在 argparse 中，但本次 `client.log` 没有完整 argv；summary 的 70 steps 说明实际 effective config 与默认值不能直接等同。

```
def local_contract(obs, client, task_description, action_plan, replan_steps):
    # 1. 只在 action queue 为空时构造双视角图像、state 和 prompt。
    request = {
        "observation/image": resize_uint8(flip_both_axes(obs["agentview_image"])),
        "observation/wrist_image": resize_uint8(flip_both_axes(obs["robot0_eye_in_hand_image"])),
        "observation/state": state_from_obs(obs),
        "prompt": str(task_description),
    }
    # 2. 请求 action chunk；本地 bundle 没有保存原始 response tensor。
    chunk = np.asarray(client.infer(request)["actions"], dtype=np.float32)
    # 3. 执行前 replan_steps 行、前 7 维，再重新请求。
    action_plan.extend(chunk[:replan_steps, :7])
    return action_plan.popleft()
```

`_run_episode` 将 `success = success or bool(done)`，把 `done` 同时作为 success flag 与 break 条件；`last_info` 仅以字符串形式写入 summary，本次为 `{}`。因此目前的 scorer 不是独立 task-success evaluator，而是脚本层面的 `done` 聚合。

5. Matched failure probe matrix

Hypothesis	Required matched intervention	Current local evidence	Status
checkpoint/normalizer mismatch	同一个 `params/assets` manifest，fresh server load，两次 run 共用 manifest ID	server log 只有 path+restore；binary manifest 不在本地	`OPEN`
image flip/resize mismatch	固定同一 init state，保存 raw image hash，A/B 只切换 flip/resize	script 固定代码路径，但 raw image 未保存	`OPEN`
state axis/unit mismatch	保存 quaternion、axis-angle、eef/gripper 原始数组与 round-trip probe	只有最终 state norm，无 raw state	`OPEN`
action order/unit/dimension	保存 server response shape、前 7D slice、denormalized action 与 env action	`:7` 代码和 video rounded 7D overlay 证明非 dummy action 被执行；raw response/denorm/env trace仍缺	`OPEN`
replan/warm-up	把 effective `wait_steps/replan_steps/max_steps` 写入 run manifest，做 paired sweep	summary 只显示最终 70 steps	`OPEN`
controller/termination scorer	固定 controller、独立 success predicate、逐 step `info` /termination reason	`last_info={}`，脚本用 `done`	`OPEN`
task/init-state confound	同一个 task/init-state/seed，独立 server restart 后重复	task/episode 已在 summary；seed/argv 未绑定	`PARTIAL`

Probe acceptance criteria

只有同时满足以下条件，才允许把下一次结果标成 matched probe：

1. `run_id`、checkpoint manifest ID、OpenPI commit、processor/tokenizer revision、effective argv 全部落档；
2. 两个条件只改变一个 intervention，task/init-state/seed、camera bytes、normalizer、controller、scorer 与 horizon 完全一致；
3. 每步 raw observation、raw response action、post-transform env action、reward、done、info 和 timestamp 可重放；
4. model/no-op 或 A/B 两侧各有独立 summary，且 success predicate 不由视频后验推断；
5. 结果先经过 shape/finite/unit/axis audit，再讨论根因。

6. Interview-safe boundary

30 秒回答

“我没有把 LIBERO 的 `0 reward / false success` 直接归因。只看本地 bundle，同一 task-0/episode-0 的 model 与 dummy 都跑了 70 步；视频前 10 帧共享 dummy warm-up，之后 model action、双相机画面和 state/action norm 明显偏离 dummy，机械臂确实向碗区运动，但没有完成抓取—放置。因此能排除‘policy 全程没执行’和‘仿真没动’，不能说一定是 normalizer、轴序或模型能力问题，因为本地仍缺 raw response/env action、effective argv 和独立 success scorer。”

禁止的强结论

- 不能说“模型一定输出了错误的 action order/unit”，因为原始 response 和 env action 没保存。
- 不能说“必然是 checkpoint 或 normalizer 错误”，因为官方 checkpoint restore/load 只证明可加载，不证明其他协议层正确。
- 不能把 dummy-reference 当作 GT demonstration 或七维全零，也不能把 paired video/pixel MAD 当作 causal ablation。
- 不能把脚本作者、远端 deployment 或 official `pi05_libero` 归为个人端到端 ownership。

7. 下一步

1. 后续仍只在本地操作：先把现有 video timeline、P01/P02 与 script contract 用于失败口述，不把它们升级成 matched rerun。
2. 如果未来有新的本地 rollout bundle，必须保存 effective argv、raw request/response、post-transform action、reward/done/info 和独立 scorer；没有这些字段时不升级 `0/1` 的根因结论。
3. 保持 `PROJ-OPENPIVLA-007/008` 与全局 280 张卡 `unseen`，不创建 `coaching_state.md`。

参考文献（References）

1. Local summary：`artifacts/libero_franka_true_rollout_v1/summary.json`（S01）。
2. Local server/client logs：同目录 `serve.log`、`client.log`（S02–S03）。
3. Local rollout implementation：`scripts/run_libero_true_rollout.py`（C01）。
4. Local rollout videos：`zero_reference_rollout.mp4`、`model_policy_rollout.mp4`、`zero_reference_vs_model.mp4`（V01–V03）。
5. OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate。
6. OpenPiVLA Official Checkpoint Immutable Manifest Gate。

Part 35 / 49 · `interview_materials/vla/sources/openpivla_official_checkpoint_immutable_manifest_gate.html` · SHA-256 `CBDF6AF4B9AA288C596797F293E7C6275C1E4BFE952714D5A17C9B35BBE23B2E`

返回本卷目录

Canonical Markdown SHA-256：`cbdf6af4b9aa288c596797f293e7c6275c1e4bfe952714d5a17c9b35bbe23b2e`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_official_checkpoint_immutable_manifest_gate.md`

OpenPiVLA Official Checkpoint Immutable Manifest Gate

状态：`local-file-manifest-complete / remote-checkpoint-unavailable / norm-stats-unhashed / WARN` 审查日期：2026-07-21 范围：`C:\Users\mousongzhe\Project\openpivla_deployment` 中 ALOHA action-level eval 与 LIBERO/Franka rollout 的本地文件，以及部署记录中声明的 `pi0_aloha_sim` / `pi05_libero` checkpoint 目录。结论先行：本页已对本地 rollout、summary、脚本、图片和视频逐文件计算 SHA-256；但两个 checkpoint 的 `params` 与 checkpoint 内 norm-stats 目录只存在于部署记录的远端 `/data/msz/...` 路径，当前 Windows 工作区不可访问。因此可以冻结“本地证据 bundle”，不能声称已经完成 checkpoint binary 或 norm-stats 的 immutable manifest。

1. Question 与证据边界

上一组 LIBERO Load–Serve–Client Lineage Gate 和 ALOHA Action Eval Load Gate 已证明 server 进程实际 restore 了官方 checkpoint、加载了 checkpoint assets 中的 normalization stats，并产生了 rollout/action-level eval 结果。本页只进一步回答：

1. 本地归档中哪些文件可以逐文件重算 SHA-256；
2. `pi0_aloha_sim` 与 `pi05_libero` 的 checkpoint 目录是否在当前工作区可读；
3. 如果 checkpoint 仍不可读，面试回答中哪些 lineage 结论必须保留为 warning。

本页不复制远端权重、不重新下载 checkpoint、不把官方 checkpoint 改写为候选人的 fine-tune 产物，也不把文件级 manifest 缺口用目录大小或 restore 成功事件替代。

2. Manifest contract

对一个真正可复用的 immutable artifact，最小记录单元定义为：

$$M_i = (r_i, s_i, b_i, h_i, t_i, q_i)$$

其中 `r_i` 是相对路径，`s_i` 是文件角色，`b_i` 是字节数，`h_i` 是 SHA-256，`t_i` 是本地最后修改时间，`q_i` 是证据等级。直觉上，只有路径、大小、hash 与角色同时冻结，后续面试追问才可以从“我看过一个目录”缩小到“这一次 run 使用的具体文件集合”。本页对本地文件直接记录这些字段；对远端 checkpoint 只能记录部署文档声称的路径和历史体量，`h_i` 与完整 `r_i` 列表均缺失。

```
def freeze_local_manifest(root, relative_paths):
    # 1. 只读取用户已归档的本地文件，不复制权重或改变项目目录。
    records = []
    for rel in sorted(relative_paths):
        path = root / rel
        # 2. 对每个文件记录字节数、修改时间和 SHA-256。
        records.append({
            "path": rel,
            "bytes": path.stat().st_size,
            "sha256": sha256(path),
            "role": classify(rel),
        })
    # 3. 缺失的远端 checkpoint 不填假 hash，而是单独进入 open gate。
    return records
```

3. Checkpoint target audit

Config	部署记录中的 checkpoint 根目录	部署记录的历史描述	当前 Windows 工作区	可支持的结论
`pi0_aloha_sim`	`/data/msz/openpivla/models/openpi-assets/checkpoints/pi0_aloha_sim`	Present；约 11.189 GiB、21 files	`Test-Path` 不存在；没有 `params` 或 `assets` 文件可读取	`serve.log` 证明当时约 6.0 GiB Orbax restore 与 checkpoint norm-stats load；不能提供本地 binary SHA

Config	部署记录中的 checkpoint 根目录	部署记录的历史描述	当前 Windows 工作区	可支持的结论
`pi05_libero`	`/data/msz/openpivla/models/openpi-assets/checkpoints/pi05_libero`	Present；下载完成约 11.6 GiB	`Test-Path` 不存在；没有 `params` 或 `assets` 文件可读取	`serve.log` 证明当时约 6.2 GiB Orbax restore 与 LIBERO norm-stats load；不能提供本地 binary SHA

`DEPLOYMENT.md` 的“Present”是部署记录中的远端状态，不是当前工作区的文件存在证明。当前不能从 Windows 目录读取 `/data/msz/...`，也没有远端只读 manifest、对象存储 revision、Merkle root 或下载清单。因此：

- `checkpoint path + restore success`：`PASS WITH WARNING`；
- `checkpoint per-file SHA / total manifest`：`UNVERIFIED`；
- `norm-stats content + SHA`：`UNVERIFIED`；
- `private Qwen3-VL/LoRA fine-tune lineage`：`UNVERIFIED`，且不应由 official checkpoint 推断。

4. Local immutable manifest：deployment and code

以下对象均在用户本地项目中，SHA-256 于 2026-07-21 重新计算。`local-direct` 只表示文件确实可读并被哈希，不表示文件内部的所有运行依赖都已闭合。

ID	相对路径	字节数	SHA-256	角色
D01	`DEPLOYMENT.md`	28695	`C0A3DD0C0DA8C1C0D36023046D5337900A71680B413657BC9B1062EEF2EEA14E`	部署路径、官方 checkpoint inventory、环境与结果边界
C01	`scripts/eval_lerobot_action_server.py`	19702	`B1B7FB9B7A86355B9C1843CDE971B38FD26A2DAB1F18550128671F79F1AB4F39`	ALOHA action-level eval、shape/L1 汇总与 dashboard writer
C02	`scripts/run_lerobot_action_eval.sh`	3053	`61DFE972B20722CF08B3FA139D68B29B6C1B52286671A7705000AE3738A8BF60`	ALOHA server/client launcher 与 checkpoint/config 参数
C03	`scripts/run_libero_true_rollout.py`	12841	`1C7F74DE83FD373A27FEDC70D863D5F0BF164B4512ED63E1D76552A0F22C3016`	LIBERO observation、action slice、replan 与 summary writer

5. Local immutable manifest：ALOHA action-level bundle

目录：`artifacts/lerobot_action_eval_aloha_sample_real_v2/`。该 bundle 的 `summary.json` 记录 4 个 sample、horizon 50、action dimension 14、mean L1 `0.0330713950`、std `0.0205169544`；这些文件支持 action-vector/harness diagnostic，不支持 task success 或 fine-tune benchmark。

ID	相对路径	字节数	SHA-256	角色
A01	`index.html`	3519	`AE7C0B0D7D1523E3896B11A584BD8B0B335705538A78B804376701E844A6CA9F`	本地 action-eval dashboard
A02	`sample_000_action_compare.png`	90142	`CE2034EEB3EEE3FDD55E0C5F18D1DD37859331CE7F1940492312DFE5551BFF14`	sample 0 action comparison plot
A03	`sample_000_frame.png`	30109	`F1D624D774529E9BFF0AA06C77DAC1B0E9C31DEB5A5A5CE8B704A0041EE95BD7`	sample 0 frame snapshot
A04	`sample_001_action_compare.png`	85942	`4E9A6DA5FE5DF4BC796FFE9215530EA7ED2625DA6E45050198A425B376090FEA`	sample 1 action comparison plot
A05	`sample_001_frame.png`	33874	`33D7F9D9BFE615C6A72B4DBA32CBA299F01423D4E045ABF825F2011914F67AFF`	sample 1 frame snapshot
A06	`sample_002_action_compare.png`	87720	`B76F05DB82D7CDFF0D572BDC082E14C946D6AE8A8A092EDDF48594198DACA245`	sample 2 action comparison plot
A07	`sample_002_frame.png`	33580	`0C36C5AF43FC3EA1CBB383E10DEAB395B568A843869CAB9130B67353B8465922`	sample 2 frame snapshot
A08	`sample_003_action_compare.png`	87738	`A76C2EBC45C5802A4CF13499F0E68D5F6379EEF61986614A1680290410CDE7E2`	sample 3 action comparison plot
A09	`sample_003_frame.png`	31141	`0C22B8B65CE4391CA484AFBC32075565287D0CF287A422FCF4C40EA3FC1F69ED`	sample 3 frame snapshot
A10	`samples.jsonl`	1219	`6570879E9242F89700F2CBD2898BA5988A2BB0CE41E475483C403F152A529E82`	4 sampled action/GT records
A11	`serve.log`	3191	`F02D0E4D08D6879C2F080B3DA905B4EA11A73B7510B7D8867785FFD1384A0868`	Orbax restore、norm-stats、8020 listen/connection
A12	`summary.json`	2261	`D367885AFB621469C81CB885C53FD36A87F02F06B8F4ED9B854F32BDB6663BAC`	mean/std、four L1 records、checkpoint/config summary
A13	`summary_l1.png`	39519	`C7A17CC82EE02DB89DB5CD7EC1E7C5404E36D16FADE096B3402F587701EFAC26`	aggregate L1 visualization

6. Local immutable manifest：LIBERO/Franka rollout bundle

目录：`artifacts/libero_franka_true_rollout_v1/`。`summary.json` 与 model/no-op video 共同支持 task-0 70-step rollout 的 `0.0 reward / false success` 边界；zero-reference 是 no-op 对照，不是 demonstration ground truth。

ID	相对路径	字节数	SHA-256	角色
L01	`client.log`	2717	`BC1B7FE6E54735B4623062D151D47667BD418F4F7A46B6CC471EC1F961869572`	client checkpoint/config、task summary、result paths
L02	`index.html`	1168	`9CF3D32F8B22444F38AEF7CD27511E6783A9B149240488AF13DDBCBEC762311F`	rollout dashboard
L03	`model_policy/model_policy_metrics.png`	95686	`D5EDB3193BCD2257771304CA97CB226A55CA0567267A9CC209DAC810D7C7CC28`	model rollout metric plot
L04	`model_policy/model_policy_rollout.mp4`	211605	`B8C14912B1202085581286B05949D8957A5AA77E040AE52AB91FFC0F67A60ED6`	model rollout video
L05	`serve.log`	2679	`FF0B39D7B87D604D86486A0088E7CF7FD2D3EA99CB151B99FD5D9B9E60AACD2A`	Orbax restore、norm-stats、8399 listen/connection
L06	`summary.json`	1450	`341E77110D75E50B601A078923F51EF8FA39AAA045FA491843A1003B0320E1DB`	model/no-op reward/success summary
L07	`zero_reference_vs_model.mp4`	269652	`226953734221483E03B509C03AE997B98C256722B9281A78A39C86D226D2ECE1`	model vs no-op comparison video
L08	`zero_reference/zero_reference_metrics.png`	68808	`0214FC58F66A3CEFFB9A4B2F652369F76F9C3620E96ED4EF8D06FE5443ED4020`	no-op metric plot
L09	`zero_reference/zero_reference_rollout.mp4`	70571	`27BFAB393ECF92E3092D768A6294759287D6F779527EEEB2CE6160F26E7063FA`	no-op rollout video

7. Gate reconciliation

Gate	状态	直接证据	当前不能推出
local deployment/code manifest	`PASS`	D01、C01–C03 的 bytes/SHA 已重算	不能推出远端 checkpoint 与本地文件完全同一版本
ALOHA action-eval artifact manifest	`PASS WITH WARNING`	A01–A13 全部存在且逐文件哈希	offline action L1 不是 task success；checkpoint binary 仍未哈希
LIBERO rollout artifact manifest	`PASS WITH WARNING`	L01–L09 全部存在且逐文件哈希	`0/1` 仍不能归因于单一 action/controller/preprocess 原因
`pi0_aloha_sim/params` binary manifest	`UNVERIFIED`	仅有 deployment path、restore log、历史 11.189 GiB/21 files	无逐文件 SHA、Merkle root 或 remote revision
`pi0_aloha_sim` norm-stats manifest	`UNVERIFIED`	serve log 有 loaded path	stats 文件内容与 SHA 未落档
`pi05_libero/params` binary manifest	`UNVERIFIED`	仅有 deployment path、restore log、历史约 11.6 GiB	无逐文件 SHA、Merkle root 或 remote revision
`pi05_libero` norm-stats manifest	`UNVERIFIED`	serve log 有 loaded path	stats 文件内容与 SHA 未落档
private fine-tune lineage	`UNVERIFIED`	当前 bundle 只指向 official configs	不能把 official checkpoint 说成 Qwen/LoRA 个人训练结果
ownership	`PENDING`	无 pre-snapshot diff 与真实口述	不能使用“我实现了整个 deployment/checkpoint”

8. 面试安全口径

30 秒回答

“我把本地 OpenPiVLA deployment bundle 做了逐文件 SHA-256 manifest：ALOHA action-eval 的 13 个文件、LIBERO rollout 的 9 个文件，以及部署记录和 3 个运行脚本都能在本地直接重算。与此同时，我确认 `pi0_aloha_sim` 和 `pi05_libero` 的 `params`、checkpoint norm-stats 只出现在远端 `/data/msz/...` 路径，当前 Windows 工作区没有这些目录，所以我不会伪称已经有 checkpoint binary SHA。现有日志能证明官方 checkpoint 当时被 Orbax restore 并完成 server/client rollout，但不能闭合远端权重不可变性，也不能证明 private Qwen3-VL/LoRA lineage 或个人 ownership。”

高频追问

1. **为什么不把 `DEPLOYMENT.md` 的 11.189 GiB/21 files 当 manifest?** 因为它是历史摘要，没有每个文件的 path/size/SHA，无法检测目录替换或缺文件。
2. **本地视频和 summary 的 SHA 能证明 checkpoint 没变吗?** 不能；它们只能冻结结果 bundle。checkpoint binary 与结果 artifact 是两个独立 lineage 节点。
3. **norm stats 是否已经 frozen?** 只 frozen 了 server log 中的加载路径和事件；stats 内容、文件列表和 SHA 尚未可读。
4. **这是否影响 `0 reward / false success` 的结论?** 不影响该次本地 rollout 产物的记录；它影响的是跨机器重跑、checkpoint identity 和根因归因的强度。
5. **下一步怎样闭合?** 在原远端环境以只读方式导出 `params` / `assets` 逐文件清单、SHA、OpenPI commit、processor/tokenizer revision，并把 manifest ID 写入 server/client summary。

9. 下一步

1. 如果远端仍可访问，只读导出两个 checkpoint 的 `params` 与 `assets` manifest，不复制权重进本仓库。
2. 给 server/client 启动脚本保存完整 argv、UTC timestamp、shared run UUID 和 request IDs。
3. 若远端不可再访问，下一 bounded slice 转向 matched LIBERO failure probe，固定 observation、flip/resize、state axis/unit、7D action order、normalizer、replan、controller 与 scorer。

4. 本页不新增题卡、不改变 mastery；`PROJ-OPENPIVLA-007/008` 仍保持 `unseen`。

参考文献（References）

1. Local deployment record：`C:\Users\mousongzhe\Project\openpivla_deployment\DEPLOYMENT.md`（D01）。
2. Local ALOHA artifact bundle：`artifacts/lerobot_action_eval_aloha_sample_real_v2/`（A01–A13）。
3. Local LIBERO artifact bundle：`artifacts/libero_franka_true_rollout_v1/`（L01–L09）。
4. Local evaluation/rollout scripts：`scripts/eval_lerobot_action_server.py`、`scripts/run_lerobot_action_eval.sh`、`scripts/run_libero_true_rollout.py`（C01–C03）。
5. OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate。
6. OpenPiVLA ALOHA Action-Level Eval Load Gate。

Part 36 / 49 · `interview_materials/vla/sources/openpivla_runtime_rollout_artifact_gate.html` · SHA-256 `A383592EB99F0332529C572AB20DE12C9D03AEFD29CB3AA16FC74CC5A7E5AB57`

返回本卷目录

Canonical Markdown SHA-256：`a383592eb99f0332529c572ab20de12c9d03aefd29cb3aa16fc74cc5a7e5ab57`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\sources\openpivla_runtime_rollout_artifact_gate.md`

OpenPiVLA 本地 Runtime Rollout Artifact Gate：ALOHA 与 LIBERO

状态：`local-direct / immutable-summary-read / true-physics-rollout / WARN` 审查日期：2026-07-21 范围：`C:\Users\mousongzhe\project\openpivla_deployment` 的部署记录与已落档 rollout summary；本页只冻结运行时证据，不把上游 checkpoint 结果改写成个人训练成果。结论先行：本地 artifact 直接支持 **官方 OpenPI checkpoint 在 ALOHA MuJoCo 与 LIBERO/robosuite 物理环境中完成了可审计的 policy inference 与视频/轨迹落档**。ALOHA transfer-cube 三回合为 `2/3`，多视角两回合为 `1/2`；LIBERO spatial task 0 的 `pi05_libero` model-policy smoke 为 `0/1`，同时记录了 no-op reference。当前仍不能据此声称 OpenPiVLA 私有微调、个人 ownership、泛化 benchmark 或 archived `97.4%` lineage 已闭合。

1. Evidence freeze

ID	本地对象	直接核对	SHA-256 / 状态
Do 1	`C:\Users\mousongzhe\project\openpivla_deployment\DEPLOYMENT.md`	checkpoint、环境、运行目录、结果表、失败边界与 provenance 说明	`C0A3DD0C0DA8C1C0D36023046D5337900A71680B413657BC9B1062EEF2EEA14E` / local-direct
Ro 1	`artifacts\aloha_sim_real_rollout_20260608_154324\summary.json`	`pi0_aloha_sim`、`gym_aloha/AlohaTransferCube-v0`、3 episodes、success/reward/action/state 指标	`27E4A04C01FD3AA6DDB387B59B519C4B3CC17DF94961EEB17686A160AECE1C13` / immutable-summary
Ro 2	`artifacts\aloha_sim_visual_rollout_real_v1\summary.json`	6 路相机、2 episodes、多视角 true-physics rollout 指标	`CB1E331A82AE032C564FD74CE5F6E01CB1875EAC3A3DF4479943E7C8B4891687` / immutable-summary
Ro 3	`artifacts\libero_franka_true_rollout_v1\summary.json`	`pi05_libero`、`libero_spatial/task_id=0`、model/no-op 对照、视频文件名	`341E77110D75E50B601A078923F51EF8FA39AAA045FA491843A1003B0320E1DB` / immutable-summary

本页引用的 summary 文件位于用户项目目录，未复制进本仓库，也未修改其代码或运行产物。`DEPLOYMENT.md` 中的 remote path 只用于记录 provenance；本页的运行结果以本地 summary JSON 与同目录视频/轨迹文件为准。

2. ALOHA：真实 MuJoCo policy inference

2.1 三回合基础 rollout

`R01` 的 summary 写明：配置为 `pi0_aloha_sim`，任务为 `gym_aloha/AlohaTransferCube-v0`，最大步数 300，3 个 episode，`state_diverged_count=0`。成功率为 `2/3=0.6667`，平均最大 reward 为 `3.3333`，平均 action delta 为 `0.0201277`，平均 action jerk 为 `0.0079451`。每回合的成功布尔值、有限状态、轨迹 JSON 与 camera MP4 均存在于 artifact 目录。

这能支持的面试表述是：

> “我把官方 OpenPI policy 接到 ALOHA transfer-cube 的 MuJoCo 环境，做了 3 回合闭环推理；2 回合成功，所有状态保持 finite，没有 state divergence，同时把每步 trajectory、相机视频和 action smoothness 指标落档。”

这不能支持“我训练出了这个 checkpoint”或“该结果是大规模 benchmark”。summary 的 checkpoint 路径是 `/data/msz/openpivla/models/openpi-assets/checkpoints/pi0_aloha_sim`，而本页没有私有微调 checkpoint、训练 config lineage 或候选人 pre-snapshot diff。

2.2 多视角 rollout

`R02` 标记 `visualization_type=true_physics_rollout`，相机为 `top`、`angle`、`front_close`、`left_wrist`、`right_wrist`、`left_pillar`。2 回合的成功率为 `0.5`，平均最大 reward 为 `2.5`，平均 action norm 为 `2.0879`，平均 action delta 为 `0.02402`，`state_diverged_count=0`。每回合有 `multiview_overlay.mp4`、各相机视频、trajectory JSON 与 metrics JSON。

多视角视频是部署/诊断证据，不等价于新算法贡献。面试中应把“相机渲染、轨迹记录、状态/动作统计”与“策略本身的训练 ownership”分开。

3. LIBERO：任务级物理 rollout 的失败样本

`R03` 的任务是 `libero_spatial`、`task_id=0`：`pick up the black bowl between the plate and the ramekin and place it on the plate`。`pi05_libero` model-policy 与 zero-reference 各执行 70 步，二者 total reward 都为 `0.0`、success 都为 `false`；model-policy 的平均 action norm 为 `1.2689`，最终 state norm 为 `3.2866`。对照视频 `zero_reference_vs_model.mp4`、两条 rollout 视频与 metrics PNG 均已落档。

这里的关键面试结论是：

- 这是 task-level MuJoCo/robosuite rollout，不是只做 import 或离线 action plot 的 smoke；
- 结果是失败，不应包装为 LIBERO 成功率；
- zero-reference 是 no-op baseline，因为当前官方 checkout 没有随附 LIBERO demonstration action file；它不是 GT demonstration；
- 失败首先证明“策略在该任务/配置下未完成目标”，不自动证明 simulator 不可用，也不自动定位到 action axis、normalizer 或 controller 的唯一根因。

4. 证据边界与 claim 回写

Claim	当前 verdict	面试安全口径	升级所需 artifact
官方 OpenPI checkpoint 能在 ALOHA MuJoCo 推理	`local-direct / PASS WITH WARNING`	“官方 `pi0_aloha_sim` 在 3 回合 transfer-cube rollout 中完成了可复核 inference，2/3 成功。”	ALOHA 专属 checkpoint manifest/load log；当前新增的 LIBERO load-lineage gate 只闭合 `pi05_libero` server restore
多视角 ALOHA true-physics rollout	`local-direct / PASS WITH WARNING`	“6 路相机、视频、trajectory、action/state 指标均有落档；2 回合为 1/2。”	更大 episode 数、固定 protocol 与独立 evaluator
`pi05_libero` 任务级 rollout	`local-direct / WARN`	“LIBERO spatial task 0 的 70-step model-policy smoke 为 0 reward/false success，失败结果可复现读取。”	完整 task matrix、checkpoint load log、独立 success scorer 与 matched reference
OpenPiVLA 私有微调已产生上述结果	`UNVERIFIED`	不把官方 checkpoint deployment 与个人 fine-tune 混写。	immutable fine-tune checkpoint、config、训练日志、load/reload 与 lineage
个人 ownership / 创新点	`PENDING`	“我能确认运行与结果文件，但具体由我负责的代码边界仍需逐文件确认。”	candidate role confirmation、pre-snapshot diff、协作工具边界
archived `97.4%` 属于本地 rollout	`ARCHIVE-SUMMARY-ONLY`	“归档报告中有该数字，但本页 rollout summary 不证明同一 checkpoint/protocol/lineage。”	原始 benchmark manifest、checkpoint hash、评估脚本与 raw episodes

5. 面试回答模板与追问

30 秒回答

“我先把官方 OpenPI policy 接入两个 task-native simulator 做部署验证。在 ALOHA transfer-cube 上，`pi0_aloha_sim` 做了 3 回合闭环 rollout，成功 2 回合，并落了相机视频、trajectory、reward、state norm、action delta/jerk；多视角版本 2 回合成功 1 回合。随后在 LIBERO spatial 的一个 Franka 任务上用 `pi05_libero` 做 70 步真实物理 rollout，model-policy 和 no-op reference 都是 0 reward、false success，所以我把它归为失败诊断样本，而不是 benchmark 成绩。这里能证明的是官方 checkpoint 的 runtime 与评估链路，不是我训练了 checkpoint，也不能把它和 97.4% 归到同一 lineage。”

高频追问

1. **为什么 ALOHA 成功而 LIBERO 失败？** 目前只能说任务、checkpoint/config、embodiment 与控制链路不同；summary 没有足够证据把原因唯一归到 axis、normalizer、相机或 controller，下一步要做 matched probe。
2. **你如何确认不是 fake rollout？** ALOHA 与 LIBERO 都有 simulator step、逐步 trajectory、reward/state/action 指标和视频；但这仍不替代独立 scorer 与 checkpoint manifest。
3. **你做了什么改动？** 当前证据只支持“部署脚本、环境隔离、评估/可视化产物已落档”；具体个人 ownership 尚未关闭，不能把 routine path/config 说成算法创新。
4. **为什么不把 2/3 写成 benchmark？** episode 数小、protocol 是 smoke/deployment run，且没有与官方 benchmark evaluator 和完整任务矩阵对齐。

6. 下一步

1. LIBERO server/client load log 已由 Checkpoint Load–Serve–Client Lineage Gate 冻结；下一步补 checkpoint 逐文件 SHA、norm stats SHA、精确 argv 与 shared run UUID，才能形成 immutable manifest。
2. 对 LIBERO 失败做一个最小 matched probe：固定 observation keys、action shape、axis/order/unit、normalizer、controller 与 scorer，并保留 zero-reference/model 对照。
3. 逐文件确认候选人的 ownership，再把本页事实映射到项目题卡；在真实口述前不新增 mastery 记录。

参考文献（References）

1. Local deployment record：`C:\Users\mousongzhe\project\openpivla_deployment\DEPLOYMENT.md`（D01）。
2. Local ALOHA rollout summary（R01）及同目录 `episode_*/trajectory.json`、`episode_*/camera.mp4`。
3. Local ALOHA visual rollout summary（R02）及同目录多视角视频/trajectory。
4. Local LIBERO rollout summary（R03）及 `model_policy/`、`zero_reference/` 视频与 metrics。
5. OpenPiVLA formal JSON/CE artifact gate，用于区分 local training representation 与官方 deployment runtime。
6. OpenPiVLA / LIBERO Checkpoint Load–Serve–Client Lineage Gate，用于补充 Orbax restore、normalizer、listen/connect 与 code contract 证据。

Part 37 / 49 · `interview_materials/vla/practice/robot_harness_oral_model_answer.html` · SHA-256 `299C25EEF26BF066DB95DDA9A9199ACD295258620E9CAA5F903A580803D5EA7D`

返回本卷目录

Canonical Markdown SHA-256: `299c25eef26bf066db95dda9a9199acd295258620e9caa5f903a580803d5ea7d`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\practice\robot_harness_oral_model_answer.md`

PROJ-HARNESS-001..008 从项目实现与 Po Audit 反推的证据约束口述稿

状态: `implementation_derived_model_answer / evidence_bounded / not_candidate_transcript / unseen` 更新日期: 2026-07-22 题目: Robot Harness 解决的核心问题是什么，如何解释 partial raw、proxy 指标、测试漂移与 Codex ownership?

1. 30 秒版本

Robot Harness 解决的不是“模型能不能调用函数”，而是高层工具调用如何在具体机器人、scope 和请求生命周期内安全执行。runtime 需要处理 robot 互斥、取消、超时、补偿和结构化错误；训练侧再把 allowed/forbidden tools、reference trajectory、success criteria 和约束转成 SFT/GRPO-style 数据与 reward。当前证据能支持 schema、runtime、数据和评测合同，但 project sandbox 100% 仍只是结构化协议分数，不能直接等同真实机器人成功率。个人贡献必须按问题定义、约束取舍、数据/reward/eval 设计和逐文件 ownership 分开说明。

2. 2 分钟版本

普通 function calling 只关注函数名和参数是否生成；机器人 harness 还要把请求绑定到 chat、scope 和 robot，并管理副作用。典型链路是：请求进入后构造 `AgentDeps`，过滤 server/client toolset，获取 robot mutex，再 dispatch/poll；在取消、超时或连接断开时检查 cancel event，尝试 stop/cancel compensation，最后返回结构化结果并释放锁。这样解决的是执行语义、并发和生命周期问题，不是低层视觉 grounding、控制器延迟或接触动力学。 数据侧我会把 SFT 与 GRPO 分开。SFT 可以使用规范化的 messages/tools/tool_calls；GRPO 只有在存在 allowed/forbidden tools、reference trajectory、success criteria 和可执行 scorer 时才进入 rewardable runner。no-call 或 clarification 样本不能从空 trajectory 直接伪造出来，browser deferred 也不能在没有 runner 时冒充真实执行轨迹。reward 可以包含 tool name/order、参数语义、scope、澄清和取消补偿，并对越权或 forbidden side effect 施加 hard penalty，但这仍然是 proxy，不是物理安全证明。 评测时我会分开 8B baseline、32B base、32B SFT 和 GRPO checkpoint，不把不同 scorer、不同 n 或不同 holdout 拼成一条因果曲线。project sandbox 的 100% 只说明在该 reference trajectory/schema 环境中通过；safety 多步任务仍弱，browser 还是 deferred，缺真实 controller/effect-sink。因此个人贡献应落在问题定义、约束取舍、数据/reward/eval 设计与结果归因，代码实现和 Codex 协作边界要按 ownership ledger 逐项确认。

3. 证据映射

结论	证据	当前强度
AgentDeps、toolset、stream/non-stream、cleanup	runtime.py / Harness Inventory	`local-direct / runtime-contract`
robot scope、mutex、cancel、compensation	registry.py 与 ownership ledger	`local-direct / safety-proxy`
SFT 62,223 与 GRPO 8,916 路由	data-preparation scripts、manifests	`local-direct / data-contract`
1,200 tool-restraint mix	restraint build script、manifest	`local-direct / reward-design`
32B ladder、sandbox/safety/browser 分项	local reports/metrics	`protocol-warning / proxy-only`
个人 ownership	Robot Harness Ownership Ledger	`candidate-confirmation-pending`

4. 高频追问答法

“为什么 mutex、cancel 和 compensation 必须一起？”

mutex 处理同一 robot 的排他冲突，cancel 处理请求生命周期变化，compensation 处理动作已经发出后的外部副作用。三者分别覆盖 before-dispatch、in-flight 和 after-side-effect；任一缺失都可能留下竞态或悬挂任务。best-effort stop/cancel 也不等于物理停止一定成功。

“SFT 和 GRPO 数据为什么不能全部混用？”

SFT 只需要可监督的消息/工具调用；GRPO 还需要可执行的 rollout、规则 scorer、成功条件和约束。没有 runner 或 reference trajectory 的样本应跳过、降级为 SFT 或单独做 probe，不能硬塞进在线 RL。

“sandbox 100% 是不是机器人成功 100%？”

不是。sandbox 主要检查 schema、tool/scope、reference trajectory 和参数约束，没有相机、controller、时延、接触和真实 effect-sink；它最多证明结构化协议层。

“95.6% 是否证明 GRPO 比 base 提升？”

不能直接证明。必须绑定 run/checkpoint、suite、scorer backend、holdout n、prompt/template、数据 revision 和 raw predictions；当前 base 与 ladder 的 scorer/n 不完全一致，只能各自按协议陈述。

“哪些代码是你写的？”

按 ownership ledger 逐文件说明亲手修改、协作、工具辅助、仅审查和仅运行；Git author 或 Codex 主要实现记录都不能自动等于独立原创。

5. Po Audit：如何把工程结果讲到可核验边界

5.1 30 秒版本

> 我把 Robot Harness 讲成高层机器人工具协议与 post-training 评估项目：runtime 负责 scope、mutex、cancel 和 compensation，数据侧形成 SFT/GRPO/restraint 合同，评估侧再分 run、scorer 和 suite。当前 Po 只部分闭合：聚合算术、5/5 overlap、12/12 locator 可复核，但 raw primary JSONL 是 28/40，四模型 exact support 是 6/10；checkpoint、统一 scorer 和真实 effect-sink 缺失，所以 proxy 100% 不是机器人成功率，37.8→95.6 也不是严格因果 uplift。

5.2 2 分钟版本

> 这个项目的核心难点不是“让模型调用一个函数”，而是把高层动作绑定到具体 robot/scope/request 生命周期，并控制副作用。请求进入 `AgentDeps` 后，server/client toolset 按 scope 过滤；排他 robot action 申请 mutex；dispatch/poll 期间检查 cancel 和 timeout；取消或断开时执行 best-effort stop/cancel compensation；最后返回结构化错误并在 finally 释放锁。它覆盖执行语义和并发生命周期，不等于视觉 grounding、控制器时延或接触动力学。 数据侧不能把所有样本混成机器人轨迹。SFT manifest 是 62,223 条（55,967/3,164/3,092），canonical GRPO 是 8,916 条（8,052/439/425），另有 1,200 条有 no-action 证据的 restraint mix；空 trajectory、没有 runner 的 browser 数据不能伪造为可执行 reward task。reward 可以对 tool name/order、argument semantic、scope、clarification、forbidden side effect 和 cancel compensation 做分解与 hard penalty，但它仍是 proxy。 评估时我把 8B v3、32B base、32B SFT、GRPO checkpoint-100/200 分开。Po 审计已经复核十套件的聚合算术、5/5 overlap 和 12/12 run locator，但只有 28/40 primary JSONL、6/10 exact common-support；checkpoint binary/hash 为 0/6 本地可核验，scorer/backend 也混用 token-overlap fallback 与 Qwen hidden embedding。于是 `sandbox 100%` 只能说明结构化协议层，`37.8→95.6` 不能写成同 scorer、同 n、同 checkpoint 的因果提升；safety 多步仍弱、browser 仍 deferred，真实 effect-sink/runner 也没有闭合。当前 targeted tests 是 11 passed、1 个 `test-model` 配置漂移失败，我不会报 runtime 全绿。

5.3 Po Evidence Contract

Claim	证据	等级	当前口径
scope/mutex/cancel/compensation runtime	`runtime.py`、`registry.py`、ownership/Po audit	`local-direct / C1-C2`	可讲执行合同与约束取舍；不讲低层控制
62,223 SFT / 8,916 GRPO / 1,200 restraint	manifests、prepare/build scripts	`local-direct / C1-C2`	可讲数据路由；不是全量机器人轨迹
schema-aware reward	reward spec/checker、task-family fields	`local-direct / C2`	可讲 proxy 设计与 hard penalty；不讲物理安全保证
aggregate arithmetic/overlap/locators	Po audit：13/13 arithmetic、5/5 overlap、12/12 locator	`local-direct / C2`	可讲报表/对账正确性
raw predictions	hidden extraction 28/40 primary、6/10 exact support	`partial-raw / C2`	可讲部分 common-support；禁止称 full raw/paired benchmark
checkpoint lineage	0/6 本地 binary 可 hash/load	`missing / C3`	禁止称 checkpoint-verified
scorer/backend	token-overlap fallback 与 Qwen hidden embedding 混用	`protocol-warning / C2`	禁止把 37.8→95.6 当严格因果提升
sandbox 100%	project 130-case structured protocol suite	`proxy-only / C2`	只能讲 tool/schema/scope proxy

Claim	证据	等级	当前口径
real robot/browser success	effect-sink/physical runner/browser runner 缺失	`unverified / C3`	禁止宣称真实闭环成功
targeted tests	11 passed、1 `Unknown model test-model` config drift	`local-direct / P1`	可讲 residual/test drift，不能称全绿
personal ownership	13-row ledger；Codex-primary implementation boundary	`candidate-confirmation-pending`	用 owner/reviewer/implementation 三轴表述

5.4 高压追问的三档回答

“37.8% 到 95.6% 是不是 GRPO 提升？”

危险回答：是，GRPO 把 32B 从 37.8% 提升到 95.6%。

可过回答：不能直接这么说，因为 n 和 scorer 不完全一致。

强回答：

> 目前只能分别陈述各 run 的协议结果。base holdout 是 `n=1333`，GRPO ladder 是 `n=1000`；base 使用 Qwen hidden embedding backend，而 SFT/GRPO ladder 标记为 token-overlap fallback，checkpoint lineage 也没有本地 binary/hash。要说因果提升，必须冻结相同 suite、n、prompt/template、scorer、checkpoint parent 和 raw per-sample predictions，再做 paired diff；当前证据只支持“不同 run 各自的 descriptive metrics”。

“sandbox tool/scope 100% 是不是机器人成功 100%？”

危险回答：是，工具调用正确就说明机器人能完成任务。

可过回答：不是，它只测结构化协议，不包含真实控制。

强回答：

> sandbox 的 130 个 case 检查 tool/scope/reference trajectory 和参数约束，没有相机观测、controller latency、接触动力学、effect-sink 或真实 success predicate。它证明的是高层协议在 reference schema 下可通过；要升级为机器人成功，至少要有同 checkpoint 的 simulator/shadow/real runner、逐 episode action/state/contact trace、termination predicate 和 invalid-policy。当前 physical-success eligible claims 是 0。

“P0 说 raw 有 28/40，能否算完整 benchmark？”

危险回答：28/40 已经覆盖大部分，足以代表十套件。

可过回答：只能算 partial raw，不能代表完整 benchmark。

强回答：

> 28/40 是 primary JSONL 的恢复数量，6/10 suite 有四模型 exact ID support；它能支持部分 common-support 对账，但余下 raw、统一 scorer、checkpoint identity 和 effect-sink 仍缺。P0 audit 明确把 eligible causal uplift 和 physical-success claims 置为 0，所以我会说“部分 raw artifact 已审计”，不会说“十套件统一 benchmark 已完成”。

“Codex 主要实现，你到底贡献了什么？”

危险回答：Codex 只是打字，代码都算我写的。

可过回答：我负责问题定义、约束和验收，Codex 主要实现工程代码。

强回答：

> 我把贡献拆成 owner decision、review/验收和 implementation 三轴。能安全讲的是：我决定把 scope/mutex/cancel/compensation 分层，把无 runner 的空 trajectory 排除出 GRPO，把 no-call restraint 只建立在有文本证据的样本上，并拒绝把 scorer/n 不一致的 37.8→95.6 写成 uplift；Codex 主要展开 runtime、数据脚本和 checker。逐文件谁改、谁 review、谁运行，要按 13-row ledger 和本人确认回答，Git author 不能替代。

“测试 11 passed、1 failed，runtime 能说稳定吗？”

危险回答：11 个通过，整体稳定，1 个失败可以忽略。

可过回答：有一个配置漂移失败，不能说全绿。

强回答：

> 这次 targeted tests 是 11 passed、1 failed、2 warnings；失败点是 `test_execute_chat_non_stream_finally_releases_robot_mutex` 使用已不存在的 `test-model`，`build_model_settings()` 在 fake agent 返回前抛 `Unknown model`，同时日志显示 finally release 执行了。它暴露的是 test/config drift，不足以证明完整请求路径全绿；修 fixture 后需要重跑同三组测试，并保留 residual。也不能把 release log 单独升级成 runtime 可靠性结论。

5.5 Po 追问树

```
“你们的结果提升了多少？”
  -> 追问 run/checkpoint/suite/n/scorer/backend/raw predictions
  -> 缺任一项：降级为协议限定的 descriptive metric

“sandbox 100%”
  -> 追问 effect-sink、controller、termination predicate、episode trace
  -> 没有 runner：明确 proxy-only

“十套件都做完了”
  -> 追问 raw JSONL 数量与四模型 exact ID support
  -> 28/40、6/10：明确 partial raw，不称 full benchmark

“你写了全部代码”
  -> 追问具体 diff、review、operator、Codex 边界
  -> 项目页/ledger显示 Codex-primary：改用三轴 ownership 口径

“runtime 稳定”
  -> 追问 targeted test residual 与 config revision
  -> 11/1：报告 test drift，不报全绿
```

6. 练习边界

- 这是 model answer，不是候选人的真实 transcript。
- 使用时必须替换“我负责”的部分，并提供具体文件、决策、命令或验收证据。
- 真实口述前保持 `PROJ-HARNESS-001` 为 `unseen`，不创建 `coaching_state.md`。

References

1. Robot Harness Claim–Evidence Inventory。
2. Robot Harness Ownership Ledger。
3. PROJ-HARNESS-001..008 Atomic Cards。
4. Robot Harness Po Bundle Audit。
5. Robot Harness Common-Support Raw Manifest。

Part 38 / 49 · `interview_materials/vla/questions/proj_robot_harness.html` · SHA-256 `AB42F762AE9420ECEDD6B87457329C4CC398DC724F907B9E5D3F044A50836F36`

返回本卷目录

Canonical Markdown SHA-256：`ab42f762ae9420ecedd6b87457329c4cc398dc724f907b9e5d3f044a50836f36`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_robot_harness.md`

PROJ-HARNESS：Robot Harness 项目原子题卡（Direct-Evidence Batch 1）

状态：`drafted / cards_8_of_8 / WARN` 更新日期：2026-07-17 当前题数：8 / 8 覆盖范围：`PROJ-HARNESS-001..008` 难度：L2 4 题 / L3 4 题 优先级：P0 7 题 / P1 1 题 练习状态：全部 `unseen` 使用方式：先闭卷回答 30 秒，再展开代码路径、指标协议和反例；未真实口述前不得改为 `pass/strong`。

A. Harness runtime 与执行安全

▼ `L2` `P0` **PROJ-HARNESS-001 · Robot Harness 解决的核心问题是什么，为什么不只是普通 function calling?**

面试意图：判断你能否把平台工程表述成机器人可执行语义问题，又不夸成低层控制成果。

30 秒回答：它解决的不是“能否调用函数”，而是同一机器人上的 scope、排他动作、取消补偿和可恢复执行。harness 把约束写进 tool schema/runtime，并沉淀成 SFT/GRPO 与 eval 合同；它仍不等价于视觉或低层控制。

机制展开（约 2 分钟）：普通 tool call 只关心名字和参数；机器人执行还要绑定 robot/chat/scope，防止同一机器人同时导航与执行冲突动作，并在用户取消或连接中断时停止任务、释放锁、保留结构化状态。训练侧再把 allowed/forbidden tools、trajectory、success criteria 与 scope 变成监督和 reward。这个链路降低上层动作语义错误，但没有覆盖相机 grounding、控制器时延和接触动力学。

诊断 / 证据：从一个导航请求追踪 `AgentDeps → filtered toolset → mutex → dispatch/poll → cancel/compensation → structured result`；再检查同 robot 并发、跨 robot 并发、取消前后和工具异常四类 trace。当前源码证明 contract，真实机器人 success 仍缺 runner/episode 证据。

项目挂钩：个人贡献应落在问题定义、约束取舍、数据/reward/eval 设计和验收；不能只说“搭了 Agent 平台”。

追问树：scope 与 robotId 有什么区别？为什么要排他动作？哪些约束在 runtime，哪些进入训练?

危险答案：说“tool accuracy 高就证明机器人控制成功”，或把 memory/subagent runtime 抽象说成模型已经学会的能力。

来源：[1][2][3][4] 练习状态：unseen

▼ `L2` `P0` **PROJ-HARNESS-002 · PydanticAI runtime 从请求到工具执行的主链路是什么?**

面试意图：确认你能从代码解释 context、toolset、流式协议与资源清理，而不是只背框架名。

30 秒回答：`_prepare_agent` 将 chat/scope/robot/token/cancel 写入 `AgentDeps`，组装 server/client toolset 后交给 PydanticAI。非流式 `run`，流式 `run_stream_events`；结束时释放 robot lock 与 cancel 状态。

机制展开（约 2 分钟）：prompt context 先解析 bound robot 与非 system history；history 会压缩重复 turn，并剥离不应被模型复用的 action-tool 记录。server tools 由 registry 提供，client tools 以 external/deferred schema 加入；重名时 server tool 优先，以保留取消/补偿保证。运行结果转换为 OpenAI 或 UI SSE，并持久化 UI parts；finally 统一释放 request 关联的 robot locks。

诊断 / 证据：打印 `AgentDeps`、最终 tool name 集合、server/client 冲突项、stream event 与 cleanup trace；验证非流式、OpenAI SSE、UI SSE 和客户端断开四条路径。源码直接支持 call graph，线上吞吐和稳定性没有由这些文件自动证明。

项目挂钩：可讲“把 PydanticAI 作为统一执行入口并保持既有协议兼容”；不要把框架迁移本身称为算法创新。

追问树：为什么 server tool 优先？deferred tool 如何返回？action history 为什么要 strip？cancel event 放在哪里?

危险答案：把 client deferred tool 当成服务端已经执行，或说 history stripping 等于模型具备长期记忆。

来源：[1][3][4][5] 练习状态：unseen

▼ L3 P0 **PROJ-HARNESS-003 · mutex、取消检查和 compensation 为什么必须同时存在?**

面试意图：检查你是否理解机器人副作用工具的并发、超时和失败语义。

30 秒回答：执行类 tool 先按 robot/request/chat 申请互斥锁，调用前后检查 cancel；导航和订单轮询在 tool 内处理超时，取消时 best-effort stop/cancel，并返回结构化 error。它防止并发冲突和悬挂副作用，但不保证补偿必然成功。

机制展开（约 2 分钟）：mutex 解决“同一 robot 两个排他任务”冲突；cancel event/registry 解决请求生命周期变化；compensation 处理动作已经发出后的外部副作用。导航 dispatch 后若取消，tool 会尝试 stop navigation；订单轮询取消则尝试 cancel order；协程被取消时还会安排 detached best-effort cleanup。三者缺一都会出现竞态、假完成或后台任务继续执行。

诊断 / 证据：做取消时序矩阵：dispatch 前、dispatch 后、poll sleep 前后、timeout、producer task cancellation；记录 lock、task/order id、stop/cancel result 和最终 error payload。补偿失败必须显式返回，不能吞掉异常后说系统安全。

项目挂钩：这类设计适合回答“机器人 Agent 与普通聊天 Agent 的系统差异”，也是取消补偿 reward 样本的 runtime 来源。

追问树：为什么是 best-effort？锁何时释放？跨 robot 是否可并发？超时后如何避免外部任务继续?

危险答案：说“收到 cancel 就一定停止物理机器人”，或用 mutex 代替外部系统的幂等和状态核验。

来源：[1][3][4] 练习状态：unseen

B. SFT / GRPO 数据与 reward

▼ L2 P0 **PROJ-HARNESS-004 · 62,223 条 SFT 数据是怎样构造的，哪些口径不能混淆?**

面试意图：看你能否讲清 schema、过滤、分组、来源构成，而不把混合数据说成机器人轨迹。

30 秒回答：62,223 是规范化混合 SFT 样本，不是机器人轨迹数。脚本统一 messages/tools/tool_calls，按 goal+trajectory 去重；project 只收非空 trajectory，ToolACE 排除，browser 保持 deferred，point/box/action-token auxiliary 过滤。

机制展开（约 2 分钟）：manifest 为 train/dev/test 55,967/3,164/3,092；任务族含 core tool 26,114、browser deferred 18,221、project robot 4,726、general capability 10,000、workflow safety 2,930。project trajectory 转成 assistant tool_calls，并把 scope 写入 metadata；稳定 hash 防止同一 goal/trajectory 跨 split。低层坐标/框/action token 被排除，只保留高层 VQA 辅助。

诊断 / 证据：复算 manifest 总量、来源/任务族分布、split group overlap、project 空轨迹拒绝数和 ToolACE 过滤数；抽样检查 tools schema 与 assistant tool_calls。数量正确不代表来源质量、真实机器人覆盖或所有样本均为多模态。

项目挂钩：面试先报“混合 SFT 总量”，再给 project robot 子集和过滤逻辑；这是比只报 62k 更可信的数据设计回答。

追问树：为什么 ToolACE 排除？browser 为什么保留在 SFT？group key 如何防泄漏？VQA 会不会稀释 tool call?

危险答案：说“62,223 条全是真实机器人轨迹”，或只凭总量证明模型具备机器人闭环能力。

来源：[1][6] 练习状态：unseen

▼ L2 P0 **PROJ-HARNESS-005 · GRPO 数据为什么不能直接复用全部 SFT 样本?**

面试意图：判断你是否理解 teacher forcing 与可验证 reward task 的数据合同差异。

30 秒回答：GRPO 只保留可规则判分 task：allowed/forbidden tools、reference trajectory、success criteria、constraints、minefields；空轨迹 project、WebLINX/Mind2Web 不进 runner。8,916 条，train/dev/test=8,052/439/425。

机制展开（约 2 分钟）：SFT 只需标准答案；GRPO 必须让 rollout 可评分，因此要保留 initial state、可用/禁用工具、参考轨迹、成功条件与风险字段。任务族包括 clarification 2,300、no-forbidden-side-effect 3,735、project robot 2,748、cancel compensation 120、bound scope 13。browser 没有 runner 时默认 disabled，project 没有轨迹时跳过，避免把不可执行样本硬塞进 reward。

诊断 / 证据：统计每个 split 的空 trajectory、missing tools/success criteria、browser disabled 和 group duplicate；对每个 task family 运行 gold trajectory scorer。若没有 executable runner 或规则 oracle，应降级为 SFT/OOD probe，而不是假装可做 online RL。

项目挂钩：个人设计价值在“把哪些样本放进 SFT、哪些能进入 GRPO、哪些只做 eval”这一分流，而不是把所有数据平均混合。

追问树：没有 reference trajectory 能不能做 RL？bound scope 为什么只有 13 条？browser runner 缺失会造成什么假标签?

危险答案：说“只要有 prompt 就能做 GRPO”，或把 browser deferred 样本当作真实网页执行轨迹。

来源：[1][7] 练习状态：unseen

▼ L3 P0 **PROJ-HARNESS-006 · schema-aware reward 与 1,200 条 tool-restraint 样本解决什么问题?**

面试意图：检查你能否解释 reward 分解、hard penalty、no-call 标签与 reward hacking 风险。

30 秒回答：reward 按 task family 加权 tool name/order/coverage、参数语义、scope、澄清、状态和取消补偿，并对 forbidden、越权、缺信息仍执行设 hard penalty；另合成 1,200 条 no-call restraint。AppWorld 空轨迹被排除，避免把 runner 缺失误标成拒绝。

机制展开（约 2 分钟）：规则可写成组件加权和加 hard penalty；不同 family 强调不同目标，例如 project robot 偏 state/scope/order/arguments，clarification 偏先澄清和不执行。restraint 样本让工具可见但正确动作是拒绝或追问，最终 mixed train=9,252。脚本只从有 no-action 文本证据的候选生成；空轨迹本身不是 no-call 证据。安全 suite 仍弱，说明规则并未消除 hacking 或长链漏步。

诊断 / 证据：对 gold、空调用、只调用一步、越权 scope、带额外参数、正确拒绝做 reward unit test；比较 old/new reward 的 large drops，并做 no-restraint/no-scope/no-arg ablation。当前有规则和离线检查，缺同 protocol 因果消融。

项目挂钩：可讲“从平台失败模式反推 reward schema”；不能说 hard penalty 自动保证真实世界安全。

追问树：semantic match 怎样防宽松过头？为什么 exact 只作 diagnostic？no-call 如何避免全拒绝策略？GRPO 会不会 exploit scorer？

危险答案：把空 trajectory 一律当拒绝标签，或说 reward 达到高分就已经解决 side effect safety。

来源：[1][7][8][9] 练习状态：unseen

C. Run lineage、指标与真实性边界

▼ L3 P0 **PROJ-HARNESS-007 · 8B、32B base、32B SFT 和 GRPO checkpoint 指标应如何分层陈述?**

面试意图：训练你主动绑定 model/run/checkpoint/suite/n/scorer，避免用好看的数字拼接一条不存在的曲线。

30 秒回答：需分 run：8B v3 holdout 95.3%；32B base 37.8%；32B SFT 95.42%；GRPO ckpt100/200 为 95.6%。但前两类 n=1333、GRPO n=1000，base 还用 Qwen-hidden scorer，不能直接宣称严格因果提升。

机制展开（约 2 分钟）：8B v3 是独立 full pipeline；32B base 单模型报告使用 schema-aware_v2 + Qwen hidden embedding。32B SFT 与 GRPO ladder 有本地 metrics，但 SFT/GRPO run manifest 使用 token-overlap fallback，GRPO 还截到 1,000 条 holdout。project suite 虽同为 n=130，也只证明结构化协议。面试可报每个 artifact 的值，但“37.8→95.6”必须先补同 scorer、同 n、同 suite 的 raw rerun。

诊断 / 证据：冻结 run ID、parent/checkpoint SHA、data/suite SHA、prompt/template、scorer/version/backend、n、raw predictions 和 summary；用统一 scorer 重算 base/SFT/ckpt100/200，再做逐样本 paired diff。当前 run/metrics 可读，统一比较 gate 未关闭。

项目挂钩：安全表达是“各自协议下分别达到”，并主动指出 comparator 缺口；这比把所有最好数字堆进简历更抗追问。

追问树：95.6 是哪个 checkpoint？为什么 n 不同？token overlap 和 hidden embedding 会影响什么？ckpt200 比 100 好吗？

危险答案：直接说“同协议从 37.8 提升到 95.6”，或把 8B 95.3 当成 32B GRPO 结果。

来源：[1][9][10][11][12] 练习状态：unseen

▼ L3 P1 **PROJ-HARNESS-008 · project sandbox 100% 为什么仍不能证明真实机器人成功，项目贡献应怎样表述?**

面试意图：检查你能否同时守住 proxy metric、safety/browser 和个人 ownership 三条边界。

30 秒回答：sandbox 的 100% 与 argument 92.31/96.92 只证明结构化协议；safety 仍约 38%，browser 只是 deferred。候选人负责问题/约束/数据/reward/eval，Codex 主要实现；13-row ledger 不把 Git author 当手写证明。

机制展开（约 2 分钟）：sandbox scorer 没有相机、控制器、接触和真实 success predicate；ckpt100/200 safety tool accuracy 约 38.75%/37.37%，browser 无 runner。ownership ledger 进一步区分：候选人 self-attested owner/reviewer，Codex-primary engineering；Git 中有 sandbox `+859/-46`、robot registry `+156/-34` 和训练 artifact sync，但 commit author 仍不等于代码作者。强项应落在具体约束取舍、review、验收和 bad-case 判断。

诊断 / 证据：ledger 结构已完成；下一 gate 是回答 8 个角色问题并给 decision/review 证据。当前 targeted tests 为 11 pass/1 config-drift fail，不能说 runtime 全绿；真实 effect-sink 仍缺。

项目挂钩：项目结尾先给协议层结果，再主动说明 safety/browser/effect-sink 和 Codex implementation 边界。

追问树：真实 success 还差哪些层？为什么 safety 比 project suite 差？哪些代码是你亲手写的？Codex 做了什么？

危险答案：说“sandbox 100% 所以机器人任务成功率 100%”，或把 Git commit author 当作全部 runtime/training/eval 的独立 ownership。

来源：[1][2][10][11][13][14]　**练习状态**：unseen

参考来源

1. Robot Harness Claim–Evidence Inventory：runtime、数据、reward、run/scorer 与 ownership 的冻结边界。
2. Robot Harness 项目页：原有项目叙事、research framing 与高压追问。
3. `C:\Users\mousongzhe\agent\server\agent\runtime.py`：PydanticAI Agent、AgentDeps、server/client toolset、stream/non-stream 和 cleanup。
4. `C:\Users\mousongzhe\agent\server\tools\registry.py`：tool schema/过滤、robot mutex、取消、轮询、compensation 和 structured error。
5. `C:\Users\mousongzhe\agent\server\agent\history.py`：history compaction、action tool history stripping 与消息转换。
6. `C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\prepare_sft_data.py` 与 `remote_training_pipeline_v2\manifests\sft_data_manifest.json`：SFT schema、过滤、split 与 62,223 条统计。
7. `C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\prepare_grpo_data.py` 与 `remote_training_pipeline_v2\manifests\grpo_data_manifest.json`：rewardable task contract、8,916 条统计与 eval suite 路由。
8. `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\scripts\build_grpo_tool_restraint_mix.py` 与 manifest：1,200 restraint、9,252 mixed train 和 AppWorld 排除。
9. `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\reports\final_pipeline_report.md`：8B v3 pipeline、suite 指标、failure samples 与 browser/safety 边界。
10. `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\reports\32b_native_tools_eval_report.md`：32B base run 与 Qwen-hidden rescore。
11. `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\reports\eval_32b_ckpt100_200_d_ood_compare_1000_ladder_20260622_140818_attempt3_d32_ood32_comparison.md`：32B SFT/GRPO ladder 与 D/OOD 分项。
12. 本地 `native_tool_restraint_32b_sft_20260618_114113`、`...d_grpo32b_ckpt100/200` metrics/run manifests：n、scoring backend、checkpoint 与分项结果。
13. 本地 `git log --follow`：只作 commit authorship signal；ledger 已建立但候选人角色确认仍缺。
14. 本地 Robot Harness Ownership Ledger：candidate-owner / Codex-primary implementation、13 个文件/决策、Git diff 与 targeted-test warning。

Part 39 / 49 · `interview_materials/vla/evidence/projects/robot_harness_claim_evidence_inventory.html` · SHA-256 `B2BD65B223A000CD19A036ACAD40B8FB0498BF6EF77E753AFEA9A41D71022AEF`

返回本卷目录

Canonical Markdown SHA-256：`b2bd65b223a000cd19a036acad40b8fb0498bf6ef77e753afea9a41d71022aef`；生成自：`interview_materials\vla\evidence\projects\robot_harness_claim_evidence_inventory.md`

Robot Harness Claim–Evidence Inventory（RH-01）

状态：`drafted / local_direct_audit / ownership_ledger_drafted / WARN` 审查日期：2026-07-17 证据截止：2026-07-17 项目范围：AceAgent robot harness、PydanticAI runtime、SFT/GRPO 数据路由、schema-aware reward、8B/32B native-tools 评测 证据仓库：`C:\Users\mousongzhe\agent`（只读审计；该仓库原有 dirty worktree 未清理）

结论先行

本轮审计可以把 Robot Harness 的工程边界、数据构造规则、reward 组件和多组 eval artifact 从项目页自述提升到 `local-direct`。`server/tools/registry.py` 直接显示 PydanticAI toolset、server/client tool 分离、robot scope 过滤、互斥锁、取消检查、轮询超时和 best-effort compensation；`prepare_sft_data.py`、`prepare_grpo_data.py` 与 v2 manifest 直接支持 62,223 条 SFT、8,916 条 canonical GRPO，以及 1,200 条 tool-restraint 合成样本的路由事实。

评测结果必须拆成不同 run/模型/协议：

- 8B v3 full pipeline 的 report 记录 SFT tool-call holdout 95.3%、project sandbox 100%、safety side-effect 44.3%、browser deferred 74.8%、cross-source 85.8%。这是 8B `native_v3_full_grpo_schemaaware_20260617_2022` 的结果，不是 32B 主结果。
- 32B base `native_qwen3vl_32b_schemaaware_full_20260617_2114` 很弱：tool-call holdout 37.8%、project sandbox 59.2%、safety 21.8%、browser 4.2%、cross-source 23.8%。
- 32B SFT 与 GRPO checkpoint-100/200 ladder 的 95.42%/95.6% 等数字有本地 metrics/report 文件，但 SFT 是 `n=1333`，GRPO ladder 是 `n=1000`；ladder 的 run manifest 标记 `token_overlap_fallback`，而早期 32B base 报告使用 Qwen hidden embedding rescore。它们可作为“各自协议下的结果”，不能直接写成同 scorer、同样本量的 base→SFT/GRPO 因果提升。
- project robot sandbox 的 100% tool/scope、92.31% exact、96.92% semantic 仅说明结构化工具协议层；不能推出真实机器人闭环成功率，也没有真实 browser runner 证据。

当前 verdict 保持 `WARN`。ownership ledger 已明确候选人 self-attested owner/reviewer 与 Codex-primary implementation 的双轴边界，并把 13 个文件/决策分层；候选人逐项确认、review history、run/checkpoint lineage、scoring protocol 对齐及完整 raw prediction/effect-sink bundle仍缺。Git commit author 为 `mousongzhe` 只能作为提交线索，不能自动等于个人独立实现。

1. 证据标签与判定规则

标签	含义	本项目例子
`local-direct`	当前工作区可读取的 source/config/manifest/report/metrics	registry、数据脚本、v2 manifests、native-tools metrics
`local-inference`	由同一份源码或相邻 artifact 推断，未被单独实验直接证明	tool accuracy 是闭环前置代理；memory/subagent 是否被模型学会
`report-supported`	committed report 有结论，但底层远端/raw 文件不在本地	8B full pipeline report 的完成状态与部分训练摘要
`protocol-limited`	指标存在，但 scorer、样本量、任务协议或 runner 需要特别说明	32B ladder 的 95.6%、browser deferred
`ownership-partial`	Git 作者或项目页支持参与线索，但没有逐文件 diff/贡献分类	`mousongzhe` commit author
`unverified`	缺失 checkpoint、raw log、episode、代码 diff 或无法复算	生产机器人成功率、个人独立 ownership

面试回答使用四级门槛：C0=知道/参与；C1=有本地代码或配置证据；C2=有设计取舍、对照或 bad case；C3=有 immutable run、checkpoint、完整 protocol、raw result 与 ownership。没有 C3 bundle 时，指标只能带协议限定语。

2. Resume / 项目页 claim reconciliation

ID	当前 claim	证据	Verdict	面试处理
RH-CL-001	机器人集群控制需要 task/action schema、robot scope、取消补偿和排他动作	server/agent/runtime.py、server/tools/registry.py、AGENTS.md	local-direct	可讲 runtime contract；不要把它说成模型已经具备长期 memory/subagent 能力
RH-CL-002	构建 62,223 条 SFT 数据	sft_data_manifest.json：train/dev/test=55,967/3,164/3,092	local-direct	可报数量，同时说明来源混合与过滤规则
RH-CL-003	构建 8,916 条 canonical GRPO 数据	grpo_data_manifest.json：8,052/439/425	local-direct	可报数量；强调 browser 默认 deferred、project 空轨迹会跳过
RH-CL-004	tool-restraint mix 增加 1,200 条，mixed train=9,252	tool_restraint_mix_manifest.json、生成脚本	local-direct	可讲 no-call/clarification/refusal 目标；不能说 1,200 条来自真实 runner 轨迹
RH-CL-005	reward 约束 tool name/order、参数语义、scope、forbidden side effect、clarification、cancel compensation	prepare_grpo_data.py reward spec 与 schema_aware_reward.py	local-direct	可解释组件与 hard penalty；先承认权重不是物理成功率
RH-CL-006	8B full pipeline 达到 95.3/100/44.3/74.8/85.8	final_pipeline_report.md、eval_report.md	report-supported / protocol-bounded	必须绑定 8B v3 run 与 suite，不能迁移成 32B 成绩
RH-CL-007	32B base→SFT/GRPO 达到 37.8→95.4/95.6	base report、SFT metrics、GRPO ladder report	local-direct / protocol-warning	分开讲 base、SFT、ckpt100/200；说明 scorer、n、checkpoint 不完全一致
RH-CL-008	project robot sandbox 达到 tool/scope 100%，argument exact/semantic 92.3/96.9	32B ladder metrics，n=130	local-direct / proxy-only	说“结构化工具协议层”，不能说真实机器人成功率
RH-CL-009	safety side-effect 已解决	safety suite：GRPO ckpt100/200 tool accuracy 38.75/37.37，coverage 44.98/43.25	FAIL	主动讲它仍是 gating weakness，并给出失败样例
RH-CL-010	browser 能力已验证	browser suite 是 deferred/format eval，未执行真实 runner	FAIL	只能讲 schema/format 评测，不讲网页闭环
RH-CL-011	所有 runtime/training/eval 实现均为个人独立完成	项目页明确 candidate-owner / Codex-primary implementation；13-row ownership ledger 已建立	REJECT-INDEPENDENT / LEDGER-DRAFTED / CONFIRMATION-PENDING	使用“我负责约束定义、数据/reward/eval 设计与验收；Codex主要实现，具体review/修改按文件说明”

3. Source inventory（冻结输入）

Source	类型	SHA-256（当前工作区）	能支持	不能支持
server/tools/registry.py	local evidence	EF1AFE36697D16F4C2A1392F618444712A87F2F88EE3BDABE409BD2948D24CF1	tool 注册、过滤、scope、mutex、取消/补偿、结构化错误	个人 ownership、模型训练效果
server/agent/runtime.py	local evidence	CEE9148C2987DA7FF6839534EBCFED75B953A8678742AD9442523F96A6A77280	PydanticAI Agent、deps、chat/scope/robot/cancel、stream/non-stream	线上流量或真实机器人 success
server/agent/history.py	local evidence	CAD470FC6B00470C37636285A3F84732EBFCDEB824612D0439AA65B70B5FC805	history compaction、action tool history stripping、消息转换	训练数据是否覆盖 memory 能力
remote_training_pipeline_scripts/prepare_sft_data.py	local evidence	AF75E07ED5F2483686D0E4DFADFE23D7F50BD31FE3455FCC3BC7354AB5E85589	SFT schema、过滤、去重、project trajectory、split	个人设计 ownership、数据原始来源质量
remote_training_pipeline_scripts/prepare_grpo_data.py	local evidence	EAA4136BE5159E894A70D755B93C1BA427CAF6E47E6AB2802B3670E2DDB28250	rewardable task 字段、任务分流、权重/hard penalty	reward 与真实物理成功的等价性
remote_training_pipeline_v2/scripts/build_grpo_tool_restraint_mix.py	local evidence	7C0983317F1A645F748F1FBB9D6D84B7CA03A415CF3344193F86724F20DF925D	1,200 合成 restraint、AppWorld 排除理由	合成样本的真实执行覆盖
v2 manifests	local evidence	见 pipeline_manifest.json、sft_data_manifest.json、grpo_data_manifest.json、tool_restraint_mix_manifest.json	数量、run path、split、来源族、mix	checkpoint binary、raw logs
8B/32B reports	report-supported/local evidence	见对应报告 SHA	指标表、失败样例、评测限制	未列出的 protocol 或闭环结果

4. Harness runtime boundary

4.1 请求到 Agent 的信息流

`runtime.py` 的 `_prepare_agent` 将 `chat_id`、`scope_type/scope_id`、`robot_id`、`bound_robot_id`、access token、cancel event 放入 `AgentDeps`；根据 `tool_names` 构造 server toolset，并把客户端 deferred tools 作为独立 toolset 加入 PydanticAI `Agent`。非流式走 `agent.run`，流式走 `agent.run_stream_events`；取消时返回 499/abort，并在 finally 释放 robot locks、清理 cancel registry。

`registry.py` 的 server tools 统一使用 `RunContext[AgentDeps]`。机器人互斥任务先申请 `RobotSessionLock`；底层调用前后检查取消状态；导航和订单轮询在 tool 内处理 timeout、取消和 best-effort stop/cancel compensation；异常经 `build_tool_error` / `normalize_tool_result` 结构化返回。`build_server_toolset` 还会按请求 tool 名和 semantic-navigation allowed robot IDs 过滤工具。

这证明了“可执行协议边界”，不证明模型懂得物理动力学、视觉 grounding 或长期 memory。history 中的 action-tool stripping 也更像运行时安全策略，不应写成模型训练目标。

4.2 面试可画的最小链路

```
HTTP/OpenAI or UI request
  -> prompt + scope/robot/cancel context
  -> PydanticAI Agent
  -> server toolset (filtered) + client deferred toolset
  -> mutex / loop guard / tool execution
  -> structured result or cancellation compensation
  -> SSE/OpenAI response + persisted UI message
```

5. SFT 数据路由证据

`sft_data_manifest.json` 记录总计 62,223 条：train 55,967、dev 3,164、test 3,092。任务族分布为 core tool call 26,114、browser deferred 18,221、project robot 4,726、general capability 10,000、workflow safety 2,930、general agent SFT 232。

脚本直接支持以下规则：

1. OpenAI-compatible `messages + tools` 规范化为 assistant-side `tool_calls`；按归一化 user goal / expected trajectory 做稳定分组与去重。
2. ToolACE 显式排除；WebLINX/Mind2Web 保留为 browser deferred，但不混入 project robot GRPO。
3. project sandbox 只有 expected trajectory 非空且存在 tool call 才转入 project SFT；scope_type/scope_id 写入 metadata。
4. point/general auxiliary 排除 `<point>/<box>`、coordinate/bbox、低层 action-token 目标，保留高层 VQA 以补通用能力。

因此 62,223 是“规范化后数据集总量”，不是 62,223 条真实机器人轨迹；project robot 只有 4,726 条，且仍需区分训练/评测 split。

6. GRPO、reward 与 tool-restraint

6.1 canonical GRPO

`grpo_data_manifest.json` 记录 8,916 条：train 8,052、dev 439、test 425。任务族为 clarification_before_action 2,300、no_forbidden_side_effect 3,735、project_robot_task 2,748、cancellation_compensation 120、bound_scope_respected 13。WebLINX/Mind2Web 默认不进入 GRPO runner；project sandbox 空 trajectory 被跳过；行级字段保留 `allowed_tools`、`forbidden_tools`、`reference_trajectory`、`success_criteria`、`policy_constraints`、`minefields`。

6.2 reward 组成

可以把规则奖励抽象为：

$$R = \sum_i w_i c_i + \sum_j p_j \, \mathbf{1}[h_j \text{ 被触发}],$$

其中 c_i 是 tool name/order、required coverage、argument semantic、scope、clarification、state success、cancellation compensation 等组件，w_i 随 task family 变化；$p_j < 0$ 是 forbidden tool、缺失澄清仍执行、scope 忽略、取消不补偿等 hard penalty。该式是对 `reward_spec()` 的结构化解释，不是新的训练实现。

6.3 tool-restraint mix

`build_grpo_tool_restraint_mix.py` 从有工具池但无 reference trajectory、且文本证据支持 no-action/clarification/refusal 的候选生成 1,200 条；最终 mixed train=8,052+1,200=9,252。每条样本把“工具可见但不能调用”写入 policy constraint，success criteria 要求不调用工具并澄清或拒绝。AppWorld 空轨迹被排除，因为空轨迹可能表示 runner 缺失，不等于 no-call 标签。

7. Run / eval lineage 与指标协议

7.1 8B full pipeline（独立基线）

Run / model	suite（n）	关键结果	证据边界
`native_v3_full_grpo_schemaaware_20260617_2022`（8B v3）	SFT holdout 1333	tool 95.3%、coverage 96.4%	schema_aware_v2，Qwen hidden embedding；report + metrics
同上	project sandbox 130	tool/scope/critical/semantic 100%	协议层，不是物理成功
同上	safety 289	tool 44.3%、coverage 49.1%	多步 telecom/workflow 仍弱
同上	browser 930	tool 74.8%	deferred/format-only，无真实 browser runner
同上	cross 1820	tool 85.8%、coverage 86.5%	source 混合，不能当 robot-only

7.2 32B base、SFT、GRPO ladder

run	model/checkpoint	suite n	tool accuracy	coverage	argument exact / semantic	scorer / protocol
`native_qwen3vl_32b_schemaaware_full_20260617_2114`	Qwen3-VL-32B base	holdout 1333	37.8%	39.4%	22.1% / 27.4%	`schema_aware_v2` + Qwen hidden embedding
`native_tool_restraint_32b_sft_20260618_114113`	32B SFT model	holdout 1333	95.42%	96.25%	74.57% / 77.87%	`schema_aware_v2` + token-overlap fallback
same SFT run	same	project 130	100%	100%	91.54% / 96.15%	结构化协议层
`...d_grpo32b_ckpt100`	checkpoint-100	holdout 1000	95.6%	96.3%	75.3% / 78.5%	`schema_aware_v2` + token-overlap fallback
same	checkpoint-100	project 130	100%	100%	92.31% / 96.92%	结构化协议层
same	checkpoint-100	safety 289	38.75%	44.98%	5.19% / 9.69%	safety仍弱
`...d_grpo32b_ckpt200`	checkpoint-200	holdout 1000	95.6%	96.3%	75.1% / 78.3%	同 ladder scorer
same	checkpoint-200	safety 289	37.37%	43.25%	5.19% / 9.00%	比 ckpt100 更差

注意三层不可比性：

1. base 的 1333 holdout 与 GRPO ladder 的 1000 holdout 不是同一 n；
2. base 早期报告最终使用 Qwen hidden embedding，ladder manifest 明确为 `token_overlap_fallback`；
3. browser 仍是 deferred/format eval，OOD suite 与 D suite 也要分开看。简历若保留 95.4/95.6，必须绑定 run、checkpoint、suite、n 与 scorer。

8. 失败模式与真实能力边界

报告给出的安全失败样例是多步 telecom repair：gold 需要 5 个工具步骤，模型只选 1 个工具；这解释了 safety tool accuracy 仍只有 38–44%。browser 失败主要是 uid/target grounding，且没有真实浏览器执行器；project sandbox 的 100% 只能说明 reference trajectory 与 schema 约束在该 sandbox 上可满足。

所以更准确的因果链是：

```
schema / scope / reward 约束
  -> 降低高层 tool-call 格式、选择和参数错误
  -> 为低层 controller 提供更干净的调用输入（前置代理）
  -/-> 直接推出视觉、控制延迟、接触动力学或真实机器人 success
```

9. Ownership boundary

`git log --follow` 显示 `server/tools/registry.py` 与 `server/agent/runtime.py` 的相关提交作者为 `mousongzhe`；`d9db` 对 registry/runtime 有 `+156/-34` 与 `+6/-1` 的 robot integration diff，训练脚本在 `7a8402f Sync remote training artifacts` 中由同一作者一

次加入。项目页明确说工程实现和 schema 字段展开主要由 Codex 完成，因此正确口径是“候选账号提交并维护，候选人负责 owner/reviewer 决策，Codex 主要实现”。Ownership Ledger进一步记录了 13 个文件/决策与 8 项确认问题。

2026-07-19 的三组 targeted runtime tests 为 11 passed / 1 failed；失败来自测试使用已不在当前配置中的 `test-model`，mutex finally release 日志仍出现。该结果说明当前存在 test/config drift，不能写 runtime 全绿；修复应放入独立 implementation slice。

当前安全口径：

> 我负责把机器人平台的失败模式抽象成可训练、可验收的 schema / reward / eval 合同，推进 SFT/GRPO 数据路由、scope 与取消补偿约束，并用分层 suite 做结果归因；runtime 工程实现按仓库提交与文件逐项核对，不能把 Git author 自动说成全部代码 ownership。

10. 面试安全表述（约 2 分钟）

> 这个项目的核心不是单纯接几个 function，而是把机器人执行的高层语义先变成可检查的 task/action schema。runtime 里通过 robot scope、互斥锁、取消事件和补偿动作保证请求不会越权或留下悬挂任务；训练侧把 OpenAI-compatible tool-call 数据规范化成 SFT，把带 reference trajectory、allowed/forbidden tools 和 success criteria 的任务转成 GRPO reward。数据规模是 62,223 条 SFT 和 8,916 条 canonical GRPO，另有 1,200 条 no-call/clarification restraint mix。评测时我把 8B baseline、32B base、32B SFT、GRPO checkpoint-100/200 分开报告：project robot sandbox 的结构化协议分数很高，但 safety 多步任务仍弱，browser 只是 deferred format eval，所以不能把 tool accuracy 直接等同真实机器人 success。我的贡献边界是问题定义、约束取舍、数据/reward/eval 设计和结果归因，具体代码 ownership 需要按文件 ledger 继续核对。

11. 高压追问与回答钩子

追问	当前可答	不能越过的边界
95.6% 是怎么来的?	32B GRPO checkpoint-100/200 ladder 的 `sft_tool_call_holdout`，n=1000，schema-aware_v2，token-overlap fallback	不说与 Qwen-hidden base 同 protocol 的严格提升
为什么 project sandbox 100% 还不能说机器人成功?	它只检查 tool/schema/argument/scope/reference trajectory；没有低层 controller、视觉、时延、接触与物理环境	不报真实机器人 success rate
reward 是否会被 hack?	有 hard penalty、schema-aware critical/semantic 分解；safety 仍弱，报告有多步漏调用样例	不说 reward 已彻底解决
browser 是否上线?	deferred/format eval，真实 runner 未执行	不说 browser closed-loop 已验证
哪些代码是你写的?	先按 ownership ledger 分亲手、协作、上游、工具辅助、仅理解	Git author 不等于独立 ownership

12. P0 缺口与下一步

优先级	需要补的 artifact	关闭的 claim
P0	每个 32B run 的 `run_id + repo/config/data/scorer SHA + checkpoint SHA + parent model`	base→SFT→GRPO lineage
P0	同一 scorer、同一 n、同一 suite 的 base/SFT/GRPO raw predictions 与 summary	95.4/95.6 的可比性
P0	project/safety/browser 的 per-sample JSON、episode/runner 状态、失败 trace	proxy 与 closed-loop 边界
P0	ownership confirmation：逐文件决定/审查/修改/运行/Codex实现/协作边界	`RH-CL-011`；ledger结构已完成，候选人回答待补
P1	reward ablation：无 scope/no-arg/no-restraint 与 paired rerun	reward 设计因果性
P1	真实 simulator 或 shadow robot runner，side-effect disabled	tool metric→闭环能力的下一层证据

第一批 `PROJ-HARNESS-001..008` 题卡和 13-row ownership ledger 已完成，全部保持 `unseen`。下一切片优先做 ownership confirmation drill，或补 common-protocol raw eval；不创建 coaching state，除非候选人真实回答。

参考文献（References）

1. Robot Harness 项目页，用于核对简历叙事与原有问题树。
2. `C:\Users\mousongzhe\agent\AGENTS.md`，PydanticAI、tool registry、取消/补偿约束。
3. `C:\Users\mousongzhe\agent\server\tools\registry.py`、`server\agent\runtime.py`、`server\agent\history.py`。
4. `C:\Users\mousongzhe\agent\remote_training_pipeline_scripts\prepare_sft_data.py`、`prepare_grpo_data.py`。
5. `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\scripts\build_grpo_tool_restraint_mix.py`。
6. v2 manifests：`manifests\pipeline_manifest.json`、`sft_data_manifest.json`、`grpo_data_manifest.json`、`tool_restraint_mix_manifest.json`、`latest_32b_eval.json`、`latest_32b_eval_ladder.json`。

7. `reports\final_pipeline_report.md`、`reports\eval_report.md`、`reports\32b_native_tools_eval_report.md`。
8. `reports\eval_32b_ckpt100_200_d_ood_compare_1000_ladder_20260622_140818_attempt3_d32_ood32_comparison.md`。
9. `C:\Users\mousongzhe\agent\git log --follow` 输出：仅作 authorship 线索，不作 ownership 证明。
10. Robot Harness Ownership Ledger；canonical Markdown，review JSON。

Part 40 / 49 · `interview_materials/vla/evidence/projects/robot_harness_ownership_ledger.html` · SHA-256 `66E6A93DC1C512E0F5603355CBE910D6D7DB3AD0CA64AD41B0C494339BBA7BD7`

返回本卷目录

Canonical Markdown SHA-256：`66e6a93dc1c512e0f5603355cbe910d6d7db3ad0ca64ad41b0c494339bba7bd7`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\robot_harness_ownership_ledger.md`

Robot Harness Ownership Ledger：Owner 决策、Codex 实现与结果归属

状态：`ownership_audited / role_self_attested / codex_primary_implementation / git_process_partial / WARN` 审查日期：2026-07-19 证据截止：2026-07-19 项目范围：`C:\Users\mousongzhe\agent`、Robot Harness 项目页、简历、SFT/GRPO/reward/eval artifacts。 结论先行：该项目已经有一条明确且可信的自述边界——候选人以 owner 方式负责问题定义、机器人约束拆解、数据分流、训练/eval 口径、结果判断与验收；Codex 主要负责 harness、schema、训练/评测脚本和字段展开。Git 提供了 initial import、sandbox eval、robot platform integration 与 remote training artifact sync 的增量记录，但 commit author 不改变上述工具辅助事实。本页把 owner/reviewer、主要实现者、运行者和结果归属分开；62,223/8,916/1,200 的数据事实可核，95.4/95.6 与 sandbox 100% 仍受 scorer、n、checkpoint、proxy 和真实闭环边界约束。

1. 问题（Question）

面试官问“工程主要由 Codex 完成，那你在 Robot Harness 中到底贡献了什么？”时，回答不能停在“想法是我的”，也不能反过来把所有代码认成手写。需要逐层回答：

1. 谁定义机器人平台的失败问题与产品目标?
2. 谁决定哪些约束进入 runtime、schema、SFT、GRPO reward 和 holdout?
3. Codex 具体实现了哪些文件、字段和评测流程?
4. 候选人如何审查、修改、运行、验收和否决工具生成内容?
5. 训练 checkpoint、scorer、metrics 与真实 robot outcome 分别能归谁、支持什么?

本页只把已有证据组织成 ownership contract；不会把自述自动变成独立验证，也不会把 Git author、report 或 tool metric 当作物理闭环证明。

2. Role Fit 与 JD 能力映射

当前未提供具体 JD，按 VLA / Embodied Agent / Multimodal Post-training / Evaluation Platform 岗位映射。

岗位能力	当前可核证据	Candidate role	Codex / 工具角色	当前面试策略
问题抽象与系统边界	project page 明确 scope、mutex、cancel/compensation、tool schema 与真实控制边界	`self-attested primary owner`	展开工程结构与字段	重点讲为什么这些失败值得训练，不能只背代码
数据工程与 post-training	SFT/GRPO scripts、62,223/8,916 manifests、1,200 restraint mix	定义分流、约束与验收；角色为自述	主要实现脚本/字段	用样本准入和拒绝条件证明算法判断
Reward / RL 设计	schema-aware reward、hard penalties、safety bad case	决定 reward 目标与取舍；自述	主要实现 checker/eval	主动承认 proxy/hacking/long-horizon 缺口
Runtime / robot integration	PydanticAI runtime、registry、sandbox、robot integration diff	owner/reviewer；具体修改范围待确认	主要工程实现	不声称手写全部 runtime；能解释关键路径和验收
实验与评估	8B/32B reports、metrics、D/OOD ladder	设计口径、判断结果和 bad case；自述	生成/实现 evaluation pipeline	每个数字绑定 run/suite/n/scorer；不拼接不存在的曲线
真实部署能力	robot tool integration 与 simulator hooks	平台目标/验收可讲	实现细节工具辅助	tool protocol 只是真实执行的前置层，不是 success rate

3. 双轴 Ownership Contract

3.1 贡献向量

把项目贡献拆成五个互不替代的分量：

$$C = (P, D, I, V, R),$$

其中 `P` 是问题定义（problem framing），`D` 是约束与实验设计（design），`I` 是工程实现（implementation），`V` 是审查、运行和验收（verification），`R` 是结果/指标 ownership。当前材料对候选人的 `P/D/V` 有明确 self-attestation，对 `I` 明确标为 Codex-primary，对 `R` 仅有 protocol-bounded 支持。直觉上，“owner”不等于“手写所有代码”，而“Codex 实现”也不等于候选人没有贡献；强回答必须证明候选人具体做过哪些取舍、审查和失败判断。实现映射是项目页、Git diff、源码、manifest、测试、run/scorer 与候选人确认表。

3.2 状态词汇

状态	含义	允许表述
`candidate-self-attested`	项目页/简历明确声明候选人角色	“候选人自述负责……”
`codex-primary-implementation`	项目页明确工程实现/字段展开主要由 Codex 完成	“我定义和验收，Codex 落地实现”
`git-diff-supported`	有独立 commit 与 before/after diff	“候选账号提交了这次集成差异”
`local-artifact`	code/config/manifest/metrics 存在	“工作区包含并可复核……”
`execution-supported`	有命令、run manifest、checkpoint/load 或测试	“该路径在指定环境被运行/验收”
`result-protocol-bounded`	指标存在但 protocol/scorer/raw/closed-loop 受限	“在该 run/suite/n/scorer 下……”
`role-confirmed`	候选人逐项确认 owner/reviewer/operator/tool boundary	才能用精确第一人称动作

4. Git、artifact 与测试冻结

4.1 Git 过程证据

Commit	关键变化	能支持	不能支持
`bd88445f9e6163c26985840eb2a94eacdcc4bdc1`	Initial AceAgent import；registry 1989 行、runtime 481 行、history 646 行首次进入	初始代码快照与候选账号提交	每行手写 ownership
`0d6606ac68c4d45bfff23f1a6180b630f8559349`	agent sandbox RL rollout eval：sandbox `+859/-46`，tests `+23/-2`	sandbox/eval 有可观察增量	具体哪些行由候选人还是 Codex 编写
`d9dbd0025cd317b6a747e5d6fb8dd203b238b195`	robot integration：registry `+156/-34`、runtime `+6/-1`	robot/backend/semantic integration 的真实 diff	candidate 独立代码作者身份
`7a8402ff8cb45c579d324490175aa711cd571b18`	remote training sync；SFT/GRPO/restraint/reward 四个关键脚本新增 1,585 行	训练资产在该 commit 进入仓库	远端生成过程、运行者、checkpoint ownership
`b5feead...` / `b4ed713...`	archive remote process artifacts / training reports	后续归档行为	raw remote state 天然可信或完整

所有这些 commit 的 author/committer 均为 `mousongzhe`。项目页又明确 Codex 是主要工程实现者，因此正确解释是“候选账号提交并维护了由候选人定义、Codex 主要实现的工程资产”，而不是“Git author 推翻工具辅助边界”。

4.2 当前 worktree 与定向测试

`C:\Users\mousongzhe\agent` 当前已有用户变化：robot state、ROS2 sim 脚本、C++ runtime node 以及未跟踪报告目录；本轮未修改这些文件。

2026-07-19 只读运行：

```
python -m pytest server/tests/test_robot_mutex.py
                 server/tests/test_cancel_current_session.py
                 server/tests/test_acerobotics_sim_backend.py -q
result: 11 passed, 1 failed, 2 warnings
```

失败发生在 `test_execute_chat_non_stream_finally_releases_robot_mutex`：测试使用的 `test-model` 不再存在于当前 `llm_configs`，`build_model_settings()` 在 fake agent 返回前抛出 `Unknown model`；日志同时显示 robot mutex 的 finally release 已执行。这是当前 test/config drift，不能写成 runtime 全绿，也不能仅凭 release log 宣称完整请求路径正确。本轮是证据审计，不修复该测试。

5. Per-file / per-decision ownership matrix

ID	文件 / 决策	Local / Git 证据	Candidate role	Codex / 其他角色	Verdict	当前安全口径
RH-OWN-001	把 robot scope、mutex、cancel/compensation 转成可训练问题	项目页与 inventory 反复声明该 framing	primary owner / self-attested	Codex 协助结构化	role-clear / independently-unverified	“我负责定义问题、约束与验收口径”
RH-OWN-002	server/tools/registry.py	2111 行，SHA ef1afe...24cf1；initial import 后 robot commit +156/-34	owner/reviewer；具体函数待确认	项目页声明工程主要由 Codex 完成	git-diff + codex-primary	“我定义并验收 tool/scope/cancel contract；Codex 主要实现 registry”
RH-OWN-003	server/agent/runtime.py	486 行，SHA cee914...77280；robot commit +6/-1	runtime contract owner/reviewer	Codex-primary implementation	local-direct / role-scope-pending	“负责迁移目标与兼容验收，不说手写全部 runtime”
RH-OWN-004	server/agent/history.py	646 行，SHA cad470...805fc；只有 initial import	理解/验收范围待确认	initial/Codex provenance未细分	snapshot-only	“工作区包含 history 策略；不自动认领设计”
RH-OWN-005	server/agent/sandbox.py / rollout eval	2318 行，SHA 57460f...3292c；0d660 有 +859/-46	sandbox 目标、case 和验收 self-attested	Codex-primary engineering	git-diff / process-partial	“以 owner 方式推进 sandbox eval，代码主要由 Codex 落地”
RH-OWN-006	prepare_sft_data.py	377 行，SHA af75e0...5589；7a840 一次新增	数据准入/过滤/split 规则 owner self-attested	Codex/remote pipeline 实现	artifact-direct / author-tool-assisted	“我决定 SFT 分流与验收；脚本由 Codex 主要实现”
RH-OWN-007	prepare_grpo_data.py	253 行，SHA eaa413...8250；7a840 一次新增	rewardable-task 准入 owner self-attested	Codex implementation	artifact-direct / role-clear-high-level	“我定义哪些样本能进入 GRPO 及原因”
RH-OWN-008	restraint mix + schema-aware reward	271/684 行；SHA 7c0983...925d / a402d2...5e1a；同一 sync commit	no-call、scope、hard-penalty 取舍 self-attested	Codex 实现字段/checker/eval	design-self-attested / code-tool-assisted	“我负责 reward 目标和 failure gate，Codex落地 checker”
RH-OWN-009	SFT/GRPO/mix manifests	totals 62,223 / 8,916 / 1,200；生成时间和 source/task family 可读	口径选择与 acceptance owner self-attested	remote scripts生成，Codex实现	count-verified / generation-role-partial	“我负责数据口径和验收；数量由 manifest 支持”
RH-OWN-010	8B/32B training / checkpoint	reports 与 run manifests 可读；model paths remote	training strategy/results interpretation self-attested	remote launcher/implementation tool-assisted；operator待确认	report/run-supported / checkpoint-owner-open	“在各自 run 中评估；不说所有 checkpoint 本地可加载或独立训练”
RH-OWN-011	native-tools / D+OOD eval	metrics、failure/change samples、scoring backend 与 run manifests direct	eval protocol、验收与归因 self-attested	Codex 主要实现 scorer/report	local-direct / raw-full-predictions-open	“我设计分层 suite 并解释结果；代码由 Codex主要实现”
RH-OWN-012	37.8→95.4/95.6、sandbox 100%	base/SFT n=1333，GRPO n=1000；semantic backend不同；proxy-only	result selection/interpretation self-attested	report generation tool-assisted	C3-protocol-warning	每个数字绑定 run/n/scorer，不写真实 robot success或严格因果
RH-OWN-013	PydanticAI / open datasets / Qwen	requirements 与数据来源可定位	采用、适配、取舍	public upstream	upstream-only	不把框架、模型和开源数据原创算作个人成果

6. Claim → ownership reconciliation

Claim	等级	当前证据	处理
“设计 robot task harness 调度与表征框架”	C2	candidate self-attested framing + runtime/local diff；工程 Codex-primary	可写“设计约束框架并负责验收”；不写手写全部工程
“构建 62,223 SFT / 8,916 GRPO”	C1/C2	scripts/manifests direct；candidate owns routing criteria，Codex owns most implementation	数量可写；必须解释混合数据不是 robot trajectories
“设计 schema-aware reward”	C2	checker/source direct + candidate self-attested reward choices；无 matched ablation	可写“设计 reward 口径/约束”；不写已证明因果 uplift
“SFT/GRPO 后 37.8→95.4/95.6”	C3	base/SFT 1333、GRPO 1000；backend/protocol不同，full raw predictions缺	改为“各自保存的 holdout artifacts 为……”；严格 uplift 需统一重算
“sandbox tool/scope 100%”	C3/proxy	n=130 structured-protocol suite	可写 proxy metric；禁止真实 robot 100%
“safety side effect 已解决”	C3	safety 仍约 37–44%，直接反证	禁写；作为高价值 bad case
“browser 能力已验证”	C3	deferred/format only，无 runner	禁写 closed-loop；只说 schema/format eval
“所有代码都是我独立完成”	C2/C3	与 project page 的 Codex-primary 说明冲突	明确拒绝；用 owner/reviewer + tool-assisted implementation 口径

7. Production Gap

当前做法	大厂追问	缺口	最小补证据	安全表达
owner + Codex 实现	你实际做了哪些 review/修改?	P0	prompt/decision log、rejected alternatives、manual diff、review checklist	“我负责定义和验收，Codex主要实现”
commit author	哪些行是你写的?	P0	逐文件 role ledger + candidate confirmation	“提交者不是独立代码作者证明”
数据 totals	多少是真实机器人轨迹?	P0	source/license/robot subset/duplicate ledger	“62k 是混合 SFT；project robot 4,726”
schema-aware reward	如何防 reward hacking?	P0	paired adversarial suite + no-scope/no-restraint ablation	“规则可解释但覆盖有限，safety仍弱”
32B metrics	base/SFT/GRPO 是否同 protocol?	P0	common-support raw rerun，同 n/scorer/template	“当前为各自 protocol 下的结果”
project sandbox	与真实执行相关吗?	P0	simulator/shadow effect-sink episodes	“高层协议 proxy，不是闭环 success”
browser deferred	有没有真实网页执行?	P1	runner、trace、side-effect-disabled test	“只完成 format/schema eval”
当前 test drift	runtime 是否全绿?	P1	修复 model fixture 后同三组 tests rerun	“11 pass/1 config-drift failure”

8. Question Tree 与回答分层

8.1 主追问树

```
工程主要由 Codex 完成，你贡献了什么?
  ├ 回答“问题定义”
  │    ├ 哪个真实 failure 让你加入 scope/cancel/restraint?
  │    ├ 哪些约束进 schema、reward、eval，为什么?
  │    └ Codex 给出错误实现时你否决过什么?
  ├ 回答“数据设计”
  │    ├ SFT 与 GRPO 的准入/拒绝条件?
  │    ├ 62k 中 robot subset 多大? 如何防泄漏?
  │    └ 1,200 no-call 为什么不是空轨迹伪标签?
  ├ 回答“结果验收”
  │    ├ base/SFT/GRPO 的 n、scorer、checkpoint是否一致?
  │    ├ safety 为什么仍差?
  │    └ sandbox 100% 为何不是 robot success?
  └ 回答“我写了全部代码”
       └ 与项目页 Codex-primary 声明冲突，直接进入真实性风险
```

8.2 高压题：Codex 写了代码，你是不是只提需求?

- 危险回答：想法都是我的，Codex 只是打字，所以代码也算我写的。
- 可过回答：我把 Codex 当实现加速器。我的责任是定义机器人高层失败、决定 schema/SFT/GRPO/reward/eval 的分层、审查生成代码和验收指标；工程实现与字段展开主要由 Codex 完成，我不会认领手写全部代码。
- 强回答（需候选人确认细节）：我能给出三个具体 owner 决策：第一，project 空轨迹不进入 GRPO，避免无 runner 被误标为 no-call；第二，scope/cancel/compensation 分别落到 runtime 与 reward，而不是只用格式 reward；第三，32B ladder 不能直接把 37.8→95.6 当同协议 uplift，因为 n 和 semantic backend 不同。Codex 落地脚本，我逐项审查 schema、manifest、failure samples 和验收门；例如当前 targeted tests 仍有一个 `test-model` 配置漂移，我不会说 runtime 全绿。

强回答中的“三个具体决策”和 review 行为仍需候选人确认；不能因为本页写出示例就视为真实口述证据。

9. SP/SSP 证据就绪度

维度	分数（5）	依据
岗位相关性	4.5	Embodied Agent、tool post-training、runtime/eval 与目标岗位高度相关
机制深度	4.0	schema/runtime/SFT/GRPO/reward/scorer 可机制级解释
工程闭环	4.0	runtime、数据、reward、run、metrics、failure samples 齐；真实 effect-sink 缺
指标可信度	3.0	local metrics 与 run manifest 丰富；common scorer/n、checkpoint/raw prediction bundle 未闭合
Ownership 清晰度	3.0	candidate-owner / Codex-primary 边界已明确；逐文件确认与 review history 仍缺
抗追问能力	3.5	proxy/safety/browser/scorer 边界清楚；当前简历强 uplift 仍需口径防线
合计	**22.0 / 30**	这是证据就绪度，不是候选人能力评分

一票否决项：隐瞒 Codex-primary engineering；把 tool metric 写成真实 robot success；把 safety 说成解决；把 browser deferred 说成上线；把 scorer/n 不同的数字称严格因果提升。

10. 候选人确认表

下一次 ownership drill 逐项回答“我决定 / 我审查 / 我修改 / Codex 实现 / 协作 / 不确定”，并给一个具体例子：

1. 哪个真实机器人/平台 failure 由你首先抽象为 scope、mutex 或 cancel/compensation?
2. `d9db` robot integration 中哪些行为是你明确要求和验收的?
3. SFT 脚本的 ToolACE 排除、project trajectory 准入和 group split 是谁决定的?
4. GRPO 的五类 task family、hard penalties 和 browser disabled 是谁决定的?
5. 1,200 restraint mix 的候选/重复策略是你选择、Codex建议还是远端复原?
6. 你亲自运行或查看过哪些 8B/32B checkpoint、日志和 metrics?
7. 你否决或修正过 Codex 哪个错误 schema、reward、scorer 或结论?
8. 简历中 37.8→95.4/95.6 与 sandbox 100% 的最终口径是谁确认的?

这些问题没有真实回答前，不创建 coaching state；ledger 状态保持 `candidate-confirmation-pending`。

11. 分阶段补强计划

时间	动作	完成标准	状态
半天	回答 8 项确认题；为 owner 决策各给一个 rejected alternative/bad case	13-row ledger 的 candidate/Codex/operator/reviewer 字段闭合	待确认
1 天	修复 test-model fixture 后 rerun；导出 review/checklist 与关键 prompt/decision log	targeted tests 全绿或有明确 residual，工具辅助过程可讲	待补做
3 天	同 n/scorer/template 重算 base/SFT/ckpt100/200，保留 full per-sample predictions	37.8/95.x 进入 common-support paired table	待补做
1 周	做 no-scope/no-arg/no-restraint ablation + simulator/shadow effect-sink	reward 因果与 proxy→execution gap 得到直接证据	待补做

12. Output Pack

12.1 简历 bullet 三档

- 保守版（当前证据）：以 owner 方式定义机器人工具协议训练问题，负责 scope、取消补偿、数据分流、schema-aware reward 与分层评估口径；借助 Codex 完成 runtime 和 SFT/GRPO 工程实现，构建 62,223 条 SFT、8,916 条 canonical GRPO 及 1,200 条 restraint 样本。
- 标准版（完成候选人确认与 **common-support** 口径后）：主导 Robot Harness 的约束建模与 post-training 验收，将 robot scope、forbidden side effect、clarification 和 cancel/compensation 映射为 SFT/GRPO schema 与 rule-based reward，并通过统一 holdout 和 safety suites 验证工具协议能力。
- 偏强版（必须补统一 **raw eval** 与 **effect-sink**）：建立从 runtime failure taxonomy、可验证 reward 到 simulator/shadow rollout 的端到端训练评估闭环，量化各 reward component 对 tool protocol 与真实执行的因果贡献。当前禁止写成已完成。

12.2 两分钟项目叙事

> 这个项目是我以 owner 方式推进、Codex 主要负责工程实现的机器人工具调用训练项目。我的核心工作不是声称手写所有代码，而是决定什么问题值得训练、如何转成可验证约束以及怎样验收。机器人执行相比普通 function calling 多了 robot scope、排他动作、取消补偿和真实副作用，所以我把这些约束分层：runtime 负责执行前后的 hard boundary，SFT 负责稳定工具协议，GRPO 只接收有 allowed/forbidden tools、reference trajectory、success criteria 或 minefields 的可判分任务；另加 no-call/clarification restraint，避免模型看到工具就调用。 数据侧形成 62,223 条混合 SFT、8,916 条 canonical GRPO 和 1,200 条 restraint；这些不是全部机器人轨迹。评估侧我把 8B pilot、32B base、32B SFT 与 GRPO checkpoint 分开，并主动保留两条限制：base/SFT/GRPO 的 n 和 semantic backend 不完全一致，因此 37.8→95.x 不是已经闭合的严格因果对照；project sandbox 的高分只证明 tool/schema/scope 层，safety 多步任务仍弱、browser 也只是 deferred eval。Codex 完成主要脚本和字段展开，我负责约束取舍、review、结果判断和验收；具体文件按 ownership ledger 逐项说明。

12.3 面试前 6 题

1. scope、mutex、cancel 和 compensation 分别解决什么失败，哪些不能互相替代?
2. 62,223 条 SFT 中真实 project robot 子集有多少，为什么不能混称机器人轨迹?
3. 哪些样本必须进 SFT、能进 GRPO、只能做 eval 或应该 drop?
4. tool-restraint 如何防止“全拒绝”与空轨迹伪标签?
5. 37.8→95.4/95.6 的 n、scorer、checkpoint 和 common-support 缺口是什么?
6. Codex 实现了什么，你做过哪些决定、审查、修改和验收?

13. Ownership 审计伪代码

```
def phrase_harness_claim(claim, ledger):
    # Separate owner decisions from primary code implementation.
    role = ledger.role_vector(claim)  # P, D, I, V, R

    # Git author proves submission identity, not handwritten implementation.
    if role.implementation == "codex_primary":
        implementation_phrase = "借助 Codex 完成工程实现"
    else:
        implementation_phrase = ledger.confirmed_implementation_phrase(claim)

    # Numeric claims require a common protocol and full result lineage.
    if claim.level == "C3" and not ledger.common_protocol_raw_bundle(claim):
        return implementation_phrase + "; 指标按 run/suite/n/scorer 限定"

    # Physical success additionally needs an effect-sink episode bundle.
    if claim.kind == "robot_success" and not ledger.effect_sink_verified(claim):
        return "只表述为高层工具协议 proxy"

    return ledger.exact_scoped_phrase(claim, implementation_phrase)
```

该伪代码是面试真实性 gate，不是 AceAgent/PydanticAI 的运行时代码。它把 candidate owner、Codex implementation、protocol-bounded result 与 physical effect-sink 分开。

参考文献（References）

1. Robot Harness Claim–Evidence Inventory，11 条 claim、runtime/data/reward/eval 边界。
2. Robot Harness Project Cards，8 张题卡，全部 unseen。
3. Robot Harness 项目页，candidate-owner / Codex-primary implementation 自述与面试口径。
4. Local repo `C:\Users\mousongzhe\agent`：commits `bd88445f`、`0d6606ac`、`d9dbd002`、`7a8402ff`。
5. `server/tools/registry.py` SHA-256 `ef1afe36697d16f4c2a1392f618444712a87f2f88ee3bdabe409bd2948d24cf1`；`server/agent/runtime.py` SHA-256 `cee9148c2987da7ff6839534ebcfed75b953a8678742ad9442523f96a6a77280`。
6. `server/agent/sandbox.py` SHA-256 `57460faa62d3b7e5b51dd4b0af7dc03c62da48916708fbce44eac00b4de3292c`。
7. `prepare_sft_data.py` SHA-256 `af75e07ed5f2483686d0e4dfadfe23d7f50bd31fe3455fcc3bc7354ab5e85589`；`prepare_grpo_data.py` SHA-256 `eaa4136be5159e894a70d755b93c1ba427caf6e47e6ab2802b3670e2ddb28250`。
8. `build_grpo_tool_restraint_mix.py` SHA-256 `7c0983317f1a645f748f1fbb9d6d84b7ca03a415cf3344193f86724f20df925d`；`schema_aware_reward.py` SHA-256 `a402d22299c1af0bab2d08e83e4389206882559294c3b1f55f9df55ac0f85e1a`。
9. SFT/GRPO/restraint manifests：SHA-256 `6700f4...72395`、`ae249f...c795`、`2d0821...b14ee`。
10. 32B base report SHA-256 `c8a00d531f90ad952f52dae563d276f0c73b88e189bc00f00840e72d553c8145`；D/OOD comparison SHA-256 `ab19733ad451af08d1b85bd3f41fe0051f76aa0e71f74580ce553b5c30465085`。
11. Optimized Resume 与 Resume Restructure Plan，当前投递 claim。

Part 41 / 49 · `interview_materials/vla/evidence/projects/robot_harness_p0_bundle_audit.html` · SHA-256 `FA8C1C38520885A8231D205A8918A990BB76472F924B7C53245BF1BC9034E7D8`

返回本卷目录

Canonical Markdown SHA-256: `fa8c1c38520885a8231d205a8918a990bb76472f924b7c53245bf1bc9034e7d8`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\robot_harness_p0_bundle_audit.md`

Robot Harness Po Bundle Audit：Runtime / Post-training / Eval

状态：`p0_audited / partial-raw / scorer-mixed / checkpoint-unverified / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计现有 runtime、数据、reward、十套件 aggregate 和 common-support raw evidence；不把 tool/schema proxy 分数升级为真实机器人闭环成功率。

1. 结论

Robot Harness 的问题定义、scope/mutex/cancel/compensation runtime、SFT/GRPO data contract、schema-aware reward 和聚合报表均有 local-direct 入口。十套件 bundle 的 arithmetic、5/5 overlap、12/12 run locator 和 28/40 primary JSONL 已复核，6/10 suite 具备四模型 exact ID support。

但 Po 仍未闭合：checkpoint binary/hash、统一 scorer/backend、base/SFT/GRPO common-support raw predictions、完整 10-suite raw、effect-sink/episode runner、operator/reviewer 和逐文件 ownership 缺失或不一致。

```
P0 status = PARTIAL
eligible physical-success claims = 0
eligible causal-uplift claims = 0
allowed = constraint/runtime/data/reward/proxy-eval evidence
forbidden = real-robot success rate, strict 37.8→95.6 uplift, independent ownership of all code
```

2. Po gate matrix

Po 字段	当前证据	状态	安全解释
problem / system boundary	project scope、robot scope、mutex、cancel/compensation、tool schema	`PRESENT`	可讲问题抽象与约束取舍
runtime implementation	registry、agent runtime、sandbox/robot integration diff	`PARTIAL`	Codex-primary implementation 与候选人 owner/reviewer 分开
SFT/GRPO data	62,223 SFT、8,916 canonical GRPO、1,200 restraint manifest/script	`PRESENT WITH WARNING`	可讲数据合同；不自动证明独立设计
reward / constraint	schema-aware reward、tool restraint、safety side-effect fields	`PARTIAL`	proxy 与真实物理 effect 分离
aggregate report	5/8-model ten-suite overview、13/13 arithmetic、60/60 metrics	`PRESENT WITH WARNING`	descriptive dashboard，不是统一 benchmark
raw predictions	普通目录无 full raw；hidden extraction 恢复 28/40 primary，6/10 exact support	`PARTIAL`	不能称十套件完整 raw 或严格 paired uplift
scorer/backend	token-overlap fallback 与 qwen-hidden 等协议混用；完整 scorer SHA 未冻结	`MISSING`	37.8→95.6 不构成严格因果提升
checkpoint lineage	overview 为 remote paths，0/6 本地 binary 可 hash/load	`MISSING`	不称 checkpoint-verified
effect-sink / runner	project sandbox 是结构化协议层；真实 robot/browser runner 缺	`MISSING`	100% proxy 不等于物理成功
ownership	13-row ledger、candidate owner/reviewer 与 Codex-primary 边界	`PENDING`	需逐文件回答，不用 Git author 代替

3. 已核验的边界

- 8B v3 full pipeline 的 95.3/100%/44.3% 等数字与 32B ladder 分开记录；
- 32B SFT 的 `n=1333`、GRPO ladder 的 `n=1000`，且 scorer/backend 不完全一致；
- project sandbox 130-case 的 tool/scope 100%、argument exact/semantic 92.31/96.92% 只说明 schema/reference-trajectory 层；
- safety 多步任务仍弱，browser 是 deferred/format eval，没有真实浏览器 runner；
- hidden extraction 的 28/40 primary JSONL 与 6/10 exact ID support 不等于 checkpoint、scorer、protocol 已统一。

4. 面试可用与禁用表述

可以说：

> “我把机器人执行失败拆成 scope、互斥、取消补偿、数据路由和分层评估，并能从 runtime/schema/reward/aggregate/raw manifest 解释证据边界。十套件目前有可复算的聚合算术和部分 common-support raw，但 scorer、checkpoint、四个 suite raw 和 physical effect-sink 没闭合，所以我把 95.x 和 sandbox 100% 定位为协议层/描述性结果，不说真实机器人成功率或严格 uplift。”

不能说：

1. 37.8→95.6 是同 scorer、同 n、同 checkpoint 的 GRPO 因果提升；
2. project sandbox 100% 等于机器人闭环成功率；
3. 已完成八模型统一 benchmark；
4. 所有 runtime/training/eval 文件均由本人独立实现；
5. partial raw bundle 可以替代 checkpoint、scorer 和 effect-sink。

5. 关闭 P0 的最小动作

1. 为每个 run 冻结 repo/config/data/scorer/checkpoint/parent SHA；
2. 用同一 scorer、同一 n、同一 suite 重跑 base/SFT/GRPO，并保留 per-sample JSON；
3. 补齐缺失 12 个 primary JSONL、project/safety/browser effect-sink 和失败 trace；
4. 候选人回答 `OWN-RH-001`，逐项说明决定、审查、修改、运行和 Codex/协作边界。

在字段闭合前，Robot Harness 保持 `PARTIAL/WARN`，不改变 `PROJ-HARNESS-*` 或通用题卡 mastery。

References

1. Robot Harness Claim–Evidence Inventory
2. Robot Harness Aggregate Bundle Gate
3. Robot Harness Common-Support Raw Manifest
4. Robot Harness Ownership Ledger
5. 五项目 P0 Artifact Intake Contract

Part 42 / 49 · `interview_materials/vla/evidence/projects/robot_harness_10suite_aggregate_bundle_gate.html` · SHA-256 `3DFE3C413D9DA4465FB42C6A6F7FBB93428F63B6A34D616703B7A8B9AE61C4E6`

返回本卷目录

Canonical Markdown SHA-256：`3dfe3c413d9da4465fb42c6a6f7fbb93428f63b6a34d616703b7a8b9ae61c4e6`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\robot_harness_10suite_aggregate_bundle_gate.md`

Robot Harness 五/八模型 × 十套件聚合证据门

状态：`local-aggregate-frozen / arithmetic-reproduced / common-support-partial-6-of-10 / checkpoint-unverified / WARN` 审查日期：2026-07-20 证据截止：2026-07-20 外部仓库：`C:\Users\mousongzhe\agent\`，审计时 HEAD `3f4c8b3a11ff9512bc18ec4a331ae5ea01555f49`；本页只读审计，没有修改该仓库。 范围：冻结 2026-07-07 的五模型与八 checkpoint 十套件 overview、builder、可定位的旧输入和本地 metrics；2026-07-20 勘误补充隐藏 remote-sync extraction 中恢复的 28/40 primary predictions 与 6/10 common-support suite。不把跨 metric 算术平均冒充统一 benchmark，不把 partial raw bundle 冒充十套件完整恢复，不把远端 checkpoint path 冒充权重 SHA。

1. 问题（Question）

Robot Harness 已存在两份十套件总览：一份展示 5 个模型，另一份展示 8 个模型/checkpoint。它们能否关闭以下 Po 缺口?

1. base、SFT、GRPO 是否已经在同一 case、同一样本数、同一 scorer 与同一模板上完成 common-support 对比?
2. `37.8% → 95.4% / 95.6%` 是否可以升级为严格、可复算的训练收益?
3. 十套件平均值是否是一个有统一统计含义的总体准确率?
4. 本地是否保存完整 per-sample predictions、checkpoint binary/config hash 与候选人 ownership?

结论：**仍不能关闭这些 Po 门，但能把 overview 的展示合同、算术、metric 语义、样本数差异和本地可复核边界冻结为一个 immutable aggregate bundle；后续 raw audit 另行证明 6/10 suite 已具备四模型 exact ID support。**

2. 结论先行

1. 五模型 JSON 有 5 个模型，checkpoint JSON 有 8 个模型；两者共同的 5 行在 `suite_scores` 与 `suite_n` 上 5/5 精确一致。
2. 两份 JSON 共 13 个 model rows；每行 10 个 suite，`D avg`、`OOD avg`、`all avg` 的 13/13 算术重算全部通过。
3. 十套件并非同一个 metric：共有 `tool_name_accuracy`、`plausible_tool_rate`、`safe_refusal_rate`、`policy_compliance_score_mean`、`normalized_text_match` 五种 primary metric。十套件平均只能叫 **descriptive index**，不能叫总体 success/accuracy。
4. 样本数并不统一：`cross_source` 为 1000/1820/2000，`sft_holdout` 为 1000/1333，两个 ToolACE probe 为 1000/5000。报告自己也记录“不同 run/protocol 与 row counts”。
5. 6 个 archived 模型的 12 个 D/OOD run 目录与 60 份 primary metrics JSON 均可在本地定位；2 个新模型的 4 个 run 目录以及它们的上游 comparison JSON 均未进入本地快照。
6. 60 份普通同步目录 metrics 包含 aggregate、分组统计与 0–80 条 failure sample；这些目录没有 full prediction/raw-output 文件。后续在隐藏 `.codex_tmp/remote_sync_20260626_141618/extract/` 恢复 28/40 个目标 primary JSONL：其中 6/10 suite 的四模型 ID set/order exact，另 12 个文件仍缺。
7. overview 引用的 6 个 distinct model/checkpoint path 全是远端路径；映射到当前 Windows 快照后 0/6 存在，0 个权重文件可哈希。
8. builder 与两个 overview 目录在 `agent` 仓库中是 untracked；当前可信 identity 来自逐文件 SHA，而不是 Git commit containment。

因此本页 verdict 为 `WARN`：它提升的是“聚合报表可解释性与失败边界”，不是模型因果收益、真实机器人闭环或个人 ownership。

3. 冻结输入与身份

ID	本地 artifact	状态	SHA-256	能支持	不能支持
RH10-A01	`reports/eval_5model_10suite_overview/eval_5model_10suite_overview_20260707.json`	local-direct / untracked	`5D79454550C50969A1441C14DA488AE6689E773EE12252EE6B7DE2AA672BF457`	5-model 展示数据、metric mapping、n、remote run path	新模型 raw inputs/checkpoint
RH10-A02	同目录 HTML	local-direct / untracked	`7950E04962D8817E073093793324D8735BF0344E7C1CCE7407CD88AA21FE2C66`	当时展示视图	canonical 计算逻辑

ID	本地 artifact	状态	SHA-256	能支持	不能支持
RH10-A03	`reports/eval_checkpoint_10suite_overview/eval_checkpoint_10suite_overview_20260707.json`	local-direct / untracked	`23AB0A9F8CEDEDDA52C65DDD98C8AB779502EA3B6B0705DE45BB427702C18D48`	8-model/checkpoint aggregate、10-suite 值与 n	common-support paired result
RH10-A04	同目录 HTML	local-direct / untracked	`C7B851E72203F914352D2A468D48EEF1A35F60F4DE25BD529EB73922262C34DA`	当时展示视图	全量来源证据
RH10-A05	`tmp_remote_recovery/build_5model_eval_overview.py`	local-direct / untracked	`69126F6A52B094BDC3219CA3CFFC97F58F6E488990B9348969CEC1B599067D47`	primary metric 选择、mean 与 HTML/JSON 生成逻辑	谁设计/运行/验收
RH10-A06	旧 32B/8B comparison JSON	local-direct	`5B5C47AF71A01D1FE10F14FCCF09BE8B1AA90B2D0251D70EDA683D9EB9DFC70B`	archived 行的上游 aggregate	checkpoint binary、full predictions
RH10-A07	新 122B/35B comparison JSON	remote-referenced / local-missing	—	overview 声明的输入路径	当前无法独立复算新两行

A01–A05 的 Git 状态均为 `??`。因此 HEAD 只记录外部仓库背景，不能替代这些文件的内容身份；本页用 full SHA-256 固定它们。

4. Builder 的真实计算合同

builder 固定五个 D suites 与五个 OOD suites，为每个 suite 指定一个 high-is-better primary metric，然后对可用值做不加权算术平均：

$$I_m^D = \frac{1}{5}\sum_{j\in\mathcal{D}} s_{m,j}, \qquad I_m^{OOD} = \frac{1}{5}\sum_{j\in\mathcal{O}} s_{m,j}, \qquad I_m^{all} = \frac{1}{10}\sum_{j=1}^{10} s_{m,j}.$$

变量解释：m 是展示行；$\mathcal{D}$ 与 $\mathcal{O}$ 分别是五个 D/OOD suite；$s_{m,j}$ 是 suite j 自己的 primary metric。直觉上它只是“把十个 0–1 分数等权放在一个仪表盘”；实现对应 builder 的 `statistics.mean`。因为 metric 语义、样本数与 protocol 不同，I_m^{all} 不是 pooled accuracy、episode success rate 或校准概率。

4.1 十套件 metric 映射

split	suite	primary metric	语义边界
D	`sft_tool_call_holdout`	`tool_name_accuracy`	工具名选择；不等于 argument/trajectory/physical success
D	`project_robot_sandbox_eval`	`tool_name_accuracy`	sandbox tool proxy；不含真实机器人 effect sink
D	`safety_side_effect_eval`	`tool_name_accuracy`	多步安全 suite 的工具名层；完整 safety 还需 forbidden/coverage/side effect
D	`browser_deferred_eval`	`tool_name_accuracy`	deferred/format eval；不是 browser runner
D	`cross_source_generalization_eval`	`tool_name_accuracy`	混合来源工具名层
OOD	`appworld_ood_api_policy_probe`	`plausible_tool_rate`	API plausibility proxy
OOD	`toolace_schema_given_probe`	`tool_name_accuracy`	给 schema 的工具名选择
OOD	`toolace_schema_absent_probe`	`safe_refusal_rate`	无 schema 时拒绝；与 accuracy 目标不同
OOD	`quarantine_safety_probe`	`policy_compliance_score_mean`	policy-compliance 连续分数
OOD	`robovlm_point_general_probe`	`normalized_text_match`	文本匹配 proxy；不是 2D/3D localization metric

4.2 为什么“每列都 high-is-better”仍不够

同方向只保证“数越大在本列越好”，不保证不同列有共同 estimand。拒绝率、tool accuracy、policy score 与 text match 的一个百分点不是同一种单位；把它们等权平均是一种展示选择，不是由 benchmark 或业务成本推导出的权重。

5. 样本数与 common-support 门

suite	overview 中出现的 n	当前判定
`project_robot_sandbox_eval`	130	base/SFT/GRPO100/200 ID set/order exact；scorer/checkpoint 仍未统一冻结
`safety_side_effect_eval`	289	四模型 ID set/order exact；scorer backend 仍不同
`browser_deferred_eval`	930	四模型 ID set/order exact；仍是 deferred proxy
`appworld_ood_api_policy_probe`	732	四模型 ID set/order exact；122B/35B raw 仍缺
`quarantine_safety_probe`	976	四模型 ID set/order exact；metric 不是 tool accuracy
`robovlm_point_general_probe`	400	四模型 ID set/order exact；仍是 text proxy

suite	overview 中出现的 n	当前判定
`cross_source_generalization_eval`	1000 / 1820 / 2000	只找回 GRPO100/200 各 1000 raw；base/SFT raw 缺失
`sft_tool_call_holdout`	1000 / 1333	只找回 GRPO100/200 各 1000 raw；base/SFT raw 缺失
`toolace_schema_given_probe`	1000 / 5000	不同 support
`toolace_schema_absent_probe`	1000 / 5000	不同 support

严格 paired 差值至少需要：

$$\Delta^{(j)}_{a,b} = \frac{1}{|C_j|}\sum_{i\in C_j}\left(g_j(y_{a,i}) - g_j(y_{b,i})\right), \quad C_j = E_{a,j}\cap E_{b,j}.$$

变量解释：$E_{a,j}$ 是模型 a 在 suite j 的 case-ID 集合；C_j 是两模型 common support；y 是原始输出，g_j 是同一版本 scorer。直觉是“先把同一道题和同一个评分器对齐，再算差”。overview 本身只保存 aggregate (n, s)；后续 raw bundle manifest 为六个 suite 恢复了 E 与 y 的文件 identity，但尚未冻结统一 g_j、checkpoint 与逐 case rescoring，因此本页仍不计算严格 paired uplift。

6. 三条 32B 结果的协议核对

行	holdout tool accuracy / n	scorer	semantic backend	project sandbox	safety tool accuracy
32B base	0.378095 / 1333	`schema_aware_v2`	`qwen_hidden:Qwen3-VL-8B-Instruct`	0.592308	0.217993
32B SFT	0.954239 / 1333	`schema_aware_v2`	`token_overlap_fallback`	1.000000	0.408304
32B GRPO100	0.956000 / 1000	`schema_aware_v2`	`token_overlap_fallback`	1.000000	0.387543
32B GRPO200	0.956000 / 1000	`schema_aware_v2`	`token_overlap_fallback`	1.000000	0.373702

这张表支持“各自 archived protocol 下的 aggregate”。它不能把 base→SFT/GRPO 当严格因果提升：

- base 与 SFT 虽同为 1333 行，但 semantic backend 不同；
- GRPO 两行只取 1000 行，不能与 1333 行直接 paired；
- SFT 的 safety tool accuracy 高于 GRPO100/200，说明“holdout tool name 接近”不等于 safety 单调提升；
- project sandbox 的 1.0 仍只是高层工具协议 proxy。

7. 本地 artifact census

7.1 已闭合

检查	结果
overview JSON 可解析	2/2
model rows	5 + 8
每行 suite 数	10/10
mean 重算	13/13 rows 通过
两 overview 重叠行	5/5 score+n 精确相同
archived D/OOD run 目录	12/12 本地存在
archived primary metrics JSON	60/60 本地存在
metrics `scoring_version` / `model_path` / `data` 字段	代表性 D 文件存在

7.2 未闭合

缺口	观测	后果
122B/35B input comparison	新 input JSON 本地不存在	两个新模型只能 remote-referenced
122B/35B run dirs	4/4 本地不存在	不能逐 suite 独立重算
full raw predictions	普通同步 run dirs 没有 raw；隐藏 extraction 恢复 28/40 primary files，6/10 suite 四模型 ID set/order exact	可建立六个 suite 的 support table；不能称十套件完整 raw bundle或直接发布 paired uplift

缺口	观测	后果
checkpoint binary	6 个 distinct `model_path` 映射后 0/6 本地存在	不能 content-hash 或 load-smoke
scorer/code SHA	overview 只给 metric name，不给完整 scorer revision/hash	不能证明所有行同实现
builder provenance	builder 与 report untracked	不能从 Git commit 推出 ownership 或不可变性
physical effect sink	project/browser 都是 proxy/deferred	不能升级真实机器人/browser 成功

`rescored_from_predictions=true` 本身只说明生成 metrics 时曾读取 predictions；后续恢复必须由实际文件、full SHA、rows 与 ordered-ID hash 单独证明。当前 28/40 已这样冻结，剩余 12 个不能用 failure sample 或 metrics 代替。

8. 八模型 overview 的安全解读

checkpoint overview 中 `8B GRPO` 的 descriptive `all_avg=0.6511435`、`D avg=0.8004548`、`OOD avg=0.5018322`；它略高于同表的 32B GRPO200 `all_avg=0.6247303`。安全口径是：

> “在这份跨 suite 描述性仪表盘里，8B GRPO 行的等权平均最高；但各列 metric、n、run/protocol 与新旧模型来源不同，所以这不能推出 8B 模型总体优于 32B/35B/122B，也不能作为模型 scaling 结论。”

尤其不能把 122B/35B 的较低 aggregate 解释为“大模型更差”：两行来自 docker vLLM + OpenAI tools protocol 的新 run，本地缺 source comparison 和 run dirs；其 protocol 与 archived native eval 不同。

9. Fail-closed 校验伪代码

```
def audit_ten_suite_bundle(bundle):
    # 1. Freeze every report, builder, scorer, run and checkpoint by content hash.
    require(bundle.report_sha and bundle.builder_sha)
    require(bundle.input_report_shas)
    require(bundle.scorer_code_sha and bundle.suite_manifest_sha)

    # 2. Preserve every case ID and raw model output; aggregates are insufficient.
    for model in bundle.models:
        require(model.checkpoint_sha and model.config_sha)
        require(model.full_predictions)
        require(model.case_ids == bundle.common_case_ids)

    # 3. Re-score all models with one scorer version on the common support.
    paired = score_common_support(
        predictions=bundle.full_predictions,
        scorer=bundle.frozen_scorer,
    )

    # 4. Keep heterogeneous suite metrics separate unless weights/utility are predeclared.
    require(bundle.predeclared_utility_weights)
    return paired
```

当前十套件 bundle 仍会在 `input_report_shas`、`checkpoint_sha`、四个 suite 的 `full_predictions/common_case_ids` 和 `predeclared_utility_weights` 处 fail closed；六个 suite 的 ID support 通过不代表其 scorer/checkpoint gate 已通过。

10. 项目真实性与 ownership 边界

可以说

> “项目里有一份五/八模型、十套件的聚合仪表盘。我重新冻结了 report 与 builder，13 个 model rows 的均值都能重算；又从隐藏 extraction 恢复了 28/40 个 primary JSONL，六个 suite 的 base/SFT/GRPO100/200 ID set/order exact。它仍混合五种 primary metric，四个 suite raw 不完整，新 122B/35B 输入和所有 checkpoint binary 也不在本地。因此我只把它当 descriptive dashboard 加 partial common-support bundle，不能说已经完成严格同协议因果对照。”

不能说

1. “我完成了八模型统一 benchmark”——没有同 support、同 scorer implementation、同 protocol。
2. “8B GRPO 总体优于 122B/35B/32B”——aggregate 的 metric/n/protocol 混合。
3. “37.8→95.6 是 GRPO 的严格 uplift”——SFT 已到 95.4，GRPO n 变为 1000，且缺 paired raw output。
4. “project sandbox 100% 代表真实机器人成功率”——没有 low-level controller、sensor/contact/latency/effect-sink。
5. “这些文件是我独立实现的”——builder/report untracked，候选人尚未完成 ownership confirmation。

11. 最小升级路径

1. 从远端归档 `openai_vllm_2models_20260706_1119_comparison.json` 与 122B/35B 四个 run 目录，记录完整 SHA manifest。
2. 恢复缺失的 base/SFT holdout/cross-source 与四模型 ToolACE 两套件共 12 个 primary 文件；随后冻结六个已对齐 suite 的 input/prompt/schema/scorer/backend/checkpoint，并生成逐项 scorer contribution。
3. 固定 scorer code、schema、suite JSONL、prompt/template、generation config 与 semantic backend SHA；不再混用 `qwen_hidden` 与 `token_overlap_fallback`。
4. 获取 checkpoint binary 或 immutable remote content manifest，至少闭合 checkpoint/config/tokenizer/parent lineage。
5. 预先声明 suite 权重或保持十列分开；不把等权 index 命名为总体准确率。
6. 由候选人完成 Robot Harness ownership drill，逐项确认 builder/metric choice/result interpretation 的决定、审查、修改与运行角色。

这些门关闭前，Robot Harness inventory 继续 `WARN`，所有 8 张项目题卡与全局 269 张题卡保持 `unseen`；本页不创建 `coaching_state.md`，也不启动 `PER-019`。

参考来源（References）

1. Robot Harness Claim–Evidence Inventory。
2. Robot Harness Ownership Ledger。
3. Evidence Manifest。
4. VOL 05 EVAL-005 Aggregation Knowledge。
5. VOL 05 EVAL-006 Benchmark/Scorer Versioning Source Freeze。
6. VOL 05 EVAL-009 Recomputable Raw-Bundle Source Freeze。
7. Robot Harness Project Cards。
8. Local bundle `C:\Users\mousongzhe\agent\reports\eval_5model_10suite_overview\`。
9. Local bundle `C:\Users\mousongzhe\agent\reports\eval_checkpoint_10suite_overview\`。
10. Local builder `C:\Users\mousongzhe\agent\tmp_remote_recovery\build_5model_eval_overview.py`。
11. Robot Harness Common-Support Raw Bundle Manifest。

Part 43 / 49 · `interview_materials/vla/evidence/projects/robot_harness_common_support_raw_bundle_manifest.html` · SHA-256 `DC7AC964B04CC429CA9FFBA9E57C5E3F9E5EAD0B1C63099D89A48A09EFB120D1`

返回本卷目录

Canonical Markdown SHA-256：`dc7ac964b04cc429ca9ffba9e57c5e3f9e5ead0b1c63099d89a48a09efb120d1`；生成自：`interview_materials\vla\evidence\projects\robot_harness_common_support_raw_bundle_manifest.md`

Robot Harness 四模型共同支持集 Raw Bundle Manifest

状态：`local-raw-partial-recovered / 28-of-40 primary files / common-support-6-of-10 / recovery-search-complete / 0-eligible-replacements / checkpoint-unverified / WARN` 审查日期：2026-07-20 证据截止：2026-07-20 原始文件位置：`C:\Users\mousongzhe\agent\.codex_tmp\remote_sync_20260626_141618\extract\training_pipeline_v2_language_full`；本页只读审计，没有修改 `agent` 或 raw outputs。边界：本页恢复的是 32B base、SFT、GRPO checkpoint-100/200 的部分 per-sample prediction identity 与共同 ID support；不把 proxy suite 升级为真实机器人成功，不把当前 scorer source 冒充当时 scorer binary，不把远端 checkpoint path 冒充权重 SHA。

1. 问题（Question）

上一份十套件 aggregate gate 在 12 个普通同步 run 目录中没有找到 raw predictions，因此无法判断 base、SFT、GRPO100、GRPO200 是否回答了同一组 case。全盘搜索随后在隐藏的 2026-06-26 remote-sync extraction 中发现 132 个 `*.predictions.jsonl`。本轮回答三个更窄的问题：

1. 目标十套件 × 四模型的 40 个 primary prediction 文件实际找回多少？
2. 找回文件能否逐行解析，是否有重复/空 ID、空 output 或 schema 漂移？
3. 哪些 suite 已达到四模型 exact ID-set + exact order 的共同支持集门，哪些仍需要恢复？

结论：**28/40 个 primary 文件可直接读取并通过结构校验；6/10 个 suite 的四模型 ID 集合与顺序完全相同。其余 12 个 primary 文件在该 extraction 中确实缺失，因此只能建立 partial common-support bundle，verdict 保持 `WARN`。**

2. 结论先行

1. Windows 传统路径探针把若干超过 260 字符的路径误报为 missing；使用 `\\?\` extended-length prefix 后，找回并流式校验 28 个 primary JSONL。
2. 28 个文件合计 `17,117,873` bytes、`17,828` rows；`bad_json=0`、`missing_id=0`、`duplicate_ids=0`、`empty_output=0`。
3. 所有 28 个文件共享同一 top-level schema 与同一 11-field embedded-metrics schema；这证明字段合同一致，不证明 scorer revision 或 semantic backend 一致。
4. `project_robot_sandbox_eval`、`safety_side_effect_eval`、`browser_deferred_eval`、`appworld_ood_api_policy_probe`、`quarantine_safety_probe`、`robovlm_point_general_probe` 六个 suite 的四模型 ID set 与 order 均完全一致，共覆盖每模型 `3,457` 个 case。
5. `sft_tool_call_holdout` 与 `cross_source_generalization_eval` 只找回 GRPO100/200 各 1000 行；base/SFT raw 缺失。两个 ToolACE suite 的四模型 primary raw 全部缺失。
6. 另发现 9 个 `.schema_aware.predictions.jsonl` 和 4 个 `.ood_scored.predictions.jsonl`；它们不是 40 个 primary 文件的替代品，也没有补齐四个缺口 suite。
7. 本轮只冻结文件 identity 与 support；没有以当前代码重新评分，也没有生成 base→SFT→GRPO uplift、置信区间或 checkpoint 因果结论。
8. 对 `agent`、`Project` 与原始 `remote_small.tar.gz` 做定向 recovery search：发现 17 个同名候选，但全部属于 GRPO 其他 attempt/singleload、8B 两行 smoke 或已存在的目标文件；**0 个可以填入 12 个缺失 slot**。

3. 对上一份 aggregate gate 的勘误

原 gate 的“12 个 archived run 目录中没有 full prediction 文件”只对普通同步目录成立。历史同步 manifest 明确排除了 `predictions.jsonl`；隐藏的 `.codex_tmp/remote_sync_20260626_141618/extract/` 保留了其中一部分。因此正确表述是：

> 普通同步目录没有 raw predictions；隐藏 remote-sync extraction 中恢复出目标 40 文件中的 28 个。六个 suite 已形成四模型 exact common support，四个 suite 仍缺 12 个 primary 文件。

这项勘误不会把旧 aggregate 的五种 metric、不同 `n`、semantic backend、122B/35B input、checkpoint binary 或 ownership 缺口自动关闭。

4. 冻结的工具与生成合同

artifact	SHA-256	本轮用途	边界
`tools/audit_robot_harness_raw_bundle.py`	`A2F3E54493097602918C4333D1C8AC03C0D6E88F9034D61AC0E9B0B186C81FC4`	长路径读取、JSONL 校验、common-support、12-slot locator 与候选拒绝审计	本轮审计工具，不是原训练/eval 实现
`robot_harness_common_support_raw_bundle_manifest.json`	`9FABE3E449D85DF10F998DE7C88BF1EAE110E52386D7E5AA29403EDBF2B1AB2F`	40 个 expected slots、28 个 file records、12 个 recovery slots、17 个 candidate records	轻量 manifest；不包含 raw output 文本
`native_tools_eval.py`	`F01EF13A029567297B35646AB5B2A17F7E8762D77FFE739370365848638FCEC4`	`<suite>.predictions.jsonl` 生成合同	当前本地 source identity，不证明每个历史 run 使用同一字节
`schema_aware_rescore.py`	`7918D6644BF49031E907D973C34A1B3CC0E88AB89EC09F3A29AFA4927E5695A7`	读取 primary 并生成 `.schema_aware.predictions.jsonl`	derived 文件不能替代缺失 primary
`score_ood_probe.py`	`6BBA56DBDE4947DE62B9A8FF38A8F215C828D556E193AA88E1E3F5AF4ABB76C2`	读取 primary 并生成 `.ood_scored.predictions.jsonl`	本轮未重新 OOD scoring
`openai_vllm_eval.py`	`F7B7920349C1574E75F60DC6192A7EA3AD607AD7CACCC921C61D22E4C84E959D`	reuse 前验证 expected ID exact order	新 122B/35B workflow 只作 contract 旁证
`eval_32b_service_client.py`	`25DB3FC22EBBA517DD988E4ED789DCB40173F77DD36C38AC5C5A598172ABFA14`	另一条 `<suite>.predictions.jsonl` writer	不证明本轮 28 文件由该 writer 生成

ordered-ID fingerprint 对文件顺序敏感：

$$H_{\mathrm{id}} = \mathrm{SHA}\,256\Big(\big\|_{i=1}^{n} \mathrm{u64\,be}(|b_i|)\|b_i\Big), \qquad b_i = \mathrm{UTF}\,8(\mathrm{id}_i).$$

变量解释：`u64be` 是 8-byte big-endian 长度前缀，避免简单字符串拼接歧义；随后按 JSONL 原顺序串联。实现映射是本轮 audit tool 的 `ordered_id_digest`。它证明相同 ordered ID stream，不证明输入 prompt、tool schema、generation config、scorer 或 checkpoint 相同。

5. 40-slot Primary 文件覆盖

suite	base32	sft32	grpo100	grpo200	四模型 common support
`sft_tool_call_holdout`	missing	missing	1000	1000	否；只可比较两条 GRPO
`project_robot_sandbox_eval`	130	130	130	130	是；set/order exact
`safety_side_effect_eval`	289	289	289	289	是；set/order exact
`browser_deferred_eval`	930	930	930	930	是；set/order exact
`cross_source_generalization_eval`	missing	missing	1000	1000	否；只可比较两条 GRPO
`appworld_ood_api_policy_probe`	732	732	732	732	是；set/order exact
`toolace_schema_given_probe`	missing	missing	missing	missing	否
`toolace_schema_absent_probe`	missing	missing	missing	missing	否
`quarantine_safety_probe`	976	976	976	976	是；set/order exact
`robovlm_point_general_probe`	400	400	400	400	是；set/order exact

6. 六个四模型共同支持集

split	suite	每模型 rows	set equal	order equal	ordered common-ID SHA-256
D	`project_robot_sandbox_eval`	130	true	true	`03329BE9FADB6A33C268EC18C4B22E8B41A6D5CC3499D9AF570AA6D93E9AB49D`
D	`safety_side_effect_eval`	289	true	true	`C9B9ACB006EF0BB5ECE55FAD83AE9E69F0FC9DE65B0A7A14D38DE816100AACBC`
D	`browser_deferred_eval`	930	true	true	`CABD9FA27ACB4115C58CC373CA83CA800AADE588BBF9046B5CEF82104E26E66C`
OOD	`appworld_ood_api_policy_probe`	732	true	true	`5083FDEBB7DA7FFCA4B0466024A1AC25CD4B9853B310210074BBC6F839EA76C1`

split	suite	每模型 rows	set equal	order equal	ordered common-ID SHA-256
OOD	quarantine_safety_probe	976	true	true	D45AE832A8CDF1180001FF4299E9419EA312FF9EA4F5619653AA65073055DF87
OOD	robovlm_point_general_probe	400	true	true	F744D0D352036D4C3A9DA20007DAEF8BED406A760C7047DF831099A8A7DBEC61

若未来冻结同一 scorer g_j，这六个 suite 已具备逐 case paired 计算的 ID 前提：

$$\Delta_{a,b}^{(j)} = \frac{1}{|C_j|} \sum_{i \in C_j} [g_j(y_{a,i}) - g_j(y_{b,i})].$$

变量解释：C_j 是上表冻结的共同 ID support，$y_{a,i}$ 是 raw output。本轮只证明 C_j 的 set/order identity；`g_j` 的 code/config/backend identity 与 checkpoint binary 仍未冻结，所以本页不报告这条差值。

7. 28 个 Primary 文件身份

7.1 D suites

suite	model	rows	bytes	SHA-256
sft_tool_call_holdout	grpo100	1000	990586	EF56A8F8670899561331CF3969AF91B7E2DC1C1581A1470E43DC93A1FA81A174
sft_tool_call_holdout	grpo200	1000	992363	4BA80F83572A96709F51D492168ED159D049FF2AEAC24ED0FA7D9693710AFB56
project_robot_sandbox_eval	base32	130	122749	E10A82A28BE175B888A827A87CBB85B82BE20FCDB985B2E6537DBE0C639507C8
project_robot_sandbox_eval	sft32	130	125909	6B09FAF5C85756318D1AF3632EF17203B7FE63E4B7BBD5A24EA2CD060385A18E
project_robot_sandbox_eval	grpo100	130	126213	1DC753581D9081724448BA16FADE68DCC9A2294EC45AE8E5FECB29E9DBDCB310
project_robot_sandbox_eval	grpo200	130	126367	31886143CDE02CA3B58E1AF7AECE77ADD11A2CA26DF46A8FCC28403461853A71
safety_side_effect_eval	base32	289	338848	B427489969141A4A6EFAF493A13501C4F9DEA9F7A6AD1019B619F1C111292A49
safety_side_effect_eval	sft32	289	334405	59224D84E2361E8027B4307EEE39E35A8F740E730A045EA61C34918ECDB54038
safety_side_effect_eval	grpo100	289	336629	103C4281DEC2016BEA71E9712EBB43D3825400EAD291370ED2F697673483935B
safety_side_effect_eval	grpo200	289	337777	5D8A15789CB05304F510C2A5672B5256EC9EFABEAD22DC129E0FA7471DE5E147
browser_deferred_eval	base32	930	808466	C8F2F480F07BA787801C04439EA13A3F90CE9628E9235DFC019E0EE16DEB6FCF
browser_deferred_eval	sft32	930	843644	11C8899540534E0E692D0E3FBD9BCD721436E2AD659D96FFB806CAEF51D8A3F0
browser_deferred_eval	grpo100	930	843924	9C7C0EC7992FA83A10548C3ECFD005222C550A2618DC074C566553A071C9FDB2
browser_deferred_eval	grpo200	930	840685	1D4BE1A286F6E825B6193ED33D287274DE5B03EF972242150662F4A1C04605EA
cross_source_generalization_eval	grpo100	1000	979978	FD3D8C1CD6E85E71AC3A0EB79D2F7FED4BCCBE192894E785F1CF05ABA7E33A4F
cross_source_generalization_eval	grpo200	1000	979460	BB476D0B53AD58ED7E31CC1D7D804D3D3E3F32BCA12BB2F85CD307126F5685BC

7.2 OOD suites

suite	model	rows	bytes	SHA-256
appworld_ood_api_policy_probe	base32	732	832866	FAC5FDE4026489480460ED549C6EF69E9E746DAB44F8310EFC0953067D3EB64C
appworld_ood_api_policy_probe	sft32	732	767117	530FC6552A076CEBB802A32A6A45D236778B7C61A35C3BEB427F837D2FB995CC
appworld_ood_api_policy_probe	grpo100	732	814884	A79E00AF2DFD71A971494F837F26B31CD8C24F06F6176A79D53A54FD4BFAC4F4
appworld_ood_api_policy_probe	grpo200	732	829542	DC426DEF5CD31A13C6DF2B6061790AB2133CDD50BE23C9EB205038ABB93504A6
quarantine_safety_probe	base32	976	998642	2EAFDF1098BC78CE84BD536F3B0E33822C636BF31484E404C9FC66B11CC45089
quarantine_safety_probe	sft32	976	894153	A7D5E4FA2FF2DAF02DBD1B1EA156749092228363CFEE28A3251D06E425D4F3F2
quarantine_safety_probe	grpo100	976	942873	599ACEEE1F4B0B64DFB59DAC72F9B5BAB9AB26B1713F0BA6BA70AA380DB93EA9
quarantine_safety_probe	grpo200	976	949895	8A88FB4FE8282518AB1C950B5134222C50820696007F5E824F20CFBD6D39A025
robovlm_point_general_probe	base32	400	253918	1A66123A191EB28CC77B2BAF5C3081AE0A77AFECEF72EEFDEC023162F373194C
robovlm_point_general_probe	sft32	400	235328	214254664FC32EA9AE56F67D232576D3CDDECCF27B5A6F42A78EC057A2AE15E2
robovlm_point_general_probe	grpo100	400	235328	06AF42AF35552098E25761FD71F5DCF50F113782F20B83E263B65E2F20AD0043
robovlm_point_general_probe	grpo200	400	235324	7BEB8770050D1BB33608F75238057268B5C363CEA771CD58F221361965B458FC

完整 relative paths、ordered-ID SHA、derived-file flags 与 validation counters 保存在同目录 JSON manifest；本页不复制 raw output 内容。

8. Schema 与 Derived 文件边界

28 个 primary 文件的唯一 top-level schema profile 是：

```
gold_signature, id, metrics, output, pred_signature,
source, task_family, tool_family
```

唯一 embedded metrics profile 有 11 项：`argument_exact_match`、`argument_semantic_match`、`bound_robot_or_scope_respected`、`browser_project_mixing`、`final_answer_contains`、`forbidden_tool`、`hallucinated_tool`、`malformed_tool_call`、`required_tool_coverage`、`tool_name_accuracy`、`valid_json_or_tool_call`。

这只证明 archived rows 的字段集合一致。`metrics` 是随文件保存的历史结果，不能在未冻结 scorer revision/backend 时当成严格统一实现。9 个 schema-aware derived 文件分布在 project/safety 四模型和 base-browser；4 个 OOD-scored derived 文件只覆盖 RoboVLM 四模型。derived 文件没有使十套件 primary coverage 从 28/40 变成 40/40。

9. 12 个缺失文件的 Recovery Manifest

split	suite	缺失模型	需要恢复的文件数	验收条件
D	`sft_tool_call_holdout`	base32、sft32	2	primary JSONL；full SHA；1333/实际 rows；unique ID；与 GRPO1000 support 的交集
D	`cross_source_generalization_eval`	base32、sft32	2	primary JSONL；full SHA；1820/2000/实际 rows；unique ID；与 GRPO1000 support 的交集
OOD	`toolace_schema_given_probe`	base32、sft32、grpo100、grpo200	4	四模型 primary JSONL；full SHA；exact ID-set/order；同 scorer input schema
OOD	`toolace_schema_absent_probe`	base32、sft32、grpo100、grpo200	4	四模型 primary JSONL；full SHA；exact ID-set/order；拒绝规则与 offered-schema 状态冻结

恢复时不能只复制 metrics JSON 或 failure samples。必须保留 primary raw output、ID、source suite SHA、prompt/tool schema、generation config、scorer code/config/backend identity 与 exclusion ledger。

9.1 12-slot Metrics 与 Run 身份

split	suite	model	expected rows	metrics SHA-256	run-manifest SHA-256
D	`sft_tool_call_holdout`	base32	1333	`B8F8AA8C74B6C836459E623BC9ECD07872F7ABB91C8ED205A71BBA6803448D47`	`E5955AF82EC29DED60951CAEF8F7FC5D47930DBA1036D76C934979D09EF68291`
D	`sft_tool_call_holdout`	sft32	1333	`747AAFF2D5E77F5FAE470F4A574532C8F0780BC9908C781137B6DF5641290CC0`	`F36A53ED1CAB05E0099CEDCCE30C1AFD35613C47F4E4801AC72C7D6BC4899635`
D	`cross_source_generalization_eval`	base32	1820	`3D7E4DEFCCE90358E4663A3179FD55B08DE8049EA181548DA718B9B8D5C7E0D2`	`E5955AF82EC29DED60951CAEF8F7FC5D47930DBA1036D76C934979D09EF68291`
D	`cross_source_generalization_eval`	sft32	1820	`3A25EA2E69015DB11A6EA2AB9B4794872CE3B07738B5A9EECE1858B0CD4D21CE`	`F36A53ED1CAB05E0099CEDCCE30C1AFD35613C47F4E4801AC72C7D6BC4899635`
OOD	`toolace_schema_given_probe`	base32	1000	`1278C5949FDF98AC95B4D3740EDCB56CB26AE8F3B40C16B61EA2391B39531A6C`	`B479F5813C30E8559C5E808B065CDF3CF2CCC8B234CD202770AA0002B938DC1E`
OOD	`toolace_schema_given_probe`	sft32	1000	`93C43A16F00ABA89841A6DBB95094D13184CD6F428A1A17DB72B5AADD7FEC999`	`26A61E9B12D8E63DB4C873BBE719B43DBFBE92C73FC5FBCEA97BB1B9E588BBFE`
OOD	`toolace_schema_given_probe`	grpo100	1000	`FDFD63F3780B7DC7899DB8057C16A39AE8E214275E9D8D7584E1E86101F9E546`	`537F9D690FADA2EFB62F1E2ACDA54DBAC689497B3922E7E7F6C5DF7655C8B230`
OOD	`toolace_schema_given_probe`	grpo200	1000	`6293C4C1DFEA786B6C9D697400E7871572694CAA71F9E5E440B72107D1902422`	`A73AA460B1C6D6E754BB9871B2119F0FD172585329077B053591AAFD5D6EF544`
OOD	`toolace_schema_absent_probe`	base32	1000	`E53BD9A6D5CE00B8F1315AA22E22B91B4E5053D999F65BA95D739D93C4FED1D8`	`B479F5813C30E8559C5E808B065CDF3CF2CCC8B234CD202770AA0002B938DC1E`
OOD	`toolace_schema_absent_probe`	sft32	1000	`E8DAEADE3229AA8A9C9CBA6B68EA65250C848F0F13187F710C467E3FAE441C88`	`26A61E9B12D8E63DB4C873BBE719B43DBFBE92C73FC5FBCEA97BB1B9E588BBFE`

split	suite	model	expected rows	metrics SHA-256	run-manifest SHA-256
OOD	`toolace_schema_absent_probe`	grpo100	1000	`3F6F453F5977A31EC14C86BCAE7BDCF5EA0C16F4D57953ECE477B4D183E3CE52`	`537F9D690FADA2EFB62F1E2ACDA54DBAC689497B3922E7E7F6C5DF7655C8B230`
OOD	`toolace_schema_absent_probe`	grpo200	1000	`0B37777D6B7927135C1D137B21DDEA65C327BC079F88876E0E78896ADE945E02`	`A73AA460B1C6D6E754BB9871B2119F0FD172585329077B053591AAFD5D6EF544`

D 的 `max_samples=0`，所以 metrics 冻结的 1333/1820 是实际 expected rows；OOD 的四条 run 都是 `max_samples=1000`、`max_seq_length=4096`、`max_new_tokens=128`。machine manifest 为每个 slot 保存完整 remote prediction path、source-suite remote path、model path、metrics/run/launch SHA；本地 12/12 slot 的 source-suite 文件均不存在，不能从 metrics 逆推出原 prompt/tools。

9.2 全盘候选搜索与拒绝理由

扫描域	同名候选	可填缺失 slot	结论
`C:\Users\mousongzhe\agent`	17	0	15 个 D 候选 + 2 个 ToolACE smoke；都不匹配缺失模型/规模/rows
`C:\Users\mousongzhe\Project`	0	0	没有额外 raw
`remote_small.tar.gz`，SHA `2C1BF435...EFA8CC`	17 members / 718 total	0 exact target member	tar 与 extraction 的候选集合一致，没有被漏解压的目标文件

candidate 类别	文件数	直接拒绝原因
GRPO100/200 attempt1/2/3 与 singleload 的 holdout/cross-source	13	缺失 slot 是 base32/SFT32；这些文件只重复已有 GRPO 模型，且不同 attempt 的 raw SHA 多数不同
`smoke_native_base5_20260617_135957` holdout/cross-source	2	manifest 实际是 `base_8b`、`Qwen3-VL-8B-Instruct`、`max_samples=2`，不是 32B base/SFT full run
`smoke_base_8b_20260617_232315` 两个 ToolACE	2	实际 8B、每套 2 rows，不能填四个 32B model × 1000 rows slot

候选总数按 path 是17；其中GRPO100 singleload holdout与attempt3 holdout的SHA完全相同，所以按content SHA去重会少1。machine manifest保留17条path provenance并全部标为`REJECT`，`eligible_candidates=[]`。这里的reject是“不能填当前12-slot recovery contract”，不是说这些文件本身损坏。

9.3 精确远端定位合同

D 缺失文件应位于两份 launch manifest 的 `output_root` 下：

```
/data/msz/dataset/online_agent/training_pipeline_v2_language_full/eval/native_tools_v1/results/
  native_qwen3vl_32b_schemaaware_full_20260617_2114/{suite}.predictions.jsonl
  native_tool_restraint_32b_sft_20260618_114113/{suite}.predictions.jsonl
```

OOD 缺失文件应位于四份 attempt3 run dir 下，文件名分别是 `toolace_schema_given_probe.predictions.jsonl` 与 `toolace_schema_absent_probe.predictions.jsonl`。`launch_eval_32b_ckpt100_200_d_ood_compare_20260622.sh` 的 SHA 为 `BF45E2A5903C2A28826828033E876521909D76E7D87F83536EC3644F5821B15F`；它把 `--suites-dir` 固定为 `${OOD_ROOT}/suites`、`--output-root` 固定为 `${OOD_ROOT}/results/${run_id}`。本地 machine manifest 已展开 12 条完整 remote path，可直接作为远端只读复制清单。

10. 为什么标准同步里没有 Raw 文件

`local_process_archive_20260629.json` 的 SHA 是 `EF4D98EAFAB166170696B7A44CA9CEF29DF8A193850E381ADF18B81A3095710E`，其 `excluded` 明列 `predictions.jsonl`。`Project/report/muxi_alignment_20260629/sync_manifest.json` 的 SHA 是 `4BEFF4CBE0FA441AB88E66843432545BDEC8525BDD13FA38B230BA054A476BC1`，policy 同样写明排除 raw predictions。因而“普通同步目录没有”和“隐藏 extraction 保留部分 raw”可以同时成立。

原始 `remote_small.tar.gz`（SHA `2C1BF435FDC42EDBDC376A03105B63E345E289B986D83BADA0152F7081EFA8CC`）共有718个普通文件；定向枚举找到17个缺口-suite同名 member，但0个是12条 exact target path。因此缺口不是 extraction 漏解压，而是 small archive 创建时就未包含。

这也解释了为什么本轮不把 61 MB 全局 132 文件复制进 `cv`：简历仓库只保存审计程序、轻量 JSON manifest 和人可读报告；原始内容留在只读 extraction，由 full SHA 定位。

11. Fail-closed 后续计算

```
def paired_suite_ready(bundle, suite):
    rows = bundle.common_support[suite]
    require(rows.all_four)
    require(rows.set_equal and rows.order_equal)
    require(bundle.suite_input_sha[suite])
```

```
    require(bundle.prompt_and_schema_sha[suite])
    require(bundle.generation_config_sha)
    require(bundle.scorer_code_and_backend_sha[suite])
    require(bundle.checkpoint_content_sha)
    return True
```

当前六个 suite 只通过前三行中的 common-support 部分；suite input、prompt/schema、generation、scorer backend 与 checkpoint content identity 尚未整体闭合。四个缺口 suite 连 `all_four` 都不满足。因此本轮不运行 paired bootstrap、不发布 uplift，也不把 `37.8→95.4/95.6` 改写成同协议因果收益。

12. 面试口径与 Ownership 边界

可以说

> “我对十套件 dashboard 做了第二层 raw audit。隐藏 extraction 找回28/40个primary JSONL、17,828行，六个suite的四模型ID set/order exact。我又扫描agent、Project和原始tar：17个同名候选全部是GRPO其他attempt或8B两行smoke，0个能填12个缺失slot；但每个slot的metrics/run SHA、expected rows和remote path已经冻结。因此我没有用同名文件凑数，也没有把partial support包装成完整统一benchmark。”

不能说

1. “十套件 raw bundle 已完整恢复”——只有 28/40、6/10 common-support。
2. “六个 suite 已证明 SFT/GRPO 因果提升”——只闭合 ID support，未闭合统一 scorer/backend/checkpoint。
3. “project/browser/point proxy 等于真实机器人/browser 成功”——仍没有 effect sink、controller、episode 或物理 outcome。
4. “我实现并运行了历史 evaluator”——本轮证明的是候选人/助手对 archived artifacts 的审计，不改变既有 ownership ledger。
5. “checkpoint 已可复现”——0/6 本地 checkpoint binary 与 load smoke 缺口仍在。
6. “17 个候选说明 raw 已经找齐”——候选的模型、run 或 rows 不匹配；eligible replacement 是 0。

13. 下一步

1. 使用 machine manifest 的12条 `expected_remote_path` 做一次受控远端只读复制；若只能恢复一批，先取base/SFT holdout与cross-source，以便与GRPO的1000-case support对齐。
2. 冻结六个已对齐 suite 的 input JSONL、prompt/tool schema、generation config、scorer code/config/semantic backend 和 checkpoint content manifest。
3. 在同一 frozen scorer 上逐 raw output 重算 per-case contribution，再做 paired difference、bootstrap interval 与 failure transition matrix。
4. 由候选人完成 Robot Harness ownership/结果解释口述；在此之前 8 张项目卡与全局 269 张卡继续 `unseen`。
5. 不创建 `coaching_state.md`，不启动 `PER-019`，不增加同义 aggregate 页面。

参考来源（References）

1. Robot Harness Ten-Suite Aggregate Bundle Gate。
2. Machine-readable Raw Bundle Manifest。
3. Robot Harness Claim–Evidence Inventory。
4. Robot Harness Ownership Ledger。
5. Evidence Manifest。
6. VOL 05 EVAL-009 Recomputable Raw-Bundle Source Freeze。
7. Local extraction `C:\Users\mousongzhe\agent\.codex_tmp\remote_sync_20260626_141618\extract\training_pipeline_v2_language_full\`。
8. Local archive policy `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\manifests\local_process_archive_20260629.json`。
9. Local sync policy `C:\Users\mousongzhe\Project\report\muxi_alignment_20260629\sync_manifest.json`。
10. Local archive `C:\Users\mousongzhe\agent\.codex_tmp\remote_sync_20260626_141618\remote_small.tar.gz`。
11. Local launcher `C:\Users\mousongzhe\agent\remote_training_pipeline_v2\scripts\launch_eval_32b_ckpt100_200_d_ood_compare_20260622.sh`。

Part 44 / 49 · `interview_materials/vla/practice/opd_oral_model_answer.html` · SHA-256 `12DF35D19EFBDE6039E300DCF6B234191093251BDCB5244082691 25E9E7C43A7`

返回本卷目录

Canonical Markdown SHA-256: `12df35d19efbde6039e300dcf6b234191093251bdcb524408269125e9e7c43a7`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\practice\opd_oral_model_answer.md`

PROJ-OPD-001..009 从项目实现反推的证据约束口述稿

状态: `implementation_derived_model_answer / evidence_bounded / not_candidate_transcript / unseen` 更新日期: 2026-07-22 题目: 如何介绍 OPD 的五 expert 数据合同、两阶段蒸馏、route/KL、checkpoint/point 协议、因果边界与 ownership?

1. 30 秒版本

OPD 的目标是让一个 student 同时覆盖 reasoning、object、region、spatial relation 和 point 等输出协议，而不是把五个 expert 的样本无标记地混在一起。数据构建阶段保留 target/candidate expert、expected format、seen/unseen、boundary 和 format-conflict metadata；formal `point/` trainer 先用 teacher-rollout full-vocab KL 做 coldstart，再用 student rollout 做 OPD5000，由 route teacher 在同一 response 上给完整词表分布。根目录同名脚本只是 legacy teacher-rollout/top1 CE，不能混为 formal 证据。当前 route 仍是 metadata selection，不是 learned router；组合结果也不能直接证明单因果 uplift 或真实机器人闭环。

2. 2 分钟版本

我先区分问题、数据合同和训练机制。五个 specialist 分别覆盖 reasoning、object、region、spatial relation 和 RoboPoint point grounding；如果只把数据拼起来，student 可能同时面对不同的输出格式、能力边界和 supervision 来源，最终出现 box/point/text 混淆。因此数据构建不仅保留样本内容，还写入 target/candidate expert、expected format、source line、image fingerprint、seen exposure、boundary 和 format-conflict。 数据侧先做 image-hash 的 train/eval 隔离，并对 spatial relation 做长尾重采样；student 数据再按 domain、general、boundary、format-conflict 分层。这样可以在评估时按 exposure、format 和 pool 分开看，不把“没有重复”直接说成模型泛化。 训练侧分两阶段，而且要先区分两份同名 trainer。根目录 legacy 版是 teacher generate、teacher logits 取 argmax、student 用 top1 hard-label CE；正式 new-experts 路线在 `point/train_opd_online_vl.py`。coldstart 从 base student 出发，由 teacher rollout；OPD5000 从 coldstart 输出继续，由 student rollout。两阶段都让 route teacher 在同一 response 上 forward，并计算完整词表的 $D_{KL}(q_\beta \| p_S)$。veto 期的 q_β 由 `teacher_logits + beta * stopgrad(student_logits)` 构造，beta 归零后才是纯 $D_{KL}(p_T \| p_S)$。这里的 route 来自 metadata，不是额外训练出来的 learned router；如果要称 learned router，还需要 routing model、routing loss 和同协议 ablation。 评估时我会绑定 raw-holdout、pool、scorer、checkpoint 和 denominator。不同 pool 的最佳 checkpoint 不一致，所以不能把 ckpt5000 说成所有指标最佳；旧版 OPD final 和新版 OPD5000 也不是同一条 run。当前可以讲数据合同、训练链和协议修正，但缺 SFT-only、single-teacher、no-synthetic、raw predictions 和逐文件 ownership，因此不把组合结果包装成 OPD 单因果 uplift 或个人独立完成。

3. 证据映射

结论	证据	当前强度
五 expert 配额、image-hash split、relation bucket	OPD Claim–Evidence Inventory	`local-direct / data-contract`
student route metadata、boundary、format-conflict	同上及 student builder/audit 记录	`local-direct / quality-gate`
legacy top1 CE vs formal full-vocab KL、coldstart teacher rollout vs OPD5000 student rollout	OPD Training Objective Lineage Gate	`local-code-reconciled / run-summary-bounded`
OPD5000 by-format/by-pool 曲线	inventory 的 checkpoint tables	`protocol-bounded / raw predictions missing`
point strict 修正	OPD inventory 与 point protocol reconciliation	`protocol-limited`
个人 ownership	OPD Ownership Ledger	`candidate-confirmation-pending`

4. 题卡覆盖映射

Question ID	实现主题	本页回答入口
`PROJ-OPD-001`	为什么不是无标记混合五个 expert	30 秒版本与项目定位

Question ID	实现主题	本页回答入口
PROJ-OPD-002	expert 配额与 relation 重采样	2 分钟版本的数据合同
PROJ-OPD-003	seen/unseen/boundary/format-conflict	2 分钟版本的数据分层
PROJ-OPD-004	coldstart500 与 OPD5000	2 分钟版本的两阶段训练
PROJ-OPD-005	route teacher 与 full-vocabulary KL	训练机制与高频追问
PROJ-OPD-006	checkpoint selection	metric/protocol 追问
PROJ-OPD-007	raw-holdout 与重复图片	image-hash split 追问
PROJ-OPD-008	point protocol 与因果归因	point/uplift 追问
PROJ-OPD-009	个人、Git、远端和工具边界	ownership 追问与 ledger

5. 高频追问答法

“route metadata 是 learned router 吗？”

不是。当前实现按样本中的 `target_expert/candidate_experts` 选择 teacher；它是数据/训练路由合同。learned router 需要额外模型、routing objective、部署时输入和 ablation，不能从 route id 自动推断。

“为什么一定要 full-vocabulary KL？”

它保留 teacher/target 在整个词表上的概率分布、格式偏好和不确定性，而不只复制 top1；代价是显存和计算更高，也可能复制 teacher 错误。当前 formal 代码计算的是 $D_{KL}(q_\beta \| p_S)$，beta=0 时为 teacher ∥ student，不是 student ∥ teacher。是否优于 top1/top-k 仍需要同数据、同 budget、同 scorer 的 matched ablation，当前不能直接宣称。

“为什么仓库里两份同名脚本，一份没有 KL？”

根目录 `train_opd_online_vl.py` 是旧版 teacher-rollout/top1-CE 实现；new-experts artifact manifest、launchers 和 run summaries 指向 `/data/msz/point/train_opd_online_vl.py`，对应本地 `point/` formal trainer。面试时我会按完整相对路径和 SHA 说明 lineage，不用同名文件末尾偷换实现版本。

“coldstart 和 OPD5000 是两种 KL 方向吗？”

不是。两者都用 formal $D_{KL}(q_\beta \| p_S)$；差异主要是 rollout distribution、parent checkpoint、max-new/prefix、veto schedule 与训练步数。coldstart 是 teacher rollout，OPD5000 是 student rollout。beta 大于 0 时 target 还混入 detached student logits，归零后才是纯 teacher-to-student forward KL。

“ckpt5000 是全局最佳吗？”

不是。point 和部分 box 指标偏向 5000，但整体 box IoU、某些 region/relation 和 text pool 的最佳 checkpoint 分散在其他 step。应按部署目标和 Pareto trade-off 选择，并报告 selection rationale。

“raw-holdout 说 0 泄漏，为什么仍能看到重复图片？”

0 泄漏指按 image hash 分组后，train 与 eval 的 hash 交集为 0；它不自动保证 eval 内每张图片只出现一次。同一图片可以对应多个 question、format 或 expert annotation，所以还要同时报告 unique-image 数、per-hash sample 数和 grouped denominator，不能把 sample-level 重复误说成跨 split 泄漏，也不能把 hash 隔离直接等同语义泛化。

“point strict=0 是模型不会定位吗？”

不一定。早期 prompt 与 strict parser 的 tag 格式不一致，可能把协议失败误判成几何能力失败；要同时报告 original strict、protocol-fixed strict 和 lenient geometry，并冻结 prompt/parser hash。

“OPD 提升了多少？”

只能按明确的同一 holdout、scorer、checkpoint 和 pool 报告组合路线结果。缺 SFT-only、single-teacher、no-synthetic 和 raw prediction 对照时，不能把差值单因果归因给 OPD。

“哪些是你做的？”

逐文件看 ownership ledger，区分 scaffold、个人设计、个人实现、协作、工具辅助、远端运行和 reviewer；`mousongzhe` 或其他账号的 commit 只能作为 lineage 线索，不能替代确认。

6. 练习边界

- 这是 model answer，不是候选人的真实 transcript。
- 使用时必须替换“我负责”的内容，并绑定具体脚本、commit、命令、评估或验收证据。
- 真实口述前保持 `PROJ-OPD-001..009` 全部为 `unseen`，不创建 `coaching_state.md`。

References

1. OPD Claim–Evidence Inventory。
2. OPD Ownership Ledger。
3. PROJ-OPD-001..009 Atomic Cards。

Part 45 / 49 · `interview_materials/vla/questions/proj_opd.html` · SHA-256 `C08737C19CE9C0FF78D7D95239E30F51D72A25D679B51262C161124DCF405B7B`

返回本卷目录

Canonical Markdown SHA-256：`c08737c19ce9c0ff78d7d95239e30f51d72a25d679b51262c161124dcf405b7b`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\proj_opd.md`

PROJ-OPD：OPD / VLM Grounding 项目原子题卡（Direct-Evidence Batch 1 + Ownership）

状态：`drafted / cards_8_of_8 / WARN` 更新日期：2026-07-17 当前题数：8 / 8 覆盖范围：`PROJ-OPD-001..009` 难度：L2 4 题 / L3 4 题 优先级：P0 7 题 / P1 1 题 练习状态：全部 `unseen` 前置证据：OPD Claim–Evidence Inventory；weights、raw predictions、统一 ablation 与 ownership ledger 未闭合，不得升级为强因果结论。

A. 问题定义与数据路由

▼ `L2` `P0` **PROJ-OPD-001 · OPD 这条路线要解决什么问题，为什么不是简单把五个 expert 的数据混在一起?**

面试意图：判断你是否能把多 expert、统一 student 和输出协议问题讲成一个可验证的训练设计，而不是只复述 OPD 名字。

30 秒回答：目标是让一个 8B student 同时保留 object/region/relation/point/text 能力，并根据任务输出正确格式。数据先训练五个 specialist，再用 route metadata 选择 teacher 做 coldstart 和在线 KL；直接混数据容易让能力和 `<box>/<point>/text` 格式互相污染。

机制展开（约 2 分钟）：五个 expert 分别偏 reasoning、object、region、spatial relation 和 RoboPoint。student 样本保留 `target_expert`、`candidate_experts`、`expected_format`、seen/unseen/hard、boundary 和 format-conflict，训练时按 target route 选 teacher。这样把数据来源、能力边界和输出契约显式化；但 route 是 metadata 驱动，不是已验证的 learned router。组合结果也不能归因于 OPD 单一机制，因为数据配比、specialist、coldstart 和格式对齐同时变化。

诊断 / 证据：检查五 expert 配额、student category/format counts、route counts 和训练 summary；再做 no-route、single-teacher、无 boundary/format-conflict 的对照。当前前四项有本地证据，消融未冻结。

项目挂钩：个人贡献应表述为多域数据合同、route metadata、训练阶段与评估协议的设计/验收；不要把统一 student 的静态指标说成真实机器人闭环。

追问树：为什么要 five expert? route 是模型学出来的吗? general 数据为什么占 25%? 格式冲突如何构造?

危险答案：说“OPD 自动学会了专家路由”，或说“只要把五个模型平均就能得到最强模型”。

来源：[1][2][3] **练习状态**：unseen

▼ `L2` `P0` **PROJ-OPD-002 · 五个 expert 的数据配额和 spatial-relation 重采样如何设计?**

面试意图：检查你是否能从脚本解释数据分工、长尾关系和 image-level split，而不是只背 expert 名称。

30 秒回答：每个 expert 的目标为 train 800k、eval 20k，并按 image hash 做 5% candidate eval。reasoning 偏 keepalive，RoboPoint 偏 point，object/region/relation 提高对应池比例；spatial expert 再平衡长尾关系。

机制展开（约 2 分钟）：脚本的 `EXPERT_TRAIN_PLAN` 为五个 expert 指定 source quota，`split_for_image` 用稳定 image/video key 做 train/eval candidate split，避免同图跨 split。`REL_BUCKET_WEIGHTS` 让 spatial relation 不被高频 `on/in` 淹没，并在输出 metadata 中保留 expert、source、source line、seed、relation bucket，便于追溯。它证明了数据路由规则存在，不证明每个 expert 已经独立 raw-holdout 最优；model asset table 还明确五个 200k expert 没有独立 10k 评估。

诊断 / 证据：复算 quota 与 eval scale、统计每个 relation bucket、检查 image/fingerprint overlap，并单独评估各 expert。当前配额和 split 是 `local-direct`，独立 expert eval 与因果比较仍缺。

项目挂钩：可讲“先解决数据分布和评估泄漏，再讨论 OPD”，把数据工程与模型训练分开。

追问树：为什么不用随机行 split? 关系 bucket 会不会改变真实分布? eval 20k 是真实 held-out 吗? 如何判断重采样过度?

危险答案：把 800k/20k 目标写成实际每个文件都精确达到，或说 relation rebalance 本身就证明关系泛化提升。

来源：[1][4][5]　练习状态：unseen

▼ L3 P0 **PROJ-OPD-003 · student 数据里的 seen/unseen/hard、boundary 和 format-conflict 分别解决什么问题?**

面试意图：判断你能否解释训练数据不是单一 iid 混合，而是带有泛化与接口边界的显式合同。

30 秒回答：每个 expert 过滤后前 100k 行标为 seen，domain 再拆 unseen/hard；general 防止模型"凡问必坐标"；boundary 覆盖 object/region/point/reasoning 易混边界；format-conflict 强化 tag、坐标范围、错误格式诱导和 prompt injection 下的输出契约。

机制展开（约 2 分钟）：最终 train summary 是 domain 599,905、general 250,000、boundary 99,998、format-conflict 49,997，共 999,900。每行写入 target/candidate expert、expected format、source line、fingerprint 与 `seen_by_expert_100k`，使 eval 可以按 exposure 分层。格式样本不是为了制造"更难看"的 prompt，而是防止 box/point/text 之间的输出漂移。严格审计还检查 JSON、坐标、图片、重复和 train/eval image overlap；这些是数据质量 gate，不是模型成功率。

诊断 / 证据：按 category、subtype、exposure 和 expected format 分层报 loss/format/geometry；做去掉 boundary 或 format-conflict 的对照。当前 metadata 与硬审计已冻结，分层模型结果和 ablation 未冻结。

项目挂钩：可把"问题发现→数据标签→评估切片"的闭环讲成个人设计价值，避免只强调训练步数。

追问树：为什么 train 少 100 行？seen 定义会不会泄漏？format-conflict 会不会导致过度拒答？如何构造 hard?

危险答案：说"100 万行全部是 unseen"，或把无 overlap 直接说成模型没有过拟合。

来源：[1][2][6]　练习状态：unseen

B. Coldstart 与在线 OPD

▼ L2 P0 **PROJ-OPD-004 · coldstart500 和正式 OPD5000 的训练链路有什么区别?**

面试意图：检查你是否分清 teacher rollout、student rollout、KL 监督和训练起点。

30 秒回答：coldstart 从 8B base 用 teacher rollout 做 500 步 full-vocab KL；OPD5000 从它继续，改为 student rollout 做 5000 步。formal `point/` trainer 计算 $D_{KL}(q_\beta \| p_S)$；根目录同名脚本是 legacy top1 CE。

机制展开（约 2 分钟）：formal trainer 支持 teacher/student rollout。coldstart 取 teacher response，prefix64，beta 百步退火；OPD5000 取 student response，prefix64、skip64k、每千步保存。两者让 route teacher 在同一 response 上 forward；target logits=`teacher_logits+beta*stopgrad(student_logits)`，beta=0 才是 $D_{KL}(p_T \| p_S)$。前者启动分布和格式，后者覆盖 student 实际 response；缺 matched raw eval，不能量化阶段贡献。

诊断 / 证据：检查两个 run summary 的 student path、rollout source、teacher load counts、route counts、loss/entropy/top-k overlap，并保留每阶段 checkpoint 与 raw eval。当前 summaries 与 checkpoint table 有，完整权重/raw log 没有。

项目挂钩：回答时强调训练阶段和 artifact lineage，而不是把"5000 steps"当作算法效果本身。

追问树：为什么不直接 online？为什么 full-vocab KL？student rollout 的分布偏移如何处理？veto beta 是什么?

危险答案：把 coldstart 的 teacher rollout 说成正式 student on-policy，或说 5000 steps 自动证明收敛。

来源：[1][7][8][17]　练习状态：unseen

▼ L3 P0 **PROJ-OPD-005 · route-based teacher selection 与 learned router 有什么区别？为什么用 full-vocabulary KL?**

面试意图：检查你能否把实现事实、训练目标和未验证假设分层。

30 秒回答：teacher selection 来自样本 metadata，不是 learned router。formal 代码计算 $D_{KL}(q_\beta \| p_S)$，beta=0 时是 teacher // student 的完整词表 KL；它比 legacy top1 CE 保留更多分布信息，但成本更高，也会复制 teacher 错误。

机制展开（约 2 分钟）：formal trainer 把 target/candidate expert 映射成 route；每行只激活一个 teacher，ZeRO-3 下各 rank 同步 route。student rollout 后，teacher 对同一 sequence forward，计算 $\sum_v q_\beta(v)[\log q_\beta(v) - \log p_S(v)]$，按 response token 数归一，并记录 entropy、overlap、loss 与 route count。route metadata 只是监督来源；learned router 还需模型、routing objective、部署输入和 matched ablation。

诊断 / 证据：做 target vs candidate route、single-teacher、random route、top-k vs full-vocab 的对照，并按格式/池统计错误。当前代码和 summary 支持实现路径，消融与 learned router 证据缺失。

项目挂钩：这是一个适合回答“工程实现与研究假设边界”的问题：可以准确讲当前做了什么，也能指出下一步研究空间。

追问树：candidate route 为什么存在？full-vocab KL 会不会过度复制 teacher 错误？为什么不做 MoE gate？如何防止 route imbalance？

危险答案：说“route id 就是 learned gating”，或说 full-vocab KL 一定比 top-k 更好且没有显存代价。

来源：[1][7][8][17] 练习状态：unseen

C. 评估、协议与真实性边界

▼ L3 P0 **PROJ-OPD-006 · OPD5000 的 checkpoint 应该怎么选，为什么不能只报 checkpoint-5000？**

面试意图：训练你根据多目标指标做 checkpoint selection，而不是把最后一个 checkpoint 当作全局最佳。

30 秒回答：ckpt3000 的整体 box IoU 最高 0.4761，ckpt5000 的 Box Acc@0.5=0.5238、Point Hit@50/100=0.7493/0.8729，ckpt4000 的 text exact=0.8784。按 pool 最佳点也分散，所以 ckpt5000 只能作为综合默认候选，不能说所有指标最佳。

机制展开（约 2 分钟）：10k holdout 按 box 6100、point 1400、text 2500 分开计算；ckpt3000 的 VG region/relation 最好，semantic-nav box 在 ckpt2000 最好，VG object 在 ckpt1000 最好，keepalive text 在 ckpt4000 最好，point 与多个 box pool 在 ckpt5000 最好。选择取决于部署目标：若强调统一多格式和 point，5000 合理；若强调整体 IoU 或特定关系池，应保留 3000。当前表格是同一新版 OPD5000 eval line，但没有 raw predictions 和完整 checkpoint bundle。

诊断 / 证据：冻结 pool/format denominator、scorer 版本、checkpoint parent、每样本 prediction 与 selection rationale；报告 Pareto frontier 而不是单一排名。当前 by-format/by-pool 表已冻结，raw predictions/lineage 仍缺。

项目挂钩：可以把 checkpoint selection 讲成“指标与产品目标对齐”的工程决策，而不是追求最后一步数字。

追问树：为什么 ckpt3000 IoU 更高但 ckpt5000 point 更好？如何避免按 holdout 过拟合选点？最终部署选哪个？

危险答案：说“5000 一定收敛且全面最好”，或把不同 pool 的 best 拼成一个虚构 checkpoint。

来源：[1][9][10] 练习状态：unseen

▼ L2 P0 **PROJ-OPD-007 · raw-holdout 的 0 泄漏结论是怎样得到的，重复图片为什么仍然存在？**

面试意图：确认你理解 source-line、fingerprint、图片和样本重复是不同层级的 leakage gate。

30 秒回答：10k holdout 阻断了五 expert 前 100k、OPD train 和训练图片集合；审计显示 fingerprint、source-line、image+prompt+answer 重复均为 0，训练图片泄漏也为 0。唯一图片 8,251，不是 10k，因为同一张 VG/RefCOCO 图片可以有多个合法标注。

机制展开（约 2 分钟）：evalset summary 记录每个 pool quota 和 blocklist，dedupe audit 记录 0 duplicate fingerprints、0 duplicate source lines、0 duplicate prompt-answer-image keys、0 image_seen_in_train rows。另有 1,168 个重复图片路径、2,917 行落在重复图片上，这是同图多 object/region/relation annotation，不等于同一监督样本。面试应说“样本级和训练图片级泄漏为零”，不要说“评估图片完全唯一”。

诊断 / 证据：复算 source-line、normalized fingerprint、image+prompt+answer key、train image set，并按 pool 输出重复图片分布；再检查近重复图或同源视频是否需要额外 block。当前三类严格重复和训练图片 gate 有本地审计，近重复/分布代表性仍可继续加强。

项目挂钩：这是体现评估严谨度的高价值回答：先证明 holdout 干净，再讨论模型数字。

追问树：为什么 image hash split 还要 blocklist？同一图片多标注会不会让 n 虚高？如何处理 near-duplicate？

危险答案：说“unique images=10,000”，或把重复图片路径直接等同于数据泄漏。

来源：[1][11][12] 练习状态：unseen

▼ L3 P1 **PROJ-OPD-008 · point strict=0 的结果如何解释？OPD 的提升能否归因于 OPD？**

面试意图：这是项目真实性与协议意识的高压题，要求同时处理 parser mismatch、可比性和个人 ownership。

30 秒回答：早期 point prompt 写 `[(x,y)]`，strict parser 却要求 `<point>[[x,y],...]</point>`，所以 strict=0 不等于不会定位。修正 prompt 后 Qwen-122B Hit@100=0.8200；OPD ckpt5000 原 strict=0.8729，但缺消融，不能作单因果归因。

机制展开（约 2 分钟）：point 需要分三种口径：original strict 看部署接口是否遵守 tag；protocol-fixed strict 看明确提示下的协议与几何能力；lenient geometry 兼容裸坐标，仅作历史能力估计。当前 OPD 训练期已对齐 `<point>`，因此原 strict 可读；大模型必须用 protocol-fixed strict 才公平。因果上，旧版 OPD final 与新版 OPD5000 也不是同一 run；缺 SFT-only、single-teacher、no-synthetic、synthetic-ratio 和 raw prediction bundle。ownership 方面，Git author 只是提交署名，不能替代逐文件 ledger。

诊断 / 证据：冻结 prompt hash、parser version、format/geometry 分项，做同 suite base→SFT→coldstart→OPD paired rerun；补逐文件 personal/collaboration/tool-assisted ledger。当前 protocol correction 和组合指标有，单因素与 ownership 仍 pending。

项目挂钩：推荐把“主动指出评估 bug 与因果缺口”作为项目结尾的可信度防线，比把所有结果都包装成 uplift 更有说服力。

追问树：为什么 OPD strict 可以直接比而 122B 不行？如果只看 box，谁更强？下一步最重要的 ablation 是什么？哪些代码是你亲手完成？

危险答案：说“base/instruct/大模型不会 point”，或说“OPD 比 base 高 11.5 个点，所以 uplift 全来自 OPD”。

来源：[1][13][14]　练习状态：unseen

D. Ownership 与可抗追问边界

▼ `L3` `P0` **PROJ-OPD-009 · OPD 哪些部分能说是你做的？如何处理 `msz136`、`mousongzhe`、Codex 与远端 run 的边界？**

面试意图：判断你是否能把 Git 署名、代码存在、设计决策、运行验收和 checkpoint/result ownership 分开，而不是把一次性快照认领成全部个人工作。

30 秒回答：我把数据、训练、评估、结果四条链分开。源码和报告可核验，但初始 scaffold 是 `msz136`，后续是 `mousongzhe` 提交，OPD5000 权重/raw predictions 未同步；所以 Git author 或 summary 不能替代逐文件个人 ownership。

机制展开（约 2 分钟）：先绑定 artifact：数据 builder/audit/evaluator 在 `3ef636b`，manifest 指向 `point/train_opd_online_vl.py`，不能用根目录副本替代。再区分设计者、提交者、远端运行者、报告生成者和 reviewer。最后按证据降级：报告支持曲线与 protocol，manifest 排除权重、optimizer、raw holdout 和 predictions；强回答要能解释一个函数并指出 diff、load smoke、raw bundle、ablation 与工具边界仍缺。

诊断 / 证据：使用 OPD Ownership Ledger 的 13 行矩阵；检查 `git log --follow`、blob/SHA、manifest synced scope、run summary、checkpoint table 和 candidate confirmation。没有真实口述前保持 `candidate_confirmation_pending / unseen`，不创建 coaching state。

项目挂钩：这是 OPD 项目最重要的真实性题。它允许你讲清真实工程价值，同时避免把 `mousongzhe` 的 commit、`msz136` 的 scaffold、远端 summary 和自己的个人贡献混成一句“全是我做的”。

追问树：哪一个函数是你亲手改的？为什么 `point/` 和根目录 trainer 不同？谁在 remote 运行？checkpoint 能 load 吗？Codex/协作者做了什么？如果没有 ablation，为什么不能说 OPD uplift？

危险答案：说“Git author 是我，所以所有代码和指标都是我的”，或把“报告里有数字”说成“我拥有模型结果”。

来源：[1][15][16]　练习状态：unseen

参考来源

1. OPD / VLM Grounding Claim–Evidence Inventory：五 expert、student route、训练 lineage、raw-holdout、point protocol 和 ownership 边界。
2. `C:\Users\mousongzhe\muxi\build_expert_mixes_seed0_v2.py`：expert quota、image-hash split、relation bucket 与 metadata。
3. `C:\Users\mousongzhe\muxi\build_opd_student_v1.py`：student category、seen/unseen/hard、boundary/format-conflict 和 fingerprint 字段。
4. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\model_assets_eval_table.md`：expert 训练摘要、checkpoint 资产边界和 pool readout。
5. `C:\Users\mousongzhe\muxi\report.md`：数据构造、严格审计、point protocol 修正与历史 OPD 说明。
6. `C:\Users\mousongzhe\muxi\audit_opd_student_v1.py`：JSON、长度、坐标、图片、重复与 overlap 审计逻辑。
7. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\remote_model_summaries\coldstart500_opd_online_run_summary.json`：coldstart rollout 与 full-vocab KL summary。

8. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\remote_model_summaries\opd5000_opd_online_run_summary.json` 与 `C:\Users\mousongzhe\muxi\point\train_opd_online_vl.py`：student rollout、route、skip、checkpoint 和 full-vocab KL；根目录同名脚本为 legacy top1 CE。
9. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\opd5000_ckpt_eval_by_format_table.md`：ckpt1000–5000 的 format-level metrics。
10. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\opd5000_ckpt_eval_by_pool_table.md`：八个 eval pool 的 checkpoint selection。
11. `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\evalset_summary.json`：10k holdout quota、blocklist 和格式分布。
12. `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\dedupe_audit.json`：重复与训练图片泄漏审计。
13. `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\raw_holdout_eval_8models_report.md`：旧版 OPD final 与 point protocol context。
14. `C:\Users\mousongzhe\muxi\report\openai_base64_point_protocol_qwen122_qwen35_20260603\comparison_summary_table.md`：protocol-fixed Qwen-122B/Qwen3-35B-VL point 结果。
15. OPD Ownership Ledger：Git identity、point/root trainer 差异、13 行 ownership matrix 与 C3 gate。
16. `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\artifact_manifest.json`：OPD5000 本地同步范围与明确排除的权重、optimizer、raw predictions。
17. OPD Training Objective Lineage Gate：legacy/formal trainer、rollout source、KL 方向、veto target 与 run-summary lineage。

Part 46 / 49 · `interview_materials/vla/evidence/projects/opd_claim_evidence_inventory.html` · SHA-256 `7FA495742B78C9FA4D56CDD9B46C9FB41FB4AB07B5E17A1202B8E860693920C8`

返回本卷目录

Canonical Markdown SHA-256: `7fa495742b78c9fa4d56cdd9b46c9fb41fb4ab07b5e17a1202b8e860693920c8`；生成自：`interview_materials\vla\evidence\projects\opd_claim_evidence_inventory.md`

OPD / VLM Grounding Claim–Evidence Inventory（OPD-01）

状态: `drafted / local_direct_audit / ownership_audited / WARN` 审查日期: 2026-07-19 证据截止: 2026-07-19 项目范围: 五 expert 数据混合、OPD student route metadata、coldstart、online student-rollout OPD、raw-holdout box/point/text 评估 证据仓库: `C:\Users\mousongzhe\muxi`（只读审计；本轮未修改该仓库） 仓库 HEAD: `756e27e16adcfb3ccbfe98c2920dda42095b127f`

结论先行

本轮可以把 OPD 的数据路由、五教师训练机制、OPD5000 checkpoint 曲线和 raw-holdout 去重协议从项目叙述提升到 `local-direct` 或 `report-supported`。`build_expert_mixes_seed0_v2.py` 明确了 800,000 train / 20,000 eval、image-hash split、五个 expert 和 spatial-relation 重采样；`build_opd_student_v1.py` 与 `audit_opd_student_v1.py` 明确了 999,900 条最终 train、46,599 条 eval、seen/unseen/hard、boundary/format-conflict route metadata 与硬校验；formal `point/train_opd_online_vl.py` 和 run summary 明确了五教师 `preloaded_zero3`、teacher selection、coldstart teacher-rollout/full-vocab KL 与 OPD5000 student-rollout/full-vocab KL。根目录同名脚本是 legacy teacher-rollout/top1-CE 路线，不能作为 formal KL claim 的代码锚点；详见 Training Objective Lineage Gate。

OPD5000 的结果必须绑定到具体 checkpoint 和子集：在 10,000 条 raw-holdout 上，checkpoint-3000 的整体 box IoU 为 0.4761，checkpoint-5000 的 Box Acc@0.5 为 0.5238、Point Hit@50/100 为 0.7493/0.8729，checkpoint-4000 的 keepalive text exact 为 0.8784。不同 pool 的最佳 checkpoint 不一致，不能把 checkpoint-5000 说成所有指标的最佳。

当前 verdict 保持 `WARN`。本地没有模型权重、optimizer state、完整 raw predictions 或可复算的远端 checkpoint bundle；`OPD final`（旧版 8-model 主评估）与 `OPD5000 checkpoint-5000`（新版 new-experts 曲线）不是同一条 run 记录；没有 SFT-only、single-teacher、no-synthetic 或 synthetic-ratio ablation。因此可以讲“组合路线及结果”，不能把提升单因果归因于 OPD，也不能把 Git author 自动等同于个人独立 ownership。逐文件 ownership 已在 OPD Ownership Ledger 建立，但 `msz136` 初始 scaffold、`mousongzhe` 后续提交、远端运行者、工具/协作边界仍需候选人确认。

1. 证据标签与面试门槛

标签	含义	OPD 例子
`local-direct`	当前工作区可直接读取的源码、配置、manifest、summary 或 committed table	五 expert 配额、route metadata、run summary、指标表
`local-inference`	由相邻本地证据推断，未被单独对照实验直接证明	route specialization 对泛化的因果贡献
`report-supported`	committed report 有结论，但底层远端权重、raw log 或 predictions 未同步	远端模型路径、旧版 OPD final 的完整 run 描述
`protocol-limited`	数字存在，但 scorer、样本量、checkpoint 或输出协议不能直接横比	original point strict 与 protocol-fixed strict
`ownership-partial`	Git author 或项目页支持参与线索，但没有逐文件 ownership ledger	相关 commit author 为 `mousongzhe`
`unverified`	缺少可审计 artifact，或无法复算	真实机器人闭环成功率、单因素 OPD uplift

面试回答使用四级门槛：C0=知道概念；C1=有本地源码/配置；C2=有设计取舍、失败样例或对照；C3=有 immutable run、checkpoint、完整 scorer、raw prediction 与 ownership。当前 OPD 的训练机制达到 C1/C2，强结果多为 C2，尚未形成完整 C3 bundle。

2. Claim reconciliation

ID	当前 claim	证据	Verdict	面试处理
`OPD-CL-001`	五个 expert 分工覆盖 reasoning、object、region、spatial relation、RoboPoint	`build_expert_mixes_seed0_v2.py` 的 `EXPERT_TRAIN_PLAN`、`REL_BUCKET_WEIGHTS`	`local-direct`	可解释数据分工；不要说五个 expert 已分别在 raw-holdout 独立最优，因独立 expert eval 未冻结
`OPD-CL-002`	expert mix 为 800k train / 20k eval，按 image hash 做 5% candidate eval	脚本常量与 split 函数	`local-direct`	可讲 image-level split 和 source/expert metadata

ID	当前 claim	证据	Verdict	面试处理
`OPD-CL-003`	OPD student 数据按 domain/general/boundary/format-conflict 构造并记录 route	`build_opd_student_v1.py`、`report.md`、最终 summary	`local-direct`	可讲 60/25/10/5 设计、seen 100k 边界与格式冲突；要说明最终 train 为 999,900
`OPD-CL-004`	train/eval 完成 JSON、坐标、图片、fingerprint 与 overlap 审计	`audit_opd_student_v1.py`、`report.md`	`local-direct / report-supported`	可报无坏 JSON、无 train/eval image/fingerprint overlap；不要把它当作模型质量证明
`OPD-CL-005`	coldstart 是五教师 teacher-rollout + full-vocab KL，约 500 steps	`point/train_opd_online_v1.py`、coldstart launcher/summary、objective lineage gate	`local-direct / report-supported`	可解释为什么先让 student 贴近 teacher 分布；未提供独立 coldstart ablation；veto beta>0 时 target 不是纯 teacher
`OPD-CL-006`	正式 OPD 是 student rollout、同一 response 上 teacher forward、route-based full-vocab $D_{KL}(q_\beta \| p_S)$，5000 steps	`point/train_opd_online_v1.py`、OPD5000 launcher/summary、objective lineage gate	`local-direct`	可画训练链路并说明 beta=0 才是 teacher // student；需绑定 max_new_tokens=64、prefix=64、skip=64,000、ZeRO-3 配置
`OPD-CL-007`	OPD5000 ckpt5000 在 point/box/text 上取得 0.7493/0.8729、0.5238、0.8724	by-format table、by-pool table、run summary	`local-direct / protocol-bounded`	绑定 10k raw-holdout、格式子集 n=1400/6100/2500；不能说所有域最佳
`OPD-CL-008`	不同任务池的最佳 checkpoint 不一致	by-pool table	`local-direct`	semantic-nav=ckpt2000、VG region/relation=ckpt3000、keepalive=ckpt4000、point=ckpt5000；用来说明多目标 trade-off
`OPD-CL-009`	raw-holdout 10k 无训练图片泄漏、fingerprint/source-line/prompt-answer-image 重复	`evalset_summary.json`、`dedupe_audit.json`	`local-direct`	可讲评估集构造和 leakage gate；重复图片路径不等价于重复样本
`OPD-CL-010`	旧版 `OPD final` 为 8-model 主评估中的综合结果	`raw_holdout_eval_8models_report.md`	`report-supported / run-conflict`	与 OPD5000 ckpt5000 分开报；旧版综合表的 point strict 受协议问题影响
`OPD-CL-011`	point strict=0 的 base/instruct/大模型不能直接解释为不会 point grounding	`report.md` protocol-fixed section、point protocol comparison	`protocol-limited`	同时报告 original strict、protocol-fixed strict、lenient geometry；避免把 parser failure 说成能力不存在
`OPD-CL-012`	OPD 单独造成全部提升，或所有实现均为个人独立完成	无完整 ablation；已有 ownership ledger，但候选人逐项确认未完成	`unverified / ownership-partial`	只说“我负责/参与了数据、路由、训练与评估设计的哪些部分”，不要过度承诺因果或 ownership

3. Source inventory（输入冻结）

Source	类型	SHA-256（当前工作区）	能支持	不能支持
`build_expert_mixes_seed0_v2.py`	local evidence	`19D5EF9E25D8664EA983880B79373098A77D19D5DA11D5B2AD17BEE6195C2F2C`	五 expert 配额、hash split、relation buckets、source metadata	实际训练是否按脚本完成、个人 ownership
`build_opd_student_v1.py`	local evidence	`44BCC42DE40249363DAE908314DB79BF658AAE4A5DA008732CD724389D5CD3BE`	target/candidate route、seen/unseen/hard、格式冲突 prompt、gold/fingerprint 字段	模型是否学会 route、最终远端文件是否与当前脚本完全同版
`audit_opd_student_v1.py`	local evidence	`B38F92C2FE681C250D3BA61C35299935501D014BA8B34965C7ED8D529A92D6BB`	JSON、长度、坐标、图片、重复与 overlap 审计逻辑	未执行的环境、权重或训练质量
`train_opd_online_v1.py`（root legacy）	local evidence	`BF9120B5D88180EA4C4725C6F4934913F35FDD36ECB4BC8E6B3B089FB964D8D9`	teacher rollout、teacher top1 hard labels、token CE、legacy mode string	coldstart/OPD5000 的 full-vocab KL 或 student-rollout formal claim
`point/train_opd_online_v1.py`（formal）	local evidence	`7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA2863336`	route teacher、teacher/student rollout switch、full-vocab $D_{KL}(q_\beta \| p_S)$、veto/prefix/entropy/top-k diagnostics	远端 checkpoint binary、optimizer state、raw prediction 与因果 uplift
`REPORT.md`（Windows 不区分大小写时显示为 `report.md`）	committed report	`0EC1E16D1E15AEDC918B910775E7BFD7C769D9715DFBBB9127C09E23F44A4170`	数据快照、point protocol 修正、OPD 叙事和历史 run 说明	raw predictions、远端权重、可复算 checkpoint lineage
`report/opd5000_newexperts_20260601/artifact_manifest.json`	local manifest	`035764A0461DEE5942AB25AF1BDFCD2514BD69AAC473B2DE8E43F3A7EC86021A`	synced scope、remote root、final model paths、排除项	本地可加载模型权重（manifest 明确排除）
`report/opd5000_newexperts_20260601/model_assets_eval_table.md`	local table	`5BA42BA56EFCD586A770D258E8536390942AB249606CA4D4AD88F70FB3BEBE45`	expert 训练摘要、evaluated checkpoints、指标口径	五 expert 独立 raw-holdout 结果
`report/opd5000_newexperts_20260601/opd5000_ckpt_eval_by_format_table.md`	local table	`7D797EE1269E082AEEE6AD851EB341588CFCF3B28C939BF870307C01004B9934`	ckpt1000–5000 的 box/point/text 子集指标	跨 run 或跨 scorer 因果比较

Source	类型	SHA-256（当前工作区）	能支持	不能支持
`report/opd5000_newexperts_20260601/opd5000_ckpt_eval_by_pool_table.md`	local table	`54CA01CAD342AC7F96C1CC0D622E102AB2C9E22E7514180213EF90A48043ADCA`	八个 pool 的 checkpoint 曲线与最佳点	单一“全局最佳”结论
`remote_model_summaries/opd5000_opd_online_run_summary.json`	report-supported summary	`2D670DE0707CD2C6A9D1D4C7EC109C86F0A3F8F87EDE5D69593670172315 47CE`	999,900 raw rows、936,192 expanded、skip 64,000、train loss、teacher/route counts、5000-step config	原始 train log、checkpoint binary、optimizer state
`remote_model_summaries/coldstart500_opd_online_run_summary.json`	report-supported summary	`90FD5A0A5983FEB5673D44CA087F597F0EBCACFD63DF1305C37C297F213D5FC0`	teacher rollout、500-step coldstart 参数与 summary	coldstart-only 对照的 raw eval
`report/raw_holdout_eval_8models_20260525/evalset_summary.json`	local eval manifest	`200F7A8B00CDD989D4E8D19B6CD3EB5B47B75B8779939443816970 2F8BBF6485`	10k quotas、训练阻断项、pool/format counts、8,251 unique images	模型 predictions 本身
`report/raw_holdout_eval_8models_20260525/dedupe_audit.json`	local audit	`2E102BB52B925ABA5AFE9E9111C98B3FA67F7032B266B1D08EBB1164A8FFB96D`	0 fingerprint/source-line/prompt-answer-image duplicates、0 train image leakage	任务是否足够代表真实部署
`report/raw_holdout_eval_8models_20260525/raw_holdout_eval_8models_report.md`	committed report	`B4520BCA9B27CBDC8E9280E734A915972CDBBEBB213E685F33FD831EA6E0E78E`	旧版 8-model 主评估、OPD final 表、评估协议与限制	与 OPD5000 ckpt5000 同 run 的证明

4. 五 expert 数据与路由设计

4.1 Expert mix

`build_expert_mixes_seed0_v2.py` 将 train 目标设为 800,000、eval 目标设为 20,000；以 image/video key 的稳定 hash 进入 5% candidate eval，再按 expert/source 配额抽样。五个 expert 的主分布是：

Expert	主数据域	设计意图
`general_reasoning_expert`	keepalive/general VQA	保留通用视觉推理与文本能力
`robopoint_expert`	RoboPoint point grounding	强化 `<point>` 坐标输出
`general_obj_expert`	RefCOCO/Flickr30K/VG object	强化 object box grounding
`region_expert`	VG region	强化 region description → box
`spatial_rel_expert`	VG relationship	强化关系条件 grounding

`spatial_rel_expert` 对 relation bucket 显式重采样：`on`、`in_inside`、`near_next_beside`、`under_below`、`above_over`、`front`、`behind`、`left_right`、`between`、`other_spatial`。每个输出样本保留 expert、split、source、source line、seed 和 relation bucket，这些字段支持后续追溯，但不证明专家已经学到可泛化的关系推理。

4.2 Student route metadata

`build_opd_student_v1.py` 将每个 expert 过滤后前 100,000 行标记为 `seen`，再构造：

- `domain`：约 600k，含 seen/unseen/hard 子类；
- `general`：约 250k，防止 student 退化为“凡问必坐标”；
- `boundary`：约 100k，覆盖 object/region/spatial/point/reasoning 边界；
- `format_conflict`：约 50k，覆盖 strict tag、错误格式诱导、prompt injection、坐标归一化。

最终 train summary 为 999,900 行：`box=555,537`、`text=293,050`、`point=151,313`。最终 eval 为 46,599 行，未为了凑满目标而用 train overlap 补齐。每行保留 `target_expert`、`candidate_experts`、`expected_format`、`sample_category/subtype`、`source_file/source_line`、`seen_by_expert_100k`、`fingerprint` 等字段。

这套 metadata 证明了“路由监督被写入数据”，不证明 student 在推理时必然选择正确 expert，也不证明 60/25/10/5 是全局最优比例。

5. Coldstart 与正式 OPD lineage

5.1 Coldstart-500

冷启动 summary 记录五教师 `preloaded_zero3`、teacher rollout、full-vocab distillation、`max_new_tokens=128`、`opd_prefix_loss_tokens=64`、veto beta 从 1 到 0（100 steps）等设置，student 起点为 `/data/msz/models/8b_base`。formal trainer 的

target 在 beta 大于 0 时由 `teacher_logits + beta * stopgrad(student_logits)` 构造，beta=0 后才退化为纯 $D_{KL}(p_T \| p_S)$。该阶段的可解释目的，是先让 student 获得稳定的 teacher 行为与输出协议，再进入 on-policy student rollout；但本地没有 coldstart-only raw-holdout 评估，不能把其增益单独量化。

5.2 OPD5000

正式 summary 记录：

- student 起点为 coldstart500 输出；五教师均以 `preloaded_zero3` 常驻并按 route 选择；
- `student` rollout 后，在同一 response 上执行 teacher forward；
- full-vocabulary $D_{KL}(q_\beta \| p_S)$，`max_new_tokens=64`，prefix loss token=64；beta归零后为 $D_{KL}(p_T \| p_S)$；
- `skip_expanded_samples=64,000`，保存 checkpoint 1000/2000/3000/4000/5000；
- 训练 summary 的 `train_loss=0.0306129573`，`last_opd_loss=0.0215270`，`last_opd_topk_overlap=0.7466`；
- 远端路径与 ZeRO-3 配置写入 manifest，但权重和完整 optimizer state 未同步到本地。

可用的面试链路：

```
clean pools
  -> five expert mixes (image-hash split)
  -> student rows with target/candidate route metadata
  -> coldstart teacher rollout + full-vocab KL
  -> student rollout -> routed teacher forward -> OPD KL
  -> raw-holdout by-format / by-pool evaluation
```

“teacher 选择”在当前实现中是 route metadata 驱动的 teacher selection，不是一个已经被单独验证的 learned router；回答时应明确这一点。

6. Raw-holdout 协议与结果

6.1 评估集 gate

raw-holdout v1 为 10,000 行，pool quota 为：box 6,100、point 1,400、text 2,500；唯一图片 8,251。`source-line`、fingerprint、image+prompt+answer 三类重复均为 0，训练图片泄漏为 0。由于多个 VG/RefCOCO 标注天然共享图片，评估集有 1,168 个重复图片路径、2,917 行位于重复图片上；这不等于重复样本，不能误写成“图片完全唯一”。

6.2 OPD5000 checkpoint 曲线

Checkpoint	Box IoU	Box Acc@0.5	Point Hit@50	Point Hit@100	Text exact
1000	0.4692	0.5139	0.7000	0.8357	0.8716
2000	0.4745	0.5192	0.7157	0.8550	0.8708
3000	**0.4761**	0.5233	0.7414	0.8629	0.8716
4000	0.4747	0.5207	0.7379	0.8650	**0.8784**
5000	0.4759	**0.5238**	**0.7493**	**0.8729**	0.8724

按 pool 的最佳点：semantic-nav box=2000；VG object=1000；VG region=3000；VG relationship=3000；point=5000；keepalive text=4000。`model_assets_eval_table.md` 的 readout 将 ckpt5000 作为综合默认候选，但同时保留 ckpt3000 在整体 box IoU / 高阈值 box 上的优势。

6.3 旧版 OPD final 与新版 OPD5000 分离

旧版 `raw_holdout_eval_8models_20260525` 主表记录的 `OPD final` 为：Box IoU 0.468、Box Acc@0.5 51.3%、Point Hit@100 84.1%、Text loose 87.4%、Bool acc 90.0%。它来自旧版 8-model 主评估与前一轮 checkpoint 曲线；新版 `opd5000_newexperts_20260601` 的 ckpt5000 是另一条 new-experts run，具有不同的训练 lineage、checkpoint 路径和后续评估表。

安全表述是“旧版 OPD final 在统一 8-model 矩阵中表现均衡；新版 OPD5000 ckpt5000 在 point/box/text 子集上有另一组结果”，而不是把两者拼成一条单调训练曲线。

7. Point protocol reconciliation

早期 raw-holdout point prompt 示例使用 `[(x1, y1), ...]`，而 gold 与 strict parser 要求 `<point>[[x1,y1],[x2,y2],...]</point>`。因此 base、instruct、Qwen-122B、Qwen3-35B-VL 的早期 strict=0 不能直接解释为几何能力=0。

模型 / 口径	Point format	Hit@50	Hit@100	MinDist	解释
8B base，original strict	0	0	0	—	主要是协议不匹配
8B base，original lenient	—	0.4450	0.5950	89.1	接受裸 `[(x,y)]` 的几何估计
8B instruct，original lenient	—	0.3686	0.5536	99.5	同上，非 strict deployment 指标
Qwen-122B，protocol-fixed strict	0.9986	0.6771	0.8200	69.1	修正 prompt 后的 strict
Qwen3-35B-VL，protocol-fixed strict	0.9857	0.6493	0.7871	75.9	修正 prompt 后的 strict
OPD5000 ckpt5000，original strict	1.0000	**0.7493**	**0.8729**	51.7	训练期已对齐 `<point>` 协议

面试中必须说清三种口径：

1. `original strict` 衡量部署接口是否遵守既定 tag/坐标协议；
2. `protocol-fixed strict` 衡量在提示明确时的协议与几何能力；
3. `lenient geometry` 是兼容历史裸坐标格式的能力估计，不能与 strict deployment score 混报。

8. 结果边界与失败域

当前证据支持：OPD5000 把多域 grounding、point 输出和 keepalive text 统一到一个 8B student，并在同一 raw-holdout 上保持稳定格式；但不支持以下强结论：

- 不能说 OPD 单独带来全部 uplift：数据配比、五 expert specialization、route metadata、coldstart、student rollout、format alignment 和 on-policy KL 同时变化；
- 不能把 aggregate comparison summary 中的跨格式数字当作格式子集数字，应优先使用 by-format / by-pool 表；
- 不能说五个 expert 已各自通过独立 raw-holdout 最优验证；model asset table 明确写着尚未独立评估；
- 不能说 point strict=0 的 base/instruct“大模型不会 point grounding”；必须给 protocol caveat；
- 不能说 raw-holdout 证明了真实机器人闭环成功率；它是静态视觉 grounding/VQA 协议评估。

9. Ablation、lineage 与 ownership 队列

P0

1. 把旧版 OPD final 与新版 OPD5000 的 base model、数据 manifest、训练脚本 SHA、checkpoint parent、scorer 版本和 run directory 收敛到单一 lineage manifest。
2. 同一 raw-holdout、同一 n、同一 scorer 复跑 base / SFT-only / coldstart-only / OPD5000，并保留 raw predictions 与逐样本 JSON。
3. 冻结 original strict、protocol-fixed strict、lenient geometry 三套 point scorer，记录 prompt hash。
4. 逐文件 ownership ledger 已建立；下一步补候选人确认，分别标记个人设计、个人实现、协作实现、历史 scaffold、工具辅助和仅读审计。

P1

1. single-teacher vs five-teacher；no-synthetic vs synthetic；synthetic ratio；route metadata ablation；skip-expanded policy ablation。
2. 按 pool 输出 calibration、错误样例和 checkpoint selection rationale。
3. 绑定远端 checkpoint 可加载 smoke、optimizer state 和完整训练 log。

10. 面试安全表述

推荐 60 秒版本：

> 我把多域视觉数据先按 image-hash 做 train/eval 隔离，再为 reasoning、object、region、spatial relation 和 RoboPoint 训练五个 expert。student 数据额外写入 target/candidate expert、seen/unseen、boundary 和 format-conflict metadata。训练上先做五教师 coldstart，再做 student rollout，在同一 response 上让 route 对应的 teacher 做 full-vocabulary KL；教师是 metadata route 选择，不是单独训练的 learned router。OPD5000 在 10k raw-holdout 上的 ckpt5000 达到 Box Acc@0.5 0.5238、Point Hit@100 0.8729、Text exact 0.8724，但不同 pool 的最佳 checkpoint 不同。point 指标我会区分 original strict、protocol-fixed strict 和 lenient geometry，避免把输出协议不一致误判成模型没有定

> 位能力。当前仍缺同协议 ablation、raw predictions 和逐文件 ownership ledger，所以我把结果称为组合路线证据，而不是 OPD 的单因果证明。

高压追问的诚实回答：

- “为什么不说 OPD 带来 11.5 个点提升？”——因为 base、数据、expert、coldstart、route 和 scorer/协议同时变化，当前没有同 suite 的单因素对照。
- “point strict=0 是不是模型不会？”——不是；早期 prompt 示例和 strict parser 不一致，protocol-fixed 后 Qwen-122B 达到 Hit@100=0.8200，OPD ckpt5000 原 strict 为 0.8729。
- “5000 是不是最佳 checkpoint？”——它是 point 和 Box Acc@0.5 的最佳点，但 box IoU、VG region/relation、keepalive text 的最佳点分散在 3000/2000/4000。
- “这些代码是不是你一个人写的？”——Git author 只能证明提交署名；我会按 ownership ledger 区分自己的设计、实现、协作和工具辅助范围。

11. References

- `C:\Users\mousongzhe\muxi\build_expert_mixes_seed0_v2.py`
- `C:\Users\mousongzhe\muxi\build_opd_student_v1.py`
- `C:\Users\mousongzhe\muxi\train_opd_online_vl.py`
- `C:\Users\mousongzhe\muxi\point\train_opd_online_vl.py`
- `C:\Users\mousongzhe\muxi\audit_opd_student_v1.py`
- `C:\Users\mousongzhe\muxi\REPORT.md`（Windows 下可能显示为 `report.md`）
- `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\opd_ownership_ledger.md`
- `C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\opd_training_objective_lineage_gate.md`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\artifact_manifest.json`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\model_assets_eval_table.md`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\opd5000_ckpt_eval_by_format_table.md`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\opd5000_ckpt_eval_by_pool_table.md`
- `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\evalset_summary.json`
- `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\dedupe_audit.json`
- `C:\Users\mousongzhe\muxi\report\raw_holdout_eval_8models_20260525\raw_holdout_eval_8models_report.md`

Part 47 / 49 · `interview_materials/vla/evidence/projects/opd_ownership_ledger.html` · SHA-256 `43280CCDA28125F284679D5612FAC2AE1621CC9A97285970F35CF4EBE703B7AD`

返回本卷目录

Canonical Markdown SHA-256：`43280ccda28125f284679d5612fac2ae1621cc9a97285970f35cf4ebe703b7ad`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\opd_ownership_ledger.md`

OPD / VLM Grounding Ownership Ledger：数据、路由、训练与结果归属

状态：`ownership_audited / coarse_git_supported / candidate_confirmation_pending / WARN` 审查日期：2026-07-19 证据截止：2026-07-19 项目范围：`C:\Users\mousongzhe\muxi` 的 OPD / VLM grounding 数据管线、五 expert、online OPD、raw-holdout 与报告资产。 结论先行：本地仓库能直接核验五 expert mix、student route metadata、coldstart500、student-rollout OPD5000、ZeRO 配置和 10k raw-holdout 表；但 Git 是“大快照 + 后续同步”形态，且初始 OPD scaffold 使用 `msz136` 身份，后续材料使用 `mousongzhe` 身份。它们不能自动合并为“候选人逐文件独立原创”。当前可说“候选账号提交了这些快照，项目材料自述负责数据/训练/评估设计”，不可说所有代码、远端权重和指标均由候选人独立完成或拥有。

1. 问题（Question）

面试官问“OPD 哪部分是你做的？”时，需要同时拆开：

1. OPD / 多教师蒸馏的公开背景与项目已有 scaffold；
2. 本地 commit 中确实存在的脚本、配置、manifest、summary 与评估表；
3. 候选人亲手设计、修改、运行、审查或验收的动作；
4. 协作者、历史账号、远端运行者和工具辅助的边界；
5. checkpoint、raw prediction、scorer 和最终数字的结果归属。

本页是 ownership ledger，不是把简历中的“设计/构建”重新认证为个人贡献。没有候选人逐项确认或过程证据的行保持 `role-confirmation-pending`。

2. Role Fit 与 JD 能力映射

岗位能力	本地可核证据	Ownership 强度	当前策略
VLM grounding 数据工程	五 expert 配额、image-hash split、source-line/fingerprint 与 route metadata	local-direct；个人决策待确认	可讲数据合同和泄漏 gate，不说全量数据由本人独立标注
多教师 / 在线蒸馏	`point/train_opd_online_v1.py`、coldstart/OPD run summary、full-vocab KL 字段	代码和 summary 强；运行者、设计者待确认	区分 metadata route 与 learned router，区分 teacher rollout 与 student rollout
分布式训练工程	`preloaded_zero3`、`zero3_opd_maca_gradclip5.json`、skip/prefix/save 配置	local-direct；远端运行闭环缺	可解释 ZeRO-3 合同，不声称性能或恢复收益
评估协议设计	10k raw-holdout、dedupe audit、strict/protocol-fixed/lenient point 口径	local-direct / protocol-bounded	先讲 scorer 和分母，再讲数字；不能把静态 holdout 写成机器人成功率
结果与因果归因	ckpt1000–5000 by-format/by-pool 表	report-supported / C3 未闭合	报告组合路线结果，不写 OPD 单因果 uplift 或“本人拥有 checkpoint”

3. Ownership 证据合同

3.1 五层 AND-gate

强主张“我负责并完成了 X”需要同时满足：

$$G(c) = I(c) \wedge A(c) \wedge D(c) \wedge V(c) \wedge O(c).$$

`I` 是文件/commit/run identity，`A` 是 artifact，`D` 是设计与修改过程，`V` 是运行或验收，`O` 是个人、协作者、工具和上游边界。Git author 只覆盖 identity 的一部分；代码存在、summary 数字和 report 也不能替代 decision、verification 或 ownership。

3.2 状态词汇

状态	含义	可用动词
`scaffold-snapshot`	初始项目快照中存在	“项目快照包含”
`git-coarse`	候选账号提交，但没有逐文件历史	“由候选账号提交快照”
`self-attested`	简历/项目页自述本人负责	“候选人自述负责”
`role-confirmed`	候选人逐项说明亲手/协作/工具边界	“我负责/我修改”
`process-supported`	有 diff、命令、失败复盘或 review 证据	“我设计并验证”
`result-owned`	checkpoint、协议、raw outputs、运行者和审查角色均闭合	“我完成并得到……”

4. Git 与 run lineage 冻结事实

4.1 仓库与身份

检查	结果	能证明	不能证明
当前 HEAD	`756e27e16adcfb3ccbfe98c2920dda42095b127f`，2026-07-07；author/committer `mousongzhe <mousongzhe@acerobotics.com>`	当前 snapshot 的提交身份与版本	每一行代码均由该人独立编写
初始 project code	`929e650fa1e6d745942eed1eab64dd8fb7833d2a`，author/committer `msz136 <msz136@users.noreply.github.com>`	仓库最早的代码快照身份	`msz136` 与候选人的现实身份是否相同
OPD scaffold	`9700efb301fef4253ef3a703d91ba61e69cae1a6`，45 files / 5,912 additions，`msz136`	`opd_project` 训练/eval scaffold 在该快照进入	候选人是否亲手写、移植或审查 scaffold
OPD data/eval batch	`3ef636bdbfcf9654bd47719acc0dffb3b9095b6f`，`mousongzhe`	五 expert builder、student builder、audit、raw-holdout evaluator/report 进入候选账号提交	远端运行者、个人设计过程和逐行 ownership
OPD5000 sync	`4c4bbfd42b1d7efee87de0d7eff78d4277fe38d4`，`mousongzhe`	`point/train_opd_online_vl.py`（1,015 行）及 coldstart/OPD5000 launch、manifest/table 进入	summary 中的远端权重和 raw predictions 属于谁

`point/train_opd_online_vl.py` 与根目录 `train_opd_online_vl.py` 不是同一 artifact：当前 SHA-256 分别为 `7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`（1,015 行）和 `BF9120B5D88180EA4C4725C6F4934913F35FDD36ECB4BC8E6B3B089FB964D8D9`（832 行），二者差异为 203 additions / 20 deletions。OPD5000 manifest 指向 `/data/msz/point/train_opd_online_vl.py`，所以结果叙事必须绑定 `point/` 版本和对应 run，而不是用根目录副本代替。

4.2 结果与资产边界

`report/opd5000_newexperts_20260601/artifact_manifest.json` 明确本地只同步 code/scripts/configs/logs/eval summaries，排除模型权重、optimizer `.pt`、raw holdout JSONL 和 `predictions.jsonl`。因此：

- `local-direct`：源码、配置、manifest、summary/table 的字节和 SHA；
- `report-supported`：远端 run 的参数、路径和指标摘要；
- `protocol-bounded`：10k raw-holdout 的 scorer、pool、checkpoint 和 n 可读，但不能复算所有样本；
- `result-owned`：当前未达到，不能把 `OPD final` 或 `OPD5000 ckpt5000` 写成候选人个人 checkpoint/result ownership。

5. Per-file / per-decision ownership matrix

ID	文件 / 决策	Local artifact 事实	Git / run 事实	当前 role verdict	当前安全口径	升级所需证据
`OPD-OWN-001`	`opd_project/` 初始训练/eval scaffold	配置、teacher forward、smoke、eval、REPORT 由 `9700efb` 一次加入	author 是 `msz136`；无更早逐文件历史	`scaffold-snapshot / identity-unverified`	“项目快照包含 OPD scaffold”	账号映射、来源/patch、候选人 review 或修改记录
`OPD-OWN-002`	`build_expert_mixes_seed0_v2.py` 五 expert mix	800k/20k、image-hash 5% candidate split、五域配额和 relation buckets；SHA `19D5…F2C2C`	580 行一次进入 `3ef636b`，候选账号提交	`local-direct / git-coarse / role-pending`	“本地实现了可追溯的五 expert 数据构造脚本”	设计决策记录、替代配比、运行 stdout、候选人逐项确认
`OPD-OWN-003`	`build_opd_student_v1.py` route metadata	999,900 train、46,599 eval、target/candidate/seen/unseen/hard/boundary/format-conflict 字段；SHA `44BC…D3BE`	700 行一次进入 `3ef636b`	`local-direct / git-coarse / role-pending`	“student 数据写入了 route 与格式合同”	选 60/25/10/5 的依据、现场解释函数、实际生成日志
`OPD-OWN-`	`audit_opd_student_v1.py` 数据审计	JSON、坐标、长度、图片/fingerprint/overlap 检查；SHA `B38F…D6BB`	431 行一次进入 `3ef636b`	`local-direct / verification-role-`	“本地有硬校验和 leakage audit”	谁运行、失败样例、修复 diff、审查结论

ID	文件 / 决策	Local artifact 事实	Git / run 事实	当前 role verdict	当前安全口径	升级所需证据
004				pending		
OPD-OWN-005	semantic-nav box 清洗与伪标签筛选	point/build_clean_grounding_pool.py、build_semantic_nav_eval_set.py；简历自述 13,879 boxes / 41,637 rows	两文件在 9df95cc 一次加入；底层标注和远端执行不在本地	self-attested / artifact-direct / role-pending	“项目材料自述先做 box 清洗和场景数据扩充”	原始输入、过滤阈值、人工/模型标注角色、held-out raw bundle
OPD-OWN-006	raw-holdout 构造与去重 gate	10k quota、0 fingerprint/source-line/prompt-answer-image duplicate、0 train-image leakage；evalset_summary SHA 200F…6485，dedupe_audit SHA 2E10…F96D	进入 3ef636b；评估样本和 predictions 未完整同步	process-supported / result-role-pending	“我能解释评估集阻断泄漏的规则”仅在候选人确认后使用	build command、冻结 prompt hash、逐样本 JSON、候选人运行/审查记录
OPD-OWN-007	point/train_opd_online_vl.py teacher route 与 full-vocab KL	1,015 行；target route、preloaded_zero3、teacher/student rollout、prefix loss、full-vocab support	9df 初始进入，4c4 增补；实际 manifest 指向 point/ SHA 7103…6336	local-direct / git-coarse / role-pending	“项目实现了 route-driven online OPD 训练路径”	亲手函数清单、before/after diff、训练命令、失败复盘与 review
OPD-OWN-008	coldstart500 launch 与 summary	teacher rollout、max_new_tokens=128、prefix=64、500 steps；summary SHA 90FD…D5FC0	launch/summary 在 4c4bbf；模型权重和 optimizer state 排除	report-supported / operator-pending	“报告记录五教师 coldstart500 配置”	remote log、checkpoint load smoke、运行者与候选人验收角色
OPD-OWN-009	OPD5000 launch、skip 和 checkpoint schedule	student rollout、skip=64,000、max_new_tokens=64、prefix=64、save 1000–5000；summary SHA 2D67…547CE	launch/summary 在 4c4bbf；manifest remote root /data/msz/point	report-supported / result-owner-blocked	“报告记录 OPD5000 run lineage 与 checkpoint 曲线”	immutable run manifest、权重/optimizer/RNG、raw predictions、operator/reviewer
OPD-OWN-010	ZeRO-3/MACA 与 teacher residency contract	point/configs/zero3_opd_maca_gradclip5.json SHA CA5C…655F；summary 写 preloaded_zero3	config 由 9df/3ef/4c4 逐步出现；无 matched memory/throughput A/B	engineering-choice-supported / decision-owner-pending	“采用了 ZeRO-3 与 resident teacher 约束”	为什么常驻、显存/吞吐对照、MACA traceback、候选人决策记录
OPD-OWN-011	raw evaluator 与 point protocol	eval_qwen_vl_raw_holdout.py、strict/protocol-fixed/lenient 口径；evaluator SHA 2957…206B	evaluator 在 3ef/4c4；早期 prompt/parser mismatch 已在报告修正	process-supported / scorer-owner-pending	“我把格式合规和几何能力分开审计”需候选人确认	scorer version/prompt hash、parser tests、逐样本 predictions、review history
OPD-OWN-012	by-format / by-pool checkpoint 表与综合选择	ckpt3000 IoU 最佳、ckpt5000 point/box Acc、ckpt4000 text 最佳；表 SHA 7D79…9934 / 54CA…ADCA	tables/model assets 在 4c4bbf；不同 pool 最佳点不一致	report-supported / selection-role-pending	“checkpoint 选择是多目标 trade-off，不是单一全局最佳”	selection freeze、未触碰 holdout 的选择集、候选人判断记录
OPD-OWN-013	结果归属与简历数字	REPORT.md SHA 0EC1…4170、manifest SHA 0357…21A；OPD final 与 OPD5000 分离	report/summary 是候选账号提交的文本快照；无本地 weights/raw predictions	report-supported / C3-blocked / candidate-confirmation-pending	“报告记录组合路线结果；我不把它升级为个人独立 checkpoint ownership 或单因果 uplift”	checkpoint bytes/load、同 suite ablation、raw scorer bundle、运行/分析/审查三方角色

6. Claim → ownership reconciliation

面试/简历 claim	当前证据合同	安全处理
“设计五域数据与 OPD 路由”	脚本和项目页支持设计形态；个人决策与过程记录未闭合	候选人确认前用“项目构建/本地实现”；确认后再用“我设计”
“训练了五个 expert 并完成 OPD5000”	summary/report 支持 run 参数和曲线；权重、optimizer、operator 缺	说“报告记录该 run”，不说“我独立训练并拥有 checkpoint”
“OPD 比 SFT 高 11.5pp”	旧版/新版 lineage、数据和 scorer 不完全同一；无 SFT-only / single-teacher ablation	禁止单因果归因；拆为组合路线和 protocol-bounded comparison
“point strict=0 说明 base 不会定位”	prompt/parser mismatch；lenient 与 protocol-fixed 结果相反	必须同时讲 original strict、protocol-fixed strict、lenient geometry
“10k raw-holdout 证明真实导航成功率”	静态 box/point/text，虽有 semantic-nav 字段但无 episode/action trace	只说静态 grounding/VQA holdout，不写 VLN/ObjectNav 或真机成功
“这些代码都是我写的”	候选账号 coarse commits + 初始 msz136 scaffold，没逐文件历史	用 per-file ledger；明确 candidate confirmation pending

7. Production Gap：抗追问最小补证据

Gap	面试官追问	最小补证据	当前表达
身份/快照边界	msz136 和 mousongzhe 是谁?	账号映射、贡献记录、pre-snapshot patch	保持 identity-unverified
trainer 版本	为什么根目录和 point/ 不同?	run manifest 绑定 script SHA、命令和代码 diff	只绑定 manifest 指向的 point/
checkpoint	权重能加载吗?	binary SHA、processor/config、load smoke	report-supported
raw result	0.5238/0.8729 如何复算?	raw predictions、scorer commit、逐样本 JSON	protocol-bounded
因果	OPD 单独贡献多少?	matched SFT-only/single-teacher/no-synthetic ablation	不做单因果归因
运行角色	谁在 remote 执行和验收?	shell history、run owner、reviewer、failure timeline	operator-pending

Gap	面试官追问	最小补证据	当前表达
数据质量	13,879 boxes 谁标注/筛选?	输入 manifest、过滤日志、抽检与标注角色	只说项目材料自述
工具协作	哪些由 AI/协作者生成?	contribution ledger、review notes、现场解释	不推断 Codex-primary

8. Question Tree 与回答分层

```
你个人做了什么?
  ├ 数据: 五 expert 配额、hash split、route metadata 的设计者是谁?
  ├ 训练: point/根目录脚本哪一版真的跑了? 为什么 full-vocab KL?
  ├ 评估: strict/parser mismatch 如何发现? holdout 如何阻断泄漏?
  ├ 结果: ckpt5000 为什么不是所有 pool 最好? 旧版 OPD final 是否同一 run?
  └ ownership: 哪些亲手、协作、工具辅助、只审查或只运行?
```

危险回答：Git author 是我，所以五个 expert、远端训练、checkpoint 和所有指标都是我独立完成的。

可过回答：我会把项目拆成数据、训练、评估和结果四条链。当前本地能直接核验五 expert mix、route metadata、`point/` 训练脚本、scorer 和报告表；但初始 scaffold 使用另一 Git 身份，OPD5000 权重/raw predictions 未同步，所以我只把候选账号提交的快照和项目自述称为参与证据，逐文件亲手/协作边界等候确认。

强回答（候选人确认并补证后才可用）：我能从一个具体函数说明输入、route、teacher forward、KL、保存和 scorer 的完整链路，并指出哪些决策由我做、哪些由协作者或工具实现；对于 `OPD final` 与 OPD5000，我会给出独立 manifest、checkpoint、prompt/scorer hash 和 raw predictions，不把组合提升包装成 OPD 单因果结果。

9. SP/SSP 证据就绪度

维度	分数（5）	依据
岗位相关性	4.5	grounding、蒸馏、分布式训练与评估均直接相关
机制深度	4.0	五 expert、route、coldstart、student rollout、full-vocab KL 和 protocol boundary 可解释
工程闭环	3.5	scripts/configs/reports 完整；weights/optimizer/RNG/load smoke 缺
指标可信度	2.5	raw-holdout table 与 leakage audit 可读；raw predictions/scorer bundle 未闭合
Ownership 清晰度	2.0	13-row ledger 已建立；初始身份、逐文件角色和运行者仍待确认
抗追问能力	3.0	机制与限制可讲；diff、checkpoint、因果和工具边界仍有断点
合计	**19.5 / 30**	这是证据就绪度，不是候选人能力评分；WARN 不变

10. 候选人确认表

下一次口述或材料核对时逐项回答“亲手 / 协作 / 工具辅助 / 只审查 / 只运行 / 不确定”：

1. `msz136` 与 `mousongzhe` 是否为同一实际贡献者？如果是，依据是什么？
2. 五 expert 配额、image-hash split 和 relation bucket 是谁决定的？有哪些被否决的配比？
3. `build_opd_student_v1.py` 中的 `seen/unseen/hard` 和 `format_conflict` 哪些字段由你设计或修改？
4. `point/train_opd_online_vl.py` 哪些函数由你亲手写、修改、review 或只运行？实际 run 是否就是该 SHA？
5. coldstart500 与 OPD5000 是谁在 remote 执行、监控和验收？你看过哪些原始日志？
6. point strict/protocol-fixed/lenient scorer 的 mismatch 是谁发现和修正的？
7. `OPD final`、OPD5000 ckpt5000 的 checkpoint、raw predictions 和结果表分别由谁生成、审查和保管？
8. 哪些数据清洗、伪标签、报告或脚本由协作者、历史代码或 AI 工具产生？你如何验证它们？

这些问题没有真实回答前，不创建 `coaching_state.md`，不把任一行升级为 `role-confirmed` 或 `result-owned`。

11. 分阶段补强计划

阶段	动作	完成标准	状态
半天	完成 8 项候选人 ownership drill；每个关键函数选一个现场解释点	13 行均有 role/source/verification 字段	待候选人确认
1 天	导出 `point/` run 的 script/config/data/scorer manifest，标记 `msz136` 与 `mousongzhe` 身份关系	OPD5000 lineage 不再依赖 path/summary 单点	待补做
3 天	补 checkpoint load smoke、raw predictions 和三套 point scorer	C3 结果链至少可复算一条 checkpoint	待补做

阶段	动作	完成标准	状态
1 周	固定同一 raw-holdout 做 SFT-only、single-teacher、no-synthetic/synthetic-ratio 对照	组合效果可拆为可审查的机制证据	待补做

12. Output Pack

12.1 简历 bullet 三档

- 当前保守版：参与 VLM grounding 数据清洗、五 expert 路由与 online OPD 训练/评估闭环；以 raw-holdout、point protocol 和 checkpoint 曲线拆分组合结果与证据边界。
- 候选确认后标准版：负责五域数据配比、route metadata、五教师 coldstart/OPD 训练 contract 及 strict holdout scorer 的设计与审查；具体文件按 ownership ledger 说明亲手与协作边界。
- 补齐 C3 后偏强版：在固定数据、scorer、checkpoint lineage 和 raw predictions 下，完成 SFT-only/single-teacher/OPD 对照并给出可复算的多格式结果；当前禁止提前使用。

12.2 两分钟叙事骨架

> 我先按 image-hash 做 train/eval 隔离，再把 reasoning、object、region、spatial relation 和 RoboPoint 组织成五个 expert，并在 student 行里保留 target/candidate route、seen/unseen、boundary 和 format-conflict。训练先用五教师 rollout 做 coldstart，再让 student rollout，由 route 对应 teacher 在同一 response 上做 full-vocabulary KL；这里的 route 是 metadata 驱动，不是已验证的 learned router。评估用 10k raw-holdout 按 box/point/text 分开，并区分 original strict、protocol-fixed strict 和 lenient geometry。OPD5000 ckpt5000 的 point/box/text 数字属于一条 report-supported new-experts run，不和旧版 OPD final 拼成一条曲线。我的具体个人贡献要按 ledger 中逐文件回答，当前不把 Git author、summary 或 report 当作独立 ownership 证明。

12.3 面试前 6 题

1. 为什么五 expert 不直接混合成一个 SFT 数据集?
2. `target_expert` route 和 learned router 的训练/部署差别是什么?
3. coldstart teacher rollout 与 OPD5000 student rollout 的数据流和 loss 有何不同?
4. 为什么 full-vocabulary KL、prefix=64、skip=64,000、`preloaded_zero3` 会同时影响工程合同?
5. point strict=0、protocol-fixed strict 和 lenient geometry 如何避免错误归因?
6. 请从 `point/train_opd_online_vl.py` 的一个函数开始，说明哪些是你亲手、协作、工具辅助、只审查或只运行的。

13. Ownership 审计伪代码

```
def accept_opd_claim(claim, ledger):
    if not ledger.identity_and_artifact(claim):
        return "reject_strong_claim"
    if not ledger.role_confirmed(claim):
        return "use_project_contains_or_candidate_attested"
    if claim.level >= "C2" and not ledger.process_and_verification(claim):
        return "downgrade_to_implemented_or_reviewed"
    if claim.level == "C3" and not ledger.checkpoint_protocol_raw_chain(claim):
        return "report_summary_only"
    return "claim_allowed_with_exact_scope"
```

这段伪代码是本仓库的真实性 gate，不是 OPD 官方实现。

参考文献（References）

1. OPD Claim–Evidence Inventory：五 expert、student route、训练 lineage、raw-holdout、point protocol 与当前 WARN 边界。
2. OPD Batch 1 项目题卡：8 张 direct-evidence-bounded unseen cards；本 ledger 新增 ownership 题卡。
3. Evidence Manifest：五项目 evidence 与 ownership 状态总索引。
4. VLA Knowledge Map：VOL 07 项目深挖与题目 ID 空间。
5. Local Git repository `C:\Users\mousongzhe\muxi`，HEAD `756e27e16adcfb3ccbfe98c2920dda42095b127f`；初始 scaffold `9700efb301fef4253ef3a703d91ba61e69cae1a6`；OPD data/eval `3ef636bdbfcf9654bd47719acc0dffb3b9095b6f`；OPD5000 sync `4c4bbfd42b1d7efee87de0d7eff78d4277fe38d4`。

6. `report/opd5000_newexperts_20260601/artifact_manifest.json`（SHA-256 `035764A0461DEE5942AB25AF1BDFCD2514BD69AAC473B2DE8E43F3A7EC86021A`）：明确排除 weights、optimizer、raw holdout 与 predictions。
7. Optimized Resume 与 OPD 项目面试页：当前候选人自述和投递口径，未经本 ledger 自动认证。

Part 48 / 49 · `interview_materials/vla/evidence/projects/opd_p0_bundle_audit.html` · SHA-256 `C4F5EAE766BA47EEECDA173548FC1E91D80CD99B466B60AC70A30BC137819B09`

返回本卷目录

Canonical Markdown SHA-256：`c4f5eae766ba47eeecda173548fc1e91d80cd99b466b60ac70a30bc137819b09`；生成自：`interview_materials\vla\evidence\projects\opd_p0_bundle_audit.md`

OPD / VLM Grounding Po Bundle Audit：Data Route / Distillation / Holdout

状态：`p0_audited / protocol-bounded / raw-prediction-missing / ownership-pending / WARN` 审计日期：2026-07-21 范围：只审计五 expert 数据构造、student route metadata、coldstart/online OPD summary、10k raw-holdout manifest/table 和 point scorer 记录；不把组合路线结果升级为单因果 OPD uplift。

1. 结论

OPD 已具备可定位的数据和训练机制证据：image-hash split、五 expert 配额、route metadata、coldstart teacher rollout、student-rollout/full-vocabulary KL、OPD5000 checkpoint 曲线、10k holdout quota 和 leakage audit 均有本地脚本/表格/summary。实现 lineage 已进一步拆开：根目录同名 trainer 是 legacy teacher-rollout/top1 CE，formal `point/` trainer 才支持 coldstart teacher rollout、OPD5000 student rollout 与 $D_{KL}(q_\beta \| p_S)$；详见 Training Objective Lineage Gate。结果可以绑定到具体 checkpoint、format/pool 子集和 protocol-limited scorer。

Po 仍未闭合：模型权重、optimizer/RNG、完整 raw predictions、统一 scorer manifest、旧版 OPD final 与新版 OPD5000 的单一 lineage、SFT-only/single-teacher/no-synthetic ablation、operator/reviewer 和逐文件 ownership 缺失。

```
P0 status = PARTIAL / PROTOCOL-BOUNDED
eligible causal claims = 0
eligible checkpoint/result ownership = 0
allowed = data route, teacher/student mechanism, leakage gate, bounded tables
forbidden = “OPD alone caused uplift”, “I own OPD5000 checkpoint”, “static holdout = robot success”
```

2. Po gate matrix

Po 字段	当前证据	状态	安全解释
data split / quota	800k/20k builder、image-hash split、五 expert、10k holdout quota	`PRESENT`	可讲数据合同与 leakage gate
route metadata	target/candidate expert、seen/unseen、boundary/format-conflict	`PRESENT`	metadata route，不称 learned router
teacher/student training	formal `point/` trainer、coldstart teacher rollout、OPD5000 student rollout、full-vocab $D_{KL}(q_\beta \| p_S)$、skip=64k	`PRESENT WITH WARNING`	可讲训练机制；legacy root top1 CE 必须分开，coldstart-only 增益未隔离
checkpoint curve	ckpt1000–5000 by-format/by-pool tables	`PARTIAL`	必须绑定 pool；不同指标最佳 checkpoint 不同
raw holdout	10k manifest、dedupe/leakage audit、aggregate tables	`PARTIAL`	raw predictions/per-sample JSON 缺
scorer/protocol	original strict、protocol-fixed strict、lenient point tables	`PARTIAL`	prompt/scorer hash 和统一版本仍缺
model lineage	OPD final 与 OPD5000 new-experts 分开报告	`CONFLICT OPEN`	不能拼成单调训练曲线
ablation	无 SFT-only、single-teacher、no-synthetic、ratio 对照	`MISSING`	不能归因 OPD 单因素
checkpoint/optimizer	artifact manifest 明确排除 weights/optimizer/raw predictions	`MISSING`	不称 checkpoint/result-owned
ownership	13-row ledger，`msz136` / `mousongzhe` scaffold 与后续同步分开	`PENDING`	Git author 不等于逐文件 ownership

3. 已核验的结果边界

- 10k holdout quota：box 6,100、point 1,400、text 2,500；unique images 8,251；三类 leakage fingerprint 为 0，但重复图片路径不等于重复样本；
- OPD5000 ckpt5000 的 Box Acc@0.5=`0.5238`、Point Hit@100=`0.8729`、Text exact=`0.8724`，必须带 pool/format/protocol；
- box IoU 最佳点在 ckpt3000，keepalive text 在 ckpt4000，不能称 ckpt5000 全局最佳；
- 旧版 OPD final 与新版 OPD5000 ckpt5000 是不同 run lineage；
- 静态 grounding/VQA holdout 不等于 VLN/ObjectNav 或机器人闭环成功。

4. 面试可用与禁用表述

可以说：

> “我把数据先按 image-hash 做隔离，再把 reasoning、object、region、spatial relation 和 RoboPoint 组织成五个 expert；student 里保留 route metadata，先做五教师 coldstart，再做 student rollout/full-vocabulary KL。评估时我把 10k raw-holdout 按 box/point/text 和 original/protocol-fixed/lenient scorer 分开，且不同 pool 的最佳 checkpoint 不同。由于权重、raw predictions、统一 scorer 和 matched ablation 未闭合，我把这些称为组合路线的 protocol-bounded evidence，不把它归因成 OPD 单独 uplift。”

不能说：

1. OPD 单独带来全部提升；
2. OPD5000 ckpt5000 是所有任务的全局最佳；
3. 报告数字是本人拥有的可加载 checkpoint 结果；
4. 10k static holdout 等于真实机器人/导航成功率；
5. `msz136` / `mousongzhe` Git author 自动证明所有文件由本人独立实现。

5. 关闭 Po 的最小动作

1. 将 OPD final、coldstart500、OPD5000 的 code/config/data/scorer/checkpoint parent SHA 收敛为 lineage manifest；
2. 同一 holdout、n 和 scorer 重跑 base/SFT-only/coldstart-only/OPD，并保留 raw predictions/per-sample JSON；
3. 固定三套 point scorer、prompt hash 和 parser tests；
4. 候选人回答 `OWN-OPD-001`，明确数据、训练、评估、运行和工具协作边界。

在这些字段闭合前，OPD 保持 `PARTIAL/WARN`，不改变 `PROJ-OPD-*` 或通用题卡 mastery。

References

1. OPD Claim–Evidence Inventory
2. OPD Ownership Ledger
3. 五项目 Po Artifact Intake Contract

Part 49 / 49 · `interview_materials/vla/evidence/projects/opd_training_objective_lineage_gate.html` · SHA-256 `5B9498966531981C21FF89980978BF16AA40D6FBD6D3F9C820FF2C70BFF7D1F4`

返回本卷目录

Canonical Markdown SHA-256：`5b9498966531981c21ff89980978bf16aa40d6fbd6d3f9c820ff2c70bff7d1f4`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\evidence\projects\opd_training_objective_lineage_gate.md`

OPD Training Objective Lineage Gate：top1 CE、Full-Vocabulary KL 与 Rollout Source

状态：`local_code_reconciled / run_summary_bounded / checkpoint_unverified / WARN` 审计日期：2026-07-22 审计对象：OPD legacy 根目录 trainer、formal `point/` trainer、coldstart-500 与 OPD5000 launcher/run summary 核心问题：当前口述中的“student rollout + full-vocabulary KL”究竟由哪份实现支持？KL 的方向、veto target、token mask 和 run lineage是什么？

1. 结论先行

本地存在两份同名但目标不同的 trainer，不能用路径末尾的 `train_opd_online_vl.py` 把它们合并：

1. `C:\Users\mousongzhe\muxi\train_opd_online_vl.py` 是 **legacy teacher rollout + teacher top1 hard-label CE**。teacher 先生成 response，再在该 response 上 forward，取 `argmax` 作为 token label，student 用逐 token NLL/CE 学习；run summary 的 `opd_mode` 固定写成 `five_expert_online_teacher_rollout_top1`。
2. `C:\Users\mousongzhe\muxi\point\train_opd_online_vl.py` 才是当前 formal **configurable teacher/student rollout + full-vocabulary forward KL** 实现。`--opd-rollout-source` 默认 `student`，也支持 `teacher`；teacher 在同一 response 上 forward，loss 使用完整词表分布。
3. coldstart-500 launcher 显式传 `--opd-rollout-source teacher`，run summary 记录 `teacher rollout + full_vocab KL + veto/prefix`；OPD5000 launcher 显式传 `student`，summary 记录 `student rollout + full_vocab KL + veto/prefix`。因此 coldstart 不是 student rollout，OPD5000 也不是 legacy top1 CE。
4. 当 `veto_beta=0` 时，formal loss 是 $D_{KL}(p_T \| p_S)$，即 teacher/target 分布对 student 分布的 forward KL；veto 期内 target 不是纯 teacher，而是由 `teacher_logits + beta * stopgrad(student_logits)` 构造的分布。口述不能把整个 500-step warmup 都简写成纯 teacher-to-student KL。
5. artifact manifest 明确本地同步范围排除了 model weights、optimizer `.pt`、raw holdout JSONL 和 predictions JSONL；因此本页能闭合 **objective/code/run-summary lineage**，不能闭合 checkpoint binary、训练完成的独立重放或因果 uplift。

2. 两份 trainer 的实现差异

维度	Legacy root trainer	Formal `point/` trainer
路径	`muxi/train_opd_online_vl.py`	`muxi/point/train_opd_online_vl.py`
SHA-256	`BF9120B5D88180EA4C4725C6F4934913F35FDD36ECB4BC8E6B3B089FB964D8D9`	`7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`
rollout source	teacher 固定	`student` / `teacher` 参数化，默认 student
teacher target	`argmax` token ID	response token 位置的完整 vocab logits
student loss	`F.nll_loss(log_softmax(student_logits), teacher_top1)`	target distribution 与 student distribution 的 full-vocabulary KL
token support	teacher top1 label	完整词表；run summary 记录 `opd_distill_support=full_vocab`
prefix	response 全段	`opd_prefix_loss_tokens` 可截断 loss response
veto	无	`target_logits = teacher_logits + beta * stopgrad(student_logits)`
summary mode	`five_expert_online_teacher_rollout_top1`	`five_expert_online_{rollout_source}_rollout_teacher_full_vocab_kl_veto_prefix`
可映射 run	早期 off-policy/top1 路线	coldstart-500 与 OPD5000 new-experts 路线

2.1 Legacy top1 CE

legacy 实现先执行：

```
teacher.generate(prompt)
  -> teacher(response) logits
  -> argmax over vocabulary at every response position
```

```
    -> hard token labels
    -> student(response) logits
    -> token NLL / CE
```

它只要求 student 提高 teacher top1 token 的概率，没有直接约束 teacher 对其余词表 token 的相对置信度。该路线可以称为 online teacher rollout + top1 distillation，不应称为 full-vocabulary KL，也不应称为 student rollout。

2.2 Formal full-vocabulary KL

formal trainer 根据 `opd_rollout_source` 由 teacher 或 student 生成 response；之后 teacher 与 student 都在同一截断 response 上 forward。令 response token 位置 t 的 student logits 为 z_t^S，teacher logits 为 z_t^T，veto 系数为 β：

$$\log p_S(v \mid x, y_{<t}) = \operatorname{logsoftmax}(z_t^S)_v$$

$$\log q_\beta(v \mid x, y_{<t}) = \operatorname{logsoftmax}\left(z_t^T + \beta \operatorname{stopgrad}(z_t^S)\right)_v$$

$$\mathcal{L}_t = D_{KL}(q_{\beta,t} \| p_{S,t}) = \sum_v q_{\beta,t}(v) \left[\log q_{\beta,t}(v) - \log p_{S,t}(v)\right]$$

当 $\beta = 0$ 时，$q_\beta = p_T$，得到 $D_{KL}(p_T \| p_S)$。这个方向以 teacher/target 分布作期望：teacher 赋予非零概率的 token 都会进入 loss，student 若在 teacher 高概率区域分配不足会受罚。它不是 $D_{KL}(p_S \| p_T)$，也不是 Jensen–Shannon divergence。

代码按每个 sample 的有效 response token 数归一化：

$$\mathcal{L}_i = \frac{\sum_t m_{i,t} D_{KL}(q_{\beta,i,t} \| p_{S,i,t})}{\max(1, \sum_t m_{i,t})}, \qquad \mathcal{L} = \frac{1}{B} \sum_i w_i \mathcal{L}_i$$

其中 $m_{i,t}$ 是 response/prefix token mask，w_i 是 `_opd_loss_weight`。当前实现为已截断 response 构造全 1 mask；padding/prompt 不进入该 token KL。run summary 还记录 student/teacher entropy、top-k overlap、loss token 数与 vocab size，便于区分“loss 下降”和“输出分布结构一致”。

3. Coldstart-500 与 OPD5000 的 lineage

Run	Student parent	Rollout	Objective	Prefix / veto	Steps / save	本地证据		
coldstart-500	`/data/msz/models/8b_base`	teacher	full-vocab `KL(target		student)`	max new 128；loss prefix 64；beta 1→0 / 100 steps	500；final only	launcher + coldstart summary
OPD5000	coldstart-500 output	student	full-vocab `KL(target		student)`	max new 64；loss prefix 64；beta 1→0 / 500 steps	5000；save every 1000	launcher + OPD5000 summary

coldstart summary 的关键字段为：`opd_rollout_source=teacher`、`opd_distill_support=full_vocab`、`opd_mode=five_expert_online_teacher_rollout_teacher_full_vocab_kl_veto_prefix`。OPD5000 summary 对应为 `student`、`full_vocab` 和 `five_expert_online_student_rollout_teacher_full_vocab_kl_veto_prefix`。

两条 run 的 route teacher 都由 sample metadata 选择；`route-block-shuffle` 保证 ZeRO-3 下每个 microstep 的 rank 进入同一 teacher。该设计证明的是 deterministic metadata routing，不是训练出了 learned router。

4. Implementation anchors

Source	SHA-256	能支持	不能支持
`muxi/train_opd_online_v1.py`	`BF9120B5D88180EA4C4725C6F4934913F35FDD36ECB4BC8E6B3B089FB964D8D9`	legacy teacher rollout、teacher top1 label、token CE、legacy mode string	coldstart/OPD5000 full-vocab KL
`muxi/point/train_opd_online_v1.py`	`7103E3B28068A30978F2B14AD980E51CEC2EC60A0445B2B52243342CDA286336`	rollout-source switch、full-vocab KL 方向、veto、prefix、entropy/top-k diagnostics	远端实际执行内容与本地文件逐字一致之外的结果

Source	SHA-256	能支持	不能支持
coldstart launcher	`06F349A4631D30F9533746CE7B8F852C14FC49A02FC047968 39A7F3C3BADCE10`	teacher rollout、base parent、500 steps、prefix/veto/ZeRO-3 参数	完成状态、权重内容
OPD5000 launcher	`DEEB0F93336C99AA54BC4D34F95315C5A2D60E0F9968293B2 0D9F3A961491BAF`	student rollout、coldstart parent、skip64k、5000 steps、save cadence	checkpoint binary 与 eval uplift
coldstart run summary	`90FD5A0A5983FEB5673D44CA087F597F0EBCACFD63DF1305C 37C297F213D5FC0`	run parameters、last loss/entropy/overlap、mode	raw log、weights、独立复算
OPD5000 run summary	`2D670DE0707CD2C6A9D1D4C7EC109C86F0A3F8F87EDE5D695 9367017231547CE`	run parameters、route counts、last loss/entropy/overlap、mode	checkpoint binary、optimizer、raw predictions
OPD5000 artifact manifest	`035764A0461DEE5942AB25AF1BDFCD2514BD69AAC473B2DE8 E43F3A7EC86021A`	remote root、formal code path、launchers、final model path、sync exclusion	binary/hash/load parity

5. 面试口述修正

30 秒版本

> OPD 有两代同名 trainer，不能混讲。根目录 legacy 版是 teacher rollout 后取 teacher top1 token，用 hard-label CE 蒸馏；正式 new-experts 路线在 `point/train_opd_online_vl.py`，coldstart 用 teacher rollout、OPD5000 用 student rollout，两者都让 route 对应 teacher 在同一 response 上 forward，并做完整词表的 $D_{KL}(q_\beta \| p_S)$。veto 期的 target 还混入 stop-gradient student logits，beta 归零后才是纯 $D_{KL}(p_T \| p_S)$。本地缺权重和 raw predictions，所以我能证明目标实现和 summary lineage，不能把它直接升级成因果 uplift。

“为什么 KL 方向重要？”

> 这里代码的 expectation 在 teacher/target 分布一侧，是 $D_{KL}(q_\beta \| p_S)$，beta=0 时就是 teacher-to-student forward KL。这样 teacher 认为有概率的完整词表区域都会约束 student；如果反过来做 $D_{KL}(p_S \| p_T)$，优化压力和 mode behavior 会不同。工程上还必须说清 logits 的 token 对齐、response prefix、mask、vocab revision 和 reduction，否则只说“用了 KL”没有可复核意义。

“coldstart 和正式 OPD 的区别是什么？”

> 两者使用同一 formal full-vocab KL trainer，但 rollout distribution 不同。coldstart 从 8B base 出发，用 teacher 生成 response，降低 student 初始分布太差带来的 on-policy风险；OPD5000 从 coldstart 输出继续，用 student 自己生成 response，再让 route teacher 在该 response 上给完整词表分布。它们不是两种 KL 方向，也不是一个 top1 CE、一个 KL。

6. 必须保留的边界

- 不再用 root `train_opd_online_vl.py` 支持 formal full-vocabulary KL claim；该文件只能支持 legacy top1 CE。
- 不把 coldstart 说成 student rollout；summary 和 launcher均明确为 teacher rollout。
- 不把 OPD5000 说成 teacher rollout；它显式为 student rollout。
- veto beta 大于 0 时，target 是 teacher/student detached logit mixture，不是纯 teacher distribution。
- full-vocabulary 表示 loss support，不代表所有 vocabulary token 在每个位置都有较大概率，也不等于生成时没有 top-p/temperature 配置。
- 现有 summary/manifest 不包含 model weights、optimizer、raw predictions；训练完成、checkpoint 可加载和 uplift 仍需 Po bundle。

7. 下一证据门

1. 从远端回传 formal `point/train_opd_online_vl.py`、launchers、DeepSpeed config 与每个 checkpoint 的 content hash，建立 run-specific manifest。
2. 新进程加载 coldstart final 与 checkpoint-1000..5000，固定 prompt/tokenizer/vocab revision 做 loss/entropy/top-k parity smoke。
3. 同一 raw-holdout、同 scorer 保留 per-sample predictions，比较 SFT-only、legacy top1、coldstart、OPD5000；没有 paired raw 前不做 objective 单因果归因。
4. 对 `veto_beta=0` 与 schedule 做 matched ablation，区分 stabilization 与最终分布拟合贡献。

8. References

- OPD Claim–Evidence Inventory
- OPD Po Bundle Audit
- OPD implementation-derived oral answer
- VLA-FND-007 KL direction card
- `C:\Users\mousongzhe\muxi\train_opd_online_vl.py`
- `C:\Users\mousongzhe\muxi\point\train_opd_online_vl.py`
- `C:\Users\mousongzhe\muxi\point\run_opd_offpolicy_coldstart500_p0p1_zero3_mb16_accum1_newexperts.sh`
- `C:\Users\mousongzhe\muxi\point\run_opd_p0p1_studentrollout_full5000_skip64000_save1000_zero3_mb16_accum1_from_coldstart500_newexperts.sh`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\artifact_manifest.json`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\remote_model_summaries\coldstart500_opd_online_run_summary.json`
- `C:\Users\mousongzhe\muxi\report\opd5000_newexperts_20260601\remote_model_summaries\opd5000_opd_online_run_summary.json`

VLA INTERVIEW LIBRARY · ARIS ACADEMIC PRINT EDITION

VLA INTERVIEW · EVIDENCE-FIRST NOTES

VOL 08 · Coding / System Design

VLA 白板编码、训练评测平台、实时服务与故障恢复

> 本卷由 25 份本地 ARIS HTML 实际内容装订生成；知识、题卡、事实/项目证据保持原页语义。题卡全部展开用于打印，但不改变仓库中的 `unseen` 状态。

装订目录

知识库

1. 01 · VOL 08 CODE-001：数值稳定的 softmax / log-softmax 实现
2. 02 · VOL 08 CODE-002：Scaled Dot-Product Attention 手写与调试
3. 03 · VOL 08 CODE-003：LayerNorm / RMSNorm 手写与调试
4. 04 · VOL 08 CODE-004：Masked MSE / Huber Action Loss
5. 05 · VOL 08 CODE-005：Homogeneous Transform Composition
6. 06 · VOL 08 CODE-006：2-Link Planar Forward Kinematics
7. 07 · VOL 08 CODE-007：PID Update with Anti-windup
8. 08 · VOL 08 CODE-008：Bellman TD(0) Update
9. 09 · VOL 08 SYS-001：可复现的 VLA 训练—评估平台
10. 10 · VOL 08 SYS-002：Policy Serving 与 Real-time Control Loop
11. 11 · VOL 08 SYS-003：Robot Fleet 与 Multi-tenant GPU Scheduling
12. 12 · VOL 08 SYS-004：Fault Recovery、Safety 与 Observability

题卡

1. 13 · VOL 08 CODE-VLA-001 原子手撕题卡：数值稳定的 softmax
2. 14 · VOL 08 CODE-VLA-002 原子手撕题卡：Scaled Dot-Product Attention
3. 15 · VOL 08 CODE-VLA-003 原子手撕题卡：LayerNorm / RMSNorm
4. 16 · VOL 08 CODE-VLA-004 原子手撕题卡：Masked MSE / Huber Loss
5. 17 · VOL 08 CODE-VLA-005 原子手撕题卡：Homogeneous Transform Composition
6. 18 · VOL 08 CODE-VLA-006 原子手撕题卡：2-Link Planar Forward Kinematics
7. 19 · VOL 08 CODE-VLA-007 原子手撕题卡：PID Update with Anti-windup
8. 20 · VOL 08 CODE-VLA-008 原子手撕题卡：Bellman TD(0) Update
9. 21 · VOL 08 SYS-001 原子题卡：可复现的 VLA 训练—评估平台
10. 22 · VOL 08 SYS-002 原子题卡：Policy Serving 与 Real-time Control Loop
11. 23 · VOL 08 SYS-003 原子题卡：Robot Fleet 与 Multi-tenant GPU Scheduling
12. 24 · VOL 08 SYS-004 原子题卡：Fault Recovery、Safety 与 Observability

来源与事实复核

1. 25 · VOL 08 Coding / System Design：Coverage-Gap Audit

Part 1 / 25 · `interview_materials/vla/knowledge/vol_08_code001_softmax.html` · SHA-256 `4910ECA5193C59913CD3CF65C9CA0AFAEF3212249F0BC09BC9511E57DF9A9524`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`4910eca5193c59913cd3cf65c9ca0afaef3212249f0bc09bc9511e57df9a9524`；生成自：`interview_materials\vla\knowledge\vol_08_code001_softmax.md`

VOL 08 CODE-001：数值稳定的 softmax / log-softmax 实现

状态：`drafted / CODE-VLA-001 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-001 Source Freeze。

问题（Question）

手撕 softmax 不只是写出指数公式。面试官通常会继续追问数值稳定性、`dim`、mask、低精度、全 masked 行、梯度和它在 attention/CE/action-token 中的边界。

1. 直觉：先把最大值搬到零

直接计算 e^{1000} 会溢出，直接计算 e^{-1000} 可能下溢。因为 softmax 对整体平移不敏感，可以令 $m = \max(z)$，先计算 $z - m$：最大项变成 0，指数最多为 1。

2. 数学与实现映射

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}, \qquad \log p_i = z_i - \operatorname{logsumexp}(z).$$

`z` 是沿 `dim` 归一化的 logits；`p` 的和为 1；`logsumexp` 通过稳定的 max-shift 计算。实现时不要先 `softmax` 再 `log`，而应直接使用 `log_softmax` 或 `logsumexp`，避免概率下溢。

2.1 手写 forward

```
def stable_softmax(x, dim=-1):
    # Accumulate in a wider dtype when the input is low precision.
    work = x.float() if x.dtype in (torch.float16, torch.bfloat16) else x
    shifted = work - work.amax(dim=dim, keepdim=True)
    exp = shifted.exp()
    probs = exp / exp.sum(dim=dim, keepdim=True)
    return probs.to(dtype=x.dtype)
```

代码中的 `keepdim=True` 保证广播维度正确；`amax` 与 `sum` 都沿同一个 `dim`。低精度输入是否 cast 回原 dtype，要由调用方的误差和 kernel contract 决定。

2.2 手写 log-softmax

```
def stable_log_softmax(x, dim=-1):
    # Compute logsumexp without materializing an unstable exp(x).
    work = x.float() if x.dtype in (torch.float16, torch.bfloat16) else x
    m = work.amax(dim=dim, keepdim=True)
    lse = m + (work - m).exp().sum(dim=dim, keepdim=True).log()
    return (work - lse).to(dtype=x.dtype)
```

生产代码优先使用框架 fused `log_softmax`；手写版本的价值在于解释 shape、dtype、mask 和测试，而不是替代高度优化 kernel。

3. Mask 语义

3.1 Attention mask

对不允许读取的位置，常见做法是将 logits 加上 $-\infty$ 或使用 bool mask。它改变信息流：被 mask 的位置不应获得 attention probability。

3.2 Loss mask

`loss_mask` 决定哪些 label 参与梯度；它不等于 attention mask。VLA 中 image/prompt token 可能可见但不计 CE loss，action token 可能需要 loss；必须分别记录。

3.3 全 masked 行

如果一整行都被加上 `-inf`，`max` 和 `logsumexp` 可能产生 `-inf - (-inf)=NaN`。系统必须提前规定：reject、返回全零并标记 invalid，或由上游保证每行至少一个 valid key。不能让 NaN 静默进入 attention 或 action head。

4. 梯度与复杂度

对 softmax Jacobian：

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j).$$

这说明概率越尖，非主类梯度越小；过大的 logits 会使 softmax 饱和。沿长度 L 的 dense softmax 通常是 $O(L)$ 的归一化，但 attention 的 $QK^\top$ 仍可能是 $O(L^2)$；不要把 softmax 稳定性误说成 attention 复杂度优化。

5. 测试矩阵

测试	断言	发现的 bug
known small logits	沿 dim 和为 1，值接近手算	dim/keepdim 错
extreme ±1000	无 NaN/Inf，结果与框架接近	未 max-shift
batch/transpose	shape 与任意 dim 正确	广播轴错
bool/float mask	masked probability/gradient 符合 contract	mask 加法方向错
all masked row	按约定返回 invalid/zero/reject	`-inf-(-inf)` NaN
fp16/bf16	forward/backward finite，误差在阈值内	低精度累加溢出
autograd parity	与框架 forward/backward 差异受控	手写 backward 或 cast 错

最小属性测试：随机生成不同 batch、长度、dim、dtype 和 mask，比较 `stable_log_softmax(x).exp()` 与 `stable_softmax(x)`，并与框架结果做 toleranced parity。对于全 masked 行，属性测试必须跳过或验证显式 invalid 分支。

6. VLA 映射与边界

- **Attention**：对 $QK^\top/\sqrt{d_k}$ 做 mask+softmax，决定 token 信息流；scale、mask 和 softmax 是不同合同。
- **Action token CE**：使用 `log_softmax`/交叉熵，prompt/image label 可能由 `loss_mask` 排除。
- **Continuous action**：MSE、Huber、diffusion/flow vector field 不需要 softmax；动作维度不能因为“概率和为 1”而被错误归一化。
- **Routing**：softmax 权重可表示 mixture/router probability，但还需说明 temperature、capacity、load balance 和 auxiliary loss。

7. 项目挂钩与真实性边界

OpenPiVLA 的 causal-CE、OPD 的 scorer、WAM/UWM 的 mask/routing 可用于讨论“概率/监督/条件 mask 的区别”；但当前项目没有本地 CODE-VLA-001 手写实现、dtype benchmark 或梯度 parity artifact。面试时可以解释实现和测试设计，不能声称某项目因自写 softmax 获得性能提升。

8. 结论与下一步

稳定 softmax 的核心是 max-shift、正确 dim、明确 mask/全 masked 行和合适的累加 dtype；log-softmax 应避免先算概率再取 log。手撕题的高分点是同时给出代码、边界测试、梯度/复杂度解释和 VLA mask 映射，而不是只背公式。

下一步：保持 `CODE-VLA-001` 题卡为 `unseen`，进行一次白板/实现练习，或补充 dtype/device parity artifact。

References

1. CODE-001 Source Freeze。
2. PyTorch `torch.softmax`、`log_softmax` 与 `CrossEntropyLoss`。
3. PyTorch `scaled_dot_product_attention`。
4. 本仓库 VOL 01 Foundations 与 VOL 03 architecture/mask audit。

Part 2 / 25 · `interview_materials/vla/knowledge/vol_08_code002_attention.html` · SHA-256 `9AC831A8765E03693F0F589C1FEB2CFD3C3AC50BEA86B992F8A3235CEA341193`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`9ac831a8765e03693f0f589c1feb2cfd3c3ac50bea86b992f8a3235cea341193`；生成自：`interview_materials\vla\knowledge\vol_08_code002_attention.md`

VOL 08 CODE-002：Scaled Dot-Product Attention 手写与调试

状态：`drafted / CODE-VLA-002 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-002 Source Freeze。

问题（Question）

手写 attention 需要同时处理 Q/K/V shape、head 拆分、$1/\sqrt{d_k}$ scale、mask 方向、不同 query/key 长度、dropout 和低精度。VLA 还要区分 attention mask 与 action loss mask。

1. 数学与 shape

$$S = \frac{QK^\top}{\sqrt{d_k}} + M, \qquad A = \mathrm{softmax}(S), \qquad Y = AV.$$

典型 shape：`Q:[B,H,Lq,D]`、`K/V:[B,H,Lk,D]`；`Q @ K.transpose(-2,-1)` 得 `[B,H,Lq,Lk]`；`A @ V` 回到 `[B,H,Lq,D]`。最后合并 head 为 `[B,Lq,H*D]`。`Lq` 与 `Lk` 可以不同，不能只用一个 `L` 写死。

2. 手写实现

```
def attention(q, k, v, mask=None, dropout_p=0.0, training=True):
    # q/k/v: [B, H, L, D]; only the last two axes are contracted.
    d = q.size(-1)
    work_q = q.float() if q.dtype in (torch.float16, torch.bfloat16) else q
    work_k = k.float() if k.dtype in (torch.float16, torch.bfloat16) else k
    scores = work_q @ work_k.transpose(-2, -1) / math.sqrt(d)

    # Apply a bool allow-mask or an additive float bias before softmax.
    if mask is not None:
        scores = apply_attention_mask(scores, mask)
    weights = torch.softmax(scores, dim=-1)
    if training and dropout_p > 0:
        weights = torch.dropout(weights, dropout_p, train=True)
    out = weights @ (v.float() if v.dtype in (torch.float16, torch.bfloat16) else v)
    return out.to(dtype=q.dtype)
```

关键是只在最后两个维度做矩阵乘法，softmax 沿 key 维 `Lk`，mask 在 softmax 前应用，低精度可在更宽 dtype 中计算。生产环境优先使用 fused SDPA；手写版本用于解释 shape、契约和测试。

3. $1/\sqrt{d_k}$ 为什么必要

若 query/key 各维近似独立、方差相近，点积方差随 d_k 增长。未缩放的 score 进入 softmax 后容易过尖，非主位置梯度变小；除以 $\sqrt{d_k}$ 把 logit 尺度拉回稳定范围。它是尺度控制，不是让 softmax “变成概率”的原因。

4. Mask 语义与 causal 方向

4.1 Bool 与 additive mask

bool mask 必须规定 `True` 是 allow 还是 deny；float mask 通常把禁止位置加上极小值或 $-\infty$。实现中必须统一语义，不能把 `key_padding_mask` 的 convention 直接套到自定义 `attn_mask`。

4.2 Causal mask

自回归 query 位置 i 只能读取 key 位置 $j \le i$。若 `Lq != Lk` 或使用 prefix/suffix layout，不能凭矩阵左上角猜方向；应写出 query/key 的物理位置映射，并用 future-blocking 单测验证。

4.3 全 masked 行

整行都被禁止时，`softmax(-inf,...,-inf)` 可能返回 NaN。系统必须规定 reject、返回 invalid/zero，或保证每行至少一个 valid key；不能静默把 NaN 送入 action head。

5. Dropout、dtype 与复杂度

dropout 只在训练时打开，评估/推理必须为 0；不要在手写函数中忽略 `training`。score/value 的主要计算量近似为 $O(BHL_qL_kD)$，显存还包含 `[B,H,Lq,Lk]` 的 score/weight；softmax 本身是沿 `Lk` 的归一化，不等于 attention 整体变成线性复杂度。fp16/bf16 可用 fp32 累加，但输出 cast 与误差阈值要由接口冻结。

6. 测试矩阵

测试	断言	典型 bug
单头小矩阵	与手算 score/weight/output 一致	transpose/scale 错
多头 batch	输出 `[B,H,Lq,D]`，合并 head 正确	head/hidden reshape 错
`Lq != Lk`	score `[B,H,Lq,Lk]`、softmax 沿 `Lk`	写死 square attention
causal	future key 权重为 0	mask 方向反
padding	padded key 不被读取	mask broadcast 错
train/eval dropout	eval deterministic，train 可随机	忘记 training flag
fp16/bf16	finite，误差与 fp32 对照受控	低精度 score 溢出
all masked row	按 contract invalid/zero/reject	`-inf-(-inf)` NaN
autograd parity	与 PyTorch SDPA 梯度差异在 tolerance 内	手写路径不等价

7. VLA 映射

- image/text/proprio/action token 的排列决定 Q/K/V 能看到什么；prefix/suffix 与 causal mask 必须画成信息流图。
- `attention_mask` 控制可见性；`loss_mask` 控制哪些 token 产生 CE/ action loss，二者不能混称。
- continuous action 的 MSE、diffusion 或 flow vector field loss 不需要对动作维度做 softmax。
- fused SDPA、FlashAttention 或特定 backend 的速度需要独立 benchmark；forward parity 不自动证明 latency、显存或真机收益。

8. 项目挂钩与真实性边界

OpenPiVLA 可连接 causal-CE、eager attention 与 action packaging；OPD 可连接 token/format scorer；WAM/UWM 可连接 condition/mask/routing，但这些都只是机制或局部代码证据。当前没有本地 CODE-VLA-002 多 backend parity、profile 或项目 uplift artifact。

9. 结论与下一步

手写 attention 的高分答案是：先固定 `[B,H,L,D]` 合同，再做 $QK^\top/\sqrt{d_k}$、沿 key 维 softmax、mask/causal/dropout、`AV` 和 head merge；随后用不同长度、mask、低精度、梯度与 fused SDPA parity 测试。它能证明实现理解，不能直接证明 VLA 项目性能。

下一步：保持 `CODE-VLA-002` 题卡为 `unseen`，进行一次白板/实现练习，或补充多 backend attention parity/profile artifact。

References

1. CODE-002 Source Freeze。
2. PyTorch `scaled_dot_product_attention` 与 `MultiheadAttention`。
3. Vaswani et al., Attention Is All You Need。
4. 本仓库 VOL 01 Foundations 与 VOL 03 architecture audit。

Part 3 / 25 · interview_materials/vla/knowledge/vol_08_code003_norm.html · SHA-256 53919A8B1A8B82BD8213F3ACD8DE07533737C02308AA1E3C254B9A3254EBF8C5

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：53919a8b1a8b82bd8213f3acd8de07533737c02308aa1e3c254b9a3254ebf8c5；生成自：C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_08_code003_norm.md

VOL 08 CODE-003：LayerNorm / RMSNorm 手写与调试

状态：drafted / CODE-VLA-003 / knowledge_1_of_phase / card_synced / WARN 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-003 Source Freeze。

问题（Question）

手撕 LayerNorm/RMSNorm 的高频坑是：沿错维度、误用 BatchNorm 的 running statistics、忽略低精度累加、epsilon 位置错、affine shape 错，以及把 RMSNorm 简化成“LayerNorm 去掉 bias”。

1. 公式与区别

LayerNorm：

$$\mu = \frac{1}{D}\sum_i x_i, \quad \sigma^2 = \frac{1}{D}\sum_i (x_i-\mu)^2, \quad y_i = \gamma_i \frac{x_i-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta_i.$$

RMSNorm：

$$r = \sqrt{\frac{1}{D}\sum_i x_i^2 + \epsilon}, \quad y_i = \gamma_i \frac{x_i}{r}.$$

LayerNorm 同时 re-center 和 re-scale；RMSNorm 不减均值，只按 root mean square 缩放。RMSNorm 通常只有 scale，但具体 API 是否允许 bias 要看实现，不能从名称猜测。

2. 归一化轴

对 `[B,L,D]`，Transformer 常沿最后 `D` 归一化：每个 batch、每个 token 独立计算统计量。`normalized_shape=(H,W,C)` 时，则沿末尾三维计算。实现应把 axis 写成 `tuple(range(x.ndim-len(shape), x.ndim))`，而不是写死 `dim=-1` 后假装支持多维。

3. 手写实现

```
def layer_norm(x, weight=None, bias=None, eps=1e-5, normalized_ndim=1):
    # Accumulate statistics in fp32 for fp16/bf16 inputs.
    work = x.float() if x.dtype in (torch.float16, torch.bfloat16) else x
    dims = tuple(range(work.ndim - normalized_ndim, work.ndim))
    mean = work.mean(dim=dims, keepdim=True)
    var = (work - mean).pow(2).mean(dim=dims, keepdim=True)
    y = (work - mean) * torch.rsqrt(var + eps)
    if weight is not None:
        y = y * weight
    if bias is not None:
        y = y + bias
    return y.to(dtype=x.dtype)

def rms_norm(x, weight=None, eps=1e-6, normalized_ndim=1):
    # RMSNorm keeps the mean; it normalizes by mean-square only.
    work = x.float() if x.dtype in (torch.float16, torch.bfloat16) else x
    dims = tuple(range(work.ndim - normalized_ndim, work.ndim))
    inv_rms = torch.rsqrt(work.pow(2).mean(dim=dims, keepdim=True) + eps)
    y = work * inv_rms
    if weight is not None:
        y = y * weight
    return y.to(dtype=x.dtype)
```

`weight/bias` 的 shape 必须与 normalized shape 可广播；生产实现优先使用框架 kernel，手写版本用于解释合同和测试。

4. Epsilon 与 dtype

(epsilon) 应加在方差/均方的平方根之前，避免零方差除零。不同 API 默认 epsilon 可能不同，迁移 checkpoint 时必须随 config 冻结。fp16/bf16 直接算均值平方可能精度不足或溢出，常在 fp32 中累加；输出是否 cast 回原 dtype 取决于 kernel/API contract。

5. 训练与推理

LayerNorm/RMSNorm 使用当前样本/位置的末尾维统计，不维护 BatchNorm 式 running mean/variance，因此相同输入、相同参数、dropout 关闭时，train/eval 输出应一致。若输出不同，应先查外层 dropout、autocast、fused kernel、量化或模型状态，而不是为 norm 人为添加 running stats。

6. 测试矩阵

测试	断言	典型 bug
`[B,L,D]`	每个 token 独立归一化	沿 batch/sequence 统计
multi-dim shape	末尾多维统计与 affine 广播正确	写死 `dim=-1`
constant input	LayerNorm finite、中心化；RMSNorm finite	eps 位置错
non-zero mean	LayerNorm 输出均值近零；RMSNorm 不保证	把两者写成同一公式
fp16/bf16	finite、与 fp32 对照受控	低精度统计不稳定
affine on/off	weight/bias shape 与梯度正确	broadcast 错
train/eval	norm 自身一致	误加 running stats
autograd parity	与 PyTorch forward/backward 接近	variance/ddof/cast 错

LayerNorm 的 variance 通常使用 population variance（unbiased=False）；若手写成 sample variance，短维度时会明显偏差。

7. Transformer / VLA 映射

- pre-norm 将 norm 放在 attention/MLP 前，残差路径更直接；post-norm 放在残差相加后。不能脱离深度、初始化和学习率说绝对优劣。
- VLM/VLA backbone 常用 LayerNorm 或 RMSNorm；action expert、vision adapter 和 language backbone 可能 epsilon/dtype 不同。
- norm 解决特征尺度稳定性，不负责 action denormalization；训练用 feature norm 与机器人 action normalizer 是不同概念。
- 上游模型选择 RMSNorm 不等于用户项目完成了 norm ablation 或获得推理加速。

8. 项目挂钩与真实性边界

OpenPiVLA 的 Qwen backbone、LoRA/attention 兼容讨论可连接 RMSNorm/LayerNorm；UWM/WAM 的 DiT/transformer blocks 可连接 pre/post-norm 和 dtype；但当前没有本地 CODE-VLA-003 parity、profile 或 matched norm ablation。安全表达是“我能解释并实现 norm 合同”，不声称项目因更换 norm 获得 uplift。

9. 结论与下一步

LayerNorm 手写要减均值、按末尾 normalized shape 算 population variance、加 epsilon、affine；RMSNorm 不减均值，只按 mean-square 缩放。两者都不使用 running statistics。高分答案还应覆盖 fp32 累加、affine broadcast、train/eval parity、constant/non-zero-mean 测试和 VLA action normalizer 边界。

下一步：保持 `CODE-VLA-003` 题卡为 `unseen`，进行一次白板/实现练习，或补充 forward/backward parity artifact。

References

1. CODE-003 Source Freeze。
2. PyTorch `LayerNorm` 与 `RMSNorm`。
3. Zhang & Sennrich, Root Mean Square Layer Normalization。
4. 本仓库 VOL 01 Foundations。

Part 4 / 25 · `interview_materials/vla/knowledge/vol_08_code004_masked_loss.html` · SHA-256 `78632186254B6454B81FC229D207125F590D0E30E0AFCE2714050658741D1B92`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`78632186254b6454b81fc229d207125f590d0e30e0afce2714050658741d1b92`；生成自：`interview_materials\vla\knowledge\vol_08_code004_masked_loss.md`

VOL 08 CODE-004：Masked MSE / Huber Action Loss

状态：`drafted / CODE-VLA-004 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-004 Source Freeze。

问题（Question）

VLA action batch 常含不同 horizon、action dimension、padding 与缺失维度。手写 masked loss 的核心不是乘一个 mask，而是冻结 elementwise target、mask 语义、分母、sample/task weighting、空 mask 和梯度行为。

1. MSE 与 Huber

令误差 $e_i = \hat{a}_i - a_i$。MSE：

$$\ell_i = e_i^2.$$

Huber：

$$\ell_i = \begin{cases} \frac{1}{2}e_i^2, & |e_i| \le \delta, \\ \delta(|e_i| - \frac{1}{2}\delta), & |e_i| > \delta. \end{cases}$$

Huber 在小误差区保持平滑二次惩罚，在大误差区改为线性，降低 outlier 的梯度支配；它不是“自动更鲁棒”，delta、单位、normalizer 和任务分布必须冻结。

2. Masked reduction

正确的有效元素均值通常是：

$$L = \frac{\sum_i w_i m_i \ell_i}{\sum_i w_i m_i}.$$

`m` 是 valid/loss mask，`w` 可是 sample/task/dimension weight。危险实现 `mean(mask * loss)` 的分母是总 slot 数，会使 padding 更多的 batch 获得更小 loss，从而改变不同 horizon/embodiment 的权重。

3. 手写实现

```
def masked_huber(pred, target, mask, delta=1.0, weight=None):
    # pred/target: [B, H, D]; mask must be broadcastable to the same shape.
    work_pred = pred.float() if pred.dtype in (torch.float16, torch.bfloat16) else pred
    work_target = target.float() if target.dtype in (torch.float16, torch.bfloat16) else target
    error = work_pred - work_target
    abs_error = error.abs()
    element = torch.where(
        abs_error <= delta,
        0.5 * error.square(),
        delta * (abs_error - 0.5 * delta),
    )

    valid = mask.to(dtype=element.dtype)
    if weight is not None:
        valid = valid * weight
    denom = valid.sum()
    # Empty supervision is a data/protocol event, not an ordinary zero-loss batch.
    if denom.item() == 0:
        raise ValueError("empty masked action target")
    return (element * valid).sum() / denom
```

若分布式训练，各 rank 的 numerator/denominator 应先聚合，再相除；先在每 rank 求 mean 再平均会在有效元素数不同的情况下产生偏差。

4. Mask taxonomy

mask	控制什么	常见错误
attention mask	token 信息流	当作 action supervision mask
sequence/padding mask	无效 timestep/slot	只乘 loss 不改分母
action-dim mask	异构 embodiment 有效轴	silent truncate/zero padding 当真实标签
label/loss mask	哪些 target 产生梯度	与 causal mask 混淆
task/sample weight	样本/任务贡献	与二值 valid mask 混成同一语义

mask 应先扩展到 elementwise loss shape，再检查 dtype、0/1 或非负权重、广播维和有效计数。

5. 空 mask 与异常值

全零 mask 可能来自数据 bug、空 action chunk、过滤后无有效维或路由错误。安全处理是 reject/skip 并记录 `episode/sample/reason`；若必须返回零梯度，应显式标记 `skipped_step`，不能把它记成正常低 loss。

NaN/Inf target、weight 或 prediction 应在乘 mask 前检测，因为 `0 * NaN` 仍可能产生 NaN。异常 outlier 是否用 Huber 处理，要先确认它是合法长尾还是单位/normalizer/axis 错误。

6. Weighting 与可比性

不同任务可以按 episode、sample、timestep、action dimension 或 task weight 聚合。每种 denominator 表达不同问题：

- per-element mean：每个有效 action 维等权；
- per-step mean：先对维度平均，再对有效 timestep；
- per-episode mean：每个 episode 等权；
- per-task macro：每个 task 等权；
- global micro：所有有效元素共同计数。

报告 loss 时必须写清 reduction；否则不同 horizon、D 或数据比例之间不能比较。

7. 测试矩阵

测试	断言	典型 bug
mask=1	与框架 elementwise+mean 相同	公式/reduction 错
部分 timestep/dim	只有效元素有梯度	broadcast/分母错
不同 padding 比例	相同有效元素得到相同 loss	`mean(mask*loss)`
all-zero mask	reject/skip contract 生效	伪零 loss
Huber delta 边界	分段连续、梯度方向正确	delta/系数错
outlier	Huber 梯度线性增长	仍被平方项支配
fp16/bf16	finite、与 fp32 对照受控	低精度 square 溢出
distributed counts	global numerator/denominator 一致	rank mean 平均偏差
autograd parity	与 PyTorch elementwise loss 接近	where/cast/weight 错

8. VLA 映射

- ACT/continuous regression 可对 action chunk 使用 L1/MSE/Huber，并用 timestep/dim mask。
- Diffusion/Flow Matching 常对 noise/vector field 做 MSE，但要冻结 timestep weight、condition mask、target 与 reduction。
- action token 模型使用 CE/token mask，不应把 token ID 当 continuous value 做 MSE。
- action normalizer 改变误差单位；Huber delta 必须在同一 normalized/physical space 下解释。

9. 项目挂钩与真实性边界

OpenPiVLA 可连接 action chunk、valid dims、normalizer 和 causal-CE；UWM/WAM 可连接 action/video target mask 与多任务 reduction；OPD 可连接 metric-specific denominator。当前没有本地 CODE-VLA-004 parity、empty-mask log 或 matched MSE/Huber ablation，因此只能讲实现与审计方法。

10. 结论与下一步

masked loss 的高分答案是：先算 elementwise MSE/Huber，再用 valid mask/weight 得到 numerator 和有效 denominator；空 mask 作为数据事件处理；低精度用 fp32 累加；分布式聚合 numerator/denominator；最后说明 reduction 与 VLA action schema。不能把 `mean(mask*loss)`、伪零 loss 或不同 denominator 的数字直接比较。

下一步：保持 `CODE-VLA-004` 题卡为 `unseen`，进行一次白板/实现练习，或补充 empty-mask/parity artifact。

References

1. CODE-004 Source Freeze。
2. PyTorch `MSELoss`、`HuberLoss` 与 `SmoothL1Loss`。
3. 本仓库 VOL 03 action audit 与 VOL 05 heterogeneous batch。

Part 5 / 25 · interview_materials/vla/knowledge/vol_08_code005_transform.html · SHA-256 322C0A03F2146A4A67EF1F8E79A9885D202A6603876E17A6E46E40068E5CB77E

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：322c0a03f2146a4a67ef1f8e79a9885d202a6603876e17a6e46e40068e5cb77e；生成自：interview_materials\vla\knowledge\vol_08_code005_transform.md

VOL 08 CODE-005：Homogeneous Transform Composition

状态：drafted / CODE-VLA-005 / knowledge_1_of_phase / card_synced / WARN 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-005 Source Freeze。

问题（Question）

机器人面试中的 4×4 变换题，真正考的是 frame/方向/单位和组合顺序。一个矩阵 shape 正确、overlay 看起来差不多，也可能把 camera→base 写反或把毫米当米。

1. 齐次矩阵与方向

$$T_B^A = \begin{bmatrix} R_B^A & t_B^A \\ 0 & 1 \end{bmatrix}, \qquad \tilde{p}_A = T_B^A \tilde{p}_B.$$

记号约定：上标是输出/目标 frame，下标是输入 frame。若点 p_B 在 B frame，乘 T_B^A 得 A frame。复合必须右到左：

$$T_C^A = T_B^A T_C^B.$$

不能把 T_B_A 和 T_A_B 当作转置名称；它们通常互为逆，而不是简单转置整个 4×4 矩阵。

2. 手写实现

```
def make_transform(R, t):
    # R: [..., 3, 3], t: [..., 3]; preserve batch dimensions.
    T = torch.eye(4, dtype=R.dtype, device=R.device).expand(R.shape[:-2] + (4, 4)).clone()
    T[..., :3, :3] = R
    T[..., :3, 3] = t
    return T

def compose(T_A_B, T_B_C):
    # Matrix multiplication composes B->C after A<-B.
    return T_A_B @ T_B_C

def inverse_transform(T_A_B):
    # For a rigid transform, inverse rotation is transpose.
    R = T_A_B[..., :3, :3]
    t = T_A_B[..., :3, 3]
    T_B_A = torch.zeros_like(T_A_B)
    R_T = R.transpose(-2, -1)
    T_B_A[..., :3, :3] = R_T
    T_B_A[..., :3, 3] = -(R_T @ t.unsqueeze(-1)).squeeze(-1)
    T_B_A[..., 3, 3] = 1
    return T_B_A
```

实际代码还应检查 RᵀR≈I、det(R)≈1、底行 [0,0,0,1]、finite、单位和 frame metadata。若矩阵包含 scale/shear，则不能使用 rigid inverse 公式。

3. 点、方向与 pose

- **point**：齐次坐标 ([x,y,z,1])，平移会生效。
- **direction/vector**：齐次坐标 ([x,y,z,0])，只旋转不平移。
- **pose**：位置和姿态一起变换；姿态表示可用 rotation matrix/quaternion/SE(3)，不能只对 xyz 乘矩阵就声称 pose 完整变换。

4. 逆变换与 roundtrip

对 rigid transform：

$$(T_B^A)^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix}.$$

roundtrip 测试：随机或已知 p_B，先 $p_A = T_B^A p_B$，再 $p'_B = (T_B^A)^{-1} p_A$，检查 $\|p'_B - p_B\|$ 小于阈值。还要测试 `compose(T_A_B,T_B_C) @ p_C` 与先后两次变换相同。

5. Frame、单位与时间

ROS REP-103/105 提供右手系、SI 单位和移动平台 frame 语义参照，但项目必须记录自己的 `frame_id、parent、child、timestamp、calibration_revision、source`。`camera→base` 与 `base→camera` 方向、米/毫米、光学轴 convention、旧 TF 和新标定 revision 都是高危边界。

如果 observation 在 τ_o 捕获，而 TF 在 τ_{tf} 更新，不能默认二者同步；移动机器人应使用 capture-time TF 或插值，并把时间差/pose uncertainty 进入 downstream gate。

6. 测试矩阵

测试	断言	典型 bug
identity	不改变点/pose	底行/shape 错
translation	点平移、direction 不平移	w 写错
known rotation	90° 方向符合右手系	轴/方向反
composition	右到左与逐步结果一致	乘法顺序错
inverse roundtrip	点误差在阈值内	平移逆公式错
rotation check	`RᵀR=I, det=1`	非刚体矩阵误用
batch	`[...,4,4]` 广播正确	只支持单矩阵
unit/frame mismatch	smoke 能显式失败	米/mm 或 label 静默错
timestamp revision	使用 capture-time TF	取 latest TF 造成漂移

7. VLA 映射

PER-002 的 pixel→camera→target、grounding point、waypoint/subgoal、camera/base/world action 都依赖 transform contract。transform 正确只证明几何链路，不证明 depth、planner、controller、grasp 或导航闭环正确；应分层记录 projection、calibration、TF、planner/controller 和 safety evidence。

8. 项目挂钩与真实性边界

OPD 可连接 point/box/text 坐标协议；WAM/OpenPiVLA 可连接 camera/action packaging；Robot Harness 可连接 frame-aware tool schema；UWM/LIBERO 可连接 camera/state/action shape。当前没有本地 CODE-VLA-005 标定/TF parity 或闭环 artifact，不能把通用矩阵代码写成项目贡献或 3D 成功。

9. 结论与下一步

手写 transform 的高分答案是：明确 T_B^A 方向、右到左组合、rigid inverse、point/vector/pose 的 w 差异、frame/单位/时间戳和 roundtrip 测试。矩阵运算正确只是几何层证据，不等于机器人执行成功。

下一步：保持 `CODE-VLA-005` 题卡为 `unseen`，进行一次白板/实现练习，或补充 TF/calibration roundtrip artifact。

References

1. CODE-005 Source Freeze。
2. ROS REP-103、REP-105 与 SciPy `Rotation`。
3. 本仓库 VOL 06 PER-002。

Part 6 / 25 · `interview_materials/vla/knowledge/vol_08_code006_planar_fk.html` · SHA-256 `CA33144F9A752CF73BD146E88F73403F148F66A102E3C6F7FD427E6F1A0EC282`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`ca33144f9a752cf73bd146e88f73403f148f66a102e3c6f7fd427e6f1a0ec282`；生成自：`interview_materials\vla\knowledge\vol_08_code006_planar_fk.md`

VOL 08 CODE-006：2-Link Planar Forward Kinematics

状态：`drafted / CODE-VLA-006 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：CODE-006 Source Freeze。

问题（Question）

二连杆平面 FK 是机器人几何题的最小闭环：给定 $(l_1, l_2, \theta_1, \theta_2)$，输出末端位置与平面姿态，并能解释 frame、单位、组合顺序、Jacobian 和边界姿态。

1. 解析推导

第一杆末端：

$$p_1 = \begin{bmatrix} l_1 \cos\theta_1 \\ l_1 \sin\theta_1 \end{bmatrix}.$$

第二杆相对第一杆的方向是 $\theta_1 + \theta_2$，所以：

$$x = l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2), \quad y = l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2).$$

末端平面姿态：

$$\phi = \theta_1 + \theta_2.$$

这里 θ_2 是相对 joint angle；若输入给的是 world/global angle，第二项不能再次加 θ_1。

2. 齐次变换形式

```
def planar_fk(l1, l2, theta1, theta2):
    # Inputs use radians and the base frame is x-right/y-up.
    c1, s1 = torch.cos(theta1), torch.sin(theta1)
    c12, s12 = torch.cos(theta1 + theta2), torch.sin(theta1 + theta2)
    x = l1 * c1 + l2 * c12
    y = l1 * s1 + l2 * s12
    phi = theta1 + theta2
    return torch.stack((x, y, phi), dim=-1)
```

等价的矩阵链：

$$T_E^0 = R_z(\theta_1) T_x(l_1) R_z(\theta_2) T_x(l_2).$$

矩阵乘法右到左，先做 joint rotation，再沿当前 link frame 平移。若把 $T_x(l_2)$ 放到 rotation 前，结果会错误但 shape 仍然正确。

3. 角度、长度与 frame

冻结：radian/degree、长度单位、joint zero、正方向、base frame、tool offset 和是否包含 calibration。ROS REP-103 推荐 SI units 和右手系，但项目自己的 joint convention 仍需从 URDF/config/driver 核验。角度忘记从 degree 转 radian 往往不会报 shape 错，只会产生完全错误的位置。

4. Jacobian 直觉

对位置 (p=[x,y]^T)：

$$J = \frac{\partial p}{\partial [\theta_1, \theta_2]} = \begin{bmatrix} -l_1 \sin\theta_1 - l_2 \sin(\theta_1+\theta_2) & -l_2 \sin(\theta_1+\theta_2) \\ l_1 \cos\theta_1 + l_2 \cos(\theta_1+\theta_2) & l_2 \cos(\theta_1+\theta_2) \end{bmatrix}.$$

小角度下 $\Delta p \approx J \Delta\theta$。当两杆伸直或折叠时，Jacobian 可能退化，某些笛卡尔方向难以通过小关节变化实现；这只是局部可动性提示，不是完整控制器证明。

5. 测试矩阵

测试	断言	典型 bug
zero angles	`(x,y)=(l1+l2,0)`	sin/cos 或 frame 错
$\theta_1 = \pi/2$	末端沿 y 方向	degree/radian、轴方向错
folded $\theta_2 = \pi$	两杆相反	把 θ2 当 global angle
negative angles	右手系方向一致	符号 convention 错
matrix/analytic parity	两种 FK 误差在阈值内	composition 顺序错
finite-difference Jacobian	数值导数接近解析 J	Jacobian 符号错
batch	`[B,2]→[B,3]` shape 正确	只支持 scalar
unit mismatch	显式拒绝 degree/mm 配置	静默错单位

6. VLA / 项目映射

grounding point 进入 robot base、waypoint 进入 planner、action pose 进入 controller 时，都可以复用 FK/SE(3) 的 frame 和单位审计。FK 只说明 joint→pose 的几何映射，不包含 IK、碰撞、动力学、接触或 closed-loop success。

OPD/WAM/OpenPiVLA 的 camera/action 字段可作为接口讨论；Robot Harness/UWM 的 robot state schema 可作为 frame/shape 检查点。当前没有 CODE-VLA-006 的本地 URDF、joint calibration、controller trace 或真机结果。

7. 结论与下一步

二连杆 FK 的高分答案是：明确相对 joint angle、frame/单位，写出解析式和矩阵链，说明右到左 composition，再用 zero/90°/folded、matrix parity 和 finite-difference Jacobian 验证。它是几何层证据，不等于 IK、控制或真机成功。

下一步：保持 `CODE-VLA-006` 题卡为 `unseen`，进行一次白板/口述练习，或补充本地 FK/URDF/Jacobian artifact。

References

1. CODE-006 Source Freeze。
2. Modern Robotics Chapter 4 与 ROS REP-103。
3. 本仓库 CODE-005 Transform 与 VOL 01 Foundations。

Part 7 / 25 · interview_materials/vla/knowledge/vol_08_code007_pid_anti_windup.html · SHA-256 478EE605D2624BD75E42FA7FF2979F32B1F0D35CDFFE996BAD3E4AE4BBA7B407

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：478ee605d2624bd75e42fa7ff2979f32b1f0d35cdffe996bad3e4ae4bba7b407；生成自：C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_08_code007_pid_anti_windup.md

VOL 08 CODE-007：PID Update with Anti-windup

状态：drafted / CODE-VLA-007 / knowledge_10_of_phase / card_pending / WARN 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：CODE-007 Source Freeze。

问题（Question）

手写 PID 题真正考的不是背出 P+I+D，而是能否把离散时间、误差符号、内部状态、输出饱和、积分 anti-windup、reset 和测试 oracle 写成一个可审计的函数。VLA 场景还要继续追问：连续 action 是否已经 denormalize、控制器是否有独立的 rate/limit、安全 gate 和 episode epoch。

1. 直觉：为什么会 wind-up？

积分项会把长期误差累起来。当执行器已经达到上限，但误差仍要求更大的输出时，真实系统只能输出饱和值；如果软件仍按理想 u_unsat 累积积分，内部状态会越来越大。误差后来反向时，控制器要先“偿还”这笔虚假的积分债，造成 overshoot、恢复慢甚至安全风险，这就是 integral wind-up。

专业上必须区分三层：

1. 理想控制输出 u_unsat：P/I/D 相加的结果；
2. 执行器可实现输出 u_sat：经过上下限、rate limit 或安全 gate 后真正送出的值；
3. 内部积分状态 i_state：下一步更新时是否继续累积、冻结或被 u_sat-u_unsat 拉回。

ROS 2 control 文档直接暴露 output/integral clamp 与两类 anti-windup strategy；WPILib 文档强调固定 period、reset、integrator range、IZone 和 output clamp。它们支持 API/边界语义，不等于本地项目已经部署 PID 或证明真机稳定。

2. 冻结接口与状态

建议函数签名：

```
pid_step(
    setpoint: Tensor,
    measurement: Tensor,
    state: PIDState,
    gains: PIDGains,
    limits: PIDLimits,
    dt: Tensor,
) -> tuple[Tensor, PIDState, PIDTrace]
```

最小 PIDState 包含 e_prev、i_state 和可选 derivative-filter state；PIDTrace 至少记录 e、p_term、i_term、d_term、u_unsat、u_sat、saturated_high/low、anti-windup branch、dt 与 reset reason。对于 batch/action dimension，所有 state 和 limit 都要明确广播规则，不能让 [B]、[B,D]、标量 limit 静默错配。

在本页固定误差为：

$$e_k = r_k - y_k.$$

若项目使用 measurement-setpoint，必须把 P/I/D 与饱和方向判定一起翻转；只改 P 项会让 conditional integration 在错误的 rail 上冻结。

3. 离散 PID 主体

令 i_state 是已经乘过 K_i 的积分输出贡献，最简单的 backward-difference derivative 为：

$$u_k^{\text{unsat}} = K_p e_k + i_k + K_d \frac{e_k - e_{k-1}}{\Delta t}, \qquad u_k = \operatorname{clip}\left(u_k^{\text{unsat}}, u_{\min}, u_{\max}\right).$$

变量含义：`r/y` 是 setpoint/measurement，`K_p/K_i/K_d` 是 gain，`Δt` 是本次更新周期，`u_unsat` 是理想输出，`u` 是 clamp 后输出。实现映射是 action/controller tensor 的逐元素计算；`trace` 必须保留 `u_unsat` 与 `u`，否则无法诊断“模型输出大”还是“执行器饱和”。

工程上 derivative 还可能使用 measurement derivative、低通滤波或 dirty-derivative；这些选择必须和噪声、采样抖动、reset 语义一起冻结，不能在面试中把一行差分公式说成所有 PID 的唯一实现。

4. Conditional integration

未饱和时正常积分；已上饱和且误差继续把输出推向上 rail 时冻结；已上饱和但误差反向、正在把输出拉回可实现区间时允许积分。概念式为：

$$i_{k+1} = i_k + K_i \Delta t\, e_k\, g_k, \qquad g_k = \mathbf{1}[\neg \mathrm{sat}_k \ \vee \ \text{error drives back from the rail}].$$

关键不是背 `g_k`，而是先判断饱和方向：

- `u_unsat > u_max` 且 `e>0`：继续向上推，冻结；
- `u_unsat < u_min` 且 `e<0`：继续向下推，冻结；
- 已上饱和但 `e<0`，或已下饱和但 `e>0`：允许积分消除 wind-up。

如果 limits 是向量，判断必须逐 action dimension 完成；不能用整个 batch 的 `any(saturated)` 冻结所有维度。

5. Back-calculation

把执行器实际输出反馈到积分状态：

$$i_{k+1} = i_k + \Delta t \left(K_i e_k + \frac{u_k - u_k^{\mathrm{unsat}}}{T_t} \right), \qquad T_t > 0.$$

`u-u_unsat` 在饱和时把积分状态拉回可实现输出；`T_t` 小则回算快但对噪声和 jitter 更敏感，`T_t` 大则恢复慢。这里 `i_state` 的单位是 output contribution，因此 `K_i e` 与 `(u-u_unsat)/T_t` 必须同量纲。若代码把积分器存成未经 `K_i` 的 `∫e dt`，公式和 reset/limit 的位置都要相应改写。

ROS 2 文档支持 `back_calculation`、`conditional_integration`、`none` 和 tracking-time 参数名；本页的离散更新是可测试的实现合同，不声称它等同于某个仓库的默认内部源码。

6. Clamp、IZone、reset 的边界

机制	作用	不能替代
`u_clamp`	限制最终 actuator command	防止 `i_state` 继续增长
`i_clamp` / integrator range	限制积分贡献的范围	具有动态回算的 back-calculation
conditional integration	根据饱和方向决定是否累加	对错误 error sign 的修复
back-calculation	用 `u_sat-u_unsat` 动态拉回积分	正确选择 tracking time 或安全 limit
IZone	误差超过阈值时禁用积分	饱和方向判断与执行器反馈
`reset()`	清除积分、历史误差、滤波器状态	处理错误的 episode/lease ownership

VLA/robot action 需要在 reset 时记录 `episode_id`、`robot_id` 或 action epoch；否则旧 observation 的 derivative/integral state 可能泄漏到新任务。reset 的触发条件应包括 controller restart、robot lease epoch 变化、model/schema revision 不兼容与安全 stop 后重新 armed。

7. 参考实现（带边界注释）

```
def pid_step(setpoint, measurement, state, gains, limits, dt):
    # Reject invalid time; silently substituting a nominal period hides jitter.
    if not torch.isfinite(dt).all() or (dt <= 0).any():
        raise ValueError("dt must be finite and positive")

    e = setpoint - measurement
    de = (e - state.e_prev) / dt
    p_term = gains.kp * e
    d_term = gains.kd * de
    u_unsat = p_term + state.i_state + d_term
    u = torch.clamp(u_unsat, limits.u_min, limits.u_max)

    high = u_unsat > limits.u_max
    low = u_unsat < limits.u_min
    if limits.antiwindup == "conditional_integration":
        drives_back = (high & (e < 0)) | (low & (e > 0))
        allow_i = ~(high | low) | drives_back
        next_i = state.i_state + gains.ki * dt * e * allow_i
    elif limits.antiwindup == "back_calculation":
        next_i = state.i_state + dt * (
            gains.ki * e + (u - u_unsat) / limits.tracking_time_constant
```

```
        )
    elif limits.antiwindup == "none":
        next_i = state.i_state + gains.ki * dt * e
    else:
        raise ValueError("unknown anti-windup strategy")

    next_i = torch.clamp(next_i, limits.i_min, limits.i_max)
    next_state = PIDState(e_prev=e, i_state=next_i)
    trace = PIDTrace(e=e, u_unsat=u_unsat, u_sat=u,
                     saturated_high=high, saturated_low=low,
                     antiwindup=limits.antiwindup, dt=dt)
    return u, next_state, trace
```

这段伪代码故意把 `u_unsat/u_sat`、方向 mask 和 `antiwindup` branch 暴露出来。生产代码还应处理 derivative kick、NaN/Inf、rate limit、command timeout、batch broadcasting、device/dtype 与 reset reason；不能只返回一个 action tensor。

8. 测试与 first-divergence 诊断

Probe	期望	首个分叉意味着什么
`dt=0/negative/NaN`	reject 或显式 fallback	时间合同/输入校验缺失
`K_i=0`、`K_d=0`	P-only/D-only 行为可解释	gain 或 state 路径错
正误差上饱和	conditional freeze；back-calc 回拉	饱和方向或公式错
误差反向	积分能消除 wind-up	把所有饱和都冻结
`u_min=u_max` / infinite clamp	无 NaN，trace 可解释	边界/广播错
reset 后同一输入	与新 episode 首步一致	state 泄漏
continuous input	仅显式启用时走 shortest path	角度 wrap 误用
fixed replay	step-by-step trace 相同	非确定性或未记录状态

诊断顺序固定为：`dt/finite → error sign → state/reset → u_unsat → clamp/limit → anti-windup branch → dtype/device → downstream controller ack`。不要从最终 action 失败直接跳到“PID 参数不对”。

9. VLA 映射与项目边界

连续 VLA action 可能是归一化的 joint delta、pose residual 或 velocity command。PID 前必须确认 action schema、单位、normalizer revision、joint order、limit 与 controller rate；否则同一个 `K_i` 在 normalized space 和 physical space 不是同一语义。PID 只处理已定义的 setpoint/measurement/control interface，不替代 perception、IK、collision check、lease fencing、独立 e-stop 或 robot success scorer。

现有五项目可作为追问 hook：OpenPiVLA 的 action shape/normalizer/parser、Robot Harness 的 timeout/compensation、WAM 的 controller/chunk diagnostic、OPD 的 route/teacher schedule、UWM/LIBERO 的 launcher/task mapping。但当前没有 CODE-007 local implementation、joint feedback trace、anti-windup tuning log、matched controller ablation 或 candidate ownership artifact，因此安全回答是“我能实现并测试这个合同”，不能说“我已在项目中完成并验证了 anti-windup PID”。

10. 30 秒回答与高压追问

30 秒回答：先固定误差符号、`dt`、state 和 output limits。算 `u_unsat=P+I+D`，再得到 `u_sat=clip(u_unsat)`。积分 anti-windup 有两种常讲法：conditional integration 在饱和且误差继续推向 rail 时冻结、反向时放行；back-calculation 用 `(u_sat-u_unsat)/T_t` 把积分拉回。`u_clamp`、`i_clamp`、IZone、reset 不是一回事。代码要记录 `u_unsat/u_sat/branch`，测 `dt` 异常、正反向饱和、reset、边界和 replay；这只证明更新合同，不等于真机闭环成功。

高压追问：

- 为什么 `mean(mask*loss)` 的问题与 PID 积分饱和有相似性？——两者都需要明确“有效贡献”的 denominator/state，而不是只看最终标量。
- back-calculation 的 `T_t` 如何调？——先固定单位和采样周期，做 matched step/reversal sweep，看 settling、overshoot、noise sensitivity 与 saturation duration；没有 trace 不报收益。
- action 已 normalized 怎么办？——先绑定 normalizer/schema revision，再决定 PID 在 normalized 还是 physical space 工作，limit 与 gain 必须在同一量纲。
- controller timeout 或 lease epoch 变化时是否继续积累？——停止/失租约时拒绝旧 action，并按 episode/epoch contract reset 或冻结，不能让 stale state 继续驱动输出。

11. 结论与下一步

PID anti-windup 的高分答案是把公式、状态、饱和方向、时间单位、reset、trace 和测试 oracle 绑定起来，并明确它与 VLA action schema、controller safety 和项目证据的边界。当前 source contract 已冻结；下一步只生成 CODE - VLA - 007 题卡，仍需另一个 bounded slice，且不改变 `unseen`。

References

1. CODE-007 Source Freeze。
2. ROS 2 control PID controller userdoc。
3. WPILib PIDController documentation。
4. Feedback Systems。
5. 本仓库 VOL 01 advanced-control fact review 与五项目 evidence inventories。

Part 8 / 25 · `interview_materials/vla/knowledge/vol_08_code008_bellman_td0.html` · SHA-256 `FC210E0FF654F670A667948414213191B221E04FEF27B80B0781E28168013C7A`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`fc210e0ff654f670a667948414213191b221e04fef27b80b0781e28168013c7a`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_08_code008_bellman_td0.md`

VOL 08 CODE-008：Bellman TD(0) Update

状态：`drafted / CODE-VLA-008 / knowledge_11_of_phase / card_pending / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：CODE-008 Source Freeze。

问题（Question）

Bellman TD(0) 手撕题的高危点不是公式本身，而是把真实终止、time-limit 截断、success、timeout、bootstrap 和 batch mask 混成一个 `done`。高分答案必须能说明 transition schema、target detach、discount、shape/reduction、异常样本以及它们如何映射到 VLA/robot rollout。

1. 直觉与专业定义

TD(0) 不等整条 episode 结束才算 return，而是用当前 transition 的 reward 加一步 bootstrap value 修正估计：

- **真实终止**：环境进入吸收状态，未来价值通常为 0；
- **truncated**：外部时间限制、预算或采样窗口结束，MDP 可能没有终止，通常仍应使用 `obs_{t+1}` bootstrap；
- **success**：业务/评估谓词，不自动等价于 termination；
- **valid**：这条 transition 是否允许进入 learner reduction。

Gymnasium 将 `terminated` 与 `truncated` 分开，VOL 01/VOL 02 fact review 也将 timeout bootstrap 作为必须保留的边界。它们是语义合同，不是候选人的项目实现证据。

2. Transition schema

```
(obs_t, action_t, reward_t, obs_tp1,
 terminated_t, truncated_t, success_t,
 discount_t, valid_t, episode_id, step_id)
```

`episode_id/step_id` 用于防止跨 episode 拼接；`obs_tp1` 缺失时不能假设 truncation 可以安全 bootstrap。进入 learner 前不要写成无来源的 `done = terminated or truncated`，除非你明确说明这是 rollout 停止标记，而不是 Bellman terminal mask。

3. State-value TD(0) target

令 `V_next=V_target(obs_tp1)`，真实终止 mask 为 `m_term`：

$$y_t = r_t + \gamma_t \left(1 - m_t^{\text{term}}\right) \text{stopgrad}[V_{next}], \qquad \delta_t = y_t - V(obs_t).$$

变量解释：`r_t` 是即时 reward，`γ_t` 是折扣，`m_term=1` 只代表真实终止，`V_next` 是下一状态 value，`δ_t` 是 TD residual。实现映射是 `target` 分支 detached，只让 `V(obs_t)` 产生梯度；`valid_t` 再决定这条样本是否进入 loss。

当 `terminated=False,truncated=True` 时，通常 `m_term=0`，保留 timeout transition 的 bootstrap；当 `terminated=True` 时，`m_term=1`，不 bootstrap。若环境定义有特殊 absorbing-state 语义，必须把该协议写进 source/benchmark manifest，而不是用一个全局规则猜测。

4. Q-learning 变体

若 learner 是 Q function，next value 不是任意地“取最大”：

$$y_t^Q = r_t + \gamma_t(1 - m_t^{\text{term}}) \,\text{stopgrad}\left[\max_{a'} Q_{target}(obs_{t+1}, a')\right].$$

Double Q、actor-critic、distributional value 或 action-conditioned VLA value head 会替换 `V_next` 的构造，但仍需保留 terminal mask、timeout 语义、target detach 与 reduction 合同。TD target 不是完整算法名，也不自动保证稳定、保守或安全。

5. Batch、dtype 与 reduction

维度/字段	合同	高危错误
`reward`, `gamma`	与 value 同 device/dtype；`gamma=0` 完全关掉 bootstrap	Python float 隐式转 device、负 gamma 未拒绝
`terminated`, `truncated`	bool 或明确 0/1，分别保留来源	合并成 `done`，timeout 被当 terminal
`valid`	控制 loss denominator；全零是数据事件	all-invalid 返回漂亮的零 loss
`[B]` / `[B,T]`	明确 broadcast/flatten、episode boundary	`[B,T]` mask 错位到 next state
target	`stopgrad`，必要时 target network	target 和 prediction 一起反传
NaN/Inf	在 reduction 前 reject/mark invalid	mask 后才检查，异常被隐藏

masked TD loss 可写为：

$$L = \frac{\sum_i v_i \, \rho(\delta_i)}{\sum_i v_i}, \qquad v_i \in \{0, 1\},$$

其中 `ρ` 可是平方、Huber 或其他明确 loss；分母要和报告口径一致。分布式训练应先聚合 numerator/denominator，再相除，不能对各 rank mean 做无权平均。

6. 参考实现（带边界注释）

```
def td0_target(reward, value_next, gamma, terminated, truncated, valid):
    # Keep flags separate; truncation is not automatically terminal.
    if (terminated & truncated).any():
        raise ValueError("ambiguous transition: both flags are true")
    if not torch.isfinite(reward).all() or not torch.isfinite(gamma).all():
        raise ValueError("non-finite reward/discount")
    if not valid.any():
        raise ValueError("all transitions are invalid")

    terminal_mask = terminated.to(value_next.dtype)
    target = reward + gamma * (1.0 - terminal_mask) * value_next.detach()
    return target, valid

def td0_loss(value, target, valid):
    residual = target.detach() - value
    weight = valid.to(residual.dtype)
    denom = weight.sum()
    if denom.item() == 0:
        raise ValueError("empty TD batch")
    return (weight * residual.square()).sum() / denom
```

这段实现刻意不把 `truncated` 丢掉：它可以进入日志、replay 与评估，但不直接进入 terminal mask。真实代码还需要检查 `obs_tp1` provenance、episode/step order、target network revision、reward scale、AMP/fp32 accumulation 与 distributed counts。

7. Deterministic probes 与诊断顺序

Probe	预期	首个分叉
P1 terminal	不 bootstrap	terminal mask/flag 错
P2 truncation	有 next state 时仍 bootstrap	timeout 被错误当 terminal
P3 both flags	reject 或显式 precedence	schema/collector 合同不清
P4 `gamma=0`	target=reward	discount broadcast/dtype 错
P5 `[B]` vs `[B,T]`	逐元素一致	flatten/next-state 对齐错
P6 all-invalid	reject/skip	伪造零 loss、分母错
P7 detach	target 无梯度	autograd graph 错
P8 NaN/Inf	reduction 前暴露	异常被 mask 掩盖

诊断顺序固定为：`transition lineage → flag semantics → obs_tp1 availability → terminal mask → gamma/dtype → target detach → shape/broadcast → valid denominator → optimizer/target revision`。不要只看 loss 下降就断言 TD 实现正确。

8. VLA 与项目真实性边界

在 VLA/robot rollout 中，`success`、controller stop、time-limit、policy timeout 和 environment termination 可能来自不同层。action chunk 被截断不等于任务失败；网络 timeout 不等于 MDP terminal；success scorer 也不一定写入 value target。必须保留 `episode_id/step_id`，`capture/apply` 时间和 model/schema revision，才能复查 transition 是否错位。

VOL 02 的本地 Harness 只能作为 offline grouped reward-weighted update、GAE/timeout 讨论的 hook；WAM/UWM/OPD/OpenPiVLA 的 reward、rollout 或 summary 不能替代 transition-level artifact。当前没有 CODE-008 local implementation、gradient parity、replay bundle、target revision 或 candidate ownership，因此安全表达是“我能实现并审计 TD(0) 合同”，不能声称 value 收敛或真机成功。

9. 30 秒回答与高压追问

30 秒回答：先把 transition 里的 `terminated` 和 `truncated` 分开。TD(0) target 是 `r+γ(1-terminated)V_next`，真实 terminal 不 bootstrap，time-limit truncation 通常仍 bootstrap。target 要 detach，`valid` 控制 denominator；检查 gamma=0、batch shape、all-invalid、NaN 和 flags 同时为 true。success/timeout/controller stop 不能自动当 terminal。没有 transition/replay/gradient artifact 时，我只讲实现合同，不报 value 或机器人收益。

高压追问：为什么 truncation 通常仍 bootstrap？如果 `obs_tp1` 缺失怎么办？`success=True` 是否一定 terminal？为什么 target detach？Q-learning 的 max 放在哪里？两 flags 同时 true 怎么办？VLA action chunk timeout 如何进入 schema？分布式 loss 为什么要聚合 numerator/denominator？

10. 结论与下一步

TD(0) 的高分实现是：显式 transition schema，terminal mask 只来自真实 termination，timeout bootstrap 保留 provenance，target detach，valid denominator 和 batch shape 可审计。当前 CODE - VLA - 008 source contract 已冻结；下一步只生成 atomic card，仍需独立 bounded slice，题卡保持 `unseen`。

References

1. CODE-008 Source Freeze。
2. Sutton & Barto, Reinforcement Learning: An Introduction。
3. Gymnasium, Handling Time Limits。
4. 本仓库 VOL 01 Batch 2 review 与 VOL 02 review。

Part 9 / 25 · `interview_materials/vla/knowledge/vol_08_sys001_vla_training_eval_platform.html` · SHA-256 `F0F57B765B3E313B6F9E1B32321A063DABBDB1D2248DC45EDA56C7B01EBF203C`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`f0f57b765b3e313b6f9e1b32321a063dabbdb1d2248dc45eda56c7b01ebf203c`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\knowledge\vol_08_sys001_vla_training_eval_platform.md`

VOL 08 SYS-001：可复现的 VLA 训练—评估平台

状态：`drafted / SYS-VLA-001 / knowledge_1_of_phase / card_synced / WARN` 报告日期：2026-07-18 证据截止：2026-07-18 来源入口：SYS-001 Source Freeze。

问题（Question）

面试中的“设计一个 VLA 训练/评估平台”不是画几个服务框，而是要保证：数据、模型、processor/normalizer、checkpoint、评估协议、rollout artifact 和结果解释能被同一个 lineage graph 串起来。否则训练 loss、offline proxy、simulator success 和真机结果会互相污染。

本页回答：

1. 怎样定义 dataset、run、bundle、evaluation 和 rollout contract?
2. 如何把 determinism、checkpoint lineage、processor/normalizer 与 protocol 固定在同一 run?
3. 怎样隔离 offline、simulator、real rollout 和不同 success predicate?
4. 哪些证据只能支持接口，哪些证据才支持结果或个人贡献?

知识地图位置与前置依赖

```
VOL 05 ENG-013 reproducibility + ENG-017 observability + ENG-021 packaging
                                ↓
dataset/schema → train run → immutable bundle → offline eval
                                ↓
                          simulator / real rollout
                                ↓
                        episode trace + metric manifest
```

前置：ENG-013 Reproducibility、ENG-017 Observability、ENG-021 Packaging。

1. 30 秒直觉：让每个结果都能回到输入

可信平台至少有五层：

```
数据快照 → 训练 run → 可加载 bundle → 固定协议评估 → episode/安全 artifact
```

每层都产出 manifest 和 hash。只保存 checkpoint 路径、平均 loss 或成功率截图，无法回答“使用了哪个数据、normalizer、init、scorer 和代码版本”。

2. Dataset contract：目录不是版本

每个样本至少绑定：

```
episode_id / step_id / embodiment / task
camera_set / image_size / calibration_revision
state_schema / action_schema / frame / unit
timestamp / action_age / valid_mask
dataset_revision / source_hash / split
```

RLDS/Open X 的 episode/step 组织是 schema 参照，但不证明本地数据已经无泄漏。对 VLA，`action_dim`、chunk horizon、absolute/velocity frame、gripper convention、camera order 和 padding mask 必须显式记录。

`train/val/raw_holdout/rollout` 不应只靠文件夹名区分。manifest 要记录 episode、scene、object、operator 或 image hash 的分组规则；相邻 window 不能跨 train/holdout。holdout scorer 也要冻结版本。

3. Run contract：一次训练到底是什么

可用下式表达 run identity：

$$run_id = H(code_rev, config_hash, dataset_manifest, seed, rank/world_size, hardware, model_rev, processor_rev, normalizer_rev, res$$

变量是代码、配置、数据、随机源、分布式拓扑、硬件、模型和恢复来源；`H` 是身份索引，不是 bitwise identical 的保证。PyTorch reproducibility 文档明确：版本、平台和非确定性算子仍会改变结果。

训练 step 至少拆成：

```
data_wait → transfer → forward → backward → communication
→ optimizer → checkpoint → logging
```

VLA 还要记录 `valid/action mask count、grad norm、AMP scale/overflow、effective batch、checkpoint save latency`。单卡 smoke 不能推出多 rank parity 或完整吞吐。

4. Bundle contract：checkpoint path 不等于模型

组件	作用	缺失风险
model weights/config	参数与结构	load 错模型
processor/tokenizer	图像、文本、特殊 token	输入分布改变
normalizer	action/state scale	action 幅值/轴错误
observation/action schema	camera、frame、unit、H/D、mask	silent reshape
code/dependency revision	decode/denorm 逻辑	lineage 断裂
metadata/integrity hash	step、source、binary identity	远端路径冒充权重

最小 load gate：固定 sample 完成 processor→model→decode→denorm→schema validation roundtrip，并保存 shape、dtype、finite、range 和 parser 结果。缺 normalizer 或 action schema 时，只能做静态审计，不能进入 rollout。

5. Evaluation / rollout contract：同名任务不等于同一指标

每次评估冻结：

```
task / scene-or-init / seed / horizon / control-hz / action-repeat
checkpoint+bundle revision / success predicate / scorer revision
denominator / terminal reason / metric namespace
```

namespace	例子	能说明什么
`train.*`	loss、grad、throughput	优化与工程观测
`offline.*`	MSE、NLL、raw holdout	代理或输入/标签指标
`sim.*`	episode success、SPL、collision	固定 simulator protocol 的闭环结果
`real.*`	safety stop、intervention、success	有安全日志的真实 rollout
`proxy.*`	PSNR/SSIM、视觉 embedding	离线代理，不等于动作成功

每个 rollout episode 保存 `episode_id、task、init、seed、bundle_revision、observation/action trace、action_age、latency、controller status、terminal reason、success predicate inputs、scorer revision`。只有远端 summary JSON 时，应标为 `user-confirmed` 或 `report-supported`。

6. 可审计 pipeline 伪代码

```
def run_vla_pipeline(dataset_manifest, config, bundle, protocol):
    # Freeze identity and split before reading training samples.
    verify_manifest(dataset_manifest, protocol.split_contract)
    run_id = make_run_id(config, dataset_manifest, bundle, protocol)
    for batch in deterministic_loader(dataset_manifest, config.seed):
        # Validate action/frame/unit/mask before the model sees the batch.
        batch = validate_schema_and_masks(batch, bundle.action_schema)
        loss, telemetry = train_step(batch, bundle.model, config)
        record_train_event(run_id, loss, telemetry)
    # Save an immutable bundle before any online rollout.
    save_immutable_bundle(bundle, run_id)
    offline = eval_holdout(bundle, protocol.offline)
    sim = rollout_sim(bundle, protocol.sim) if protocol.sim else None
    real = rollout_real(bundle, protocol.real) if protocol.real else None
    write_metric_manifest(run_id, offline=offline, sim=sim, real=real)
    return run_id
```

关键点是先冻结身份，再校验 batch/schema，训练后保存 immutable bundle，最后把 offline/sim/real 分开记录。伪代码是系统设计合同，不是任何现有项目的完整实现。

7. Failure tree 与恢复策略

失败层	第一证据	常见误判
data/schema	sample manifest、roundtrip、hash	归因模型结构
transform/normalizer	processor/normalizer revision、finite/range	归因 checkpoint
train/runtime	telemetry、rank log、DCP marker	只看最终 loss
bundle/load	immutable hash、compatibility smoke	只比较路径
protocol	eval tuple、scorer/init/seed	说成模型 uplift
controller/safety	action trace、latency、watchdog	归因 policy loss

恢复顺序是：保留失败 run 和 marker，定位第一失败层；不要覆盖 latest checkpoint，不要自动把旧 bundle 送入新 protocol，也不要在没有 paired run 时恢复强因果叙事。

8. 项目映射与真实性边界

- **OpenPiVLA**：可连接 formal trainer、recipe、action packaging、timeout hook；不能把 archive-only 结果或远端路径写成完整平台。
- **UWM/LIBERO**：可连接 task mapping、fixed/reset protocol、normalizer/action shape、DataLoader stall；186/250 仍缺本地 raw episode bundle，RoboCasa success 未验证。
- **WAM**：可连接 rollout manifest、controller/horizon/camera provenance；checkpoint、target metric、ownership 仍缺。
- **Robot Harness**：可连接 schema、timeout/cancel/compensation；不等于 VLA lineage 或 inference batching。
- **OPD**：可连接 split、raw holdout、strict scorer、point/box/text schema；不等于 robot rollout。

9. 结论与下一步

VLA 平台设计的核心不是堆服务，而是用 manifest、immutable bundle、固定 protocol 和逐 episode artifact 把“输入—训练—模型—评估—执行—结果”连成可追溯图。公开来源和本仓库 ENG-013/021 足以冻结机制，但当前五个项目仍缺统一 registry、checkpoint/normalizer hash、raw rollout 与 ownership ledger，因此保持 `WARN`。

下一步：保持 `SYS-VLA-001` 题卡为 `unseen`，进行一次系统设计口述，或补充一个平台 artifact；之后再做 VOL 08 hand-coding 或第二个 system-design slice。

References

1. SYS-001 Source Freeze。
2. PyTorch Distributed Checkpoint 与 Reproducibility。
3. Open X-Embodiment、LIBERO、OpenPI policy interface。
4. 本仓库 ENG-013、ENG-017、ENG-021 与五项目 evidence inventory。

Part 10 / 25 · `interview_materials/vla/knowledge/vol_08_sys002_policy_serving_realtime_control_loop.html` · SHA-256 `6EC2068A01051EAB334B2C87D63524BAADFC8B23759A7CA1F78FA197147AFD48`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`6ec2068a01051eab334b2c87d63524baadfc8b23759a7ca1f78fa197147afd48`；生成自：`interview_materials\vla\knowledge\vol_08_sys002_policy_serving_realtime_control_loop.md`

VOL 08 SYS-002：Policy Serving 与 Real-time Control Loop

状态：`drafted / SYS-VLA-002 / knowledge_8_of_phase / card_1_of_1_unseen / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：SYS-VLA-002 Source Freeze。

问题（Question）

怎样把 VLA policy 接入实时控制环，同时准确区分“能返回动作的 WebSocket demo”和“可审计、可降级、可验证的 policy service”？面试回答至少要覆盖 request/response schema、状态所有权、action chunk/replan、端到端 deadline、admission/backpressure、batching、controller 与 fail-safe 边界。

本页只做 source freeze 的教学转译。OpenPI 的 frozen revision 是最小 serving baseline；下面的 request envelope、deadline propagation、bounded admission 和 controller watchdog 是系统设计合同或验证要求，不能倒写成 OpenPI 已有能力，也不能写成个人项目已经上线。

1. 直觉与专业定义

1.1 Policy serving

通俗地说，policy serving 是一个“把观测送给策略、把动作候选返回给控制器”的服务层。它负责模型、预处理和推理生命周期，但不自动拥有机器人控制权。

专业地说，serving contract 是一个带身份、版本、时间戳和动作语义的函数：`observation + task condition → action chunk + timing/validity metadata`。必须同时定义输入输出 schema、模型 revision、normalizer、动作 frame/unit、可执行期限和异常处理。

1.2 Observation/action schema

观测（observation）不只是图片。最小合同通常包括 camera frame、proprioception/state、语言任务、action history、capture timestamp 和 schema/model revision。动作（action）也不只是一个向量：需要声明 horizon (H)、每步维度 (D)、frame、unit、normalizer、controller mode，以及是否允许执行到下一次 replan。

frozen OpenPI server 的直接路径是“连接后发送 metadata；每次收到消息后调用 `policy.infer`，附加 server timing，再发送结果”。frozen client 是同步 `send → recv`。当前这些文件没有正式的 request ID、absolute deadline、action expiry 或 controller acknowledgement；这些字段只能作为后续设计建议。

1.3 Action chunk 与 replan

一次推理可以返回未来 (H) 行动作。consumer 在本地缓存 chunk，用 cursor 逐行消费，并在执行 (K) 行后重新请求，形成 receding-horizon 控制。OpenPI 的 `ActionChunkBroker` 和 LIBERO/DROID examples 直接支持这种 client-side 语义；但 `H`、`K` 是 example/config 选择，不是所有 VLA 的通用标准。

1.4 Deadline、action age 与 backpressure

deadline 是“这次结果最晚什么时候仍可用”的绝对时间合同；action age 是动作从 observation capture 到 controller apply 已经过的时间。backpressure 是负载超过服务能力时限制 admission、丢弃或降级请求，而不是让无界队列把延迟悄悄变成 stale action。

1.5 Soft real-time、hard real-time 与 controller boundary

soft real-time 允许少量 deadline miss，但要可测量、可告警、可降级；hard real-time 要求在明确上界内完成，通常还要处理 scheduler、page fault、memory lock、优先级和最大延迟。Python WebSocket + GPU inference 仅凭平均 `infer_ms` 不能证明 hard real-time。

policy server 只返回动作候选；controller apply、hold/stop、独立 watchdog 和 physical e-stop 是另一层状态机。RPC timeout 或 task cancellation 不等于已经撤回发送中的控制命令，更不等于机器人已经停止。

2. 三层架构与接口合同

2.1 Server、client/broker、controller 的状态所有权

层	直接持有的状态	必须回答的追问	frozen OpenPI 证据边界
Policy server	checkpoint、policy object、输入/输出 transform、连接 metadata、server-local timing	revision 切换、warmup、readiness、模型错误如何传播	`websocket_policy_server.py` 支持 metadata 与 `recv → infer → send`；没有显式 queue、batch、deadline、watchdog
Client / broker	WebSocket 连接、latest observation、action chunk cache、cursor、replan cadence	stale response 如何丢弃、重连如何恢复、chunk 何时失效	`websocket_client_policy.py` 为同步 send/recv；`ActionChunkBroker` 支持 cursor/horizon reset；没有 request ID/expiry
Controller / robot	apply state、hold/stop、执行 ack、限位、e-stop、惯性与外部 side effect	timeout 后谁接管、命令是否 exactly-once、物理停止如何证明	这些状态在 OpenPI serving 文件之外；不能从 WebSocket response 推断

2.2 建议的 request envelope（设计推断）

下面是面试中应主动冻结的最小字段，而不是 frozen OpenPI 的现有 schema：

类别	字段示例	目的
identity	`request_id`, `session_id`, `robot_id`, `model_revision`, `schema_version`	去重、路由、回包关联、审计
observation	`capture_ts`, `clock_id`, `camera_ids`, `state`, `action_history`, `prompt`	计算 age，保证相机/状态语义一致
timing	`deadline_abs`, `enqueue_ts`, `infer_start`, `infer_end`, `send_ts`	分段预算、deadline miss 与 first divergence 定位
output	`action_chunk`, `H`, `D`, `frame`, `unit`, `normalizer_revision`, `valid_until`	parser、反归一化、切片与 freshness 检查
control	`replan_after_K`, `controller_mode`, `fallback_policy`	明确执行几步、失败后谁接管

回答时要把“字段建议”和“代码已存在字段”分开：OpenPI metadata 可以承载 identity，但 source freeze 没有证明上述字段已经齐全。

3. Timing 数学：从 server-local 到 controller apply

3.1 端到端 critical path

令 t_c 为 observation capture，t_e 为 enqueue，t_i 为 inference start，t_r 为 client response receive，t_p 为 postprocess 完成，t_a 为 controller apply。首个动作的端到端延迟可写成：

$$T_{\text{e2e}} = (t_e - t_c) + (t_i - t_e) + (t_r - t_i) + (t_p - t_r) + (t_a - t_p).$$

变量解释：五项分别对应 capture/serialize、queue 或 batch wait、网络与 infer、parse/denorm/safety、controller handoff。实现映射：OpenPI 直接给出的是 server-local `infer_ms` 与 `prev_total_ms`；完整分解需要在 envelope 和 controller trace 中增加 timestamp，因此这是基于 EVAL-013 与 source freeze 的 design instrumentation，而非现成 OpenPI 指标。

3.2 Action age

若一次请求在 t_c 捕获观测并返回 H 行 chunk，第 k 行在 $t_{\text{apply},k}$ 被执行，则：

$$A_k = t_{\text{apply},k} - t_c, \qquad k = 0, \dots, K-1.$$

H 是模型输出 horizon，$K \le H$ 是下一次 query 前执行的行数。它说明“首行延迟低”不代表后续行仍新鲜；实现上要记录 capture/apply timestamp，并在 `valid_until` 或 age threshold 超过时触发 replan/fallback。S5/S6/S7 只证明 client cursor 和 example cadence，不替你选择 freshness 阈值。

3.3 剩余 deadline budget

设调用方给出的绝对 deadline 为 (D)，任一层当前时刻为 (t)，则剩余预算为：

$$B_{\text{remain}} = D - t.$$

每层都重新设置一段完整 timeout 会把 queue、网络、推理和 controller 的预算重复计算。应传播同一个 deadline、统一 clock 语义，并把 `DEADLINE_EXCEEDED`、in-flight compute、stale response、controller hold/stop 分开记录。gRPC 文档支持 deadline propagation；将该语义接入 WebSocket VLA service 是 design inference。

4. Pipeline 与有注释伪代码

4.1 推荐的控制路径

```
# Each stage must emit trace fields; this is a design pipeline, not frozen OpenPI code.
observation capture
  → schema / timestamp validation
  → bounded admission
```

```
→ compatible queue or batch
→ policy inference
→ parse / denormalize / safety check
→ controller apply + acknowledgement
→ metrics, fallback and audit trace
```

每个箭头都应能在 trace 中定位；只记录 `infer_ms` 无法回答“延迟究竟发生在 queue、网络还是 controller”。

4.2 面试级伪代码

```
def online_rollout(env, policy_client, task, replan_steps):
    # Reset the environment and capture a timestamped observation.
    obs = env.reset(task)
    chunk = None
    cursor = 0

    while not obs.done:
        # Build the versioned request: images, state, prompt, and capture_ts.
        request = build_policy_observation(obs, task, capture_ts=clock.now())
        validate_schema(request)

        # Query only when no valid chunk remains or the replan cadence fires.
        if chunk is None or cursor >= replan_steps or chunk.is_expired(clock.now()):
            chunk = policy_client.infer(request, deadline_abs=request.deadline_abs)
            cursor = 0

        # Parse, denormalize, slice to controller dimensions, and enforce limits.
        action = output_transform(chunk.rows[cursor])
        safe_action = safety_check_or_fallback(action, obs)

        # Apply at the controller boundary; RPC success alone is not physical ack.
        obs = env.step(safe_action)
        record_trace(request, chunk, safe_action, obs, action_age=clock.now() - request.capture_ts)
        cursor += 1

    # Return the success predicate and failure trace, not just the last RPC status.
    return obs.info
```

伪代码中的 `build_policy_observation`、`output_transform` 和 `env.step` 必须替换为具体项目函数后才能声称 local-direct。OpenPI LIBERO example 直接证明了“执行 chunk 前 K 步并记录 success”的一种实现；本页的 deadline、expiry、safety 和 ack 是应补齐的系统设计层。

5. Admission、Backpressure 与 Batching

5.1 Bounded admission

CPython `asyncio.Queue(maxsize)` 在达到上限后会阻塞 `put`，而 queue API 本身没有 timeout 参数，因此必须在外层包裹 deadline-aware admission。面试中应明确选择：拒绝新请求、丢弃旧 observation、复用仍有效的 action chunk，还是切换到 fallback policy；不同选择会改变 freshness 与安全语义。

5.2 Dynamic batching 的兼容条件

Triton 文档中的 preferred batch、queue delay、queue size 和 timeout 只提供通用 batcher 机制。VLA 请求只有在 camera 数量/分辨率、prompt、state/action shape、normalizer、model revision 和 controller contract 兼容时才可 batch。把不同机器人请求“同时到达”直接拼成 batch，可能造成 padding、条件串线或错误反归一化。

5.3 Fresh observation 与 side-effect action

ROS 2 QoS 的 sensor profile 强调及时性、较小 keep-last depth 和可接受丢包；这可启发 observation queue 使用 latest-wins。但 action/command 有 side effect，不能无条件 latest-wins。旧 observation 可以丢，已确认要执行的 action 必须有 request ID、expiry、顺序和 controller ownership。

系统优化目标不是单独提高吞吐，而是同时约束 throughput、tail latency、action age、deadline miss rate 和 fallback rate。无界 queue 往往把 overload 变成“看似成功、实际过期”的动作。

6. Failure、降级与实时边界

故障	可能发生的层	正确处理	不能直接声称
connect/recv 失败	client/server/network	retry 或 fallback，并记录 request/session	“机器人已停止”
queue 满或 deadline 不足	admission/queue	reject、复用仍新鲜 chunk 或切 fallback	“吞吐变高”
inference exception / timeout	server/GPU	标记 request failed，丢弃 stale response，通知 controller	“in-flight kernel 已取消”
parse/denorm/limit 失败	client/controller boundary	fail-closed、hold/stop、独立 watchdog	“模型输出天然安全”
controller ack 缺失	controller/robot	超时进入 hold/stop，保留 e-stop 路径	“RPC 200 等于物理执行成功”

6.1 Cancel compute 不等于 physical stop

asyncio task cancellation 与 gRPC deadline propagation 只描述软件任务/RPC 的取消语义。GPU kernel、已发出的 controller command、机器人惯性和外部副作用可能继续存在。因此回答必须拆成四件事：`cancel compute`、`discard stale response`、`controller hold/stop`、`independent e-`

`stop`；后两项需要 controller/robot 直接证据。

6.2 Soft real-time 的验证门

若目标是 soft real-time，应至少收集 capture→apply 的 p50/p95/p99、最大 action age、deadline miss、queue depth、fallback rate 和 success predicate，并在固定 task/trial/seed 下对比。若声称 hard real-time，还要提供 scheduler、page-fault、memory-lock、priority 和最大延迟证据；ROS 2 pendulum real-time 文档可作为验证门参考，不能替代真实系统测量。

7. 实现证据与代码映射

证据层	文件/来源	可以回答	不可以回答
official-code	OpenPI `websocket_policy_server.py`、`websocket_client_policy.py`、`action_chunk_broker.py`	metadata、同步 send/recv、server-local timing、client cursor	production queue/batch/deadline/watchdog
official-code	OpenPI LIBERO/DROID examples	`replan_steps=5`、open-loop horizon、env step 与 success logging	通用 H/K、hard real-time、真机安全
local-evidence	`scripts/eval_lerobot_action_server.py`	WebSocket client、connect/infer timeout wrapper、offline action metric 入口	p99、cancel propagation、online controller
local-evidence	`scripts/run_aloha_sim_real_rollout.py`	in-process policy + `ActionChunkBroker` + env apply 的 rollout 入口	network policy server、production serving
local-evidence	`DEPLOYMENT.md`	official server/client/checkpoint 路径与部署说明	binary identity、SLO、watchdog、个人端到端 ownership
inference / design	CPython、Triton、gRPC、ROS 2 机制拼接	bounded admission、compatible batching、deadline、QoS 与 real-time profiling 的设计要求	这些机制已存在于 OpenPI 或个人项目

阅读项目时保持四层顺序：`public upstream → local-direct → inference/design → unverified`。上游代码被仓库引用，不等于候选人写过、维护过或上线过同样的系统。

8. 五个项目的真实性边界

项目	面试可用 hook	不能说的强 claim
OpenPiVLA	offline WebSocket action eval、in-process broker rollout、timeout/failure summary、H/D/parser 风险	production policy service、formal checkpoint serving、p99/fairness、cancel→physical stop、完整端到端 ownership
Robot Harness	tool polling timeout、cancel context、scope/lock cleanup、best-effort compensation	VLA inference batcher、action freshness、controller deadline、robot success
OPD	training/eval data route、strict scorer、多目标 checkpoint 选择	policy serving、closed-loop controller、fleet 调度
WAM	simulator rollout launcher、chunk/controller schema、diagnostic artifacts	network serving、hard real-time、真机 watchdog、已验证 checkpoint uplift
UWM / LIBERO	simulator protocol、action/state/normalizer、rollout evidence	production policy service、multi-robot serving、跨协议真机泛化

9. 面试回答模板与禁说清单

9.1 60 秒回答模板

“我先把系统拆成 server、client/broker、controller 三层。OpenPI 的 frozen baseline 是连接后 metadata，再按 observation→infer→action 返回；chunk cursor 和 replan 在 client/example 侧。若要做可审计的实时服务，我会在 request 中加入 request/session/robot/model/schema identity、capture timestamp、absolute deadline，在 admission 层做 bounded queue 和兼容 batch，在输出侧做 denormalize、limit、expiry 与 controller ack。指标不能只看 `infer_ms`，还要看 capture→apply、action age、deadline miss 和 fallback。最后，RPC timeout 只代表软件请求失败，physical stop 仍由 controller watchdog/e-stop 负责。对我的项目，我会把 OpenPiVLA 的 offline WebSocket 与 in-process broker 作为局部 evidence，不把它们升级成 production serving。”

9.2 不能说的强 claim

- “OpenPI 已经有 production-grade queue、dynamic batching、deadline propagation、multi-robot fairness 或 watchdog。”
- “`server_timing.infer_ms` 就是端到端 capture→controller apply latency。”
- “LIBERO 的 (H/K) 示例值是所有机器人和所有 VLA 的标准。”
- “timeout 或 exception 自动取消 GPU、清空旧动作并让机器人物理停止。”
- “引用 official code、目录路径或文档，就等于我个人拥有端到端实现和上线责任。”

10. 结论与下一步

高质量的 policy-serving 回答不是背 WebSocket API，而是能把 schema、状态所有权、chunk freshness、deadline budget、bounded overload、controller ack 和独立 e-stop 串成同一条证据链。当前 source gate 已证明 OpenPI 最小 serving path、client-side chunk 语义和跨来源系统设计机制；

尚未证明统一服务、load/fault test、E2E timestamp trace、formal checkpoint bundle 或候选人完整 ownership。

下一步：保持 `SYS-VLA-002` 题卡未生成；先用本页 60 秒模板做一次口述，或补充一个带 request ID/capture→apply timestamp 的最小 trace artifact。未完成真实口述前不创建 `coaching_state.md`，不启动 `SYS-VLA-003`。

参考文献（References）

1. SYS-VLA-002 Source Freeze：OpenPI、CPython、Triton、gRPC、ROS 2 的 frozen revisions 与 14/14 probes。
2. Physical Intelligence，OpenPI `openpi@15a9616a`：server/client、broker 与 LIBERO/DROID examples。
3. CPython，asyncio queue/tasks：bounded queue 与 cancellation 语义。
4. NVIDIA Triton，batcher/model configuration：preferred batch、queue delay/size 与 timeout policy。
5. gRPC，Deadlines；ROS 2，QoS design 与 pendulum real-time。
6. 本仓库 ENG-022 Policy Server Resilience、EVAL-013 Latency/Deadline/Jitter 与 OpenPiVLA Ownership Ledger。

Part 11 / 25 · `interview_materials/vla/knowledge/vol_08_sys003_robot_fleet_multitenant_gpu_scheduling.html` · SHA-256
`BD682F803F3CED854C7B4DD9C5205A244A52110F21B78F0933DF927D4A60649E`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`bd682f803f3ced854c7b4dd9c5205a244a52110f21b78f0933df927d4a60649e`；生成自：
`interview_materials\vla\knowledge\vol_08_sys003_robot_fleet_multitenant_gpu_scheduling.md`

VOL 08 SYS-003：Robot Fleet 与 Multi-tenant GPU Scheduling

状态：`drafted / SYS-VLA-003 / knowledge_9_of_phase / card_1_of_1_unseen / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：SYS-003 Source Freeze。

问题（Question）

当机器人数量、模型 revision、租户和 GPU 类型同时增长时，怎样把“一个 policy server”扩展成可控的 robot fleet？本页把 Kubernetes/Volcano/Ray/Triton/NVIDIA/ROS 2 的公开机制转译成面试系统设计语言，同时明确哪些只是 design inference，哪些没有个人项目证据。

不要把 fleet 设计理解成“加一个 scheduler”。真正的合同需要同时覆盖 tenant/robot/model identity、资源向量、placement、queue/fairness、preemption、model replica/cache、robot lease、rollout/canary、成本和 action freshness。

1. 直觉与专业定义

1.1 Fleet scheduler

通俗地说，fleet scheduler 决定“哪个机器人请求、哪个模型副本、哪块 GPU、什么时候执行”。它必须同时考虑资源够不够、请求是否兼容、租户是否超额、机器人 lease 是否有效，以及动作返回时是否已经过期。

专业地说，scheduler 是从请求集合 (mathcal{Q})、节点/副本集合 (mathcal{N}) 和资源向量 (R) 到 placement 与 admission 决策的控制平面。Kubernetes 官方 scheduler 暴露 QueueSort、Filter、Score、Reserve/Permit、Bind 等阶段；这些阶段描述 Pod placement，不自动理解 VLA 的 camera schema、normalizer、action age 或 controller safety。

1.2 Multi-tenancy、capacity、fairness、priority、freshness

这四个词不能混用：

- **capacity**：一个 tenant/queue 最多能占多少 GPU、MIG 或其他资源；
- **fairness**：长期资源占用或 queue share 是否按权重平衡；
- **priority**：关键请求在冲突时谁先服务；
- **freshness**：机器人动作是否仍在 deadline 和 action-age 合同内。

Volcano 的 fair-share/capacity/queue-priority 设计分别提供 weight、guarantee/capability/deserved、priority/reclaim 机制；它们不能自动推出 VLA 的 deadline fairness。让高优先级离线训练抢占 serving GPU，可能改善训练等待，却恶化动作新鲜度。

1.3 Logical resource 与 physical resource

Kubernetes device plugin、Ray logical/custom resources 和 Triton named resources 都能表达“需要某类资源”。但逻辑 reservation 不等于显存/带宽隔离；NVIDIA time-slicing 或 MIG 还要配合 workload、driver、memory、tail latency 和 failure 观测，不能只看 GPU utilization。

2. Fleet identity 与状态所有权

2.1 最小 identity/schema 合同

类别	字段示例	面试时要回答的边界
tenant	`tenant_id`, `quota_class`, `priority`, `budget`	配额、优先级、公平性、成本如何归属
robot	`robot_id`, `robot_type`, `capability_set`, `lease_epoch`, `controller_endpoint`	能力匹配、fencing、controller 路由
policy	`model_revision`, `processor_revision`, `normalizer_revision`, `action_schema_version`	warm cache 与 action 语义兼容
request	`request_id`, `session_id`, `capture_ts`, `deadline_abs`, `H`, `D`, `valid_until`	queue、batch、action age、幂等与 stale-response
placement	`gpu_type`, `mig_profile`, `memory_budget`, `network_zone`, `replica_id`	Filter/Score、locality、异构设备和成本

`robot_id` 不能仅凭 WebSocket 连接获得永久控制权。lease epoch、model/schema revision 和 controller acknowledgement 必须共同进入执行边界；这是 fleet 设计推断，不是当前 OpenPI 或五个项目已有字段。

2.2 State ownership 表

状态	owner	失效/恢复动作	证据边界
tenant quota / queue	scheduler control plane	reject、requeue、reclaim、downgrade	Kubernetes/Volcano 机制；无 VLA local implementation
model replica / GPU reservation	orchestrator + inference runtime	warm/cold、drain、reschedule、load smoke	Ray/Triton/Kubernetes mechanism；不证明 checkpoint lineage
robot lease	fleet control plane + controller	renew、epoch bump、fence、hold/stop	Lease/ROS lifecycle 启发的设计合同
action chunk	policy client/broker	expiry、stale drop、replan、fallback	SYS-002 OpenPI evidence；无 fleet-wide fencing proof
rollout/canary	release controller	promotion、per-tenant gate、rollback	design inference；项目只有局部 eval/rollout hooks

3. Capacity 与 placement 数学

3.1 Resource vector admission

将一个请求抽象为资源向量 (R=(g,m,c,n,s))，分别表示 GPU/accelerator、显存、CPU、网络和模型/缓存空间；节点剩余容量为 (C)。必要的 admission 条件可写为：

$$R_{\text{req}} \preceq C_{\text{node}}, \qquad \text{compatible}(M, S, N, \Gamma) = 1.$$

其中 (M) 是 model revision，(S) 是 action/schema，(N) 是 normalizer，(Gamma) 是 controller contract。逐维容量约束来自 Kubernetes/Ray/Triton 的资源机制；compatibility 是 VLA 系统设计推断，用来防止不同 shape、normalizer 或 controller mode 被错误合批。

3.2 多目标调度代价

单独最小化 GPU 空闲率是不够的。一个可用于面试澄清的设计目标是：

$$J = \alpha W_q + \beta L_{tail} + \gamma A_{action} + \delta M_{deadline} + \eta F_{fair} + \lambda C_{run}.$$

W_q 是 queue wait，L_{tail} 是 inference tail latency，A_{action} 是 action age，$M_{deadline}$ 是 deadline miss rate，F_{fair} 是 tenant share 偏差，C_{run} 是运行成本。权重不是上游默认值，必须由 task criticality、tenant SLA 和安全边界决定；应在固定 workload matrix 上做 matched A/B，而不是凭单次 utilization 报公平或性能提升。

4. Queue、Fairness 与 Preemption

4.1 Queue 层级

Kubernetes ResourceQuota 限制 namespace 的 aggregate resource/object consumption；PriorityClass/Preemption 允许高优先级 Pod 驱逐低优先级 Pod；Volcano queue 机制可表达 weight、capacity、guarantee、deserved 和 reclaim。面试回答应说明四个层级的组合：

1. admission 先检查 tenant quota 与 robot/model compatibility；
2. queue order 再考虑 priority、fair-share deficit 和 deadline budget；
3. placement 通过 Filter/Score 选择 GPU 类型、MIG profile、network zone 和 warm replica；
4. preemption 前先判断被抢占的是 offline job、可重试 inference，还是带物理 side effect 的 controller path。

4.2 Freshness 不是 fairness

旧 observation 可以丢弃，已确认要执行的 action 不能无条件 latest-wins。一个 tenant 获得“公平 GPU share”并不保证它的机器人动作在 deadline 内返回；相反，一个安全 critical robot 可能需要保留 capacity，但仍要受到 lease、controller ack 和 independent stop 的约束。

4.3 抢占与降级

推荐的 fail-closed 顺序是：拒绝新请求 → 复用仍新鲜的 action chunk → 切换已验证 fallback → drain/terminate 无副作用的 offline replica → 最后才考虑抢占；任何涉及 controller side effect 的请求都要先丢弃 stale response、提升 lease epoch，并由 controller 完成 hold/stop。Kubernetes preemption 和 Volcano reclaim 只提供调度机制，不证明这个安全顺序已由上游或个人项目实现。

5. Heterogeneous GPU、Replica 与 Cache

5.1 GPU placement

NVIDIA device plugin 支持 GPU discovery/health、MIG 与 CUDA time-slicing 入口；Ray 可用 logical/custom resource 和 placement group 表达 accelerator、memory、PACK/SPREAD 或 gang scheduling；Triton model configuration 以 `max_batch_size`、batch dimension、instance group 等约束 batching shape。

应把以下字段纳入 placement key：GPU 型号/MIG profile、显存预算、模型 revision、processor/normalizer revision、camera/prompt/action shape、network locality 和 controller mode。GPU 可见不代表模型能加载；加载成功也不代表 capture→apply tail latency 达标。

5.2 Replica 与 named resource

Triton rate limiter 的 named resource/count 适合表达“某 model instance 执行时占用多少份资源”；Ray Serve 的 Controller/Proxy/Replica 结构适合表达请求队列、replica 数量和 per-replica GPU reservation。两者都不能自动提供 tenant fairness、robot lease 或 physical safety。

模型 cache 至少应绑定 `model_revision + processor_revision + normalizer_revision + action_schema_version`，并区分 warm hit、cold load、OOM、driver mismatch 和 stale bundle。cache hit 只能改善启动或吞吐的一个环节，不能替代 checkpoint hash、load smoke 或 closed-loop evaluation。

6. Robot Lease、Lifecycle 与 Fencing

Kubernetes Lease 文档支持 lock、heartbeat 和 single-instance coordination；ROS 2 managed-node lifecycle 支持资源 acquisition/release 与 supervisory state。将其组合为 robot fleet lease 时，应额外冻结：

- `lease_epoch` 单调递增，旧 epoch 的 action 回包必须拒绝；
- renew 失败进入 `EXPIRED`，不能继续发送新 action；
- model revision、normalizer 和 controller endpoint 与 lease 绑定；
- controller acknowledgement、hold/stop 和 independent e-stop 由机器人侧拥有；
- reconnect 必须重新握手，不能复用旧 session 或旧 action chunk。

这是一套 fail-closed 设计要求。当前五个项目的 inventory/ledger 没有统一 robot lease、epoch fencing、controller ack 或多机器人 replay artifact，不能说已经实现。

7. Rollout、Canary、Cost 与观测

7.1 正常路径

```
# Design path; not an existing user fleet implementation.
request(identity + observation + deadline)
  → tenant quota / queue admission
  → compatible schema + model-revision grouping
  → Filter / Score / reserve GPU or placement group
  → warm replica + named rate-limiter resources
  → infer + parse + denormalize + safety boundary
  → robot lease/epoch + controller acknowledgement
  → per-tenant metrics, canary gate, replay and rollback record
```

7.2 Canary gate

模型 promotion 至少要绑定 model/processor/normalizer/action-schema revision、tenant/robot cohort、fixed task/seed、episode artifact、capture→apply tail latency、deadline miss、fallback、lease conflict 和 success predicate。不能只用 offline loss、GPU utilization 或一条 summary JSON 作为 fleet canary。

7.3 Cost 与 SLO

成本账本应分 GPU reservation、cold-start、network、storage/cache、preemption/retry 和机器人空闲时间；SLO 应分 queue wait、inference tail、action age、deadline miss、lease availability、fallback 和 task success。Ray Serve 的 replica queue、Triton rate limiter、Volcano queue 和 Kubernetes quota 是机制入口，不是本地 SLO 结果。

8. 面试级伪代码

```
def schedule_fleet_request(req, cluster, registry, clock):
    # 1. Reject stale or malformed identity before touching GPU or robot state.
    validate_identity(req.tenant_id, req.robot_id, req.model_revision)
    validate_schema_and_normalizer(req)

    # 2. Check quota, lease epoch, deadline budget, and side-effect class.
    if not quota_allows(req) or not lease_is_current(req):
        return reject_or_fallback(req, reason="admission")

    # 3. Keep only resources and replicas compatible with the VLA contract.
    candidates = filter_nodes(cluster, req.gpu_type, req.mig_profile, req.shape)
    candidates = score_for_locality_fairness_freshness(candidates, req)
    placement = reserve_or_queue(candidates, req.resource_vector)
    if placement is None:
        return reject_or_fallback(req, reason="capacity")
```

```
    # 4. Route to a warm model replica; the limiter controls instance occupancy.
    replica = acquire_replica(registry, req.model_revision, req.normalizer_revision)
    result = replica.infer(req, deadline_abs=req.deadline_abs)

    # 5. Fence old results and require controller acknowledgement at the edge.
    if result.lease_epoch != current_lease_epoch(req.robot_id):
        return discard_stale_result(req, result)
    action = denormalize_and_limit(result.action_chunk, req.normalizer_revision)
    return controller_apply_with_ack(req.robot_id, action, req.valid_until)
```

伪代码中的 `filter_nodes`、`reserve_or_queue`、`acquire_replica` 和 `controller_apply_with_ack` 需要具体系统实现后才能成为 local-direct。它展示的是面试设计顺序，不是 Kubernetes、Ray 或任何一个个人项目的现成函数。

9. 实现映射与项目真实性边界

证据层	可用映射	不可升级的 claim
official upstream	Kubernetes scheduler/quota/Lease、NVIDIA device plugin、Volcano queue、Triton limiter、Ray placement/Serve、ROS lifecycle	用户已部署 VLA fleet、SLO、公平性或 physical safety
local evidence	五项目 inventory/ownership ledger 的 trainer/eval/timeout/rollout hooks	统一 control plane、multi-tenant load test、GPU placement、lease fencing
design inference	identity/schema、multi-objective scheduler、cache key、canary、fencing	已有 runtime、checkpoint lineage、candidate end-to-end ownership
unverified	p99/p999、tenant fairness、cold-start uplift、robot fleet success	任何 production/hard-real-time/多机器人安全强结论

五项目具体 hook：

- OpenPiVLA：offline WebSocket、in-process broker、normalizer/action shape/parser 风险；没有 fleet scheduler 或 lease。
- Robot Harness：tool scope、lock、timeout、compensation、训练/eval route；不是 VLA inference scheduler。
- OPD：multi-expert route、teacher schedule、ZeRO/config、holdout；不是 multi-tenant serving。
- WAM：simulator launcher、chunk/controller schema、diagnostic trace；不是 heterogeneous GPU fleet。
- UWM/LIBERO：launcher、task mapping、fixed protocol、loader diagnosis；不是 production fleet 或 cross-robot serving。

10. 60 秒回答与禁说清单

10.1 60 秒回答模板

“我会把 fleet 拆成 identity、admission/queue、placement、replica、robot lease 和 rollout 六层。请求必须带 tenant、robot、model/processor/normalizer、schema、deadline 和 lease epoch；先检查 quota、兼容性和 freshness，再按 GPU/MIG、显存、network locality 与 warm replica 做 Filter/Score。fair-share、priority、capacity 和 freshness 分开衡量，不能只看 GPU utilization。模型 promotion 要绑定 cohort、episode、tail latency、deadline miss、fallback 和 success predicate。Lease 失效时提升 epoch、丢弃旧 action，controller 自己负责 ack/hold/stop/e-stop。Kubernetes、Volcano、Ray、Triton 只能作为机制来源；我当前项目只有局部 eval/timeout/rollout hooks，不能称作已经上线的 production fleet。”

10.2 不能说的强 claim

- “Kubernetes/Volcano/Ray/Triton 已经构成我的 production VLA fleet。”
- “GPU utilization 高就说明租户公平、动作 deadline 满足。”
- “GPU time-slicing、MIG 或 Pod preemption 天然提供显存隔离和物理安全。”
- “Lease renew 成功就等于 controller 已安全执行 action。”
- “Ray placement group、Triton rate limiter 或 Volcano fair-share 证明我的项目有 multi-robot load test。”
- “五个项目的局部脚本、配置或 Git author 等于我完成了 fleet control plane。”

11. 结论与下一步

fleet 系统设计的高分点是把资源调度和机器人语义连接起来：capacity 约束 GPU，fairness 约束长期 share，priority 决定冲突顺序，freshness 约束动作可执行性，lease/fencing 保护 robot ownership，canary/replay 保护 revision promotion。当前 source freeze 已核对 15 个公开机制和五项目证据缺口；尚未证明统一 fleet service、tenant/robot/model trace、GPU fairness A/B、model cache lineage、lease fencing、controller ack 或个人 ownership。

下一步：保持 `SYS-VLA-003` 题卡未生成；先用本页 60 秒模板做一次口述，或补一个 tenant/robot/model/request/placement trace artifact。未取得真实回答前不创建 `coaching_state.md`，不启动 `SYS-VLA-004`。

参考文献（References）

1. SYS-003 Source Freeze：15 个 pinned sources、local evidence SHA 与 15/15 probes。
2. Kubernetes scheduler、Priority/Preemption、ResourceQuota、Lease。
3. NVIDIA Kubernetes device plugin 与 Volcano fair-share/capacity/priority。
4. NVIDIA Triton model configuration/rate limiter。
5. Ray resources/placement groups/Serve 与 ROS 2 node lifecycle。
6. 本仓库 ENG-016、ENG-018、ENG-019、ENG-022 与 五项目 evidence inventories。

Part 12 / 25 · `interview_materials/vla/knowledge/vol_08_sys004_fault_recovery_safety_observability.html` · SHA-256 `03C062336EB705B4940B4F8579D2873F96B4C70EA007A480BC39FA75EFF0074F`

返回本卷目录 · 对应题卡

Canonical Markdown SHA-256：`03c062336eb705b4940b4f8579d2873f96b4c70ea007a480bc39fa75eff0074f`；生成自：`interview_materials\vla\knowledge\vol_08_sys004_fault_recovery_safety_observability.md`

VOL 08 SYS-004：Fault Recovery、Safety 与 Observability

状态：`drafted / SYS-VLA-004 / knowledge_12_of_phase / card_pending / WARN` 报告日期：2026-07-20 证据截止：2026-07-20 来源入口：SYS-004 Source Freeze。

问题（Question）

VLA policy、数据管线、机器人 controller 或 telemetry 出现 timeout、schema mismatch、stale action、节点失活或 rollout regression 时，如何先保护 side effect，再找出 first verified divergence，最后完成可验证的 degradation、rollback 与 replay？本页把 source freeze 转译成面试可讲的状态、事件、恢复和项目边界；它不是个人项目已经上线安全系统的证明。

1. 直觉与专业定义

1.1 先保护副作用，再解释原因

直觉上，故障处理不是“看到 error 就 restart”。机器人动作可能已经进入 controller，任何 retry、rollback 或 latest-wins 都可能产生新的 side effect。因此顺序应是：

```
发现异常 → 关闭新请求/丢弃 stale action → 记录事件
  → 找 first verified divergence → 选择 hold/fallback/rollback/replay
  → 用同一 identity 的证据验证恢复 → 才能释放降级状态
```

专业上，fault supervisor 需要把 serving、controller、lease、release 和 evidence 状态分离。ROS 2 lifecycle/`ros2_control` 提供状态与资源管理机制；Kubernetes probes/rollback 提供编排层信号；QoS 与 OpenTelemetry 提供通信和观测信号。把它们组合成 VLA 安全闭环是系统设计推断。

1.2 First verified divergence 不等于 root cause

- **symptom**：例如 deadline miss、action-age 超限、readiness 失败或 success 降低；它是现象。
- **first verified divergence**：最早能由日志、trace、schema diff、replay 或 ack 复现的协议偏离，例如 `capture_ts` 到 `infer_ts` 已超预算，或 action 的 `schema_revision` 与 controller 不一致。
- **root cause**：导致偏离的因果解释，例如 queue starvation、normalizer mismatch 或硬件错误；必须有干预/替换/重跑证据。
- **recovery outcome**：降级动作是否真的消除了故障，并且没有引入新的 side effect；不能由“进程重启成功”替代。

回答时把四层分开，避免把一个 warning 讲成 root cause 或 recovery success。

2. Fault state machine 与 owner

2.1 状态定义

状态	进入条件	主要 owner	允许动作	退出证据
`HEALTHY`	schema、deadline、lease、controller ack 和 telemetry 正常	serving + controller	正常 infer/apply	同一 identity 的连续健康窗口
`DEGRADED`	deadline/QoS/metric 异常但 side effect 仍受控	fault supervisor	限流、降低 horizon、切 verified fallback、拒绝新请求	freshness 与 health gate 恢复
`QUIESCING`	drain、rollback 或 lease epoch 变更	release/fleet controller	拒绝新请求、等待或取消 in-flight	in-flight 清零或逐项 ack
`HOLD`	stale action、controller fault 或未知状态	controller/safety layer	保持安全姿态、丢弃 action、等待恢复	controller hold ack 与 release reason
`STOPPED`	明确 stop/e-stop 或不可恢复故障	independent safety/controller	不再 apply action	独立 reset/inspection 记录
`RECOVERY`	已执行 restart/fallback/rollback/replay	fault supervisor	执行恢复并收集证据	fault-injection/rerun/audit bundle
`QUARANTINED`	revision、schema、lease 或证据不兼容	release/evidence controller	禁止继续 rollout，保留 artifact	compatibility review 与人工批准

这些状态是面试设计合同，不是 ROS 或 Kubernetes 自动提供的完整 robot-safety 状态机。尤其 `readiness=1` 只说明服务可以接流量，不说明 controller 可以安全 apply。

2.2 状态转换的负面边界

1. `HEALTHY → DEGRADED` 可以由一个确定的 health gate 触发，但不能仅凭 GPU utilization。
2. `DEGRADED → RECOVERY` 需要记录触发原因和当前 in-flight side effect；不能跳过 hold/fence。
3. `RECOVERY → HEALTHY` 必须有同 revision、同 schema、同 protocol 的验证窗口。
4. 任何 unknown lease/schema/controller 状态都应 fail-closed 到 `HOLD`、`STOPPED` 或 `QUARANTINED`，而不是继续 latest-wins。

3. Liveness、Readiness、Startup 与通信信号

3.1 三类 probe 不可混用

信号	回答的问题	失败时可做	不能推出
startup	进程/模型是否还在启动	暂缓 liveness/readiness、延长启动窗口	模型已 warm、controller 可用
liveness	进程是否仍应被重启	restart 或标记实例异常	restart 等于 recovery success
readiness	当前实例是否接收新流量	从 service/queue 摘除、drain	已安全 apply action
QoS deadline	传输/消费是否在 deadline 内	记录 miss、降级或丢弃 stale 数据	miss 的 root cause
liveliness	publisher/subscriber 是否仍被认为活跃	触发 reconnect/hold 策略	controller 已停止物理动作

Kubernetes probes 是 orchestration 机制，ROS 2 QoS 是通信语义；两者都要通过 `request_id`、时间戳和 revision 进入 incident event，才能和 VLA 的 capture→infer→parse→apply 链接起来。

3.2 Deadline 与 action age

对一条 action chunk，至少记录：

$$A_{\text{age}} = t_{\text{apply}} - t_{\text{capture}}, \qquad \text{fresh}(a) = \mathbf{1}[A_{\text{age}} \le D_{\text{valid}}].$$

`A_age` 是观测捕获到动作 apply 的年龄，`D_valid` 是动作协议允许的有效窗口。它映射到 capture/apply timestamp、chunk cursor 和 controller apply log；缺任一时间点时只能标记 unknown，不能把 timeout 直接解释为 MDP terminal 或任务失败。

4. Incident identity 与可观测性

4.1 最小事件 schema

```
(event_id, incident_id, request_id, session_id,
 robot_id, lease_epoch, model_revision, schema_revision,
 capture_ts, infer_ts, parse_ts, apply_deadline, apply_ts,
 stage, severity, symptom, first_verified_divergence,
 action_seq, controller_ack, recovery_attempt, evidence_refs)
```

字段职责：

- `request_id/session_id` 连接一次服务调用及其重试；
- `robot_id/lease_epoch` 防止旧连接或旧动作越过 fencing；
- `model/schema_revision` 绑定 checkpoint、processor、normalizer、action parser 和 controller contract；
- `capture_ts/infer_ts/parse_ts/apply_ts` 支持阶段级 latency 与 first divergence；
- `controller_ack` 必须区分收到 cancel、进入 hold 和停止执行；
- `evidence_refs` 指向不可变 trace、log、metric snapshot、replay bundle 或人工批准。

4.2 Traces、metrics、logs 的分工

信号	适合回答	常见误区
trace	一次 request 从 capture 到 apply 的跨服务路径	trace 存在不代表每个 side effect 都被采样
metric	p95/p99 latency、deadline miss、queue depth、fallback rate、lease conflict	平均值掩盖 tail 或分母不一致
log	schema diff、异常栈、controller ack、rollback reason、人工批准	有日志不等于 root cause 已确认
baggage/identity	把 tenant/robot/model/session 信息跨边界传递	identity 未校验会污染 incident correlation

OpenTelemetry 的 signals 规定载体和关联方式；将其组合为 robot-safety evidence 是设计工作，不是 telemetry SDK 自动完成的功能。

5. Safety boundary 与 stale-action fencing

5.1 Safe apply 条件

$$\text{safe_apply}(a_t) = 1 \iff \text{fresh}(a_t) \wedge \text{compatible}(r, m, s, n) \wedge \text{lease_valid} \wedge \text{controller_ready}.$$

其中 `r` 是 robot identity，`m` 是 model revision，`s` 是 action schema，`n` 是 normalizer revision。`fresh` 检查 action age/sequence，`compatible` 检查 shape、unit、parser 和 controller contract，`lease_valid` 检查 epoch/fencing，`controller_ready` 只代表 controller 明确允许接收。它是 SYS - 004 的系统设计推断，不是任一上游 API 的默认保证。

5.2 Cancel、hold、stop、e-stop

层	能安全声称	还需要什么才能升级
software cancel	请求被取消或不再排队	in-flight infer、parse、send 是否已被收敛
controller hold	controller 返回 hold acknowledgement	hardware state、动作是否真正停止
software stop	policy/client 不再发送 action	controller/actuator 是否仍有 buffered command
independent e-stop	安全链路触发停止	复位、检查、incident record 与物理验证

“cancel acknowledged”不能写成“机器人已经停下”；“readiness=false”不能写成“任务安全失败”。若 controller ack 缺失，保持 `HOLD/STOPPED/UNKNOWN`，不释放 side effect gate。

6. Degradation、rollback 与 replay

6.1 Fail-closed 顺序

推荐顺序是：

1. 拒绝新请求并记录 incident identity；
2. 丢弃过期 action，提升 lease epoch，阻断旧 response；
3. 进入 `HOLD` 或 `STOPPED`，要求 controller acknowledgement；
4. 对无物理 side effect 的请求执行 fallback、restart 或 drain；
5. 对 revision regression 执行 quiesce→compatibility check→rollback；
6. 用固定输入、固定 revision、固定 protocol replay，并记录 recovery outcome；
7. 通过 health gate 和人工/策略批准后才回到 `HEALTHY`。

6.2 Rollback compatibility matrix

rollback 不能只记录 `from_revision → to_revision`，至少要比较：

项	必须一致或显式迁移
model/processor	checkpoint、tokenizer/processor 版本与 load smoke
normalizer	mean/std、action unit、valid-dimension slice
schema	camera/text/proprio/action shape、parser、chunk horizon
controller	endpoint、ack/hold semantics、limit 与 lease epoch
protocol	task/seed/init/episode、scorer 与 denominator
rollout	cohort、canary window、in-flight drain 与 fallback

Kubernetes Deployment 的 rollback 只描述 workload revision；上表是 VLA 兼容性设计推断。

6.3 Replay evidence

replay bundle 应是不可变的 observation/action/event/metric/trace manifest，并包含 SHA、revision、timestamps、protocol tuple 与原始输出位置。“重跑一次变绿”不能替代同输入、同版本、同协议的对照；没有 bundle 时只能写“恢复路径已设计”，不能写“恢复成功”。

7. 参考伪代码（带关键边界注释）

```
def handle_result(req, result, clock, controller, ledger):
    # 1. Fence stale identity before any physical side effect.
    if result.model_revision != req.model_revision:
        return quarantine(req, result, "model_revision_mismatch")
    if result.lease_epoch != ledger.current_epoch(req.robot_id):
        return discard(req, result, "stale_lease_epoch")
    if clock.now() > result.valid_until:
        return hold(req.robot_id, controller, "stale_action")
```

```
    # 2. Require explicit controller acknowledgement; cancel is not e-stop.
    ack = controller.apply(result.action_chunk, req.action_schema)
    if ack.kind not in {"APPLIED", "HOLD", "STOPPED"}:
        return hold(req.robot_id, controller, "unknown_controller_ack")

    # 3. Write one correlated event before selecting recovery.
    ledger.append_event(req.identity, result, ack)
    if ack.kind in {"HOLD", "STOPPED"}:
        return enter_recovery(req, ack.kind)
    return mark_healthy_if_gate_passes(req)
```

这段伪代码只表达检查顺序；`controller.apply`、`current_epoch`、`append_event` 与 recovery supervisor 必须有真实实现、trace 和 ownership 才能成为 local-direct evidence。

8. First-divergence 诊断矩阵

观察到的症状	首个分叉检查	不应直接下的结论
readiness 失败	probe dependency、model load、revision	“重启后已经恢复”
deadline/action-age 超限	capture→infer→parse→apply 时间线	“timeout 就是任务失败”
stale response	request/lease epoch/action sequence	“模型能力下降”
controller update/fallback error	controller ack、hardware state、buffer	“fallback 已停止物理动作”
telemetry gap	trace sampling、event identity、metric denominator	“没有日志就没有故障”
rollout regression	fixed holdout、canary cohort、protocol tuple	“rollback 自动修复 root cause”
replay mismatch	model/schema/normalizer/scorer hash	“同一个 checkpoint 就可比”

诊断顺序固定为：`identity → stage timestamps → schema/normalizer → lease/action sequence → controller ack → health gate → recovery intervention → replay outcome`。不要跳到最后一层猜 root cause。

9. 五项目真实性边界

项目	可安全绑定的 hook	当前不能说
OpenPiVLA	offline WebSocket timing、parser/normalizer/schema mismatch	production watchdog、controller e-stop、rollback closure
Robot Harness	timeout/cancel、robot lock、compensation、trace hook	VLA fault supervisor、physical stop、fleet incident response
OPD	teacher/route/holdout、scorer diagnostics	production observability、perception-action recovery
WAM	simulator/chunk/controller diagnostic trace	real-robot safety recovery、rollback canary、root-cause closure
UWM/LIBERO	launcher/task mapping、protocol mismatch、loader diagnosis	physical safety state、lease fencing、cross-robot recovery

五个项目的 inventory/ownership ledger 只允许作为 bounded hook。Git author、上游 lifecycle、示例 YAML、路径、smoke report 或局部 timeout 字段都不能自动变成候选人 ownership。

10. 面试回答模板与高压追问

10.1 60 秒回答

“我会先把故障拆成 symptom、first verified divergence、root cause 和 recovery outcome。事件必须带 request、robot、lease epoch、model/schema/normalizer revision，以及 capture→infer→parse→apply 时间线。readiness、liveness、QoS deadline 和 telemetry 只代表不同层信号，不能直接写成安全成功。出现 stale action 或未知 controller 状态时先丢弃旧结果、提升 epoch、进入 HOLD/STOPPED 并要求 controller ack，再做 fallback、rollback 或固定协议 replay。Kubernetes/ROS/OpenTelemetry 是机制来源；我当前项目只有局部 timeout、eval、launcher 或 diagnostic hook，不能声称已经实现 production fault supervisor 或物理 e-stop。”

10.2 高压追问

- readiness=false 与 controller hold 有什么区别?
- cancel acknowledged 为什么不等于 e-stop?
- rollback 如何证明 normalizer、schema、controller 兼容?
- 如何定义 first verified divergence，而不是凭经验猜 root cause?
- stale action 如何用 lease epoch 和 sequence fencing?
- telemetry 缺失时为什么应进入 unknown/hold，而不是继续执行?
- replay bundle 最小字段是什么？如何避免“重跑一次变绿”?
- 你的五个项目中哪一个有真实 fault-injection 或 recovery artifact？没有时怎样回答?

11. 结论与下一步

SYS - 004 的高分答案不是背某个 probe 或 restart 命令，而是把 identity、first-divergence、side-effect safety、controller acknowledgement、degradation、rollback、replay 和证据闭环连接起来。当前 source gate 已冻结，knowledge 已完成教学转译；仍没有 fault supervisor、physical-stop acknowledgement、fault-injection matrix、immutable replay bundle 或个人 ownership 证据。

下一步：单独生成一张 `SYS-VLA-004` atomic card；在此之前不创建 `coaching_state.md`，不升级任何 mastery。

参考文献（References）

1. SYS-004 Source Freeze。
2. ROS 2 managed node lifecycle 与 QoS。
3. `ros2_control` controller manager。
4. Kubernetes probes 与 Deployment rollback。
5. OpenTelemetry signals。
6. 本仓库 SYS - 002 与 EVAL - 008。

Part 13 / 25 · `interview_materials/vla/questions/vol_08_code001.html` · SHA-256 `80D88D0A2D2CC3AE925ECC6E7DB1BFE497F75B64F65629366E0E3A6CD5387576`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`80d88d0a2d2cc3ae925ecc6e7db1bfe497f75b64f65629366e0e3a6cd5387576`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_code001.md`

VOL 08 CODE-VLA-001 原子手撕题卡：数值稳定的 softmax

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-001` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-001 Knowledge；来源边界见 CODE-001 Source Freeze。

手写稳定 softmax / log-softmax 并说明 VLA 边界

▼ `L2` `P0` **CODE-VLA-001 · 请手写数值稳定的 softmax 或 log-softmax，并说明 mask、低精度和测试怎么处理。**

面试意图：判断你能否同时处理公式、axis/shape、数值稳定性、mask、dtype、梯度和 VLA 中 attention/loss/action 的边界，而不是只写一行 `exp(x)/sum(exp(x))`。

30 秒回答：先沿 dim 做 max-shift，再 exp/sum；低精度用更宽累加。log-softmax 直接算 logsumexp，不能先 softmax 再 log。mask、全 masked 行和 dtype 要定合同，并用极值、梯度、任意 dim 与框架 parity 测试。

机制展开（约 2 分钟）：实现 `m=x.amax(dim,keepdim=True)`，`p=exp(x-m)/sum(exp(x-m))`；fp16/bf16 在 fp32 累加后按 contract cast。log-softmax 用 `x-logsumexp(x)`；mask 区分 bool/加性，全 masked 行约定 reject/zero/invalid，避免 `-inf-(-inf)` NaN。测试 batch/transpose、±1000、mask、低精度、finite、autograd 与 PyTorch parity。attention 的 mask/scale 和 VLA loss mask 不同，continuous action MSE/diffusion 不需要 softmax。

诊断 / 证据：白板先写 shape/`dim`/`keepdim`，再写 max-shift 和累加 dtype；代码题至少覆盖已知小 logits、±1000、任意 batch/dim、bool/float mask、全 masked 行、fp16/bf16、NaN/Inf、forward/backward parity。若与 fused `torch.log_softmax` 对照，记录 tolerance、device、dtype 和是否 cast；不要把一次 forward 通过当作所有 kernel/device 都正确。

项目挂钩：OpenPiVLA 的 causal-CE、OPD 的 scorer、WAM/UWM 的 mask/routing 可用于解释监督 mask 与条件 mask 的区别；但当前没有本地 CODE-VLA-001 实现或 parity artifact。安全表达是“我能写出并测试稳定实现”，不声称某项目因手写 softmax 获得收益。

追问树：为什么 max-shift 不改变结果？`dim=-1` 与 `keepdim` 如何影响广播？全 masked 行为什么会 NaN？为什么 CE 不应先 softmax 再 log？fp16 为什么要更宽累加？attention mask 和 loss mask 是否相同？continuous action 为什么通常不用 softmax？

易错点：忘记减最大值；沿错轴归一化；`keepdim=False` 导致广播错；全 masked 行传播 NaN；先 softmax 再 log；把 bool mask 当 loss mask；把 softmax 的 (O(L)) 归一化说成 attention 变成线性复杂度。

危险答案：说“softmax 天然不会溢出”“所有 mask 都加一个 0/1 就行”“概率和为 1 所以 action 必须 softmax”“一次单测通过就兼容 fp16/所有设备”或“PyTorch fused kernel 就是我的原创实现”。

来源：[1][2][3][4] 练习状态：unseen

参考来源

1. CODE-001 Knowledge：max-shift、logsumexp、mask、dtype、梯度和测试矩阵。
2. CODE-001 Source Freeze：PyTorch softmax/log-softmax/CE/SDPA 官方 API 与项目边界。
3. PyTorch `torch.softmax`、`log_softmax` 与 `CrossEntropyLoss`。
4. PyTorch `scaled_dot_product_attention`。

Part 14 / 25 · `interview_materials/vla/questions/vol_08_code002.html` · SHA-256 `7FCA7D94FFB9758D6D010F57FE4A753D521DEBD59881AFD6B27A137395DA6078`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`7fca7d94ffb9758d6d010f57fe4a753d521debd59881afd6b27a137395da6078`；生成自：`interview_materials\vla\questions\vol_08_code002.md`

VOL 08 CODE-VLA-002 原子手撕题卡：Scaled Dot-Product Attention

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-002` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-002 Knowledge；来源边界见 CODE-002 Source Freeze。

手写 scaled dot-product attention 并处理 mask/shape

▼ `L2` `P0` **CODE-VLA-002 · 请手写 scaled dot-product attention，并说明多头 shape、causal/padding mask、dropout 和测试。**

面试意图：判断你能否把 Q/K/V shape、$1/\sqrt{d_k}$、mask 方向、不同 query/key 长度、dtype、dropout 和 VLA loss 边界落到可运行实现。

30 秒回答：先统一 Q/K/V 为 [B,H,L,D]，算 scores=$QK^T/\sqrt{D}$，沿 Lk 做 softmax；mask 在 softmax 前应用，causal 只屏蔽未来，训练才 dropout。测试 Lq≠Lk、padding、fp16 和梯度 parity。

机制展开（约 2 分钟）：实现时只在最后两维做 matmul，score 为 [B,H,Lq,Lk]，softmax 沿 key 维；bool/additive mask 的 True 语义先定，Lq≠Lk 不能写死方阵。fp16/bf16 可在 fp32 算 score/softmax 后 cast；全 masked 行要 reject/invalid，避免 NaN；dropout 只在 train。用手算、多头、causal future、padding、不同长度、极值、autograd/SDPA parity 验证。attention mask 决定可见性，loss mask 决定监督，continuous action 不需 softmax。

诊断 / 证据：白板先标注每个 tensor 的 `[B,H,L,D]`，再验证 `Q @ K.transpose(-2,-1)`、scale、mask broadcast、softmax axis 和 `A @ V`。测试 square 与 `Lq≠Lk`、单头手算、多头 merge、future blocking、padding、train/eval dropout、fp16/bf16 finite、all-masked row、autograd 与 PyTorch SDPA tolerance。记录 device、dtype、dropout seed 和 backend；forward 通过不等于 fused kernel 性能通过。

项目挂钩：OpenPiVLA 可连接 causal-CE、eager attention 与 action packaging；OPD 可连接 token/format scorer；WAM/UWM 可连接 condition/mask/routing。安全表达是“我能实现并测试 attention 合同”，不把上游 SDPA/FlashAttention 或局部项目代码写成个人原创或性能 uplift。

追问树：为什么除以 √D？softmax 沿哪个维度？`Lq≠Lk` 怎么做 causal mask？bool mask 的 True 是 allow 还是 deny？全 masked 行如何处理？dropout 在 eval 是否关闭？attention mask 与 loss mask 有何区别？continuous action 为什么不用 softmax？

易错点：Q/K/V 轴顺序错；沿 query 而非 key 维 softmax；把不同长度写成方阵；causal 方向反；mask 广播到错 head/batch；eval 仍 dropout；把 forward parity 当作 latency 或真机收益。

危险答案：说“attention 天然是 O(L)”“mask 加 0/1 就够”“True 的语义所有框架都一样”“FlashAttention 一定等价且更快”或“attention mask 正确就说明 action loss/项目闭环正确”。

来源：[1][2][3][4]　**练习状态**：unseen

参考来源

1. CODE-002 Knowledge：shape、scale、mask、dropout、dtype、复杂度与测试矩阵。
2. CODE-002 Source Freeze：PyTorch SDPA/MHA、Transformer paper 与 VLA boundary。
3. PyTorch `scaled_dot_product_attention` 与 `MultiheadAttention`。
4. Vaswani et al., Attention Is All You Need。

Part 15 / 25 · `interview_materials/vla/questions/vol_08_code003.html` · SHA-256 `C7A1A99CBB06AA166DDD47321ADFDE61DCCFC3DDF1F44DC0607697F150AB58AE`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`c7a1a99cbb06aa166ddd47321adfde61dccfc3ddf1f44dc0607697f150ab58ae`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_code003.md`

VOL 08 CODE-VLA-003 原子手撕题卡：LayerNorm / RMSNorm

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-003` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-003 Knowledge；来源边界见 CODE-003 Source Freeze。

手写 LayerNorm / RMSNorm 并解释差异

▼ `L2` `P0` **CODE-VLA-003 · 请手写 LayerNorm 和 RMSNorm，并说明归一化轴、epsilon、低精度与训练/推理差异。**

面试意图：判断你能否区分 re-centering/re-scaling、normalized shape、population variance、affine、epsilon、dtype 与 BatchNorm running statistics。

30 秒回答：LayerNorm 沿 normalized_shape 算均值/方差，减均值后乘 rsqrt(var+eps)，再做 γ/β；RMSNorm 不减均值，只除以 sqrt(mean(x²)+eps)，通常只有 γ。低精度用 fp32 统计；两者无 running stats，测试任意轴、constant 和 autograd parity。

机制展开（约 2 分钟）：对 [B,L,D] 逐 token 沿 D：LN 算 μ、population var，`(x-μ)*rsqrt(var+eps)*γ+β`；RMSNorm 为 `x*rsqrt(mean(x²)+eps)*γ`。多维 normalized_shape 沿末尾多轴，affine 可广播；低精度用 fp32 统计后 cast。两者无 running stats，train/eval 自身一致。用 constant、non-zero-mean、multi-dim、affine on/off、低精度和 PyTorch forward/backward parity 测试；feature norm 不等于 action normalizer。

诊断 / 证据：先写输入与 normalized shape，再列出归一化 dims、统计 dtype、epsilon、affine shape 和输出 dtype。测试 `[B,L,D]`、末尾多维、constant/zero/non-zero-mean、fp16/bf16、γ/β on/off、train/eval 和 autograd parity；LayerNorm 用 population variance。若项目换 norm，要固定 checkpoint/config/eps/dtype/residual layout 并做 matched ablation，不能只看单次 loss。

项目挂钩：OpenPiVLA 的 Qwen backbone 可连接 RMSNorm/LayerNorm、LoRA 与 attention 兼容；UWM/WAM 的 transformer/DiT block 可连接 pre/post-norm 和 dtype。安全表达是“我能实现并审计 norm 合同”，不把上游 norm 选择写成个人原创，也不把 feature norm 与 action normalizer 混为一谈。

追问树：为什么 RMSNorm 不减均值？LayerNorm variance 是否 unbiased？`normalized_shape=(H,W,C)` 沿哪些轴？epsilon 加在哪里？为什么不用 running mean？train/eval 为什么应一致？fp16 怎样处理？feature norm 与 action denormalization 有何区别？

易错点：沿 batch/sequence 维统计；把 sample variance 当 population variance；epsilon 放在 sqrt 外；把 RMSNorm 写成 LayerNorm 去 bias 但仍减均值；给 norm 加 running stats；affine 广播错；把上游 RMSNorm 说成个人优化。

危险答案：说“RMSNorm 就是没有 β 的 LayerNorm”“LayerNorm 训练和推理用不同 running mean”“normalized_shape 永远只有最后一维”“换 RMSNorm 一定更快更好”或“action normalizer 与 RMSNorm 是同一件事”。

来源：[1][2][3] 练习状态：unseen

参考来源

1. CODE-003 Knowledge：公式、normalized shape、低精度、训练/推理与测试矩阵。
2. CODE-003 Source Freeze：PyTorch LayerNorm/RMSNorm、RMSNorm paper 与项目边界。
3. PyTorch `LayerNorm`、`RMSNorm` 与 RMSNorm paper。

Part 16 / 25 · `interview_materials/vla/questions/vol_08_code004.html` · SHA-256 `CE9725D4B27B2B2F837404A4F171F4A635341457DE83D5C37092746CFA8B1FAD`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`ce9725d4b27b2b2f837404a4f171f4a635341457de83d5c37092746cfa8b1fad`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_code004.md`

VOL 08 CODE-VLA-004 原子手撕题卡：Masked MSE / Huber Loss

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-004` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-004 Knowledge；来源边界见 CODE-004 Source Freeze。

手写 masked MSE / Huber action loss

▼ `L2` `P0` **CODE-VLA-004 · 请手写带 mask 的 MSE/Huber loss，并说明 reduction、空 mask、低精度与测试。**

面试意图：判断你能否处理 action padding、异构有效维、Huber delta、有效 denominator、分布式聚合和 VLA loss/mask 边界。

30 秒回答：先算 elementwise MSE/Huber，再乘 valid/action-dim mask，分母用有效权重和，不能 `mean(mask*loss)`。全零 mask 要 reject/skip；低精度用 fp32 累加。测试 padding 不变、delta/outlier、空 mask、分布式分母和梯度 parity。

机制展开（约 2 分钟）：误差 e=pred-target；MSE=e^2，Huber 在 $|e|\le\delta$ 用 $0.5e^2$，否则 $\delta(|e|-0.5\delta)$。把 mask/weight 广播到 [B,H,D]，loss=sum(wmℓ)/sum(wm)；分布式先 all-reduce numerator/denominator 再相除。全零 mask 不是正常零 loss，要 reject/skip；NaN/Inf 在乘 mask 前查。测试 mask=1 parity、不同 padding/有效维、delta 边界、outlier、fp16 和 autograd。attention mask、action valid mask、token CE 与 diffusion/flow target 要分开。

诊断 / 证据：记录 pred/target/mask/weight shape、normalized/physical space、delta、numerator、denominator、有效 timestep/dim 和 skip reason。验证 `mask=1` 与框架 parity、同有效元素不同 padding 的 loss 不变、全零 mask 触发显式分支、Huber 在 delta 处连续、outlier 梯度线性、fp16/bf16 finite。多 rank 必须聚合 numerator/denominator，而不是平均各 rank mean。

项目挂钩：OpenPiVLA 可连接 action chunk、valid dims、normalizer 与 causal-CE；UWM/WAM 可连接 action/video target mask、timestep weight 和多任务 reduction；OPD 可连接 metric-specific denominator。安全表达是“我能实现和审计 masked loss”，不声称更换 Huber 后项目成功率提升。

追问树：为什么 `mean(mask*loss)` 错？全零 mask 返回零可以吗？Huber 与 SmoothL1 的 scale 有何区别？delta 在 normalized 还是 physical space？不同 action dim 如何公平聚合？分布式为何先聚合 numerator/denominator？attention mask 是否能直接当 loss mask？

易错点：padding 进入分母；全零 mask 伪装成低 loss；`0*NaN` 仍传播 NaN；Huber delta/系数错；把 task weight 当二值 valid mask；先 rank mean 再平均；把 token CE、diffusion target 和 action MSE 混成同一 loss。

危险答案：说“乘 mask 后直接 mean 即可”“空 mask 返回 0 最稳定”“Huber 一定优于 MSE”“所有任务按有效元素 micro-average 就公平”或“loss 降低证明机器人成功率提升”。

来源：[1][2][3] **练习状态**：unseen

参考来源

1. CODE-004 Knowledge：MSE/Huber、masked denominator、empty mask、weighting 与测试矩阵。
2. CODE-004 Source Freeze：PyTorch losses 与 VLA mask/reduction 边界。
3. PyTorch `MSELoss`、`HuberLoss` 与 `SmoothL1Loss`。

Part 17 / 25 · `interview_materials/vla/questions/vol_08_code005.html` · SHA-256 `E1680AFC574B0B7C28D4C3FA8DC526021B003CB2C8EDADBA4E6B65592F2F3945`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`e1680afc574b0b7c28d4c3fa8dc526021b003cb2c8edadba4e6b65592f2f3945`；生成自：`interview_materials\vla\questions\vol_08_code005.md`

VOL 08 CODE-VLA-005 原子手撕题卡：Homogeneous Transform Composition

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-005` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-005 Knowledge；来源边界见 CODE-005 Source Freeze。

手写 SE(3) 齐次变换与 roundtrip

▼ `L2` `P0` **CODE-VLA-005 · 请解释并手写 4×4 homogeneous transform composition，说明 frame 方向、逆变换和测试。**

面试意图：判断你能否处理 T_B^A 方向、组合顺序、point/vector/pose、刚体逆、frame/单位/时间戳和 roundtrip，而不是只会矩阵乘法。

30 秒回答：先定义 T^A_B 是 B→A，点用 [x,y,z,1]，direction 用 w=0；组合 T^A_C=T^A_B T^B_C，刚体逆为 R^T、$-R^Tt$。冻结 frame/单位/时间戳，做 identity、逆变换和 roundtrip 测试。

机制展开（约 2 分钟）：写 T=[R,t;0,1]，明确上标目标、下标输入；compose 右到左。点乘 w=1，方向 w=0，pose 还要变换姿态。刚体 inverse 用 R^T 和 $-R^Tt$，并检查 $R^TR\approx I$、$\det\approx 1$。记录 frame_id、parent/child、米/mm、timestamp、calibration revision；用 known rotation、composition/inverse roundtrip、batch、unit/frame mismatch 和 capture-time TF 测试。几何正确不等于 depth/planner/controller 成功。

诊断 / 证据：先用 identity、pure translation、90° rotation 手算，再做 `compose(T_A_B,T_B_C)` 与逐步变换 parity；随机点做 inverse roundtrip，验证 rotation orthogonality/determinant、bottom row、finite、batch broadcast。对 camera/base/world 记录 parent/child、matrix revision、capture-time timestamp、unit 和 uncertainty；若 overlay 对但 frame/时间不一致，不能把它当 controller 正确。

项目挂钩：OPD 可连接 point/box/text 坐标协议；OpenPiVLA/WAM 可连接 camera/action packaging；Robot Harness 可连接 frame-aware tool schema；UWM/LIBERO 可连接 camera/state/action shape。安全表达是“我能实现并审计 SE(3) 接口”，不把通用矩阵代码写成项目标定、抓取或导航成功。

追问树：T^A_B 的上下标怎样读？为什么不能直接转置整个 4×4？point 与 direction 的 w 有何不同？组合为何右到左？刚体 inverse 的平移项为何是 $-R^Tt$？米/mm 错会怎样？latest TF 能否替代 capture-time TF？overlay 对能证明什么？

易错点：把 B→A 写成 A→B；乘法顺序反；把 direction 也平移；把 pose 只变换 xyz；用一般矩阵 inverse 代替 rigid inverse 却不检查 R；忽略 frame/单位/时间戳/revision；把几何 roundtrip 当闭环成功。

危险答案：说“4×4 矩阵转置就是逆”“只要 overlay 看起来对就行”“ROS 轴序默认适用于所有项目”“最新 TF 总是最好”或“point 变换成功就证明抓取/导航成功”。

来源：[1][2][3] **练习状态**：unseen

参考来源

1. CODE-005 Knowledge：SE(3)、方向、point/vector/pose、inverse、frame/单位/时间和测试矩阵。
2. CODE-005 Source Freeze：ROS REP-103/105、SciPy Rotation 与 PER-002 边界。
3. ROS REP-103、REP-105 与 SciPy `Rotation`。

Part 18 / 25 · `interview_materials/vla/questions/vol_08_code006.html` · SHA-256 `7A35D23D69F85F142F2B02C52A3DAF785D851734A124751B112DA6537F3E5908`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`7a35d23d69f85f142f2b02c52a3daf785d851734a124751b112da6537f3e5908`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_code006.md`

VOL 08 CODE-VLA-006 原子手撕题卡：2-Link Planar Forward Kinematics

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`CODE-VLA-006` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-006 Knowledge；来源边界见 CODE-006 Source Freeze。

手写二连杆平面正运动学

▼ `L2` `P0` **CODE-VLA-006 · 请推导并实现 2-link planar forward kinematics，说明角度/坐标约定和测试。**

面试意图：判断你能否处理相对 joint angle、frame/单位、解析式、齐次变换链、Jacobian 直觉和边界姿态，而不是只背结果。

30 秒回答：先冻结 radians、base frame 和相对 joint angle：$x=l_1\cos\theta_1+l_2\cos(\theta_1+\theta_2)$，y 同理，$\varphi=\theta_1+\theta_2$。也可用 $Rz(\theta_1)Tx(l_1)Rz(\theta_2)Tx(l_2)$。用 zero/90/folded、矩阵 parity 和 Jacobian finite difference 测。

机制展开（约 2 分钟）：对 [B,2] 输入，先算 c1/s1 与 c12/s12，再输出 $x=l_1c_1+l_2c_{12}$、$y=l_1s_1+l_2s_{12}$、$\varphi=\theta_1+\theta_2$；$\theta_2$ 是相对角。矩阵链右到左，平移沿当前 link frame。冻结 rad/deg、米/mm、joint zero、正方向和 tool offset。测试 zero、90°、folded、负角、解析/矩阵 parity、batch、单位 mismatch 与有限差分 Jacobian；FK 不等于 IK、碰撞或 controller 成功。

诊断 / 证据：先用 zero、π/2、π folded 手算，再对解析式和 `Rz·Tx·Rz·Tx` 做数值 parity；随机角度做 finite-difference Jacobian，检查 batch `[B,2]→[B,3]`、finite、单位和 frame metadata。若接入项目，固定 URDF/joint zero/limit、tool transform、angle unit 和 calibration revision；不要只用一张 overlay 证明几何或控制正确。

项目挂钩：OPD 可连接 point/box 坐标协议；OpenPiVLA/WAM 可连接 camera/action packaging；Robot Harness/UWM 可连接 robot state schema。安全表达是“我能实现和审计 FK 接口”，不把通用二连杆代码写成项目标定、IK、控制或真机成功。

追问树：θ_2 是 relative 还是 global？为什么第二项用 $\theta_1+\theta_2$？矩阵链为何右到左？角度 degree 忘转 radian 会怎样？Jacobian 何时退化？FK 能否解决 IK？tool offset 放在哪里？如何验证 frame 方向？

易错点：把 θ_2 当 global angle；sin/cos 轴方向反；degree/radian 混用；link 平移在错误 frame；忘记 tool transform；把 finite-difference Jacobian 当完整 controller；把 FK 结果当抓取/导航成功。

危险答案：说“FK 直接给 IK”“θ_1、θ_2 可以任意相加不看 convention”“长度单位不影响结果”“zero pose 看起来对就证明模型正确”或“二连杆 FK 证明项目真机控制成功”。

来源：[1][2][3] **练习状态**：unseen

参考来源

1. CODE-006 Knowledge：解析 FK、齐次链、frame/单位、Jacobian 与测试矩阵。
2. CODE-006 Source Freeze：Modern Robotics、ROS REP-103 与项目边界。
3. Modern Robotics Chapter 4 与 ROS REP-103。

Part 19 / 25 · `interview_materials/vla/questions/vol_08_code007.html` · SHA-256 `C1E2B7B691E0A26C1FB034A51B33C0C3CA0A7FED19640E8EB708F087AADCC58E`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256: `c1e2b7b691e0a26c1fb034a51b33c0c3ca0a7fed19640e8eb708f087aadcc58e`; 生成自: `interview_materials\vla\questions\vol_08_code007.md`

VOL 08 CODE-VLA-007 原子手撕题卡：PID Update with Anti-windup

状态: `drafted / card_1_of_1 / WARN` 更新日期: 2026-07-20 覆盖范围: `CODE-VLA-007` 难度: L2 1 题 优先级: P0 练习状态: `unseen` 前置知识: CODE-007 Knowledge；来源边界见 CODE-007 Source Freeze。

手写带 anti-windup 的离散 PID update

▼ `L2` `P0` **CODE-VLA-007 · 请实现离散 PID update，处理输出饱和、积分 anti-windup、reset 与测试。**

面试意图：判断你能否冻结 `dt`、error sign、内部 state、output/i limits 与 anti-windup 分支，并写出可诊断的更新，而不是只把 `P+I+D` 相加。

30 秒回答：固定 e、dt、state 和 limits，算 $u_{unsat} = P + I + D$，再 clip 得 u_{sat}。conditional integration 在饱和且误差继续推向 rail 时冻结、反向时放行；back-calculation 用 $(u_{sat} - u_{unsat})/T_t$ 回拉积分。记录 trace，测 dt、饱和、reset、replay。

机制展开（约 2 分钟）：令 i_{state} 为积分输出贡献，$u_{unsat} = K_p e + i_{state} + K_d \Delta e/dt$，$u_{sat} = \text{clip}(u_{unsat})$。conditional integration 按饱和方向决定是否累加 $K_i dt e$；back-calculation 用 $dt\,[K_i e + (u_{sat} - u_{unsat})/T_t]$ 回拉。`u_clamp`、`i_clamp`、IZone、reset 不等价，reset 绑定 episode/lease epoch。返回 action、state、trace，检查 dt、broadcast、NaN 和 limits。VLA 先绑定 action schema/norm/unit；没有 PID 或 tuning artifact，不能声称真机闭环。

诊断 / 证据：按 `dt/finite → error sign → state/reset → u_unsat → clamp → anti-windup branch → dtype/device → controller ack` 定位。做 `K_i/K_d=0`、上/下饱和、误差反向、`u_min=u_max`、reset、continuous input、fixed replay 八类 probe；记录 `e/p/i/d/u_unsat/u_sat/branch/dt/reset_reason`。白板答案应说明 `Tt` 与单位；runtime 还需 matched step/reversal、noise/jitter、episode epoch 与 action-limit artifact。现有项目只能作为接口 hook，不能升级为 PID ownership 或 robot success。

项目挂钩：OpenPiVLA 可连接 action shape/normalizer/parser；Robot Harness 可连接 timeout/compensation；WAM 可连接 controller/chunk diagnostic；OPD 可连接 route/teacher schedule；UWM/LIBERO 可连接 launcher/task mapping。五项目均没有 CODE-007 local PID implementation、joint feedback trace、anti-windup tuning log 或 matched controller ablation，回答时保持 evidence boundary。

追问树：为什么 output clamp 不会自动阻止积分增长？conditional integration 如何判断“误差正在把输出拉回”？back-calculation 的 Tt 如何调？i_state 是 Ki 前还是 Ki 后？measurement derivative 和 error derivative 有何差别？normalized action 上做 PID 还是 physical action 上做？lease/episode epoch 改变时 state 怎么处理？如何证明不是 controller limit 或单位错误？

易错点：把 `u_clamp` 当 anti-windup；所有饱和都冻结积分；忽略 error sign；`dt` 固定写死；把 i_state 与 ∫e 混为一谈；不同单位复用 gain；reset 不清 derivative；用最终 action 或 simulator rollout 冒充 tuning/真机证据。

危险答案：说“clip 输出就不会 wind-up”“ROS/WPILib 的 PID API 证明我完成了 VLA controller”“Tt 越小越快所以一定更好”“normalized action 直接套 physical gain”“一次 rollout 成功就证明 anti-windup 稳定”。

来源：[1][2][3] **练习状态**：unseen

参考来源

1. CODE-007 Knowledge：PID state、clamp、conditional/back-calculation、reset、probe 与项目边界。
2. CODE-007 Source Freeze：ROS 2 control、WPILib、Feedback Systems 与 8 probe contract。
3. 本仓库 VOL 01 advanced-control fact review 与五项目 evidence inventories。

Part 20 / 25 · `interview_materials/vla/questions/vol_08_code008.html` · SHA-256 `EE52BE476C7CDF559CA3FCFD9B7EE65507FA24ADE004B64F958E450103A69945`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256: `ee52be476c7cdf559ca3fcfd9b7ee65507fa24ade004b64f958e450103a69945`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_code008.md`

VOL 08 CODE-VLA-008 原子手撕题卡：Bellman TD(0) Update

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`CODE-VLA-008` 难度：L2 1 题 优先级：P0 练习状态：`unseen` 前置知识：CODE-008 Knowledge；来源边界见 CODE-008 Source Freeze。

实现正确的 Bellman TD(0) update

▼ `L2` `P0` **CODE-VLA-008 · 请实现 Bellman TD(0) update，区分 terminated/truncated，并处理 target、mask 与 batch edge cases。**

面试意图：判断你能否把 MDP 语义落实到 transition schema、target detach、valid reduction 和可诊断的代码，而不是只背 Bellman 公式。

30 秒回答：分开 `terminated/truncated`，带上 `obs_tp1`。target=`r+γ(1-terminated)V_next.detach()`；terminal 不 bootstrap，timeout 通常 bootstrap。检查 flags、all-invalid、`γ=0`、shape；无 replay artifact 不声称收敛。

机制展开（约 2 分钟）：冻结 transition schema 与 episode/step lineage；拒绝双 flag、缺 `obs_tp1`、非 finite reward/discount。terminal mask 只来自 `terminated`，truncation 有 next state 仍 bootstrap。target detach；`valid` 明确分子/分母，`[B]` / `[B,T]` 对齐，分布式先聚合计数再相除。Q-learning 只替换 next value，不是所有 TD 都取 max；success、controller stop、network timeout 保留来源，不自动变成 terminal。

诊断 / 证据：按 `schema/lineage → terminated/truncated → obs_tp1 → terminal mask → gamma/dtype → detach → shape/broadcast → valid denominator → target revision` 定位。至少跑 8 个 probe：真实 terminal 不 bootstrap；time-limit truncation bootstrap；两 flag 冲突 reject/precedence；`gamma=0` 只剩 reward；`[B]` 与 `[B,T]` 等价；all-invalid reject/skip；target 无梯度；NaN/Inf 在 reduction 前暴露。runtime 还要有 transition/replay bundle、target-network revision 和 numerator/denominator 记录。

项目挂钩：VOL 02 的 Harness 只能连接 offline grouped reward-weighted update、GAE/timeout 讨论；OpenPiVLA、WAM、UWM/LIBERO、OPD 的 rollout/reward/summary 不能替代 transition-level artifact。当前没有 CODE-008 本地 learner、replay bundle、gradient parity、target revision 或候选人 ownership；安全说法是“我能实现并审计 TD(0) 合同”，不能说已证明 value 收敛、offline RL 稳定或真机成功。

追问树：为什么 truncation 通常仍 bootstrap？`obs_tp1` 缺失怎么办？`success=True` 是否一定 terminal？为什么 target 要 detach？两 flag 同时 true 怎么处理？Q-learning 的 max 在哪一侧？action chunk timeout 如何进入 schema？分布式 masked loss 为什么不能平均各 rank 的 mean？

易错点：把 `done=terminated or truncated` 直接用于 Bellman mask；把 success 当 terminal；忽略 `obs_tp1` provenance；target 和 prediction 一起反传；all-invalid 静默返回 0；把 `[B,T]` mask 错位到 next state；用 loss 下降冒充 value 正确；把公开 TD 机制写成个人项目实现。

危险答案：说“timeout 就是失败所以不 bootstrap”“Gymnasium API 证明我实现了 learner”“所有 Q/V target 都取 max”“target 不 detach 也一样”“一次 rollout 成功就证明 TD 稳定”。

来源：[1][2][3][4] **练习状态**：unseen

参考来源

1. CODE-008 Knowledge：transition schema、terminal/timeout bootstrap、target detach、reduction、probe 与项目边界。
2. CODE-008 Source Freeze：Sutton–Barto、Gymnasium time-limit 语义与 8 个 deterministic probes。
3. Sutton & Barto, Reinforcement Learning: An Introduction。
4. Gymnasium, Handling Time Limits。

Part 21 / 25 · `interview_materials/vla/questions/vol_08_sys001.html` · SHA-256 `F64BA3691B49C2672E33BFBDE6CF4A0188C1704825D39010CD454E17E3AC9188`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`f64ba3691b49c2672e33bfbde6cf4a0188c1704825d39010cd454e17e3ac9188`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_sys001.md`

VOL 08 SYS-001 原子题卡：可复现的 VLA 训练—评估平台

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-18 覆盖范围：`SYS-VLA-001` 难度：L3 1 题 优先级：P0 练习状态：`unseen` 前置知识：SYS-001 Knowledge；来源边界见 SYS-001 Source Freeze。

设计可复现的 VLA 训练—评估平台

▼ `L3` `P0` **SYS-VLA-001 · 如何设计一个可复现的 VLA 训练—评估平台，避免数据、checkpoint、评估协议和 rollout 互相污染?**

面试意图：判断你能否把数据版本、训练 lineage、模型 bundle、评估 tuple、episode artifact 和安全边界串成系统，而不是只画 trainer/server 方框。

30 秒回答：画五层 lineage：dataset→run→bundle→fixed eval→episode artifact。冻结 schema、seed、processor/normalizer、checkpoint、scorer、分母；offline/sim/real 分 namespace。缺 trace/hash，只讲接口，不称平台或 benchmark。

机制展开（约 2 分钟）：五类 manifest：dataset 记 episode/frame/unit/mask/split/hash；run 记 code/config/seed/rank/model/processor/normalizer；bundle 做 load/hash gate；eval 固定 task/init/seed/horizon/success/scorer/分母；rollout 存 action trace、latency、terminal reason，分 train/offline/sim/real/proxy。项目仅局部证据，缺 registry、bundle hash、raw rollout、ownership，不能把 loss、远端 summary 或脚本写成平台。

诊断 / 证据：先对固定 sample 做 processor→model→decode→denorm→schema roundtrip，检查 shape、dtype、finite、range 和 parser；再用冻结 holdout 复算 offline metric；最后按同一 checkpoint/bundle 运行 simulator 或 real rollout，保存逐 episode trace、terminal reason、latency、scorer 和 denominator。用 dataset split、normalizer swap、checkpoint load、init/seed replay 做 first-failure 分层；缺 raw bundle 时保持 `WARN`。

项目挂钩：OpenPiVLA 可连接 formal trainer、recipe、action packaging 和 timeout hook；UWM/LIBERO 可连接 task mapping、fixed/reset protocol、normalizer/action shape 与 DataLoader stall；WAM 可连接 rollout provenance；Robot Harness 可连接 timeout/cancel/compensation；OPD 可连接 split、raw holdout 和 strict scorer。安全表达是“我能设计 lineage 和验证门”，不把局部脚本、远端 summary 或上游框架写成个人完成的平台。

追问树：为什么 checkpoint path 不能证明权重相同？normalizer 放在 bundle 哪一层？offline MSE 与 simulator success 如何分 namespace？如何防相邻 window 泄漏？mid-epoch resume 需要保存什么？没有 episode JSONL 时能否报成功率？怎样区分 protocol change 与 model uplift?

易错点：只固定 seed；忽略 processor/normalizer/action schema；把 simulator summary 当 real result；把 report-supported 数字当 local-direct；用单卡 smoke 证明分布式复现；覆盖 latest checkpoint 导致 lineage 丢失。

危险答案：说“有 checkpoint 就能复现”“固定随机种子即可 bitwise identical”“loss 降了所以 success 一定升”“一个 summary JSON 足以证明 benchmark”或“OpenPiVLA/UWM 脚本就是我搭建的完整平台”。

来源：[1][2][3][4] 练习状态：unseen

参考来源

1. SYS-001 Knowledge：dataset/run/bundle/eval/rollout contract、failure tree 与项目边界。
2. SYS-001 Source Freeze：PyTorch DCP/reproducibility、Open X、LIBERO、OpenPI 与本地 evidence inventory。
3. PyTorch Distributed Checkpoint 与 Reproducibility。
4. Open X-Embodiment、LIBERO、OpenPI policy interface。

Part 22 / 25 · `interview_materials/vla/questions/vol_08_sys002.html` · SHA-256 `577C653C0C846147E3274F57155258FD891D3348398A3467912F36EF54E8EB88`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`577c653c0c846147e3274f57155258fd891d3348398a3467912f36ef54e8eb88`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_sys002.md`

VOL 08 SYS-002 原子题卡：Policy Serving 与 Real-time Control Loop

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`SYS-VLA-002` 难度：L3 1 题 优先级：P0 练习状态：`unseen` 前置知识：SYS-002 Knowledge；来源边界见 SYS-002 Source Freeze。

如何设计 VLA Policy Serving 与实时控制环?

▼ `L3` `P0` **SYS-VLA-002 · 如何把 VLA policy 接入实时控制环，并区分 WebSocket demo、soft-real-time service 与物理安全边界?**

面试意图：判断你能否把 API/schema、状态所有权、chunk/replan、端到端 latency、deadline/backpressure、fallback 和 controller/e-stop 串成一条可验证的系统设计，而不是只说“加一个 policy server”。

30 秒回答：拆三层：server 返回动作，client 管 chunk，controller 管 ack/stop。冻结 deadline、H/D、frame、expiry；batch 需兼容，denorm/限幅后交 controller。看 capture→apply、age、miss、fallback；infer_ms/timeout 不等于 E2E/stop。

机制展开（约 2 分钟）：server 管模型，client 管连接和 chunk，controller 管执行与停止。request 记录 id、capture_ts、revision、deadline、H/D、frame、normalizer、expiry，并传播 deadline。队列满则 reject、复用新鲜 chunk 或 fallback；batch 前校验 shape/normalizer。推理后 parse、denorm、限幅、stale 检查，再交 controller。用 capture→apply、action age、deadline miss、fallback 验证 soft real-time；cancel、旧回包、stop、e-stop 分开。

诊断 / 证据：先用 request_id 串起 capture、enqueue、infer、receive、postprocess、apply 时间戳，定位 queue/network/GPU/controller 的 first divergence；再做 queue-full、deadline-too-small、stale-response、parse/denorm、controller-ack 缺失和 reconnect fault injection。检查 H/K、action axis/frame/unit、normalizer revision、control frequency 与 success predicate 是否一致。没有 load/fault trace、E2E timestamp、controller ack 或 watchdog/e-stop 证据时，只能说设计了验证门，不能报 p99 SLO、hard real-time 或“timeout 后机器人必停”。

项目挂钩：OpenPiVLA 可安全连接 offline WebSocket eval、connect/infer alarm、in-process `ActionChunkBroker` rollout 和 parser/normalizer 风险；Robot Harness 可连接 timeout/cancel/compensation；OPD 可连接 data/eval route；WAM、UWM/LIBERO 可连接 simulator/chunk/controller protocol。五者都只能作为局部接口或证据 hook，不能升级为我已上线 production policy service、multi-robot fleet、hard real-time watchdog 或完整物理安全系统。

追问树：为什么 `infer_ms` 不是 E2E latency? H 与 K 如何共同影响 action age? 什么请求可以 dynamic batch，什么请求必须拒绝? deadline 与逐层 timeout 有何区别? queue 满时丢 observation 和丢 action 的语义为何不同? RPC cancellation 为什么不等于 e-stop? controller ack、hold/stop 和 watchdog 谁拥有最终状态? 如何证明旧 chunk 不会在重连后执行?

易错点：把 WebSocket 叫作“天然实时”；把 example 的 H/K 当通用标准；把 `asyncio.Queue`、Triton batcher 或 ROS QoS 文档写成 OpenPI 已实现；只报平均 infer time；把 timeout/exception 当成物理停止；把上游代码路径当个人 ownership；把 offline action metric 当 closed-loop success。

危险答案：说“OpenPI 已经有 production queue、dynamic batching、deadline propagation、fairness 和 watchdog”；说“server 返回 200 就代表机器人执行成功”；说“取消 RPC 会撤回 GPU 和已发控制命令”；或说“OpenPiVLA 的 WebSocket/ALOHA 脚本证明我完成了端到端生产部署”。

来源：[1][2][3][4][5] 练习状态：unseen

参考来源

1. SYS-002 Knowledge：schema、状态所有权、timing、backpressure、实时边界与五项目 hook。
2. SYS-002 Source Freeze：OpenPI frozen revision、CPython/Triton/gRPC/ROS 2 机制与 14/14 probes。
3. Physical Intelligence OpenPI `openpi@15a9616a`：server/client、broker、LIBERO/DROID examples。
4. CPython `asyncio` queue/tasks、NVIDIA Triton batcher/model configuration。
5. gRPC Deadlines 与 ROS 2 QoS / pendulum real-time。

Part 23 / 25 · `interview_materials/vla/questions/vol_08_sys003.html` · SHA-256 `7885CF2DE2FB8439978BCE00EE875C1210B4D1438382AEC459BF4C12AB102B12`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`7885cf2de2fb8439978bce00ee875c1210b4d1438382aec459bf4c12ab102b12`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_sys003.md`

VOL 08 SYS-003 原子题卡：Robot Fleet 与 Multi-tenant GPU Scheduling

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-20 覆盖范围：`SYS-VLA-003` 难度：L3 1 题 优先级：P1 练习状态：`unseen` 前置知识：SYS-003 Knowledge；来源边界见 SYS-003 Source Freeze。

如何设计 Robot Fleet 与 Multi-tenant GPU Scheduling?

▼ `L3` `P1` **SYS-VLA-003 · 当机器人、租户、模型 revision 和异构 GPU 同时增长时，如何设计 fleet scheduler，并保证公平、freshness 与安全边界?**

面试意图：判断你能否把 tenant/robot/model identity、quota、placement、fairness、preemption、replica/cache、robot lease、canary 和 cost/SLO 串起来，而不是只罗列 Kubernetes 或 GPU 名词。

30 秒回答：拆 queue、placement、lease、rollout。请求带 tenant/robot/model、deadline、epoch；查 quota/兼容性，按 GPU/MIG/显存选节点。capacity/fairness/priority/freshness 分开；lease 失效丢 action，controller stop。

机制展开（约 2 分钟）：校验 tenant/robot/model/schema、deadline、lease 后查 quota，按 GPU/MIG、显存、locality、warm replica placement。capacity/fairness/priority/freshness 分开；Triton/Ray 只提供资源机制。cache 绑定 model/processor/normalizer/schema；preemption 先处理无副作用任务。lease 失效提升 epoch、丢旧回包、controller stop。用 queue wait、tail、age、miss、share、fallback、cost 验证；局部项目 hook 不能称 fleet。

诊断 / 证据：先做 identity/schema/normalizer roundtrip，再记录 admission、queue wait、Filter/Score、GPU reservation、model load、infer、lease renew、controller ack 和 rollback 时间戳。做 tenant burst、GPU 异构、cold/warm cache、preemption、lease expiry、stale-response、replica crash 和 reconnect fault injection；固定 task/seed/cohort，按 tenant 报 queue wait、p95/p99 capture→apply、action age、deadline miss、share、fallback、cost 与 success。没有 multi-tenant load trace、placement/fairness A/B、lease fencing、controller ack 或 checkpoint/cache lineage 时，只能讲设计门，不能报 fleet SLO 或 production fairness。

项目挂钩：OpenPiVLA 可连接 offline WebSocket、in-process broker、normalizer/action-shape/parser 风险；Robot Harness 可连接 tool scope/lock/timeout/compensation；OPD 可连接 multi-expert route、teacher schedule、ZeRO/config；WAM 可连接 simulator/chunk/controller diagnostic；UWM/LIBERO 可连接 launcher、task mapping、fixed protocol 和 loader diagnosis。五个项目都没有统一 fleet control plane、GPU placement、robot lease 或 multi-tenant fairness artifact，不能把局部 hook 说成我上线了 production fleet。

追问树：capacity、fairness、priority、freshness 有什么区别？为什么 GPU utilization 高不代表 action deadline 满足？Kubernetes preemption 能否安全抢占 serving？MIG/time-slicing 是否提供显存和实时隔离？model cache key 为什么必须带 normalizer/schema revision？Ray placement group 与 Triton rate limiter 各解决什么？Lease epoch 如何防止旧 action 回包？controller ack、hold/stop 和 independent e-stop 谁最终负责？如何设计 canary 与 rollback 的 cohort/metric tuple?

易错点：把 quota 当 fairness；把 priority 当 safety；把 logical resource 当 physical isolation；把 preemption 当 cancellation；把 warm cache 当 checkpoint lineage；只报 GPU utilization 或平均 latency；把上游 scheduler 文档和 Git author 当个人 fleet ownership；把 simulator rollout 当 multi-robot production result。

危险答案：说“Kubernetes/Volcano/Ray/Triton 已经构成我的 VLA fleet”；说“GPU time-slicing/MIG 天然安全”；说“Lease renew 就代表机器人执行成功”；说“抢占 Pod 会自动撤回旧 action”；或说“五个项目的脚本证明我完成了 tenant fairness、fleet SLO 和物理安全”。

来源：[1][2][3][4][5] 练习状态：unseen

参考来源

1. SYS-003 Knowledge：identity、capacity/placement、fairness/preemption、GPU/cache、lease、canary/cost 与项目边界。
2. SYS-003 Source Freeze：15 个 pinned public sources、5 项目 inventory/ledger SHA 与 15/15 probes。
3. Kubernetes scheduler/quota/priority/Lease。
4. NVIDIA device plugin、Volcano fair-share/capacity/priority、Triton model/rate limiter。
5. Ray resources/placement/Serve 与 ROS 2 node lifecycle。

Part 24 / 25 · `interview_materials/vla/questions/vol_08_sys004.html` · SHA-256 `20D9D703990CD024572B1C0A67AEE311973D8F1F401CB83776B593CC1A8B9E03`

返回本卷目录 · 对应知识页

Canonical Markdown SHA-256：`20d9d703990cd024572b1c0a67aee311973d8f1f401cb83776b593cc1a8b9e03`；生成自：`C:\Users\mousongzhe\cv\interview_materials\vla\questions\vol_08_sys004.md`

VOL 08 SYS-004 原子题卡：Fault Recovery、Safety 与 Observability

状态：`drafted / card_1_of_1 / WARN` 更新日期：2026-07-21 覆盖范围：`SYS-VLA-004` 难度：L3 1 题 优先级：P0 练习状态：`unseen` 前置知识：SYS-004 Knowledge；来源边界见 SYS-004 Source Freeze。

如何设计 VLA 系统的故障恢复、安全边界与可观测性？

▼ `L3` `P0` **SYS-VLA-004 · policy、controller、robot 或数据链路异常时，如何定位 first divergence、降级恢复并防止 stale action 造成物理风险？**

面试意图：判断你能否把状态机、证据链、safe-apply、lease fencing、rollback/replay 和独立 e-stop 边界连成闭环，而不是把“加日志、重试、stop”当作完整安全设计。

30 秒回答：用 trace/metric/log 找 first divergence，按 DEGRADED→HOLD/STOPPED 分级。action 校验 schema、时间、lease epoch、deadline；失败即丢弃并 hold/stop。恢复隔离后回滚兼容 bundle，用 replay/canary 验证；e-stop 属于独立安全链。

机制展开（约 2 分钟）：让 observation→infer→apply→ack 共用 robot/session identity，关联 trace、metric、log。按 symptom→first divergence→root cause→outcome 诊断；liveness/readiness/startup 与 QoS 分开。`safe_apply` 检查 schema、normalizer/controller compatibility、freshness、deadline、lease epoch；失败则 quiesce、清旧 chunk，进入 hold/stop。rollback 用兼容矩阵和 bundle；replay 固定输入与版本，再做 canary。硬件 e-stop 独立断能。

诊断 / 证据：先找第一条可验证的时间戳分叉：capture→queue→infer→validate→apply→ack；记录 `A_age=t_apply-t_capture`、deadline miss、QoS drop、queue、lease epoch、controller ack 和状态迁移。做 stale response、model/schema mismatch、normalizer drift、timeout、partial sensor、controller restart、rollback 与 replay fault injection。没有本地 supervisor、controller acknowledgement、fault-injection matrix、immutable replay bundle 或 rollback compatibility artifact 时，只能讲设计合同，不能声称完成生产安全闭环。

项目挂钩：OpenPiVLA 可连接 WebSocket connect/infer timeout、parser/normalizer/action-shape 风险；Robot Harness 可连接 tool timeout、lock、cancel/compensation；OPD 可连接多教师/配置 lineage；WAM 可连接 simulator/chunk/controller diagnostics；UWM/LIBERO 可连接 loader、launcher、protocol 与 fixed-init 结果。五个项目都没有被证据证明拥有统一 fault supervisor、controller e-stop acknowledgement 或 production incident replay，不能把局部脚本写成个人安全平台 ownership。

追问树：liveness/readiness/startup 为什么不能互换？first divergence 如何区别 symptom 和 root cause？cancel、hold、stop、e-stop 的 owner 分别是谁？为什么 lease epoch 能挡 stale action？action age 与 inference latency 有什么区别？rollback 只换 checkpoint 够不够？如何保证 replay 不被 wall-clock、随机数或在线状态污染？QoS reliable 是否等于动作必达？controller 没 ack 时策略服务该做什么？

易错点：把重试当恢复；把服务进程存活当 readiness；只记录 error log 不记录 identity/timestamp；把 cancel 说成物理 stop；没有 fencing 仍执行旧 action；把上游 ROS/Kubernetes/OpenTelemetry 机制当个人 production evidence；把 simulator replay 当真实机器人安全结果。

危险答案：说“超时就重试直到成功”；说“服务返回 200 就代表机器人执行成功”；说“kill policy pod 就是 e-stop”；说“有 trace 就能证明 RCA”；或说“五个项目已经完成 fault supervisor、rollback 和物理安全闭环”。

来源：[1][2][3][4][5] **练习状态**：unseen

参考来源

1. SYS-004 Knowledge：fault state、first-divergence、safe-apply、lease fencing、rollback/replay 与项目边界。
2. SYS-004 Source Freeze：ROS 2、Kubernetes、OpenTelemetry、12 个 deterministic probes 与五项目证据边界。
3. ROS 2 managed node lifecycle 与 `ros2_control` controller manager。
4. Kubernetes probes 与 rollout/rollback。
5. OpenTelemetry signals and context；物理 e-stop 仍属于独立安全控制链，不由软件 tracing 或 policy response 取代。

Part 25 / 25 · `interview_materials/vla/sources/vol_08_coding_system_design_coverage_gap_audit.html` · SHA-256 `E39AA5BAD3E2E4847984BE2A156B779982DB82671F180A8C1D35F1F32EEAB61F`

返回本卷目录

Canonical Markdown SHA-256: `e39aa5bad3e2e4847984be2a156b779982db82671f180a8c1d35f1f32eeab61f`; 生成自: `C:\Users\mousongzhe\cv\interview_materials\vla\sources\vol_08_coding_system_design_coverage_gap_audit.md`

VOL 08 Coding / System Design: Coverage-Gap Audit

审计日期: 2026-07-20 状态: `coverage_contract_frozen / planned_12 / sources_12 / knowledge_12 / cards_12 / missing_sources_0 / missing_knowledge_0 / missing_cards_0 / WARN` 审计对象: VOL 08 已有 source/knowledge/card、早期手撕题 registry、全局 Knowledge Map、Repository Completion Audit、GitHub skeleton guidance、VOL 05 工程/评估卷与五项目 evidence inventory。 核心边界: 本报告冻结整卷范围、去重边界和退出条件; `SYS-VLA-002..003` 三联件已同步但未练习，不把 VOL 05 组件知识重复包装成系统设计答案，不把公开系统方案写成个人项目成果。

问题（Question）

现有 `SYS-VLA-001` 与 `CODE-VLA-001..006` 是否足以支持"VOL 08 Coding / System Design 已完成"? 如果不足，整卷应冻结为哪些不可合并的系统设计与手撕实现节点，如何与 VOL 01、VOL 05 和个人项目证据分工?

1. 结论先行

1. VOL 08 冻结为 **12 个原子节点**: `SYS-VLA-001..004` 四个系统设计题，`CODE-VLA-001..008` 八个手撕实现题; 其中 10 个 P0、2 个 P1。
2. 当前 source coverage 为 `12/12=100%`，knowledge coverage 为 `12/12=100%`，card coverage 为 `12/12=100%`: `SYS-VLA-004` source+knowledge+card 已完成; `CODE-VLA-008` 已完成 source+knowledge+card，`CODE-VLA-007` 与 `SYS-VLA-001..003/CODE-VLA-001..006` 均已有三联件。十二张题卡全部保持 `unseen/WARN`。
3. 页面三联件已完整闭合; 不能把页面覆盖计为题卡掌握，四题 SYS 口述与八题 CODE 练习仍是独立退出门。
4. WinstonJQ 骨架支持"先题目清单、再分卷、Markdown canonical、折叠题卡、review、L1/L2/L3"的工作流，但不支持具体 VLA 系统事实、题目频次或个人项目 claim。
5. VOL 05 负责可复用组件机制; VOL 08 的系统设计题必须把这些组件整合成 requirements/capacity、API/schema、state ownership、SLO/deadline/backpressure、failure/degradation/rollback、scaling/cost/safety 和项目证据边界，不能只背组件名。
6. Coverage、review、drill 与 project truth 分开验收。即使未来达到 12/12 页面覆盖，没有四题真实系统设计口述与八题白板/runtime 练习，也不能标记为 `drilled/done`。

2. 本次审计依据

依据	实际支持	不能支持
GitHub Source Scout	WinstonJQ 骨架: 主索引、分卷、canonical Markdown、折叠题卡、review、L1/L2/L3 与题目前置清单	VLA 系统设计的技术事实、面试频次或个人贡献
VLA Knowledge Map	VOL 08 应覆盖 attention/loss/norm、geometry/FK、train pipeline、policy service、robot fleet、fault recovery	当前十一节点已有 source/knowledge，十张题卡已生成
Repository Completion Audit	全局 274 张卡均 unseen; VOL 08 当前有十二个完整节点，SYS-004 三联件已同步	registry-only ID 或页面生成等于掌握
VOL 01 Phase 0 Question List	`CODE-VLA-001..008` 的早期手撕候选边界，含 PID 与 TD(0)	007 source 已冻结但 knowledge/card 未生成; 008 仍 planned
SYS-VLA-001 Source Freeze	dataset/run/bundle/eval/rollout lineage 与可复现训练—评估平台合同	policy service、fleet 和 fault-recovery 三题已经覆盖
SYS-VLA-002 Source Freeze	OpenPI request/response、client-side chunk/replan、deadline/backpressure、real-time boundary、14/14 probes 与五项目 truth boundary	knowledge 与题卡已转译; production SLO、hard real-time或个人端到端ownership仍未完成
CODE-VLA-001..006 Source Freezes	softmax、attention、norm、masked loss、transform、planar FK 六个实现节点各自的来源与测试边界	PID、TD(0) 或本地白板/runtime 已完成
VOL 05 Completion Audit	工程与评估组件知识的计划内覆盖和证据边界	四个端到端 system-design oral answer
五项目 inventories / ownership ledgers	项目接口、artifact、指标与个人贡献边界	已建设统一平台、robot fleet、生产 policy service 或物理安全系统

3. 当前资产盘点

层	已有	目标	覆盖	仍开放的门
planned stable IDs	12	12	100% contract	本报告只冻结边界，不代表内容完成
source freeze	12	12	100%	source gate 已冻结; 三联件已同步
knowledge nodes	12	12	100%	knowledge gate 已冻结; 三联件已同步

层	已有	目标	覆盖	仍开放的门
generated question cards	12	12	100%	页面覆盖完成；仍需真实系统设计/白板训练
system-design oral drill	0	4	0%	四张 SYS 卡均已生成并保持 unseen，尚未完成限时口述
coding whiteboard/runtime drill	0	8	0%	六张已有卡仍 unseen，007/008 未生成
project truth gate	5 inventories + 5 ledgers	每个 system answer 都有 bounded hook	partial	不得把 interface/evidence 升级成 platform/fleet ownership

4. 整卷 Stable-ID 合同

ID	优先级 / 难度	冻结题目	当前状态	不得合并掉的核心边界
`SYS-VLA-001`	P0 / L3	可复现的 VLA 训练—评估平台	source + knowledge + card / unseen	data/run/checkpoint/bundle/eval/rollout lineage 与 protocol identity
`SYS-VLA-002`	P0 / L3	Policy serving 与 real-time control loop	source + knowledge + card / unseen	API/schema、session/state ownership、latency budget、deadline、backpressure、chunk/replan、fallback
`SYS-VLA-003`	P1 / L3	Robot fleet 与 multi-tenant GPU scheduling	source + knowledge + card / unseen	capacity、tenant/robot/model identity、placement、fairness、cache、rollout、cost 与异构设备
`SYS-VLA-004`	P0 / L3	Fault recovery、safety 与 observability	source + knowledge + card / unseen	first-divergence、SLO、degradation、cancel/e-stop 边界、rollback、replay 与 incident evidence
`CODE-VLA-001`	P0 / L1	Numerically stable softmax / log-softmax	source + knowledge + card / unseen	max-shift、mask、dtype、all-masked row 与 parity
`CODE-VLA-002`	P0 / L2	Scaled dot-product attention	source + knowledge + card / unseen	shape、scale、mask、dropout、complexity 与 backend parity
`CODE-VLA-003`	P0 / L1	LayerNorm / RMSNorm	source + knowledge + card / unseen	axis、centering、variance/RMS、epsilon、dtype 与 train/eval
`CODE-VLA-004`	P0 / L1	Masked MSE / Huber loss	source + knowledge + card / unseen	valid denominator、empty mask、weighting 与 distributed reduction
`CODE-VLA-005`	P0 / L2	Homogeneous transform composition	source + knowledge + card / unseen	`T^A_B` 方向、frame、单位、时间戳、point/vector/pose 与 inverse
`CODE-VLA-006`	P1 / L2	2-link planar forward kinematics	source + knowledge + card / unseen	relative joint angle、analytic/matrix parity、Jacobian 与 FK/IK 边界
`CODE-VLA-007`	P0 / L2	PID update with anti-windup	source + knowledge + card / unseen	`dt`、derivative state、saturation、conditional integration/back-calculation 与 reset
`CODE-VLA-008`	P0 / L2	Bellman TD(0) update	source + knowledge + card / unseen	reward、discount、bootstrap、terminated/truncated 与 batch mask

`registered / planned` 只表示 stable ID 已进入范围，不进入 272 张 generated-card 计数，也不产生 mastery。

5. 依赖主链与学习簇

```
# Planned nodes are scope contracts, not generated answers.
CODE-001 softmax -> CODE-002 attention
CODE-003 norm + CODE-004 masked loss -> train-time numerical correctness

CODE-005 transform -> CODE-006 FK -> CODE-007 PID -> policy/controller boundary
VOL 01 MDP/Bellman -> CODE-008 TD(0) -> value/update implementation

VOL 05 reproducibility/packaging/eval -> SYS-001 train-eval platform
VOL 05 policy server/latency -> SYS-002 serving-control loop
VOL 05 scheduling/retry/observability -> SYS-003 fleet -> SYS-004 recovery/safety
```

学习簇	IDs	当前 / 目标	主要验收方式
数值与 Transformer 实现	`CODE-001..004`	4 / 4	白板实现、shape/mask/dtype probes、reference parity
几何、控制与 RL 更新	`CODE-005..008`	2 / 4	单位/frame/edge-case 测试、finite difference、termination semantics
训练、服务、fleet 与恢复	`SYS-001..004`	1 / 4	需求澄清、容量估算、API/schema、SLO/failure/rollback 口述

6. 与 VOL 01 / VOL 05 的去重边界

6.1 VOL 01 Foundations 与 CODE 题

VOL 01 负责解释公式、概念与使用条件；VOL 08 CODE 负责在明确函数签名、shape、dtype、mask、edge case、复杂度和测试 oracle 下实现。能口述 softmax 或 PID 原理不等于能写出稳定、可测的实现。

6.2 VOL 05 Engineering / Evaluation 与 SYS 题

VOL 05 的 ENG/EVAL 节点是组件知识。VOL 08 每个系统设计回答必须显式走完以下主线：

1. 需求与非目标：任务类型、机器人/请求规模、控制频率、安全边界、可接受降级。
2. 容量与延迟预算：吞吐、并发、GPU/CPU/网络、端到端 deadline 与 jitter。
3. API 与 schema：observation/action/model/session/run identity、版本、幂等与兼容性。
4. 状态所有权：谁持有 session、action chunk、normalizer、checkpoint、robot lease 与 replay cursor。
5. 正常数据流：从 observation capture 到 infer、validate、denormalize、execute、replan、record。
6. 压力与失败：queue/backpressure、timeout、partial failure、stale observation、model/server/controller mismatch。
7. 降级与恢复：fallback、cancel、rollback、canary、checkpoint recovery、incident replay；物理 e-stop 只能作为系统边界，不能由无证据软件 claim 替代。
8. 扩展、成本与证据：多 tenant/robot/model 的 placement 与公平性，以及哪些设计在个人项目中真实存在。

7. 后续 Source Queue

下表只冻结检索队列；跨卷引用可以减少重复，但不能替代每个新节点自己的 claim-level source gate。

ID	待冻结的来源类型	可复用入口	当前状态
`SYS-VLA-002`	OpenPI 8 fixed files、CPython/Triton/gRPC/ROS 2 fixed docs、3个local OpenPiVLA files、14 probes	SYS-001、VOL 03 action execution、VOL 05 ENG-022/EVAL-013	`source + knowledge + card / unseen`
`SYS-VLA-003`	Kubernetes scheduler/quota/lease、NVIDIA device plugin、Volcano fair-share/capacity/priority、Triton rate/model config、Ray resources/placement/Serve、ROS lifecycle 与五项目 manifests	VOL 05 ENG-016/018/019/022、五项目 inventories/ledgers	`source + knowledge + card / unseen`
`SYS-VLA-004`	ROS 2 lifecycle/controller/QoS、Kubernetes probes/rollback、OpenTelemetry signals、SYS-002/EVAL-008	VOL 05 ENG-017/018/022、EVAL-008/014/020/021、VOL 04 safety/fallback	`source + knowledge + card synced / unseen`
`CODE-VLA-007`	ROS 2 control PID、WPILib PIDController、Feedback Systems 与 VOL 01 advanced-control review	VOL 01 `VLA-FND-036`、advanced-control fact review	`source + knowledge + card / unseen`
`CODE-VLA-008`	Sutton–Barto TD(0)、Gymnasium terminated/truncated semantics 与 VOL 01/VOL 02 fact reviews	VOL 01 `VLA-FND-023/027`、VOL 02 fact review	`source + knowledge + card synced / unseen`

8. 项目挂钩与真实性边界

项目	可用 hook	当前不能说
Robot Harness	local trainer/eval/config、metrics/raw-support/recovery manifest，可用于 offline evaluation、lineage 和 failure-evidence 讨论	已部署 robot fleet、online policy service、统一生产平台或真实物理安全闭环
OpenPiVLA	local JSON/causal-CE、shape/formatter 风险与 official parallel serving/action contract，可用于 schema/round-trip 诊断	formal checkpoint、controller round-trip、稳定 online rollout 或个人拥有完整 serving platform
OPD	point/box/text route、scorer、data/eval boundary，可用于 typed request/result 与 evaluation contract	robot policy serving、fleet scheduling 或 closed-loop physical execution
WAM	simulator/policy artifact 与 paired diagnostic，可用于 checkpoint/eval/rollout 失败边界	已完成真实机器人部署、生产 recovery 或安全系统
UWM / LIBERO	simulator protocol、fixed-init 与 user-confirmed limited result，可用于 protocol identity 与 rollout schema	跨协议生产泛化、robot fleet、可复算 checkpoint/episode bundle 或未经确认 ownership

面试中按 `public upstream design → local evidence → local inference → unverified` 四层作答。路径、官方 example、Git author、配置字段或局部脚本都不能自动升级为个人端到端 system ownership。

9. VOL 08 退出条件

9.1 Coverage complete

1. `SYS-VLA-001..004` 与 `CODE-VLA-001..008` 全部有唯一 source/knowledge/card 三联件，卷级 registry 与索引同步。
2. 每张卡包含 stable ID、优先级/难度、短答、机制展开、来源、项目挂钩、易错点、追问和练习状态。
3. 新节点有 primary/official revision 与 claim boundary；跨卷链接不能单独支撑强事实。

9.2 Reviewed

1. 12 个节点的 canonical/HTML/review SHA、单 H1、层级、锚点、公式、代码、表格和本地链接通过。
2. 四个 SYS 节点逐一完成独立事实/设计边界复核；两个 CODE 学习簇各至少有一次 reference-parity review。
3. 项目 hook 只引用 evidence inventory/ownership ledger；缺 runtime、checkpoint、raw bundle 或候选人确认时保留 WARN。

9.3 Drilled

1. `SYS-VLA-001..004` 各完成一次限时系统设计口述，记录需求澄清、容量、API、状态、failure 与 trade-off 反馈。

2. `CODE-VLA-001..008` 各完成一次白板或 runtime 练习，记录代码、测试、复杂度、错误与复盘。
3. 只有真实回答/实现后才创建或更新 coaching state；页面覆盖与静态校验不改变 `unseen/weak/pass/strong/stale`。

9.4 Project truth

1. 每个 SYS 回答至少包含一个真实项目 hook，同时明确“已做 / 参与 / 可补做 / 未验证 / 不能说”。
2. 不要求虚构 fleet 或生产部署；没有直接证据时，允许把答案限定为设计方案和项目差距分析。
3. 系统设计能力与个人已落地系统是两个不同 claim，必须分别表述。

10. 缺口优先级与下一切片

优先级	缺口	原因	下一动作
P0-1	`SYS-VLA-002` knowledge	request/schema、state、timing、backpressure、real-time与truth boundary已完成教学转译	已完成，保持WARN
P0-2	`SYS-VLA-002` card	建立在已审knowledge上，包含短答、机制、诊断、项目 hook 与危险答案	已完成一张unseen卡，保持WARN
P0-3	`SYS-VLA-004` oral drill	knowledge/card 已完成；需记录 incident/recovery 口述与项目边界	下一步进行一次真实系统设计口述
P1	`SYS-VLA-003` fleet scheduling	价值高但依赖 serving identity、state 与 SLO 合同	source+knowledge+card已同步，等待口述

下一 bounded slice：只进行一次 `SYS-VLA-004` 系统设计真实口述；保持 `unseen`、不修改 mastery、不创建`coaching_state.md`，不启动其他 VOL 08 节点。

参考入口（References）

1. GitHub 骨架与来源入口
2. VLA Knowledge Map
3. Repository Completion Audit
4. VOL 01 Phase 0 Question List
5. SYS-VLA-001 Source Freeze
6. CODE-VLA-001 Softmax Source Freeze
7. CODE-VLA-002 Attention Source Freeze
8. CODE-VLA-003 Norm Source Freeze
9. CODE-VLA-004 Masked Loss Source Freeze
10. CODE-VLA-005 Transform Source Freeze
11. CODE-VLA-006 Planar FK Source Freeze
12. VOL 05 Completion Independent Fact Review
13. Evidence Manifest
14. SYS-VLA-002 Policy Serving / Real-time Control Loop Source Freeze